AiM5101HXX 芯片

用户指南

文件版本：V1.1

发布日期：2024-2-27

版权所有 © 成都爱旗科技有限公司 2023。保留一切权利。

非经成都爱旗科技有限公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

商标声明

 、爱旗和其他爱旗商标均为成都爱旗科技有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

注意

您购买的产品、服务或特性等应受成都爱旗科技有限公司商业合同和条款的约束，本文档中描述的全部或部分产品、服务或特性可能不在您的购买或使用范围之内。除非合同另有约定，成都爱旗科技有限公司对本文档内容不做任何明示或默示的声明或保证。

由于产品版本升级或其他原因，本文档内容会不定期进行更新。除非另有约定，本文档仅作为使用指导， 本文档中的所有陈述、信息和建议不构成任何明示或暗示的保证，包括但不限于适销性或者适用于某一特定目的的保证。

成都爱旗科技有限公司

地址: 四川省成都市高新区天华二路 219 号 C12 栋 17/18 层

邮编: 610041

官网: http://www.aichtech.com

客户服务邮箱: support@aichtech.com

前 言

概述

本文档主要介绍 AiM5101HXX 系列芯片的各项基本功能，常见应用模式的配置方法。

本文档提供 AiM5101HXX 系列芯片的应用配置方法。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
AiM5101HXX 系列芯片	V10

读者对象

本文档主要适用于以下工程师：

- 单板硬件开发工程师
- 软件工程师
- 技术支持工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下：

符号	说明
⚠危险	表示如不避免则将会导致死亡或严重伤害的具有高等级风险的危害。
⚠警告	表示如不避免则可能导致死亡或严重伤害的具有中等级风险的危害。
⚠注意	表示如不避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
V1.0	2023-4-22	第一次正式发布版本。
V1.1	2024-2-27	1.刷新 TSC、QDM、KEYSCAN、ACPM 功能描述。 2.刷新 GPIO、QDM、QSPI、BACKUP、ACPM 寄存器描述。

目 录

前 言 3
目 录 5
插图目录 43
表格目录 48
1 系统 51
1.1 系统架构 51
1.2 存储器构成 52
1.2.1 简介 53
1.2.2 存储器映射和寄存器边界地址 53
1.3 嵌入式 SRAM 55
1.4 Flash 概述 56
1.5 自举配置 56
1.6 中断映射表 57
2 处理器（CPU） 61
2.1 CPU 主要特性 61
2.2 CPU 编程模型 61
2.2.1 工作模式及寄存器视图 61
2.2.2 通用寄存器 63
2.2.3 浮点寄存器 64
2.2.4 机器模式控制寄存器 64
2.2.5 用户模式控制寄存器 72
2.2.6 数据格式 74
2.3 CPU 指令集 75
2.4 CPU 异常与中断 89
2.4.1 异常 89
2.4.2 中断 94
2.4.3 NMI 99
2.5 CLIC 中断控制器 100
2.5.1 CLIC 寄存器地址映射 100

2.5.2 CLIC 寄存器描述 101
3 嵌入式 Flash 控制器（EFC） 105
3.1 EFlash 控制器简介 105
3.2 EFlash 控制器主要特性 105
3.2.1 存储体 105
3.2.2 EFlash 接口特性 106
3.3 EFlash 控制器与系统 106
3.3.1 存储器与控制器地址 106
3.3.2 自启动与 IP Trim 106
3.3.3 Option Byte 107
3.3.4 BOOT 模式 107
3.4 EFlash 控制器功能描述 108
3.4.1 Flash 空检查 108
3.4.2 Flash 计时基准 109
3.4.3 Flash 读访问延迟 109
3.4.4 Flash 擦除 109
3.4.5 Flash 编程 110
3.4.6 Flash 重载选项字节 110
3.4.7 Flash 读取 111
3.5 Flash 内容访问保护 112
3.5.1 写入保护区域 112
3.5.2 读访问保护等级 113
3.5.3 PCROP 保护区 114
3.6 EFlash 控制器寄存器概览 114
3.7 EFlash 控制器寄存器描述 116
3.7.1 EF_ACC_CTRL 116
3.7.2 EF_OPKEY 116
3.7.3 EF_OPTRG 117
3.7.4 EF_OPSET 118
3.7.5 EF_STAT 118
3.7.6 EF_OPADDR 119
3.7.7 EF_RDBUF0 120
3.7.8 EF_RDBUF1 120
3.7.9 EF_PROGBUF0 121
3.7.10 EF_PROGBUF1 121
3.7.11 EF_WRP0 122

3.7.12 EF_WRP0_SHD 122
3.7.13 EF_WRP1 123
3.7.14 EF_WRP1_SHD 123
3.7.15 EF_SYS_OPT 124
3.7.16 EF_SYS_OPT_SHD 124
3.7.17 EF_PCROP0 125
3.7.18 EF_PCROP0_SHD 126
3.7.19 EF_PCROP1 126
3.7.20 EF_PCROP1_SHD 126
3.7.21 EF_ECC 127
3.7.22 INT_EN 127
3.7.23 INT_RAW 129
3.7.24 INT_STAT 130
3.7.25 INT_CLR 131
3.7.26 INT_INJ 132
3.7.27 UID0 134
3.7.28 UID1 134
3.7.29 UID2 134
3.7.30 UID3 135
4 时钟和复位控制（CRG） 136
4.1 CRG 简介 136
4.2 复位 136
4.2.1 复位源描述 136
4.2.2 复位控制域描述 137
4.3 时钟 138
4.3.1 源时钟 139
4.3.2 PLL 140
4.3.3 衍生时钟 141
4.3.4 输出时钟 142
4.3.5 寄存器写保护 143
4.3.6 系统时钟切换流程 143
4.4 CRG 寄存器概览 144
4.5 CRG 寄存器描述 147
4.5.1 PERI_CRG0 147
4.5.2 PERI_CRG1 148
4.5.3 PERI_CRG2 148

4.5.4 PERI_CRG3 148
4.5.5 PERI_CRG4 149
4.5.6 PERI_CRG5 150
4.5.7 PERI_CRG7 150
4.5.8 PERI_CRG8 151
4.5.9 PERI_CRG21 152
4.5.10 PERI_CRG22 152
4.5.11 PERI_CRG23 153
4.5.12 PERI_CRG24 154
4.5.13 PERI_CRG25 155
4.5.14 PERI_CRG26 155
4.5.15 PERI_CRG27 156
4.5.16 PERI_CRG28 157
4.5.17 PERI_CRG29 157
4.5.18 PERI_CRG30 158
4.5.19 PERI_CRG31 158
4.5.20 PERI_CRG33 159
4.5.21 PERI_CRG35 159
4.5.22 PERI_CRG37 160
4.5.23 PERI_CRG40 161
4.5.24 PERI_CRG41 162
4.5.25 PERI_CRG42 162
4.5.26 PERI_CRG44 163
4.5.27 PERI_CRG46 164
4.5.28 PERI_CRG47 165
4.5.29 PERI_CRG49 165
4.5.30 PERI_CRG50 166
4.5.31 PERI_CRG51 166
4.5.32 PERI_CRG52 167
4.5.33 PERI_CRG53 168
4.5.34 PERI_CRG54 168
4.5.35 PERI_CRG57 169
4.5.36 PERI_CRG58 169
4.5.37 PERI_CRG59 170
4.5.38 PERI_CRG60 171
4.5.39 PERI_CRG61 172

4.5.40 PERI_CRG62 172
4.5.41 PERI_CRG63 173
4.5.42 PERI_CRG64 173
4.5.43 PERI_CRG65 174
4.5.44 PERI_CRG66 175
4.5.45 PERI_CRG67 175
4.5.46 PERI_CRG68 176
4.5.47 PERI_CRG69 176
4.5.48 PERI_CRG70 177
4.5.49 PERI_CRG71 178
4.5.50 PERI_CRG72 178
4.5.51 PERI_CRG73 179
4.5.52 PERI_CRG74 180
4.5.53 PERI_CRG75 181
5 时钟频偏检测（CMM） 183
5.1 CMM 简介 183
5.2 CMM 主要特性 183
5.3 CMM 功能描述 183
5.4 CMM 工作方式 184
5.4.1 频偏监测流程 184
5.4.2 中断处理流程 185
5.5 CMM 寄存器概览 185
5.6 CMM 寄存器描述 186
5.6.1 CMM_EN 186
5.6.2 CMM_CFG1 186
5.6.3 CMM_CFG2 187
5.6.4 CMM_STS 188
5.6.5 CMM_INT_EN 188
5.6.6 CMM_INT_STS 189
5.6.7 CMM_INT_RAW 189
5.6.8 CMM_INT_INJ 190
5.6.9 CMM_INT_CLR 190
6 时钟失效检测（CFD） 192
6.1 CFD 简介 192
6.2 CFD 主要特性 192
6.3 CFD 功能描述 192

6.4 CFD 工作方式 193
6.4.1 失效监测流程 193
6.4.2 中断处理流程 194
6.5 CFD 寄存器概览 194
6.6 CFD 寄存器描述 195
6.6.1 CFD_EN 195
6.6.2 CFD_CFG 195
6.6.3 CFD_STS 196
6.6.4 CFD_INT_EN 196
6.6.5 CFD_INT_STS 197
6.6.6 CFD_INT_RAW 198
6.6.7 CFD_INT_INJ 198
6.6.8 CFD_INT_CLR 199
7 通用 I/O（GPIO） 200
7.1 GPIO 简介 200
7.2 GPIO 主要特性 200
7.3 GPIO 功能描述 200
7.4 GPIO 寄存器概览 202
7.5 GPIO 寄存器描述 203
7.5.1 GPIO_DATA 203
7.5.2 GPIO_DIR 203
7.5.3 GPIO_IS 204
7.5.4 GPIO_IBE 204
7.5.5 GPIO_IEV 204
7.5.6 GPIO_IE 205
7.5.7 GPIO_RIS 205
7.5.8 GPIO_MIS 206
7.5.9 GPIO_IC 206
7.5.10 DATA_EN 207
8 管脚复用（IO MUX） 208
8.1 简介 208
8.2 IO 主要特性 208
8.3 IO 模拟模式 208
8.4 IO 数字模式 209
8.5 CORE IO 寄存器概览 209
8.6 CORE IO 寄存器描述 211

8.6.1 PA0_IOCFG 211
8.6.2 PA1_IOCFG 213
8.6.3 PA2_IOCFG 214
8.6.4 PA3_IOCFG 215
8.6.5 PA9_IOCFG 216
8.6.6 PA10_IOCFG 217
8.6.7 PA11_IOCFG 218
8.6.8 PA12_IOCFG 220
8.6.9 PA13_IOCFG 221
8.6.10 PA14_IOCFG 222
8.6.11 PA15_IOCFG 223
8.6.12 PB0_IOCFG 225
8.6.13 PB1_IOCFG 226
8.6.14 PB2_IOCFG 227
8.6.15 PB3_IOCFG 229
8.6.16 PB4_IOCFG 230
8.6.17 PB5_IOCFG 231
8.6.18 PB6_IOCFG 233
8.6.19 PB7_IOCFG 234
8.6.20 PB8_IOCFG 235
8.6.21 PB9_IOCFG 237
8.6.22 PB10_IOCFG 238
8.6.23 PB11_IOCFG 239
8.6.24 PB12_IOCFG 241
8.6.25 PB13_IOCFG 242
8.6.26 PB14_IOCFG 243
8.6.27 PB15_IOCFG 245
8.6.28 PC0_IOCFG 246
8.6.29 PC1_IOCFG 247
8.6.30 PC2_IOCFG 248
8.6.31 PC3_IOCFG 250
8.6.32 PC4_IOCFG 251
8.6.33 PC5_IOCFG 252
8.6.34 PC6_IOCFG 254
8.6.35 PC7_IOCFG 255
8.6.36 PC8_IOCFG 256

8.6.37 PC9_IOCFG 257
8.6.38 PC10_IOCFG 259
8.6.39 PC11_IOCFG 260
8.6.40 PC12_IOCFG 262
8.6.41 PC13_IOCFG 263
8.6.42 PC14_IOCFG 264
8.6.43 PC15_IOCFG 266
8.6.44 PD0_IOCFG 267
8.6.45 PD1_IOCFG 268
8.6.46 PD2_IOCFG 269
8.6.47 PD3_IOCFG 270
8.6.48 PD4_IOCFG 271
8.6.49 PD5_IOCFG 272
8.6.50 PD6_IOCFG 274
8.6.51 PD7_IOCFG 275
8.6.52 PD8_IOCFG 276
8.6.53 PD9_IOCFG 277
8.6.54 PD10_IOCFG 279
8.6.55 PD11_IOCFG 280
8.6.56 PD12_IOCFG 281
8.6.57 PD13_IOCFG 282
8.6.58 PD14_IOCFG 284
8.6.59 PD15_IOCFG 285
8.6.60 PE0_IOCFG 286
8.6.61 PE1_IOCFG 288
8.6.62 PE2_IOCFG 289
8.6.63 PE3_IOCFG 290
8.6.64 PE4_IOCFG 291
8.6.65 PE5_IOCFG 292
8.6.66 PE6_IOCFG 294
8.6.67 PE7_IOCFG 295
8.6.68 PE8_IOCFG 296
8.6.69 PE9_IOCFG 297
8.6.70 PE10_IOCFG 298
8.6.71 PE11_IOCFG 300
8.6.72 PE12_IOCFG 301

8.6.73 PE13_IOCFG 302
8.6.74 PE14_IOCFG 303
8.6.75 PE15_IOCFG 304
8.6.76 PF0_IOCFG 306
8.6.77 PF1_IOCFG 307
8.6.78 BOOT_IOCFG 308
8.6.79 PF2_IOCFG 309
8.6.80 PF3_IOCFG 310
8.6.81 PF4_IOCFG 312
8.6.82 PF5_IOCFG 313
8.7 BACKUP IO 寄存器概览 314
8.8 BACKUP IO 寄存器描述 314
8.8.1 PA4_IOCFG 314
8.8.2 PA5_IOCFG 316
8.8.3 PA6_IOCFG 317
8.8.4 PA7_IOCFG 318
8.8.5 PA8_IOCFG 319
9 系统控制器 (SYSCTRL) 322
9.1 SYSCTRL 简介 322
9.2 SYSCTRL 寄存器概览 322
9.3 SYSCTRL 寄存器描述 323
9.3.1 SYS_RST_REQ 323
9.3.2 SYSTICK_CTRL 324
9.3.3 SYSTICK_CNT_INIT_VAL 324
9.3.4 INT_SOFTWARE 325
9.3.5 LOCKEN 325
9.3.6 SRAM_PARITY_ERR 326
9.3.7 SRAM_PARITY_ERR_CLR 327
9.3.8 SYS_RSV_CTRL 327
9.3.9 MODULE_CTRL 328
9.3.10 USB_CTRL 329
9.3.11 DMA_REQ_SEL 330
9.3.12 SC_RST_CNT0 330
9.3.13 SC_RST_CNT1 331
9.3.14 VERSION 331
9.3.15 USER_REG0 332

9.3.16 USER_REG1 332
9.3.17 USER_HRST_REG0 332
9.3.18 USER_HRST_REG1 333
9.3.19 USER_POR_REG0 333
9.3.20 USER_POR_REG1 334
10 外设互联矩阵 335
10.1 外设互联矩阵简介 335
10.2 连接汇总 335
10.3 连接信息 336
10.3.1 GPT0~3、APT0、APT1、ADC 之间的相互同步 336
10.3.2 CMP0~3、CPU、PVD、SRAM、EFC、CFD 到 APT0~1 的连接 337
10.3.3 GPT0~3、APT0、APT1 到 ADC0~1 的连接 337
11 电源控制（PMC） 338
11.1 PMC 简介 338
11.2 PMC 主要特性 338
11.3 电源 338
11.3.1 概述 338
11.3.2 电源监控 340
11.4 低功耗模式 340
11.4.1 概述 340
11.4.2 工作模式 342
11.4.3 睡眠模式 343
11.4.4 深睡眠模式 343
11.4.5 关机模式 344
11.4.6 VBAT 模式 345
11.5 PMC 寄存器概览 345
11.6 PMC 寄存器描述 346
11.6.1 IWDG_CRG 346
11.6.2 LOWPOWER_MODE 347
11.6.3 STATE_EN 347
11.6.4 LOWPOWER_STATUS 348
11.6.5 LOWPOWER_STATUS_CLR 349
11.6.6 PMU_CTRL 349
11.6.7 INT_PVD_STATE 350
11.6.8 INT_PVD_CLR 351
11.6.9 PMU_STATE 351

11.6.10 AON_USER_REG0 352
11.6.11 AON_USER_REG1 352
11.6.12 AON_USER_REG2 353
11.6.13 AON_USER_REG3 353
12 循环冗余校验计算单元（CRC） 354
12.1 CRC 简介 354
12.2 CRC 主要特性 354
12.3 CRC 功能描述 354
12.3.1 CRC 典型配置 355
12.4 CRC 寄存器概览 355
12.5 CRC 寄存器描述 356
12.5.1 CRC_EN 356
12.5.2 CRC_IN 356
12.5.3 CRC_OUT 357
13 直接存储器访问控制器（DMAC） 358
13.1 DMAC 简介 358
13.2 DMAC 主要特性 358
13.3 DMAC 逻辑实现 358
13.3.1 DMAC 逻辑框图 358
13.3.2 DMAC 工作/配置流程 359
13.3.3 DMAC 请求映射 360
13.3.4 DMAC 流控制器及传输类型 364
13.4 DMAC 功能描述 365
13.4.1 DMAC 握手接口 365
13.4.2 DMAC 链表 369
13.4.3 DMAC 循环模式 371
13.4.4 DMAC 时钟门控 372
13.4.5 DMAC 初始化 372
13.4.6 DMAC 通道编程 373
13.4.7 DMAC 中断处理 375
13.5 DMAC 寄存器概览 376
13.6 DMAC 寄存器描述 378
13.6.1 SAR 378
13.6.2 DAR 378
13.6.3 LLP 378
13.6.4 CTL_L 379

13.6.5 CTL_H 381
13.6.6 SSTAT 382
13.6.7 DSTAT 382
13.6.8 SSTATAR 382
13.6.9 DSTATAR 383
13.6.10 CFG_L 383
13.6.11 CFG_H 385
13.6.12 SGR 386
13.6.13 DSR 386
13.6.14 RawTfr 386
13.6.15 RawBlock 387
13.6.16 RawSrcTran 387
13.6.17 RawDstTran 388
13.6.18 RawErr 388
13.6.19 StatusTfr 389
13.6.20 StatusBlock 389
13.6.21 StatusSrcTran 390
13.6.22 StatusDstTran 390
13.6.23 StatusErr 391
13.6.24 MaskTfr 391
13.6.25 MaskBlock 392
13.6.26 MaskSrctran 392
13.6.27 MaskDstTran 393
13.6.28 MaskErr 393
13.6.29 ClearTfr 394
13.6.30 ClearBlock 394
13.6.31 ClearSrcTran 395
13.6.32 ClearDstTran 395
13.6.33 ClearErr 395
13.6.34 StatusInt 396
13.6.35 ReqSrcReg 397
13.6.36 ReqDstReg 397
13.6.37 SglRqSrcReg 398
13.6.38 SglRqDstReg 398
13.6.39 LstSrcReg 399
13.6.40 LstDstReg 400

13.6.41 DmaCfgReg 400
13.6.42 ChEnReg 401
14 通用定时器（GPT0~4) 402
14.1 GPT 简介 402
14.2 GPT 主要特性（每个 GPT） 402
14.3 GPT 逻辑实现 403
14.4 GPT 功能描述 404
14.4.1 GPT 默认配置 404
14.4.2 Timer 模式的一般用法 405
14.4.3 PWM 模式的一般用法 405
14.4.4 CAPTURE 模式的一般用法 405
14.4.5 ETR/软件同步功能 408
14.5 GPT 应用示例 410
14.5.1 HALL 电机应用 410
14.6 GPT 寄存器概览 412
14.7 GPT 寄存器描述 413
14.7.1 GENE_CR 413
14.7.2 SYNC_SELR 416
14.7.3 IN_SELR 418
14.7.4 TSR 418
14.7.5 TSR_DIV 419
14.7.6 EAR 419
14.7.7 EAR_CMP 420
14.7.8 TSR_PHS 420
14.7.9 ECR0 420
14.7.10 ECR1 421
14.7.11 ECR2 421
14.7.12 ECR3 421
14.7.13 PWM_PRIOD 422
14.7.14 PWM_DUTY 422
14.7.15 ECSEQR 422
14.7.16 FTCR 423
14.7.17 CCR0 423
14.7.18 CCR1 425
14.7.19 INT_EN 426
14.7.20 INT_RAW 428

14.7.21 INT_STAT 428
14.7.22 INT_CLR 429
14.7.23 INT_INJ 430
15 正交解码模块（QDM） 431
15.1 QDM 简介 431
15.2 QDM 主要特性 431
15.3 QDM 逻辑实现 431
15.3.1 QDM 与引脚、对端器件的连接 432
15.3.2 QDM 与片内逻辑的连接 434
15.3.3 QDM 逻辑框图 434
15.4 QDM 功能描述 439
15.4.1 QDM 默认配置 439
15.4.2 配置解码模式 440
15.4.3 选择速度计算方法 440
15.4.4 电机参数检查 440
15.4.5 ΔX/T 计算速度 441
15.4.6 X/ΔT 计算速度 441
15.5 QDM 寄存器概览 442
15.6 QDM 寄存器描述 443
15.6.1 GENE_CTRL0 443
15.6.2 GENE_CTRL1 443
15.6.3 POS_CNT 444
15.6.4 POS_INIT 444
15.6.5 POS_MAX 445
15.6.6 POS_CMP 445
15.6.7 POSI_LAT 445
15.6.8 POSS_LAT 446
15.6.9 POS_LAT 446
15.6.10 POS_CTRL 446
15.6.11 UTMR 447
15.6.12 UPRD 447
15.6.13 WDTMR 448
15.6.14 WDPRD 448
15.6.15 QD_CTRL 448
15.6.16 QEP_CTRL 450
15.6.17 QEP_STAT 452

15.6.18 CAP_CTRL 452
15.6.19 CAP_TMR 453
15.6.20 CAP_PRD 453
15.6.21 CAP_TMRLAT 454
15.6.22 CAP_PRDLAT 454
15.6.23 STRB_SELR 455
15.6.24 QMA_CTRL 455
15.6.25 INT_EN 455
15.6.26 INT_RAW 457
15.6.27 INT_STAT 458
15.6.28 INT_CLR 459
15.6.29 INT_INJ 460
16 高级定时器（APT） 462
16.1 APT 简介 462
16.2 APT 主要特性 462
16.3 APT 逻辑实现 462
16.3.1 APT 与引脚、对端器件的连接 464
16.3.2 APT 与片内逻辑的连接 464
16.3.3 APT 单通道逻辑框图 464
16.4 APT 功能描述 467
16.4.1 Time_Gen（TG）子模块 467
16.4.2 Function_Control（FC）子模块 470
16.4.3 Dead-Band（DB）子模块 472
16.4.4 Trip_Zone（TZ）子模块 473
16.4.5 Event_Trigger（ET）子模块 475
16.4.6 SYNC_MUX 子模块 476
16.4.7 Blank_gen 子模块 477
16.5 APT 寄存器概览 477
16.6 APT 寄存器详述 479
16.6.1 TBCTL 479
16.6.2 TBSTS 481
16.6.3 TBPHS 482
16.6.4 TBCTR 482
16.6.5 TBPRD 483
16.6.6 CMPCTL 483
16.6.7 CMPA 484

16.6.8 CMPB 484
16.6.9 CPR1 485
16.6.10 CPR2 485
16.6.11 AQCTLA 485
16.6.12 AQCTLB 487
16.6.13 AQSFRC 488
16.6.14 AQCSFRC 489
16.6.15 DBCTL 490
16.6.16 DBRED 490
16.6.17 DBFED 491
16.6.18 BLKCTLA 491
16.6.19 BLKCTLB 492
16.6.20 TZSEL 492
16.6.21 TZCTL 495
16.6.22 TZEINT 498
16.6.23 TZFLG 498
16.6.24 TZCLR 499
16.6.25 TZFRC 499
16.6.26 ETSEL 500
16.6.27 ETPS 501
16.6.28 ETFLG 502
16.6.29 ETCLR 503
16.6.30 ETFRC 504
16.6.31 TBCTL_RO 504
16.6.32 TBCTL_W1P 505
16.6.33 TBSTS_CLR 505
16.6.34 TBCTR_RW 506
16.6.35 AQSFRC_W1P 506
16.6.36 ETPS_RO 507
17 窗口看门狗（WWDG） 508
17.1 WWDG 简介 508
17.2 WWDG 主要特性 508
17.3 WWDG 功能描述 509
17.4 WWDG 使用流程 509
17.5 WWDG 寄存器概述 512
17.6 WWDG 寄存器描述 513

17.6.1 WDT_CTR 513
17.6.2 WDT_SCALE 513
17.6.3 WDT_SRV 514
17.6.4 WDT_WLB 514
17.6.5 WDT_WUB 515
17.6.6 WDT_LOCK 515
17.6.7 WDT_COUNT 516
17.6.8 WDT_STATUS 516
18 独立看门狗（IWDG） 517
18.1 IWDG 简介 517
18.2 IWDG 主要特性 517
18.3 IWDG 功能描述 517
18.4 IWDG 典型配置 519
18.4.1 计数初值配置 519
18.4.2 系统初始化配置 519
18.4.3 中断处理过程 520
18.4.4 关闭 WatchDog 520
18.5 IWDG 寄存器概览 520
18.6 IWDG 寄存器描述 521
18.6.1 WDG_LOAD 521
18.6.2 WDG_VALUE 521
18.6.3 WDG_CONTROL 522
18.6.4 WDG_INTCLR 522
18.6.5 WDG_RIS 522
18.6.6 WDG_MIS 523
18.6.7 WDG_LOCK 523
19 键盘扫描模块（KEYSCAN） 525
19.1 KEYSCAN 简介 525
19.2 KEYSCAN 主要特性 525
19.3 KEYSCAN 功能说明 525
19.3.1 概述 525
19.3.2 按键识别功能 527
19.3.3 按键扫描功能 527
19.3.4 注意事项 527
19.4 KEYSCAN 寄存器概览 528
19.5 KEYSCAN 寄存器描述 529

19.5.1 SCAN_CTRL 529
19.5.2 DEBOUNCE_NUM 529
19.5.3 T_LOW_LEVEL 530
19.5.4 SCAN_EN 531
19.5.5 SCAN_STAT 531
19.5.6 KEYPRESS_IER 532
19.5.7 KEYPRESS_CLR 534
19.5.8 KEYPRESS_STAT 535
19.5.9 KEYPRESS_RAW 537
19.5.10 KEYOUT_SEL 540
20 触摸传感控制器（TSC） 543
20.1 TSC 简介 543
20.2 TSC 主要特性 543
20.3 TSC 逻辑实现 543
20.4 TSC 连接关系 545
20.5 TSC 功能描述 545
20.5.1 TSC 通道轮询 545
20.5.2 TSC 轮询模式 546
20.5.3 TSC 消抖 546
20.5.4 TSC 中断 546
20.6 TSC 数模控制 547
20.7 TSC 配置流程 547
20.8 TSC 寄存器概览 548
20.9 TSC 寄存器描述 549
20.9.1 TOUCH_ANA_CTRL0 549
20.9.2 TS_CNT0 550
20.9.3 TS_MAX 551
20.9.4 TS_THD 551
20.9.5 CCR0 551
20.9.6 CCR1 552
20.9.7 SHD_REC 552
20.9.8 ERR_REC 553
20.9.9 NORM_REC 553
20.9.10 INT_EN 554
20.9.11 INT_RAW 556
20.9.12 INT_STAT 558

20.9.13 INT_CLR 561
20.9.14 INT_INJ 563
20.9.15 VEC_CFG 564
20.9.16 VEC_REC 565
21 同步串行外设接口（SPI） 566
21.1 SPI 简介 566
21.2 SPI 主要特性 566
21.3 SPI 逻辑实现 567
21.4 SPI 功能描述 570
21.4.1 发送与接收传输模式 570
21.4.2 主设备模式与从设备模式 570
21.4.3 RX 采样延迟 572
21.4.4 DMA 控制器数据传输 573
21.5 SPI 接口时序 576
21.5.1 Motorola SPI 接口 576
21.5.2 TI 同步串行接口 581
21.5.3 Microwire 接口 581
21.6 SPI 工作方式 585
21.6.1 工作模式 585
21.6.2 时钟与复位 585
21.6.3 中断信号处理 585
21.6.4 初始化 585
21.6.5 轮询模式下的数据传输 586
21.6.6 中断模式下的数据传输 586
21.6.7 DMA 模式下的数据传输 586
21.7 SPI 寄存器概览 587
21.8 SPI 寄存器描述 588
21.8.1 CTRLR0 588
21.8.2 CTRLR1 591
21.8.3 SSIENR 591
21.8.4 MWCR 592
21.8.5 SER 593
21.8.6 BAUDR 593
21.8.7 TXFTLR 594
21.8.8 RXFTLR 594
21.8.9 TXFLR 594

21.8.10 RXFLR 595
21.8.11 SR 595
21.8.12 IMR 596
21.8.13 ISR 597
21.8.14 RISR 597
21.8.15 TXOICR 598
21.8.16 RXOICR 598
21.8.17 RXUICR 599
21.8.18 MSTICR 599
21.8.19 ICR 600
21.8.20 DMACR 600
21.8.21 DMATDLR 601
21.8.22 DMARDLR 601
21.8.23 IDR 601
21.8.24 SSI_VERSION_ID 602
21.8.25 DR 602
21.8.26 RX_SAMPLE_DLY 602
22 扩展同步串行外设接口（QSPI） 604
22.1 QSPI 简介 604
22.2 QSPI 主要特性 604
22.3 QSPI 逻辑实现 605
22.4 QSPI 功能描述 605
22.4.1 时钟与复位 605
22.4.2 标准 SPI 模式 605
22.4.3 增强型 SPI 模式 606
22.5 QSPI 接口时序 607
22.5.1 增强型 SPI 接口时序 607
22.5.2 初始化配置流程 610
22.6 QSPI 寄存器概览 611
22.7 QSPI 寄存器描述 612
22.7.1 CTRLR0 612
22.7.2 CTRLR1 615
22.7.3 SSIENR 615
22.7.4 MWCR 616
22.7.5 SER 616
22.7.6 BAUDR 617

22.7.7 TXFTLR 617
22.7.8 RXFTLR 618
22.7.9 TXFLR 618
22.7.10 RXFLR 619
22.7.11 SR 619
22.7.12 IMR 620
22.7.13 ISR 621
22.7.14 RISR 622
22.7.15 TXOICR 622
22.7.16 RXOICR 623
22.7.17 RXUICR 623
22.7.18 MSTICR 624
22.7.19 ICR 624
22.7.20 DMACR 624
22.7.21 DMATDLR 625
22.7.22 DMARDLR 625
22.7.23 IDR 626
22.7.24 SSI_VERSION_ID 626
22.7.25 DR 627
22.7.26 RX_SAMPLE_DLY 627
22.7.27 QSPI_CTRLR0 627
22.7.28 XIP_MODE_BITS 629
22.7.29 XIP_INCR_INST 630
22.7.30 XIP_WRAP_INST 630
22.7.31 XIP_CTRL 631
22.7.32 XIP_SER 633
22.7.33 XRXOICR 633
22.7.34 XIP_CNT_TIME_OUT 634
23 集成电路接口（I2C） 635
23.1 I2C 简介 635
23.2 I2C 主要特性 635
23.3 I2C 功能描述 636
23.3.1 I2C 时钟 636
23.3.2 I2C 工作模式 636
23.3.3 I2C 总线行为与模块实现 636
23.3.4 SCL/SDA 输入毛刺过滤 638

23.3.5 可选的 Slave 地址 639
23.3.6 总线清除功能 639
23.3.7 SCL 高低电平配置 639
23.3.8 SDA 保持时间 642
23.3.9 I2C 接口时序 644
23.4 SMBus 功能 645
23.4.1 时钟扩展功能 645
23.4.2 SMBus 协议 646
23.4.3 SMBus 地址解析协议（ARP） 648
23.4.4 SMBus 永久 Slave 与非永久 Slave 649
23.4.5 SMBus 告警信号 649
23.5 中断寄存器 650
23.6 工作方式 651
23.6.1 Master 在中断方式下的数据传输 651
23.6.2 Master 在轮询方式下的数据传输 652
23.6.3 Master 在 DMA 模式下的写操作 654
23.6.4 Master 在 DMA 模式下的读操作 654
23.6.5 Slave 在中断方式下的数据传输 655
23.6.6 Slave 在轮询方式下的数据传输 656
23.6.7 Slave 在 DMA 模式下的写操作 657
23.6.8 Slave 在 DMA 模式下的读操作 658
23.6.9 SCL 和 SDA 总线恢复流程 658
23.6.10 Master 模式下 SMBus 超时处理流程 659
23.6.11 Slave 模式下 SMBus 超时处理流程 660
23.6.12 Host 模式下的 SMBus Alert 信号处理 660
23.6.13 Device 模式下的 SMBus Alert 信号处理 662
23.7 I2C 寄存器概览 663
23.8 I2C 寄存器描述 665
23.8.1 CON 665
23.8.2 TAR 668
23.8.3 SAR 669
23.8.4 DATA_CMD 669
23.8.5 SS_SCL_HCNT 670
23.8.6 SS_SCL_LCNT 671
23.8.7 FS_SCL_HCNT 671
23.8.8 FS_SCL_LCNT 671

23.8.9 INTR_STAT 672
23.8.10 INTR_MASK 673
23.8.11 RAW_INTR_STAT 675
23.8.12 RX_TL 677
23.8.13 TX_TL 678
23.8.14 CLR_INTR 678
23.8.15 CLR_RX_UNDER 679
23.8.16 CLR_RX_OVER 679
23.8.17 CLR_TX_OVER 679
23.8.18 CLR_RD_REQ 680
23.8.19 CLR_TX_ABRT 680
23.8.20 CLR_RX_DONE 680
23.8.21 CLR_ACTIVITY 681
23.8.22 CLR_STOP_DET 681
23.8.23 CLR_START_DET 682
23.8.24 CLR_GEN_CALL 682
23.8.25 ENABLE 682
23.8.26 STATUS 684
23.8.27 TXFLR 686
23.8.28 RXFLR 686
23.8.29 SDA_HOLD 687
23.8.30 TX_ABRT_SOURCE 687
23.8.31 DMA_CR 690
23.8.32 DMA_TDLR 690
23.8.33 DMA_RDLR 691
23.8.34 SDA_SETUP 691
23.8.35 ACK_GENERAL_CALL 692
23.8.36 ENABLE_STATUS 692
23.8.37 FS_SPKLEN 694
23.8.38 CLR_RESTART_DET 694
23.8.39 SCL_STUCK_AT_LOW_TIMEOUT 694
23.8.40 SDA_STUCK_AT_LOW_TIMEOUT 695
23.8.41 CLR_SCL_STUCK_DET 695
23.8.42 DEVICE_ID 696
23.8.43 SMBUS_CLK_LOW_SEXT 696
23.8.44 SMBUS_CLK_LOW_MEXT 697

23.8.45 SMBUS_THIGH_MAX_IDLE_COUNT 697
23.8.46 SMBUS_INTR_STAT 698
23.8.47 SMBUS_INTR_MASK 699
23.8.48 SMBUS_RAW_INTR_STAT 700
23.8.49 CLR_SMBUS_INTR 702
23.8.50 OPTIONAL_SAR 703
23.8.51 SMBUS_UDID_LSB 703
24 集成电路内置音频总线（I2S） 705
24.1 I2S 简介 705
24.2 I2S 主要特性 705
24.3 I2S 逻辑实现 706
24.4 I2S 功能描述 708
24.4.1 I2S 接口时序 708
24.4.2 主模式和从模式选择 709
24.4.3 I2S 模块全局使能 709
24.4.4 I2S 发送功能 709
24.4.5 I2S 接收功能 711
24.4.6 时钟产生（仅主模式） 712
24.4.7 DMA 功能 712
24.4.8 全双工和半双工控制 713
24.4.9 门控延迟功能 713
24.5 I2S 配置流程 713
24.5.1 I2S 普通发送 714
24.5.2 I2S 普通接收 714
24.5.3 I2S DMA 模式发送 715
24.5.4 I2S DMA 模式接收 715
24.6 I2S 寄存器概览 716
24.7 I2S 寄存器描述 718
24.7.1 IER 718
24.7.2 IRER 718
24.7.3 ITER 718
24.7.4 CER 719
24.7.5 CCR 720
24.7.6 RXFFR 720
24.7.7 TXFFR 721
24.7.8 LRBRx 721

24.7.9 LTHRx 722
24.7.10 RRBRx 722
24.7.11 RTHRx 723
24.7.12 RERx 723
24.7.13 TERx 724
24.7.14 RCRx 724
24.7.15 TCRx 725
24.7.16 ISRx 725
24.7.17 IMRx 726
24.7.18 RORx 727
24.7.19 TORx 727
24.7.20 RFCRx 728
24.7.21 TFCRx 729
24.7.22 RFFx 729
24.7.23 TFFx 730
24.7.24 I2S_COMP_PARAM_2 731
24.7.25 I2S_COMP_PARAM_1 731
24.7.26 I2S_COMP_VERSION 732
24.7.27 I2S_COMP_TYPE 732
24.7.28 DMACR 732
24.7.29 RXDMA_CHx 733
24.7.30 TXDMA_CHx 734
24.7.31 EXT_CTRL 734
24.7.32 GATECLKDLY 735
25 通用异步收发传输器（UART） 737
25.1 UART 简介 737
25.2 UART 主要特性 737
25.3 UART 逻辑实现 738
25.4 UART 功能描述 739
25.4.1 UART 串行协议 739
25.4.2 9-bit 数据传输 739
25.4.3 RS485 串行协议 740
25.4.4 小数分频 741
25.4.5 IrDA1.0 SIR 协议 741
25.4.6 中断功能 742
25.4.7 自动流控 743

25.4.8 DMA 功能 745
25.4.9 时钟与复位 745
25.5 UART 配置流程 746
25.5.1 典型发送配置操作流程 746
25.5.2 典型接收配置操作流程 746
25.5.3 9-bit 数据格式发送模式 0 748
25.5.4 9-bit 数据格式发送模式 1 749
25.5.5 9-bit 数据格式接收硬件地址匹配模式 750
25.5.6 9-bit 数据格式接收软件地址匹配模式 751
25.6 UART 寄存器概览 751
25.7 UART 寄存器详细描述 753
25.7.1 RBR 753
25.7.2 THR 754
25.7.3 DLL 754
25.7.4 DLH 755
25.7.5 IER 755
25.7.6 FCR 756
25.7.7 IIR 757
25.7.8 LCR 758
25.7.9 MCR 759
25.7.10 LSR 761
25.7.11 MSR 763
25.7.12 SCR 764
25.7.13 MISC_CTRL 765
25.7.14 SRBRn 766
25.7.15 STHRn 767
25.7.16 FAR 767
25.7.17 TFR 768
25.7.18 RFW 768
25.7.19 USR 769
25.7.20 TFL 770
25.7.21 RFL 770
25.7.22 SRR 771
25.7.23 SRTS 771
25.7.24 SBCR 772
25.7.25 Reserved 772

25.7.26 SFE 773
25.7.27 SRT 773
25.7.28 STET 773
25.7.29 HTX 774
25.7.30 DMASA 774
25.7.31 TCR 775
25.7.32 DE_EN 776
25.7.33 RE_EN 776
25.7.34 DET 777
25.7.35 TAT 777
25.7.36 DLF 778
25.7.37 RAR 778
25.7.38 TAR 779
25.7.39 LCR_EXT 779
26 控制器局域网总线（CAN） 781
26.1 CAN 简介 781
26.2 CAN 主要特性 781
26.3 CAN 逻辑实现 781
26.4 CAN 功能描述 782
26.4.1 工作模式 782
26.4.2 自接收功能 784
26.4.3 消息过滤 785
26.4.4 波特率和采样设置 787
26.4.5 总线仲裁 788
26.4.6 错误处理 788
26.5 CAN 寄存器概览 789
26.6 CAN 寄存器描述 790
26.6.1 MOD 790
26.6.2 CMR 791
26.6.3 SR 791
26.6.4 IR 793
26.6.5 IER 793
26.6.6 Reserved 794
26.6.7 BTR0 795
26.6.8 BTR1 795
26.6.9 OCR 796

26.6.10 Reserved 796
26.6.11 Reserved 797
26.6.12 ALC 797
26.6.13 ECC 798
26.6.14 EWLR 799
26.6.15 RXERR 800
26.6.16 TXERR 800
26.6.17 FRAME_INFO_ACR0 800
26.6.18 ID1_ACR1 801
26.6.19 ID2_ACR2 802
26.6.20 ID3_ACR3 802
26.6.21 ID4_AMR0 803
26.6.22 Data1_AMR1 803
26.6.23 Data2_AMR2 804
26.6.24 Data3_AMR3 804
26.6.25 Data4 805
26.6.26 Data5 805
26.6.27 Data6 806
26.6.28 Data7 806
26.6.29 Data8 806
26.6.30 RMC 807
26.6.31 RBSA 807
26.6.32 CDR 808
26.6.33 RX_FIFO 808
26.6.34 TX_BUFF 809
27 通用串行总线（USB） 810
27.1 USB_FS 简介 810
27.2 USB_FS 主要特性 810
27.2.1 通用特性 810
27.2.2 主机模式特性 811
27.2.3 设备模式特性 811
27.3 USB_FS 逻辑实现 811
27.3.1 USB_FS 框图 811
27.3.2 USB_FS PHY 812
27.3.3 USB_FS 控制器 812
27.4 USB_FS 功能描述 813

27.4.1 双角色设备(DRD) 813
27.4.2 USB_FS 设备模式 814
27.4.3 USB_FS 主机模式 817
27.4.4 USB_FS SOF 触发 820
27.4.5 低功耗模式 821
27.4.6 动态更新 HFIR 寄存器 821
27.4.7 数据 FIFO 822
27.4.8 系统性能 827
27.4.9 中断 827
27.5 USB_FS 编程流程 828
27.5.1 控制器全局初始化 828
27.5.2 主机初始化 829
27.5.3 设备初始化 829
27.5.4 主机编程流程 830
27.5.5 设备编程流程 832
27.6 USB_FS 寄存器概览 834
27.7 USB_FS 寄存器 838
27.7.1 GOTGCTL 838
27.7.2 GOTGINT 839
27.7.3 GAHBCFG 839
27.7.4 GUSBCFG 840
27.7.5 GRSTCTL 842
27.7.6 GINTSTS 845
27.7.7 GINTMSK 850
27.7.8 GRXSTSR 853
27.7.9 GRXSTSP 855
27.7.10 GRXFSIZ 856
27.7.11 GNPTXFSIZ 856
27.7.12 GNPTXSTS 857
27.7.13 GSNPSID 858
27.7.14 GHWCFG1 858
27.7.15 GHWCFG2 859
27.7.16 GHWCFG3 861
27.7.17 GHWCFG4 863
27.7.18 GDFIFOCFG 864
27.7.19 HPTXFSIZ 865

27.7.20 DIEPTXF1 865
27.7.21 DIEPTXF2 866
27.7.22 DIEPTXF3 866
27.7.23 DIEPTXF4 867
27.7.24 DIEPTXF5 867
27.7.25 HCFG 868
27.7.26 HFIR 869
27.7.27 HFNUM 870
27.7.28 HPTXSTS 870
27.7.29 HAINT 871
27.7.30 HAINTMSK 872
27.7.31 HPRT 873
27.7.32 HCCHAR(n) 875
27.7.33 HCINT(n) 876
27.7.34 HCINTMSK(n) 878
27.7.35 HCTSIZ(n) 879
27.7.36 DCFG 880
27.7.37 DCTL 881
27.7.38 DSTS 883
27.7.39 DIEPMSK 884
27.7.40 DOEPMSK 885
27.7.41 DAINT 887
27.7.42 DAINTMSK 888
27.7.43 DTHRCTL 889
27.7.44 DIEPEMPMSK 890
27.7.45 DIEPCTL(m) 891
27.7.46 DIEPINT(m) 894
27.7.47 DIEPTSIZ(m) 896
27.7.48 DTXFSTS(m) 896
27.7.49 DOEPCTL(m) 897
27.7.50 DOEPINT(m) 900
27.7.51 DOEPTSIZ(m) 902
27.7.52 PCGCCTL 903
27.7.53 PCGCCTL1 904
27.7.54 EP0/CH0 FIFO 904
27.7.55 EP1/CH1 FIFO 905

27.7.56 EP2/CH2 FIFO 905
27.7.57 EP3/CH3 FIFO 905
27.7.58 EP4/CH4 FIFO 906
27.7.59 EP5/CH5 FIFO 906
27.7.60 CH6 FIFO 906
27.7.61 CH7 FIFO 907
27.7.62 CH8 FIFO 907
27.7.63 CH9 FIFO 907
27.7.64 CH10 FIFO 908
27.7.65 CH11 FIFO 908
27.7.66 U2PHY_LS_CTRL 908
28 安全数字输入输出接口（SDIO） 910
28.1 SDIO 简介 910
28.2 SDIO 主要特性 910
28.3 SDIO 功能说明 910
28.3.1 SDIO 框图 910
28.3.2 SDIO 端口 911
28.3.3 命令与响应 912
28.3.4 数据传输 915
28.4 SDIO 工作方式 917
28.4.1 SDIO Host 复位和时钟配置 917
28.4.2 SDIO Host 初始化 918
28.4.3 无数据传输命令 919
28.4.4 单块或多块读数据 919
28.4.5 单块或多块写数据 920
28.5 SDIO 寄存器概述 921
28.6 SDIO 寄存器描述 922
28.6.1 SDMASA_R 922
28.6.2 BLOCKSIZE_R 922
28.6.3 BLOCKCOUNT_R 923
28.6.4 ARGUMENT_R 924
28.6.5 XFER_MODE_R 924
28.6.6 CMD_R 925
28.6.7 RESP01_R 926
28.6.8 RESP23_R 926
28.6.9 RESP45_R 926

28.6.10 RESP67_R 927
28.6.11 BUF_DATA_R 927
28.6.12 PSTATE_REG 928
28.6.13 HOST_CTRL1_R 929
28.6.14 PWR_CTRL_R 930
28.6.15 BGAP_CTRL_R 931
28.6.16 WUP_CTRL_R 931
28.6.17 CLK_CTRL_R 932
28.6.18 TOUT_CTRL_R 933
28.6.19 SW_RST_R 933
28.6.20 NORMAL_INT_STAT_R 934
28.6.21 ERROR_INT_STAT_R 935
28.6.22 NORMAL_INT_STAT_EN_R 936
28.6.23 ERROR_INT_STAT_EN_R 937
28.6.24 NORMAL_INT_SIGNAL_EN_R 938
28.6.25 ERROR_INT_SIGNAL_EN_R 939
28.6.26 AUTO_CMD_STAT_R 940
28.6.27 HOST_CTRL2_R 941
28.6.28 CAPABILITIES1_R 942
28.6.29 ADMA_ERR_STAT_R 943
28.6.30 ADMA_SA_LOW_R 944
28.6.31 MSHC_CTRL_R 944
28.6.32 EMMC_CTRL_R 944
29 备份寄存器（BACKUP） 946
29.1 BACKUP 简介 946
29.2 BACKUP 主要特性 946
29.3 BACKUP 功能说明 947
29.3.1 框图 947
29.3.2 BACKUP 时钟结构 948
29.3.3 BACKUP 复位结构 949
29.3.4 寄存器写保护 949
29.3.5 入侵检测 950
29.3.6 定时唤醒器 950
29.3.7 时间戳 950
29.3.8 外部 WKUP 管脚 951
29.3.9 备份数据寄存器 BACKUP_STORE 951

29.3.10 BACKUP 相关的 IO 951
29.4 BACKUP 使用流程 951
29.4.1 初始化流程 951
29.4.2 处理事件记录标志 953
29.4.3 处理唤醒 953
29.4.4 处理中断 953
29.4.5 处理时间戳 954
29.5 BACKUP_CTRL 寄存器概览 954
29.6 BACKUP_CTRL 寄存器描述 956
29.6.1 BACKUP_CRG0 956
29.6.2 BACKUP_CRG1 956
29.6.3 BACKUP_EVENT_CTRL0 958
29.6.4 BACKUP_EVENT_CTRL1 959
29.6.5 BACKUP_EVENT_CTRL2 961
29.6.6 BACKUP_TAMPER_ERASE_TRGWKT_EN 961
29.6.7 BACKUP_TS_EN 962
29.6.8 BACKUP_WKT_CFG 963
29.6.9 BACKUP_INT_RAW 964
29.6.10 BACKUP_INT_CLR 965
29.6.11 BACKUP_INT_EN 965
29.6.12 BACKUP_INT_STATUS 967
29.6.13 BACKUP_RECORD_FLAG 968
29.6.14 BACKUP_RECORD_FLAG_CLR 969
29.6.15 BACKUP_WKUPREQ_EN 970
29.6.16 BACKUP_WKUPREQ_STATUS 971
29.6.17 BACKUP_TS_YMDW 972
29.6.18 BACKUP_TS_HMS 972
29.6.19 LSE_32K_CTRL 973
29.6.20 BACKUP_TEST_0 974
29.7 BACKUP_STORE 备份数据寄存器概览 975
29.7.1 BACKUP_STORE 975
29.8 BACKUP_STORE 备份数据寄存器描述 975
29.8.1 BACKUP_STORE 975
30 实时时钟 (RTC) 976
30.1 RTC 简介 976
30.2 RTC 主要特性 976

30.3 RTC 功能说明 976
30.3.1 RTC 框图 976
30.3.2 时间日历 977
30.3.3 可编程闹钟 ALARM 977
30.3.4 RTC 时钟校准 978
30.3.5 RTC 相关的 IO 978
30.4 RTC 使用流程 979
30.4.1 初始化流程 979
30.4.2 时间校准配置流程 979
30.4.3 时间日历使用流程 979
30.4.4 闹钟功能使用流程 979
30.5 RTC 寄存器概览 980
30.6 RTC 寄存器列表 980
30.6.1 RTCCR 980
30.6.2 RTCTR 982
30.6.3 RTCWDA 982
30.6.4 RTCWTA 983
30.6.5 RTCWDC 984
30.6.6 RTCWTC 985
30.6.7 RTCLOCK 986
31 数模转换器（DAC） 987
31.1 DAC 简介 987
31.2 DAC 主要特性 987
31.3 DAC 功能说明 987
31.3.1 DAC 框图 987
31.3.2 DAC 工作时钟 988
31.3.3 DAC 引脚与连接关系 988
31.3.4 DAC 使能 988
31.3.5 DAC 转换 989
31.3.6 DAC 触发选择 989
31.3.7 DMA 数据传输 990
31.3.8 DAC 测试数据生成 991
31.3.9 DAC 参考电压选择 991
31.3.10 DAC 模式选择 991
31.4 DAC 工作方式 992
31.4.1 中断处理 992

31.4.2 测试模式 993
31.4.3 触发模式 993
31.5 DAC 寄存器概览 993
31.6 DAC 寄存器描述 994
31.6.1 DAC_CTRL 994
31.6.2 DAC_DHR 995
31.6.3 DAC_VALUE 995
31.6.4 DAC_CFG 996
31.6.5 DAC_STATE 997
31.6.6 DAC_SCR 998
31.6.7 DAC_SWTRG 999
31.6.8 DAC_COM 1000
32 模数转换器（ADC） 1002
32.1 ADC 简介 1002
32.2 ADC 主要特性 1002
32.3 ADC 性能指标 1003
32.4 ADC 功能 1004
32.4.1 ADC 框图 1004
32.4.2 ADC 时钟 1004
32.4.3 ADC 分辨率 1004
32.4.4 ADC 参考电压 1004
32.4.5 ADC 输入信号 1005
32.4.6 ADC 启动和关闭 1005
32.4.7 ADC 自校准 1006
32.4.8 ADC 通道选择 1007
32.4.9 ADC 采样时间 1007
32.4.10 ADC 采样结果 1008
32.4.11 ADC 转换时间 1008
32.4.12 ADC 外部触发和触发极性的转换 1008
32.4.13 ADC 注入通道采样 1009
32.4.14 ADC 单次转换模式 1010
32.4.15 ADC 连续转换模式 1011
32.4.16 ADC 非连续转换模式 1012
32.4.17 ADC 启动转换 1013
32.4.18 ADC 停止正在进行中的转换 1014
32.4.19 ADC 注入组转换队列功能 1014

32.4.20 ADC 单通道转换结束 1021
32.4.21 ADC 转换序列结束 1022
32.4.22 ADC 数据管理 1022
32.4.23 ADC 自动延迟转换功能 1023
32.4.24 ADC DMA 搬数 1023
32.4.25 ADC 模拟看门狗 1024
32.4.26 ADC 过采样 1026
32.4.27 温度传感器 1027
32.4.28 双 ADC 模式 1028
32.4.29 ADC 中断 1029
32.5 ADC 寄存器概述 1030
32.6 ADC 寄存器描述 1032
32.6.1 ADC_ISR 1032
32.6.2 ADC_ISR_CLR 1034
32.6.3 ADC_IER 1035
32.6.4 ADC_CSR 1038
32.6.5 ADC_CMDR 1039
32.6.6 ADC_CFGR 1040
32.6.7 ADC_OVSR 1043
32.6.8 ADC_SMPR0 1045
32.6.9 ADC_SMPR1 1045
32.6.10 ADC_SMPR2 1046
32.6.11 ADC_SMPR3 1046
32.6.12 ADC_SMPR4 1046
32.6.13 ADC_CVS 1047
32.6.14 ADC_TR1 1047
32.6.15 ADC_TR2 1048
32.6.16 ADC_TR3 1049
32.6.17 ADC_SQRL 1049
32.6.18 ADC_SQR1 1050
32.6.19 ADC_SQR2 1051
32.6.20 ADC_SQR3 1051
32.6.21 ADC_SQR4 1052
32.6.22 ADC_DR 1052
32.6.23 ADC_JSQR 1053
32.6.24 ADC_CUR_JSQR 1054

32.6.25 ADC_JDR1 1056
32.6.26 ADC_JDR2 1056
32.6.27 ADC_JDR3 1057
32.6.28 ADC_JDR4 1057
32.6.29 ADC_AWD2CR 1058
32.6.30 ADC_AWD3CR 1058
32.6.31 ADC_CALFACT_AUTO 1059
32.6.32 ADC_CALFACT_SOFT 1059
32.6.33 ADC_DIFSEL 1060
32.6.34 ADC_SEL_CAL_CFG 1060
32.6.35 ADC_ANA_CTRL 1061
32.7 ADC_DUAL 寄存器描述 1062
32.7.1 ADC_COM_ISR 1062
32.7.2 ADC_COM_CSR 1063
32.7.3 ADC_COM_DR 1064
33 可编程增益放大器（PGA） 1065
33.1 PGA 简介 1065
33.2 PGA 主要特性 1065
33.3 PGA 结构框图 1065
33.4 PGA 通道选择及输出连接 1066
33.5 PGA 输入摆幅选择 1067
33.6 PGA 增益配置及内外电阻模式选择 1068
33.7 PGA 寄存器概述 1068
33.8 PGA 寄存器描述 1068
33.8.1 PGA0_CTRL 1068
33.8.2 PGA1_CTRL 1070
33.8.3 PGA2_CTRL 1071
33.8.4 PGA3_CTRL 1073
34 模拟比较器（ACMP） 1075
34.1 ACMP 简介 1075
34.2 ACMP 主要特性 1075
34.3 ACMP 功能说明 1075
34.3.1 ACMP 框图 1076
34.3.2 ACMP 输入源选择 1076
34.3.3 ACMP 比较极性选择 1077
34.3.4 ACMP 迟滞电压选择 1077

34.3.5 窗口比较器功能 1078
34.3.6 过压欠压检测 1078
34.3.7 比较器消隐功能 1079
34.3.8 去抖功能 1079
34.4 ACMP 配置流程 1080
34.5 ACMP 寄存器概览 1080
34.6 ACMP 寄存器描述 1081
34.6.1 CMP(m)_CTRL 1081
34.6.2 CMP(m)_DCR 1084
34.6.3 CMP_OUT 1085
35 其它模拟 IP 寄存器 1086
35.1 寄存器概览 1086
35.2 寄存器描述 1086
35.2.1 VREFBUF_CTRL 1086
35.2.2 IBIAS_SEL_2 1088
35.2.3 ADC_BGR_CTRL 1088
36 调试接口（DBG) 1090
36.1 DBG 简介 1090
36.2 DBG 主要特性 1090
36.3 DBG 框图 1090
36.4 DBG 引脚 1091
36.4.1 JTAG 5 线接口引脚 1091
36.4.2 二线串行调试接口引脚 1091
37 器件电子签名 1093
37.1 简介 1093
37.2 唯一器件标识符寄存器（UID） 1093
37.3 Flash 大小数据寄存器 1094

插图目录

图 1-1 系统架构 51
图 1-2 总线互联矩阵 52
图 2-1 工作模式及寄存器视图 62
图 4-1 时钟系统框图 139
图 7-1 GPIO 模块框图 201
图 11-1 芯片电源域示意图 339
图 11-2 低功耗工作模式 341
图 12-1 CRC 结构图 355
图 13-1 DMAC 逻辑框图 359
图 13-2 软件握手框图 365
图 13-3 非外设流控 368
图 13-4 外设流控 368
图 13-5 硬件握手时序 369
图 13-6 链表更新通道寄存器示意图 370
图 13-7 链表数据结构及节点链接 370
图 14-1 GPT 逻辑结构图 403
图 14-2 滤波模式示意图 406
图 14-3 输入分频示意图 406
图 14-4 霍尔传感器电机输出波形 410
图 15-1 正交编码器输出波形示意图 432
图 15-2 脉冲-方向编码输出波形示意图 433
图 15-3 脉冲-递增/递减编码器输出波形示意图 433
图 15-4 非标准模式 1 编码器输出波形示意图 433
图 15-5 非标准模式 2 编码器输出波形示意图 434
图 15-6 QDM 逻辑框图 435
图 15-7 滤波模式示意图（OUTPUT 为数字滤波器输出） 435
图 15-8 正交模式解码分辨率示意图 436
图 15-9 脉冲-方向模式解码分辨率示意图 437

图 16-1 APT 集成连接关系示意图 463
图 16-2 APT 单个通道——EPWM 单元逻辑框图 465
图 16-3 Time_Gen 模块逻辑框图 467
图 16-4 Time_Gen 计数分频示意图 469
图 16-5 DB 模块结构图 472
图 16-6 死区插入效果示例 473
图 16-7 刹车功能各模式对 PWM 输出的影响 474
图 16-8 消隐信号（blanking）与 pwm 信号的关系 477
图 17-1 WWDG 框图 509
图 17-2 WWDG 软件操作流程 509
图 18-1 IWDG 框图 518
图 19-1 KEYSCAN 概述图 526
图 20-1 TSC 逻辑框图 543
图 20-2 定长检测原理 547
图 20-3 快速检测原理 547
图 21-1 SPI 框图 568
图 21-2 主模式下 Master 与多 Slave 外设通信 571
图 21-3 从模式下多 Slave 与 Master 通信 572
图 21-4 RXD 采样延迟示意图 573
图 21-5 DMA 传输数据量为突发长度整数倍 574
图 21-6 DMA 传输数据量为非突发长度整数倍 574
图 21-7 发送方向 FIFO 水线配置 575
图 21-8 接收方向 FIFO 水线配置 576
图 21-9 SPI 接口时序概览图 577
图 21-10 SPI 单帧帧格式（SPO=0、SPH=0） 577
图 21-11 SPI 连续帧帧格式（SPO=0、SPH=0） 577
图 21-12 SPI 单帧帧格式（SPO=0、SPH=1） 578
图 21-13 SPI 连续帧帧格式（SPO=0、SPH=1） 578
图 21-14 SPI 单帧帧格式（SPO=1、SPH=0） 579
图 21-15 SPI 连续帧帧格式（SPO=1、SPH=0） 579
图 21-16 SPI 单帧帧格式（SPO=1、SPH=1） 580
图 21-17 SPI 连续帧帧格式（SPO=1、SPH=1） 580
图 21-18 TI 同步串行单帧帧格式 581
图 21-19 TI 同步串行连续帧帧格式 581

图 21-20 Microwire Non-sequential 模式连续读操作（MDD=0，MWMOD=0） 582
图 21-21 Microwire Sequential 模式连续读操作（MDD=0，MWMOD=1） 582
图 21-22 Microwire 连续写操作（MDD=1，MWMOD=0） 583
图 21-23 Microwire 带握手功能连续写操作（MWHS=1，MDD=1，MWMOD=0） 583
图 21-24 单次读操作（MWHS=0，MDD=0，MWMOD=0） 584
图 21-25 单次写操作（MWHS=0，MDD=1，MWMOD=0） 584
图 22-1 典型的 2/4 线增强型 SPI 模式写操作 608
图 22-2 指令和地址采用 Standard SPI 格式传输 608
图 22-3 指令采用 Standard SPI，地址采用增强型 SPI 传输 608
图 22-4 只传输指令模式 609
图 22-5 指令和地址都采用 Standard SPI 帧格式 609
图 22-6 指令采用 Standard SPI 帧格式，地址采用增强 SPI 格式 609
图 22-7 指令和地址都采用增强 SPI 帧格式 610
图 22-8 无指令与地址读操作 610
图 23-1 I2C 总线协议图 637
图 23-2 7bit 地址格式 637
图 23-3 滤毛刺示例 639
图 23-4 SCL 上升时间和下降时间对生成 SCL 的影响 641
图 23-5 SDA_HOLD 寄存器 642
图 23-6 块接收数据时，SDA_HOLD.ic_sda_rx_hold 配置>=3 643
图 23-7 模块接收数据时，SDA_HOLD.ic_sda_rx_hold 配置为 2 643
图 23-8 Master 写操作时序 644
图 23-9 Master 读操作时序 644
图 23-10 Master 和 Slave 的时钟扩展超时间隔 646
图 23-11 中断方案 651
图 23-12 Host 模式下的 SMBus Alert 信号处理 661
图 23-13 Device 模式下的 SMBus Alert 信号处理 662
图 24-1 I2S 功能框图 707
图 24-2 主从设计对接 708
图 24-3 帧格式 709
图 24-4 ws 信号宽度 712
图 25-1 UART 框图 738
图 25-2 串行数据格式 739
图 25-3 9-bit 数据格式 740

图 25-4 IrDA SIR 数据格式 742
图 25-5 自动流控框图 744
图 25-6 典型发送 746
图 25-7 典型接收 747
图 25-8 硬件发送地址 748
图 25-9 普通 9bit 发送 749
图 25-10 硬件地址匹配 750
图 25-11 软件地址匹配 751
图 26-1 CAN 模块框图 782
图 26-2 标准帧格式单过滤 ID 模式 786
图 26-3 标准帧格式双过滤 ID 模式 786
图 26-4 扩展帧格式单过滤 ID 模式 786
图 26-5 扩展帧格式双过滤 ID 模式 787
图 26-6 CAN 的比特周期结构图 787
图 27-1 全速 USB 模块框图 812
图 27-2 USB_FS 双角色设备连接 813
图 27-3 动态更新 HFIR 821
图 27-4 非动态更新 HFIR 822
图 27-5 设备模式下的 FIFO 地址映射 823
图 27-6 主机模式下的 FIFO 地址映射 824
图 27-7 中断汇聚 828
图 28-1 SDIO 结构框图 911
图 28-2 命令与响应时序 915
图 28-3 单块与多块读数据传输 916
图 28-4 单块与多块写数据传输 916
图 28-5 1 线模式数据传输格式 917
图 28-6 4 线模式数据传输格式 917
图 29-1 BACKUP 电源域整体框图 947
图 29-2 Backup 事件处理模块框图 948
图 30-1 RTC 框图 977
图 31-1 DAC 架构图 988
图 31-2 DAC 转换时序图 989
图 31-3 ramp 序列 991
图 32-1 ADC 实现框图 1004

图 32-2 启动队列功能时的队列缓存示意图 1016
图 32-3 jqm=0 队列关闭时的缓存示意图 1016
图 32-4 注入组队列功能示例（相同触发源） 1017
图 32-5 注入组队列功能示例（切换触发源） 1017
图 32-6 注入组队列功能示例（转换前溢出） 1018
图 32-7 注入组队列功能示例（转换过程中溢出） 1018
图 32-8 注入组队列清空示例 1（jqm=0） 1019
图 32-9 注入组队列清空示例 2（jqm=1） 1019
图 32-10 注入组队列清空示例 3（jqm=0） 1019
图 32-11 注入组队列清空示例 4（jqm=0） 1020
图 32-12 数据对齐方式 1022
图 32-13 规则转换数据溢出 1023
图 32-14 模拟看门狗阈值保护 1025
图 32-15 过采样的数据截取 1026
图 32-16 过采样的数据截取示例 1026
图 32-17 过采样时触发注入（丢弃注入前的过采样数据） 1027
图 33-1 PGA 结构框图 1066
图 34-1 比较器结构框图 1076
图 34-2 窗口比较器功能图 1078
图 34-3 消除功能时序图 1079
图 36-1 调试系统示意图 1091

表格目录

表 1-1 地址空间映射表.....53
表 1-2 自举配置（其中 x 表示该位的状态不起作用）.....57
表 1-3 中断号分配表.....57
表 2-1 CPU 通用寄存器.....63
表 2-2 CPU 浮点寄存器.....64
表 2-3 CPU 机器模式控制寄存器.....64
表 2-4 CPU 机器模式扩展控制寄存器.....66
表 2-5 CPU 用户模式控制寄存器.....72
表 2-6 CPU 用户模式扩展控制寄存器.....72
表 2-7 CPU RV32I 整型指令集.....75
表 2-8 CPU RV32M 乘除法指令集.....77
表 2-9 CPU RV32A 原子指令集.....78
表 2-10 CPU RV32F 单精度浮点指令.....78
表 2-11 CPU RVC 压缩指令集.....80
表 2-12 CPU RV32P DSP 指令集.....81
表 2-13 CPU 异常向量表.....90
表 2-14 CPU 异常优先级.....91
表 2-15 CLIC 地址映射.....100
表 2-16 nlbits 与 CPU 保存中断优先级编码示例.....104
表 2-17 nlbits 与 CLICINTCTL 划分示例.....104
表 3-1 Flash 总线地址映射表.....106
表 3-2 Boot 方式映射表（其中 x 表示该位的状态不起作用）.....108
表 3-3 RDP 保护等级映射表.....113
表 4-1 复位控制域.....137
表 10-1 互连关系图.....336
表 11-1 低功耗模式汇总.....341
表 11-2 低功耗模式下电源状态.....342
表 12-1 多项式参数.....355
表 13-1 DMA 请求映射关系.....361

表 13-2 传输类型与流控选择 364
表 13-3 接口信号描述 367
表 13-4 自动重载设置 371
表 13-5 CTL_L.src_msize/dest_msize 与对应 burst length 374
表 13-6 CTL_L.src_tr_width/dst_tr_width 与对应传输位宽 374
表 13-7 外设请求接口配置选择 374
表 14-1 GPT 通道地址偏移映射 404
表 14-2 通过边沿事件中断确认转子位置对照表 411
表 14-3 通过实时电平状态查询确认电机转子位置对照表 411
表 16-1 EPWM 通道 465
表 16-2 TG 模块关键信号释义 468
表 16-3 AQCTLA/B 寄存器设置在事件处使 PWM 波形产生的变化 470
表 16-4 递增时 PWM 变化事件优先级 471
表 16-5 递减时 PWM 变化事件优先级 471
表 20-1 管脚连接关系 545
表 21-1 SPI 实现 567
表 22-1 QSPI 实现及与 SPI 差别 605
表 23-1 I2C/SMBus 第一个字节的各 bit 定义 638
表 23-2 I2C 时序参数与对应的寄存器配置 640
表 23-3 MIN_SCL_HIGHtime 和 MIN_SCL_LOWtime 在不同速率下的要求 641
表 23-4 不同速率下 SDA_HOLD.ic_sda_rx_hold 的最大值 643
表 23-5 SMBus 总线协议在模块中的实现 646
表 23-6 SMBus ARP 命令在模块中实现 648
表 23-7 中断寄存器 RAW_INTR_STAT 各 bit 的置位与清除 650
表 24-1 I2S 实现 706
表 25-1 中断控制功能描述 742
表 27-1 USB_FS 支持的速度 810
表 28-1 SDIO 端口 911
表 28-2 命令格式 913
表 28-3 R1/R1b(普通响应)格式 913
表 28-4 R3(OCR 响应)格式 913
表 28-5 R4(CMD5 命令的响应)格式 913
表 28-6 R4(CMD39 命令的响应)格式 914
表 28-7 R5(CMD52 命令的响应)格式 914
表 28-8 R5(CMD40 命令的响应)格式 914

表 28-9 R6(RCA 响应)格式 914
表 28-10 R7(卡接口条件响应)格式 915
表 28-11 R2（CID/CSD 响应）格式 915
表 31-1 DAC 的带开关输出连接关系 988
表 31-2 DAC 触发源选择 990
表 31-3 DAC 内部参考源 991
表 31-4 DAC 输出 buffer 特性 992
表 32-1 ADC 的特性 1003
表 32-2 RAIN 的最大值与 t_s 的关系 1003
表 32-3 ADC 输入信号连接关系 1005
表 32-4 单端输入电压转 12bit 数据表 1008
表 32-5 差分输入电压转 12bit 数据表 1008
表 32-6 规则组外部触发的极性配置 1009
表 32-7 注入组外部触发的极性配置 1009
表 32-8 AWD1 检测通道配置 1025
表 32-9 温度传感器参数 1027
表 32-10 每个 ADC 的中断列表 1030
表 33-1 PGA 的输入通道选择 1066
表 33-2 各个 PGA 的端口连接 1066
表 33-3 PGA 增益选择 1068
表 34-1 ACMP 输入源选择 1076
表 34-2 各个 CMP 的输入端口连接 1077
表 34-3 输出极性选择 1077
表 34-4 ACMP 迟滞电压选择 1078

1 系统

1.1 系统架构

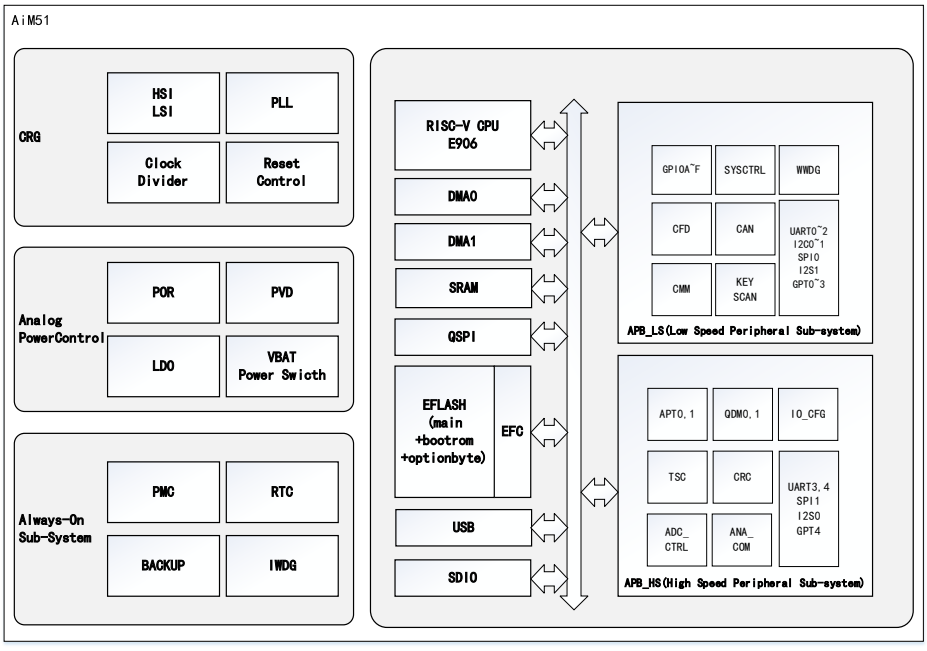
主系统包含以下部件：

时钟和复位控制（CRG），电源控制（PMC），常开电源域（AON）和总线系统，其中总线系统包含若干逻辑模块，分别为：

- Masters：RISC-V CPU、DMA0、DMA1、SDIO；
- Slaves：SRAM、EFC、QSPI、USB、高速以及低速外设。

其中时钟和复位控制以及常开电源域也通过总线接口作为 Slave 与总线系统连接，整体框图如图 1-1 所示。

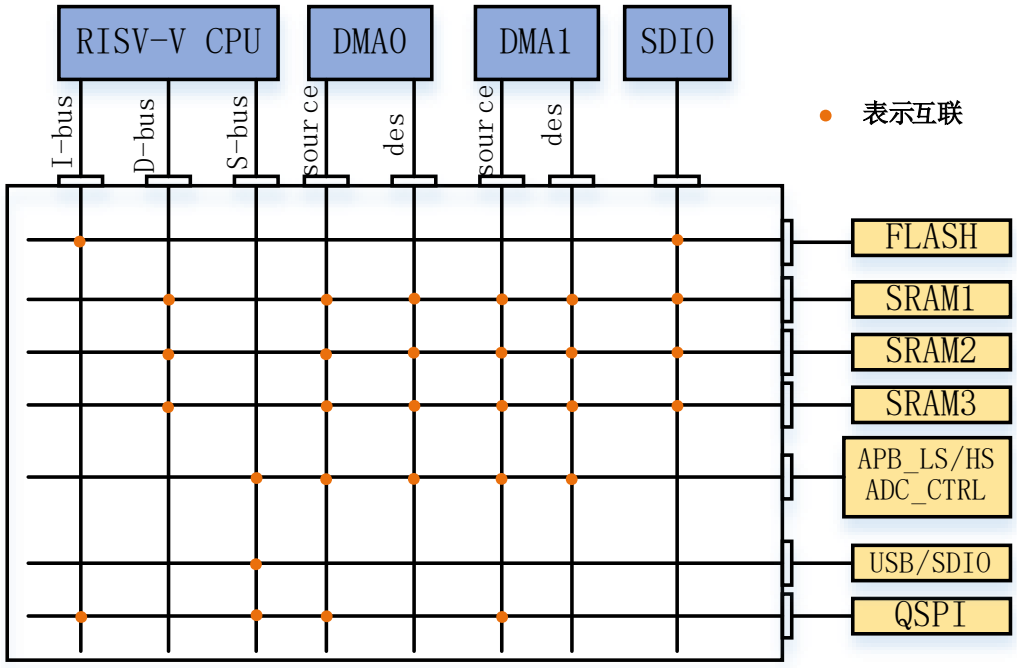
图 1-1 系统架构



AHB_BUS_Matrix

总线矩阵将 Master 与 Slave 互联，管理内核系统、DMA 与 SDIO 总线之间的仲裁。仲裁采用循环调度算法。总线矩阵由主控总线（CPU、DMA 与 SDIO）和被控总线（eFlash 接口、SRAM 、APB_LS、APB_HS、SDIO、USB、ADC_CTRL 与 QSPI）组成。总线互联矩阵如图 1-2 所示。

图 1-2 总线互联矩阵



APB_LS

此外设总线将低速外设连接到 AHB_BUS_Matrix，时钟频率为 BUS 时钟频率的 1/2。

APB_HS

此外设总线将高速外设连接到 AHB_BUS_Matrix，时钟频率为 BUS 时钟频率。

QSPI/USB/ADC_CTRL

为了提升性能，QSPI、USB 与 ADC_CTRL 连接到 AHB_BUS_Matrix，时钟频率为 BUS 时钟频率。

1.2 存储器构成

1.2.1 简介

程序存储器、数据存储器、寄存器和 I/O 端口排列在同一个线性（即地址连续）的 4GB 地址空间内。各字节按小端格式对齐（字中编号最低的字节被视为该字的最低有效字节，而编号最高的字节被视为最高有效字节）。

1.2.2 存储器映射和寄存器边界地址

所有未分配给片上存储器和外设的地址空间均视为“保留区”。有关可用存储器和寄存器区域的详细映射，请参考表 1-1。

⚠注意

ANA_COM 这段地址空间包含了 DAC、PGA、ACMP 这 3 个模拟 IP 的寄存器地址，详情可参考这 3 个 IP 的寄存器描述章节。

表 1-1 地址空间映射表

起始地址	结束地址	功能	大小
0x00000000	0x00083FFF	BOOT_ADDR	528KB
0x00084000	0x07FFFFFF	Reserved	-
0x08000000	0x08083FFF	EFLASH	528KB
0x08084000	0x2FFFFFFF	Reserved	-
0x30000000	0x37FFFFFF	QSPI_I	128MB
0x38000000	0x3FFFFFFF	Reserved	-
0x40000000	0x40007FFF	RAM0	32KB
0x40008000	0x40017FFF	RAM1	64KB
0x40018000	0x4002FFFF	RAM2	96KB
0x40030000	0x5FFFFFFF	Reserved	-
0x60000000	0x600000FF	I2C0	256B
0x60000100	0x60000FFF	Reserved	-
0x60001000	0x600010FF	I2C1	256B
0x60001100	0x6000FFFF	Reserved	-
0x60010000	0x600100FF	UART0	256B
0x60010100	0x60010FFF	Reserved	-
0x60011000	0x600110FF	UART1	256B
0x60011100	0x60011FFF	Reserved	-
0x60012000	0x600120FF	UART2	256B

起始地址	结束地址	功能	大小
0x600121FF	0x6002FFFF	Reserved	-
0x60030000	0x600300FF	SPI0	256B
0x60030100	0x6003FFFF	Reserved	-
0x60040000	0x60040FFF	WWDG	4KB
0x60041000	0x6004FFFF	Reserved	-
0x60050000	0x60050FFF	GPIOA	4KB
0x60051000	0x60051FFF	GPIOB	4KB
0x60052000	0x60052FFF	GPIOC	4KB
0x60053000	0x60053FFF	GPIOD	4KB
0x60054000	0x60054FFF	GPIOE	4KB
0x60055000	0x60055FFF	GPIOF	4KB
0x60056000	0x6005FFFF	Reserved	-
0x60060000	0x600603FF	I2S1	1KB
0x60060400	0x6007FFFF	Reserved	-
0x60080000	0x60080FFF	CMM	4KB
0x60081000	0x60081FFF	CFD	4KB
0x60082000	0x60082FFF	KEYSCAN	4KB
0x60083000	0x60083FFF	GPT0	4KB
0x60084000	0x60084FFF	GPT1	4KB
0x60085000	0x60085FFF	GPT2	4KB
0x60086000	0x60086FFF	GPT3	4KB
0x60087000	0x600873FF	CAN	1KB
0x60087400	0x60087FFF	Reserved	-
0x60088000	0x600880FF	USB_PHY	256B
0x60088100	0x60088FFF	Reserved	-
0x60089000	0x60089FFF	Reserved	-
0x6008A000	0x60FFFFFF	Reserved	-
0x61000000	0x610007FF	CRG	2KB
0x61000800	0x61000FFF	SYSCTRL	2KB
0x61001000	0x610017FF	EFC	2KB
0x61001800	0x61001FFF	IOCFG	2KB
0x61002000	0x610027FF	CRC	2KB
0x61002800	0x61002FFF	GPT4	2KB
0x61003000	0x610037FF	QDM0	2KB
0x61003800	0x61003FFF	QDM1	2KB
0x61004000	0x610047FF	APT0	2KB

起始地址	结束地址	功能	大小
0x61004800	0x61004FFF	APT1	2KB
0x61005000	0x610057FF	ANA_COM	2KB
0x61005800	0x610059FF	PMC	512B
0x61005A00	0x61005BFF	IWDG	512B
0x61005C00	0x61005FFF	Reserved	-
0x61006000	0x610061FF	BACKUP_CTRL	512B
0x61006200	0x610063FF	RTC	512B
0x61006400	0x610065FF	BACK_IO	512B
0x61006600	0x610067FF	BACKUP_STORE	512B
0x61006800	0x61006FFF	TSC	2KB
0x61007000	0x610073FF	I2S0	1KB
0x61007400	0x610074FF	SPI1	256B
0x61007500	0x610075FF	UART3	256B
0x61007600	0x610076FF	UART4	256B
0x61007700	0x62FFFFFF	Reserved	-
0x63000000	0x6303FFFF	USB	256B
0x63020000	0x6305FFFF	Reserved	-
0x63060000	0x63060FFF	SDIO	4KB
0x63061000	0x63FFFFFF	Reserved	-
0x64000000	0x64000FFF	DMA0	4KB
0x64001000	0x64001FFF	DMA1	4KB
0x64002000	0x64FFFFFF	Reserved	-
0x65000000	0x65000FFF	QSPI_S	4KB
0x65001000	0x650017FF	ADC_CTRL	2KB
0x65001800	0xDFFFFFFF	Reserved	-
0xE0000000	0xEFFFFFFF	CPU internal Peripherals	256MB
0xF0000000	0xFFFFFFFF	Reserved	-

1.3 嵌入式 SRAM

本芯片支持 192KB 嵌入式 SRAM，能够以最大系统时钟频率按字节、半字（16 位）或全字（32 位）访问，无需等待周期。

SRAM 支持奇偶校验功能，数据总线宽度为 36 位，其中有 4 位用于奇偶校验（每字节 1 位）。奇偶校验位在写入 SRAM 时进行计算和保存，然后在读取时自动进行校验。如果某一位失败，则将生成校验错误中断。该错误也连接到 APT0~1 的系统事件，用于刹车控制。系统控制寄存器 SRAM_PARITY_ERR.sram_parity_err 位域存放该 SRAM 奇偶校验错误标志。

说明

SRAM 奇偶校验功能默认开启，建议软件在系统启动时初始化整个 SRAM，以免在读取非初始化位置时出现奇偶校验错误。

1.4 Flash 概述

Flash 由两个不同的物理区域组成：

- 主 Flash 块，它包含应用程序和用户数据。
- 信息块，它包含三部分：
  - 用于硬件和存储器保护用户配置的选项字节。
  - 包含专有自举程序代码的系统存储器。
  - OTP（一次性可编程）区域。

更多详细信息，详细参考 3 嵌入式 Flash 控制器（EFC）章节。

Flash 接口根据 AHB 协议执行指令访问和数据访问，并通过 Flash 寄存器控制执行 Flash 操作（编程/擦除）。

1.5 自举配置

本芯片支持 Main Memory boot，Boot Memory boot 和 SRAM boot 三种不同的启动模式，通过 BOOT0 引脚的状态，以及 Option Byte（用户选项字节）中的配置位 EF_SYS_OPT.boot、EF_SYS_OPT.boot_sel 共同决定系统的启动模式。如表 1-2 所示，详细参考 3.3.4 BOOT 模式。

表 1-2 自举配置（其中 x 表示该位的状态不起作用）

RDP	BOOT0 引脚	EF_SYS_OPT.boot (二进制)	EF_SYS_OPT.boot_sel (二进制)	所选启动区域
0/1	0	xx	0	Main Memory
0/1	1	1x	0	Boot Memory
0/1	1	0x	0	SRAM
0/1	x	x0	1	Main Memory
0/1	x	11	1	Boot Memory
0/1	x	01	1	SRAM
2	x	xx	x	Main Memory

说明

系统上电时，BOOT0、EF_SYS_OPT.boot 和 EF_SYS_OPT.boot_sel 的值共同决定系统启动位置。

当 RDP 保护等级为 2 时，芯片固定从 Main Memory 启动。

1.6 中断映射表

芯片共支持 86 个外设中断，对应 CPU 中断号 16~101。外设中断映射如表 1-3 所示。中断处理详细信息，参考 2.4 异常与中断章节。

表 1-3 中断号分配表

CPU 中断号	中断源
16	SOFTWARE
17	IWDG
18	WWDG
19	PVD
20	EFC
21	EFC_ERR
22	SRAM0_PARITY_ERR
23	SRAM1_PARITY_ERR
24	SRAM2_PARITY_ERR
25	DMA0
26	DMA1

CPU 中断号	中断源
27	GPT0
28	GPT1
29	GPT2
30	GPT3
31	GPT0_CH0
32	GPT0_CH1
33	GPT0_CH2
34	GPT0_CH3
35	GPT1_CH0
36	GPT1_CH1
37	GPT1_CH2
38	GPT1_CH3
39	GPT2_CH0
40	GPT2_CH1
41	GPT2_CH2
42	GPT2_CH3
43	GPT3_CH0
44	GPT3_CH1
45	GPT3_CH2
46	GPT3_CH3
47	ADC0
48	ADC1
49	I2C0
50	I2C1
51	SPI0
52	SPI1
53	UART0
54	UART1
55	UART2
56	UART3
57	UART4
58	I2S0
59	I2S1
60	APT0
61	APT1
62	ATP0_TZ

CPU 中断号	中断源
63	ATP1_TZ
64	CAN
65	GPT4
66	QDM0
67	QDM1
68	QSPI
69	USB
70	SDIO_CARD_PULL_OUT
71	SDIO_CARD_VALID_WAKEUP
72	KEYSCAN
73	CMM
74	CFD
75	GPIOA
76	GPIOB
77	GPIOC
78	GPIOD
79	GPIOE
80	GPIOF
81	RTC
82	TSC
83	DAC0
84	DAC1
85	DAC2
86	CMP0
87	CMP1
88	CMP2
89	CMP3
90	DMA0_CH0
91	DMA0_CH1
92	DMA0_CH2
93	DMA0_CH3
94	DMA0_CH4
95	DMA0_CH5
96	DMA1_CH0
97	DMA1_CH1
98	DMA1_CH2

CPU 中断号	中断源
99	DMA1_CH3
100	DMA1_CH4
101	DMA1_CH5

2 处理器（CPU）

2.1 CPU 主要特性

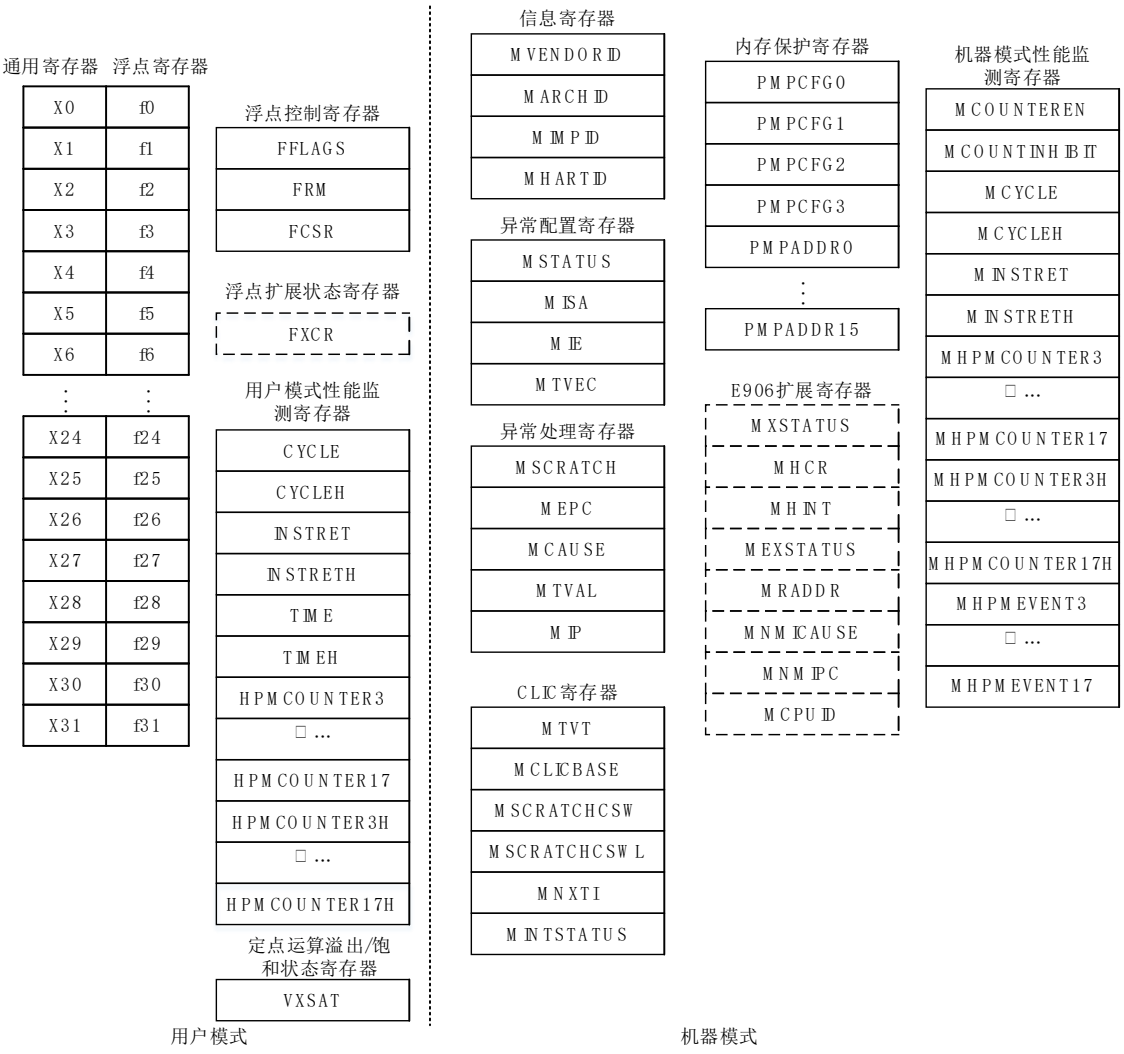
AiM51 MCU 中的处理器是一款基于 RISC-V 指令集的 32bit 高能效比嵌入式处理器，具有以下特点：

- 支持 RV32IMAFCP 指令集
- 支持机器模式和普通用户模式
- 支持物理内存保护(Physical Memory Protection，PMP)
- 单发射 5 级整型流水线微架构
- 支持动态分支预测
- 支持最大 4G Byte 地址空间
- 支持指令总线和数据总线同时访问代码和数据，同时支持通过系统总线访问外设 IP
- 支持矢量中断及非矢量中断模式

2.2 CPU 编程模型

2.2.1 工作模式及寄存器视图

图 2-1 工作模式及寄存器视图



芯片的 CPU 实现了两种 RISC-V 编程模式：机器模式和普通用户模式。

当 MXSTATUS 寄存器内 PM 位被置位，处理器就在机器模式下执行程序。处理器复位后，工作在机器模式。

机器模式：

- 允许访问所有通用寄存器、浮点寄存器、控制寄存器和所有紧耦合 IP 的寄存器
- 允许使用 CPU 支持的所有指令

普通用户模式：

- 允许访问通用寄存器、浮点寄存器、浮点控制寄存器以及事件监测寄存器等
- 允许使用绝大多数除了对系统产生重大影响的特权指令如 WFI，MRET，CSR 和平头

哥扩展的 CACHE 指令之外的指令

- 使用 ECALL 指令进入机器模式

两种编程模式对应不同的操作权限，区别主要体现在以下几个方面：

- 对控制寄存器的访问
- 特权指令的使用
- 对紧耦合 IP 的寄存器访问

2.2.2 通用寄存器

表 2-1 列出了普通用户编程模式下的 32 个 32 位通用寄存器（x0~x31），其中 x0 是零值寄存器。

表 2-1 CPU 通用寄存器

寄存器	ABI 名称	描述
x0	zero	零值
x1	ra	返回地址
x2	sp	堆栈指针
x3	gp	全局指针
x4	tp	线程指针
x5	t0	临时/备用链接寄存器
x6-7	t1-2	临时寄存器
x8	s0/fp	保留寄存器/帧指针
x9	s1	保留寄存器
x10-11	a0-a1	函数参数/返回值
x12-17	a2-a7	函数参数
x18-27	s2-s11	保留寄存器
x28-31	t3-t6	临时寄存器

通用寄存器通常用于存储指令操作数和结果以及地址信息。软硬件上约定这些通用寄存器作为子程序的链接调用，参数传递以及堆栈指针等功能。

此外，普通用户编程模式寄存器还包括处理器执行地址寄存器（PC）。

2.2.3 浮点寄存器

当 CPU 配置了硬件浮点单元时，按照 RISC-V SPEC 定义，CPU 内部会额外实现一组 32 个浮点寄存器（f0~f31），如下表 2-2 所示。根据浮点配置的不同，当配置单精度浮点时，f0~f31 每个寄存器位宽为 32 比特，当配置双精度浮点时，f0~f31 每个寄存器位宽为 64 比特。按照 RISC-V 标准定义，配置双精度浮点时必须同时实现单精度浮点，因此单精度浮点使用 f0~f31 中寄存器的低 32 位，高 32 位为全 1。

表 2-2 CPU 浮点寄存器

寄存器	ABI 名称	描述
f0~f7	ft0-7	浮点临时寄存器
f8~f9	fs0-1	浮点保留寄存器
f10~f11	fa0-1	浮点参数/返回值寄存器
f12~f17	fa2-7	浮点参数寄存器
f18~f27	fs2-11	浮点保留寄存器
f28~f31	ft8-11	浮点临时寄存器

2.2.4 机器模式控制寄存器

CPU 中实现的 RISC-V 标准定义的机器模式控制寄存器如下表 2-3 所示。

表 2-3 CPU 机器模式控制寄存器

名称	读写权限	寄存器编号	描述
机器模式信息寄存器组			
MVENDORID	机器模式只读	0xF11	厂商编号寄存器
MARCHID	机器模式只读	0xF12	架构编号寄存器
MIMPID	机器模式只读	0xF13	微体系结构编号寄存器
MHARTID	机器模式只读	0xF14	线程编号寄存器
机器模式异常配置寄存器组			
MSTATUS	机器模式读写	0x300	机器模式处理器状态寄存器
MISA	机器模式读写	0x301	机器模式处理器指令集信息寄存器
MIE	机器模式读写	0x304	机器模式中断使能控制寄存器
MTVEC	机器模式读写	0x305	机器模式异常向量基址寄存器
MTVT	机器模式读写	0x307	机器模式矢量中断基址寄存器
机器模式异常处理寄存器组			
MSCRATCH	机器模式读写	0x340	机器模式数据备份寄存器

名称	读写权限	寄存器编号	描述
MEPC	机器模式读写	0x341	机器模式异常程序计数器
MCAUSE	机器模式读写	0x342	机器模式异常原因寄存器
MTVAL	机器模式读写	0x343	机器模式异常向量寄存器
MIP	机器模式读写	0x344	机器模式中断等待寄存器
MNXTI	机器模式读写	0x345	机器模式等待中断向量地址和中断使能寄存器
MINTSTATUS	机器模式读写	0x346	机器模式中断状态寄存器
MSCRATCHCSW	机器模式读写	0x348	机器模式多模式数据备份寄存器
MSCRATCHCSWL	机器模式读写	0x349	机器模式中断数据备份寄存器
MCLICBASE	机器模式读写	0x350	机器模式中断控制器基址寄存器
机器模式性能监测寄存器组			
MCOUNTEREN	机器模式读写	0x306	机器模式计数器授权寄存器
MCOUNTINHIBIT	机器模式读写	0x320	机器模式计数禁止寄存器
MHPMEVENT3	机器模式读写	0x323	机器模式 3 号事件选择寄存器
……			
MHPMEVENT17	机器模式读写	0x331	机器模式 17 号事件选择寄存器
MCYCLE	机器模式读写	0xB00	机器模式周期计数器
MINSTRET	机器模式读写	0xB02	机器模式退休指令计数器
MCYCLEH	机器模式读写	0xB80	机器模式周期计数器高 32 位
MINSTRETH	机器模式读写	0xB82	机器模式退休指令计数器高 32 位
MHPMCOUNTER3	机器模式读写	0xB03	机器模式 3 号事件计数器
……			
MHPMCOUNTER17	机器模式读写	0xB11	机器模式 17 号事件计数器
MHPMCOUNTER3H	机器模式读写	0xB83	机器模式 3 号事件计数器高 32 位
……			
MHPMCOUNTER17H	机器模式读写	0xB91	机器模式 17 号事件计数器高 32 位

CPU 中扩展的控制寄存器如下表 2-4 所示：

表 2-4 CPU 机器模式扩展控制寄存器

名称	读写权限	寄存器编号	描述
机器模式扩展寄存器组			
MXSTATUS	机器模式读写	0x7C0	扩展状态寄存器
MHCR	机器模式读写	0x7C1	硬件控制寄存器
MHINT	机器模式读写	0x7C5	隐式操作寄存器
MRADDR	机器模式读写	0x7E0	处理器复位启动地址指示寄存器
MEXSTATUS	机器模式读写	0x7E1	扩展异常状态寄存器
MNMICAUSE	机器模式读写	0x7E2	NMI 现场保存寄存器
MNMIPC	机器模式读写	0x7E3	NMI 异常程序计数器
机器模式处理器型号扩展寄存器组			
MCPUID	机器模式读写	0xFC0	处理器型号寄存器

1. 扩展状态寄存器（MXSTATUS）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PM		Res.							THEADISAEE	Res.					
RW	RW	-	-	-	-	-	-	-	RW	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MM	Res.	PMDM	Res.		PMDU	Res.									
RW	-	RW	-	-	RW	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

PMDU-用户模式事件监测计数器开关位：

0：用户模式下事件监测计数器可以正常计数

1：用户模式下事件监测计数器禁止计数

该域复位值为 0，机器模式读写。

PMDM-机器模式事件检测计数器开关位：

0：机器模式下事件监测计数器可以正常计数

1：机器模式下事件监测计数器禁止计数

该域复位值为 0，机器模式读写。

MM-非对齐访问控制位：

1：当内存访问出现地址非对齐时，不会上报非对齐访问异常，硬件会对非对齐访问请求进行拆分处理；

0：当内存访问出现地址非对齐时，上报非对齐访问异常；

该域复位值为 1，机器模式读写。

THEADISAEE-THEAD 扩展指令集使能位：

当 THEADISAEE 为 0 时，执行所有 T-Head 自定义扩展指令产生非法指令异常；

当 THEADISAEE 为 1 时，处理器可以正常执行 T-Head 自定义扩展指令集。

该域复位值为 0，机器模式读写。

PM-当前中断特权模式位:

表示当前处理器的所处的特权模式，该域为 2'b11 时处理器为机器模式，2'b00 时为用户模式。

该域复位值为 2'b11，机器模式只读。

2. 硬件配置寄存器（MHCR)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			BTB	Res.						BPE	RS	WA	WB	DE	IE
-	-	-	RW	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

BTB-分支目标预测使能位：

当 BTB 值为 0 时，分支目标预测关闭；当 BTB 值为 1 时，分支目标预测开启。

该域复位值为 0，机器模式读写。

BPE-允许预测跳转设置位：

当 BPE 为 0 时，预测跳转关闭；当 BPE 为 1 时，预测跳转开启。

该域复位值为 0，机器模式读写。

RS-地址返回栈设置位：

当 RS 为 0 时，返回栈关闭；当 RS 为 1 时，返回栈开启。

该域复位值为 0，机器模式读写。

WA-高速缓存写分配有效设置：

该位表示处理器执行 Store 指令且数据 cache 缺失时，是否在数据 cache 中分配缺失的缓存行。当 WA 为 0 时，数据 cache 为 write non-allocate 模式，即在数据 cache 缺失时不会将缺失的数据缓存行从内存加载到数据 cache 内；当 WA 为 1 时，数据 cache 为 write allocate 模式，即在 Store 指令导致的数据 cache 缺失时会将该缺失的数据缓存行从内存中加载到数据 cache 中。

该域复位值为 0，机器模式读写。

WB-高速缓存写回设置位：

当 WB 为 0 时，数据 cache 为写直模式；当 WB 为 1 时，数据 cache 为写回模式。

该域复位值为 0，机器模式读写。

DE-数据高速缓存设置位：

当 DE 为 0 时，数据 cache 关闭；当 DE 为 1 时，数据 cache 开启。

该域复位值为 0，机器模式读写。

IE-指令高速缓存设置位：

当 IE 为 0 时，指令 cache 关闭；当 IE 为 1 时，指令 cache 开启。

该域复位值为 0，机器模式读写。

3. 隐式操作寄存器（MHINT）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.											AEE	Res.			
-	-	-	-	-	-	-	-	-	-	-	RW	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

AEE-精确异常使能位：

当 AEE 为 1 时，处理器处于精确异常工作模式。由于 Load/Store 指令访问总线产生的总线访问错误异常信号返回延迟不可预期，处理器在执行 Load/Store 指令时会堵塞流水线，后续指令不会下发被执行。直到异常信号返回处理器，进入异常服务程序，MEPC 将保存这条产生总线访问异常的 Load/Store 指令的 PC，从而实现精确异常。

当 AEE 为 0 时，处理器处于非精确异常工作模式。处理器在执行 Load/Store 指令时不会堵塞流水线，若后续指令不为 Load/Store 指令（结构竞争）且与该 Load/Store 指令无数据相关性时，可以继续执行。当总线访问错误异常信号传送至处理器，此时进入异常服务程序时，MEPC 将保存流水线 EX 级中正在被执行的指令的 PC，不一定为该出现内存访问错误异常的 Load/Store 指令的 PC。

需要注意的是，即便 AEE 配置为 0，执行 Load/Store 指令因 PMP 权限不匹配导致的内存访问错误异常是精确的。另外，非精确异常工作模式下，MTVAL 寄存器的更新值是精确的，即产生内存访问错误异常的地址被精确保存了下来。

该域复位值为 0，机器模式读写。

4. 扩展异常状态寄存器（MEXSTATUS）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.														SPSWAPEN	SPUSHEN
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							BUSERR	NMISTS	LOCKUP	EXPTVLD	WFEEN	LPMD		SOFT_RST	
RW	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

SOFT_RST-软件复位模式：

向 SOFT_RST 写 2'b00 时，表示不需要进行软复位；向 SOFT_RST 写 2'b01 时，表示需要复位。

向 SOFT_RST 写 2'b10 时，表示需要复位整个系统；向 SOFT_RST 写 2'b11 为 reserved。

该域复位值为 2'b00，机器模式读写。

LPMD-lowpower 模式选择：

当 LPMD 为 2'b00 时，下次使用 WFI 指令进入深睡眠；当 LPMD 为 2'b01 时，下次使用 WFI 指令进入浅睡眠。

该域复位值为 2'b00，机器模式读写。

WFEEN-wait for event 模式使能：

当 WFEEN 为 1 时，执行 WFI 指令实际为 WFE 功能，event 可以唤醒，中断不需要考虑优先级即可唤醒；当 WFEEN 为 0 时，WFI 指令仅能由中断进行唤醒，而且需要考虑中断优先级。

该域复位值为 1，机器模式读写。

EXPT_VLD-异常有效：

当 EXPT_VLD 为 1 时，表示处理器进入异常处理状态。

该域复位值为 0，机器模式只读。

LOCKUP-lockup 有效：

当 LOCKUP 为 1 时，表示处理器进入锁定状态。

该域复位值为 0，机器模式只读。

NMI_STS-NMI 有效：

当 NMI_STS 为 1 时，表示处理器进入 NMI 处理状态。

该域复位值为 0，机器模式只读。

BUSERR-BUSERR 异常：

当 BUSERR 为 1 时，表示处理器上一次发生的异常为 BUSERR 异常。

该域复位值为 0，机器模式读写。

SPUSHEN-中断自动压栈开关位：

该位为 0 时，不使能硬件自动压栈机制；该位为 1 时，使能硬件自动压栈机制，响应中断时，硬件投机执行一条 IPUSH 指令。

该域复位值为 0，机器模式读写。

SPSWAPEN-自动中断栈切换位：

该位为 0 时，不使能自动中断栈切换机制；该位为 1 时，使能自动中断栈切换机制，当响应第 0 层中断时，自动将 mscratchcswl 寄存器的值写入 sp 寄存器。第 0 层中断判断条件：当前 MIL 为 0。

该域复位值为 0，机器模式读写。

5. 复位地址指示寄存器（MRADDR）

该寄存器用于指示 CPU 的复位启动地址，CPU 复位启动地址由系统集成时指定，该寄存器仅用于软件查询，机器模式只读。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MRADDR															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MRADDR															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

6. NMI 状态寄存器（MNMICAUSE）

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NMI_INTR	Res.	NMI_MPP		NMI_MPIE	Res.										
RO	-	RO	RO	RO	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				NMI_VECTOR											
-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

NMI_VECTOR:

保存 NMI 响应时 MCAUSE 内 exception code 寄存器的值

NMI_MPIE:

保存 NMI 响应时 MSTATUS 内 MPIE 寄存器的值

NMI_MPP:

保存 NMI 响应时 MSTATUS 内 MPP 寄存器的值

NMI_INTR:

保存 NMI 响应时 MCAUSE 内 INTR 寄存器的值

7. NMI 异常程序计数器（MNMIPC）

该寄存器位宽为 32 位，仅在机器模式下可读写，用户模式下访问该寄存器会触发非法指令异常。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PC															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PC															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

8. 处理器型号寄存器（MCPUID）

处理器型号寄存器（MCPUID）存储了处理器型号信号，其复位值由产品本身决定，具体定义遵循《平头哥产品 ID 定义规范（5.0 版）》。

2.2.5 用户模式控制寄存器

用户模式可访问的控制寄存器如下表 2-5 和表 2-6 所示。

表 2-5 CPU 用户模式控制寄存器

名称	读写权限	寄存器编号	描述
用户模式浮点控制寄存器组			
FFLAGS	用户模式读写	0x001	浮点异常累积状态寄存器
FRM	用户模式读写	0x002	浮点动态舍入模式寄存器
FCSR	用户模式读写	0x003	浮点控制寄存器
用户模式性能监测计数器			
CYCLE	用户模式只读	0xC00	用户模式运行周期数计数器低 32 位
TIME	用户模式只读	0xC01	用户模式计时器低 32 位
INSTRET	用户模式只读	0xC02	用户模式指令退休计数器低 32 位
CYCLEH	用户模式只读	0xC80	用户模式运行周期数计数器高 32 位
TIMEH	用户模式只读	0xC81	用户模式计时器高 32 位
INSTERTH	用户模式只读	0xC82	用户模式指令退休计数器高 32 位
HPMCOUNTER3	用户模式只读	0xC03	用户模式 3 号事件计数器
……			
HPMCOUNTER17	用户模式只读	0xC11	用户模式 17 号事件计数器
HPMCOUNTERH3	用户模式只读	0xC83	用户模式 3 号事件计数器高 32 位
……			
HPMCOUNTERH17	用户模式只读	0xC91	用户模式 17 号事件计数器高 32 位

表 2-6 CPU 用户模式扩展控制寄存器

名称	读写权限	寄存器编号	描述
用户模式浮点控制寄存器组			
FXCR	机器模式只读	0x800	用户模式浮点扩展控制寄存器

用户模式浮点扩展控制寄存器（FXCR）

用户模式浮点扩展控制寄存器（FXCR）用于浮点扩展功能开关和浮点异常累积位，用户模式可读写。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					RM			DQNaN	Res.						
-	-	-	-	-	RW	RW	RW	RW	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										FE	NV	DZ	OF	UF	NX
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

NX-非精确异常：

FCSR 对应位的映射。

UF-下溢异常：

FCSR 对应位的映射。

OF-上溢异常：

FCSR 对应位的映射。

DZ-除 0 异常：

FCSR 对应位的映射。

NV-无效操作数异常：

FCSR 对应位的映射。

FE-浮点异常累积位：

当有任何一个浮点异常发生时，该位将被置为 1。

DQNaN-输出 QNaN 模式位：

当 DQNaN 为 0 时，计算输出的 QNaN 值为 RISC-V 规定的固定值，即 0x7FC0_0000。

当 DQNaN 为 1 时，计算输出的 QNaN 值跟 IEEE754 标准一致。

RM-舍入模式：

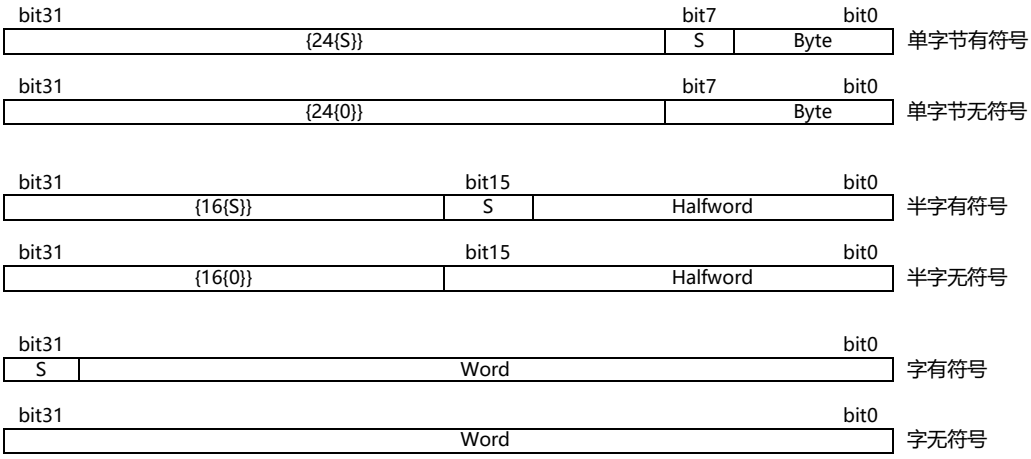
FCSR 对应位的映射。

对 NX/UF/OF/DZ/NV/RM 位域的写操作会同时体现在 FCSR 寄存器上。

2.2.6 数据格式

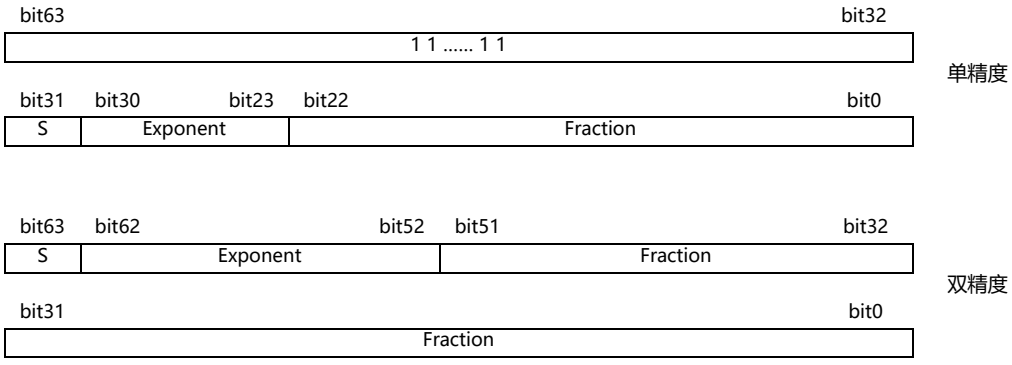
1. 整型数据格式

寄存器内部的数值存在有符号和无符号的区别。其格式均为从右至左表示逻辑低位到高位的排布，如下图所示：



2. 浮点数据格式

CPU 浮点单元遵从 RISC-V 标准，兼容 IEEE 754-2008 浮点协议，支持单精度浮点运算，数据格式如下图所示。当当 CPU 配置为双精度浮点时，单精度数据仅使用 64 位浮点寄存器的低 32 位，高 32 位需要全部为 1，否则会被当做非数处理。当 CPU 配置为单精度浮点时，浮点寄存器位宽为 32 位。



3. 小端

存储器数据有大小端的区分，CPU 仅支持小端模式，即数据高位存放至物理内存的高地址。

Byte3	Byte2	Byte1	Byte0	Word at A
Byte3	Byte2	Byte1	Byte0	Half word at A
Byte3	Byte2	Byte1	Byte0	Byte at A
A+3	A+2	A+1	A+0	

2.3 CPU 指令集

本章主要介绍了 CPU 中实现的 RV32IMAFDCP 指令集，包括标准的整型指令集（RV32I），RV 乘除法指令集（RV32M），RV 原子指令集（RV32A），RV 单精度浮点指令集(RV32F)，RV 压缩指令（RVC）和 RV DSP 指令集（RV32P）。

1. RV32I 整型指令集

本节介绍了 32 位基本整型指令，如下表 2-7 所示。基本整型指令集按功能可以分为以下类型：

- 加减法指令
- 逻辑操作指令
- 移位指令
- 比较指令
- 数据传输指令
- 分支跳转指令
- 内存存取指令
- 控制寄存器操作指令
- 低功耗指令
- 异常返回指令
- 特殊功能指令

表 2-7 CPU RV32I 整型指令集

指令名称	指令描述	执行延时
加减法指令		
ADD	有符号加法指令	1
ADDI	有符号立即数加法指令	1
SUB	有符号减法指令	1
逻辑操作指令		
AND	按位与指令	1
ANDI	立即数按位与指令	1
OR	按位或指令	1
ORI	立即数按位或指令	1

指令名称	指令描述	执行延时
XOR	按位异或指令	1
XORI	立即数按位异或指令	1
移位指令		
SLL	逻辑左移指令	1
SLLI	立即数逻辑左移指令	1
SRL	逻辑右移指令	1
SRLI	立即数逻辑右移指令	1
SRA	算术右移指令	1
SRAI	立即数算术右移指令	1
比较指令		
SLT	有符号比较小于置位指令	1
SLTU	无符号比较小于置位指令	1
SLTI	有符号立即数比较小于置位指令	1
SLTIU	无符号立即数比较小于置位指令	1
数据传输指令		
LUI	高位立即数装载指令	1
AUIPC	PC 高位立即数加法指令	1
分支跳转指令		
BEQ	相等分支指令	1
BNE	不等分支指令	1
BLT	有符号小于分支指令	1
BGE	有符号大于等于分支指令	1
BLTU	无符号小于分支指令	1
BGEU	无符号大于等于分支指令	1
JAL	直接跳转子程序指令	1
JALR	寄存器跳转子程序指令	1
内存存取指令		
LB	有符号扩展字节加载指令	2（cache 命中）
LBU	无符号扩展字节加载指令	
LH	有符号扩展半字加载指令	
LHU	无符号扩展半字加载指令	
LW	有符号扩展字加载指令	
SB	字节存储指令	
SH	半字存储指令	
SW	字存储指令	
控制寄存器操作指令		

指令名称	指令描述	执行延时
CSRRW	控制寄存器读写传送指令	阻塞执行
CSRRS	控制寄存器置位传送指令	
CSRRC	控制寄存器清零传送指令	
CSRRWI	控制寄存器立即数读写传送指令	
CSRRSI	控制寄存器立即数置位传送指令	
CSRRCI	控制寄存器立即数清零传送指令	
低功耗指令		
WFI	进入低功耗模式指令	不可预期
异常返回指令		
MRET	机器模式异常返回指令	阻塞执行
特殊功能指令		
FENCE	存储同步指令	不可预期
FENCE.I	指令流同步指令	阻塞执行
ECALL	环境异常调用指令	1
EBREAK	断点指令	1

2. RV32M 乘除法指令集

RV32M 乘除法指令如下表 2-8 所示。

表 2-8 CPU RV32M 乘除法指令集

指令名称	指令描述	执行延时
乘除法指令		
MUL	有符号乘法指令	2
MULH	有符号乘法取高位指令	2
MULHSU	有符号与无符号乘法取高位指令	2
MULHU	无符号乘法取高位指令	2
DIV	有符号除法指令	1-33
DIVU	无符号除法指令	1-33
REM	有符号取余指令	1-33
REMU	无符号取余指令	1-33

3. RV32A 原子指令集

RV32A 原子指令如下表 2-9 所示。

表 2-9 CPU RV32A 原子指令集

指令名称	指令描述	执行延时
原子指令		
LR.W	字加载保留指令	拆分为多条原子指令执行，可能拆分出 fence 指令且延时不可预期
SC.W	字条件存储指令	
AMOSWAP.W	原子交换指令	
AMOADD.W	原子加法指令	
AMOXOR.W	原子按位异或指令	
AMOAND.W	原子按位与指令	
AMOOR.W	原子按位或指令	
AMOMIN.W	原子有符号取最小值指令	
AMOMAX.W	位原子有符号取最大值指令	
AMOMINU.W	原子无符号取最小值指令	
AMOMAXU.W	原子无符号取最大值指令	

4. RV32F 单精度浮点指令

本节主要介绍 32 位单精度浮点指令，如下表 2-10 所示。按功能可以分为以下类型：

- 运算指令
- 内存存取指令
- 类型转换及数据传输指令
- 比较指令
- 分类指令

表 2-10 CPU RV32F 单精度浮点指令

指令名称	指令描述	执行延时
运算指令		
FADD.S	单精度浮点加法指令	3
FSUB.S	单精度浮点减法指令	3
FMADD.S	单精度浮点乘累加指令	4
FMSUB.S	单精度浮点乘累减指令	4
FNMSUB.S	单精度浮点乘累减取反指令	4
FNMADD.S	单精度浮点乘累加取反指令	4
FMUL.S	单精度浮点乘法指令	3
FDIV.S	单精度浮点除法指令	18

指令名称	指令描述	执行延时
FSQRT.S	单精度浮点开方指令	18
FMIN.S	单精度浮点取最小值指令	3
FMAX.S	单精度浮点取最大值指令	3
内存存取指令		
FLW	单精度浮点内存加载指令	2（cache 命中）
FSW	单精度浮点内存存储指令	2（cache 命中）
类型转换及数据传输指令		
FSGNJ.S	单精度浮点符号注入指令	1
FSGNJN.S	单精度浮点符号取反注入指令	1
FSGNJX.S	单精度浮点符号异或注入指令	1
FCVT.W.S	单精度浮点转换为有符号整型指令	3
FCVT.WU.S	单精度浮点转换为无符号整型指令	3
FCVT.S.W	有符号整型转换为单精度浮点指令	3
FCVT.S.WU	无符号整型转换为单精度浮点指令	3
FMV.X.W	单精度浮点读传送指令	1
FMV.W.X	单精度浮点写传送指令	1
比较指令		
FEQ.S	单精度相等比较指令	3
FLT.S	单精度浮点小于比较指令	3
FLE.S	单精度浮点小于等于比较指令	3
分类指令		
FCLASS.S	单精度数值类型分类指令	1

5. RVC 压缩指令集

本节主要介绍 16 位压缩指令，如下表 2-11 所示。压缩指令集按功能可以分为以下类型：

- 加减法指令
- 逻辑操作指令
- 移位指令
- 数据传输指令
- 分支跳转指令
- 立即数偏移存取指令

表 2-11 CPU RVC 压缩指令集

指令名称	指令描述	执行延时
加减法指令		
C.ADD	有符号加法指令	1
C.ADDI	有符号立即数加法指令	1
C.SUB	有符号减法指令	1
C.ADDI16SP	堆栈指针有符号自加指令	1
C.ADDI4SPN	堆栈指针无符号加法指令	1
逻辑操作指令		
C.AND	按位与指令	1
C.ANDI	立即数按位与指令	1
C.OR	按位或指令	1
C.XOR	按位异或指令	1
移位指令		
C.SLLI	立即数逻辑左移指令	1
C.SRLI	立即数逻辑右移指令	1
C.SRAI	立即数算术右移指令	1
数据传输指令		
C.MV	数据传送指令	1
C.LI	低位立即数传送指令	1
C.LUI	高位立即数传送指令	1
分支跳转指令		
C.BEQZ	等于零分支指令	1
C.BNEZ	不等于零分支指令	1
C.J	无条件跳转指令	1
C.JR	寄存器跳转指令	1
C.JAL	无条件跳转子程序指令	1
C.JALR	寄存器跳转子程序指令	1
内存存取指令		
C.LW	字加载指令	2(cache 命中)
C.SW	字存储指令	
C.LWSP	字堆栈加载指令	
C.SWSP	字堆栈存储指令	
特殊指令		
C.NOP	空操作指令	1
C.EBREAK	调试断点指令	1

6. RV32P DSP 指令集

本节主要介绍 RV32P DSP 指令集，CPU 实现了选配的 Zp64 指令子集，如下表 2-12 所示。DSP 指令集按功能可以分为以下类型：

- SIMD 指令
- 部分 SIMD 指令
- 64 位运算指令
- 非 SIMD 指令

表 2-12 CPU RV32P DSP 指令集

指令名称	指令描述	执行延迟
SIMD 指令		
16 位加减法指令		
add16	16 位加法指令	1
radd16	16 位有符号加法减半指令	1
uradd16	16 位无符号加法减半指令	1
kadd16	16 位有符号加法饱和指令	1
ukadd16	16 位无符号加法饱和指令	1
sub16	16 位减法指令	1
rsub16	16 位有符号减法减半指令	1
ursub16	16 位无符号减法减半指令	1
ksub16	16 位有符号减法饱和指令	1
uksub16	16 位无符号减法饱和指令	1
cras16	16 位交叉加减指令	1
rcras16	16 位有符号交叉加减减半指令	1
urcras16	16 位无符号交叉加减减半指令	1
kcras16	16 位有符号交叉加减饱和指令	1
ukcras16	16 位无符号交叉加减饱和指令	1
crsa16	16 位交叉减加指令	1
rcrsa16	16 位有符号交叉减加减半指令	1
urcrsa16	16 位无符号交叉减加减半指令	1
kcrsa16	16 位有符号交叉减加饱和指令	1
ukcrsa16	16 位无符号交叉减加饱和指令	1
stas16	16 位对应半字加减指令	1
rstas16	16 位有符号对应半字加减减半指令	1
urstas16	16 位无符号对应半字加减减半指令	1
kstas16	16 位有符号对应半字加减饱和指令	1

指令名称	指令描述	执行延迟
ukstas16	16 位无符号对应半字加减饱和指令	1
stsa16	16 位对应半字减加指令	1
rstsa16	16 位有符号对应半字减加减半指令	1
urstsa16	16 位无符号对应半字减加减半指令	1
kstsa16	16 位有符号对应半字减加饱和指令	1
ukstsa16	16 位无符号对应半字减加饱和指令	1
8 位加减法指令		
add8	8 位加法指令	1
radd8	8 位有符号加法减半指令	1
uradd8	8 位无符号加法减半指令	1
kadd8	8 位有符号加法饱和指令	1
ukadd8	8 位无符号加法饱和指令	1
sub8	8 位减法指令	1
rsub8	8 位有符号减法减半指令	1
ursub8	8 位无符号减法减半指令	1
ksub8	8 位有符号减法饱和指令	1
uksub8	8 位无符号减法饱和指令	1
16 位移位指令		
sra16	16 位寄存器算术右移指令	1
srai16	16 位立即数算术右移指令	1
sra16.u	16 位寄存器算术右移舍入指令	2
srai16.u	16 位立即数算术右移舍入指令	2
srl16	16 位寄存器逻辑右移指令	1
srli16	16 位立即数逻辑右移指令	1
srl16.u	16 位寄存器逻辑右移舍入指令	2
srli16.u	16 位立即数逻辑右移舍入指令	2
sll16	16 位寄存器逻辑左移指令	1
slli16	16 位立即数逻辑左移指令	1
ksll16	16 位寄存器逻辑左移饱和指令	1
kslli16	16 位立即数逻辑左移饱和指令	1
kslra16	16 位寄存器逻辑左移饱和或算术右移指令	1
kslra16.u	16 位寄存器逻辑左移饱和或算术右移舍入指令	2
8 位移位指令		
sra8	8 位寄存器算术右移指令	1
srai8	8 位立即数算术右移指令	1

指令名称	指令描述	执行延迟
sra8.u	8 位寄存器算术右移舍入指令	2
srai8.u	8 位立即数算术右移舍入指令	2
srl8	8 位寄存器逻辑右移指令	1
srli8	8 位立即数逻辑右移指令	1
srl8.u	8 位寄存器逻辑右移舍入指令	2
srli8.u	8 位立即数逻辑右移舍入指令	2
sll8	8 位寄存器逻辑左移指令	1
slli8	8 位立即数逻辑左移指令	1
ksll8	8 位寄存器逻辑左移饱和指令	1
kslli8	8 位立即数逻辑左移饱和指令	1
kslra8	8 位寄存器逻辑左移饱和或算术右移指令	1
kslra8.u	8 位寄存器逻辑左移饱和或算术右移舍入指令	2
16 位比较指令		
cmpeq16	16 位相等比较指令	1
scmplt16	16 位有符号小于比较指令	1
scmple16	16 位有符号小于等于比较指令	1
ucmplt16	16 位无符号小于比较指令	1
ucmple16	16 位无符号小于等于比较指令	1
8 位比较指令		
cmpeq8	8 位相等比较指令	1
scmplt8	8 位有符号小于比较指令	1
scmple8	8 位有符号小于等于比较指令	1
ucmplt8	8 位无符号小于比较指令	1
ucmple8	8 位无符号小于等于比较指令	1
16 位乘法指令		
smul16	16 位有符号乘法指令	2
smulx16	16 位有符号交叉乘法指令	2
umul16	16 位无符号乘法指令	2
umulx16	16 位无符号交叉乘法指令	2
khm16	Q15 有符号饱和乘法指令	2
khmx16	Q 15 有符号交叉饱和乘法指令	2
8 位乘法指令		
smul8	8 位有符号乘法指令	2
smulx8	8 位有符号交叉乘法指令	2
umul8	8 位无符号乘法指令	2

指令名称	指令描述	执行延迟
umulx8	8 位无符号交叉乘法指令	2
khm8	Q7 有符号饱和乘法指令	2
khmx8	Q7 有符号交叉饱和乘法指令	2
16 位杂类指令		
smin16	16 位有符号求最小值指令	1
umin16	16 位无符号求最小值指令	1
smax16	16 位有符号求最大值指令	1
umax16	16 位无符号求最大值指令	1
sclip16	16 位有符号缩减指令	1
uclip16	16 位无符号缩减指令	1
kabs16	16 位求绝对值指令	1
clrs16	16 位求前导符号位位宽指令	1
clz16	16 位求前导零位宽指令	1
clo16	16 位求前导 1 位宽指令	1
swap16	16 位交换指令	1
8 位杂类指令		
smin8	8 位有符号求最小值指令	1
umin8	8 位无符号求最小值指令	1
smax8	8 位有符号求最大值指令	1
umax8	8 位无符号求最大值指令	1
kabs8	8 位求绝对值指令	1
sclip8	8 位有符号缩减指令	1
Uclip8	8 位无符号缩减指令	1
clrs8	8 位求前导符号位位宽指令	1
clz8	8 位求前导零位宽指令	1
clo8	8 位求前导 1 位宽指令	1
swap8	8 位交换指令	1
8 位拆解指令		
sunpkd810	有符号字节 1，字节 0 拆解指令	1
sunpkd820	有符号字节 2，字节 0 拆解指令	1
sunpkd830	有符号字节 3，字节 0 拆解指令	1
sunpkd831	有符号字节 3，字节 1 拆解指令	1
sunpkd832	有符号字节 3，字节 2 拆解指令	1
zunpkd810	无符号字节 1，字节 0 拆解指令	1
zunpkd820	无符号字节 2，字节 0 拆解指令	1
zunpkd830	无符号字节 3，字节 0 拆解指令	1

指令名称	指令描述	执行延迟
zunpkd831	无符号字节 3，字节 1 拆解指令	1
zunpkd832	无符号字节 3，字节 2 拆解指令	1
部分 SIMD 指令		
16 位封装指令		
pkbb16	两个低半字封装指令	1
pkbt16	低半字和高半字封装指令	1
pktb16	高半字和低半字封装指令	1
pktt16	两个高半字封装指令	1
32x32 高位乘累加指令		
smmul	有符号 32x32 取高字指令	2
smmul.u	有符号 32x32 舍入后取高字指令	2
kmmac	有符号 32x32 取高字后饱和加指令	2
kmmac.u	有符号 32x32 舍入后取高字再饱和加指令	2
kmmsb	有符号 32x32 取高字后饱和减指令	2
kmmsb.u	有符号 32x32 舍入后取高字再饱和减指令	2
kwmmul	有符号 3 2x32 倍增后饱和取高字指令	2
kwmmul.u	有符号 32x32 倍增再舍入后饱和取高字指令	2
32x16 高位乘累加指令		
smmwb	有符号 32x 低 16 取高字指令	2
smmwb.u	有符号 32x 低 16 舍入后取高字指令	2
smmwt	有符号 32x 高 16 取高字指令	2
smmwt.u	有符号 32x 高 16 舍入后取高字指令	2
kmmawb	有符号 32x 低 16 取高字饱和加指令	2
kmmawb.u	有符号 32x 低 16 舍入后取高字再饱和加指令	2
kmmawt	有符号 32x 高 16 取高字饱和加指令	2
kmmawt.u	有符号 32x 高 16 舍入后取高字再饱和加指令	2
kmmwb2	有符号 32x 低 16 倍增后取高字指令	2
kmmwb2.u	有符号 32x 低 16 倍增后再舍入取高字指令	2
kmmwt2	有符号 32x 高 16 倍增后取高字指令	2
kmmwt2.u	有符号 32x 高 16 倍增后再舍入取高字指令	2
kmmawb2	有符号 32x 低 16 倍增后取高字再饱和加	2

指令名称	指令描述	执行延迟
	指令	
kmmawb2.u	有符号 32x 低 16 倍增后再舍入取高字后饱和加指令	2
kmmawt2	有符号 32x 高 16 倍增后取高字再饱和加指令	2
kmmawt2.u	有符号 32x 高 16 倍增后再舍入取高字后饱和加指令	2
16x32 位加减指令		
smbb16	有符号低半字乘法指令	2
smbt16	有符号低半字和高半字乘法指令	2
smtt16	有符号高半字乘法指令	2
kmda	有符号半字乘法后相加指令	2
kmxda	有符号半字交叉乘法后相加指令	2
smds	有符号半字乘法后相减指令	2
smdrs	有符号半字乘法后反向相减指令	2
smxds	有符号半字交叉乘法后相减指令	2
kmabb	有符号低半字乘法后加 32 位运算指令	2
kmabt	有符号低半字和高半字乘法后加 32 位运算指令	2
kmatt	有符号高半字乘法后加 32 位运算指令	2
kmada	有符号半字乘法后相加再加 32 位运算指令	3
kmaxda	有符号半字交叉乘法后相加再加 32 位运算指令	3
kmads	有符号半字乘法后相减再加 32 位运算指令	3
kmadrs	有符号半字乘法后反向相减再加 32 位运算指令	3
kmaxds	有符号半字交叉乘法后相减再加 32 位运算指令	3
kmsda	有符号半字乘法后相减再被 32 位减运算指令	3
kmsxda	有符号半字交叉乘法后相减再被 32 位减运算指令	3
16x64 位加减法指令		
smal	16x16 再加 64 位运算指令	3
杂类指令		
sclip32	32 位有符号缩减指令	1

指令名称	指令描述	执行延迟
uclip32	32 位无符号缩减指令	1
clrs32	32 位求前导符号位位宽指令	1
clz32	32 位求前导零位宽指令	1
clo32	32 位求前导 1 位宽指令	1
pbsad	字节差分绝对值求和指令	1
pbsada	字节差分绝对值求和再累加指令	1
8x32 位加法指令		
smaqa	8 位有符号乘法累加再加 32 位运算指令	3
umaqa	8 位无符号乘法累加再加 32 位运算指令	3
smaqa.su	8 位有符号和无符号乘法累加再加 32 位运算指令	3
64 位 Profile 指令		
64 位加减法指令		
add64	64 位加法指令	2
radd64	64 位有符号加法后减半指令	2
uradd64	64 位无符号加法后减半指令	2
kadd64	64 位有符号饱和加法指令	2
ukadd64	64 位无符号饱和加法指令	2
sub64	64 位减法指令	2
rsub64	64 位有符号减法后减半指令	2
ursub64	64 位无符号减法后减半指令	2
ksub64	64 位有符号饱和减法指令	2
uksub64	64 位无符号饱和减法指令	2
32 位相乘与 64 位加减法指令		
smar64	有符号 32x32 再加 64 位指令	2
smsr64	有符号 32x32 被 64 位减指令	2
umar64	无符号 32x32 再加 64 位指令	2
umsr64	无符号 32x32 被 64 位减指令	2
kmar64	有符号 32x32 再加 64 位后饱和指令	2
kmsr64	有符号 32x32 被 64 位减后饱和指令	2
ukmar64	无符号 32x32 再加 64 位后饱和指令	2
ukmsr64	无符号 32x32 被 64 位减后饱和指令	2
16 位相乘与 64 位加减法指令		
smalbb	有符号低 16x 低 16 再加 64 位指令	3
smalbt	有符号低 16x 高 16 再加 64 位指令	3
smaltt	有符号高 16x 高 16 再加 64 位指令	3

指令名称	指令描述	执行延迟
smalda	有符号半字相乘累加后再加 64 位指令	3
smalxda	有符号半字交叉相乘累加后再加 64 位指令	3
smalds	有符号半字相乘累减后再加 64 位指令	3
smaldrs	有符号半字相乘反向累减后再加 64 位指令	3
smalxds	有符号半字交叉相乘累减后再加 64 位指令	3
smslda	有符号半字相乘累减后再被 64 位减指令	3
smslxda	有符号半字交叉相乘累减后再被 64 位减指令	3
非 SIMD 指令		
Q15 饱和指令		
kaddh	低半字相加后饱和指令	2
ksubh	低半字相减后饱和指令	1
khmbb	低半字相乘后饱和到 Q15 指令	2
khmbt	低半字和高半字相乘后再饱和到 Q15 指令	2
khmtt	高半字相乘后饱和到 Q15 指令	2
ukaddh	无符号低半字相加后饱和指令	2
uksubh	无符号低半字相减后饱和指令	1
Q31 饱和指令		
kaddw	32 位相加饱和到 Q31 指令	1
ukaddw	无符号 32 位相加饱和指令	1
ksubw	32 位相减饱和到 Q31 指令	1
uksubw	无符号 32 位相减饱和指令	1
kdmbb	低半字相乘倍增后饱和到 Q31 指令	2
kdmbt	低半字和高半字相乘倍增后饱和到 Q31 指令	2
kdmtt	高半字相乘倍增后饱和到 Q31 指令	2
kslraw	寄存器逻辑左移后饱和或算术右移指令	1
kslraw.u	寄存器逻辑左移后饱和或算术右移后舍入指令	2
ksllw	寄存器逻辑左移后饱和指令	1
kslliw	立即数逻辑左移后饱和指令	1
kdmabb	低半字相乘后倍增再和 32 位饱和加之后再饱和指令	2
kdmabt	低半字和高半字相乘后倍增再和 32 位饱和加之后再饱和指令	2
kdmatt	高半字相乘后倍增再和 32 位饱和加之后再	2

指令名称	指令描述	执行延迟
	饱和指令	
kabsw	32 位求绝对值后再饱和指令	1
32 位计算指令		
raddw	32 位有符号加后减半指令	1
uraddw	32 位无符号加后减半指令	1
rsubw	32 位有符号减后减半指令	1
ursubw	32 位无符号减后减半指令	1
maxw	32 位有符号求最大值指令	1
minw	32 位有符号求最小值指令	1
mulr64	无符号 32x32 运算指令	2
mulsr64	有符号 32x32 运算指令	2
msubr32	32x 32 取低字再被 32 位累减指令	2
饱和/溢出状态操作指令		
Rdov	读 vxsat.OV 指令	阻塞执行
clrov	清除 vxsat.OV 指令	阻塞执行
杂类指令		
ave	求平均值后舍入指令	1
sra.u	寄存器算术右移后舍入指令	2
srai.u	立即数算术右移后舍入指令	2
bitrev	按寄存器比特位倒序指令	1
bitrevi	按立即数比特位倒序指令	1
wext	按寄存器从 64 位中抽取 32 位指令	1
wexti	按立即数从 64 位中抽取 32 位指令	1
bpick	按寄存器值选择比特封装指令	1
insb	字节插入指令	1
maddr32	3 2x32 取低字再累加 32 位指令	2
msubr32	32x 32 取低字再被 32 位累减指令	2

2.4 CPU 异常与中断

2.4.1 异常

异常处理（包括指令异常和外部中断）是处理器的一项重要技术，在异常事件产生时，用来使处理器转入对异常事件的处理。

异常处理是处理器根据内部或外部的异常事件从正常的程序处理转入特定的异常处理程

序。引起异常的外部事件包括：外部设备的中断请求、读写访问错误；引起异常的内部事件包括：非法指令和非对齐访问错误（misaligned error）。ECALL 和 EBREAK 指令正常执行时也会产生异常。异常处理利用异常向量表跳转到异常服务程序的入口。

异常处理的关键是在异常发生时，保存 CPU 当前指令运行的状态，在退出异常处理时恢复异常处理前的状态。异常能够在指令流水线的各个阶段被识别，并使后面的指令不会改变 CPU 的状态。异常在指令的边界上被处理，即 CPU 在指令退休时响应异常，并保存退出异常处理时下一条被执行的指令的地址。即使异常指令退休前被识别，异常也要在相应的指令退休时才会被处理。CPU 根据异常识别时的指令是否完成决定异常地址寄存器存储哪一条指令的地址。例如，如果异常事件是外部中断服务请求，被中断的指令将正常退休并改变 CPU 的状态，它的下一条指令的地址（PC+2/PC+4，根据当前指令是 16 位或 32 位决定 +2 或者 +4）将被保存在异常保留程序计数器（MEPC）中作为中断返回时指令的入口；如果异常事件是由访问错误指令产生的，因为这条指令不能完成，它将异常退休但不改变 CPU 的状态（即不改变寄存器的值），这条访问错误地址指令的地址（PC）将被保存在异常保留程序计数器（MEPC）中，CPU 从异常服务程序返回时继续执行这条访问错误指令。

CPU 兼容 RISC-V 标准的异常向量号，如下表 2-13 所示：

表 2-13 CPU 异常向量表

中断标记	向量号	异常中断类型
1	0-2	未实现/保留
1	3	机器模式软件中断
1	4-6	未实现/保留
1	7	机器模式计时器中断
1	8-10	未实现/保留
1	11	机器模式外部中断
1	>=12	保留
1	16	CLIC 外接中断 0（pad_clic_int_vld[0]）
1	……	……
1	16+i	CLIC 外接中断 i（pad_clic_int_vld[i]）
1	……	……
1	101	CLIC 外接中断 85（pad_clic_int_vld[85]）
0	0	未实现
0	1	取指令访问错误异常
0	2	非法指令异常
0	3	调试断点异常
0	4	加载指令非对齐访问异常

中断标记	向量号	异常中断类型
0	5	加载指令访问错误异常
0	6	存储指令非对齐访问异常
0	7	存储指令访问错误异常
0	8	用户模式环境调用异常
0	9	未实现
0	10	保留
0	11	机器模式环境调用异常
0	12~23	未实现/保留
0	24	NMI
0	>=25	未实现/保留

1. 异常响应

RISC-V 编程模型中没有异常使能寄存器，因此一旦触发异常即可进行响应。按照 RISC-V 标准定义，中断优先级高于异常，异常内部优先级定义如下表 2-14 所示。CPU 将 NMI 的异常向量号定义为 24，并将其设置为所有中断和异常中优先级最高的异常类型。

表 2-14 CPU 异常优先级

优先级	向量号	异常类型
最高	3	调试断点异常
↓	1	取指令访问错误异常
↓	2	非法指令异常
↓	8	用户模式环境调用异常
↓	11	机器模式环境调用异常
↓	6	存储指令非对齐访问异常
↓	4	加载指令非对齐访问异常
↓	7	存储指令访问错误异常
↓	5	加载指令访问错误异常

异常响应会打断 CPU 正常的程序执行轨迹，转而处理该异常事件，因此在异常响应时需要对 CPU 现场状态进行保存，并在 CPU 退出异常服务程序时恢复现场并执行异常响应前的程序流。异常响应按如下步骤进行现场保存，所有步骤在一个处理器时钟周期完成：

1) 处理器保存 PC 到异常保留程序计数器（MEPC）中。

2) 将 MCAUSE 中的异常向量号 Exception Code 域更新为当前发生的异常向量号。

3) 将 MSTATUS 中的中断使能位 MIE 保存到 MPIE 中。

4) 将 MSTATUS 中的中断使能位 MIE 位清零，禁止响应中断。

5) 将 MXSTATUS 中的 PM 域保存到 MSTATUS 寄存器的 MPP 域，并将 PM 设置为机器模式，即异常响应后 CPU 进入机器模式。

6) 将产生异常事件的原因更新到 MTVAL 寄存器。

7) 处理器根据向量基址寄存器 MTVEC 中的基址得到异常服务程序入口地址进行跳转执行。

上述第 1 步中，在精确异常模式下保存触发异常的指令 PC 值到 MEPC 寄存器中，在非精确异常模式下，MEPC 不能精确保存触发异常的指令 PC 值，而有可能是指令流执行顺序上触发异常指令后面的指令对应 PC 值，但这种情况下 MTVAL 仍是正确更新。上述第 6 步中，响应 NMI 时不会更新 MTVAL 寄存器，可以保证即便在异常服务程序中响应 NMI 请求也不会覆盖 MTVAL 寄存器。关于 NMI 的相关处理描述请参考 NMI。所有异常的跳转入口均由 MTVEC 寄存器定义，从该地址取回的第一笔数据即为异常服务程序的第一条指令。因为 CPU 只实现 MTVEC.mode=3 这一模式，因此要求异常服务程序的入口地址为 64 字节对齐。具体请参考机器模式异常向量基址寄存器（MTVEC）。

2. 异常处理

如上节所述，所有异常响应时的跳转执行入口均由 MTVEC 寄存器指定，因此在 CPU 跳转到该入口执行程序时，软件可依据 MCAUSE 寄存器中的异常向量号来决定是否在异常服务程序中实现再次跳转到各自对应的服务程序进行处理。需要注意的是，除了在异常响应时对必要的 CSR 寄存器及 PC 等现场状态进行保存外，在异常服务程序入口需要软件对 GPR 等需要用到的寄存器进行压栈处理。

3. 异常返回

异常服务程序的返回需要通过执行 MRET 指令实现。MRET 指令执行时会将异常响应时保存的 CPU 现场进行恢复，主要包括如下方面：

1) 将 PC 恢复成 MEPC 寄存器的值，精确异常模式下可以保证 CPU 从异常服务程序返回后可以从触发异常的地方重新执行指令。这就要求上节的异常服务程序中将触发该异常的事件进行修复，避免再次触发异常。

2) MSTATUS.MIE 被恢复成 MSTATUS.MPIE 的值，MPIE 被设置为 1。

3) MXSTATUS.PM 域被恢复成 MSTATUS.MPP 的值，MPP 被设置为 2'b00（硬件实现用户模式时）或者 2'b11（硬件仅实现机器模式时）。需要说明的是在从异常服务程序返回时，硬件不会对 MCAUSE 寄存器里面的 EXCEPTION.CODE 域进行清除。

4. 锁定

如前文所述，RISC-V 编程模型中没有定义异常的使能寄存器，在从异常服务程序返回到正

常程序流时也没有对 MCAUSE 寄存器中的异常向量号进行清除，因此软件开发人员不能方便的从编程模型所定义寄存器中得知 CPU 当前正在处理异常还是运行正常程序轨迹。CPU 中实现了扩展异常状态寄存器（MEXSTATUS），当 CPU 响应异常时会将该寄存器的 EXPT_VLD 域置为 1，通过 MRET 指令从异常服务程序返回后该域被清零。

当 CPU 响应异常时会判断当前程序流是否处于异常服务程序中（通过 MEXSTATUS 的 EXPT_VLD 域），如果 CPU 正在从异常服务程序返回时又触发新的异常，将会导致 CPU 被锁定。CPU 被锁定时会置位 CPU 顶层输出信号（cpu_pad_lockup）为高，告知 SoC CPU 处于锁定状态，同时 CPU 停止指令取指、执行并保持 PC 为触发 CPU 锁定的指令 PC，MEXSTATUS 寄存器中的 LOCKUP 域被设置为 1。

需要说明的是，在 CPU 响应异常时，即便再次响应 NMI 请求也不会导致 CPU 的锁定，反之如果在 NMI 的服务程序中触发异常，CPU 则会进入锁定状态。

调试请求和下文描述的 NMI 请求可以将 CPU 的锁定状态打断，CPU 传递给 SoC 的锁定指示信号被拉低，此时，在调试模式下可以正常调试代码，分析 CPU 被锁定的原因。NMI 打断 CPU 锁定状态时，NMI 服务程序也可以正常执行。

在 CPU 从 NMI 处理函数返回后仍将处于锁定状态。在 CPU 从调试模式退出后有所差异，在调试模式下如果软件将 CPU 退出调试模式跳转执行的 PC 设为 0xEFFFFFFC，CPU 在退出调试后仍将处于锁定状态。如果软件不将 CPU 退出调试模式跳转执行的 PC 设为 0xEFFFFFFC，CPU 在退出调试模式时锁定状态也被清除（包括传递给 SoC 的指示 CPU 处于锁定状态的 LOCKUP 有效信号以及 MEXSTATUS.LOCKUP），以及导致 CPU 进入锁定状态的异常信息（MEXSTATUS.EXPT_VLD），CPU 继续执行指令。

当 CPU 处于锁定状态时，建议系统设计人员将 CPU 核进行复位。特殊说明的是，执行 ECALL 或者 EBREAK 主动触发的异常在和任何异常类型（包括 NMI）进行嵌套时都不会触发 CPU 的锁定。

5. 非对齐访问异常

在 RISC-V 标准定义中，当取指或者内存数据读写操作地址不对齐时会上报相应的非对齐访问异常。CPU 取指令地址在 CPU 复位启动后由硬件维护，不会产生非对齐访问异常。内存数据数据操作的地址由软件定义，因此当访存的数据位宽和地址不对齐时按照 RISC-V 标准需产生加载指令或者存储指令非对齐访问异常。对于双精度浮点来说，双精度内存加载或者存储地址需要是双字对齐。

为了降低软件编程复杂度，CPU 扩展实现了非对其访问异常掩码开关，由扩展状态寄存器 MXSTATUS 中的扩展状态寄存器（MXSTATUS）中的 MM 位来控制非对齐内存访问是按照标准定义触发非对齐访问异常抑或不上报异常，硬件做拆分访问处理。

6. 精确与非精确异常

处理器在发生异常时，会把发生异常时处理器的 PC 值存入 MEPC 寄存器中，如果 MEPC

指向的是真正引起异常的指令，则称之为精确异常，反之则是非精确异常。

CPU 支持总线返回错误引发的访问错误异常精确响应和非精确响应两种方式。由扩展隐式操作寄存器（MHINT)中的精确异常使能位控制，默认模式为总线访问错误非精确响应。当精确异常使能位打开后，总线访问错误异常得到精确响应。对于非精确总线错误异常，存入 MEPC 寄存器的 PC 值不一定指向真正引发总线错误的指令，但是 MTVAL 存放的内存访问地址是引发总线错误的地址。

在非精确异常模式下，除了 Load 和 Store 指令访问内存触发的内存访问错误异常外，其他异常都是精确异常，包括 PMP 权限违反导致的内存访问错误异常。但是因 PMP 权限违反导致的内存加载或存储访问错误异常与外部总线返回错误导致的访问错误异常共享异常向量号，因此 CPU 在 MEXSTATUS 寄存器中扩展实现了 BUSERR 指示位，用于区分具体是由哪种情况导致的内存访问错误异常。对于精确响应的异常，CPU 响应异常时存入 MEPC 和 MTVAL 的值都是触发异常的指令对应的信息。非精确异常模式可以使得 CPU 的 Load/Store 内存访问性能得到进一步的提升。在非精确异常模式下调试内存访问错误异常（5 号和 7 号）时，可以根据异常向量号和 MEPC 回溯到最近的一条 Load 或者 Store 指令，然后根据 MEXSTATUS 中的 BUSERR 位和 MTVAL 寄存器来定位分析。

2.4.2 中断

中断作为外部事件请求，处理器在响应中断时需要进行如下判断：

- 中断使能位是否开启以及中断优先级是否足够；
- 处理器需要保存的现场有哪些；
- 处理器跳转执行的中断服务程序入口在哪里；
- 在从中断服务程序返回时处理器现场如何恢复；

CPU 设计实现了 RISC-V 标准的 CLINT 中断，包括机器模式软件中断、机器模式计时器中断以及机器模式外部中断，RISC-V 标准的中断控制器 CLIC。

在 CLIC 中只有符合条件的中断源才会参与仲裁。需满足的条件如下：

- 中断源处于等待状态（IP=1）；
- 中断优先级大于 MINTTHRESH 域值寄存器设定的域值（中断优先级非零才可正常参与仲裁）；
- CLIC 中该中断使能位为 1（IE=1）。

当 CLIC 中有多个中断处于等待状态时，CLIC 仲裁出优先级最高的中断。CLIC 中中断优先级配置寄存器(CLICINTCTL) 的值越大，优先级越高，优先级为 0 的中断无效；如多个中断拥有相同的优先级，则中断 ID 较大的优先处理。CLIC 会将仲裁结果包括中断 ID，优先级，特权态是否为矢量中断的信息传递给 CPU 流水线核心。其中，中断 ID 作为中断号进

行处理，CLINT 中断对应中断号为 0~15，CLIC 外接的中断源中断号为 16~101。CPU 内部设计实现的 CLIC 兼容 CLIC SPEC-0.8 版本，按照 SPEC 定义，硬件实现 CLIC 时，MTVEC.mode[1:0]扩展出 2’ b11 这一模式，支持硬件矢量中断和非矢量中断，中断服务程序入口可由 MTVT 寄存器指定。CPU 只实现了 MTVEC.mode[1:0]=3 这一模式，本节主要对该模式下矢量中断和非矢量中断的处理进行描述。

2.4.2.1 矢量中断

可通过将 CLIC 中每个中断的中断属性寄存器（CLICINTATTR）中的 shv 域设置为 1 来指示该中断为硬件矢量中断。

中断优先级

CPU 支持中断优先级有效位 2-5 位硬件可配，最多支持 32 个中断优先级。中断优先级的设置分为两步：

1) 设置 CLIC 寄存器 CLICCFG 中的 nlbits 域，该位指示了可配置的中断优先级个数的多少；
2) 设置每个中断所对应寄存器 CLICINTCTL 的 int_ctl 域，指示该中断参与优先级仲裁的有效值及保存在 MINTSTATUS.MIL 寄存器域的有效值

具体寄存器配置请参考 CLIC 中断控制器。

中断响应

为了保证中断被正常响应，必须保证全局中断使能位 MSTATUS.MIE 为 1 以及该中断对应的中断使能寄存器（CLICINTIE）中的使能位为 1。

中断响应按以下步骤被处理，所有步骤在一个处理器时钟周期完成：

1) 处理器保存 PC 到异常保留程序计数器（MEPC）中。因为中断优先级比异常高，如果响应中断的指令本身同时触发异常，保存在 MEPC 寄存器中的值会是响应中断的指令本身，否则为响应中断指令的下一条指令。
2) 将 MCAUSE 中的异常向量号 Exception Code 域更新为当前有效的中断号，具体向量号如 RIS 异常向量号所示。并且将 MCAUSE 寄存器最高位置为 1，表示 CPU 响应的是中断。
3) 将 MSTATUS 中的中断使能位 MIE 保存到 MPIE 中。
4) 将 MSTATUS 中的中断使能位 MIE 位清零，禁止响应中断。
5) 将 MXSTATUS 中的 PM 域保存到 MSTATUS 寄存器的 MPP 域，并将 PM 设置为机器模式 3，即中断响应后 CPU 进入机器模式。
6) 将 MCAUSE 寄存器的 MPIL 域更新为 MINTSTATUS 寄存器的 MIL 域，MINTSTATUS 寄存器的 MIL 域被更新为当前被响应的中断的优先级。

7) 对于脉冲中断而言，CPU 会自动清除其对应 CLICINTIP 寄存器中的 pending 位，而对于电平中断而言则不会清除，需要软件在异常服务程序中清除。

在 MTVEC.mode=3 时，硬件矢量中断的服务程序入口是由 MTVT 寄存器指定的基址加中断号所指定的偏移量决定的，具体请参考机器模式矢量中断基址寄存器（MTVT）。

中断处理

在处理器跳转到中断服务程序执行时，首先需要对处理器之前正常运行的通用寄存器现场进行保存。在运行实时操作系统时，为了减少每个任务的中断现场保存所占空间，会需要一个单独的所有任务共享的中断栈。RISC-V 定义了 MSCRATCHCSWL 寄存器，它的实体寄存器为 MSCRATCH。假如当前中断是 CPU 从正常执行程序流跳转过来响应的，对 MSCRATCHCSWL 寄存器的读写可实现 sp 与 MSCRATCH 寄存器的交换，而 MSCRATCH 寄存器可以保存中断栈的指针值，进而在中断服务程序中使用单独的中断栈进行现场保存和恢复。

在实时操作系统中，CPU 所配套的 SDK 采用 3 号机器模式软件中断来进行 task 的切换，在 3 号中断的服务程序中所保存和恢复的 CPU 现场需要存储在各 task 的栈空间中，因此在 3 号中断服务程序中所用的 SP 不能是中断栈，另外 NMI 的服务程序中所用栈指针也不是中断栈指针。假如需要嵌套响应中断，需要对当前的 MSTATUS，MCAUSE，MINTSTATUS 等寄存器进行保存并重新打开 MSTATUS 中的中断使能位。这些操作均属于真正的中断处理操作之前的必须操作。

为了加速中断的响应，尽快开始真正的中断事务处理，首先在扩展异常状态寄存器 MEXSTATUS 中定义了 SPSWAPEN 位，可由软件控制是否开启上述中断栈的硬件切换功能。另外，CPU 上定义实现了扩展中断加速压栈指令 IPUSH，该指令可实现对 ABI 所定义的中断入口所要保存的 GPR（X1，X5-X7，X10-X17，X28-X31，共 16 个 GPR）进行压栈，并且对 MEPC 以及 MCAUSE 两个在中断嵌套时需要保存 CSR 进行压栈。该指令执行的最后一个功能是设置 MSTATUS 寄存器中的 MIE 位，使能中断。

为了进一步加快中断的响应，扩展异常状态寄存器 MEXSTATUS 中定义了 SPUSHEN 位，当该位设置为 1 时，CPU 在响应中断时，会投机的执行一条 IPUSH 指令。执行执行过程中会与从中断服务程序入口取回的指令码进行比对，判断是否投机成功。当投机失败时，即从中断服务程序入口取回的第一条指令不是 IPUSH 指令，投机执行的 IPUSH 指令不产生任何实际效果。

中断服务程序入口的 IPUSH 指令可由编译器编译生成，用户只需在高阶语言实现的中断服务函数上添加指定描述符即可。

如果中断服务程序入口需要对浮点寄存器现场进行压栈保存，可由软件通过浮点 Store 指令完成。

中断返回

中断服务程序的退出必须使用 MRET 指令完成，在执行 MRET 指令之前需要软件将中断服务程序入口压栈保存的现场进行弹栈处理。通过执行 MRET 指令将响应中断之前的 CPU 现场进行恢复，主要有如下操作：

1) 将 PC 恢复成 MEPC 寄存器的值；
2) 将 MXSTATUS 寄存器中的 PM 域恢复成 MSTATUS.MPP，MPP 被设置为 2'b00（硬件实现用户模式时）或者 2'b11（硬件仅实现机器模式时）；
3) MSTATUS.MIE 恢复成 MSTATUS.MPIE 的值；
4) MINTSTATUS.MIL 被更新成 MCAUSE.MPIL 的值。

当中断服务程序入口是 IPUSH 指令完成的现场压栈时，中断返回操作的同样可由 CPU 定义实现的 IPOP 指令完成。IPOP 指令会先将 MSTATUS 寄存器中的中断使能位关闭，并对 IPUSH 指令压栈的现场进行对应的按序弹栈，最后拆分出 MRET 指令进行中断返回。同 IPUSH 指令一样，IPOP 指令同样可由编译器编译生成，因为 IPOP 最后会拆分出 MRET 指令，因此在中断服务程序的最后无需再加上 MRET 指令。为了在 IPOP 执行过程中加快对更高优先级中断的响应，一旦 CLIC 仲裁出更高优先级中断会打断 IPOP 的执行，转而响应更高优先级的中断，在高优先级中断返回后重新执行这条 IPOP 指令。如果中断服务程序中需要对浮点寄存器现场进行弹栈操作，需要在 IPOP 指令之前通过浮点 Load 指令完成。

中断咬尾

中断咬尾是指在中断服务程序中中断事务处理完成后，处理器现场弹栈之前，对 CLIC 中等待处理的较低优先级中断（如果存在的话）进行查询，如果该优先级比 MCAUSE.mpil 寄存器值大的话，会优先响应该等待处理的中断，跳过当前按中断服务程序中的处理器现场弹栈操作，在新响应的中断服务程序中再进行现场的弹栈。如果采用 IPOP 指令来实现中断的返回，在 IPOP 指令执行之初，会对 CLIC 仲裁出来的等待响应的低优先级中断进行查询响应，并投机的认为新响应中断的入口是 IPUSH 指令，实现中断的咬尾操作。因为处理器现场是在新响应的中断服务程序的最后进行弹栈回复，这就要求咬尾响应的中断入口必须是 IPUSH 指令，结尾同样通过 IPOP 指令实现中断的返回。

成功进行中断咬尾时，当前中断的 IPOP 和咬尾中断的 IPUSH 指令会立即执行完成，同时不产生任何实际效果。如果 IPOP 指令执行最后确实需要中断返回，即没有咬尾中断，如果 MCAUSE.MPIL 为 0，代表当前中断为嵌套响应的最外层中断，在扩展异常状态寄存器 MEXSTATUS 中定义了 SPSWAPEN 位为 1 时，硬件会将中断栈和 SP 寄存器进行切换。

2.4.2.2 非矢量中断

可通过将 CLIC 中每个中断的配置寄存器 CLICINTATTR 中的 shv 域设置为 0 来指示该中断为非矢量中断。

中断响应

跟中断的 pending 位，无论脉冲中断还是电平中断。需要软件在中断服务程序中使用读 MNXTI 寄存器的方式触发 pending 位的清除，相关介绍请参考中矢量中断的中断响应类似，CPU 会保存响应中断时的处理器现场，所不同的是 CPU 在响应中断时不会主动清除 CLIC 内对应断咬尾。另外，对于非矢量中断而言，中断服务程序入口都是统一由 MTVEC 寄存器指定，不同于硬件矢量中断的由 MTV 加中断号偏移量指定的模式。另外，从该地址取回的数据即为中断服务程序的第一条指令。

中断处理

因为非矢量中断的中断服务入口和异常处理入口相同，都由 MTVEC 寄存器指定，因此非矢量中断服务程序的第一条指令无法使用 IPUSH 指令（因为它同样是异常处理程序的第一条指令）。在中断服务程序的入口，可通过 MCAUSE 寄存器的 INTR 域分辨出异常和中断，进而跳转到指定中断服务程序处理。真正的中断事务处理前的处理器现场可通过标准的 Store 指令完成。

中断返回

与上节的中断处理类似，中断返回前可先通过标准的 Load 指令完成处理器现场的弹栈，接着执行 MRET 指令实现从中断服务程序返回到响应中断之前的执行程序流。

中断咬尾

CLIC SPEC 在非硬件矢量中断模式下定义了中断咬尾处理的功能，同时支持在中断服务程序入口响应晚到且高优先级中断的功能。因为在非硬件矢量中断模式下，中断服务程序的入口统一由 MTVEC 寄存器指定，因此可以在中断服务程序入口做统一的寄存器压栈操作。在压栈完成之后可以读取 MNXTI 寄存器的值，如果返回非零值即当前处于等待状态的 CLIC 仲裁出来的最高优先级的中断（可能是当前正在被处理的中断也有可能是新近的更高优先级的中断），并且该中断在 CLIC 内部的 pending 位被清除（仅限脉冲中断），同时通过读写 MNXTI 寄存器的操作可以主动设置 MSTATUS.MIE 值为 1。

同样，在非矢量中断本身的中断事务处理完后，在进行现场弹栈及执行 MRET 返回前，可通过执行读写 MNXTI 寄存器的指令，获取 CLIC 仲裁出来的比 MCAUSE.MPIL 优先级还高的中断入口地址，该地址由 MTVT 加中断号指定的偏移量决定。软件可据此地址从内存加载该等待响应的中断的服务程序入口地址并跳转执行。如果返回非零值，即不需要处理 CLIC 仲裁出来的中断（或者 CLIC 根本没有待处理的中断），可以直接进行弹栈操作和中断返回。

2.4.3 NMI

NMI 作为外部事件，E906 将其异常向量号定义为 24，优先级在所有的中断和异常中最高。因为 NMI 不受全局中断使能位 MSTATUS.MIE 的控制，因此为了使得任何时候响应 NMI 并返回之后，CPU 都可以恢复正常运行程序流，E906 扩展实现了 MNMICAUSE 寄存器和 MNMIPC 寄存器用于保存响应 NMI 时 CPU 的现场。

NMI 响应

CPU 在响应 NMI 时会将当前的状态信息保存下来，主要有：

- 将 MEPC 寄存器的值保存在 MNMIPC 寄存器；
- 将响应 NMI 请求时的指令 PC 保存在 MEPC 寄存器，便于 NMI 处理结束后 CPU 返回正常程序轨迹；
- 将 MCAUSE 寄存器域的 Exception Code 以及中断指示位保存在 MNMICAUSE 寄存器，并更新 MCAUSE 寄存器的 Exception Code 为 12 ‘h18，bit[31]中断标志位设置为 0；
- 将 MSTATUS 寄存器域的 MPP 及 MPIE 域保存在 MNMICAUSE 寄存器，并将 MXSTATUS 寄存器的 PM 域更新到 MPP，将 MSTATUS 的 MIE 域保存在 MPIE；
- 将扩展异常状态寄存器 MEXSTATUS 中的 NMI 域值置为 1，表明处理器正在处理 NMI；
- 硬件不会更新 MTVAL 寄存器。

在上述 CPU 状态保存的同时，CPU 会依据 MTVEC 寄存器指定的地址统一异常入口地址，跳转执行 NMI 的服务程序。

在 NMI 被响应直到 CPU 返回正常程序轨迹之前，新的 NMI 请求以及中断请求都无法被响应，当在 NMI 服务程序中触发异常时会使 CPU 进入锁定的状态，如 4.3.3 节所述。因为 NMI 状态维护依赖 MCAUSE 寄存器中的 Exception Code 域，因此软件要避免在 NMI 服务程序中改写该寄存器域。

NMI 返回

在 NMI 服务程序返回时需要通过执行 MRET 指令将 CPU 的现场进行恢复，主要有如下操作：

- 将 PC 恢复成 MEPC 寄存器的值，将 MEPC 寄存器的值恢复成 MNMIPC 寄存器的值；
- 将 MCAUSE.exception_code，中断指示位，恢复成 MNMICAUSE 中保存的值；
- MXSTATUS.PM 被恢复成 MSTATUS.MPP 域的值；
- 将 MSTATUS.MPIE 和 MSTATUS.MPP 的值恢复成 MNMICAUSE 中保存的值；

- MNMICAUSE 中的 NMI_MPP 被恢复成 2 ‘b00（硬件实现用户模式时）或者 2’b11（硬件仅实现机器模式时）。

锁定

在 NMI 的服务程序中如果触发异常，CPU 会进入锁定状态。如锁定节所述，CPU 会停止取指和执行，并设置 MEXSTATUS 寄存器的 LOCKUP 域为 1。这时即便新的 NMI 请求也不能将 CPU 的锁定状态打断。来自 SoC 的输入复位请求信号可以将 CPU 的锁定状态彻底打断，使得 CPU 重新从复位启动地址开始执行程序。

2.5 CLIC 中断控制器

核内局部中断控制器（以下简称 CLIC），仅用于对中断源进行采样，优先级仲裁和分发。CLIC 仲裁来源包括处理器各个模式下触发的中断。

E906 实现的 CLIC 单元基本功能如下：

- 支持 RISC-V 标准 CLIC spec-0.8 版本；
- 最多支持 86 个外部中断源可配，支持电平中断，脉冲中断，加上兼容 CLINT 的至多 16 个中断（目前仅实现机器模式软件中断和机器模式计时器中断），CLIC 共支持 102 个中断处理；
- 中断优先级有效位 CLICINTCTLBITS 2-5 可配，最多 32 个级别的中断优先级；
- 每个中断目标拥有 4 个 memory-mapped 的控制寄存器；
- 通过写相应中断源的控制寄存器可以配置此中断源的各个属性；
- 支持了可选的 MSCRATCHCSW，MSCRATCHCSWL 寄存器，供中断处理函数内快速交换栈指针使用。

2.5.1 CLIC 寄存器地址映射

CLIC 中断控制器占据 20KiB 内存空间，其地址映射如下表 2-15 所示。其中 CLICCFG，CLICINFO，MINTTHRESH 寄存器仅支持地址字对齐的访问。

表 2-15 CLIC 地址映射

地址	名称	类型	初始值	描述
0xE0800000	CLICCFG	RW	0x1	CLIC 配置寄存器
0xE0800004	CLICINFO	RO	xx	CLIC 信息寄存器
0xE0800008	MINTTHRESH	RW	0x0	中断阈值寄存器
0xE0801000+4*i	CLICINTIP[i]	R or RW	0x0	中断源 i 等待寄存器

地址	名称	类型	初始值	描述
0xE0801001+4*i	CLICINTIE[i]	RW	0x0	中断源 i 使能寄存器
0xE0801002+4*i	CLICINTATTR[i]	RW	0x0	中断源 i 属性寄存器
0xE0801003+4*i	CLICINTCTL[i]	RW	0x0	中断源 i 控制寄存器

2.5.2 CLIC 寄存器描述

1. CLIC 配置寄存器（CLICCFG）

CLIC 有一个 8-bit 的全局配置寄存器 CLICCFG，其中定义了支持的中断响应特权态，CLICINTCTL[i]的划分，以及是否支持硬件矢量中断。

7	6	5	4	3	2	1	0
Res.	nmbits		nlbits				nvbits
-	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0

nmbits-特权态有效位数：

CLICINTATTR[i].mode 中有效的位数。由于 E906 仅支持机器模式下处理中断，所以该位绑为 0。CLICINTATTR[i].mode 域中的值无论为何值均认为是机器模式响应中断。

nlbits-中断优先级有效位数：

CLICINTCTL[i]中作为中断优先级的位数。CLIC 产生的中断优先级位数为固定 8 位，不足的部分由 1 进行填充。

nvbits-硬件矢量中断实现标志位：

代表 CLIC 控制器是否支持矢量模式中断，该位恒为 1，表示支持硬件矢量模式中断。开启中断硬件矢量模式需要将中断对应的 CLICINTATTR.shv 置 1。硬件矢量中断的服务程序入口地址采用硬件两级跳转的方式获取，首先在保存完处理器现场后 CPU 从 MTVT+中断 ID*4 的地址处取数据，该数据被认为是该中断 ID 的中断服务程序的入口地址，CPU 跳转到该地址去执行中断服务程序。非硬件矢量中断的服务程序入口是 MTVEC[31:6]<<6，CPU 跳转到该地址去处理中断。

2. CLIC 信息寄存器（CLICINFO）

CLICINFO 为一个只读寄存器，里面提供了 CLIC 的部分信息。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							CLICCTLBITS				version				
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	x	x	x	x	x

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
version			num interrupt												
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
x	x	x	0	0	0	0	0	0	0	0	0	0	0	0	0

CLICCTLBITS-CLICINTCTL 有效位数：

CLICINTCTL 寄存器内优先级有效位数。实现的有效位数在 CLICINTCTL[i]中左对齐。

version-版本信息：

其中低 4 位为硬件实现的修改版本；高四位为 CLIC 架构版本信息。

num_interrupt-中断源数量：

代表该 CLIC 控制器硬件支持的中断源个数。

3. 中断阈值寄存器（MINTTHRESH）

阈值寄存器定义了当前处于等待状态的中断请求能够向 CPU 流水线核心发起中断请求的优先级临界值。中断阈值寄存器为每一个特权模式提供了一个 8 位的域作为相应的中断阈值。处于等待状态的中断请求的优先级必须高于 MINTTHRESH 寄存器定义的阈值才能向处理器发起中断。

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
mth								Res.							
RW	RW	RW	RW	RW	RW	RW	RW	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

mth-机器模式阈值：

机器模式中断的阈值。由于 E906 仅支持机器模式中断，所以仅有一个阈值域。

4. 中断等待寄存器（CLICINTIP）

7	6	5	4	3	2	1	0
Res.							IP
-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0

IP-中断等待：

中断源是否有中断等待响应。该位在电平中断和边缘中断情况下有不同的置位与清除逻辑。

电平中断模式下，CLICINTIP 为只读寄存器。更改 CLICINTIP 的值需要通过直接对外部中断源进行操作来实现，外部中断源为高则 IP 为 1，外部中断源为低则 IP 为 0。

边缘中断模式下，CLICINTIP 为可读可写寄存器，且带有自动清除 IP 位的功能。当中断配置为硬件矢量模式时，CPU 响应中断的同时会自动清除该中断的 IP 位；对于非硬件矢量模式的中断，软件建议通过执行一条 CSR 访问指令同时产生对 MNXTI 寄存器有效的读写操作来获取中断等待状态，同时该操作可以清除在等待响应的对应的中断，CPU 去跳转处理中断。如果不是采用读写 MNXTI 寄存器的方式去获取非硬件矢量模式的中断而是该中断主动将中断信息传递给 CPU 流水线核心，那么 CPU 在响应该中断时硬件无法清除其中断

等待状态，需要通过软件清除。

5. 中断使能寄存器（CLICINTIE)

该寄存器最低位被置起表示对应的中断源被使能，在满足条件的情况下 CPU 可以响应该中断。

7	6	5	4	3	2	1	0
Res.							IE
-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0

IE-中断使能：

1：对应中断被使能；

0：对应中断未使能。

6. 中断属性寄存器（CLICINTATTR)

7	6	5	4	3	2	1	0
mode		Res.			trig		shv
RO	RO	-	-	-	RW	RW	RW
1	1	0	0	0	0	0	0

mode-中断特权态：

该域用于配置中断的特权态，由于 E906 仅支持机器模式下响应中断，所以该域被绑为 2'b11，代表机器模式中断。

trig-中断触发方式：

该域用于区分脉冲中断和电平中断，当 trig[0]为 0 时，代表为电平中断。当 trig[0]为 1 时，trig[1]为 0 代表上升沿中断，trig[1]为 1 代表下降沿中断。

shv-矢量中断使能：

代表该中断是否为硬件矢量中断。

7. 中断控制寄存器（CLICINTCTL)

该寄存器用于表示每一个中断源参与仲裁的优先级，同时配合 CLICCFG.nlbits 产生给 CPU 的中断优先级。给 CPU 的中断优先级固定为 8 位。

7	0
int_ctl	hardware tied to 1
RW	-
0	{(CLICINTCTLBITS’{1b’1}

int_ctl-参与仲裁优先级：

该位域有效位为 CLICINTCTLBITS 位。有效位域内的数值全部作为 CLIC 内部进行中断仲裁的优先级，但仅有高 CLICCFG.nlbits 位用于传递给 CPU 流水线核心表征仲裁出的中断的优先级（该优先级用于更新 MINTSTATUS 寄存器的 mil 域）。

CLICCFG.nlbits 与最终 CLIC 传递给 CPU 的中断优先级编码不完全示例如下表 2-16 所示。

当 nlbits > CLICINTCTLBITS 时，nlbits 被认为等于 CLICINTCTLBITS。

表 2-16 nlbits 与 CPU 保存中断优先级编码示例

nlbits	CLICINTCTL 编码	可配中断优先级
0	xxxxxxxx	255
1	lxxxxxxx	127, 255
2	llxxxxxx	63, 127, 191, 255
3	lllxxxxx	31, 63, 95, 127, 159, 191, 223, 255
4	llllxxxx	15, 31, 47, 63, 79, 95, 111, 127, 143, 159, 175, 191, 207, 223, 239, 255

CLICCFG.nlbits 与 CLICINTCTL 划分不完全示例如下表 2-17 所示。

表 2-17 nlbits 与 CLICINTCTL 划分示例

CLICINTCTLBITS	nlbits	编码	可配中断优先级
2	2	llxxxxxx	63, 127, 191, 255
2	1	lpxxxxxx	127, 255
2	0	ppxxxxxx	255
3	2	llpxxxxx	63, 127, 191, 255
4	2	llppxxxx	63, 127, 191, 255
5	1	lppppxxx	127, 255

3 嵌入式 Flash 控制器（EFC）

3.1 EFlash 控制器简介

嵌入式闪存控制器——Embedded Flash Controller，简称 EFLASH 控制器（EFC），支持 CPU 通过指令总线读取 Flash 内容以直接执行指令或读取数据；或者通过寄存器配置进行以下 Flash 存储器内容操作：编程（PROGRAM），页擦除(PAGE ERASE)，全片擦除(MASS ERASE)，读取(READ)，重载选项字节（RELOAD OPTION BYTE）。

3.2 EFlash 控制器主要特性

3.2.1 存储体

AiM51 系列芯片存储容量说明如下：

- Main Memory 有如下 3 种规格：
  - 128 KB
  - 256 KB
  - 512 KB
- Boot Memory（NVR）：12 KB
- Page Size：1 KB
- Row Size：512 B
- 数据读取宽度为 72 位（64 位 + 8 个 ECC 位）
- 数据写入宽度为 72 位（64 位 + 8 个 ECC 位）
- 单页或连续多页擦除和 Main Memory 全片擦除

3.2.2 EFlash 接口特性

- Flash 直接执行指令
- Flash 读操作
- Flash 编程/擦除操作
- 由选项字节（RDP）激活的读保护
- 由选项字节（WRPx）选择的两个写保护区域
- 由选项字节（PCROPx）选择的两个专有代码保护区域
- Flash 为空检查
- 预取缓冲器(PFBUF)
- 误码校正（ECC），支持 1bit 纠正，2bit 检出
- 选项字节重载功能

3.3 EFlash 控制器与系统

3.3.1 存储器与控制器地址

表 3-1 Flash 总线地址映射表

起始地址	末尾地址	容量	名称
0x08000000	0x0807FFFF	512KB	Main Memory
	0x0803FFFF	256KB	
	0x0801FFFF	128KB	
0x08080000	0x08082FFF	12KB	Boot Memory
0x08083000	0x080837FF	2KB	ROM
		1KB	Option Byte

3.3.2 自启动与 IP Trim

上电复位或系统复位后，CPU 复位被 Flash 控制器阻塞，FLASH 控制器逻辑自动完成下列动作：

步骤 1 等待 Flash IP 供电建立稳定（20us）；

步骤 2 获取 NVR_CFG 内的 Flash IP 配置 CBD0~7 与坏块信息 RR0/1 并锁存；

步骤 3 配置 CBD0~7 到 Flash IP 内部寄存器；

步骤 4 读取系统 IP trim 与器件 ID 并锁存、输出，后续 CPU 复位释放后，可查询各 TRIM 寄存器获取。

步骤 5 读取 Option Byte 并锁存，后续 CPU 复位释放后，可查询 EF_OPTB,EF_WDP0/1 获取。

须知

步骤 4 结束后，系统上的部分模拟 IP 都会获取 FLASH 控制器送出的初始 Trim 信息；对于用户而言，该流程是自动且不可介入的，流程结束后，CPU 即释放复位，调试器可接入。

3.3.3 Option Byte

芯片通过 Flash 保存一些芯片设置，包括：

- EF_SYS_OPT.boot：芯片 boot 位置选择，详见 3.3.4 BOOT 模式
- EF_SYS_OPT.boot_sel ：芯片 boot 位置选择位高位来源，详见 3.3.4 BOOT 模式
- EF_SYS_OPT.rdp ：读保护等级，详见 3.5.2 读访问保护等级
- EF_WRP0.wrp0_start_page ~ EF_WRP0.wrp0_end_page ：写保护区域 0 区间，详见 3.5.1 写入保护区域
- EF_WRP1.wrp1_start_page ~ EF_WRP1.wrp1_end_page ：写保护区域 1 区间，详见 3.5.1 写入保护区域
- EF_PCROP0.pcrop0_start_page ~ EF_PCROP0.pcrop0_end_page：专有代码读出保护区域 0 区间，详见 3.5.3PCROP 保护区
- EF_PCROP1.pcrop1_start_page ~ EF_PCROP0.pcrop1_end_page：专有代码读出保护区域 1 区间，详见 3.5.3PCROP 保护区

想要修改这些选项，必须使用 3.4.6 Flash 重载选项字节 所述的操作，芯片复位之前，选项字节的有效值不会改变。

3.3.4 BOOT 模式

芯片支持三种 boot 模式，分别是 Main Memory boot，Boot Memory boot 与 SRAM boot。系统上电时，通过 BOOT0 引脚的状态、Option Byte 中的 EF_SYS_OPT.boot 和 EF_SYS_OPT.boot_sel 的值共同决定系统的 Boot 模式，如下表 3-2 所示。

Option Byte 中的 EF_SYS_OPT.boot 和 EF_SYS_OPT.boot_sel 的值可通过软件修改，EF_SYS_OPT.boot 的值由寄存器 EF_SYS_OPT_SHD.boot 配置，EF_SYS_OPT.boot_sel 的值由寄存器 EF_SYS_OPT_SHD.boot_sel 配置。EF_SYS_OPT.boot 默认配置为 2'b10，

EF_SYS_OPT.boot_sel 默认配置为 1'b0。

表 3-2 Boot 方式映射表（其中 x 表示该位的状态不起作用）

RDP	BOOT0 引脚	EF_SYS_OPT.boot (二进制值)	EF_SYS_OPT.boot_sel (二进制值)	所选启动区域
0/1	0	xx	0	Main Memory
0/1	1	1x	0	Boot Memory
0/1	1	0x	0	SRAM
0/1	x	x0	1	Main Memory
0/1	x	11	1	Boot Memory
0/1	x	01	1	SRAM
2	x	xx	x	Main Memory

须知

如果要修改系统 boot 模式，在对 BOOT0 引脚或者 Option Byte 的内容做好相应的设置后需要复位系统，系统复位后修改才能生效。

当 RDP 保护等级为 2 时，芯片固定从 Main Memory boot，关于 RDP 保护等级的描述详见 3.5.2 读访问保护等级。

当 Main Memory 内容为空时，芯片以 Boot Memory boot 或者 SRAM boot（取决于 EF_SYS_OPT.boot 的配置）取代 Main Memory boot，关于 Flash 空检查的描述，详见 3.4.1 Flash 空检查 章节。

3.4 EFlash 控制器功能描述

3.4.1 Flash 空检查

FLASH 控制器在初始化阶段会自动校验 Main Memory 的第一个地址，当它的值为 0xFFFFFFFFFFFFFFFF 时，代表 FLASH 控制器认为 Flash 是空的；此时即使 boot 模式选择 Main Memory boot 模式，也会从 SRAM boot 或者 Boot Memory boot（取决于 EF_SYS_OPT. boot 的设置）。

另见 3.3.4BOOT 模式 。

3.4.2 Flash 计时基准

为了满足 Flash 器件擦除/编程等操作的时序关系与时长，Flash 控制器内部按照计时基准的倍数完成计时，这个基准为 125ns（8MHz）。EF_ACC_CTRL.time_base 的含义是：系统时钟频率(MHz)/8MHz-1（除法向上取整）。

3.4.3 Flash 读访问延迟

Flash 器件准备数据需要 30ns，FLASH 控制器通过插入读取 latency（延迟）满足时序要求。

FLASH 控制器通过时基设置自动对应 latency。

⚠注意

需要改变 FLASH 控制器工作频率时必须按照以下顺序：

- 提频时先改变 FLASH 控制器时基配置 EF_ACC_CTRL.time_base，再配置 CRG 切频。
- 降频时先改变 CRG 配置切频，再调整 FLASH 控制器时基配置 EF_ACC_CTRL.time_base。
- 切频的配置过程中禁止插入其他 flash 操作，否则可能产生不确定的后果。

3.4.4 Flash 擦除

步骤 1 查询 EF_STAT.busy 位，该位为 0 时，可以操作 Flash；

步骤 2 配置 EF_OPKEY.cfg_key 域为 0x6969；

步骤 3 如果写入正确，可查询到 EF_STAT.cfg_lock 位为 0，代表操作锁已解除，如果该位为 1，代表 cfg_key 写入错误，后续步骤无效；操作完全完成后，该位会自动置 1，重新锁定；

步骤 4 若进行页擦除，则配置 EF_OPADDR.nvr_sel 和 EF_OPADDR.page_sel；

步骤 5 若进行多页擦除，则配置 EF_OPADDR.mpe_end_page；当 mpe_end_page > page_sel 时，两页间（包含这两页）的所有页都会被擦除；当 mpe_end_page ≤ page_sel 时只有 page_sel 选择的页会被擦除；

步骤 6 配置 EF_OPSET.flash_cmd 域为 1 页/多页擦除操作指令，或者 2 全片擦除操作指令；

步骤 7 对 EF_OPTRG.sop 位写 1 开始擦除操作；

步骤 8 查询 EF_STAT.busy 位，擦除操作进行中，该位为 1，操作完成后该位为 0，表示可以进行后续操作；

步骤 9 擦除完成后， INT_RAW. int_eop_raw 置 1，可通过写 INT_CLR.int_eop_clr 清 0，如果激活中断 INT_EN.int_eop_en，FLASH 控制器会送出中断，并且状态位 INT_STAT.int_eop 会等同于 INT_RAW.int_eop_raw。

须知

Flash 擦除操作进行时，任何总线访问都将被挂起，直到擦除完成再完成正确响应。

危险

Flash 擦除操作进行时芯片掉电，Flash 及内部存储内容存在损坏风险。

3.4.5 Flash 编程

步骤 1 查询 EF_STAT.busy 位，该位为 0 时，可以操作 Flash；

步骤 2 配置 EF_OPKEY.cfg_key 域为 0x6969；

步骤 3 如果写入正确，可查询到 EF_STAT.cfg_lock 位为 0，代表操作锁已解除，如果该位为 1，代表 cfg_key 写入错误，后续步骤无效；操作完全完成后，该位会自动置 1，重新锁定；

步骤 4 配置 EF_OPADDR 编程目标地址；

步骤 5 配置 EF_PROGBUF0/1 编程数据；

步骤 6 配置 EF_OPSET.flash_cmd 域为 3 编程操作指令；

步骤 7 对 EF_OPTRG.sop 位写 1 开始编程；

步骤 8 查询 EF_STAT.busy 位，编程操作进行中，该位为 1，操作完成后该位为 0，表示可以进行后续操作；

步骤 9 编程完成后， INT_RAW. int_eop_raw 置 1，可通过写 INT_CLR.int_eop_clr 清 0，如果激活中断 INT_EN.int_eop_en，FLASH 控制器会送出中断，并且状态位 INT_STAT.int_eop 会等同于 INT_RAW.int_eop_raw。

须知

Flash 编程操作进行时，任何总线访问都将被挂起，直到擦除完成再完成正确响应。

危险

Flash 编程操作进行时芯片掉电，Flash 及内部存储内容存在损坏风险

3.4.6 Flash 重载选项字节

FLASH 控制器通过用户配置自动重载（擦除对应区域并编程）选项字节，指令 7 设置后暂不生效，直到完成系统复位后才会生效，而指令 8 会在修改完毕后立刻生效。

步骤 1 查询 EF_STAT.busy 位，该位为 0 时，可以操作 Flash；

步骤 2 配置 EF_OPKEY.optb_key 域为 0xC3C3；

步骤 3 如果写入正确，可查询到 EF_STAT.optb_lock 位为 0，代表操作锁已解除，如果该位为 1，代表 optb_key 写入错误，后续 Option Byte 配置步骤无效；

步骤 4 根据需要配置 EF_WRP0/1_SHD, EF_SYS_OPT_SHD，EF_PCROP0/1_SHD；

步骤 5 配置 EF_OPKEY.cfg_key 域为 0x6969；

步骤 6 如果写入正确，可查询到 EF_STAT.cfg_lock 位为 0，代表操作锁已解除，如果该位为 1，代表 cfg_key 写入错误，后续步骤无效；操作完全完成后，该位会自动置 1，重新锁定；

步骤 7 配置 EF_OPSET.flash_cmd 域为 7 或者 8 重载选项字节操作指令；

步骤 8 对 EF_OPTRG.sop 位写 1 开始重载选项字节动作；

步骤 9 查询 EF_STAT.busy 位，编程操作进行中，该位为 1，操作完成后该位为 0，表示可以进行后续操作；

步骤 10 重载完成后， INT_RAW. int_eop_raw 置 1，可通过写 INT_CLR.int_eop_clr 清 0，如果激活中断 INT_EN.int_eop_en，FLASH 控制器会送出中断，并且状态位 INT_STAT.int_eop 会等同于 INT_RAW.int_eop_raw。

须知

Flash 重载选项字操作进行时，任何总线访问都将被挂起，直到擦除完成再完成正确响应。

危险

Flash 重载选项字操作进行时芯片掉电，Flash 及内部存储内容存在损坏风险。

3.4.7 Flash 读取

FLASH 控制器通过寄存器接口访问 flash 内容，使用与执行程序不同的逻辑通路，适用于查询某些特殊的标志位或者监视 flash 内存储的程序。

步骤 1 查询 EF_STAT.busy 位，该位为 0 时，可以操作 Flash；

步骤 2 配置 EF_OPKEY.cfg_key 域为 0x6969；

步骤 3 如果写入正确，可查询到 EF_STAT.cfg_lock 位为 0，代表操作锁已解除，如果该位为 1，代表 cfg_key 写入错误，后续步骤无效；操作完全完成后，该位会自动置 1，重新锁定；

步骤 4 配置 EF_OPSET.flash_cmd 域为 4 读取操作指令；

步骤 5 配置 EF_OPADDR 设置地址；

步骤 6 对 EF_OPTRG.sop 位写 1 开始读取；

步骤 7 查询 EF_STAT.busy 位，读取操作进行中，该位为 1，操作完成后该位为 0，表示可以进行后续操作；

步骤 8 读取完成后， INT_RAW. int_eop_raw 置 1，可通过写 INT_CLR.int_eop_clr 清 0，如果激活中断 INT_EN.int_eop_en，FLASH 控制器会送出中断，并且状态位 INT_STAT.int_eop 会等同于 INT_RAW.int_eop_raw；

步骤 9 读取一次 EF_STAT.rdata_vld 位，EF_RDBUF0/1 的数据有效时，该位为 1；

步骤 10 读取 EF_ RDBUF0/1 获取数据。

须知

Flash 调试读取操作进行时，任何总线访问都将被挂起，直到读取完成再完成响应。

3.5 Flash 内容访问保护

3.5.1 写入保护区域

写入保护区域（WRP）的用途是规划只读区域，防止误操作，在读保护（RDP）等级为 2 的时候，由于 Option Byte 不可被改写，所以此时 WRP 区域等效于 ROM。

Write Protection 是 Option Byte 的一部分，由 WRP 划定的区域不可以被编程或者擦除，如果发生冲突，发起的编程/擦除操作将被忽略并且产生写保护错误中断 INT_RAW.int_wrp_err_raw，可通过写 INT_CLR.int_wrp_err_clr 可清除写保护错误中断，如果激活中断 INT_EN.int_wrp_err_en，FLASH 控制器会把该中断上报 CPU，并且状态位 INT_STAT.int_wrp_err 会置 1。

FLASH 控制器中有两个 Main Memory WRP 区域。用 EF_WRPx.wrpx_start_page ~ EF_WRPx.wrpx_end_page（x 取值 0~1） 定义两个 Main Memory 的 WRP 区域。

此外，当 end page 地址在 start page 之前，相当于禁用写保护。而任意写保护区域存在时，全片擦除操作会被忽略，并且产生写保护错误中断 INT_RAW.int_wrp_err_raw。当 WRP 区域重叠时，取并集。

3.5.2 读访问保护等级

RDP（Read Protection）是 Option Byte 的一部分，由 EF_SYS_OPT_SHD.rdp 配置值译码的保护等级对 Main Memory 存储区生效，如下表 3-3 所示。RDP 用于保护用户数据、核心代码不被第三方改写或读出。

表 3-3 RDP 保护等级映射表

EF_SYS_OPT_SHD.rdp	rdp_level
0xAA	0
其他	1
0xCC	2

RDP 有以下三种保护等级：

0：无保护

可使用任何方式对 Main Memory 区域执行读取、编程和擦除操作。也可更改选项字节内容。（编程和擦除仍受到 WRP 以及 PCROP 限制）。

0 级可以被任意程序更改为任何其他等级

1：读保护，调试保护。

在 Main Memory / Boot Memory boot：可对主 Flash、选项字节执行所有操作（编程和擦除仍受到 WRP 以及 PCROP 限制）。

调试/ SRAM boot 时，Main Memory 完全不可访问。在 RDP 等级=1 时，对 flash 进行读访问或写访问会被忽略，并且产生调试口读保护错误中断 INT_RAW.int_rdp_err_raw，与 WRP 类似，开启中断使能后会产生中断上报。

⚠注意

1 级可以被非第三方程序更改为任何其他等级，但是当从 1 级降为 0 级时，会引起 main flash 全片擦除。这也代表原本位于 Flash 内的程序将无法正常执行。

2：禁止调试，强制启动区间

系统此时禁止任何调试；

系统将强制从 Main Memory boot

其余保护能力与等级 1 相当

⚠警告

2 级为单向进入，无法通过任何方式改变为其他值。

须知

保护等级通过重载选项字节操作更改，AiM51 系列芯片出厂默认保护等级为 0，并且 Main Memory 区域全空。

3.5.3 PCROP 保护区

PCROP 保护区是指专有代码读出保护（Proprietary Code ReadOut Protection）区域，该区域的内容只可被 CPU 执行（Excute）,不可读（Read）写（Write），可以防止第三方读取或者改写。

AiM51 系列芯片可配置两个专有代码保护区 PCROP0 和 PCROP1。作为 Option Byte 的一部分，类似 WRP0/1 区域的配置，3.4.6 章节中的操作流程适用于配置两个 PCROP 区域，EF_PCROPx.pcropx_start_page ~ EF_PCROPx.pcropx_end_page（x 取值 0~1）之间的区域受到 PCROP 保护；当 end page 地址在 start page 之前，相当于禁用保护。保护存在时，尝试读取该区域内容，会产生调试口读保护错误中断 INT_RAW.int_rdp_err_raw；尝试编程或擦除该区域，会产生写保护错误中断 INT_RAW.int_wrp_err_raw。

须知

强烈建议用户将敏感代码所处区段设为 PCROP 保护区域。

⚠注意

需要注意的是，如果想将一个被 PCROP 保护的 page，改为非 PCROP 保护的，那么这次对 PCROP 的修改（更改选项字节操作）必须伴随 RDP 等级从 1 改为 0。

由于中断向量表对 CPU 而言是数据访问而非取指访问，如果将其所在的区域设置成 PCROP 保护区域，会导致任何程序都无法正常执行。因此，严禁将中断向量表所在区域设为 PCROP 保护区。

一旦不慎将中断向量表或者作为数据使用的空间设为 PCROP，则需要在满足上述约束的情况下取消 PCROP，并重新烧写被降级擦除的数据。

3.6 EFlash 控制器寄存器概览

EFlash 控制器寄存器基地如下：

- EFlash 控制器寄存器基地为：0x61001000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	EF_ACC_CTRL	0x0	32	RW_POR	0x00000002	时基配置寄存器
2	EF_OPKEY	0x4	32	RW	0x00000000	秘钥配置寄存器
3	EF_OPTRG	0x8	32	RW	0x00000000	操作使能控制寄存器
4	EF_OPSET	0xC	32	RW	0x00000000	操作模式配置寄存器
5	EF_STAT	0x10	32	RO	0x00000310	状态寄存器
6	EF_OPADDR	0x14	32	RW	0x00000000	操作地址配置寄存器
7	EF_RDBUF0	0x18	32	RO	0x00000000	数据读取寄存器 0
8	EF_RDBUF1	0x1C	32	RO	0x00000000	数据读取寄存器 1
9	EF_PROGBUF0	0x20	32	RW	0x00000000	编程数据寄存器 0
10	EF_PROGBUF1	0x24	32	RW	0x00000000	编程数据寄存器 1
11	EF_WRP0	0x28	32	RO	0x00000000	写保护区域 0 地址读取寄存器
12	EF_WRP0_SHD	0x2C	32	RW	0x00000000	写保护区域 0 地址配置寄存器
13	EF_WRP1	0x30	32	RO	0x00000000	写保护区域 1 地址读取寄存器
14	EF_WRP1_SHD	0x34	32	RW	0x00000000	写保护区域 1 地址配置寄存器
15	EF_SYS_OPT	0x38	32	RO	0x000000AA	系统选项字状态读取寄存器
16	EF_SYS_OPT_SHD	0x3C	32	RW	0x000000AA	系统选项字配置寄存器
17	EF_PCROP0	0x40	32	RO	0x00000001	专有代码保护区 0 地址读取寄存器
18	EF_PCROP0_SHD	0x44	32	RW	0x00000001	专有代码保护区 0 地址配置寄存器
19	EF_PCROP1	0x48	32	RO	0x00000001	专有代码保护区 1 地址读取寄存器
20	EF_PCROP1_SHD	0x4C	32	RW	0x00000001	专有代码保护区 1 地址配置寄存器
21	EF_ECC	0x50	32	RO	0x00000000	ECC 状态寄存器
22	INT_EN	0x54	32	RW	0x00000000	中断屏蔽寄存器
23	INT_RAW	0x58	32	RO	0x00000000	原始中断状态寄存器
24	INT_STAT	0x5C	32	RO	0x00000000	中断状态寄存器
25	INT_CLR	0x60	32	RO	0x00000000	中断状态清除寄存器
26	INT_INJ	0x64	32	RO	0x00000000	中断注入寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
27	UID0	0xC0	32	RO	0x00000000	芯片唯一标识码寄存器 0
28	UID1	0xC4	32	RO	0x00000000	芯片唯一标识码寄存器 1
29	UID2	0xC8	32	RO	0x00000000	芯片唯一标识码寄存器 2
30	UID3	0xCC	32	RO	0x000E000E	芯片唯一标识码寄存器 3

3.7 EFlash 控制器寄存器描述

3.7.1 EF_ACC_CTRL

◆ **偏移地址：0x0　　复位值：0x00000002**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.											time_base				
-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
time_base	[4:0]	RW_POR	计时时基(timing base)。 运算公式为：（工作时钟频率/8-1），其中工作时钟频率来自 CRG 配置。	0x2

3.7.2 EF_OPKEY

◆ **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
optb_key															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cfg_key															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
optb_key	[31:16]	RW	选项字节密钥(Option Byte Key)。 写入 0xC3C3 以解锁以下寄存器的写操作： EF_SYS_OPT_SHD EF_WRP0_SHD EF_WRP1_SHD 写入其他值：lock 位不会从 1 置位为 0。	0x0
cfg_key	[15:0]	RW	配置密钥(configuration key)。 写入 0x6969 以解锁以下寄存器的写操作： EF_OPTRG EF_OPSET EF_OPADDR EF_PROGBUF0 EF_PROGBUF1 写入其他值：lock 位不会从 1 置位为 0。	0x0

3.7.3 EF_OPTRG

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															sop
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
sop	[0]	W1P	操作开始触发位（Start Of Process）： 1：开始 EF_OPSET 寄存器所配置的操作； 0：无效。	0x0

3.7.4 EF_OPSET

◆ **偏移地址：0xC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												flash_cmd			
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
flash_cmd	[3:0]	RW	闪存命令(Flash Command)： 8：无需复位立即生效的选项字更改； 7：重载选项字； 4：读操作； 3：编程操作； 2：全片擦除操作； 1：页擦除操作； 0 以及其他值：无操作。	0x0

3.7.5 EF_STAT

◆ **偏移地址：0x10　　复位值：0x00000310**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

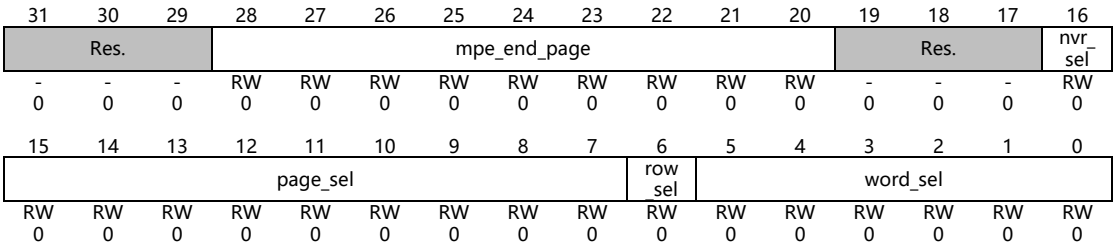
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		boot_mode		Res.		optb_lock	cfg_lock	Res.			flash_empty	Res.		rdata_vld	busy
-	-	RO	RO	-	-	RO	RO	-	-	-	RO	-	-	RO	RO
0	0	0	0	0	0	1	1	0	0	0	1	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
boot_mode	[13:12]	RO	boot 模式选择： 0/2：Main Memory boot； 1：SRAM boot； 3：Boot Memory boot。	0x2
reserved	[11:10]	-	保留。	0x0
optb_lock	[9]	RO	选项字节锁定(Option Byte Locked)： 0：key 写入正确，后续配置可生效	0x1

Name	Bits	R/W	Description	Default
			1：key 错误，后续配置将不生效。	
cfg_lock	[8]	RO	配置锁定(Configuration Locked)： 0：key 写入正确，后续配置可生效 1：key 错误，后续配置将不生效。	0x1
reserved	[7:5]	-	保留。	0x0
flash_empty	[4]	RO	空片状态(Flash is empty)： 0：flash 进行过生产阶段的编程 1：flash 未进行过生产阶段编程。	0x1
reserved	[3:2]	-	保留。	0x0
rdata_vld	[1]	RO	读数据有效性(read data valid)： 1：EF_RDBUF0\EF_RDBUF1 对应当前地址 0：EF_RDBUF0\EF_RDBUF1 内容无效。	0x0
busy	[0]	RO	繁忙状态(Busy)： 0：flash 空闲，可以直接读访问或者发起操作 1：flash 繁忙，此时的读访问将推迟响应，编程/擦除等动作会被忽略。不允许系统复位或者下电。	0x0

3.7.6 EF_OPADDR

◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
mpe_end_page	[28:20]	RW	擦除专用额外页数选择。	0x0
reserved	[19:17]	-	保留。	0x0
nvr_sel	[16]	RW	额外闪存区域片选(NVR Selection)： 0：地址指向 Main Array； 1：地址指向 NVR 区域。 由于 NVR 区域与 Main array 共享 offset，所以靠这一位选通 NVR 区域。	0x0
page_sel	[15:7]	RW	页选择(Page Selection)： 0：Page 1； 1：Page 2；	0x0

Name	Bits	R/W	Description	Default
			... 511：Page 512。 选择操作 0~511 Page，每页大小为 1KB。不同型号芯片的 flash 大小不一样，具体可查看 UID3.flash_mem_size。	
row_sel	[6]	RW	行选择(Row Selection)： 选择操作每 Page 的某一行 0：第一行； 1：第二行。	0x0
word_sel	[5:0]	RW	字选择(Word selection) 选择操作每一行的某个双字， 对应 eflash YADR 0：每行第一个双字； 1：每行第二个双字； ... 63：每行最后一个双字。	0x0

3.7.7 EF_RDBUF0

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
read_data_l32															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
read_data_l32															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
read_data_l32	[31:0]	RO	读取数据的低 32 位(read data low 32 bit)。	0x0

3.7.8 EF_RDBUF1

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
read_data_h32															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
read_data_h32															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
read_data_h32	[31:0]	RO	读取数据的高 32 位（read data high 32 bit)。	0x0

3.7.9 EF_PROGBUF0

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
prog_data_l32															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
prog_data_l32															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
prog_data_l32	[31:0]	RW	编程数据的低 32 比特(program data low 32bit)。	0x0

3.7.10 EF_PROGBUF1

◆ **偏移地址：0x24　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
prog_data_h32															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
prog_data_h32															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
prog_data_h32	[31:0]	RW	编程数据的高 32 比特(program data high 32bit)。	0x0

3.7.11 EF_WRP0

◆ **偏移地址：0x28　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							wrp0_end_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							wrp0_start_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
wrp0_end_page	[24:16]	RO	写保护区域 0 末尾页(write protected area0 end page)。	0x0
reserved	[15:9]	-	保留。	0x0
wrp0_start_page	[8:0]	RO	写保护区域 0 起始页(write protected area0 start page)。	0x0

3.7.12 EF_WRP0_SHD

◆ **偏移地址：0x2C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							wrp0_end_page_shd								
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							wrp0_start_page_shd								
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
wrp0_end_page_shd	[24:16]	RW	写保护区域 0 末尾页设置(write protected area0 end page shadow)。 写后进行重载选项字节（EF_OPSET=0x7）操作使之生效。 当末尾页在起始页之前时，禁用写保护功能。	0x0
reserved	[15:9]	-	保留。	0x0
wrp0_start_page_shd	[8:0]	RW	写保护区域 0 起始页设置(write protected area0 start page shaow)。 写入后进行重载选项字节（EF_OPSET=0x7）操作使之生效。	0x0

3.7.13 EF_WRP1

◆ **偏移地址：0x30　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							wrp1_end_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							wrp1_start_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
wrp1_end_page	[24:16]	RO	写保护区域 1 末尾页(write protected area1 end page)。	0x0
reserved	[15:9]	-	保留。	0x0
wrp1_start_page	[8:0]	RO	写保护区域 1 起始页(write protected area1 start page)。	0x0

3.7.14 EF_WRP1_SHD

◆ **偏移地址：0x34　　复位值：0x00000000**

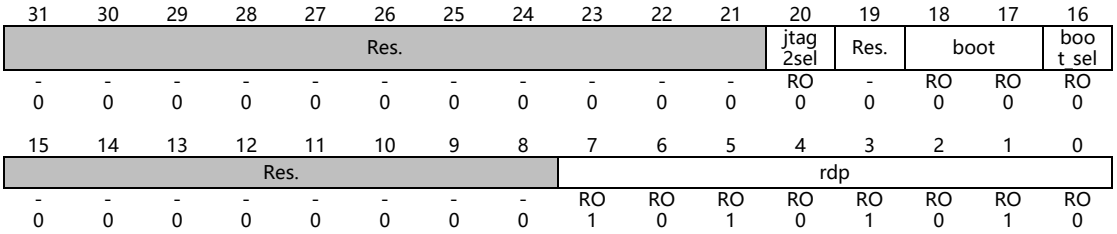
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							wrp1_end_page_shd								
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							wrp1_start_page_shd								
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
wrp1_end_page_shd	[24:16]	RW	写保护区域 1 末尾页设置(write protected area1 end page shadow)。 写入后进行重载选项字节（EF_OPSET=0x7）操作使之生效。 当末尾页在起始页之前时，禁用写保护功能。	0x0
reserved	[15:9]	-	保留。	0x0
wrp1_start_page_shd	[8:0]	RW	写保护区域 1 起始页设置(write protected area1 start page shadow)。 写入后进行重载选项字节（EF_OPSET=0x7）操作使之生效。	0x0

3.7.15 EF_SYS_OPT

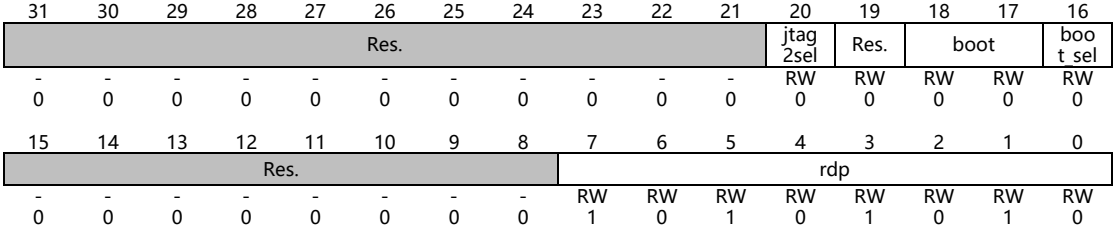
◆ **偏移地址：0x38　　复位值：0x000000AA**



Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
jtag2sel	[20]	RO	JTAG 模式选择： 0：5 线模式； 1：2 线模式。	0x0
reserved	[19]	-	保留。	0x0
boot	[18:17]	RO	自举位置选择： 0/2：Main Memory； 1：SRAM； 3：Boot Memory。 注意：启动位置由 BOOT0、EF_SYS_OPT.boot 和 EF_SYS_OPT.boot_sel 的值共同决定。	0x2
boot_sel	[16]	RO	自举位置来源选择： 0：使用引脚； 1：使用上述 EF_SYS_OPT.boot 定义的位置。	0x0
reserved	[15:8]	-	保留。	0x0
rdp	[7:0]	RO	保护等级设置： 0xAA：保护等级为 0； 0xCC：保护等级为 2； 其他值：保护等级为 1。	0xAA

3.7.16 EF_SYS_OPT_SHD

◆ **偏移地址：0x3C　　复位值：0x000000AA**



Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
jtag2sel	[20]	RW	JTAG 模式选择： 0：5 线模式； 1：2 线模式。	0x0
reserved	[19]	-	保留。	0x0
boot	[18:17]	RW	自举位置选择： 0/2：Main Memory； 1：SRAM； 3：Boot Memory。 注意：启动位置由 BOOT0、EF_SYS_OPT.boot 和 EF_SYS_OPT.boot_sel 的值共同决定。	0x2
boot_sel	[16]	RW	自举位置来源选择： 0：使用引脚； 1：使用上述 EF_SYS_OPT_SHD.boot 定义的位置。	0x0
reserved	[15:8]	-	保留。	0x0
rdp	[7:0]	RW	保护等级设置： 0xAA：保护等级为 0； 0xCC：保护等级为 2； 其他值：保护等级为 1。	0xAA

3.7.17 EF_PCROP0

◆ **偏移地址：0x40　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							pcrop0_end_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							pcrop0_start_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
pcrop0_end_page	[24:16]	RO	专有代码保护区 0 已生效结束地址。	0x0
reserved	[15:9]	-	保留。	0x0
pcrop0_start_page	[8:0]	RO	专有代码保护区 0 已生效起始地址。	0x1

3.7.18 EF_PCROP0_SHD

◆ **偏移地址：0x44　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							pcrop0_end_page_shd								
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							pcrop0_start_page_shd								
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
pcrop0_end_page_shd	[24:16]	RW	配置专有代码保护区 0 结束地址。	0x0
reserved	[15:9]	-	保留。	0x0
pcrop0_start_page_shd	[8:0]	RW	配置专有代码保护区 0 起始地址。	0x1

3.7.19 EF_PCROP1

◆ **偏移地址：0x48　　复位值：0x00000001**

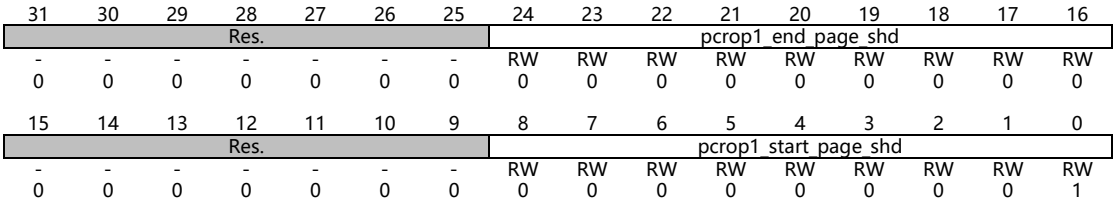
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							pcrop1_end_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							pcrop1_start_page								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
pcrop1_end_page	[24:16]	RO	专有代码保护区 1 已生效结束地址。	0x0
reserved	[15:9]	-	保留。	0x0
pcrop1_start_page	[8:0]	RO	专有代码保护区 1 已生效起始地址。	0x1

3.7.20 EF_PCROP1_SHD

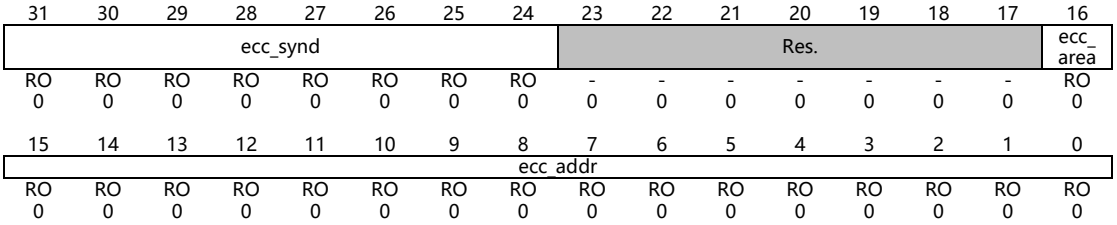
◆ **偏移地址：0x4C　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
pcrop1_end_page_shd	[24:16]	RW	配置专有代码保护区 1 结束地址。	0x0
reserved	[15:9]	-	保留。	0x0
pcrop1_start_page_shd	[8:0]	RW	配置专有代码保护区 1 起始地址。	0x1

3.7.21 EF_ECC

◆ **偏移地址：0x50　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
ecc_synd	[31:24]	RO	ECC 错误特征码，用于识别出错位置。	0x0
reserved	[23:17]	-	保留。	0x0
ecc_area	[16]	RO	指明发生 ECC 错误的区域： 0：Main array 发生 ECC 错误； 1：NVR 发生 ECC 错误。	0x0
ecc_addr	[15:0]	RO	ECC 错误发生的具体地址。	0x0

3.7.22 INT_EN

◆ **偏移地址：0x54　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							int_oor_err_en	int_optv_err_en	int_rdp_err_en	int_wrp_err_en	int_progsize_err_en	int_progseq_err_en	int_eccd_en	int_eccc_en	int_eop_en
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
int_oor_err_en	[8]	RW	操作地址越界使能 (operation break error enable)： 1：开启； 0：屏蔽	0x0
int_optv_err_en	[7]	RW	选项字校验错误使能 (Option Byte varifying error enable)： 1：开启； 0：屏蔽。	0x0
int_rdp_err_en	[6]	RW	调试口读保护错误使能 (read protect error enable)： 1：开启； 0：屏蔽。	0x0
int_wrp_err_en	[5]	RW	写保护错误使能 (write protect enable)： 1：开启； 0：屏蔽。	0x0
int_progsize_err_en	[4]	RW	编程尺寸错误使能 (program size error enable)： 1：开启； 0：屏蔽。	0x0
int_progseq_err_en	[3]	RW	编程未擦除区域错误使能 (program sequence error enable)： 1：开启； 0：屏蔽。	0x0
int_eccd_en	[2]	RW	ECC nbit 检错中断使能 (ecc error detected enable)： 1：开启； 0：屏蔽。	0x0
int_eccc_en	[1]	RW	ECC 完成 1bit 纠错使能 (ecc error correcting)： 1：开启； 0：屏蔽。	0x0
int_eop_en	[0]	RW	操作完成使能 (end of process enable)： 1：开启；	0x0

Name	Bits	R/W	Description	Default
			0：屏蔽。	

3.7.23 INT_RAW

◆ **偏移地址：0x58　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							int_oor_err_raw	int_optv_err_raw	int_rdp_err_raw	int_wrp_err_raw	int_progsize_err_raw	int_progseq_err_raw	int_eccd_raw	int_eccc_raw	int_eop_raw
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留	0x0
int_oor_err_raw	[8]	RO	操作越界原始中断（erase error raw interrupt）： 1：有效； 0：未发生。	0x0
int_optv_err_raw	[7]	RO	选项字校验错误原始中断（Option Byte varifying error raw interrupt）： 1：有效； 0：未发生。	0x0
int_rdp_err_raw	[6]	RO	调试口读保护错误原始中断（read protect error raw interrupt）： 1：有效； 0：未发生。	0x0
int_wrp_err_raw	[5]	RO	写保护错误原始中断（write protect raw interrupt）： 1：有效； 0：未发生。	0x0
int_progsize_err_raw	[4]	RO	编程尺寸错误原始中断（program size error raw interrupt）： 1：有效； 0：未发生。	0x0
int_progseq_err_raw	[3]	RO	编程未擦除区域错误原始中断（program sequence error raw interrupt）： 1：有效； 0：未发生。	0x0

Name	Bits	R/W	Description	Default
int_eccd_raw	[2]	RO	ECC nbit 检错原始中断 (ecc error detected raw interrupt)： 1：有效； 0：未发生。	0x0
int_eccc_raw	[1]	RO	ECC 完成 1bit 纠错原始中断 (ecc error correcting raw interrupt)： 1：有效； 0：未发生。	0x0
int_eop_raw	[0]	RO	操作完成原始中断 (ecc error detected raw interrupt)： 1：有效； 0：未发生。	0x0

3.7.24 INT_STAT

◆ **偏移地址：0x5C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							int_oor_err	int_optv_err	int_rdp_err	int_wrp_err	int_progsize_err	int_progseq_err	int_eccd	int_eccc	int_eop
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
int_oor_err	[8]	RO	操作越界中断状态 (erase error interrupt status)： 1：有效； 0：未发生。	0x0
int_optv_err	[7]	RO	选项字校验错误中断状态 (Option Byte varifying error interrupt status)： 1：有效； 0：未发生。	0x0
int_rdp_err	[6]	RO	调试口读保护错误中断状态 (read protect error interrupt status)： 1：有效； 0：未发生。	0x0
int_wrp_err	[5]	RO	写保护错误中断状态 (write protect interrupt status)： 1：有效；	0x0

Name	Bits	R/W	Description	Default
			0：未发生。	
int_progsize_err	[4]	RO	编程尺寸错误中断状态 (program size error interrupt status)： 1：有效； 0：未发生。	0x0
int_progseq_err	[3]	RO	编程未擦除区域错误中断状态 (program sequence error interrupt status)： 1：有效； 0：未发生。	0x0
int_eccd	[2]	RO	ECC nbit 检错中断状态 (ecc error detected interrupt status)： 1：有效； 0：未发生。	0x0
int_eccc	[1]	RO	ECC 完成 1bit 纠错中断状态 (ecc error correcting interrupt status)： 1：有效； 0：未发生。	0x0
int_eop	[0]	RO	操作完成中断状态 (ecc error detected interrupt status)： 1：有效； 0：未发生。	0x0

3.7.25 INT_CLR

◆ **偏移地址：0x60　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							int_oor_err_clr	int_optv_err_clr	int_rdp_err_clr	int_wrp_err_clr	int_progsize_err_clr	int_progseq_err_clr	int_eccd_clr	int_eccc_clr	int_eop_clr
-	-	-	-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
int_oor_err_clr	[8]	W1P	操作地址越界中断清除 (erase error clear)： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0

Name	Bits	R/W	Description	Default
int_optv_err_clr	[7]	W1P	选项字校验错误中断清除（Option Byte varifying error clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_rdp_err_clr	[6]	W1P	调试口读保护错误中断清除（read protect error clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_wrp_err_clr	[5]	W1P	写保护错误中断清除（write protect clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_progsize_err_clr	[4]	W1P	编程尺寸错误中断清除（program size error clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_progseq_err_clr	[3]	W1P	编程未擦除区域错误中断清除（program sequence error clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_eccd_clr	[2]	W1P	ECC nbit 检错中断清除（ecc error detected clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_eccc_clr	[1]	W1P	ECC完成写1bit纠错中断清除（ecc error correcting clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_eop_clr	[0]	W1P	操作完成中断清除（ecc error detected clear）： 写 1：清除 INT_RAW 寄存器对应位； 写 0：无效。	0x0

3.7.26 INT_INJ

- **偏移地址：0x64　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

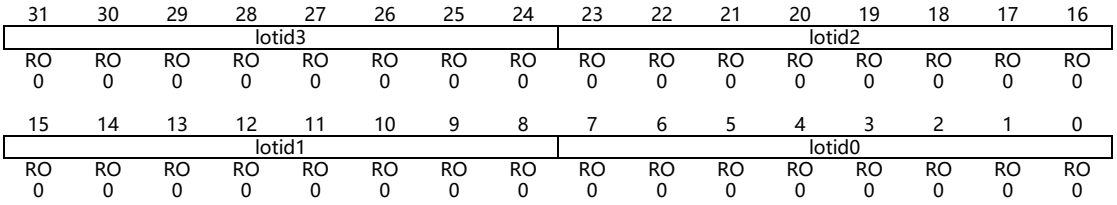
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							int_oor_err_inj	int_optv_err_inj	int_rdp_err_inj	int_wrp_err_inj	int_progsize_err_inj	int_progseq_err_inj	int_eccd_inj	int_eccc_inj	int_eop_inj
-	-	-	-	-	-	-	WP	WP	WP	WP	WP	WP	WP	WP	WP
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
int_oor_err_inj	[8]	W1P	操作越界中断注入(operation break error inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。；	0x0
int_optv_err_inj	[7]	W1P	选项字校验错误中断注入(Option Byte varifying error inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。；	0x0
int_rdp_err_inj	[6]	W1P	调试口读保护错误中断注入(read protect error inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。；	0x0
int_wrp_err_inj	[5]	W1P	写保护错误中断注入(write protect inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。；	0x0
int_progsize_err_inj	[4]	W1P	编程尺寸错误中断注入(program size error inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_progseq_err_inj	[3]	W1P	编程未擦除区域错误中断注入(program sequence error inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。	0x0
int_eccd_inj	[2]	W1P	ECC nbit 检错中断注入 (ecc error detected inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。；	0x0
int_eccc_inj	[1]	W1P	ECC 完成写 1bit 纠错中断注入(ecc error correcting inject)： 写 1：注入 INT_RAW 寄存器对应位； 写 0：无效。；	0x0

Name	Bits	R/W	Description	Default
int_eop_inj	[0]	W1P	操作完成中断注入(ecc error detected inject)： 写 1：注入 INT_RAW 寄存器对应位 写 0：无效。	0x0

3.7.27 UID0

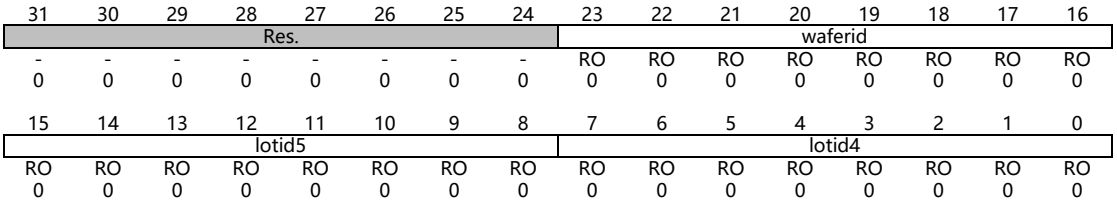
◆ **偏移地址：0xC0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
lotid3	[31:24]	RO	该芯片的批号第 3 字段。	0x0
lotid2	[23:16]	RO	该芯片的批号第 2 字段。	0x0
lotid1	[15:8]	RO	该芯片的批号第 1 字段。	0x0
lotid0	[7:0]	RO	该芯片的批号第 0 字段。	0x0

3.7.28 UID1

◆ **偏移地址：0xC4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	器件签名。	0x0
waferid	[23:16]	RO	该芯片所在的 wafer 识别码。	0x0
lotid5	[15:8]	RO	该芯片的批号第 5 字段。	0x0
lotid4	[7:0]	RO	该芯片的批号第 4 字段。	0x0

3.7.29 UID2

◆ **偏移地址：0xC8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.								package_type							
-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
diey_ct								diex_ct							
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
package_type	[23:16]	RO	芯片封装类型代码（请参考文章开头的章节查找映射）： 0：C 类； 2：R 类； 3：V 类。	0x0
diey_ct	[15:8]	RO	该芯片的 wafer 坐标位置，Y 轴。	0x0
diex_ct	[7:0]	RO	该芯片的 wafer 坐标位置，X 轴。	0x0

3.7.30 UID3

◆ **偏移地址：0xCC　　复位值：0x000E000E**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sram_mem_size															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
flash_mem_size															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0

Name	Bits	R/W	Description	Default
sram_mem_size	[31:16]	RO	SRAM 存储器容量： 11：64Kbyte； 12：128Kbyte； 14：192Kbyte； 其他：无效。	0xE
flash_mem_size	[15:0]	RO	eflash 容量： 11：128Kbyte； 12：256Kbyte； 14：512Kbyte； 其他：无效。	0xE

4 时钟和复位控制（CRG）

4.1 CRG 简介

CRG（clock & reset generater）时钟复位生成器为芯片各模块提供时钟和复位控制。选取时钟源进行 PLL 配置，并将相应时钟分发到系统总线和各个外设，也对时钟进行门控管理用于低功耗控制。

4.2 复位

芯片共有 5 种类型的复位源：电源复位、管脚复位、电压监控复位、看门狗复位、软件复位。

4.2.1 复位源描述

- 电源复位

电源复位由以下事件触发：

  - power-on reset (POR) ；
  - 从 shutdown 模式退出 ；
  - 从 deepsleep 模式退出 。

- 管脚复位

当芯片处于 WORK 模式下，复位管脚 RESETN 被外部驱动为低电平且持续 100us 以上时，触发管脚复位。

- 电压监控复位

在 eflash 中使能掉电保护功能，当 PVD 模块检测到 DVDD 供电电压低于 PVD 阈值时，触发电压监控复位。

- 看门狗复位

当芯片看门狗模块（WWDG/IWDG）未被正常刷新时，触发看门狗复位。

- 软件复位

当对系统控制器（SYSCTRL）的 SYS_RST_REQ.soft_rst_req 位域进行写操作时，触发软件复位。

4.2.2 复位控制域描述

复位触发后可对芯片不同属性或工作域的寄存器进行复位，如表 4-1 所示。其中，Y 表示能复位；N 表示不能复位。

表 4-1 复位控制域

电源域	寄存器	电源复位				管脚复位	PVD 监测异常复位	看门狗复位	SYS 软复位	BACKUP 软复位
		BPOR	AON POR	退出 SHUT DOWN 模式	退出 DEEP SLEEP 模式					
备份域 (BACKUP)	软复位请求寄存器	**Y**	N	N	N	N	N	N	N	N
	其他寄存器	**Y**	N	N	N	N	N	N	N	Y
常电域 (AON)	所有寄存器	N	**Y**	**Y**	N	N	N	N	N	N
掉电域 (CORE)	CRG、SYSCTRL RW_POR 属性寄存器、EFC RW_POR 属性寄存器	N	**Y**	**Y**	**Y**	N	N	N	N	N
	SYSCTRL RW_HRST 属性寄存器	N	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	N	N
	其他寄存器	N	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	**Y**	N

说明

电源域划分详见 PMC 11.3.1 电源章节。

- 电源复位

POR 或从 shutdown 模式退出时，会将芯片内部除备份域外的所有寄存器复位； 从

deepsleep 模式退出时，CORE 电源域所有寄存器被复位。

- 管脚复位、PVD 监测异常复位、看门狗复位

  芯片内部 CORE 电源域寄存器被复位，但不包含：

  - CRG 所有配置寄存器；
  - 系统控制器（SYSCTRL 和 EFC）中 RW_POR 属性的寄存器。

- SYS 软复位

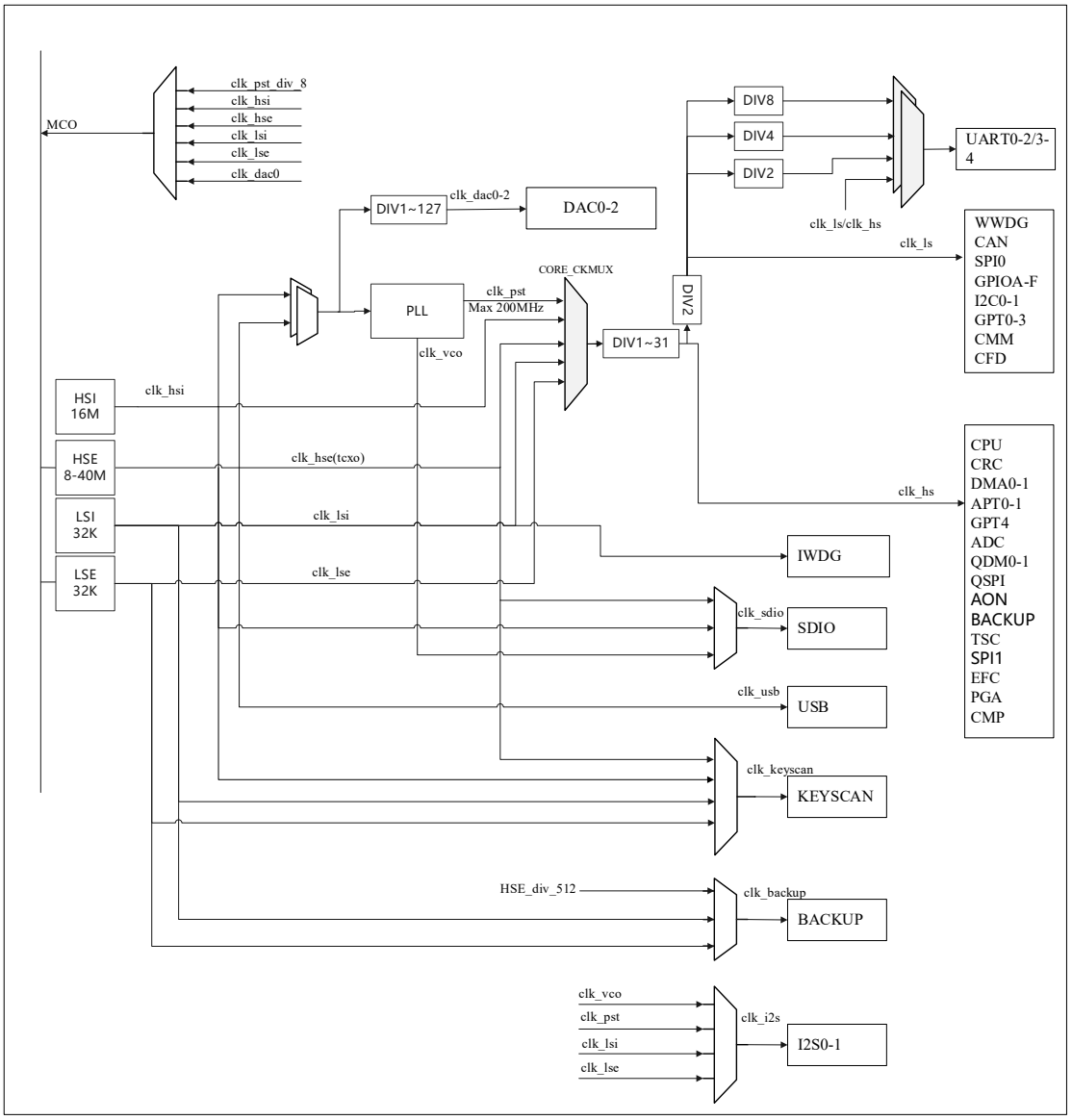
  当对系统控制器（SYSCTRL）的 SYS_RST_REQ.soft_rst_req 位域进行写操作时，触发软件复位。 软件复位会将 CORE 电源域内寄存器复位，但不包含：

  - CRG 中所有配置寄存器；
  - 系统控制器（SYSCTRL）中 RW_HRST 以及 （SYSCTRL 和 EFC）RW_POR 属性的寄存器。

4.3 时钟

本章节介绍芯片的时钟源和时钟域，以及如何在应用中配置时钟。芯片的时钟系统如图 4-1 所示。

图 4-1 时钟系统框图



4.3.1 源时钟

源时钟用于衍生芯片各模块所需的时钟。芯片共有四个源时钟资源：HSI、LSI、HSE、LSE。

- **HSI**：HSI 是芯片集成的高精度高速晶振，提供 16MHz 时钟 clk_hsi。clk_hsi 是芯片 CORE 电源域的默认时钟；
- **LSI**：LSI 是芯片集成的低速晶振，提供 32.768KHz 时钟 clk_lsi。clk_lsi 是芯片 AON 电源域的默认时钟，也为 IWDG 模块提供计数时钟；

- **HSE**：HSE 支持高速外部晶体或外部时钟源，提供 8~40MHz 时钟 clk_hse；
- **LSE**：LSE 是外部晶体或外部时钟源，提供 32.768KHz 时钟 clk_lse。

4.3.2 PLL

- PLL 时钟计算公式
  - 整数分频公式：

$$clk_pst = \frac{clk_pll_ref * fbdiv}{refdiv * postdiv}$$

  - 小数分频公式：

$$clk_{pst} = \frac{clk_{pll_{ref}}}{refdiv} \frac{fbdiv + \frac{frac}{2^{24}}}{postdiv}$$

- PLL 分频器
  - 支持前置分频器 refdiv，1~63 分频系数可配；
  - 支持反馈分频器 fbdiv，整数分频 16~2400 范围可配，小数分频 20~240 范围可配；
  - 支持后置分频器 postdiv1，1~7 分频系数可配，postdiv1>= postdiv2；
  - 支持后置分频器 postdiv2，1~7 分频系数可配，postdiv1>= postdiv2。
- PLL 频率范围
  - PLL 输入参考时钟 clk_pll_ref，允许输入参考时钟频率范围 Fclk_pll_ref：整数分频 1MHz~800MHz，小数分频 10MHz~800MHz；
  - PLL 内部参考时钟 clk_pfd，其频率 Fclk_pfd = Fclk_pll_ref / refdiv，允许频率范围：整数分频 1MHz~150MHz，小数分频 10MHz~120MHz；
  - PLL VCO 输出时钟 clk_vco，其频率 Fclk_vco = Fclk_pfd * fbdiv，允许频率范围： 600MHz~2400MHz；
  - PLL 输出时钟 clk_pst，其频率 Fclk_pst = Fclk_vco /（postdiv1* postdiv2） = Fclk_pll_ref / prediv * fbdiv /（postdiv1* postdiv2），允许频率最大：200MHz。
- PLL 配置流程

步骤 1 解锁 CRG 寄存器写保护，当系统控制器（SYSCTRL）的 crg_cfg_lock 寄存器位域为 0 时，对 PLL 配置寄存器的写操作才有效；

步骤 2 关闭 PLL 锁定检测，通过 CRG 的 PERI_CRG5 寄存器 pll_lockdet_pd 位域置位为值 1 实现；

步骤 3 关闭 PLL，通过 CRG 的 PERI_CRG4 寄存器 pll_pd 位域置位为值 1 实现；

步骤 4 PLL 参考时钟 pll_ref_cksel 源选择，通过 CRG 的 PERI_CRG0 寄存器实现；

步骤 5 PLL 内部分频器 refdiv、fbdiv、postdiv1/2 以及 frac(PLL 小数分频反馈分频比)配置，通过 CRG 的 PERI_CRG1~3 寄存器实现；

步骤 6 如果需要小数分频，PERI_CRG4 寄存器 pll_dsmpd 和 pll_dacpd 位域置为 0；

步骤 7 使能 PLL，通过 CRG 的 PERI_CRG4 寄存器 pll_pd 位域清零为值 0 实现；

步骤 8 使能 PLL 锁定检测，通过 CRG 的 PERI_CRG5 寄存器 pll_lockdet_pd 位域清零为值 0 实现；

步骤 9 等待 PLL 锁定，通过查询 CRG 的 PERI_CRG8 寄存器实现；

步骤 10 关闭 PLL bypass，通过 CRG 的 PERI_CRG0 寄存器 pll_bypass 位域清零为值 0 实现；

步骤 11 重新锁定 CRG 寄存器写保护，当系统控制器（SYSCTRL）的 crg_cfg_lock 寄存器位域为 1 时，对 PLL 配置寄存器的写操作无效。

4.3.3 衍生时钟

clk_hs

clk_hs 为系统主工作时钟，最大支持 200MHz。为 CPU、SRAM、高速外设以及总线 SYS_BUS 提供时钟。

- clk_hs 时钟源：
  - clk_hsi（默认） clk_hse clk_pst clk_lsi clk_lse
- 高速外设：
  - DMAC0~1 CRC APT0~1 GPT4 QDM0~1 QSPI UART3~4 SPI1 TSC ADC0~1 SDIO USB DAC0~2 EFLASH BACKUP

clk_ls

clk_ls 是 clk_hs 的二分频时钟，为系统低速外设总线提供时钟，也为低速外设提供时钟。还有部分低速外设可以选择 clk_ls 的 2/4/8 分频作为其工作时钟。

- clk_ls 时钟源：
  - clk_hs 的二分频
- 低速外设：
  - I2C0~1 I2S0~1 GPIOA~F SPI0 GPT0~3 WWDG IWDG CAN KEYSCAN
- 可选 clk_ls 分频的低速外设：
  - UART0~2

其它衍生时钟：

clk_dac0/1

clk_dac0/1 为 DAC 工作时钟，最大支持 40MHz。

- clk_dac0/1 时钟源有：
  - clk_hsi（默认） clk_hse

clk_sdio

clk_sdio 为 SDIO 内部工作时钟，最大支持 25MHz。

- clk_sdio 时钟源有：
  - clk_hsi（默认） clk_vco clk_hse

clk_usb

clk_usb 为 usb 工作时钟，最大支持 48MHz。

- clk_usb 时钟源有：
  - clk_hse（默认）

4.3.4 输出时钟

可以将芯片内部时钟输出到 IO 管脚提供给外部使用（此管脚为 PD9，此时 PD9 要复用成 MCO 功能）。通过 PERI_CRG67 寄存器选择要输出的时钟。

- 可输出时钟有：
  - clk_pst_div8（clk_pst 的 8 分频） clk_hsi clk_hse clk_lsi clk_lse clk_dac0

4.3.5 寄存器写保护

为避免非法配置导致系统异常，CRG 部分寄存器支持写保护功能。该功能由系统控制器（SYSCTRL）的 LOCKEN 寄存器 crg_cfg_lock 位域控制。当 crg_cfg_lock 位域值为 1 时，对 CRG 写保护寄存器进行写操作无效。CRG 中支持写保护的寄存器为 PLL、BACKUP 和 AON 的配置寄存器，详见寄存器描述。

4.3.6 系统时钟切换流程

切换到 clk_pst

上电之后系统主时钟默认工作在 clk_hsi，等待 PLL 锁定，切换到 clk_pst。

步骤 1 首先配置 PERI_CRG7.core_cksel[3]=0 、PERI_CRG7.core_cken[1]=1，再配置 PERI_CRG7.core_cksel[4]=1；

步骤 2 软件等待 1us；

步骤 3 然后配置 PERI_CRG7.core_cksel[1]=1，PERI_CRG7.core_cken[0]=1，再配置 PERI_CRG7.core_cksel[4]=0；

此时 CORE_CKMUX 输出 clk_pst 时钟作为系统主时钟。

系统工作在 clk_pst 之后才可以切换到其它时钟源作为系统时钟；无论当前系统工作在哪个时钟源，切回到 pll 时钟的配置都如上面第 3 步；无论从当前时钟切回到 pll 时钟，还是 pll 时钟换到其它时钟，都要保证当前时钟和被切换时钟有效。

切换到 clk_hsi

步骤 1 切回到 pll 时钟，配置 PERI_CRG7.core_cksel[1]=1，PERI_CRG7.core_cken[0]=1，再配置 PERI_CRG7.core_cksel[4]=0；

步骤 2 软件等待 1us；

步骤 3 然后配置 PERI_CRG7.core_cksel[0]=0，PERI_CRG7.core_cksel[3]=0，PERI_CRG7.core_cken[1]=1，再配置 PERI_CRG7.core_cksel[4]=1。

切换到 clk_hse

步骤 1 切回到 pll 时钟，配置 PERI_CRG7.core_cksel[1]=1，PERI_CRG7.core_cken[0]=1，再配置 PERI_CRG7.core_cksel[4]=0；

步骤 2 软件等待 1us；

步骤 3 然后配置 PERI_CRG7.core_cksel[0]=1，PERI_CRG7.core_cksel[3]=0，PERI_CRG7.core_cken[1]=1，再配置 PERI_CRG7.core_cksel[4]=1。

切换到 clk_lsi

步骤 1 切回到 pll 时钟，配置 PERI_CRG7.core_cksel[1]=1，PERI_CRG7.core_cken[0]=1，再配置 PERI_CRG7.core_cksel[4]=0；

步骤 2 软件等待 0.2ms；

步骤 3 然后配置 PERI_CRG7.core_cksel[2]=0，PERI_CRG7.core_cksel[3]=1，PERI_CRG7.core_cken[1]=1，PERI_CRG7.core_cken[2]=1，再配置 PERI_CRG7.core_cksel[4]=1。

切换到 clk_lse

步骤 1 切回到 pll 时钟，配置 PERI_CRG7.core_cksel[1]=1，PERI_CRG7.core_cken[0]=1，再配置 PERI_CRG7.core_cksel[4]=0；

步骤 2 软件等待 0.2ms；

步骤 3 然后配置 PERI_CRG7.core_cksel[2]=1，PERI_CRG7.core_cksel[3]=1，PERI_CRG7.core_cken[1]=1，PERI_CRG7.core_cken[2]=1，再配置 PERI_CRG7.core_cksel[4]=1。

4.4 CRG 寄存器概览

CRG 寄存器基址如下：

- CRG 寄存器基址为：0x61000000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	PERI_CRG0	0x0	32	RW	0x00010000	PLL 寄存器 0
2	PERI_CRG1	0x4	32	RW	0x00000001	PLL 寄存器 1
3	PERI_CRG2	0x8	32	RW	0x00000032	PLL 寄存器 2
4	PERI_CRG3	0xC	32	RW	0x00000104	PLL 寄存器 3
5	PERI_CRG4	0x10	32	RW	0x00000309	PLL 寄存器 4
6	PERI_CRG5	0x14	32	RW	0x00000100	PLL 寄存器 5
7	PERI_CRG7	0x1C	32	RW	0x00010300	系统时钟复位控制寄存器 7
8	PERI_CRG8	0x20	32	RO	0x00000000	PLL 输出 lock 信息寄存器
9	PERI_CRG21	0x54	32	RW	0x00000001	uart0 时钟及软复位控制寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
10	PERI_CRG22	0x58	32	RW	0x00000000	uart1 时钟及软复位控制寄存器
11	PERI_CRG23	0x5C	32	RW	0x00000000	uart2 时钟及软复位控制寄存器
12	PERI_CRG24	0x60	32	RW	0x00000001	uart3 时钟及软复位控制寄存器
13	PERI_CRG25	0x64	32	RW	0x00000000	uart4 时钟及软复位控制寄存器
14	PERI_CRG26	0x68	32	RW	0x00000000	gpt0 时钟及软复位控制寄存器
15	PERI_CRG27	0x6C	32	RW	0x00000000	gpt1 时钟及软复位控制寄存器
16	PERI_CRG28	0x70	32	RW	0x00000000	gpt2 时钟及软复位控制寄存器
17	PERI_CRG29	0x74	32	RW	0x00000000	gpt3 时钟及软复位控制寄存器
18	PERI_CRG30	0x78	32	RW	0x00000000	i2c0 时钟及软复位控制寄存器
19	PERI_CRG31	0x7C	32	RW	0x00000000	i2c1 时钟及软复位控制寄存器
20	PERI_CRG33	0x84	32	RW	0x00000000	spi0 时钟及软复位控制寄存器
21	PERI_CRG35	0x8C	32	RW	0x00000000	spi1 时钟及软复位控制寄存器
22	PERI_CRG37	0x94	32	RW	0x00000001	wwdg 时钟及软复位控制寄存器
23	PERI_CRG40	0xA0	32	RW	0x0000003F	gpio 时钟及软复位控制寄存器
24	PERI_CRG41	0xA4	32	RW	0x00000000	can 时钟及软复位控制寄存器
25	PERI_CRG42	0xA8	32	RW	0x01000000	i2s0 mclk 时钟及软复位控制寄存器
26	PERI_CRG44	0xB0	32	RW	0x00000000	i2s1 mclk 时钟及软复位控制寄存器
27	PERI_CRG46	0xB8	32	RW	0x01000000	keyscan 时钟及软复位控制寄存器
28	PERI_CRG47	0xBC	32	RW	0x00000000	tsc 时钟及软复位控制寄存器
29	PERI_CRG49	0xC4	32	RW	0x00000001	crc/adc 时钟及软复位控制寄存器

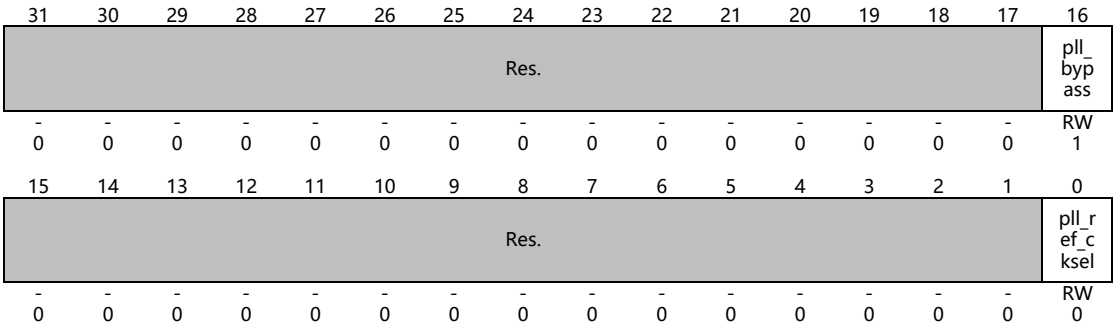
No.	Names	Offset Address	SIZE	R/W	Default	Description
30	PERI_CRG50	0xC8	32	RW	0x00000000	apt0 时钟及软复位控制寄存器
31	PERI_CRG51	0xCC	32	RW	0x00000000	apt1 时钟及软复位控制寄存器
32	PERI_CRG52	0xD0	32	RW	0x00000000	qdm0 时钟及软复位控制寄存器
33	PERI_CRG53	0xD4	32	RW	0x00000000	qdm1 时钟及软复位控制寄存器
34	PERI_CRG54	0xD8	32	RW	0x00000000	gpt4 时钟及软复位控制寄存器
35	PERI_CRG57	0xE4	32	RW	0x00000100	dac0 时钟及软复位控制寄存器
36	PERI_CRG58	0xE8	32	RW	0x00000100	dac1 时钟及软复位控制寄存器
37	PERI_CRG59	0xEC	32	RW	0x00000100	dac2 时钟及软复位控制寄存器
38	PERI_CRG60	0xF0	32	RW	0x00000000	qspi 时钟及软复位控制寄存器
39	PERI_CRG61	0xF4	32	RW	0x00C80000	sdio 时钟及软复位控制寄存器
40	PERI_CRG62	0xF8	32	RW	0x00000001	sdio tx 时钟及软复位控制寄存器
41	PERI_CRG63	0xFC	32	RW	0x00000000	sdio rx 时钟及软复位控制寄存器
42	PERI_CRG64	0x100	32	RW	0x00000A00	sdio card 时钟及软复位控制寄存器
43	PERI_CRG65	0x104	32	RW	0x00000000	usb 时钟及软复位控制寄存器
44	PERI_CRG66	0x108	32	RW	0x00010001	dmac 时钟及软复位控制寄存器
45	PERI_CRG67	0x10C	32	RW	0x00000000	时钟输出控制寄存器
46	PERI_CRG68	0x110	32	RO	0x00000000	clk_hse ready 状态寄存器
47	PERI_CRG69	0x114	32	RW	0x00000400	aon_cfg 时钟及软复位控制寄存器
48	PERI_CRG70	0x118	32	RW	0x000000A0	backup_cfg 时钟及软复位控制寄存器
49	PERI_CRG71	0x11C	32	RW	0x00000000	cmm cfd 复位控制寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
50	PERI_CRG72	0x120	32	RW	0x00000000	hpm 时钟复位控制寄存器
51	PERI_CRG73	0x124	32	RW	0x00004E82	xo 时钟控制寄存器 0 crg_to_ana_x_xo_ctrl 0
52	PERI_CRG74	0x128	32	RW	0x00005E00	xo 时钟控制寄存器 1 crg_to_ana_x_xo_ctrl 1
53	PERI_CRG75	0x12C	32	RW	0x00000000	hsi 时钟控制寄存器 crg_to_ana_x_hsi_ctrl

4.5 CRG 寄存器描述

4.5.1 PERI_CRG0

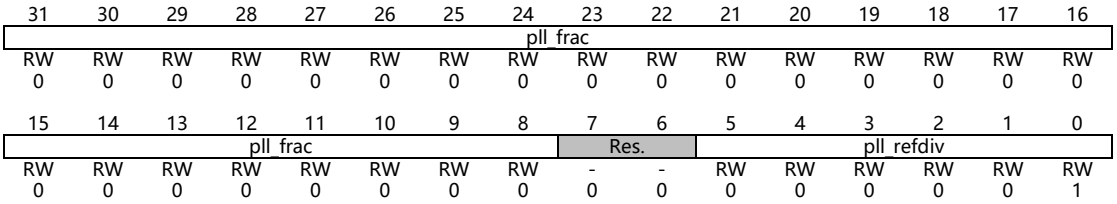
◆ **偏移地址：0x0　　复位值：0x00010000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
pll_bypass	[16]	RW	PLL 旁路控制信号： 0：不旁路，即使用 PLL 时钟； 1：旁路，即不使用 PLL 时钟。	0x1
reserved	[15:1]	-	保留。	0x0
pll_ref_cksel	[0]	RW	PLL 输入参考时钟 clk_pll_ref 时钟源选择信号： 0：选择 clk_hsi； 1：选择 clk_hse。	0x0

4.5.2 PERI_CRG1

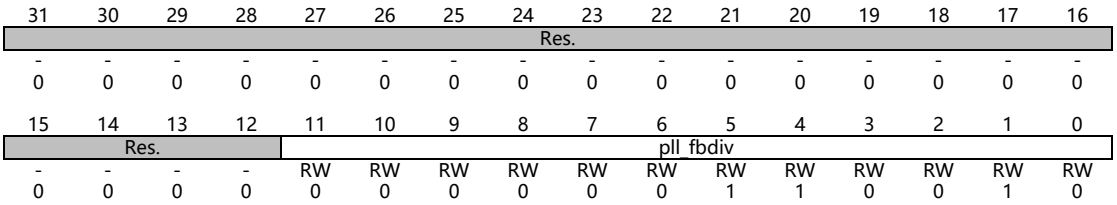
◆ **偏移地址：0x4　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
pll_frac	[31:8]	RW	PLL 小数分频反馈分频比： 24 比特可配。	0x0
reserved	[7:6]	-	保留。	0x0
pll_refdiv	[5:0]	RW	PLL 内部前置分频器分频比： 允许配置范围(1~63)。	0x1

4.5.3 PERI_CRG2

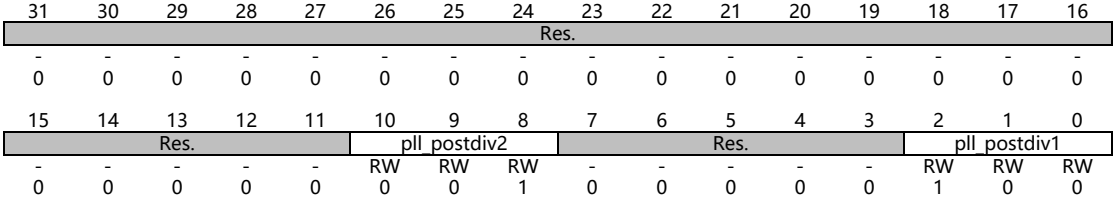
◆ **偏移地址：0x8　　复位值：0x00000032**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留。	0x0
pll_fbdiv	[11:0]	RW	PLL 内部反馈分频比： 允许配置范围整数分频 16~2400；小数分频 20~240。	0x32

4.5.4 PERI_CRG3

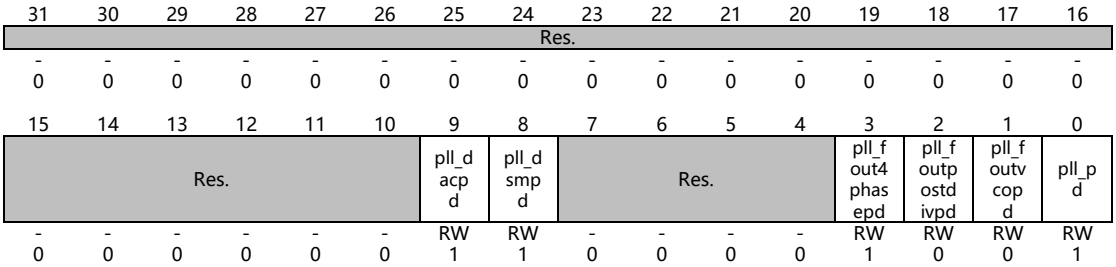
◆ **偏移地址：0xC　　复位值：0x00000104**



Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
pll_postdiv2	[10:8]	RW	PLL 内部后置分频器分频比 2： 允许配置范围（1~7），必须 pll_postdiv1 配置 7 之后才能使用 pll_postdiv2。	0x1
reserved	[7:3]	-	保留。	0x0
pll_postdiv1	[2:0]	RW	PLL 内部后置分频器分频比 1： 允许配置范围（1~7），必须 pll_postdiv1 配置 7 之后才能使用 pll_postdiv2。	0x4

4.5.5 PERI_CRG4

◆ **偏移地址：0x10　　复位值：0x00000309**

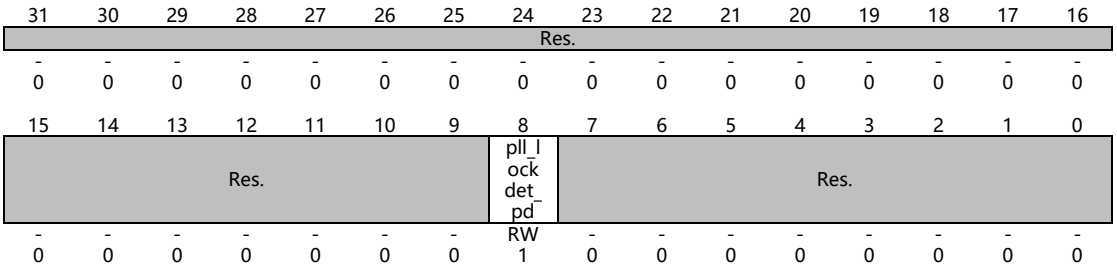


Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留。	0x0
pll_dacpd	[9]	RW	FRAC 模式下使能 noise canceling DAC： 0：DAC 上电使能； 1：DAC 下电关闭。	0x1
pll_dsmpd	[8]	RW	DSM 小数分频使能控制： 0：DSM 小数分频上电使能； 1：DSM 小数分频下电使能。	0x1
reserved	[7:4]	-	保留。	0x0
pll_fout4phasepd	[3]	RW	FOUT 相位产生使能控制： 0：FOUT 相位产生上电使能（必须 pll_foutpostdivpd 上电使能）； 1：FOUT 相位产生下电使能。	0x1

Name	Bits	R/W	Description	Default
pll_foutpostdivpd	[2]	RW	FOUTPOSTDIV 使能控制： 0：上电使能； 1：下电使能。	0x0
pll_foutvcopd	[1]	RW	FOUTVCO 使能控制： 0：上电使能； 1：下电使能。	0x0
pll_pd	[0]	RW	PLL 使能控制： 0：上电使能； 1：下电使能。	0x1

4.5.6 PERI_CRG5

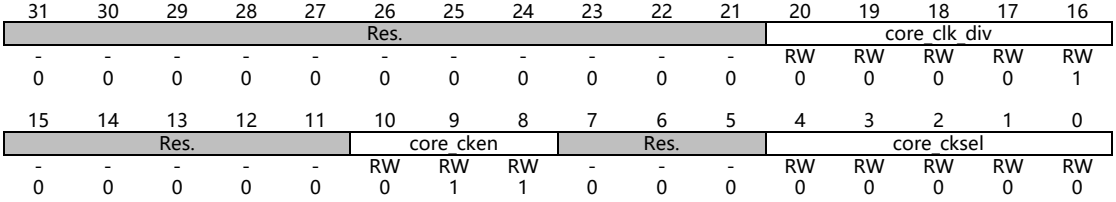
◆ **偏移地址：0x14　　复位值：0x00000100**



Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
pll_lockdet_pd	[8]	RW	PLL 锁定检测控制： 0：PLL 锁定检测使能； 1：PLL 锁定检测关闭。 PLL 下电关闭时需关闭锁定检测，PLL 上电使能后需打开锁定检测。	0x1
reserved	[7:0]	-	保留。	0x0

4.5.7 PERI_CRG7

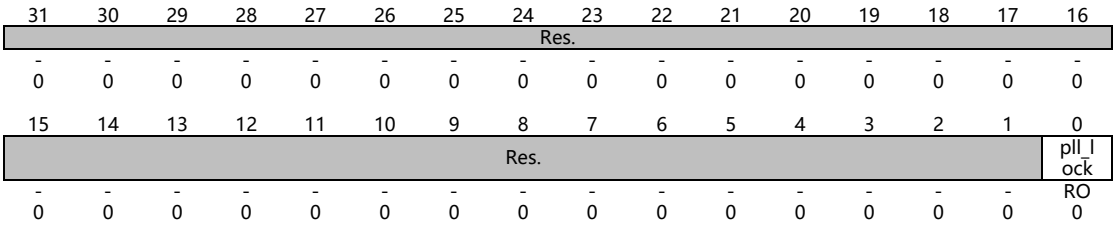
◆ **偏移地址：0x1C　　复位值：0x00010300**



Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
core_clk_div	[20:16]	RW	系统主时钟分频器分频比： 允许配置范围(1~31)。	0x1
reserved	[15:11]	-	保留。	0x0
core_cken	[10:8]	RW	系统时钟门控，需要与 core_cksel 配合选择系统时钟，具体操作请参考用户指南 4.3.6 章节	0x3
reserved	[7:5]	-	保留。	0x0
core_cksel	[4:0]	RW	系统时钟选择信号,需要与 core_cken 配合选择系统时钟，可选时钟有： clk_hsi clk_pst clk_hse clk_lsi clk_lse 默认选择 clk_hsi，具体操作请参考用户指南 4.3.6 章节。	0x0

4.5.8 PERI_CRG8

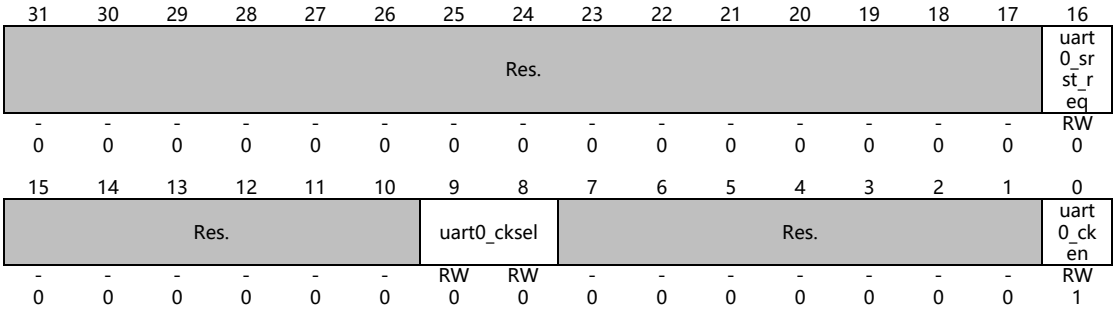
◆ **偏移地址：0x20　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
pll_lock	[0]	RO	pll lock 信号。 0：pll 没锁住； 1：pll 锁住。	0x0

4.5.9 PERI_CRG21

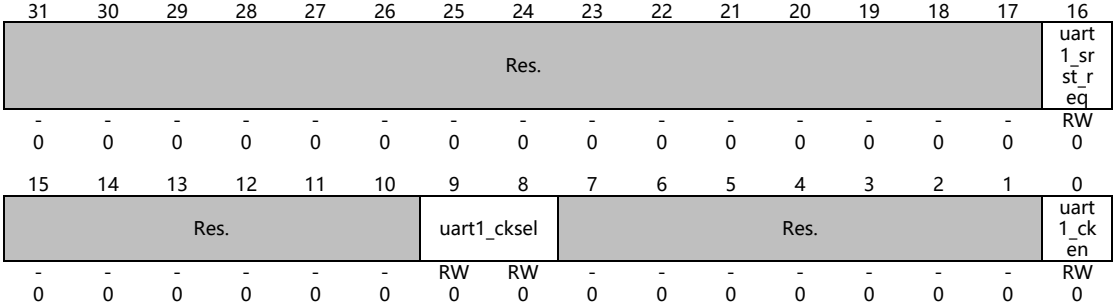
◆ **偏移地址：0x54　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
uart0_srst_req	[16]	RW	uart0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
uart0_cksel	[9:8]	RW	uart0 时钟选择信号： 0：选择 clk_ls 时钟； 1：选择 clk_ls/2 时钟； 2：选择 clk_ls/4 时钟； 3：选择 clk_ls/8 时钟。	0x0
reserved	[7:1]	-	保留。	0x0
uart0_cken	[0]	RW	uart0 时钟门控： 0：关断； 1：打开。	0x1

4.5.10 PERI_CRG22

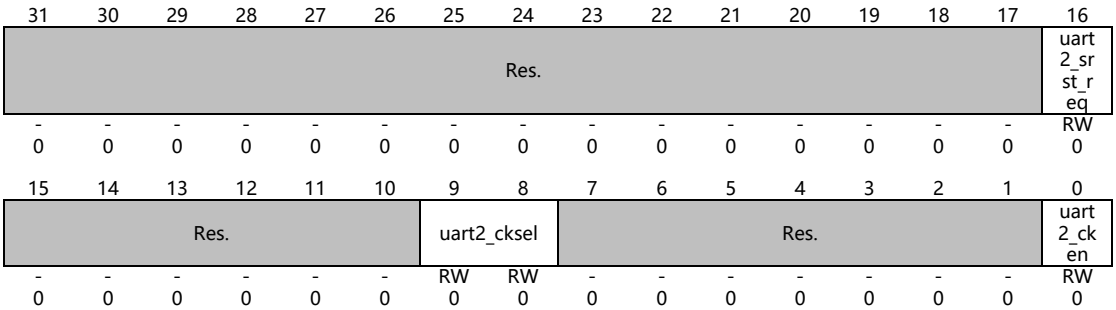
◆ **偏移地址：0x58　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
uart1_srst_req	[16]	RW	uart1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
uart1_cksel	[9:8]	RW	uart1 时钟选择信号： uart1 时钟选择信号： 0：选择 clk_ls 时钟； 1：选择 clk_ls/2 时钟； 2：选择 clk_ls/4 时钟； 3：选择 clk_ls/8 时钟	0x0
reserved	[7:1]	-	保留。	0x0
uart1_cken	[0]	RW	uart1 时钟门控： 0：关断； 1：打开。	0x0

4.5.11 PERI_CRG23

◆ **偏移地址：0x5C　　复位值：0x00000000**

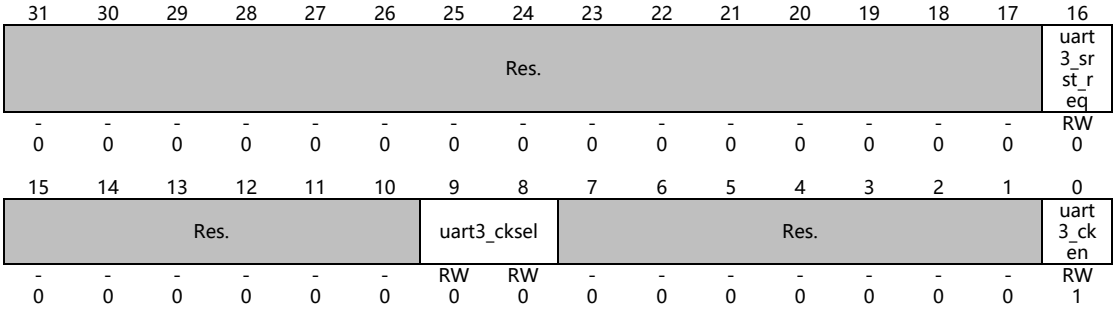


Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0

Name	Bits	R/W	Description	Default
uart2_srst_req	[16]	RW	uart2 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
uart2_cksel	[9:8]	RW	uart2 时钟选择信号： 0：选择 clk_ls 时钟； 1：选择 clk_ls/2 时钟； 2：选择 clk_ls/4 时钟； 3：选择 clk_ls/8 时钟	0x0
reserved	[7:1]	-	保留。	0x0
uart2_cken	[0]	RW	uart2 时钟门控： 0：关断； 1：打开。	0x0

4.5.12 PERI_CRG24

◆ **偏移地址：0x60　　复位值：0x00000001**

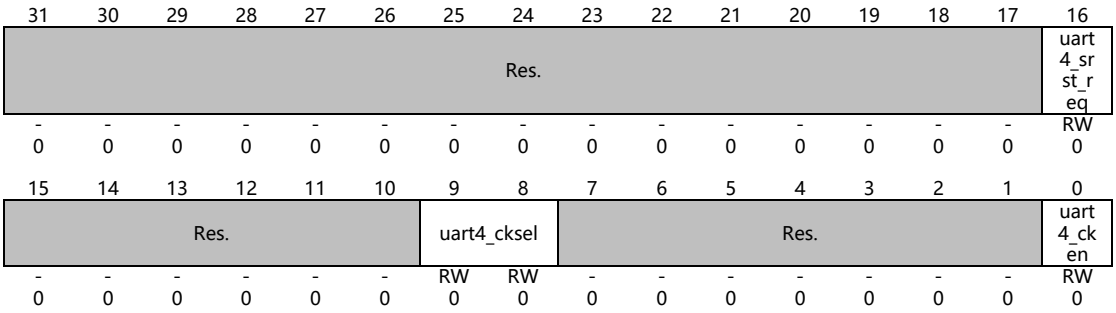


Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
uart3_srst_req	[16]	RW	uart3 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
uart3_cksel	[9:8]	RW	uart3 时钟选择信号： 0：选择 clk_hs 时钟； 1：选择 clk_hs/2 时钟； 2：选择 clk_hs/4 时钟； 3：选择 clk_hs/8 时钟。	0x0
reserved	[7:1]	-	保留。	0x0
uart3_cken	[0]	RW	uart3 时钟门控： 0：关断；	0x1

Name	Bits	R/W	Description	Default
			1：打开。	

4.5.13 PERI_CRG25

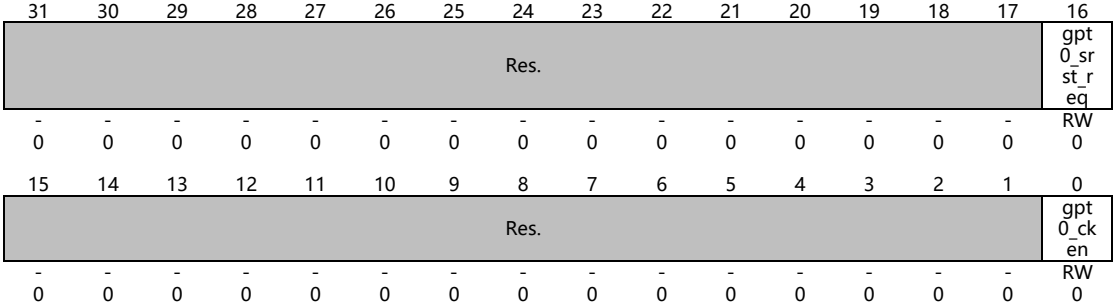
◆ **偏移地址：0x64　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
uart4_srst_req	[16]	RW	uart4 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
uart4_cksel	[9:8]	RW	uart4 时钟选择信号： 0：选择 clk_hs 时钟； 1：选择 clk_hs/2 时钟； 2：选择 clk_hs/4 时钟； 3：选择 clk_hs/8 时钟。	0x0
reserved	[7:1]	-	保留。	0x0
uart4_cken	[0]	RW	uart4 时钟门控： 0：关断； 1：打开。	0x0

4.5.14 PERI_CRG26

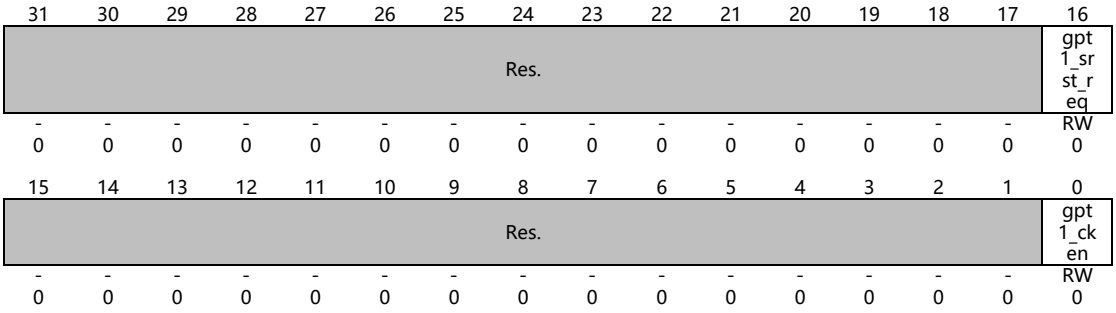
◆ **偏移地址：0x68　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
gpt0_srst_req	[16]	RW	gpt0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
gpt0_cken	[0]	RW	gpt0 时钟门控： 0：关断； 1：打开。	0x0

4.5.15 PERI_CRG27

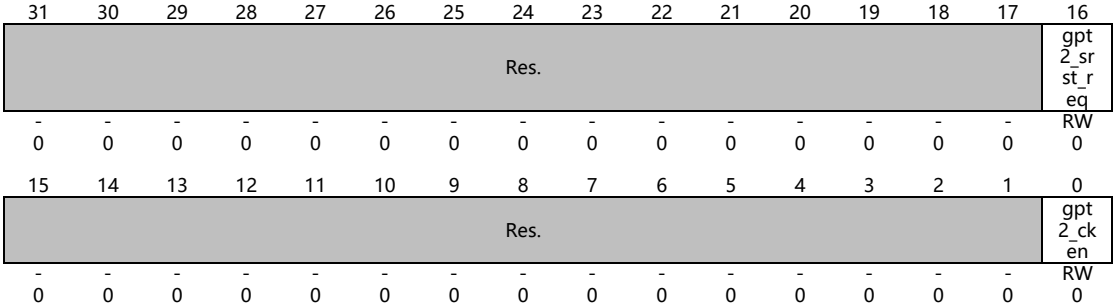
◆ **偏移地址：0x6C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
gpt1_srst_req	[16]	RW	gpt1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
gpt1_cken	[0]	RW	gpt1 时钟门控： 0：关断； 1：打开。	0x0

4.5.16 PERI_CRG28

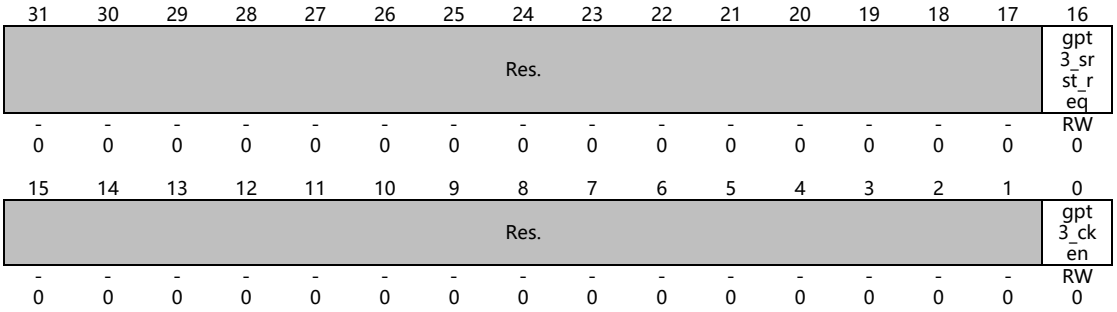
◆ **偏移地址：0x70　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
gpt2_srst_req	[16]	RW	gpt2 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
gpt2_cken	[0]	RW	gpt2 时钟门控： 0：关断； 1：打开。	0x0

4.5.17 PERI_CRG29

◆ **偏移地址：0x74　　复位值：0x00000000**

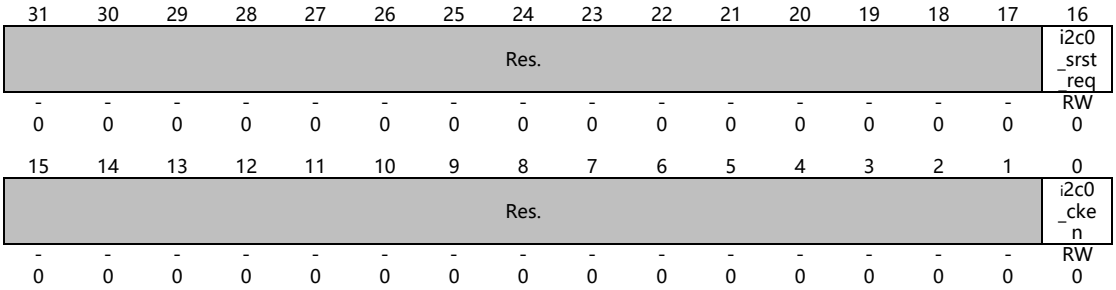


Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
gpt3_srst_req	[16]	RW	gpt3 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0

Name	Bits	R/W	Description	Default
gpt3_cken	[0]	RW	gtp3 时钟门控： 0：关断； 1：打开。	0x0

4.5.18 PERI_CRG30

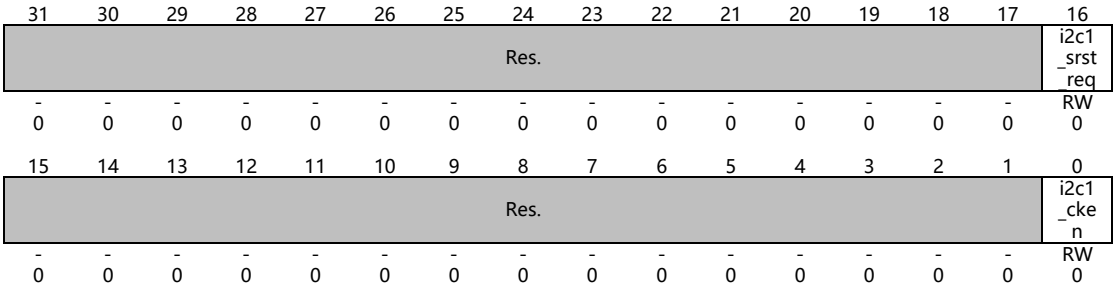
◆ **偏移地址：0x78　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
i2c0_srst_req	[16]	RW	i2c0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
i2c0_cken	[0]	RW	i2c0 时钟门控： 0：关断； 1：打开。	0x0

4.5.19 PERI_CRG31

◆ **偏移地址：0x7C　　复位值：0x00000000**

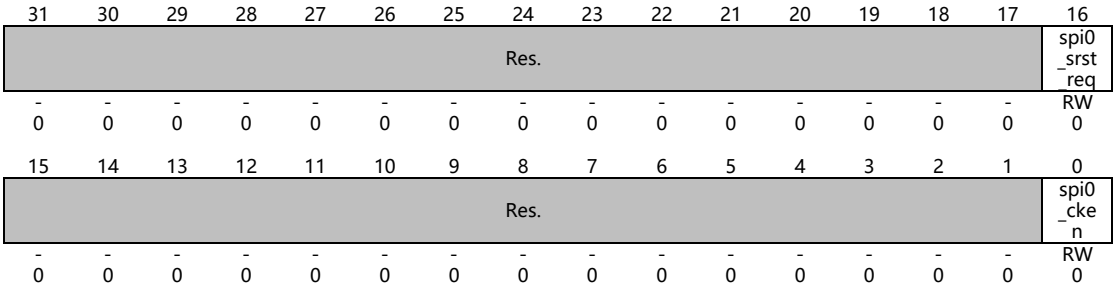


Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0

Name	Bits	R/W	Description	Default
i2c1_srst_req	[16]	RW	i2c1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
i2c1_cken	[0]	RW	i2c1 时钟门控： 0：关断； 1：打开。	0x0

4.5.20 PERI_CRG33

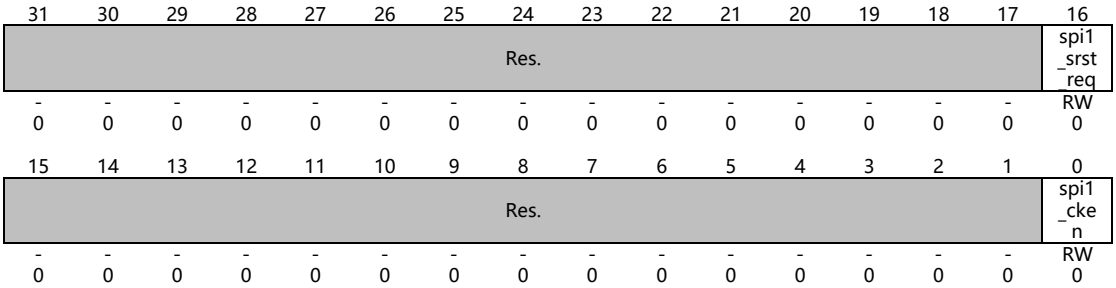
◆ **偏移地址：0x84　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
spi0_srst_req	[16]	RW	spi0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
spi0_cken	[0]	RW	spi0 时钟门控： 0：关断； 1：打开。	0x0

4.5.21 PERI_CRG35

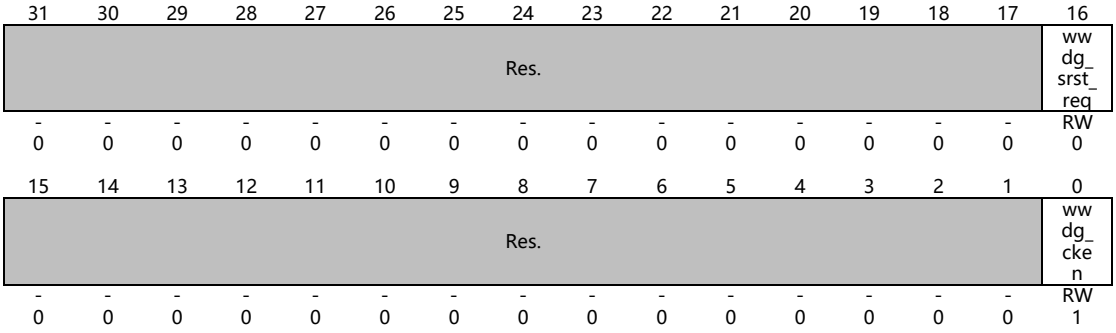
◆ **偏移地址：0x8C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
spi1_srst_req	[16]	RW	spi1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
spi1_cken	[0]	RW	spi1 时钟门控： 0：关断； 1：打开。	0x0

4.5.22 PERI_CRG37

◆ **偏移地址：0x94　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
wwdg_srst_req	[16]	RW	wwdg 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
wwdg_cken	[0]	RW	wwdg 时钟门控： 0：关断； 1：打开。	0x1

4.5.23 PERI_CRG40

◆ **偏移地址：0xA0　　复位值：0x0000003F**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.										gpiof_srst_req	gpioe_srst_req	gpiod_srst_req	gpioc_srst_req	gpiob_srst_req	gpioa_srst_req
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										gpiof_cken	gpioe_cken	gpiod_cken	gpioc_cken	gpiob_cken	gpioa_cken
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:22]	-	保留。	0x0
gpiof_srst_req	[21]	RW	gpiof 软复位请求： 0：不复位； 1：复位。	0x0
gpioe_srst_req	[20]	RW	gpioe 软复位请求： 0：不复位； 1：复位。	0x0
gpiod_srst_req	[19]	RW	gpiod 软复位请求： 0：不复位； 1：复位。	0x0
gpioc_srst_req	[18]	RW	gpioc 软复位请求： 0：不复位； 1：复位。	0x0
gpiob_srst_req	[17]	RW	gpiob 软复位请求： 0：不复位； 1：复位。	0x0
gpioa_srst_req	[16]	RW	gpioa 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:6]	-	保留。	0x0
gpiof_cken	[5]	RW	gpiof 时钟门控： 0：关断； 1：打开。	0x1
gpioe_cken	[4]	RW	gpioe 时钟门控： 0：关断； 1：打开。	0x1
gpiod_cken	[3]	RW	gpiod 时钟门控：	0x1

Name	Bits	R/W	Description	Default
			0：关断； 1：打开。	
gpioc_cken	[2]	RW	gpioc 时钟门控： 0：关断； 1：打开。	0x1
gpiob_cken	[1]	RW	gpiob 时钟门控： 0：关断； 1：打开。	0x1
gpioa_cken	[0]	RW	gpioa 时钟门控： 0：关断； 1：打开。	0x1

4.5.24 PERI_CRG41

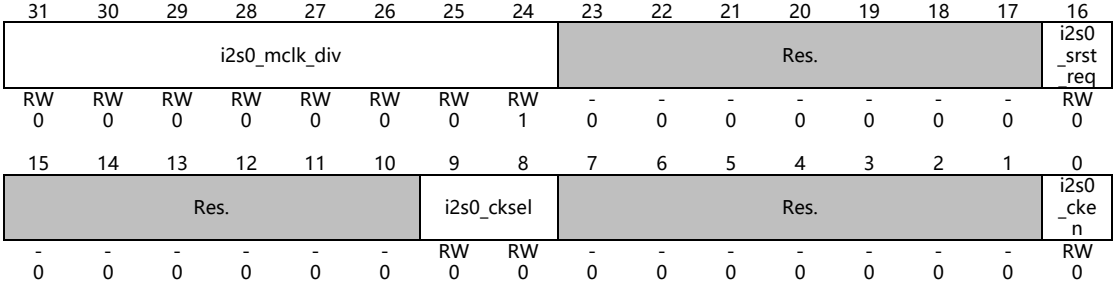
◆ **偏移地址：0xA4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															can_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															can_cken
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
can_srst_req	[16]	RW	can 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
can_cken	[0]	RW	can 时钟门控： 0：关断； 1：打开。	0x0

4.5.25 PERI_CRG42

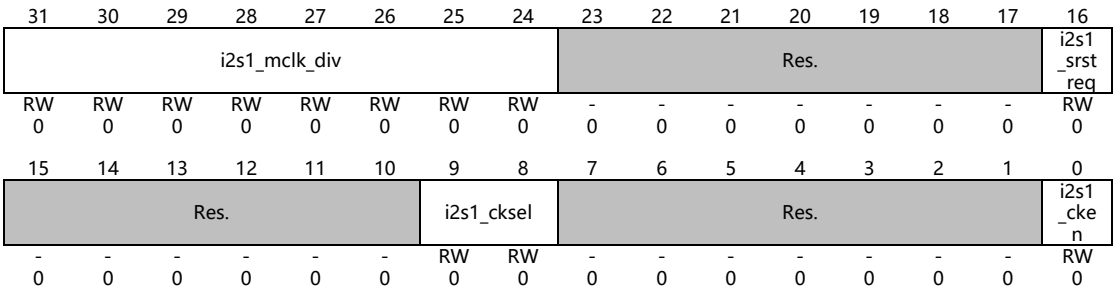
◆ **偏移地址：0xA8　　复位值：0x01000000**



Name	Bits	R/W	Description	Default
i2s0_mclk_div	[31:24]	RW	i2s0 主时钟分频器分频比: 允许配置范围(1~255)。	0x1
reserved	[23:17]	-	保留。	0x0
i2s0_srst_req	[16]	RW	i2s0 软复位请求: 0：不复位; 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
i2s0_cksel	[9:8]	RW	i2s0 时钟选择信号: 0：选择 clk_vco 时钟; 1：选择 clk_pst 时钟; 2：选择 clk_hsi 时钟; 3：选择 clk_hse 时钟。	0x0
reserved	[7:1]	-	保留。	0x0
i2s0_cken	[0]	RW	i2s0 时钟门控: 0：关断; 1：打开。	0x0

4.5.26 PERI_CRG44

◆ **偏移地址：0xB0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
i2s1_mclk_div	[31:24]	RW	i2s1 主时钟分频器分频比:	0x0

Name	Bits	R/W	Description	Default
			允许配置范围(1~255)。	
reserved	[23:17]	-	保留。	0x0
i2s1_srst_req	[16]	RW	i2s1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
i2s1_cksel	[9:8]	RW	i2s1 时钟选择信号： 0：选择 clk_vco 时钟； 1：选择 clk_pst 时钟； 2：选择 clk_hsi 时钟； 3：选择 clk_hse 时钟。	0x0
reserved	[7:1]	-	保留。	0x0
i2s1_cken	[0]	RW	i2s1 时钟门控： 0：关断； 1：打开。	0x0

4.5.27 PERI_CRG46

◆ **偏移地址：0xB8　　复位值：0x01000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.		keyscan_div						Res.							keyscan_srst_req
-	-	RW	RW	RW	RW	RW	RW	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.						keyscan_cksel		Res.							keyscan_cken
-	-	-	-	-	-	RW	RW	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:30]	-	保留。	0x0
keyscan_div	[29:24]	RW	keyscan 时钟分频器分频比： 允许配置范围(1~63)。	0x1
reserved	[23:17]	-	保留。	0x0
keyscan_srst_req	[16]	RW	keyscan 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:10]	-	保留。	0x0
keyscan_cksel	[9:8]	RW	keyscan 时钟选择信号： 0：选择 clk_lsi 时钟； 1：选择 clk_lse 时钟；	0x0

Name	Bits	R/W	Description	Default
			2：选择 clk_hsi 时钟； 3：选择 clk_hse 时钟。	
reserved	[7:1]	-	保留。	0x0
keyscan_cken	[0]	RW	keyscan 时钟门控： 0：关断； 1：打开。	0x0

4.5.28 PERI_CRG47

◆ **偏移地址：0xBC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															tsc_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															tsc_cken
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
tsc_srst_req	[16]	RW	tsc 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
tsc_cken	[0]	RW	tsc 时钟门控： 0：关断； 1：打开。	0x0

4.5.29 PERI_CRG49

◆ **偏移地址：0xC4　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															adc_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															crc_cken/adc_rst_cken
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
adc_srst_req	[16]	RW	adc 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
crc_cken/adc_rst_cken	[0]	-	crc 时钟门控，adc 复位门控： 0：关断； 1：打开。	0x1

4.5.30 PERI_CRG50

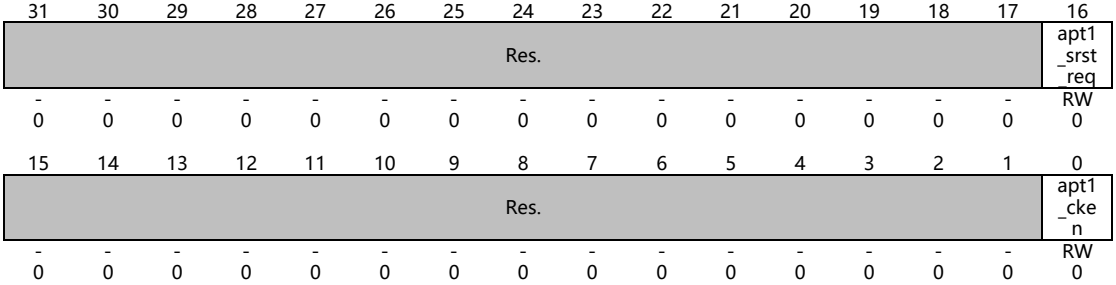
◆ **偏移地址：0xC8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															apt0_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															apt0_cken
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
apt0_srst_req	[16]	RW	apt0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
apt0_cken	[0]	RW	apt0 时钟门控： 0：关断； 1：打开。	0x0

4.5.31 PERI_CRG51

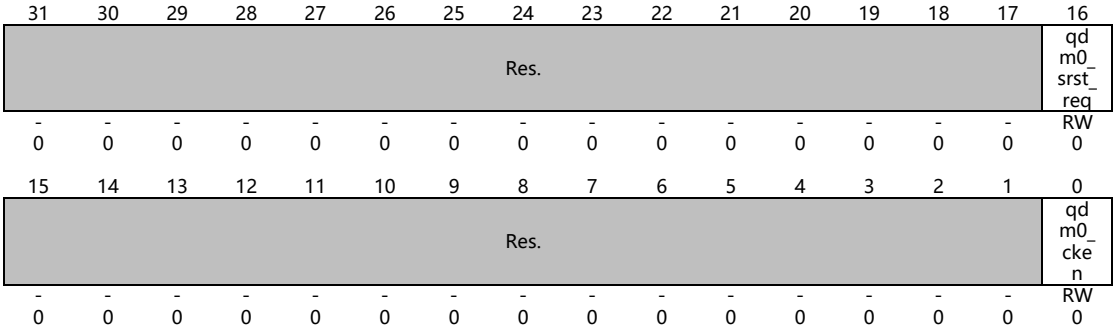
◆ **偏移地址：0xCC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
apt1_srst_req	[16]	RW	apt1 软复位请求: 0：不复位; 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
apt1_cken	[0]	RW	apt1 时钟门控: 0：关断; 1：打开。	0x0

4.5.32 PERI_CRG52

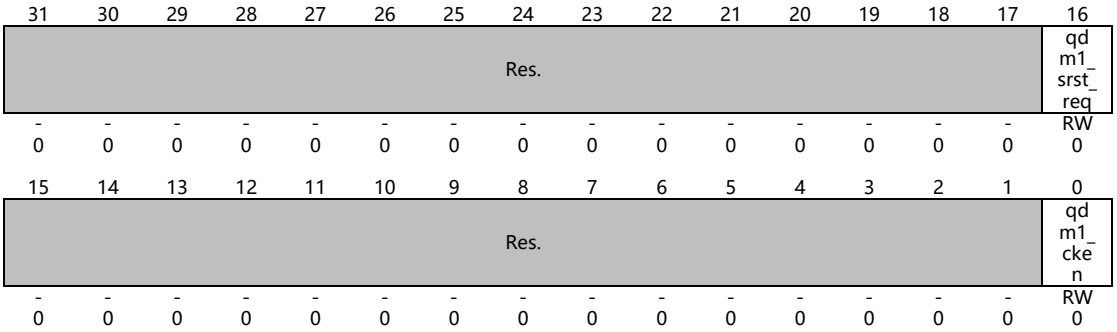
◆ **偏移地址：0xD0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
qdm0_srst_req	[16]	RW	qdm0 软复位请求: 0：不复位; 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
qdm0_cken	[0]	RW	qdm0 时钟门控: 0：关断; 1：打开。	0x0

4.5.33 PERI_CRG53

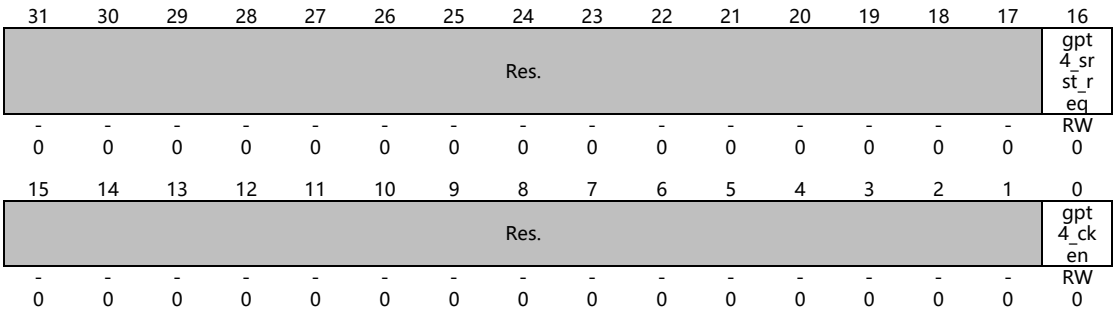
◆ **偏移地址：0xD4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
qdm1_srst_req	[16]	RW	qdm1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
qdm1_cken	[0]	RW	qdm1 时钟门控： 0：关断； 1：打开。	0x0

4.5.34 PERI_CRG54

◆ **偏移地址：0xD8　　复位值：0x00000000**

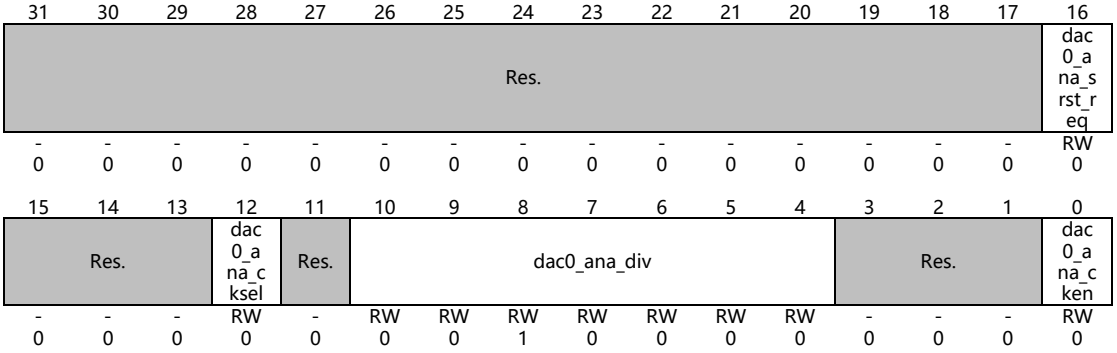


Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
gpt4_srst_req	[16]	RW	gpt4 软复位请求： 0：不复位； 1：复位。	0x0

Name	Bits	R/W	Description	Default
reserved	[15:1]	-	保留。	0x0
gpt4_cken	[0]	RW	gpt4 时钟门控： 0：关断； 1：打开。	0x0

4.5.35 PERI_CRG57

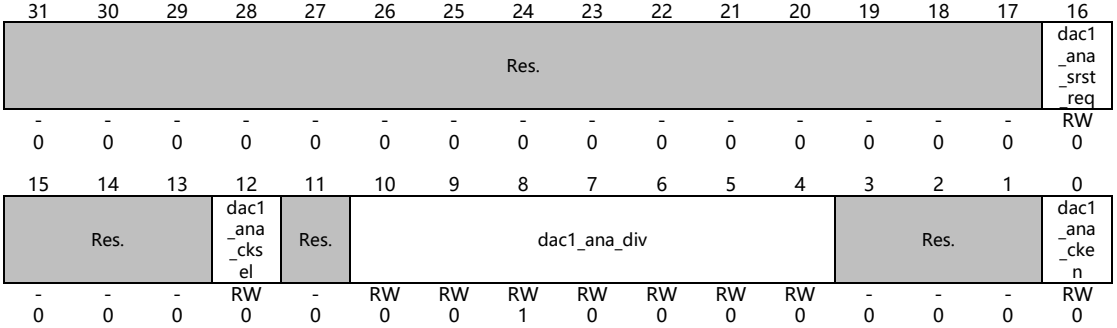
◆ **偏移地址：0xE4　　复位值：0x00000100**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
dac0_ana_srst_req	[16]	RW	dac0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:13]	-	保留。	0x0
dac0_ana_cksel	[12]	RW	dac0 时钟选择信号： 0：选择 clk_hsi 时钟； 1：选择 clk_hse 时钟。	0x0
reserved	[11]	-	保留。	0x0
dac0_ana_div	[10:4]	RW	dac0 时钟分频器分频比： 允许配置范围(1~127)。	0x10
reserved	[3:1]	-	保留。	0x0
dac0_ana_cken	[0]	RW	dac0 时钟门控： 0：关断； 1：打开。	0x0

4.5.36 PERI_CRG58

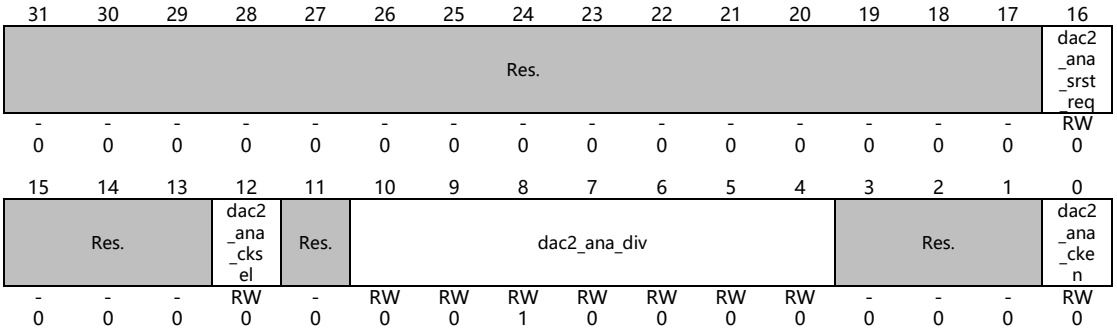
◆ **偏移地址：0xE8　　复位值：0x00000100**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
dac1_ana_srst_req	[16]	RW	dac1 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:13]	-	保留。	0x0
dac1_ana_cksel	[12]	RW	dac1 时钟选择信号： 0：选择 clk_hsi 时钟； 1：选择 clk_hse 时钟。	0x0
reserved	[11]	-	保留。	0x0
dac1_ana_div	[10:4]	RW	dac1 时钟分频器分频比： 允许配置范围(1~127)。	0x10
reserved	[3:1]	-	保留。	0x0
dac1_ana_cken	[0]	RW	dac1 时钟门控： 0：关断； 1：打开。	0x0

4.5.37 PERI_CRG59

◆ **偏移地址：0xEC　　复位值：0x00000100**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
dac2_ana_srst_req	[16]	RW	dac2 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:13]	-	保留。	0x0
dac2_ana_cksel	[12]	RW	dac2 时钟选择信号： 0：选择 clk_hsi 时钟； 1：选择 clk_hse 时钟。	0x0
reserved	[11]	-	保留。	0x0
dac2_ana_div	[10:4]	RW	dac2 时钟分频器分频比： 允许配置范围(1~127)。	0x10
reserved	[3:1]	-	保留。	0x0
dac2_ana_cken	[0]	RW	dac2 时钟门控： 0：关断； 1：打开。	0x0

4.5.38 PERI_CRG60

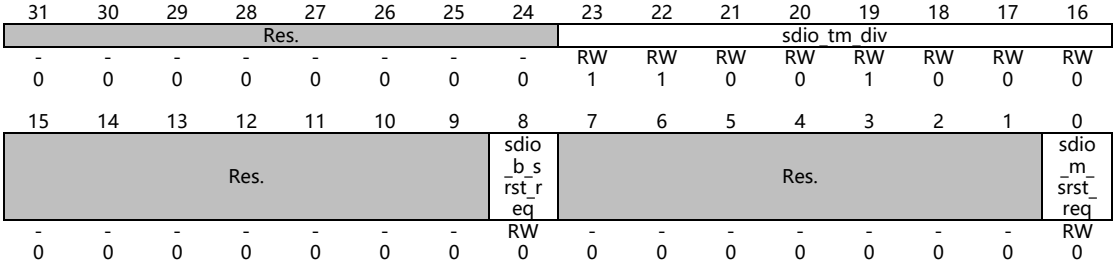
◆ **偏移地址：0xF0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															qspi_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															qspi_cken
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
qspi_srst_req	[16]	RW	qspi 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
qspi_cken	[0]	RW	qspi 时钟门控： 0：关断； 1：打开。	0x0

4.5.39 PERI_CRG61

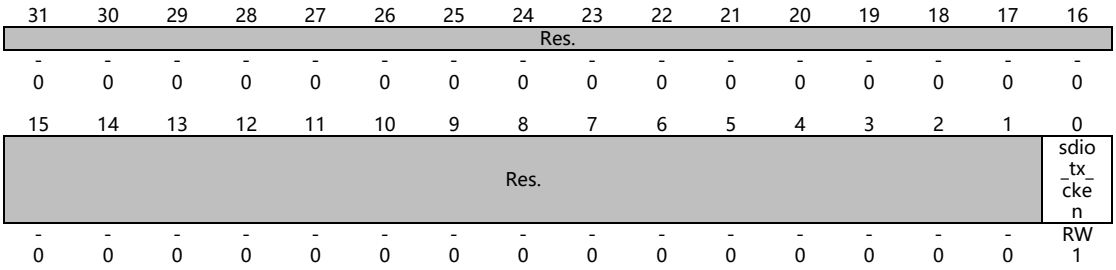
◆ **偏移地址：0xF4　　复位值：0x00C80000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
sdio_tm_div	[23:16]	RW	sdio timer 时钟分频器分频比： 允许配置范围(1~255)。	0xC8
reserved	[15:9]	-	保留。	0x0
sdio_b_srst_req	[8]	RW	sdio host 基本工作时钟软复位请求： 0：不复位； 1：复位。	0x0
reserved	[7:1]	-	保留。	0x0
sdio_m_srst_req	[0]	RW	sdio 内置 DMA 和 ahb master 软复位请求： 0：不复位； 1：复位。	0x0

4.5.40 PERI_CRG62

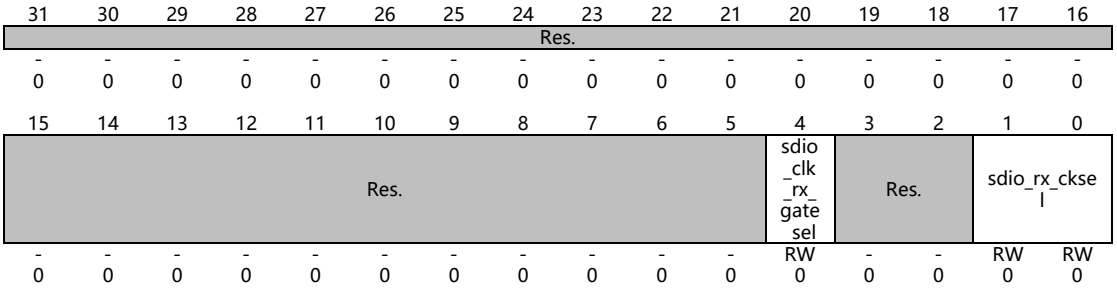
◆ **偏移地址：0xF8　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
sdio_tx_cken	[0]	RW	sdio tx 时钟门控： 0：关断； 1：打开。	0x1

4.5.41 PERI_CRG63

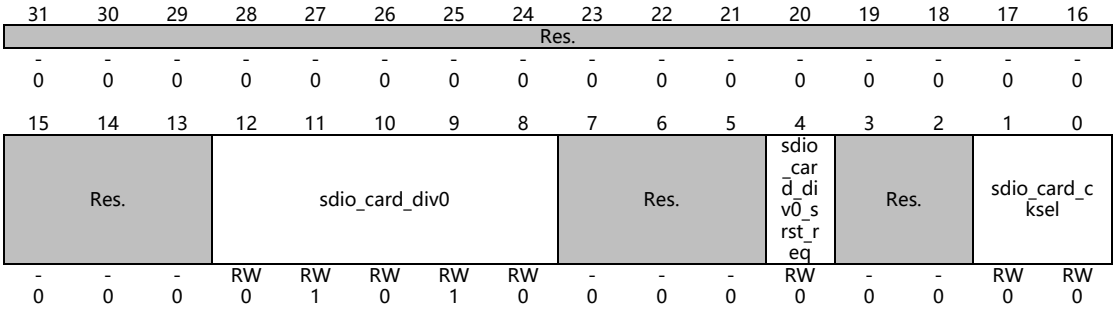
◆ **偏移地址：0xFC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
sdio_clk_rx_gate_sel	[4]	RW	sdio rx 时钟门控选择信号： 0：选择 clk2card_on 时钟； 1：选择 clk2card_on_d1 时钟。	0x0
reserved	[3:2]	-	保留。	0x0
sdio_rx_cksel	[1:0]	RW	sdio rx 时钟选择信号： 0：选择 clk_sdio_card 0 度相位时钟； 1：选择 clk_sdio_card 90 度相位时钟； 2：选择 clk_sdio_card 180 相位时钟； 3：选择 clk_sdio_card 270 相位时钟。	0x0

4.5.42 PERI_CRG64

◆ **偏移地址：0x100　　复位值：0x00000A00**



Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
sdio_card_div0	[12:8]	RW	sdio card 时钟分频器分频比： 允许配置范围(1~31)。	0xA

Name	Bits	R/W	Description	Default
reserved	[7:5]	-	保留。	0x0
sdio_card_div0_srst_req	[4]	RW	sdio card 时钟软复位请求： 0：不复位； 1：复位。	0x0
reserved	[3:2]	-	保留。	0x0
sdio_card_cksel	[1:0]	RW	sdio card 时钟选择信号： 0：选择 clk_hsi 时钟； 1：选择 clk_vco 时钟； 2：选择 clk_hse 时钟。	0x0

4.5.43 PERI_CRG65

◆ **偏移地址：0x104　　复位值：0x00000000**

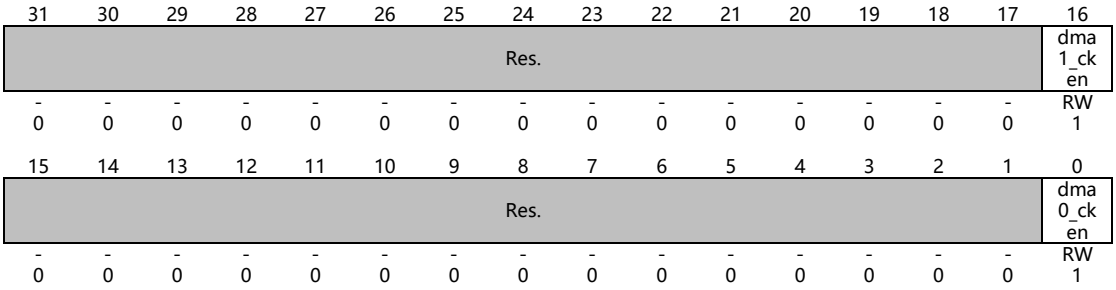
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															usb_ctrl_phy_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							usb_phy_srst_req	Res.			usb_srst_req	Res.			usb_cken
-	-	-	-	-	-	-	RW	-	-	-	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
usb_ctrl_phy_srst_req	[16]	RW	usb_ctrl_phy 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:9]	-	保留。	0x0
usb_phy_srst_req	[8]	RW	usb phy 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[7:5]	-	保留。	0x0
usb_srst_req	[4]	RW	usb 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[3:1]	-	保留。	0x0
usb_cken	[0]	RW	usb 时钟门控： 0：关断； 1：打开。	0x0

4.5.44 PERI_CRG66

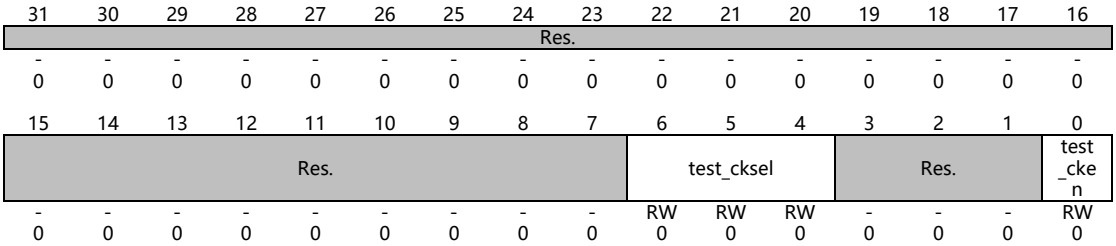
◆ **偏移地址：0x108　　复位值：0x00010001**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
dma1_cken	[16]	RW	dma1 时钟门控： 0：关断； 1：打开。	0x1
reserved	[15:1]	-	保留。	0x0
dma0_cken	[0]	RW	dma0 时钟门控： 0：关断； 1：打开。	0x1

4.5.45 PERI_CRG67

◆ **偏移地址：0x10C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留。	0x0
test_cksel	[6:4]	RW	test 时钟选择信号： 0：选择 clk_pst_div8 时钟； 1：选择 clk_hsi 时钟； 2：选择 clk_hse 时钟； 4：选择 clk_lsi 时钟； 5：选择 clk_lse 时钟；	0x0

Name	Bits	R/W	Description	Default
			6：选择 clk_dac0 时钟。	
reserved	[3:1]	-	保留。	0x0
test_cken	[0]	RW	test 时钟门控： 0：关断； 1：打开。	0x0

4.5.46 PERI_CRG68

◆ **偏移地址：0x110　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															tcxo_ready
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
tcxo_ready	[0]	RO	外部晶振时钟 clk_hse 稳定信号： 0：clk_hse 未稳定； 1：clk_hse 稳定。	0x0

4.5.47 PERI_CRG69

◆ **偏移地址：0x114　　复位值：0x00000400**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															aon_cfg_srst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

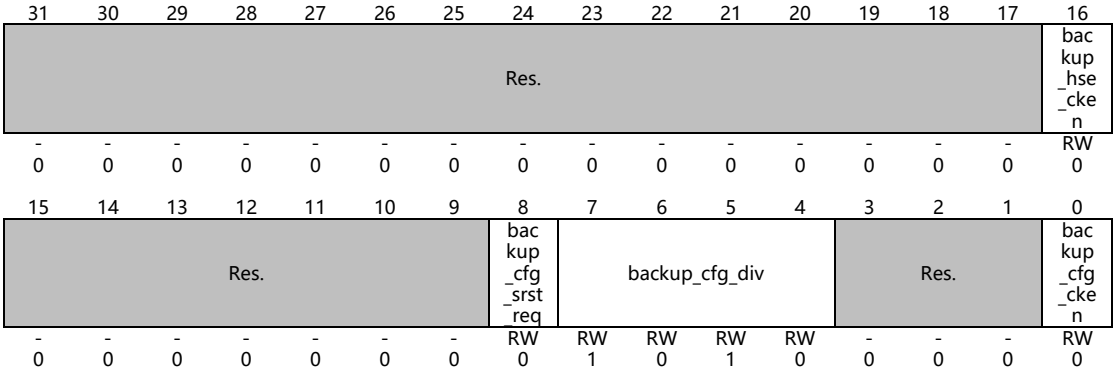
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					aon_cfg_div			Res.							aon_cfg_cken
-	-	-	-	-	RW	RW	RW	-	-	-	-	-	-	-	RW
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
aon_cfg_srst_req	[16]	RW	aon 软复位请求： 0：不复位；	0x0

Name	Bits	R/W	Description	Default
			1：复位。	
reserved	[15:11]	-	保留。	0x0
aon_cfg_div	[10:8]	RW	aon 配置时钟分频器分频比： 允许配置范围(1~7)。	0x4
reserved	[7:1]	-	保留。	0x0
aon_cfg_cken	[0]	RW	aon 配置时钟门控： 0：关断； 1：打开。	0x0

4.5.48 PERI_CRG70

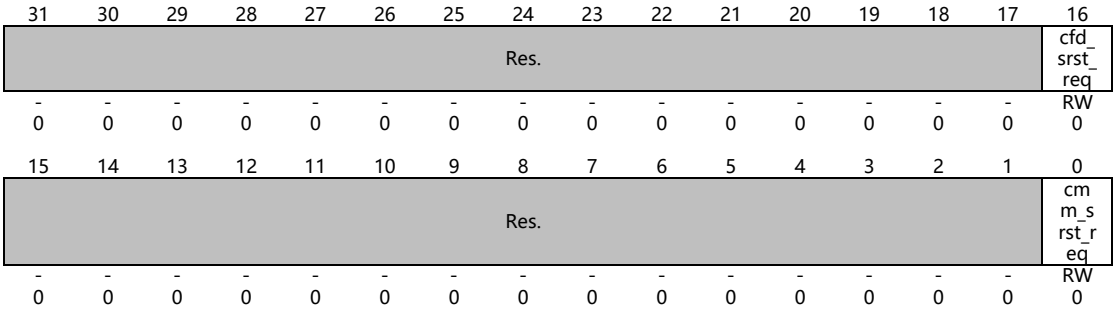
◆ **偏移地址：0x118　　复位值：0x000000A0**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
backup_hse_cken	[16]	RW	backup hse 配置时钟门控： 0：关断； 1：打开。	0x0
reserved	[15:9]	-	保留。	0x0
backup_cfg_srst_req	[8]	RW	backup 软复位请求： 0：不复位； 1：复位。	0x0
backup_cfg_div	[7:4]	RW	backup 配置时钟分频器分频比： 允许配置范围(1~15)。	0xA
reserved	[3:1]	-	保留。	0x0
backup_cfg_cken	[0]	RW	backup 配置时钟门控： 0：关断； 1：打开。	0x0

4.5.49 PERI_CRG71

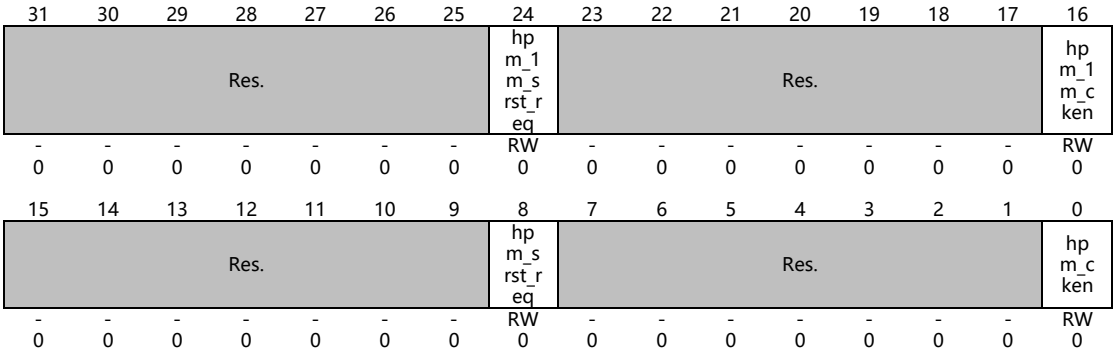
◆ **偏移地址：0x11C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
cfd_srst_req	[16]	RW	cfd 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:1]	-	保留。	0x0
cmm_srst_req	[0]	RW	cmm 软复位请求： 0：不复位； 1：复位。	0x0

4.5.50 PERI_CRG72

◆ **偏移地址：0x120　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
hpm_1m_srst_req	[24]	RW	hpm 1m 软复位请求： 0：不复位； 1：复位。	0x0

Name	Bits	R/W	Description	Default
reserved	[23:17]	-	保留。	0x0
hpm_1m_cken	[16]	RW	hpm 1m 配置时钟门控： 0：关断； 1：打开。	0x0
reserved	[15:9]	-	保留。	0x0
hpm_srst_req	[8]	RW	hpm 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[7:1]	-	保留。	0x0
hpm_cken	[0]	RW	hpm 配置时钟门控： 0：关断； 1：打开。	0x0

4.5.51 PERI_CRG73

◆ **偏移地址：0x124　　复位值：0x00004E82**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xo_res_tm		xo_cap_tm		xo_buf_res_tm		xo_drvmux_tm		xo_drive_tm				xo_ex_en	xo_force_rdy_flag_en	xo_rdy_en	xo_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	1	0	0	1	1	1	0	1	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
xo_res_tm	[15:14]	RW	xo16M 限制 drive level 的电阻 trim 控制： 0：限流电阻为 0； 1：限流电阻为 1K； 2：限流电阻为 2K； 3：限流电阻为 3K。	0x1
xo_cap_tm	[13:12]	RW	xo16M 片内 cap 容值控制： 0：Cap 值最小； 3：Cap 值最大。	0x0
xo_buf_res_tm	[11:10]	RW	xo16M buffer 反馈 res 控制： 0：反馈 Res=30K； 1：反馈 Res=90K； 2：反馈 Res=210K； 3：反馈 Res=270K。	0x3

Name	Bits	R/W	Description	Default
xo_drvmux_tm	[9:8]	RW	xo16M mux 驱动能力控制： 0：驱动能力设置为最低； 3：驱动能力设置为最高。	0x2
xo_drive_tm	[7:4]	RW	xo16M drive core 驱动能力控制： 0：驱动能力设置为最低； 15：驱动能力设置为最高。 注意：为了支持绝大多数晶体的使用场景，保证充足的起振裕度推荐使用默认档位，非必要不建议使用档位 1 和 2。	0x8
xo_ex_en	[3]	RW	xo16M 外灌使能控制信号，当 xo_en=1 时： 0：关闭外灌时钟通路； 1：使能外灌时钟通路，同时 xo 通路关闭。	0x0
xo_force_rdy_flag_en	[2]	RW	xo16M ready 信号的打拍时钟强制保持使能： 0：ready 信号的打拍时钟在 ready 信号给出后的几拍后关闭； 1：ready 信号的打拍时钟在 ready 信号给出后依然保持。	0x0
xo_rdy_en	[1]	RW	xo16M ready 信号输出使能： 0：ready 信号始终保持 0； 1：ready 信号在时钟稳定后输出 1。	0x1
xo_en	[0]	RW	xo16M 使能控制信号： 0：关闭 xo16M 电路； 1：使能 xo16M 电路。	0x0

4.5.52 PERI_CRG74

◆ **偏移地址：0x128　　复位值：0x00005E00**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xo_tbuf_div_sel		xo_res_thold_tm			xo_mos_thold_tm		xo_tbuf_input_en	xo_test_sel		hsi_ldo_vout_sel				hsi_ldo_op_lowbias	hsi_ldo_enb
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	1	0	1	1	1	1	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
xo_tbuf_div_sel	[15:14]	RW	xo16M testbuffer 内部除率选择： 0：div1； 1：div2；	0x1

Name	Bits	R/W	Description	Default
			2：div4； 3：div8。	
xo_res_thold_tm	[13:11]	RW	xo16M 内部比较器阈值 trim 电阻调节： 0：阈值电压设置为最低； 7：阈值电压设置为最高。	0x3
xo_mos_thold_tm	[10:9]	RW	xo16M 内部比较器迟滞电压调节： 0：迟滞电压设置为最低； 3：迟滞电压设置为最高。	0x3
xo_tbuf_input_en	[8]	RW	xo16Mtestbuffer 的输入 buffer 使能开关： 0：关闭 testbuffer 的输入 buffer； 1：使能 testbuffer 的输入 buffer。	0x0
xo_test_sel	[7:6]	RW	xo16M 中测试信号选择： 0：测试模块关闭； 1：xo16M 输出时钟测试； 2：ready 信号输出测试； 3：比较器阈值电压测试。	0x0
hsi_ldo_vout_sel	[5:2]	RW	HSI_LDO 的输出电压粗调： 0：1.8； 1：2.06； 2：1.99； 3：1.92 4：1.86； 5：1.74； 6：1.69； 7：1.64 8：1.59； 9：1.55； 10：2.3。 注意：除列出码字外其他码字无效。	0x0
hsi_ldo_op_lowbias	[1]	RW	HSI_LDO 的功耗模式调整： 0：高功耗； 1：低功耗。	0x0
hsi_ldo_enb	[0]	RW	HSI 的 LDO 使能控制信号，默认 0（上电自启，可配置关闭） 0：使能； 1：关闭。	0x0

4.5.53 PERI_CRG75

◆ **偏移地址：0x12C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		clk_ibias_enb	hsi_res_sel						hsi_ibias_sel		hsi_reserved	hsi_test_div		hsi_test_en	hsi_enb
-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
clk_ibias_enb	[13]	RW	XLDO/OSC16M/XO16M/PVD_VDDA 的偏置电流使能控制信号： 0：使能； 1：不使能。	0x0
hsi_res_sel	[12:7]	RW	HSI 的电阻选择控制信号： 0：无温度系数修正； 7：最大温度系数修正。	0x0
hsi_ibias_sel	[6:5]	RW	HSI 功耗模式选择控制信号 0：高功耗； 1：低功耗。	0x0
hsi_reserved	[4]	RW	保留。	0x0
hsi_test_div	[3:2]	RW	保留。	0x0
hsi_test_en	[1]	RW	HSI 测试使能控制信号： 0：关闭测试通路； 1：开启测试通路。	0x0
hsi_enb	[0]	RW	HSI 的使能控制信号（上电自启，可配置关闭）： 0：正常工作； 1：关闭。	0x0

5 时钟频偏检测（CMM）

5.1 CMM 简介

CMM（Clock Monitor Module）模块实现对芯片内关键时钟的频率检测，用于判断待检测时钟的频率是否在设置的有效频率范围内，支持参考时钟可选，支持待检测时钟可选。

5.2 CMM 主要特性

CMM 的主要功能特点有：

- 支持模块功能打开或者关闭。
- 支持参考时钟可选。
- 支持待检测时钟可选。
- 支持参考时钟与待检测时钟分频系数可配置。
- 支持频率检测上下限阈值可配置。
- 支持检测结果上报。
- 支持中断功能。

5.3 CMM 功能描述

参考时钟与待检测时钟在正常工作时两者保持固定的比例关系。考虑到参考时钟和待检测时钟两者各自引入的偏差，两者间的比例关系应该保持在一定的阈值门限内。因此通过对参考时钟或待检测时钟进行分频，让两者满足一定比例关系，然后设置合理的上下门限即可判断待检测时钟是否有频偏。

具体实现为使用参考时钟对待检测时钟计数，在每个参考时钟的上沿对计数器清零并重新开始计数，计数值保存到 cnt_det 寄存器中，同时将计数值与上下门限进行比较判断，完

成一次检测，如此循环往复。

根据实际需要选择待检测时钟和参考时钟，下面通过实际的例子讲解，这里参考时钟选择低频时钟 CLK_LSI（32KHz），待检测时钟选择为高频时钟 CLK_HSI（16MHz）。按照检测方法，默认低频时钟 CLK_LSI 最小做 2 分频，此处设置为 2 分频，则两者之间的比例关系为 CLK_HSI/(CLK_LSI/2)=500*2=1000，以 1%精度考虑（此处需要考虑参考时钟和待检测时钟两者的各自引入的时钟偏差），检测上限设置为 1000*1.01=1010，检测下限 1000*0.99=990。

当待检测时钟的频率为 16.17599M 时，cnt_det 计数值为 1011，大于上门限，因此上报时钟监测错误状态，以及上报时钟监测错误中断。

CMM 有两种中断状态，分别为时钟监测错误中断，时钟监测正确中断。

CMM 支持中断注入功能。

5.4 CMM 工作方式

5.4.1 频偏监测流程

以功能描述示例为例，具体步骤如下：

步骤 1 配置 CMM_CFG1.clk_div_sel=0，将参考时钟分频，使用待检测时钟计数。

步骤 2 配置 CMM_CFG1.clk_tgt_sel=1，选择待检测时钟为 CLK_HSI 。

步骤 3 配置 CMM_CFG1.clk_ref_sel=0，选择参考时钟为 CLK_LSI。

步骤 4 配置 CMM_CFG1.set_div=2，配置分频系数为 2。

步骤 5 配置 CMM_CFG2.set_cnt_up=1010，配置检测上限。

步骤 6 配置 CMM_CFG2.set_cnt_dn=990，配置检测下限。

步骤 7 打开中断使能 CMM_INT_EN.cmm_err_int_en 与 CMM_INT_EN.cmm_ok_int_en（可根据需求，单独打开或关闭）。

步骤 8 打开模块使能 CMM_EN=1，启动 CMM 模块。

⚠注意

时钟选择采用有毛刺时钟切换，在模块使能打开前时钟已经成功切换，但第一个时钟周期占空比可能偏小，但不影响后续检测结果。

CMM 的两个中断事件，cmm_err_int 代表的是检测到时钟状态从 ok 变为 error，cmm_ok_int 代表的是检测到时钟状态从 error 变为 ok。

每一次重新将 CMM_EN.cmm_en 从 0 设置为 1 时，CMM 模块的时钟状态初始化设定为 error。

因此，每一次重新将 CMM_EN.CMM_en 从 0 设置为 1 时，如果 CMM 模块检测到 cmm_error 事件，也不会上报 cmm_err_int 中断，因为时钟状态仍然保持在 error 并未发生变化，此时可以通过查询 CMM_STS 来判断当前的检测状态。

与之对应的，每一次重新将 CMM_EN.CMM_en 从 0 设置为 1 时，如果 CMM 模块检测到 cmm_ok 事件，因为时钟状态从 error 变为 ok，则会上报 cmm_ok_int 中断，也可以通过查询 CMM_STS 来判断当前的检测状态。

在后续的检测过程中，只要是时钟状态发生变化，包含从 ok 变为 error，或者从 error 变为 ok，都可以正常上报一次中断。

5.4.2 中断处理流程

步骤 1 关闭中断使能。

步骤 2 读取原始中断。

步骤 3 读取状态寄存器，查看检测结果和检测计数。

步骤 4 清除原始中断。

说明

中断注入功能：通过软件配置发起中断，用于测试模块的中断是否能够正常上报至 CPU。

5.5 CMM 寄存器概览

CMM 的基地址如下：

- CMM 寄存器基地址为：0x60080000

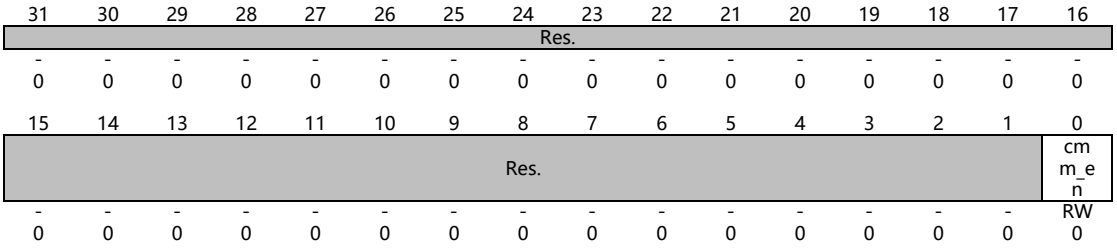
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CMM_EN	0x0	32	RW	0x00000000	CMM 使能寄存器
2	CMM_CFG1	0x4	32	RW	0x00101000	CMM 配置寄存器 1
3	CMM_CFG2	0x8	32	RW	0x80004000	CMM 配置寄存器 2
4	CMM_STS	0xC	32	RO	0x00000000	CMM 状态寄存器
5	CMM_INT_EN	0x10	32	RW	0x00000000	CMM 中断使能寄存器
6	CMM_INT_STS	0x14	32	RO	0x00000000	CMM 中断状态寄存器
7	CMM_INT_RAW	0x18	32	RO	0x00000000	CMM 原始中断寄存器
8	CMM_INT_INJ	0x1C	32	W1P	0x00000000	CMM 中断注入寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
9	CMM_INT_CLR	0x20	32	W1P	0x00000000	CMM 中断清除寄存器

5.6 CMM 寄存器描述

5.6.1 CMM_EN

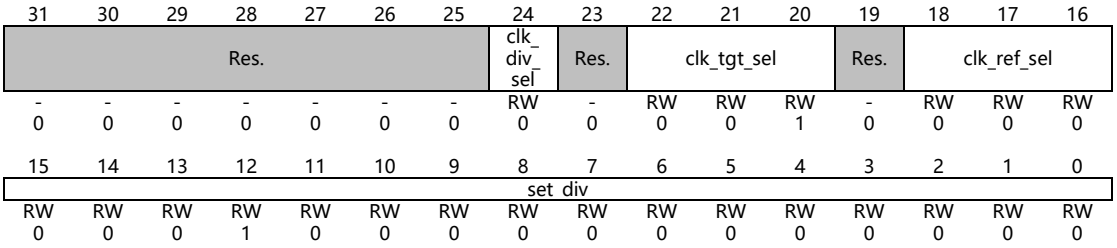
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
cmm_en	[0]	RW	CMM 使能信号 0：CMM 不使能 1：CMM 使能	0x0

5.6.2 CMM_CFG1

◆ **偏移地址：0x4　　复位值：0x00101000**



Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留	0x0
clk_div_sel	[24]	RW	分频时钟选择 0：将参考时钟分频，使用待检测时钟计数	0x0

Name	Bits	R/W	Description	Default
			1：将待检测时钟分频，使用参考时钟计数	
reserved	[23]	-	保留	0x0
clk_tgt_sel	[22:20]	RW	待检测时钟选择信号 0：CLK_HS 1：CLK_HSI 2：CLK_HSE 3：CLK_PST_DIV8 4：CLK_LS 5：CLK_LSI 6：CLK_LSE 7：CLK_HS	0x1
reserved	[19]	-	保留	0x0
clk_ref_sel	[18:16]	RW	参考时钟选择信号 0：CLK_LSI 1：CLK_HSI 2：CLK_HSE 3：CLK_LSE 4：CLK_PST_DIV8 5~7：CLK_LSI	0x0
set_div	[15:0]	RW	待检测时钟分频系数配置，要求配置为大于等于 2 的偶数，奇数的最低比特会被忽略，配置为 0 或 1 分频时，检测会失败	0x1000

5.6.3 CMM_CFG2

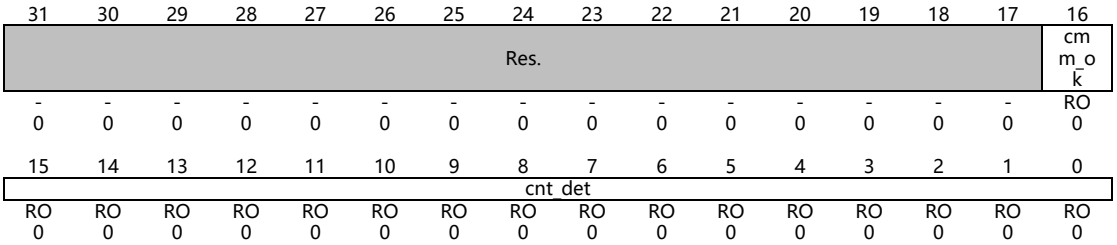
◆ **偏移地址：0x8　　复位值：0x80004000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
set_cnt_up															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
set_cnt_dn															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
set_cnt_up	[31:16]	RW	待检测时钟计数上限	0x8000
set_cnt_dn	[15:0]	RW	待检测时钟计数下限	0x4000

5.6.4 CMM_STS

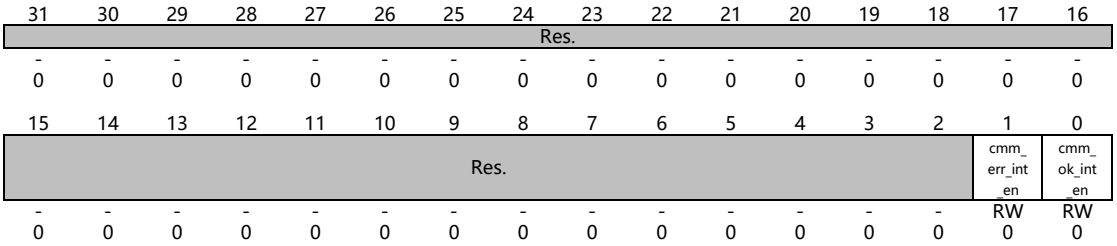
◆ **偏移地址：0xC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留	0x0
cmm_ok	[16]	RO	时钟监测结果 0：时钟失效 1：时钟正常	0x0
cnt_det	[15:0]	RO	时钟检测计数取值	0x0

5.6.5 CMM_INT_EN

◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
cmm_err_int_en	[1]	RW	时钟监测错误中断使能 0：不使能 1：使能	0x0
cmm_ok_int_en	[0]	RW	时钟监测正确中断使能 0：不使能 1：使能	0x0

5.6.6 CMM_INT_STS

◆ **偏移地址：0x14　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														cmm_err_int_sts	cmm_ok_int_sts
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
cmm_err_int_sts	[1]	RO	时钟监测错误中断状态 0：无中断上报 1：有中断上报	0x0
cmm_ok_int_sts	[0]	RO	时钟监测正确中断状态 0：无中断上报 1：有中断上报	0x0

5.6.7 CMM_INT_RAW

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														cmm_err_int_raw	cmm_ok_int_raw
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
cmm_err_int_raw	[1]	RO	时钟监测错误原始中断 0：无原始中断 1：有原始中断	0x0
cmm_ok_int_raw	[0]	RO	时钟监测正确原始中断 0：无原始中断	0x0

Name	Bits	R/W	Description	Default
			1：有原始中断	

5.6.8 CMM_INT_INJ

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														cmm_err_int_inj	cmm_ok_int_inj
-	-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
cmm_err_int_inj	[1]	W1P	时钟监测错误中断注入 0：无中断注入 1：有中断注入	0x0
cmm_ok_int_inj	[0]	W1P	时钟监测正确中断注入 0：无中断注入 1：有中断注入	0x0

5.6.9 CMM_INT_CLR

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														cmm_err_int_clr	cmm_ok_int_clr
-	-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0

Name	Bits	R/W	Description	Default
cmm_err_int_clr	[1]	W1P	时钟监测错误中断清除 0：无中断清除 1：执行中断清除	0x0
cmm_ok_int_clr	[0]	W1P	时钟监测正确中断清除 0：无中断清除 1：执行中断清除	0x0

6 时钟失效检测（CFD）

6.1 CFD 简介

CFD（Clock Failure Detector）模块实现对芯片内关键时钟的失效检测，用于判断待检测时钟在设置的参考时钟溢出计数时间内是否有翻转，支持待检测时钟可选。

6.2 CFD 主要特性

- 支持模块功能打开或者关闭。
- 支持待检测时钟可选。
- 支持待检测时钟分频系数可配置。
- 支持检测超时窗口可配置。
- 支持检测到失效后产生系统事件送到 APT。
- 支持检测到失效后产生系统事件送到时钟切换模块。
- 支持检测结果上报。
- 支持中断功能。

6.3 CFD 功能描述

CFD 检测时钟失效方法：对参考时钟（即系统选择的 32KHz 时钟，可以是 CLK_LSI 或 CLK_LSE）进行计数，计数器被待检测时钟（分频后的时钟）的上升沿及下降沿清零。若待检测时钟失效，则计数器不会清零且持续累加，超过设定的阈值后，上报时钟失效错误及中断标志。

下面通过实际的例子讲解，参考时钟使用 CLK_LSI（32KHz），待检测时钟根据实际需要选择，这里以 CLK_HSI（16MHz）为例进行分析。

计算待检测时钟分频系数：按照现有检测方法，需首先对待检测时钟 CLK_HSI（16MHz）进行分频，得到分频后时钟 CLK_TGT_DIV。使用 CLK_LSI（32KHz）对 CLK_TGT_DIV 计数，要求 CLK_LSI（32KHz）的时钟频率大于等于 CLK_TGT_DIV 的 2 倍（根据采样定律），小于 CLK_TGT_DIV 的 255 倍（根据检测周期配置上限），建议配置为 4 倍。也就是 CLK_TGT_DIV=CLK_LSI/4=32K/4=8KHz。因此待检测时钟分频系数配置为 CLK_HSI/CLK_TGT_DIV=16M/8K=2000。

配置参考时钟计数溢出值上限：要求大于 CLK_LSI/CLK_TGT_DIV，即大于 4（32K/8K），考虑双方时钟引入的偏差，建议配置为 16。注意，如果该配置不满足以上约束，可能会导致频繁交替上报时钟监测错误中断 clock_fail_int 和时钟检测正确中断 clock_pass_int。

当 CLK_HSI（16MHz）失效时，其分频时钟的翻转，没有在参考时钟计数溢出时间内被检测到，导致 CLK_LSI（32KHz）的计数器持续累加，当计数值大于等于设定的阈值后，产生错误上报及中断，且计数器重新清零累加。

6.4 CFD 工作方式

6.4.1 失效监测流程

以功能描述示例为例，具体步骤如下：

步骤 1 配置待检测时钟选择 CFD_CFG.clk_tgt_sel=1，选择 CLK_HSI（16MHz）。

步骤 2 配置待检测时钟分频系数 CFD_CFG.set_tgt_div=2000，是 CLK_LSI（32KHz）时钟频率是 CLK_TGT_DIV 的 4 倍。

步骤 3 配置参考时钟溢出配置上限 CFD_CFG.set_ref_cnt=16，将计数超时门限配置为 CLK_TGT_DIV 的 16 倍。

步骤 4 配置 CFD_CFG.clock_fail_crg_en，根据应用需要选择是否打开检测结果控制自动时钟切换。

步骤 5 配置 CFD_CFG.clock_fail_apt_en，根据应用需要选择是否打开检测结果控制 APT 系统保护。

步骤 6 打开中断使能 CFD_INT_EN=3，默认 clock_fail_int 和 clock_pass_int 的中断使能都开启。

步骤 7 打开模块使能 CFD_EN.cfd_en=1，启动 CFD 模块。

⚠注意

时钟选择采用有毛刺时钟切换，在模块使能打开前时钟已经成功切换，但第一个时钟周期占空比可能偏小，但不影响后续检测结果。

CFD 有两个中断事件，其中 clock_fail_int 中断代表的是检测到时钟状态从 pass 变为 fail，clock_pass_int 中断代表的是检测到时钟状态从 fail 变为 pass。

每一次重新将 CFD_EN.cfd_en 从 0 设置 1 时，CFD 模块的时钟状态初始化设定为 pass。

因此，每一次重新将 CFD_EN.cfd_en 从 0 设置为 1 时，如果 CFD 模块检测到 clock_pass 事件，也不会上报 clock_pass_int 中断，因为时钟状态仍然保持在 pass 并未发生变化，此时可以通过查询 CFD_STS 来判断当前的检测状态。

与之对应的，每一次重新将 CFD_EN.cfd_en 从 0 设置为 1 时，如果 CFD 模块检测到 clock_fail 事件，因为时钟状态从 pass 变为 fail，则会上报 clock_fail_int 中断，也可以通过查询 CFD_STS 来判断当前的检测状态。

在后续的检测过程中，只要是时钟状态发生变化，包含从 pass 变为 fail，或者从 fail 变为 pass，都可以正常上报一次中断。

6.4.2 中断处理流程

步骤 1 关闭中断使能。

步骤 2 读取原始中断。

步骤 3 清除原始中断。

说明

中断注入功能：通过软件配置发起中断，用于测试模块的中断是否能够正常上报至 CPU。

6.5 CFD 寄存器概览

CFD 的基地址如下。

- CFD 寄存器基地址为：0x60081000

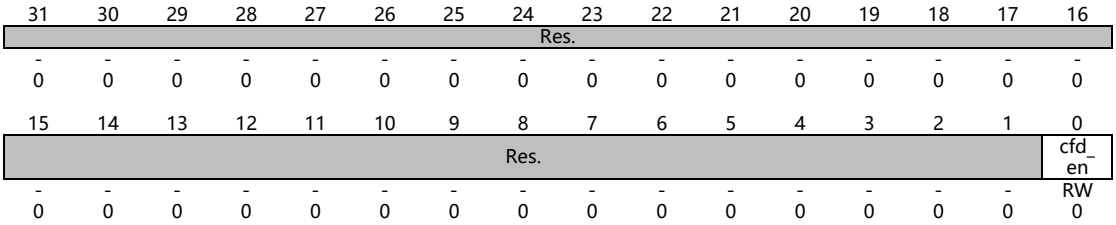
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CFD_EN	0x0	32	RW	0x00000000	CFD 使能寄存器
2	CFD_CFG	0x4	32	RW	0x031F1000	CFD 配置寄存器
3	CFD_STS	0x8	32	RO	0x00000000	CFD 状态寄存器
4	CFD_INT_EN	0x10	32	RW	0x00000000	CFD 中断使能寄存器
5	CFD_INT_STS	0x14	32	RO	0x00000000	CFD 中断状态寄存器
6	CFD_INT_RAW	0x18	32	RO	0x00000000	CFD 原始中断寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
7	CFD_INT_INJ	0x1C	32	W1P	0x00000000	CFD 中断注入寄存器
8	CFD_INT_CLR	0x20	32	W1P	0x00000000	CFD 中断清除寄存器

6.6 CFD 寄存器描述

6.6.1 CFD_EN

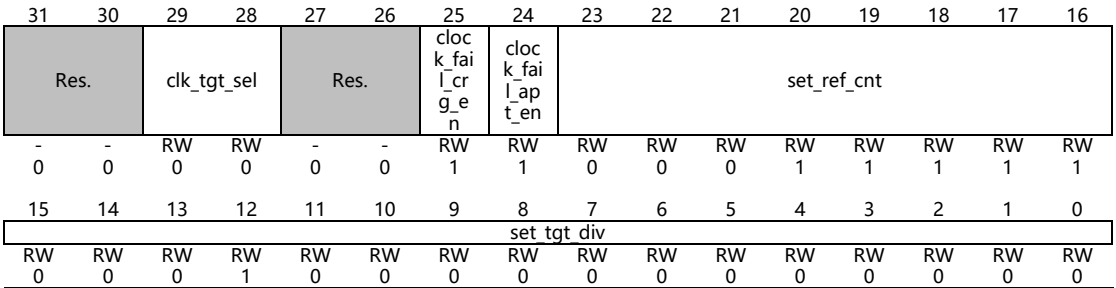
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
cfd_en	[0]	RW	CFD 使能信号 0：CFD 不使能 1：CFD 使能	0x0

6.6.2 CFD_CFG

◆ **偏移地址：0x4　　复位值：0x031F1000**



Name	Bits	R/W	Description	Default
reserved	[31:30]	-	保留	0x0
clk_tgt_sel	[29:28]	RW	待检测时钟选择信号	0x0

Name	Bits	R/W	Description	Default
			0：CLK_HS 1：CLK_HSI 2：CLK_HSE 3：CLK_PST_DIV8	
reserved	[27:26]	-	保留	0x0
clock_fail_crg_en	[25]	RW	时钟失效应用于 CRG 控制自动时钟切换 0：不使能 1：使能	0x1
clock_fail_apt_en	[24]	RW	时钟失效应用于 APT 自动保护 0：不使能 1：使能	0x1
set_ref_cnt	[23:16]	RW	参考时钟计数溢出配置	0x1F
set_tgt_div	[15:0]	RW	待检测时钟分频系数配置，要求配置为大于等于 2 的偶数，奇数的最低比特会被忽略，配置为 0 或 1 分频时，检测会失败	0x1000

6.6.3 CFD_STS

◆ **偏移地址：0x8　　复位值：0x00000000**

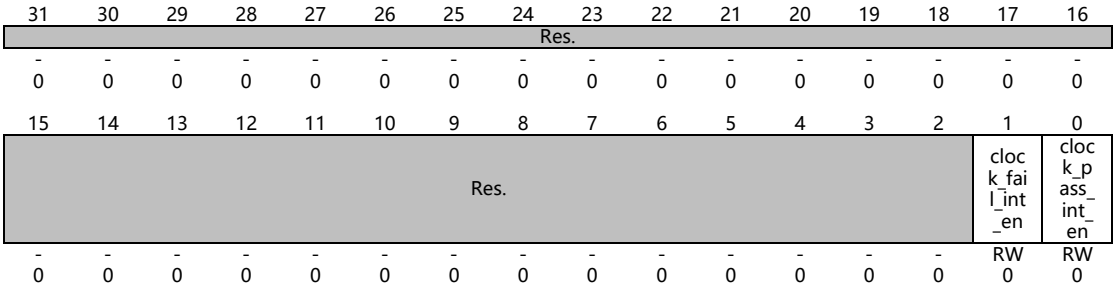
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															clock_fail
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
clock_fail	[0]	RO	时钟监测结果 0：时钟正常 1：时钟失效	0x0

6.6.4 CFD_INT_EN

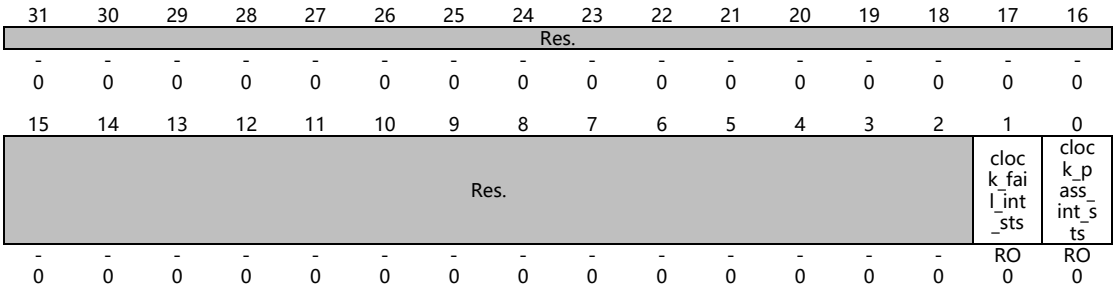
◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
clock_fail_int_en	[1]	RW	时钟监测错误中断使能 0：不使能 1：使能	0x0
clock_pass_int_en	[0]	RW	时钟监测正确中断使能 0：不使能 1：使能	0x0

6.6.5 CFD_INT_STS

◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
clock_fail_int_sts	[1]	RO	时钟监测错误中断状态 0：无中断上报 1：有中断上报	0x0
clock_pass_int_sts	[0]	RO	时钟监测正确中断状态 0：无中断上报 1：有中断上报	0x0

6.6.6 CFD_INT_RAW

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														clock_fail_int_raw	clock_pass_int_raw
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
clock_fail_int_raw	[1]	RO	时钟监测错误原始中断 0：无原始中断 1：有原始中断	0x0
clock_pass_int_raw	[0]	RO	时钟监测正确原始中断 0：无原始中断 1：有原始中断	0x0

6.6.7 CFD_INT_INJ

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														clock_fail_int_inj	clock_pass_int_inj
-	-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
clock_fail_int_inj	[1]	W1P	时钟监测错误中断注入 0：无中断注入 1：有中断注入	0x0

Name	Bits	R/W	Description	Default
clock_pass_int_inj	[0]	W1P	时钟监测正确中断注入 0：无中断注入 1：有中断注入	0x0

6.6.8 CFD_INT_CLR

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														clock_fail_int_clr	clock_pass_int_clr
-	-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
clock_fail_int_clr	[1]	W1P	时钟监测错误中断清除 0：无中断清除 1：执行中断清除	0x0
clock_pass_int_clr	[0]	W1P	时钟监测正确中断清除 0：无中断清除 1：执行中断清除	0x0

7 通用 I/O（GPIO）

7.1 GPIO 简介

芯片支持 6 组 GPIO（General Purpose Input/Output）控制器，即 GPIOA ~ GPIOF。除最后一组只有 6 个可编程的 GPIO 外，其余每组 GPIO 控制器提供 16 个可编程的 GPIO。

每个 GPIO 可以独立配置为输入或者输出。这些 GPIO 用于生成特定应用的输出信号或采集特定应用的输入信号。作为输入 GPIO 时，可作为中断源；作为输出 GPIO 时，每个 GPIO 都可以独立地清 0 或置 1。

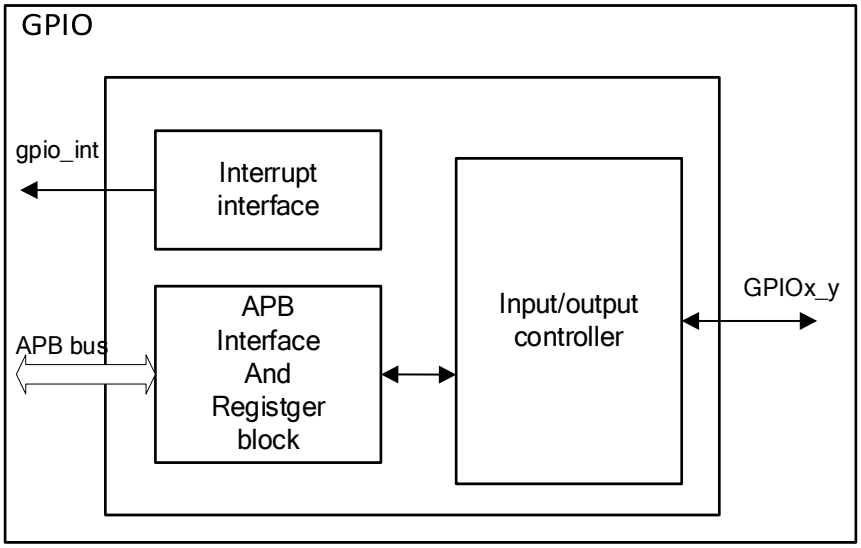
GPIO 控制器可以根据电平或跳变值产生可屏蔽的中断。GPIO_RIS（General Purpose Input/Output Interrupt State）信号给中断控制器一个指示，表示有中断发生。

7.2 GPIO 主要特性

- 每个 GPIO 均可独立配置为输入、输出。
- 作为输入 GPIO 时，可作为中断源。中断触发类型可以灵活选择。
- 作为输出 GPIO 时，每个 GPIO 都可以独立地清 0 或置 1，且能独立查询。
- 支持中断屏蔽。

7.3 GPIO 功能描述

图 7-1 GPIO 模块框图



GPIO 控制器通过总线配置寄存器的值，以控制 GPIO 为输入/输出，当作为输入时可以产生可屏蔽的中断信号。GPIO 的逻辑框图如图 7-1 所示。

GPIOx_y 是 GPIO 控制器的输入/输出信号，x 取 A~F，对应芯片支持的 6 组 GPIO 控制器。y 取 0~15，代表每组 GPIO 控制器提供的 16 个可编程 GPIO(最后一组实际只有 6 个可用)。

可以通过配置 GPIO_DIR 对每个 GPIO 单独配置为输入/输出：

- 当 GPIO 作为输出时，可以通过 GPIO_DATA 对每个 GPIO 单独清 0 或置 1。
- 当 GPIO 作为输入时，可以通过 GPIO_DATA 读取相应 GPIO 的输入值。

可以通过配置 DATA_EN 设置写 GPIO_DATA 的生效 bit 位置。

中断功能

GPIO 控制器的中断由 GPIO_IS、GPIO_IBE、GPIO_IEV、GPIO_IE、GPIO_RIS、GPIO_MIS、GPIO_IC 寄存器控制，可以配置 GPIO 控制器的中断触发类型以及清除中断等。

GPIO_IS、GPIO_IBE、GPIO_IEV 三个寄存器共同决定了中断源的特性和中断触发类别。GPIO_IS 决定中断是边沿触发还是电平触发。当配置边沿触发时， GPIO_IBE 决定中断是单边沿还是双边沿触发。 GPIO_IEV 决定中断是下降沿/低电平触发还是上升沿/高电平触发。

可以配置 GPIO_IE 以屏蔽相应中断。可以通过读 GPIO_RIS 和 GPIO_MIS 来查询中断的原始状态和屏蔽后状态。 中断状态还支持通过 GPIO_IC 进行清除。

GPIO 控制器触发后产生可屏蔽的中断信号，中断信号送至中断控制器，表示 GPIO 控制器有中断产生，系统可以进入中断函数执行相应操作。

7.4 GPIO 寄存器概览

GPIO 实例 6 组，其基地址如下：

- GPIOA 寄存器基地址为：0x60050000
- GPIOB 寄存器基地址为：0x60051000
- GPIOC 寄存器基地址为：0x60052000
- GPIOD 寄存器基地址为：0x60053000
- GPIOE 寄存器基地址为：0x60054000
- GPIOF 寄存器基地址为：0x60055000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	GPIO_DATA	0x0	32	RW	0x00000000	GPIO 数据寄存器
2	GPIO_DIR	0x400	32	RW	0x00000000	GPIO 输入输出控制寄存器
3	GPIO_IS	0x404	32	RW	0x00000000	GPIO 中断模式配置寄存器
4	GPIO_IBE	0x408	32	RW	0x00000000	GPIOI 双边沿中断配置寄存器
5	GPIO_IEV	0x40C	32	RW	0x00000000	GPIO 单边沿或者电平中断配置寄存器
6	GPIO_IE	0x410	32	RW	0x00000000	GPIO 中断屏蔽寄存器
7	GPIO_RIS	0x414	32	RO	0x00000000	GPIO 原始中断状态寄存器
8	GPIO_MIS	0x418	32	RO	0x00000000	GPIO 中断状态寄存器
9	GPIO_IC	0x41C	32	W1P	0x00000000	GPIO 中断状态清除寄存器
10	DATA_EN	0x424	32	RW	0x00000000	GPIO 数据使能寄存器

7.5 GPIO 寄存器描述

7.5.1 GPIO_DATA

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpiodata															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpiodata	[15:0]	RW	GPIO 数据寄存器。该寄存器表示输出到 GPIO 管脚的值或者从 GPIO 管脚上输入的值，该寄存器只有在 DATA_EN 使能后才有效。16bit 每个 bit 相互独立。 0：输出的值是 0 或者输入的值是 0 1：输出的值是 1 或者输入的值是 1	0x0

7.5.2 GPIO_DIR

◆ **偏移地址：0x400　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpiodir															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpiodir	[15:0]	RW	GPIO 输入输出控制寄存器。16bit 每个 bit 相互独立。 0：输入 1：输出	0x0

7.5.3 GPIO_IS

◆ **偏移地址：0x404　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpiois															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpiois	[15:0]	RW	GPIO 中断模式配置寄存器。可配置为电平触发中断或者边沿触发中断。16bit 每个 bit 相互独立。 0：边沿触发中断 1：电平触发中断	0x0

7.5.4 GPIO_IBE

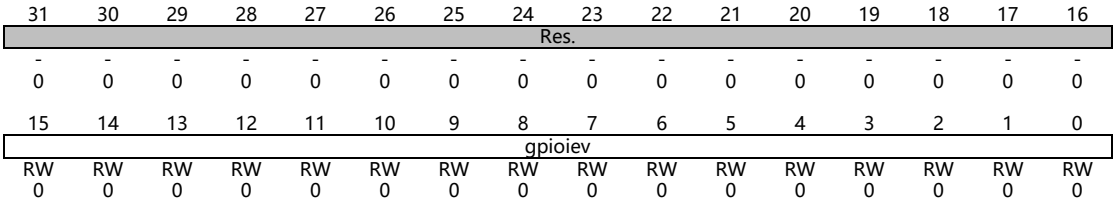
◆ **偏移地址：0x408　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpioibe															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpioibe	[15:0]	RW	GPIOI 双边沿中断配置寄存器。16bit 每个 bit 相互独立。 0：中断触发条件参考 GPIO_IEV 寄存器 1：双沿触发中断	0x0

7.5.5 GPIO_IEV

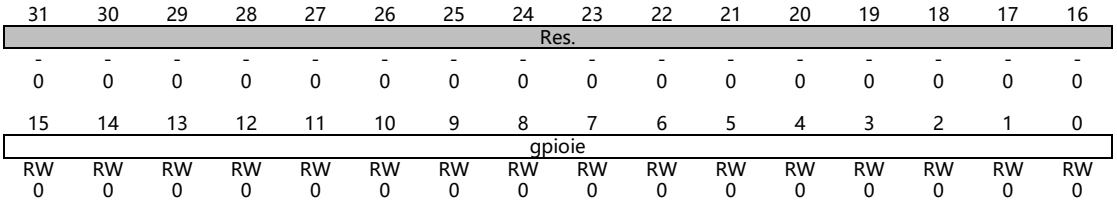
◆ **偏移地址：0x40C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpioiev	[15:0]	RW	GPIO 单边沿或者电平中断配置寄存器。16bit 每个 bit 相互独立。 0：下降沿或低电平触发中断 1：上升沿或高电平触发中断	0x0

7.5.6 GPIO_IE

◆ **偏移地址：0x410　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpioie	[15:0]	RW	GPIO 中断屏蔽寄存器，屏蔽后，中断状态不会上报到 GPIO_MIS 寄存器。 0：中断屏蔽 1：中断使能	0x0

7.5.7 GPIO_RIS

◆ **偏移地址：0x414　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpioris															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpioris	[15:0]	RO	GPIO 原始中断状态寄存器。16bit 每个 bit 相互独立。 0：无中断 1：产生中断	0x0

7.5.8 GPIO_MIS

◆ **偏移地址：0x418　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpiomis															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpiomis	[15:0]	RO	GPIO 中断状态寄存器。16bit 每个 bit 相互独立。 0：无中断 1：产生中断	0x0

7.5.9 GPIO_IC

◆ **偏移地址：0x41C　　复位值：0x00000000**

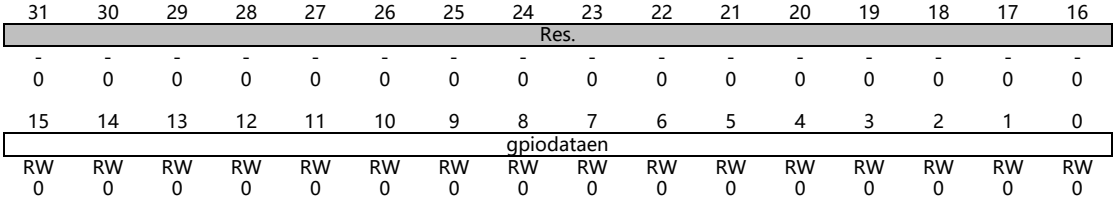
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gpioic															
W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpioic	[15:0]	W1P	GPIO 中断状态清除寄存器。16bit 每个 bit 相互独立。 0：无效果 1：清除中断状态	0x0

7.5.10 DATA_EN

◆ **偏移地址：0x424　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
gpiodataen	[15:0]	RW	GPIO 数据使能寄存器。16bit 每个 bit 相互独立。该寄存器使能有效 GPIO_DATA 才会生效。 0：数据无效 1：数据使能	0x0

8 管脚复用 (IO MUX)

8.1 简介

IO MUX 实现对芯片 IO 管脚的状态控制和功能复用管理。除特殊说明外，所有 IO 管脚默认为 GPIO 输入功能。

8.2 IO 主要特性

IO MUX 能够为 IO 管脚提供以下可配置特性：

- 上/下拉
- 输出状态：推挽或开漏
- 输入状态：浮空、上拉/下拉
- 模拟功能
- 可为每个 IO 选择不同的驱动能力、输出速率
- 复用功能选择

8.3 IO 模拟模式

芯片部分 IO 支持模拟模式。当 IO 管脚配置寄存器 sw 位为 1 时，IO 处于模拟模式。

⚠注意

模拟模式下，应禁用 IO 上拉、下拉配置（即 pu 和 pd 配置为 0，在模拟模式下使能上下拉会导致模拟信号质量下降）。

IO 配置为 GPIO 功能（即 funcsel 配置为 0），但是需要在 GPIO 控制器中关闭 GPIO 使能开关（即 gpiodataen 配置为 0），否则 GPIO 数字电路会影响模拟信号质量。

IO 复用功能选择为 ADC、ACMP、PGA 模拟输入或 DAC、PGA 模拟输出等模拟功能时，还需要在各模块配置中使能相应功能。

8.4 IO 数字模式

芯片部分 IO 为数模混合 IO，部分为纯数字 IO。对于数模混合 IO，IO 管脚配置寄存器 sw 位为 0 时，IO 处于数字模式，处于数字模式的 IO（包括纯数字 IO），可根据场景配置 IO 的具体参数，如上下拉、驱动能力等。

PA5 管脚在芯片上电时有特殊用途，用于判断芯片是否进入 test mode。如果 PA5 管脚作为普通 GPIO 使用，在硬件设计上就需要保证芯片上电时 PA5 处于低电平状态，否则芯片可能无法进入功能模式。

8.5 CORE IO 寄存器概览

CORE IO 寄存器基地如下：

- CORE IO 寄存器基地址为：0x61001800

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	PA0_IOCFG	0x0	32	RW	0x00000000	PA0 管脚配置寄存器
2	PA1_IOCFG	0x4	32	RW	0x00000000	PA1 管脚配置寄存器
3	PA2_IOCFG	0x8	32	RW	0x00000000	PA2 管脚配置寄存器
4	PA3_IOCFG	0xC	32	RW	0x00000000	PA3 管脚配置寄存器
5	PA9_IOCFG	0x10	32	RW	0x00000000	PA9 管脚配置寄存器
6	PA10_IOCFG	0x14	32	RW	0x00080000	PA10 管脚配置寄存器
7	PA11_IOCFG	0x18	32	RW	0x00080000	PA11 管脚配置寄存器
8	PA12_IOCFG	0x1c	32	RW	0x00000000	PA12 管脚配置寄存器
9	PA13_IOCFG	0x20	32	RW	0x00000000	PA13 管脚配置寄存器
10	PA14_IOCFG	0x24	32	RW	0x00000000	PA14 管脚配置寄存器
11	PA15_IOCFG	0x28	32	RW	0x00000000	PA15 管脚配置寄存器
12	PB0_IOCFG	0x2C	32	RW	0x00000000	PB0 管脚配置寄存器
13	PB1_IOCFG	0x30	32	RW	0x00000000	PB1 管脚配置寄存器
14	PB2_IOCFG	0x34	32	RW	0x00000000	PB2 管脚配置寄存器
15	PB3_IOCFG	0x38	32	RW	0x00000000	PB3 管脚配置寄存器
16	PB4_IOCFG	0x3c	32	RW	0x00000000	PB4 管脚配置寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
17	PB5_IOCFG	0x40	32	RW	0x00000000	PB5 管脚配置寄存器
18	PB6_IOCFG	0x44	32	RW	0x00000000	PB6 管脚配置寄存器
19	PB7_IOCFG	0x48	32	RW	0x00000000	PB7 管脚配置寄存器
20	PB8_IOCFG	0x4c	32	RW	0x00000000	PB8 管脚配置寄存器
21	PB9_IOCFG	0x50	32	RW	0x00000000	PB9 管脚配置寄存器
22	PB10_IOCFG	0x54	32	RW	0x00000000	PB10 管脚配置寄存器
23	PB11_IOCFG	0x58	32	RW	0x00000000	PB11 管脚配置寄存器
24	PB12_IOCFG	0x5C	32	RW	0x00000000	PB12 管脚配置寄存器
25	PB13_IOCFG	0x60	32	RW	0x00000000	PB13 管脚配置寄存器
26	PB14_IOCFG	0x64	32	RW	0x00000000	PB14 管脚配置寄存器
27	PB15_IOCFG	0x68	32	RW	0x00000000	PB15 管脚配置寄存器
28	PC0_IOCFG	0x6C	32	RW	0x00000000	PC0 管脚配置寄存器
29	PC1_IOCFG	0x70	32	RW	0x00000000	PC1 管脚配置寄存器
30	PC2_IOCFG	0x74	32	RW	0x00000000	PC2 管脚配置寄存器
31	PC3_IOCFG	0x78	32	RW	0x00000000	PC3 管脚配置寄存器
32	PC4_IOCFG	0x7C	32	RW	0x00000000	PC4 管脚配置寄存器
33	PC5_IOCFG	0x80	32	RW	0x00000000	PC5 管脚配置寄存器
34	PC6_IOCFG	0x84	32	RW	0x00000000	PC6 管脚配置寄存器
35	PC7_IOCFG	0x88	32	RW	0x00000000	PC7 管脚配置寄存器
36	PC8_IOCFG	0x8C	32	RW	0x00000000	PC8 管脚配置寄存器
37	PC9_IOCFG	0x90	32	RW	0x00000000	PC9 管脚配置寄存器
38	PC10_IOCFG	0x94	32	RW	0x00000000	PC10 管脚配置寄存器
39	PC11_IOCFG	0x98	32	RW	0x00000000	PC11 管脚配置寄存器
40	PC12_IOCFG	0x9C	32	RW	0x00000000	PC12 管脚配置寄存器
41	PC13_IOCFG	0xA0	32	RW	0x00000000	PC13 管脚配置寄存器
42	PC14_IOCFG	0xA4	32	RW	0x00000000	PC14 管脚配置寄存器
43	PC15_IOCFG	0xA8	32	RW	0x00000000	PC15 管脚配置寄存器
44	PD0_IOCFG	0xAC	32	RW	0x00000000	PD0 管脚配置寄存器
45	PD1_IOCFG	0xB0	32	RW	0x00000000	PD1 管脚配置寄存器
46	PD2_IOCFG	0xB4	32	RW	0x00000000	PD2 管脚配置寄存器
47	PD3_IOCFG	0xB8	32	RW	0x00000000	PD3 管脚配置寄存器
48	PD4_IOCFG	0xBC	32	RW	0x00000000	PD4 管脚配置寄存器
49	PD5_IOCFG	0xC0	32	RW	0x00000000	PD5 管脚配置寄存器
50	PD6_IOCFG	0xC4	32	RW	0x00000000	PD6 管脚配置寄存器
51	PD7_IOCFG	0xC8	32	RW	0x00000000	PD7 管脚配置寄存器
52	PD8_IOCFG	0xCC	32	RW	0x00000000	PD8 管脚配置寄存器
53	PD9_IOCFG	0xD0	32	RW	0x00000000	PD9 管脚配置寄存器
54	PD10_IOCFG	0xD4	32	RW	0x00000000	PD10 管脚配置寄存器
55	PD11_IOCFG	0xD8	32	RW	0x00000000	PD11 管脚配置寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
56	PD12_IOCFG	0xDC	32	RW	0x00000000	PD12 管脚配置寄存器
57	PD13_IOCFG	0xE0	32	RW	0x00000000	PD13 管脚配置寄存器
58	PD14_IOCFG	0xE4	32	RW	0x00082000	PD14 管脚配置寄存器
59	PD15_IOCFG	0xE8	32	RW	0x00000000	PD15 管脚配置寄存器
60	PE0_IOCFG	0xEC	32	RW	0x00002000	PE0 管脚配置寄存器
61	PE1_IOCFG	0xF0	32	RW	0x00002000	PE1 管脚配置寄存器
62	PE2_IOCFG	0xF4	32	RW	0x00000000	PE2 管脚配置寄存器
63	PE3_IOCFG	0xF8	32	RW	0x00000000	PE3 管脚配置寄存器
64	PE4_IOCFG	0xFC	32	RW	0x00000000	PE4 管脚配置寄存器
65	PE5_IOCFG	0x100	32	RW	0x00000000	PE5 管脚配置寄存器
66	PE6_IOCFG	0x104	32	RW	0x00000000	PE6 管脚配置寄存器
67	PE7_IOCFG	0x108	32	RW	0x00000000	PE7 管脚配置寄存器
68	PE8_IOCFG	0x10C	32	RW	0x00000000	PE8 管脚配置寄存器
69	PE9_IOCFG	0x110	32	RW	0x00000000	PE9 管脚配置寄存器
70	PE10_IOCFG	0x114	32	RW	0x00000000	PE10 管脚配置寄存器
71	PE11_IOCFG	0x118	32	RW	0x00000000	PE11 管脚配置寄存器
72	PE12_IOCFG	0x11C	32	RW	0x00000000	PE12 管脚配置寄存器
73	PE13_IOCFG	0x120	32	RW	0x00000000	PE13 管脚配置寄存器
74	PE14_IOCFG	0x124	32	RW	0x00002000	PE14 管脚配置寄存器
75	PE15_IOCFG	0x128	32	RW	0x00000000	PE15 管脚配置寄存器
76	PF0_IOCFG	0x12C	32	RW	0x00000000	PF0 管脚配置寄存器
77	PF1_IOCFG	0x130	32	RW	0x00000000	PF1 管脚配置寄存器
78	BOOT_IOCFG	0x134	32	RW	0x00002000	BOOT 管脚配置寄存器
79	PF2_IOCFG	0x138	32	RW	0x00000000	PF2 管脚配置寄存器
80	PF3_IOCFG	0x13C	32	RW	0x00000000	PF3 管脚配置寄存器
81	PF4_IOCFG	0x140	32	RW	0x00000000	PF4 管脚配置寄存器
82	PF5_IOCFG	0x144	32	RW	0x00000000	PF5 管脚配置寄存器

8.6 CORE IO 寄存器描述

8.6.1 PA0_IOCFG

- **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 1：GPT3_CH0; 2：KEY_IN0; 其他：保留。	0x0

8.6.2 PA1_IOCFG

◆ **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。	0x0

Name	Bits	R/W	Description	Default
			0：GPIO； 1：GPT3_CH1； 2：KEY_IN1； 其他：保留。	

8.6.3 PA2_IOCFG

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

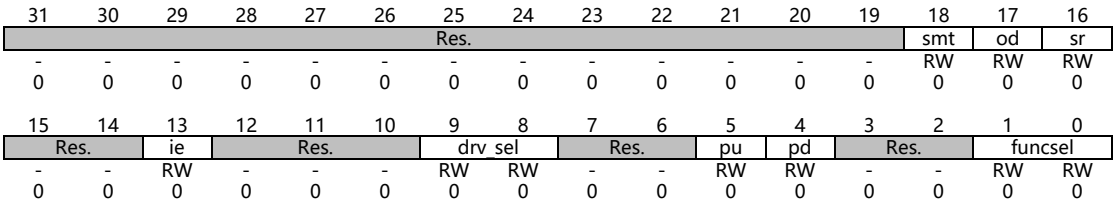
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0

Name	Bits	R/W	Description	Default
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 1：GPT3_CH2; 2：KEY_IN2; 其他：保留。	0x0

8.6.4 PA3_IOCFG

◆ **偏移地址：0xC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA；	0x0

Name	Bits	R/W	Description	Default
			3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 1：GPT3_CH3; 2：KEY_IN3; 其他：保留。	0x0

8.6.5 PA9_IOCFG

◆ **偏移地址：0x10　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0

Name	Bits	R/W	Description	Default
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.6.6 PA10_IOCFG

◆ **偏移地址：0x14　　复位值：0x00080000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x1
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0

Name	Bits	R/W	Description	Default
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：I2C1_SDA; 其他：保留。	0x0

8.6.7 PA11_IOCFG

◆ **偏移地址：0x18　　复位值：0x00080000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0

Name	Bits	R/W	Description	Default
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x1
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：I2C1_SCL; 其他：保留。	0x0

8.6.8 PA12_IOCFG

◆ **偏移地址：0x1c　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			func sel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0

Name	Bits	R/W	Description	Default
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 1：GPT3_ETR。	0x0

8.6.9 PA13_IOCFG

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			func sel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR；	0x0

Name	Bits	R/W	Description	Default
			1：禁止 SR。	
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.6.10 PA14_IOCFG

◆ **偏移地址：0x24　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

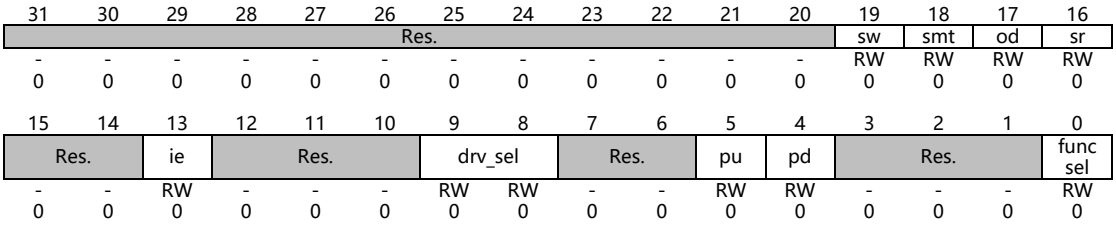
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			func sel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开	0x0

Name	Bits	R/W	Description	Default
			关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.6.11 PA15_IOCFG

◆ **偏移地址：0x28　　复位值：0x00000000**

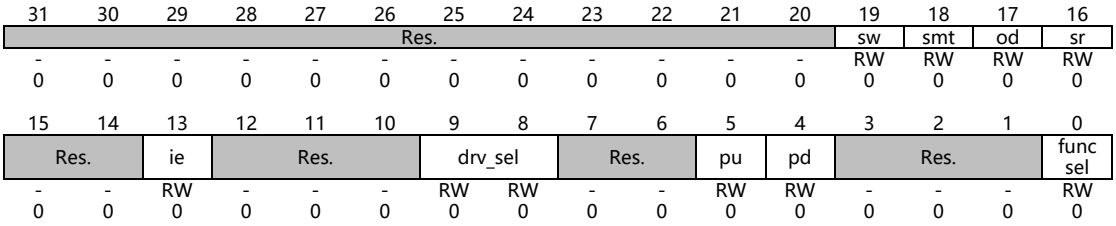


Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0

Name	Bits	R/W	Description	Default
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.6.12 PB0_IOCFG

◆ **偏移地址：0x2C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。	0x0

Name	Bits	R/W	Description	Default
			0：关闭输入通路； 1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.6.13 PB1_IOCFG

◆ **偏移地址：0x30　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。	0x0

Name	Bits	R/W	Description	Default
			0：禁止 SMT； 1：使能 SMT。	
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART1_CTS； 2：KEY_IN4； 5：GPT2_CH0； 其他：保留。	0x0

8.6.14 PB2_IOCFG

- **偏移地址：0x34　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART1_RTS_DE； 2：KEY_IN5； 5：GPT2_CH1； 其他：保留。	0x0

8.6.15 PB3_IOCFG

◆ **偏移地址：0x38　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。	0x0

Name	Bits	R/W	Description	Default
			0：关闭输入通路； 1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART1_TX； 2：KEY_IN6； 5：GPT2_CH2； 其他：保留。	0x0

8.6.16 PB4_IOCFG

◆ **偏移地址：0x3c　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

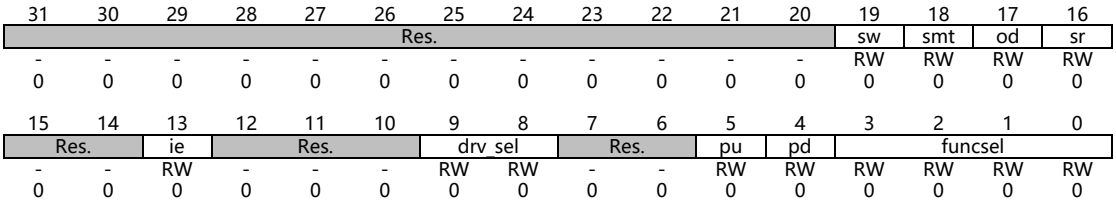
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭	0x0

Name	Bits	R/W	Description	Default
			（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART1_RX； 2：KEY_IN7； 5：GPT2_CH3； 其他：保留。	0x0

8.6.17 PB5_IOCFG

◆ **偏移地址：0x40　　复位值：0x00000000**



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN8； 3：SPI0_NSS0； 4：GPT1_CH0； 5：GPT2_ETR； 其他：保留。	0x0

8.6.18 PB6_IOCFG

◆ **偏移地址：0x44　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0

Name	Bits	R/W	Description	Default
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN9； 3：SPI0_SCK； 4：GPT1_CH1； 其他：保留。	0x0

8.6.19 PB7_IOCFG

◆ **偏移地址：0x48　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭	0x0

Name	Bits	R/W	Description	Default
			（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN10； 3：SPI0_MISO； 4：GPT1_CH2； 其他：保留。	0x0

8.6.20 PB8_IOCFG

◆ **偏移地址：0x4c　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN11； 3：SPI0_MOSI； 4：GPT1_CH3； 其他：保留。	0x0

8.6.21 PB9_IOCFG

◆ **偏移地址：0x50　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。	0x0

Name	Bits	R/W	Description	Default
			0：关闭输入通路； 1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：UART0_TX； 4：GPT1_ETR； 其他：保留。	0x0

8.6.22 PB10_IOCFG

◆ **偏移地址：0x54　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

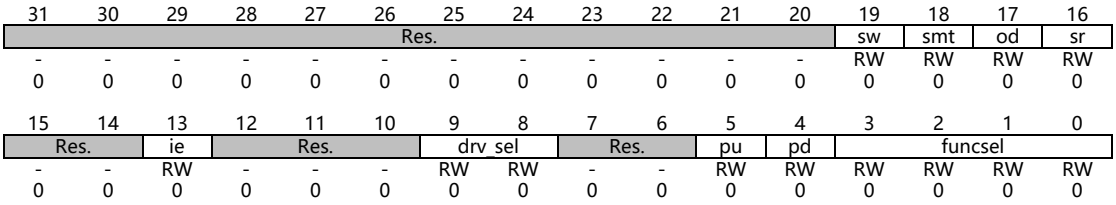
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭	0x0

Name	Bits	R/W	Description	Default
			（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 1：UART0_RX; 2：KEY_IN12; 其他：保留。	0x0

8.6.23 PB11_IOCFG

◆ **偏移地址：0x58　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN13； 其他：保留。	0x0

8.6.24 PB12_IOCFG

◆ **偏移地址：0x5C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0

Name	Bits	R/W	Description	Default
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN14； 其他：保留。	0x0

8.6.25 PB13_IOCFG

◆ **偏移地址：0x60　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

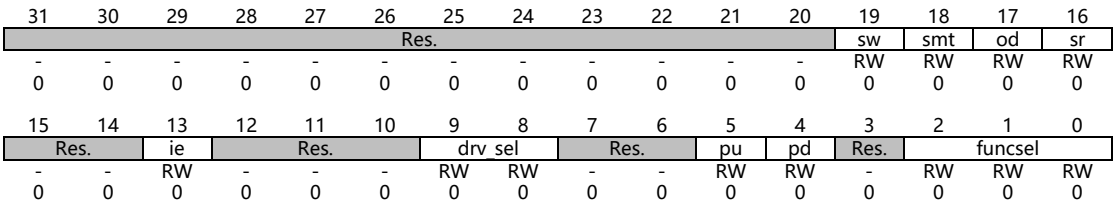
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0

Name	Bits	R/W	Description	Default
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：KEY_IN15； 其他：保留。	0x0

8.6.26 PB14_IOCFG

◆ **偏移地址：0x64　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0

Name	Bits	R/W	Description	Default
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_ETRIN； 其他：保留。	0x0

8.6.27 PB15_IOCFG

◆ **偏移地址：0x68　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_CH0N； 其他：保留。	0x0

8.6.28 PC0_IOCFG

◆ **偏移地址：0x6C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0

Name	Bits	R/W	Description	Default
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_CH0； 其他：保留。	0x0

8.6.29 PC1_IOCFG

◆ **偏移地址：0x70　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

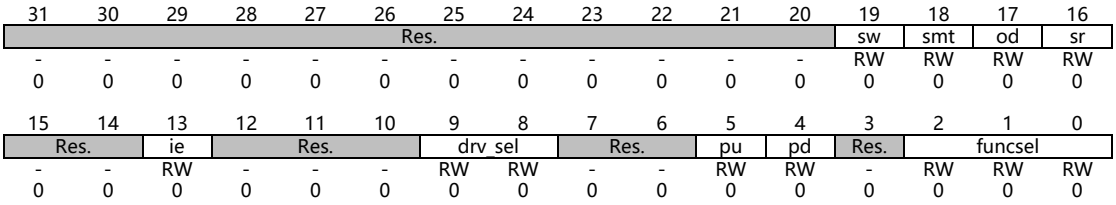
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭	0x0

Name	Bits	R/W	Description	Default
			（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_CH1N； 其他：保留。	0x0

8.6.30 PC2_IOCFG

- **偏移地址：0x74　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_CH1； 其他：保留。	0x0

8.6.31 PC3_IOCFG

◆ **偏移地址：0x78　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路；	0x0

Name	Bits	R/W	Description	Default
			1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_CH2N； 其他：保留。	0x0

8.6.32 PC4_IOCFG

◆ **偏移地址：0x7C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。	0x0

Name	Bits	R/W	Description	Default
			0：禁止 SMT； 1：使能 SMT。	
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_CH2； 其他：保留。	0x0

8.6.33 PC5_IOCFG

◆ **偏移地址：0x80　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 4：APT0_BKIN；	0x0

Name	Bits	R/W	Description	Default
			其他：保留。	

8.6.34 PC6_IOCFG

◆ **偏移地址：0x84　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA；	0x0

Name	Bits	R/W	Description	Default
			3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 5：UART2_RX； 其他：保留。	0x0

8.6.35 PC7_IOCFG

◆ **偏移地址：0x88　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0

Name	Bits	R/W	Description	Default
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 5：UART2_TX； 其他：保留。	0x0

8.6.36 PC8_IOCFG

◆ **偏移地址：0x8C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

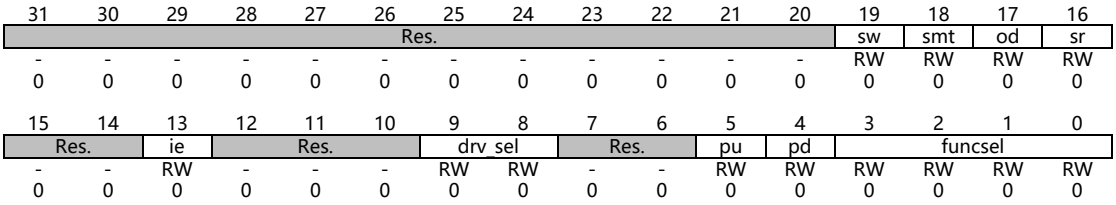
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。	0x0

Name	Bits	R/W	Description	Default
			注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 3：SPI1_NSS0； 5：UART2_RX； 其他：保留。	0x0

8.6.37 PC9_IOCFG

◆ **偏移地址：0x90　　复位值：0x00000000**



31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：GPT4_CH3； 3：SPI1_NSS1； 4：APT0_BKIN； 6：I2S0_WS； 其他：保留。	0x0

8.6.38 PC10_IOCFG

◆ **偏移地址：0x94　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0

Name	Bits	R/W	Description	Default
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：GPT4_CH2； 3：SPI1_SCK； 4：APT0_CH0N； 5：UART2_CTS； 6：I2S0_CK； 其他：保留。	0x0

8.6.39 PC11_IOCFG

◆ **偏移地址：0x98　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。	0x0

Name	Bits	R/W	Description	Default
			注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：GPT4_CH1； 3：SPI1_MISO； 4：APT0_CH1N； 5：UART2_RTS_DE； 6：I2S0_SDI0； 其他：保留。	0x0

8.6.40 PC12_IOCFG

◆ **偏移地址：0x9C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：打开。	
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：GPT4_CH0； 3：SPI1_MOSI； 4：APT0_CH2N； 6：I2S0_SDO0； 其他：保留。	0x0

8.6.41 PC13_IOCFG

◆ **偏移地址：0xA0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。	0x0

Name	Bits	R/W	Description	Default
			0：使能 SR； 1：禁止 SR。	
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT7； 5：UART2_TX； 6：I2S0_SDI1； 其他：保留。	0x0

8.6.42 PC14_IOCFG

◆ **偏移地址：0xA4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟使能开关。 0：关闭模拟功能；	0x0

Name	Bits	R/W	Description	Default
			1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。模拟功能复用选择请参考见各个模拟 IP 的芯片用户指南。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT6； 5：UART2_RX； 6：I2S0_SDO1； 其他：保留。	0x0

8.6.43 PC15_IOCFG

◆ **偏移地址：0xA8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。	0x0

Name	Bits	R/W	Description	Default
			0：GPIO； 1：KEY_OUT5； 6：I2S0_SDI2； 其他：保留。	

8.6.44 PD0_IOCFG

◆ **偏移地址：0xAC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0

Name	Bits	R/W	Description	Default
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT4； 5：UART2_CTS； 6：I2S0_SDO2； 其他：保留。	0x0

8.6.45 PD1_IOCFG

◆ **偏移地址：0xB0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA；	0x0

Name	Bits	R/W	Description	Default
			2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT3； 4：GPT0_CH0； 5：UART2_RTS_DE； 其他：保留。	0x0

8.6.46 PD2_IOCFG

◆ **偏移地址：0xB4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。	0x0

Name	Bits	R/W	Description	Default
			0：关闭输入通路； 1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT2； 4：GPT0_CH1； 7：SDIO_VDD1_ON； 其他：保留。	0x0

8.6.47 PD3_IOCFG

◆ **偏移地址：0xB8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0

Name	Bits	R/W	Description	Default
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT1； 4：GPT0_CH2； 7：SDIO_WP； 其他：保留。	0x0

8.6.48 PD4_IOCFG

◆ **偏移地址：0xBC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0

Name	Bits	R/W	Description	Default
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT0； 4：GPT0_CH3； 7：SDIO_CD； 其他：保留。	0x0

8.6.49 PD5_IOCFG

◆ **偏移地址：0xC0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT5； 2：GPT4_CH0； 3：QDM1_STRB；	0x0

Name	Bits	R/W	Description	Default
			4：GPT0_ETR； 6：I2S0_SDI1； 7：SDIO_D7； 其他：保留。	

8.6.50 PD6_IOCFG

◆ **偏移地址：0xC4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0

Name	Bits	R/W	Description	Default
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT4； 2：GPT4_CH1； 3：QDM1_INDEX； 6：I2S0_SDO1； 7：SDIO_D6； 其他：保留。	0x0

8.6.51 PD7_IOCFG

◆ **偏移地址：0xC8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。	0x0

Name	Bits	R/W	Description	Default
			0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT3； 2：GPT4_CH2； 3：QDM1_B； 6：I2S0_SDI2； 7：SDIO_D5； 其他：保留。	0x0

8.6.52 PD8_IOCFG

◆ **偏移地址：0xCC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。	0x0

Name	Bits	R/W	Description	Default
			0：使能 SR； 1：禁止 SR。	
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT2； 2：GPT4_CH3； 3：QDM1_A； 4：APT0_BKIN； 6：I2S0_SDO2； 7：SDIO_D4； 其他：保留。	0x0

8.6.53 PD9_IOCFG

◆ **偏移地址：0xD0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：KEY_OUT1； 3：QDM1_A； 4：APT0_CH0； 5：I2C1_SMBA；	0x0

Name	Bits	R/W	Description	Default
			6：I2S0_MCK； 7：SDIO_D3； 8：MCO； 其他：保留。	

8.6.54 PD10_IOCFG

◆ **偏移地址：0xD4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0

Name	Bits	R/W	Description	Default
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART0_TX； 2：KEY_OUT0； 3：QDM1_B； 4：APT0_CH1； 5：I2C1_SCL； 7：SDIO_D2； 其他：保留。	0x0

8.6.55 PD11_IOCFG

◆ **偏移地址：0xD8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。	0x0

Name	Bits	R/W	Description	Default
			0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART0_RX； 3：QDM1_INDEX； 4：APT0_CH2； 5：I2C1_SDA； 7：SDIO_D1； 其他：保留。	0x0

8.6.56 PD12_IOCFG

◆ **偏移地址：0xDC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

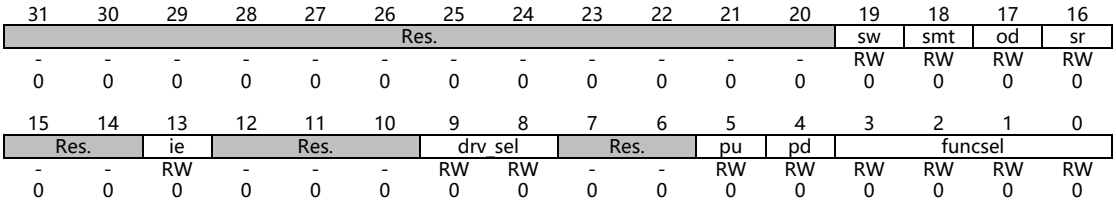
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟 USB_DM 使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT；	0x0

Name	Bits	R/W	Description	Default
			1：使能 SMT。	
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART0_CTS； 3：QDM0_INDEX； 4：APT0_CH2N； 6：CAN_RX； 7：SDIO_D0； 其他：保留。	0x0

8.6.57 PD13_IOCFG

◆ **偏移地址：0xE0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟 USB_DP 使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭；	0x0

Name	Bits	R/W	Description	Default
			1：打开。	
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART0_RTS_DE； 3：QDM0_B； 4：APT0_CH1N； 6：CAN_TX； 7：SDIO_CMD； 其他：保留。	0x0

8.6.58 PD14_IOCFG

◆ **偏移地址：0xE4　　复位值：0x00082000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟 USB_ID 使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。	0x1

Name	Bits	R/W	Description	Default
			0：关闭输入通路； 1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x1
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART2_CTS； 3：QDM0_A； 4：APT0_CH0N； 6：SWDIO_JTAG_TMS； 7：SDIO_CLK； 其他：保留。	0x6

8.6.59 PD15_IOCFG

◆ **偏移地址：0xE8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式；	0x0

Name	Bits	R/W	Description	Default
			1：使能 Open drain，即输出为开漏模式。	
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART2_RTS_DE； 3：QDM0_STRB； 4：I2C0_SMBA； 5：USBFS_SOF； 7：SDIO_RSTN 其他：保留。	0x0

8.6.60 PE0_IOCFG

◆ **偏移地址：0xEC　　复位值：0x00002000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x1
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x1
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART2_TX； 2：SPI1_NSS0； 3：SPI0_NSS0； 4：I2C0_SDA； 6：SWCLK_JTAG_TCK； 7：SDIO_CLK； 其他：保留。	0x6

8.6.61 PE1_IOCFG

◆ **偏移地址：0xF0　　复位值：0x00002000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x1
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x1
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART2_RX； 2：SPI1_NSS1；	0x6

Name	Bits	R/W	Description	Default
			3：SPI0_NSS1； 4：I2C0_SCL； 5：I2S1_WS； 6：JTAG_TDI； 7：SDIO_CMD； 8：USBFS_DRVVBUS； 其他：保留。	

8.6.62 PE2_IOCFG

◆ **偏移地址：0xF4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0

Name	Bits	R/W	Description	Default
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：UART1_TX； 2：SPI1_SCK； 3：UART3_TX； 4：GPT3_CH0； 其他：保留。	0x0

8.6.63 PE3_IOCFG

◆ **偏移地址：0xF8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0

Name	Bits	R/W	Description	Default
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：UART1_RX； 2：SPI1_MISO； 3：UART3_RX； 4：GPT3_CH1； 其他：保留。	0x0

8.6.64 PE4_IOCFG

◆ **偏移地址：0xFC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0

Name	Bits	R/W	Description	Default
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：SPI1_MOSI； 3：UART4_TX； 4：GPT3_CH2； 7：SDIO_CD； 其他：保留。	0x0

8.6.65 PE5_IOCFG

- **偏移地址：0x100　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：UART3_CTS； 3：UART4_CTS； 6：CAN_RX；	0x0

Name	Bits	R/W	Description	Default
			7：SDIO_VDD1_ON； 其他：保留。	

8.6.66 PE6_IOCFG

◆ **偏移地址：0x104　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭；	0x0

Name	Bits	R/W	Description	Default
			1：打开。	
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：UART3_RTS_DE； 3：UART4_RTS_DE； 6：CAN_TX； 7：SDIO_RSTN； 其他：保留。	0x0

8.6.67 PE7_IOCFG

◆ **偏移地址：0x108　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA；	0x0

Name	Bits	R/W	Description	Default
			3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSSN； 3：UART4_RX； 4：GPT3_CH3； 7：SDIO_WP； 其他：保留。	0x0

8.6.68 PE8_IOCFG

◆ **偏移地址：0x10C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。	0x0

Name	Bits	R/W	Description	Default
			0：关闭输入通路； 1：打开输入通路。	
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSCK； 3：UART1_CTS； 4：GPT1_CH0； 其他：保留。	0x0

8.6.69 PE9_IOCFG

◆ **偏移地址：0x110　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0

Name	Bits	R/W	Description	Default
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSIO3； 3：UART1_RTS_DE； 4：GPT1_CH1； 7：SDIO_D7； 其他：保留。	0x0

8.6.70 PE10_IOCFG

- **偏移地址：0x114　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSIO2； 3：UART1_TX； 7：SDIO_D6；	0x0

Name	Bits	R/W	Description	Default
			其他：保留。	

8.6.71 PE11_IOCFG

◆ **偏移地址：0x118　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0

Name	Bits	R/W	Description	Default
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSIO1； 3：UART1_RX； 4：GPT1_CH3； 7：SDIO_D5； 其他：保留。	0x0

8.6.72 PE12_IOCFG

◆ **偏移地址：0x11C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0

Name	Bits	R/W	Description	Default
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSIO0； 4：GPT1_CH2； 5：I2S1_MCK； 7：SDIO_D4； 其他：保留。	0x0

8.6.73 PE13_IOCFG

◆ **偏移地址：0x120　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0

Name	Bits	R/W	Description	Default
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART1_TX； 3：SPI0_SCK； 4：APT1_CH0N； 5：I2S1_CK； 6：JTAG_TDO； 7：SDIO_D3； 8：GPT0_ETR； 其他：保留。	0x6

8.6.74 PE14_IOCFG

◆ **偏移地址：0x124　　复位值：0x00002000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

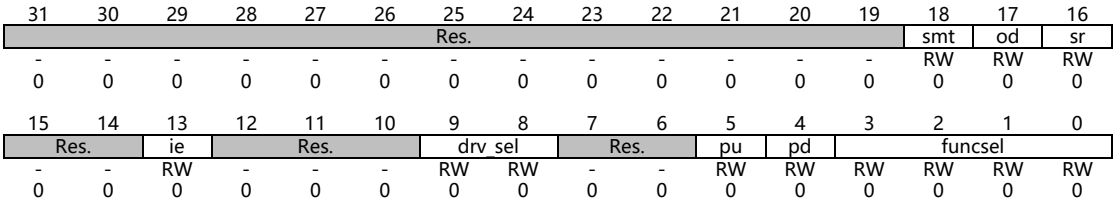
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式；	0x0

Name	Bits	R/W	Description	Default
			1：使能 Open drain，即输出为开漏模式。	
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x1
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x2
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART1_RX； 2：QSPI_QSIO3； 3：SPI0_MISO； 4：APT1_CH1N； 5：I2S1_SDI0； 6：JTAG_TRST； 7：SDIO_D2； 8：GPT0_CH0； 其他：保留。	0x6

8.6.75 PE15_IOCFG

◆ **偏移地址：0x128　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSIO2； 3：SPI0_MOSI； 4：APT1_CH2N； 5：I2S1_SDO0；	0x0

Name	Bits	R/W	Description	Default
			6：I2C0_SMBA； 7：SDIO_D1； 8：GPT0_CH1； 其他：保留。	

8.6.76 PF0_IOCFG

◆ **偏移地址：0x12C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0

Name	Bits	R/W	Description	Default
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART0_TX； 2：QSPI_QSIO1； 4：APT1_CH0； 6：I2C0_SCL； 7：SDIO_D0； 8：GPT0_CH2； 其他：保留。	0x0

8.6.77 PF1_IOCFG

◆ **偏移地址：0x130　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。	0x0

Name	Bits	R/W	Description	Default
			0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：UART0_RX； 2：QSPI_QSIO0； 4：APT1_CH1； 6：I2C0_SDA； 8：GPT0_CH3； 其他：保留。	0x0

8.6.78 BOOT_IOCFG

◆ **偏移地址：0x134　　复位值：0x00002000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			func sel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。	0x0

Name	Bits	R/W	Description	Default
			0：使能 SR； 1：禁止 SR。	
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x1
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x1
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0x0：BOOT0； 其他：保留。	0x0

8.6.79 PF2_IOCFG

◆ **偏移地址：0x138　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式；	0x0

Name	Bits	R/W	Description	Default
			1：使能 Open drain，即输出为开漏模式。	
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO；； 2：QSPI_QSCK； 4：APT1_CH2； 5：CAN_RX； 6：I2C0_SCL； 其他：保留。	0x0

8.6.80 PF3_IOCFG

◆ **偏移地址：0x13C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 2：QSPI_QSSN； 4：APT1_BKIN； 5：CAN_TX；	0x0

Name	Bits	R/W	Description	Default
			6：I2C0_SDA； 其他：保留。	

8.6.81 PF4_IOCFG

◆ **偏移地址：0x140　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.	funcsel		
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭；	0x0

Name	Bits	R/W	Description	Default
			1：打开。	
reserved	[3]	-	保留。	0x0
funcsel	[2:0]	RW	功能选择。 0：GPIO； 1：UART0_TX； 4：APT1_ETRIN； 其他：保留。	0x0

8.6.82 PF5_IOCFG

◆ **偏移地址：0x144　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			func sel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3： idle（无驱动能力）。	0x0

Name	Bits	R/W	Description	Default
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 1：UART0_RX； 其他：保留。	0x0

8.7 BACKUP IO 寄存器概览

BACKUP IO 寄存器基地如下：

- BACKUP IO 寄存器基地址为：0x61006400

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	PA4_IOCFG	0x0	32	RW	0x00000000	PA4 管脚配置寄存器
2	PA5_IOCFG	0x4	32	RW	0x00002010	PA5 管脚配置寄存器
3	PA6_IOCFG	0x8	32	RW	0x00080000	PA6 管脚配置寄存器
4	PA7_IOCFG	0xC	32	RW	0x00080000	PA7 管脚配置寄存器
5	PA8_IOCFG	0x10	32	RW	0x00000000	PA8 管脚配置寄存器

8.8 BACKUP IO 寄存器描述

8.8.1 PA4_IOCFG

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.		funcsel	
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3：idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:2]	-	保留。	0x0
funcsel	[1:0]	RW	功能选择。 0：GPIO； 1：WKUP2； 2：RTC_TAMP2； 3：LSCO；	0x0

Name	Bits	R/W	Description	Default
			其他：保留。	

8.8.2 PA5_IOCFG

◆ **偏移地址：0x4　　复位值：0x00002010**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	funcsel			
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	RW	RW	RW	RW
0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x1
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3：idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x1

Name	Bits	R/W	Description	Default
funcsel	[3:0]	RW	功能选择。 0：GPIO； 1：WKUP1； 2：RTC_TAMP1； 3：RTC_TS； 4：RTC_ALARM_OUT； 5：TEST_MODE； 6：RTC_1HZ_OUT； 7：RTC_WTF_OUT； 8：LSCO； 其他：保留。 注意：TEST_MODE 信号，上电启动时该信号为高，则芯片进入 test mode 模式；上电启动时该信号为低，则芯片进入正常的功能模式。	0x0

8.8.3 PA6_IOCFG

◆ **偏移地址：0x8　　复位值：0x00080000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			funcsel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟 OSC32_IN 使能开关。 0：关闭模拟功能； 1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。	0x1
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0

Name	Bits	R/W	Description	Default
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3：idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.8.4 PA7_IOCFG

◆ **偏移地址：0xC　　复位值：0x00080000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sw	smt	od	sr
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

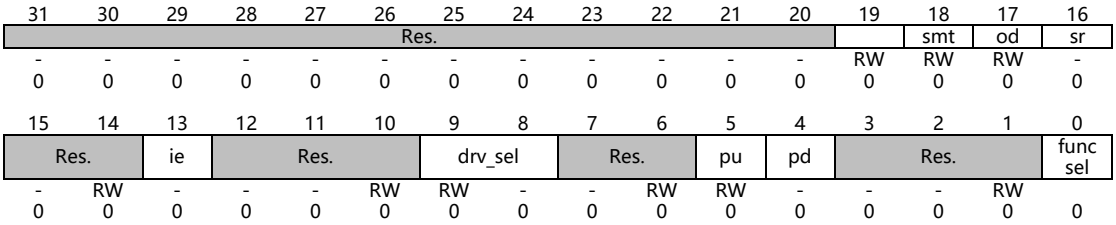
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		ie	Res.			drv_sel		Res.		pu	pd	Res.			func sel
-	-	RW	-	-	-	RW	RW	-	-	RW	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
sw	[19]	RW	模拟 OSC32_OUT 使能开关。 0：关闭模拟功能；	0x1

Name	Bits	R/W	Description	Default
			1：打开模拟功能。 注意：此时需要把该管脚复用成 GPIO 功能（即 funcsel=0）且在 GPIO 控制器中关闭 GPIO 使能开关（gpiodataen=0），同时把管脚上下拉都关闭（pu/pd=0）。	
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号，默认打开，如果需要节省功耗可以考虑关掉。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3：idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 其他：保留。	0x0

8.8.5 PA8_IOCFG

◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
smt	[18]	RW	使能（SMT）施密特功能，针对输入有效。 0：禁止 SMT； 1：使能 SMT。	0x0
od	[17]	RW	Open drain 使能信号。 0：禁止 Open drain，即输出为推挽模式； 1：使能 Open drain，即输出为开漏模式。	0x0
sr	[16]	RW	压摆率调整信号。 0：使能 SR； 1：禁止 SR。	0x0
reserved	[15:14]	-	保留。	0x0
ie	[13]	RW	IO 输入使能。 0：关闭输入通路； 1：打开输入通路。	0x0
reserved	[12:10]	-	保留。	0x0
drv_sel	[9:8]	RW	输出驱动档位选择。 0：三档驱动能力，输出电流最小 12mA； 1：二档驱动能力，输出电流最小 6mA； 2：一档驱动能力，输出电流最小 3mA； 3：idle（无驱动能力）。	0x0
reserved	[7:6]	-	保留。	0x0
pu	[5]	RW	管脚上拉控制。 0：关闭； 1：打开。	0x0
pd	[4]	RW	管脚下拉控制。 0：关闭； 1：打开。	0x0
reserved	[3:1]	-	保留。	0x0
funcsel	[0]	RW	功能选择。 0：GPIO； 1：WKUP0； 2：RTC_TAMP0； 3：LSCO；	0x0

Name	Bits	R/W	Description	Default
			其他：保留。	

9 系统控制器（SYSCTRL)

9.1 SYSCTRL 简介

系统控制器 SYSCTRL（System Controller）包含一组配置寄存器，其主要用途如下：

- 为系统关键寄存器提供写保护控制
- 系统复位事件次数统计
- 触发系统软件复位和软件中断
- 系统状态查询
- USB 控制信号
- DMA 请求信号源复用映射配置（使用详情参考 13.3.3 DMAC 请求映射章节）
- SPI 模式使能配置
- SYSTICK 控制

9.2 SYSCTRL 寄存器概览

SYSCTRL 基地址如下：

- SYSCTRL 寄存器基地址为：0x61000800

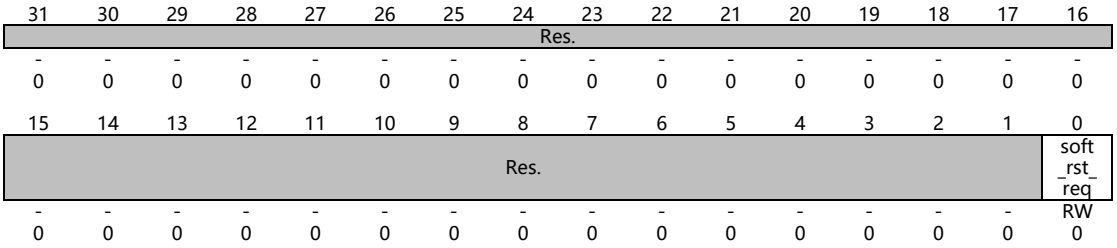
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	SYS_RST_REQ	0x0	32	RW	0x00000000	系统软复位请求寄存器
2	SYSTICK_CTRL	0x4	32	RW	0x00000001	SYSTICK 配置寄存器
3	SYSTICK_CNT_INIT_VAL	0x8	32	RW	0x00000000	SYSTICK 计数初始值设置寄存器
4	INT_SOFTWARE	0xC	32	RW	0x00000000	软件中断控制寄存器
5	LOCKEN	0x10	32	RW	0x00000007	写保护寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
6	SRAM_PARITY_ERR	0x14	32	RO	0x00000000	SRAM 奇偶校验错误状态寄存器
7	SRAM_PARITY_ERR_CLR	0x18	32	RW	0x00000000	SRAM 奇偶校验错误状态清除寄存器
8	SYS_RSV_CTRL	0x24	32	RW	0x00000000	系统散落控制信号
9	MODULE_CTRL	0x2C	32	RW	0x00000000	模块控制寄存器
10	USB_CTRL	0x44	32	RW	0x00072002	USB 控制寄存器
11	DMA_REQ_SEL	0x60+0x4 *reqnum	32	RW	0x00000000	DMA 请求复用选择寄存器
12	SC_RST_CNT0	0x100	32	RO	0x00000000	系统复位次数统计寄存器 0
13	SC_RST_CNT1	0x104	32	RO	0x00000000	系统复位次数统计寄存器 1
14	VERSION	0x108	32	RO	0x20220920	版本寄存器
15	USER_REG0	0x200	32	RW	0x00000000	用户专用寄存器 0
16	USER_REG1	0x204	32	RW	0x00000000	用户专用寄存器 1
17	USER_HRST_REG0	0x208	32	RW_HRST	0x00000000	用户专用硬复位寄存器 0
18	USER_HRST_REG1	0x20C	32	RW_HRST	0x00000000	用户专用硬复位寄存器 1
19	USER_POR_REG0	0x210	32	RW_POR	0x00000000	用户专用上电复位寄存器 0
20	USER_POR_REG1	0x214	32	RW_POR	0x00000000	用户专用上电复位寄存器 1

9.3 SYSCTRL 寄存器描述

9.3.1 SYS_RST_REQ

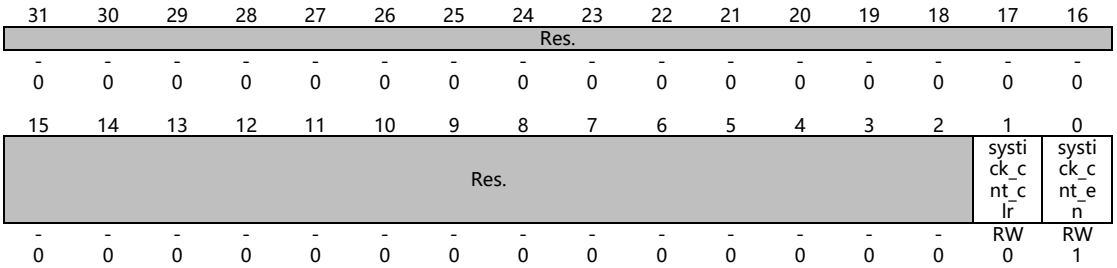
- **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
soft_rst_req	[0]	RW	系统软复位请求： 0：软复位请求撤离； 1：产生软复位请求，除 PMC/CRG 寄存器，以及 RW_POR/RW_HRST 属性寄存器外，其他寄存器全复位。	0x0

9.3.2 SYSTICK_CTRL

◆ **偏移地址：0x4　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
systick_cnt_clr	[1]	RW	systick 计数清零配置： 0：systick 计数不清零； 1：systick 计数清零（清零后恢复到计数初始值 SYSTICK_CNT_INIT_VAL，默认值是 0）。 注意：软件需要先配置 1 再配置 0，清零才生效。	0x0
systick_cnt_en	[0]	RW	systick 计数使能配置： 0：禁止； 1：使能。	0x1

9.3.3 SYSTICK_CNT_INIT_VAL

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
systick_cnt_init_val															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
systick_cnt_init_val															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
systick_cnt_init_val	[31:0]	RW	systick 计数初始值，systick_cnt_clr=1 时，systick 计数从该初始值开始。	0x0

9.3.4 INT_SOFTWARE

◆ **偏移地址：0xC　　复位值：0x00000000**

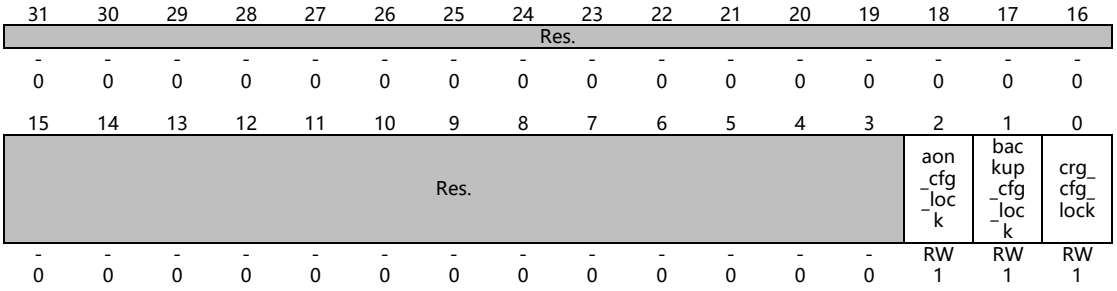
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															int_software
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
int_software	[0]	RW	软件中断产生控制（调试使用） 0：不产中断； 1：产生中断。	0x0

9.3.5 LOCKEN

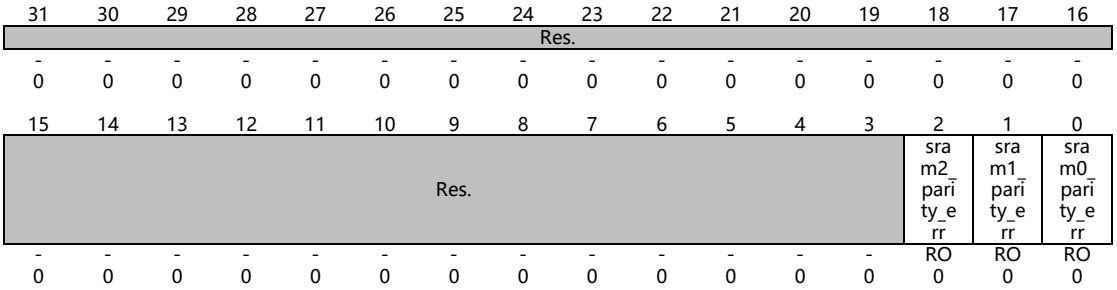
◆ **偏移地址：0x10　　复位值：0x00000007**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
aon_cfg_lock	[2]	RW	aon 写保护设置： 0：允许写操作； 1：禁止写操作。	0x1
backup_cfg_lock	[1]	RW	backup 写保护设置： 0：允许写操作； 1：禁止写操作。	0x1
crg_cfg_lock	[0]	RW	CRG 写保护设置： 0：允许写操作； 1：禁止写操作。	0x1

9.3.6 SRAM_PARITY_ERR

◆ **偏移地址：0x14　　复位值：0x00000000**

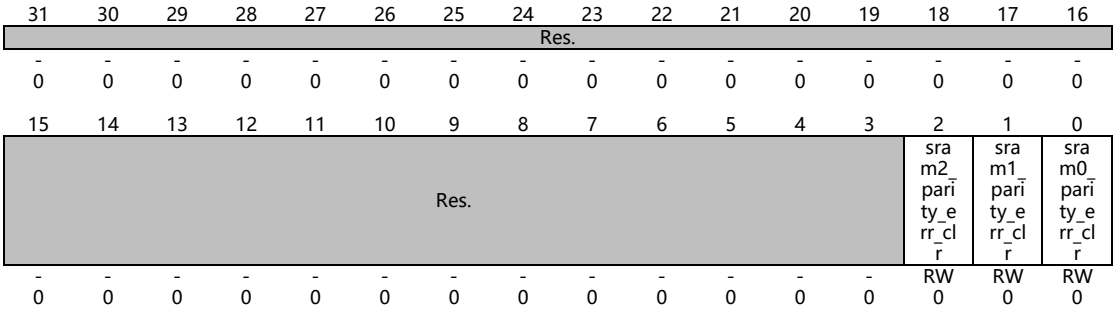


Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
sram2_parity_err	[2]	RO	SRAM2 奇偶校验错误状态。 0：无错误； 1：有错误。	0x0
sram1_parity_err	[1]	RO	SRAM1 奇偶校验错误状态。 0：无错误； 1：有错误。	0x0

Name	Bits	R/W	Description	Default
sram0_parity_err	[0]	RO	SRAM0 奇偶校验错误状态。 0：无错误； 1：有错误。	0x0

9.3.7 SRAM_PARITY_ERR_CLR

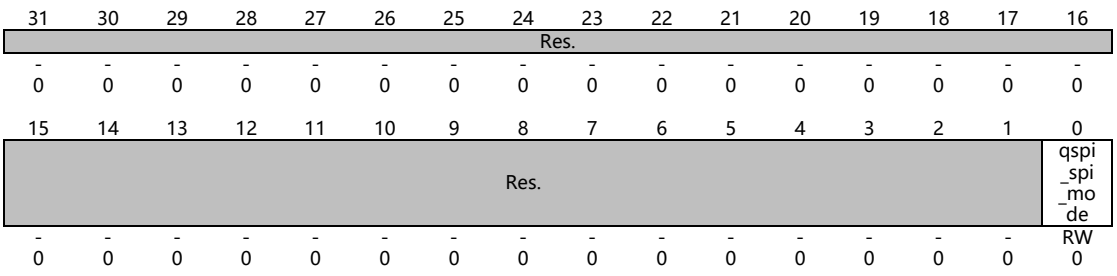
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
sram2_parity_err_clr	[2]	RW	SRAM2 奇偶校验错误状态清除设置。 0：不清除； 1：清除。	0x0
sram1_parity_err_clr	[1]	RW	SRAM1 奇偶校验错误状态清除设置。 0：不清除； 1：清除。	0x0
sram0_parity_err_clr	[0]	RW	SRAM0 奇偶校验错误状态清除设置。 0：不清除； 1：清除。	0x0

9.3.8 SYS_RSV_CTRL

◆ **偏移地址：0x24　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
qspi_spi_mode	[0]	RW	QSPI 和 SPI 模式配置： 0：QSPI 配置为 QSPI 模式； 1：QSPI 配置为 SPI 模式，且为 TI 帧格式。其它如 MOTO、MW 帧格式无需配置。	0x0

9.3.9 MODULE_CTRL

◆ **偏移地址：0x2C　　复位值：0x00000000**

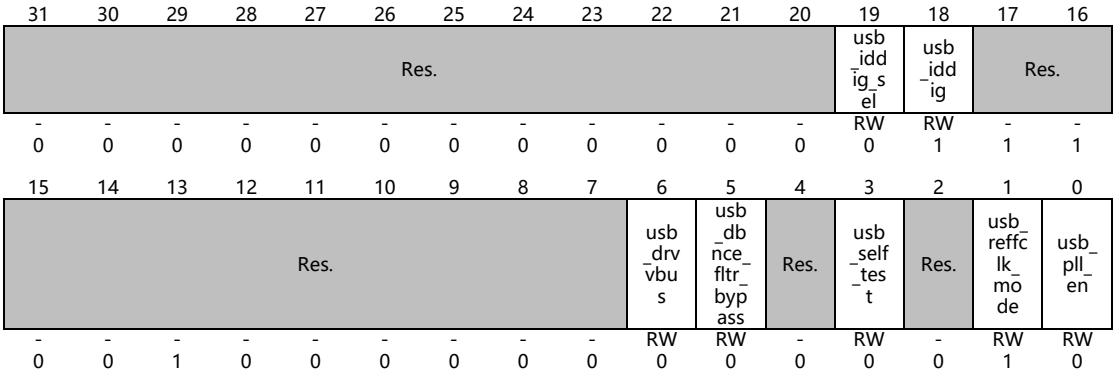
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.														dma1_soft_rst_req	dma0_soft_rst_req
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					qspi_mode_sel	spi1_mode_sel	spi0_mode_sel	Res.							
-	-	-	-	-	RW	RW	RW	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:18]	-	保留。	0x0
dma1_soft_rst_req	[17]	RW	dma1 软复位请求： 0：不复位； 1：复位。	0x0
dma0_soft_rst_req	[16]	RW	dma0 软复位请求： 0：不复位； 1：复位。	0x0
reserved	[15:11]	-	保留。	0x0
qspi_mode_sel	[10]	RW	qspi 模式选择： 0：兼容 SYNOPSYS/INTEL 芯片的 TI 模式； 1：兼容 ST 芯片的 TI 模式。	0x0
spi1_mode_sel	[9]	RW	spi1 模式选择： 0：兼容 SYNOPSYS/INTEL 芯片的 TI 模式； 1：兼容 ST 芯片的 TI 模式。	0x0
spi0_mode_sel	[8]	RW	spi0 模式选择： 0：兼容 SYNOPSYS/INTEL 芯片的 TI 模式； 1：兼容 ST 芯片的 TI 模式。	0x0
reserved	[7:0]	-	保留。	0x0

9.3.10 USB_CTRL

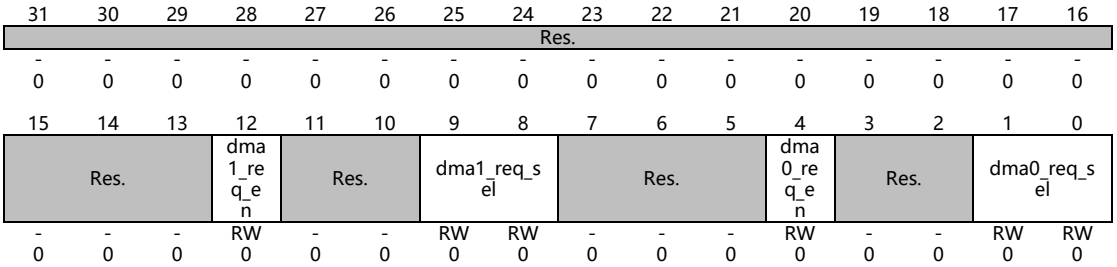
◆ **偏移地址：0x44　　复位值：0x00072002**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
usb_iddig_sel	[19]	RW	选择 HOST/DEVICE 模式的方式： 0：根据寄存器位 usb_iddig 配置识别； 1：根据 ID PIN 电平识别。	0x0
usb_iddig	[18]	RW	USB 模式选择： 0：USB 配置成 HOST； 1：USB 配置成 DEVICE。	0x1
reserved	[17:7]	-	保留。	0x0
usb_drvvbus	[6]	RW	用于控制 USBFS_DRVVBUS 管脚（PE1 管脚）的输出状态，从而控制 5V VBUS 电源开关： 0：USBFS_DRVVBUS 管脚输出低电平； 1：USBFS_DRVVBUS 管脚输出高电平。	0x0
usb_dbnce_fltr_bypass	[5]	RW	去抖动滤波器使能控制： 0：开启滤波器； 1：关闭滤波器。	0x0
reserved	[4]	-	保留。	0x0
usb_self_test	[3]	RW	PHY BIST 模式开关： 0：正常功能模式； 1：BIST 模式。	0x0
reserved	[2]	-	保留。	0x0
usb_reffclk_mode	[1]	RW	参考时钟选择： 0：25MHz； 1：12MHz。	0x1
usb_pll_en	[0]	RW	USB PHY PLL 使能控制： 0：PHY 的 PLL 状态由 USB 控制器自动控制； 1：关闭 PHY 的 PLL。	0x0

9.3.11 DMA_REQ_SEL

◆ **偏移地址：0x60+0x4*reqnum　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
dma1_req_en	[12]	RW	DMA1 req0~15 使能: 0：不使能; 1：使能。 注意:reqnum 表示 0~15。	0x0
reserved	[11:10]	-	保留	0x0
dma1_req_sel	[9:8]	RW	DMA1 req 复用关系选择。 复用关系请参考 DMAC 请求映射章节中的 DMA 请求映射关系表。	0x0
reserved	[7:5]	-	保留。	0x0
dma0_req_en	[4]	RW	DMA0 req0~15 使能: 0：不使能; 1：使能。 注意:reqnum 表示 0~15。	0x0
reserved	[3:2]	-	保留。	0x0
dma0_req_sel	[1:0]	RW	DMA0 req 复用关系选择。 复用关系请参考 DMAC 请求映射章节中的 DMA 请求映射关系表。	0x0

9.3.12 SC_RST_CNT0

◆ **偏移地址：0x100　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ext_rst_cnt															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
soft_rst_cnt															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ext_rst_cnt	[31:16]	RO	RESETN 管脚复位次数统计。	0x0
soft_rst_cnt	[15:0]	RO	软复位次数统计。	0x0

9.3.13 SC_RST_CNT1

◆ **偏移地址：0x104　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
wwdg_rst_cnt															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
iwdg_rst_cnt															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
wwdg_rst_cnt	[31:16]	RO	wwdg 复位次数统计。	0x0
iwdg_rst_cnt	[15:0]	RO	iwdg 复位次数统计。	0x0

9.3.14 VERSION

◆ **偏移地址：0x108　　复位值：0x20220920**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
version															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	1	0	0	0	0	0	0	0	1	0	0	0	1	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
version															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
version	[31:0]	RO	版本寄存器。	0x20220920

9.3.15 USER_REG0

◆ **偏移地址：0x200　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
user_reg0															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
user_reg0															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
user_reg0	[31:0]	RW	用户专用寄存器 0。系统复位可复位该寄存器，预留用户使用。	0x0

9.3.16 USER_REG1

◆ **偏移地址：0x204　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
user_reg1															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
user_reg1															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
user_reg1	[31:0]	RW	用户专用寄存器 1。系统复位可复位该寄存器，预留用户使用。	0x0

9.3.17 USER_HRST_REG0

◆ **偏移地址：0x208　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
user_hrst_reg0															
RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
user_hrst_reg0															
RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0

Name	Bits	R/W	Description	Default
user_hrst_reg0	[31:0]	RW_HRST	用户专用硬复位寄存器 0。硬复位可复位该寄存器，预留用户使用。	0x0

9.3.18 USER_HRST_REG1

◆ **偏移地址：0x20C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
user_hrst_reg1															
RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
user_hrst_reg1															
RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0	RW_HRST 0

Name	Bits	R/W	Description	Default
user_hrst_reg1	[31:0]	RW_HRST	用户专用硬复位寄存器 1。硬复位可复位该寄存器，预留用户使用。	0x0

9.3.19 USER_POR_REG0

◆ **偏移地址：0x210　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
user_por_reg0															
RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
user_por_reg0															
RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0	RW_POR 0

Name	Bits	R/W	Description	Default
user_por_reg0	[31:0]	POR_RW	用户专用上电复位寄存器 0。上电复位可复位该寄存器，预留用户使用。	0x0

9.3.20 USER_POR_REG1

◆ **偏移地址：0x214　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
user_por_reg1															
RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
user_por_reg1															
RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR	RW_POR
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
user_por_reg1	[31:0]	POR_RW	用户专用上电复位寄存器 1。上电复位可复位该寄存器，预留用户使用。	0x0

10 外设互联矩阵

10.1 外设互联矩阵简介

外设互联矩阵是一种硬件设计技术，它可以将多个外设之间直接连接起来，形成一个互联网络。每个外设都有一个或多个输入输出端口，通过这些端口可以与其他外设进行数据传输、同步信号传递等操作。相对于 CPU 或其他主控芯片的软件实现，外设互联矩阵具有以下优点：

- **节省 CPU 资源**：传统的方式是通过 CPU 管理所有外设之间的通信，但随着外设数量增加，软件处理的负担越来越重。使用外设互联矩阵可以让多个外设之间直接进行通信，减轻了 CPU 的负担，降低了系统功耗。
- **提高实时性**：由于外设互联矩阵是硬件实现的，因此其处理速度更快，响应时间更短。这使得外设之间的通信可以更加精确和及时，提高了系统的实时性能。
- **更好的可靠性**：外设互联矩阵可以提供更为可靠的数据传输和同步服务。它们可以实现数据的纠错、校验和自适应调节等功能，在传输过程中能够有效地保证数据的完整性和正确性。

总之，外设互联矩阵是一种高效、可靠的硬件设计技术，能够有效降低系统功耗，提高实时性能，并提供更为可靠的数据传输和同步服务。在众多嵌入式系统和电子设备中得到了广泛的应用。

10.2 连接汇总

表 10-1 互连关系图

目的 / 源端	GPT0	GPT1	GPT2	GPT3	APT0	APT1	ADC0~1
GPT0	-	10.3.1	10.3.1	10.3.1	10.3.1	10.3.1	10.3.3
GPT1	10.3.1	-	10.3.1	10.3.1	10.3.1	10.3.1	10.3.3
GPT2	10.3.1	10.3.1	-	10.3.1	10.3.1	10.3.1	10.3.3
GPT3	10.3.1	10.3.1	10.3.1	-	10.3.1	10.3.1	10.3.3
APT0	10.3.1	10.3.1	10.3.1	10.3.1	-	-	10.3.3
APT1	10.3.1	10.3.1	10.3.1	10.3.1	-	-	10.3.3
ADC0~1	10.3.1	10.3.1	10.3.1	10.3.1	10.3.1	10.3.1	-
CMP0~3	-	-	-	-	10.3.2	10.3.2	-
CPU	-	-	-	-	10.3.2	10.3.2	-
PVD	-	-	-	-	10.3.2	10.3.2	-
SRAM	-	-	-	-	10.3.2	10.3.2	-
EFC	-	-	-	-	10.3.2	10.3.2	-
CFD	-	-	-	-	10.3.2	10.3.2	-

说明

“10.3.X”：表示有互联关系，点击链接到对应章节；“-”：表示没有互联关系。

10.3 连接信息

10.3.1 GPT0~3、APT0、APT1、ADC 之间的相互同步

GPT0 可被 GPT1、GPT2、GPT3、APT0、APT1 同步，ADC 有 ETR（外部触发）信号到 GPT0。

GPT1 可被 GPT0、GPT2、GPT3、APT0、APT1 同步，ADC 有 ETR（外部触发）信号到 GPT1。

GPT2 可被 GPT0、GPT1、GPT3、APT0、APT1 同步，ADC 有 ETR（外部触发）信号到 GPT2。

GPT3 可被 GPT0、GPT1、GPT2、APT0、APT1 同步，ADC 有 ETR（外部触发）信号到 GPT3。

APT0 可被 GPT0、GPT1、GPT2、GPT3 同步，ADC 有 ETR（外部触发）信号到 APT0。

APT1 可被 GPT0、GPT1、GPT2、GPT3 同步，ADC 有 ETR（外部触发）信号到 APT1。

说明

GPTx 的同步功能参考 14.4.5 ETR/软件同步功能。APTx 的同步功能参考 16.4.6 SYNC_MUX 子模块。

10.3.2 CMP0~3、CPU、PVD、SRAM、EFC、CFD 到 APT0~1 的连接

CMP0~3、CPU、PVD、SRAM、EFC、CFD 到 APT0 的连接可控制 APT0 刹车。

CMP0~3、CPU、PVD、SRAM、EFC、CFD 到 APT1 的连接可控制 APT1 刹车。

说明

APTx 的刹车功能参考 16.4.4 Trip_Zone（TZ）子模块。

10.3.3 GPT0~3、APT0、APT1 到 ADC0~1 的连接

GPT0~3、APT0、APT1 到 ADC0 的连接可触发 ADC0 采样。

GPT0~3、APT0、APT1 到 ADC1 的连接可触发 ADC1 采样。

说明

触发 ADC 采样功能参考 32.4.12 ADC外部触发和触发极性的转换。

11 电源控制（PMC）

11.1 PMC 简介

PMC（Power Manage Controller）通过对芯片内部电源的管理，实现芯片的上/下电时序控制、开/关机功能和低功耗模式切换。

11.2 PMC 主要特性

PMC 具备以下特性：

- 上/下电时序控制；
- 低功耗模式切换；
- 支持 POR/PDR；
- 支持 PVD；
- 支持定时/IWDG/管脚唤醒；
- 管脚唤醒源选择；

11.3 电源

11.3.1 概述

芯片有 5 个电源，分别为：DVDD、VDDA、VAON、VCORE 和 VBAT。其中 DVDD、VDDA 和 VBAT 需要外部提供，VAON 和 VCORE 是芯片内部数字电源。每个电源的介绍如下：

- **DVDD = 2.4V~3.6V**

DVDD 是为内部调压器（ALDO、CLDO）、POR、PVD、PLL、HSE、HSI、LSI、BGR、BGR_PRE、EFLASH Macro 和 I/O 管脚供电的电源，通过 DVDD 管脚从外部提供。

- **VDDA = 2.4V ~3.6V**

VDDA 是为 ADC、DAC、CMP、PGA、电压参考缓冲器(VREF BUF)、温度传感器（Tsensor）、TSC、IBIAS 供电的模拟电源，通过 VDDA 管脚从外部提供。

- **VCORE≈ 1.1V**

VCORE 是为芯片内部 CORE 域（掉电域）供电的数字电源，为 CPU、数字外设（包含 UART、I2C、SPI、WWDG、I2S、GPIO、CFD、CMM、GPT、CAN、KEYSCAN、APT、QDM、CRC、TSC、SYSCTRL、CRG、IO_CFG、ANA_COM、QSPI、SDIO、USB、ADC_CTRL、DMA）、SRAM 和 EFLASH 供电。 VCORE 由嵌入式线性调压器 CLDO 供电。用户根据应用场景可选择进入深睡眠模式，系统会关闭 CLDO 供电。

- **VAON ≈ 1.1V**

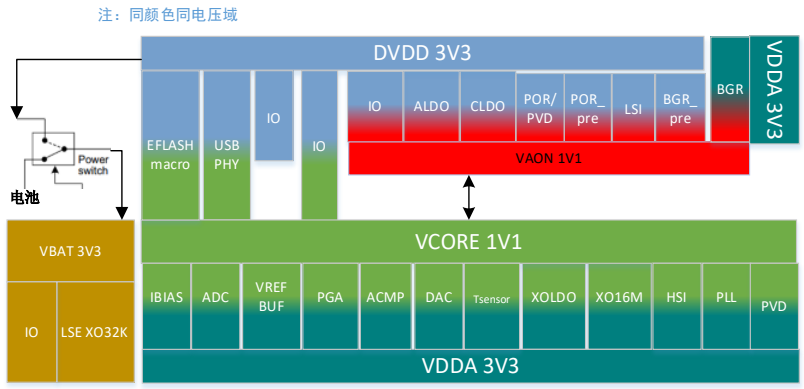
VAON 是为芯片内部 AON 域（常电域）供电的数字电源，为常电电路（PMC 与 IWDG）供电。VAON 由嵌入式线性调压器 ALDO 供电。AON 域由 LSI 提供工作时钟。用户根据应用场景可选择进入关机模式，系统会关闭 CLDO 与 ALDO 供电。

- **VBAT = 1.6V~3.6V**

VBAT 是为 BACKUP 域（备份域）供电的电源。当 DVDD 没有掉电时，VBAT 由 DVDD 供电；当 DVDD 掉电后，VBAT 会无缝切换到外部电池供电，BACKUP 域模块仍然可以正常工作。VBAT 模式详情可参考 29 备份寄存器（BACKUP）章节。

根据这 5 个电源，芯片划分了 5 个电源域，芯片电源域划分如图 11-1 所示。

图 11-1 芯片电源域示意图



11.3.2 电源监控

上电复位（POR）/掉电复位（PDR）

芯片具有一个集成的上电复位（POR）/掉电复位（PDR），在所有功耗模式下都有效。上电期间，POR 将使芯片保持复位状态，直到 DVDD 电源电压达到 POR 阈值。此时，将释放芯片复位信号并使能 ALDO 与 CLDO，系统可以启动。在掉电期间，DVDD 电源电压降至 PDR 阈值时，芯片再次被置于复位状态。

可编程电压检测器(PVD)

PVD 用于监视 DVDD 电源电压，将 PMU_CTRL.pvd_en 位置 1 即可使能 PVD。用户可配置 PMU_CTRL.pvd_sel 位域来选择监测电压阈值，在 PVD 使能的情况下，PVD 将 DVDD 电压值与监测电压阈值进行比较，当 DVDD 电压下降至监测电压阈值以下时，会产生 PVD 告警；当 DVDD 电压上升至监测电压阈值以上时，PVD 告警解除，可以通过读取 INT_PVD_STATE.int_pvd_hsi 位域获取 PVD 告警状态。用户可以在中断服务程序中执行紧急关闭系统的任务。

如果使能了 eflash 掉电保护功能，当 DVDD 降至 PVD 下降沿阈值时，将产生系统硬复位，可写 Option Byte 关闭 eflash 掉电保护功能（请参考 3.3.3 Option Byte 章节）。

⚠注意

建议打开芯片掉电保护功能，在 3.3V 电源供电的情况下需要满足：

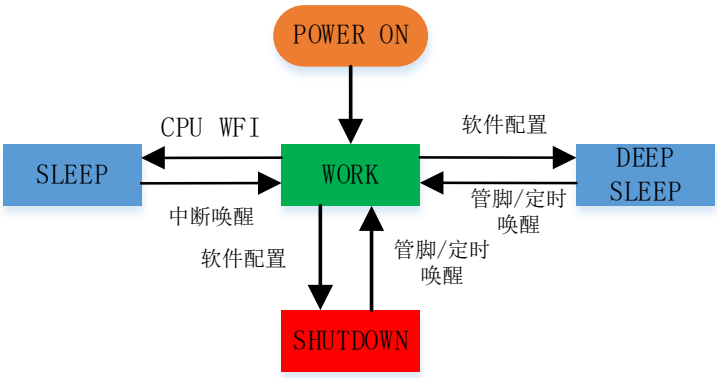
- PVD 档位设置为 6 或 7 档位；
- 从该档位电压下降到 2.2V 的时间大于等于 300us。

11.4 低功耗模式

11.4.1 概述

默认情况下，系统复位或上电复位后，芯片进入工作模式。系统提供了多个低功耗模式，可在 CPU 不需要运行时节省功耗。由用户根据应用选择具体的低功耗模式，以在低功耗、短启动时间和可用唤醒源之间寻求最佳平衡。如图 11-2 所示。

图 11-2 低功耗工作模式



WORK：工作模式。所有 LDO 供电，整芯片正常工作。

SLEEP：睡眠模式。CPU 时钟关闭，但所有外设都可以运行，并在发生中断时唤醒 CPU。

DEEPSLEEP：深睡眠模式。保留 ALDO 供电，CORE 域断电（即关闭 CLDO），此时 CORE 域中的所有时钟都会停止，PLL、HSI 和 HSE 振荡器被禁止。

SHUTDOWN：关机模式。仅保留 DVDD/VDDA 给芯片供电，所有的 LDO 关闭。

VBAT 模式：DVDD/VDDA 停止供电，仅保留备份域 RTC 工作。开启 DVDD/VDDA 电源后，系统按照上电流程重新启动。

此外，可通过下列方法之一降低运行模式下的功耗：

- 降低系统时钟速度。
- 不使用外设时，将对应的外设时钟关闭。

低功耗模式进入和退出条件、相关系统状态如下表 11-1 所示。

表 11-1 低功耗模式汇总

模式	进入条件	退出条件	唤醒系统时钟	对时钟的影响	电压
SLEEP	CPU 指令 WFI=1	任意中断	与进入前相同	CPU 时钟关闭，对其他任何时钟无影响	VCORE=1.1V
DEEPSLEEP	soft_deepsleep_req =1 + WFI	BACKUP 事件、RESET_N、IWDG 复位	HSI_16M	除 LSI 和 LSE 之外的所有时钟关闭	VCORE 关闭

模式	进入条件	退出条件	唤醒系统时钟	对时钟的影响	电压
SHUTDOWN	soft_shutdown_req=1 + WFI	BACKUP 事件、RESET_N	HSI_16M	除 LSI 和 LSE 之外的所有时钟关闭	除 VBAT 外，其他 LDO 全关闭

说明

BACKUP 事件包含 TAMPER 事件、WKUP 事件、TS 事件、WKT 事件、闹钟事件，详细参考 29 备份寄存器（BACKUP）章节。

表 11-2 低功耗模式下电源状态

模式	DVDD	VDDA	ALDO	CLDO	VBAT	CPU	时钟	Eflash	外设
WORK	on	on	on	on	on	on	任意	on	all
SLEEP	on	on	on	on	on	off	任意	on	all
DEEPSLEEP	on	on	on	down	on	down	LSI、LSE	down	RSET_N、备份域 IO
SHUTDOWN	on	on	down	down	on	down	LSI、LSE	down	RSET_N、备份域 IO

说明

down 表示掉电，off 表示关时钟。

11.4.2 工作模式

默认情况下，系统复位或上电复位后，芯片进入工作模式，系统主时钟由 HSI 提供。

降低系统时钟频率

在工作模式下，可以通过对 CRG 分频器配置（降低系统和外设的时钟频率）的方式减少系统功耗。在进入睡眠模式之前，也可以使用这些分频器降低外设频率。有关详细信息，请参考 4 时钟和复位控制（CRG)章节。

外设时钟门控

在工作模式下，可以关闭各外设的时钟来降低功耗。如需进一步降低睡眠模式的功耗，可在执行 WFI 指令之前关闭外设时钟。外设时钟门控由 CRG 寄存器控制。有关详细信息参考 4 时钟和复位控制（CRG)章节。

11.4.3 睡眠模式

CPU WFI 指令响应后，进入该模式。睡眠模式下，CPU 停止运行，处于低功耗状态（仅保留中断唤醒功能），除此之外与工作模式相同。如需进一步降低睡眠模式的功耗，可在进入睡眠模式之前通过配 CRG 相关寄存器降低系统时钟频率和关闭外设时钟。

睡眠模式下的 I/O 状态

在睡眠模式下，所有 I/O 管脚的状态与工作模式下相同。

进入睡眠模式

在工作模式下，当系统没有中断挂起且 CPU 执行 WFI 指令后，芯片将进入睡眠模式。

退出睡眠模式

任一未屏蔽的外设中断都能唤醒系统。

11.4.4 深睡眠模式

深睡眠模式下，CLDO 被关闭，CORE 域中的所有时钟都会停止，PLL、HSI 和 HSE 振荡器被禁止。所有寄存器（AON 域中的寄存器除外）内容和 SRAM 内容都将丢失。

深睡眠模式下的 I/O 状态

深睡眠模式下，RESET_N 管脚状态和备份域管脚状态与工作模式相同。其他 I/O 管脚状态为浮动输入。仅保留 ALDO 供电，CORE 域断电（即关闭 CLDO），此时，CORE 域中的所有时钟都会停止，PLL、HSI 和 HSE 振荡器被禁止。LSI/LSE 可以保持运行。可通过 RESET_N 管脚、IWDG、BACKUP 事件唤醒。

进入深睡眠模式

在工作模式下，配置 STATE_EN.state_en 位域为 1，配置 LOWPOWER_MODE。soft_deepsleep_req 位域置 1，等待芯片 CPU WFI 指令响应后，芯片将进入深睡眠模式。

退出深睡眠模式

芯片可以通过以下方式退出深睡眠模式：

- RESET_N 管脚触发：RESET_N 管脚持续低电平时间大于 100us。
- WKUP 事件或 TAMPER 事件触发：WKUP 事件或 TAMPER 事件属于 BACKUP 事件，可通过 WKUP 或 TAMPER 功能管脚唤醒深睡眠。芯片最大支持 3 个 WKUP 或 3 个 TAMPER 管脚，WKUP 与 TAMPER 管脚复用，可独立配置触发方式和是否使能，详细配置参考 29.3.5 入侵检测和 29.3.8 外部 WKUP 管脚章节。

- 闹钟触发：闹钟触发属于 BACKUP 事件，事件触发可唤醒深睡眠，闹钟的配置流程参考 30.4.4 闹钟功能使用流程相关章节。
- IWDG 复位触发：进入低功耗前，请参考 18 独立看门狗（IWDG)相关章节，使用 18.4 IWDG 典型配置流程进行配置，IWDG 复位触发唤醒深睡眠。

⚠注意

IWDG 复位产生时间需要大于系统进入深睡眠的消耗时间，即需要保证系统进入低功耗后 IWDG 复位才唤醒深睡眠。

- 定时唤醒器触发（WKT 事件）：定时唤醒器属于 BACKUP 事件，采用备份域时钟驱动的计数器计数到目标值产生触发信号，详细参考 29 备份寄存器（BACKUP）的 29.3.6 定时唤醒器章节。
- 时间戳触发（TS 事件）：TS 管脚输入事件属于 BACKUP 事件，事件产生时支持触发唤醒，软件可使能是否触发唤醒。详细参考 29 备份寄存器（BACKUP）的 29.3.7 时间戳章节。

11.4.5 关机模式

关机模式下，所有线性调压器都被关闭，仅保留 DVDD/VDDA 给芯片供电。所有寄存器内容和 SRAM 内容都将丢失。

关机模式下的 I/O 状态

关机模式下，RESET_N 管脚状态和备份域管脚状态与工作模式相同。其他 I/O 管脚状态为浮动输入。

进入关机模式

在工作模式下，配置 STATE_EN.state_en 位域为 1，配置 LOWPOWER_MODE.soft_shutdown_req 位域置 1，等待芯片 CPU WFI 指令响应后，芯片将进入关机模式。

退出关机模式

退出关机模式除了不支持 IWDG 唤醒方式外，其他唤醒方式与深睡眠模式唤醒方式一致，参考 11.4.4 深睡眠模式章节的退出深睡眠模式。

11.4.6 VBAT 模式

VBAT 为备份域进行供电，当 DVDD 停止供电时，启用 VBAT 供电，整芯片仅保留 RTC 工作。

VBAT 模式下的 I/O 状态

VBAT 模式下，LSE、WKUP/RTC 管脚状态与工作模式相同。其他 I/O 管脚状态为浮动输入。

进入 VBAT 模式

在任意低功耗模式下，DVDD 掉电，备份域自动切换到 VBAT 供电模式。

退出 VBAT 模式

进入 VBAT 模式后，芯片不可重新回到工作模式，需要重新上电，按照上电流程启动芯片。

说明

VBAT 模式仅用于备份域模块，关于 VBAT 的其他内容可参考 29 备份寄存器（BACKUP）章节。

11.5 PMC 寄存器概览

PMC 寄存器基地址如下：

- PMC 的寄存器基地址为：0x61005800

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	IWDG_CRG	0x4	32	RW	0x00000000	IWDG 配置寄存器
2	LOWPOWER_MODE	0x8	32	W1P	0x00000000	低功耗模式配置寄存器
3	STATE_EN	0xC	32	RW	0x00000000	低功耗使能配置寄存器
4	LOWPOWER_STATUS	0x10	32	RO	0x00000000	低功耗唤醒状态寄存器
5	LOWPOWER_STATUS_CLR	0x14	32	W1P	0x00000000	低功耗唤醒状态清除寄存器
6	PMU_CTRL	0x50	32	RW	0x00000000	PMU 控制寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
7	INT_PVD_STATE	0x80	32	RO	0x00000000	PVD 中断状态寄存器
8	INT_PVD_CLR	0x84	32	RW	0x00000000	PVD 中断清除寄存器
9	PMU_STATE	0x88	32	RO	0x00000000	PMU 状态寄存器
10	AON_USER_REG0	0x8C	32	RW	0x00000000	AON 电源域通用寄存器 0
11	AON_USER_REG1	0x90	32	RW	0x00000000	AON 电源域通用寄存器 1
12	AON_USER_REG2	0x94	32	RW	0x00000000	AON 电源域通用寄存器 2
13	AON_USER_REG3	0x98	32	RW	0x00000000	AON 电源域通用寄存器 3

11.6 PMC 寄存器描述

11.6.1 IWDG_CRG

◆ **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													iwdg_pr_div		
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													iwdg_srst_reg	iwdg_cken	Res.
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
iwdg_pr_div	[18:16]	RW	IWDG 计数时钟分频设置。 0：4 分频 1：8 分频 2：16 分频 3：32 分频 4：64 分频 5：128 分频 6：256 分频 7：禁止配置	0x0

Name	Bits	R/W	Description	Default
reserved	[15:3]	-	保留。	0x0
iwdg_srst_req	[2]	RW	iwdg 软复位请求。 0：软复位请求撤离 1：软复位请求 注意：需要写 1 再写 0，写 1 需要保持 32us 以上。	0x0
iwdg_cken	[1]	RW	iwdg 时钟门控使能设置。 0：时钟关闭 1：时钟开启	0x0
reserved	[0]	-	保留。	0x0

11.6.2 LOWPOWER_MODE

◆ **偏移地址：0x8　　复位值：0x00000000**

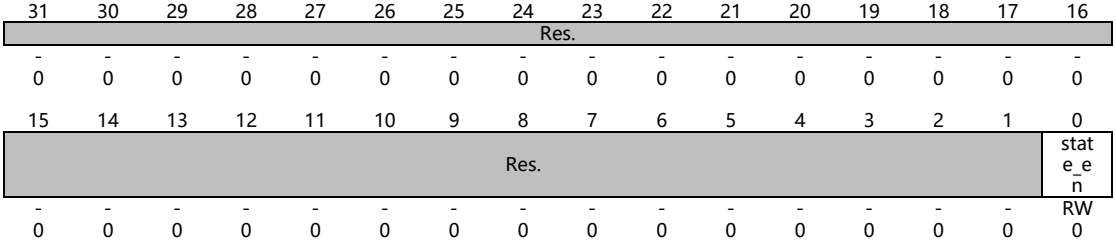
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													soft_deepsleep_req	Res.	soft_shutdown_req
-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	-	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
soft_deepsleep_req	[2]	W1P	系统进入 deepsleep 模式请求。 对此寄存器位域写 1 操作将触发芯片进入 deepsleep 模式流程。	0x0
reserved	[1]	-	保留。	0x0
soft_shutdown_req	[0]	W1P	系统进入 shutdown 模式请求。 对此寄存器位域写 1 操作将触发芯片进入 shutdown 模式流程。	0x0

11.6.3 STATE_EN

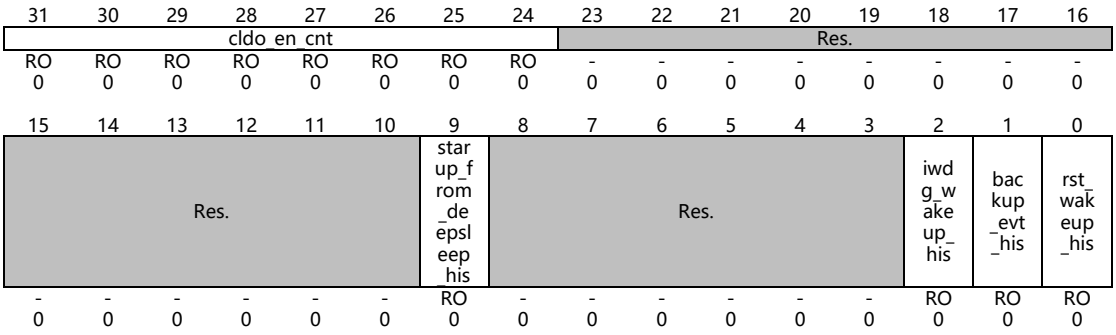
◆ **偏移地址：0xC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
state_en	[0]	RW	上电与低功耗使能 0：上电 1：低功耗 注意：进入 deepsleep 或 shutdown 模式前需要先配置该寄存器为 1。	0x0

11.6.4 LOWPOWER_STATUS

◆ **偏移地址：0x10　　复位值：0x00000000**

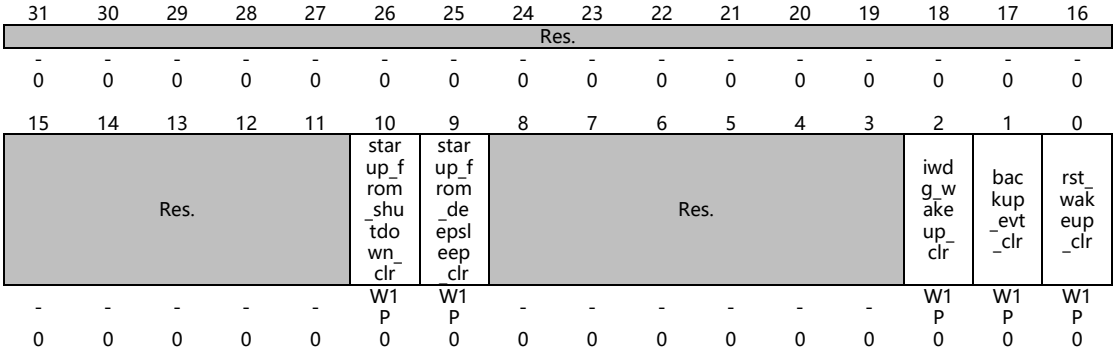


Name	Bits	R/W	Description	Default
cldo_en_cnt	[31:24]	RO	cldo en 关断次数统计。	0x0
reserved	[23:10]	-	保留。	0x0
starup_from_deepsleep_his	[9]	RO	从 deepsleep 状态启动标识历史值。	0x0
reserved	[8:3]	-	保留。	0x0
iwdg_wakeup_his	[2]	RO	IWDG 事件唤醒历史值，1 表示有效。	0x0
backup_evt_his	[1]	RO	BACKUP 事件唤醒历史值（wakup 或 rtc 或 tamp），1 表示有效。	0x0

Name	Bits	R/W	Description	Default
rst_wakeup_his	[0]	RO	RESETN 管脚唤醒历史值，1 表示有效。	0x0

11.6.5 LOWPOWER_STATUS_CLR

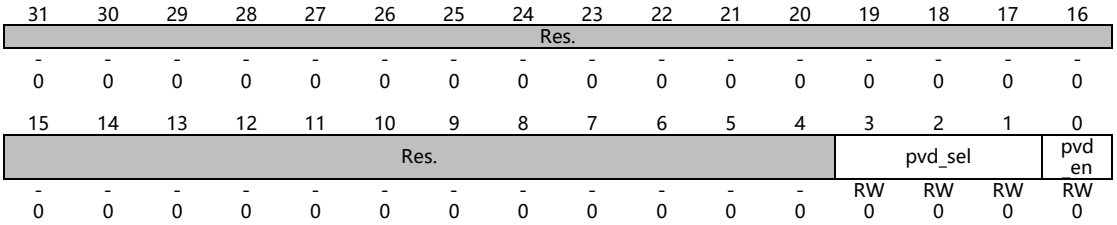
◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
starup_from_shutdown_clr	[10]	W1P	shutdown 低功耗状态清除，写 1 清。	0x0
starup_from_deepsleep_clr	[9]	W1P	deepsleep 低功耗状态清除，写 1 清。	0x0
reserved	[8:3]	-	保留。	0x0
iwdg_wakeup_clr	[2]	W1P	IWDG 事件唤醒历史值清除，写 1 清。	0x0
backup_evt_clr	[1]	W1P	BACKUP 事件唤醒历史值清除，写 1 清。	0x0
rst_wakeup_clr	[0]	W1P	RESETN 管脚唤醒历史值清除，写 1 清。	0x0

11.6.6 PMU_CTRL

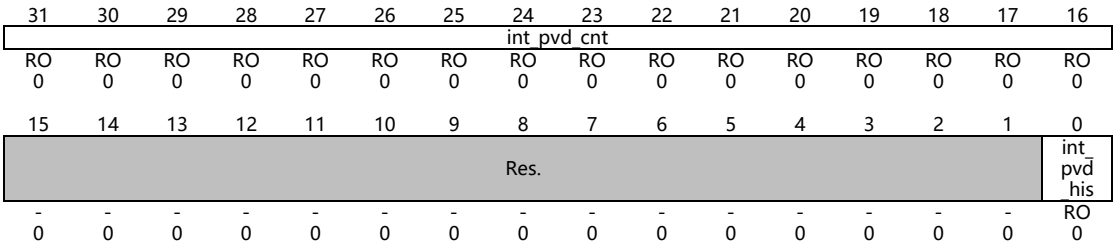
◆ **偏移地址：0x50　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
pvd_sel	[3:1]	RW	PVD 的档位选择信号。用于配置 DVDD 电压上升时或者下降时的监控电压档位。 0：上升沿 2.38V；下降沿 2.28V 1：禁止使用 2：禁止使用 3：上升沿 2.48V；下降沿 2.38V 4：上升沿 2.58V；下降沿 2.48V 5：上升沿 2.68V；下降沿 2.58V 6：上升沿 2.78V；下降沿 2.68V 7：上升沿 2.88V；下降沿 2.78V	0x0
pvd_en	[0]	RW	PVD 的使能控制信号。 0：不使能 1：使能	0x0

11.6.7 INT_PVD_STATE

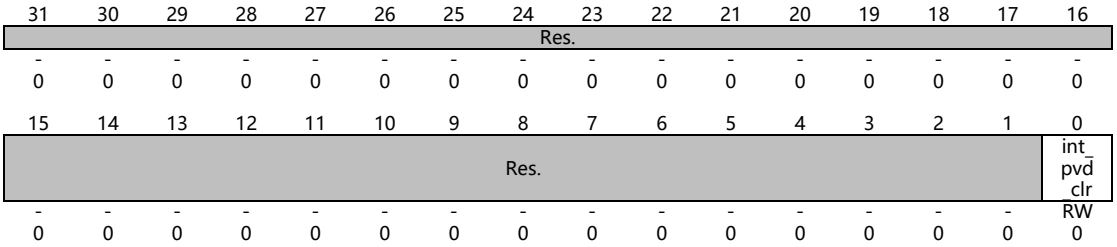
◆ **偏移地址：0x80　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
int_pvd_cnt	[31:16]	RO	PVD 中断次数统计。	0x0
reserved	[15:1]	-	保留。	0x0
int_pvd_his	[0]	RO	PVD 中断上报历史值。	0x0

11.6.8 INT_PVD_CLR

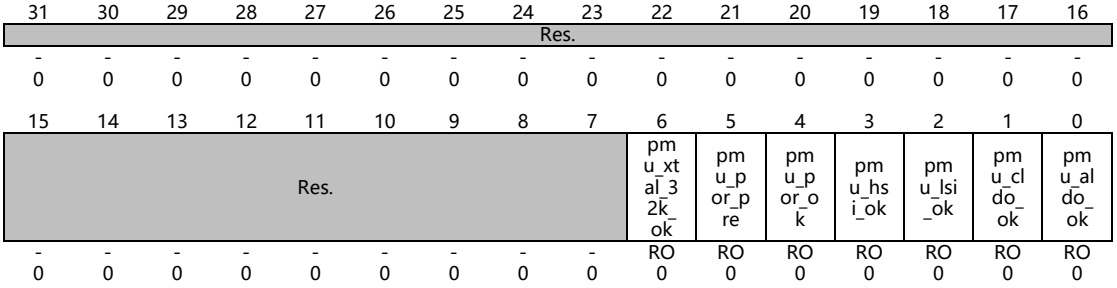
◆ **偏移地址：0x84　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
int_pvd_clr	[0]	RW	PVD 中断上报历史值写清（先写 1，然后延时 64us，再写 0）	0x0

11.6.9 PMU_STATE

◆ **偏移地址：0x88　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留。	0x0
pmu_xtal_32k_ok	[6]	RO	外部 32K 晶振稳定标识。 0：未稳定 1：已稳定	0x0
pmu_por_pre	[5]	RO	POR_PRE(低阈值 POR 信号)	0x0
pmu_por_ok	[4]	RO	PMU POR 上电稳定标识。 0：未稳定 1：已稳定	0x0
pmu_hsi_ok	[3]	RO	冷启动或 SHUTDOWN 唤醒 PMU HSI 16M 时钟稳定标识。 0：未稳定 1：已稳定	0x0

Name	Bits	R/W	Description	Default
pmu_lsi_ok	[2]	RO	PMU LSI 32K 时钟稳定标识。 0：未稳定 1：已稳定	0x0
pmu_cldo_ok	[1]	RO	PMU CLDO 电压稳定标识。 0：未稳定 1：已稳定	0x0
pmu_aldo_ok	[0]	RO	PMU ALDO 电压稳定标识。 0：未稳定 1：已稳定	0x0

11.6.10 AON_USER_REG0

◆ **偏移地址：0x8C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
aon_user_reg0															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
aon_user_reg0															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
aon_user_reg0	[31:0]	RW	AON 电源域通用寄存器 0。未控制任何逻辑，给用户预留。	0x0

11.6.11 AON_USER_REG1

◆ **偏移地址：0x90　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
aon_user_reg1															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
aon_user_reg1															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
aon_user_reg1	[31:0]	RW	AON 电源域通用寄存器 1。未控制任何逻辑，给用户预留。	0x0

11.6.12 AON_USER_REG2

◆ **偏移地址：0x94　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
aon_user_reg2															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
aon_user_reg2															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
aon_user_reg2	[31:0]	RW	AON 电源域通用寄存器 2。未控制任何逻辑，给用户预留。	0x0

11.6.13 AON_USER_REG3

◆ **偏移地址：0x98　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
aon_user_reg3															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
aon_user_reg3															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
aon_user_reg3	[31:0]	RW	AON 电源域通用寄存器 3。未控制任何逻辑，给用户预留。	0x0

12 循环冗余校验计算单元（CRC）

12.1 CRC 简介

CRC 是循环冗余校验计算单元的简称，根据传输的数据通过复杂的计算产生一个数。发送设备在发送数据前进行这个计算，然后将结果发送给接收设备。接收设备在接收后，重复同样的运算，如果两个设备的运算结果相同，就认为传输无误。

CRC 校验通常用来校验数据传输或者存储过程的完整性，是数据通信领域最常用的一种差错校验码。

本芯片包含一个 CRC 循环冗余校验计算单元，支持 1 种算法多项式，其中 CRC 初始值固定为 0xFFFFFFFF，可以被使用以产生传输过程中的 CRC 码值或接收输入的 CRC 码值进行校验。

12.2 CRC 主要特性

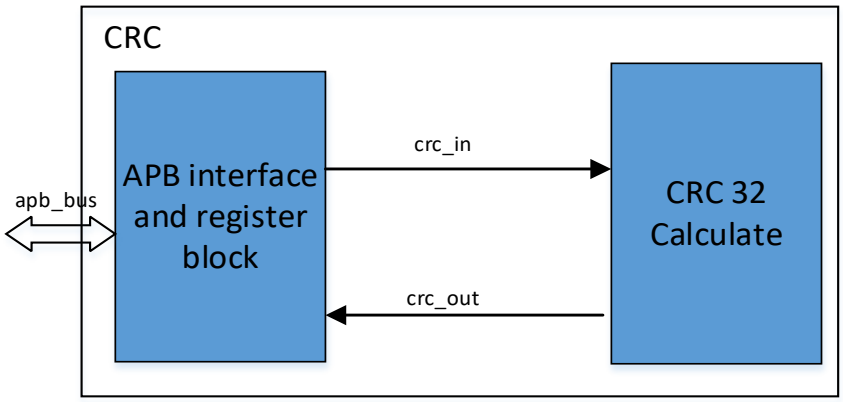
CRC 模块有以下特点：

- 支持 CRC32-0x04C11DB7 算法多项式。
- 输入数据有效位宽为 32bit。

12.3 CRC 功能描述

CRC 模块整体框如图 12-1 所示。

图 12-1 CRC 结构图



CRC 模块算法多项式、初值等参数固化，通过总线读出 CRC 的计算码值。多项式参数参考表 12-1。

表 12-1 多项式参数

CRC 参数	CRC32-0x04C11DB7
宽度	32
初始值	0xFFFFFFFF
结果异或值	0xFFFFFFFF

12.3.1 CRC 典型配置

操作步骤如下：

步骤 1 配置 CRC_EN 寄存器为 1，使能 CRC。

步骤 2 重复写数据到 CRC_IN 寄存器，向 CRC 输入数据，直到达到 CRC 计算长度。如果最后一个数据不足一个 word，则高 bit 需要写 0 。

步骤 3 读取 CRC_OUT 寄存器，获取 CRC 计算结果。

步骤 4 配置 CRC_EN 寄存器为 0，关闭 CRC。

12.4 CRC 寄存器概览

CRC 基地址如下：

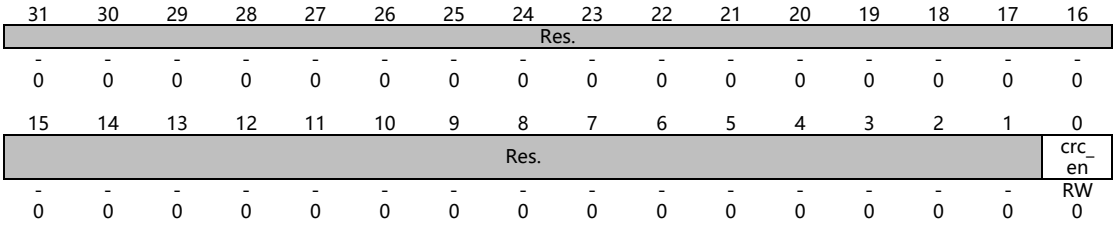
- CRC 寄存器基地址为：0x61002000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CRC_EN	0x0	32	RW	0x00000000	CRC 使能配置
2	CRC_IN	0x4	32	RW	0x00000000	CRC 输入数据
3	CRC_OUT	0xC	32	RO	0x00000000	CRC 计算结果

12.5 CRC 寄存器描述

12.5.1 CRC_EN

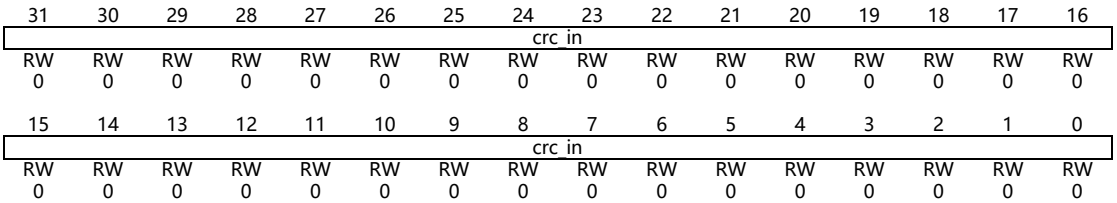
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
crc_en	[0]	RW	CRC 使能配置： 0：禁止 1：使能 注意：一次完整的 crc 计算从使能有效开始计算，到使能无效时结束。	0x0

12.5.2 CRC_IN

◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
crc_in	[31:0]	RW	CRC 输入 32bit 数据。	0x0

Name	Bits	R/W	Description	Default
			注意：如果最后一个输入数据不足一个 word，则高位无效位需要置 0。	

12.5.3 CRC_OUT

◆ **偏移地址：0xC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
crc_out															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
crc_out															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
crc_out	[31:0]	RO	CRC 计算结果。	0x0

13 直接存储器访问控制器（DMAC）

13.1 DMAC 简介

DMA（Direct Memory Access）是一种可以不通过 CPU，直接访问存储器数据的数据传输操作。DMA 控制器（DMAC）以该种传输方式控制实现 Peripheral to Memory、Memory to Peripheral、Memory to Memory 之间的直接数据传输，减轻 CPU 数据处理负担并减少 CPU 中断处理开销。

13.2 DMAC 主要特性

本芯片支持两个 DMAC 模块，分别为 DMAC0 和 DMAC1。DMAC 主要特性如下：

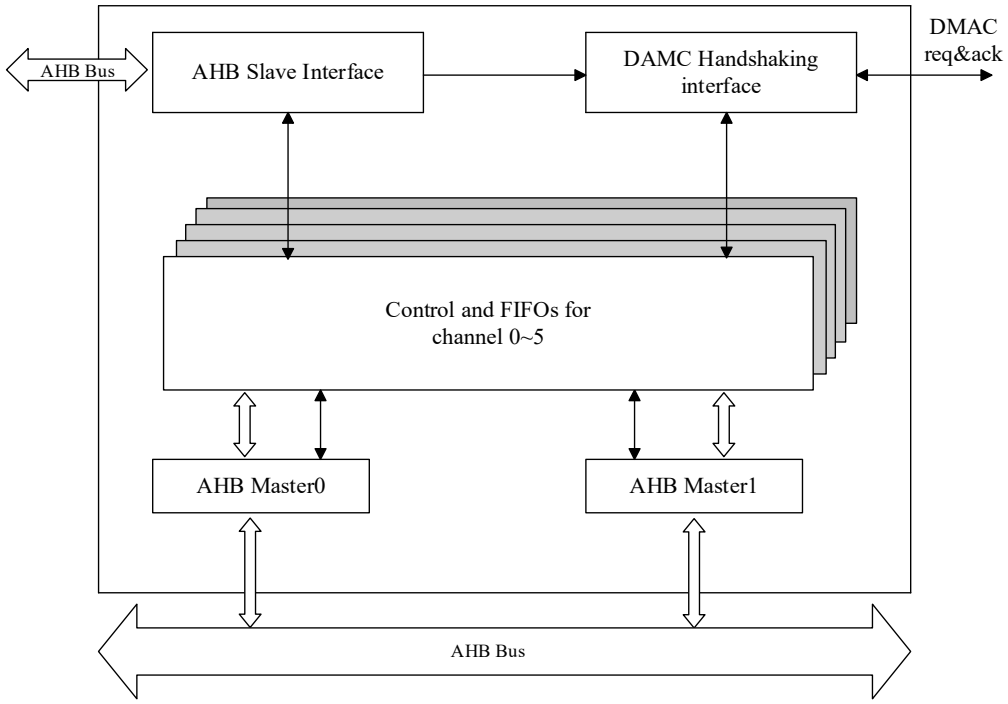
- 支持 6 通道，各通道独立配置。
- 提供 2 个位宽为 32bit 的 Master 接口用于数据传输。
- 支持 8bit、16bit、32bit 数据传输位宽。
- 支持硬件 DMA 请求和软件 DMA 请求。
- 支持软件配置 DMA 传输的 Burst Length 大小。
- 支持软件配置源端地址和目的端地址为自增或不自增。
- 支持循环模式、链表模式。
- 支持 gather、scatter 模式。
- 支持 DMAC 流控。

13.3 DMAC 逻辑实现

13.3.1 DMAC 逻辑框图

DMAC 逻辑框图如图 13-1 所示。

图 13-1 DMAC 逻辑框图



说明

DMAC 通道优先级可由软件配置，默认从低到高依次为 0~5，且该优先级是指 DMAC 通道访问 Master 的优先级。

DMAC 的每一个通道均含有传输控制逻辑和 FIFO，传输控制逻辑根据配置自动完成以下过程：

步骤 1 根据通道流控配置，当满足开始传输条件，则进行不同通道之间的优先级仲裁，优先级高的通道先开始传输。

步骤 2 从源端地址读取数据。

步骤 3 数据缓存到内部 FIFO 中。

步骤 4 从 FIFO 中取出数据。

步骤 5 数据写入目的端地址，上报传输状态及相关中断。

13.3.2 DMAC 工作/配置流程

详细流程如下：

步骤 1 确认软件需要使用的 DMAC 通道，并配置该通道相关信息，源端起始地址（SAR）、目的端起始地址（DAR）、通道控制寄存器（CTL_L 和 CTL_H）、通道配置寄存器（CFG_L 和 CFG_H）等寄存器，以明确传输的源地址、目的地址、链表使能及链表头指针、待传输数据个数、传输模式（Memory to Memory、Memory to Peripheral、Peripheral to Memory）、源端和目的端需响应的 DMA 请求、burst length、源端和目的端使用的 Master、流控模式、开启通道使能等信息，等待 DMA 请求以开始传输。

步骤 2 源设备向 DMAC 发起 DMA 请求，DMAC 响应其请求，完成一次 single/burst 传输。（需知：如果源设备为外设，则需要选择硬件握手接口，如果源设备为存储器，则需要选择软件握手接口；握手接口选择详见 CFGx.hs_sel_src，同时需要选择需要被响应的 DMA 请求线，详见 CFG_H.src_per；且支持选择 DMA 请求有效电平，详见 CFG_L.src_hs_pol，默认有效电平为高电平）

步骤 3 从源端获取的数据会缓存在通道 FIFO 中。

步骤 4 目的设备发起 DMA 请求，DMAC 响应其请求，完成一次 single/burst 传输。（需知：如果目的设备为外设，则需要选择硬件握手接口，如果目的设备是存储器，则需要选择软件握手接口，详见 CFG_L.hs_sel_dst，同时需要选择需要被响应的 DMA 请求线，详见 CFG_H.dst_per）

步骤 5 从 FIFO 中拿出数据写入目的设备。

步骤 6 流程 2、3 和流程 4、5 可能是并行执行的，取决于源端设备和目的端设备发起请求的时机以及源端和目的端 Master 选择。且流程 2、3 和流程 4、5 是在重复进行的，直到分别完成源端和目的端所有待传数据（CTL_H.block_ts）的传输，就会发出传输完成中断及状态（中断可屏蔽）。当出现 DMAC 通道 FIFO 被写满且目的设备的 DMA 请求仍未产生，或者 DMAC 通道 FIFO 被读空且源端设备的 DMA 请求仍未产生，DMAC 自动阻塞源端或者目的端设备的 DMA 请求，直到 FIFO 结束空满状态。

步骤 7 链表模式使能开启情况下，DMAC 将自动从链表指针获取下一次传输所需要的配置信息，并完成传输，详见寄存器 CTL_L.llp_src_en、CTL_L.llp_dst_en、LLP（需知：链表模式未使能，则只重复执行 2-5，跳过 7）。

13.3.3 DMAC 请求映射

DMAC 请求支持 16bit 位宽，支持 burst request 和 single request 两种请求；DMAC 请求的位宽是有限的，同时系统内 DMA 请求数量很多，就需要通过 DMAMUX 实现 DMA 请求的分时复用。详细映射关系见表 13-1。映射关系配置寄存器参考 9 系统控制器(SYSCTRL)章节 9.3.11 DMA_REQ_SEL 寄存器。

表 13-1 DMA 请求映射关系

DMA 请求编号	寄存器描述	DMA0 对应外设	DMA1 对应外设
0	dma_req0_sel[1:0]=0	UART0 Tx 通道 DMA 请求	UART3 Tx 通道 DMA 请求
	dma_req0_sel[1:0]=1	UART1 Tx 通道 DMA 请求	I2C0 Tx 通道 DMA 请求
	dma_req0_sel[1:0]=2	QSPI Tx 通道 DMA 请求	QSPI Tx 通道 DMA 请求
	dma_req0_sel[1:0]=3	ADC0 DMA 请求	ADC0 DMA 请求
1	dma_req1_sel[1:0]=0	UART0 Rx 通道 DMA 请求	UART3 Rx 通道 DMA 请求
	dma_req1_sel[1:0]=1	UART1 Rx 通道 DMA 请求	I2C0 Rx 通道 DMA 请求
	dma_req1_sel[1:0]=2	QSPI Rx 通道 DMA 请求	QSPI Rx 通道 DMA 请求
	dma_req1_sel[1:0]=3	ADC1 DMA 请求	ADC1 DMA 请求
2	dma_req2_sel[1:0]=0	UART0 Tx 通道 DMA 请求	UART3 Tx 通道 DMA 请求
	dma_req2_sel[1:0]=1	UART1 Tx 通道 DMA 请求	UART4 Tx 通道 DMA 请求
	dma_req2_sel[1:0]=2	UART2 Tx 通道 DMA 请求	I2C1 Tx 通道 DMA 请求
	dma_req2_sel[1:0]=3	I2S0 CH0 通道 DMA 请求	I2S0 CH0 Tx 通道 DMA 请求
3	dma_req3_sel[1:0]=0	UART0 Rx 通道 DMA 请求	UART3 Rx 通道 DMA 请求
	dma_req3_sel[1:0]=1	UART1 Rx 通道 DMA 请求	UART4 Rx 通道 DMA 请求
	dma_req3_sel[1:0]=2	UART2 Rx 通道 DMA 请求	I2C1 Rx 通道 DMA 请求
	dma_req3_sel[1:0]=3	I2S0 CH0 Rx 通道 DMA 请求	I2S0 CH0 Rx 通道 DMA 请求
4	dma_req4_sel[1:0]=0	UART2 Tx 通道 DMA 请求	UART4 Tx 通道 DMA 请求
	dma_req4_sel[1:0]=1	SPI0 Tx 通道 DMA 请求	SPI0 Tx 通道 DMA 请求

DMA 请求编号	寄存器描述	DMA0 对应外设	DMA1 对应外设
	dma_req4_sel[1:0]=2	I2S0 CH0 Tx 通道 DMA 请求	I2S0 CH0 Tx 通道 DMA 请求
	dma_req4_sel[1:0]=3	ADC0 DMA 请求	ADC0 DMA 请求
5	dma_req5_sel[1:0]=0	UART2 Rx 通道 DMA 请求	UART4 Rx 通道 DMA 请求
	dma_req5_sel[1:0]=1	SPI0 Rx 通道 DMA 请求	SPI0 Rx 通道 DMA 请求
	dma_req5_sel[1:0]=2	I2S0 CH0 Rx 通道 DMA 请求	I2S0 CH0 Rx 通道 DMA 请求
	dma_req5_sel[1:0]=3	ADC1 DMA 请求	ADC1 DMA 请求
6	dma_req6_sel[1:0]=0	QSPI Tx 通道 DMA 请求	QSPI Tx 通道 DMA 请求
	dma_req6_sel[1:0]=1	SPI1 Tx 通道 DMA 请求	SPI1 Tx 通道 DMA 请求
	dma_req6_sel[1:0]=2	I2S0 CH1 Tx 通道 DMA 请求	I2S0 CH1 Tx 通道 DMA 请求
	dma_req6_sel[1:0]=3	CAPM CH0 DMA 请求	CAPM CH0 DMA 请求
7	dma_req7_sel[1:0]=0	QSPI Rx 通道 DMA 请求	QSPI Rx 通道 DMA 请求
	dma_req7_sel[1:0]=1	SPI1 Rx 通道 DMA 请求	SPI1 Rx 通道 DMA 请求
	dma_req7_sel[1:0]=2	I2S0 CH1 Rx 通道 DMA 请求	I2S0 CH1 Rx 通道 DMA 请求
	dma_req7_sel[1:0]=3	CPAM CH1DMA 请求	CPAM CH1DMA 请求
8	dma_req8_sel[1:0]=0	I2C0 Tx 通道 DMA 请求	I2C0 Tx 通道 DMA 请求
	dma_req8_sel[1:0]=1	SPI0 Tx 通道 DMA 请求	SPI0 Tx 通道 DMA 请求
	dma_req8_sel[1:0]=2	I2S0_CH2 Tx 通道 DMA 请求	I2S0_CH2 Tx 通道 DMA 请求
	dma_req8_sel[1:0]=3	CPAM CH2 DMA 请求	CPAM CH2 DMA 请求
9	dma_req9_sel[1:0]=0	I2C0 Rx 通道 DMA 请求	I2C0 Rx 通道 DMA 请求

DMA 请求编号	寄存器描述	DMA0 对应外设	DMA1 对应外设
	dma_req9_sel[1:0]=1	SPI0 Rx 通道 DMA 请求	SPI0 Rx 通道 DMA 请求
	dma_req9_sel[1:0]=2	I2S0 CH2 Rx 通道 DMA 请求	I2S0 CH2 Rx 通道 DMA 请求
	dma_req9_sel[1:0]=3	CPAM CH3 DMA 请求	CPAM CH3 DMA 请求
10	dma_req10_sel[1:0]=0	I2C0 Tx 通道 DMA 请求	UART0 Tx 通道 DMA 请求
	dma_req10_sel[1:0]=1	I2C1 Tx 通道 DMA 请求	I2C1 Tx 通道 DMA 请求
	dma_req10_sel[1:0]=2	SPI11 Tx 通道 DMA 请求	SPI1 Tx 通道 DMA 请求
	dma_req10_sel[1:0]=3	I2S0_CH1 Tx 通道 DMA 请求	I2S0_CH1 Tx 通道 DMA 请求
11	dma_req11_sel[1:0]=0	I2C0 Rx 通道 DMA 请求	UART0 Rx 通道 DMA 请求
	dma_req11_sel[1:0]=1	I2C1 Rx 通道 DMA 请求	I2C1 Rx 通道 DMA 请求
	dma_req11_sel[1:0]=2	SPI1 Rx 通道 DMA 请求	SPI1 Rx 通道 DMA 请求
	dma_req11_sel[1:0]=3	I2S0_CH1 Rx 通道 DMA 请求	I2S0_CH1 Rx 通道 DMA 请求
12	dma_req12_sel[2:0]=0	UART3 Tx 通道 DMA 请求	UART1 Tx 通道 DMA 请求
	dma_req12_sel[2:0]=1	I2S1 Tx 通道 DMA 请求	I2S1 Tx 通道 DMA 请求
	dma_req12_sel[2:0]=2	I2S0_CH2 Tx 通道 DMA 请求	I2S0_CH2 Tx 通道 DMA 请求
	dma_req12_sel[2:0]=3	DAC0 通道 DMA 请求	DAC0 通道 DMA 请求
13	dma_req13_sel[2:0]=0	UART3 Rx 通道 DMA 请求	UART1 Rx 通道 DMA 请求
	dma_req13_sel[2:0]=1	I2S1 Rx 通道 DMA 请求	I2S1 Rx 通道 DMA 请求
	dma_req13_sel[2:0]=2	I2S0_CH2 Rx 通道 DMA 请求	I2S0_CH2 Rx 通道 DMA 请求

DMA 请求编号	寄存器描述	DMA0 对应外设	DMA1 对应外设
	dma_req13_sel[2:0]=3	DAC1 通道 DMA 请求	DAC1 通道 DMA 请求
14	dma_req14_sel[2:0]=0	UART4 Tx 通道 DMA 请求	UART2 Tx 通道 DMA 请求
	dma_req14_sel[2:0]=1	I2C1 Tx 通道 DMA 请求	I2S1 Tx 通道 DMA 请求
	dma_req14_sel[2:0]=2	I2S1 Tx 通道 DMA 请求	QSPI Tx 通道 DMA 请求
	dma_req14_sel[2:0]=3	DAC2 通道 DMA 请求	DAC2 通道 DMA 请求
15	dma_req15_sel[2:0]=0	UART4 Rx 通道 DMA 请求	UART2 Rx 通道 DMA 请求
	dma_req15_sel[2:0]=1	I2C1 Rx 通道 DMA 请求	I2S1 Rx 通道 DMA 请求
	dma_req15_sel[2:0]=2	I2S1 Rx 通道 DMA 请求	QSPI Rx 通道 DMA 请求
	dma_req15_sel[2:0]=3	ADC0 通道 DMA 请求	ADC 1 通道 DMA 请求

13.3.4 DMAC 流控制器及传输类型

流控制器指控制传输数据长度的设备。在传输配置中必须指定 DMAC、源端设备、目的端设备其中之一作为流控制器。DMAC 作为流控制器时，需要配置寄存器 CTL_H.BLOCK_TS 位域。

传输类型支持 Memory to Memory、Memory to Peripheral、Peripheral to Memory。DMAC 每个通道均支持 DMAC 流控和外设流控。需要注意的是，芯片内集成的外设都不支持外设流控。表 13-2 说明了传输类型与流控选择的组合情况。

说明

存储器指的是地址空间映射的 RAM 区域，以及地址空间映射的外设寄存器。

部分外设未设置 DMA 硬件请求线，但外设相关寄存器映射到地址空间（即 DMAC 可以寻址访问），那么这些外设寄存器可以视为存储器被访问的。

表 13-2 传输类型与流控选择

CTL_L.TT_FC 位域（二进制值）	传输类型	流控制器
000	Memory to Memory	DMAC
001	Memory to Peripheral	DMAC
010	Peripherl to Memory	DMAC

DMAC 作为流控制器时，软件需要在 DMAC 侧配置 CTL_H.block_ts 位域和 CTL_L.tt_fc 位域，以指定传输数据长度、传输类型和 DMAC 流控模式。当外设作为流控制器时，DMAC 侧是不知道传输数据长度，所以外设需要在传输结束时反馈结束信号给 DMAC。

13.4 DMAC 功能描述

13.4.1 DMAC 握手接口

DMAC 握手接口在传输级控制 single/burst 传输，完成一次握手即完成一次 single/burst 传输。

DMAC 握手接口：

- 硬件握手
- 软件握手

硬件握手接口是一套硬件接口，有请求信号及应答信号。软件握手接口无硬件信号，通过映射内存寄存器完成。硬件握手接口根据外设是否是流控制器有相应的不同。

⚠注意

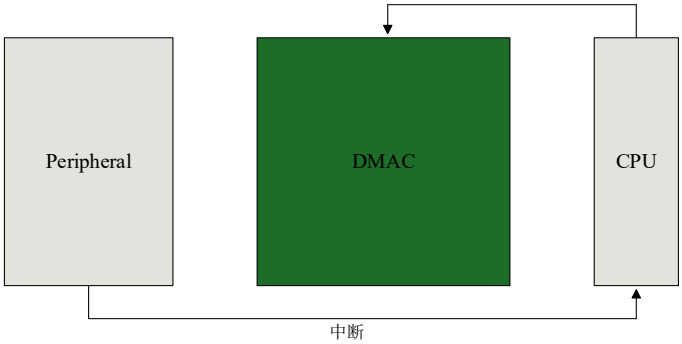
源端和目的端的握手接口是可以独立配置选择的；详见 CFG_L 寄存器的 CFG_L.hs_sel_src 和 CFG_L.hs_sel_dst。

13.4.1.1 DMAC 软件握手

软件握手即通过映射内存寄存器完成握手，需要 CPU 通过软件触发 DMA 进行传输。以下是中断触发示例。

软件握手框图如图 13-2 所示。

图 13-2 软件握手框图



软件握手接口寄存器：

- ReqSrcReg 源端软件传输请求寄存器
- ReqDstReg 目的端软件传输请求寄存器
- SglRqSrcReg 源端软件 single 传输请求寄存器
- SglRqDstReg 目的端软件 single 传输请求寄存器
- LstSrcReg 源端软件最后一次传输请求寄存器
- LstDstReg 目的端软件最后一次传输请求寄存器

说明

软件握手接口下，软件程序配置 DMAC 的请求寄存器，并开启通道使能；传输类型及流控选择依据使用场景配置即可。

需注意的是软件握手不需要硬件请求信号，如果传输类型为 Memory to Memory，其数据准备就绪，打开 DMAC 通道使能就直接可以开始传输数据。所以对于无硬件请求信号的外设可以选择软件握手实现数据传输，也可以直接通过 Memory to Memory 的方式实现传输。

配置举例：

步骤 1 源端或目的端为外设，且采用 DMAC 流控，需配置 CTL_L.tt_fc 选择为 Peripheral to Memory 或者 Memory to Peripheral。

步骤 2 配置 CTL_H.BLOCK_TS。

步骤 3 配置源端和目的端传输位宽，CTL_L.src_tr_width、CTL_L.dst_tr_width。

步骤 4 配置源端和目的端传输 burst length，CTL_L.src_msize、CTL_L.dest_msize。

步骤 5 配置源端或目的端握手接口为软件握手接口，CFG_L.hs_sel_src、CFG_L.hs_sel_dst。

步骤 6 当源端外设数据缓存达到 FIFO 水线（比如 SPI 外设）则发出中断，在中断任务中配置 DMAC 的软件请求寄存器（ReqSrcReg 和 SglRqSrcReg 均要配置），以触

发 DMAC 搬移源端设备数据到目的端；完成本次握手 DMAC 发出传输完成中断。

步骤 7 当目的端外设 FIFO 数据低于水线则发出中断，同样在中断任务中配置 DMAC 的软件请求寄存器（ReqDstReg 和 SglRqDstReg 均要配），以触发 DMAC 搬移数据到目的端设备；完成本次握手 DMAC 发出传输完成中断。

步骤 8 重复步骤 6（源端为外设）或者步骤 7（目的端为外设）直到完成全部 BLOCK_TS 数据传输。

⚠注意

上述流程只涉及 DMAC 侧的配置，外设的配置视具体外设而定。

13.4.1.2 DMAC 硬件握手

外设发起请求，DMAC 完成数据搬移后应答为一次完整握手；硬件握手接口信号如表 13-3 所示。

表 13-3 接口信号描述

信号	方向	信号描述
dma_ack	Output	DMAC 对外设的应答信号：dma_ack 在完成本次握手中最后的 AHB 传输（single 或 burst）后置 1；在 single 传输中，dma_ack 在外设的 dma_single 信号无效后的下一时钟周期置 0；在 burst 传输中，dma_ack 在外设的 dma_req 信号无效后的下一时钟周期置 0.
dma_finish	Output	DMAC 对外设的完成一次块传输的应答信号：dma_finish 在完成一次块传输（即搬移完成一次 CTLx.BLOCK_TS）后置 1；在最后一次外设的 dma_single 或 dma_burst 信号无效后的下一时钟周期置 0；
dma_last	Input	块传输的最后一次传输指示信号：当外设为流控制器时，该信号表示本次传输为块传输的最后一次传输；该信号与最后一次的 dma_single 或者 dma_req 同时有效，持续一个时钟周期后置 0；DMAC 完成本次请求/应答握手即停止传输；当 DMAC 为流控制器时，该信号被忽略；
dma_req	Input	外设的 burst 传输请求：外设向 DMAC 发起的 burst 传输请求信号，该信号有效时，不论 dma_single 是何种值，均进行 burst 传输；该信号一经发起，要持续到 DMAC 的应答信号 dma_ack 有效的下一时钟周期才会置 0.一旦 dma_req 置 0，DMAC 也将 dma_ack 置 0，即完成握手。
dma_single	Input	外设的 single 传输请求：外设向 DMAC 发起的 single 传输

信号	方向	信号描述
		请求信号，该信号有效时，DMAC 完成一次 single 数据传输；该信号一经发起，要持续到 DMAC 的应答信号 dma_ack 有效后的下一时钟周期才会置 0。一旦 dma_single 置 0，DMAC 也将 dma_ack 置 0，即完成握手。

硬件握手接口由外设是否是流控制器而有所区别，图 13-3 和图 13-4 分别展示了外设不作为流控制器和作为流控制器的接口结构。非外设流控，则接口只关注 DMAC 请求信号和 DMAC 的应答信号；外设流控，则还需要外设的最后的请求信号以告知 DMAC 传输的是最后的数据。接口握手时序如图 13-5 所示。

图 13-3 非外设流控

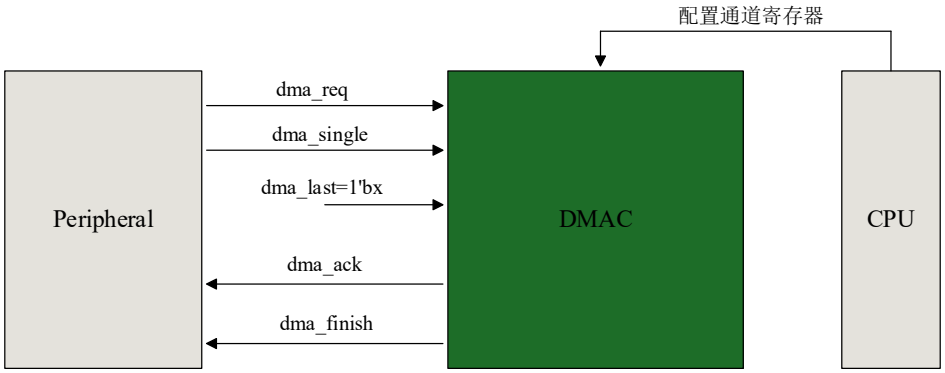
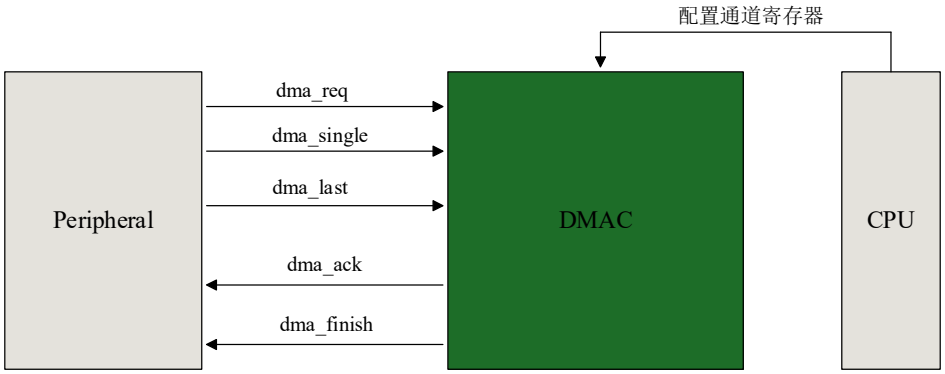


图 13-4 外设流控



配置举例：

步骤 1 源端或目的端均为外设，DMAC 流控，需配置 CTL_L.TT_FC 为 Peripheral to Memory 或者 Memory to Peripheral DMAC 流控。

步骤 2 配置源端起始地址、目的端起始地址，SAR、DAR。

步骤 3 配置传输位宽、源端和目的端 Master 口。

步骤 4 配置传输数据长度，CTL_H.block_ts；配置源端和目的端 burst length；

步骤 5 配置选择源端或目的端对应需要被响应的 DMA 请求信号。

步骤 6 配置握手接口为硬件握手，配置请求信号有效电平，寄存器 CFG_L。

步骤 7 配置源端和目的端地址递增或者不递增。

步骤 8 配置传输模式，如：循环模式、单次模式、链表模式等。

步骤 9 配置该通道优先级。

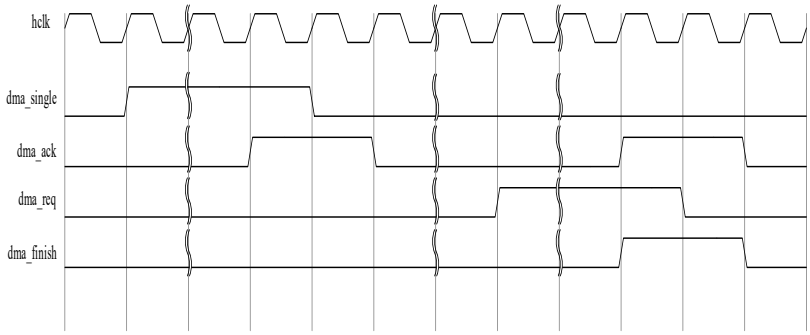
步骤 10 开启通道使能和中断使能，等待外设 DMA 请求，开始数据传输。

⚠注意

DMA 响应外设请求时，burst 请求与 single 请求同时有效时，会进行 burst 传输；只有 single 请求时进行 single 传输。

当 BLOCK_TS 可以被 SRC(DEST)_MSIZE 整除，则以 burst 传输完成全部数据搬移；当 BLOCK_TS 不可以被 SRC(DEST)_MSIZE 整除，则剩余不能整除的部分按 single 传输完成；该机制取决于外设能发起的请求。

图 13-5 硬件握手时序



13.4.2 DMAC 链表

链表模式为 DMAC 提供的一种自动获取配置节点信息的功能；能够依照用户的设置实现数据搬移；链表节点数据结构为：

- 通道寄存器 SAR：设置源端起始地址
- 通道寄存器 DAR：设置目的端起始地址

- 通道寄存器 LLP：设置下一节点地址

通道寄存器 CTL_L 和 CTL_H，设置 Master 选择、传输位宽、transfer size、burst length、地址递增等参数；

链表更新通道寄存器示意如图 13-6 所示，链表数据结构及链表节点访问方式如图 13-7 所示。

图 13-6 链表更新通道寄存器示意图

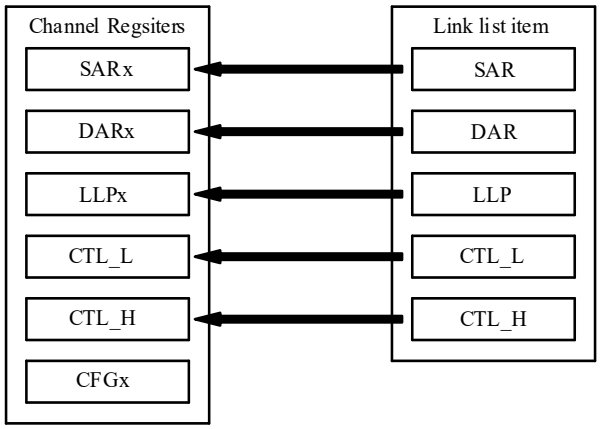
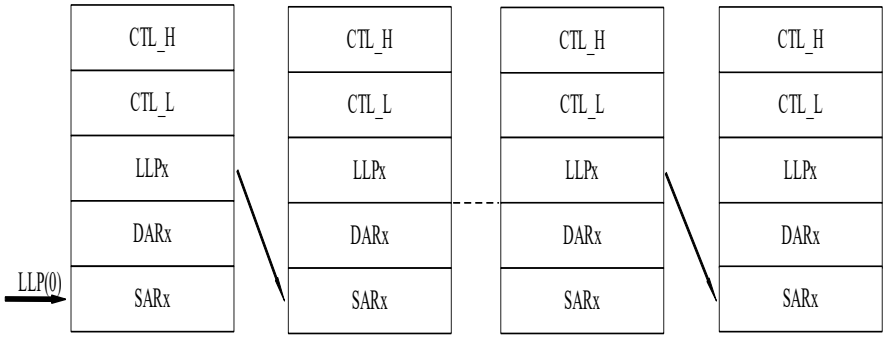


图 13-7 链表数据结构及节点链接



⚠注意

链表节点信息存储的地址必须连续。

为保证链表节点加载效率，建议链表保存按 8 word 对齐。

如果链表 LLP 寄存器的值为 0，表示当前节点就是链表的链尾，则完成本节点传输即关闭通道。

13.4.3 DMAC 循环模式

DMAC 提供循环传输模式，其自动重载设置通过配置通道寄存器 CFG_L 实现。表 13-4 展示自动重载设置方式。

配置流程举例：

步骤 1 读 ChEnReg 寄存器，选择空闲通道。

步骤 2 配置 CTL_L、CTL_H、CFG_L、CFG_H、SAR、DAR 等寄存器，设置传输类型、传输位宽、请求源等信息，开启自动重载使能。

步骤 3 使能 DMA 通道。

步骤 4 配置中断寄存器，使能 block 中断。

步骤 5 查询 block 中断状态，block 中断响应后清除中断，不做其他操作则自动重载配置进行循环；何时退出循环由软件决定。

步骤 6 若未配置中断使能，则一经触发 DMA 持续进行循环搬移，直到软件更改自动重载配置关闭自动重载使能。

表 13-4 自动重载设置

传输类型	LLP.LOC=0	LLP_SRC_EN(CTLx)	RELOAD_SRC (CFGx)	LLP_DST_EN (CTLx)	RELOAD_DST (CFGx)	CTLx和LLPx更新方式	SARx更新方式	DARx更新方式
单块或多块最后一次	YES	0	0	0	0	NONE	NONE	NONE
多块传输自动重载及配置源端	YES	0	0	0	1	CTLx,LLPx为初始配置	用户配置	自动重载
多块传输自动重载及配置目的端	YES	0	1	0	0	CTLx,LLPx为初始配置	自动重载	用户配置
多块传输自动重载	YES	0	1	0	1	CTLx,LLPx为初始配置	自动重载	自动重载
单块或多块最后一次传输	NO	0	0	0	0	NONE	NONE	NONE
链表多块传输及配置源端	NO	0	0	1	0	CTLx，LLPx从下一链表节点加载	用户配置	链表节点重载
链表多块传	NO	0	1	1	0	CTLx，LLPx	自动重	链表

传输类型	LLP.LOC=0	LLP_SRC_EN(CTLx)	RELOAD_SRC (CFGx)	LLP_DST_EN (CTLx)	RELOAD_DST (CFGx)	CTLx和LLPx更新方式	SARx更新方式	DARx更新方式
输及自动重载源端						从下一链表节点加载	载	节点重载
链表多块传输及配置目的端	NO	1	0	0	0	CTLx，LLPx从下一链表节点加载	链表节点加载	用户配置
链表多块传输及自动重载目的端	NO	1	0	0	1	CTLx，LLPx从下一链表节点加载	链表节点加载	自动重载
链表多块传输	NO	1	0	1	0	CTLx，LLPx从下一链表节点加载	链表节点加载	链表节点加载

链表模式推荐使用表 13-4 中链表多块传输方式。

13.4.4 DMAC 时钟门控

DMAC 支持本模块时钟门控，以在必要时候降低功耗；当以下情况时可以考虑软件关闭时钟门控：

- DMAC 各通道均处于空闲状态且没有 DMA 传输请求；
- 查询到 DMAC 寄存器 ChEnReg.ch_en[x:0](x 表示通道数)长时间等于 0，即 DMAC 所有通道处于关闭状态；

关断 DMAC 时钟步骤：

步骤 1 对 DMAC 寄存器 ChEnReg.ch_en[x:0]写 0，关闭所有通道。

步骤 2 对 CRG 寄存器 PERI_CRG66.dmax_cken 写 0（x 表示 DMAC0 或者 DMAC1），关闭对应 DMAC 时钟门控，最终关闭 DMAC 时钟。

13.4.5 DMAC 初始化

初始化 DMAC 步骤：

步骤 1 对 CRG 寄存器 PERI_CRG66.dmax_cken 写 1（x 表示 DMAC0 或者 DMAC1），打开时钟门控，DMAC 时钟正常。

步骤 2 对 DMAC 寄存器 ClearTfr、ClearBlock、ClearSrcTran、ClearDstTran、

ClearErr 寄存器写 0，清除所有中断。

13.4.6 DMAC 通道编程

DMAC 初始化完成后，需要配置 DMAC 通道，才可以使用 DMAC 进行数据传输。

编程示例如下：

步骤 1 回读 DMAC 寄存器 ChEnReg，查询通道空闲状态，选择当前空闲的通道进行配置。

步骤 2 写 DMAC 寄存器 ClearTfr、ClearBlock、ClearSrcTran、ClearDstTran、ClearErr，清除需使用通道的中断。

步骤 3 配置 DMAC 通道寄存器 CTL_L 和 CTL_H，设置 transfer size、链表使能与否、源端与目的端 Master 口选择、传输类型及流控方式、Scatter/Gather 模式使能、源端与目的端 burst length、地址自增、传输位宽等信息、通道中断使能。

步骤 4 配置 DMAC 通道寄存器 SAR，设置源端设备起始地址。

步骤 5 配置 DMAC 通道寄存器 DAR，设置目的端设备起始地址。

步骤 6 配置 DMAC 通道寄存器 LLP，设置链表节点，如果不使用链表模式，则链表节点配置默认值或者 0；如果使用链表模式，则需要配置链表的头指针。

步骤 7 配置 DMAC 通道寄存器 CFG_L 和 CFG_H，设置源端与目的端响应的请求、自动重载使能、源端和目的端握手信号有效电平（芯片支持高有效 ）、握手接口选择、通道优先级等信息。

步骤 8 配置 DMAC 寄存器 MaskTfr、MaskBlock、MaskSrctran、MaskDstTran、MaskErr，设置中断屏蔽。

步骤 9 配置 DMAC 寄存器 DmaCfgReg，开启 DMAC 使能。

步骤 10 配置 DMAC 寄存器 ChEnReg，开启相应通道使能，启动该通道。

说明

Scatter 模式是用于目的端传输，可以定义地址跳变次数和间隔量实现分散式的地址访问；如配置 DSR.dsc 为 4，DSR.DSI 为 16，那么在传输位宽为 64bit 情况下，地址跳变值为 DSR.dsi*8=0x80，共访问 DSR.dsc-1 段地址段，每个地址段内访问地址数为 DSR.dsc*（传输位宽/8），如以 A0 为起始地址，第一段为 A0~A0+（DSR.dsc-1）*（传输位宽/8）。

Gather 模式用于源端传输，可以定义地址跳变次数和间隔量实现跳跃式地址访问；如配置 SGR.sgc 为 4，SGR.sgi 为 1，那么在传输为 32bit 的情况下，地址跳变值为 SGR.sgi*4=0x4，共访问 SGR.sgc-1 段地址段；每个地址段内访问地址数为 SGR.sgc*（传输位宽/8），如以 A0 为起始地址，第一段为 A0~A0+（SGR.sgc-1）*（传输位宽

/8）。

通道寄存器 CTL_L.src_msize/dest_msize 配置与其 burst length 关系如下表 13-5 所示。

表 13-5 CTL_L.src_msize/dest_msize 与对应 burst length

src_msize 或 dst_msize（二进制值）	Burst length
000	1
001	4
010	8
011	16
100	32
101	64
110	128
111	256

通道寄存器 CTL_L.src_tr_width/dst_tr_width 与传输位宽的关系如下表 13-6 所示。

表 13-6 CTL_L.src_tr_width/dst_tr_width 与对应传输位宽

src_tr_width 或 dst_tr_width（二进制值）	传输位宽
000	Byte（8bit）
001	Half Word（16bit）
010	Word（32bit）
011	Reserved
100	Reserved
101	Reserved
110	Reserved
111	Reserved

外设请求接口选择如下表 13-7 所示。

表 13-7 外设请求接口配置选择

src_per/dest_per（二进制值）	对应请求
0000	dma_req[0]/dma_single[0]
0001	dma_req[1]/dma_single[1]
0010	dma_req[2]/dma_single[2]

0011	dma_req[3]/dma_single[3]
0100	dma_req[4]/dma_single[4]
0101	dma_req[5]/dma_single[5]
0110	dma_req[6]/dma_single[6]
0111	dma_req[7]/dma_single[7]
1000	dma_req[8]/dma_single[8]
1001	dma_req[9]/dma_single[9]
1010	dma_req[10]/dma_single[10]
1011	dma_req[11]/dma_single[11]
1100	dma_req[12]/dma_single[12]
1101	dma_req[13]/dma_single[13]
1110	dma_req[14]/dma_single[14]
1111	dma_req[15]/dma_single[15]

13.4.7 DMAC 中断处理

DMAC 通道完成传输或者传输过程发生错误，通道的状态会上报到中断寄存器；所有通道的中断信号通过一个中断请求线上报给 CPU，CPU 接收到中断请求后可查询各个通道的状态。中断处理如下：

步骤 1 读取中断状态寄存器 StatusTfr、StatusBlock、StatusSrcTran、StatusDstTran、StatusErr，查询发出中断的通道，并判断中断类型。

步骤 2 查询到通道完成传输中断，则执行 3；查询到通道错误中断，则执行 4。

步骤 3 传输完成中断：

1) 写 ClearTfr、ClearBlock、ClearSrcTran、ClearDstTran、ClearErr 寄存器对应位，清除中断。
2) 处理目的地址数据。
3) 退出中断。

步骤 4 传输错误中断：

1) 写 ClearErr 寄存器对应位，清除对应通道错误中断。
2) 记录错误信息，视必要重新配置启动该通道。
3) 退出中断。

⚠注意

DMAC 中断寄存器，是按中断类型分配的；如 StatusErr 中断寄存器，是所有通道错误中断指示寄存器。

13.5 DMAC 寄存器概览

DMAC 实例 2 个，其基地址如下：

- DMAC0 寄存器基地址为：0x64000000
- DMAC1 寄存器基地址为：0x64001000

说明

其中 x 表示通道数，取值 0~5。

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	SAR	0x0+0x58*x	32	RW	0x00000000	通道的源端地址寄存器
2	DAR	0x8+0x58*x	32	RW	0x00000000	通道的目的端地址寄存器
3	LLP	0x10+0x58*x	32	RW	0x00000000	通道的链表指针寄存器
4	CTL_L	0x18+0x58*x	32	RW	0x00004825	通道的控制寄存器低 32 位
5	CTL_H	0x1C+0x58*x	32	RW	0x00000002	通道的控制寄存器高 32 位
6	SSTAT	0x20+0x58*x	32	RW	0x00000000	通道的源端状态寄存器
7	DSTAT	0x28+0x58*x	32	RW	0x00000000	通道的目的端状态寄存器
8	SSTATAR	0x30+0x58*x	32	RO	0x00000000	通道的源端状态地址寄存器
9	DSTATAR	0x38+0x58*x	32	RO	0x00000000	通道的目的端状态地址寄存器
10	CFG_L	0x40+0x58*x	32	RW	0x00000E00	通道的配置寄存器低 32 位
11	CFG_H	0x44+0x58*x	32	RW	0x00000000	通道的配置寄存器高 32 位
12	SGR	0x48+0x58*x	32	RW	0x00000000	通道的源端 Gather 寄存器
13	DSR	0x50+0x58*x	32	RW	0x00000000	通道的目的端 Scatter 寄存器
14	RawTfr	0x2c0	32	RW	0x00000000	Transfer 完成的原始中断
15	RawBlock	0x2c8	32	RW	0x00000000	Block 完成的原始中断
16	RawSrcTran	0x2d0	32	RW	0x00000000	源端 Transfer 完成的原始中断
17	RawDstTran	0x2d8	32	RW	0x00000000	目的端 Transfer 完成的原始中断
18	RawErr	0x2e0	32	RW	0x00000000	传输错误原始中断

No.	Names	Offset Address	SIZE	R/W	Default	Description
19	StatusTfr	0x2e8	32	RO	0x00000000	Transfer 完成的中断状态
20	StatusBlock	0x2f0	32	RO	0x00000000	Block 完成的中断状态
21	StatusSrcTran	0x2f8	32	RO	0x00000000	源端 Transfer 完成的中断状态
22	StatusDstTran	0x300	32	RO	0x00000000	目的端 Transfer 完成的中断状态
23	StatusErr	0x308	32	RO	0x00000000	传输错误中断状态
24	MaskTfr	0x310	32	RW	0x00000000	Transfer 完成中断的屏蔽信号
25	MaskBlock	0x318	32	RW	0x00000000	Block 完成中断的屏蔽信号
26	MaskSrctran	0x320	32	RW	0x00000000	源端 Transfer 完成中断的屏蔽信号
27	MaskDstTran	0x328	32	RW	0x00000000	目的端 Transfer 完成中断屏蔽信号
28	MaskErr	0x330	32	RW	0x00000000	传输错误中断的屏蔽信号
29	ClearTfr	0x338	32	WO	0x00000000	Transfer 完成中断的清除信号
30	ClearBlock	0x340	32	WO	0x00000000	Block 完成中断的清除信号
31	ClearSrcTran	0x348	32	WO	0x00000000	源端 Transfer 完成中断的清除信号
32	ClearDstTran	0x350	32	WO	0x00000000	目的端 Transfer 完成中断清除信号
33	ClearErr	0x358	32	WO	0x00000000	传输错误中断的清除信号
34	StatusInt	0x360	32	RO	0x00000000	中断状态汇总寄存器
35	ReqSrcReg	0x368	32	RW	0x00000000	源端软件传输请求寄存器
36	ReqDstReg	0x370	32	RW	0x00000000	目的端软件传输请求寄存器
37	SglRqSrcReg	0x378	32	RW	0x00000000	源端软件 single 传输请求寄存器
38	SglRqDstReg	0x380	32	RW	0x00000000	目的端软件 single 传输请求寄存器
39	LstSrcReg	0x388	32	RW	0x00000000	源端软件最后一次传输请求寄存器
40	LstDstReg	0x390	32	RW	0x00000000	目的端软件最后一次传输请求寄存器
41	DmaCfgReg	0x398	32	RW	0x00000000	DMAC 通道配置寄存器
42	ChEnReg	0x3a0	32	RW	0x00000000	DMAC 通道使能寄存器

说明

1. 寄存器表中的 x 表示通道数，取值 0~5。

2. 此处说明几个概念：如寄存器概览中提到的 DMAC 的几个中断，Transfer 完成中断是指一次握手完成的中断；Block 完成中断是指当前配置的 CTL_H.BLOCK_TS 传输完成的中断；源端 Transfer 完成中断是指源端的一次握手完成的中断；目的端 Transfer 完成中断是指目的端一次握手完成的中断。

13.6 DMAC 寄存器描述

13.6.1 SAR

◆ **偏移地址：0x0+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sar															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sar															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
sar	[31:0]	RW	源端起始地址	0x0

13.6.2 DAR

◆ **偏移地址：0x8+0x58*x　　复位值：0x00000000**

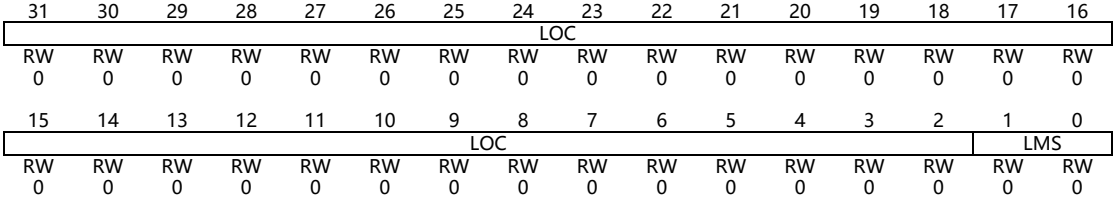
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
dar															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dar															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
dar	[31:0]	RW	目的端起始地址	0x0

13.6.3 LLP

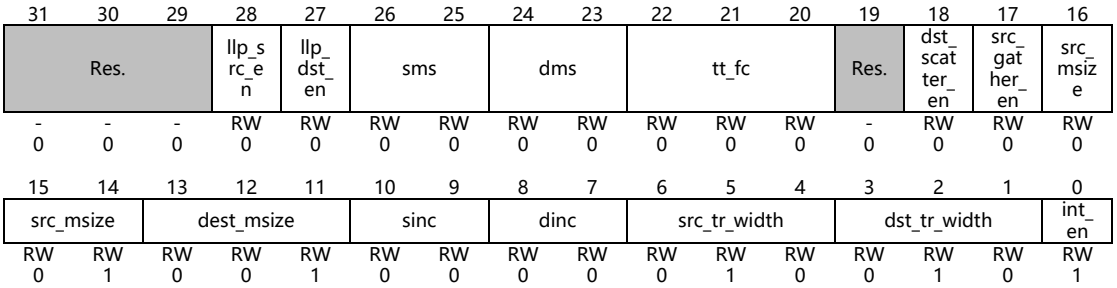
◆ **偏移地址：0x10+0x58*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
loc	[31:2]	RW	链表节点起始地址	0x0
lms	[1:0]	RW	链表信息获取 master 口选择 0：master 1 1：master 2 2：reserved 3：reserved	0x0

13.6.4 CTL_L

◆ **偏移地址：0x18+0x58*x　　复位值：0x00004825**



Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留	0x0
llp_src_en	[28]	RW	该信号使能源端链表模式 0：源端链表模式不使能 1：源端链表模式使能	0x0
llp_dst_en	[27]	RW	该信号使能目的端链表模式 0：目的端链表模式不使能 1：目的端链表模式使能	0x0
sms	[26:25]	RW	设置访问源端设备的 master 0：使用 master1 访问 1：使用 master2 访问	0x0
dms	[24:23]	RW	设置访问目的端设备的 master 0：使用 master1 访问 1：使用 master2 访问	0x0

Name	Bits	R/W	Description	Default
tt_fc	[22:20]	RW	设置传输类型和流控制器 0：memory to memory； 1：memory to Peripheral； 2：Peripheral to memory；	0x0
reserved	[19]	-	保留	0x0
dst_scatter_en	[18]	RW	目的端 scatter 模式使能设置。该位域用于使能目的端 scatter 模式，scatter 模式只有设置了目的端地址递增或递减时才适用 0：目的端 scatter 不使能 1：目的端 scatter 使能	0x0
src_gather_en	[17]	RW	源端 gather 模式使能设置。该位域用于使能源端 gather 模式，gather 模式只有设置了源端地址递增或递减时才适用 0：源端 gather 不使能 1：源端 gather 使能	0x0
src_msize	[16:14]	RW	设置源端 burst length。每次接受到 burst request 时，从源端读取的数据量 0：读取的数据量是 1； 1：读取的数据量是 4； 2：读取的数据量是 8； 3：读取的数据量是 16； 4：读取的数据量是 32； 5：读取的数据量是 64； 6：读取的数据量是 128； 7：读取的数据量是 256； 注意：该值与 AHB 的 burst 无关	0x1
dest_msize	[13:11]	RW	设置目的端 burst length。每次接受到 burst request 时，向目的端发送的数据量 0：发送的数据量是 1； 1：发送的数据量是 4； 2：发送的数据量是 8； 3：发送的数据量是 16； 4：发送的数据量是 32； 5：发送的数据量是 64； 6：发送的数据量是 128； 7：发送的数据量是 256； 注意：该值与 AHB 的 burst 无关	0x1
sinc	[10:9]	RW	设置源端地址自增 0：源端地址递增 1：源端地址递减 2 或者 3：源端地址固定不变	0x0

Name	Bits	R/W	Description	Default
dinc	[8:7]	RW	设置目的端地址自增 0：目的端地址递增 1：目的端地址递减 2 或者 3：目的端地址固定不变	0x0
src_tr_width	[6:4]	RW	设置源端传输数据位宽 0：数据位宽是 8bit； 1：数据位宽是 16bit； 2：数据位宽是 32bit； 3：数据位宽是 64bit； 4：数据位宽是 128bit； 5：数据位宽是 256bit； 6：数据位宽是 256bit； 7：数据位宽是 256bit；	0x2
dst_tr_width	[3:1]	RW	设置目的端传输数据位宽 0：数据位宽是 8bit； 1：数据位宽是 16bit； 2：数据位宽是 32bit； 3：数据位宽是 64bit； 4：数据位宽是 128bit； 5：数据位宽是 256bit； 6：数据位宽是 256bit； 7：数据位宽是 256bit；	0x2
int_en	[0]	RW	设置中断使能，该位用于设置所有中断产生使能 0：中断不使能 1：中断使能	0x1

13.6.5 CTL_H

◆ **偏移地址：0x1C+0x58*x　　复位值：0x00000002**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			done	block_ts											
-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留	0x0
done	[12]	RW	该位是完成指示位；如果状态回写使能，则 CTL_H 将在传输结束时写入链表节点的 CTL_H 位置，并置	0x0

Name	Bits	R/W	Description	Default
			位 DONE bit；软件可以轮询链表节点中的 CTL_H 的 DONE bit 查看是否完成传输，在启动通道之前，应清除链表节点中的 DONE； 0：该位不指示块传输完成 1：在块传输完成后置位该位	
block_ts	[11:0]	RW	通过该位域设置 DMAC 块传输的数据长度，当 DMAC 为流控制器时，可以通过设置该位域来设置块传输的数据总数。	0x2

13.6.6 SSTAT

◆ **偏移地址：0x20+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sstat															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sstat															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
sstat	[31:0]	RW	源端状态信息，该信息包含于 SSTATAR 寄存器指向的地址中	0x0

13.6.7 DSTAT

◆ **偏移地址：0x28+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
dstat															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dstat															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
dstat	[31:0]	RW	目的端状态信息，该信息包含于 DSTATAR 寄存器指向的地址中	0x0

13.6.8 SSTATAR

◆ **偏移地址：0x30+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sstatar															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sstatar															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
sstatar	[31:0]	RO	指示源端状态信息所在的地址；该寄存器指向的地址的源端状态信息会在下一次块传输开始前写入链表节点对应位置	0x0

13.6.9 DSTATAR

◆ **偏移地址：0x38+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
dstatar															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dstatar															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
dstatar	[31:0]	RO	指示目的端状态信息所在的地址；该寄存器指向的地址的目的端状态信息会在下一次块传输开始前写入链表节点对应位置	0x0

13.6.10 CFG_L

◆ **偏移地址：0x40+0x58*x　　复位值：0x00000E00**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
reload_dst	reload_src	Res.										src_hs_pol	dst_hs_pol	lock_b	lock_ch
RW	RW	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
lock_b_l		lock_ch_l		hs_sel_src	hs_sel_dst	fifo_empty	ch_susp	ch_prior			Res.				
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	-	-	-	-	-
0	0	0	0	1	1	1	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reload_dst	[31]	RW	目的端地址自动重载使能设置	0x0

Name	Bits	R/W	Description	Default
			0：目的端地址重载不使能 1：目的端地址重载使能	
reload_src	[30]	RW	源端地址自动重载使能设置 0：源端地址重载不使能 1：源端地址重载使能	0x0
reserved	[29:20]	-	保留	0x0
src_hs_pol	[19]	RW	源端握手接口电平设置 0：源端握手电平高有效 1：源端握手电平低有效（本芯片只支持高电平）	0x0
dst_hs_pol	[18]	RW	目的端握手接口电平设置 0：目的端握手电平高有效 1：目的端握手电平低有效（本芯片只支持高电平）	0x0
lock_b	[17]	RW	总线 LOCK 位 0：禁止总线 lock 1：使能总线 lock	0x0
lock_ch	[16]	RW	通道 LOCK 位 0：禁止通道 lock 1：使能通道 lock	0x0
lock_b_l	[15:14]	RW	设置总线 LOCK 位生效时机 0：DMA transfer 完成 1：DMA block transfer 完成 2 或者 3：DMA 一次搬移完成	0x0
lock_ch_l	[13:12]	RW	设置通道 LOCK 位生效时机 0：DMA transfer 完成 1：DMA block transfer 完成 2 或者 3：DMA 一次搬移完成	0x0
hs_sel_src	[11]	RW	源端握手接口选择 0：选择硬件握手接口 1：选择软件握手接口	0x1
hs_sel_dst	[10]	RW	目的端握手接口选择 0：选择硬件握手接口 1：选择软件握手接口	0x1
fifo_empty	[9]	RW	通道 FIFO 状态位 0：通道 FIFO 非空 1：通道 FIFO 空	0x1
ch_susp	[8]	RW	通道挂起位；可以设置暂停 DMA 传输，直到该位清除； 0：不挂起源端的 DMA 传输 1：挂起源端的 DMA 传输	0x0
ch_prior	[7:5]	RW	通道优先级	0x0

Name	Bits	R/W	Description	Default
			该位域用于设置通道使用 master 的优先级，0 级最低； 0：通道优先级 0 1：通道优先级 1 2：通道优先级 2 3：通道优先级 3 4：通道优先级 4 5：通道优先级 5 6 或者 7：保留	
reserved	[4:0]	-	保留	0x0

13.6.11 CFG_H

◆ **偏移地址：0x44+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	dest_per				src_per				ss_upd_en	ds_upd_en	Res.				
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留	0x0
dest_per	[14:11]	RW	目的端硬件请求设置 该位域用于设置目的端被使用的请求信号；	0x0
src_per	[10:7]	RW	源端硬件请求设置 该位域用于设置源端被使用的请求信号；	0x0
ss_upd_en	[6]	RW	源端状态更新使能 该位域为设置为高，则源端状态信息从 SSTATAR 指向的位置获取并存储到 SSTAT 中； 0：禁止源端状态信息更新 1：使能源端状态信息更新	0x0
ds_upd_en	[5]	RW	目的端状态更新使能 该位域为设置为高，则目的端状态信息从 DSTATAR 指向的位置获取并存储到 DSTAT 中； 0：禁止目的端状态信息更新 1：使能目的端状态信息更新	0x0
reserved	[4:2]	-	保留	0x0
reserved	[1]	-	保留	0x0
reserved	[0]	-	保留	0x0

13.6.12 SGR

◆ **偏移地址：0x48+0x58*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sgc												sgi			
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
sgi															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
sgc	[31:20]	RW	源端 Gather 计数器 计数源端地址跳转次数	0x0
sgi	[19:0]	RW	源端地址间隔数（单位 type）	0x0

13.6.13 DSR

◆ **偏移地址：0x50+0x58*x　　复位值：0x00000000**

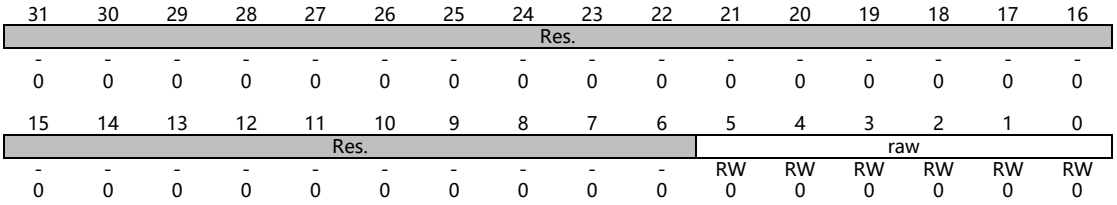
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
dsc												dsi			
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dsi															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
dsc	[31:20]	RW	目的端 Scatter 计数器 计数目的端地址跳转次数	0x0
dsi	[19:0]	RW	目的端地址间隔数（单位 type）	0x0

13.6.14 RawTfr

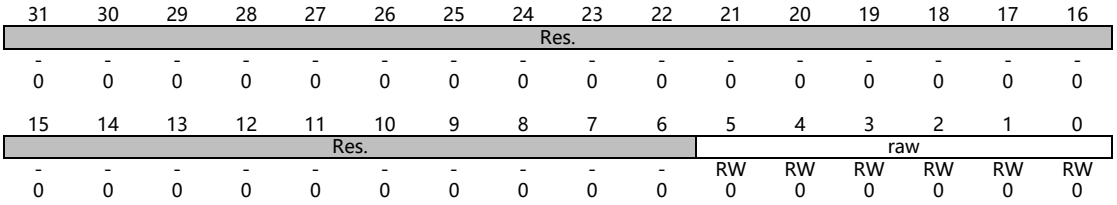
◆ **偏移地址：0x2c0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
raw	[5:0]	RW	原始中断状态。该位域上报各通道 Transfer 完成中断，bit0-5 依次对应通道 0-5。 0：原始中断无效 1：有原始中断	0x0

13.6.15 RawBlock

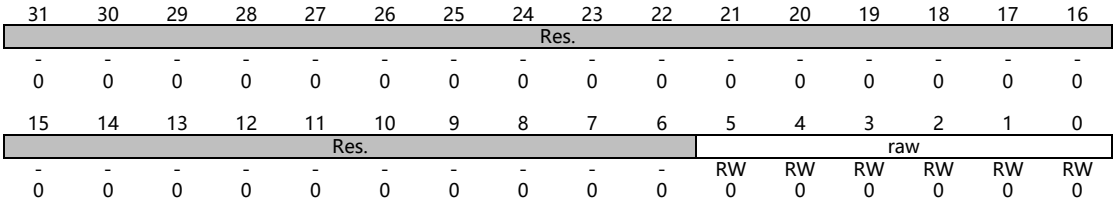
◆ **偏移地址：0x2c8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
raw	[5:0]	RW	Block 完成原始中断；该位域上报各通道块传输完成原始中断，bit0-5 依次对应通道 0-5。 0：原始中断无效 1：有原始中断	0x0

13.6.16 RawSrcTran

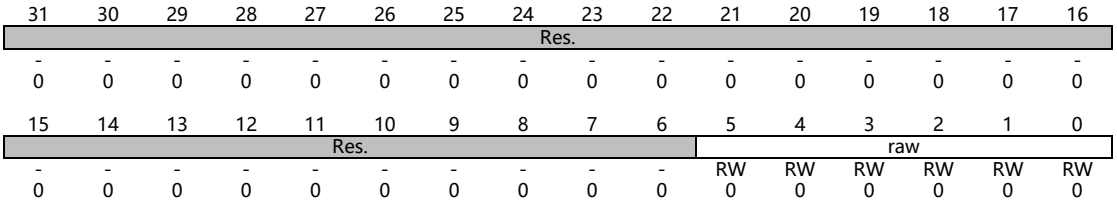
◆ **偏移地址：0x2d0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
raw	[5:0]	RW	源端 Transfer 完成原始中断；该位域上报各通道源端 transfer 完成原始中断，bit0-5 依次对应通道 0-5。 0：原始中断无效 1：有原始中断	0x0

13.6.17 RawDstTran

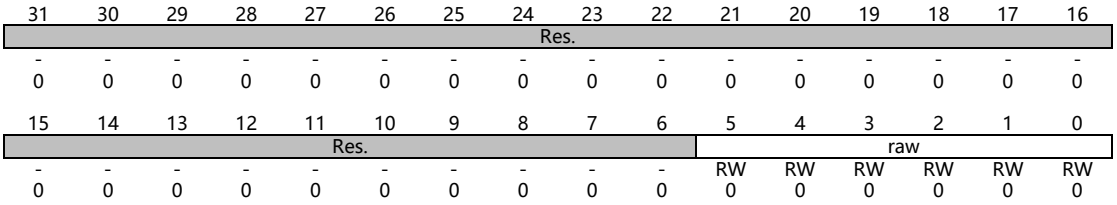
◆ **偏移地址：0x2d8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
raw	[5:0]	RW	目的端 Transfer 完成原始中断状态；该位域上报各通道目的端 transfer 完成原始中断，bit0-5 依次对应通道 0-5。 0：原始中断无效 1：有原始中断	0x0

13.6.18 RawErr

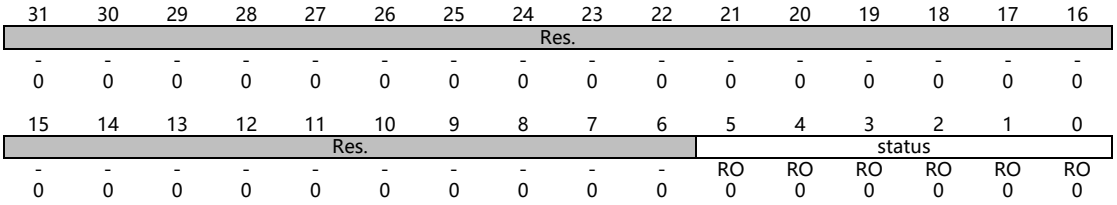
◆ **偏移地址：0x2e0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
raw	[5:0]	RW	传输错误原始中断；该位域上报各通道传输错误原始中断，bit0-5 依次对应通道 0-5。 0：原始中断无效 1：有原始中断	0x0

13.6.19 StatusTfr

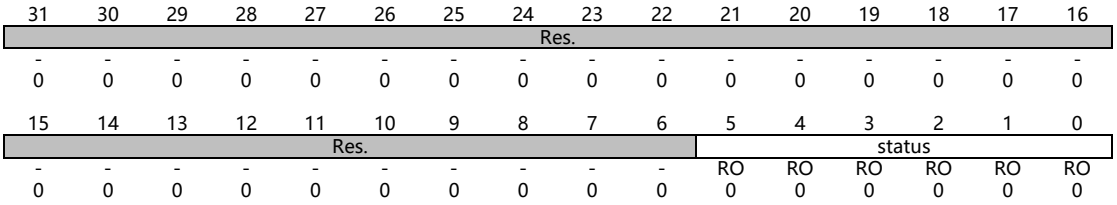
◆ **偏移地址：0x2e8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
status	[5:0]	RO	Transfer 中断状态；该位域上报各通道 Transfer 完成中断；bit0-5 依次对应通道 0-5。 0：中断状态无效 1：有中断状态	0x0

13.6.20 StatusBlock

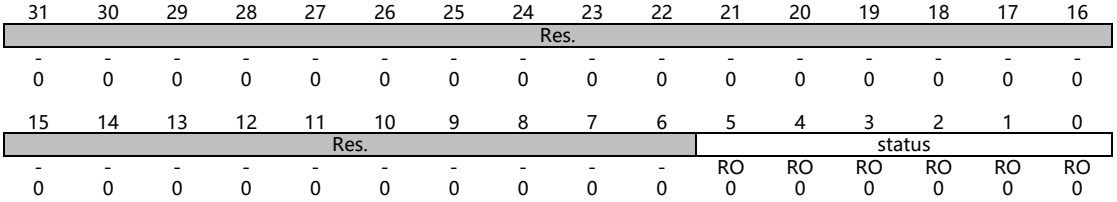
◆ **偏移地址：0x2f0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
status	[5:0]	RO	块传输完成中断状态；该位域上报各通道块传输完成中断；bit0-5 依次对应通道 0-5。 0：中断状态无效 1：有中断状态	0x0

13.6.21 StatusSrcTran

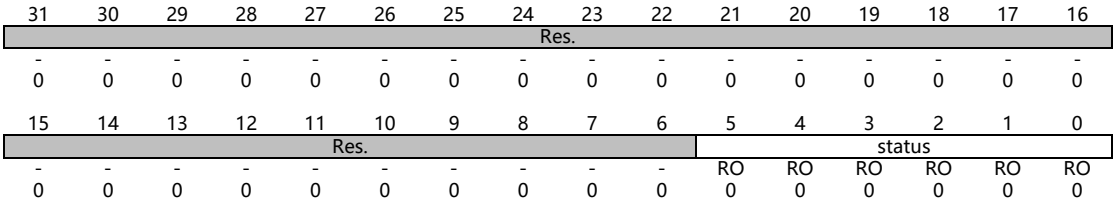
◆ **偏移地址：0x2f8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
status	[5:0]	RO	源端 Transfer 完成中断状态；该位域上报各通道源端 Transfer 完成中断；bit0-5 依次对应通道 0-5。 0：中断状态无效 1：有中断状态	0x0

13.6.22 StatusDstTran

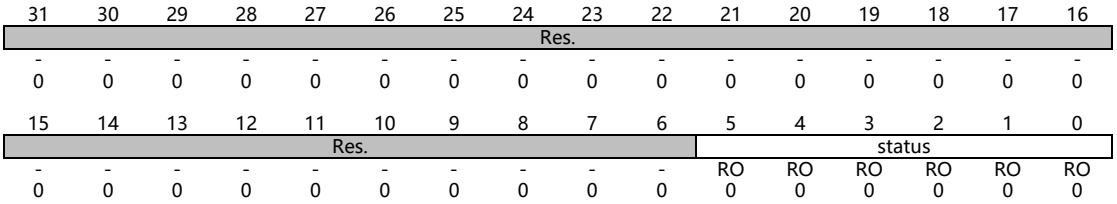
◆ **偏移地址：0x300　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
status	[5:0]	RO	目的端 Transfer 完成中断状态；该位域上报各通道目的端 Transfer 完成中断；bit0-5 依次对应通道 0-5。 0：中断状态无效 1：有中断状态	0x0

13.6.23 StatusErr

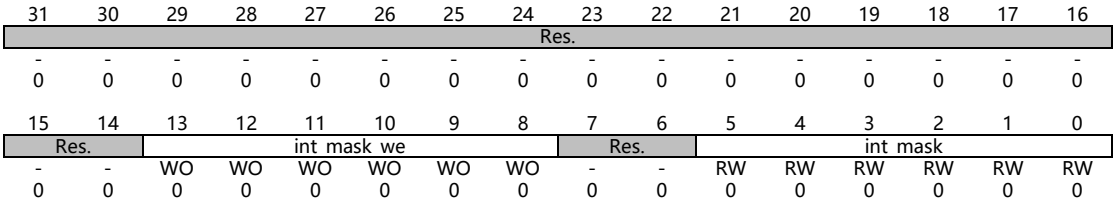
◆ **偏移地址：0x308　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
status	[5:0]	RO	传输错误中断状态；该位域上报各通道传输错误中断；bit0-5 依次对应通道 0-5。 0：中断状态无效 1：有中断状态	0x0

13.6.24 MaskTfr

◆ **偏移地址：0x310　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
int_mask_we	[13:8]	WO	Transfer 中断屏蔽写使能位；该位域控制使能屏蔽位写使能；只有对该位域写 1，中断屏蔽位才会被有效写入；bit 从低到高依次对应通道 0-5； 0：禁止写中断屏蔽 1：允许写中断屏蔽	0x0
reserved	[7:6]	-	保留	0x0
int_mask	[5:0]	RW	Transfer 中断屏蔽位；该位域控制中断屏蔽；bit 从低到高依次对应通道 0-5； 0：屏蔽中断 1：不屏蔽中断	0x0

13.6.25 MaskBlock

◆ **偏移地址：0x318　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		int_mask_we						Res.		int_mask					
-	-	WO	WO	WO	WO	WO	WO	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
int_mask_we	[13:8]	WO	块传输完成中断屏蔽写使能位；该位域控制使能屏蔽位写使能；只有对该位域写 1，中断屏蔽位才会被有效写入；bit 从低到高依次对应通道 0-5； 0：禁止写中断屏蔽 1：允许写中断屏蔽	0x0
reserved	[7:6]	-	保留	0x0
int_mask	[5:0]	RW	块传输完成（block）中断屏蔽位；该位域控制中断屏蔽；bit 从低到高依次对应通道 0-5； 0：屏蔽中断 1：不屏蔽中断	0x0

13.6.26 MaskSrctran

◆ **偏移地址：0x320　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

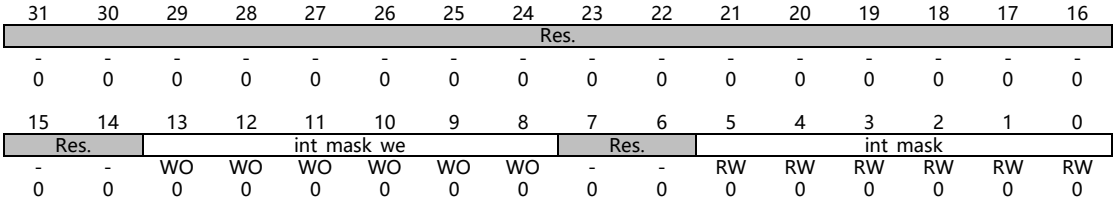
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		int_mask_we						Res.		int_mask					
-	-	WO	WO	WO	WO	WO	WO	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
int_mask_we	[13:8]	WO	源端 Transfer 完成中断屏蔽写使能位；该位域控制使能屏蔽位写使能；只有对该位域写 1，中断屏蔽位才会被有效写入；bit 从低到高依次对应通道 0-5； 0：禁止写中断屏蔽 1：允许写中断屏蔽	0x0
reserved	[7:6]	-	保留	0x0

Name	Bits	R/W	Description	Default
int_mask	[5:0]	RW	源端 Transfer 完成中断屏蔽位；该位域控制中断屏蔽；bit 从低到高依次对应通道 0-5； 0：屏蔽中断 1：不屏蔽中断	0x0

13.6.27 MaskDstTran

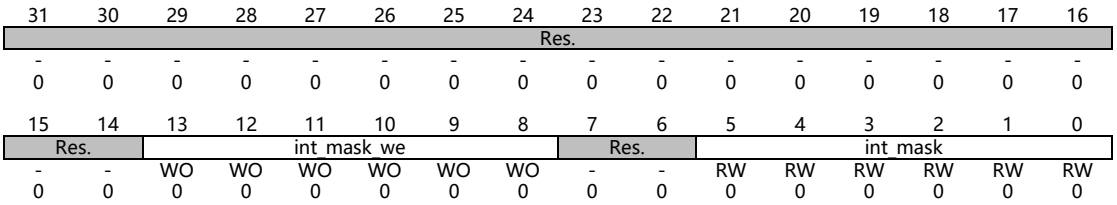
◆ **偏移地址：0x328　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
int_mask_we	[13:8]	WO	目的端Transfer完成中断屏蔽写使能位；该位域控制使能屏蔽位写使能；只有对该位域写 1，中断屏蔽位才会被有效写入；bit 从低到高依次对应通道 0-5； 0：禁止写中断屏蔽 1：允许写中断屏蔽	0x0
reserved	[7:6]	-	保留	0x0
int_mask	[5:0]	RW	目的端Transfer完成中断屏蔽位；该位域控制中断屏蔽；bit 从低到高依次对应通道 0-5； 0：屏蔽中断 1：不屏蔽中断	0x0

13.6.28 MaskErr

◆ **偏移地址：0x330　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0

Name	Bits	R/W	Description	Default
int_mask_we	[13:8]	WO	传输错误中断屏蔽写使能位；该位域控制使能屏蔽位写使能；只有对该位域写 1，中断屏蔽位才会被有效写入；bit 从低到高依次对应通道 0-5； 0：禁止写中断屏蔽 1：允许写中断屏蔽	0x0
reserved	[7:6]	-	保留	0x0
int_mask	[5:0]	RW	传输错误中断屏蔽位；该位域控制中断屏蔽；bit 从低到高依次对应通道 0-5； 0：屏蔽中断 1：不屏蔽中断	0x0

13.6.29 ClearTfr

◆ **偏移地址：0x338　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										clear					
-	-	-	-	-	-	-	-	-	-	WO	WO	WO	WO	WO	WO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
clear	[5:0]	WO	Transfer 完成中断清除位； 0：对中断无影响 1：清除对应通道中断	0x0

13.6.30 ClearBlock

◆ **偏移地址：0x340　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										clear					
-	-	-	-	-	-	-	-	-	-	WO	WO	WO	WO	WO	WO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
clear	[5:0]	WO	块传输（Block）完成中断清除位；	0x0

Name	Bits	R/W	Description	Default
			0：对中断无影响 1：清除对应通道中断	

13.6.31 ClearSrcTran

◆ **偏移地址：0x348　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										clear					
-	-	-	-	-	-	-	-	-	-	WO	WO	WO	WO	WO	WO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
clear	[5:0]	WO	源端 Transfer 传输完成中断清除位； 0：对中断无影响 1：清除对应通道中断	0x0

13.6.32 ClearDstTran

◆ **偏移地址：0x350　　复位值：0x00000000**

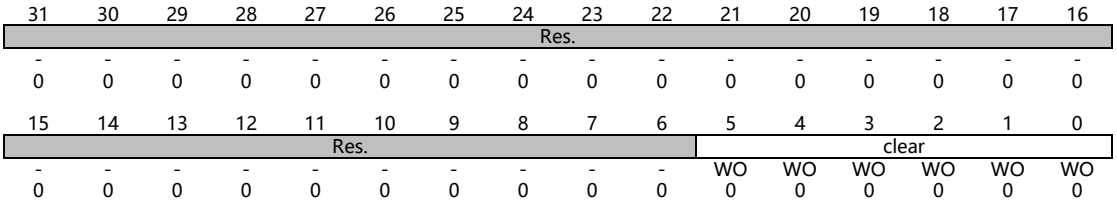
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										clear					
-	-	-	-	-	-	-	-	-	-	WO	WO	WO	WO	WO	WO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
clear	[5:0]	WO	目的端 Transfer 传输完成中断清除位； 0：对中断无影响 1：清除对应通道中断	0x0

13.6.33 ClearErr

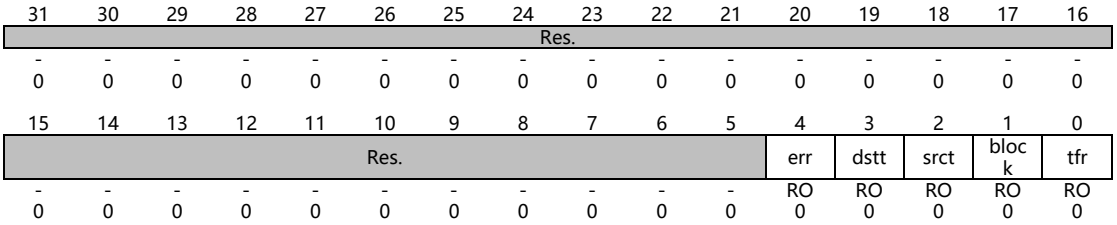
◆ **偏移地址：0x358　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
clear	[5:0]	WO	传输错误中断清除位； 0：对中断无影响 1：清除对应通道中断	0x0

13.6.34 StatusInt

◆ **偏移地址：0x360　　复位值：0x00000000**

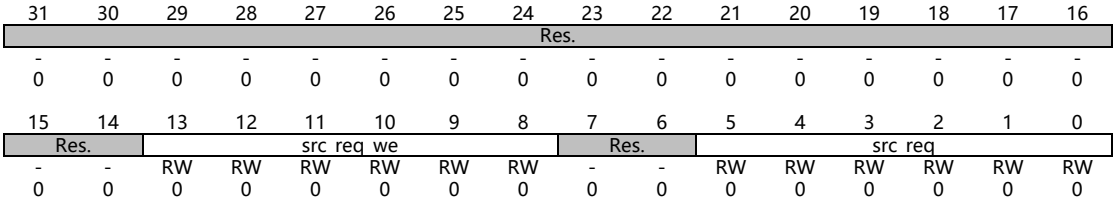


Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
err	[4]	RO	各通道传输错误中断状态汇总位；该位为各通道传输错误中断状态相或的结果 0：无错误中断 1：有错误中断	0x0
dstt	[3]	RO	各通道目的端 Transfer 完成中断状态汇总位；该位为各通道目的端 Transfer 完成中断状态相或的结果 0：无 Transfer 完成中断 1：有 Transfer 完成中断	0x0
srct	[2]	RO	各通道源端 Transfer 完成中断状态汇总位；该位为各通道源端 Transfer 完成中断状态相或的结果 0：无 Transfer 完成中断 1：有 Transfer 完成中断	0x0
block	[1]	RO	各通道块传输（Block）完成中断状态汇总位；该位为各通道块传输完成中断状态相或的结果 0：无块传输完成中断 1：有块传输完成中断	0x0

Name	Bits	R/W	Description	Default
tfr	[0]	RO	各通道Transfer完成中断状态汇总位；该位为各通道Transfer 传输完成中断状态相或的结果 0：无 Transfer 完成中断 1：有 Transfer 完成中断	0x0

13.6.35 ReqSrcReg

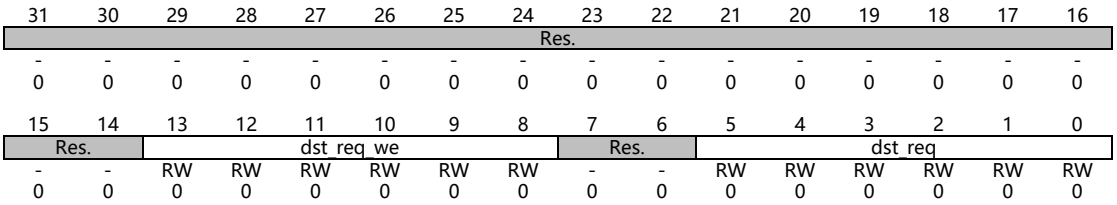
◆ **偏移地址：0x368　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
src_req_we	[13:8]	RW	源端软件传输请求写使能。该位域控制源端软件传输请求位域（SRC_REQ）是否被允许写，从低到高位对应通道 0-5，只有该位置 1，对应通道的软件传输请求写入才有效。 0：源端软件传输请求写入无效； 1：源端软件传输请求写入有效；	0x0
reserved	[7:6]	-	保留	0x0
src_req	[5:0]	RW	源端软件传输请求；从低到高位对应通道 0-5； 0：源端软件传输请求无效； 1：源端软件传输请求有效；	0x0

13.6.36 ReqDstReg

◆ **偏移地址：0x370　　复位值：0x00000000**

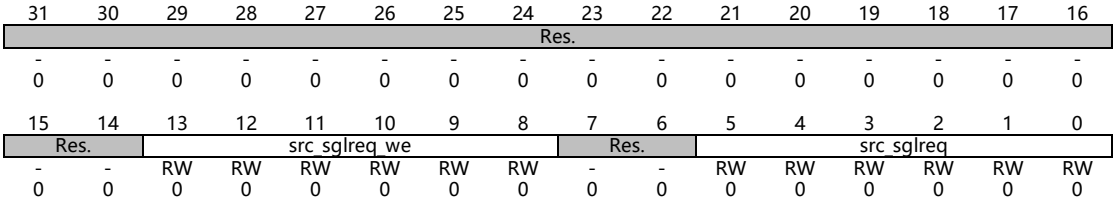


Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0

Name	Bits	R/W	Description	Default
dst_req_we	[13:8]	RW	目的端软件传输请求写使能。该位域控制目的端软件传输请求位域（DST_REQ）是否被允许写，从低到高位对应通道 0-5，只有该位置 1，对应通道的软件传输请求写入才有效。 0：目的端软件传输请求写入无效； 1：目的端软件传输请求写入有效；	0x0
reserved	[7:6]	-	保留	0x0
dst_req	[5:0]	RW	目的端软件传输请求；从低到高位对应通道 0-5； 0：目的端软件传输请求无效； 1：目的端软件传输请求有效；	0x0

13.6.37 SglRqSrcReg

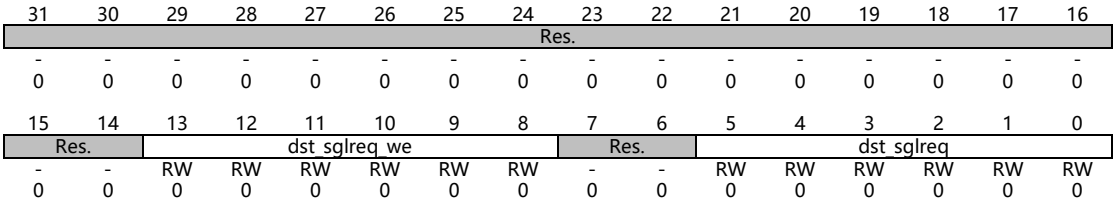
◆ **偏移地址：0x378　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
src_sglreq_we	[13:8]	RW	源端软件 single 传输请求写使能。该位域控制源端软件 single 传输请求位域（SRC_SGLREQ）是否被允许写，从低到高位对应通道 0-5，只有该位置 1，对应通道的软件传输请求写入才有效。 0：源端软件 single 传输请求写入无效； 1：源端软件 single 传输请求写入有效；	0x0
reserved	[7:6]	-	保留	0x0
src_sglreq	[5:0]	RW	源端软件传输 single 请求；从低到高位对应通道 0-5； 0：源端软件 single 传输请求无效； 1：源端软件 single 传输请求有效；	0x0

13.6.38 SglRqDstReg

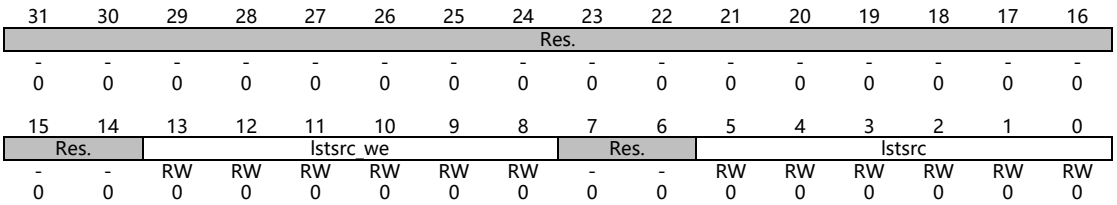
◆ **偏移地址：0x380　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
dst_sglreq_we	[13:8]	RW	目的端软件 single 传输请求写使能。该位域控制目的端软件 single 传输请求位域（DST_SGLREQ）是否被允许写，从低到高位对应通道 0-5，只有该位置 1，对应通道的软件传输请求写入才有效。 0：目的端软件 single 传输请求写入无效； 1：目的端软件 single 传输请求写入有效；	0x0
reserved	[7:6]	-	保留	0x0
dst_sglreq	[5:0]	RW	目的端软件 single 传输请求；从低到高位对应通道 0-5； 0：目的端软件 single 传输请求无效； 1：目的端软件 single 传输请求有效；	0x0

13.6.39 LstSrcReg

◆ **偏移地址：0x388　　复位值：0x00000000**

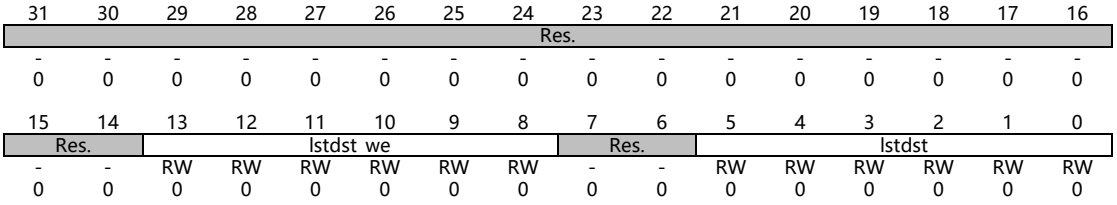


Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
lstsrc_we	[13:8]	RW	源端软件最后一次传输请求写使能。该位域控制源端软件最后一次传输请求位域（LSTSRC）是否被允许写，从低到高位对应通道 0-5，只有该位置 1，对应通道的软件传输请求写入才有效。 0：源端软件最后一次传输请求写入无效； 1：源端软件最后一次传输请求写入有效；	0x0
reserved	[7:6]	-	保留	0x0

Name	Bits	R/W	Description	Default
lstsrc	[5:0]	RW	源端软件最后一次传输请求；从低到高位对应通道 0-5； 0：源端软件最后一次传输请求无效； 1：源端软件最后一次传输请求有效；	0x0

13.6.40 LstDstReg

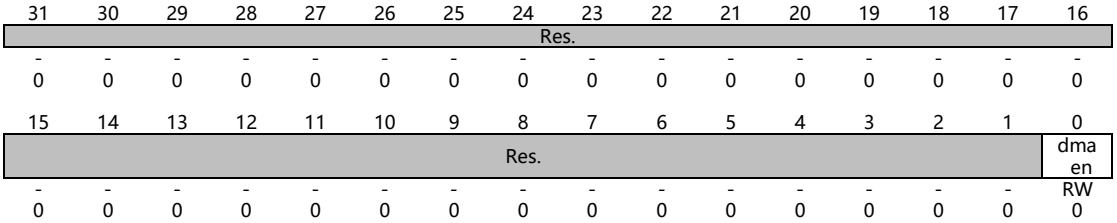
◆ **偏移地址：0x390　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
lstdst_we	[13:8]	RW	目的端软件最后一次传输请求写使能。该位域控制目的端软件最后一次传输请求位域（LSTDST）是否被允许写，从低到高位对应通道 0-5，只有该位置 1，对应通道的软件传输请求写入才有效。 0：目的端软件最后一次传输请求写入无效； 1：目的端软件最后一次传输请求写入有效；	0x0
reserved	[7:6]	-	保留	0x0
lstdst	[5:0]	RW	目的端软件最后一次传输请求；从低到高位对应通道 0-5； 0：目的端软件最后一次传输请求无效； 1：目的端软件最后一次传输请求有效；	0x0

13.6.41 DmaCfgReg

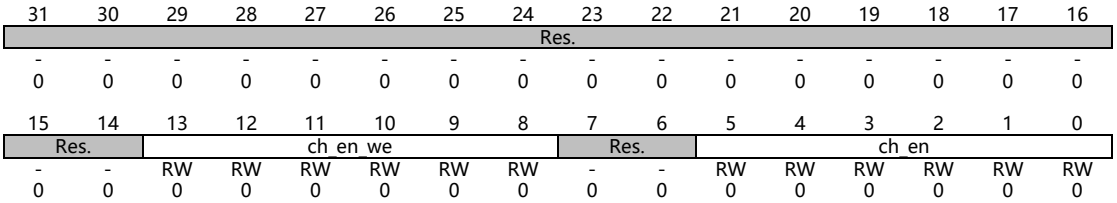
◆ **偏移地址：0x398　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
dma_en	[0]	RW	DMAC 总使能； 0：DMAC 不使能 1：DMAC 使能	0x0

13.6.42 ChEnReg

◆ **偏移地址：0x3a0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
ch_en_we	[13:8]	RW	通道使能写使能；位域从低到高一次对应通道 0-5 0：通道使能写无效 1：通道使能写有效	0x0
reserved	[7:6]	-	保留	0x0
ch_en	[5:0]	RW	DMAC 各通道使能；位域从低到高依次对应通道 0-5 0：通道不使能 1：对应通道使能	0x0

14 通用定时器（GPT0~4）

14.1 GPT 简介

GPT（General Purpose Timer）模块可用于捕获芯片管脚输入信号的边沿，记录边沿对应的时间戳，可用于通用数字信号捕获场景，也可实现电机专用场景，如：霍尔传感器，PWM 波通信等应用。

另外当 GPT 模块不用于输入捕获时，用户也可以用它实现单通道非对称脉宽调制（PWM）波形生成器（32 位）。计数器以计数模式运行，所以该模块可用作通用 timer。

14.2 GPT 主要特性（每个 GPT）

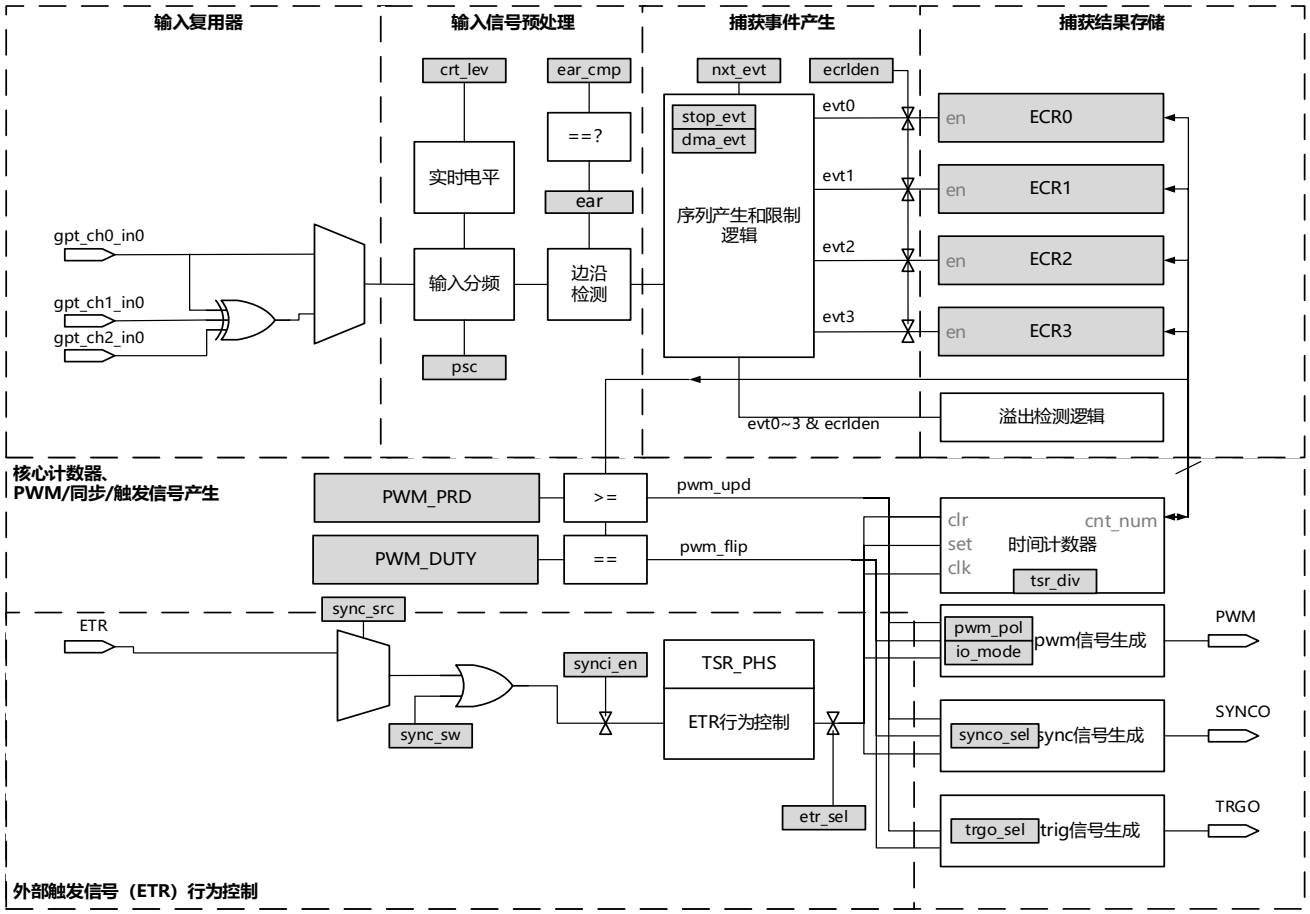
- 最多支持 4 通道捕获；
- 支持输入数字滤波（支持过滤毛刺宽度 1~4096 个时钟周期），输入预分频（2~510 偶数分频），信号在滤波后延迟的长度为配置的滤波周期；
- 支持输入电平实时检测；
- 支持边沿计数，支持边沿数量比较；
- 支持 32 比特时间戳计数，支持计数分频；
- 支持单个 GPT 通道捕获最多 4 个捕获事件，保存最多 4 个捕获事件位置的计数值，支持捕获溢出检测；
- （GPT4）支持触发 DMA 搬取捕获结果，仅支持 DMA BURST-INCR 4 传输，支持 DMA 溢出检测；
- 最多支持 4 通道 PWM 输出；
- 支持 PWM 极性配置；
- 支持 PWM 翻转与更新事件以及对应中断；
- 支持比较匹配事件脉冲同步输出；

- 支持比较匹配事件产生触发信号；
- 支持循环工作模式（捕获或产生 PWM）或者单轮次工作模式（捕获或产生 PWM）；
- 支持仿真器接入停止计数器；
- （GPT0~3）支持时间计数相位同步，APT 硬件同步，支持软件同步；

14.3 GPT 逻辑实现

图 14-1 显示了实现捕获以及 PWM 功能的逻辑框架。

图 14-1 GPT 逻辑结构图



对于 GPT0~3，它们集成在低速总线，工作频率为 CPU 频率的一半，支持与管脚、与 APT 以及其他 GPT 同步；

对于 GPT4，它集成在高速总线，工作频率与 CPU 相同，精度更好，通道之间的片内传输延迟非常小，但不支持与其他模块同步。

对于输入捕获，将输入信号进行预处理——滤波，分频后检测边沿，根据配置进行匹配，产生对应的捕获事件，捕获事件将 TSR 计数器的值锁存到 ECR 完成时间戳捕获。

对于 PWM 输出，TSR 与配置的 PWM_DUTY 和 PWM_PRIOD 分别比较，产生翻转和更新事件，事件控制 PWM 波形输出，还能产生中断或者作为 master 触发其他 Timer 行为。

由于所有 GPT 模块的所有通道的输入和输出信号都分别使用相同的引脚，并且捕获事件间时间计数和 PWM 周期计数使用唯一的计数器，所以 GPT 的输入和输出模式是互斥的，模式之间通过寄存器配置进行切换。

14.4 GPT 功能描述

表 14-1 GPT 通道地址偏移映射

OFFSET	DESCRIPTION
0x000	同时配置/控制每一个 GPT 通道的一部分功能
0x100	配置/控制 GPT 通道 0
0x200	配置/控制 GPT 通道 1
0x300	配置/控制 GPT 通道 2
0x400	配置/控制 GPT 通道 3

说明

以下配置方法针对单个 GPT 通道进行说明，如需使用多通道，请为每个通道配置。

14.4.1 GPT 默认配置

- 滤波功能默认关闭，仅同步到工作时钟，不过滤毛刺；
- 输入信号不进行预分频；
- TSR 时间计数为 0 且不计数，计数步长不分频；
- PWM_PRIOD 寄存器为 0xFFFFFFFF，PWM_DUTY 寄存器为 0x0000FFFF；
- 模式为 Capture 模式；
- 捕获终点/循环边界为 EVT3，捕获模式为循环捕获；
- DMA 触发事件为 EVT3；
- ECR 不会更新，不产生捕获事件，事件不会触发 TSR 复位；
- 同步/触发信号功能不使能；

- 不会上报任何中断；

14.4.2 Timer 模式的一般用法

步骤 1【可选】配置 INT_EN.int_pwmflip_en/int_pwmupd_en=1，开启数值比较中断；

步骤 2 配置 PWM_DUTY,PWM_PRIOD 寄存器为两个比较点，PWM_DUTY>=PWM_PRIOD，只产生 pwmupd 事件，PWM_DUTY < PWM_PRIOD,先产生 pwmflip 事件，再产生 pwmupd 事件；

步骤 3【可选】设置时间计数步长 TSR_DIV.tsr_div；

步骤 4【可选】配置 CCR1.cont_mode =0/1，选择执行一周期计数或按连续循环计数；

步骤 5 配置 CCR1.io_mode=1，开启 PWM 模式；

步骤 6 配置 CCR1.tsr_run_mode =1，启动计数器；

步骤 7【可选】如果是单周期模式，对 GENE_CR.restart_gptx 写 1 开始一次新的计数。

须知

此时对应引脚如果复用为该模块的 PWM 输出，将会输出 PWM 波形。

14.4.3 PWM 模式的一般用法

步骤 1 配置 CCR1.io_mode=1，开启 PWM 模式；

步骤 2 配置 CCR1. pwm_pol_sel 选择 PWM 波极性；

步骤 3【可选】配置 INT_EN.int_pwmflip/int_pwmupd=1，开启数值比较中断；

步骤 4 配置 PWM_DUTY,PWM_PRIOD 寄存器设置占空比和周期（要求 PWM_DUTY < PWM_PRIOD）；

步骤 5【可选】设置时间计数步长 TSR_DIV.tsr_div；

步骤 6 配置 CCR1.cont_mode =0/1，选择产生一个 PWM 周期或连续产生 PWM 波；

步骤 7 配置 CCR1.tsr_run_mode =1，启动计数器；

步骤 8【可选】配置 GENE_CR.stat_clr_gptx，清空一次工作状态；

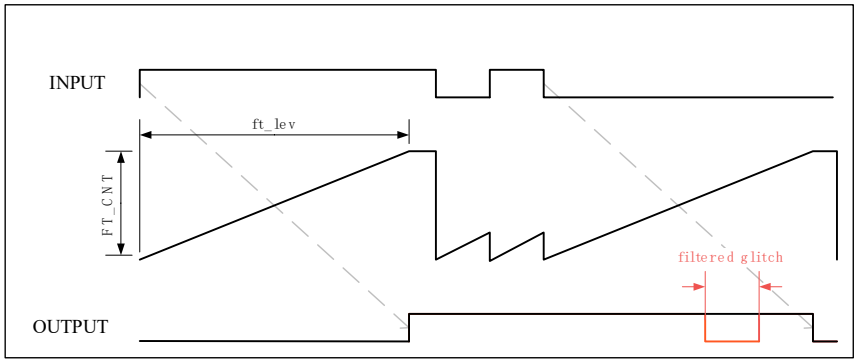
步骤 9 如果是单次模式，对 GENE_CR.restart_gptx 写 1 开始产生新的 PWM。

14.4.4 CAPTURE 模式的一般用法

步骤 1【可选】信号预处理

- 【可选】设置时间计数步长 TSR_DIV.tsr_div；
- 【可选】设置输入信号滤波 FTCR.ft_lev；

图 14-2 滤波模式示意图

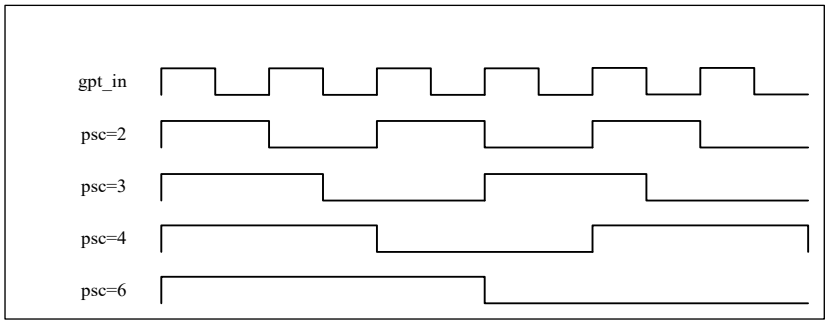


说明

如上图 14-2 所示，滤波功能将会导致滤波模块的输出 OUT 信号相对输入 IN 信号有一定延迟。

- 【可选】设置输入信号预分频系数 CCR0.psc；

图 14-3 输入分频示意图



说明

输入信号分频会使得 GPT 的边沿检测逻辑采样的信号波形如上图 14-3 所示：gpt_in 是 GPT 通道输入的待捕获信号。

步骤 2【可选】中断配置

1) 中断使能：INT_EN.int_cevt_en / int_tsrovf_en / int_ecrovf_en / int_dmaovf_en 开启或关闭;

2) 中断注入：对 INT_INJ. int_cevt_inj / int_tsrovf_inj / int_ecrovf_inj / int_dmaovf_inj 写 1;
3) 中断查询：读 INT_RAW. int_cevt_raw / int_tsrovf_raw / int_ecrovf_raw / int_dmaovf_raw 获悉原始中断状态;
4) 中断查询：读 INT_STAT. int_cevt / int_tsrovf / int_ecrovf / int_dmaovf 获悉 CPU 中断标志;
5) 中断清除：对 INT_CLR.int_cevt_clr / int_tsrovf_clr / int_ecrovf_clr / int_dmaovf_clr 写 1 清除。

说明

中断注入是指通过写寄存器直接触发相应中断，适用于在难以创造中断事件的情况下触发中断，便于调试对应的断响应程序。

步骤 3 配置工作模式

1) 配置 CCR1.io_mode=0，维持 Capture 模式;
2) 配置 CCR1.cont_mode =0/1，选择捕获一轮或按捕获事件数连续捕获;
3) 配置 CCR1.stop_evt 选择使用的事件数（序列终点/循环边界）;
4) 配置 CCR0.evtx_edge_sel 选择捕获的边沿，CCR0.evtx_clr_en 配置捕获边沿时是否清零数器;
5) 配置 CCR0.ecrlden=1，开启捕获寄存器加载使能;
6) 配置 CCR1.tsr_run_mode =1，启动计数器;
7) 【可选】配置 GENE_CR.stat_clr_gptx，清空一次工作状态;
8) 如果是单轮次模式，对 GENE_CR.restart_gptx 写 1 开始一次新的计数。

步骤 4 获取捕获时间戳。软件有如下 3 种方式读取有效捕获结果（时间戳）：

- 查询方式

  查询 ECSEQR.nxt_evt 可知捕获序列中的下一个捕获事件（也就是下一个将被更新的捕获事件的时间戳寄存器 ECR）。当 nxt_evt=x 时，表示下一个捕获事件是 EVTx，因为捕获序列是顺序递增的，也就是说捕获事件（x-1）的时间戳寄存器有效可读。

- 中断方式

  配置捕获事件的中断使能 INT_EN. int_cevtx_en (x=0,1,2,3)，在相应的捕获事件有效（也就是捕获事件的时间戳寄存器 ECR 被更新）时，上报中断。因为捕获序列是顺序递增的，也就是在中断状态 INT_STAT. int_cevtx 有效时，捕获事件(x-1)的时间戳寄存器有效可读。

- DMA 方式

  配置 DMA 请求选择 CCR1. dma_evt_sel，选择相应的捕获事件有效（也就是捕获事件的时间戳寄存器 ECR 被更新）时，产生 DMA 请求，然后从内存中读取。

须知

如果 ECR 的旧值未被读取，又有新的捕获事件触发，ECR 寄存器会被更新，并产生捕获溢出原始中断状态（若使能了捕获溢出中断，则会产生中断上报）。如果该事件 DMA 请求未撤销，又捕获到了 DMA 请求事件，则产生 DMA 溢出中断。

如果输入波形已经在输送，那么 GPT 模块刚刚使能时的捕获结果可能是不符合预期的，需要在完成捕获配置后，进行 GENE_CR.stat_clr_gptx 写 1 清除工作状态再开始使用 ECRx 内的数据。

14.4.5 ETR/软件同步功能

ETR 信号源与功能选择：

步骤 1 配置 SYNC_SELR.sync_src_gptx 选择硬件同步源；

步骤 2 配置 CCR1.etr_behavior 选择 ETR 信号功能。

说明

若同步源选择为 0，则信号来自芯片外部，需要 IO_MUX 的配置匹配，还需要额外配置 SYNC_SELR.sync_ft_lev；同时，用户硬件连接的 ETR 信号也需要注意匹配类型。

若同步源选择不为 0，则信号来自芯片内部，此时 CCR1.etr_behavior 寄存器选择的 ETR 信号功能 2 与功能 3 将不能够支持。

ETR 硬件同步 TSR 计数器：

步骤 1 配置 CCR1.etr_behavior 为 1，选择 ETR 信号功能为同步脉冲；

步骤 2 配置 TSR_PHS 寄存器设置计数器同步的相位值；

步骤 3 配置 SYNC_SELR.sync_src_gptx 选择硬件同步源；

步骤 4 配置 CCR1.synci_en =1 使能同步功能。当硬件同步源发出同步信号时，对应通道将 TSR 置位为 TSR_PHS，同时产生同步输出。

ETR 作为 TSR 计数器使能：

步骤 1 配置 CCR1.etr_behavior 为 2，选择 ETR 信号功能为高有效电平使能；

步骤 2 无需配置 CCR1.tsr_run_mode，即可使 TSR 正常计数。

ETR 使能并同步 TSR 计数器：

步骤 1 配置 CCR1.etr_behavior 为 3，选择 ETR 信号功能为高有效电平使能以及上升沿同步；

步骤 2 无需配置 CCR1.tsr_run_mode，即可使 TSR 正常计数，并且 TSR 将从 TSR_PHS 设置的值开始计数。

说明

如果在不配置 CCR0.tsr_run_mode 的情况下启用此功能，且 GPT 设置为 PWM 输出模式，那么在完成 PWM_DUTY 和 PWM_PRIOD 配置后需要配置寄存器 GENE_CR.restart_gptx 将影子模式的 PWM_DUTY 和 PWM_PRIOD 载入实际生效的寄存器，否则 ETR 上升沿处，加载动作将会覆盖同步动作。

ETR 信号触发强制 PWM 输出 1 或者 0：

步骤 1 配置 CCR1.etr_behavior 为 4 或者 5，选择 ETR 信号功能为强制 PWM 输出；

步骤 2 配置 CCR1.io_mode=1 使能输出模式；

步骤 3 ETR 信号上升沿，PWM 输出信号将会立刻变成 0 或 1，但是如果 tsr_run_mode=1，那么该强制输出只持续一个 PWM_PRIOD 的时间。

ETR 信号作为 TSR 计数时钟：

步骤 1 配置 CCR1.etr_behavior 为 6，选择 ETR 信号作为 TSR 计数时钟；

步骤 2 配置 CCR1.tsr_run_mode=1，即可使 TSR 于 ETR 信号上升沿+1。

软件同步 GPT：

步骤 1 配置 TSR_PHS 寄存器设置 TSR 置位值；

步骤 2 软件对寄存器 GENE_CR.sync_sw_gptx 写 1（多 bit 同时写 1 可以实现多通道同步）。此时，通道将 TSR 置位为 TSR_PHS。

同步/触发其他 Timer 类模块：

步骤 1 配置 CCR1.synco_sel 选择同步事件源，GPT 将在对应源事件发生时以同步信号的形式同步其他模块；

步骤 2 配置 CCR1.trgo_sel 选择触发信号事件源，GPT 将在对应源事件发生时以触发信号的形式同步其他模块。

说明

对于 GPT0~3，每个 GPT 模块会把 4 通道的同步输出结果进行逻辑或操作，或完后的结果再作为其他模块使用的同步输入信号。

14.5 GPT 应用示例

说明

以下示例并不提供一种具体的解决方案，请用户结合实际情况利用 GPT 实现具体方案。

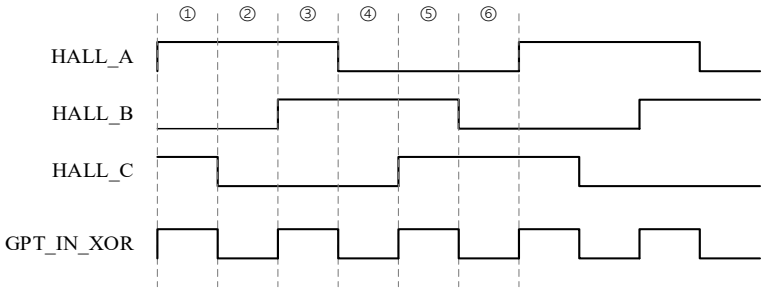
14.5.1 HALL 电机应用

霍尔编码器输出波形为 HALL_A /B /C，GPT 每通道都可以选为通道 0、1、2 的输入相异或的信号 GPT_IN_XOR，通过对霍尔信号的解析，可以得到电机的以下信息（不限于）：

- 磁对极数量（电机每转一周，其中一相的周期数）
- 转速
- 当前位置与起转位置的转角差
- 换向情况

对于图 14-4 所示的霍尔波形，①~⑥对应的是电机所在扇区。

图 14-4 霍尔传感器电机输出波形



使用三个 GPT 通道的方法

1. 连线与配置

配置 IN_SELR.input_src_gpt0/1/2=0，此时该通道的输入是芯片引脚 GPT0/1/2 的原始信号，并进行相应的捕获模式配置。

2. 运算转速

电机的磁对极数为 M，输入霍尔信号不进行分频，上升/下降沿事件时钟周期个数间距 GPT_CH0.ECR2- GPT_CH0.ECR0 = N，系统时钟频率为 f，预分频系 1 数为 SC，那么电机的实时转速 R 为：

$$R = \frac{1}{\left(SC \times N \times \frac{1}{f}\right) \times M}$$

3. 判断位置

1) **软件中断方式**：当产生 GPTx.INT_STAT.int_cevtx 事件中断时，读取 GPT 的 0,1,2 通道的 ECSEQR.nxt_evt，按照表 14-2 方式对应：

表 14-2 通过边沿事件中断确认转子位置对照表

POSITION	HALL-A/B/C	GPT_CH0.NXT_EVT	GPT_CH1.NXT_EVT	GPT_CH2.NXT_EVT
①	101	01b / 11b	00b / 10b	01b / 11b
②	100	01b / 11b	00b / 10b	00b / 10b
③	110	01b / 11b	01b / 11b	00b / 10b
④	010	00b / 10b	01b / 11b	00b / 10b
⑤	011	00b / 10b	01b / 11b	01b / 11b
⑥	001	00b / 10b	00b / 10b	01b / 11b

2) **软件轮询方式**：任意时刻，读取 GPT 通道 0,1,2 的 ECSEQR.crt_lev，按照表 14-3 方式对应：

表 14-3 通过实时电平状态查询确认电机转子位置对照表

POSITION	HALL-A/B/C	GPT_CH0.CRT_LEV	GPT_CH1.CRT_LEV	GPT_CH2.CRT_LEV
①	101	1	0	1
②	100	1	0	0
③	110	1	1	0
④	010	0	1	0
⑤	011	0	1	1
⑥	001	0	0	1

仅使用一通道 PWM 的方法

1. 接线与配置

配置 IN_SELR.input_src_gpt0/1/2=2，此时该通道的输入是芯片引脚 GPT0/1/2 相异或的信号。

额外配置 EAR_CMP，设置边沿事件数量；

额外配置 CCR0.cnt_edge_sel 选择计数边沿；

执行相应 CAPTURE 模式配置；

2. 运算转速

电机的磁对极数为 M，输入霍尔信号不进行分频，上升/下降沿事件时钟周期个数间距 GPT0.ECR2- GPT0.ECR0=N，系统时钟频率为 f，预分频系 1 数为 SC，那么此时的实时转速 R 为：

$$R = \frac{1}{3\left(SC \times N \times \frac{1}{f}\right) \times M}$$

3. 判断位置

电机的磁对极数为 M，EAR 计数值为 K，计数边沿为双沿，那么电机距离上一次 EAR 清零，已经转过的弧度 Arc 为：

$$Arc = \frac{\pi}{3} \times \frac{K}{M}$$

14.6 GPT 寄存器概览

芯片集成 5 个 GPT，其基地址分别为：

- GPT0 寄存器基地址为：0x60083000
- GPT1 寄存器基地址为：0x60084000
- GPT2 寄存器基地址为：0x60085000
- GPT3 寄存器基地址为：0x60086000
- GPT4 寄存器基地址为：0x61002800

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	GENE_CR	0x0	32	W1P	0x00000000	通道通用控制寄存器
2	SYNC_SELR	0x4	32	RW	0x00000000	同步信号源选择寄存器
3	IN_SELR	0x8	32	RW	0x00000000	输入信号源选择寄存器
4	TSR	0x100+0x100*k	32	RO	0x00000000	时间计数器

No.	Names	Offset Address	SIZE	R/W	Default	Description
5	TSR_DIV	0x104+0x100*k	32	RW	0x00000000	计数分频寄存器
6	EAR	0x108+0x100*k	32	RO	0x00000000	边沿数量计数器
7	EAR_CMP	0x10C+0x100*k	32	RW	0x0000FFFF	边沿数量比较寄存器
8	TSR_PHS	0x110+0x100*k	32	RW	0x00000000	计数器同步相位寄存器
9	ECR0	0x114+0x100*k	32	RO	0x00000000	捕获事件处计数值寄存器 0
10	ECR1	0x118+0x100*k	32	RO	0x00000000	捕获事件处计数值寄存器 1
11	ECR2	0x11C+0x100*k	32	RO	0x00000000	捕获事件处计数值寄存器 2
12	ECR3	0x120+0x100*k	32	RO	0x00000000	捕获事件处计数值寄存器 3
13	PWM_PRIOD	0x124+0x100*k	32	RW	0xFFFFFFFF	PWM 周期设置寄存器
14	PWM_DUTY	0x128+0x100*k	32	RW	0x0000FFFF	PWM 翻转位置设置寄存器
15	ECSEQR	0x12C+0x100*k	32	RO	0x00000000	捕获序列寄存器
16	FTCR	0x130+0x100*k	32	RW	0x00000000	输入滤波控制寄存器
17	CCR0	0x134+0x100*k	32	RW	0x00000000	控制与配置寄存器 0（信号预处理与捕获行为）
18	CCR1	0x138+0x100*k	32	RW	0x000000F0	控制与配置寄存器 1（工作模式设置）
19	INT_EN	0x13C+0x100*k	32	RW	0x00000000	中断使能寄存器
20	INT_RAW	0x140+0x100*k	32	RO	0x00000000	原始中断寄存器
21	INT_STAT	0x144+0x100*k	32	RO	0x00000000	中断状态寄存器
22	INT_CLR	0x148+0x100*k	32	W1P	0x00000000	中断清除寄存器
23	INT_INJ	0x14C+0x100*k	32	W1P	0x00000000	中断注入寄存器

14.7 GPT 寄存器描述

14.7.1 GENE_CR

- **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												sync_sw_gpt3	sync_sw_gpt2	sync_sw_gpt1	sync_sw_gpt0
-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				stat_clr_gpt3	stat_clr_gpt2	stat_clr_gpt1	stat_clr_gpt0	Res.				restart_gpt3	restart_gpt2	restart_gpt1	restart_gpt0
-	-	-	-	W1P	W1P	W1P	W1P	-	-	-	-	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

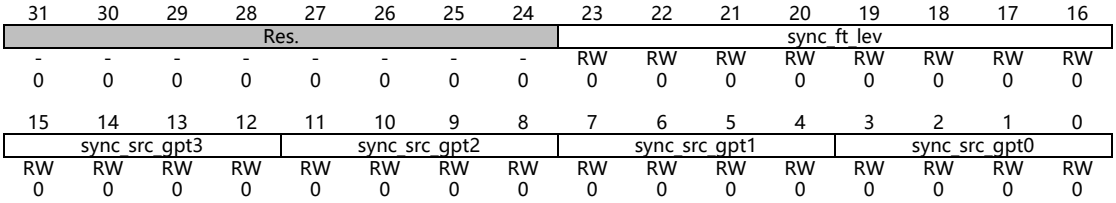
Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留	0x0
sync_sw_gpt3	[19]	W1P	写 1 触发软件同步通道 3： 使 TSR 载入 TSR_PHSR 的值 捕获模式：重置序列 PWM 模式：立即开始重新产生 PWM（应用 shadow 寄存器设置）	0x0
sync_sw_gpt2	[18]	W1P	写 1 触发软件同步通道 2： 使 TSR 载入 TSR_PHSR 的值 捕获模式：重置序列 PWM 模式：立即开始重新产生 PWM（应用 shadow 寄存器设置）	0x0
sync_sw_gpt1	[17]	W1P	写 1 触发软件同步通道 1： 使 TSR 载入 TSR_PHSR 的值 捕获模式：重置序列 PWM 模式：立即开始重新产生 PWM（应用 shadow 寄存器设置）	0x0
sync_sw_gpt0	[16]	W1P	写 1 触发软件同步通道 0： 使 TSR 载入 TSR_PHSR 的值 捕获模式：重置序列 PWM 模式：立即开始重新产生 PWM（应用 shadow 寄存器设置）	0x0
reserved	[15:12]	-	保留	0x0
stat_clr_gpt3	[11]	W1P	写 1 触发通道 3 工作状态复位： 计数分频计数器 TSR 计数器 EAR 计数器 捕获模式：ECR0~3,所有中断标记，捕获序列 PWM 模式：所有中断标记 该位适合用于将选定通道设为一个确定状态的起点	0x0
stat_clr_gpt2	[10]	W1P	写 1 触发通道 2 工作状态复位：	0x0

Name	Bits	R/W	Description	Default
			计数分频计数器 TSR 计数器 EAR 计数器 捕获模式：ECR0~3,所有中断标记，捕获序列 PWM 模式：所有中断标记 该位适合用于将选定通道设为一个确定状态的起点	
stat_clr_gpt1	[9]	W1P	写 1 触发通道 1 工作状态复位： 计数分频计数器 TSR 计数器 EAR 计数器 捕获模式：ECR0~3,所有中断标记，捕获序列 PWM 模式：所有中断标记 该位适合用于将选定通道设为一个确定状态的起点	0x0
stat_clr_gpt0	[8]	W1P	写 1 触发通道 0 工作状态复位： 计数分频计数器 TSR 计数器 EAR 计数器 捕获模式：ECR0~3,所有中断标记，捕获序列 PWM 模式：所有中断标记 该位适合用于将选定通道设为一个确定状态的起点	0x0
reserved	[7:4]	-	保留	0x0
restart_gpt3	[3]	W1P	写 1 触发通道 3 以下行为： 单次捕获模式，重置捕获序列，开始一次捕获 单次 PWM 模式，应用当前 SHADOW 配置，产生一周期 PWM 连续捕获模式，重置捕获序列 连续 PWM 模式，立即应用当前 SHADOW 配置，继续产生 PWM（默认应用时刻是 PWM 周期完成时）	0x0
restart_gpt2	[2]	W1P	写 1 触发通道 2 以下行为： 单次捕获模式，重置捕获序列，开始一次捕获 单次 PWM 模式，应用当前 SHADOW 配置，产生一周期 PWM 连续捕获模式，重置捕获序列 连续 PWM 模式，立即应用当前 SHADOW 配置，继续产生 PWM（默认应用时刻是 PWM 周期完成时）	0x0
restart_gpt1	[1]	W1P	写 1 触发通道 1 以下行为： 单次捕获模式，重置捕获序列，开始一次捕获 单次 PWM 模式，应用当前 SHADOW 配置，产生一周期 PWM 连续捕获模式，重置捕获序列 连续 PWM 模式，立即应用当前 SHADOW 配置，继续产生 PWM（默认应用时刻是 PWM 周期完成时）	0x0

Name	Bits	R/W	Description	Default
restart_gpt0	[0]	W1P	写 1 触发通道 0 以下行为： 单次捕获模式，重置捕获序列，开始一次捕获 单次 PWM 模式，应用当前 SHADOW 配置，产生一周期 PWM 连续捕获模式，重置捕获序列 连续 PWM 模式，立即应用当前 SHADOW 配置，继续产生 PWM（默认应用时刻是 PWM 周期完成时）	0x0

14.7.2 SYNC_SELR

◆ **偏移地址：0x4　　复位值：0x00000000**



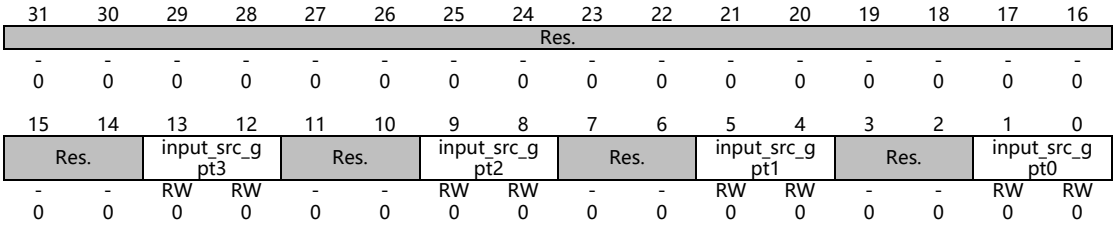
Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留	0x0
sync_ft_lev	[23:16]	RW	同步输入选择为 0，即来自 PAD 时，滤波等级选择宽度为（配置值-1）*时钟周期	0x0
sync_src_gpt3	[15:12]	RW	硬件同步源选择，12 选 1 0：ETR PAD 1：GPT1.sync_out_hub 2：GPT2.sync_out_hub 3：GPT3.sync_out_hub 对于GPT1,2,3，同步源分别为GPT0/2/3, GPT0/1/3, GPT0/1/2 4：APT0.sync_out_hub 5：APT1.sync_out_hub 6：ADC0.awd_out1 7：ADC0.awd_out2 8：ADC0.awd_out3 9：ADC1.awd_out1 10：ADC1.awd_out2 11：ADC1.awd_out3 其他：无效	0x0
sync_src_gpt2	[11:8]	RW	硬件同步源选择，12 选 1 0：ETR PAD 1：GPT1.sync_out_hub	0x0

Name	Bits	R/W	Description	Default
			2：GPT2.sync_out_hub 3：GPT3.sync_out_hub 对于 GPT1,2,3，同步源分别为 GPT0/2/3,GPT0/1/3,GPT0/1/2 4：APT0.sync_out_hub 5：APT1.sync_out_hub 6：ADC0.awd_out1 7：ADC0.awd_out2 8：ADC0.awd_out3 9：ADC1.awd_out1 10：ADC1.awd_out2 11：ADC1.awd_out3 其他：无效	
sync_src_gpt1	[7:4]	RW	硬件同步源选择，12 选 1 0：ETR PAD 1：GPT1.sync_out_hub 2：GPT2.sync_out_hub 3：GPT3.sync_out_hub 对于 GPT1,2,3，同步源分别为 GPT0/2/3,GPT0/1/3,GPT0/1/2 4：APT0.sync_out_hub 5：APT1.sync_out_hub 6：ADC0.awd_out1 7：ADC0.awd_out2 8：ADC0.awd_out3 9：ADC1.awd_out1 10：ADC1.awd_out2 11：ADC1.awd_out3 其他：无效	0x0
sync_src_gpt0	[3:0]	RW	硬件同步源选择，12 选 1 0：ETR PAD 1：GPT1.sync_out_hub 2：GPT2.sync_out_hub 3：GPT3.sync_out_hub 对于 GPT1,2,3，同步源分别为 GPT0/2/3,GPT0/1/3,GPT0/1/2 4：APT0.sync_out_hub 5：APT1.sync_out_hub 6：ADC0.awd_out1 7：ADC0.awd_out2 8：ADC0.awd_out3 9：ADC1.awd_out1	0x0

Name	Bits	R/W	Description	Default
			10：ADC1.awd_out2 11：ADC1.awd_out3 其他：无效	

14.7.3 IN_SELR

◆ **偏移地址：0x8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
input_src_gpt3	[13:12]	RW	通道 3 输入源选择， 0：gpt_in0 2：gpt_in_xor(通道 0，1，2 的 gpt_in0 相互异或) 其他：关闭信号输入	0x0
reserved	[11:10]	-	保留	0x0
input_src_gpt2	[9:8]	RW	通道 2 输入源选择， 0：gpt_in0 2：gpt_in_xor(通道 0，1，2 的 gpt_in0 相互异或) 其他：关闭信号输入	0x0
reserved	[7:6]	-	保留	0x0
input_src_gpt1	[5:4]	RW	通道 1 输入源选择， 0：gpt_in0 2：gpt_in_xor(通道 0，1，2 的 gpt_in0 相互异或) 其他：关闭信号输入	0x0
reserved	[3:2]	-	保留	0x0
input_src_gpt0	[1:0]	RW	通道 0 输入源选择， 0：gpt_in0 2：gpt_in_xor(通道 0，1，2 的 gpt_in0 相互异或) 其他：关闭信号输入	0x0

14.7.4 TSR

◆ **偏移地址：0x100+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
tsr															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tsr															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
tsr	[31:0]	RO	读取计数器同步相位值	0x0

14.7.5 TSR_DIV

◆ **偏移地址：0x104+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tsr_div															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
tsr_div	[15:0]	RW	计数步长设置，计数步长=tsr_div+1 个 GPT 工作时钟周期	0x0

14.7.6 EAR

◆ **偏移地址：0x108+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ear															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
ear	[15:0]	RO	边缘数量计数器，对 CCR0 寄存器 cnt_edge_sel 域设置的边缘类型进行计数	0x0

14.7.7 EAR_CMP

◆ **偏移地址：0x10C+0x100*k　　复位值：0x0000FFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ear_cmp															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
ear_cmp	[15:0]	RW	边缘检测数量设置，当 EAR=EAR_CMP 时，产生内部比较匹配事件 该事件可作为中断上报	0xFFFF

14.7.8 TSR_PHS

◆ **偏移地址：0x110+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
tsr_phs															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tsr_phs															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
tsr_phs	[31:0]	RW	设置计数器同步相位值	0x0

14.7.9 ECR0

◆ **偏移地址：0x114+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ecr0															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ecr0															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ecr0	[31:0]	RO	捕获事件 0 发生时，TSR 寄存器的值锁存至该寄存器	0x0

14.7.10 ECR1

◆ **偏移地址：0x118+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ecr1															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ecr1															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ecr1	[31:0]	RO	捕获事件 1 发生时，TSR 寄存器的值锁存至该寄存器	0x0

14.7.11 ECR2

◆ **偏移地址：0x11C+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ecr2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ecr2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ecr2	[31:0]	RO	捕获事件 2 发生时，TSR 寄存器的值锁存至该寄存器	0x0

14.7.12 ECR3

◆ **偏移地址：0x120+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ecr3															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ecr3															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ecr3	[31:0]	RO	捕获事件 3 发生时，TSR 寄存器的值锁存至该寄存器	0x0

14.7.13 PWM_PRIOD

◆ **偏移地址：0x124+0x100*k** **复位值：0xFFFFFFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pwm_priod															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pwm_priod															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
pwm_priod	[31:0]	RW	TSR=PWM_PRIOD，PWM 周期完成，TSR 清零	0xFFFFFFFF

14.7.14 PWM_DUTY

◆ **偏移地址：0x128+0x100*k** **复位值：0x0000FFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pwm_duty															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pwm_duty															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
pwm_duty	[31:0]	RW	TSR=PWM_DUTY 时，PWM 输出电平翻转	0x0000FFFF

14.7.15 ECSEQR

◆ **偏移地址：0x12C+0x100*k** **复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

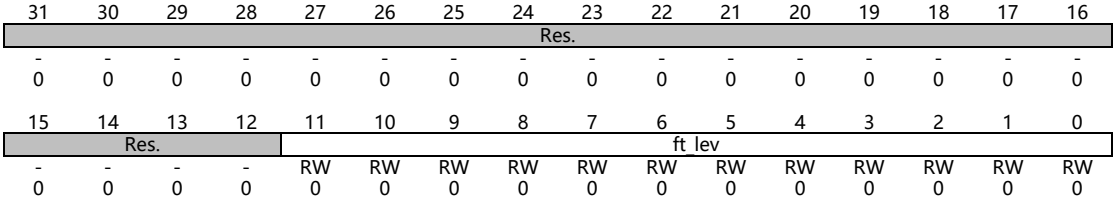
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.											crt_lev	Res.		nxt_evt	
-	-	-	-	-	-	-	-	-	-	-	RO	-	-	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0

Name	Bits	R/W	Description	Default
crt_lev	[4]	RO	指示实时电平 0：低电平 1：高电平	0x0
reserved	[3:2]	-	保留	0x0
nxt_evt	[1:0]	RO	指示即将被加载的 ECR 0：ECR0 1：ECR1 2：ECR2 3：ECR3	0x0

14.7.16 FTCR

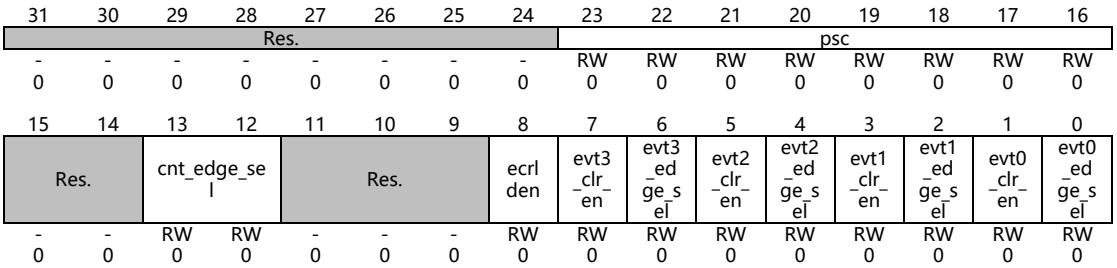
◆ **偏移地址：0x130+0x100*k　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
ft_lev	[11:0]	RW	滤波等级配置。 0：不过滤毛刺； 1：过滤最宽 1 时钟周期宽度毛刺； ... 4095：过滤最宽 4095 时钟周期宽度毛刺。	0x0

14.7.17 CCR0

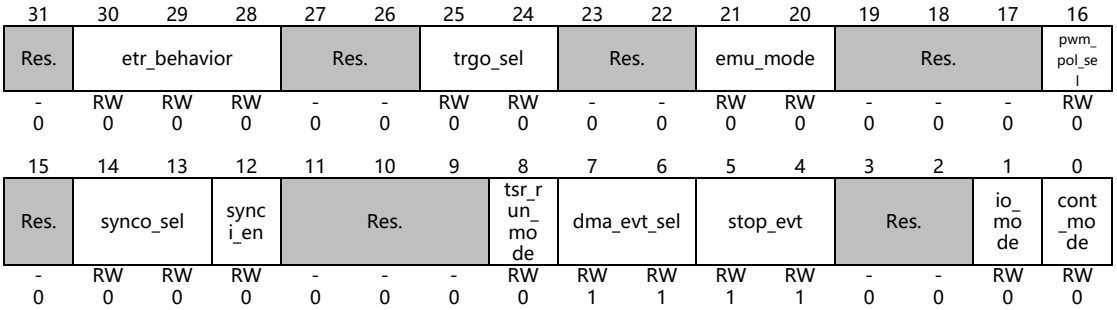
◆ **偏移地址：0x134+0x100*k　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留	0x0
psc	[23:16]	RW	输入信号预分频系数，GPT 识别的信号周期将等于 psc*2 倍的原始输入信号周期	0x0
reserved	[15:14]	-	保留	0x0
cnt_edge_sel	[13:12]	RW	计数边沿类型选择 0：不计数 1：对上升沿计数 2：对下降沿计数 3：对双沿计数	0x0
reserved	[11:9]	-	保留	0x0
ecrlden	[8]	RW	产生捕获事件并在事件发生时将 TSR 锁存至对应 ECR 0：禁能 1：使能	0x0
evt3_clr_en	[7]	RW	事件 3 产生时清零 TSR： 0：不清零 1：清零	0x0
evt3_edge_sel	[6]	RW	事件 3 的对应输入信号边沿类型： 0：上升沿 1：下降沿	0x0
evt2_clr_en	[5]	RW	事件 2 产生时清零 TSR： 0：不清零 1：清零	0x0
evt2_edge_sel	[4]	RW	事件 2 的对应输入信号边沿类型： 0：上升沿 1：下降沿	0x0
evt1_clr_en	[3]	RW	事件 1 产生时清零 TSR： 0：不清零 1：清零	0x0
evt1_edge_sel	[2]	RW	事件 1 的对应输入信号边沿类型： 0：上升沿 1：下降沿	0x0
evt0_clr_en	[1]	RW	事件 0 产生时清零 TSR： 0：不清零 1：清零	0x0
evt0_edge_sel	[0]	RW	事件 0 的对应输入信号边沿类型： 0：上升沿 1：下降沿	0x0

14.7.18 CCR1

◆ **偏移地址：0x138+0x100*k　　复位值：0x000000F0**



Name	Bits	R/W	Description	Default
reserved	[31]	-	保留	0x0
etr_behavior	[30:28]	RW	外部触发信号行为（通过 SYNC_SELR 寄存器设置每通道的外部触发信号源） 0：无任何效果 1：ETR 上升沿，同步 TSR 2：ETR 有效时，ETR 高电平代替 tsr_run_mode 高电平 3：ETR 上升沿同步一次 TSR 并于 ETR 有效期间使能 TSR 4：ETR 有效时，强制 PWM 输出 1 5：ETR 有效时，强制 PWM 输出 0 6：TSR 在 ETR 上升沿计数 其他：无任何效果 注意当 sync_src 的配置不为 0（来自引脚）时，ETR 无法使用 2/3 这两个配置。	0x0
reserved	[27:26]	-	保留	0x0
trgo_sel	[25:24]	RW	产生触发信号输出的事件选择： 0/3：无触发 1：PWM 翻转 2：PWM 更新（周期完成）	0x0
reserved	[23:22]	-	保留	0x0
emu_mode	[21:20]	RW	调试器接入时 TSR 的行为选择： 0：TSR 立即停止 1：TSR=0 时（完成最后一个 PWM 周期时）停止 2，3：调试器接入不影响 TSR	0x0
reserved	[19:17]	-	保留	0x0
pwm_pol_sel	[16]	RW	PWM 模式波形极性选择： 0：在一周期内，先输出高电平再输出低电平	0x0

Name	Bits	R/W	Description	Default
			1：在一周期内，先输出低电平再输出高电平	
reserved	[15]	-	保留	0x0
synco_sel	[14:13]	RW	产生同步输出信号的事件选择： 0：禁用同步输出 1：将软件同步事件直接输出 2：PWM 模式下将比较事件（翻转点）作为同步信号输出 3：PWM 模式下将更新事件作为同步信号输出	0x0
synci_en	[12]	RW	TSR 同步功能开关： 0：关闭相位同步功能 1：可以被软件或者外部触发信号（ETR）信号同步	0x0
reserved	[11:9]	-	保留	0x0
tsr_run_mode	[8]	RW	TSR 运行模式选择： 0：TSR 不计数（捕获没有时间戳，无法产生 PWM） 1：连续模式下 TSR free run，单次模式捕获序列，或者产生一周期 PWM 后，TSR 复位	0x0
dma_evt_sel	[7:6]	RW	触发 DMA 的事件选择： 0：事件 0 触发 DMA 1：事件 1 触发 DMA 2：事件 2 触发 DMA 3：事件 3 触发 DMA	0x3
stop_evt	[5:4]	RW	捕获序列长度选择： 0：只产生事件 0 1：产生事件 0，1 2：产生事件 0，1，2 3：产生事件 0，1，2，3	0x3
reserved	[3:2]	-	保留	0x0
io_mode	[1]	RW	0：输入信号捕获（Capture）模式 1：PWM 输出模式	0x0
cont_mode	[0]	RW	0：连续捕获/产生 PWM 模式 1：单次捕获/产生 PWM 模式	0x0

14.7.19 INT_EN

◆ **偏移地址：0x13C+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.						int_dmaovf_en	int_ecrovf_en	int_tsrovf_en	int_pwmupd_en	int_pwmflip_en	int_earcmp_en	int_cevt3_en	int_cevt2_en	int_cevt1_en	int_cevt0_en
-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留	0x0
int_dmaovf_en	[9]	RW	DMA 溢出中断使能 0：不使能 1：使能	0x0
int_ecrovf_en	[8]	RW	事件捕获溢出中断使能 0：不使能 1：使能	0x0
int_tsrovf_en	[7]	RW	计数器溢出中断使能 0：不使能 1：使能	0x0
int_pwmupd_en	[6]	RW	PWM 周期更新事件中断使能 0：不使能 1：使能	0x0
int_pwmflip_en	[5]	RW	PWM 翻转事件中断使能 0：不使能 1：使能	0x0
int_earcmp_en	[4]	RW	边沿计数比较匹配中断使能 0：不使能 1：使能	0x0
int_cevt3_en	[3]	RW	捕获事件 3 中断使能 0：不使能 1：使能	0x0
int_cevt2_en	[2]	RW	捕获事件 2 中断使能 0：不使能 1：使能	0x0
int_cevt1_en	[1]	RW	捕获事件 1 中断使能 0：不使能 1：使能	0x0
int_cevt0_en	[0]	RW	捕获事件 0 中断使能 0：不使能 1：使能	0x0

14.7.20 INT_RAW

◆ **偏移地址：0x140+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.						int_dmaovf_raw	int_ecrovf_raw	int_tsrovf_raw	int_pwmupd_raw	int_pwmflip_raw	int_earcmp_raw	int_cevt3_raw	int_cevt2_raw	int_cevt1_raw	int_cevt0_raw
-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留	0x0
int_dmaovf_raw	[9]	RO	DMA 溢出中断原始状态（不受使能影响）	0x0
int_ecrovf_raw	[8]	RO	事件捕获溢出中断原始状态（不受使能影响）	0x0
int_tsrovf_raw	[7]	RO	计数器溢出中断原始状态（不受使能影响）	0x0
int_pwmupd_raw	[6]	RO	PWM 周期更新事件中断原始状态（不受使能影响）	0x0
int_pwmflip_raw	[5]	RO	PWM 翻转事件中断原始状态（不受使能影响）	0x0
int_earcmp_raw	[4]	RO	边沿计数比较匹配中断原始状态（不受使能影响）	0x0
int_cevt3_raw	[3]	RO	捕获事件 3 中断原始状态（不受使能影响）	0x0
int_cevt2_raw	[2]	RO	捕获事件 2 中断原始状态（不受使能影响）	0x0
int_cevt1_raw	[1]	RO	捕获事件 1 中断原始状态（不受使能影响）	0x0
int_cevt0_raw	[0]	RO	捕获事件 0 中断原始状态（不受使能影响）	0x0

14.7.21 INT_STAT

◆ **偏移地址：0x144+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.						int_dmaovf	int_ecrovf	int_tsrovf	int_pwmupd	int_pwmflip	int_earcmp	int_cevt3	int_cevt2	int_cevt1	int_cevt0
-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留	0x0
int_dmaovf	[9]	RO	DMA 溢出中断状态	0x0
int_ecrovf	[8]	RO	事件捕获溢出中断状态	0x0
int_tsrovf	[7]	RO	计数器溢出中断状态	0x0
int_pwmupd	[6]	RO	PWM 周期更新事件中断状态	0x0
int_pwmflip	[5]	RO	PWM 翻转事件中断状态	0x0
int_earcmp	[4]	RO	边沿计数比较匹配中断状态	0x0
int_cevt3	[3]	RO	捕获事件 3 中断状态	0x0
int_cevt2	[2]	RO	捕获事件 2 中断状态	0x0
int_cevt1	[1]	RO	捕获事件 1 中断状态	0x0
int_cevt0	[0]	RO	捕获事件 0 中断状态	0x0

14.7.22 INT_CLR

◆ **偏移地址：0x148+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.						int_dmaovf_clr	int_ecrovf_clr	int_tsrovf_clr	int_pwmupd_clr	int_pwmflip_clr	int_earcmp_clr	int_cevt3_clr	int_cevt2_clr	int_cevt1_clr	int_cevt0_clr
-	-	-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留	0x0
int_dmaovf_clr	[9]	W1P	DMA 溢出中断清除（写 1 清）	0x0
int_ecrovf_clr	[8]	W1P	事件捕获溢出中断清除（写 1 清）	0x0
int_tsrovf_clr	[7]	W1P	计数器溢出中断清除（写 1 清）	0x0

Name	Bits	R/W	Description	Default
int_pwmupd_clr	[6]	W1P	PWM 周期更新事件中断清除（写 1 清）	0x0
int_pwmflip_clr	[5]	W1P	PWM 翻转事件中断清除（写 1 清）	0x0
int_earcmp_clr	[4]	W1P	边沿计数比较匹配中断清除（写 1 清）	0x0
int_cevt3_clr	[3]	W1P	捕获事件 3 中断清除（写 1 清）	0x0
int_cevt2_clr	[2]	W1P	捕获事件 2 中断清除（写 1 清）	0x0
int_cevt1_clr	[1]	W1P	捕获事件 1 中断清除（写 1 清）	0x0
int_cevt0_clr	[0]	W1P	捕获事件 0 中断清除（写 1 清）	0x0

14.7.23 INT_INJ

◆ **偏移地址：0x14C+0x100*k　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.						int_dmaovf_inj	int_ecrovf_inj	int_tsrovf_inj	int_pwmupd_inj	int_pwmflip_inj	int_earcmp_inj	int_cevt3_inj	int_cevt2_inj	int_cevt1_inj	int_cevt0_inj
-	-	-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留	0x0
int_dmaovf_inj	[9]	W1P	DMA 溢出中断注入（写 1）	0x0
int_ecrovf_inj	[8]	W1P	事件捕获溢出中断注入（写 1）	0x0
int_tsrovf_inj	[7]	W1P	计数器溢出中断注入（写 1）	0x0
int_pwmupd_inj	[6]	W1P	PWM 周期更新事件中断注入（写 1）	0x0
int_pwmflip_inj	[5]	W1P	PWM 翻转事件中断注入（写 1）	0x0
int_earcmp_inj	[4]	W1P	边沿计数比较匹配中断注入（写 1）	0x0
int_cevt3_inj	[3]	W1P	捕获事件 3 中断注入（写 1）	0x0
int_cevt2_inj	[2]	W1P	捕获事件 2 中断注入（写 1）	0x0
int_cevt1_inj	[1]	W1P	捕获事件 1 中断注入（写 1）	0x0
int_cevt0_inj	[0]	W1P	捕获事件 0 中断注入（写 1）	0x0

15 正交解码模块（QDM）

15.1 QDM 简介

QDM（Quard-encode-pulse Decode Module）模块用于解析芯片管脚输入的正交/旋转编码器数字信号（由物理量编码形成），并转换为位置和时间信息供应用程序计算。

本章节介绍了 QDM 模块的工作原理，具备的逻辑组件以及实现某些功能的配置顺序。

AiM51 芯片有 2 个相互独立的 QDM 模块。

15.2 QDM 主要特性

- 支持 6 种输入模式解析
- 支持输入数字滤波，连接关系/极性交换
- I(index),S(strobe)可配置为位置基准输入或者位置比较匹配同步输出
- 支持 32bit 单位时间计数器 UTMR，计数周期 UPRD 可配
- 支持 32bit 位置计数器 POS_CNT，支持 4 种复位模式，3 种初始化模式，3 种锁存模式
- 支持 M 法，又称 ΔX/T 法计算速度，即在单位时间内采集位置变量
- 支持最大 32bit 计数器采集移动 1~2048（2^n）个单位位置所经过的时间
- 支持 16bit 专用看门狗进行输入丢失检测，无需软件喂狗，看门狗的计时单位为 QDM 工作时钟的 4096 倍
- 支持多种位置信息上报，详见 QEP_STAT 寄存器，位置信息与异常中断，中断支持原始状态上报，屏蔽以及注入

15.3 QDM 逻辑实现

15.3.1 QDM 与引脚、对端器件的连接

QDM 模块具有两个输入方向接口和两个双向接口：

QDMx_A & QDMx_B ：编码器的输入，连接正交编码器时对接编码器 A、B 相，连接脉冲-方向编码器时，分别对接编码器 XCLK 和 XDIR 相。

QDMx_INDEX ：作为输入方向时，为编码器 0 位置脉冲信号；作为输出方向时，可选为位置匹配同步脉冲输出。

QDMx_STROBE：作为输入方向时，为可额外用于锁存和清除位置计数器的控制信号；作为输出方向时，可选为位置匹配同步脉冲输出。

说明

同一时间 QDMx_INDEX 和 QDMx_STROBE 只有一个可以被配置为输出方向。

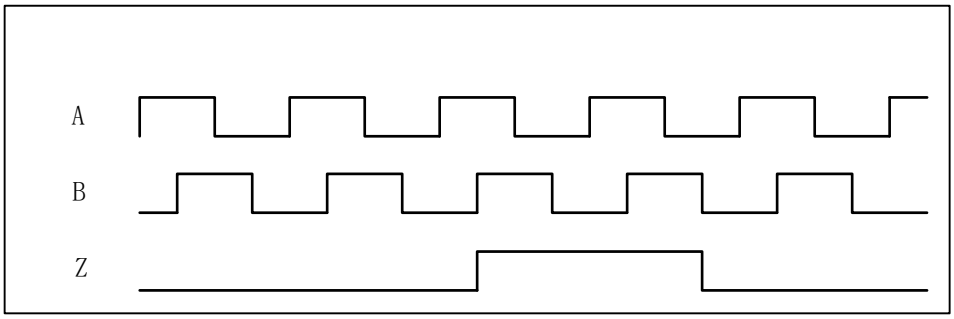
使用 QDM 模块时，请参考 IO MUX 章节将芯片引脚的 IO 模式配置成 QDMx_A / QDMx_B / QDMx_INDEX / QDMx_STROBE。

通常情况下，市售的光电旋转编码器的 A、B、Z 线分别与 QDMx_A / QDMx_B / QDMx_INDEX 一一对应。

QDM 模块支持以下几种编码模式的解码：

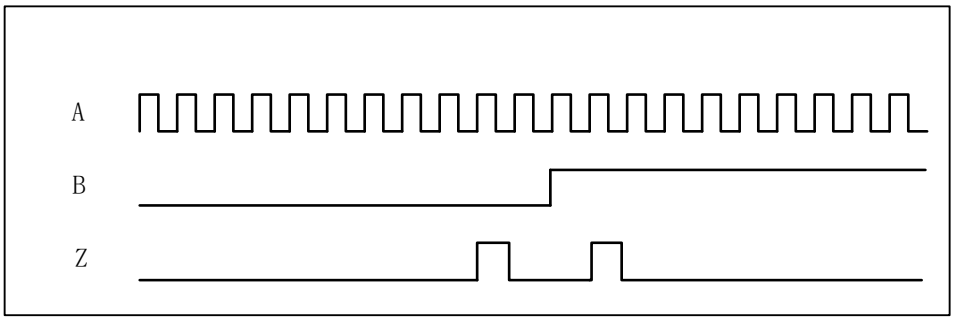
1. 正交模式

图 15-1 正交编码器输出波形示意图



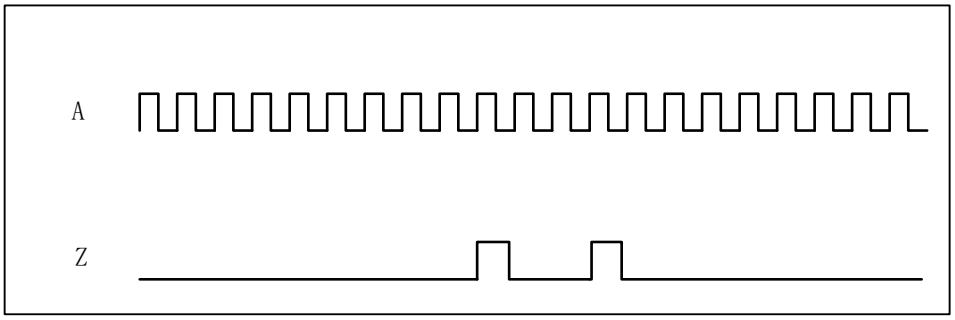
2. 脉冲-方向模式

图 15-2 脉冲-方向编码输出波形示意图



3. 脉冲递增或递减模式

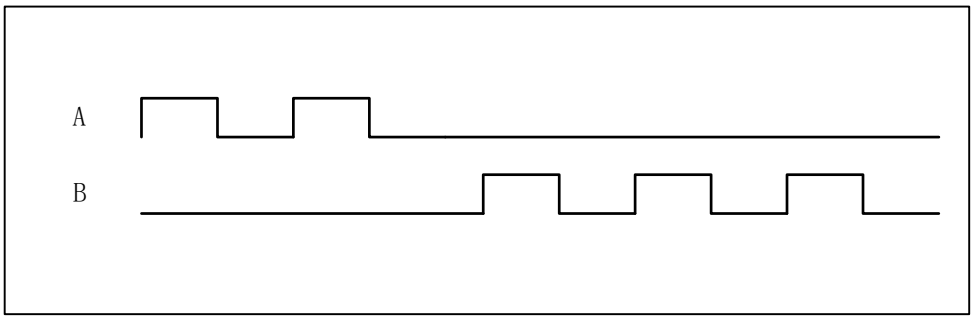
图 15-3 脉冲-递增/递减编码器输出波形示意图



(脉冲递增或者递减模式的“增”和“减”判断是通过 QDM 寄存器配置的)

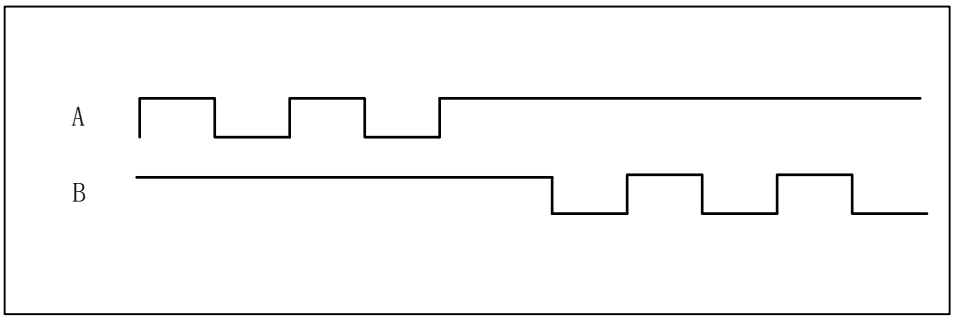
4. 非标准模式 1

图 15-4 非标准模式 1 编码器输出波形示意图



5. 非标准模式 2

图 15-5 非标准模式 2 编码器输出波形示意图



（QDM 通过配置适配模式将 A 和 B 信号拟合成脉冲-方向模式）

15.3.2 QDM 与片内逻辑的连接

QDM 的所有逻辑工作时钟都与它所挂载的高速 AHB 总线子系统相同，在最多 200MHz 的主频下能够提供 5ns 的分辨精度。请注意，在利用 QDM 的计时器结果进行计算时，会频繁使用到“工作时钟周期”这一概念。

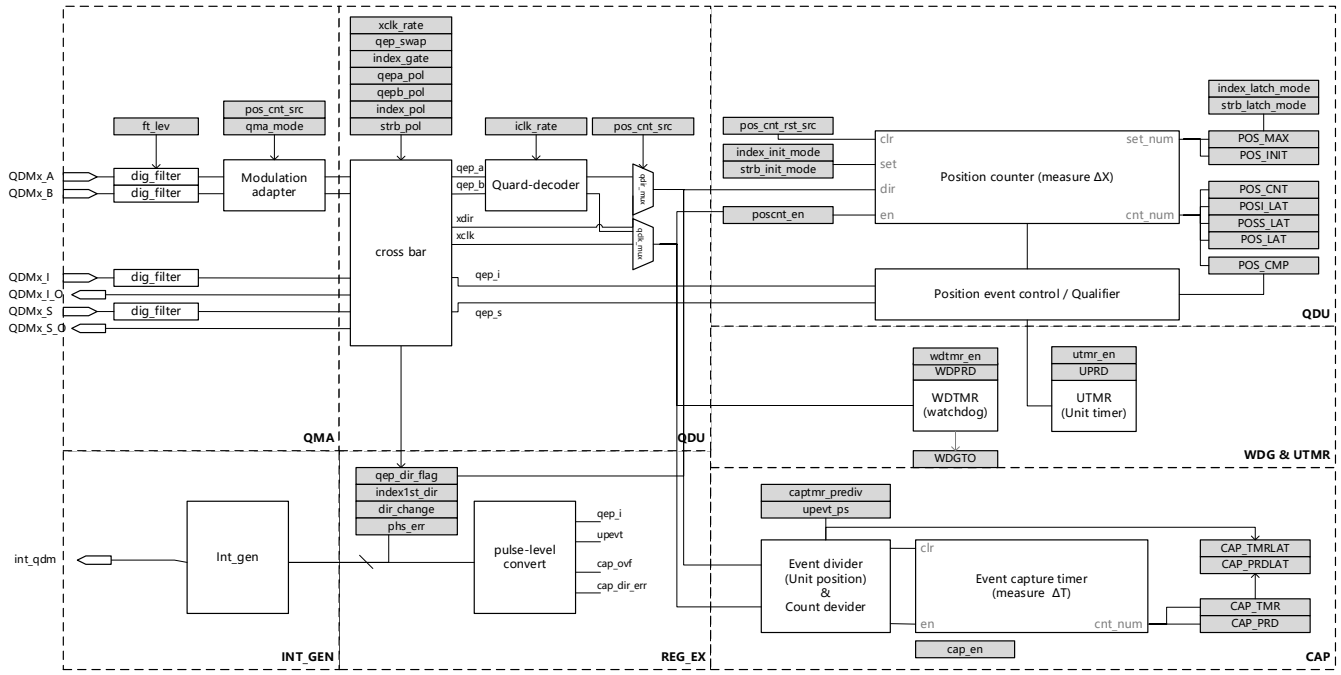
每个 QDM 支持一个多中断源汇聚的中断请求，每个中断源可以单独使能，支持原始中断事件和已使能中断状态的查询；还支持中断注入功能，在真实的中断触发条件难以达成时通过配置寄存器的方式直接触发中断。

另外，在 CPU CLIC 的配置中也可以完全屏蔽某一个 QDM 的中断请求。

15.3.3 QDM 逻辑框图

QDM 逻辑框图如下图 15-6 所示。

图 15-6 QDM 逻辑框图



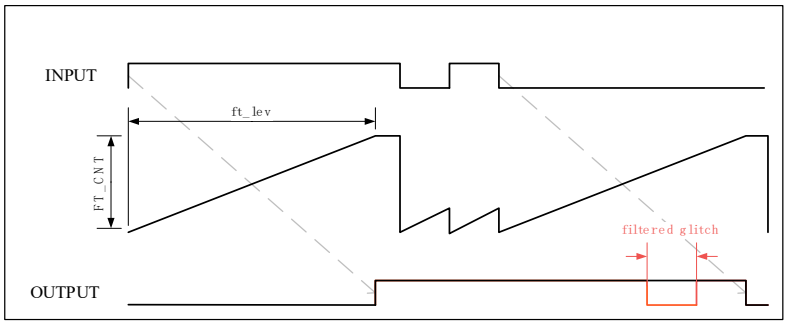
灰色部分代表这部分逻辑和软件交互的部分

15.3.3.1 正交适配器 QMA（QEP Modulation Adapter）

它的功能是进行信号预处理以及非标准正交波形拟合。如果输入信号不符合配置的解码模式，QMA 还可以产生异常提示信息。

软件可以通过配置 ft_lev 消除引脚输入信号 ft_lev*工作时钟周期的毛刺从而减少误差，但是这将引入与 ft_lev*工作时钟周期长度的信号延迟。

图 15-7 滤波模式示意图（OUTPUT 为数字滤波器输出）



配置适配模式：

如果当前输入信号的编码模式是非标准模式 1 或 2，可以配置 QMA_CTRL.qma_mode =1 或 2，将输入信号适配为脉冲-方向模式；如果当前输入信号的编码模式是标准的正交或脉冲方向模式，保持 QMA_CTRL.qma_mode=0。

15.3.3.2 正交解码单元 QDU（QEP Decoding Unit）

它的功能是将原始或适配后的 A 和 B 相信号、INDEX 和 STROB 相信号进行极性的配置、调换 A 和 B 的相位、使用 STROBE 钳位（逻辑与）INDEX 相信号。参考 15.7.15 QD_CTRL 寄存器。

极性选取与换相后，将 A\B 相解析为转动方向信号 QDIR 和位置计数信号 QCLK。

通过 QD_CTRL.pos_cnt_src 设置解码模式；

通过 QD_CTRL.iclk_rate 设置正交模式的倍频系数（分辨率），通过 QD_CTRL.xclk_rate 设置其他模式的倍频系数；

正交编码器输出信号或者脉冲-方向信号会以图 15-8 的方式被解析为 QDM 逻辑内部使用的 QCLK 计数脉冲。此外正交模式下，芯片内部逻辑会识别 A 与 B 相信号的相位关系自动判断旋转方向，产生 QDIR 信号。脉冲-方向模式下，QDIR=XDIR；脉冲递增或递减模式下，QDIR=1 或 0。

图 15-8 正交模式解码分辨率示意图

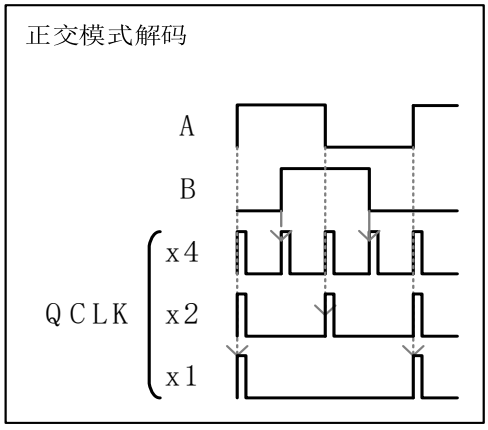
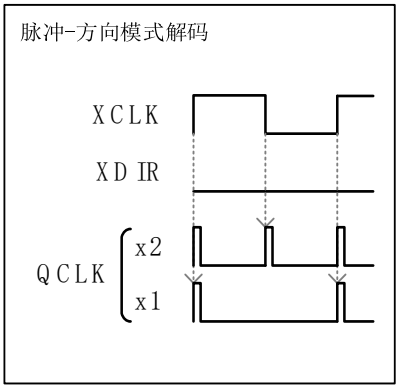


图 15-9 脉冲-方向模式解码分辨率示意图



QDU 还会识别 INDEX 相的上升/下降沿或者 INDEX 有效期间的第一个 QCLK 作为“INDEX 事件（Index event）”，当第一个 INDEX 事件发生，事件状态和事件发生时的方向都可以在 QEP_STAT 寄存器查询。

说明

INDEX 事件在物理上可以被认为是编码器上的一个精确的零位置。本章 15.5.4 电机参数检查小节就是利用这一特性的示例。

15.3.3.3 位置计数控制单元 PCCU（Position Counting Control Unit）

根据解码得到的计数脉冲和方向信息，进行位置计数。

计数实时状态可以通过 POS_CNT 寄存器查询，适用于调试情形下确认运行状态。

POS_CNT 位置计数器的使能开关是 QEP_CTRL.pos_cnt_en，在使能时，PCCU 有以下特性：

1) 当 QCLK 到来，若 QDIR=1，则递增，否则递减。

2) 当 QEP_CTRL.pos_cnt_rst_src 设置的条件满足时，若 QDIR=1，复位到 0；否则复位到 POS_MAX 寄存器的设定值。

3) 当 QEP_CTRL.index_init_mode 或 QEP_CTRL.strb_init_mode 设置的条件满足时，亦或是 QEP_CTRL.sw_init_pos_cnt 被写入 1；POS_CNT 被置位为 POS_INIT 寄存器设定的值。

4) 当 QEP_CTRL.index_latch_mode 设置的条件满足时，POS_CNT 的值将被锁存到 POSI_LAT 寄存器。

5) 当 QEP_CTRL.strobe_latch_mode 设置的条件满足时，POS_CNT 的值将被锁存到 POSS_LAT 寄存器。

6) 当 QEP_CTRL.cap_latch_mode=0 时，软件读取 CAP_TMR、CAP_PRD 或者 POS_CNT 任意一个寄存器的同时，POS_CNT 的值将被锁存到 POS_LAT 寄存器；而当 QEP_CTRL.cap_latch_mode=1 时，单位时间事件产生时，POS_CNT 的值将被锁存到 POS_LAT 寄存器。

PCCU 还可以通过设置 POS_CMP 来设定一个位置计数比较值，每当 POS_CNT = POS_CMP，且 INDEX 或者 STROBE 被配置为输出，将输出一个通过 POS_CTRL 寄存器设置脉宽和极性的同步脉冲信号（POS_CTRL 还包含其他相关设置，详见寄存器描述）。

15.3.3.4 单位时间计数器 UTMR（Unit Timer）

通过固定时钟周期数的计数，周期性地提供单位时间事件。

UTMR 的使能开关是 QEP_CTRL.utmr_en，当使能开启，32bit 的 UTMR 计数器开始递增， 当达到 UPRD 时，计数器清零并产生单位时间事件。该事件可产生中断。

每隔 1 个高速 AHB 总线时钟周期，UTMR 计数器计数值加 1。

说明

POS_CNT 可以在 UTEVT（单位时间事件）处发生锁存，此时 POS_LAT 与 UPRD 寄存器分别提供了单位时间内的位移量。可用于速度计算，即 V =ΔX / T。

15.3.3.5 事件捕获模块 CAP（Capture unit）

事件捕获模块可以每隔数个 QCLK 产生一个 UPEVT（单位位置事件），通过寄存器 CAP_CTRL.upevt_ps 设置，每间隔 2 的 upevt_ps 次幂个 QCLK 产生一个 UPEVT。

当 CAP_CTRL.cap_en 使能开关打开，位宽 16bit 的 CAP_TMR 的计数器将开始递增（并且该计数器支持最多 8 分频，分频通过 CAP_CTRL.captmr_prediv 配置），每当 UPEVT 有效，CAP_TMR 的值将被锁存到 CAP_PRD 寄存器中。CAP_TMR 自身被清零。

为了消除软件依次查询造成的误差，当 CAP_TMR，CAP_PRD，POS_CNT 任何一个寄存器被读取时，其余的寄存器内容将分别对应地被锁存到 CAP_TMRLAT，CAP_PRDLAT，POS_LAT。

说明

CAP_TMR 可以在 UPEVT（单位位置事件）处发生锁存，此时 CAP_PRD 与 CAP_CTRL.upevt_ps 寄存器分别提供了固定位移经过的时间。也可用于速度计算，即 V =X / ΔT。

如果在两个 UPEVT 之间 qdir 反转，CAP_PRD 的值将为 0xFFFF，CAP 模块会产生指示上述状态的中断事件，代表这一组测量值是无效的。

15.3.3.6 看门狗模块 WDG（WatchDog）

16bit 递增计数器用于检测过宽的 QCLK 间隔，QCLK 间隔越长，说明正交输入信号的变化越缓慢，这可能代表着对端器件的异常。

通过 QEP_CTRL.wdg_en 使能后，每当检测到 QCLK，WDTMR 计数器就会清零，当 WDTMR 达到 WDPRD 寄存器所设定的值，说明已有足够长时间未产生 QCLK，此时 WDG 模块产生异常事件，该事件可产生中断。

每隔 64 个高速 AHB 总线时钟周期，WDTMR 计数器计数值加 1。

15.4 QDM 功能描述

说明

以下配置方法针对单个 QDM 进行说明，两个 QDM 的使用是完全独立的。

15.4.1 QDM 默认配置

- 滤波功能默认关闭，仅同步到工作时钟，不过滤毛刺。
- 所有信号均为输入方向，不进行任何适配或预处理。
- POS_CNT，CAP_TMR，WDTMR，UTMR 计数值为 0 且未使能，WDTMR 具有默认分频而其他计数器不具有分频。
- 不会进行位置比较。
- 不会上报任何中断，但 index 事件、方向改变等事件会有状态上报。

15.4.2 配置解码模式

步骤 1 配置适配模式

- 如果是非标准模式 1 或 2，配置 QMA_CTRL.qma_mode=1(适配模式 1) / 2(适配模式 2)；
- 如果是标准的正交或脉冲方向模式，保持 QMA_CTRL.qma_mode=0；

步骤 2 配置解码模式。如果是非标准模式 1 或 2，配置脉冲方向模式，其他情况参考 QD_CTRL.pos_cnt_src 寄存器描述；

步骤 3 配置连接交换或者极性取反 QD_CTRL[6:0]，通常硬件连接正常的情况下无需配置。如果希望交换参考方向，可配置这个寄存器；

步骤 4 配置解码倍率，正交模式使用 QD_CTRL.iclk_rate；其他模式使 QD_CTRL.xclk_rate；

步骤 5 如果希望 QDM 输出一个参考位置信号，配置 QD_CTRL.synco_en=1；

步骤 6 通过 QD_CTRL.synco_sel 选择使用 I 相引脚还是 S 相引脚；

15.4.3 选择速度计算方法

QDM 提供两种速度计算方法：

V = ΔX / T 使用 UTMR 设定单位时间，使用 PCCU 测量位移；这种方法适用于编码器电机高速旋转的情况，可以较精确的测量一段时间内的平均转速或者极短时间的瞬时速度。

V = X / ΔT 使用 CAP 设定单位位置并测量时间间隔；这种方式在编码器电机转速较慢的时候可以得到较精确的测量结果，并且当单位位移间隔很小，测量结果可以认为是瞬时速度。但是在转速较快时，单位位移的时间太短，按周期数计时引入的误差会越来越大，所以不适用于高转速测量。

15.4.4 电机参数检查

说明

本小节开始的后续小节都默认芯片与编码器的物理连接与对应解码模式相关寄存器设置已经完成。

步骤 1 连接编码器电机到 AiM51 芯片引脚；

步骤 2 配置 QD_CTRL.iclk_rate = 2 或 xclk_rate =0 ，选择 1 倍分辨率；

步骤 3 设置 QEP_CTRL.index_latch_mode = 3，Index 事件发生时，POS_CNT 计数值将会锁存在 POSI_LAT 寄存；

步骤 4 设置 QEP_CTRL.pos_cnt_rst_src = 0， Index 事件发生时，POS_CNT 计数值将会清零；

步骤 5 配置 QEP_CTRL.pos_cnt_en = 1，启动 POS_CNT 运转；

步骤 6 令编码器电机单向旋转至少完整的 2 圈；

步骤 7 读取 POSI_LAT 寄存器，与编码器电机规格书的标定刻度对比。从而确认电机误差；或者测量出对于规格未标定的编码器电机的分辨率。

15.4.5 ΔX/T 计算速度

步骤 1 设置 UTMR 计数周期 UPRD；

步骤 2 设置 POS_MAX 为数倍的电机分辨率；

步骤 3 设置 QEP_CTRL.pos_cnt_rst_src = 3， 每当单位时间事件产生时复位；

步骤 4 配置 QEP_CTRL.pos_cnt_en / utmr_en = 1，启动 POS_CNT 运转；

步骤 5 利用 UTEVT 中断，读取 POS_LAT 寄存器，得到 UPRD 时间周期内，编码器转动的位移量。

编码器电机转速即为：

$$RPM = \left(\frac{1}{N} \times \frac{POS_LAT}{clk_rate} / \frac{UPRD}{Fclk}\right) \div 60$$

其中，N 为编码器刻度数量，clk_rate 为倍频解码参数（分辨率），Fclk 为 QDM 工作时钟周期。

15.4.6 X/ΔT 计算速度

步骤 1 设置 CAP_CTRL.upevt_ps / captmr_prediv ；

步骤 2 设置 QEP_CTRL.pos_cnt_rst_src = 0，每当 Index 事件产生时复位；

步骤 3 设置 CAP_CTRL.cap_en = 1 开启捕获计数器运转；

步骤 4 读取 CAP_PRD。

编码器电机转速为：

$$RPM = \left(\frac{1}{N} \times \frac{2^{upevt_ps}}{clk_rate} / \frac{CAP_PRD \times captmr_prediv}{Fclk}\right) \div 60$$

其中，N 为编码器刻度数量，clk_rate 为倍频解码参数（分辨率），Fclk 为 QDM 工作时钟周期。

15.5 QDM 寄存器概览

QDM 实例 2 个，其基地址如下：

- QDM0 寄存器基地址为：0x61003000
- QDM1 寄存器基地址为：0x61003800

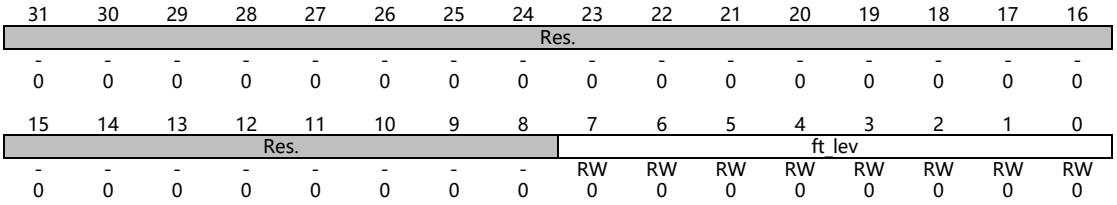
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	GENE_CTRL0	0x0	32	RW	0x00000000	一般控制寄存器 0
2	GENE_CTRL1	0x4	32	W1P	0x00000000	一般控制寄存器 1
3	POS_CNT	0x8	32	RO	0x00000000	位置计数器
4	POS_INIT	0xC	32	RW	0x00000000	位置初始化寄存器
5	POS_MAX	0x10	32	RW	0x00000000	最大位置寄存器
6	POS_CMP	0x14	32	RW	0x00000000	位置比较寄存器
7	POSI_LAT	0x18	32	RO	0x00000000	位置计数 INDEX 锁存寄存器
8	POSS_LAT	0x1C	32	RO	0x00000000	位置计数 STROBE 锁存寄存器
9	POS_LAT	0x20	32	RO	0x00000000	位置计数锁存寄存器
10	POS_CTRL	0x24	32	RW	0x00000000	位置计数控制寄存器
11	UTMR	0x28	32	RO	0x00000000	单位时间计数器
12	UPRD	0x2C	32	RW	0x00000000	单位时间周期寄存器
13	WDTMR	0x30	32	RO	0x00000000	看门狗计数器
14	WDPRD	0x34	32	RW	0x00000000	看门狗周期寄存器
15	QD_CTRL	0x38	32	RW	0x00000000	正交解码控制寄存器
16	QEP_CTRL	0x3C	32	RW	0x00000000	正交编码波形控制寄存器
17	QEP_STAT	0x40	32	RO	0x00000020	正交编码波形状态寄存器
18	CAP_CTRL	0x44	32	RW	0x00000000	事件捕获控制寄存器
19	CAP_TMR	0x48	32	RO	0x00000000	捕获事件时间计数器
20	CAP_PRD	0x4C	32	RO	0x00000000	捕获时间周期记录寄存器
21	CAP_TMRLAT	0x50	32	RO	0x00000000	捕获事件时间锁存寄存器
22	CAP_PRDLAT	0x54	32	RO	0x00000000	捕获时间周期记录锁存寄存器
23	STRB_SELR	0x58	32	RW	0x00000000	STROBE 信号选择寄存器
24	QMA_CTRL	0x60	32	RW	0x00000000	正交适配控制寄存器
25	INT_EN	0x64	32	RW	0x00000000	中断使能寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
26	INT_RAW	0x68	32	RO	0x00000000	原始中断寄存器
27	INT_STAT	0x6C	32	RO	0x00000000	上报中断状态寄存器
28	INT_CLR	0x70	32	W1P	0x00000000	中断清除寄存器
29	INT_INJ	0x74	32	W1P	0x00000000	中断注入寄存器

15.6 QDM 寄存器描述

15.6.1 GENE_CTRL0

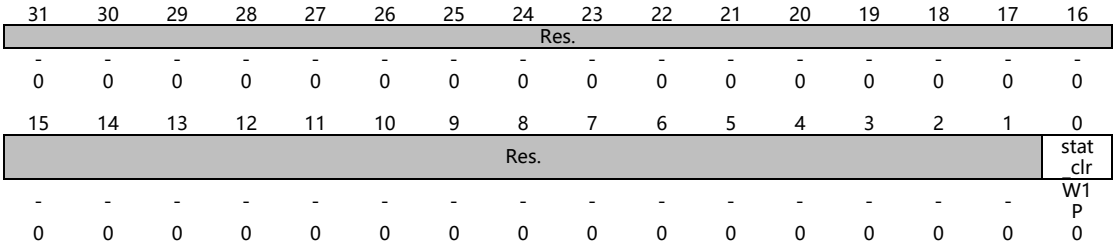
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
ft_lev	[7:0]	RW	滤波等级 A,B,I,S 相共享 0：仅同步 1：过滤 1 工作周期毛刺，信号延迟 1 工作时钟周期 ... 255：过滤 255 工作时钟周期毛刺，信号延迟 255 工作时钟周期	0x0

15.6.2 GENE_CTRL1

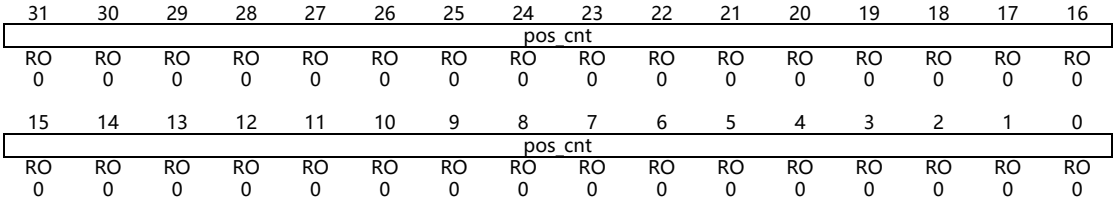
◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
stat_clr	[0]	W1P	状态清除，除静态配置以外的所有电路状态复位（即所有 RO 寄存器以及内部分频计数器）	0x0

15.6.3 POS_CNT

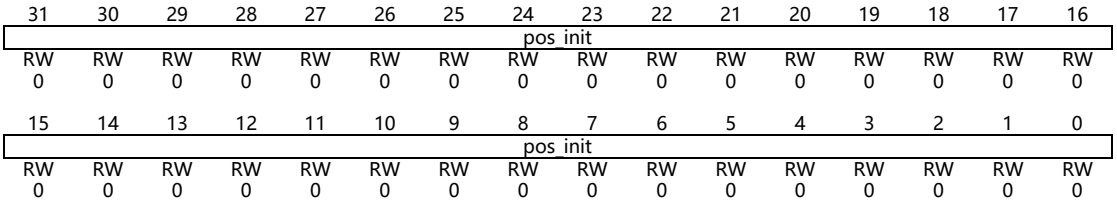
◆ **偏移地址：0x8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
pos_cnt	[31:0]	RO	位置计数器	0x0

15.6.4 POS_INIT

◆ **偏移地址：0xC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
pos_init	[31:0]	RW	位置计数器初始化设置	0x0

15.6.5 POS_MAX

◆ **偏移地址：0x10　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pos_max															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pos_max															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
pos_max	[31:0]	RW	位置计数器最大值设置	0x0

15.6.6 POS_CMP

◆ **偏移地址：0x14　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pos_cmp															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pos_cmp															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
pos_cmp	[31:0]	RW	位置计数器比较值	0x0

15.6.7 POSI_LAT

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pos_index_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pos_index_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
pos_index_latch	[31:0]	RO	Index 事件引起的位置锁存	0x0

15.6.8 POSS_LAT

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pos_strb_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pos_strb_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
pos_strb_latch	[31:0]	RO	Strobe 事件引起的位置锁存	0x0

15.6.9 POS_LAT

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
pos_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pos_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
pos_latch	[31:0]	RO	UnitTimer 事件引起的位置锁存	0x0

15.6.10 POS_CTRL

◆ **偏移地址：0x24　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pos_cmp_shd_en	pos_cmp_load_mode	pos_cmp_pol	pos_cmp_en	pos_cmp_pw											
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
pos_cmp_shd_en	[15]	RW	影子模式使能: 0：POS_CMP 写入立即生效 1：POS_CMP 在加载事件处生效	0x0
pos_cmp_load_mode	[14]	RW	影子模式下，比较值寄存器生效的加载事件 0：比较匹配 1：POS_CNT 复位	0x0
pos_cmp_pol	[13]	RW	比较匹配事件输出极性 0：高有效 1：低有效	0x0
pos_cmp_en	[12]	RW	位置比较开关 0：关闭 1：打开	0x0
pos_cmp_pw	[11:0]	RW	比较输出事件脉冲宽度配置 脉冲宽度=该位域配置值对应十进制数值个工作时钟周期	0x0

15.6.11 UTMR

◆ **偏移地址：0x28　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
utmr															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
utmr															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
utmr	[31:0]	RO	单位时间计数器	0x0

15.6.12 UPRD

◆ **偏移地址：0x2C　　复位值：0x00000000**

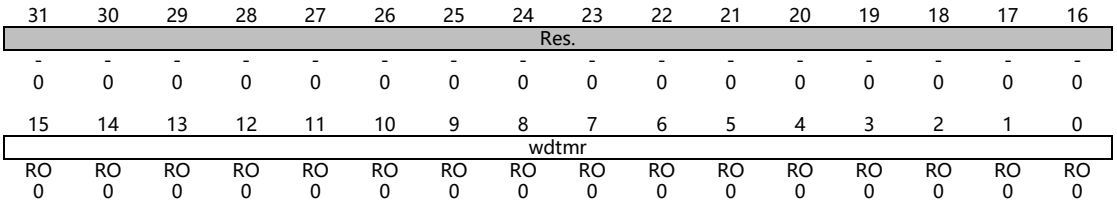
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
uprd															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
uprd															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
uprd	[31:0]	RW	单位时间计数周期配置寄存器	0x0

15.6.13 WDTMR

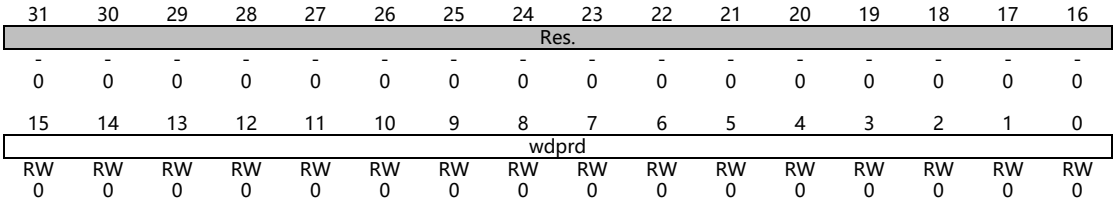
◆ **偏移地址：0x30　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
wdtmr	[15:0]	RO	专用看门狗计数器	0x0

15.6.14 WDPRD

◆ **偏移地址：0x34　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
wdprd	[15:0]	RW	专用看门狗周期配置寄存器	0x0

15.6.15 QD_CTRL

◆ **偏移地址：0x38　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		pos_cnt_src		synco_en	synco_sel	iclk_rate		xclk_rate	qep_swap	index_gate	qepa_pol	qepb_pol	index_pol	strb_pol	Res.
-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留	0x0
pos_cnt_src	[13:12]	RW	位置计数触发信号源模式 0：正交信号解码后的边沿与方向 1：脉冲方向模式的输入的边沿与方向 2：增计数模式 3：减计数模式	0x0
synco_en	[11]	RW	同步脉冲输出使能 0：禁用 1：使能	0x0
synco_sel	[10]	RW	同步输出通道选择 0：从 index 输出 1：从 strobe 输出	0x0
iclk_rate	[9:8]	RW	正交模式解码得到的计数脉冲倍频模式 0：x4（A\B 相双沿） 1：x2（仅 A 相双沿） 2：x1（仅 A 相上升沿） 3：禁止配置为该值	0x0
xclk_rate	[7]	RW	外部输入脉冲方向模式下倍频方式 0：x1（仅 A 相上升沿） 1：x2（A 相双沿）	0x0
qep_swap	[6]	RW	正交输入换相 0：无特殊处理 1：A 相与 B 相输入互换	0x0
index_gate	[5]	RW	使用 strobe 有效窗口门控 index 选择 0：不门控 1：使能门控	0x0
qepa_pol	[4]	RW	A 相输入的有效极性 0：高有效 1：低有效	0x0
qepb_pol	[3]	RW	B 相输入的有效极性 0：高有效 1：低有效	0x0
index_pol	[2]	RW	I 相（index）输入的有效极性	0x0

Name	Bits	R/W	Description	Default
			0：高有效 1：低有效	
strb_pol	[1]	RW	S 相（strobe）输入的有效极性 0：高有效 1：低有效	0x0
reserved	[0]	-	保留	0x0

15.6.16 QEP_CTRL

◆ **偏移地址：0x3C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
emu_mode		pos_cnt_rst_src		strb_init_mode		index_init_mode		sw_init_pos_cnt	strb_latch_mode	index_latch_mode		pos_cnt_en	cap_latch_mode	utmr_en	wdg_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

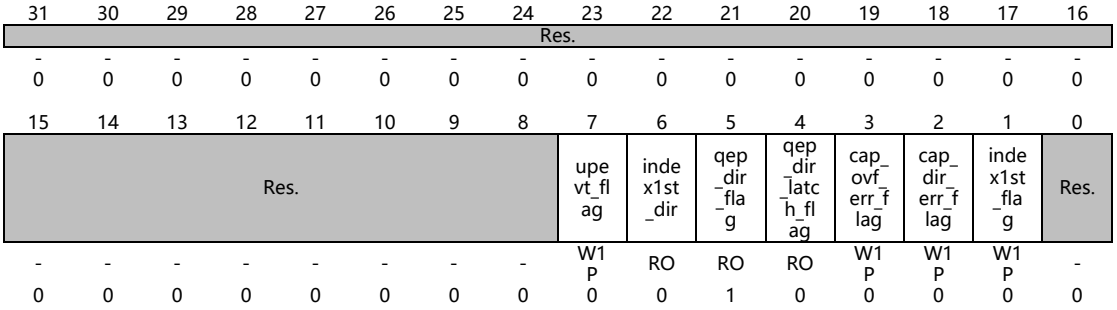
Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
emu_mode	[15:14]	RW	仿真器接入行为 0： POS_CNT 立即停止计数 UTMR 立即停止计数 WDTMR 立即停止计数 CAP_TMR 立即停止计数 (停止在 0) 1： POS_CNT 溢出后停止 UTMR 到达周期 UPRD 清零后停止 WDTMR 到达周期 WDPRD 清零后停止 CAP_TMR 到达下一个单位位置事件后停止 2： POS_CNT 不受影响 UTMR 不受影响 WDTMR 不受影响 CAP_TMR 不受影响 3： POS_CNT 不受影响 UTMR 不受影响	0x0

Name	Bits	R/W	Description	Default
			WDTMR 不受影响 CAP_TMR 不受影响	
pos_cnt_rst_src	[13:12]	RW	POS_CNT 复位事件选择 0：Index 事件 1：溢出事件 2：第一个 index 事件 3：单位时间事件	0x0
strb_init_mode	[11:10]	RW	strobe 初始化 POS_CNT 的模式 0：禁用 1：禁用 2：strobe 极性为高有效时，上升沿；反之，下降沿 3：正转时的 strobe 上升沿，反转时的 strobe 下降沿	0x0
index_init_mode	[9:8]	RW	index 初始化 POS_CNT 的模式 0：禁用 1：禁用 2：index 上升沿 3：index 下降沿	0x0
sw_init_pos_cnt	[7]	W1P	软件写 1 初始化 POS_CNT 计数器 此寄存器不可读，直接写 1 即可触发操作	0x0
strb_latch_mode	[6]	RW	strobe 锁存 POS_CNT 到 pos_strb_latch 的模式 0：strobe 极性为高有效时，上升沿；反之，下降沿 1：正转时的 strobe 上升沿，反转时的 strobe 下降沿	0x0
index_latch_mode	[5:4]	RW	index 锁存 POS_CNT 到 pos_index_latch 的模式 0：禁用 1：index 上升沿 2：index 下降沿 3：index 事件或者软件初始化 POS_CNT 事件	0x0
pos_cnt_en	[3]	RW	位置计数器使能，关闭时，所有相关信息寄存器全部清零 0：关闭 1：使能	0x0
cap_latch_mode	[2]	RW	POS_CNT 锁存模式 0：CPU 读取位置计数器时触发锁存 1：单位时间事件产生时触发锁存	0x0
utmr_en	[1]	RW	单位时间计数器使能，关闭时，所有相关信息寄存器全部清零 0：关闭 1：使能	0x0

Name	Bits	R/W	Description	Default
wdg_en	[0]	RW	专用看门狗使能，关闭时，所有相关信息寄存器全部清零 0：关闭 1：使能	0x0

15.6.17 QEP_STAT

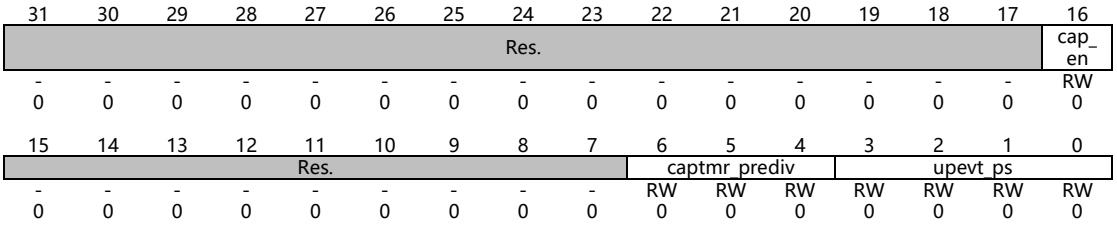
◆ **偏移地址：0x40　　复位值：0x00000020**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
upevt_flag	[7]	W1P	单位位置事件标志，写 1 清	0x0
index1st_dir	[6]	RO	第一个 index 事件发生时的电机旋转方向	0x0
qep_dir_flag	[5]	RO	电机实时旋转方向	0x1
qep_dir_latch_flag	[4]	RO	index 事件发生后的电机转向标志	0x0
cap_ovf_err_flag	[3]	W1P	两个单位位置事件之间，CAP_TMR 发生溢出标志，写 1 清	0x0
cap_dir_err_flag	[2]	W1P	两个单位位置事件之间出现换向标志，写 1 清	0x0
index1st_flag	[1]	W1P	第一个 index 标志，写 1 清	0x0
reserved	[0]	-	保留	0x0

15.6.18 CAP_CTRL

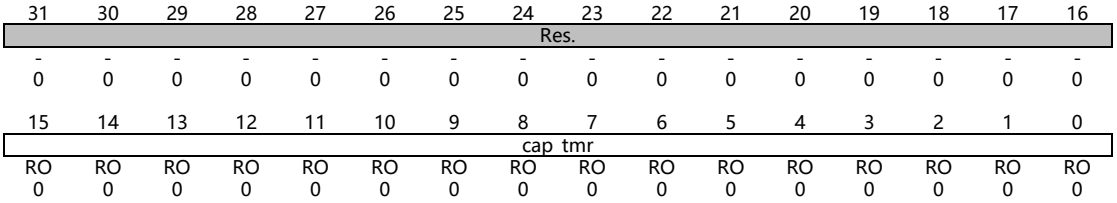
◆ **偏移地址：0x44　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留	0x0
cap_en	[16]	RW	单位位置捕获功能使能，关闭时，所有相关计数器和信息寄存器全部清零 0：关闭 1：使能	0x0
reserved	[15:7]	-	保留	0x0
captmr_prediv	[6:4]	RW	捕获计数器预分频 分频系数= 配置值+1	0x0
upevt_ps	[3:0]	RW	单位位置事件分频， （2^配置值）个 qclk 产生一次单位位置事件	0x0

15.6.19 CAP_TMR

◆ **偏移地址：0x48　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
cap_tmr	[15:0]	RO	单位位置捕获计数器	0x0

15.6.20 CAP_PRD

◆ **偏移地址：0x4C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cap_prd															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
cap_prd	[15:0]	RO	单位位置捕获周期结果（实时更新）	0x0

15.6.21 CAP_TMRLAT

◆ **偏移地址：0x50　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cap_tmr_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
cap_tmr_latch	[15:0]	RO	单位位置捕获计数器锁存	0x0

15.6.22 CAP_PRDLAT

◆ **偏移地址：0x54　　复位值：0x00000000**

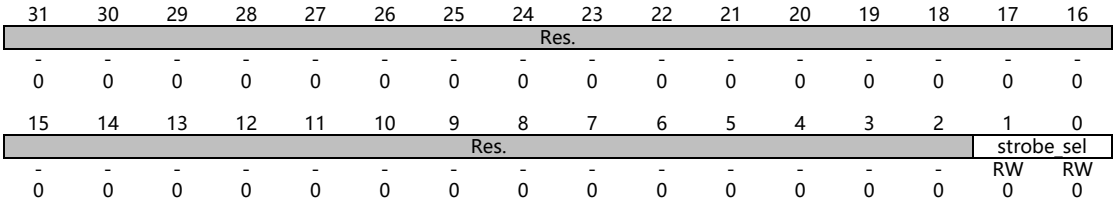
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cap_prd_latch															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
cap_prd_latch	[15:0]	RO	单位位置捕获周期结果锁存	0x0

15.6.23 STRB_SELR

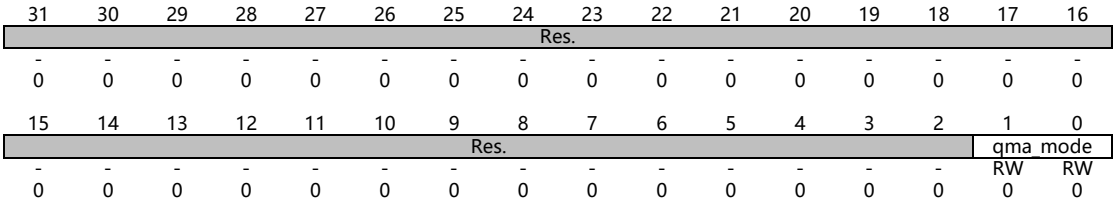
◆ **偏移地址：0x58　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
strobe_sel	[1:0]	RW	Strobe 信号类型 0：极性选择后的 S 相 1：极性选择后的 S 相 2：极性选择后的 S 相与 APTx.pwm0_soca 相或 3：极性选择后的 S 相与 APTx.pwm0_socb 相或	0x0

15.6.24 QMA_CTRL

◆ **偏移地址：0x60　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
qma_mode	[1:0]	RW	QEP 适配模式 0：bypass 模式 1：适配模式（棘轮模式）1 2：适配模式（棘轮模式）2 3：bypass 模式	0x0

15.6.25 INT_EN

◆ **偏移地址：0x64　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				qma_err_en	utcmp_match_en	index_latch_en	strb_latch_en	pos_cnt_match_en	pos_cmp_ready_en	pos_cnt_ovf_en	pos_cnt_udf_en	wdgto_en	dir_change_en	phase_err_en	Res.
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
qma_err_en	[11]	RW	识别到输入模式不匹配适配模式错误中断使能 0：不使能 1：使能	0x0
utcmp_match_en	[10]	RW	单位时间比较匹配中断使能 0：不使能 1：使能	0x0
index_latch_en	[9]	RW	index 引起 POS_CNT 锁存中断使能 0：不使能 1：使能	0x0
strb_latch_en	[8]	RW	strobe 引起 POS_CNT 锁存中断使能 0：不使能 1：使能	0x0
pos_cnt_match_en	[7]	RW	位置计数比较匹配中断使能 0：不使能 1：使能	0x0
pos_cmp_ready_en	[6]	RW	POS_CMP 寄存器生效事件中断使能 0：不使能 1：使能	0x0
pos_cnt_ovf_en	[5]	RW	位置计数器增计数上溢中断使能 0：不使能 1：使能	0x0
pos_cnt_udf_en	[4]	RW	位置计数器减计数下溢中断使能 0：不使能 1：使能	0x0
wdgto_en	[3]	RW	看门狗超时中断使能 0：不使能 1：使能	0x0
dir_change_en	[2]	RW	QEP 换向中断使能 0：不使能 1：使能	0x0
phase_err_en	[1]	RW	相位关系错误中断使能	0x0

Name	Bits	R/W	Description	Default
			0：不使能 1：使能	
reserved	[0]	-	保留	0x0

15.6.26 INT_RAW

◆ **偏移地址：0x68　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				qma_err_raw	utcmp_match_raw	index_latch_raw	strb_latch_raw	pos_cnt_match_raw	pos_cmp_ready_raw	pos_cnt_ovf_raw	pos_cnt_udf_raw	wdgto_raw	dir_change_raw	phase_err_raw	Res.
-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
qma_err_raw	[11]	RO	识别到输入模式不匹配适配模式错误原始中断状态 0：没有产生中断 1：产生中断	0x0
utcmp_match_raw	[10]	RO	单位时间比较匹配原始中断状态 0：没有产生中断 1：产生中断	0x0
index_latch_raw	[9]	RO	index 引起 POS_CNT 锁存原始中断状态 0：没有产生中断 1：产生中断	0x0
strb_latch_raw	[8]	RO	strobe 引起 POS_CNT 锁存原始中断状态 0：没有产生中断 1：产生中断	0x0
pos_cnt_match_raw	[7]	RO	位置计数比较匹配原始中断状态 0：没有产生中断 1：产生中断	0x0
pos_cmp_ready_raw	[6]	RO	POS_CMP 寄存器生效事件原始中断状态 0：没有产生中断 1：产生中断	0x0
pos_cnt_ovf_raw	[5]	RO	位置计数器增计数上溢原始中断状态 0：没有产生中断 1：产生中断	0x0

Name	Bits	R/W	Description	Default
pos_cnt_udf_raw	[4]	RO	位置计数器减计数下溢原始中断状态 0：没有产生中断 1：产生中断	0x0
wdgto_raw	[3]	RO	看门狗超时原始中断状态 0：没有产生中断 1：产生中断	0x0
dir_change_raw	[2]	RO	QEP 换向原始中断状态 0：没有产生中断 1：产生中断	0x0
phase_err_raw	[1]	RO	相位关系错误原始中断状态 0：没有产生中断 1：产生中断	0x0
reserved	[0]	-	保留	0x0

15.6.27 INT_STAT

◆ **偏移地址：0x6C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				qma_err_stat	utcmp_match_stat	index_latch_stat	strb_latch_stat	pos_cnt_match_stat	pos_cmp_ready_stat	pos_cnt_ovf_stat	pos_cnt_udf_stat	wdgto_stat	dir_change_stat	phase_err_stat	Res.
-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
qma_err_stat	[11]	RO	识别到输入模式不匹配适配模式错误中断标志 0：没有产生中断 1：产生中断	0x0
utcmp_match_stat	[10]	RO	单位时间比较匹配中断标志 0：没有产生中断 1：产生中断	0x0
index_latch_stat	[9]	RO	index 引起 POS_CNT 锁存中断标志 0：没有产生中断 1：产生中断	0x0
strb_latch_stat	[8]	RO	strobe 引起 POS_CNT 锁存中断标志 0：没有产生中断 1：产生中断	0x0

Name	Bits	R/W	Description	Default
pos_cnt_match_stat	[7]	RO	位置计数比较匹配中断标志 0：没有产生中断 1：产生中断	0x0
pos_cmp_ready_stat	[6]	RO	POS_CMP 寄存器生效事件中断标志 0：没有产生中断 1：产生中断	0x0
pos_cnt_ovf_stat	[5]	RO	位置计数器增计数上溢中断标志 0：没有产生中断 1：产生中断	0x0
pos_cnt_udf_stat	[4]	RO	位置计数器减计数下溢中断标志 0：没有产生中断 1：产生中断	0x0
wdgto_stat	[3]	RO	看门狗超时中断标志 0：没有产生中断 1：产生中断	0x0
dir_change_stat	[2]	RO	QEP 换向中断标志 0：没有产生中断 1：产生中断	0x0
phase_err_stat	[1]	RO	相位关系错误中断标志 0：没有产生中断 1：产生中断	0x0
reserved	[0]	-	保留	0x0

15.6.28 INT_CLR

◆ **偏移地址：0x70　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				qma_err_clr	utcmp_match_clr	index_latch_clr	strb_latch_clr	pos_cnt_match_clr	pos_cmp_ready clr	pos_cnt_ovf_clr	pos_cnt_udf_clr	wdgto_clr	dir_change_clr	phase_err_clr	Res.
-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
qma_err_clr	[11]	W1P	识别到输入模式不匹配适配模式错误中断清除（写 1 清）	0x0

Name	Bits	R/W	Description	Default
utcmp_match_clr	[10]	W1P	单位时间比较匹配中断清除（写 1 清）	0x0
index_latch_clr	[9]	W1P	index 引起 POS_CNT 锁存中断清除（写 1 清）	0x0
strb_latch_clr	[8]	W1P	strobe 引起 POS_CNT 锁存中断清除（写 1 清）	0x0
pos_cnt_match_clr	[7]	W1P	位置计数比较匹配中断清除（写 1 清）	0x0
pos_cmp_ready_clr	[6]	W1P	POS_CMP 寄存器生效事件中断清除（写 1 清）	0x0
pos_cnt_ovf_clr	[5]	W1P	位置计数器增计数上溢中断清除（写 1 清）	0x0
pos_cnt_udf_clr	[4]	W1P	位置计数器减计数下溢中断清除（写 1 清）	0x0
wdgto_clr	[3]	W1P	看门狗超时中断清除（写 1 清）	0x0
dir_change_clr	[2]	W1P	QEP 换向中断清除（写 1 清）	0x0
phase_err_clr	[1]	W1P	相位关系错误中断清除（写 1 清）	0x0
reserved	[0]	-	保留	0x0

15.6.29 INT_INJ

◆ **偏移地址：0x74　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				qma_err_inj	utcmp_match_inj	index_latch_inj	strb_latch_inj	pos_cnt_match_inj	pos_cmp_ready_inj	pos_cnt_ovf_inj	pos_cnt_udf_inj	wdgto_inj	dir_change_inj	phase_err_inj	Res.
-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
qma_err_inj	[11]	W1P	识别到输入模式不匹配适配模式错误中断注入（写 1）	0x0
utcmp_match_inj	[10]	W1P	单位时间比较匹配中断注入（写 1）	0x0
index_latch_inj	[9]	W1P	index 引起 POS_CNT 锁存中断注入（写 1）	0x0
strb_latch_inj	[8]	W1P	strobe 引起 POS_CNT 锁存中断注入（写 1）	0x0

Name	Bits	R/W	Description	Default
pos_cnt_match_inj	[7]	W1P	位置计数比较匹配中断注入（写 1）	0x0
pos_cmp_ready_inj	[6]	W1P	POS_CMP 寄存器生效事件中断注入（写 1）	0x0
pos_cnt_ovf_inj	[5]	W1P	位置计数器增计数上溢中断注入（写 1）	0x0
pos_cnt_udf_inj	[4]	W1P	位置计数器减计数下溢中断注入（写 1）	0x0
wdgto_inj	[3]	W1P	看门狗超时中断注入（写 1）	0x0
dir_change_inj	[2]	W1P	QEP 换向中断注入（写 1）	0x0
phase_err_inj	[1]	W1P	相位关系错误中断注入（写 1）	0x0
reserved	[0]	-	保留	0x0

16 高级定时器（APT）

16.1 APT 简介

APT (Advanced PWM timer) 可通过软件控制周期、极性、相位、时序关系，输出 6 路 PWM 信号。

APT 模块适用于主流的无刷直流电机变频控制，可对半 H 桥（HHB，Half H-Bridge）、3 相电机、3 相交错 DC/DC、全 H 桥（FHB，Full H-Bridge）等系统进行控制。

16.2 APT 主要特性

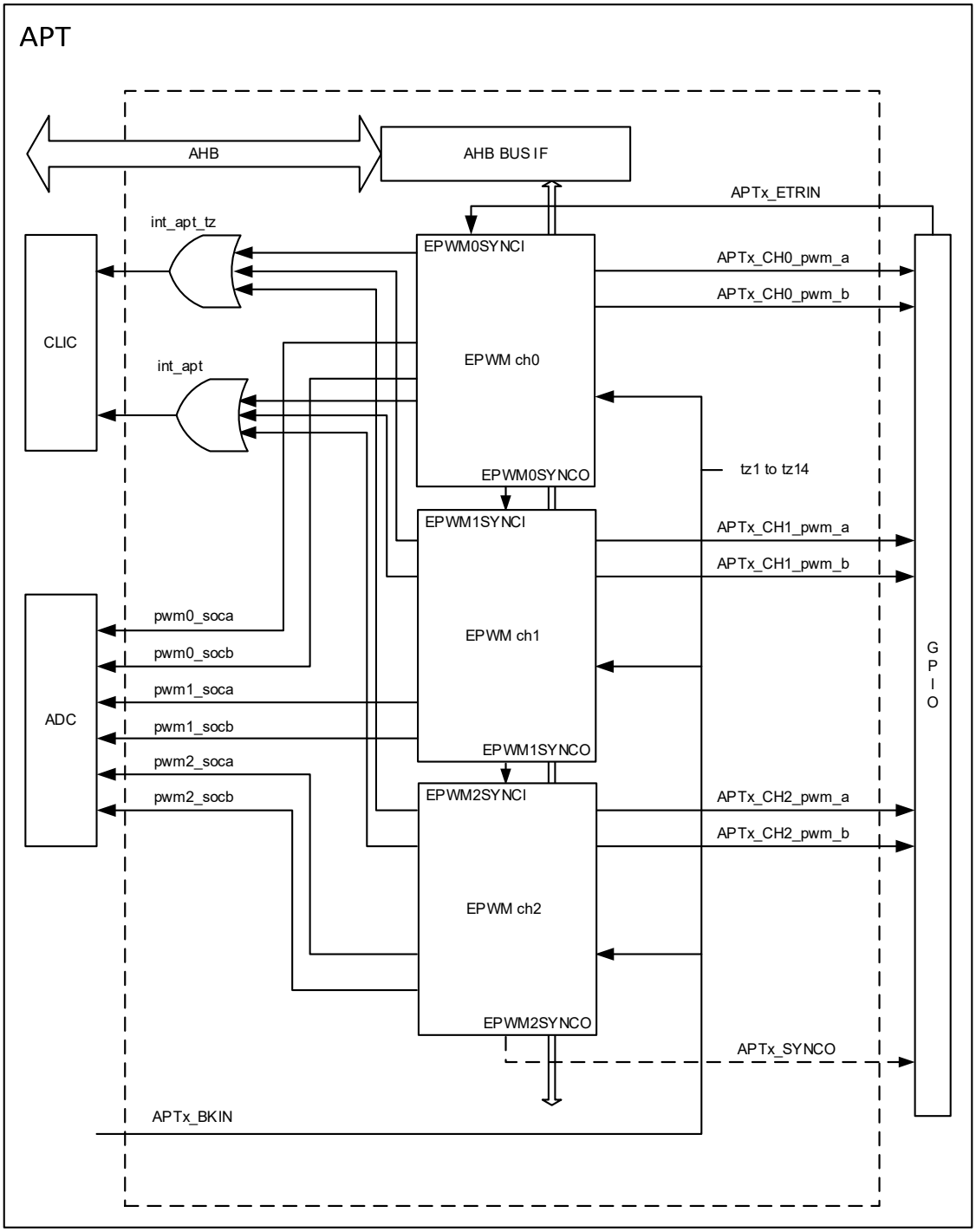
AiM51 系列芯片集成了 2 个 APT 模块，每个 APT 模块划分为 3 个 EPWM（Enhanced PWM）通道，每个通道具有如下功能：

- 外部或软件控制 Timer 开始/结束；
- 一个 16-bit PWM 周期计数器；
- 一对 PWM 输出（pwm_a 和 pwm_b）；
- 支持中心对称或非对称的 PWM 输出；
- 相对于其他通道 PWM 的相位可调；
- 软件或硬件触发的同步控制；
- 16-bit 死区时间寄存器，上升沿或下降沿的延迟时间可单独调节；
- 外设刹车可将 PWM 输出强制成高电平、低电平、高阻态；
- 软件可配置选择用于触发 CPU 中断或 ADC 转换的信号；

16.3 APT 逻辑实现

APT 集成连接关系如下图 16-1 所示。

图 16-1 APT 集成连接关系示意图



16.3.1 APT 与引脚、对端器件的连接

AiM51 系列芯片集成了 2 个相互独立的 APT 模块，每个 APT 模块具有：

6 路 PWM 输出：APTx_CH0、APTx_CH0N、APTx_CH1、APTx_CH1N、APTx_CH2、APTx_CH2N

1 个外部刹车信号接口：APTx_BKIN

1 个双向同步互联接口：APTx_ETR

说明

同一时间 APT_ETR 只可被配置为一个方向：作为外部同步功能 Master 或者 Slave。

使用 APT 模块时，请参考 IO MUX 章节将芯片引脚的 IO 模式配置成 APT 模式。

16.3.2 APT 与片内逻辑的连接

APT 的所有逻辑工作时钟都与它所挂载的高速 AHB 总线子系统相同，在最多 200MHz 的主频下能够提供 5ns 的 PWM 分辨精度。请注意，在计算 APT 的输出波形宽度时，会频繁使用到“工作时钟周期”这一概念。

每个 APT 支持一个多中断源汇聚的普通中断请求和一个刹车中断请求，每个请求的每个中断源可以单独使能，支持原始中断事件和已使能中断状态的查询。

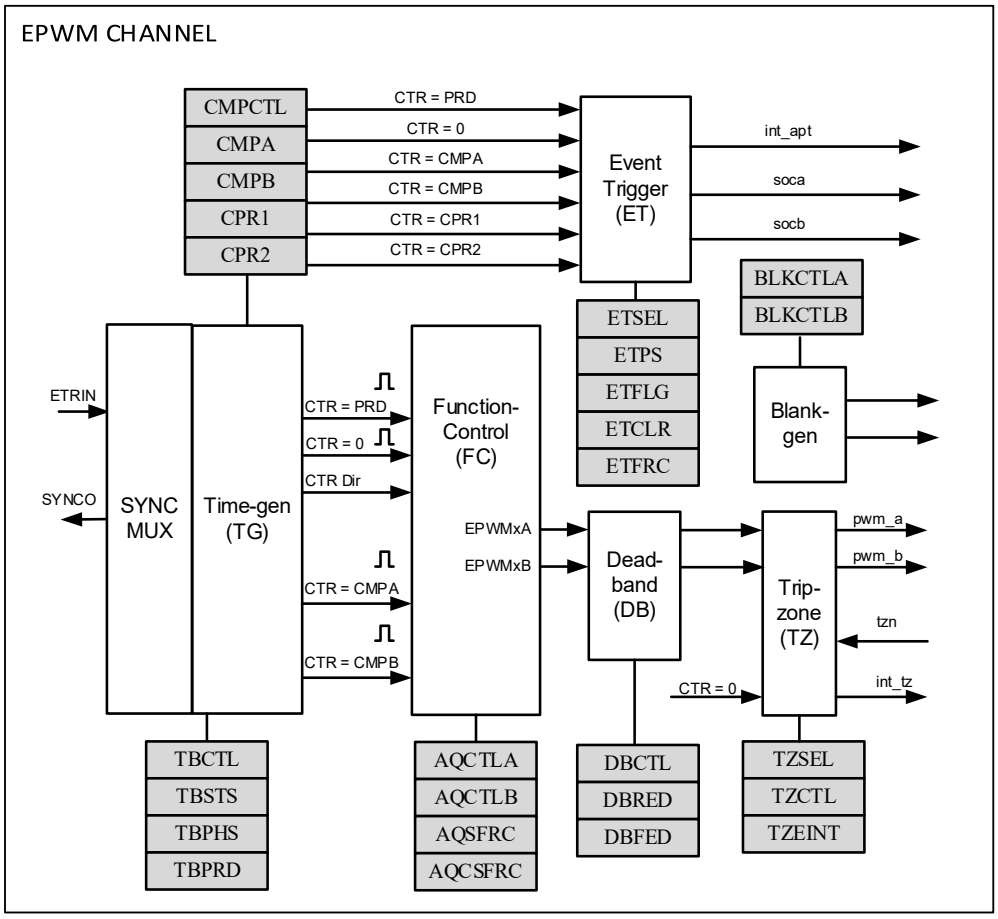
另外，在 CPU CLIC 的配置中也可以完全屏蔽某一个 APT 的中断请求。

APT 每个 EPWM 通道输出两个 ADC SOC 转换触发信号送往 ADC 模块触发对外部电机模组相电流的采样。

16.3.3 APT 单通道逻辑框图

APT 单个通道逻辑框图如下图 16-2 所示。

图 16-2 APT 单个通道——EPWM 单元逻辑框图



每个 EPWM 通道包含数个子模块，这些子模块各自在 PWM 波形的生成中发挥特定功能。灰色的部分是软件可配置的寄存器。表 16-1 汇总列出了各个模块主要的功能和对应可配置项。比如用户希望改变 PWM 波形在一周期中翻转的位置，则需要在 Time-Gen 模块进一步确认相关细节。

表 16-1 EPWM 通道

模块	功能与配置
SYNC_MUX	1、配置 ETR 信号源。 2、配置 ETR 信号功能。
Time-Gen	1、配置 PWM 时基计数器（TBCTR）频率、周期以及相对工作时钟的分频比。 2、设置时基计数器的模式： （1）递增模式：用于不对称 PWM； （2）递减模式：用于不对称 PWM；

模块	功能与配置
	（3）增减计数模式：用于对称 PWM； 3、配置相对于另一个 ePWM 模块的时基相位（TBPHS）。 4、通过硬件或软件同步通道或其他模块之间的时基计数器（TBCTL）。 5、配置同步事件后时基计数器的方向（递增或递减）。 6、配置调试器接入芯片时时基计数器的行为。 7、指定通道同步输出的事件来源。 8、指定提示 FC 子模块执行 pwm_a 或 pwm_b 输出信号发生翻转的时间比较点。 9、指定产生具有可配置周期间隔的中断和 ADC 转换触发信号的独立时间比较点。
Function Control	1、指定时基计数器比较匹配事件、TZ 子模块刹车事件发生时采取的操作类型： （1）未采取任何行动； （2）输出 pwm_a 和/或 pwm_b 切换为高电平； （3）输出 pwm_a 和/或 pwm_b 切换为低电平； （4）输出 pwm_a 和/或 pwm_b 翻转； 2、通过软件控制强制 PWM 输出状态，并设置强制控制的时机。
Dead Band	1、控制两相 pwm 之间的传统互补死区关系。 2、指定输出上升沿延迟值。 3、指定输出下降沿延迟值。 4、pwm 信号完全绕过死区模块。在这种情况下，PWM 波形不会被修改。 5、允许 pwm_b 相对于 pwm_a 输出进行相移。
Trip Zone	配置 ePWM 模块对某一个、全部 TZ 输入信号或数字比较事件作出反应。 1、指定 TZ 信号有效时采取的刹车动作： （1）强制 pwm_a 和/或 pwm_b 高； （2）强制 pwm_a 和/或 pwm_b 低； （3）强制 pwm_a 和/或 pwm_b 进入高阻抗状态； （4）使 pwm_a 和/或 pwm_b 忽略任何跳闸条件。 2、配置 ePWM 立即或一周期后，仅单周期或者逐周期受刹车信号影响。 3、启用跳闸区域以启动中断。 4、完全绕过跳闸区域模块。 5、逐周期跳闸清除的可编程选项 。
Event Trigger	1、配置产生中断的时间比较事件。 2、配置用于触发 ADC 转换的时间比较事件。 3、指定上述事件触发的频率（每个 N 个 PWM 周期）。

模块	功能与配置
	4、查询、屏蔽或清除事件标志。
Blank-gen	1、配置是否产生消隐信号。 2、配置消隐信号产生的时机。 3、配置消隐信号的脉冲宽度。

16.4 APT 功能描述

16.4.1 Time_Gen（TG）子模块

Time_Gen 模块为 APT 中每个 PWM 通道的核心时基计数器，它通过计数与比较产生在一个计数周期内位置特定的事件，使其他子模块可以使用这些事件进一步生成灵活可控的 PWM 波形。

图 16-3 显示了 TG 模块核心计数器 TBCTR 的相关逻辑框架和寄存器接口，表 16-2 阐述了图中关键信号的含义。

图 16-3 Time_Gen 模块逻辑框图

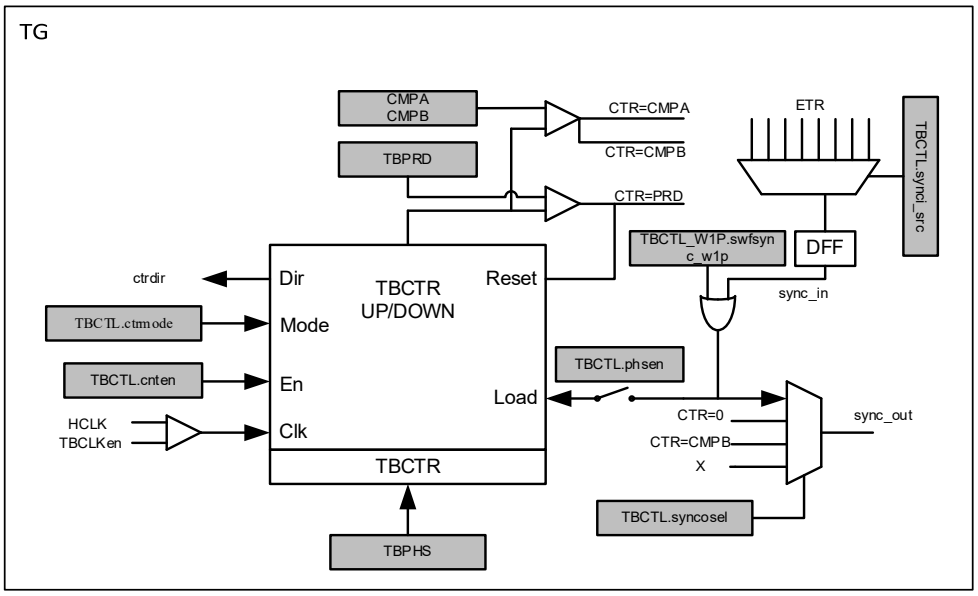


表 16-2 TG 模块关键信号释义

信号	释义
ETR	外部触发信号，它们来自芯片 IO，GPT0~3 同步输出，ADC0/1 的电压监测告警。
sync_in	TBCTR 同步输入信号，通过 TBCTL.synci_src 选择并生效的 ETR 信号。
sync_out	通过 TBCTL.syncosel 选择的时间比较匹配事件产生输出同步脉冲信号，对于其他模块来讲，它也是 ETR 之一。
HCLK	APT 模块的工作主时钟，最高频率为 200MHz。
TBCLKen	分频计数使能，将会每间隔数个 HCLK 时钟周期有效一次，当 TBCLKen 有效时，TBCTR 才会计数。
CTR=CMPA	当 TBCTR=CMPA 寄存器设置的值时，产生的计数值匹配事件，它是 TG 模块的输出。
CTR=CMPB	当 TBCTR=CMPB 寄存器设置的值时，产生的计数值匹配事件，它是 TG 模块的输出。
CTR=PRD CTR=0	TBPRD 是 TBCTR 计数的最大值，递增方向计数时，TBCTR=TBPRD 发生后，TBCTR 被清零；递减方向计数时，TBCTR=0 发生后被置位到 TBPRD；这两个信号也是 TG 模块的输出。
ctrdir	指示计数器的递增或递减方向。

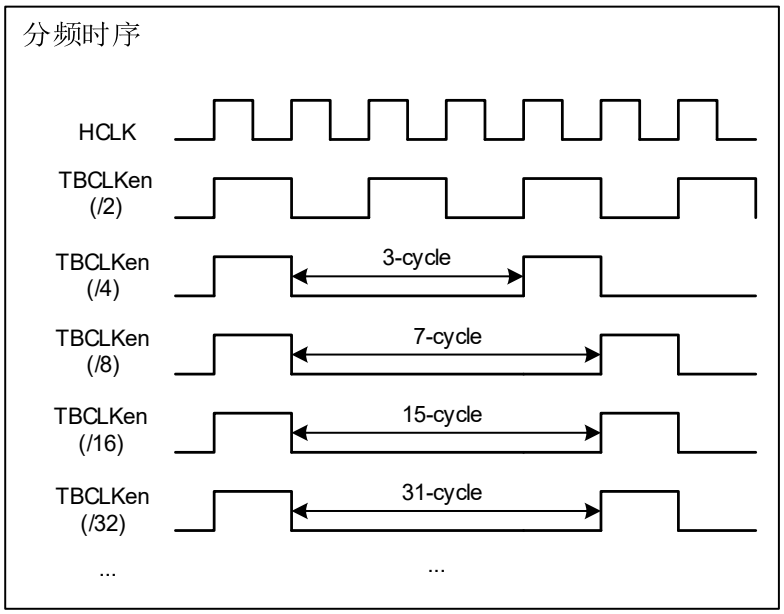
计数器分频和周期控制

APT 具有一个时钟分频模块，使用不同于 TBCTR 的另一个计数器控制时序，当计数器值=TBCTL.clkdiv-1 时，TBCTR 计数使能 TBCLKen 有效，可以设置的分频系数有 1、2、4、8...128（2 的 clkdiv 次幂）。

TBCLKen 的频率为 f_{HCLK}/2、f_{HCLK}/4、f_{HCLK}/8、f_{HCLK}/16、f_{HCLK}/32、f_{HCLK}/64、f_{HCLK}/128。

图 16-4 显示了 TBCLKen 信号与工作时钟 HCLK 之间的关系。

图 16-4 Time_Gen 计数分频示意图



用户可以通过 TBCTL.clkdiv 寄存器和 TBPRD 寄存器设置计数周期

TBCTR 的总计数周期时长为 HCLK 周期×2^TBCTL.clkdiv×TBPRD 。大部分时候，这也是 PWM_A 和 PWM_B 信号的周期。

计数器比较匹配事件

TG 会输出提供四个比较寄存器 CMPA、CMPB、CPR1、CPR2 的数值设置，根据 ctrdir 分别产生 12 种内部比较匹配事件：

ctrdir=0 或 ctrdir=1 时的 CTR=0, CTR=PRD, CTR=CMPA, CTR=CMPB 被用于 PWM 的波形控制、同步信号输出和产生 PWM 中断，以及 ADC 转换触发。

而 ctrdir=0 或 ctrdir=1 时的 CTR=CPR1, CTR=CPR2 只被用于 ADC 转换触发。

计数器使能开关与模式设置

TBCTL.cnten 为 TBCTR 计数器总开关。

TBCTL.ctrmode 设置 TBCTR 计数模式。有递增，递减，增减交替模式可以选择。

TBCTL.free_soft 可以控制 TBCTR 计数器运行状态，当配置 0 时，计数器立即停止；配置为 1 时，计数周期完成后停止；配置为 2 或 3 时，计数器继续运行。

计数值同步功能

用户可以通过 TBCTL.etr_behavior 和 TBCTL.synci_src 选择 ETR 信号的功能以及 ETR 信

号源，请参考 16.4.6 SYNC_MUX 子模块章节获取 ETR 信号的其他功能细节。当 TBCTL.etr_behavior=1，即 ETR 信号选择同步功能时，使能 TBCTL.phsen 即可在由 TBCTL.synci_src 选择的 ETR 信号有效时或者软件对 TBCTL_W1P.swfsync_w1p 写 1 时，将 TBCTR 的值置为预先设置在 TBPHS 寄存器的值。

同步触发后，TBCTR 还会立即根据 TBCTL.phsdir 的设置改变计数方向。

计数器同步功能可以保证多通道/多模块的时基计数器同步计数值或者产生固定的相位，并向预期的方向计数。进而产生多路边沿对齐或有固定相位关系的 PWM 信号。

用户还可以通过 TBCTL.syncosel 选择当前 PWM 通道依据何种事件产生一个供其他模块/通道使用的 ETR 信号。

此外，用户可以通过对 TBCTR_RW 寄存器写一个值使当前通道 TBCTR 值立即改变。

计数器周期加载时机控制

用户通过 TBCTL.prdld 设置 TBPRD 寄存器立即生效或是在完成上一个周期后生效。

计数器状态监控

用户可以通过 TBSTS 获悉 TBCTR 计数状态，TBSTS 提供

- TBCTR 发生过溢出（TBSTS.ctrmax）
- TBCTR 发生过同步（TBSTS.synci）
- TBCTR 当前的计数方向（TBSTS.ctrdir）

这些信息,其中 TBSTS.ctrmax 和 TBSTS.synci 状态可以通过 TBSTS_CLR.ctrmax_clr 和 TBSTS_CLR.synci_clr 清除。

16.4.2 Function_Control（FC）子模块

Function Control 模块根据软件设置的行为，通过接收 Time_Gen 提供的事件发生标记信号，并根据寄存器配置设定这些事件标记信号的作用，以及独立控制在事件发生时对 PWM 信号 pwm_a 和 pwm_b 的操作：无操作、置低电平、置高电平、翻转。产生初始 PWM 波形。

表 16-3 显示了 Time-gen 模块的事件有效时和软件强制寄存器被写入时可配置的 PWM 信号行为与 PWM 信号的变化。

表 16-3 AQCTLA/B 寄存器设置在事件处使 PWM 波形产生的变化

方向		ctrdir=UP	ctrdir=DOWN		pwm_a/b

比较事件	TBCTR=0	TBCTR=CMPA	TBCTR=CMPB	TBCTR=CMPA	TBCTR=CMPB	TBCTR=TBPRD	
AQSFRC.actsfb	ZRO	CAU	CBU	CAD	CBD	PRD	
0x0	0x0	0x0	0x0	0x0	0x0	0x0	无动作
0x1	0x1	0x1	0x1	0x1	0x1	0x1	置低
0x2	0x2	0x2	0x2	0x2	0x2	0x2	置高
0x3	0x3	0x3	0x3	0x3	0x3	0x3	翻转

因存在多个条件同时达成的情况，存在优先级问题，多优先级的划分见下表所示。表 16-4 是递增计数时的优先级，表 16-5 是递减计数时的优先级。

表 16-4 递增时 PWM 变化事件优先级

优先等级	条件
1(最高)	软件强制（SOFTWARE FORCE）
2	TBCTR=TBPRD
3	TBCTR=CMPB
4	TBCTR=CMPA
5(最低)	TBCTR=0

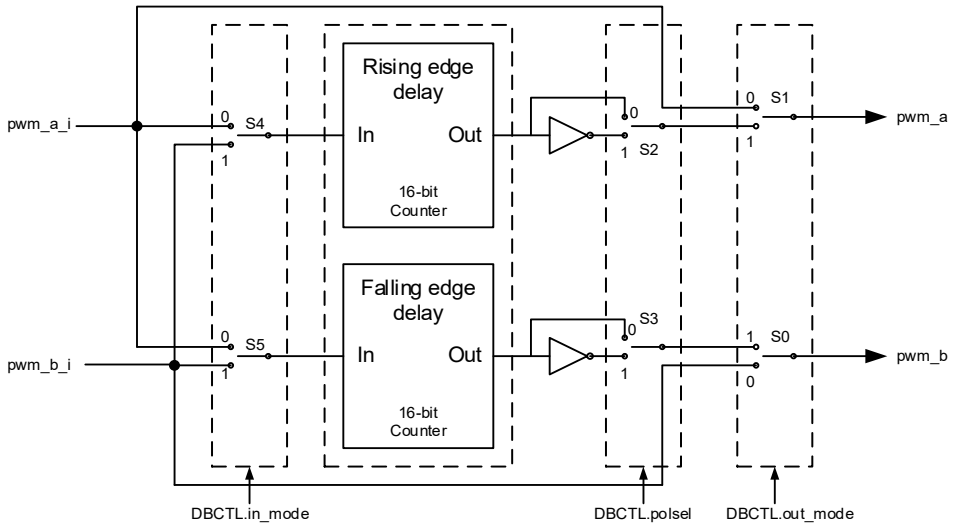
表 16-5 递减时 PWM 变化事件优先级

优先等级	条件
1(最高)	软件强制（SOFTWARE FORCE）
2	TBCTR=0
3	TBCTR=CMPA
4	TBCTR=CMPB
5(最低)	TBCTR= TBPRD

16.4.3 Dead-Band（DB）子模块

处理 FC 模块输出的 pwm_a 和 pwm_b 信号，推移 pwm_a 和 pwm_b 的上升沿或下降沿，得到一对具有死区时间的 PWM 信号 pwm_a 和 pwm_b。DB 模块结构图如下图 16-5 所示。

图 16-5 DB 模块结构图



通过 DBCTL.in_mode 选择在 pwm_a、pwm_b 的上升沿或下降沿插入死区时间。

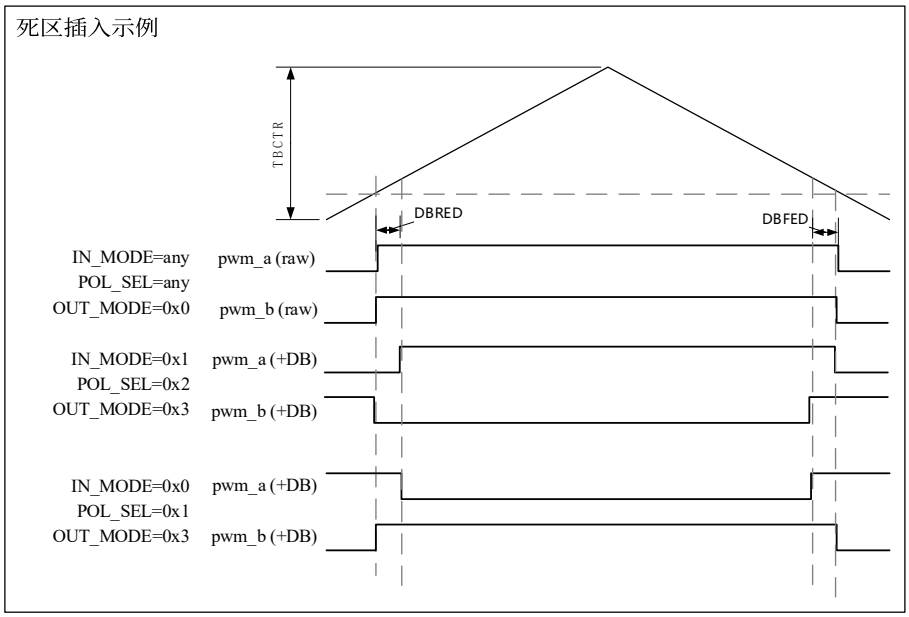
通过 DBCTL.polsel 选择将 pwm_a、pwm_b 是否需要反向输出。

通过 DBCTL.outmode 选择 pwm_a、pwm_b 是否插入死区时间。

用户可以在 DBRED 和 DBFED 寄存器中设置上升沿延时和下降沿延时时间。

图 16-6 显示了两种死区控制有效的情况下，原始 PWM 波形在 DB 模块处理的最终输出结果。

图 16-6 死区插入效果示例



16.4.4 Trip_Zone（TZ）子模块

APT 模块的三通道共用 14 个 TZ 信号（TZ1 ~TZ14），该信号可用于在外部异常时快速中止 PWM 输出，且中止状态的 PWM 极性可控制为高电平、低电平或高阻态。且每个 TZ 信号具有独立使能与滤波控制。

芯片刹车信号源分别为：

- TZ1：芯片引脚
- TZ2：CPU_LOCKUP 事件
- TZ3：PVD 低电压检测事件
- TZ4：SRAM0 奇偶校验错误事件
- TZ5：SRAM1 奇偶校验错误事件
- TZ6：SRAM2 奇偶校验错误事件
- TZ7：eFlash 控制器检测到错误中断事件
- TZ8：CFD 检测到时钟失效事件
- TZ9~TZ12：ACMP0~3 比较超阈值事件
- TZ13：CPU Halt 进入调试模式事件
- TZ14：无

说明

CPU_LOCKUP 事件：CPU 在异常处理程序中遇到异常。

CPU_LOCKUP 事件、PVD 事件、SRAM 奇偶校验错误事件、eFlash 错误事件、CFD 检测到时钟失效事件、ACMP 事件和 CPU Halt 事件都是高电平有效的。

通过 TZCTL.tza 和 TZCTL.tzb 分别预设刹车时 pwm_a，pwm_b 的极性；

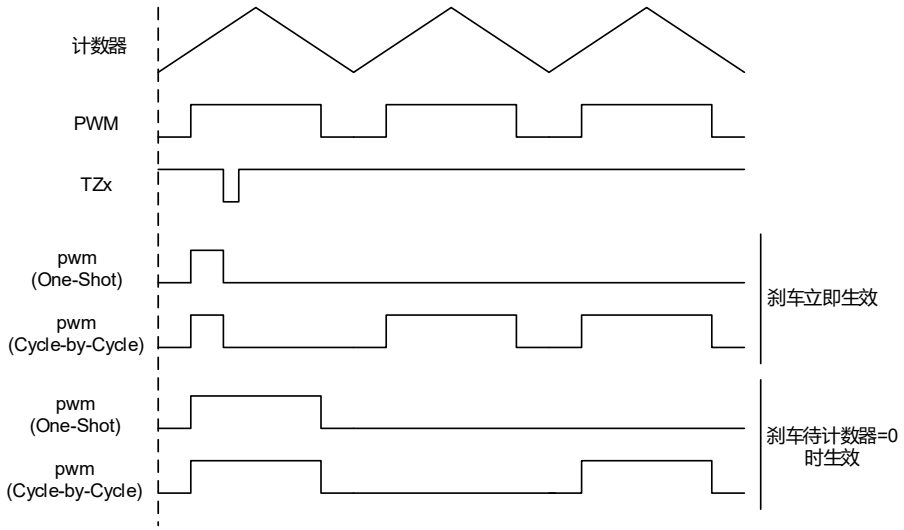
通过 TZCTL.tzx_sel（x 取值 1~14）选择刹车信号的有效极性，并通过 TZCTL.tzx_en（x 取值 1~14）选择是否对指定刹车信号进行数字滤波；

通过 TZSEL.cbcx 和 TZSEL.oshtx（x 取值 1~14）选择每个 TZ 信号仅一周期生效还是在每周期生效；

通过 TZSEL.cbcmode 和 TZSEL.oshtmode 选择当 TZ 信号有效时，是立刻中止 pwm 还是等待 TBCTR 计数周期完成后中止。

图 16-7 显示了在每种模式下刹车信号对 PWM 信号产生影响的时机。

图 16-7 刹车功能各模式对 PWM 输出的影响



刹车事件发生后，可以通过 TZEINT, TZFLG, TZCLR 这组寄存器设置连续或单次生效的刹车中断上报、状态查询与清除。

TZFRC 寄存器触发强制刹车，它的作用相当于软件刹车，用户可以使用软件识别 14 个 TZ 信号以外的其他事件来使 PWM 产生刹车动作。但是这种方式的实时性要弱于使用 TZ1~14 刹车信号。

16.4.5 Event_Trigger（ET）子模块

该模块将 Time_Gen 模块的比较匹配事件配置为对 ADC 的采样触发信号或产生系统中断信号。

通过 ETSEL.inten 选择是否输出系统中断上报，通过 ETSEL.socaen 和 ETSEL.socben 选择是否使能 ADC 采样触发信号 pwm_soca/pwm_socb。

通过 ETSEL.intsel 设置产生系统中断的条件：

- 当 TBCTR=0；
- 当 TBCTR=PRD；
- 当 TBCTR=0 或 PRD；
- 当 TBCTR=CMPA 且 ctrdir 递增；
- 当 TBCTR=CMPA 且 ctrdir 递减；
- 当 TBCTR=CMPB 且 ctrdir 递增；
- 当 TBCTR=CMPB 且 ctrdir 递减；

通过 ETPS.intprd 设置系统中断的产生频率：

- 从不产生；
- 事件发生时；
- 事件发生两次；
- 事件发生三次；

使用 2-bit 计数器统计事件发生次数，可在寄存器 ETPS_RO.socacnt_ro 和 ETPS_RO.socbcnt_ro 查询，该计数器和寄存器会在产生中断后清零。

通过 ETSEL.socasel 和 ETSEL.socbsel 选择 ADC 转换触发信号事件源：

- 当 TBCTR=0；
- 当 TBCTR=PRD（周期）；
- 当 TBCTR=0 或 PRD；
- 当 TBCTR=CMPA 且 ctrdir 递增；
- 当 TBCTR=CMPA 且 ctrdir 递减；
- 当 TBCTR=CMPB 且 ctrdir 递增；
- 当 TBCTR=CMPB 且 ctrdir 递减；
- 当 TBCTR=CPR1 且 ctrdir 递增；
- 当 TBCTR=CPR1 且 ctrdir 递减；

- 当 TBCTR=CPR2 且 ctrdir 递增；
- 当 TBCTR=CPR2 且 ctrdir 递减；

通过 ETPS_RO.socacnt 或者 ETPS_RO.socbcnt 设定 ADC 转换触发信号频率：

- 从不产生；
- 事件发生时；
- 事件发生两次；
- 事件发生三次；

使用 2-bit 计数器统计事件发生次数，通过计数器 ETPS_RO.socacnt 或者 ETPS_RO.socbcnt 可查询，且计数器和寄存器在 ADC 转换触发信号发生时清零。

16.4.6 SYNC_MUX 子模块

APT 提供 11 选 1 的 ETR 信号配置。通过 TBCTL.synci_src 设置，这些信号分别是：

- 芯片 IO 输入；
- GPT0 的同步输出；
- GPT1 的同步输出；
- GPT2 的同步输出；
- GPT3 的同步输出；
- 3bit ADC0.AWDG 超门限告警信号；
- 3bit ADC1.AWDG 超门限告警信号；

ETR 信号可通过 TBCTL.etr_behavior 寄存器设置为以下功能：

- 作为脉冲使 TBCTR 同步；
- 代替 cnten 和 freesoft 配置使能 TBCTR 计数（不影响计数步长）；
- 在 ETR 上升沿时使 TBCTR 计数；
- 强制 PWM 的输出值；

16.4.7 Blank_gen 子模块

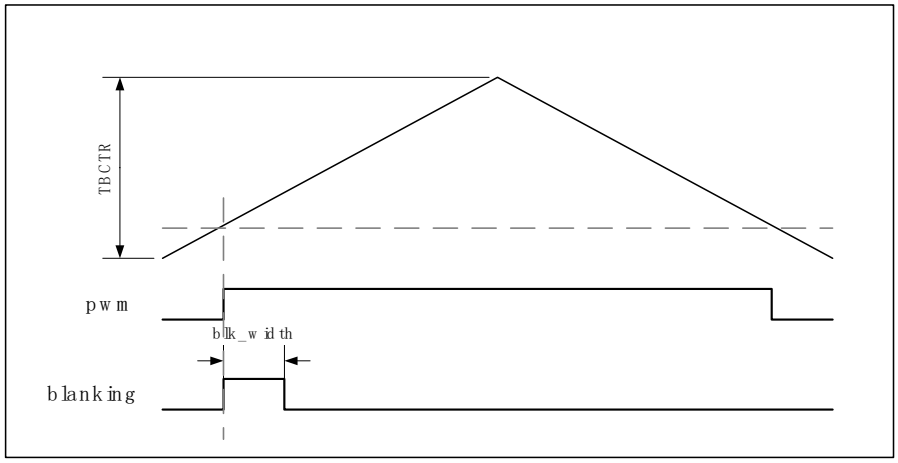
该模块使用 TG 模块产生的计数比较事件产生 pwm_a 和 pwm_b 信号的宽度可配置的随路比较器消隐信号送到模拟比较器 ACMP；

通过 BLKCTLA.blk_a_en 和 BLKCTLA.blk_b_en 设置 EPWM 通道是否输出消隐信号。

通过 BLKCTLA.blk_a_start 和 BLKCTLA.blk_b_start 设置消隐信号生效的起始位置。

通过 BLKCTLA.blk_a_width 和 BLKCTLA.blk_b_width 设置消隐信号维持的宽度。

图 16-8 消隐信号（blanking）与 pwm 信号的关系



⚠注意

配置时需注意 blk_a_width+CMPA<TBPRD , CMPA-blk_a_witdh>0；

或者 blk_b_width+CMPB<TBPRD , CMPB-blk_b_witdh>0，否则可能导致消隐功能失效。

16.5 APT 寄存器概览

APT 实例 2 个，其基地址如下：

- APT0 基地址 0x61004000
- APT1 基地址 0x61004800

说明

以 APT0 基地址举例：

0x61004000+offset 可以同时配置所有通道的寄存器（但是做不到同时读取）；

0x61004200+offset 可以配置 EPWM 通道 0 的寄存器；

0x61004400+offset 可以配置 EPWM 通道 1 的寄存器；

0x61004600+offset 可以配置 EPWM 通道 2 的寄存器；

APT1 同理，两个 APT 的使用是完全独立的。

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	TBCTL	0x0	32	RW	0x00000000	TB 控制寄存器
2	TBSTS	0x4	32	RO	0x00000001	TB 状态寄存器
3	TBPHS	0x8	32	RW	0x00000000	相位寄存器
4	TBCTR	0xC	32	RO	0x00000000	计数器状态寄存器
5	TBPRD	0x10	32	RW	0x0000FFFF	周期寄存器
6	CMPCTL	0x14	32	RW	0x00000000	比较控制寄存器
7	CMPA	0x18	32	RW	0x00000000	CMPA 寄存器
8	CMPB	0x1C	32	RW	0x00000000	CMPB 寄存器
9	CPR1	0x20	32	RW	0x00000000	CPR1 寄存器
10	CPR2	0x24	32	RW	0x00000000	CPR2 寄存器
11	AQCTLA	0x30	32	RW	0x00000000	功能控制寄存器 A
12	AQCTLB	0x34	32	RW	0x00000000	功能控制寄存器 B
13	AQSFRC	0x38	32	RW	0x00000000	软件单次强制寄存器
14	AQCSFRC	0x3C	32	RW	0x00000000	软件连续强制寄存器
15	DBCTL	0x40	32	RW	0x00000000	死区控制寄存器
16	DBRED	0x44	32	RW	0x00000000	死区上升沿延时寄存器
17	DBFED	0x48	32	RW	0x00000000	死区下降沿延时寄存器
18	BLKCTLA	0x58	32	RW	0x00000000	A 路消隐信号控制寄存器
19	BLKCTLB	0x5C	32	RW	0x00000000	B 路消隐信号控制寄存器
20	TZSEL	0x60	32	RW	0x00000000	刹车选择寄存器
21	TZCTL	0x64	32	RW	0x00000000	刹车控制寄存器
22	TZEINT	0x68	32	RW	0x00000000	刹车中断使能寄存器
23	TZFLG	0x6C	32	RO	0x00000000	刹车中断状态寄存器
24	TZCLR	0x70	32	W1P	0x00000000	刹车中断清除寄存器
25	TZFRC	0x74	32	W1P	0x00000000	刹车强制寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
26	reserved	0x78	32	RO	0x00000000	预留寄存器
27	reserved	0x7C	32	RO	0x00000000	预留寄存器
28	ETSEL	0x80	32	RW	0x00000000	事件选择寄存器
29	ETPS	0x84	32	RW	0x00000000	事件控制寄存器
30	ETFLG	0x88	32	RO	0x00000000	事件标志状态寄存器
31	ETCLR	0x8C	32	W1P	0x00000000	事件标志清除寄存器
32	ETFRC	0x90	32	W1P	0x00000000	事件标志强制寄存器
33	TBCTL_RO	0x94	32	RO	0x00000000	TB 控制状态寄存器
34	TBCTL_W1P	0x98	32	W1P	0x00000000	TB 控制脉冲寄存器
35	TBSTS_CLR	0x9C	32	RW	0x00000000	TB 状态清除寄存器
36	TBCTR_RW	0xA0	32	RW	0x00000000	计数器强制寄存器
37	AQSFRC_W1P	0xA4	32	W1P	0x00000000	软件强制脉冲寄存器
38	ETPS_RO	0xA8	32	RO	0x00000000	事件状态寄存器

16.6 APT 寄存器详述

16.6.1 TBCTL

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
etr_ft_lev								etr_behavior				synci_src			
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
free_soft		phs dir	clkdiv			cnte n	res.			syncosel		prdl d	phs en	ctrmode	
RW	RW	RW	RW	RW	RW	RW	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
etr_ft_lev	[31:24]	RW	ETR 信号滤波等级。 0：不滤波 1~255: 滤除配置值数量+2 时钟周期的毛刺	0x0
etr_behavior	[23:20]	RW	由 synci_src 信号选择到的 ETR 信号动作。 0：无任何效果 1：ETR 上升沿处，phsen 有效时，同步 phsdir 和 TBPHS 至计数器 2：ETR 高电平，使能 TBCTR，无视 cnten 3：ETR 上升沿处，同步一次 TBCNT 并于 ETR 高电平或者 cnten 有效期间使能 TBCTR	0x0

Name	Bits	R/W	Description	Default
			4：ETR 有效时，A 相 PWM 强制输出 1，B 相 PWM 强制输出 1 5：ETR 有效时，A 相 PWM 强制输出 0，B 相 PWM 强制输出 0 6：ETR 有效时，A 相 PWM 强制输出 1，B 相 PWM 强制输出 0 7：ETR 有效时，A 相 PWM 强制输出 0，B 相 PWM 强制输出 1 8：TBCTR 在 ETR 上升沿计数（根据当前递增/减配置） 其他：无任何效果 注意当 synci_src 的配置不为 0（来自引脚）时，ETR 无法使用 etr_behavior=2 或者 3 时的功能	
synci_src	[19:16]	RW	ETR 输入源选择。 0：PAD 1：GPT0 2：GPT1 3：GPT2 4：GPT3 5~7：ADC0.AWDG 超门限告警 8~10：ADC1.AWDG 超门限告警 11~15：禁用	0x0
free_soft	[15:14]	RW	当计数器完成一个周期计数时的行为。 0：停止 1：递增模式：当 TBCTR=TBPRD 时停止 递减模式：当 TBCTR=0 时停止 增减模式：当 TBCTR=0 时停止 2 或者 3：继续运行	0x0
phsdir	[13]	RW	选择当同步信号有效或软件强制同步或 TBPHS 值刷新后，计数器计数的方向（递增/递减）。仅当计数器运行在增减模式下该寄存器才有效。 0：同步后以递减模式运行 1：同步后以递增模式运行	0x0
clkdiv	[12:10]	RW	计数器计数时钟（TBCLK）分频参数。 TBCLK=(HCLK)/(clkdiv) 0：1 分频（默认值） 1：2 分频 2：4 分频 3：8 分频 4：16 分频 5：32 分频 6：64 分频	0x0

Name	Bits	R/W	Description	Default
			7：128 分频	
cnten	[9]	RW	计数器计数使能。 0：停止工作 1：写 1 后计数器进入计数工作模式	0x0
reserved	[8:6]	-	保留	0x0
syncosel	[5:4]	RW	同步信号输出控制。作用于 EPWMxSYNCO（同步输出信号）。 0：等于 EPWMxSYNCI（输入同步信号） 1：TBCTR=0 2：TBCTR =CMPB 3：不输出同步信号	0x0
prdld	[3]	RW	控制 TBPRD 寄存器值何时生效。 0：仅当 TBCTR =0 时，TBPRD 值生效 1：TBPRD 值立即生效	0x0
phsen	[2]	RW	TBPHS 赋值给 TBCTR 使能。 etr_behavior=1 并且 ETR 信号有上升沿或者 swfsync_w1p 被写入 1 时， 0：禁止 TBPHS 赋值给 TBCTR 1：允许 TBPHS 赋值给 TBCTR	0x0
ctrmode	[1:0]	RW	TBCTR 计数方式。 0：递增模式 1：递减模式 2：增减模式 3：预留	0x0

16.6.2 TBSTS

◆ **偏移地址：0x4　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													ctrmax	synci	ctrdir
-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
ctrmax	[2]	RO	计数器最大值状态位。 0：计数器未达到最大值。写无效	0x0

Name	Bits	R/W	Description	Default
			1：计数器达到最大值 0xFFFF,(清除状态见 TBSTS_CLR)	
synci	[1]	RO	TBCTR 同步状态。 0：未接收到任何同步信号，写无效 1：接收到外部同步信号（EPWMxSYNCI），(清除状态见 TBSTS_CLR)	0x0
ctrdir	[0]	RO	TBCTR 计数方向，需要先配置 TBCTL.ctrmode。 0：计数器在递减 1：计数器在递增	0x1

16.6.3 TBPHS

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tbphs															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
tbphs	[15:0]	RW	设定在 EPWMxSYNCI 同步 TBCTR 的初始值，从而控制 PWM 的相位。 当 TBCTL.phsen=0，忽略同步信号。 当 TBCTL.phsen=1，当 EPWMxSYNCI 有效或软件强制同步时，计数器值等于 TBPHS。	0x0

16.6.4 TBCTR

◆ **偏移地址：0xC　　复位值：0x00000000**

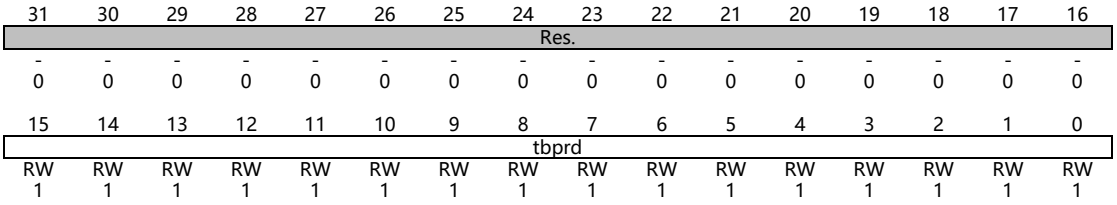
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tbctr ro															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0

Name	Bits	R/W	Description	Default
tbctr_ro	[15:0]	RO	该寄存器值等于计数器值，读取该寄存器可获取当前计数器值。 (可在 TBCTR_RW 直接设置 TBCTR 计数值)	0x0

16.6.5 TBPRD

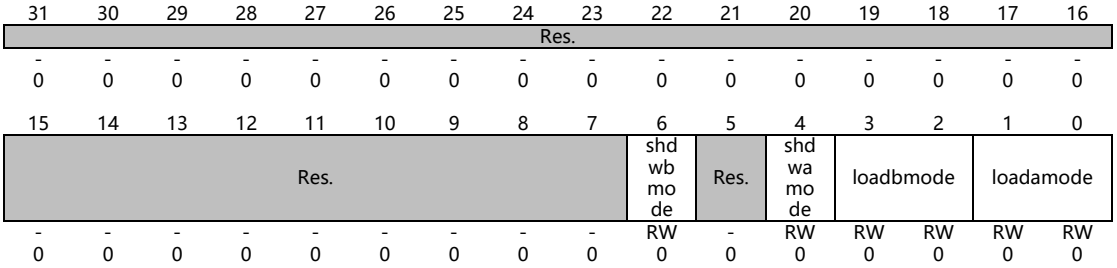
◆ **偏移地址：0x10　　复位值：0x0000FFFF**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
tbprd	[15:0]	RW	用于设定 Time_Gen 中计数器的最大值。	0xFFFF

16.6.6 CMPCTL

◆ **偏移地址：0x14　　复位值：0x00000000**

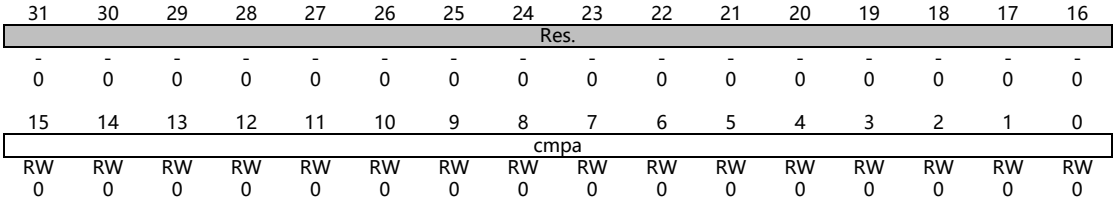


Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留。	0x0
shdwbmode	[6]	RW	CMPB 寄存器模式选择。 0：间接模式，CPU 写 CMPB 的值不会立刻生效 1：直接模式，CPU 写 CMPB 的值立刻生效	0x0
reserved	[5]	-	保留。	0x0
shdwamode	[4]	RW	CMPA 寄存器模式选择。 0：间接模式，CPU 写 CMPA 的值不会立刻生效 1：直接模式，CPU 写 CMPA 的值立刻生效	0x0
loadbmode	[3:2]	RW	间接模式（CMPCTL.shdwbmode=0）CMPB 何时生效。	0x0

Name	Bits	R/W	Description	Default
			0：当 TBCTR =0 时 1：当 TBCTR =PRD 时 2 或者 3：当 TBCTR =0 或 TBCTR =PRD 时	
loadamode	[1:0]	RW	间接模式（CMPCTL.shdwamode=0）CMPA 何时生效。 0：当 TBCTR =0 时 1：当 TBCTR =PRD 时 2 或者 3：当 TBCTR =0 或 TBCTR =PRD 时	0x0

16.6.7 CMPA

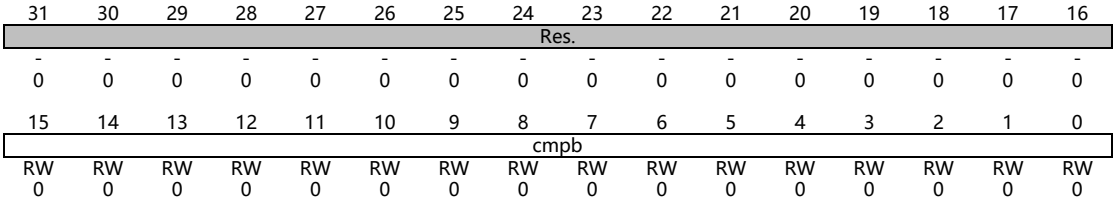
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
cmpa	[15:0]	RW	用于与 TBCTR 做比较，当 TBCTR=CMPA 时，FC 模块根据 AQCTLA、AQCTLB 寄存器的配置产生相应的动作。	0x0

16.6.8 CMPB

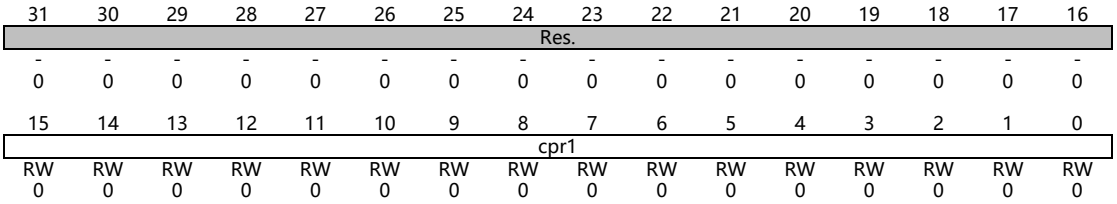
◆ **偏移地址：0x1C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
cmpb	[15:0]	RW	用于与 TBCTR 做比较，当 TBCTR=CMPB 时，FC 模块根据 AQCTLA、AQCTLB 寄存器的配置产生相应的动作。	0x0

16.6.9 CPR1

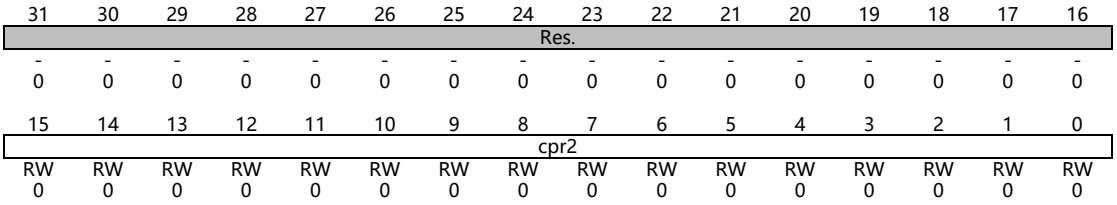
◆ **偏移地址：0x20　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
cpr1	[15:0]	RW	用于与 TBCTR 做比较，当 TBCTR = CPR1 时，ET 模块可产生 ADC 触发信号。	0x0

16.6.10 CPR2

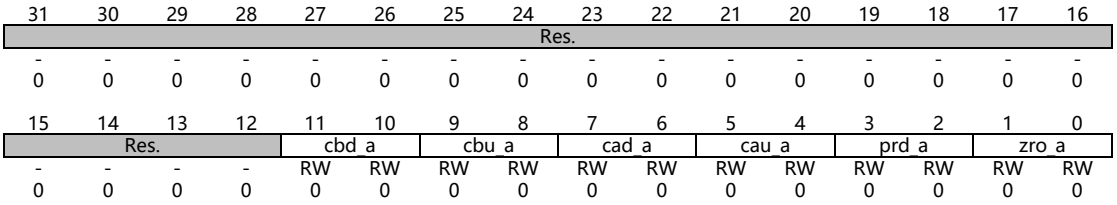
◆ **偏移地址：0x24　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
cpr2	[15:0]	RW	用于与 TBCTR 做比较，当 TBCTR = CPR2 时，ET 模块可产生 ADC 触发信号。	0x0

16.6.11 AQCTLA

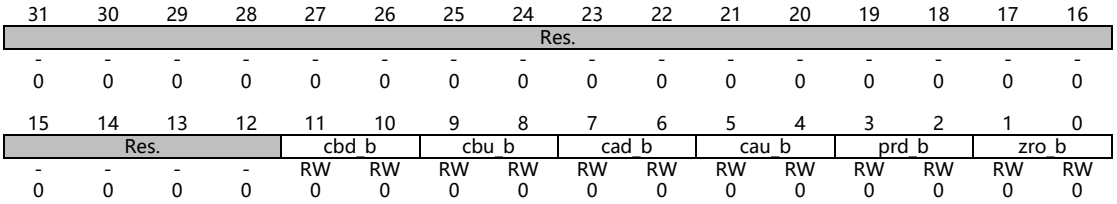
◆ **偏移地址：0x30　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留。	0x0
cbd_a	[11:10]	RW	当 TBCTR=CMPB 且计数器递减阶段时 pwm_a 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
cbu_a	[9:8]	RW	当 TBCTR=CMPB 且计数器递增阶段时 pwm_a 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
cad_a	[7:6]	RW	当 TBCTR=CMPA 且计数器递减阶段时 pwm_a 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
cau_a	[5:4]	RW	当 TBCTR=CMPA 且计数器递增阶段时 pwm_a 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
prd_a	[3:2]	RW	当 TBCTR=PRD，pwm_a 输出的操作。 注意：在增减模式下，当 TBCTR=PRD 时，计数器计数方向为递减，即 ctrdir=0。 0：无动作 1：置低 2：置高 3：翻转	0x0
zro_a	[1:0]	RW	当 TBCTR=0，pwm_a 输出的操作。 注意：在增减模式下，当 TBCTR= 0 时，计数器计数方向为递增，即 ctrdir=1。 0：无动作 1：置低 2：置高 3：翻转	0x0

16.6.12 AQCTLB

◆ **偏移地址：0x34　　复位值：0x00000000**

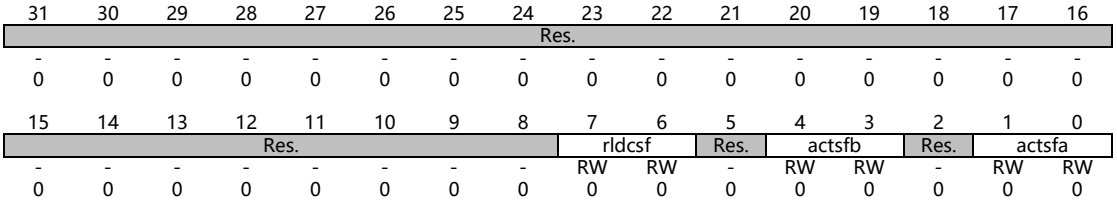


Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留。	0x0
cbd_b	[11:10]	RW	当 TBCTR=CMPB 且计数器递减阶段时 pwm_b 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
cbu_b	[9:8]	RW	当 TBCTR=CMPB 且计数器递增阶段时 pwm_b 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
cad_b	[7:6]	RW	当 TBCTR=CMPA 且计数器递减阶段时 pwm_b 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
cau_b	[5:4]	RW	当 TBCTR=CMPA 且计数器递增阶段时 pwm_b 输出的操作。 0：无动作 1：置低 2：置高 3：翻转	0x0
prd_b	[3:2]	RW	当 TBCTR=PRD，pwm_b 输出的操作。 注意：在增减模式下，当 TBCTR=PRD 时，计数器计数方向为递减，即 ctrdir=0。 0：无动作 1：置低 2：置高	0x0

Name	Bits	R/W	Description	Default
			3：翻转	
zro_b	[1:0]	RW	当 TBCTR=0，pwm_b 输出的操作。 注意：在增减模式下，当 TBCTR= 0 时，计数器计数方向为递增，即 ctrdir=1。 0：无动作 1：置低 2：置高 3：翻转	0x0

16.6.13 AQSFRC

◆ **偏移地址：0x38　　复位值：0x00000000**

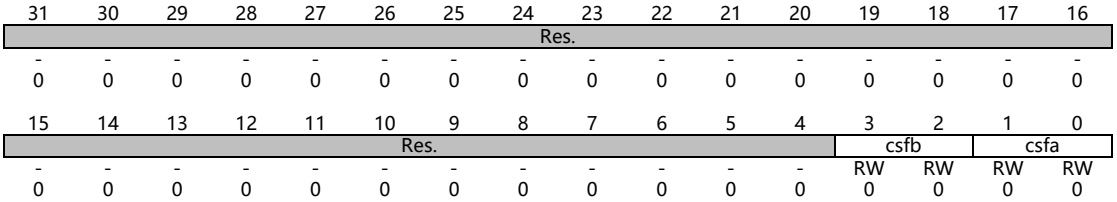


Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
rldcsf	[7:6]	RW	选择软件连续（CBC）强制寄存器（AQCSFRC）值何时生效。 0：间接模式，TBCTR=0 时 1：间接模式，TBCTR=PRD 时 2：间接模式，TBCTR=0 或 TBCTR=PRD 时 3：直接模式，写 AQCSFRC 寄存器后立刻生效	0x0
reserved	[5]	-	保留	0x0
actsfb	[4:3]	RW	选择软件单次（One-Shot）信号产生后 pwm_b 的操作。 0：无动作 1：置低 2：置高 3：翻转 注意：软件单次（One-Shot）强制信号生效时间段：从配置本字段时的 TBCTR 值到下一个比较点的值（下一个比较点可能是 0 或 CMPA 或 CMPB 或 PRD）。	0x0
reserved	[2]	-	保留。	0x0
actsfa	[1:0]	RW	选择软件单次（One-Shot）强制信号产生后 pwm_a 的操作。	0x0

Name	Bits	R/W	Description	Default
			0：无动作 1：置低 2：置高 3：翻转 注意：软件单次（One-Shot）强制信号生效时间段：从配置本字段时的 TBCTR 值到下一个比较点的值（下一个比较点可能是 0 或 CMPA 或 CMPB 或 PRD）。	

16.6.14 AQCSFRC

◆ **偏移地址：0x3C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
csfb	[3:2]	RW	选择软件连续（CBC）强制信号产生后 pwm_b 的操作。 在直接模式下，强制动作在写本寄存器后的下一个 TBCLK 沿作用。 在间接模式下，强制动作在软件 CBC 强制寄存器生效后的下一个 TBCLK 沿作用。 0：软件强制无效 1：置低 2：置高 3：无操作	0x0
csfa	[1:0]	RW	选择软件连续（CBC）强制信号产生后 pwm_a 的操作。 在直接模式下，强制动作在写本寄存器后的下一个 TBCLK 沿作用。 在间接模式下，强制动作在软件 CBC 强制寄存器生效后的下一个 TBCLK 沿作用。 0：软件强制无效 1：置低 2：置高 3：无操作	0x0

16.6.15 DBCTL

◆ **偏移地址：0x40　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										in_mode		polsel		out_mode	
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留。	0x0
in_mode	[5:4]	RW	Dead-band 输入模式选择。 0：pwm_a 上升沿与下降沿都延迟 1：pwm_b 上升沿延迟，pwm_a 下降沿延迟 2：pwm_a 上升沿延迟，pwm_b 下降沿延迟 3：pwm_b 上升沿与下降沿都延迟	0x0
polsel	[3:2]	RW	极性选择。 0：pwm_a 与 pwm_b 均不反向（default） 1：pwm_a 反向 2：pwm_b 反向 3：pwm_a 与 pwm_b 均反向	0x0
out_mode	[1:0]	RW	Dead-band 输出模式选择。 0：关闭 Dead-band 功能，对 pwm_a 与 pwm_b 不作延时处理。在此模式下，polsel 和 in_mode 无效。 1：关闭上升沿延时功能 2：关闭下降沿延时功能 3：同时打开上升沿延时和下降沿延时功能	0x0

16.6.16 DBRED

◆ **偏移地址：0x44　　复位值：0x00000000**

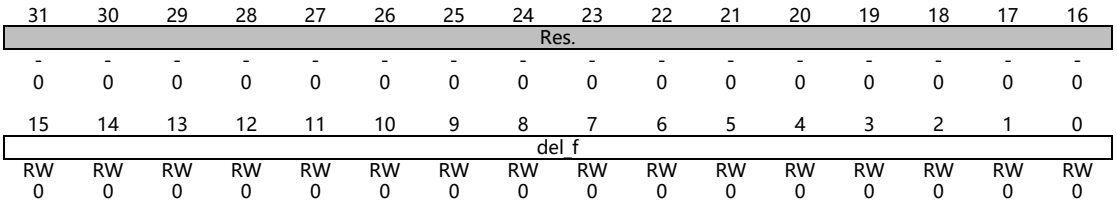
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
del_r															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
del_r	[15:0]	RW	上升沿延时计数值，6-bit 计数器。	0x0

16.6.17 DBFED

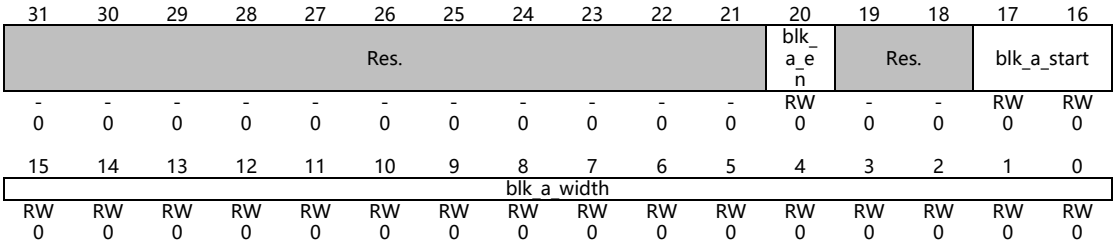
◆ **偏移地址：0x48　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
del_f	[15:0]	RW	下降沿延时计数值，16-bit 计数器。	0x0

16.6.18 BLKCTLA

◆ **偏移地址：0x58　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
blk_a_en	[20]	RW	pwm_a 随路 blanking 信号输出使能。 0：不输出 blanking 信号 1：输出 blanking 信号	0x0
reserved	[19:18]	-	保留。	0x0
blk_a_start	[17:16]	RW	pwm_a 随路 blanking 信号的生效起始位置。 0：TBCTR=0 1：TBCTR=TBPRD 2：TBCTR=CMPA 3：TBCTR=CMPB	0x0

Name	Bits	R/W	Description	Default
blk_a_width	[15:0]	RW	pwm_a 随路 blanking 信号的宽度。 注意宽度设置，不合理的宽度设置可能导致 blanking 信号常有效。 例如于比较点 A 产生 blanking 信号，则该位域+CMPA 的值应该小于 TBPRD 的值。	0x0

16.6.19 BLKCTLB

◆ **偏移地址：0x5C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.											blk_b_en	Res.		blk_b_start	
-	-	-	-	-	-	-	-	-	-	-	RW	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
blk_b_width															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
blk_b_en	[20]	RW	pwm_b 随路 blanking 信号输出使能。 0：不输出 blanking 信号 1：输出 blanking 信号	0x0
reserved	[19:18]	-	保留。	0x0
blk_b_start	[17:16]	RW	pwm_b 随路 blanking 信号的生效起始位置。 0：TBCTR=0 1：TBCTR=TBPRD 2：TBCTR=CMPA 3：TBCTR=CMPB	0x0
blk_b_width	[15:0]	RW	pwm_b 随路 blanking 信号的宽度。 注意宽度设置，不合理的宽度设置可能导致 blanking 信号常有效。 例如于比较点 A 产生 blanking 信号，则该位域+CMPA 的值应该小于 TBPRD 的值。	0x0

16.6.20 TZSEL

◆ **偏移地址：0x60　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
osht14	osht13	osht12	osht11	osht10	osht9	osht8	osht7	cbc14	cbc13	cbc12	cbc11	cbc10	cbc9	cbc8	cbc7
RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	oshtmode	osht6	osht5	osht4	osht3	osht2	osht1	Res.	cbcmode	cbc6	cbc5	cbc4	cbc3	cbc2	cbc1
- 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	- 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0

Name	Bits	R/W	Description	Default
osht14	[31]	RW	TZ14 开关控制。 0：关闭 TZ14 作为单周期刹车信号 1：打开 TZ14 作为单周期刹车信号	0x0
osht13	[30]	RW	TZ13 开关控制。 0：关闭 TZ13 作为单周期刹车信号 1：打开 TZ13 作为单周期刹车信号	0x0
osht12	[29]	RW	TZ12 开关控制。 0：关闭 TZ12 作为单周期刹车信号 1：打开 TZ12 作为单周期刹车信号	0x0
osht11	[28]	RW	TZ11 开关控制。 0：关闭 TZ11 作为单周期刹车信号 1：打开 TZ11 作为单周期刹车信号	0x0
osht10	[27]	RW	TZ10 开关控制。 0：关闭 TZ10 作为单周期刹车信号 1：打开 TZ10 作为单周期刹车信号	0x0
osht9	[26]	RW	TZ9 开关控制。 0：关闭 TZ9 作为单周期刹车信号 1：打开 TZ9 作为单周期刹车信号	0x0
osht8	[25]	RW	TZ8 开关控制。 0：关闭 TZ8 作为单周期刹车信号 1：打开 TZ8 作为单周期刹车信号	0x0
osht7	[24]	RW	TZ7 开关控制。 0：关闭 TZ7 作为单周期刹车信号 1：打开 TZ7 作为单周期刹车信号	0x0
cbc14	[23]	RW	TZ14 开关控制。 0：关闭 TZ14 作为持续刹车信号 1：打开 TZ14 作为持续刹车信号	0x0
cbc13	[22]	RW	TZ13 开关控制。 0：关闭 TZ13 作为持续刹车信号 1：打开 TZ13 作为持续刹车信号	0x0
cbc12	[21]	RW	TZ12 开关控制。 0：关闭 TZ12 作为持续刹车信号	0x0

Name	Bits	R/W	Description	Default
			1：打开 TZ12 作为持续刹车信号	
cbc11	[20]	RW	TZ11 开关控制。 0：关闭 TZ11 作为持续刹车信号 1：打开 TZ11 作为持续刹车信号	0x0
cbc10	[19]	RW	TZ10 开关控制。 0：关闭 TZ10 作为持续刹车信号 1：打开 TZ10 作为持续刹车信号	0x0
cbc9	[18]	RW	TZ9 开关控制。 0：关闭 TZ9 作为持续刹车信号 1：打开 TZ9 作为持续刹车信号	0x0
cbc8	[17]	RW	TZ8 开关控制。 0：关闭 TZ8 作为持续刹车信号 1：打开 TZ8 作为持续刹车信号	0x0
cbc7	[16]	RW	TZ7 开关控制。 0：关闭 TZ7 作为持续刹车信号 1：打开 TZ7 作为持续刹车信号	0x0
reserved	[15]	-	保留。	0x0
oshtmode	[14]	RW	OSHT(One-Shot)使能。可控制每个刹车信号的开/关。刹车触发后，刹车动作会持续直到用户清除 TZCLR 寄存器。 OSHT 刹车时是立即刹车或等待 CTR=0 再刹车 0：立即刹车 1：等待 CTR=0 时刹车	0x0
osht6	[13]	RW	TZ6 开关控制。 0：关闭 TZ6 作为单周期刹车信号 1：打开 TZ6 作为单周期刹车信号	0x0
osht5	[12]	RW	TZ5 开关控制。 0：关闭 TZ5 作为单周期刹车信号 1：打开 TZ5 作为单周期刹车信号	0x0
osht4	[11]	RW	TZ4 开关控制。 0：关闭 TZ4 作为单周期刹车信号 1：打开 TZ4 作为单周期刹车信号	0x0
osht3	[10]	RW	TZ3 开关控制。 0：关闭 TZ3 作为单周期刹车信号 1：打开 TZ3 作为单周期刹车信号	0x0
osht2	[9]	RW	TZ2 开关控制。 0：关闭 TZ2 作为单周期刹车信号 1：打开 TZ2 作为单周期刹车信号	0x0
osht1	[8]	RW	TZ1 开关控制。 0：关闭 TZ1 作为单周期刹车信号 1：打开 TZ1 作为单周期刹车信号	0x0

Name	Bits	R/W	Description	Default
reserved	[7]	-	保留。	0x0
cbcmode	[6]	RW	Cycle-by-Cycle(CBC)使能。当 CBC 发生后，会在 TBCTR 值为 0 时自动解除刹车自动清除 CBC 状态 CBC 刹车时是立即刹车或等待 CTR=0 再刹车。 0：立即刹车 1：等待 CTR=0 再刹车	0x0
cbc6	[5]	RW	TZ6 开关控制。 0：关闭 TZ6 作为持续刹车信号 1：打开 TZ6 作为持续刹车信号	0x0
cbc5	[4]	RW	TZ5 开关控制。 0：关闭 TZ5 作为持续刹车信号 1：打开 TZ5 作为持续刹车信号	0x0
cbc4	[3]	RW	TZ4 开关控制。 0：关闭 TZ4 作为持续刹车信号 1：打开 TZ4 作为持续刹车信号	0x0
cbc3	[2]	RW	TZ3 开关控制。 0：关闭 TZ3 作为持续刹车信号 1：打开 TZ3 作为持续刹车信号	0x0
cbc2	[1]	RW	TZ2 开关控制。 0：关闭 TZ2 作为持续刹车信号 1：打开 TZ2 作为持续刹车信号	0x0
cbc1	[0]	RW	TZ1 开关控制。 0：关闭 TZ1 作为持续刹车信号 1：打开 TZ1 作为持续刹车信号	0x0

16.6.21 TZCTL

◆ **偏移地址：0x64　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
tz14_en	tz13_en	tz12_en	tz11_en	tz10_en	tz9_en	tz8_en	tz7_en	tz14_sel	tz13_sel	tz12_sel	tz11_sel	tz10_sel	tz9_sel	tz8_sel	tz7_sel
RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tz6_en	tz5_en	tz4_en	tz3_en	tz2_en	tz1_en	tz6_sel	tz5_sel	tz4_sel	tz3_sel	tz2_sel	tz1_sel	tzb		tza	
RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0

Name	Bits	R/W	Description	Default
tz14_en	[31]	RW	TZ14 滤波使能选择。 0：Bypass 1：滤波	0x0
tz13_en	[30]	RW	TZ13 滤波使能选择。	0x0

Name	Bits	R/W	Description	Default
			0：Bypass 1：滤波	
tz12_en	[29]	RW	TZ12 滤波使能选择。 0：Bypass 1：滤波	0x0
tz11_en	[28]	RW	TZ11 滤波使能选择。 0：Bypass 1：滤波	0x0
tz10_en	[27]	RW	TZ10 滤波使能选择。 0：Bypass 1：滤波	0x0
tz9_en	[26]	RW	TZ9 滤波使能选择。 0：Bypass 1：滤波	0x0
tz8_en	[25]	RW	TZ8 滤波使能选择。 0：Bypass 1：滤波	0x0
tz7_en	[24]	RW	TZ7 滤波使能选择。 0：Bypass 1：滤波	0x0
tz14_sel	[23]	RW	TZ14 极性选择。 0：高有效 1：低有效	0x0
tz13_sel	[22]	RW	TZ13 极性选择。 0：高有效 1：低有效	0x0
tz12_sel	[21]	RW	TZ12 极性选择。 0：高有效 1：低有效	0x0
tz11_sel	[20]	RW	TZ11 极性选择。 0：高有效 1：低有效	0x0
tz10_sel	[19]	RW	TZ10 极性选择。 0：高有效 1：低有效	0x0
tz9_sel	[18]	RW	TZ9 极性选择。 0：高有效 1：低有效	0x0
tz8_sel	[17]	RW	TZ8 极性选择。 0：高有效 1：低有效	0x0

Name	Bits	R/W	Description	Default
tz7_sel	[16]	RW	TZ7 极性选择。 0：高有效 1：低有效	0x0
tz6_en	[15]	RW	TZ6 滤波使能选择。 0：Bypass 1：滤波	0x0
tz5_en	[14]	RW	TZ5 滤波使能选择。 0：Bypass 1：滤波	0x0
tz4_en	[13]	RW	TZ4 滤波使能选择。 0：Bypass 1：滤波	0x0
tz3_en	[12]	RW	TZ3 滤波使能选择。 0：Bypass 1：滤波	0x0
tz2_en	[11]	RW	TZ2 滤波使能选择。 0：Bypass 1：滤波	0x0
tz1_en	[10]	RW	TZ1 滤波使能选择。 0：Bypass 1：滤波	0x0
tz6_sel	[9]	RW	TZ6 极性选择。 0：高有效 1：低有效	0x0
tz5_sel	[8]	RW	TZ5 极性选择。 0：高有效 1：低有效	0x0
tz4_sel	[7]	RW	TZ4 极性选择。 0：高有效 1：低有效	0x0
tz3_sel	[6]	RW	TZ3 极性选择。 0：高有效 1：低有效	0x0
tz2_sel	[5]	RW	TZ2 极性选择。 0：高有效 1：低有效	0x0
tz1_sel	[4]	RW	TZ1 极性选择。 0：高有效 1：低有效	0x0
tzb	[3:2]	RW	当刹车命令到来时，对 pwm_b 输出的动作。 0：高阻态	0x0

Name	Bits	R/W	Description	Default
			1：置高 2：置低 3：无动作	
tza	[1:0]	RW	当刹车命令到来时，对 pwm_a 输出的动作。 0：高阻态 1：置高 2：置低 3：无动作	0x0

16.6.22 TZEINT

◆ **偏移地址：0x68　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													ost_en	cbc_en	Res.
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
ost_en	[2]	RW	One-Shot 中断使能。 0：关闭 One-Shot 中断 1：打开 One-Shot 中断。OST 事件将触发 EPWMxTZINT	0x0
cbc_en	[1]	RW	CBC 中断使能。 0：关闭 CBC 中断 1：打开 CBC 中断。CBC 事件将触发 EPWMxTZINT	0x0
reserved	[0]	-	保留。	0x0

16.6.23 TZFLG

◆ **偏移地址：0x6C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													ost_flg	cbc_flg	tzint_flg
-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
ost_flg	[2]	RO	OST 状态标记，供查询。 0：未发生 OST 1：处于 OST 事件。当写 TZCLR 寄存器时，OST 标记将清除	0x0
cbc_flg	[1]	RO	CBC 状态标记，供查询。 0：未发生 CBC 1：处于 CBC 事件。用户写 TZCLR 寄存器，CBC 标记会立即置 0。	0x0
tzint_flg	[0]	RO	中断状态标记，供查询。 0：未发生中断 1：产生了 EPWMxTZINT 中断	0x0

16.6.24 TZCLR

◆ **偏移地址：0x70　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													ost_clr	cbc_clr	tzint_clr
-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
ost_clr	[2]	W1P	清除 OST 状态标记。 0：无效，任何时候读都为 0 1：清除 OST 状态	0x0
cbc_clr	[1]	W1P	清除 CBC 状态标记。 0：无效，任何时候读都为 0 1：清除 CBC 状态	0x0
tzint_clr	[0]	W1P	清除中断状态标记。 0：无效，任何时候读都为 0 1：清除中断状态	0x0

16.6.25 TZFRC

◆ **偏移地址：0x74　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													ost_frc	cbc_frc	Res.
-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
ost_frc	[2]	W1P	强制进入 OST。 0：无效，任何时候读都为 0 1：强制进入 OST	0x0
cbc_frc	[1]	W1P	强制进入 CBC。 0：无效，任何时候读都为 0 1：强制进入 CBC	0x0
reserved	[0]	-	保留。	0x0

16.6.26 ETSEL

◆ **偏移地址：0x80　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

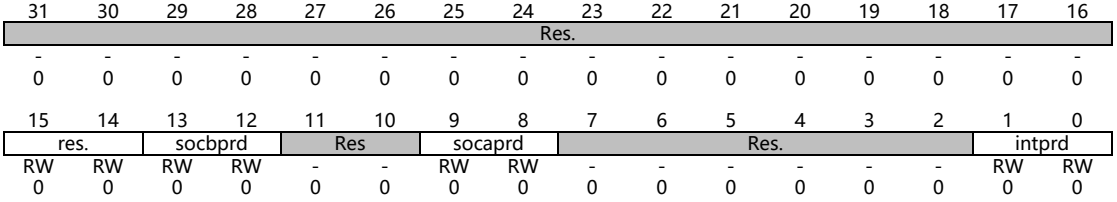
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
socben	socbsel				socaen	socasel				Res.		inten	intsel		
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
socben	[15]	RW	pwm_soc_b 输出使能。 0：关闭 1：打开	0x0
socbsel	[14:11]	RW	pwm_soc_b 在何时产生脉冲。 0：不产生脉冲 1：当 TBCTR=0 2：当 TBCTR=PRD 3：当 TBCTR=0 或 PRD 4：当 TBCTR=CMPA 且 ctrdir 递增 5：当 TBCTR=CMPA 且 ctrdir 递减 6：当 TBCTR=CMPB 且 ctrdir 递增	0x0

Name	Bits	R/W	Description	Default
			7：当 TBCTR=CMPB 且 ctrdir 递减 8：当 TBCTR=CPR1 且 ctrdir 递增 9：当 TBCTR=CPR1 且 ctrdir 递减 10：当 TBCTR=CPR2 且 ctrdir 递增 11：当 TBCTR=CPR2 且 ctrdir 递减 其他：禁用	
socaen	[10]	RW	pwm_soc_a 输出使能。 0：关闭 1：打开	0x0
socasel	[9:6]	RW	pwm_soc_a 在何时产生脉冲。 0：不产生脉冲 1：当 TBCTR=0 2：当 TBCTR=PRD 3：当 TBCTR=0 或 PRD 4：当 TBCTR=CMPA 且 ctrdir 递增 5：当 TBCTR=CMPA 且 ctrdir 递减 6：当 TBCTR=CMPB 且 ctrdir 递增 7：当 TBCTR=CMPB 且 ctrdir 递减 8：当 TBCTR=CPR1 且 ctrdir 递增 9：当 TBCTR=CPR1 且 ctrdir 递减 10：当 TBCTR=CPR2 且 ctrdir 递增 11：当 TBCTR=CPR2 且 ctrdir 递减 其他：禁用	0x0
reserved	[5:4]	-	保留。	0x0
inten	[3]	RW	ePWM 中断（EPWMxINT）使能。 0：关闭 1：打开	0x0
intsel	[2:0]	RW	ePWM 中断在何时产生。 0：不产生 1：当 TBCTR=0 2：当 TBCTR=PRD 3：当 TBCTR=0 或 PRD 4：当 TBCTR=CMPA 且 ctrdir 递增 5：当 TBCTR=CMPA 且 ctrdir 递减 6：当 TBCTR=CMPB 且 ctrdir 递增 7：当 TBCTR=CMPB 且 ctrdir 递减	0x0

16.6.27 ETPS

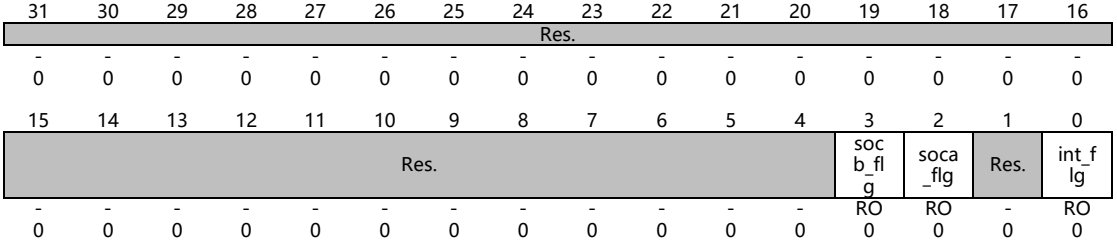
◆ **偏移地址：0x84　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
socbprd	[13:12]	RW	设定多少个 ETSEL.socbsel 事件触发一次 ADC。 0：从不产生 EPWMxSOCB 脉冲 1：当 ETPS_RO.socbcnt_ro=1 时，产生 EPWMxSOCB 脉冲 2：当 ETPS_RO.socbcnt_ro=2 时，产生 EPWMxSOCB 脉冲 3：ETPS_RO.socbcnt_ro=3 时，产生 EPWMxSOCB 脉冲	0x0
reserved	[11:10]	-	保留。	0x0
socaprd	[9:8]	RW	设定多少个 ETSEL.socasel 事件触发一次 ADC。 0：从不产生 EPWMxSOCA 脉冲 1：当 ETPS_RO.socacnt_ro=1 时，产生 EPWMxSOCA 脉冲 2：当 ETPS_RO.socacnt_ro=2 时，产生 EPWMxSOCA 脉冲 3：当 ETPS_RO.socacnt_ro=3 时，产生 EPWMxSOCA 脉冲	0x0
reserved	[7: 2]	-	保留。	0x0
intprd	[1:0]	RW	设定多少个 ETSEL.intsel 事件触发一次中断。 0：从不产生中断 1：1 个 intsel 事件产生中断脉冲 2：2 个 intsel 事件产生中断脉冲 3：3 个 intsel 事件产生中断脉冲	0x0

16.6.28 ETFLG

◆ **偏移地址：0x88　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
socb_flg	[3]	RO	EPWMxSOCB 触发 ADC 状态标记。 0：未发生 1：表示 EPWMxSOCB 已触发 ADC	0x0
soca_flg	[2]	RO	EPWMxSOCA 触发 ADC 状态标记。 0：未发生 1：表示 EPWMxSOCA 已触发 ADC	0x0
reserved	[1]	-	保留。	0x0
int_flg	[0]	RO	中断（EPWMxINT）状态标记 0：未发生 1：表明已产生中断。只有清除该状态才会产生下一个中断脉冲。	0x0

16.6.29 ETCLR

◆ **偏移地址：0x8C　　复位值：0x00000000**

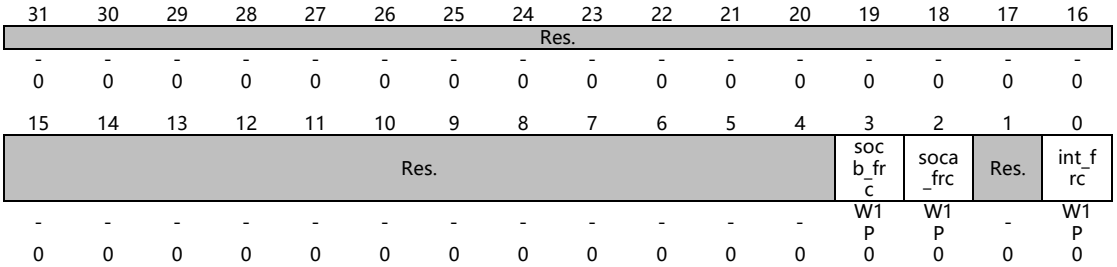
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												socb_clr	soca_clr	Res.	int_clr
-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	-	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
socb_clr	[3]	W1P	EPWMxSOCB 触发 ADC 状态标记清除。 0：无效 1：清除 ETFLG.socb_flg 标记位	0x0
soca_clr	[2]	W1P	EPWMxSOCA 触发 ADC 状态标记清除。 0：无效 1：清除 ETFLG.soca_flg 标记位	0x0
reserved	[1]	-	保留。	0x0
int_clr	[0]	W1P	中断（EPWMxINT）状态标记清除。 0：无效 1：清除 ETFLG.int_flg 标记位	0x0

16.6.30 ETFRC

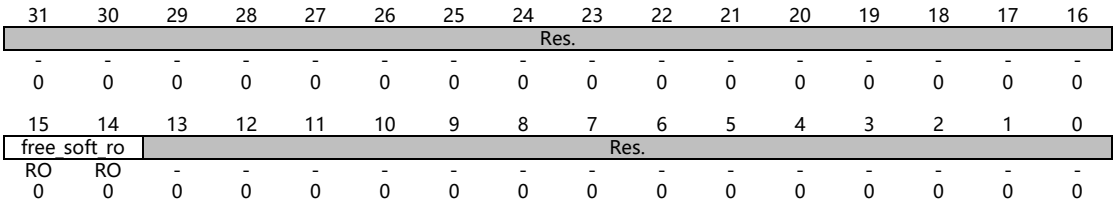
◆ **偏移地址：0x90　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
socb_frc	[3]	W1P	强制产生 EPWMxSOCB 事件脉冲。 0：无意义 1：强制产生 EPWMxSOCB 事件脉冲并置 SOCBFLG 寄存器	0x0
soca_frc	[2]	W1P	强制产生 EPWMxSOCA 事件脉冲。 0：无意义 1：强制产生 EPWMxSOCA 事件脉冲并置 SOCAFLG 寄存器	0x0
reserved	[1]	-	保留。	0x0
int_frc	[0]	W1P	强制产生中断事件。 0：无意义 1：强制产生中断事件	0x0

16.6.31 TBCTL_RO

◆ **偏移地址：0x94　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
free_soft_ro	[15:14]	RO	当计数器完成一个周期时计数器的行为。 0：停止 1：递增模式：当 TBCTR= TBPRD 时停止	0x0

Name	Bits	R/W	Description	Default
			递减模式：当 TBCTR=0 时停止 增减模式：当 TBCTR=0 时停止 2 或者 3：继续运行	
reserved	[13:0]	-	保留。	0x0

16.6.32 TBCTL_W1P

◆ **偏移地址：0x98　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.									swfsync_w1p	Res.					
-	-	-	-	-	-	-	-	-	W1P	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
swfsync_w1p	[6]	W1P	软件强制同步控制。该同步动作只有在 syncosel=0 时生效。 0：写 0 后无同步动作，任何时候读该寄存器值都为 0 1：写 1 后硬件将产生同步脉冲	0x0
reserved	[5:0]	-	保留	0x0

16.6.33 TBSTS_CLR

◆ **偏移地址：0x9C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													ctrmax_clr	synci_clr	Res.
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
ctrmax_clr	[2]	RW	计数器最大值状态位清除。 0：不清除	0x0

Name	Bits	R/W	Description	Default
			1：清除	
synci_clr	[1]	RW	TBCTR 同步状态位清除。 0：不清除 1：清除	0x0
reserved	[0]	-	保留。	0x0

16.6.34 TBCTR_RW

◆ **偏移地址：0xA0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
tbctr															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
tbctr	[15:0]	RW	写该寄存器将会立刻将当前 TBCTR 计数器值刷新为寄存器值。	0x0

16.6.35 AQSFRC_W1P

◆ **偏移地址：0xA4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

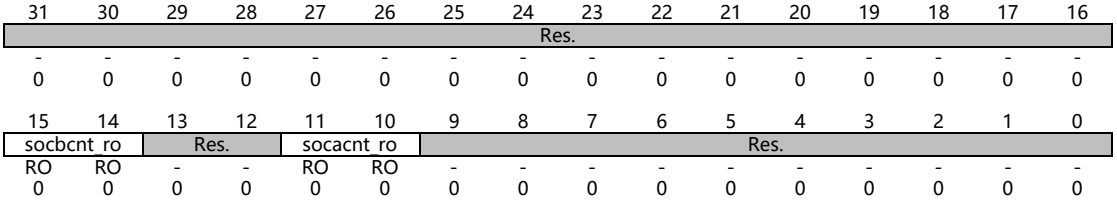
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										otsfb	Res.		otsfa	Res.	
-	-	-	-	-	-	-	-	-	-	W1P	-	-	W1P	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留。	0x0
otsfb	[5]	W1P	软件强制控制 pwm_b。 0：无操作。任何时候读都为 0。 1：产生软件强制信号。One-shot 类型，当写 1 后，软件强制信号产生短脉冲后清零。	0x0
reserved	[4:3]	-	保留。	0x0
otsfa	[2]	W1P	软件强制控制 pwm_a。	0x0

Name	Bits	R/W	Description	Default
			0：无操作。任何时候读都为 0。 1：产生软件强制信号。One-shot 类型，当写 1 后，软件强制信号产生短脉冲后清零。	
reserved	[1:0]	-	保留	0x0

16.6.36 ETPS_RO

◆ **偏移地址：0xA8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
socbcnt_ro	[15:14]	RO	ETSEL.socbsel 事件计数器。当产生 EPWMxSOCB 时，该寄存器将清零。 0：0 1：1 2：2 3：3	0x0
reserved	[13:12]	-	保留	0x0
socacnt_ro	[11:10]	RO	ETSEL.socasel 事件计数器。当产生 EPWMxSOCA 时，该寄存器将清零。 0：0 1：1 2：2 3：3	0x0
reserved	[9:0]	-	保留。	0x0

17 窗口看门狗（WWDG)

17.1 WWDG 简介

窗口看门狗 Window Watchdog（WWDG）通常用于检测由于外部干扰等导致的软件故障。该看门狗支持两种工作模式：窗口模式和普通模式。工作在窗口模式时，计数到上边界减 1 时产生中断；工作在普通模式时，当计数到上限时产生中断，再次计数到上限且未清除中断时，会产生系统复位请求信号。该窗口看门狗的时钟可通过 WDT_SCALE 进行分频从而实现更大时间范围的监测。

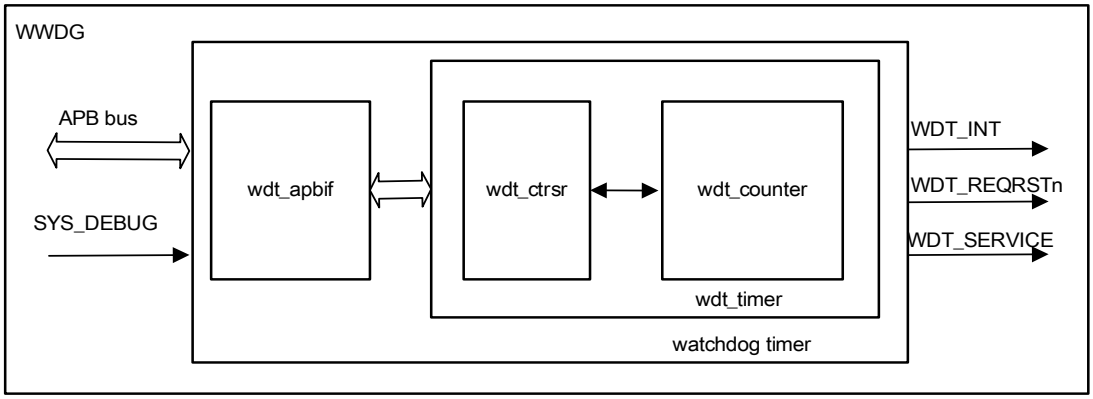
17.2 WWDG 主要特性

WWDG 的主要功能特点有：

- 支持 APB 总线接口，工作时钟为 clk_ls。
- 计时步长(分频系数)可通过 WDT_SCALE 配置。
- 支持基于窗口的喂狗方式，CPU 必须在窗口范围内喂狗，窗口的下边界和上边界可配置，达到窗口上边界减 1 时产生中断。
- 支持窗口模式和普通模式。
- 窗口模式，喂狗时机错误（窗口功能打开）、计数器越界、喂狗特征值写错可立即产生复位信号。
- 支持复位信号的产生与屏蔽
- 普通模式（同 IWDG），即第一次越界产生中断，第二次越界时如果中断没有清除则产生复位输出；喂狗特征值写错或喂狗时机错误（窗口功能打开时）也会产生复位输出。
- 看门狗运行中若要配置寄存器，需先将寄存器解锁再配置，否则寄存器不能写入。
- 为方便产品程序 debug，支持当 CPU 处于 HALT 时，看门狗所有逻辑停止工作，包括 APB 配置逻辑。

17.3 WWDG 功能描述

图 17-1 WWDG 框图



WWDG 主要用于系统软件运行状态监测，避免系统软件跑飞陷入死循环，导致系统异常。WWDG 的主要框图如图 17-1 所示，其中 SYS_DEBUG 表示 CPU 是否 HALT，高电平有效。

CPU 配置 WWDG 的控制寄存器，包括计时周期、喂狗窗口等，然后 watchdog timer 根据配置从 0 开始递增计时，CPU 须在窗口范围内喂狗，喂狗特征值写错、在范围外喂狗或超过上边界还未喂狗则都认为程序异常，为方便系统应用，可根据配置产生中断或复位。

此模块支持两种工作方式：窗口模式和普通模式。在窗口模式时，只要内部 counter 超过窗口的上边界、在窗口的下边界以内喂狗（窗口功能打开）、喂狗值写错均产生复位。普通模式（同 IWDG），在 counter 第一次超过窗口的上边界会产生中断，在 counter 第二次超过上边界时如果中断还没清除则会产生复位；如果窗口功能打开，在窗口下边界以内喂狗以及喂狗写错都会产生复位输出。

WWDG 逻辑全在片内工作，且均采用基于总线的 CPU 控制模式，所以主要考虑程序操作失误的情况，设置了寄存器配置 lock 机制，操作寄存器前必须先 unlock，否则不能配置，避免程序误操作；对喂狗特征值也有要求，喂狗 value 必须符合设定值，否则会引起复位；比常规喂狗机制更可靠，程序必须在一段窗口范围内喂狗，早于或超过均会异常。通过这三种方式，降低系统程序出错导致系统错误的概率。

17.4 WWDG 使用流程

图 17-2 WWDG 软件操作流程

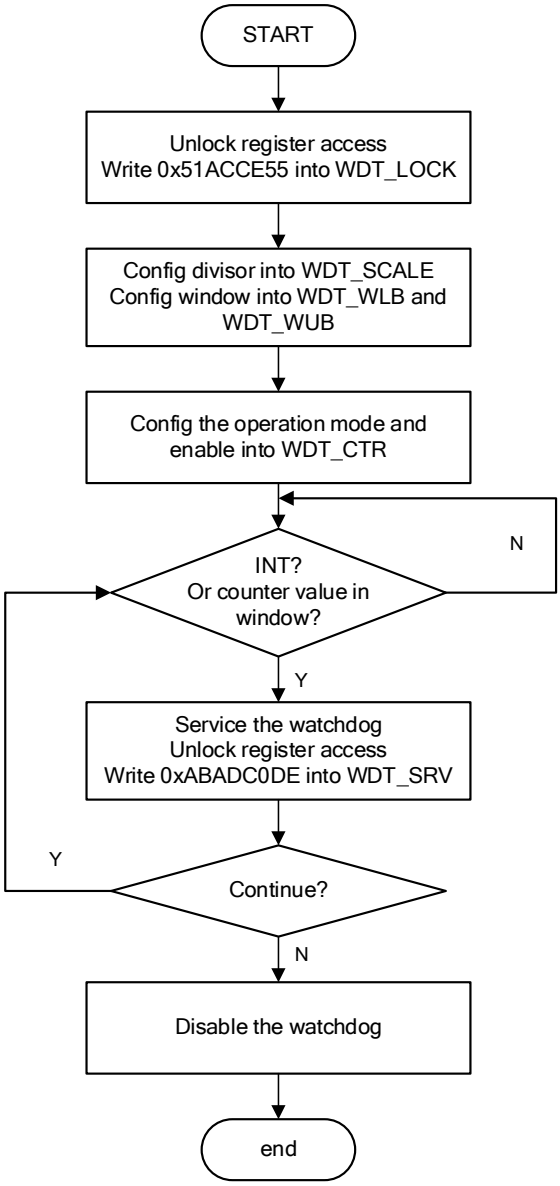


图 17-2 为 WWDG 的软件操作流程。

在窗口模式时，WWDG 会在窗口上边界减 1 处产生中断，这是喂狗的最后时机，需要保证在窗口内进行喂狗；在普通模式（同 IWDG）时，中断发生后，则需喂狗，若第二次越界且上次中断未被清除则产生复位输出。

关于窗口模式和普通模式的功能，请参考 17.3 章节的描述。

操作流程描述如下：

步骤 1 系统上电复位完成后，各时钟稳定且 CPU 程序 load 完成，并进行 IP 初始化；

步骤 2 开启 WWDG 功能，配置寄存器，打开使能、配置窗口范围、分频参数等；

步骤 3 WWDG 开始工作。窗口模式时，若 counter 越界、喂狗值错误、喂狗时机错误（窗口功能打开）则直接产生复位，这就要求 CPU 必须在 counter 处于窗口范围内喂狗且喂狗值正确；普通模式（同 IWDG）时，第一次越界会产生中断，CPU 接收到中断后应立即喂狗，若在 counter 第二次越界时中断还未清除则会产生复位输出；如果喂狗时机错误（窗口功能打开）或喂狗特征值写错，则都会产生复位输出；

步骤 4 CPU 响应中断，喂狗，WWDG 继续工作；

使用 WWDG 时需要注意以下内容：

- 正常使用时，为确保 WWDG 功能有效，必须将中断和复位使能打开；
- WDT_CTR.glb_enb 相当于软件复位，写 0 则内部逻辑会被清零；
- 窗口模式实际使用场合比较稀少，对软件程序的要求很高，通常适用于 MCU 级的应用，且程序对时间把握很精准；逻辑支持计数到上边界减 1 时产生中断，提醒已到最后喂狗时机。
- 在写除 WDT_LOCK 外的其他寄存器前均需先 unlock，写完后 lock，确保寄存器不被错误访问。
- counter 被清零的条件：
  - 向 WDT_CTR.glb_enb 写 0；
  - 写入正确的喂狗特征值；
  - counter 超过窗口上边界；
  - 改变窗口上边界或下边界的值；
  - 产生复位输出。
- counter 停止计数的条件：
  - 系统调试有效且 HALT 功能被使能，如果刚好在上边界上有效，则 counter 停在边界处，不会产生中断和复位输出；
  - 产生了复位请求后，内部 counter 不计数。
- 产生中断的条件：
  - 窗口模式，窗口上边界值减 1 处产生中断；
  - 普通模式（同 IWDG），中断使能打开，第一次 counter 超边界产生中断。

- 产生复位的条件：
  - 窗口模式，复位使能打开，喂狗不在窗口范围内（窗口功能打开）；
  - 窗口模式，复位使能打开，counter 超边界；
  - 窗口模式，复位使能打开，喂狗特征值写错；
  - 普通模式（同 IWDG），复位使能打开，在跨边界中断已产生且没有清除的情况下再产生 counter 超边界；
  - 普通模式（同 IWDG），复位使能打开，喂狗时机错误（窗口功能打开）；
  - 普通模式（同 IWDG），复位使能打开，喂狗特征值写入错误。
- 产生正常喂狗的条件：
  - 窗口功能打开，在窗口范围内喂狗且喂狗特征值正确；
  - 窗口功能关闭，在上边界以内喂狗且喂狗特征值正确。
- 为保证可靠性，喂狗时喂狗特征值必须正确，不正确时会产生复位；
- 窗口功能打开后，窗口的上边界必须大于下边界，否则模块不工作；
- 如果要改边界值，建议在喂狗特征值写了后再修改；
- 系统调试有效时，内部逻辑停止工作，包括 APB 寄存器配置逻辑。

17.5 WWDG 寄存器概述

WWDG 寄存器基地址如下：

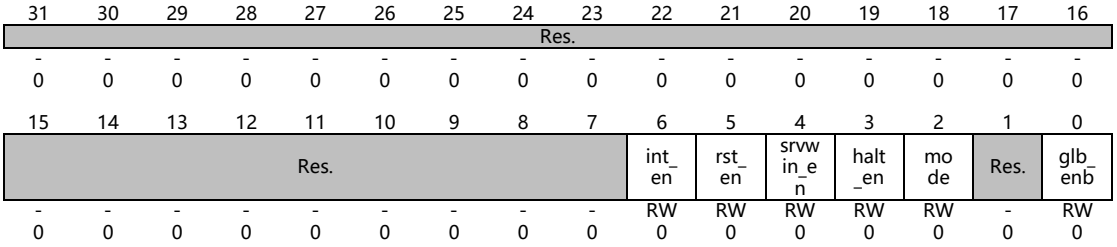
- WWDG 寄存器基地址：0x60040000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	WDT_CTR	0x0	32	RW	0x00000000	控制寄存器
2	WDT_SCALE	0x4	32	RW	0x00000001	分频寄存器
3	WDT_SRV	0x8	32	RW	0x00000000	喂狗寄存器
4	WDT_WLB	0xC	32	RW	0x00000000	窗口上边界配置寄存器
5	WDT_WUB	0x10	32	RW	0xFFFFFFFF	窗口下边界配置寄存器
6	WDT_LOCK	0x14	32	RW	0x00000000	写保护寄存器
7	WDT_COUNT	0x18	32	RO	0x00000000	计数寄存器
8	WDT_STATUS	0x1C	32	RO	0x00000000	状态寄存器

17.6 WWDG 寄存器描述

17.6.1 WDT_CTR

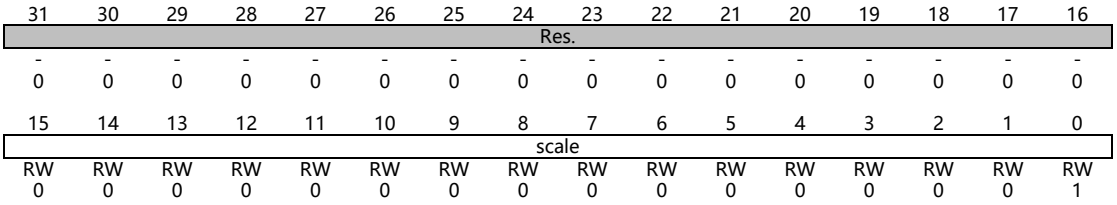
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留。	0x0
int_en	[6]	RW	中断产生使能，高有效。 0：禁止中断 1：使能中断	0x0
rst_en	[5]	RW	复位信号产生使能，高有效。 0：禁止产生复位信号 1：使能产生复位信号	0x0
srvwin_en	[4]	RW	窗口功能开关，高有效。 0：窗口功能关，只要在上边界以内喂狗即可 1：窗口功能开，必须在窗口范围内喂狗	0x0
halt_en	[3]	RW	Halt 功能使能。 0：当系统调试时，看门狗继续计数 1：当系统调试时，看门狗停止计数，且逻辑停止工作，不能写寄存器	0x0
mode	[2]	RW	工作模式选择。 0：窗口模式 1：普通模式	0x0
reserved	[1]	-	保留。	0x0
glb_enb	[0]	RW	看门狗定时器使能。 0：禁止 1：使能 注意：当写为 0 后，内部逻辑被清零。	0x0

17.6.2 WDT_SCALE

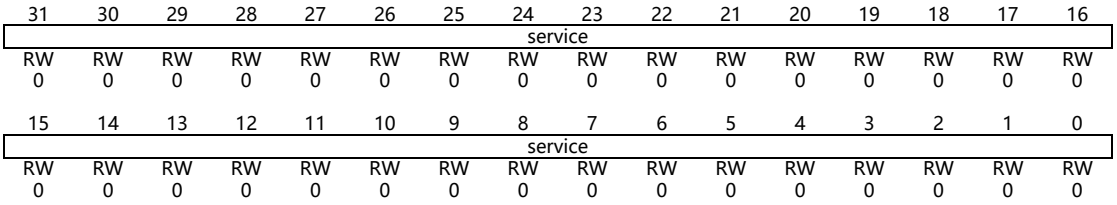
◆ **偏移地址：0x4　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
scale	[15:0]	RW	WWDG 计时参考时钟分频配置。总线时钟经过 scale 分频后，作为 WWDG 的计时参考时钟。 注意：该寄存器推荐的配置范围是：0x80~0xFFFF。当分频系数较小时，WWDG 的计数频率会很快，容易导致喂狗不及时从而产生系统复位。	0x1

17.6.3 WDT_SRV

◆ **偏移地址：0x8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
service	[31:0]	RW	喂狗特征值寄存器。在窗口内写入 0xABADC0DE 时即表示喂狗。若写入其他值会立即产生复位输出。如果有中断发生，写喂狗特征值会将中断清除。	0x0

17.6.4 WDT_WLB

◆ **偏移地址：0xC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
low_bound															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
low_bound															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
low_bound	[31:0]	RW	窗口下边界设置。	0x0

17.6.5 WDT_WUB

◆ **偏移地址：0x10　　复位值：0xFFFFFFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
up_bound															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
up_bound															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
up_bound	[31:0]	RW	窗口上边界设置。必须大于 WDT_WLB 中的值。	0xFFFFFFFF

17.6.6 WDT_LOCK

◆ **偏移地址：0x14　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
lock															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
lock															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
lock	[31:0]	RW	写保护寄存器。当向此寄存器写入 0x51ACCE55 时，寄存器配置被 unlock，可配置其他寄存器；当写入其他值时，寄存器配置被 lock，不能配置其他寄存器。 当读时作为 Lock 状态指示。 1：寄存器配置 unlock，可以配置寄存器	0x0

Name	Bits	R/W	Description	Default
			0：寄存器配置 lock，不能配置寄存器	

17.6.7 WDT_COUNT

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
count															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
count															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
count	[31:0]	RO	WWDG 内部 counter 当前计数值，供软件读取，了解当前 counter 计时。	0x0

17.6.8 WDT_STATUS

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															cnt_ext
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
cnt_ext	[0]	RO	Counter 计数越界。 1：counter 计数已超过上边界值 0：counter 计数未超过上边界值	0x0

18 独立看门狗（IWDG）

18.1 IWDG 简介

独立看门狗 IWDG 用于系统异常情况下，一定时间内发出中断/复位信号，以复位系统或使系统进入中断函数执行相应操作。

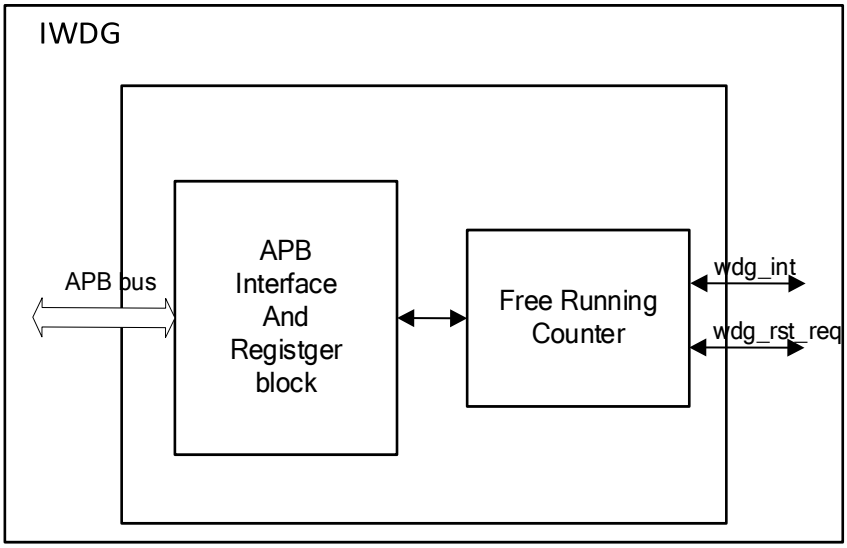
18.2 IWDG 主要特性

IWDG 具备以下特点：

- 内部具有一个 32bit 减法计数器，计数初值可配置。
- 计数时钟为 32.768K 的分频时钟，分频系数通过 IWDG_CRG 寄存器配置（该寄存器在 PMC 寄存器中）。
- 支持寄存器锁定，防止寄存器被误改。
- 支持超时中断产生。
- 支持复位信号产生与屏蔽。
- 支持调试模式，调试模式下自动停止 IWDG 计数。

18.3 IWDG 功能描述

图 18-1 IWDG 框图



WatchDog 模块框图如图 18-1 所示。

系统通过总线配置 WatchDog 计数初值、复位/中断信号使能等参数。WatchDog 内部计数器开始计数，定时发出 wdg_int 中断请求给系统，并在系统没有响应中断的情况下（如：死机），发出 wdg_rst_req 复位信号，使系统复位，达到监控系统运行的目的。

功能原理

WatchDog 的运行基于 1 个 32bit 减法计数器，计数初值由 WDG_LOAD 载入。在 WatchDog 时钟使能情况下，计数值在每个计数时钟（时钟可配）的上升沿减 1。当计数值递减到 0，WatchDog 将产生一个中断信号 wdg_int。然后在下一个计数时钟上升沿，计数器又从 WDG_LOAD 中重新载入计数初值，开始递减计数。WatchDog 计数器当前的计数值可以从 WDG_VALUE 查询。如果计数器的计数值第二次计数递减到 0 时，CPU 还没有清除 WatchDog 中断，则 WatchDog 将发出复位信号 wdg_rst_req，以复位系统，同时计数器停止计数。

中断和复位

根据实际应用需要，可通过配置 WDG_CONTROL 使能或者禁止 WatchDog 产生中断和复位信号：

- 当禁止产生中断时，计数器将停止计数。
- 当重新开启中断时，WatchDog 将从 WDG_LOAD 的设定值开始计数，而不是从计数器上次停止时的计数值开始计数。在中断到来之前，可以重新载入初值。

WatchDog 的原始中断状态和屏蔽后中断状态可通过 WDG_RIS 和 WDG_MIS 查询。可以通过 WDG_INTCLR 清除中断。

寄存器锁定

通过配置 WDG_LOCK，可以禁止对 WatchDog 内部寄存器进行写操作：

- 向 WDG_LOCK 写入 0x1ACC_E551，可以打开所有 WatchDog 寄存器的写权限。
- 向 WDG_LOCK 写入其他任何值，可以关闭所有 WatchDog 寄存器（WDG_LOCK 除外）的写权限。

该特性保护 WatchDog 的寄存器不被软件错误修改，从而使得在异常情况下，WatchDog 不至于被软件错误地中止操作。

调试模式

在调试模式下，WatchDog 自动关闭，以防止干扰正常的调试操作。

18.4 IWDG 典型配置

18.4.1 计数初值配置

在向 WDG_LOAD 载入计数初值之前需要进行计算。

$Value_{WDG_LOAD} = T_{WDG} \times f_{clk}$

说明

其中上式中，各参数代表的意思分别是：

- T_{WDG} 表示 WatchDog 计数时间；
- $Value_{WDG_LOAD}$ 表示 WatchDog 计数初值；
- f_{clk} 表示 WatchDog 计数时钟频率。

18.4.2 系统初始化配置

系统上电复位后 WatchDog 计数器处于停止计数状态，在系统初始化过程中需要将 WatchDog 初始化并启动其运行。WatchDog 的初始化过程如下：

步骤 1 根据想要设置的超时时长和 WatchDog 的时钟频率，计算计数初值。写 WDG_LOAD，设定计数初值。

步骤 2 写 WDG_CONTROL，解除中断/复位屏蔽并启动 WatchDog 计数。

步骤 3 写除 0x1ACCE551 外的任意值给 WDG_LOCK，给 WatchDog 上锁，防止软件错误修改 WatchDog 的配置。

18.4.3 中断处理过程

收到 WatchDog 发出的中断后，系统应及时清除其中断状态，并使其载入计数初值重新开始计数。若不及时清除中断状态，则在 WatchDog 第二次计数值递减到 0 后，会产生复位信号复位整个系统，WatchDog 停止工作。WatchDog 中断处理的过程如下：

步骤 1 向 WDG_LOCK 写 0x1ACC_E551，解除 WatchDog 寄存器写保护。

步骤 2 写 WDG_INTCLR，清除 WatchDog 的中断状态，同时也使 WatchDog 自动载入计数初值重新开始计数。

步骤 3 向 WDG_LOCK 写入 0x1ACC_E551 以外的任何值，给 WatchDog 重新上锁。

18.4.4 关闭 WatchDog

向 WDG_CONTROL [int_en]控制位写入 0 或 1 控制 WatchDog 的状态：

- 0：关闭 WatchDog；
- 1：打开 WatchDog。

18.5 IWDG 寄存器概览

IWDG 寄存器基地如下：

- IWDG 寄存器基地址为：0x61005A00

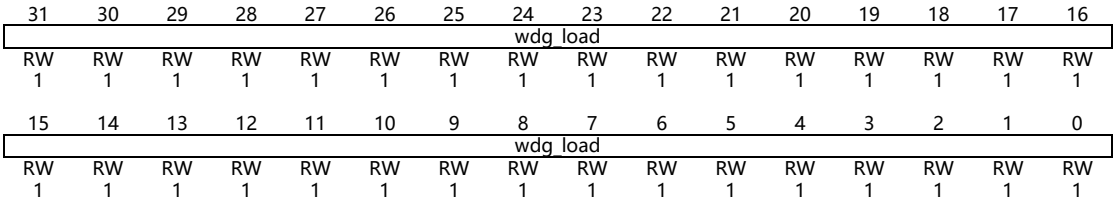
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	WDG_LOAD	0x0	32	RW	0xFFFFFFFF	WatchDog 计数初值寄存器
2	WDG_VALUE	0x4	32	RO	0xFFFFFFFF	WatchDog 当前值寄存器
3	WDG_CONTROL	0x8	32	RW	0x00000000	WatchDog 控制寄存器
4	WDG_INTCLR	0xC	32	WO	0x00000000	WatchDog 中断清除寄存器
5	WDG_RIS	0x10	32	RO	0x00000000	WatchDog 原始中断寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
6	WDG_MIS	0x14	32	RO	0x00000000	WatchDog 屏蔽后中断寄存器
7	WDG_LOCK	0x100	32	RW	0x00000000	WatchDog LOCK 寄存器

18.6 IWDG 寄存器描述

18.6.1 WDG_LOAD

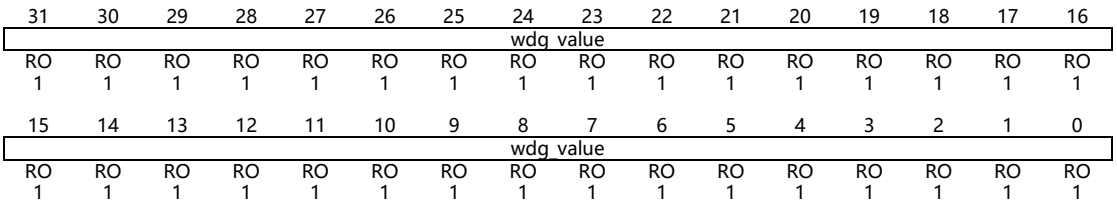
◆ **偏移地址：0x0　　复位值：0xFFFFFFFF**



Name	Bits	R/W	Description	Default
wdg_load	[31:0]	RW	计数初值。当 WatchDog 正在运行时向此寄存器写入数据，计数器马上从新值开始重新计数。 注意：WatchDog 加载此寄存器的值还可以通过： 1、计数值递减到 0； 2、重新使能 WDG_CONTROL.int_en 信号； 3、写入 WDG_INTCLR 清除中断。	0xFFFFFFFF

18.6.2 WDG_VALUE

◆ **偏移地址：0x4　　复位值：0xFFFFFFFF**



Name	Bits	R/W	Description	Default
wdg_value	[31:0]	RO	WatchDog 计数器当前值。	0xFFFFFFFF

18.6.3 WDG_CONTROL

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														res_en	int_en
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
res_en	[1]	RW	WatchDog 复位信号输出使能。 0：禁止； 1：使能。	0x0
int_en	[0]	RW	WatchDog 中断信号输出使能。 0：计数器停止计数，计数值保持当前值不变，WatchDog 被关闭； 1：既启动计数器又使能中断， WatchDog 被启动。	0x0

18.6.4 WDG_INTCLR

◆ **偏移地址：0xC　　复位值：0x00000000**

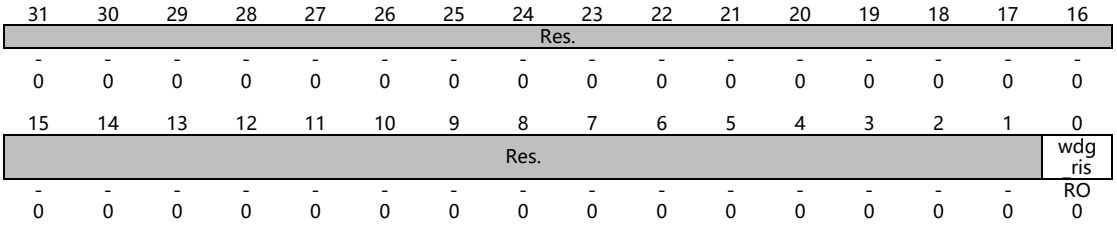
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
wdg_intclr															
WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
wdg_intclr															
WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO	WO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
wdg_intclr	[31:0]	WO	对该寄存器写入任意值均可清除 WatchDog 的中断，并使 WatchDog 从寄存器 WDG_LOAD 中重新载入初值计数。	0x0

18.6.5 WDG_RIS

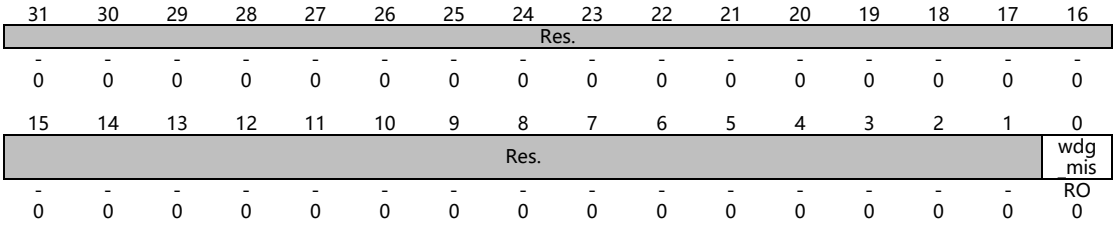
◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
wdg_ris	[0]	RO	WatchDog 原始中断状态，当计数器的计数值递减到 0 时，该位置 1。 0：未产生中断； 1：已产生中断。	0x0

18.6.6 WDG_MIS

◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
wdg_mis	[0]	RO	WatchDog 屏蔽后的中断状态。 0：未产生中断或者中断被屏蔽； 1：已产生中断。	0x0

18.6.7 WDG_LOCK

◆ **偏移地址：0x100　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
wdg_lock															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
wdg_lock															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
wdg_lock	[31:0]	RW	向该寄存器写入 0x1ACC_E551，可打开所有寄存器的写权限；向该寄存器写入其他值则关闭写权限。 读该寄存器返回加锁的状态而不是写入该寄存器的值： 0：允许写访问（未加锁）； 1：禁止写访问（已加锁）。	0x0

19 键盘扫描模块（KEYSCAN)

19.1 KEYSCAN 简介

AiM5101HXX 芯片包含一个键盘扫描模块(KEYSCAN)。KEYSCAN 模块支持外部键盘阵列(行列)的扫描功能，列是由独立的扫描输出 KEY_OUT(0~7)驱动，行 KEY_IN(0~15)是作为中断输入被检测。本模块是通过列扫描，行查询的方法实现按键识别功能。

19.2 KEYSCAN 主要特性

- 最大支持 8 列*16 行的扫描规格
- 列输出 KEY_OUT 个数通过寄存器配置,灵活映射
- 间隔一定时间依次输出低电平来扫描键盘阵列
- KEY_IN 可通过寄存器独立选择行输入
- 扫描时间可配置
- 中断上报后停止扫描，软件可根据列扫描状态和行输入中断定位按下的键
- 重新启动扫描后，列扫描在上次停止列的下一列开始扫描
- 时钟来源可选择
- 按键输入消抖
- 支持同一列多按键识别

19.3 KEYSCAN 功能说明

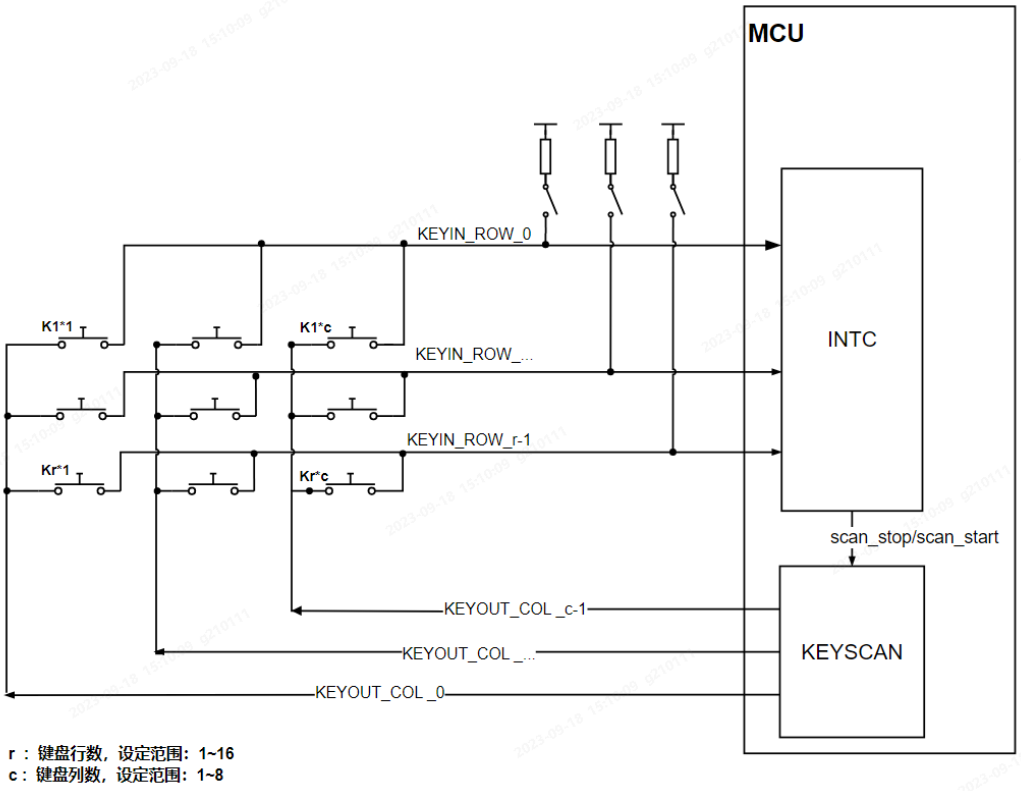
19.3.1 概述

KEYSCAN 模块包含扫描列输出的控制逻辑、行输入按键的去抖等功能，去抖后信号通过

中断线连接到 CPU，CPU 通过中断寄存器和行输出状态寄存器确定按键的位置。

KEYSCAN 的概述图如下图 19-1 框图所示。

图 19-1 KEYSCAN 概述图



KEYSCAN 模块依据软件配置输出各个扫描列 KEYOUT_COL(对应 KEY_OUT)，外接外部键盘的列端，键盘为 r*c 的矩阵键盘，矩阵键盘的行输入到芯片内部，去抖后产生中断汇报到软件。

列输出依据用户配置的个数，不断顺序扫描各列，某列拉低认为此列正在扫描，直到某按键被按下，行信号拉低，通过中断传输到软件，软件端通过此时的列扫描状态，输入的行中断共同确认按下键的位置；同时，按键中断被检测后，KEYSCAN 会停止列的扫描，直到软件识别到对应按键后，清除中断，列扫描将继续开始。

KEYSCAN 的输入端需要上拉操作，用户可在芯片的外部电路实现上拉，也可以使用芯片内部 I/O MUX 配置上拉。

KEYSCAN 扫描输出依赖上拉，用户可在芯片外部实现上拉，也可以使用芯片内部 I/O MUX 配置上拉。

19.3.2 按键识别功能

某按键被按下，键盘的行和列短接，行产生下降沿，从而产生中断标志，软件端通过此时的列扫描状态 SCAN_STAT，输入的行中断 KEYPRESS_STAT 共同确认按下键的位置；同时，按键中断被检测后，KEYSCAN 会停止列的扫描，直到软件识别到对应按键后，配置清除中断，列扫描将继续开始。

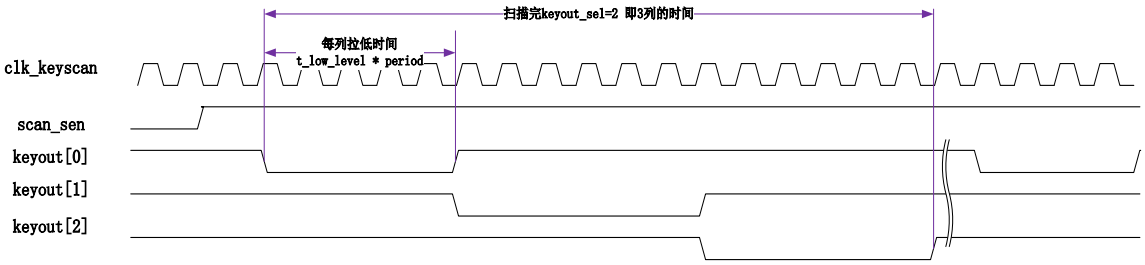
KEYIN_ROW 可以从行输入选择寄存器 SCAN_CTRL.keyin_sel 中独立选择，并可以通过消抖周期配置寄存器 DEBOUNCE_NUM 配置输入行按键的消抖时间，通过列 channel 输出选择寄存器 SCAN_CTRL.col_chan_sel 选择要输出的列 channel，通过列通道使能寄存器 KEYOUT_SEL 选择上述列 channel 的输出选择，从而实现输出列对管脚的灵活映射，最大可以支持 16 行*8 列的键盘阵列。

19.3.3 按键扫描功能

键盘扫描的处理方式为：循环将键盘阵列的列输出低电平，当有键按下时，会产生对应的行输入中断。

当扫描使能开关寄存器 SCAN_EN 置 1 后，第 0 列输出低电平，第 1 列~第 n 列为高阻 HIZ，经过列扫描时间寄存器 T_LOW_LEVEL 设定的时间之后，第 1 列输出为低电平，其余列输出管脚为高阻 HIZ，依此类似循环。当有键按下并去抖产生行中断后，键盘扫描功能停止，对应的中断标志清零后，扫描会自动重启，并在上次扫描列的下一列开始循环。

图 19-2 KEYSCAN 扫描功能示意图



19.3.4 注意事项

本模块驱动键盘列，通过输出列低电平实现各个列的扫描；而键盘行检测则由去抖模块和中断控制模块功能实现，需要行信号的下降沿检测，并配置合适的去抖时间。

列选择配置需要列 channel 输出选择寄存器 SCAN_CTRL.col_chan_sel 和列通道使能寄存器 KEYOUT_SEL 共同配置确认。

本模块扫描输出依赖上拉，用户可选择外部电路上拉或者 I/O MUX 配置上拉。

本模块要求 KEYSCAN 所有寄存器只能在扫描使能开关寄存器 SCAN_EN=0 时修改。

本模块存在多个时钟源，默认为低速振荡器 32KHz，默认时钟为 32KHz; 如果选择其他时钟源，注意 KEYSCAN 允许修改工作时钟，选择较大时钟时，要注意列扫描时间寄存器 T_LOW_LEVEL 和消抖周期配置寄存器 DEBOUNCE_NUM 的对应修改。

为实现最优的按键检测效果，需要设定合适的去抖时间和扫描时间。

使用行中断检测时，可使用中断状态寄存器 KEYPRESS_STAT 或者中断原始状态寄存器 KEYPRESS_RAW，使用中断状态寄存器 KEYPRESS_STAT 需要通过中断使能寄存器 KEYPRESS_IER 使能。

本模块检测到按键中断后会停止扫描，在软件中断标志清零后会立即开始扫描，因用户按键时长的不确定性，为规避同一按键的重复识别，用户可选用适当的扫描时间或适当的延迟时间（检测按键中断后到清除中断的时间）。

如果使用芯片内部上拉电阻，则请参考管脚复用（I/O MUX)。

19.4 KEYSCAN 寄存器概览

KEYSCAN 的基地址如下：

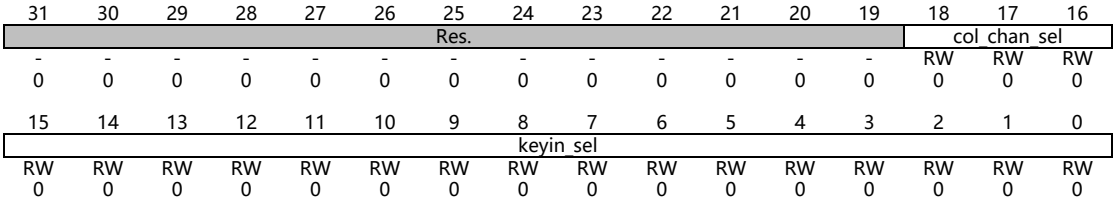
- KEYSCAN 寄存器基地址为：0x60082000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	SCAN_CTRL	0x0	32	RW	0x00000000	按键配置寄存器
2	DEBOUNCE_NUM	0x4	32	RW	0x00000040	按键去抖周期配置寄存器
3	T_LOW_LEVEL	0x8	32	RW	0x00000A00	按键扫描周期配置
4	SCAN_EN	0xC	32	RW	0x00000000	按键使能寄存器
5	SCAN_STAT	0x10	32	RO	0x00000000	按键状态寄存器
6	KEYPRESS_IER	0x18	32	RW	0x00000000	中断使能寄存器
7	KEYPRESS_CLR	0x1C	32	WIP	0x00000000	中断清除寄存器
8	KEYPRESS_STAT	0x20	32	RO	0x00000000	中断状态寄存器
9	KEYPRESS_RAW	0x24	32	RO	0x00000000	中断原始状态寄存器
10	KEYOUT_SEL	0x28	32	RW	0x00000000	列选择寄存器

19.5 KEYSCAN 寄存器描述

19.5.1 SCAN_CTRL

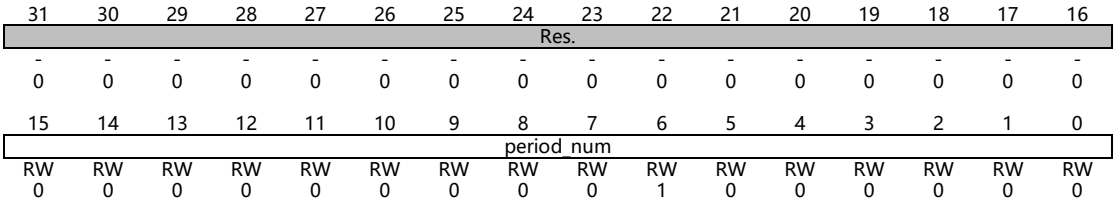
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留	0x0
col_chan_sel	[18:16]	RW	列 channel 输出个数选择 设定值 ：输出选择 0 : channel0 1 : channel0~channel1 2 : channel0~channel2 3 : channel0~channel3 4 : channel0~channel4 5 : channel0~channel5 6 : channel0~channel6 7 : channel0~channel7 注意：此寄存器禁止在 scan_sen=1 时改变，只可以在 scan_sen=0 时修改	0x0
keyin_sel	[15:0]	RW	行输入选择位，选择的行作为键盘阵列的行使用，并会作为中断被检测 keyin_sel[n]=0 的话，keyin[n]不用作为键盘阵列行使用 keyin_sel[n]=1 的话，keyin[n]用作为键盘阵列行使用 n:范围 0~15 注意：此寄存器禁止在 scan_sen=1 时改变，只可以在 scan_sen=0 时修改	0x0

19.5.2 DEBOUNCE_NUM

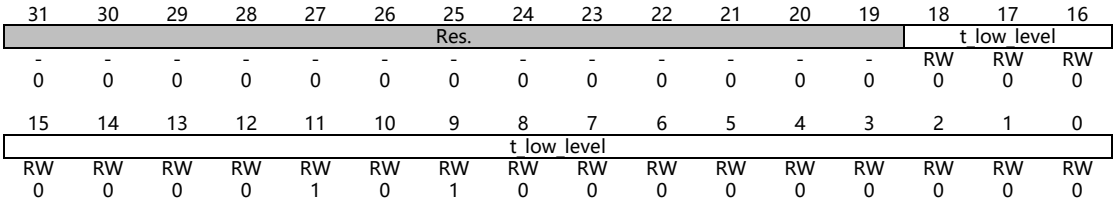
◆ **偏移地址：0x4　　复位值：0x00000040**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
period_num	[15:0]	RW	此寄存器用作配置去抖时对应的单位时钟周期个数； 按键消抖模块使用 10 个单位时间(period_num*时钟周期)来进行按键消抖； 由于模块时钟可变，此寄存器配置值乘以模块时钟周期后为单位时间，而按键消抖会使用 10 个单位时间进行消抖； 默认时钟 32KHz，默认周期数 0x40，即对应的单位时间为 2ms，而对应的消抖时间是 20ms； 注意：16'h0000 设定禁止，此寄存器只能在 scan_sen=0 时设定有效	0x0040

19.5.3 T_LOW_LEVEL

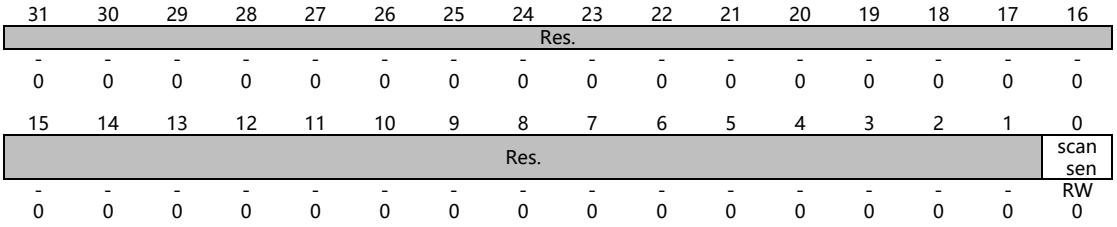
◆ **偏移地址：0x8　　复位值：0x00000A00**



Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留	0x0
t_low_level	[18:0]	RW	单列扫描输出低电平的时间(扫描时钟数)： 输出的低电平时间=(t_low_level)*时钟周期 注意：此寄存器只能在 scan_sen=0 时设定有效，且 19'h0000h 和 19'h0001 设定禁止,最大值可以设定值为 19'h7FFFE 单列扫描时间 t_low_level 不小于 DEBOUNCE_NUM. period_num 寄存器值*11 所对应时间	0x00A00

19.5.4 SCAN_EN

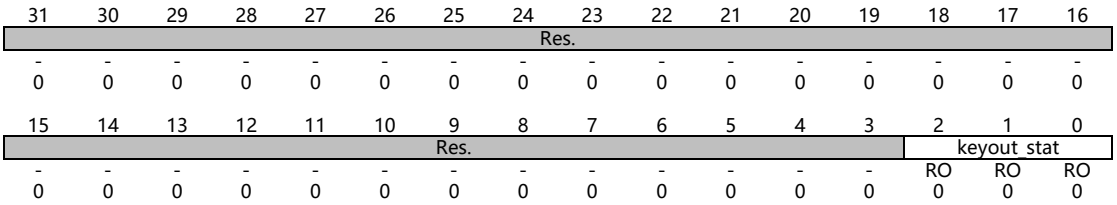
◆ **偏移地址：0xC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
scan_sen	[0]	RW	扫描使能 0：扫描禁止 1：扫描使能 模块内部运行依赖于此扫描使能寄存器	0x0

19.5.5 SCAN_STAT

◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
keyout_stat	[2:0]	RO	当前正在扫描的列管脚索引位 0 :当前工作的 scan 管脚为 KEY_OUT0 1 :当前工作的 scan 管脚为 KEY_OUT1 2 :当前工作的 scan 管脚为 KEY_OUT2 3 :当前工作的 scan 管脚为 KEY_OUT3 4 :当前工作的 scan 管脚为 KEY_OUT4 5 :当前工作的 scan 管脚为 KEY_OUT5 6 :当前工作的 scan 管脚为 KEY_OUT6 7 :当前工作的 scan 管脚为 KEY_OUT7	0x0

19.5.6 KEYPRESS_IER

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
keypress15_ien	keypress14_ien	keypress13_ien	keypress12_ien	keypress11_ien	keypress10_ien	keypress9_ien	keypress8_ien	keypress7_ien	keypress6_ien	keypress5_ien	keypress4_ien	keypress3_ien	keypress2_ien	keypress1_ien	keypress0_ien
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
keypress15_ien	[15]	RW	第 15 行按键按下中断使能 0：禁用第 15 行扫描输入中断 1：启用第 15 行扫描输入中断 注意： 仅当 keypress15_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress14_ien	[14]	RW	第 14 行按键按下中断使能 0：禁用第 14 行扫描输入中断 1：启用第 14 行扫描输入中断 注意： 仅当 keypress14_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress13_ien	[13]	RW	第 13 行按键按下中断使能 0：禁用第 13 行扫描输入中断 1：启用第 13 行扫描输入中断 注意： 仅当 keypress13_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress12_ien	[12]	RW	第 12 行按键按下中断使能 0：禁用第 12 行扫描输入中断 1：启用第 12 行扫描输入中断 注意： 仅当 keypress12_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress11_ien	[11]	RW	第 11 行按键按下中断使能 0：禁用第 11 行扫描输入中断 1：启用第 11 行扫描输入中断 注意： 仅当 keypress11_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress10_ien	[10]	RW	第 10 行按键按下中断使能 0：禁用第 10 行扫描输入中断 1：启用第 10 行扫描输入中断	0x0

Name	Bits	R/W	Description	Default
			注意： 仅当 keypress10_stat 清零时，才允许通过软件对此位执行写操作。	
keypress9_ien	[9]	RW	第 9 行按键按下中断使能 0：禁用第 9 行扫描输入中断 1：启用第 9 行扫描输入中断 注意： 仅当 keypress9_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress8_ien	[8]	RW	第 8 行按键按下中断使能 0：禁用第 8 行扫描输入中断 1：启用第 8 行扫描输入中断 注意： 仅当 keypress8_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress7_ien	[7]	RW	第 7 行按键按下中断使能 0：禁用第 7 行扫描输入中断 1：启用第 7 行扫描输入中断 注意： 仅当 keypress7_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress6_ien	[6]	RW	第 6 行按键按下中断使能 0：禁用第 6 行扫描输入中断 1：启用第 6 行扫描输入中断 注意： 仅当 keypress6_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress5_ien	[5]	RW	第 5 行按键按下中断使能 0：禁用第 5 行扫描输入中断 1：启用第 5 行扫描输入中断 注意： 仅当 keypress5_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress4_ien	[4]	RW	第 4 行按键按下中断使能 0：禁用第 4 行扫描输入中断 1：启用第 4 行扫描输入中断 注意： 仅当 keypress4_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress3_ien	[3]	RW	第 3 行按键按下中断使能 0：禁用第 3 行扫描输入中断 1：启用第 3 行扫描输入中断 注意： 仅当 keypress3_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress2_ien	[2]	RW	第 2 行按键按下中断使能 0：禁用第 2 行扫描输入中断 1：启用第 2 行扫描输入中断	0x0

Name	Bits	R/W	Description	Default
			注意： 仅当 keypress2_stat 清零时，才允许通过软件对此位执行写操作。	
keypress1_ien	[1]	RW	第 1 行按键按下中断使能 0：禁用第 1 行扫描输入中断 1：启用第 1 行扫描输入中断 注意： 仅当 keypress1_stat 清零时，才允许通过软件对此位执行写操作。	0x0
keypress0_ien	[0]	RW	第 0 行按键按下中断使能 0：禁用第 0 行扫描输入中断 1：启用第 0 行扫描输入中断 注意： 仅当 keypress0_stat 清零时，才允许通过软件对此位执行写操作。	0x0

19.5.7 KEYPRESS_CLR

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
keypress15_iclr	keypress14_iclr	keypress13_iclr	keypress12_iclr	keypress11_iclr	keypress10_iclr	keypress9_iclr	keypress8_iclr	keypress7_iclr	keypress6_iclr	keypress5_iclr	keypress4_iclr	keypress3_iclr	keypress2_iclr	keypress1_iclr	keypress0_iclr
W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
keypress15_iclr	[15]	W1P	第 15 行按键按下状态中断清除 写 1 清	0x0
keypress14_iclr	[14]	W1P	第 14 行按键按下状态中断清除 写 1 清	0x0
keypress13_iclr	[13]	W1P	第 13 行按键按下状态中断清除 写 1 清	0x0
keypress12_iclr	[12]	W1P	第 12 行按键按下状态中断清除 写 1 清	0x0
keypress11_iclr	[11]	W1P	第 11 行按键按下状态中断清除 写 1 清	0x0
keypress10_iclr	[10]	W1P	第 10 行按键按下状态中断清除 写 1 清	0x0
keypress9_iclr	[9]	W1P	第 9 行按键按下状态中断清除 写 1 清	0x0

Name	Bits	R/W	Description	Default
keypress8_icl r	[8]	W1P	第 8 行按键按下状态中断清除 写 1 清	0x0
keypress7_icl r	[7]	W1P	第 7 行按键按下状态中断清除 写 1 清	0x0
keypress6_icl r	[6]	W1P	第 6 行按键按下状态中断清除 写 1 清	0x0
keypress5_icl r	[5]	W1P	第 5 行按键按下状态中断清除 写 1 清	0x0
keypress4_icl r	[4]	W1P	第 4 行按键按下状态中断清除 写 1 清	0x0
keypress3_icl r	[3]	W1P	第 3 行按键按下状态中断清除 写 1 清	0x0
keypress2_icl r	[2]	W1P	第 2 行按键按下状态中断清除 写 1 清	0x0
keypress1_icl r	[1]	W1P	第 1 行按键按下状态中断清除 写 1 清	0x0
keypress0_icl r	[0]	W1P	第 0 行按键按下状态中断清除 写 1 清	0x0

19.5.8 KEYPRESS_STAT

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
keypress15_stat	keypress14_stat	keypress13_stat	keypress12_stat	keypress11_stat	keypress10_stat	keypress9_stat	keypress8_stat	keypress7_stat	keypress6_stat	keypress5_stat	keypress4_stat	keypress3_stat	keypress2_stat	keypress1_stat	keypress0_stat
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
keypress15_stat	[15]	RO	键盘 15 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress15_ien 使能，否则此值持续为 0	0x0
keypress14_stat	[14]	RO	键盘第 14 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断	0x0

Name	Bits	R/W	Description	Default
			依赖于 keypress14_ien 使能，否则此值持续为 0	
keypress13_stat	[13]	RO	键盘第 13 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress13_ien 使能，否则此值持续为 0	0x0
keypress12_stat	[12]	RO	键盘第 12 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress12_ien 使能，否则此值持续为 0	0x0
keypress11_stat	[11]	RO	键盘第 11 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress11_ien 使能，否则此值持续为 0	0x0
keypress10_stat	[10]	RO	键盘第 10 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress10_ien 使能，否则此值持续为 0	0x0
keypress9_stat	[9]	RO	键盘第 9 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress9_ien 使能，否则此值持续为 0	0x0
keypress8_stat	[8]	RO	键盘第 8 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress8_ien 使能，否则此值持续为 0	0x0
keypress7_stat	[7]	RO	键盘第 7 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress7_ien 使能，否则此值持续为 0	0x0
keypress6_stat	[6]	RO	键盘第 6 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress6_ien 使能，否则此值持续为 0	0x0
keypress5_stat	[5]	RO	键盘第 5 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress5_ien 使能，否则此值持续为 0	0x0
keypress4_stat	[4]	RO	键盘第 4 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress4_ien 使能，否则此值持续为 0	0x0

Name	Bits	R/W	Description	Default
keypress3_stat	[3]	RO	键盘第 3 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress3_ien 使能，否则此值持续为 0	0x0
keypress2_stat	[2]	RO	键盘第 2 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress2_ien 使能，否则此值持续为 0	0x0
keypress1_stat	[1]	RO	键盘第 1 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress1_ien 使能，否则此值持续为 0	0x0
keypress0_stat	[0]	RO	键盘第 0 行按键按下中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 依赖于 keypress0_ien 使能，否则此值持续为 0	0x0

19.5.9 KEYPRESS_RAW

◆ **偏移地址：0x24　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
keypress15_raw	keypress14_raw	keypress13_raw	keypress12_raw	keypress11_raw	keypress10_raw	keypress9_raw	keypress8_raw	keypress7_raw	keypress6_raw	keypress5_raw	keypress4_raw	keypress3_raw	keypress2_raw	keypress1_raw	keypress0_raw
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
keypress15_raw	[15]	RO	键盘第 15 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress15_stat 区别在于不依赖于 keypress15_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress14_raw	[14]	RO	键盘第 14 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断	0x0

Name	Bits	R/W	Description	Default
			此列与 keypress14_stat 区别在于不依赖于 keypress14_ien 使能，存在行输入即会拉起此状态寄存器	
keypress13_raw	[13]	RO	键盘第 13 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress13_stat 区别在于不依赖于 keypress13_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress12_raw	[12]	RO	键盘第 12 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress12_stat 区别在于不依赖于 keypress12_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress11_raw	[11]	RO	键盘第 11 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress11_stat 区别在于不依赖于 keypress11_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress10_raw	[10]	RO	键盘第 10 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress10_stat 区别在于不依赖于 keypress10_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress9_raw	[9]	RO	键盘第 9 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress9_stat 区别在于不依赖于 keypress9_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress8_raw	[8]	RO	键盘第 8 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress8_stat 区别在于不依赖于 keypress8_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress7_raw	[7]	RO	键盘第 7 行输入原始中断状态寄存 0：未有键盘按下中断	0x0

Name	Bits	R/W	Description	Default
			1：存在键盘按下中断 此列与 keypress7_stat 区别在于不依赖于 keypress7_ien 使能，存在行输入即会拉起此状态寄存器	
keypress6_raw	[6]	RO	键盘第 6 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress6_stat 区别在于不依赖于 keypress6_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress5_raw	[5]	RO	键盘第 5 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress5_stat 区别在于不依赖于 keypress5_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress4_raw	[4]	RO	键盘第 4 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress4_stat 区别在于不依赖于 keypress4_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress3_raw	[3]	RO	键盘第 3 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress3_stat 区别在于不依赖于 keypress3_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress2_raw	[2]	RO	键盘第 2 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress2_stat 区别在于不依赖于 keypress2_ien 使能，存在行输入即会拉起此状态寄存器	0x0
keypress1_raw	[1]	RO	键盘第 1 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress1_stat 区别在于不依赖于 keypress1_ien 使能，存在行输入即会拉起此状态寄存器	0x0

Name	Bits	R/W	Description	Default
keypress0_raw	[0]	RO	键盘第 0 行输入原始中断状态寄存 0：未有键盘按下中断 1：存在键盘按下中断 此列与 keypress0_stat 区别在于不依赖于 keypress0_ien 使能，存在行输入即会拉起此状态寄存器	0x0

19.5.10 KEYOUT_SEL

◆ **偏移地址：0x28　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	keyout_sel7			Res.	keyout_sel6			Res.	keyout_sel5			Res.	keyout_sel4		
-	RW	RW	RW	-	RW	RW	RW	-	RW	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	keyout_sel3			Res.	keyout_sel2			Res.	keyout_sel1			Res.	keyout_sel0		
-	RW	RW	RW	-	RW	RW	RW	-	RW	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31]	-	保留	0x0
keyout_sel7	[30:28]	RW	channel7 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	0x0
reserved	[27]	-	保留	0x0
keyout_sel6	[26:24]	RW	channel6 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能	0x0

Name	Bits	R/W	Description	Default
			7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	
reserved	[23]	-	保留	0x0
keyout_sel5	[22:20]	RW	channel5 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	0x0
reserved	[19]	-	保留	0x0
keyout_sel4	[18:16]	RW	channel4 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	0x0
reserved	[15]	-	保留	0x0
keyout_sel3	[14:12]	RW	channel3 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改	0x0

Name	Bits	R/W	Description	Default
			要求 keyout_selx(x=0~7)设置不同	
reserved	[11]	-	保留	0x0
keyout_sel2	[10:8]	RW	channel2 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	0x0
reserved	[7]	-	保留	0x0
keyout_sel1	[6:4]	RW	channel1 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	0x0
reserved	[3]	-	保留	0x0
keyout_sel0	[2:0]	RW	channel0 输出选择 0：选取 KEY_OUT0 使能 1：选取 KEY_OUT1 使能 2：选取 KEY_OUT2 使能 3：选取 KEY_OUT3 使能 4：选取 KEY_OUT4 使能 5：选取 KEY_OUT5 使能 6：选取 KEY_OUT6 使能 7：选取 KEY_OUT7 使能 注意：此寄存器禁止在 scan_sen=1 时设置，只可以在 scan_sen=0 时修改 要求 keyout_selx(x=0~7)设置不同	0x0

20 触摸传感控制器（TSC）

20.1 TSC 简介

TSC（Touch sense controller）触摸传感控制器用于检测压力传感电路的被触摸状态。该 MCU 芯片集成了 24 路常规检测通道和 1 路 shield 通道。

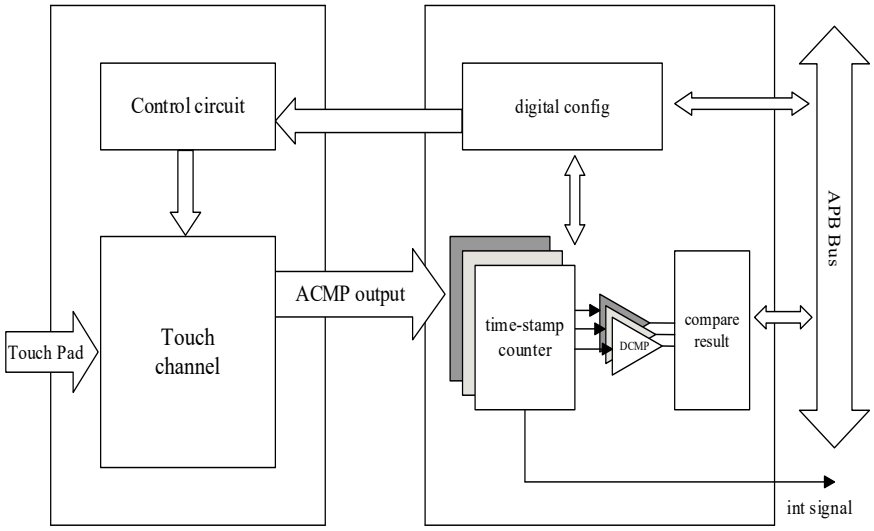
20.2 TSC 主要特性

- 电源电压：2.4~3.6V；
- 最高支持 24 个常规检测通道，每个通道可独立使能；
- 支持两种通道扫描模式：定长轮询、快速轮询；
- 支持自感模式；
- 支持 shield 通道；
- 支持检测门限配置；
- 支持充电超时检测；
- 支持消抖；

20.3 TSC 逻辑实现

TSC 逻辑实现框图如图 20-1 所示。

图 20-1 TSC 逻辑框图



Control circuit 示意控制器对模拟电路的控制逻辑；包括模拟电路电流设置、参考电压设置、模拟电路使能等；通过寄存器 TOUCH_ANA_CTRL0.touch_sel_ib_glb 设置 ibias 全局电流档位；配置寄存器 TOUCH_ANA_CTRL0.touch_sel_ib_gpx（x=0~2）设置对应 touch 通道充电电流；配置寄存器 TOUCH_ANA_CTRL0.touch_sel_cmp 设置当前需使用的模拟比较器 ACMP；模拟比较器的参考电压值档位与参考电压使能通过配置寄存器 TOUCH_ANA_CTRL0.touch_sel_verf 和 TOUCH_ANA_CTRL0.touch_en_verf 设置。

Touch channel 示意本模块检测通道，包括一个 shield 检测通道与 24 个常规检测通道。

time-stamp counter 示意数字电平捕获计数电路，数字电路对模拟比较器输出电平信号进行捕获计数，以判断并上报当前通道捕获计数信息。

DCMP 示意数字比较器，对计数时间戳的计数结果与检测门限进行比较得到当前检测通道检测情况。

compare result 示意数字电路比较结果上报与汇总，数字电路提供各监测通道比较结果查询以及所有检测通道检测结果汇总。

digital config 示意控制器配置信息；配置模拟电路控制信息，配置各通道向量使能、通道检测门限、轮询模式、中断使能等。

说明

模拟电路的 ibias 电流与通道充电电流大小和通道检测速度有关；ibias 电流、通道充电电流越大，通道检测速度越快；另外电流越大，通道抗干扰能力越强。

模拟比较器的参考电压档位与参考电压使能需要在本模块设置，具体模拟比较器参考电压输入源设置需通过模拟比较器章节的寄存器配合使用。

20.4 TSC 连接关系

TSC 内部电路共分成 3 组，每一组控制 8 个通道，当对应的控制信号为 1 时，该通道使能。这里的控制信号就是寄存器 VEC_CFGx.vec_vld。VEC_CFG0-7 为第一组对应通道 0-7，VEC_CFG8-15 为第二组对应通道 8-15，VEC_CFG16-23 为第三组对应通道 16-23。TSC（touch_en）使能后，TSC_SHIELD 通道会默认使能。管脚连接关系如表 20-1 所示。

表 20-1 管脚连接关系

TSC 通道	管脚名	TSC 通道	管脚名
TSC_IO0	PA12	TSC_IO12	PB9
TSC_IO1	PA13	TSC_IO13	PB10
TSC_IO2	PA14	TSC_IO14	PB11
TSC_IO3	PA15	TSC_IO15	PB12
TSC_IO4	PB1	TSC_IO16	PB13
TSC_IO5	PB2	TSC_IO17	PC8
TSC_IO6	PB3	TSC_IO18	PC9
TSC_IO7	PB4	TSC_IO19	PC10
TSC_IO8	PB5	TSC_IO20	PC11
TSC_IO9	PB6	TSC_IO21	PC12
TSC_IO10	PB7	TSC_IO22	PC13
TSC_IO11	PB8	TSC_IO23	PC14
TSC_SHIELD	PC7		

20.5 TSC 功能描述

20.5.1 TSC 通道轮询

TSC 的检测通道包括一个 shield 通道和 24 个常规通道；轮询顺序为顺序轮询，每次轮询从 shield 通道开始，依照通道向量 0-23 完成 24 个常规通道检测；TSC 通道使能寄存器 VEC_CFGx（x 代表通道值）去独立控制 24 个常规通道的开启与关闭，对应使能开启该通道即进行检测，关闭则不进行检测。shield 通道的测试结果可作为防水、防污、抗干扰能力的比较基准。

20.5.2 TSC 轮询模式

TSC 提供两种轮询模式：定长轮询、快速轮询；配置 TSC 寄存器 CCR1.rr_mode 可以设置轮询模式。

通道检测的有效时长阈值通过寄存器 TS_THD 设置。

定长轮询模式，通道检测时长固定，每个通道均检测相同时间；配置 TSC 寄存器 TS_MAX 设置定长轮询模式每个通道检测时长。

快速轮询模式，通道检测时长不固定，通道检测最大时长通过 TSC 寄存器 TS_MAX 设置，检测到有效触摸即结束当前通道进行下一被使能的通道；快速轮询模式支持检测超时机制，当前通道检测达到最大时长仍没有有效触摸即上报通道检测超时中断，通过查询寄存器 INT_STAT.int_charge_ot_stat 获取，也可查询寄存器 ERR_REC.ot_vec 实时获取超时通道信息。

TSC 提供多次轮询之间插入等待时间，通过配置寄存器 CCR1.rr_wait 设置；当软件完成一轮检测结果查询后结束等待。

20.5.3 TSC 消抖

TSC 提供检测比较结果直接上报，同时提供了消抖处理；TSC 会将检测比较结果存入寄存器 VEC_REC.cmp_result，软件可直接查询对应通道检测结果；同时允许串行记录该通道多次轮询检测结果，软件可以通过读取连续轮询的结果以达到消抖作用，得到更准确的检测结果；除此之外 TSC 也提供 24 个通道检测比较结果汇总上报，软件可查询寄存器 NORM_REC 获取本轮检测情况。

TSC 也提供通道检测计数结果直接上报，该计数为当前通道捕获计数值，软件可通过查询寄存器 VEC_REC.time_stamp，自行与检测门限比较得到当前通道触摸情况。

20.5.4 TSC 中断

TSC 提供快速模式下通道超时检测中断、轮询结束中断、shield 通道检测完成中断、常规通道检测完成中断；软件可通过查询上述中断得到当前运行状态及结果等信息。

⚠注意

- 如果轮询间不插入等待，那么新的轮询结果会覆盖旧的轮询结果。

20.6 TSC 数模控制

TSC 提供触控电路电流档位设置、触控电路充电电流设置、触控电路参考电压档位设置、参考电压源设置，这些参数可通过配置寄存器 TOUCH_ANA_CTRL0 设置。图 20-2 与图 20-3 展示了触控电路检测触摸的原理与时序。

图 20-2 定长检测原理

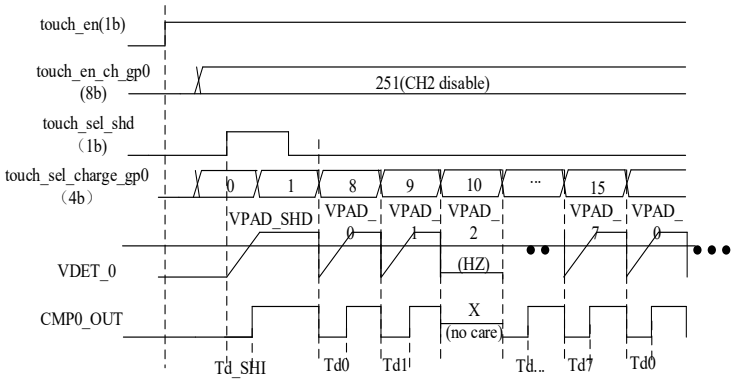
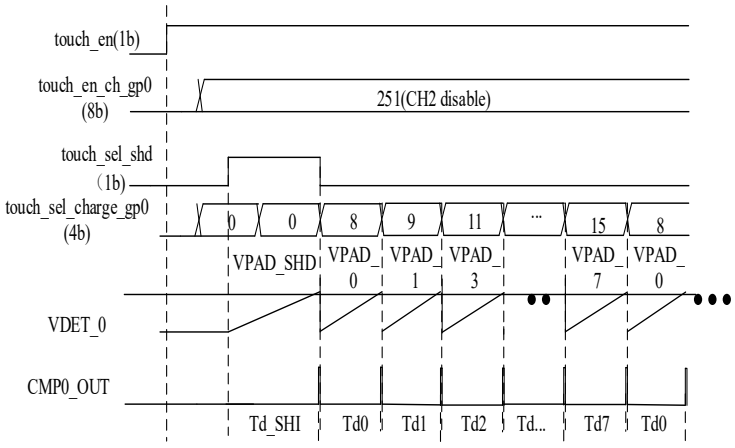


图 20-3 快速检测原理



20.7 TSC 配置流程

步骤 1 查询触控使能，寄存器 TOUCH_ANA_CTRL0.touch_en；若为 0，则表明当前触控

电路空闲，若为 1，表明正在进行检测；

步骤 2 若为空闲，写中断寄存器 INT_CLR，清除所有中断；

步骤 3 配置寄存器 TOUCH_ANA_CTRL0，设置触控电路；

步骤 4 配置寄存器 INT_EN，打开中断使能；

步骤 5 配置寄存器 VEC_CFG，设置检测通道使能；

步骤 6 配置寄存器 TOUCH_ANA_CTRL0.touch_en 为 1 进行检测，等待触控监测结果；

20.8 TSC 寄存器概览

TSC 的寄存器基地址如下：

- TSC 寄存器基地址为：0x61006800

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	TOUCH_ANA_CTRL0	0x0	32	RW	0x00000000	模拟控制寄存器
2	TS_CNT0	0x4	32	RO	0x00000000	通道检测计时寄存器
3	TS_MAX	0x10	32	RW	0x000FFFFF	周期寄存器
4	TS_THD	0x14	32	RW	0x000FFFFF	定长轮询检测阈值寄存器
5	CCR0	0x18	32	W1P	0x00000000	控制寄存器 0
6	CCR1	0x1C	32	RW	0x00000000	控制寄存器 1
7	SHD_REC	0x20	32	RO	0x00000000	Shield 通道计时寄存器
8	ERR_REC	0x24	32	RO	0x00000000	错误状态寄存器
9	NORM_REC	0x28	32	RO	0x00000000	检测比较结果汇总寄存器
10	INT_EN	0x30	32	RW	0x00000000	中断使能寄存器
11	INT_RAW	0x34	32	RO	0x00000000	原始中断寄存器
12	INT_STAT	0x38	32	RO	0x00000000	中断状态寄存器
13	INT_CLR	0x3C	32	W1P	0x00000000	中断清除寄存器
14	INT_INJ	0x40	32	W1P	0x00000000	中断注入寄存器
15	VEC_CFG	0x100+0x4*j	32	RW	0x00010000	通道使能寄存器（j 代表通道，取值 0~23）
16	VEC_REC	0x200+0x4*j	32	RO	0x00000000	通道结果缓存寄存器（j 代表通道，取值 0~23）

20.9 TSC 寄存器描述

20.9.1 TOUCH_ANA_CTRL0

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
touch_sel_ib_glb		touch_sel_ib_gp2		touch_sel_ib_gp1		touch_sel_ib_gp0		touch_sel_cmp		touch_sel_vref			Res.	touch_en	touch_en_vref
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
touch_sel_ib_glb	[15:14]	RW	touch ibias 全局电流档位控制信号： 0x0：输出电流为 1 倍 0x1：输出电流为 0.5 倍 0x2：输出电流为 2 倍 0x3：输出电流为 1 倍	0x0
touch_sel_ib_gp2	[13:12]	RW	touch group2 电路的充电电流大小控制信号。在 1 倍输出电流模式下的输出电流为： 0x0：输出 10uA 0x1：输出 20uA 0x2：输出 30uA 0x3：输出 40uA	0x0
touch_sel_ib_gp1	[11:10]	RW	touch group1 电路的充电电流大小控制信号。在 1 倍输出电流模式下的输出电流为： 0x0：输出 10uA 0x1：输出 20uA 0x2：输出 30uA 0x3：输出 40uA	0x0
touch_sel_ib_gp0	[9:8]	RW	touch group0 电路的充电电流大小控制信号。在 1 倍输出电流模式下的输出电流为： 0x0：输出 10uA 0x1：输出 20uA 0x2：输出 30uA 0x3：输出 40uA	0x0
touch_sel_cmp	[7:6]	RW	touch 的待检测电压连接到 ACMP 的选择控制信号： 0：reserved；	0x0

Name	Bits	R/W	Description	Default
			1：group0、group1、group2 都使用 ACMP0 检测； 2：group0、group1、group2 都使用 ACMP1 检测； 3：group0、group1、group2 都使用 ACMP2 检测；	
touch_sel_vref	[5:3]	RW	模拟比较器的参考电压值选择控制信号： 0：VREF=1/9*VDDA； 1：VREF=2/9*VDDA； 2：VREF=3/9*VDDA； 3：VREF=4/9*VDDA； 4：VREF=5/9*VDDA； 5：VREF=6/9*VDDA； 6：VREF=7/9*VDDA； 7：VREF=8/9*VDDA；	0x0
reserved	[2]	-	保留。	0x0
touch_en	[1]	RW	touch 数字控制器总使能控制信号： 0：不使能； 1：使能。	0x0
touch_en_vref	[0]	RW	模拟比较器的参考电压使能控制信号，作为 ACMP 的备选参考电压： 0：不使能； 1：使能。	0x0

20.9.2 TS_CNT0

◆ **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.								ts_cnt0							
-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ts_cnt0															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
ts_cnt0	[23:0]	RO	时间戳计数器（供调试读取 touch_sel_cmp 时的对应比较输出信号捕获的时间）。	0x0

20.9.3 TS_MAX

◆ **偏移地址：0x10　　复位值：0x000FFFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.								ts_max							
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ts_max															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
ts_max	[23:0]	RW	快速轮询模式下的检测超时计数值。 定长轮询模式下的检测周期计数值。	0xFFFFF

20.9.4 TS_THD

◆ **偏移地址：0x14　　复位值：0x000FFFFF**

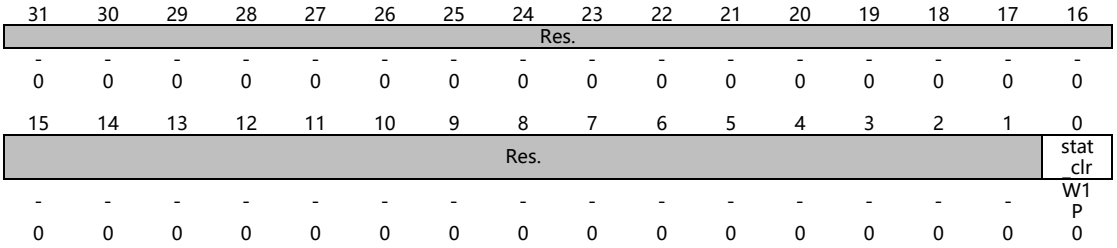
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.								ts_thd							
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ts_thd															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
ts_thd	[23:0]	RW	定长轮询模式下触摸有效的计数阈值。	0xFFFFF

20.9.5 CCR0

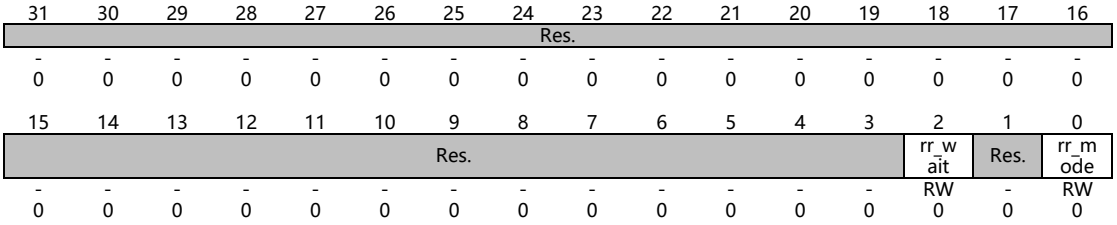
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
stat_clr	[0]	W1P	清除工作状态。所有计时器，序列器，结果标记等将清除，配置信息会保留。	0x0

20.9.6 CCR1

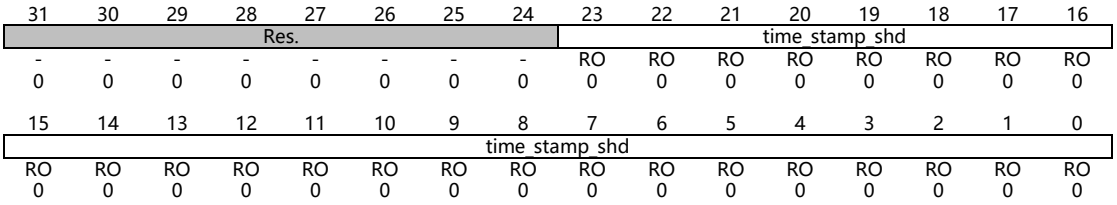
◆ **偏移地址：0x1C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
rr_wait	[2]	RW	选择是否在轮询轮次之间插入自动等待： 0：不插入，直接进行后续轮询； 1：插入自动等待，等待软件完成读取再自动开始后续轮询。	0x0
reserved	[1]	-	保留。	0x0
rr_mode	[0]	RW	轮询模式选择： 0：定长模式； 1：快速模式。	0x0

20.9.7 SHD_REC

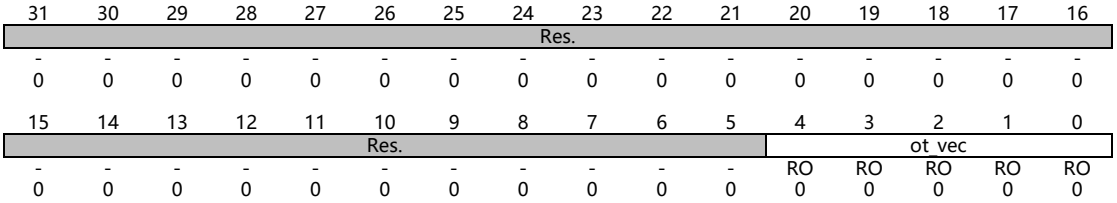
◆ **偏移地址：0x20　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
time_stamp_shd	[23:0]	RO	shield 通道时间戳捕获结果。	0x0

20.9.8 ERR_REC

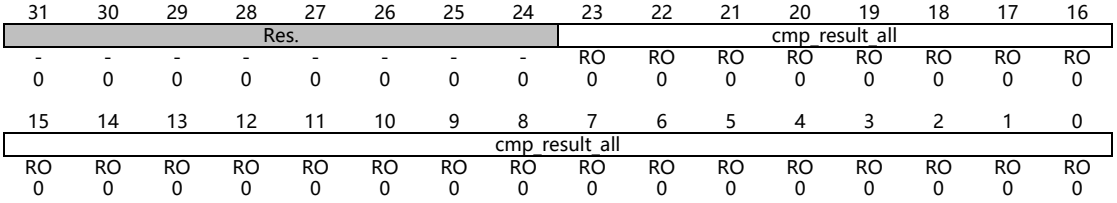
◆ **偏移地址：0x24　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
ot_vec	[4:0]	RO	当前检测超时通道指示（实时刷新）。	0x0

20.9.9 NORM_REC

◆ **偏移地址：0x28　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
cmp_result_all	[23:0]	RO	全部通道检测比较结果（每一 bit 对应一个通道，如：0bit 对应 0 通道）。	0x0

20.9.10 INT_EN

◆ **偏移地址：0x30　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					int_charge_ot_en	int_rrfin_en	int_shd_sence_en	int_vec23_sence_en	int_vec22_sence_en	int_vec21_sence_en	int_vec20_sence_en	int_vec19_sence_en	int_vec18_sence_en	int_vec17_sence_en	int_vec16_sence_en
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
int_vec15_sence_en	int_vec14_sence_en	int_vec13_sence_en	int_vec12_sence_en	int_vec11_sence_en	int_vec10_sence_en	int_vec9_sence_en	int_vec8_sence_en	int_vec7_sence_en	int_vec6_sence_en	int_vec5_sence_en	int_vec4_sence_en	int_vec3_sence_en	int_vec2_sence_en	int_vec1_sence_en	int_vec0_sence_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
int_charge_ot_en	[26]	RW	快速模式检测超时中断使能。 0：不使能； 1：使能。	0x0
int_rrfin_en	[25]	RW	轮询完成中断使能。 0：不使能； 1：使能。	0x0
int_shd_sence_en	[24]	RW	shield 通道检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec23_sence_en	[23]	RW	通道 23 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec22_sence_en	[22]	RW	通道 22 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec21_sence_en	[21]	RW	通道 21 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec20_sence_en	[20]	RW	通道 20 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec19_sence_en	[19]	RW	通道 19 检测完成中断使能。 0：不使能； 1：使能。	0x0

Name	Bits	R/W	Description	Default
int_vec18_sence_en	[18]	RW	通道 18 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec17_sence_en	[17]	RW	通道 17 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec16_sence_en	[16]	RW	通道 16 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec15_sence_en	[15]	RW	通道 15 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec14_sence_en	[14]	RW	通道 14 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec13_sence_en	[13]	RW	通道 13 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec12_sence_en	[12]	RW	通道 12 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec11_sence_en	[11]	RW	通道 11 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec10_sence_en	[10]	RW	通道 10 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec9_sence_en	[9]	RW	通道 9 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec8_sence_en	[8]	RW	通道 8 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec7_sence_en	[7]	RW	通道 7 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec6_sence_en	[6]	RW	通道 6 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec5_sence_en	[5]	RW	通道 5 检测完成中断使能。 0：不使能；	0x0

Name	Bits	R/W	Description	Default
			1：使能。	
int_vec4_sence_en	[4]	RW	通道 4 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec3_sence_en	[3]	RW	通道 3 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec2_sence_en	[2]	RW	通道 2 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec1_sence_en	[1]	RW	通道 1 检测完成中断使能。 0：不使能； 1：使能。	0x0
int_vec0_sence_en	[0]	RW	通道 0 检测完成中断使能。 0：不使能； 1：使能。	0x0

20.9.11 INT_RAW

◆ **偏移地址：0x34　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					int_charge_ot_raw	int_rrfin_raw	int_shd_sence_raw	int_vec23_sence_raw	int_vec22_sence_raw	int_vec21_sence_raw	int_vec20_sence_raw	int_vec19_sence_raw	int_vec18_sence_raw	int_vec17_sence_raw	int_vec16_sence_raw
-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
int_vec15_sence_raw	int_vec14_sence_raw	int_vec13_sence_raw	int_vec12_sence_raw	int_vec11_sence_raw	int_vec10_sence_raw	int_vec9_sence_raw	int_vec8_sence_raw	int_vec7_sence_raw	int_vec6_sence_raw	int_vec5_sence_raw	int_vec4_sence_raw	int_vec3_sence_raw	int_vec2_sence_raw	int_vec1_sence_raw	int_vec0_sence_raw
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
int_charge_ot_raw	[26]	RO	快速模式检测超时原始中断。 0：没有中断； 1：产生中断。	0x0
int_rrfin_raw	[25]	RO	轮询完成原始中断。 0：没有中断； 1：产生中断。	0x0

Name	Bits	R/W	Description	Default
int_shd_sence_raw	[24]	RO	shield 通道检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec23_sence_raw	[23]	RO	通道 23 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec22_sence_raw	[22]	RO	通道 22 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec21_sence_raw	[21]	RO	通道 21 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec20_sence_raw	[20]	RO	通道 20 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec19_sence_raw	[19]	RO	通道 19 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec18_sence_raw	[18]	RO	通道 18 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec17_sence_raw	[17]	RO	通道 17 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec16_sence_raw	[16]	RO	通道 16 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec15_sence_raw	[15]	RO	通道 15 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec14_sence_raw	[14]	RO	通道 14 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec13_sence_raw	[13]	RO	通道 13 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec12_sence_raw	[12]	RO	通道 12 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec11_sence_raw	[11]	RO	通道 11 检测完成原始中断。 0：没有中断；	0x0

Name	Bits	R/W	Description	Default
			1：产生中断。	
int_vec10_sence_raw	[10]	RO	通道 10 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec9_sence_raw	[9]	RO	通道 9 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec8_sence_raw	[8]	RO	通道 8 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec7_sence_raw	[7]	RO	通道 7 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec6_sence_raw	[6]	RO	通道 6 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec5_sence_raw	[5]	RO	通道 5 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec4_sence_raw	[4]	RO	通道 4 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec3_sence_raw	[3]	RO	通道 3 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec2_sence_raw	[2]	RO	通道 2 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec1_sence_raw	[1]	RO	通道 1 检测完成原始中断。 0：没有中断； 1：产生中断。	0x0
int_vec0_sence_raw	[0]	RO	通道 0 检测完成原始中断。	0x0

20.9.12 INT_STAT

◆ **偏移地址：0x38　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					int_charge_ot_stat	int_rrfin_stat	int_shd_sence_stat	int_vec23_sence_stat	int_vec22_sence_stat	int_vec21_sence_stat	int_vec20_sence_stat	int_vec19_sence_stat	int_vec18_sence_stat	int_vec17_sence_stat	int_vec16_sence_stat
-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
int_vec15_sence_stat	int_vec14_sence_stat	int_vec13_sence_stat	int_vec12_sence_stat	int_vec11_sence_stat	int_vec10_sence_stat	int_vec9_sence_stat	int_vec8_sence_stat	int_vec7_sence_stat	int_vec6_sence_stat	int_vec5_sence_stat	int_vec4_sence_stat	int_vec3_sence_stat	int_vec2_sence_stat	int_vec1_sence_stat	int_vec0_sence_stat
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
int_charge_ot_stat	[26]	RO	快速模式检测超时中断状态。 0：没有中断； 1：产生中断。	0x0
int_rrfin_stat	[25]	RO	轮询完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_shd_sence_stat	[24]	RO	shield 通道检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec23_sence_stat	[23]	RO	通道 23 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec22_sence_stat	[22]	RO	通道 22 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec21_sence_stat	[21]	RO	通道 21 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec20_sence_stat	[20]	RO	通道 20 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec19_sence_stat	[19]	RO	通道 19 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec18_sence_stat	[18]	RO	通道 18 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0

Name	Bits	R/W	Description	Default
int_vec17_sence_stat	[17]	RO	通道 17 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec16_sence_stat	[16]	RO	通道 16 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec15_sence_stat	[15]	RO	通道 15 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec14_sence_stat	[14]	RO	通道 14 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec13_sence_stat	[13]	RO	通道 13 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec12_sence_stat	[12]	RO	通道 12 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec11_sence_stat	[11]	RO	通道 11 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec10_sence_stat	[10]	RO	通道 10 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec9_sence_stat	[9]	RO	通道 9 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec8_sence_stat	[8]	RO	通道 8 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec7_sence_stat	[7]	RO	通道 7 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec6_sence_stat	[6]	RO	通道 6 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec5_sence_stat	[5]	RO	通道 5 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec4_sence_stat	[4]	RO	通道 4 检测完成中断状态。 0：没有中断；	0x0

Name	Bits	R/W	Description	Default
			1：产生中断。	
int_vec3_sence_stat	[3]	RO	通道 3 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec2_sence_stat	[2]	RO	通道 2 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec1_sence_stat	[1]	RO	通道 1 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0
int_vec0_sence_stat	[0]	RO	通道 0 检测完成中断状态。 0：没有中断； 1：产生中断。	0x0

20.9.13 INT_CLR

◆ **偏移地址：0x3C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					int_charge_ot_clr	int_rrfin_clr	int_shd_sence_clr	int_vec23_sence_clr	int_vec22_sence_clr	int_vec21_sence_clr	int_vec20_sence_clr	int_vec19_sence_clr	int_vec18_sence_clr	int_vec17_sence_clr	int_vec16_sence_clr
-	-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
int_vec15_sence_clr	int_vec14_sence_clr	int_vec13_sence_clr	int_vec12_sence_clr	int_vec11_sence_clr	int_vec10_sence_clr	int_vec9_sence_clr	int_vec8_sence_clr	int_vec7_sence_clr	int_vec6_sence_clr	int_vec5_sence_clr	int_vec4_sence_clr	int_vec3_sence_clr	int_vec2_sence_clr	int_vec1_sence_clr	int_vec0_sence_clr
W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
int_charge_ot_clr	[26]	W1P	快速模式检测超时中断清除信号（写 1 清）。	0x0
int_rrfin_clr	[25]	W1P	轮询完成中断清除信号（写 1 清）。	0x0
int_shd_sence_clr	[24]	W1P	shield 通道检测完成中断清除信号（写 1 清）。	0x0
int_vec23_sence_clr	[23]	W1P	通道 23 检测完成中断清除信号（写 1 清）。	0x0
int_vec22_sence_clr	[22]	W1P	通道 22 检测完成中断清除信号（写 1 清）。	0x0

Name	Bits	R/W	Description	Default
int_vec21_sence_clr	[21]	W1P	通道 21 检测完成中断清除信号（写 1 清）。	0x0
int_vec20_sence_clr	[20]	W1P	通道 20 检测完成中断清除信号（写 1 清）。	0x0
int_vec19_sence_clr	[19]	W1P	通道 19 检测完成中断清除信号（写 1 清）。	0x0
int_vec18_sence_clr	[18]	W1P	通道 18 检测完成中断清除信号（写 1 清）。	0x0
int_vec17_sence_clr	[17]	W1P	通道 17 检测完成中断清除信号（写 1 清）。	0x0
int_vec16_sence_clr	[16]	W1P	通道 16 检测完成中断清除信号（写 1 清）。	0x0
int_vec15_sence_clr	[15]	W1P	通道 15 检测完成中断清除信号（写 1 清）。	0x0
int_vec14_sence_clr	[14]	W1P	通道 14 检测完成中断清除信号（写 1 清）。	0x0
int_vec13_sence_clr	[13]	W1P	通道 13 检测完成中断清除信号（写 1 清）。	0x0
int_vec12_sence_clr	[12]	W1P	通道 12 检测完成中断清除信号（写 1 清）。	0x0
int_vec11_sence_clr	[11]	W1P	通道 11 检测完成中断清除信号（写 1 清）。	0x0
int_vec10_sence_clr	[10]	W1P	通道 10 检测完成中断清除信号（写 1 清）。	0x0
int_vec9_sence_clr	[9]	W1P	通道 9 检测完成中断清除信号（写 1 清）。	0x0
int_vec8_sence_clr	[8]	W1P	通道 8 检测完成中断清除信号（写 1 清）。	0x0
int_vec7_sence_clr	[7]	W1P	通道 7 检测完成中断清除信号（写 1 清）。	0x0
int_vec6_sence_clr	[6]	W1P	通道 6 检测完成中断清除信号（写 1 清）。	0x0
int_vec5_sence_clr	[5]	W1P	通道 5 检测完成中断清除信号（写 1 清）。	0x0
int_vec4_sence_clr	[4]	W1P	通道 4 检测完成中断清除信号（写 1 清）。	0x0
int_vec3_sence_clr	[3]	W1P	通道 3 检测完成中断清除信号（写 1 清）。	0x0
int_vec2_sence_clr	[2]	W1P	通道 2 检测完成中断清除信号（写 1 清）。	0x0

Name	Bits	R/W	Description	Default
int_vec1_sence_clr	[1]	W1P	通道 1 检测完成中断清除信号（写 1 清）。	0x0
int_vec0_sence_clr	[0]	W1P	通道 0 检测完成中断清除信号（写 1 清）。	0x0

20.9.14 INT_INJ

◆ **偏移地址：0x40　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					int_charge_ot_inj	int_rrfin_inj	int_shd_sence_inj	int_vec23_sence_inj	int_vec22_sence_inj	int_vec21_sence_inj	int_vec20_sence_inj	int_vec19_sence_inj	int_vec18_sence_inj	int_vec17_sence_inj	int_vec16_sence_inj
-	-	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

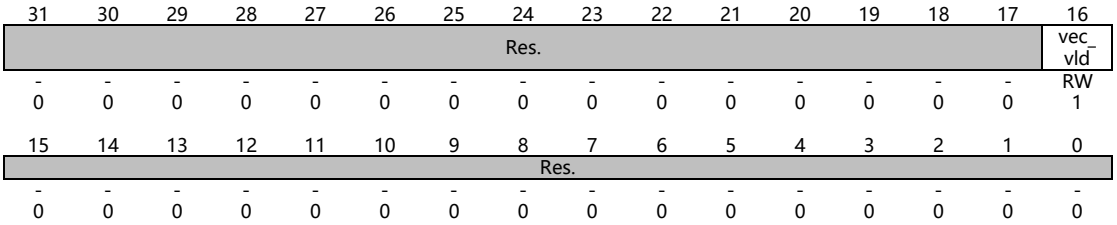
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
int_vec15_sence_inj	int_vec14_sence_inj	int_vec13_sence_inj	int_vec12_sence_inj	int_vec11_sence_inj	int_vec10_sence_inj	int_vec9_sence_inj	int_vec8_sence_inj	int_vec7_sence_inj	int_vec6_sence_inj	int_vec5_sence_inj	int_vec4_sence_inj	int_vec3_sence_inj	int_vec2_sence_inj	int_vec1_sence_inj	int_vec0_sence_inj
W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
int_charge_ot_inj	[26]	W1P	快速模式检测超时中断注入信号（写 1）。	0x0
int_rrfin_inj	[25]	W1P	轮询完成中断注入信号（写 1）。	0x0
int_shd_sence_inj	[24]	W1P	shield 通道检测完成中断注入信号（写 1）。	0x0
int_vec23_sence_inj	[23]	W1P	通道 23 检测完成中断注入信号（写 1）。	0x0
int_vec22_sence_inj	[22]	W1P	通道 22 检测完成中断注入信号（写 1）。	0x0
int_vec21_sence_inj	[21]	W1P	通道 21 检测完成中断注入信号（写 1）。	0x0
int_vec20_sence_inj	[20]	W1P	通道 20 检测完成中断注入信号（写 1）。	0x0
int_vec19_sence_inj	[19]	W1P	通道 19 检测完成中断注入信号（写 1）。	0x0
int_vec18_sence_inj	[18]	W1P	通道 18 检测完成中断注入信号（写 1）。	0x0

Name	Bits	R/W	Description	Default
int_vec17_sence_inj	[17]	W1P	通道 17 检测完成中断注入信号（写 1）。	0x0
int_vec16_sence_inj	[16]	W1P	通道 16 检测完成中断注入信号（写 1）。	0x0
int_vec15_sence_inj	[15]	W1P	通道 15 检测完成中断注入信号（写 1）。	0x0
int_vec14_sence_inj	[14]	W1P	通道 14 检测完成中断注入信号（写 1）。	0x0
int_vec13_sence_inj	[13]	W1P	通道 13 检测完成中断注入信号（写 1）。	0x0
int_vec12_sence_inj	[12]	W1P	通道 12 检测完成中断注入信号（写 1）。	0x0
int_vec11_sence_inj	[11]	W1P	通道 11 检测完成中断注入信号（写 1）。	0x0
int_vec10_sence_inj	[10]	W1P	通道 10 检测完成中断注入信号（写 1）。	0x0
int_vec9_sence_inj	[9]	W1P	通道 9 检测完成中断注入信号（写 1）。	0x0
int_vec8_sence_inj	[8]	W1P	通道 8 检测完成中断注入信号（写 1）。	0x0
int_vec7_sence_inj	[7]	W1P	通道 7 检测完成中断注入信号（写 1）。	0x0
int_vec6_sence_inj	[6]	W1P	通道 6 检测完成中断注入信号（写 1）。	0x0
int_vec5_sence_inj	[5]	W1P	通道 5 检测完成中断注入信号（写 1）。	0x0
int_vec4_sence_inj	[4]	W1P	通道 4 检测完成中断注入信号（写 1）。	0x0
int_vec3_sence_inj	[3]	W1P	通道 3 检测完成中断注入信号（写 1）。	0x0
int_vec2_sence_inj	[2]	W1P	通道 2 检测完成中断注入信号（写 1）。	0x0
int_vec1_sence_inj	[1]	W1P	通道 1 检测完成中断注入信号（写 1）。	0x0
int_vec0_sence_inj	[0]	W1P	通道 0 检测完成中断注入信号（写 1）。	0x0

20.9.15 VEC_CFG

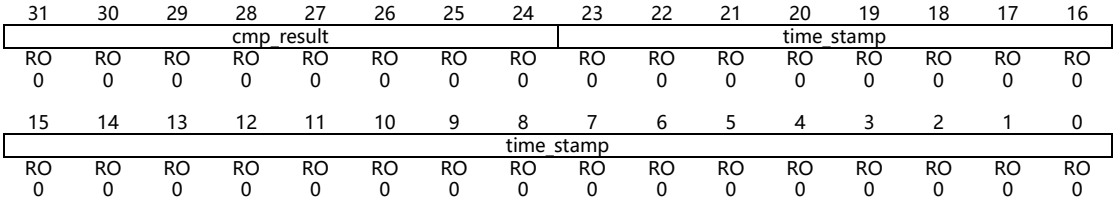
- **偏移地址：0x100+0x4*j　　复位值：0x00010000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
vec_vld	[16]	RW	通道使能（j 代表通道，取值 0~23） 0：无效； 1：有效。	0x1
reserved	[15:0]	-	保留。	0x0

20.9.16 VEC_REC

◆ **偏移地址：0x200+0x4*j　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
cmp_result	[31:24]	RO	检测结果缓存，显示对应通道最近 8 次的检测结果（j 代表通道，取值 0~23）。其中每一位表示： 0：无触摸； 1：有触摸。 注意：最低 bit 表示最近一次检测结果，依次往高 bit 类推。	0x0
time_stamp	[23:0]	RO	通道对应的检测时间戳（每完成一次检测刷新一次）。	0x0

21 同步串行外设接口（SPI）

21.1 SPI 简介

SPI 控制器实现数据的串并、并串转换，可以作为 Master 与外部设备进行同步串行通信，也可作 Slave 与外部 Master 设备对接。支持 Motorola SPI、TI SSP 和 MicroWire 三种外设接口协议。芯片支持 2 个 SPI。

21.2 SPI 主要特性

SPI 的功能特点有：

- 支持接口时钟 SCK 频率可编程。
- 支持主模式和从模式。
- 支持标 SPI 模式。
- 支持主模式下接收串行数据位采样时间可编程延迟。
- 支持双片选。
- 支持 TX(发送) FIFO 和 RX(接收) FIFO 深度为 16。
- 支持 4bit～16bit 可编程串行数据帧长度。
- 支持单帧和连续帧格式。
- 支持 Motorola SPI 全双工工作模式，时钟极性、相位可配置。
- 支持 MicroWire 半双工工作模式。
- 支持 TI 同步串行接口全双工工作模式。
- 支持 DMA 操作。
- 支持位反序功能。
- 支持中断功能。

21.3 SPI 逻辑实现

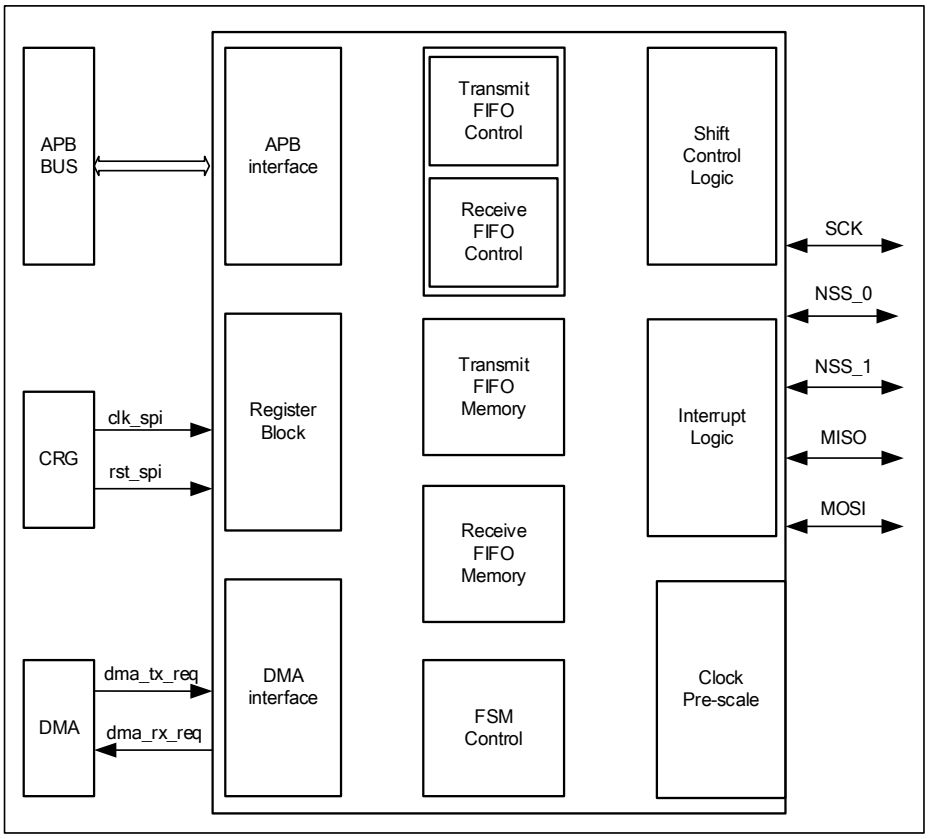
表 21-1 介绍了芯片中 SPI0、SPI1 模块在器件中所具备的功能。

表 21-1 SPI 实现

SPI 特性	SPI0（LS）	SPI1（HS）
支持 TI 模式	YES	YES
支持 MOTO 模式	YES	YES
支持 MW 模式	YES	YES
最高串行输出时钟	主模式：25Mhz 从模式：20Mhz	主模式：25Mhz 从模式：20Mhz
2/4 线模式	NO	NO
增强型 SPI 模式	NO	NO
XIP 模式	NO	NO
数据帧长	4~16	4~16
FIFO 深度	TX FIFO:16 RX FIFO:16	TX FIFO:16 RX FIFO:16

SPI 支持同步串行方式与外部器件进行通信。CPU 通过总线配置 SPI 控制器，SPI 控制器与外围 SPI 设备对接通信，需要将待发送的数据写入 TX FIFO，然后转换成 SPI 总线协议发送给外设，也可以从 RX FIFO 中读取外设发送给 SPI 控制器的数据。SPI 控制器支持中断模式，向 CPU 上报中断。同时 SPI 控制器也支持 DMA 模式，不经过 CPU 处理，在 SPI 控制器与存储器之间直接进行数据传输。软件可通过轮询 BUSY 状态标志或使用 SPI 专用中断对通信进行管理。SPI 的主要组件及其交互方式如下面的图 21-1 框图所示。

图 21-1 SPI 框图



- SPI 控制器支持接口时钟 SCK 频率可编程

接口时钟 SCK 由 SPI 的工作时钟 Fclk_spi 分频产生，计算公式为：

SCK= Fclk_spi/分频系数

其中 Fclk_spi 是 SPI 的工作时钟，SPI0 的工作时钟为 APB_LS 时钟，最大为 100Mhz；SPI1 的工作时钟为 APB_HS，最大为 200Mhz。分频系数通过 BAUDR.sckdv 寄存器配置，支持配置为 2~65534 之间的任意偶数值，如果 BAUDR.sckdv 配置为 0，则 SCK 被禁用。SCK 支持的最大速率请参考表 21-1，使用时请根据 SPI 的工作时钟灵活配置分频系数。

⚠注意

当 SPI 的串行输出时设置比较高的时候，中断和轮询模式可能不能及时处理数据，将导致数据传输异常，建议使用 DMA 模式搬运数据。

- SPI 控制器支持主、从模式可配置

通过配置 CTRLR0.spi_mst_slv_sel 位选择主/从模式。SPI 协议规定了 2 个 SPI 设备之间通

信必须由主设备 Master 来控制从设备 Slave。其中 Master 可以通过时钟 SCK 以及片选信号来控制多个 Slave 设备；而 Slave 设备只能接收来自 Master 的 SCK 时钟，本身不能产生与控制 SCK 信号，如果没有 SCK 则 Slave 设备不能正常工作。

- SPI 控制器支持双片选

SPI 作为 Master，当连接 2 个 Slave 时，可通过配置 SER.ser 位域来设置片选信号，选择与哪个 slave 通信。

- SPI 控制器支持 TX FIFO 和 RX FIFO

TX FIFO 和 RX FIFO 深度均为 16。写操作时，写入的数据通过 DRx.dr 位域进入 TX FIFO; 读操作时，通过 DRx.dr 位域读取进入 RX FIFO 的数据。此外，通过配置 TXFTLR.tft 和 RXFTLR.rft 位域，设置 TX FIFO 和 RX FIFO 的水线。用户通过读取 TXFLR.txtfl 和 RXFLR.rxtfl 寄存器，可以获取到当前 TX FIFO 和 RX FIFO 的状态。

- SPI 控制器支持可编程的串行数据帧长度

串行数据帧长度支持 4bit ~ 16bit 可编程范围，通过配置 CTRLR0.dfs 位域实现。

- SPI 控制器支持的工作模式

SPI 控制器支持 Motorola SPI 全双工工作模式、MicroWire 半双工工作模式和 TI 同步串行接口全双工工作模式，通过配置 CTRLR0.frf 位域选择工作模式。在 Motorola SPI 全双工工作模式下，通过配置 CTRLR0.spo 位域和 CTRLR0.sph 位域以分别设置时钟极性和时钟相位。

- SPI 控制器支持 DMA 操作。

支持从内存搬运数据至 TX FIFO 再发送至从机，也支持将读回的数据存入 RX FIFO 再搬运至内存。SPI 控制器支持 signle 和 burst DMA 操作，burst DMA 操作的 TX FIFO 和 RX FIFO 的水线分别通过配置 DMATDLR.dmatdl 和 DMARDLR.dmardl 位域设置。具体工作方式见 21.4.4 DMA 控制器数据传输章节。

- SPI 通过 4 个专用引脚与外部器件通讯：
  - **MISO**：主输入/从输出引脚。即主模式下接收数据，从模式下发送数据。
  - **MOSI**：主输出/从输入引脚。即主模式下发送数据，从模式下接收数据。
  - **SCK**：串行比特率时钟引脚，主模式下发送到外设，从模式下从外设接收。
  - **NSS**：从设备选择引脚。根据 Motorola SPI、MicroWire、TI 模式设置。

21.4 SPI 功能描述

21.4.1 发送与接收传输模式

SPI 在串行总线上传输数据，可通过如下 4 种模式进行传输。

1. 发送与接收模式

当 CTRLR0.TMOD 为 0 时，发送和接收功能都有效。根据选择的帧格式，数据传输可以正常进行。待发送的数据将从 TX FIFO 中弹出，并通过 MOSI 引脚发送到目标设备。相应的目标设备将返回数据到 MISO 引脚上，在每个数据帧的末端，目标设备返回的数据将从接收移位寄存器移动到接收 FIFO 中。

2. 只发送模式

当 CTRLR0.TMOD 为 1 时，接收数据无效，不会存储在接收 FIFO 中。根据选择的帧格式，发送数据可以正常进行。待发送的数据从 TX FIFO 中弹出，并通过 MOSI 引脚发送到目标设备。相应的目标设备将返回数据到 MISO 引脚上，在数据帧的末端，接收移位寄存器不会把新接收的数据保存到 RX FIFO 中，接收移位寄存器中的数据将被下一个数据所覆盖。在此模式下，需要软件屏蔽来自接收逻辑的中断信号。

3. 只接收模式

当 CTRLR0.TMOD 为 2 时，发送数据无效。当配置为从设备时，TX FIFO 的数据在该模式下不会发送出去，发送引脚在数据传输期间保持在恒定的逻辑电平。根据选择的帧格式，接收数据可以正常进行，在每个数据帧的末端，来自外部目标设备的数据将从接收移位寄存器移动到 RX FIFO 中。在此模式下，需要软件屏蔽来自发送逻辑的中断信号。

4. EEPROM 读模式

当 CTRLR0.TMOD 为 3 时，此模式仅配置为主设备有效，主设备发送数据用于向 EEPROM 设备发送操作码及地址。通常这需要三个数据帧（8 位操作码、8 位高地址和 8 位低地址）。在操作码和地址的传输过程中，主设备的接收逻辑不会捕获任何数据（只要发送引脚上有传输数据，接收引脚上的数据就会被忽略）。主设备只有在 TX FIFO 为空时才停止发送数据。当 TX FIFO 为空后，接收引脚上的数据会被保存到 RX_FIFO 中，在此期间，发送引脚将保持在恒定的逻辑电平。

⚠ 注意

当 SPI 配置为 TI 模式时，将不支持 EEPROM 读模式。

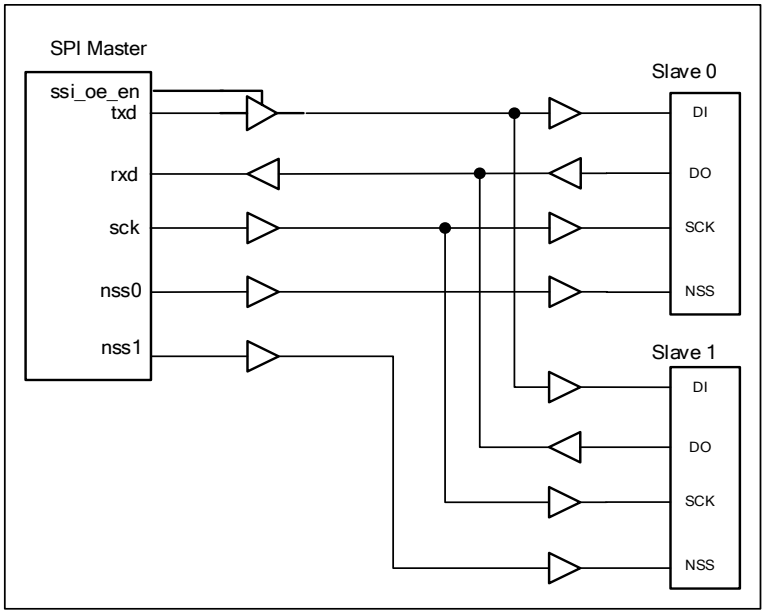
21.4.2 主设备模式与从设备模式

SPI 支持配置为主模式（Master）或者从模式（Slave）。

1. 主模式（Master Mode）

此模式下可与外部从设备进行串行通信。当配置为主模式时，SPI 将启动并控制所有串行传输。图 21-2 描述了 1 个 SPI Master 连接 2 个 Slave 时的连接关系。Master 的 nss0 和 Slave0 的 NSS 相连，Master 的 nss1 和 Slave1 的 NSS 相连，通过配置 SER.ser 位域来设置片选信号，当 SER.ser[0]位域配置为 1 时，选通 Slave0；当 SER.ser[1]配置为 1 时，则选通 Slave1。

图 21-2 主模式下 Master 与多 Slave 外设通信

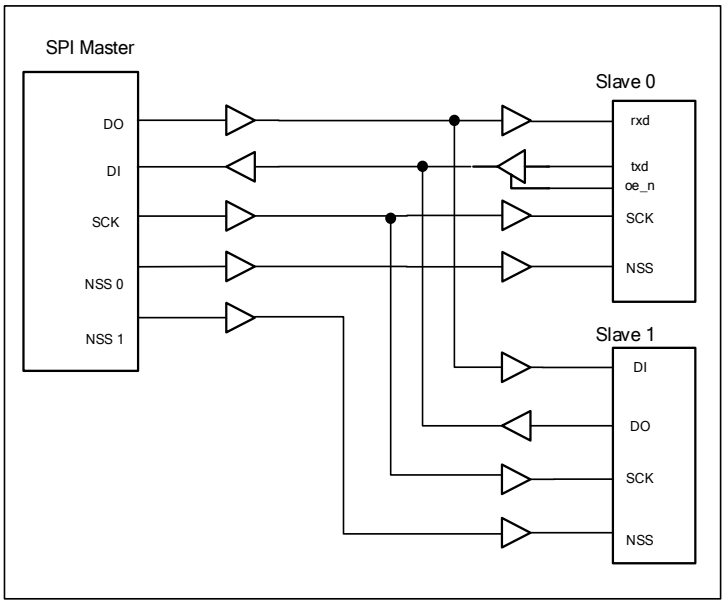


由 SPI Master 生成和控制的串行比特率时钟在 SCK 信号上输出。当 SPI 被禁用（SSI_EN = 0）时，不会发生串行传输，SCK 处于“非活动”状态，由它当前运行的串行协议定义。传输由主设备 Master 启动，当 SPI 使能时（SSI_EN=1）,传输 FIFO 中至少存在一个有效数据。当数据传输开始后，状态寄存器 SR 中的 busy 标志（BUSY）将拉高。必须等待其状态清除后，才能进行新的数据传输。

2. 从模式（Slave Mode）

此模式下可与外部主设备进行串行通信。当配置为从模式时，所有的串行传输都由主设备启动和控制。图 21-3 显示了在单主总线系统中配置为从设备的示例。

图 21-3 从模式下多 Slave 与 Master 通信



当 SPI 配置为 Slave 模式且被片选信号选中后，txd 信号将可以通过串行总线发出。所有进出从设备的数据传输，都在来自主设备输出的串行时钟线 SCK 上进行控制。数据在串行时钟总线的边沿进行传输，在另一个时钟边沿进行采样。当从设备没有被选中时，txd 输出呈高阻态，因此不会干扰主设备与其它从设备的数据传输。

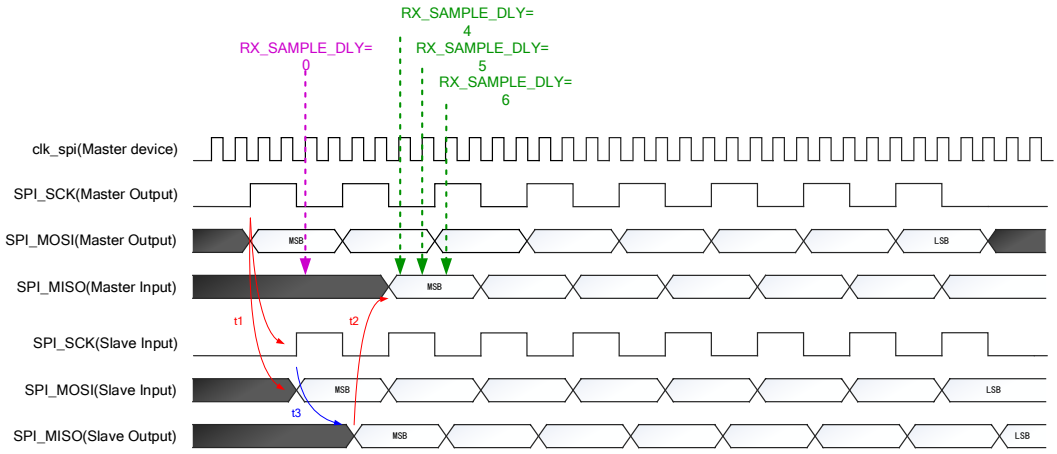
21.4.3 RX 采样延迟

当 SPI 被配置为主模式且传输速率较高时，由于往返路径（板级走线）的延迟，可能导致主设备对 rxd_mst 信号（接收到的数据）的采样时间偏离了正确的采样时间，这将导致数据采样错误，从而导致主设备接收到错误的数据。

为了解决上述问题，SPI 内部逻辑支持配置采样延迟寄存器 RX_SAMPLE_DLY，以便延迟主设备对 rxd_mst 信号的采样时间，从而提 SPI 在高速模式下数据传输的正确性。

图 21-4 是 RX 采样延迟示意图，其中红色箭头表示 SCK 信号在主设备和从设备之间的传输延迟，蓝色箭头表示从设备接收 SCK 信号（sclk_in）到从设备发送数据（txd_slv）之间的延迟。t1+t2+t3 即表示主设备发出 SCK 信号开始到主设备接收到第一个数据所需时间，在高速模式下，由于这个时间的延迟，如果主设备仍然从默认位置（上图紫色虚线处）对 rxd_mst 信号进行采样，这将导致采样数据错误。

图 21-4 RXD 采样延迟示意图



在 RX 采样延迟模式下，通过配置采样延迟寄存器 RX_SAMPLE_DLY，SPI 主设备内部逻辑将 rxd_mst 信号的采样时间从默认位置（上图紫色虚线处，RX_SAMPLE_DLY 配置为 0）往后延迟，延迟时间是 ssi_clk（clk_spi）时钟周期的整倍数，具体时间由 RX_SAMPLE_DLY 寄存器的配置值决定，通过这种方式可以确保主设备采样的 setup 时序满足要求，从而提高 SPI 在高速模式下数据传输的正确性。

21.4.4 DMA 控制器数据传输

SPI 使用握手信号与芯片内 DMA 控制器交互，通过 APB 总线完成与 DMA 控制器的数据传输，且由 DMA 控制传输数据块的大小（DMA 寄存器介绍请参考 DMA 相关章节）。

1. 数据块传输方式

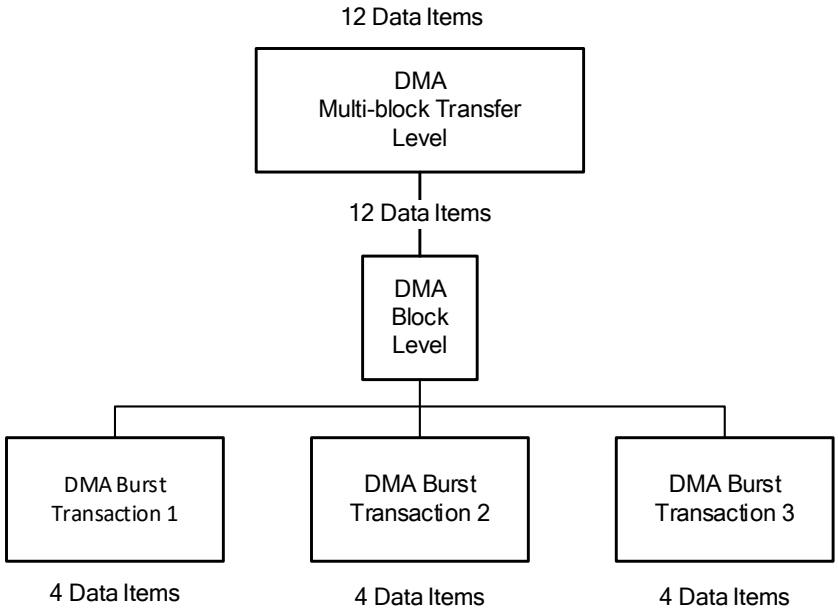
SPI 有 2 个 DMA 通道，一个用于发送数据，另一个用于接收数据。通过 3 个寄存器来实现 DMA 传输：

- DMACR：DMA 传输使能寄存器
- DMATDLR：发送 FIFO 水线，当 FIFO 中有效数据少于或等于它，SPI 将向 DMA 控制器发送 DMA 发请求
- DMARDLR：接收 FIFO 水线，当 FIFO 中有效数据大于或等于它，SPI 将向 DMA 控制器发送 DMA 读请求

为实现 DMA 传输控制，需要通过 CPU 配置 DMAC 控制器，包括数据块长度（DMA.CTLx.block_ts）、突发长度（DMA.CTLx.src_msize/dest_msize）。

图 21-5 显示了一个单个数据块传输，其中数据块大小设置为 12，突发长度设置为 4。在这种情况下，数据块大小是突发长度(burst length)的倍数，因此 DMA 传输就由 3 个突发传输组成。

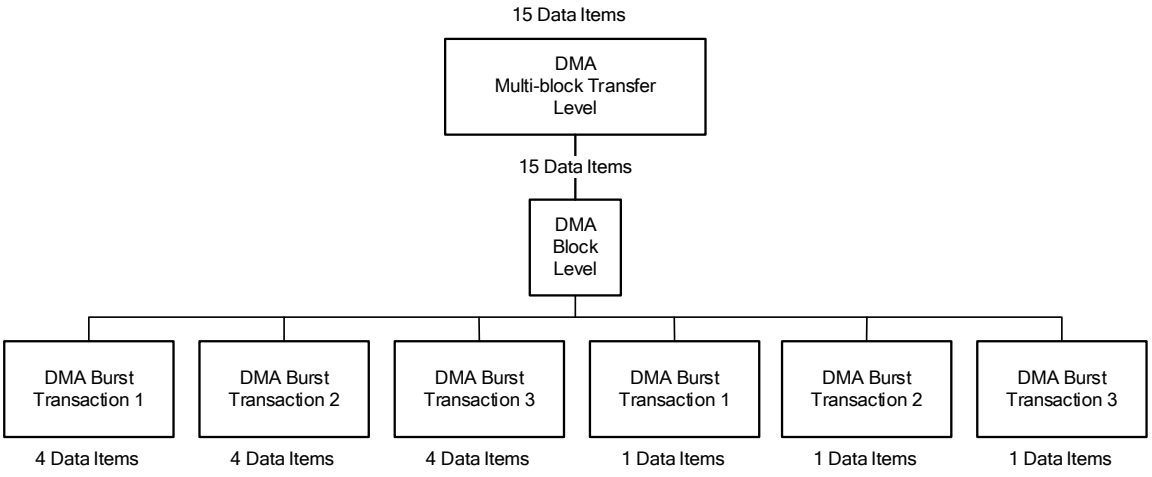
图 21-5 DMA 传输数据量为突发长度整数倍



DMA块传输数据长度：DMA.CTLx.BLOCK_TS = 12
每个突发事务处理的数据量：DMA.CTLx.SRC_MSIZE = 4
SPI DMA请求发起水线：SSI.DMARDLR + 1 = DMA.CTLx.SRC_MSIZE = 4

当需要传输的数据块大小（配置到 DMAC 中的数据块大小）不是突发长度的倍数时，如下图 21-6 所示，就需要多个突发传输及单次传输来完成整个数据块的传输。

图 21-6 DMA 传输数据量为非突发长度整数倍



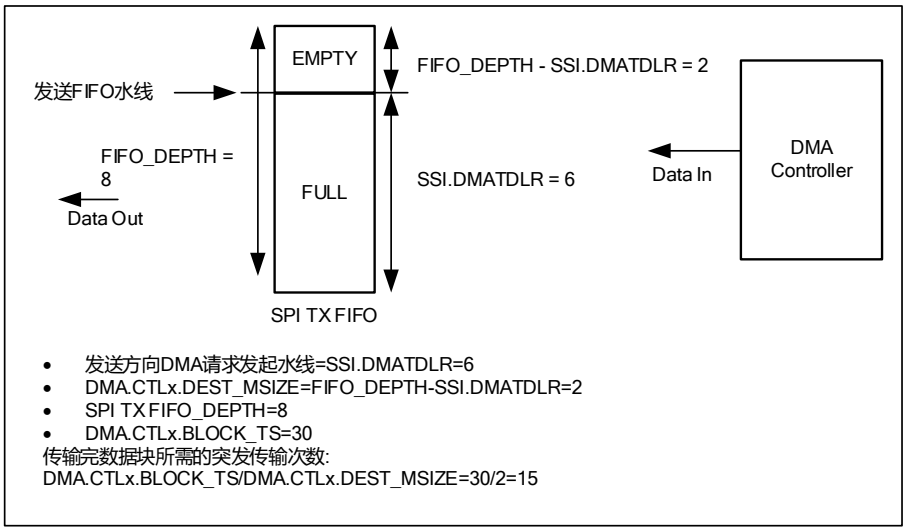
DMA块传输数据长度：DMA.CTLx.BLOCK_TS = 15
每个突发事务处理的数据量：DMA.CTLx.SRC_MSIZE = 4
SPI DMA请求发起水线：SSI.DMARDLR + 1 = DMA.CTLx.SRC_MSIZE = 4

2. FIFO 水线配置

SPI 发送方向

FIFO 下溢：SPI 发送数据期间，当 FIFO 中数据量小于等于 DMATDLR 时，SPI 就会向 DMAC 控制器发送请求，DMAC 响应并发送突发长度的数据到 FIFO 中。为达到连续传输目的，SPI 应及时向 DMAC 请求数据，否则 FIFO 被清空产生 FIFO 下溢。

图 21-7 发送方向 FIFO 水线配置



在这个数据块传输中共有 15 个突发传输，其中 DMATDLR 配置值较高，因此 SPI 出现下溢的概率很低，因为 DMAC 控制器在 SPI TX FIFO 变空前有足够的时间来服务突发传输，但随之而来的是总线的效率降低。

从图 21-7 可以看出，若 DMA.CTLx.dest_msize 设置为大于水线，当 SPI TX FIFO 中没有足够空间来服务突发请求时，FIFO 将会溢出。因此为了避免出现溢出，必须遵循如下公式：

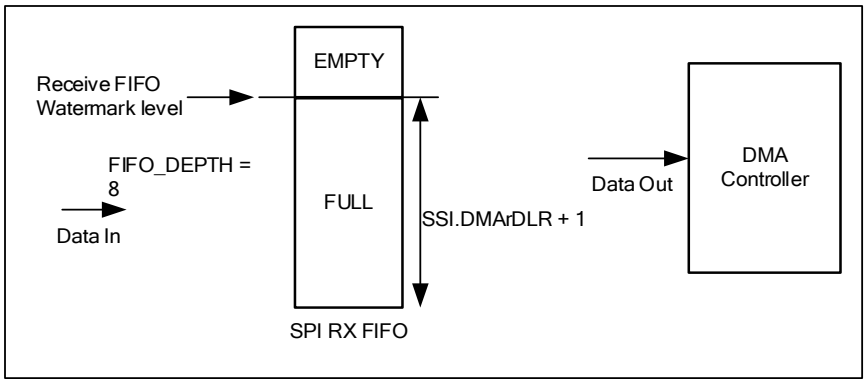
$$DMA.CTLx.dest_msize = SSI.fifo_depth - SSI.dmatdlr$$

在示例中 DMATDLR 设置为 6，在向 DMAC 发出请求时 TX FIFO 中的剩余空间等于目标突发长度 DMA.CTLx.dest_msize。因此在突发传输完成时，TX FIFO 可能是满足的，但并没有溢出。

SPI 接收方向

FIFO 上溢：SPI 读取数据期间，当 FIFO 中数据量大于等于 DMARDLR 时，SPI 就会向 DMAC 控制器发送请求，DMAC 响应并从 FIFO 中读取突发长度的数据。为达到连续传输目的，DMA 应及时从 DMAC 读取数据，否则 FIFO 可能被填满产生 FIFO 上溢。

图 21-8 接收方向 FIFO 水线配置



与发送方向水线设置类似，接收水线 DMARDLR 的设置要最小化 RX FIFO 溢出的概率。如图 21-8 所示，需要权衡 DMA 突发长度与 RX FIFO 溢出概率。若突发长度设置为大于水线，若 FIFO 没有足够的数据供突发传输时，将会发生下溢。如果 RX FIFO 中的数据量等于突发传输长度时，突发传输完成后 RX FIFO 被读空，但 FIFO 并不会下溢。因此推荐的水线设置为：

$$DMA.CTLx.src_msize = SSI.dmardlr + 1$$

这样可以减少突发传输的次数，从而提高总线的效率。

21.5 SPI 接口时序

图 21-9～图 21-23 中的缩略语含义为：

- MSB：Most Significant Bit（最高有效位）
- LSB：Least Significant Bit（最低有效位）
- Q：Q is an undefined signal（未定义信号）

⚠注意

时序图中的 SPI_CS_N 为配置 SER.ser 位域后的 SPI 片选信号。

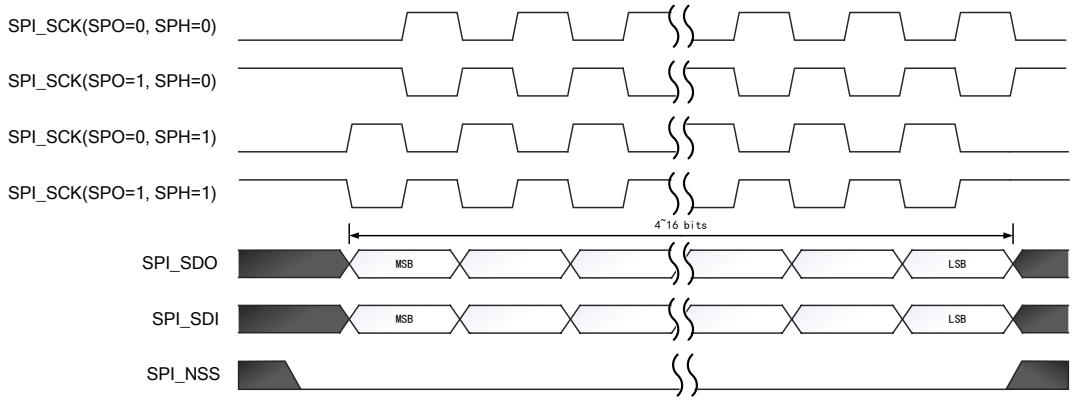
21.5.1 Motorola SPI 接口

说明

SPO 表示 SPI_CLK 的极性，SPH 表示 SPI_CLK 的相位。

SPI 接口时序概览如图 21-9 所示。

图 21-9 SPI 接口时序概览图



1. SPO=0、SPH=0

SPI 单帧和连续帧帧格式分别如图 21-10 和图 21-11 所示。

图 21-10 SPI 单帧帧格式（SPO=0、SPH=0）

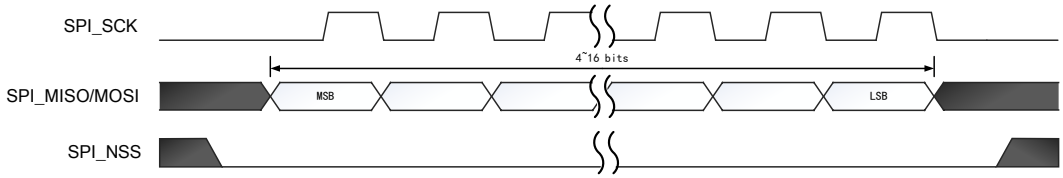
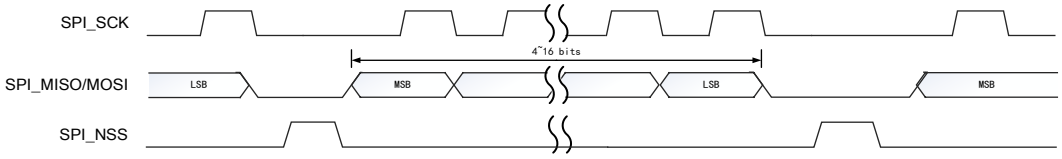


图 21-11 SPI 连续帧帧格式（SPO=0、SPH=0）



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为低
- SPI_CS_N 信号设置为高
- 发送数据线 SPI_TX 强制为低

当 SPI 处于使能状态，而且发送 FIFO 内存在有效数据时，设置 SPI_CS_N 信号为低，表示开始传输数据。来自 Slave 的数据立刻发送到 Master 的接收数据线 SPI_RX。半个 SPI_CLK 时钟周期之后，有效的 Master 数据传输到 SPI_TX。此时 Master 和 Slave 数据都已经有效，SPI_CLK 管脚在接下来的半个 SPI_CLK 时钟周期之后变为高电平。数据在 SPI_CLK 时钟的上升沿被捕获，在时钟的下降沿被传送。

如果传输单个帧，当捕捉到最后 1bit 数据时，SPI_CS_N 在接下来的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

如果是传输连续帧，SPI_CS_N 信号在每个帧传输之间必须将 SPI_CLK 时钟拉高一个时钟周期。这是因为 SPH 为 0 时，Slave 选择管脚会固定其内部串行设备寄存器的数据，使它不会变化。因此在连续传输时，主设备必须在每个帧传输之间将 SPI_CS_N 信号拉高。连续传输结束时，SPI_CS_N 在捕获到最后 1bit 之后的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

2. SPO=0、SPH=1

SPI 单帧和连续帧帧格式分别如图 21-12 和图 21-13 所示。

图 21-12 SPI 单帧帧格式（SPO=0、SPH=1）

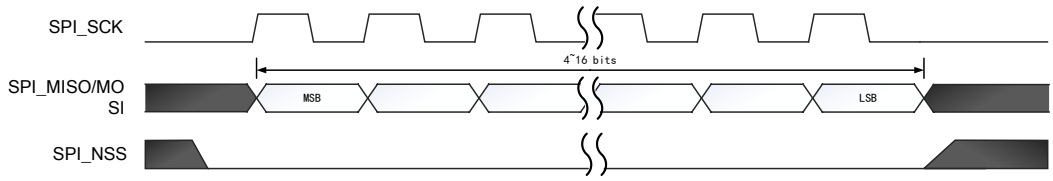
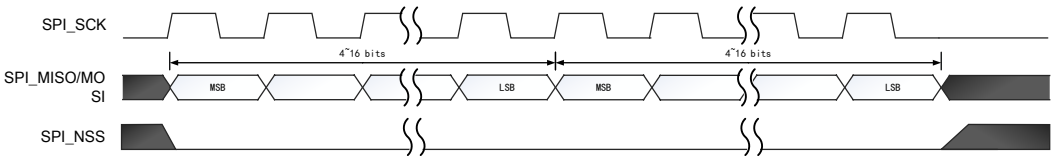


图 21-13 SPI 连续帧帧格式（SPO=0、SPH=1）



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为低
- SPI_CS_N 设置为高
- 发送数据线 SPI_TX 强制为低

当 SPI 为使能状态，而且发送 FIFO 内存在有效数据时，设置 SPI_CS_N 信号为低表示开始传输数据。半个 SPI_CLK 时钟周期之后，Master 和 Slave 的有效数据分别在各自

的传输线上有效。同时，SPI_CLK 从第一个上升沿开始有效。数据在 SPI_CLK 时钟的下降沿被捕获，在时钟的上升沿被传送。

如果传输单个帧，当捕捉到最后 1bit 数据时，SPI_CS_N 在接下来的 1 个 SPI_CLK 时钟之后恢复为高电平。

当连续传输时，在传输数据帧之间 SPI_CS_N 保持为低。连续传输结束时，SP_CS_N 在最后 1bit 捕获之后的 1 个 SPI_CLK 时钟之后恢复为高电平。

3. SPO=1、SPH=0

SPI 单帧和连续帧帧格式分别如图 21-14 和图 21-15 所示。

图 21-14 SPI 单帧帧格式（SPO=1、SPH=0）

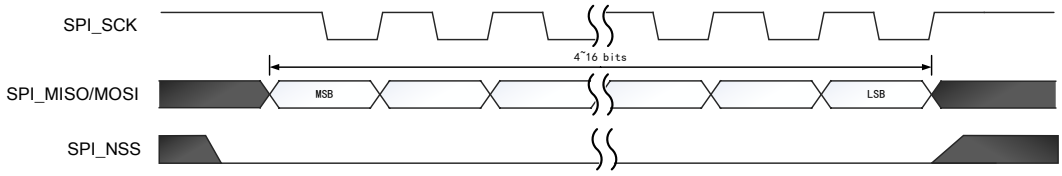
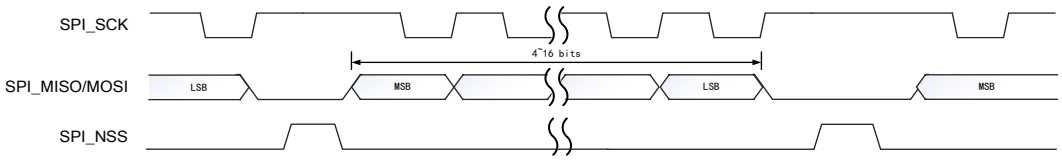


图 21-15 SPI 连续帧帧格式（SPO=1、SPH=0）



在该配置下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为高
- SPI_CS_N 信号设置为高
- 发送数据线 SPI_TX 强制为低

当 SPI 为使能状态，而且发送 FIFO 内存在有效数据时，设置 SPI_CS_N 信号为低表示开始传输数据。此时 Slave 的数据立刻发送到 Master 的接收数据线 SPI_RX。半个 SPI_CLK 周期之后，Master 的有效数据传送到 SPI_TX。再过半个 SPI_CLK 时钟周期之后，SPI_CLK Master 管脚设置为低。这表示数据在 SPI_CLK 时钟的下降沿被捕获，在 SPI_CLK 时钟的上升沿被传送。

如果传输单个帧，当捕捉到最后 1bit 数据时，SPI_CS_N 在接下来的 1 个 SPI_CLK 时钟之后恢复为高电平。

如果是连续的传输，SPI_CS_N 信号在每个帧传输之间必须拉高。这是因为当 SPH 为 0 时，Salve 选择管脚固定其内部串行设备寄存器的数据，使它不会变化。SPI_CS_N 在捕获到最后 1bit 数据之后的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

4. SPO=1、SPH=1

SPI 单帧和连续帧帧格式分别如图 21-16 和图 21-17 所示。

图 21-16 SPI 单帧帧格式（SPO=1、SPH=1）

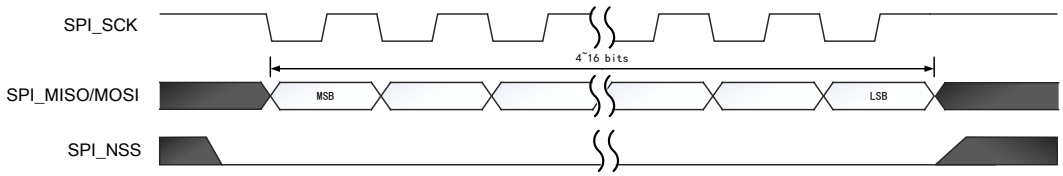
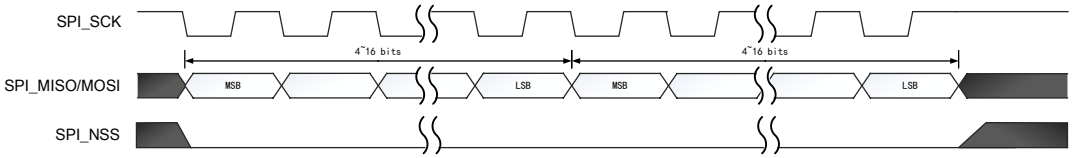


图 21-17 SPI 连续帧帧格式（SPO=1、SPH=1）



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 信号设置为高
- SPI_CS_N 信号设置为高
- 发送数据线 SPI_TX 强制为低

当 SPI 为使能状态，而且发送 FIFO 内存在有效数据时，设置 SPI_CS_N Master 信号为低表示开始传输数据。半个 SPI_CLK 时钟周期后，Master 和 Slave 数据在各自的传输线上有效。同时，时钟 SPI_CLK 从 1 个下降沿开始有效。数据在 SPI_CLK 时钟的上升沿被捕获，在时钟的下降沿被传送。

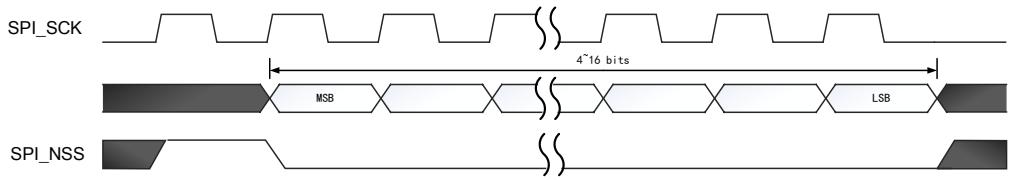
当传输单个帧时，SPI_CS_N 在传输的最后 1bit 捕获之后的 1 个 SPI_CLK 时钟周期之后恢复为高电平。

如果是连续传输，SPI_CS_N 信号始终保持为低。SPI_CS_N 在捕获到最后 1bit 之后的 1 个 SPI_CLK 时钟周期之后恢复到高状态。对于连续传输来说，SPI_CS_N 在传输过程中一直保持为低，结束方式与单个传输方式相同。

21.5.2 TI 同步串行接口

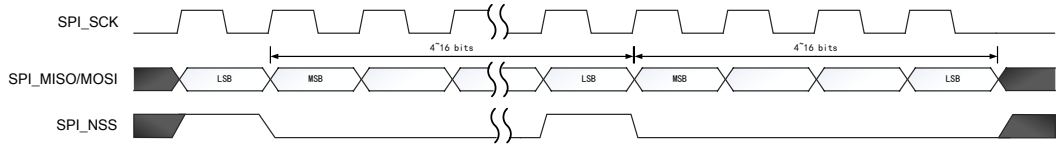
TI 同步串行单帧帧格式如图 21-18 所示。

图 21-18 TI 同步串行单帧帧格式



TI 同步串行连续帧帧格式如图 21-19 所示。

图 21-19 TI 同步串行连续帧帧格式



在该模式下，当 SPI 处于空闲状态时：

- SPI_CLK 为低电平
- SPI_CS_N 为低电平
- 传输数据线 SPI_TX 保持为高阻

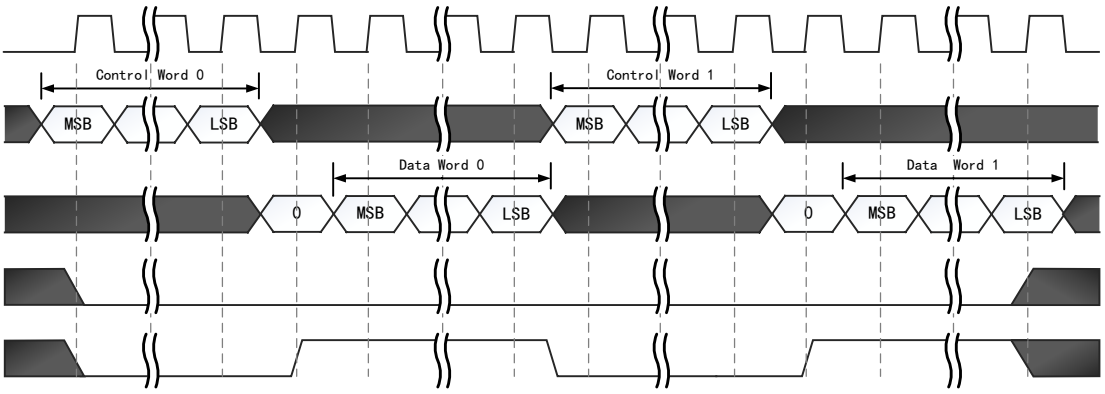
一旦发送 FIFO 有数据，SPI_CS_N 就会产生一个 SPI_CLK 时钟周期的高电平脉冲，将被发送的数据就会从发送 FIFO 传送到发送逻辑串行移位寄存器。在 SPI_CLK 时钟的下一个上升沿，4bit～16bit 数据帧的 MSB 就会从 SPI_TX 移位输出。同样，从外部串行 Slave 设备接收数据的 MSB 会从 SPI_RX 管脚移位输入。

SPI 和片外串行设备在 SPI_CLK 时钟的下降沿将数据存入串行移位寄存器。接收串行寄存器在接收到 LSB 之后的第一个 SPI_CLK 时钟上升沿将数据送给接收 FIFO。

21.5.3 Microwire 接口

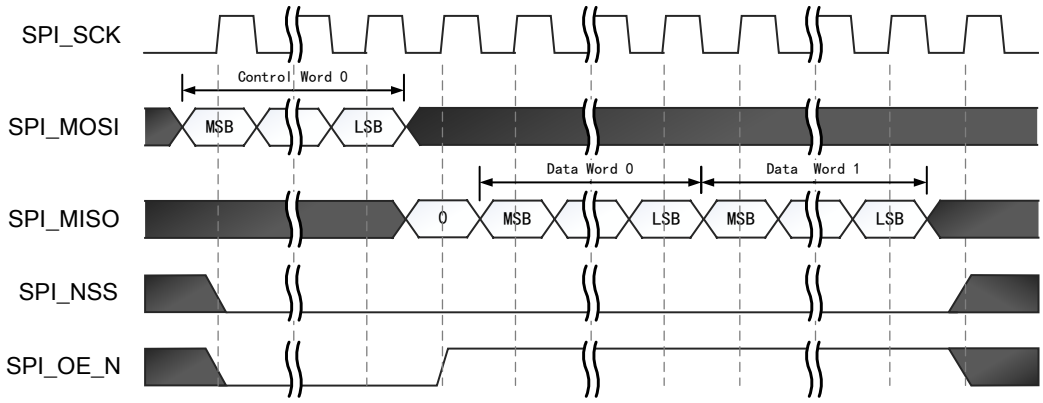
1. 在 Microwire 模式下配置为 Master

图 21-20 Microwire Non-sequential 模式连续读操作（MDD=0，MWMOD=0）



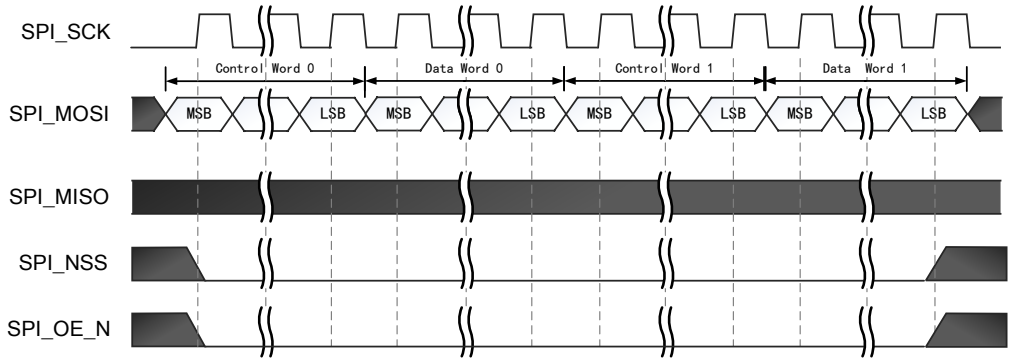
执行 Non-sequential 连续读操作，所需的动作是将更多的控制字写入传输 FIFO 缓冲区。在上图 21-20 中，从外部串行从设备读取两个数据字。

图 21-21 Microwire Sequential 模式连续读操作（MDD=0，MWMOD=1）



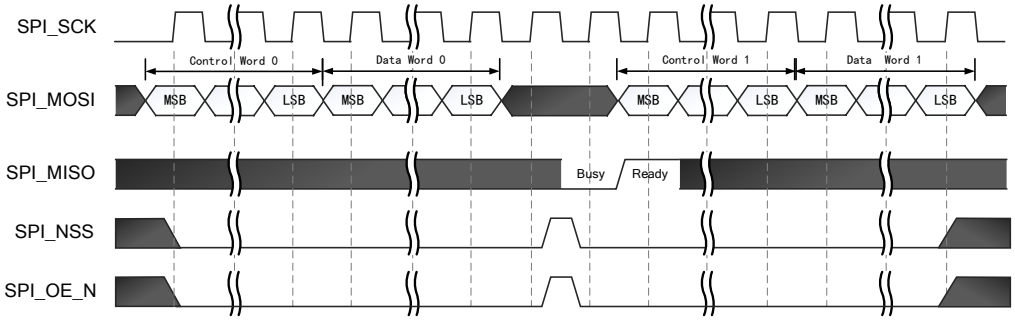
在 Sequential 连续读期间，只有一个控制字从 SPI Master 中输出，如上图 21-21 所示。传输的启动方式与 Non-sequential 的读取操作相同，但是循环会读更多的数据。Slave 设备自动从下一地址获取数据。可以以这种方式读取任意数量的位置，当接收数据个数达到设定个数（CTRLR1 寄存器中的值+1）时，表明数据传输完成，Master 设备自动结束传输。

图 21-22 Microwire 连续写操作（MDD=1，MWMOD=0）



通过不断将 Ctrl Word 与 Data word 交替写入 TX FIFO，就可以完成连续写操作，如上图 21-22 所示。

图 21-23 Microwire 带握手功能连续写操作（MWHS=1，MDD=1，MWMOD=0）



握手接口也可以用于对外部从设备的写操作，如上图 21-23 所示。要启用握手功能，必须将 1 写入 MWCR 寄存器上的 MHS 位字段（第 2 位）。当 MHS 设置为 1 时，SPI Master 在完成传输之前检查从设备的就绪状态，或为连续传输传输下一个控制字。

在第一个数据字被传送到串行从设备之后，SPI Master 轮询从从设备等待就绪状态的 rxd 输入。在接收到就绪状态后，SPI Master 开始传输下一个控制字。假定 Slave 将保持准备状态，直到它收到来自 Master 的任何确认，这是 Master 恢复时钟时提供的。在最后一个数据帧的传输完成后，SPI Master 传输一个开始位，以在完成传输之前清除 Slave 设备的就绪状态。

2. 在 Microwire 模式下配置为 Slave

当 SPI 被配置为 Slave 时，数据传输从片选信号（ss_in_n）的下降边缘开始。在半串行时钟（sclk_in）周期之后，控制位的第一 bit 出现在 rxd 线上。控制字的长度可以在 1 到 16

位的范围内，通过在 CTRLR0 寄存器中写入位字段 CFS 来设定。

数据字的方向由寄存器 MWCR.mdd 位域控制。当 MDD 为 0 时，这表示 SPI Slave 将从外部 Master 接收数据。在传输控制字后，Master 立即开始将数据帧驱动到 SPI Slave rxd 线上。数据在串行时钟的下降边缘发送，并在上升边缘被捕获。从设备选择信号在传输期间保持活动低，并在数据传输完成后半个时钟周期拉高。SPI 从设备输出启用信号（ssi_oe_n）在传输期间保持非活动状态。

图 21-24 单次读操作（MWHS=0，MDD=0，MWMOD=0）

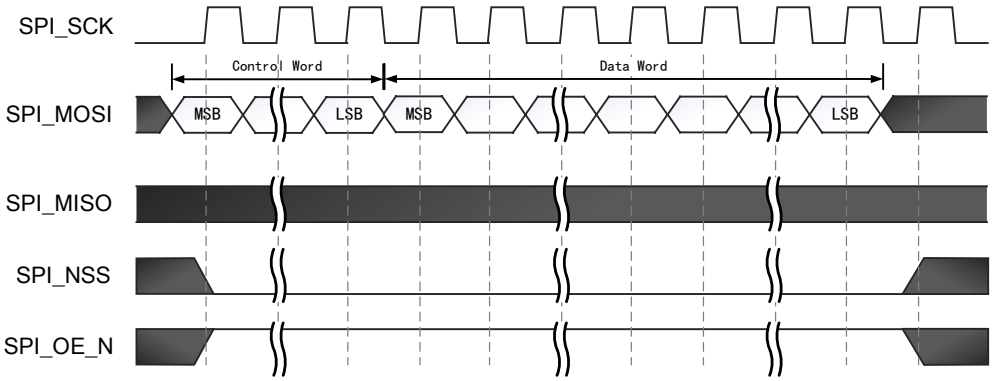
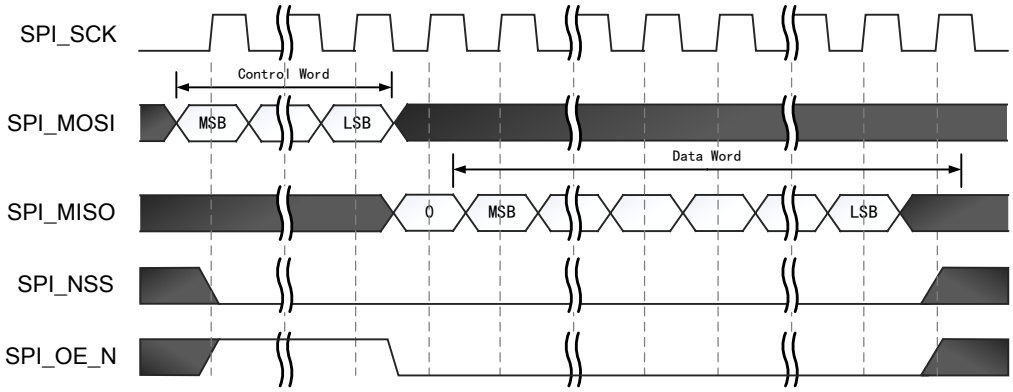


图 21-25 单次写操作（MWHS=0，MDD=1，MWMOD=0）



SPI Slave 的连续传输与 SPI Master 配置指定的方式相同。SPI Slave 不支持握手。

21.6 SPI 工作方式

21.6.1 工作模式

SPI 支持轮询模式、中断模式和 DMA 模式下的数据传输。

21.6.2 时钟与复位

SPI0 的工作时钟为 APB_LS 时钟，最大为 100Mhz；SPI1 的工作时钟为 APB_HS，最大为 200Mhz。每个 SPI 支持独立软复位和时钟门控，具体请参考 4 时钟和复位控制(CRG)。

21.6.3 中断信号处理

SPI 有 5 个独立中断源、可屏蔽、高电平有效。

- RXFIS：接收 FIFO 满中断。
- RXOIS：接收 FIFO 溢出中断。
- RXUIS：接收 FIFO 下溢中断。
- TXOIS：发送 FIFO 溢出中断。
- TXEIS：发送 FIFO 空中断。

SPI 的 5 个原始中断可以通过 RISR 寄存器查询；也可以通过配置 IMR 相应位域进行中断屏蔽，屏蔽后的中断可以通过 ISR 寄存器查询。

21.6.4 初始化

初始化步骤如下：

步骤 1 配置寄存器 CTRLR0.spi_mst_slv_sel，设定 SPI 为 Master 或 Slave。

步骤 2 配置寄存器 SSIENR.ssi_en 为 0，禁止 SPI。

步骤 3 若步骤 1 配置为 Master 模式，需配置 SER.SER 位域，选择片选信号；若步骤 1 配置为 Slave，则跳过此步。

步骤 4 配置寄存器 CTRLR0，配置帧格式及传输数据位宽等参数。

步骤 5 若 TMOD = 10 或者 TMOD = 11，配置寄存器 CTRLR1.ndf 接收数据帧数。

步骤 6 配置寄存器 BAUDR.sckdv，设定时钟分频因子。

步骤 7 配置 TX/RX FIFO 水线寄存器，TXFTLR.tft 和 RXFTLR.rft。

步骤 8 设置寄存器 IMR，使能相应中断信号；

步骤 9 DMA 模式下，设置 DMATDLR 和 DMARDLR 寄存器，用于控制 FIFO 在哪个阈值发送 DMA 请求。

步骤 10 DMA 模式下，配置寄存器 DMACR，使能 SPI 的 DMA 功能。

21.6.5 轮询模式下的数据传输

具体步骤如下：

步骤 1 向寄存器 SSIENR.ssi_en 写 1，使能 SPI。

步骤 2 将需发送的数据连续写到数据寄存器 DRx。

步骤 3 轮询寄存器 SR，直到 SR.busy =0（表示总线不忙）、同时发送方向需要 SR.tfe =1（表示发送 FIFO 已空），而接收方向需要 SR.RFNE =1（表示接收 FIFO 非空）。

步骤 4 读出数据，需保证读空接收 FIFO（可通过查询 SR.RFNE 得到）。

步骤 5 向寄存器 SSIENR.ssi_en 写 0，禁止 SPI。

须知

- Motorola SPI/TI 同步串行接口的全双工特性，每发一个数据就接收一个数据，即使只需发送数据，也需要清空接收 FIFO。

21.6.6 中断模式下的数据传输

具体步骤如下：

步骤 1 向寄存器 SSIENR.ssi_en 写 1，使能 SPI。

步骤 2 将需发送的数据连续写到数据寄存器 DRx。

步骤 3 等待中断，读出数据。循环直到读出所有数据。

步骤 4 向寄存器 SSIENR.ssi_en 写 0，禁止 SPI。

须知

- Motorola SPI/TI 同步串行接口的全双工特性，每发一个数据就接收一个数据，即使只需发送数据，也需要清空接收 FIFO。

21.6.7 DMA 模式下的数据传输

具体步骤如下：

步骤 1 获取一个 DMAC 通道。

步骤 2 向寄存器 SSIENR.ssi_en 写 1，使能 SPI。

步骤 3 发送数据：

1) 配置该发送 DMAC 通道的配置寄存器和控制寄存器中的相关参数。
2) 启动 DMAC，响应 SPI 发送 FIFO 的 DMA 请求进行数据传输。
3) 通过 DMA 中断上报，判断数据是否发送完成，如果完成则关闭 SPI 的 DMA 功能。

步骤 4 接收数据：

1) 配置该接收 DMAC 通道的配置寄存器和控制寄存器中的相关参数。
2) 启动 DMAC，响应 SPI 接收 FIFO 的 DMA 请求进行数据传输。
3) 通过 DMA 中断上报，判断数据是否接收完成，如果完成则关闭 SPI 的 DMA 功能。

步骤 5 向寄存器 SSIENR.ssi_en 写 0，禁止 SPI。

21.7 SPI 寄存器概览

SPI 实例 2 个 SPI 的基地址如下：

- SPI0（LS）寄存器基地址为：0x60030000
- SPI1（HS）寄存器基地址为：0x61007400

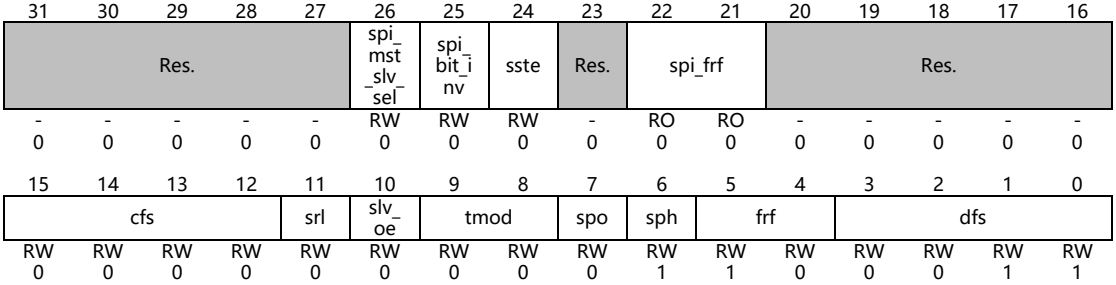
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CTRLR0	0x0	32	RW	0x01000007	SPI 控制寄存器 0
2	CTRLR1	0x4	32	RW	0x00000000	SPI 控制寄存器 1
3	SSIENR	0x8	32	RW	0x00000000	SPI 使能寄存器
4	MWCR	0xC	32	RW	0x00000000	Microwire 模式控制寄存器
5	SER	0x10	32	RW	0x00000000	Slave 使能寄存器
6	BAUDR	0x14	32	RW	0x00000000	波特率配置寄存器
7	TXFTLR	0x18	32	RW	0x00000000	发送 FIFO 水线配置寄存器
8	RXFTLR	0x1C	32	RW	0x00000000	接收 FIFO 水线配置寄存器
9	TXFLR	0x20	32	RO	0x00000000	发送 FIFO 有效数据寄存器
10	RXFLR	0x24	32	RO	0x00000000	接收 FIFO 有效数据寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
11	SR	0x28	32	RO	0x00000006	状态寄存器
12	IMR	0x2C	32	RW	0x0000001F	中断屏蔽寄存器，中断被屏蔽后，不会上报到 ISR 寄存器
13	ISR	0x30	32	RO	0x00000000	中断状态寄存器
14	RISR	0x34	32	RO	0x00000000	原始中断状态寄存器
15	TXOICR	0x38	32	RO	0x00000000	发送 FIFO 溢出中断清除寄存器
16	RXOICR	0x3C	32	RO	0x00000000	接收 FIFO 溢出中断清除寄存器
17	RXUICR	0x40	32	RO	0x00000000	接收 FIFO 下溢中断清除寄存器
18	MSTICR	0x44	32	RO	0x00000000	多 Master 竞争中断清除寄存器
19	ICR	0x48	32	RO	0x00000000	中断清除寄存器
20	DMACR	0x4C	32	RW	0x00000000	DMA 控制寄存器
21	DMATDLR	0x50	32	RW	0x00000000	DMA 发送流水线寄存器
22	DMARDLR	0x54	32	RW	0x00000000	DMA 接收水线寄存器
23	IDR	0x58	32	RO	0x00009528	SPI ID 寄存器
24	SSI_VERSION_ID	0x5C	32	RO	0x3430322A	SPI 版本号寄存器
25	DR	0x60	32	RW	0x00000000	数据寄存器
26	RX_SAMPLE_DLY	0xf0	32	RW	0x00000000	RX 采样延迟配置寄存器

21.8 SPI 寄存器描述

21.8.1 CTRLR0

◆ **偏移地址：0x0　　复位值：0x01000007**



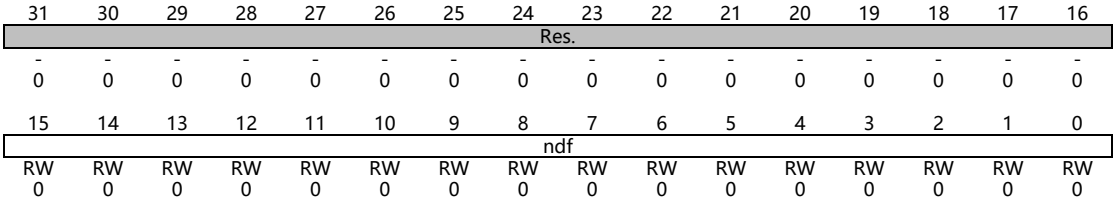
Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留	0x0
spi_mst_slv_sel	[26]	RW	MASTER 与 SLAVE 选择寄存器： 0：SLAVE 1：MASTER	0x0
spi_bit_inv	[25]	RW	SPI 帧反序控制寄存器： 0：SPI 帧不反序 1：SPI 帧反序	0x0
sste	[24]	RW	从设备选择切换启用。当在时钟阶段（sph）设置为 0 的 SPI 模式下操作时，此寄存器控制数据帧之间的从选择线（ss_x_n）的行为。 0：ss_0_n 将保持低电平，sclk 将在传输期间持续运行 1：ss_0_n 电平状态将在连续数据帧之间切换，当 ss_0_n 为高电平时 sclk 保持为默认值 注意：ss_x_n，x 取值范围 0~1	0x1
reserved	[23]	-	保留	0x0
spi_frf	[22:21]	RO	SPI 帧格式： 0：Standard SPI 帧格式（标准 SPI） 1：Dual SPI 帧格式（双线 SPI，当前 IP 不支持）	0x0
reserved	[20:16]	RO	保留	0x0
cfs	[15:12]	RW	控制帧长度，用于在 Microwire 帧格式下选择控制帧长度： 0：1-bit 1：2-bit 2：3-bit 3：4-bit 4：5-bit 5：6-bit 6：7-bit 7：8-bit 8：9-bit 9：10-bit	0x0

Name	Bits	R/W	Description	Default
			10：11-bit 11：12-bit 12：13-bit 13：14-bit 14：15-bit 15：16-bit	
srl	[11]	RW	移位寄存器操作模式。 0：正常模式操作 1：测试模式， Tx & Rx 移位寄存器连接 注意：测试模式仅用于测试目的，将发送移位寄存器输出连接到接收移位寄存器输入。	0x0
slv_oe	[10]	RW	SLAVE 输出使能控制。仅当 SPI 被配置为 SLAVE 才有用。当配置为 MASTER 时，此位字段没有任何功能。此位用于控制 SPI SLAVE 的 ssi_oe_n 输出。当 SLV_OE = 1 时，ssi_oe_n 输出处于非激活状态。ssi_oe_n 控制 SLAVE txd 输出上的三态门，当 SLV_OE = 1 时，SLAVE 端 txd 输出为高阻状。当 MASTER 以广播模式传输（MASTER 将数据传输到所有 SLAVE）时，只有一个 SLAVE 可以响应 MASTER。 0：SLAVE 输出已启用 1：SLAVE 输出已禁用	0x0
tmod	[9:8]	RW	传输模式： 0: Transmit & Receive（发送接收同时模式） 1: Transmit Only （只发送） 2: Receive Only （只接收） 3: EEPROM Read（EEPROM 读）	0x0
spo	[7]	RW	串行时钟极性： 0：串行时钟的非活动状态为低 1：串行时钟的非活动状态为高	0x0
sph	[6]	RW	串行时钟相位： 0：第一个数据位中间的串行时钟切换 1：第一个数据位开始时的串行时钟切换	0x1
frf	[5:4]	RW	帧格式，选择由哪个串行协议传输数据： 0：Motorola 帧格式 1：TI 帧格式 2：MICROWIRE 帧格式 3：保留	0x2
dfs	[3:0]	RW	数据帧大小： 0：保留 1：保留	0x7

Name	Bits	R/W	Description	Default
			2：保留 3：4-bit 串行数据传输 4：5-bit 串行数据传输 5：6-bit 串行数据传输 6：7-bit 串行数据传输 7：8-bit 串行数据传输 8：9-bit 串行数据传输 9：10-bit 串行数据传输 10：11-bit 串行数据传输 11：12-bit 串行数据传输 12：13-bit 串行数据传输 13：14-bit 串行数据传输 14：15-bit 串行数据传输 15：16-bit 串行数据传输	

21.8.2 CTRLR1

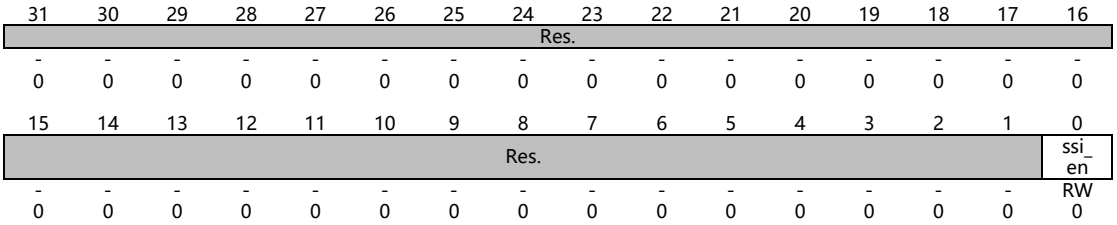
◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
ndf	[15:0]	RW	数据帧个数： 当 CTRLR0.tmod = 2 或 3 时，该寄存器字段设置了 SPI 要连续接收的数据帧数。SPI 继续接收串行数据，直到接收到的数据帧数等于这个寄存器值加上 1。当 SPI 配置为串行从设备时，只要主设备选中该从设备，传输就会持续进行。因此，当 SPI 被配置为从设备时，这个寄存器没有任何作用，也不存在。	0x0

21.8.3 SSIENR

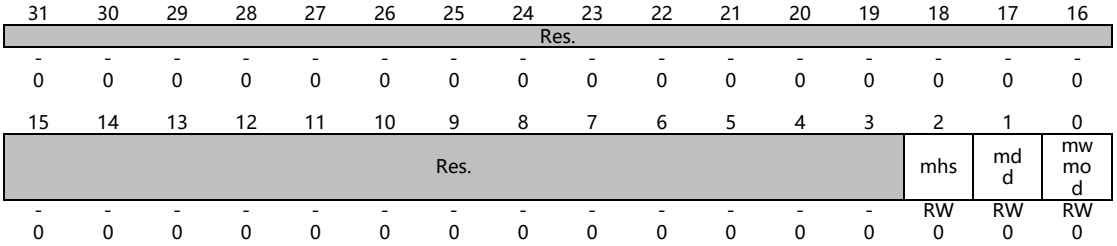
◆ **偏移地址：0x8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
ssi_en	[0]	RW	SPI 使能配置： 0：不使能 1：SPI 使能 SPI 不使能时，所有串行传输都将立即停止。当设备被禁用时，将清除传输和接收 FIFO 缓冲区。当使能时，不能对某些 SPI 控制寄存器进行编程。	0x0

21.8.4 MWCR

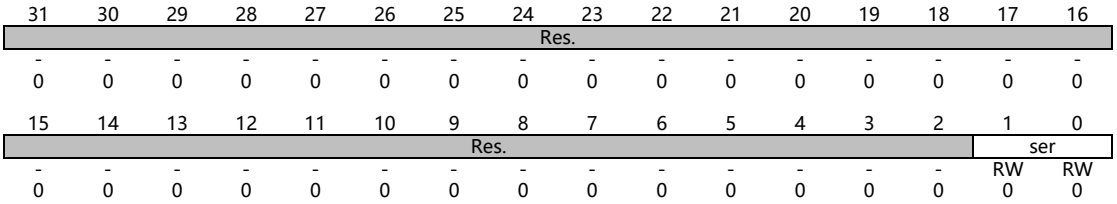
◆ **偏移地址：0xC　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
mhs	[2]	RW	Microwire 握手使能： 0：不启用握手功能 1：启用握手功能 仅当 SPI 被配置为 Master 才启用，当配置为 Slave，此位字段没有任何功能。用于启用和禁用 Microwire 协议的繁忙/准备就绪握手接口。	0x0
mdd	[1]	RW	Microwire 接收发送方向控制： 0：SPI 接收数据 1：SPI 发送数据	0x0
mwmod	[0]	RW	Microwire 传输模式： 0：非连续传输 1：连续传输	0x0

21.8.5 SER

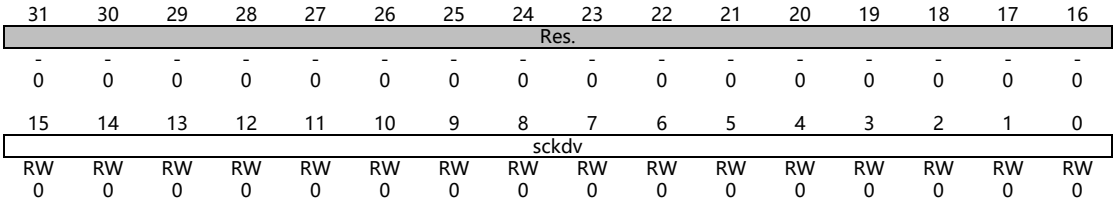
◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
ser	[1:0]	RW	Slave 片选使能配置，其中 bit0 是 Slave 0 选择寄存器；bit1 是 Slave 1 选择寄存器： 0：没有选择此 Slave 1：选择此 Slave	0x0

21.8.6 BAUDR

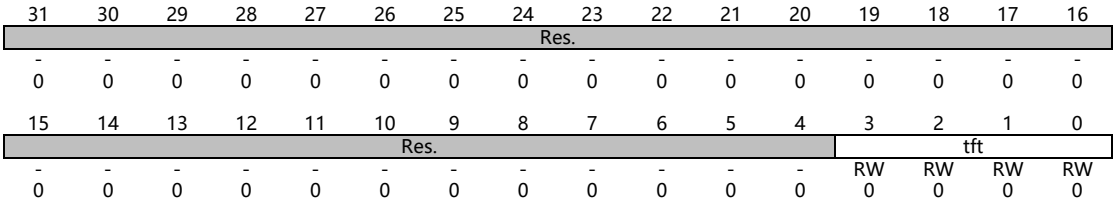
◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
sckdv	[15:0]	RW	SPI 时钟分频器。 此字段的 LSB 总是设置为 0，并且不受写操作的影响，此字段可配置为 2~65534 之间的任意偶数值，如果配置为 0，则会禁用串行输出时钟 SCK。 SCK 的计算公式为：SCK=Fclk_spi/sckdv。 SCK 支持的最大速率请参考表 21-1，使用时请根据 SPI 的工作时钟 Fclk_spi 灵活配置分频系数。例如：当 Fclk_spi=100MHz 且 SPI 工作做在主模式时，根据表 21-1 可知此时 SCK 最大支持 25MHz，则 sckdv 最小可配置为 4，若 sckdv 小于 4 则可能导致数据传输异常。	0x0

21.8.7 TXFTLR

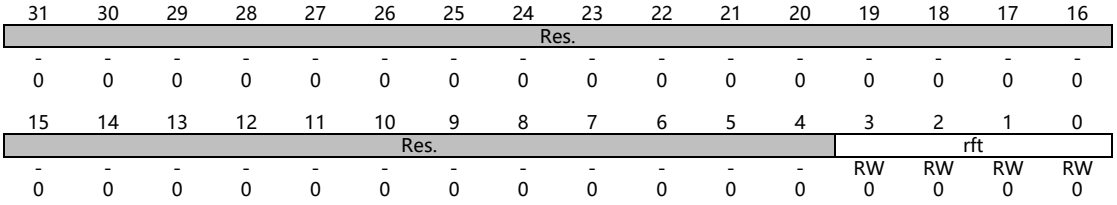
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
tft	[3:0]	RW	发送 FIFO 水线配置	0x0

21.8.8 RXFTLR

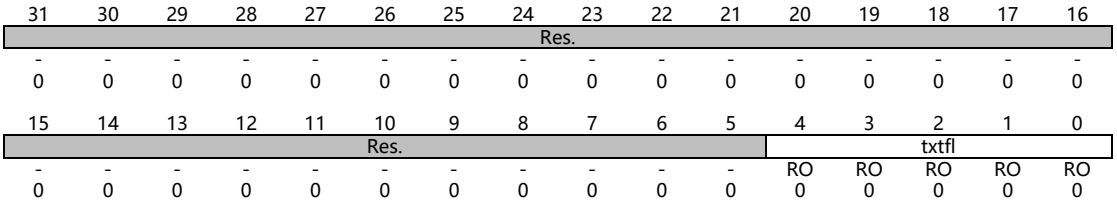
◆ **偏移地址：0x1C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
rft	[3:0]	RW	接收 FIFO 水线配置	0x0

21.8.9 TXFLR

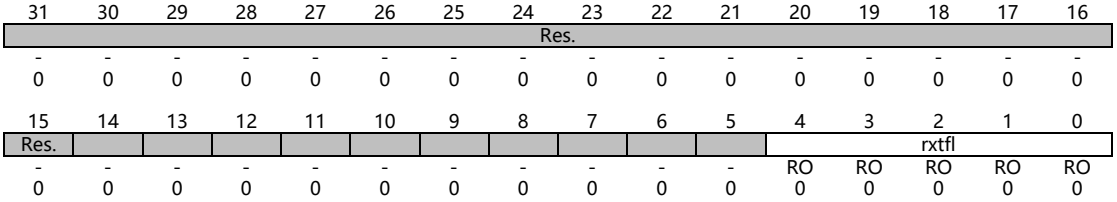
◆ **偏移地址：0x20　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
txtfl	[4:0]	RO	发送 FIFO 中的有效数据数量。	0x0

21.8.10 RXFLR

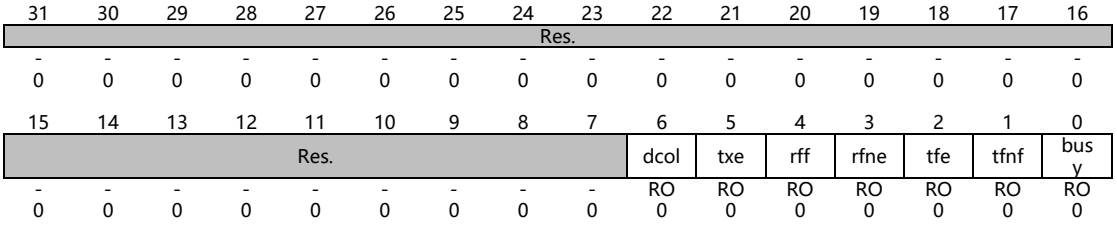
◆ **偏移地址：0x24　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
rxtfl	[4:0]	RO	接收 FIFO 中的有效数据数量。	0x0

21.8.11 SR

◆ **偏移地址：0x28　　复位值：0x00000006**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
dcol	[6]	RO	数据冲突标志： 0：无冲突 1：数据冲突错误 仅当 SPI 被配置为 Master 时有效。当 SPI 配置成 Master 模式并且与 slave 器件正在通信时，其他 Master 尝试访问该 slave，此时产生数据冲突，此位将置 1。	0x0
txe	[5]	RO	发送错误标志： 0：无错误 1：传输错误 注意：此寄存器只在 SPI 配置为 Slave 时有效，当传输开始时，若 FIFO 为空则置位	0x0
rff	[4]	RO	接收 FIFO 满标志： 0：接收 FIFO 未满 1：接收 FIFO 已满	0x0

Name	Bits	R/W	Description	Default
rfne	[3]	RO	接收 FIFO 非空标志： 0：接收 FIFO 为空 1：接收 FIFO 不为空	0x0
tfe	[2]	RO	发送 FIFO 空标志： 0：发送 FIFO 非空 1：发送 FIFO 空	0x1
tfnf	[1]	RO	发送 FIFO 非满标志： 0：发送 FIFO 满 1：发送 FIFO 未满	0x1
busy	[0]	RO	SPI 忙标志： 0：SPI 空闲或禁用 1：SPI 正在传输数据	0x0

21.8.12 IMR

◆ **偏移地址：0x2C　　复位值：0x0000001F**

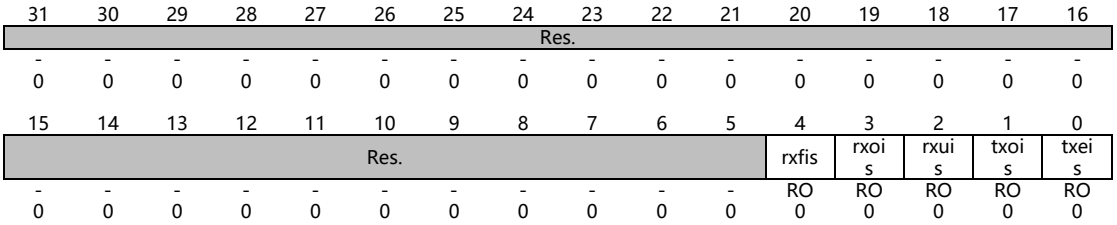
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.											rxfim	rxoim	rxuim	txoim	txeim
-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
rxfim	[4]	RW	RX FIFO 满中断屏蔽： 0：ssi_rxf_intr 中断被屏蔽 1：ssi_rxf_intr 中断未屏蔽	0x1
rxoim	[3]	RW	RX FIFO 溢出中断屏蔽： 0：ssi_rxo_intr 中断被屏蔽 1：ssi_rxo_intr 中断未被屏蔽	0x1
rxuim	[2]	RW	RX FIFO 下溢中断屏蔽： 0：ssi_rxu_intr 中断被屏蔽 1：ssi_rxu_intr 中断未被屏蔽	0x1
txoim	[1]	RW	发送 FIFO 溢出中断屏蔽： 0：ssi_txo_intr 中断被屏蔽 1：ssi_txo_intr 中断未被屏蔽	0x1
txeim	[0]	RW	发送 FOFO 空中断屏蔽： 0：ssi_txe_intr 中断被屏蔽 1：ssi_txe_intr 中断未被屏蔽	0x1

21.8.13 ISR

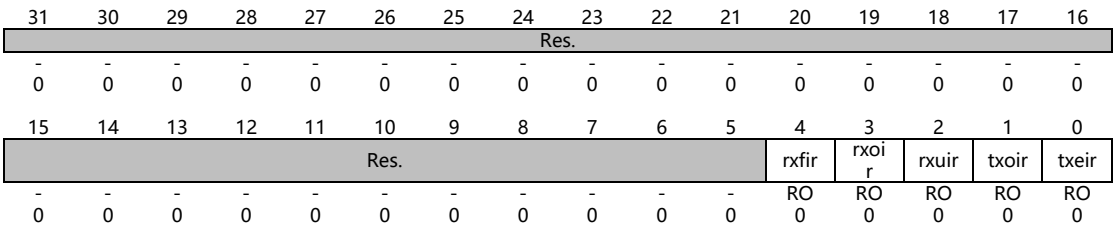
◆ **偏移地址：0x30　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
rxfis	[4]	RO	接收 FIFO 满中断： 0：无中断 1：产生中断	0x0
rxois	[3]	RO	接收 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
rxuis	[2]	RO	接收 FIFO 下溢中断： 0：无中断 1：产生中断	0x0
txois	[1]	RO	发送 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
txeis	[0]	RO	发送 FIFO 空中断： 0：无中断 1：产生中断	0x0

21.8.14 RISR

◆ **偏移地址：0x34　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0

Name	Bits	R/W	Description	Default
rxfir	[4]	RO	接收 FIFO 满中断： 0：无中断 1：产生中断	0x0
rxoir	[3]	RO	接收 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
rxuir	[2]	RO	接收 FIFO 下溢中断： 0：无中断 1：产生中断	0x0
txoir	[1]	RO	发送 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
txeir	[0]	RO	发送 FIFO 空中断： 0：无中断 1：产生中断	0x0

21.8.15 TXOICR

◆ **偏移地址：0x38　　复位值：0x00000000**

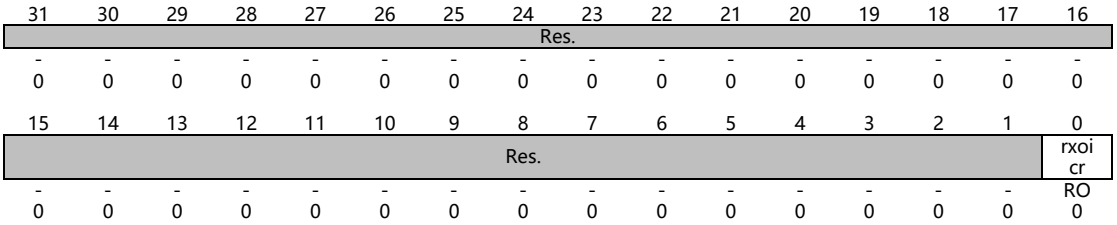
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															txoicr
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txoicr	[0]	RO	这个寄存器反映了中断的状态。从这个寄存器读取会清除 ssi_txo_intr 发送 FIFO 溢出中断；写入没有效果。	0x0

21.8.16 RXOICR

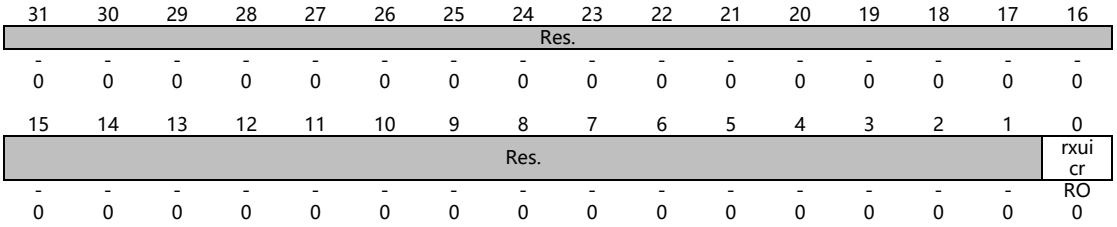
◆ **偏移地址：0x3C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxoicr	[0]	RO	这个寄存器反映了中断的状态。从这个寄存器读取会清除 ssi_rxo_intr 接收 FIFO 溢出中断；写入没有效果。	0x0

21.8.17 RXUICR

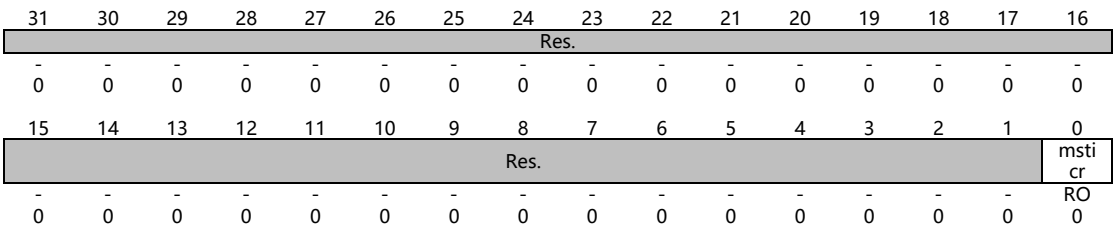
◆ **偏移地址：0x40　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxuicr	[0]	RO	这个寄存器反映了中断的状态。从这个寄存器读取会清除 ssi_rxu_intr 接收 FIFO 下溢中断；写入没有效果。	0x0

21.8.18 MSTICR

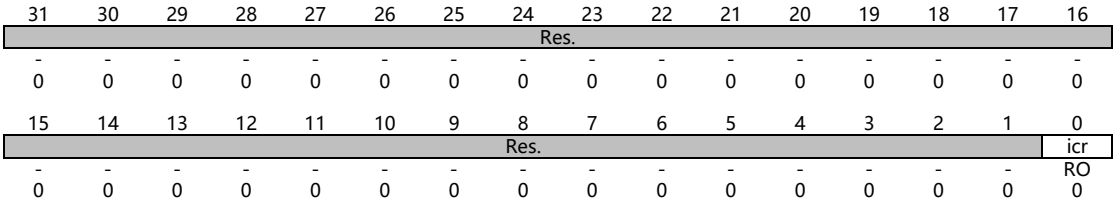
◆ **偏移地址：0x44　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
msticr	[0]	RO	这个寄存器反映了中断的状态。从这个寄存器读取会清除 ssi_mst_intr 多 Master 竞争中断；写入没有效果。	0x0

21.8.19 ICR

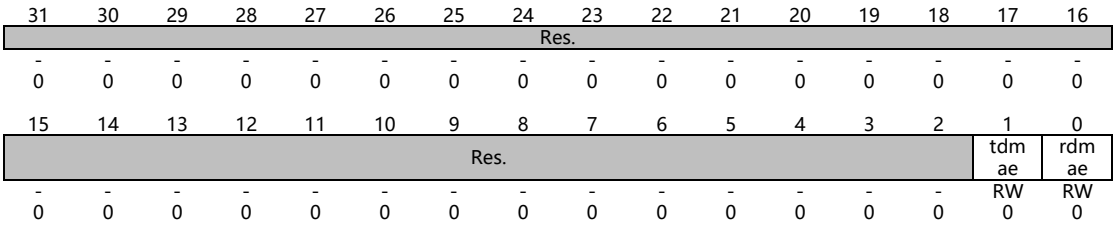
◆ **偏移地址：0x48　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
icr	[0]	RO	如果下面的任何一个中断有效，则此寄存器将上报中断。读取将清除 ssi_txo_intr、ssi_rxu_intr、ssi_rxo_intr 和 ssi_mst_intr 中断。	0x0

21.8.20 DMACR

◆ **偏移地址：0x4C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
tdmae	[1]	RW	发送 DMA 使能： 0：发送 DMA 禁用 1：启用发送 DMA	0x0
rdmae	[0]	RW	接收 DMA 使能： 0：接收 DMA 禁用	0x0

Name	Bits	R/W	Description	Default
			1：接收 DMA 启用	

21.8.21 DMATDLR

◆ **偏移地址：0x50　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												dmatdl			
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
dmatdl	[3:0]	RW	此寄存器配置 DMA 请求发起水线（发送方向），当 FIFO 中数据个数小于等于此水线值时，dma_tx_req 将拉高，向 DMA 发送数据发送请求。	0x0

21.8.22 DMARDLR

◆ **偏移地址：0x54　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												dmardl			
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
dmardl	[3:0]	RW	此寄存器配置 DMA 请求发起水线（接收方向），当 FIFO 中数据个数等于或者大于此寄存器值+1 时，dma_rx_req 将拉高，向 DMA 发送数据搬移请求。	0x0

21.8.23 IDR

◆ **偏移地址：0x58　　复位值：0x00009528**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IDCODE															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
idcode															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
idcode	[31:0]	RO	SPI ID 号	0x9528

21.8.24 SSI_VERSION_ID

◆ **偏移地址：0x5C　　复位值：0x3430322A**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ssi_comp_version															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ssi_comp_version															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ssi_comp_version	[31:0]	RO	SPI 版本号	0x3430322A

21.8.25 DR

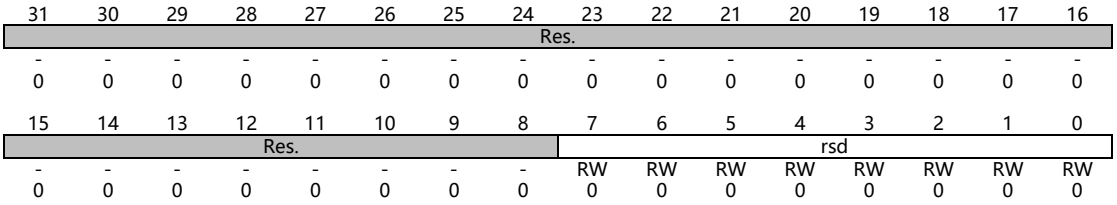
◆ **偏移地址：0x60　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dr															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
dr	[15:0]	RW	数据寄存器，右对齐。	0x0

21.8.26 RX_SAMPLE_DLY

◆ **偏移地址：0xf0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
rsd	[7:0]	RW	Rxd 延迟配置寄存器，此寄存器用于 rxd 输入数据的延迟打拍。每个值代表 rxd 样本上的一个 ssi_clk 延迟。	0x0

22 扩展同步串行外设接口（QSPI）

22.1 QSPI 简介

QSPI 扩展自 SPI，在 SPI 基础上支持 Enhanced 协议，包括 2/4 线 SPI 及指令/地址等模式，支持 XIP 功能。

说明

- QSPI 具备 SPI 全部功能（QSPI 仅支持主模式除外），QSPI 与 SPI 的差异及 QSPI 扩展功能将在本章节进行介绍。

22.2 QSPI 主要特性

在 SPI 基础上，QSPI 扩展功能及与 SPI 差异包括：

- 仅支持主模式，不支持从模式。主模式与 SPI 一样支持 RXD 采样延迟，具体见 SPI 的 21.4.3 RX 采样延迟章节。
- 支持 Dual/Quad SPI。
- 支持 4bit ~ 32bit 可编程串行数据帧长度。
- 支持 TX(发送)FIFO 深度为 8，RX(接收)FIFO 深度为 16，XIP FIFO 深度为 16。
- 支持增强型 SPI 模式，包括指令阶段与地址阶段。
- 支持 Execute in Place (XIP)模式。
- 支持 XIP 模式下可编程指令、地址长度、等待周期和数据帧大小。
- 支持 XIP 模式下连续传输模式。
- 支持 XIP 模式下预取功能。
- 支持 XIP 模式下并行传输模式。

- 支持 XIP 模式下 Mode Bits（模式位）发送功能。

22.3 QSPI 逻辑实现

表 22-1 介绍了芯片 QSPI 模块在器件中所具备的功能，及与 SPI 的差异。

表 22-1 QSPI 实现及与 SPI 差别

SPI 特性	SPI0~1	QSPI
支持 TI 模式	YES	YES
支持 MOTO 模式	YES	YES
支持 MW 模式	YES	YES
最高串行输出时钟	主模式：25Mhz 从模式：20Mhz	主模式：25Mhz
2/4 线模式	NO	YES
增强型 SPI 模式	NO	YES
XIP 模式	NO	YES
数据帧长	4~16	4~32
FIFO 深度	TX FIFO:16 RX FIFO:16	TX FIFO:8 RX FIFO:16 XIP FIFO:16

⚠注意

- 当 QSPI 的串行输出时设置比较高的时候，中断和轮询模式可能不能及时处理数据，将导致数据传输异常，建议使用 DMA 模式搬运数据。

22.4 QSPI 功能描述

22.4.1 时钟与复位

QSPI 的工作时钟为 APB_HS，最大为 200Mhz。QSPI 支持独立软复位和时钟门控，具体请参考 4 时钟和复位控制（CRG)章节。

22.4.2 标准 SPI 模式

QSPI 模块支持标准 SPI 模式，使用方法和 SPI 一样，详见 21 同步串行外设接口(SPI)相关

章节。

22.4.3 增强型 SPI 模式

增强型 SPI 模式是指 QSPI 支持多线（2/4 线）模式与外部器件通信，并且具备可编程的指令、地址长度、等待周期和数据帧大小。

须知

- QSPI 模块支持增强型 SPI 模式和标准 SPI 模式，SPI 模块仅支持标准 SPI 模式。

1. Execute in Place（XIP）模式

QSPI 提供了可以直接通过 AHB 总线操作执行内存读取的功能，即 XIP 模式。其中 QSPI 充当到 FLASH 的内存映射接口。

⚠注意

- 仅支持 XIP 读操作，不支持 XIP 写操作。XIP 操作仅在 2/4 线模式中支持。

2. XIP 与非 XIP 并行操作

QSPI 模块在 TX FIFO 与 RX FIFO 之外，还有一个 XIP FIFO，它的存在确保了 XIP 与非 XIP 传输数据之间的独立性。

以实际场景举例：在一个系统内，QSPI 与多个总线 Master 对接，Master-A 负责 DMA 传输，Master-B 负责 XIP 传输，这两个 Master 都可以在任何时候进行传输请求。

DMA 数据传输启动前（Master-A 发出的非 XIP 操作），QSPI 需要等待 FIFO 中存储的数据达到水线，确保整个 DMA 传输在一次操作中完成。在等待 FIFO 达到阈值的过程中，若 Master-B 发出 XIP 事务，QSPI 将立即响应，不必等待 DMA 操作完成。

3. XIP 连续传输模式

QSPI 在接收到一个 XIP 请求时，将来自 AHB 接口上的地址直接传输到 SPI 串行接口上。若不使能 XIP 连续传输模式，每个新的 XIP 操作都需要经过指令、地址、数据阶段，这导致了系统延迟较大。通过使用 XIP 连续传输模式，可以将两个或者多个 AHB 突发传输请求合并到一个 SPI 操作中，多个操作仅需要传输一次指令与地址，因此极大提高传输效率。

要实现 XIP 连续传输，不仅要通过配置使能连续传输，也需要满足以下条件，否则将退出连续传输模式：

1) 在 XIP 接口上接收到非 XIP 命令

2) 当 AHB 事务是一个非连续的地址时，片选信号将失效，然后 QSPI 启动一个新的 XIP

请求

4. 数据预取功能

通过使能 QSPI 中的数据预取功能，控制器可以预取当前 XIP 事务中连续传输的数据。如果下一个传输请求是对连续地址发出的，则可以直接从 RX FIFO 读取数据，而不需要等待将新的地址和数据发送到设备。这提高了系统的整体性能。

要预取的数据量应等于最后一个 AHB 请求的突发长度或 FIFO 深度（以较低者为准）。例如，如果 AHB 定义了从地址 0x00 开始的 16 个突发长度，那么 QSPI 获取 16 个数据帧来完成当前的传输，然后将再次获取 16 个数据帧并保存到数据寄存器中。如果 AHB 总线再次请求数据且数据地址是连续的（即地址是从上次传输结束的地址开始），那么请求的数据将从 RX FIFO 本身直接发送到设备；同时，QSPI 再次启动 XIP 传输，以预取下一个数据块。如果 AHB 总线没有请求连续地址的数据，则当前预取的数据将从 FIFO 中刷新，并且 QSPI 控制器将启动一个新的 XIP 传输。

22.5 QSPI 接口时序

图 22-1 ~ 图 22-8 中的缩略语含义为：

- MSB：Most Significant Bit（最高有效位）
- LSB：Least Significant Bit（最低有效位）
- Q：Q is an undefined signal（未定义信号）

⚠注意

时序图中的 SPI_CS_N 为配置 SER.ser 位域后的 SPI 片选信号。

22.5.1 增强型 SPI 接口时序

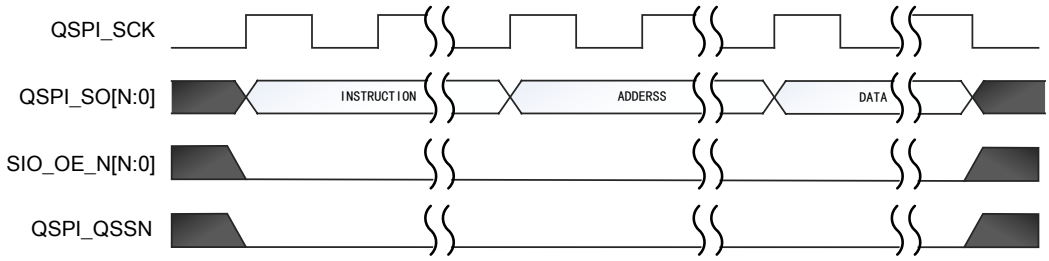
当 QSPI 被配置为 Dual、Quad 模式时，txd、rxd 和 ssi_oe_n 信号也都将相应变为 2 线或 4 线，它的工作原理与标准 SPI 模式相同。

1. 增强型 SPI 写操作

增强型 SPI 写操作被分为三个部分：指令阶段、地址阶段、数据阶段。

一条指令占用一个 FIFO 位置，而地址可以占用多个 FIFO 位置。指令和地址都必须在数据寄存器（DR）中进行编程，QSPI 控制器等待两者都被编程以启动写入操作，具体启动时机由 TXFTLR.txfthr 寄存器控制。

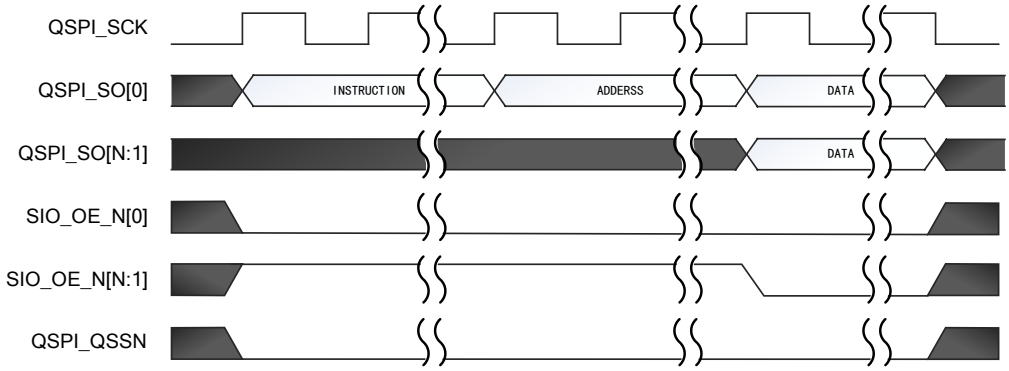
图 22-1 典型的 2/4 线增强型 SPI 模式写操作



Dual 模式时，N 为 1；Quad 模式时，N 为 3。

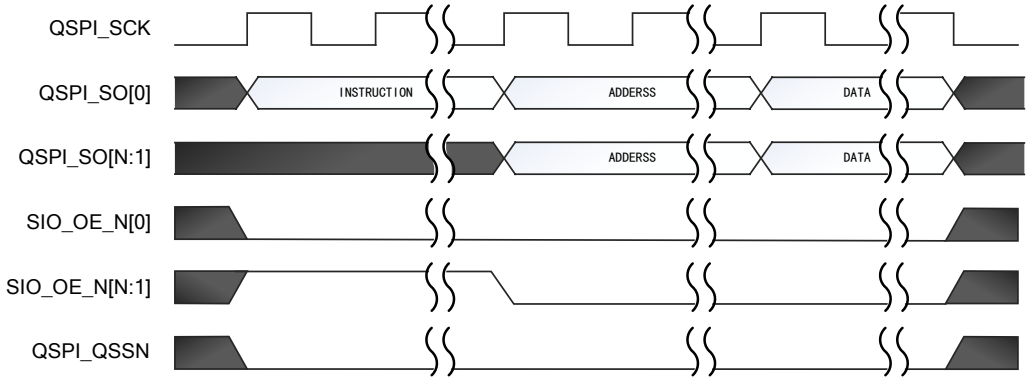
指令和地址，不仅可以如上图 22-1 所示都采用 2/4 线传输（指令/地址/数据都以增强型 SPI 传输），也可以分别设置为标准、Dual 或者 Quad 模式传输，具体的模式设置请参考 QSPI_CTRLR0.trans_type 寄存器。

图 22-2 指令和地址采用 Standard SPI 格式传输



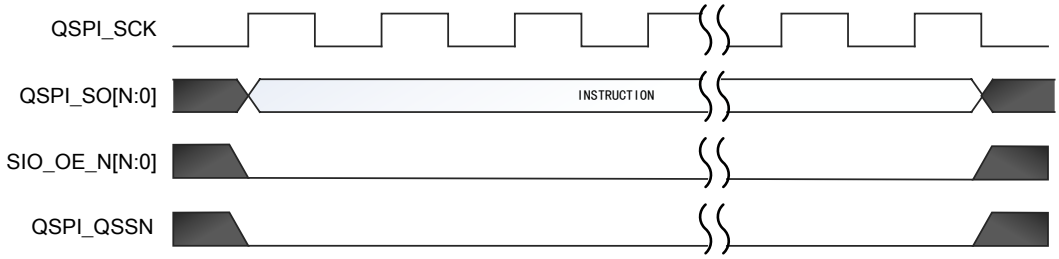
上图 22-2 显示了指令和地址都以标准 SPI 格式传输，数据以增强型 SPI 传输时的时序图。

图 22-3 指令采用 Standard SPI，地址采用增强型 SPI 传输



上图 22-3 显示了指令以标准 SPI 格式传输，地址和数据以增强型 SPI 传输时的时序图。

图 22-4 只传输指令模式



上图 22-4 显示了仅传输指令的增强型 SPI 传输时序图。

2. 增强型 SPI 读操作

增强型 SPI 读操作分为如下四个部分：指令阶段、地址阶段、等待阶段、数据阶段，其中等待周期根据对接的不同器件而不同。

对于读操作，QSPI 控制器发送一次指令和控制数据，直到它接收到 CTRLR1.ndf 数据帧数，然后取消 Slave 选择信号。

图 22-5 指令和地址都采用 Standard SPI 帧格式

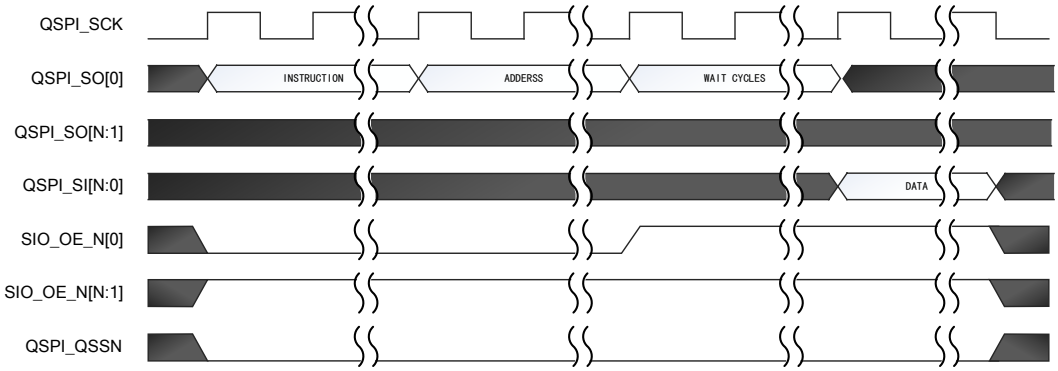


图 22-6 指令采用 Standard SPI 帧格式，地址采用增强 SPI 格式

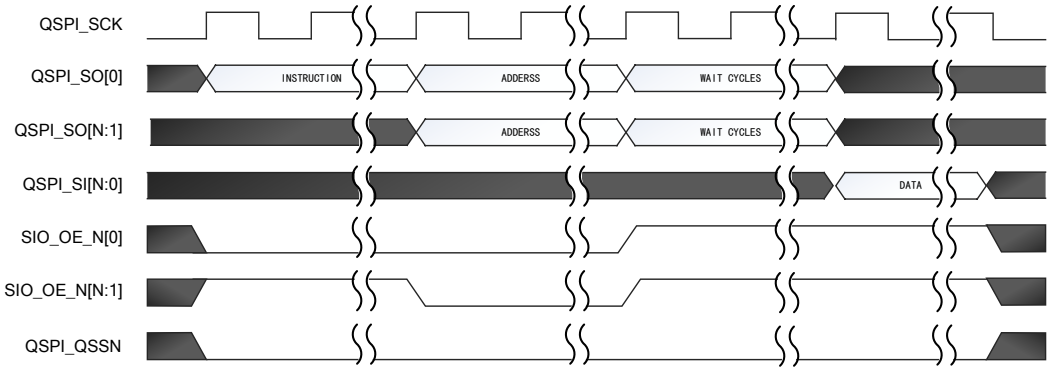


图 22-7 指令和地址都采用增强 SPI 帧格式

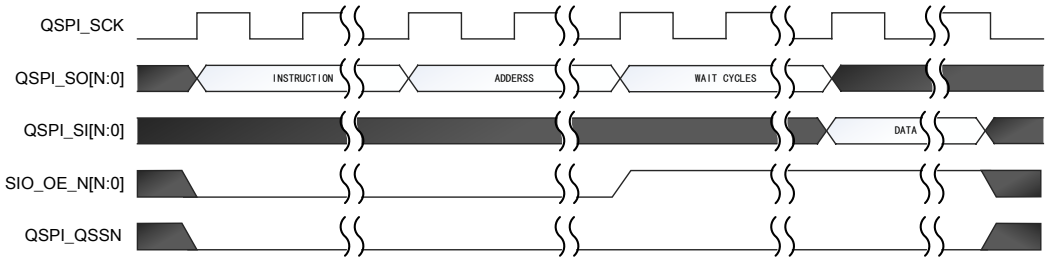
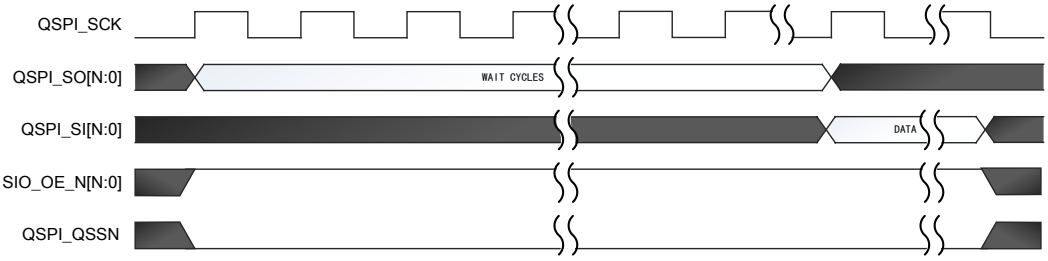


图 22-8 无指令与地址读操作



22.5.2 初始化配置流程

初始化步骤如下：

步骤 1 配置寄存器 SSIENR.ssic_en 为 0，禁止 QSPI。

步骤 2 配置寄存器 CTRLR0，设置 QSPI 帧格式、传输数据位宽等参数；

步骤 3 配置寄存器 BAUDR.sckdv，设置时钟分频参数；

步骤 4 配置 TXFTLR 和 RXFTLR 水线寄存器，设置 TX/RX FIFO 阈值；

步骤 5 配置 IMR 中断屏蔽寄存器，使能相应的中断；

步骤 6 配置 SER.ser 为 1，使能片选信号；

步骤 7 根据发送给 Flash 器件的命令，配置 CTRLR0，从而实现对指令长度，地址长度，dummy cycle 等的配置；

步骤 8 DMA 模式下，设置 DMATDLR 和 DMARDLR 寄存器，用于控制 FIFO 在哪个阈值发送 DMA 请求；

步骤 9 DMA 模式下，配置寄存器 DMACR，使能 QSPI 的 DMA 功能；

步骤 10 配置 SSIENR.ssic_en 为 1，使能 QSPI；

步骤 11 配置 DR 寄存器，写入要发送给 Flash 器件的命令；

步骤 12 查询 SR.busy 状态，获取 QSPI 的工作状态；

XIP 相关功能还需要配置 XIP_INCR_INST，XIP_WRAP_INST，XIP_CTRL，XIP_SER 等寄

存器。

22.6 QSPI 寄存器概览

QSPI 寄存器基地址如下：

- QSPI 寄存器基地址为：0x65000000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CTRLR0	0x0	32	RW	0x0080441F	QSPI 控制寄存器 0
2	CTRLR1	0x4	32	RW	0x00000000	QSPI 控制寄存器 1
3	SSIENR	0x8	32	RW	0x00000001	QSPI 使能寄存器
4	MWCR	0xC	32	RW	0x00000000	Microwire 模式控制寄存器
5	SER	0x10	32	RW	0x00000001	Slave 使能寄存器
6	BAUDR	0x14	32	RW	0x00000002	波特率配置寄存器
7	TXFTLR	0x18	32	RW	0x00000000	发送 FIFO 水线配置寄存器
8	RXFTLR	0x1C	32	RW	0x00000000	接收 FIFO 水线配置寄存器
9	TXFLR	0x20	32	RO	0x00000000	发送 FIFO 有效数据寄存器
10	RXFLR	0x24	32	RO	0x00000000	接收 FIFO 有效数据寄存器
11	SR	0x28	32	RO	0x00000006	状态寄存器
12	IMR	0x2C	32	RW	0x0000007F	中断屏蔽寄存器，中断被屏蔽后，不会上报到 ISR 寄存器
13	ISR	0x30	32	RO	0x00000005	中断状态寄存器
14	RISR	0x34	32	RO	0x00000005	原始中断状态寄存器
15	TXOICR	0x38	32	RO	0x00000000	发送 FIFO 溢出中断清除寄存器
16	RXOICR	0x3C	32	RO	0x00000000	接收 FIFO 溢出中断清除寄存器
17	RXUICR	0x40	32	RO	0x00000001	接收 FIFO 下溢中断清除寄存器
18	MSTICR	0x44	32	RO	0x00000000	多 Master 竞争中断清除寄存器
19	ICR	0x48	32	RO	0x00000000	中断清除寄存器
20	DMACR	0x4C	32	RW	0x00000000	DMA 控制寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
21	DMATDLR	0x50	32	RW	0x00000000	DMA 发送水线寄存器
22	DMARDLR	0x54	32	RW	0x00000000	DMA 接收水线寄存器
23	IDR	0x58	32	RO	0x00009529	QSPI ID 寄存器
24	SSI_VERSION_ID	0x5C	32	RO	0x3130312A	QSPI 版本号寄存器
25	DR	0x60	32	RW	0x00000000	数据寄存器
26	RX_SAMPLE_DLY	0xf0	32	RW	0x00000000	RX 采样延迟配置寄存器
27	QSPI_CTRLR0	0xF4	32	RW	0x00000200	QSPI 控制寄存器
28	XIP_MODE_BITS	0xFC	32	RW	0x00000000	XIP 模式寄存器
29	XIP_INCR_INST	0x100	32	RW	0x00000000	XIP INCR 传输模式操作码寄存器
30	XIP_WRAP_INST	0x104	32	RW	0x00000000	XIP WRAP 传输模式操作码寄存器
31	XIP_CTRL	0x108	32	RW	0x08440482	XIP 模式控制寄存器
32	XIP_SER	0x10C	32	RW	0x00000000	Slave 使能寄存器
33	XRXOICR	0x110	32	RW	0x00000000	XIP 接收 FIFO 溢出中断清除寄存器
34	XIP_CNT_TIME_OUT	0x114	32	RW	0x00000000	XIP 连续传输超时寄存器

22.7 QSPI 寄存器描述

22.7.1 CTRLR0

◆ **偏移地址：0x0　　复位值：0x0080441F**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.								spi_frf		Res.		cfs			
-	-	-	-	-	-	-	-	RW	RW	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	sste	srl	Res.	tmod		spo	sph	frf		Res.	dfs				
-	RW	RW	-	RW	RW	RW	RW	RW	RW	-	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	1	1	0	0	0	0	0	1	1

Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留	0x0
spi_frf	[23:22]	RW	QSPI 帧格式： 0：Standard SPI 格式 1：Dual SPI 格式 2：Quad SPI 格式	0x2

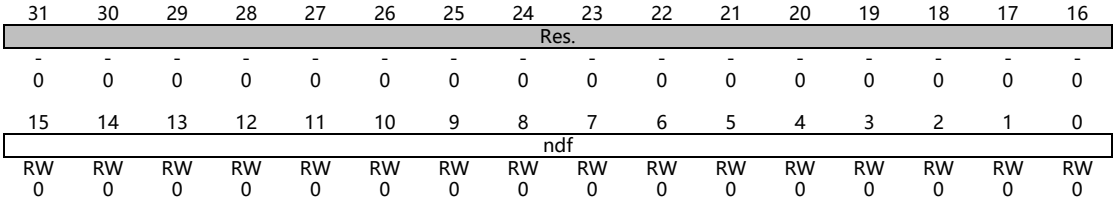
Name	Bits	R/W	Description	Default
reserved	[21:20]	-	保留	0x0
cfs	[19:16]	RW	控制帧长度，用于在 Microwire 帧格式下选择控制帧长度： 0：1-bit 1：2-bit 2：3-bit 3：4-bit 4：5-bit 5：6-bit 6：7-bit 7：8-bit 8：9-bit 9：10-bit 10：11-bit 11：12-bit 12：13-bit 13：14-bit 14：15-bit 15：16-bit	0x0
reserved	[15]	-	保留	0x0
sste	[14]	RW	片选信号翻转配置： 在时钟阶段（sph）设置为 0 的 SPI 模式下操作时，此寄存器控制数据帧之间的从属选择线（ss_0_n）的行为。 0：ss_0_n 将保持低电平，sclk 将在传输期间持续运行 1：ss_0_n 电平状态将在连续数据帧之间切换，当 ss_0_n 为高电平时 sclk 保持为默认值	0x1
srl	[13]	RW	移位寄存器循环。 仅用于测试目的。将发送移位寄存器输出连接到接收移位寄存器输入。 0：正常模式操作 1：测试模式： Tx & Rx 移位寄存器连接	0x0
reserved	[12]	-	保留	0x0
tmod	[11:10]	RW	传输模式： 0：发送和接收；不适用于增强型 SPI 操作模式（SPI） 1：仅发送模式（SPI）；或增强型 SPI 写操作模式（QSPI） 2：仅接收模式（SPI）；或增强型 SPI 读操作模式（QSPI）	0x1

Name	Bits	R/W	Description	Default
			3：EEPROM 读取模式；不适用于增强 SPI 操作模式（SPI）	
spo	[9]	RW	串行时钟极性： 0：串行时钟的非活动状态为低（SCLK_LOW） 1：串行时钟的非活动状态为高（SCLK_HIGH）	0x0
sph	[8]	RW	串行时钟相位： 0：第一个数据位中间的串行时钟切换 1：第一个数据位开始时的串行时钟切换	0x0
frf	[7:6]	RW	帧格式，选择由哪个串行协议传输数据： 0：Motorola SPI 帧格式 1：TI SSP 帧格式 2：National Microwire 帧格式 3：保留	0x0
reserved	[5]	-	保留	0x0
dfs	[4:0]	RW	数据帧大小： spi_frf=1（dual），则 dfs 值必须为 2 的倍数； spi_frf=2（quad），则 dfs 值必须为 4 的倍数； 0：保留 1：保留 2：保留 3: 4-bit 串行数据传输 4: 5-bit 串行数据传输 5: 6-bit 串行数据传输 6: 7-bit 串行数据传输 7: 8-bit 串行数据传输 8: 9-bit 串行数据传输 9: 10-bit 串行数据传输 10：11-bit 串行数据传输 11：12-bit 串行数据传输 12：13-bit 串行数据传输 13：14-bit 串行数据传输 14：15-bit 串行数据传输 15：16-bit 串行数据传输 16：17-bit 串行数据传输 17：18-bit 串行数据传输 18：19-bit 串行数据传输 19：20-bit 串行数据传输 20：21-bit 串行数据传输 21：22-bit 串行数据传输 22：23-bit 串行数据传输 23：24-bit 串行数据传输 24：25-bit 串行数据传输	0x1F

Name	Bits	R/W	Description	Default
			25：26-bit 串行数据传输 26：27-bit 串行数据传输 27：28-bit 串行数据传输 28：29-bit 串行数据传输 29：30-bit 串行数据传输 30：31-bit 串行数据传输 31：32-bit 串行数据传输	

22.7.2 CTRLR1

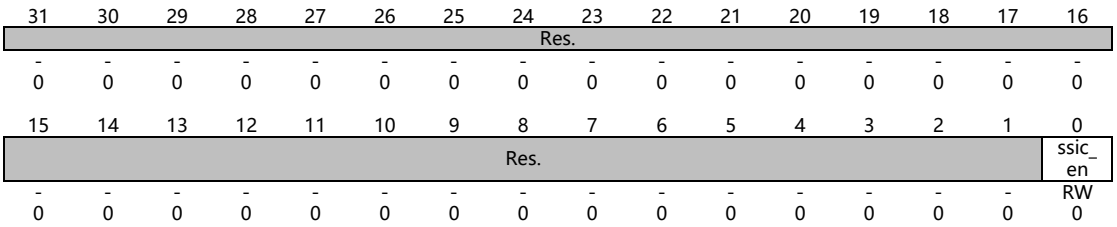
◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
ndf	[15:0]	RW	数据帧个数： 当 CTRLR0.tmod = 2 或 TMOD = 3 时，该寄存器字段设置了 QSPI 要连续接收的数据帧数。	0x0

22.7.3 SSIENR

◆ **偏移地址：0x8　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
ssic_en	[0]	RW	QSPI 使能配置： 0：不使能 1：使能	0x1

Name	Bits	R/W	Description	Default
			QSPI 不使能时，所有串行传输都将立即停止。当设备被禁用时，将清除传输和接收 FIFO 缓冲区。当使能时，不能对某些 QSPI 控制寄存器进行编程。	

22.7.4 MWCR

◆ **偏移地址：0xC　　复位值：0x00000000**

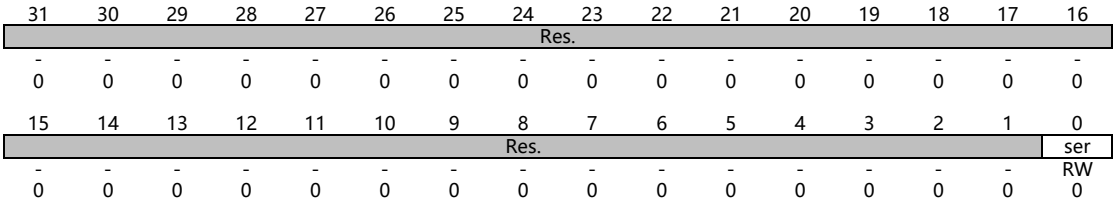
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													mhs	mdd	mwmod
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
mhs	[2]	RW	Microwire 握手使能： 0：不启用握手功能 1：启用握手功能 仅当 QSPI 被配置为 Master 才启用，当配置为 Slave，此位字段没有任何功能。用于启用和禁用 Microwire 协议的繁忙/准备就绪握手接口。	0x0
mdd	[1]	RW	Microwire 接收发送方向控制： 0：接收数据 1：发送数据	0x0
mwmod	[0]	RW	Microwire 传输模式： 0：非连续传输 1：连续传输	0x0

22.7.5 SER

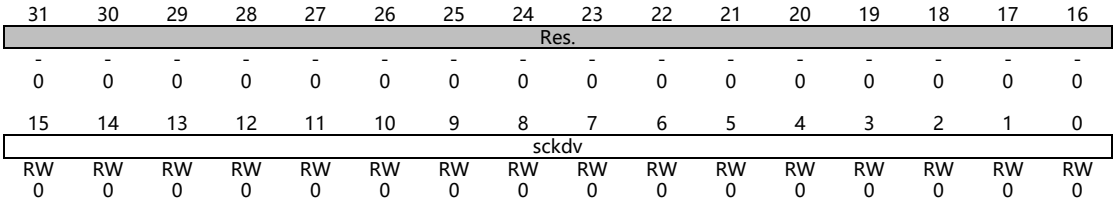
◆ **偏移地址：0x10　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
ser	[0]	RW	Slave 使能配置： 0：没有选择此 Slave 1：选择此 Slave	0x1

22.7.6 BAUDR

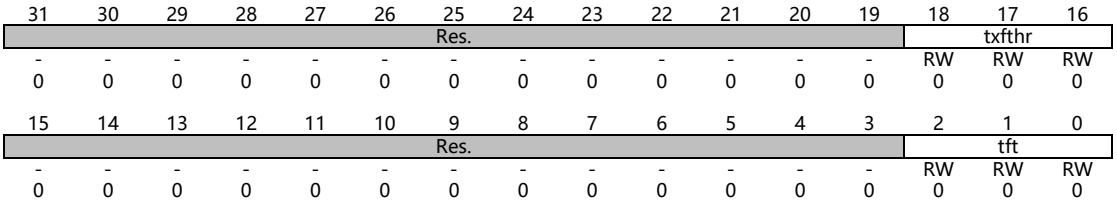
◆ **偏移地址：0x14　　复位值：0x00000002**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
sckdv	[15:0]	RW	QSPI 时钟分频器。 此字段的 LSB 总是设置为 0，并且不受写操作的影响，此字段可配置为 2~65534 之间的任意偶数值，如果配置为 0，则会禁用串行输出时钟 SCK。 SCK 的计算公式为：SCK=Fclk_qspi/sckdv。 SCK 支持的最大速率请参考表 22-1，使用时请根据 QSPI 的工作时钟 Fclk_qspi 灵活配置分频系数。例如：当 Fclk_qspi=200MHz 且 QSPI 工作做在主模式时，根据表 22-1 可知此时 SCK 最大支持 25MHz ，则 sckdv 最小可配置为 8，若 sckdv 小于 8 则可能导致数据传输异常。	0x2

22.7.7 TXFTLR

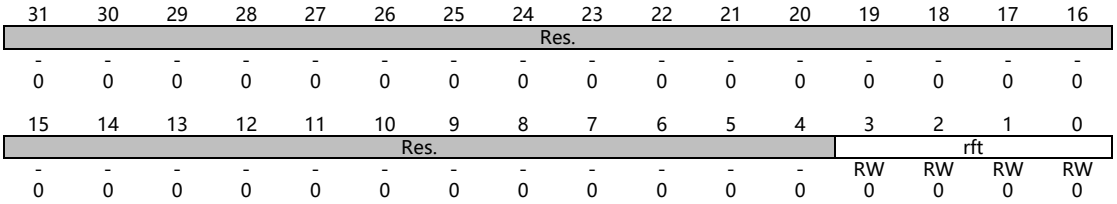
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留	0x0
txfthr	[18:16]	RW	发送 FIFO 传输开始水线配置： 用于控制发送 FIFO 在什么时候启动数据传输，当发送 FIFO 内的数据量超过该水线时，才能启动传输。此寄存器可确保在传输开始之前， 发送 FIFO 中存在足够的数据。仅对主操作模式有效。	0x0
reserved	[15:3]	-	保留	0x0
tft	[2:0]	RW	发送 FIFO 水线配置	0x0

22.7.8 RXFTLR

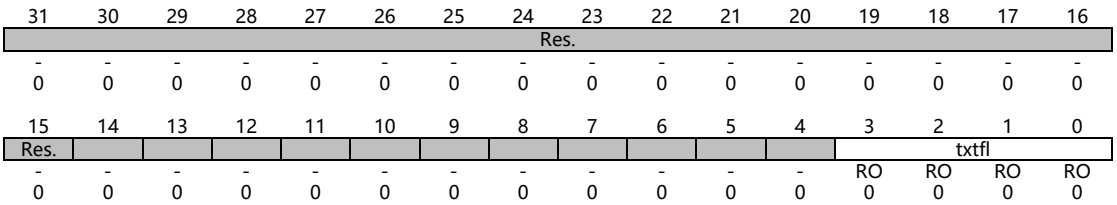
◆ **偏移地址：0x1C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
rft	[3:0]	RW	接收 FIFO 水线配置	0x0

22.7.9 TXFLR

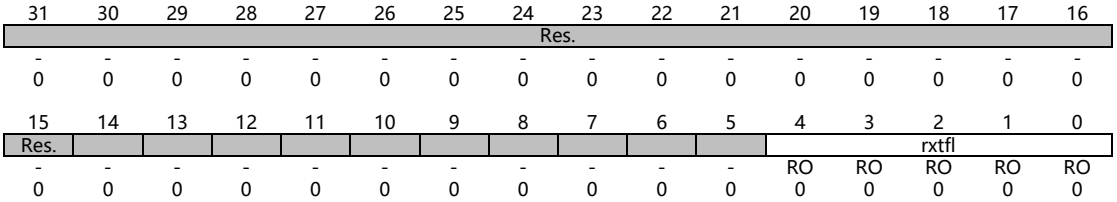
◆ **偏移地址：0x20　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
txtfl	[3:0]	RO	发送 FIFO 中的有效数据的数量。	0x0

22.7.10 RXFLR

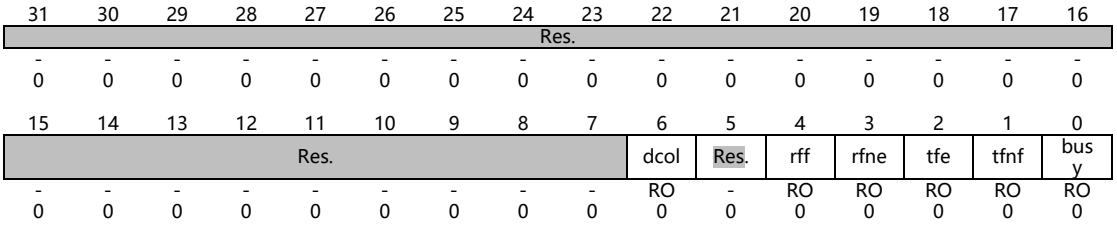
◆ **偏移地址：0x24　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
rxtfl	[4:0]	RO	接收 FIFO 中的有效数据的数量。	0x0

22.7.11 SR

◆ **偏移地址：0x28　　复位值：0x00000006**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
dcol	[6]	RO	数据冲突标志： 0：无冲突 1：数据冲突错误 当 QSPI 配置成 Master 模式并且与 slave 器件正在通信时，其他 Master 尝试访问该 slave，此时产生数据冲突，此位将置 1。	0x0
reserved	[5]	-	保留。	0x0
rff	[4]	RO	接收 FIFO 满标志： 0：接收 FIFO 未满 1：接收 FIFO 已满	0x0
rfne	[3]	RO	接收 FIFO 非空标志：	0x0

Name	Bits	R/W	Description	Default
			0：接收 FIFO 为空 1：接收 FIFO 不为空	
tfe	[2]	RO	发送 FIFO 空标志： 0：发送 FIFO 不是空的 1：发送 FIFO 是空的	0x1
tfnf	[1]	RO	发送 FIFO 非满标志： 0：发送 FIFO 已满 1：发送 FIFO 未满	0x1
busy	[0]	RO	QSPI 忙标志： 0：QSPI 空闲或不活动 1：QSPI 正在传输数据	0x0

22.7.12 IMR

◆ **偏移地址：0x2C　　复位值：0x00000007F**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.									xrxoim	mstim	rxfim	rxoim	rxuim	txoim	txeim
-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
xrxoim	[6]	RW	XIP 接收 FIFO 溢出中断屏蔽： 0：ssi_xrxo_intr 中断被屏蔽 1：ssi_xrxo_intr 中断未被屏蔽	0x1
mstim	[5]	RW	多 Master 竞争中断上报屏蔽： 0：ssi_mst_intr 中断被屏蔽 1：ssi_mst_intr 中断未被屏蔽	0x1
rxfim	[4]	RW	RX FIFO 满中断屏蔽： 0：ssi_rxf_intr 中断被屏蔽 1：ssi_rxf_intr 中断未被屏蔽	0x1
rxoim	[3]	RW	RX FIFO 溢出中断屏蔽： 0：ssi_rxo_intr 中断被屏蔽 1：ssi_rxo_intr 中断未被屏蔽	0x1
rxuim	[2]	RW	RX FIFO 下溢中断屏蔽： 0：ssi_rxu_intr 中断被屏蔽 1：ssi_rxu_intr 中断未被屏蔽	0x1
txoim	[1]	RW	发送 FIFO 溢出中断屏蔽：	0x1

Name	Bits	R/W	Description	Default
			0：ssi_txo_intr 中断被屏蔽 1：ssi_txo_intr 中断未被屏蔽	
txeim	[0]	RW	发送 FOFO 空中断屏蔽： 0：ssi_txe_intr 中断被屏蔽 1：ssi_txe_intr 中断未被屏蔽	0x1

22.7.13 ISR

◆ **偏移地址：0x30　　复位值：0x00000005**

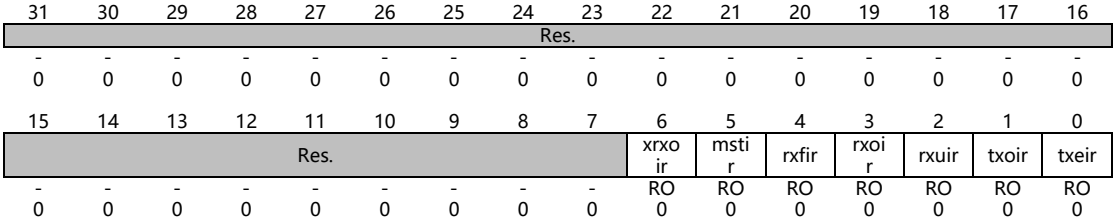
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.									xrxois	mstis	rxfis	rxois	rxuis	txois	txeis
-	-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
xrxois	[6]	RO	XIP 接收 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
mstis	[5]	RO	多 Master 竞争中断上报： 0：无中断 1：产生中断	0x0
rxfis	[4]	RO	接收 FIFO 满中断： 0：无中断 1：产生中断	0x0
rxois	[3]	RO	接收 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
rxuis	[2]	RO	接收 FIFO 下溢中断： 0：无中断 1：产生中断	0x1
txois	[1]	RO	发送 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
txeis	[0]	RO	发送 FIFO 空中断： 0：无中断 1：产生中断	0x1

22.7.14 RISR

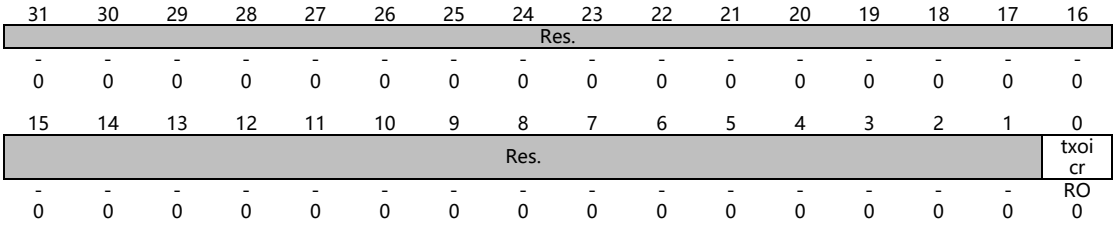
◆ **偏移地址：0x34　　复位值：0x00000005**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
xrxoir	[6]	RO	XIP 接收 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
mstir	[5]	RO	多 Master 竞争中断上报： 0：无中断 1：产生中断	0x0
rxfir	[4]	RO	接收 FIFO 满中断： 0：无中断 1：产生中断	0x0
rxoir	[3]	RO	接收 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
rxuir	[2]	RO	接收 FIFO 下溢中断： 0：无中断 1：产生中断	0x1
txoir	[1]	RO	发送 FIFO 溢出中断： 0：无中断 1：产生中断	0x0
txeir	[0]	RO	发送 FIFO 空中断： 0：无中断 1：产生中断	0x1

22.7.15 TXOICR

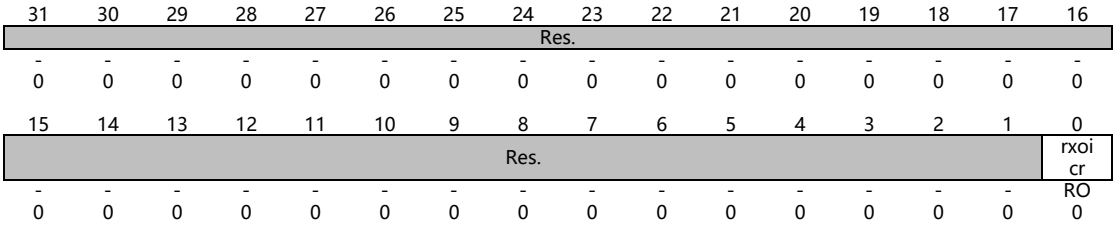
◆ **偏移地址：0x38　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txoicr	[0]	RO	从这个寄存器读取会清除 ssi_txo_intr 中断；写入没有效果。	0x0

22.7.16 RXOICR

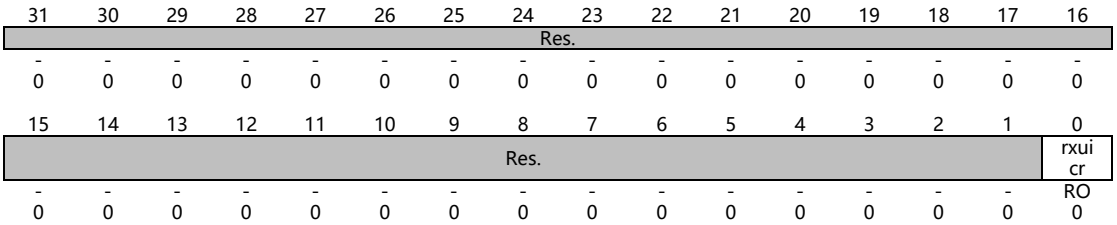
◆ **偏移地址：0x3C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxoicr	[0]	RO	从这个寄存器读取会清除 ssi_rxo_intr 中断；写入没有效果。	0x0

22.7.17 RXUICR

◆ **偏移地址：0x40　　复位值：0x00000001**

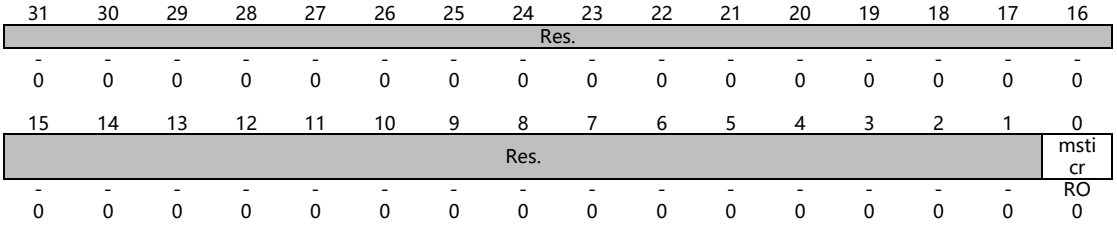


Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0

Name	Bits	R/W	Description	Default
rxuicr	[0]	RO	从这个寄存器读取会清除 ssi_rxu_intr 中断；写入没有效果。	0x1

22.7.18 MSTICR

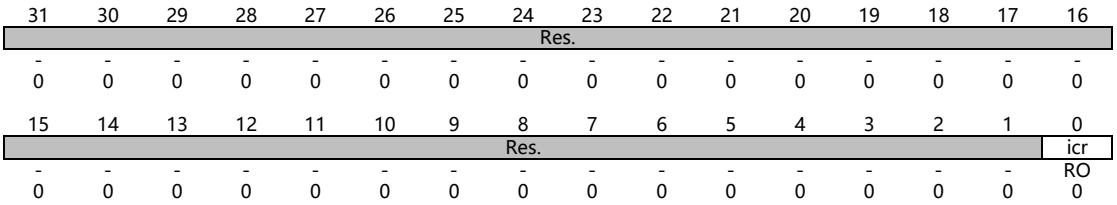
◆ **偏移地址：0x44　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
msticr	[0]	RO	从这个寄存器读取会清除 ssi_mst_intr 中断；写入没有效果。	0x0

22.7.19 ICR

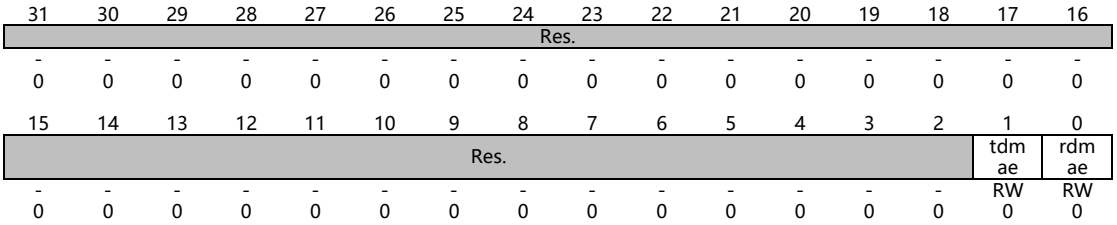
◆ **偏移地址：0x48　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
icr	[0]	RO	读取将清除 ssi_txo_intr、ssi_rxu_intr、ssi_rxo_intr 和 ssi_mst_intr 中断。写入此寄存器没有效果。	0x0

22.7.20 DMACR

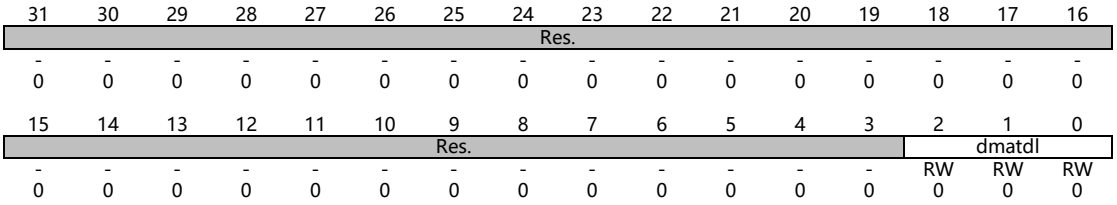
◆ **偏移地址：0x4C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留	0x0
tdmae	[1]	RW	发送 DMA 使能： 0：禁用发送 DMA 1：启用发送 DMA	0x0
rdmae	[0]	RW	接收 DMA 使能： 0：禁用接收 DMA 1：启用接收 DMA	0x0

22.7.21 DMATDLR

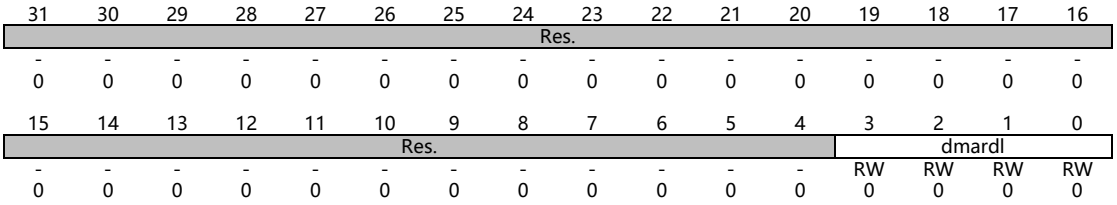
◆ **偏移地址：0x50　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
dmatdl	[2:0]	RW	此寄存器配置 DMA 请求发起水线（发送方向），当 FIFO 中数据个数小于等于此水线值时，dma_tx_req 将拉高，向 DMA 发送数据发送请求。	0x0

22.7.22 DMARDLR

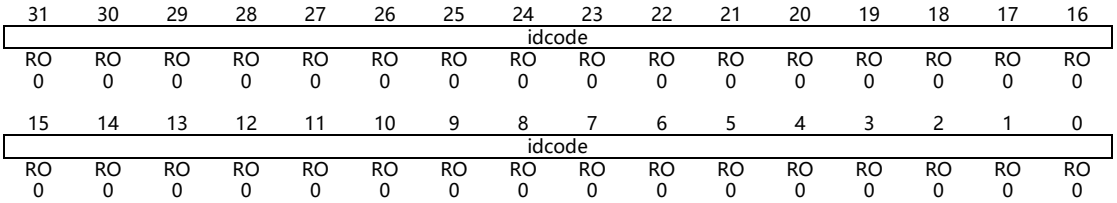
◆ **偏移地址：0x54　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
dmardl	[3:0]	RW	此寄存器配置 DMA 请求发起水线（接收方向），当 FIFO 中数据个数等于或者大于此寄存器值+1 时，dma_rx_req 将拉高，向 DMA 发送数据搬移请求。	0x0

22.7.23 IDR

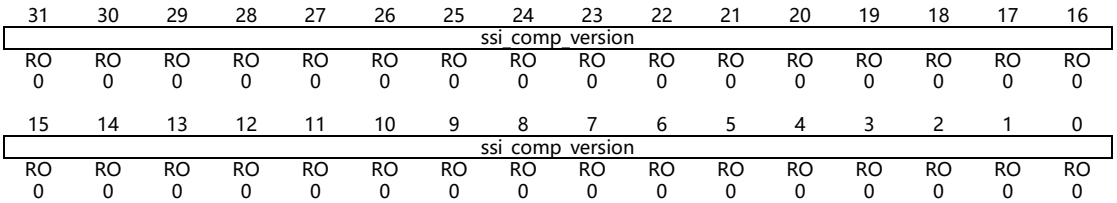
◆ **偏移地址：0x58　　复位值：0x00009529**



Name	Bits	R/W	Description	Default
idcode	[31:0]	RO	QSPI ID 号	0x9529

22.7.24 SSI_VERSION_ID

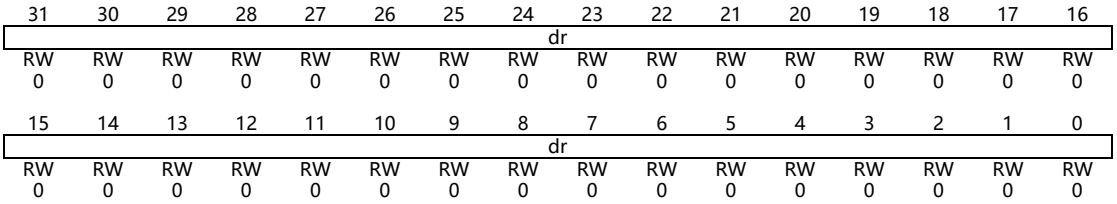
◆ **偏移地址：0x5C　　复位值：0x3130312A**



Name	Bits	R/W	Description	Default
ssi_comp_version	[31:0]	RO	QSPI 版本号	0x3130312A

22.7.25 DR

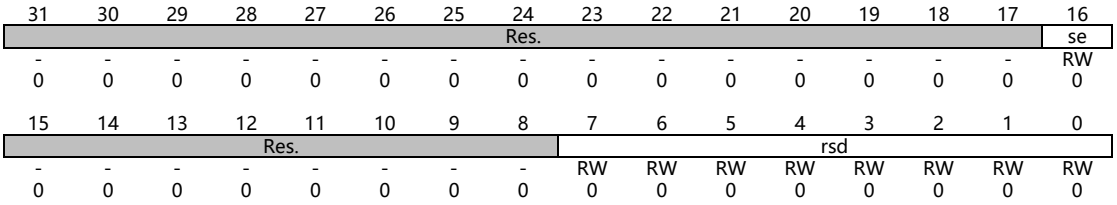
◆ **偏移地址：0x60　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
dr	[31:0]	RW	数据寄存器，右对齐	0x0

22.7.26 RX_SAMPLE_DLY

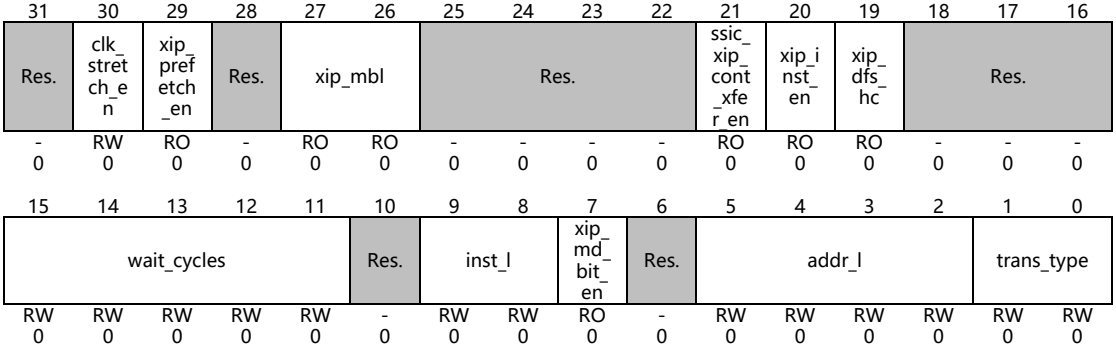
◆ **偏移地址：0xf0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留	0x0
se	[16]	RW	接收数据（rxd）采样边缘。该寄存器用于用 ssi_clk 确定 RXD 信号的采样边。将此位设置为 1，使用 ssi_clk 的负边沿对输入的数据进行采样，否则将使用正边沿进行采样。	0x0
reserved	[15:8]	-	保留	0x0
rsd	[7:0]	RW	Rxd 延迟配置寄存器，此寄存器用于延迟 rxd 输入端口的样本。每个值代表 rxd 样本上的一个 ssi_clk 延迟。	0x0

22.7.27 QSPI_CTRLR0

◆ **偏移地址：0xF4　　复位值：0x00000200**



Name	Bits	R/W	Description	Default
reserved	[31]	-	保留	0x0
clk_stretch_en	[30]	RW	在 QSPI 传输中启用时钟拉伸功能。 0：不使能 1：使能	0x0
xip_prefetch_en	[29]	RO	在 XIP 模式下预取功能使能状态寄存器。 0：不使能 1：使能	0x0
reserved	[28]	-	保留	0x0
xip_mbl	[27:26]	RO	XIP 模式位长度读取寄存器。 0：模式位长度等于 2 1：模式位长度等于 4 2：模式位长度等于 8 3：模式位长度等于 16	0x0
reserved	[25:22]	-	保留	0x0
ssic_xip_cont_xfer_en	[21]	RO	XIP 模式下连续传输使能状态寄存器。 0：不使能 1：使能	0x0
xip_inst_en	[20]	RO	XIP 指令传输使能状态寄存器。 0：不使能 1：使能	0x0
xip_dfs_hc	[19]	RO	XIP 传输的 DFS 状态寄存器。 0：数据帧的大小和要获取的数据帧数将由 HSIZE 和 HBURST 信号决定。 1：XIP 传输的数据帧大小将被固定为 CTRLR0 中的编程值。DFS要获取的数据帧数将由 HSIZE 和 HBURST 信号决定。	0x0
reserved	[18:16]	-	保留	0x0
wait_cycles	[15:11]	RW	2 线、4 线模式控制帧传输和数据接收之间的等待周期数。此值被指定为 SPI 时钟周期数。	0x0
reserved	[10]	-	保留	0x0

Name	Bits	R/W	Description	Default
inst_l	[9:8]	RW	指令长度： 0：0-bit (无指令) 1：4-bit 2：8-bit 3：16-bit	0x1
xip_md_bit_en	[7]	RO	XIP 模式位使能状态寄存器。 0：不使能 1：使能	0x0
reserved	[6]	-	保留	0x0
addr_l	[5:2]	RW	地址长度： 0：0-bit 1：4-bit 2：8-bit 3：12-bit 4：16-bit 5：20-bit 6：24-bit 7：28-bit 8：32-bit 9：36-bit 10：40-bit 11：44-bit 12：48-bit 13：52-bit 14：56-bit 15：60-bit	0x0
trans_type	[1:0]	RW	选择 QSPI 是在标准 SPI 模式下或在 CTRLR0 中选择的 SPI 模式下传输指令/地址： 0：指令和地址将以标准 SPI 模式发送。 1：指令将以标准 SPI 模式发送，地址将以 CTRLR0 指定的模式发送。 2：指令和地址都将以 SPI_FRF 指定的模式发送。 3：保留。	0x0

22.7.28 XIP_MODE_BITS

◆ **偏移地址：0xFC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xip_md_bits															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
xip_md_bits	[15:0]	RW	XIP 传输模式下，地址阶段后发送完成后，发送的 XIP 模式位。	0x0

22.7.29 XIP_INCR_INST

◆ **偏移地址：0x100　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
incr_inst															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
incr_inst	[15:0]	RW	XIP INCR 传输操作码。	0x0

22.7.30 XIP_WRAP_INST

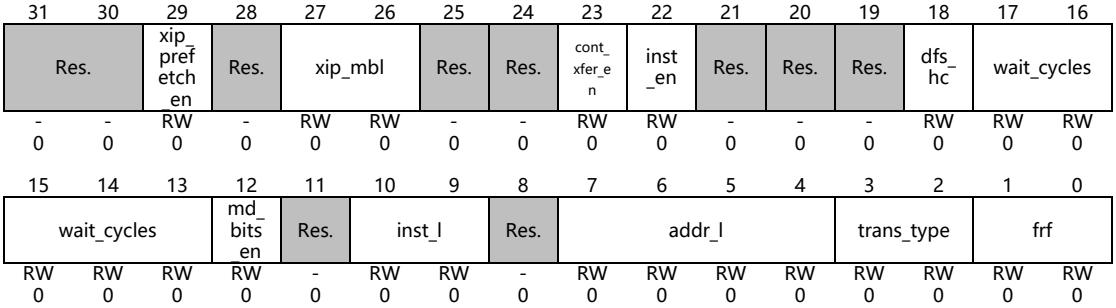
◆ **偏移地址：0x104　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
wrap_inst															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
wrap_inst	[15:0]	RW	XIP WRAP 传输操作码。	0x0

22.7.31 XIP_CTRL

◆ **偏移地址：0x108　　复位值：0x08440482**



Name	Bits	R/W	Description	Default
reserved	[31:30]	-	保留位	0x0
xip_prefetch_en	[29]	RW	在 XIP 模式下预取功能使能控制。一旦启用，QSPI 控制器将从下一个连续位置预取数据帧，以减少传输间的延迟；如果下一个 XIP 请求不是连续的，则预取的比特将被丢弃。 0：不使能 1：使能	0x0
reserved	[28]	-	保留	0x0
xip_mbl	[27:26]	RW	XIP 模式位长度设置。 设置 XIP 模式操作中的模式位的长度。这些位仅在 SPI_CTRLR0.xip_md_bit_en 被设置为 1 时才有效。 0：模式位长度等于 2 1：模式位长度等于 4 2：模式位长度等于 8 3：模式位长度等于 16	0x2
reserved	[25]	-	保留	0x0
reserved	[24]	-	保留	0x0
cont_xfer_en	[23]	RW	XIP 模式下的连续传输使能控制。 0：不使能 1：使能	0x0
inst_en	[22]	RW	XIP 指令传输使能控制。如果这个位被设置为 1，那么 XIP 传输也将有指令阶段。指令操作码将根据 AHB 传输类型从 XIP_INCR_INST 或 XIP_WRAP_INST 寄存器中选择。 0：不使能 1：使能	0x1
reserved	[21]	-	保留	0x0
reserved	[20]	-	保留	0x0

Name	Bits	R/W	Description	Default
reserved	[19]	-	保留	0x0
dfs_hc	[18]	RW	XIP 传输的 DFS 设置寄存器。 0：数据帧的大小和要获取的数据帧数将由 HSIZE 和 HBURST 信号决定。 1：XIP 传输的数据帧大小将被固定为 CTRLR0 中的编程值。DFS要获取的数据帧数将由 HSIZE 和 HBURST 信号决定。	0x1
wait_cycles	[17:13]	RW	2 线、4 线分模式控制帧传输和数据接收之间的等待周期数。此值被指定为 QSPI 时钟周期数。	0x0
md_bits_en	[12]	RW	模式位在 XIP 模式下启用。如果这个位被设置为 1，那么在 XIP 的操作模式中，QSPI 将在地址阶段之后插入模式位。这些位是在寄存器 XIP_MODE_BITS 寄存器中设置的。模式位的长度总是设置为 8 位。 0：不使能 1：使能	0x0
reserved	[11]	-	保留	0x0
inst_l	[10:9]	RW	指令长度： 0：0-bit (无指令) 1：4-bit 2：8-bit 3：16-bit	0x2
reserved	[8]	-	保留	0x0
addr_l	[7:4]	RW	地址长度： 0：0-bit 1：4-bit 2：8-bit 3：12-bit 4：16-bit 5：20-bit 6：24-bit 7：28-bit 8：32-bit 9：36-bit 10：40-bit 11：44-bit 12：48-bit 13：52-bit 14：56-bit 15：60-bit	0x8
trans_type	[3:2]	RW	选择 QSPI 是在标准 SPI 模式下或在 CTRLR0 中选择的 SPI 模式下传输指令/地址：	0x0

Name	Bits	R/W	Description	Default
			0：指令和地址将以标准 SPI 模式发送。 1：指令将以标准 SPI 模式发送，地址将以 CTRLR0 指定的模式发送。 2：指令和地址都将以 SPI_FRF 指定的模式发送。 3：保留。	
frf	[1:0]	RW	QSPI 帧格式： 0：保留 1：Dual SPI Format 2：Quad SPI Format 3：Octal SPI Format	0x2

22.7.32 XIP_SER

◆ **偏移地址：0x10C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															ser
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
ser	[0]	RW	Slave 选择寄存器： 1：选中 0：未选中	0x0

22.7.33 XRXOICR

◆ **偏移地址：0x110　　复位值：0x00000000**

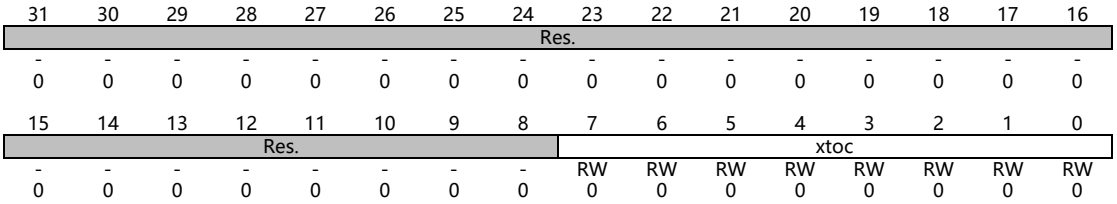
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															xrxoicr
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
xrxoicr	[0]	RO	清除 XIP 接收 FIFO 溢出中断。	0x0

22.7.34 XIP_CNT_TIME_OUT

◆ **偏移地址：0x114　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
xtoc	[7:0]	RW	XIP 超时值计数器阈值，以 clk_hs 为单位。在连续 XIP 模式中 slave 被选中，如果在计数器达到阈值后未响应，由 Slave 片选信号将失效。	0x0

23 集成电路接口（I2C）

23.1 I2C 简介

I2C（Inter-Integrated Circuit）接口控制与串行 I2C 总线间的通信，可以作为 Master 与外部设备进行通信，也可作为 Slave 与外设对接。支持 I2C 总线标准 6.0 版本，兼容 SMBus 规范 3.0 版本以及 PMBus 规范 1.2 版本。

芯片共有 2 个 I2C 接口。

23.2 I2C 主要特性

I2C 主要特性如下：

- 支持 Master 模式和 Slave 模式
- 支持标准模式（最高 100Kbps）、快速模式（最高 400Kbps）和超快速模式（最高 1Mbps）
- 支持 7 位寻址模式
- 支持配置 2 个 7 位从地址（Slave 模式）
- 支持 General Call
- 支持 DMA 操作
- 总线上的数据建立时间和保持时间可软件配置
- 支持对输入的 SDA 和 SCL 信号进行滤波
- 支持总线清除功能，当数据线（SDA）卡在低电平时，可以尝试把数据线恢复正常
- 支持 TX FIFO 和 RX FIFO，FIFO 深度都是 8
- 支持读写交替访问
- 支持 SMBus/PMBus

- 支持 SMBus Slave 检测和响应 ARP 命令

23.3 I2C 功能描述

23.3.1 I2C 时钟

I2C 的工作时钟（ic_clk）与 I2C 的配置时钟一致。ic_clk 由芯片顶层的 CRG（Clock and Reset Generator）模块产生。

23.3.2 I2C 工作模式

I2C 可以工作在下面四种模式：

- Slave 发送数据模式：响应读请求，向总线发送读数据。
- Slave 接收数据模式：接收写请求和写数据。
- Master 发送数据模式：发送写请求，写数据。
- Master 接收数据模式：发送读请求，接收读数据。

23.3.3 I2C 总线行为与模块实现

I2C 总线行为

Master 负责生成时钟并控制数据传输。读写传输都由 Master 主动发起，Slave 仅响应读请求返回读数据或者响应写请求接收写数据。数据确认由接收数据的设备发送，该设备可以是 Master 或 Slave。I2C 协议还允许多个 Master 在 I2C 总线上，并使用仲裁来确定总线所有权。

每个 I2C Slave 都有确定的地址。当 Master 想要与 Slave 通信时，Master 发送 START/RESTART 条件，然后跟随 Slave 的地址和控制位（R/W），以确定 Master 是想发送数据给 Slave，或是接收 Slave 发送的数据，Slave 在确认地址之后发送（ACK）信号。

如果 Master 正在向 Slave 发送数据，在每发送完 1 字节数据后，Master 会等待 Slave 返回 ACK 信号确认 Slave 已经接收到数据，Master 发送完所有数据后，将发起 STOP 条件终止数据传输。

如果 Master 正在从 Slave 端读取数据，则 Slave 每向 Master 发送一帧数据后，Master 会返回 ACK 信号以确认接收到数据。在 Master 接收到最后 1 字节数据后，Master 通过发送 NACK 信号来确认终止本次传输，之后 Master 发出 STOP 条件停止数据传输，或者

是发出 RESTART 条件后寻址另一个 Slave。

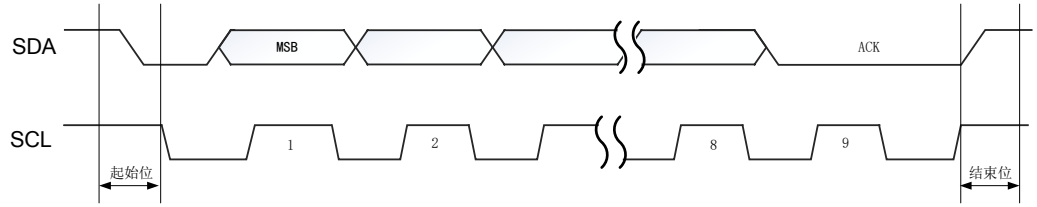
I2C 模块实现

当 I2C 总线上是 idle 态时，SCL 和 SDA 信号都是高电平状态，这需要靠外部上拉电阻实现。

当本模块作为 Master 时，往本模块的 TX FIFO 中存入数据时，本模块将在 I2C 总线上产生一个 START 条件，直到往寄存器 DATA_CMD.stop 中写入 1 时，即本模块检测到带有 STOP 标记的数据，在完成传输后即发出 STOP 条件。如果 TX FIFO 变为空并且最后一个字节没有设置 STOP 位，则本模块通过保持 SCL 线为低来暂停传输。

当本模块作为 Slave 时，不产生 START 和 STOP 条件。如果模块接收到读请求时没有可发送的读数据，则将 SCL 线保持为低电平以暂停传输，直到读数据被提供给模块，或是通过将模块的寄存器 ENABLE.enable 置 0 以关闭 I2C 模块。

图 23-1 I2C 总线协议图



在 7 位地址格式中，第一个字节的前 7 位（bit 7:1）设置 Slave 地址，LSB 位（bit0）是$\mathbf{R}/\overline{\mathbf{W}}$位，即读写标志位，如图 23-2 所示。当该位设置为 0 时，Master 向 Slave 发送写数据；当该位设置为 1 时，Master 从 Slave 读取数据。

图 23-2 7bit 地址格式

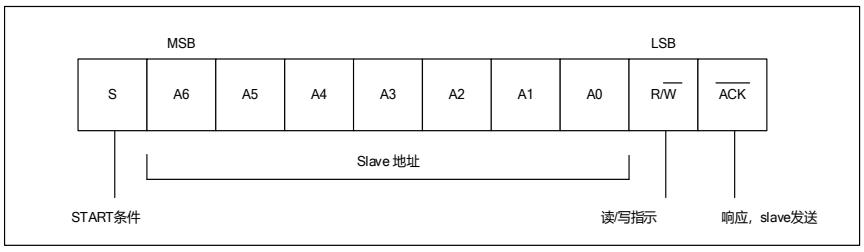


表 23-1 中给出了第一个字节的专用地址和保留地址。本模块并不限制使用这些保留地址，但是在使用这些保留地址时，可能会遇到与其他 I2C 组件不兼容的问题。

表 23-1 I2C/SMBus 第一个字节的各 bit 定义

Slave 地址	R/W bit	说明
0000 000	0	广播地址，本模块把数据放在接收缓存中，并发出广播中断
0000 000	1	START byte
0000 001	X	CBUS 地址，本模块忽略这些访问
0000 010	X	保留
0000 011	X	保留
0000 1XX	X	高速模式下 Master code，本模块不支持高速模式
1111 1XX	X	保留
1111 0XX	X	10 位 Slave 寻址，本模块不支持 10 位地址寻址
0001 000	X	SMBus 主机
0001 100	X	SMBus Alert 告警响应地址
1100 001	X	SMBus 器件默认地址

⚠注意

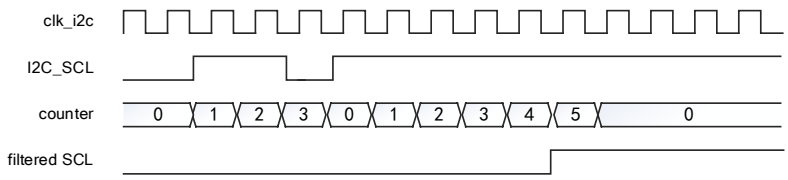
START BYTE 传输协议是为没有板级专用 I2C 硬件模块的系统设置的。当本模块是 Slave 时，以支持的最高速度对 I2C 总线进行采样，从而不需要 START BYTE；当本模块是 Master 时，支持在 Slave 需要的场景下，每次传输开始时产生 START BYTE。

23.3.4 SCL/SDA 输入毛刺过滤

模块包含毛刺过滤逻辑，符合 I2C 总线规范对标准/快速模式的要求。基于对输入信号(SCL 和 SDA)进行监视的计数器，检查输入信号在内部采样之前是否在预定量的 ic_clk（I2C 工作时钟）周期内保持稳定。SCL 和 SDA 各有一个单独的计数器。ic_clk 周期的数量可以由软件配置，配置时需考虑 ic_clk 的频率和相关的尖峰长度规范来计算。

I2C 总线协议中，标准/快速模式下，要求过滤毛刺的最大长度是 50ns，以 ic_clk 频率 100MHz 为例，FS_SPKLEN 应配置为 5。配置寄存器 FS_SPKLEN 来设置标准/快速模式下的过滤毛刺的最大长度。该寄存器能配置的最小值是 1，当配置值小于 1 时，也是按照 1 写入。过滤时，0 到 1 和 1 到 0 的毛刺都会过滤。

图 23-3 滤毛刺示例



示例中 ic_clk 的周期是 10ns，那么在标准/快速模式下，FS_SPKLEN 应配置为 5。

23.3.5 可选的 Slave 地址

模块作 Slave 时，除了 SAR 地址，还支持第二个 Slave 地址，该 Slave 的地址配置寄存器为 OPTIONAL_SAR。当总线上发送的 Slave 地址与这两个寄存器中的任意一个设备地址相匹配时，模块均能正常通信。

OPTIONAL_SAR 寄存器是否启用由 CON.optional_sar_ctrl 控制。

23.3.6 总线清除功能

总线清除功能通过配置 CON.bus_clear_feature_ctrl 开关。I2C 和 SMBus 模式下都支持。SMBus 相关内容请参考 23.4 SMbus 。

- SDA/SMBDAT 低电平超时恢复

I2C 的数据线 SDA 或者 SMBus 的数据线 SMBDAT 保持低电平的时间长度超过寄存器 SDA_STUCK_AT_LOW_TIMEOUT 设定的门限时，将产生 TX_ABRT 中断，同时对应的 TX_ABRT_SOURCE.abrt_sda_stuck_at_low 会置 1。对数据线的低电平超时检测与恢复流程，I2C 请参考 23.6.9 SCL 和 SDA 总线恢复流程 ，SMBus 请参考 23.6.10 Master 模式下 SMBus 超时处理流程 和 23.6.11 Slave 模式下 SMBus 超时处理流程 。

- SCL/SMBCLK 低电平超时恢复

I2C 的时钟线 SCL 或者 SMBus 的时钟线 SMBCLK 保持低电平的时间长度超过寄存器 SCL_STUCK_AT_LOW_TIMEOUT 设定的门限时，将产生 SCL_STUCK_AT_LOW 中断。对时钟线的低电平超时检测与恢复流程，I2C 请参考 23.6.9 SCL 和 SDA 总线恢复流程 ，SMBus 请参考 23.6.10 Master 模式下 SMBus 超时处理流程 和 23.6.11 Slave 模式下 SMBus 超时处理流程 。

23.3.7 SCL 高低电平配置

在 I2C 总线开始传输数据之前，根据实际使用的速率模式（标准、快速、超快速模式）配置*SCL_LCNT 和*SCL_HCNT 寄存器，确保正确的 I/O 时序。

tBUF 和 START/STOP/RESTART 的建立/保持时间由*SCL_LCNT 和*SCL_HCNT 寄存器设置，如表 23-2 所示。

表 23-2 I2C 时序参数与对应的寄存器配置

时序参数	符号	标准模式	快速/超快速模式
SCL 的低电平周期	tLOW	SS_SCL_LCNT	FS_SCL_LCNT
SCL 的高电平周期	tHIGH	SS_SCL_HCNT	FS_SCL_HCNT
START/RESTART 的建立时间	tSU:STA	SS_SCL_LCNT	FS_SCL_LCNT
START/RESTART 的保持时间	tHD:STA	SS_SCL_HCNT	FS_SCL_HCNT
STOP 的建立时间	tSU:STO	SS_SCL_HCNT	FS_SCL_HCNT
STOP 与 START 之间的总线空闲时间	tBUF	SS_SCL_LCNT	FS_SCL_LCNT
滤毛刺长度	tSP	FS_SPKLEN	FS_SPKLEN
数据保持时间	tHD:DAT	SDA_HOLD	SDA_HOLD
数据建立时间	tSU:DAT	SDA_SETUP	SDA_SETUP

⚠注意

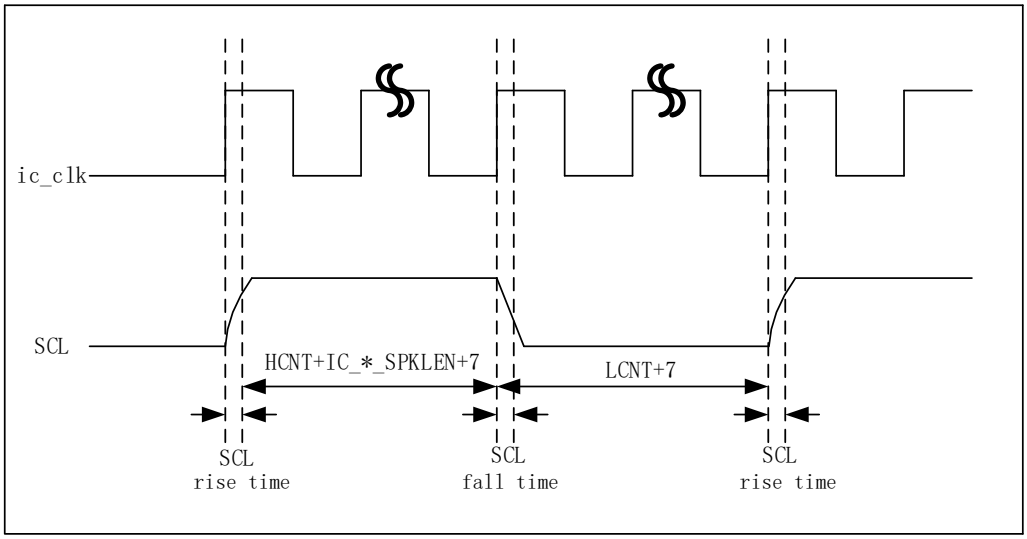
*SCL_LCNT 和*SCL_HCNT 寄存器仅用于确认模块作为 Master 时的 SCL 时序要求，所以当模块仅作为 Slave 使用时，不需要配置*SCL_LCNT 和*SCL_HCNT 寄存器。

当模块作为 I2C Master，在发送或者接收传输时：

- SS_SCL_LCNT 和 FS_SCL_LCNT 的配置值必须大于 FS_SPKLEN+7，原因是模块在 SCL 下降沿之后驱动 SDA 需要时间。
- SS_SCL_HCNT 和 FS_SCL_HCNT 的配置值必须大于 FS_SPKLEN+5，原因是模块在 SCL 为高电平期间采样 SDA 需要时间。

模块在产生 SCL 低电平时，由于计数器计数的原因，低电平实际长度是*SCL_LCNT+1；模块在产生 SCL 高电平时，由于计数器计数以及逻辑处理等原因，高电平实际长度是*SCL_HCNT+FS_SPKLEN+7。

图 23-4 SCL 上升时间和下降时间对生成 SCL 的影响



⚠注意

模块作为 Master 时产生的 SCL 信号的总高电平长度和低电平长度，也受 SCL 线的上升时间和下降时间影响。如图 23-4 所示 SCL 线的上升时间和下降时间因外部因素而异，比如：IO 驱动的特性、上拉电阻的大小、SCL 线上的总电容等，这些都不在模块的控制范围内。

不考虑 RC 的影响，计算*SCL_LCNT 和*SCL_HCNT 的公式如下：

- SCL 高电平长度=[(*SCL_HCNT+ FS_SPKLEN+7)*ic_clk]
- SCL 低电平长度=[(*SCL_LCNT+1)*ic_clk]

考虑 RC 时序的影响，计算 HCNT 和 LCNT 的公式如下：

- SCL 高电平长度=[(*SCL_HCNT+ FS_SPKLEN+7)*ic_clk]+SCL_Fall_time
- SCL 低电平长度=[(*SCL_LCNT+1)*ic_clk]-SCL_Fall_time+SCL_Rise_time

表 23-3 MIN_SCL_HIGHtime 和 MIN_SCL_LOWtime 在不同速率下的要求

速率	MIN_SCL_HIGHtime	MIN_SCL_LOWtime
100kbps	4000ns	4700ns
400kbps	600ns	1300ns
1000kbps	260ns	500ns

当时钟频率确定后，根据时钟频率和协议规定的 SCL 高低电平最小时间长度要求，计算出当前的*SCL_LCNT 和*SCL_HCNT 值。下面举例说明，在不考虑 RC 的影响下，假定 I2C 速率 100Kbps，ic_clk 频率 100MHz：

- ic_clk 周期为 10ns
- FS_ SPKLEN=5
- SCL 周期 10000ns，占空比设定 50%，高低电平长度各 5000ns，满足表 23-3 要求，则：

带入高电平公式有：(SS_SCL_HCNT+5+7)*10=5000，得到 SS_SCL_HCNT=488；

带入低电平公式有：(SS_SCL_LCNT+1)*10=5000，得到 SS_SCL_LCNT=499；

其他速率下也按照相同的方法计算，SCL 的占空比设定需要满足 I2C 速率下的高低电平最小时间要求。

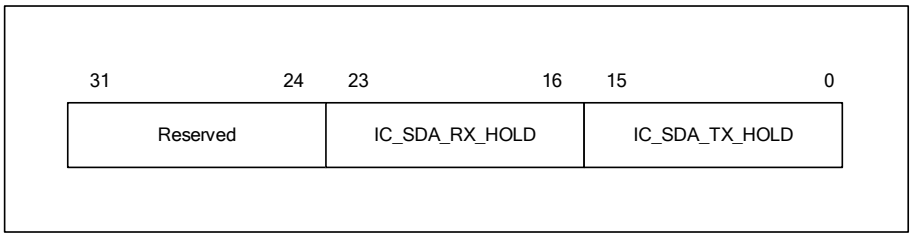
23.3.8 SDA 保持时间

I2C 协议要求 SDA 信号的保持时间(tHD:DAT)在标准模式和快速模式下为 300ns；在超快速模式下要保持一定时间，从而保证能包裹住 SCL 信号从逻辑 1 变成逻辑 0 的这段下降沿时间。

SCL 和 SDA 信号有板级延迟，这意味着 I2C 作为 Master 时可能已经满足了保持时间的要求，但由于板级延迟，可能导致对端的 Slave 侧保持时间却不满足；反之亦然。本模块包含了可配寄存器 SDA_HOLD，以实现 SDA 保持时间的动态调整。

在 Master 或者 Master 模式发送数据期间，SDA_HOLD.ic_sda_tx_hold 用于控制当 SCL 从高变为低之后 SDA 的保持时间。在 Master 或者 Slave 模式接收数据期间，SDA_HOLD.ic_sda_rx_hold 用于扩展当 SCL 为高电平时 SDA 的保持时间。SDA_HOLD 需要根据支持的速度进行配置，并且在模块处于关闭时才可以配置。

图 23-5 SDA_HOLD 寄存器



接收数据时的 SDA 保持时序

当模块在接收数据时，根据 I2C 协议，在 SCL 从逻辑 1 降到逻辑 0 这段时间内，模块要在内部保持住 SDA 信号。SDA_HOLD.ic_sda_rx_hold 可用于更改模块应用于传输 SDA 线路的内部保持时间。SDA_HOLD.ic_sda_rx_hold 的单位是 ic_clk 的周期，最小值 0。仅在 SCL 为高时适用，SCL 内部变低后，接收器不扩展 SDA。如图 23-6 所示，当 SDA_HOLD.ic_sda_rx_hold=3 时，sda_int 扩展了 3 拍；当 SDA_HOLD.ic_sda_rx_hold>3 时，因为 scl_int 拉低了，sda_int 也只能扩展 3 拍。

图 23-6 块接收数据时，SDA_HOLD.ic_sda_rx_hold 配置>=3

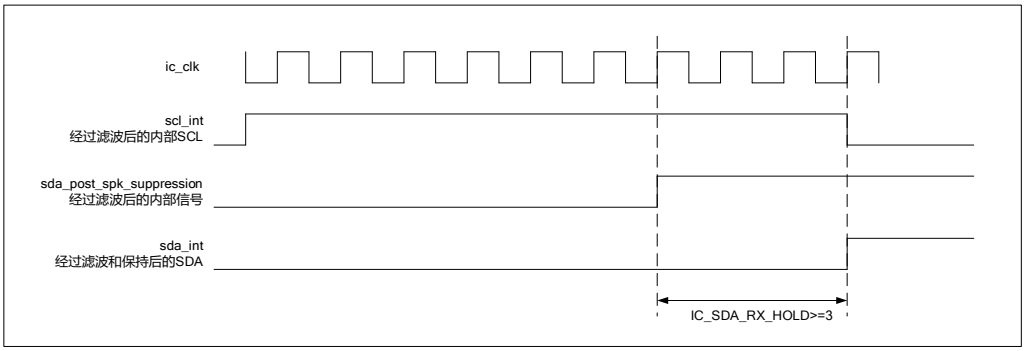


图 23-7 模块接收数据时，SDA_HOLD.ic_sda_rx_hold 配置为 2

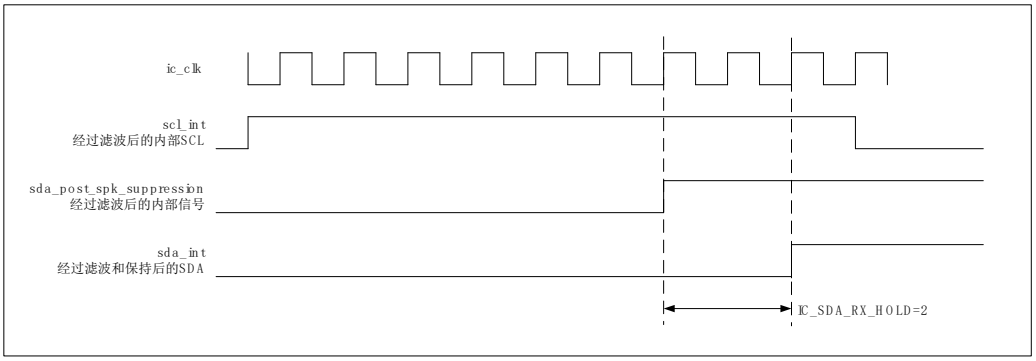


表 23-4 不同速率下 SDA_HOLD.ic_sda_rx_hold 的最大值

速度模式	SDA_HOLD.ic_sda_rx_hold 最大值
标准模式	SS_SCL_HCNT-FS_SPKLEN-3
快速模式/超快速模式	FS_SCL_HCNT-FS_SPKLEN-3

⚠注意

表 23-4 中给出的最大值仅适用于模块是 Master 模式。当模块是 Slave 时，需要确保 SDA_HOLD.ic_sda_rx_hold 不超过最大 SCL 下降时间。

发送数据时的 SDA 保持时序

SDA_HOLD.ic_sda_tx_hold 用于模块产生内部的 SDA(ic_data_oe)信号，其单位也是 ic_clk 的周期。

当模块作为 Master 时，tHD:DAT 的最小值是 1 个 ic_clk 周期，因此，当 SDA_HOLD.ic_sda_tx_hold 是 0 时，模块将在 SCL(ic_clk_oe)驱动到逻辑 0 后一个 ic_clk 周期驱动 SDA(ic_data_oe)；当 SDA_HOLD.ic_sda_tx_hold 大于 0 时，模块将 SCL(ic_clk_oe)驱动至逻辑 0 后，SDA(ic_data_oe)驱动发生在 SDA_HOLD.ic_sda_tx_hold 个 ic_clk 周期时。

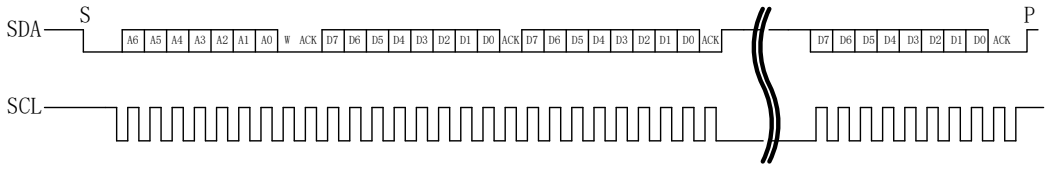
当模块作为 Slave 时，tHD:DAT 的最小值是 FS_SPKLEN+7 个 ic_clk 周期，因此，当 SDA_HOLD.ic_sda_tx_hold 配置值小于 FS_SPKLEN+7 时，在 SCL(ic_clk_in)到逻辑 0 后的 FS_SPKLEN+7 个 ic_clk 周期时模块驱动 SDA(ic_data_oe)；当 SDA_HOLD.ic_sda_tx_hold 配置值不小于 FS_SPKLEN+7 时，SCL(ic_clk_in_a)转换到逻辑 0 后，SDA 上的驱动(ic_data_oe)发生在 SDA_HOLD.ic_sda_tx_hold 个 ic_clk 周期时。

⚠注意

配置 SDA_HOLD.ic_sda_tx_hold 时，要保证在任何时候都不能超过 SCL 低电平持续时间。因此 SDA_HOLD.ic_sda_tx_hold 的配置不能大于 N_SCL_LOW-2，其中 N_SCL_LOW 是以 ic_clk 周期测量的 SCL 周期的低电平持续时间。

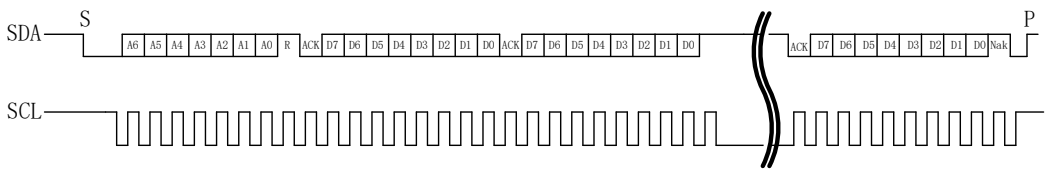
23.3.9 I2C 接口时序

图 23-8 Master 写操作时序



当模块作为 Master 发起写操作时，先发送 Slave 地址，再发送写数据；中间 TX FIFO 为空时，拉低 SCL 线，等到 TX FIFO 非空时又恢复正常传输；当检测到 STOP bit 时，产生 STOP 条件。写操作时序如上图 23-8 所示。

图 23-9 Master 读操作时序



模块作为 Master 接收数据时，TX FIFO 中缓存的是读命令。当 TX FIFO 为空时且一直没有检测到要发送 STOP 条件，则拉低 SCL 线，等到 TX FIFO 非空再继续恢复 SCL 线开始传输。写操作时序如上图 23-9 所示。

23.4 SMBus 功能

SMBus 全称 System Management Bus（系统管理总线），设计用于连接系统中的各种设备，如传感器、控制器、存储器等，以实现管理和监控系统内部的各种操作。与 SMBUS 规范兼容。

系统管理总线规范涉及三类设备。

- 从设备：用于接收或响应命令。
- 主设备：用于发出命令、生成时钟和中止传输。
- 主机：专用的主设备，可提供连接系统 CPU 的主接口。主机必须具有主 - 从设备功能，并且必须支持 SMBus 主机通知协议。系统中只允许存在一个主机。

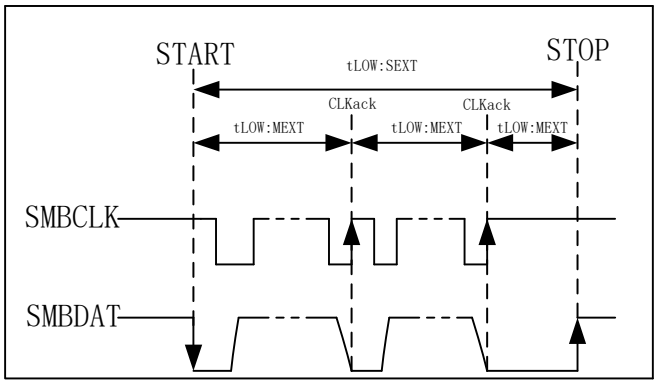
该外设可配置为主设备或从设备，也可配置为主机。

23.4.1 时钟扩展功能

tLOW: MEXT是 Master 的时钟扩展时间，是在消息的每个字节（定义为 START 到 ACK、 ACK 到 ACK 、ACK 到 STOP）内时钟信号可延展的时间。模块做为 Master 时，使用寄存器 SMBUS_CLK_LOW_MEXT 来设定检测主设备时钟扩展超时的门限，当超过门限后，生成 SMBUS_CLK_LOW_MEXT 中断。

tLOW: SEXT是 Slave 的扩展时间，是在一条消息的最初起始到停止期间时钟信号可延展的时间。模块作为 Slave 时，使用寄存器 SMBUS_CLK_LOW_SEXT 来设定从设备时钟扩展超时的门限，超过门限后生成 SMBUS_CLK_LOW_SEXT 中断。

图 23-10 Master 和 Slave 的时钟扩展超时间隔



23.4.2 SMBus 协议

典型的 SMBus 设备具有一组命令，通过这些命令可以读取和写入数据。所有命令的长度都是一个字节，它们的参数和返回值长度可能不同。根据 SMBus 规范，首先传输最高有效位 MSB。对于任何给定的设备，都有十一种可能的命令协议，分别是快速命令、发送字节、接收字节、写入字节、写入字、读取字节、读取字、进程调用、块读取、块写入和块写入-块读取进程调用。

用于消息事务的 SMBus 协议通常不同于 I2C 数据传输命令，但仍可以配置 SMBus Master 发送 I2C 数据传输命令。在 SMBus Master 模式中，所有接收数据字节都可在 RX FIFO 中使用；在 SMBus Slave 模式中，所有总线协议命令代码和数据字节都在 RX FIFO 中接收，读取请求数据字节必须使用 TX FIFO 发送，类似于 I2C 模式。

下表 23-5 描述了通过 TX FIFO 命令实现 SMBus 总线协议。

表 23-5 SMBus 总线协议在模块中的实现

协议命令	TX FIFO 需要的命令数	命令/数据：DATA_CMD.dat	命令 bit：DATA_CMD.cmd	STOP bit：DATA_CMD.stop	备注
快速命令	1	NA	配置命令为读或者写	置 1	配置 TAR.special 和 TAR.smbus_quick_cmd 为 1
发送字节	1	数据字节	置 0	置 1	
接收字节	1	NA	置 1	置 1	
写入字节	2	命令	置 0	置 0	
		数据	置 0	置 1	

协议命令	TX FIFO 需要的命令数	命令/数据: DATA_CMD.dat	命令 bit: DATA_CMD.cmd	STOP bit: DATA_CMD.stop	备注
写入字	3	命令	置 0	置 0	
		数据低位字节	置 0	置 0	
		数据高位字节	置 0	置 1	
读取字节	2	命令	置 0	置 0	
		NA	置 1	置 1	
读取字	3	命令	置 0	置 0	
		NA	置 1	置 0	
		NA	置 1	置 1	
进程调用	5	命令	置 0	置 0	
		数据低位字节	置 0	置 0	
		数据高位字节	置 0	置 0	
		NA	置 1	置 0	
		NA	置 1	置 1	
块写入	N+2	命令	置 0	置 0	
		数据	置 0	置 0	
		N+1) 数据 N	置 0	置 1	
块读取	N+2	命令	置 0	置 0	
		NA	置 0	置 0	
		N+1) NA	置 0	置 1	
块写入-块读取调用进程	M+N+2	命令	置 0	置 0	
		数据 1	置 0	置 0	
		M+1) 数据 M	置 0	置 0	
		M+2) NA	置 1	置 0	
		M+3) NA	置 1	置 0	
		M+N+1) NA	置 1	置 1	
SMBUS 主机通知协议	3	器件地址	置 0	置 0	配置 TAR.ic_tar 为 SMB 主机地址 (0001000)
		数据低位字节	置 0	置 0	
		数据高位字节	置 0	置 1	

模块作为 Slave 时，可以通过配置 CON.slave_quick_cmd_en 为 1 来设定为仅接收快速命令。此时不接收协议中的其他命令。当检测到快速命令后，模块会产生 SMBUS_QUICK_DET 中断。

SMBus 通过在总线协议的末尾添加 PEC（Packet Error checking）字节，引入了一种分组错误检查机制。这可以通过添加一个额外的命令（PEC 字节）来实现，同时通过软件进行传输和解码。

23.4.3 SMBus 地址解析协议（ARP）

SMBus 从设备地址冲突可以通过主机为每个从设备动态分配一个新的唯一地址来解决。此功能允许设备“热插拔”到系统中。

SMBus 为系统中的每个设备引入了一个 128 位的唯一设备 ID（UDID），来隔离每个设备以进行地址分配。本模块使用寄存器 SMBUS_UDID_LSB 来设定 UDID 的低 32bit，也可以使用 CON.persistant_slv_addr_en 域段来设定是否支持持久 Slave 地址。

表 23-6 SMBus ARP 命令在模块中实现

ARP 命令	TX FIFO 需要的命令数	命令/数据：DATA_CMD.dat	命令 bit：DATA_CMD.cmd	STOP bit：DATA_CMD.stop	备注
ARP 准备	2	命令= ‘00000001’	置 0	置 0	配置 TAR.ic_tar 为 SMBus 默认地址 1100 001
		PEC 字节	置 0	置 1	
复位器件（广播）	2	命令= ‘00000010’	置 0	置 0	配置 TAR.ic_tar 为 SMBus 默认地址 1100 001
		PEC 字节	置 0	置 1	
获取 UDID（广播）	20	命令= ‘00000011’	置 0	置 0	配置 TAR.ic_tar 为 SMBus 默认地址 1100 001 读 16 次获取 128bit 的 UDID 字节 最后一个读命令读取 Slave 地址
		NA	置 1	置 0	
		NA	置 1	置 0	
		NA	置 1	置 0	
		PEC 字节	置 1	置 1	
分配地址	20	命令= ‘00000100’	置 0	置 0	配置 TAR .ic_tar 为 SMBus 默认地址 1100 001 写 16 次完成 128bit 的 UDID 字节 最后一个写命令写入 Slave 地址
		Byte 计数=17	置 0	置 0	
		UDID 字节 15 到 0	置 0	置 0	
		分配地址	置 0	置 0	
		PEC 字节	置 0	置 1	
获取 UDID（定向）	20	命令= ‘Slave 地址[6:0] | 1’	置 0	置 0	配置 TAR.ic_tar 为 SMBus 默认地址 1100 001 读一次获取字节数 读 16 次获取 UDID
		NA	置 1	置 0	
		NA（16 次）	置 1	置 0	

ARP 命令	TX FIFO 需要的命令数	命令/数据: DATA_CMD.dat	命令 bit: DATA_CMD.cmd	STOP bit: DATA_CMD.stop	备注
		NA	置 1	置 0	最后一次读 Slave 地址
		PEC 字节	置 1	置 1	
复位器件（定向）	2	命令 = ‘Slave 地址[6:0] | 0’	置 0	置 0	配置 TAR.ic_tar 为 SMBus 默认地址 1100 001
		PEC 字节	置 0	置 1	
通知 ARP 主机	3	器件地址 = ‘1100 0010’	置 0	置 0	配置 TAR.ic_tar 为 SMBus 主机地址 0001 000
		数据低位字节 = ‘00000000’	置 0	置 0	
		数据高位字节 = ‘00000000’	置 0	置 1	

⚠注意

模块作为 Slave 时，硬件能处理 PEC 的生成、检测和以及不匹配时自动返回 NACK。使用 CRC-8 多项式 C(X)=X^8+X^2+X^1+1。硬件不处理非 ARP 命令的 PEC。

模块作 Master 时，硬件不处理 ARP 和非 ARP 命令的 PEC。

23.4.4 SMBus 永久 Slave 与非永久 Slave

永久 Slave（Persistent Slave，PSA）是一种 SMBus 设备，它始终处于连接状态，即使总线上没有通信也是如此，保持实时与总线连接，以随时准备处理请求。常见的例子是电源管理控制器和温度传感器等。当本模块作为 PSA 时，地址有效标志位(STATUS.smbus_slave_addr_valid)始终设置为 1。

非永久 Slave（非 PSA）是一种 SMBus 设备，只在收到请求时才会加入总线，并在完成请求后断开连接。常见的例子包括 EEPROM 和 I/O 扩展器等。当本模块作为非 PSA 时，会清除广播和定向重置 ARP 命令的地址有效标志。

23.4.5 SMBus 告警信号

模块支持 SMBus Alert 信号。从设备可以用它来请求和主机通信。告警信号是线与信号，由从设备发出，输入给主机。

当模块作为 Master 时，在检测到 Alert 告警后，主机发送 alert 响应地址给总线上的所有

从设备，该从设备（发出 alert 告警的从设备）再发送其设备地址给主机并撤销 alert 告警，主机获取到从设备地址后就可以进行正常的通信。如果主机一直检测到 alert 告警在上报，就会一直重复发送告警响应地址。当模块作为 Master 时，检测到输入的 alert 告警信号，可以产生 Alert 中断，当前输入的 alert 信号状态可以通过读寄存器 STATUS.smbus_alert_status 获得。

当模块作为 Slave 时，通过配置寄存器 ENABLE.smbus_alert_ctrl 为 1，使模块产生 alert 告警；之后就等待 Master 发送 alert 告警响应地址。接收到这个地址后，模块再把配置在 SAR.ic_sar 的设备地址发送给对端 Master；都完成后，模块自动清除 SMBUS_ALERT_CTRL，并停止输出 alert 告警。

23.5 中断寄存器

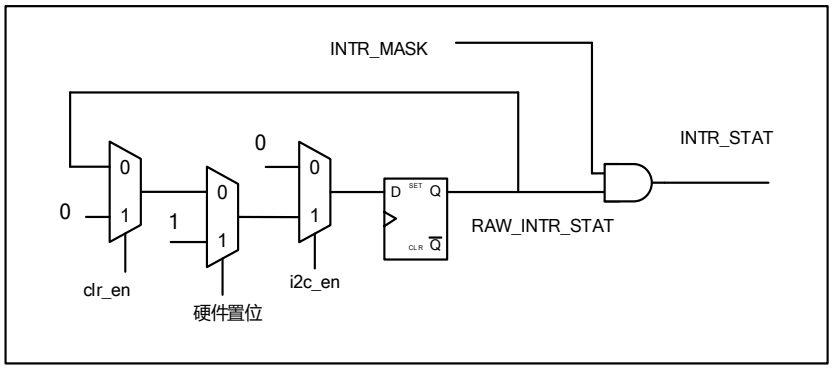
- 中断寄存器有 RAW_INTR_STAT 和 SMBUS_RAW_INTR_STAT。
- 中断寄存器 RAW_INTR_STAT 的各域段有“硬件置位软件清除”和“硬件置位和清除”两种中断置位清除方式。如下表所示，Y 表示支持，-表示不支持。

表 23-7 中断寄存器 RAW_INTR_STAT 各 bit 的置位与清除

中断寄存器各 bit 域段	硬件置位，软件清除	硬件置位和清除
scl_stuck_at_low	Y	-
master_on_hold	-	Y
gen_call	Y	-
start_det	Y	-
stop_det	Y	-
activity	Y	-
rx_done	Y	-
tx_abrt	Y	-
rd_req	Y	-
tx_empty	-	Y
tx_over	Y	-
rx_full	-	Y
rx_over	Y	-

中断寄存器各 bit 域段	硬件置位，软件清除	硬件置位和清除
rx_under	Y	-

图 23-11 中断方案



RAW_INTR_STAT 中软件自清的中断，可以通过读 CLR_INTR 或者读各自的读清中断寄存器清除。硬件自清的中断无法通过软件读 CLR_INTR 清除。

- 中断寄存器 SMBUS_RAW_INTR_STAT 的各个域段都是“硬件置位软件清除”。

结构与 RAW_INTR_STAT 相同，只是清除方式有差异，可以通过读 CLR_INTR 或向 CLR_SMBUS_INTR 对应 bit 写 1 清除中断。

INTR_STAT 和 SMBUS_ INTR_STAT 汇聚后输出。

23.6 工作方式

23.6.1 Master 在中断方式下的数据传输

以标准速率模式为例，配置流程如下：

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 1，关闭 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 1，使能 Master 模式。

步骤 3 配置寄存器 TAR.ic_tar，设定 Slave 的设备地址。

步骤 4 配置寄存器 SS_SCL_HCNT.ic_ss_scl_hcnt 设定 SCL 信号的高电平持续时间。

步骤 5 配置寄存器 SS_SCL_LCNT.ic_ss_scl_lcnt 设定 SCL 信号的低电平持续时间。

步骤 6 配置寄存器 INTR_MASK，根据需要使能相应的中断，允许中断上报到 CPU。

步骤 7 配置寄存器 RX_TL.rx_tl，根据需要设定 RX FIFO 的阈值门限。

步骤 8 配置寄存器 TX_TL.tx_tl，根据需要设定 TX FIFO 的阈值门限。

步骤 9 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 10 配置寄存器 DATA_CMD。如果要发送数据，则配置 DATA_CMD.cmd 为 0，并把要发送的内容写入到 DATA_CMD. dat，再跳转到步骤 13；如果要读取数据，则配置 DATA_CMD.cmd 为 1，跳转到步骤 11。

步骤 11 CPU 会监控 RX_FULL 中断（INTR_STAT.r_rx_full）产生，当中断产生后，表明 RX FIFO 中接收的数据量达到 RX_TL.rx_tl 设置的阈值，CPU 可通过读 DATA_CMD. dat 位域来读取接收到的数据。

步骤 12 软件判断是否还要读取数据，如果还要读取，则跳转到步骤 10，否则跳转到步骤 15。

步骤 13 CPU 等待 TX_EMPTY 中断（INTR_STAT.r_tx_empty）产生，当中断产生后，表明当前的数据发送完成。

步骤 14 软件判断是否还要继续发送数据，如果还要发送数据，则跳转到步骤 10，否则跳转到步骤 15。

步骤 15 等待寄存器 STATUS.mst_activity 上报为 0。

步骤 16 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

当支持的最高速率是快速模式或超快速模式时，流程与上述相同，只是根据速率配置 CON.speed 为 2，并通过 FS_SCL_HCNT.ic_fs_scl_hcnt 和 FS_SCL_LCNT.ic_fs_scl_lcnt 设置 SCL 信号的高电平和低电平持续时间。

23.6.2 Master 在轮询方式下的数据传输

以标准速率模式为例，配置流程如下：

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 1，关闭 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 1，使能 Master 模式。

步骤 3 配置寄存器 TAR.ic_tar，设定 Slave 的设备地址。

步骤 4 配置寄存器 SS_SCL_HCNT.ic_ss_scl_hcnt 设定 SCL 信号的高电平持续时间。

步骤 5 配置寄存器 SS_SCL_LCNT.ic_ss_scl_lcnt 设定 SCL 信号的低电平持续时间。

步骤 6 配置寄存器 INTR_MASK，根据需要使能相应的中断（轮询模式也可以忽略步骤 6，软件查询原始中断状态即可）。

步骤 7 配置寄存器 RX_TL.rx_tl，根据需要设定设定 RX FIFO 的阈值门限。

步骤 8 配置寄存器 TX_TL.tx_tl，根据需要设定设定 TX FIFO 的阈值门限。

步骤 9 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 10 配置寄存器 DATA_CMD。如果要发送数据，则配置 DATA_CMD.cmd 为 0，并把要发送的内容写入到 DATA_CMD. dat，再跳转到步骤 13；如果要读取数据，则配置 DATA_CMD.cmd 为 1，跳转到步骤 11。

步骤 11 软件不断查询原始中断寄存器 RAW_INTR_STAT.rx_full 位域，当该 rx_full 为 1 时，表明 RX FIFO 中接收的数据量达到 RX_TL.rx_tl 设置的阈值，CPU 可通过读 DATA_CMD. dat 来读取接收到的数据。

步骤 12 软件判断是否还要读取数据，如果还要读取，则跳转到步骤 10，否则跳转到步骤 15。

步骤 13 软件不断查询原始中断寄存器 RAW_INTR_STAT.tx_empty 位域，直到该 bit 为 1，表明当前的这一字节数据发送完成。

步骤 14 软件判断是否还要继续发送数据，如果还要发送数据，则跳转到步骤 10，否则跳转到步骤 15。

步骤 15 等待寄存器 STATUS.mst_activity 上报为 0。

步骤 16 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

当支持的最高速率是快速模式或超快速模式时，流程与上述相同，只是根据速率配置 CON.speed 为 2，并通过 FS_SCL_HCNT.ic_fs_scl_hcnt 和 FS_SCL_LCNT.ic_fs_scl_lcnt 设置 SCL 信号的高电平和低电平持续时间。

23.6.3 Master 在 DMA 模式下的写操作

以标准速率模式下为例，流程如下：

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 1，关闭 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 1，使能 Master 模式。

步骤 3 配置寄存器 TAR.ic_tar，设定 Slave 的设备地址。

步骤 4 配置寄存器 SS_SCL_HCNT.ic_ss_scl_hcnt 设定 SCL 信号的高电平持续时间。

步骤 5 配置寄存器 SS_SCL_LCNT.ic_ss_scl_lcnt 设定 SCL 信号的低电平持续时间。

步骤 6 配置寄存器 INTR_MASK，根据需要使能相应的中断。

步骤 7 配置寄存器 RX_TL.rx_tl，根据需要设定设定 RX FIFO 的阈值门限。

步骤 8 配置寄存器 TX_TL.tx_tl，根据需要设定设定 TX FIFO 的阈值门限。

步骤 9 配置寄存器 DMA_CR.tdmae 为 1，使能发送 FIFO DMA 通道。

步骤 10 配置寄存器 DMA_TDLR.dmatdl，设定发送 TX DMA 请求的阈值。

步骤 11 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 12 启动 DMA 模块。当 TX FIFO 内的数据量低于 DMA_TDLR.dmatdl 的设定值时，I2C 模块将发送 dma_tx_req 请求信号给 DMA 模块，DMA 响应该请求信号后将自动发送数据到 I2C 模块并返回 dma_ack 信号，此时 DMA 搬运过来的数据将在 I2C 总线上发送出去。

步骤 13 如果 Master 还要发送数据，则跳到步骤 12；否则，配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

当速率是快速模式或超快速模式时，流程与上述相同，只是根据速率配置 CON.speed 为 2，并通过 FS_SCL_HCNT.ic_fs_scl_hcnt 和 FS_SCL_LCNT.ic_fs_scl_lcnt 设置 SCL 信号的高电平和低电平持续时间。

23.6.4 Master 在 DMA 模式下的读操作

以标准速率模式下为例，流程如下：

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 1，关闭 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 1，使能 Master 模式。

步骤 3 配置寄存器 TAR.ic_tar，设定 Slave 的设备地址。

步骤 4 配置寄存器 SS_SCL_HCNT.ic_ss_scl_hcnt 设定 SCL 信号的高电平持续时间。

步骤 5 配置寄存器 SS_SCL_LCNT.ic_ss_scl_lcnt 设定 SCL 信号的低电平持续时间。

步骤 6 配置寄存器 INTR_MASK，根据需要使能相应的中断。

步骤 7 配置寄存器 RX_TL.rx_tl，根据需要设定设定 RX FIFO 的阈值门限。

步骤 8 配置寄存器 TX_TL.tx_tl，根据需要设定设定 TX FIFO 的阈值门限。

步骤 9 配置寄存器 DMA_CR.rdmae 为 1，使能接收 FIFO DMA 通道。

步骤 10 配置寄存器 DMA_RDLR.dmardl，设定发送 RX DMA 请求的阈值。

步骤 11 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 12 启动 DMA 模块。I2C 模块发送读命令给 Slave，Slave 返回的数据将缓存到 RX FIFO 中，当 RX FIFO 中接收到的数据量达到 DMA_RDLR.dmardl 的设定值时，I2C 模块将发送 dma_rx_req 请求信号给 DMA 模块，请求 DMA 搬运数据。

步骤 13 如果 Master 还要读取数据，则跳到步骤 12；否则，配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

当速率是快速模式或超快速模式时，流程与上述相同，只是根据速率配置 CON.speed 为 2，并通过 FS_SCL_HCNT.ic_fs_scl_hcnt 和 FS_SCL_LCNT.ic_fs_scl_lcnt 设置 SCL 信号的高电平和低电平持续时间。

23.6.5 Slave 在中断方式下的数据传输

以标准速率模式为例：

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 SAR.ic_sar，设定响应地址。

步骤 3 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 0，使能 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 0，关闭 Master 模式。

步骤 4 配置寄存器 INTR_MASK，根据需要使能相应的中断。

步骤 5 配置寄存器 RX_TL.rx_tl，根据需要设定设定 RX FIFO 的阈值门限。

步骤 6 配置寄存器 TX_TL.tx_tl，根据需要设定设定 TX FIFO 的阈值门限。

步骤 7 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 8 CPU 自动检测 rd_req 中断（INTR_STAT.r_rd_req）或者 rx_full 中断（INTR_STAT.r_rx_full）是否上报。如果 rd_req 中断为 1，表明接收到读请求，跳转到步骤 9；如果 rx_full 中断为 1，表明 RX FIFO 中接收的数据量达到 RX_TL.rx_tl 设置的阈值，跳转到步骤 12。

步骤 9 软件检查 TX_ABRT 中断（INTR_STAT. r_tx_abrt），如果为 1，则读寄存器 CLR_TX_ABRT 清除 tx_abrt 中断。

步骤 10 配置 DATA_CMD.cmd 为 0，表示发送数据，并把要发送的数据写入 DATA_CMD.dat。

步骤 11 读寄存器 CLR_RD_REQ 清除 rd_req 中断。

步骤 12 读寄存器 DATA_CMD.dat，获取 I2C 接收到的数据。

步骤 13 等待寄存器 STATUS.mst_activity 上报为 0。

步骤 14 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

当支持的最高速率为快速模式或者超快速模式时，需要配置 CON.speed 为 2，其他流程与上述一致。

23.6.6 Slave 在轮询方式下的数据传输

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 SAR.ic_sar，设定响应地址。

步骤 3 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 0，使能 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 0，关闭 Master 模式。

步骤 4 配置寄存器 INTR_MASK，根据需要使能相应的中断。

步骤 5 配置寄存器 RX_TL.rx_tl，根据需要设定设定 RX FIFO 的阈值门限。

步骤 6 配置寄存器 TX_TL.tx_tl，根据需要设定设定 TX FIFO 的阈值门限。

步骤 7 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 8 软件不断查询 rd_req 中断（RAW_INTR_STAT.rd_req）或者 rx_full 中断（RAW_INTR_STAT.rx_full）是否上报；如果 rd_req 为 1，表明接收到读请求，跳转到步骤 9；如果 rx_full 为 1，表明 RX FIFO 中接收的数据量达到 RX_TL.rx_tl 设置的阈值，跳转到步骤 12。

步骤 9 软件查询 raw_intr_stat.tx_abrt，如果 tx_abrt 为 1，则读寄存器 CLR_TX_ABRT 清除 tx_abrt 中断。

步骤 10 配置 DATA_CMD.cmd 为 0，表示发送数，并把要发送的数据写入 DATA_CMD.dat。

步骤 11 读寄存器 CLR_RD_REQ 清除 rd_req 中断。

步骤 12 读 DATA_CMD.dat，获取接收到的数据。

步骤 13 等待寄存器 STATUS.mst_activity 上报为 0。

步骤 14 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

当支持的最高速率为快速模式或者超快速模式时，需要配置 CON.speed 为 2，其他流程与上述一致。

23.6.7 Slave 在 DMA 模式下的写操作

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 SAR.ic_sar，设定响应地址。

步骤 3 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 0，使能 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 0，关闭 Master 模式。

步骤 4 配置寄存器 INTR_MASK，根据需要使能相应的中断。

步骤 5 配置寄存器 DMA_CR.tdmae 为 1，使能发送 FIFO DMA 通道。

步骤 6 配置寄存器 DMA_TDLR.dmatdl 设定发送 TX DMA 请求的阈值。

步骤 7 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 8 等待 rd_req 中断（INTR_STAT.r_rd_req）产生，如果 rd_req 中断为 1，表明接收到对端 Master 的读请求。

步骤 9 启动 DMA 模块。当 TX FIFO 内的数据量低于 DMA_TDLR.dmatdl 的设定值时，I2C 模块将发送 dma_tx_req 请求信号给 DMA 模块，DMA 响应该请求信号后将自动发送数据到 I2C 模块并返回 dma_ack 信号，此时 DMA 搬运过来的数据将在 I2C 总线上发送出去。

步骤 10 如果 Slave 还要发送数据，则跳到步骤 8；否则，配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

23.6.8 Slave 在 DMA 模式下的读操作

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 SAR.ic_sar，设定响应地址。

步骤 3 配置寄存器 CON 的多个域段：

- 配置 CON.ic_slave_disable 为 0，使能 Slave 模式；
- 配置 CON.ic_restart_en 为 1，使能 restart 模式；
- 配置 CON.speed 为 1，使能标准速率模式；
- 配置 CON.master_mode 为 0，关闭 Master 模式。

步骤 4 配置寄存器 INTR_MASK，根据需要使能相应的中断。

步骤 5 配置寄存器 DMA_CR.rdmae 为 1，使能接收 FIFO DMA 通道。

步骤 6 配置寄存器 DMA_RDLR.dmardl，设定发送 RX DMA 请求的阈值。

步骤 7 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 8 等待 rx_full 中断（RAW_INTR_STAT.rx_full），如果 rx_full 中断为 1，表明 Slave 的 RX FIFO 中接收的数据量达到 RX_TL.rx_tl 设置的阈值。

步骤 9 启动 DMA 模块。I2C Slave 接收到对端 Master 发送的数据后，缓存到 RX FIFO 中，当 RX FIFO 中接收到的数据量达到 DMA_RDLR.dmardl 的设定值时，I2C 模块发送 dma_rx_req 请求信号给 DMA 模块，请求 DMA 搬运数据。

步骤 10 如果还有数据要接收，则跳回步骤 8；否则，配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

23.6.9 SCL 和 SDA 总线恢复流程

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 CON. bus_clear_feature_ctrl 为 1，使能总线恢复功能。

步骤 3 配置寄存器 SCL_STUCK_AT_LOW_TIMEOUT 和 SDA_STUCK_AT_LOW_TIMEOUT 分别设定 SCL、SDA 的超时门限。

步骤 4 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 5 读取原始中断寄存器 RAW_INTR_STAT，检查是否有 scl_stuck_at_low 中断或者 tx_abrt 中断产生。如果有 scl_stuck_at_low 中断产生，跳到步骤 10；如果有 tx_abrt 中断产生并且寄存器 TX_ABRT_SOURCE.abrt_sda_stuck_at_low=1，跳到步骤 6。

步骤 6 配置 Master 的 ENABLE.sda_stuck_recovery_enable 为 1，启动 SDA 总线恢复机制。

步骤 7 循环读 Master 的 ENABLE.sda_stuck_recovery_enable 直到回读值是 0。

步骤 8 读寄存器 STATUS.sda_stuck_not_recovered，如果回读值是 1，表示 SDA 未恢复，跳到步骤 10；如果回读值是 0，表示 SDA 恢复，跳到步骤 9。

步骤 9 读寄存器 CLR_TX_ABRT，清除 tx_abrt 中断，开始正常传输。

步骤 10 硬件复位 I2C 系统。

23.6.10 Master 模式下 SMBus 超时处理流程

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置 SMBus 超时检测相关的寄存器，SCL_STUCK_AT_LOW_TIMEOUT、SDA_STUCK_AT_LOW_TIMEOUT、SMBUS_CLK_LOW_SEXT 以及 SMBUS_CLK_LOW_MEXT。配置寄存器 CON. bus_clear_feature_ctrl 为 1，使能总线恢复功能。

步骤 3 配置寄存器 ENABLE.enable 为 1，使能 I2C 模块。

步骤 4 检测 RAW_INTR_STAT 和 SMBUS_RAW_INTR_STAT 中断寄存器状态，如果出现 RAW_INTR_STAT.scl_stuck_at_low 中断为 1 或者 SMBUS_RAW_INTR_STAT.slv_clock_extnd_timeout 中断为 1，或者 SMBUS_RAW_INTR_STAT.mst_clock_extnd_timeout 中断为 1，则跳转步骤 5；如果出现 RAW_INTR_STAT.tx_abrt 中断为 1 并且 TX_ABRT_SOURCE.abrt_sda_stuck_at_low 为 1，则跳转步骤 7；否则跳转步骤 9。

步骤 5 配置寄存器 ENABLE.abort 为 1，总线上产生 STOP 标志，并中断当前传输。跳转步骤 6。

步骤 6 循环读寄存器 ENABLE.abort，直到回读值是 0，跳转步骤 10。

步骤 7 配置寄存器 ENABLE.smbus_clk_reset 为 1，将 SMBCLK 保持 SCL_STUCK_AT_LOW_TIMEOUT 长度的低电平，复位 SMBus。

步骤 8 循环读寄存器 ENABLE.smbus_clk_reset，直到回读值是 0。

步骤 9 正常传输。

23.6.11 Slave 模式下 SMBus 超时处理流程

步骤 1 配置寄存器 ENABLE.enable 为 0，关闭 I2C 模块。

步骤 2 配置寄存器 SCL_STUCK_AT_LOW_TIMEOUT 设置检测 SMBCLK 低电平持续时间的超时门限。

步骤 3 配置寄存器 CON. bus_clear_feature_ctrl 为 1，使能总线恢复功能。

步骤 4 配置寄存器 ENABLE 为 1 使能 I2C 模块。

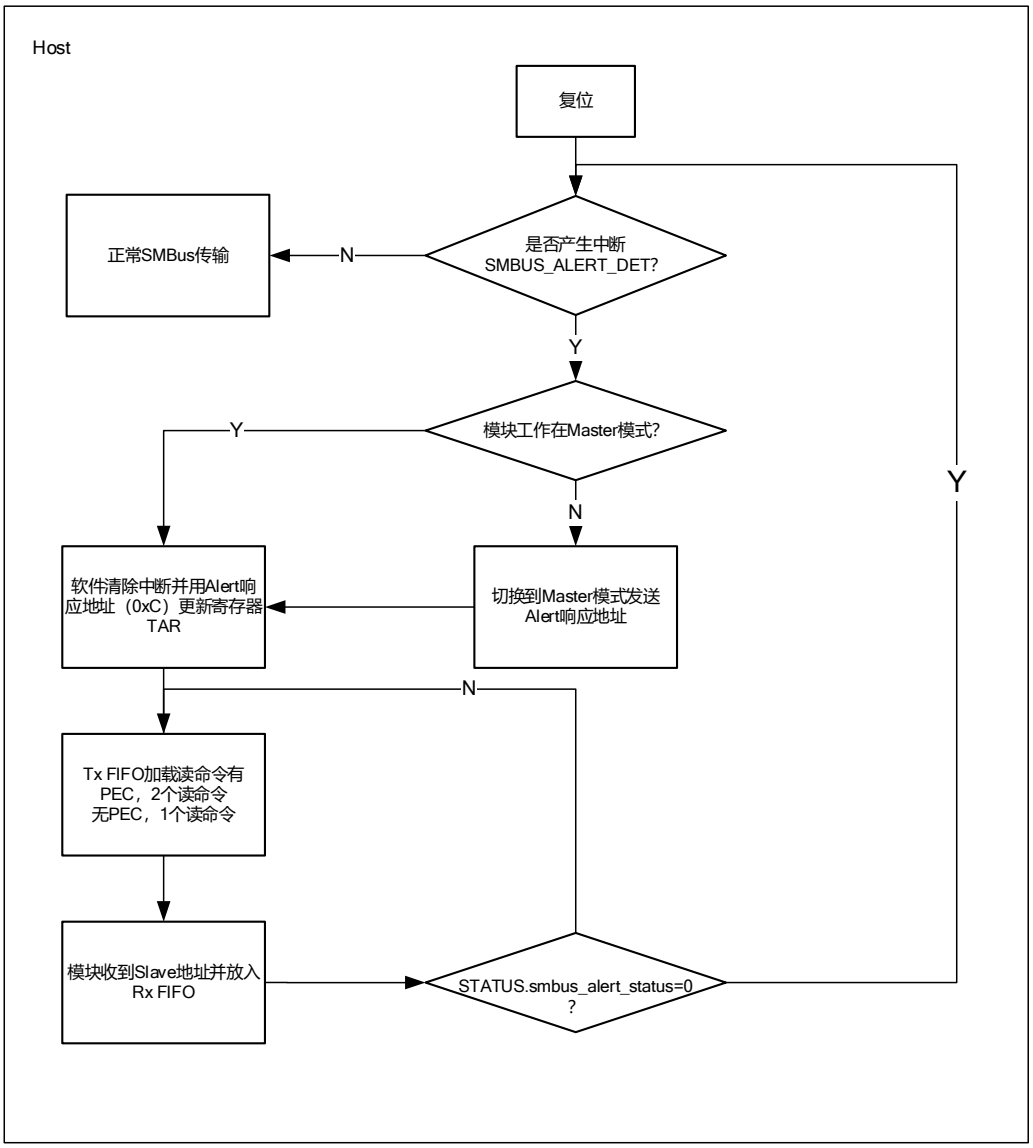
步骤 5 读取原始中断寄存器 RAW_INTR_STAT，如果 RAW_INTR_STAT.scl_stuck_at_low 中断为 1，则跳转步骤 6；否则跳转步骤 7。

步骤 6 SCL 和 SDA 硬件自动复位，清空内部 FIFO。

步骤 7 正常传输。

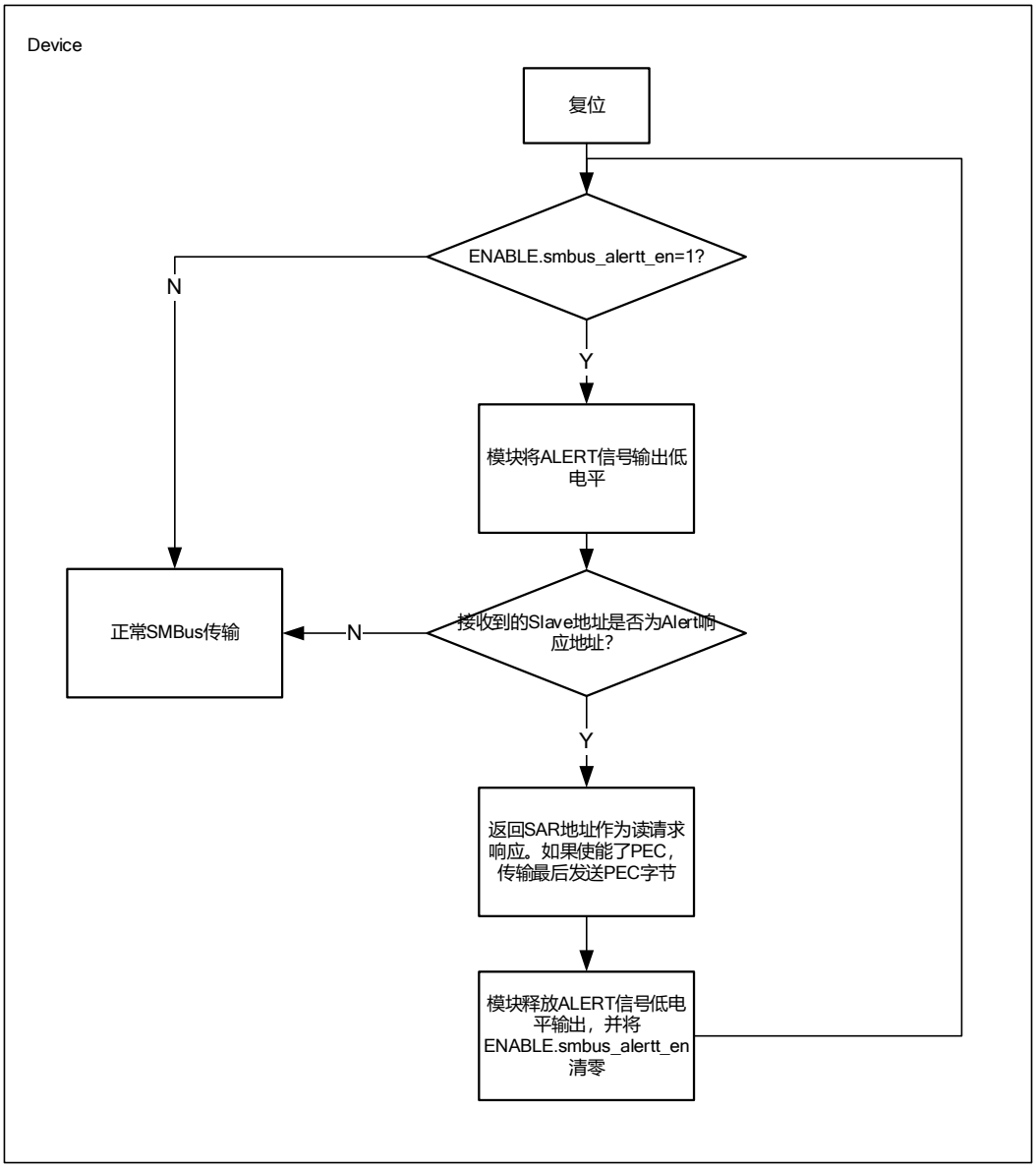
23.6.12 Host 模式下的 SMBus Alert 信号处理

图 23-12 Host 模式下的 SMBus Alert 信号处理



23.6.13 Device 模式下的 SMBus Alert 信号处理

图 23-13 Device 模式下的 SMBus Alert 信号处理



23.7 I2C 寄存器概览

I2C 实例 2 个，其基地址如下。

- I2C0 寄存器基地址：0x60000000
- I2C1 寄存器基地址：0x60001000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CON	0x0	32	RW	0x00000024	I2C 控制寄存器
2	TAR	0x4	32	RW	0x00000055	I2C 目标地址寄存器
3	SAR	0x8	32	RW	0x00000055	I2C Slave 地址寄存器
4	DATA_CMD	0x10	32	RW	0x00000000	I2C RX/TX 数据缓存与命令寄存器
5	SS_SCL_HCNT	0x14	32	RW	0x000001F4	标准速率模式下 I2C SCL 高电平计数寄存器
6	SS_SCL_LCNT	0x18	32	RW	0x0000024C	标准速率模式下 I2C SCL 低电平计数寄存器
7	FS_SCL_HCNT	0x1C	32	RW	0x00000004B	快速模式或超快速模式下I2C SCL高电平计数寄存器
8	FS_SCL_LCNT	0x20	32	RW	0x000000A3	快速模式或超快速模式下I2C SCL低电平计数寄存器
9	INTR_STAT	0x2C	32	RO	0x00000000	I2C 中断状态寄存器
10	INTR_MASK	0x30	32	RW	0x000048FF	I2C 中断屏蔽寄存器中断被屏蔽后，不会上报到 INTR_STAT 寄存器
11	RAW_INTR_STAT	0x34	32	RO	0x00000000	I2C 原始中断状态寄存器
12	RX_TL	0x38	32	RW	0x00000004	I2C 接收 FIFO 阈值寄存器
13	TX_TL	0x3C	32	RW	0x00000004	I2C 传输 FIFO 阈值寄存器
14	CLR_INTR	0x40	32	RO	0x00000000	中断清除寄存器
15	CLR_RX_UNDER	0x44	32	RO	0x00000000	rx_under 中断清除寄存器

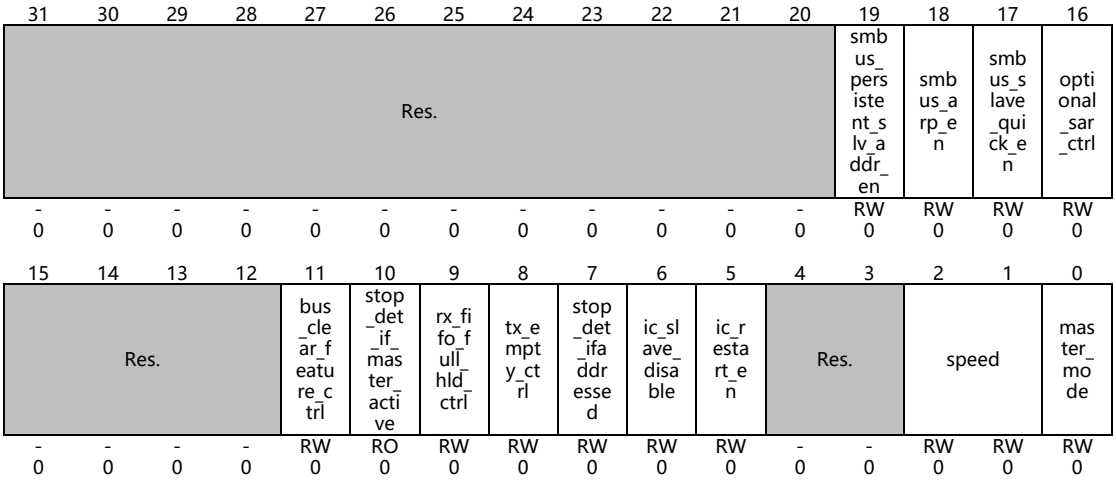
No.	Names	Offset Address	SIZE	R/W	Default	Description
16	CLR_RX_OVER	0x48	32	RO	0x00000000	rx_over 中断清除寄存器
17	CLR_TX_OVER	0x4C	32	RO	0x00000000	tx_over 中断清除寄存器
18	CLR_RD_REQ	0x50	32	RO	0x00000000	rd_req 中断清除寄存器
19	CLR_TX_ABRT	0x54	32	RO	0x00000000	tx_abrt 中断清除寄存器
20	CLR_RX_DONE	0x58	32	RO	0x00000000	rx_done 中断清除寄存器
21	CLR_ACTIVITY	0x5C	32	RO	0x00000000	activity 中断清除寄存器
22	CLR_STOP_DET	0x60	32	RO	0x00000000	stop_det 中断清除寄存器
23	CLR_START_DET	0x64	32	RO	0x00000000	start_det 中断清除寄存器
24	CLR_GEN_CALL	0x68	32	RO	0x00000000	gen_call 中断清除寄存器
25	ENABLE	0x6C	32	RW	0x00000000	I2C 使能寄存器
26	STATUS	0x70	32	RO	0x00000006	I2C 状态寄存器
27	TXFLR	0x74	32	RO	0x00000000	I2C 发送 FIFO 等级寄存器
28	RXFLR	0x78	32	RO	0x00000000	I2C 接收 FIFO 等级寄存器
29	SDA_HOLD	0x7C	32	RW	0x00000001	I2C SDA 保持时间长度寄存器
30	TX_ABRT_SOURCE	0x80	32	RO	0x00000000	I2C 传输中止源寄存器
31	Reserved	0x84	32	-	0x00000000	保留。寄存器
32	DMA_CR	0x88	32	RW	0x00000000	DMA 控制寄存器
33	DMA_TDLR	0x8C	32	RW	0x00000000	DMA 发送数据阈值寄存器
34	DMA_RDLR	0x90	32	RW	0x00000000	DMA 接收数据阈值寄存器
35	SDA_SETUP	0x94	32	RW	0x00000064	I2C SDA 设置寄存器
36	ACK_GENERAL_CALL	0x98	32	RW	0x00000001	I2C 广播响应寄存器
37	ENABLE_STATUS	0x9C	32	RO	0x00000000	I2C 使能状态寄存器
38	FS_SPKLEN	0xA0	32	RW	0x00000007	I2C SS,FS 或 FM+峰值抑制极限寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
39	CLR_RESTART_DET	0xA8	32	RO	0x0000000	RESTART_DET 中断清除寄存器
40	SCL_STUCK_AT_LOW_TIMEOUT	0xAC	32	RW	0xFFFFFFFF	I2C SCL 低超时寄存器
41	SDA_STUCK_AT_LOW_TIMEOUT	0xB0	32	RW	0xFFFFFFFF	I2C SDA 低超时寄存器
42	CLR_SCL_STUCK_DET	0xB4	32	RO	0x00000000	SCL_STUCK_AT_LOW 中断清除寄存器
43	DEVICE_ID	0xB8	32	RO	0x00000000	I2C Device-ID 寄存器
44	SMBUS_CLK_LOW_SEXT	0xBC	32	RW	0xFFFFFFFF	SMBus Slave 时钟扩展超时门限寄存器
45	SMBUS_CLK_LOW_MEXT	0xC0	32	RW	0xFFFFFFFF	SMBus Master 时钟扩展超时门限寄存器
46	SMBUS_THIGH_MAX_IDLE_COUNT	0xC4	32	RW	0x0000FFFF	SMBus Master 总线空闲状态时间段设置寄存器
47	SMBUS_INTR_STAT	0xC8	32	RO	0x00000000	SMBus 中断状态寄存器
48	SMBUS_INTR_MASK	0xCC	32	RW	0x000007FF	SMBus 中断屏蔽寄存器
49	SMBUS_RAW_INTR_STAT	0xD0	32	RO	0x00000000	SMBus 原始中断上报寄存器
50	CLR_SMBUS_INTR	0xD4	32	RW	0x00000000	SMBus 清中断寄存器
51	OPTIONAL_SAR	0xD8	32	RW	0x00000000	I2C 可选 Slave 地址寄存器
52	SMBUS_UDID_LSB	0xDC	32	RW	0xFFFFFFFF	SMBUS ARP UDID LSB 寄存器

23.8 I2C 寄存器描述

23.8.1 CON

- **偏移地址：0x0　　复位值：0x00000024**



Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
smbus_persistent_slv_addr_en	[19]	RW	SMbus 永久 Slave（Persistent Slave，PSA）使能控制： 0：SMBus PSA 不使能； 1：SMBus PSA 使能。 注意：仅在 Slave 模式下可用。	0x0
smbus_arp_en	[18]	RW	SMBus 地址解析功能（ARP）使能控制。Slave 模式下将解码地址解析协议命令并做出响应，还包括为 ARP 命令产生 PEC 字节以及正确性检查。 仅在 Slave 模式下可用。 0：SMBus ARP 关闭； 1：SMBus ARP 使能。	0x0
smbus_slave_quick_en	[17]	RW	Slave 模式下，SMBus 快速命令（Quick commands）使能控制： 0：不使能，此模式下，接收除快速命令之外的所有 SMBus 命令； 1：使能，此模式下仅接收快速命令。	0x0
optional_sar_ctrl	[16]	RW	可选 slave 地址（OPTIONAL_SAR）使能控制： 0：不使能 OPTIONAL_SAR； 1：使能 OPTIONAL_SAR。 注意：启用 OPTIONAL_SAR 功能时，必须先配置一个有效的设备地址到 OPTIONAL_SAR 寄存器，再使能 OPTIONAL_SAR。	0x0
reserved	[15:12]	-	保留。	0x0
bus_clear_feature_ctrl	[11]	RW	总线清除功能（bus clear）使能控制： 0：不使能； 1：使能。	0x0

Name	Bits	R/W	Description	Default
			注意：此功能仅 Mastet 模式下可用，Slave 模式下不可用。	
stop_det_if_master_active	[10]	RO	Master 模式下，stop_det 中断信号控制： 0：仅在 Master 处于活动状态时才能发出 stop_det 中断； 1：无论 Master 是否处于活动状态，都能发出 stop_det 中断。	0x0
rx_fifo_full_hld_ctrl	[9]	RW	当 RX_FIFO 满时，I2C 控制器的行为： 0：RX FIFO 数据溢出； 1：保持住总线，直到 RX FIFO 有空间后才允许接收。	0x0
tx_empty_ctrl	[8]	RW	tx_empty 中断的生成方式选择： 0：当 TX FIFO 中的缓存不高于寄存器 TX_TL 中设定的水线时，产生 tx_empty 中断； 1：当 TX FIFO 中的缓存不高于寄存器 TX_TL 中设定的水线，且来自内部移位寄存器的地址/数据也传输完成时，产生 tx_empty 中断。	0x0
stop_det_ifaddressed	[7]	RW	Slave 模式下，stop_det 中断信号控制。 0：无论寻址与否，都会发出 stop_det 中断 1：仅在被寻址时发出 stop_det 中断 注意：在响应广播地址时，设定为 1，即使返回 ACK 响应广播地址也不会发出 stop_det 中断。stop_det 中断仅在传输的地址与 Slave 地址匹配时生成（SAR）。	0x0
ic_slave_disable	[6]	RW	Slave 模式使能控制： 0：Slave 模式使能； 1：Slave 模式关闭。	0x0
ic_restart_en	[5]	RW	Master 模式，restart 功能使能控制： 0：Master restart 关闭； 1：Master restart 使能。	0x1
reserved	[4:3]	-	保留。	0x0
speed	[2:1]	RW	I2C 速率选择： 0：禁止； 1：标准模式（100kbit/s）； 2：快速模式（不超过 400kbit/s）或者超快速模式（不超过 1000kbit/s）； 3：高速模式，当前模块不支持；	0x2
master_mode	[0]	RW	Master 模式使能控制： 0：Master 模式关闭； 1：Master 模式使能。	0x0

Name	Bits	R/W	Description	Default
			注意：当模块作为 Master 时，该字段配置 1，ic_slave_disable 配置 1；当模块作为 Slave 时，该字段配置 0，ic_slave_disable 配置 0。	

23.8.2 TAR

◆ **偏移地址：0x4　　复位值：0x00000055**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															smbus_quick_cmd
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

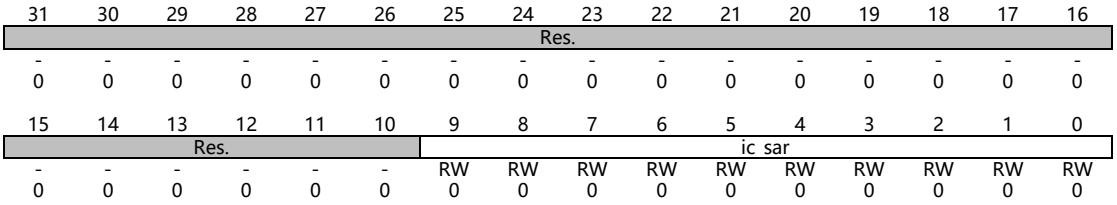
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		device_id	Res.	special	gc_or_start	ic_tar									
-	-	RW	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
smbus_quick_cmd	[16]	RW	当 special 为 1 时，用于使能 smbus 快速命令发送： 0：不使能 smbus 快速命令； 1：使能 smbus 快速命令。	0x0
reserved	[15:14]	-	保留。	0x0
device_id	[13]	RW	当 special 为 1 时，用于使能 Master 模式下的 Device-ID 传输： 0：不使能， 1：使能。 注意：作为 Slave 时，可以读取 DEVICE_ID 寄存器来获取 Device ID。	0x0
reserved	[12]	-	保留。	0x0
special	[11]	RW	用于使能是否发送 Device-ID、广播命令、START 字节命令： 0：不使能，此时将正常发送 ic_tar 配置的设备地址； 1：使能。	0x0
gc_or_start	[10]	RW	当 special=1 且 device_id=0 时，用于控制发送广播命令还是 START 字节： 0：广播命令； 1：START 字节。	0x0

Name	Bits	R/W	Description	Default
ic_tar	[9:0]	RW	Master 模式下，用于配置从设备地址。当发送广播地址时，忽略该域段；当产生 START 字节时，需要配置一次该域段。	0x55

23.8.3 SAR

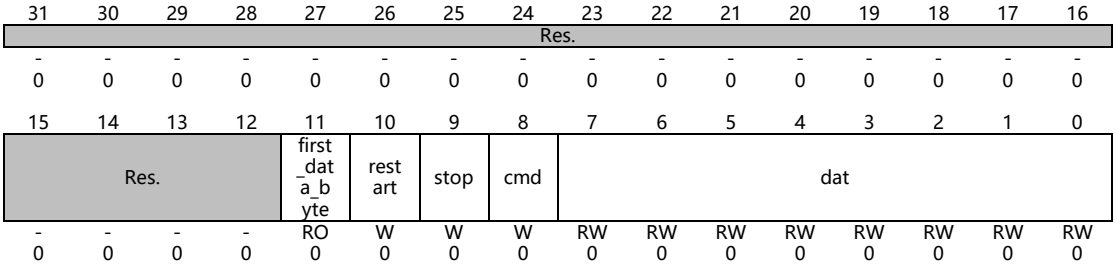
◆ **偏移地址：0x8　　复位值：0x00000055**



Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留。	0x0
ic_sar	[9:0]	RW	模块作为 Slave 时的地址，只能在 ENABLE.enable=0 时配置。不要配置为 I2C/SMBus 的保留地址。	0x55

23.8.4 DATA_CMD

◆ **偏移地址：0x10　　复位值：0x00000000**

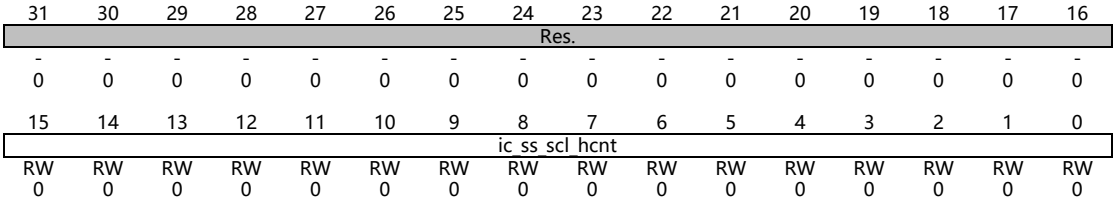


Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留。	0x0
first_data_byte	[11]	RO	指示在 Master 接收数据或者 Slave 接收数据时，接收到的数据是否是跟地址之后的第一个字节数据： 0：接收到的数据不是地址之后的第一个字节； 1：接收到的数据是地址之后的第一个字节。	0x0
restart	[10]	W	在字节发送或者接收前，是否发送 RESTART 条件： 0：当前命令前不发送 RESTART；	0x0

Name	Bits	R/W	Description	Default
			1：当前命令前发送 RESTART。	
stop	[9]	W	在发送或者接收一字节数据后，是否发送 STOP 条件： 0：不发送 STOP 条件。如果 TX FIFO 不为空，则继续发送；如果为空，保持 SCL 为低电平直到有新数据进入 TX FIFO。 1：发送 STOP 条件。如果 TX FIFO 不为空，Master 将立即尝试发送 START 发起新的传输，并竞争总线；如果为空，则停止传输。	0x0
cmd	[8]	W	Master 模式下，控制读写方向： 0：Master 写命令； 1：Master 读命令。 Slave 模式接收数据时，不关心该 bit 位；在 Slave 模式发送数据时，该位要填 0，表示要传输 DATA_CMD[7:0]中的数据。 注意： 1、除非 TAR.special 已被清除，否则在发送广播命令后尝试执行读取操作会导致 TX_ABRT 中断（RAW_INTR_STAT 的 bit6） 2、如果在接收到 RD_REQ 中断后向该位写入 1，则发生 TX_ABRT 中断	0x0
dat	[7:0]	RW	I2C 总线上要发送或者接收的数据。当发送读命令时，该域段被忽略。	0x0

23.8.5 SS_SCL_HCNT

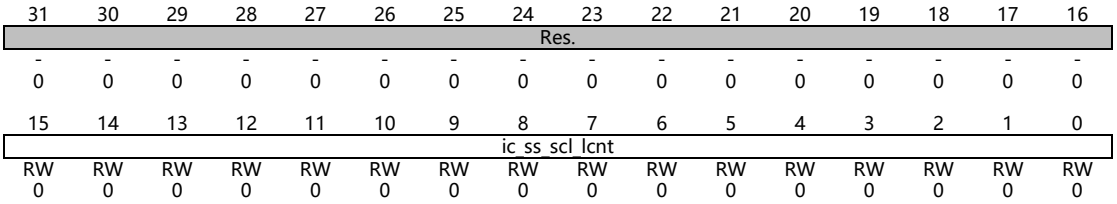
◆ **偏移地址：0x14　　复位值：0x000001F4**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
ic_ss_scl_hcnt	[15:0]	RW	标准模式下的 SCL 信号高电平持续时间，单位是 ic_clk 的周期，配置值不能小于 6，也不能超过 65525。 ENABLE.enable =0 时才可以配置	0x1F4

23.8.6 SS_SCL_LCNT

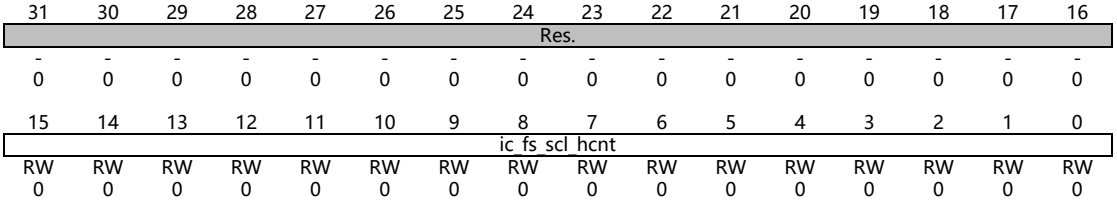
◆ **偏移地址：0x18　　复位值：0x0000024C**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
ic_ss_scl_lcnt	[15:0]	RW	标准模式下的 SCL 信号低电平持续时间，单位是 ic_clk 的周期，配置值不能小于 8。 ENABLE.enable =0 时才可以配置。	0x24C

23.8.7 FS_SCL_HCNT

◆ **偏移地址：0x1C　　复位值：0x0000004B**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
ic_fs_scl_hcnt	[15:0]	RW	快速模式或超快速模式下的 SCL 信号高电平持续时间，单位是 ic_clk 的周期，配置值不能小于 6。 当前最高速率是标准模式时，只读，返回值 0。 ENABLE.enable =0 时才可以配置。	0x4B

23.8.8 FS_SCL_LCNT

◆ **偏移地址：0x20　　复位值：0x000000A3**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ic_fs_scl_lcnt															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
ic_fs_scl_lcnt	[15:0]	RW	快速模式或超快速模式下的 SCL 信号低电平持续时间，单位是 ic_clk 的周期，配置值不能小于 8。 当前最高速率是标准模式时，只读，返回值 0。 ENABLE.enable=0 时才可以配置。	0xA3

23.8.9 INTR_STAT

◆ **偏移地址：0x2C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	r_scl_stuck_at_low	r_master_on_hold	r_restart_det	r_gen_call	r_start_det	r_stop_det	r_activity	r_rx_done	r_tx_abrt	r_rd_req	r_tx_empty	r_tx_over	r_rx_full	r_rx_over	r_rx_under
-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留。	0x0
r_scl_stuck_at_low	[14]	RO	scl_stuck_at_low 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_master_on_hold	[13]	RO	master_on_hold 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_restart_det	[12]	RO	restart_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_gen_call	[11]	RO	gen_calL 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_start_det	[10]	RO	start_det 屏蔽后的上报结果：	0x0

Name	Bits	R/W	Description	Default
			0：中断事件未发生； 1：中断事件发生。	
r_stop_det	[9]	RO	stop_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_activity	[8]	RO	activity 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_rx_done	[7]	RO	rx_done 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_tx_abrt	[6]	RO	tx_abrt 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_rd_req	[5]	RO	rd_req 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_tx_empty	[4]	RO	tx_empty 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_tx_over	[3]	RO	tx_over 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_rx_full	[2]	RO	rx_full 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_rx_over	[1]	RO	rx_over 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_rx_under	[0]	RO	rx_under 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0

23.8.10 INTR_MASK

◆ **偏移地址：0x30　　复位值：0x000048FF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	m_scl_stuck_at_low	m_master_on_hold	m_restart_det	m_gen_call	m_start_det	m_stop_det	m_activity	m_rx_done	m_tx_abrt	m_rd_req	m_tx_empty	m_tx_over	m_rx_full	m_rx_over	m_rx_under
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留。	0x0
m_scl_stuck_at_low	[14]	RW	scl_stuck_at_low 中断屏蔽控制，结果上报 r_scl_stuck_at_low: 0：中断不使能; 1：中断使能。	0x1
m_master_on_hold	[13]	RW	master_on_hold 中断屏蔽控制，结果上报 r_master_on_hold: 0：中断不使能; 1：中断使能。	0x0
m_restart_det	[12]	RW	restart_deT 中断屏蔽控制，结果上报 r_restart_det: 0：中断不使能; 1：中断使能。	0x0
m_gen_call	[11]	RW	gen_call 中断屏蔽控制，结果上报 r_gen_call: 0：中断不使能; 1：中断使能。	0x1
m_start_det	[10]	RW	start_deT 中断屏蔽控制，结果上报 r_start_det: 0：中断不使能; 1：中断使能。	0x0
m_stop_det	[9]	RW	stop_det 中断屏蔽控制，结果上报 r_stop_det: 0：中断不使能; 1：中断使能。	0x0
m_activity	[8]	RW	activity 中断屏蔽控制，结果上报 r_activity: 0：中断不使能; 1：中断使能。	0x0
m_rx_done	[7]	RW	rx_done 中断屏蔽控制，结果上报 r_rx_done: 0：中断不使能; 1：中断使能。	0x1
m_tx_abrt	[6]	RW	tx_abrt 断屏蔽控制，结果上报 r_tx_abrt: 0：中断不使能; 1：中断使能。	0x1

Name	Bits	R/W	Description	Default
m_rd_req	[5]	RW	rd_req 中断屏蔽控制，结果上报 r_rd_req： 0：中断不使能； 1：中断使能。	0x1
m_tx_empty	[4]	RW	tx_empty 中断屏蔽控制，结果上报 r_tx_empty： 0：中断不使能； 1：中断使能。	0x1
m_tx_over	[3]	RW	tx_over 中断屏蔽控制，结果上报 r_tx_over： 0：中断不使能； 1：中断使能。	0x1
m_rx_full	[2]	RW	rx_full 中断屏蔽控制，结果上报 r_rx_full： 0：中断不使能； 1：中断使能。	0x1
m_rx_over	[1]	RW	rx_over 断屏蔽控制，结果上报 r_rx_over： 0：中断不使能； 1：中断使能。	0x1
m_rx_under	[0]	RW	rx_under 中断屏蔽控制，结果上报 r_rx_under： 0：中断不使能； 1：中断使能。	0x1

23.8.11 RAW_INTR_STAT

◆ **偏移地址：0x34　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	scl_stuck_at_low	master_on_hold	restart_det	gen_call	start_det	stop_det	activity	rx_done	tx_abrt	rd_req	tx_empty	tx_over	rx_full	rx_over	rx_under
-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留。	0x0
scl_stuck_at_low	[14]	RO	当 SCL 保持在低电平的时间超过在 SCL_STUCK_LOW_TIMEOUT 寄存器配置的 ic_clk 周期时，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0
master_on_hold	[13]	RO	当 TX FIFO 为空，模块控制 SCL 保持在低电平时，产生该中断： 0：中断事件未发生；	0x0

Name	Bits	R/W	Description	Default
			1：中断事件发生。	
restart_det	[12]	RO	模块作为 Slave 被寻址，RESTART 条件发生时，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0
gen_call	[11]	RO	当接收到广播地址并且响应时，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0
start_det	[10]	RO	无论模块是作为 Slave 还是 Master，当 I2C 总线上产生了 START 或者 RESTART 条件，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0
stop_det	[9]	RO	无论模块是作为 Slave 还是 Master，当 I2C 总线上产生了 STOP 条件时，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0
activity	[8]	RO	当模块处于活动状态时，产生该中断： 0：中断事件未发生； 1：中断事件发生。 注意：如果没有清除，该中断将一直保持，有四种清除方式： 1、关闭模块 2、读 CLR_ACTIVITY 寄存器 3、读 CLR_INTR 寄存器 4、系统复位	0x0
rx_done	[7]	RO	当模块作为 Slave 发送数据时，对端 Master 没有发送 ACK 信号时上报该中断，这发生在传输的最后一个字节，表示传输已完成： 0：中断事件未发生； 1：中断事件发生。	0x0
tx_abrt	[6]	RO	指示当模块在发送数据时，传输异常中止，即无法正常发送完 TX FIFO 内的数据时，产生该中断： 0：中断事件未发生； 1：中断事件发生。 注意：当该中断发生时，TX_ABRT_SOURCE 寄存器指示发生中止的原因。	0x0
rd_req	[5]	RO	当模块作为 Slave 收到读请求时，产生该中断： 0：中断事件未发生； 1：中断事件发生。 注意：模块收到读请求后，将 I2C 总线保持在等待状态（低电平状态），直到处理器将请求的数据写入	0x0

Name	Bits	R/W	Description	Default
			DATA_CMD 寄存器，读取 CLR_RD_REQ 寄存器可该中断。	
tx_empty	[4]	RO	根据CON.tx_empty_ctrl产生中断，具体见该寄存器的描述。当 TX FIFO 的内容超过水线时，逻辑自动清除中断。 0：中断事件未发生； 1：中断事件发生。	0x0
tx_over	[3]	RO	当 TX FIFO 中的数据量到达 FIFO 深度，继续配置 DATA_CMD 寄存器尝试写入命令时，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0
rx_full	[2]	RO	当 RX FIFO 中缓存的数据量达到或者超过 RX_TL 中的设置的阈值时，产生该中断。当低于这个门限后，逻辑自动清除中断。 0：中断事件未发生； 1：中断事件发生。	0x0
rx_over	[1]	RO	当 RX FIFO 中缓存的数据量达到 FIFO 深度，又从总线上收到了数据时，产生该中断： 0：中断事件未发生； 1：中断事件发生。 注意：当 CON.rx_fifo_full_hld_ctrl 配置为 1 时，该中断不会产生。	0x0
rx_under	[0]	RO	当 RX FIFO 为空，软件通过读取 DATA_CMD 寄存器来获取数据时，产生该中断： 0：中断事件未发生； 1：中断事件发生。	0x0

23.8.12 RX_TL

◆ **偏移地址：0x38　　复位值：0x00000004**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													rx_tl		
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0

Name	Bits	R/W	Description	Default
rx_tl	[2:0]	RW	RX FIFO 阈值门限设置，用于控制 rx_full 中断的产生，当进入 RX FIFO 的数据量大于等于这个阈值时，会产生 rx_full 中断。配置值范围为 0~7，实际阈值为配置值加 1，即实际阈值范围是 1~8。	0x4

23.8.13 TX_TL

◆ **偏移地址：0x3C　　复位值：0x00000004**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													tx_tl		
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
tx_tl	[2:0]	RW	TX FIFO 阈值门限设置，用于控制 tx_empty 中断的产生，当 TX FIFO 中的数据量低于等于这个阈值时，会产生 tx_empty 中断。配置值范围为 0~7，实际阈值也是 0~7。	0x4

23.8.14 CLR_INTR

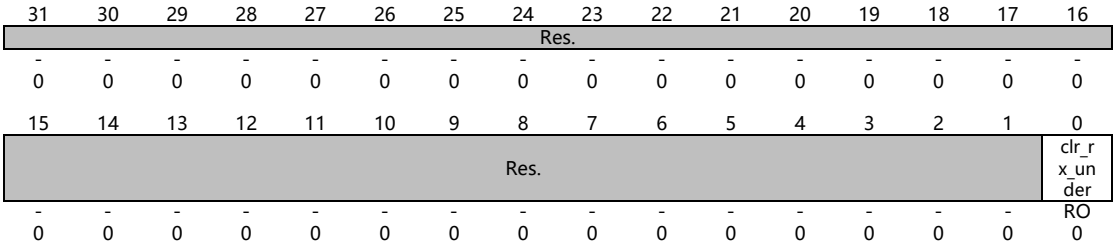
◆ **偏移地址：0x40　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															clr_intr
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_intr	[0]	RO	读寄存器可以清除 RAW_INTR_STAT、SMBUS_RAW_INTR_STAT 中断寄存器，以及 TX_ABRT_SOURCE 寄存器。	0x0

23.8.15 CLR_RX_UNDER

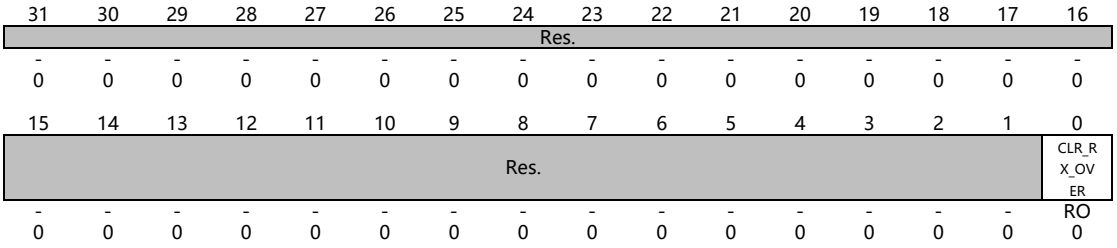
◆ **偏移地址：0x44　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_rx_under	[0]	RO	读寄存器清除 rx_under 中断。	0x0

23.8.16 CLR_RX_OVER

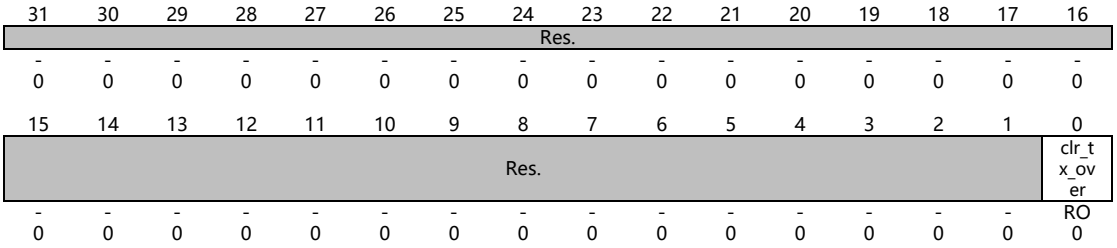
◆ **偏移地址：0x48　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_rx_over	[0]	RO	读寄存器清除 rx_over 中断。	0x0

23.8.17 CLR_TX_OVER

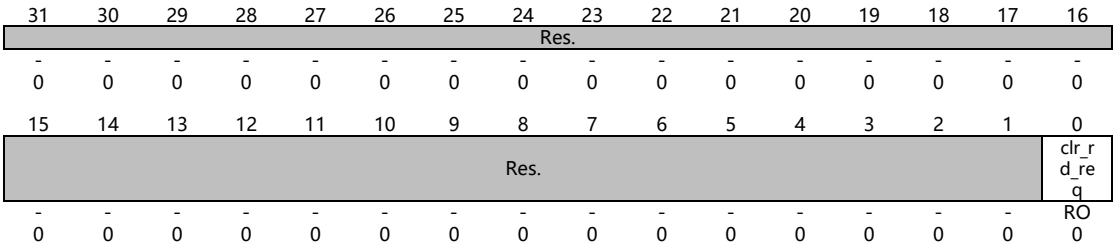
◆ **偏移地址：0x4C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_tx_over	[0]	RO	读寄存器清除 tx_over 中断。	0x0

23.8.18 CLR_RD_REQ

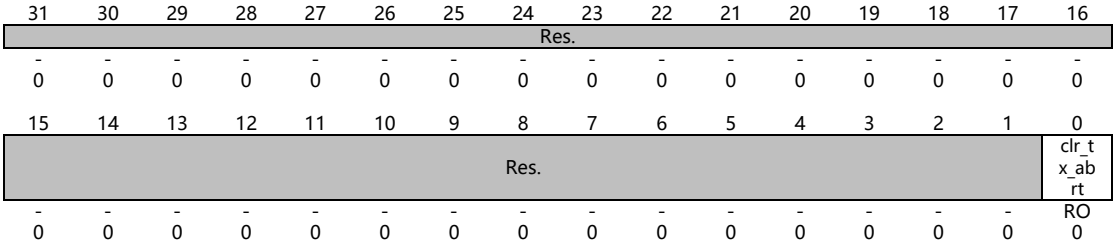
◆ **偏移地址：0x50　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_rd_req	[0]	RO	读寄存器清除 rd_req 中断。	0x0

23.8.19 CLR_TX_ABRT

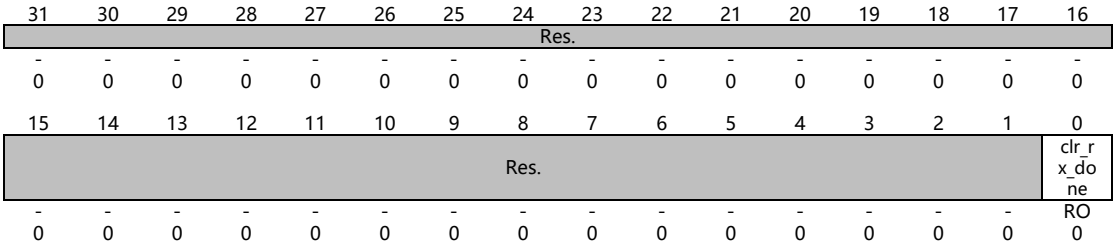
◆ **偏移地址：0x54　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_tx_abrt	[0]	RO	读寄存器清除 tx_abrt 中断。	0x0

23.8.20 CLR_RX_DONE

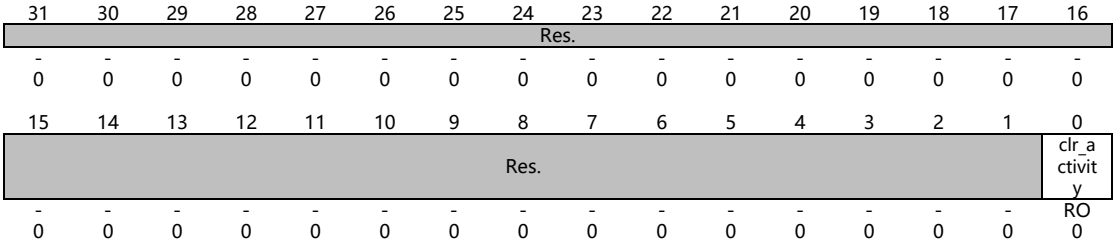
◆ **偏移地址：0x58　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_rx_done	[0]	RO	读寄存器清除 rx_done 中断。	0x0

23.8.21 CLR_ACTIVITY

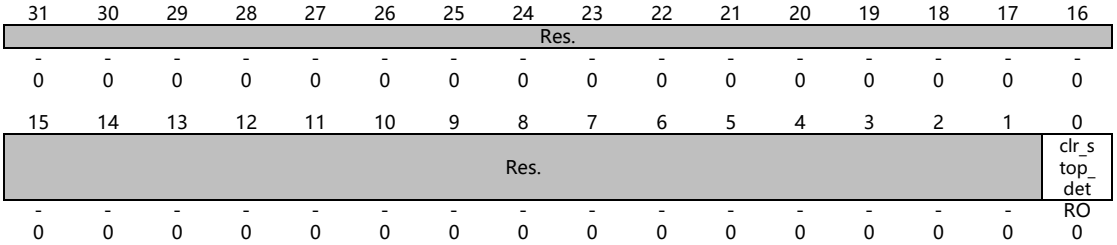
◆ **偏移地址：0x5C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_activity	[0]	RO	读寄存器清除 activity 中断。	0x0

23.8.22 CLR_STOP_DET

◆ **偏移地址：0x60　　复位值：0x00000000**

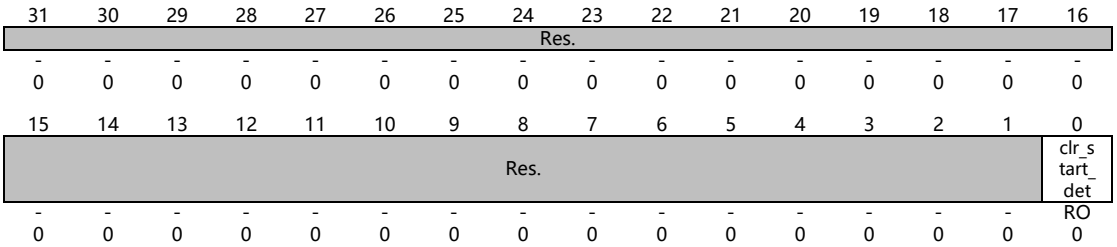


Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0

Name	Bits	R/W	Description	Default
clr_stop_det	[0]	RO	读寄存器清除 stop_det 中断。	0x0

23.8.23 CLR_START_DET

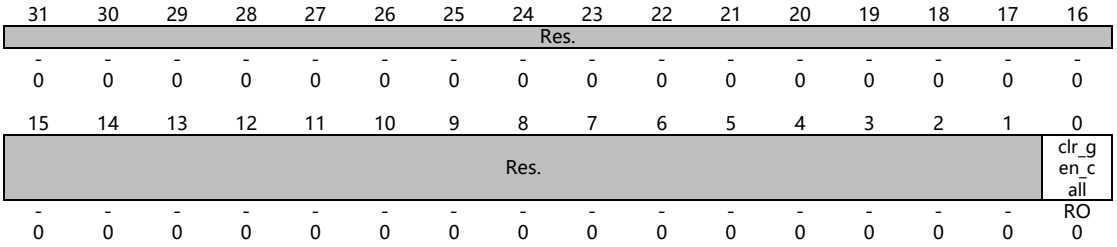
◆ **偏移地址：0x64　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_start_det	[0]	RO	读寄存器清除 start_det 中断。	0x0

23.8.24 CLR_GEN_CALL

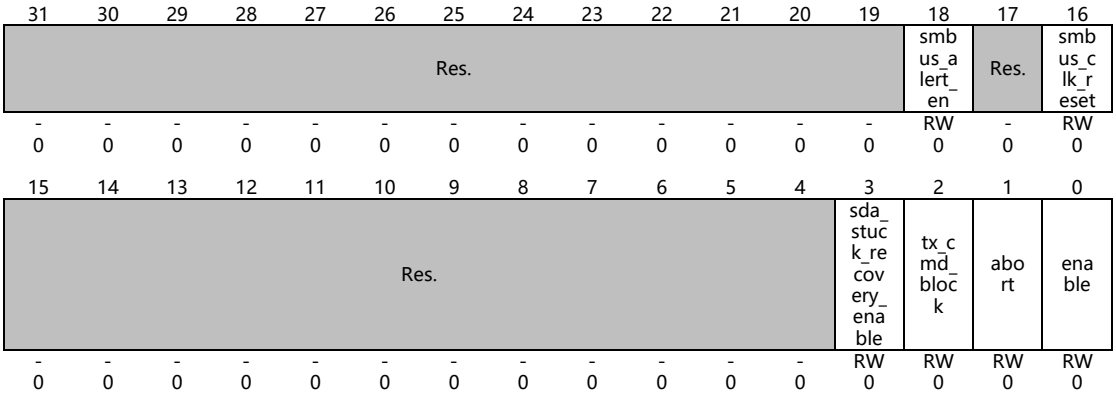
◆ **偏移地址：0x68　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_gen_call	[0]	RO	读寄存器清除 gen_call 中断。	0x0

23.8.25 ENABLE

◆ **偏移地址：0x6C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
smbus_alert_en	[18]	RW	SMBUS Slave 模式下，ALERT 信号产生控制配置。 软件配置 1 时产生 ALERT 信号，该域段在接收到 Master 发送的 ACK 信号后自动清除。 0：不产生 ALERT 信号； 1：产生 ALERT 信号。	0x0
reserved	[17]	-	保留。	0x0
smbus_clk_reset	[16]	RW	SMBus Master 模式下的时钟重置控制。只有 Master 处于空闲状态时，才启用此位。每当该位被启用时，SMBCLK 在 SCL_STUCK_AT_LOW_TIMEOUT 寄存器配置的 ic_clk 周期内保持低电平，以重置 SMBus Slave。 0：不使能； 1：使能。	0x0
reserved	[15:4]	-	保留。	0x0
sda_stuck_recovery_enable	[3]	RW	SDA 恢复机制启用控制，通过 tx_abrt 中断（TX_ABRT_SOURCE.abrt_sda_stuck_at_low）可以知道 SDA 一直处于低电平，可通过此位启动 SDA 恢复机制（即，最多发送 9 个 SCL 时钟并停止以释放 SDA 线路），此位由逻辑自动清除。 0：不启动 SDA 恢复机制； 1：启动 SDA 恢复机制。	0x0
tx_cmd_block	[2]	RW	Master 模式下，I2C 发送数据控制： 0：一旦 TX FIFO 中的第一个数据可用，I2C 模块就自动开始发送数据； 1：停止 I2C 总线上的数据传输，即使 TX FIFO 有数据。 注意：要停止 I2C 总线上的数据传输，仅在 TX FIFO 为空（STATUS.tfe=1）且 Master 处于空闲状态	0x0

Name	Bits	R/W	Description	Default
			（STATUS.mst_activity=0）时，软件设置 tx_cmd_block 为 1。在 tx_cmd_block 置 0 前，禁止写数据到 TX FIFO。	
abort	[1]	RW	I2C 传输中止控制： 0：不中止当前的传输或传输已经中止； 1：中止当前的传输。 I2C 模块作为主器件时，软件可以配置该位以中止 I2C 传输，一旦置位后不能立即清除，置位后 I2C 模块会在完成当前传输后产生 STOP 条件并清空 TX FIFO 并产生 tx_abrt 中断。该 abort 位会在中止操作结束后自动清零。	0x0
enable	[0]	RW	I2C 模块使能开关： 0：不使能； 1：使能。	0x0

23.8.26 STATUS

◆ **偏移地址：0x70　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.											smbus_alert_status	Res.	smbus_slave_addr_resolved	smbus_slave_addr_valid	smbus_quick_cmd_bit
-	-	-	-	-	-	-	-	-	-	-	RO	-	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				sda_stuck_not_recovered	slv_hold_rx_fifo_full	slv_hold_tx_fifo_empty	mst_hold_rx_fifo_full	mst_hold_tx_fifo_empty	slv_activity	mst_activity	rff	rfne	tfe	tfnf	activity
-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
smbus_alert_status	[20]	RO	SMBus Alert 信号状态上报： 0：没有产生 alert 信号； 1：产生 alert 信号。	0x0
reserved	[19]	-	保留。	0x0
smbus_slave_addr_resolved	[18]	RO	指示 ARP Master 是否解析了 Slave 地址： 0：SMBUS Slave 地址未解析； 1：SMBUS Slave 地址已解析。	0x0

Name	Bits	R/W	Description	Default
smbus_slave_addr_valid	[17]	RO	指示 Slave 地址是否可用： 0：SMBUS Slave 地址不可用； 1：SMBUS Slave 地址可用。	0x0
smbus_quick_cmd_bit	[16]	RO	指示接收到的 SMBUS 快速命令中的读写位： 0：SMBUS 快速命令的读写位是 0； 1：SMBUS 快速命令的读写位是 1； 注意：用户读取这个域段后会清 0。	0x0
reserved	[15:12]	-	保留。	0x0
sda_stuck_not_recovered	[11]	RO	Master 模式下，在启用 SDA 恢复机制后，SDA 线的恢复状态： 0：SDA 已恢复正常； 1：SDA 未恢复正常。 注意：在 Slave 模式下，该域段不可用。	0x0
slv_hold_rx_fifo_full	[10]	RO	Slave 模式下，RX FIFO 已满且接收到额外字节而导致总线保持的情况： 0：Slave没有保持住总线或者总线保持住但不是因为 RX FIFO 满了； 1：Slave 因为 RX FIFO 满而保持住总线。	0x0
slv_hold_tx_fifo_empty	[9]	RO	Slave 模式下，因为 TX FIFO 空而将总线保持住的情况： 0：Slave没有保持住总线或者总线保持住但不是因为 TX FIFO 空了； 1：Slave 因为 TX FIFO 空而保持住总线。	0x0
mst_hold_rx_fifo_full	[8]	RO	Master 模式下,RX FIFO 已满且接收到额外字节而导致总线保持的情况： 0：Master 没有保持住总线或者显现保持住但不是因为 RX FIFO 满了； 1：Master 因为 RX FIFO 满保持住总线。	0x0
mst_hold_tx_fifo_empty	[7]	RO	Master 模式下，在没有设置 STOP 位时，因为 TX FIFO 空而导致 Master 保持住总线的情况： 0：Master 没有保持住总线或者总线保持住不是因为 TX FIFO 空了； 1：Master 因为 TX FIFO 空保持住总线。	0x0
slv_activity	[6]	RO	Slave 状态机活动状态上报： 0：Slave 状态机是 IDLE 态； 1：Slave 状态机不是 IDLE 态，模块在活动中。	0x0
mst_activity	[5]	RO	Master 状态机活动状态上报： 0：Master 状态机是 IDLE 态； 1：Master 状态机不是 IDLE 态，模块在活动中。	0x0
rff	[4]	RO	RX FIFO 完全满状态上报： 0：RX FIFO 未满；	0x0

Name	Bits	R/W	Description	Default
			1：RX FIFO 已满。 注意：当 RX FIFO 出现空位时，自动清零。	
rfne	[3]	RO	RX FIFO 非空状态上报： 0：RX FIFO 空； 1：RX FIFO 非空。	0x0
tfe	[2]	RO	TX FIFO 完全空状态上报： 0：TX FIFO 非空； 1：TX FIFO 空。 注意：当有数据进入 TX FIFO 时，自动清零。	0x1
tfnf	[1]	RO	TX FIFO 非满状态上报： 0：TX FIFO 已满； 1：TX FIFO 未满。	0x1
activity	[0]	RO	I2C 模块活动状态上报： 0：处于 IDLE 态； 1：处于活动中。	0x0

23.8.27 TXFLR

◆ **偏移地址：0x74　　复位值：0x00000000**

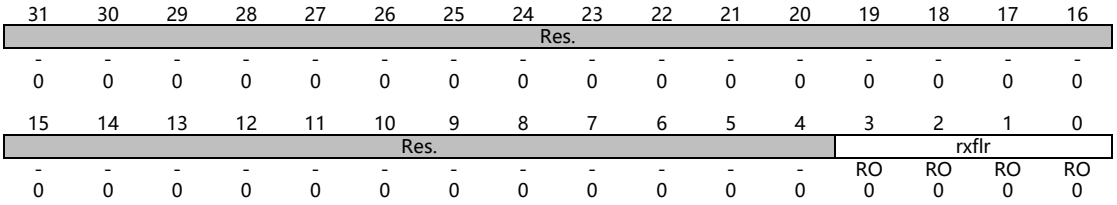
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												txflr			
-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
txflr	[3:0]	RO	TX FIFO 中可用数据的数量。	0x0

23.8.28 RXFLR

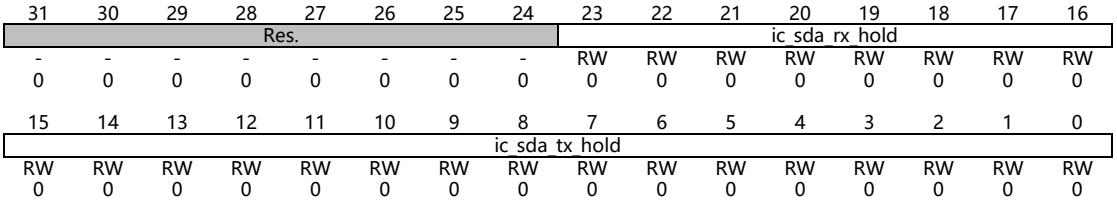
◆ **偏移地址：0x78　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
rxflr	[3:0]	RO	RX FIFO 中可用数据的数量。	0x0

23.8.29 SDA_HOLD

◆ **偏移地址：0x7C　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
ic_sda_rx_hold	[23:16]	RW	模块作为 Master 接收数据或 Slave 接收数据时，配置 SDA 的保持时间，单位是 ic_clk 周期，ENABLE.enable=0 时该寄存器可配。	0x0
ic_sda_tx_hold	[15:0]	RW	模块作为 Master 发送数据或 Slave 发送数据时，配置 SDA 的保持时间，单位是 ic_clk 周期，ENABLE.enable=0 时该寄存器可配。在任何时候都不能超过 SCL 低电平持续的时间。	0x1

23.8.30 TX_ABRT_SOURCE

◆ **偏移地址：0x80　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
tx_flush_cnt									Res.		abrt_device_write	abrt_device_slvaddr_noack	abrt_device_noack	abrt_sda_stuck_at_low	abrt_user_abrt
RO	RO	RO	RO	RO	RO	RO	RO	RO	-	-	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
abrt_slvrd_intx	abrt_slv_arblost	abrt_slvflush_txfifo	arb_lost	abrt_master_dis	abrt_10b_rd_norstrt	abrt_sbyte_norstrt	Res.	abrt_sbyte_ackdet	Res.	abrt_gcall_read	abrt_gcall_noack	abrt_txdata_noack	Res.		abrt_7b_addr_noack
RO	RO	RO	RO	RO	RO	RO	-	RO	-	RO	RO	RO	-	-	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
tx_flush_cnt	[31:23]	RO	统计模块作为 Master 发送数据或 Slave 发送数据时，由于 tx_abrt 中断而导致 TX FIFO 数据丢失的次数。	0x0
reserved	[22:21]	-	保留。	0x0
abrt_device_write	[20]	RO	模块作为 Master 初始化 DEVICE_ID 时，TX FIFO 中都是写命令： 0：未出现此场景； 1：出现此场景。	0x0
abrt_device_slvaddr_noack	[19]	RO	模块作为 Master 初始化 DEVICE_ID 时，发送的 Slave 地址没有 Slave 确认 0：未出现此场景； 1：出现此场景。	0x0
abrt_device_noack	[18]	RO	模块作为 Master 初始化 DEVICE_ID 时，发送的 device id 没有 Slave 确认： 0：未出现此场景； 1：出现此场景。	0x0
abrt_sda_stuck_at_low	[17]	RO	模块作为 Master 时，检测到 SDA 保持了 SDA_STUCK_AT_LOW_TIMEOUT 个 ic_clk 周期的低电平： 0：未出现此场景； 1：出现此场景。	0x0
abrt_user_abrt	[16]	RO	模块作为 Master 发送数据时，检测到传输中止信号（ENABLE.abort 置 1）： 0：未出现此场景； 1：出现此场景。	0x0
abrt_slvrd_intx	[15]	RO	模块作为 Slave 响应远端 Master 的读请求，Slave 在发送数据时，用户往 DATA_CMD.cmd 写入 1： 0：未出现此场景；	0x0

Name	Bits	R/W	Description	Default
			1：出现此场景。	
abrt_slv_arblost	[14]	RO	模块作为 Slave，在发送数据给远端 Master 时丢失总线控制权： 0：未出现此场景； 1：出现此场景。 注意：此场景发生时，TX_ABRT_SOUTRCE.arb_lost 也会置位。	0x0
abrt_slvflush_txfifo	[13]	RO	模块作为 Slave，接收到一个读命令但此时 TX FIFO 中还存在一些数据，Slave 通过产生 tx_abrt 中断来清空 TX FIFO 中的数据： 0：未出现此场景； 1：出现此场景。	0x0
arb_lost	[12]	RO	模块作为 Master 发送数据时丢失总线控制权，或者 TX_ABRT_SOUTRCE. abrt_slv_arblost 场景出现： 0：未出现此场景； 1：出现此场景。	0x0
abrt_master_dis	[11]	RO	用户在 Master 模式处于关闭状态时执行 Master 初始化操作： 0：未出现此场景； 1：出现此场景。	0x0
abrt_10b_rd_norstrt	[10]	RO	模块作为 Master，当 RESTART 功能关闭时（CON.ic_restart_en=0），模块发送了 10bit 地址模式的读命令： 0：未出现此场景； 1：出现此场景。	0x0
abrt_sbyte_norstrt	[9]	RO	模块作为 Master，当 RESTART 功能关闭时，用户尝试发送 START 字节： 0：未出现此场景； 1：出现此场景。 要清除该位，需要配置 CON.ic_restart_en =1，清除 TAR.special=0 或者 TAR.GC_OR_START=0，之后该位与该寄存器中的其他域段一样，可以用相同的方式清除。	0x0
reserved	[8]	-	保留。	0x0
abrt_sbyte_ackdet	[7]	RO	模块作为 Master 时，发送 START 字节后并被确认（这是错误行为）： 0：START 字节没有 ACK 响应； 1：START 字节检测到 ACK 响应。	0x0
reserved	[6]	-	保留。	0x0

Name	Bits	R/W	Description	Default
abrt_gcall_read	[5]	RO	模块作为 Master，在发送广播后，用户紧接着配置 i2c 总线发送了 1 个读请求（DATA_CMD.cmd=1）： 0：未出现此场景； 1：出现此场景。	0x0
abrt_gcall_noack	[4]	RO	模块作为 Master，发送广播后无 Slave 响应： 0：未出现此场景； 1：出现此场景。	0x0
abrt_txdata_noack	[3]	RO	模块作为 Master，在发送地址被响应后，再发送数据无 Slave 响应： 0：未出现此场景； 1：出现此场景。	0x0
reserved	[2:1]	-	保留。	0x0
abrt_7b_addr_noack	[0]	RO	模块作为 Master 时，发送的 7bit 地址未被任何 Slave 响应： 0：未出现此场景； 1：出现此场景。	0x0

23.8.31 DMA_CR

◆ **偏移地址：0x88　　复位值：0x00000000**

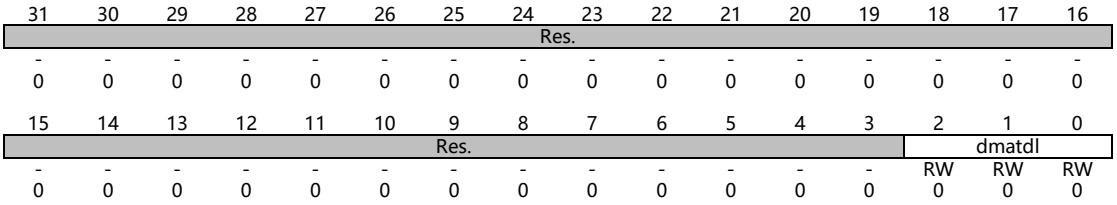
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														tdmae	rdmae
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
tdmae	[1]	RW	TX DMA 使能控制： 0：关闭 TX DMA 通道； 1：使能 TX DMA 通道。	0x0
rdmae	[0]	RW	RX DMA 使能控制： 0：关闭 RX DMA 通道； 1：使能 RX DMA 通道。	0x0

23.8.32 DMA_TDLR

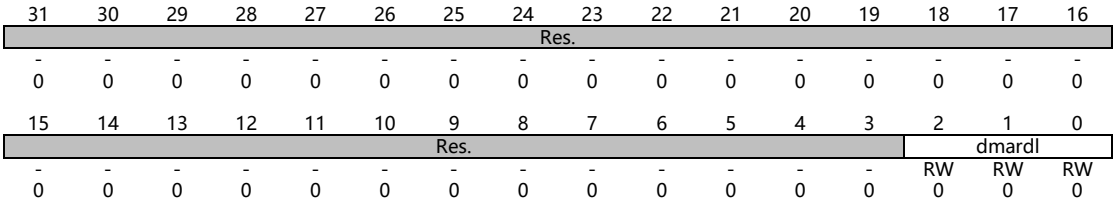
◆ **偏移地址：0x8C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
dmatdl	[2:0]	RW	DMA 发送数据阈值。当 TX FIFO 中的可用数据个数≤dmatdl 并且 DMA_CR.tdmae=1 时，产生 DMA 搬运请求。	0x0

23.8.33 DMA_RDLR

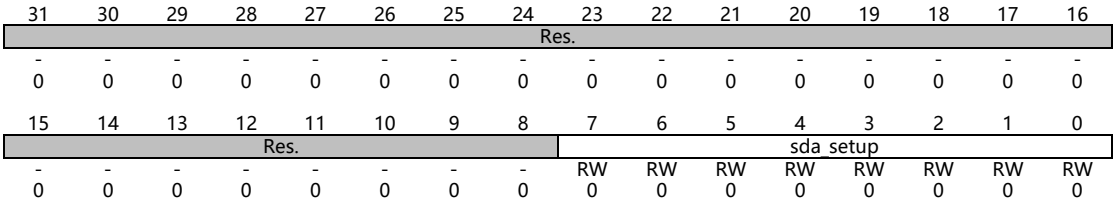
◆ **偏移地址：0x90　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
dmardl	[2:0]	RW	DMA 接收数据阈值。当 RX FIFO 中的可用数据个数≥dmardl+1 并且 DMA_CR.rdmae=1 时，产生 DMA 搬运请求。	0x0

23.8.34 SDA_SETUP

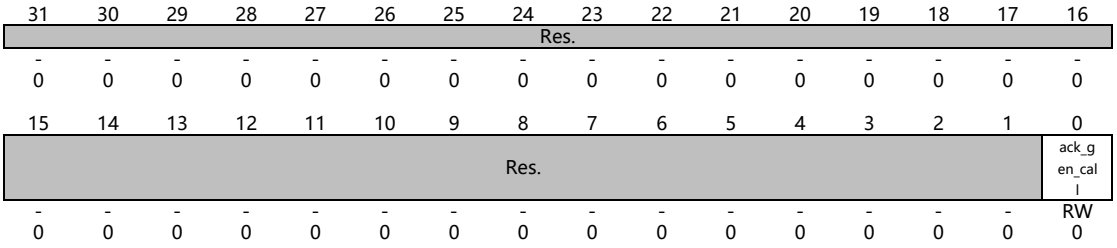
◆ **偏移地址：0x94　　复位值：0x00000064**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
sda_setup	[7:0]	RW	配置 SDA 建立时间。 当模块作为 Slave 响应读操作时需要设定，设定要求见 I2C 总线定义。单位是 ic_clk 的周期，最小值 2。 SDA 的建立时间=(配置值-1)*ic_clk 周期。 ENABLE.enable=0 时才可以配置。	0x64

23.8.35 ACK_GENERAL_CALL

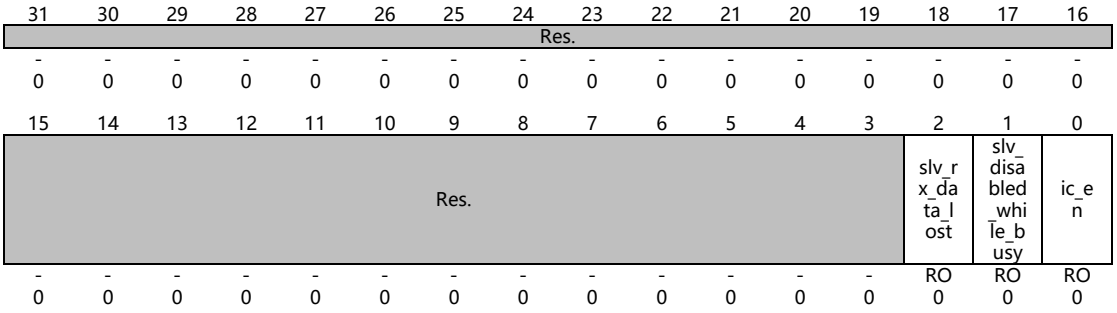
◆ **偏移地址：0x98　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
ack_gen_call	[0]	RW	广播响应返回 ACK 控制。模块作为 Slave 时使用。 0：接收到广播地址时返回 ACK； 1：接收到广播地址时返回 NACK。	0x1

23.8.36 ENABLE_STATUS

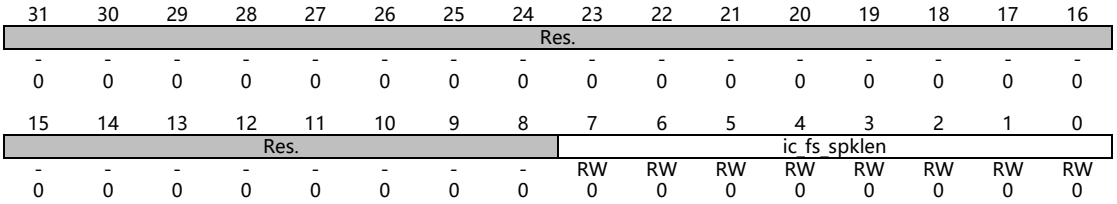
◆ **偏移地址：0x9C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
slv_rx_data_lost	[2]	RO	Slave 接收数据丢失上报。该 bit 位指示 Slave 接收的数据是否由于 ENABLE.enable 从 1 变成 0 导致丢失，此时 Slave 可能已经从 i2c 总线上接收到了至少一个字节。 0：Slave 接收数据未丢失 1：Slave 接收数据丢失 注意：1、在模块返回 NACK 前，远端 Master 发送 STOP 条件停止传输，并且模块 ENABLE.enable 已设置为 0，那么此时上报也是 1； 2、当 ENABLE.enable 是 0 时，本域段仅在本寄存器的 bit0 是 0 的时候有效； 3、当 ENABLE.enable 是 1 时，本域段回读值是 0。	0x0
slv_disabled_while_busy	[1]	RO	当 Slave 处于忙时被关闭状态上报。当 ENABLE.enable 从 1 变成 0 时，Slave 可能正在接收远端 Master 发送的地址或数据，或者是正在处理这些接收到的地址或数据，此时需要停止后续处理。 0：Slave 被关闭时处于 idle 态 1：Slave 被关闭时处于活动状态 注意：1、在模块返回 NACK 前，远端 Master 发送 STOP 条件停止传输，并且模块 ENABLE.enable 已设置为 0，那么此时上报也是 1； 2、当 ENABLE.enable 是 0 时，本域段仅在本寄存器的 bit0 是 0 的时候有效； 3、当 ENABLE.enable 是 1 时，本域段回读值是 0。	0x0
ic_en	[0]	RO	I2C 模块状态： 0：模块处于使能状态 1：模块处于关闭状态 注意：CPU 可以随时读取本域段，当本域段是 0 时，本寄存器的 bit[2:1]才有效。	0x0

23.8.37 FS_SPKLEN

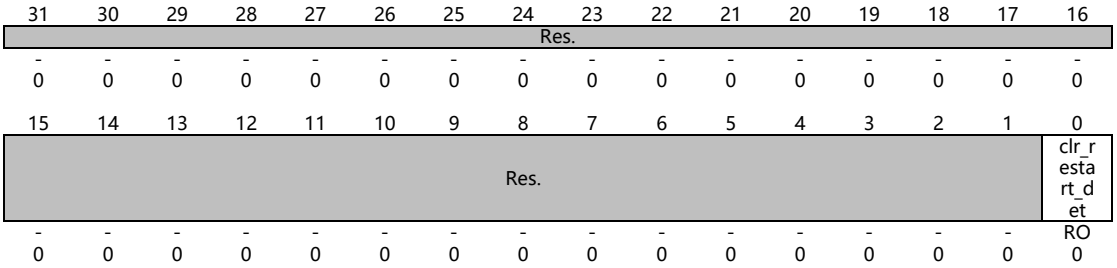
◆ **偏移地址：0xA0　　复位值：0x00000007**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
ic_fs_spklen	[7:0]	RW	I2C 标准模式，快速模式，超快速模式下，对 SCL 或 SDA 线进行毛刺过滤的最大尖峰长度，单位是 ic_clk 周期，最小值 1。 ENABLE.enable=0 时才可以配置。	0x7

23.8.38 CLR_RESTART_DET

◆ **偏移地址：0xA8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_restart_det	[0]	RO	读该寄存器清除 RAW_INTR_STAT.restart_det 中断。	0x0

23.8.39 SCL_STUCK_AT_LOW_TIMEOUT

◆ **偏移地址：0xAC　　复位值：0xFFFFFFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ic_scl_stuck_low_timeout															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ic_scl_stuck_low_timeout															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ic_scl_stuck_low_timeout	[31:0]	RW	SCL 低电平超时门限。当 SCL 保持在低电平的时间达到本寄存器配置个数的 ic_clk 周期时，模块产生 scl_stuck_at_low 中断。 ENABLE.enable=0 时才可以配置。	0xFFFFFFFF

23.8.40 SDA_STUCK_AT_LOW_TIMEOUT

◆ **偏移地址：0xB0　　复位值：0xFFFFFFFF**

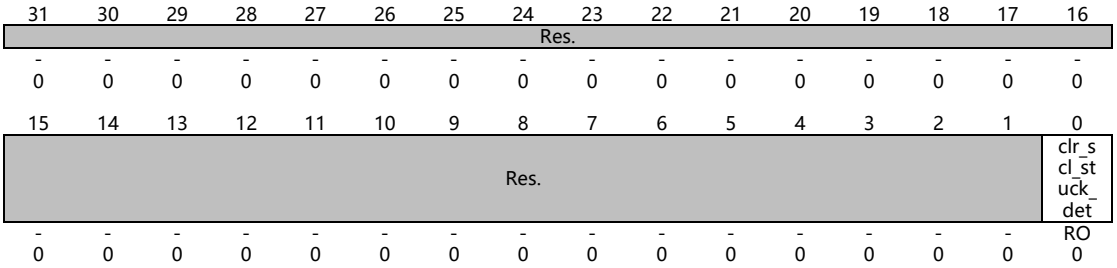
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ic_sda_stuck_low_timeout															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ic_sda_stuck_low_timeout															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ic_sda_stuck_low_timeout	[31:0]	RW	SDA 低电平超时门限。当模块检测到 SDA 线保持在低电平的时间达到本寄存器配置个数的 ic_clk 周期时，通过使能 ENABLE.sda_stuck_recovery_enable 来启动 SDA 线的恢复。	0xFFFFFFFF

23.8.41 CLR_SCL_STUCK_DET

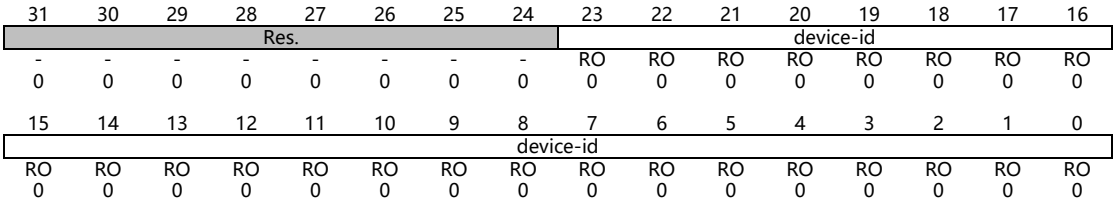
◆ **偏移地址：0xB4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
clr_scl_stuck_det	[0]	RO	读该寄存器清除 RAW_INTR_STAT.scl_stuck_at_low 中断。	0x0

23.8.42 DEVICE_ID

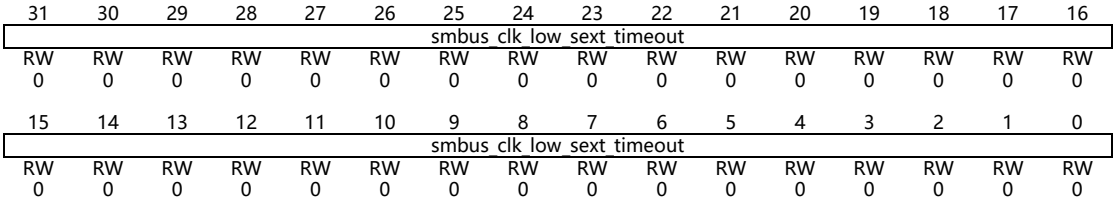
◆ **偏移地址：0xB8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
device-id	[23:0]	RO	设备 ID。	0x0

23.8.43 SMBUS_CLK_LOW_SEXT

◆ **偏移地址：0xBC　　复位值：0xFFFFFFFF**



Name	Bits	R/W	Description	Default
smbus_clk_low_sext_timeout	[31:0]	RW	包含用于确定一次传输（从 START 到 STOP）中的 SMBus Slave 时钟扩展超时的超时值。单位为 ic_clk 周期。 ENABLE.enable=0 时才可以配置。	0xFFFFFFFF

23.8.44 SMBUS_CLK_LOW_MEXT

◆ **偏移地址：0xC0　　复位值：0xFFFFFFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
smbus_clk_low_mext_timeout															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
smbus_clk_low_mext_timeout															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
smbus_clk_low_mext_timeout	[31:0]	RW	包含用于确定传输一个字节中的 SMBus Master 时钟扩展超时的超时值（从 START 到 ACK，ACK 到 ACK，ACK 到 STOP）。单位为 ic_clk 周期。 ENABLE.enable=0 时才可以配置。	0xFFFFFFFF

23.8.45 SMBUS_THIGH_MAX_IDLE_COUNT

◆ **偏移地址：0xC4　　复位值：0x0000FFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
smbus_thigh_max_bus_idle_cnt															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
smbus_thigh_max_bus_idle_cnt	[15:0]	RW	该寄存器用于设定所需的总线 IDLE 时间段。当 Master 已被动态添加到总线并且可能未检测到 SMBCLK 或 SMBDAT 线上的状态转换时使用。在这种情况下，Master 必须等待足够长的时间，以确保当前没有进行传输。单位为 ic_clk 周期。 ENABLE.enable=0 时才可以配置。	0xFFFF

23.8.46 SMBUS_INTR_STAT

◆ **偏移地址：0xC8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					r_smbus_alert_det	r_smbus_suspend_det	r_slv_rx_pec_nack	r_arp_assgn_addr_cmd_det	r_arp_get_udid_cmd_det	r_arp_rst_cmd_det	r_arp_prepare_cmd_det	r_host_notify_mst_det	r_quick_cmd_det	r_mst_clock_extnd_timeout	r_slv_clock_extnd_timeout
-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
r_smbus_alert_det	[10]	RO	smbus_alert_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_smbus_suspend_det	[9]	RO	smbus_suspend_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_slv_rx_pec_nack	[8]	RO	slv_rx_pec_nack 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_arp_assgn_addr_cmd_det	[7]	RO	arp_assgn_addr_cmd_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_arp_get_udid_cmd_det	[6]	RO	arp_get_udid_cmd_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_arp_rst_cmd_det	[5]	RO	arp_rst_cmd_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_arp_prepare_cmd_det	[4]	RO	arp_prepare_cmd_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_host_notify_mst_det	[3]	RO	host_notify_mst_det 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_quick_cmd_det	[2]	RO	quick_cmd_det 屏蔽后的上报结果： 0：中断事件未发生；	0x0

Name	Bits	R/W	Description	Default
			1：中断事件发生。	
r_mst_clock_extnd_timeout	[1]	RO	mst_clock_extnd_timeout 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0
r_slv_clock_extnd_timeout	[0]	RO	slv_clock_extnd_timeout 屏蔽后的上报结果： 0：中断事件未发生； 1：中断事件发生。	0x0

23.8.47 SMBUS_INTR_MASK

◆ **偏移地址：0xCC　　复位值：0x000007FF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					m_smbus_alert_det	m_smbus_suspend_det	m_slv_rx_pec_nack	m_arp_assgn_addr_cmd_det	m_arp_get_uid_cmd_det	m_arp_rst_cmd_det	m_arp_prepare_cmd_det	m_host_notify_mst_det	m_quick_cmd_det	m_mst_clock_extnd_timeout	m_slv_clock_extnd_timeout
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x1
m_smbus_alert_det	[10]	RW	smbus_alert_det 中断屏蔽控制，结果上报 r_smbus_alert_det： 0：中断不使能； 1：中断使能。	0x1
m_smbus_suspend_det	[9]	RW	smbus_suspend_det 中断屏蔽控制，结果上报 r_smbus_suspend_det： 0：中断不使能； 1：中断使能。	0x1
m_slv_rx_pec_nack	[8]	RW	slv_rx_pec_nack 中断屏蔽控制，结果上报 r_slv_rx_pec_nack： 0：中断不使能； 1：中断使能。	0x1
m_arp_assgn_addr_cmd_det	[7]	RW	arp_assgn_addr_cmd_det 中断屏蔽控制，结果上报 r_arp_assgn_addr_cmd_det： 0：中断不使能； 1：中断使能。	0x1

Name	Bits	R/W	Description	Default
m_arp_get_udid_cmd_det	[6]	RW	arp_get_udid_cmd_det 中断屏蔽控制，结果上报 r_arp_get_udid_cmd_det: 0：中断不使能; 1：中断使能。	0x1
m_arp_rst_cmd_det	[5]	RW	arp_rst_cmd_det 中断屏蔽控制，结果上报 r_arp_rst_cmd_det: 0：中断不使能; 1：中断使能。	0x1
m_arp_prepare_cmd_det	[4]	RW	arp_prepare_cmd_det 中断屏蔽控制，结果上报 r_arp_prepare_cmd_det: 0：中断不使能; 1：中断使能。	0x1
m_host_notify_mst_det	[3]	RW	host_notify_mst_det 中断屏蔽控制，结果上报 r_host_notify_mst_det: 0：中断不使能; 1：中断使能。	0x1
m_quick_cmd_det	[2]	RW	quick_cmd_det 中断屏蔽控制，结果上报 r_quick_cmd_det: 0：中断不使能; 1：中断使能。	0x1
m_mst_clock_extnd_timeout	[1]	RW	mst_clock_extnd_timeout 中断屏蔽控制，结果上报 r_mst_clock_extnd_timeout: 0：中断不使能; 1：中断使能。	0x1
m_slv_clock_extnd_timeout	[0]	RW	slv_clock_extnd_timeout 中断屏蔽控制，结果上报 r_slv_clock_extnd_timeout: 0：中断不使能; 1：中断使能。	0x1

23.8.48 SMBUS_RAW_INTR_STAT

◆ **偏移地址：0xD0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					smbus_alert_det	smbus_suspend_det	slv_rx_pec_nack	arp_assgn_addr_cmd_det	arp_get_udid_cmd_det	arp_rst_cmd_det	arp_prepare_cmd_det	host_ntfy_mst_det	quick_cmd_det	mst_clock_extnd_timeout	slv_clock_extnd_timeout
-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
smbus_alert_det	[10]	RO	输入的 SMBALERT 信号被 Slave 拉低中断： 0：中断事件未发生； 1：中断事件发生。	0x0
smbus_suspend_det	[9]	RO	输入的 SMBSUS 信号被 Master 拉低中断： 0：中断事件未发生； 1：中断事件发生。	0x0
slv_rx_pec_nack	[8]	RO	指示作为 ARP Slave 工作时，是否由于 PEC 不匹配而发送了 NACK： 0：中断事件未发生； 1：中断事件发生。	0x0
arp_assgn_addr_cmd_det	[7]	RO	指示是否已接收到分配地址 ARP 命令： 0：中断事件未发生； 1：中断事件发生。	0x0
arp_get_udid_cmd_det	[6]	RO	指示是否已接收到 Get-UDID ARP 命令： 0：中断事件未发生； 1：中断事件发生。	0x0
arp_rst_cmd_det	[5]	RO	指示是否已接收到广播或者定向复位 ARP 命令： 0：中断事件未发生； 1：中断事件发生。	0x0
arp_prepare_cmd_det	[4]	RO	指示是否已接收到准备 ARP 命令： 0：中断事件未发生； 1：中断事件发生。	0x0
host_ntfy_mst_det	[3]	RO	指示是否通知 ARP Master 已接收到 ARP 命令： 0：中断事件未发生； 1：中断事件发生。	0x0
quick_cmd_det	[2]	RO	指示是否在 SMBus 接口上接收到快速命令，无论模块是 Slave 还是 Master： 0：中断事件未发生； 1：中断事件发生。	0x0

Name	Bits	R/W	Description	Default
mst_clock_extnd_timeout	[1]	RO	指示 Master 传输事务的每个字节（从 START 到 STOP）是否超过 SMBUS_CLOCK_LOW_MEXT 时间，仅在 CON.master_mode=1 时可用： 0：中断事件未发生； 1：中断事件发生。	0x0
slv_clock_extnd_timeout	[0]	RO	指示从 Slave 的传输事务（即从 START 到 STOP）是否超过 SMBUS_CLK_LOW_SEXT 时间，仅在 CON.master_mode=1 时可用： 0：中断事件未发生； 1：中断事件发生。	0x0

23.8.49 CLR_SMBUS_INTR

◆ **偏移地址：0xD4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					clr_smbus_alert_det	clr_smbus_suspend_det	clr_slv_rx_pec_nack	clr_arp_assgn_addr_cmd_det	clr_arp_get_udid_cmd_det	clr_arp_rst_cmd_det	clr_arp_prepare_cmd_det	clr_host_notify_mst_det	clr_quick_cmd_det	clr_mst_clock_extnd_timeout	clr_slv_clock_extnd_timeout
-	-	-	-	-	W	W	W	W	W	W	W	W	W	W	W
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
clr_smbus_alert_det	[10]	W	写 1 清除 SMBUS_ALERT_DET 中断。	0x0
clr_smbus_suspend_det	[9]	W	写 1 清除 SMBUS_SUSPEND_DET 中断。	0x0
clr_slv_rx_pec_nack	[8]	W	写 1 清除 SLV_RX_PEC_NACK 中断。	0x0
clr_arp_assgn_addr_cmd_det	[7]	W	写 1 清除 ARP_ASSGN_ADDR_CMD_DET 中断。	0x0
clr_arp_get_udid_cmd_det	[6]	W	写 1 清除 ARP_GET_UDID_CMD_DET 中断。	0x0
clr_arp_rst_cmd_det	[5]	W	写 1 清除 ARP_RST_CMD_DET 中断。	0x0

Name	Bits	R/W	Description	Default
clr_arp_prepare_cmd_det	[4]	W	写 1 清除 ARP_PREPARE_CMD_DET 中断。	0x0
clr_host_notify_mst_det	[3]	W	写 1 清除 HOST_NOTIFY_MST_DET 中断。	0x0
clr_quick_cmd_det	[2]	W	写 1 清除 QUICK_CMD_DET 中断。	0x0
clr_mst_clock_extnd_timeout	[1]	W	写 1 清除 MST_CLOCK_EXTND_TIMEOUT 中断。	0x0
clr_slv_clock_extnd_timeout	[0]	W	写 1 清除 SLV_CLOCK_EXTND_TIMEOUT 中断。	0x0

23.8.50 OPTIONAL_SAR

◆ **偏移地址：0xD8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.									optional_sar						
-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留。	0x0
optional_sar	[6:0]	RW	作为 Slave 时的可选 Slave 地址，与 SAR 寄存器作用相同。	0x0

23.8.51 SMBUS_UDID_LSB

◆ **偏移地址：0xDC　　复位值：0xFFFFFFFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
smbus_udid_lsb															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
smbus_udid_lsb															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
smbus_udid_lsb	[31:0]	RW	用于存储 ARP 中使用的 UHID 的 LSB 32bit。 ENABLE.enable=0 时才可以配置。	0xFFFFFFFF

24 集成电路内置音频总线（I2S）

24.1 I2S 简介

集成电路内置音频总线（Inter-IC sound，I2S）也是同步串行通信接口，是用于两个设备直接进行音频数据传输的总线，它可以在从模式或主模式下进行全双工通信，支持 Philips 的 I2S 串行协议。芯片内部集成两个 I2S 模块。

24.2 I2S 主要特性

I2S 的功能特点有：

- 支持主模式和从模式。
- 接收和发送数据格式基于 Philips 的 I2S 串行协议标准。
- 支持 12、16、20、24、32 位音频数据分辨率。
- WS 信号支持 16、32 位宽度可配置。
- 最高支持 192KHz 音频采样率（WS 信号频率）。
- 支持中断以及中断标志查询，包括发送方向的满溢出、空中断，接收方向的满溢出和数据有效中断。
- 支持 DMA 功能。
- 接收或者发送的各个通道支持独立的 DMA 握手信号。
- 支持全双工和半双工模式，半双工模式发送和接收共用一个管脚。
- TX 或者 RX 的左声道、右声道的 FIFO 独立，FIFO 深度都为 4，宽度为 32bit，最大可缓存 4 帧数据。
- FIFO 阈值可配置。
- 支持输出 mclk 时钟给外部器件。

24.3 I2S 逻辑实现

芯片内部实现两个独立的 I2S 模块，两个 I2S 模块的主要实现如下表 24-1 所示。

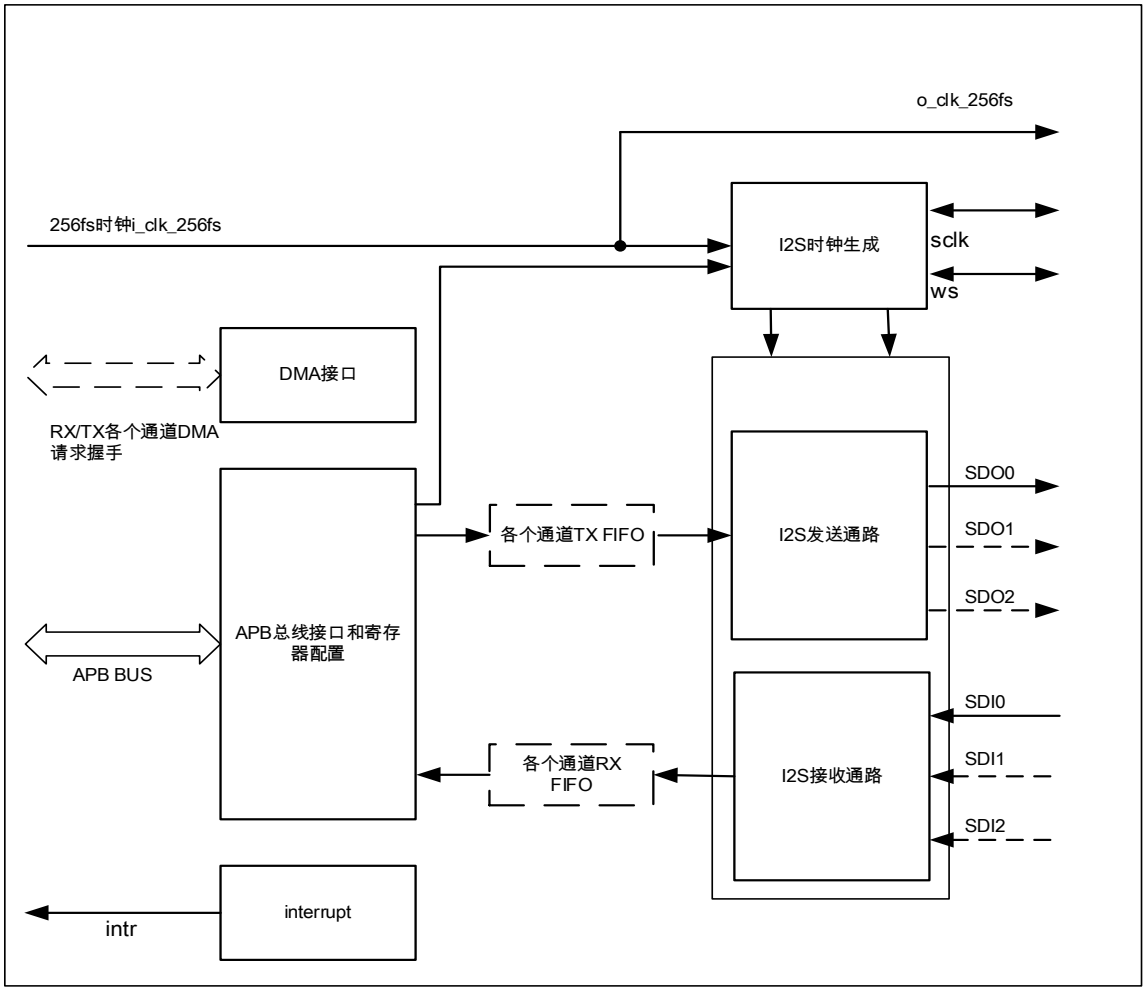
表 24-1 I2S 实现

I2S 特性	I2S0	I2S1
MASTER 模式	支持	支持
SLAVE 模式	支持	支持
接收通道数	3 个	1 个
发送通道数	3 个	1 个
每个通道 FIFO 深度	4	4

后续文档描述中会提到的“x”代表的通道号，其中 I2S0 的 TX 和 RX 通道都是 3 个，I2S1 的 TX 和 RX 通道都是 1 个。

I2S 模块可以使得 MCU 芯片通过 I2S 总线同外部设备进行音频数据传输，内置的 I2S 模块可以根据连接设备的模式配置成 MASTER 或者 SLAVE 模式。I2S 模块的每个 RX 或者 TX 通道都有独立 DMA 接口，可以配置系统中的 DMA 模块和 I2S 配合完成高效音频数据传输。具体实现框图参考图 24-1。

图 24-1 I2S 功能框图



功能框图中虚线部分表示内部实现根据不同的通道数改变。

I2S 总线传输接口

- O_clk_256fs：在主模式或者从模式时可以输出一个 256 倍采样率的时钟，外部器件可以用该时钟作为工作的 MCLK 时钟。
- Sclk：数据采样时钟，主设备输出给从设备。
- WS：左右通道指示， 主设备输出给从设备。
- SDOx：发送数据引脚。
- SDIx：接收数据引脚，在半双工模式下，SDO 和 SDI 的同一组（如 SDO0 和 SDI0）可共用一个引脚。

I2S 与芯片内部的信息传输主要通过以下三个方式：

- DMA 请求握手：与 DMA 模块进行握手。
- APB BUS：用于配置模块内部寄存器和数据收发。
- intr：上报模块中断给 CPU。

I2S 支持 RX 和 TX 各个通道独立的 DMA 握手信号。

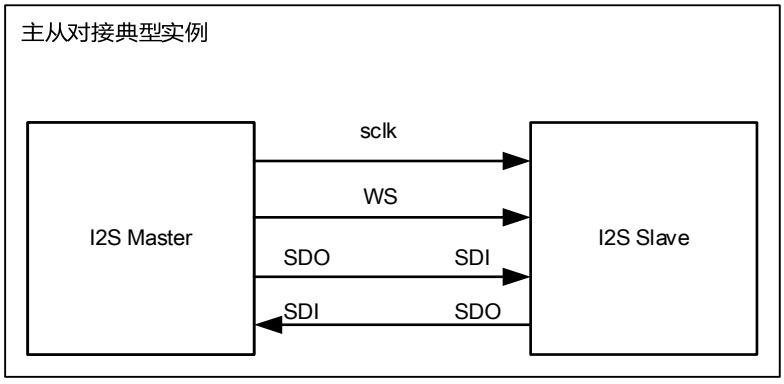
- **I2S0**：内部有 3 个 RX 通道，3 个 TX 通道，对应到 DMA 接口有 6 套（3 套 RX DMA 和 3 套 TX DMA 请求和握手），相互独立。与 DMA 模块进行数据传输，且每个 TX 或者 RX 通道模块都有一个独立使用的 FIFO（宽度 32bit，深度为 4）。
- **I2S1**：内部有 1 个 RX 通道，1 个 TX 通道，对应到 DMA 接口有 2 套（1 套 RX DMA 和 1 套 TX DMA 请求和握手），相互独立。用于同 DMA 模块进行数据传输，且每个 TX 或者 RX 通道模块都有一个独立使用的 FIFO（宽度 32bit，深度为 4）。

24.4 I2S 功能描述

24.4.1 I2S 接口时序

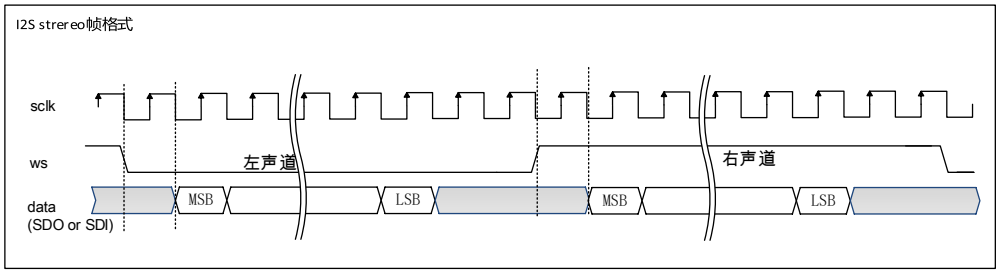
典型的 I2S 应用是主设备和从设备对接，用于数据传输，简要框图如图 24-2 所示，图中为全双工对接方式。当有多个通道并行传输时，数据线个数跟通道数匹配。

图 24-2 主从设计对接



串行数据以二进制补码格式传输，首先传输是最高有效位（MSB）。ws 用于立体声数据区分选择，当 ws 为低时，正在传输的是左立体声数据；当 ws 为高时，正在传输的是右立体声数据。帧格式如图 24-3 所示。对于标准 I2S 格式，数据的 MSB 在 ws 变化后一个 sclk 周期发送。发射端 SDO 发送的串行数据可以与 sclk 信号的负边缘或正边缘同步。然而接收端必须在 sclk 的上升沿锁存串行数据。

图 24-3 帧格式



24.4.2 主模式和从模式选择

I2S 模块支持主模式和从模式，主从模式的选择通过配置扩展配置寄存器 EXT_CTRL. ms_cmd 进行选择，配置 0 为从模式，配置 1 为主模式。初始化配置 I2S 模块时，首先需要配置 EXT_CTRL. ms_cmd，这样才能保证后续配置能正确映射到主或者从的寄存器组中。

24.4.3 I2S 模块全局使能

I2S 模块有全局使能开关信号，必须开启全局使能后才能将数据接收或传输到 FIFO。要启用 I2S 时，请将 I2S 的全局使能寄存器 IER.ien 设置为 1；要禁用 I2S 时，请将 IER.ien 设置为 0。当全局使能关闭后会有以下影响：

- TX FIFO 和 RX FIFO 被清除；
- 发送或接收过程中的任何数据丢失；
- 模块中的所有其他可编程使能（如 TX/RX 使能开关和单个 TX/RX 通道开关）被覆盖为关闭状态；
- 主模式时钟信号 sclk、ws 输出被关闭。

24.4.4 I2S 发送功能

发送块使能

I2S 的发送块全局使能寄存器 ITER，可用于开关所有的 TX 通道。当关闭发送全局使能时，会产生以下影响：

- 输出数据丢失，通道输出保持低电平；
- TX FIFO 中的数据被保留，FIFO 可以被写入；
- 保留 TX 通道的现有配置（如字大小、阈值电平等的变化）；
- 任何单独的 TX 通道启用都被覆盖为禁用。

禁用发送块时，可以执行以下步骤：

步骤 1 配置 TX 通道寄存器；

步骤 2 通过配置 TX FIFO 复位寄存器 TXFFR（TXFFR.txffr=1）清空 TX FIFO；

步骤 3 通过配置通道 x 发送 FIFO 清空寄存器 TFFx（其中 x 是通道号），清空单个通道的 TX FIFO（例如清空通道 0 发送 FIFO，则配置 TFF0.xchfr=1）。

发送通道使能

每个发送通道都有自己独立的使能控制位，通过通道 x 发送使能寄存器 TERx 控制。当某个通道使能禁用后会有如下影响。

- 发送的立体声数据丢失；
- 通道输出保持低电平；
- TX FIFO 中的数据被保留，FIFO 可以被写入；
- TX 通道寄存器的配置都将被保留，寄存器可以进一步重新配置。

发送通道音频数据分辨率

TX 通道的每个 channel 的数据分辨率都是可以独立配置的，可支持的数据分辨率范围为 16、24、32 位。通过通道 x 发送配置寄存器 TCRx 配置。

发送通道 FIFO

TX 通道的每个 channel 有两个 FIFO 用于左右声道立体数据存放，FIFO 的深度为 4，宽度为 32bit。

如果要复位清空各自通道的 FIFO，需关闭发送块，或者关闭该通道的使能控制位。TX FIFO 的空阈值也是可以配置的，匹配时空阈值也是需要关闭通道使能。

通过 TX 通道发送数据

当使用 TX 通道发送立体声数据时，要把待发送的数据写入左声道发送数据缓存寄存器 LTHRx 和右声道发送数据缓存寄存器 RTHRx。所有立体声数据对必须使用以下两阶段过程进行写入：

1. 将左声道立体声数据写入 LTHRx；
2. 将右声道立体声数据写入 RTHRx。

当启用 I2S 的 TX 通道时，如果 TX FIFO 为空，并且在下一个左周期之前未将数据写入 FIFO，则通道在整个帧（左声道和右声道）都输出零。只有在转换到左声道之前 TX FIFO 中有数据时，传输才开始。换句话说，如果错过了帧的开头，则通道输出将一直空闲到下

一个可用帧。

24.4.5 I2S 接收功能

接收块使能

I2S 的接收块全局使能寄存器 IRER，可用于开关所有的 RX 通道。当关闭接收全局使能时，会产生以下影响。

- 输入数据丢失；
- RX FIFO 中的数据被保留，FIFO 可以被读取；
- 保留 RX 通道的现有配置（如字大小、阈值电平等的变化）；
- 任何单独的 RX 通道启用都被覆盖为禁用。

禁用 RX 块时，可以执行以下步骤：

步骤 1 配置 RX 通道寄存器；

步骤 2 通过配置 RX FIFO 复位寄存器 RXFFR（RXFFR.rxffr=1）清空 RX FIFO；

步骤 3 通过配置通道 x 接收 FIFO 清空寄存器 RFFx，清空单个通道的 RX FIFO（例如清空通道 0 接收 FIFO，则配置 RFF0.rxchfr =1）。

接收通道使能

每个接收通道都有自己独立的使能控制位，通过通道 x 接收使能寄存器 RERx 控制。当某个通道的通道使能禁用后会有如下影响。

- 接收的立体声数据丢失；
- RX FIFO 中的数据被保留，FIFO 仍可以被读取；
- RX 通道寄存器的配置都将被保留，寄存器可以进一步重新配置。

传输通道音频数据分辨率

RX 通道的每个 channel 的数据分辨率都是可以独立配置的，可支持的数据分辨率范围为 16、24、32 位。通过通道 x 接收配置寄存器 RCRx 控制。

传输通道 FIFO

RX 通道的每个 channel 有两个 FIFO 用于左右声道立体数据存放，FIFO 的深度为 4，宽度为 32bit。

如果要复位清空各自通道的 FIFO，需关闭接收块，或者关闭该通道的使能控制位。RX FIFO 的空阈值也是可以配置的，匹配时空阈值也是需要关闭通道使能。

通过 RX 通道读取数据

通过 RX 通道接收到的立体声数据会写入到 RX FIFO，可以通过左声道接收数据缓存寄存器 LRBRx 和右声道接收数据缓存寄存器 RRBRx 读取。所有立体声数据对必须使用以下两阶段过程进行读取：

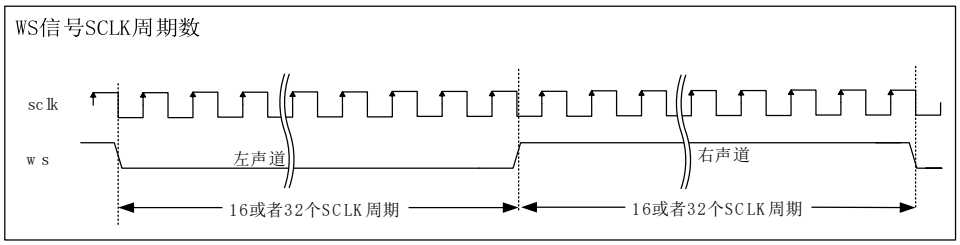
1. 左声道从 LRHRx 读取；
2. 右声道从 RRHRx 读取。

24.4.6 时钟产生（仅主模式）

当 I2S 配置成 master 模式时，需要输出 sclk 和 ws 信号给对接的从设备，时钟产生模块用于产生输出 sclk 和 ws 信号。o_clk_256fs 是从系统配置产生的 256 倍采样率时钟，o_clk_256fs 的分频产生配置需要参考 4 时钟和复位控制 (CRG)。时钟产生器会根据时钟配置寄存器 CCR.wss 的值分频和输出匹配的 sclk 和 ws。

在主模式是输出的 ws 信号指示左/右声道的宽度可以配置为 16 或者 32 位宽，如下图 24-4 所示：

图 24-4 ws 信号宽度



时钟输出使能寄存器 CER，用于开启或者关闭 ws 和 sclk 的输出，当时钟输出使能关闭时，ws 输出保持为 0。

当打开时钟输出使能时，默认 sclk 在 ws 为低时开始输出。可以通过配置输出门控时钟相位控制寄存器 GATECLKDLY 对输出的 sclk 进行延迟，调节 sclk 在 ws 为高电平的时候才开始输出，可匹配对 ws 和 sclk 启动时间关系有不同要求的从设备。

24.4.7 DMA 功能

每个发送和接收通道的都有专用 DMA 握手接口，I2S0 总共有 6 组 DMA 握手信号，I2S1 有 2 组 DMA 握手信号。每个通道的 TX FIFO 和 RX FIFO 阈值可由用户配置，当满足阈值触发条件时向 DMA 发送数据传输请求。

配置 DMACR 寄存器用于开启 RX 和 TX 通道的 DMA 功能，在 DMA 模式下，DMA 模式通道 x 接收数据寄存器 RXDMA_CHx 和 DMA 模式通道 x 发送数据寄存器 TXDMA_CHx 用于读取和写入立体声数据。

由于以上因素叠加限制，需要配置一次 DMA 握手传输的数据个数为 4 个，以及设定好 FIFO 正确阈值。

建议配置 DMA 和 I2S 一次 DMA 握手传输数据个数为 4，建议把相匹配的通道 x 接收 FIFO 配置寄存器 RFCRx 配置为 2，通道 x 发送 FIFO 配置寄存器 TFCRx 配置为 2。

24.4.8 全双工和半双工控制

I2S 模块内部有 RX 和 TX 通道，可配置同时工作，也可配置一个时刻只有 RX 或者 TX 在工作。

半双工模式

配置通道 x 扩展配置寄存器 EXT_CTRL.ds_cmd=0，对应通道 x 工作在半双工模式。数据输入管脚 SDIx 和输出管脚 SDOx 公共使用一个引脚，此时通道 x 只能使用发送数据或者接收数据。

通道 x 半双工模式开启时，通过 EXT_CTR.sd_oen_cmd 控制数据的发送和接收。EXT_CTR.sd_oen_cmd=0,引脚配置为输出模式，通道 x 只可以发送数据，不能接收数据；EXT_CTR.sd_oen_cmd=1,引脚配置为输入模式，通道 x 不能发送数据，只能接收数据。

全双工模式

配置通道 x 扩展配置寄存器 EXT_CTRL.ds_cmd=1，对应通道 x 工作在全双工模式。数据输入引脚 SDIx 和输出引脚 SDOx 分别使用独立的引脚。全双工时 EXT_CTR.sd_oen_cmd 的配置不会起作用。

24.4.9 门控延迟功能

I2S 模块在主模式时，可以通过配置输出门控时钟相位控制寄存器 GATECLKDLY 控制主模式发送的第一个 sclk 和 ws 信号的关系，可实现 ws 信号为低时输出 sclk 或者 ws 信号为高时输出 sclk，用于适配不同需求的外部 I2S 从设备。

24.5 I2S 配置流程

24.5.1 I2S 普通发送

具体步骤如下：

步骤 1 向扩展配置寄存器 EXT_CTRL.ms_cmd 写 1 或者 0，配置为主或者从模式。

步骤 2 向扩展配置寄存器 EXT_CTRL. ds_cmd 写 1 或者 0，配置为全双工模式或者半双工模式。如果配置为半双工模式，向扩展配置寄存器 EXT_CTRL. sd_oen_cmd 写 0，配置数据引脚为输出方向。

步骤 3 向全局使能寄存器 IER.ien 写入 1，打开全局使能。

步骤 4 配置数据位宽、ws 宽度、FIFO 阈值。

步骤 5 如果配置为主模式：配置输出门控时钟相位控制寄存器 GATECLKDLY 的 sclk 相位延迟值，配置时钟输出使能寄存器（仅主模式）CER. clken 为 1，允许 sclk 和 ws 输出。

步骤 6 配置通道 x 发送使能寄存器 TERx.txchenx 为 0 或者为 1，开启、关闭需要使用的发送通道。

如果需要开启发送数据，将数据写入左声道发送数据缓存寄存器 LTHRx 和右声道发送数据缓存寄存器 RTHRx 来填充 TX-FIFO。保持先左后右的顺序写入，直到 FIFO 充满数据。I2S 模块会用 sclk 检测到 ws 从 1 变为 0 后开始第一个左声道立体声数据发送。

1. 向发送块使能寄存器 ITER.txen 中写入 1 来启用 I2S 发射器块。
2. 通过中断响应或者查询方式等待通道 x 中断状态寄存 ISRx.txfe 为 1，继续写入需要发送的数。
3. 当需要发送的数传输完成，可配置关闭对应通道使能。

24.5.2 I2S 普通接收

具体步骤如下：

步骤 1 向扩展配置寄存器 EXT_CTRL.ms_cmd 写 1 或者 0，配置为主或者从模式。

步骤 2 向扩展配置寄存器 EXT_CTRL. ds_cmd 写 1 或者 0，配置为全双工模式或者半双工模式。如果配置为半双工模式，向扩展配置寄存器 EXT_CTRL. sd_oen_cmd 写 1，配置数据引脚为输入方向。

步骤 3 向全局使能寄存器 IER.ien 写入 1，打开全局使能。

步骤 4 配置数据位宽、ws 宽度、FIFO 阈值。

步骤 5 如果配置为主模式：配置输出门控时钟相位控制寄存器 GATECLKDLY 的 sclk 相位延迟值，配置时钟输出使能寄存器（仅主模式）CER. clken 为 1，允许 sclk 和 ws 输出。

步骤 6 配置通道 x 接收使能寄存器 RERx. rxchenx 为 0 或者为 1，开启、关闭需要使用的接收通道。

步骤 7 向接收块使能寄存器 IRER .rxen 中写入 1 来启用 I2S 接收数据。

步骤 8 通过中断响应或者查询方式等待通道 x 中断状态寄存器 ISRx.rxda 为 1，读取 RX FIFO 中接收到的立体声数据。

步骤 9 当需要接收的数传输完成，可配置关闭对应通道使能。

24.5.3 I2S DMA 模式发送

流程以 I2S 通道 0 使用 DMA 方式进行传输数据来体现配置流程。具体步骤如下：

步骤 1 配置 DMA 模块。DMA 模块发送数据的目的地址是 I2S 模块的 TXDMA_CH0 寄存器。

步骤 2 向扩展配置寄存器 EXT_CTRL.ms_cmd 写 1 或者 0，配置为主或者从模式。

步骤 3 向扩展配置寄存器 EXT_CTRL. ds_cmd 写 1 或者 0，配置为全双工模式或者半双工模式。如果配置为半双工模式，向扩展配置寄存器 EXT_CTRL. sd_oen_cmd0 写 0，配置数据引脚为输出方向。

步骤 4 向全局使能寄存器 IER.ien 写入 1，打开全局使能。

步骤 5 配置数据位宽、ws 宽度、FIFO 阈值等。

步骤 6 如果配置为主模式：配置输出门控时钟相位控制寄存器 GATECLKDLY 的 sclk 相位延迟值，配置时钟输出使能寄存器（仅主模式）CER. clken 为 1，允许 sclk 和 ws 输出。

步骤 7 配置通道 x 发送使能寄存器 TERx.txchenx 为 0 或者为 1，开启、关闭需要使用的发送通道。

步骤 8 向发送块使能寄存器 ITER.txen 中写入 1 来启用 I2S 发射器块。

步骤 9 向 DMACR.dmaen_txch_0 写入 1,打开通道 0 的 DMA 发送功能。

步骤 10 DMA 模块自动和 I2S 进行 DMA 握手并往 TXDMA_CH0 寄存器中填充数据。

步骤 11 DMA 传输完所有数据后关闭 I2S 通道 0。

24.5.4 I2S DMA 模式接收

流程以 I2S 通道 0 使用 DMA 方式进行传输数据来体现配置流程。具体步骤如下：

步骤 1 配置 DMA 模块。DMA 模块读取数据的源地址是 I2S 模块的 RXDMA_CH0 寄存器。

步骤 2 向扩展配置寄存器 EXT_CTRL.ms_cmd 写 1 或者 0，配置为主或者从模式。

步骤 3 向扩展配置寄存器 EXT_CTRL. ds_cmd 写 1 或者 0，配置为全双工模式或者半双工

模式。如果配置为半双工模式，向扩展配置寄存器 EXT_CTRL. sd_oen_cmd 写 1，配置数据引脚为输入方向。

步骤 4 向全局使能寄存器 IER.ien 写入 1，打开全局使能。

步骤 5 配置数据位宽、ws 宽度、FIFO 阈值等。

步骤 6 如果配置为主模式：配置输出门控时钟相位控制寄存器 GATECLKDLY 的 sclk 相位延迟值，配置时钟输出使能寄存器（仅主模式）CER. clken 为 1，允许 sclk 和 ws 输出。

步骤 7 配置通道 x 接收使能寄存器 RERx.rxchenx 为 0 或者为 1，开启、关闭需要使用的接收通道。

步骤 8 向接收块使能寄存器 IRER .rxen 中写入 1 来启用 I2S 接收数据。

步骤 9 向 DMACR.DMAEN_RXCH_0 写入 1,打开通道 0 的 DMA 接收功能。

步骤 10 DMA 模块自动和 I2S 进行 DMA 握手，并从 RXDMA_CH0 寄存器中读取数据。

步骤 11 DMA 传输完所有数据后关闭 I2S 通道 0。

24.6 I2S 寄存器概览

I2S 实例 2 个 ，各自基地址如下：

- I2S0 寄存器基地址为：0x61007000
- I2S1 寄存器基地址为：0x60060000

说明

寄存描述中，寄存器后缀“x”取值：I2S0 模块包含 3 个通道，因此“x”可以为 0，1，2。I2S1 模块只有一个通道“x”只能是 0。

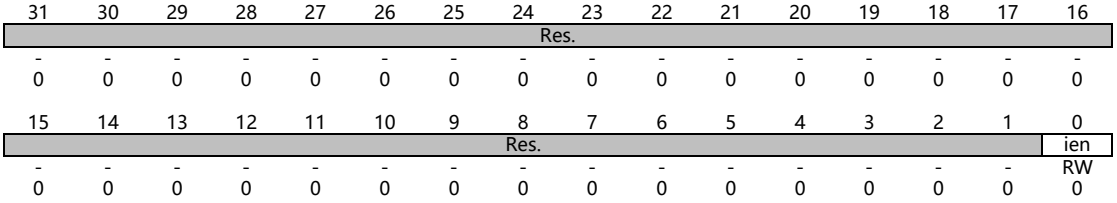
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	IER	0x0	32	RW	0x00000000	全局使能
2	IRER	0x4	32	RW	0x00000000	接收块使能
3	ITER	0x8	32	RW	0x00000000	发送块使能
4	CER	0xc	32	RW	0x00000000	时钟输出使能（仅主模式）
5	CCR	0x10	32	RW	0x00000001	时钟配置（仅主模式）

No.	Names	Offset Address	SIZE	R/W	Default	Description
6	RXFFR	0x14	32	RO	0x00000000	接收 FIFO 复位
7	TXFFR	0x18	32	RO	0x00000000	发送 FIFO 复位
8	LRBRx	0x20+0x40*x	32	RO	0x00000000	左声道接收数据缓存。
9	LTHRx	0x20+0x40*x	32	RO	0x00000000	左声道发送数据缓存
10	RRBRx	0x24+0x40*x	32	RO	0x00000000	右声道接收数据缓存
11	RTHRx	0x24+0x40*x	32	RO	0x00000000	右声道发送数据缓存
12	RERx	0x28+0x40*x	32	RW	0x00000001	通道 x 接收使能
13	TERx	0x2c+0x40*x	32	RW	0x00000001	通道 x 发送使能
14	RCRx	0x30+0x40*x	32	RW	0x00000005	通道 x 接收配置
15	TCRx	0x34+0x40*x	32	RW	0x00000005	通道 x 发送配置
16	ISRx	0x38+0x40*x	32	RO	0x00000010	通道 x 中断状态
17	IMRx	0x3c+0x40*x	32	RW	0x00000033	通道 x 中断屏蔽
18	RORx	0x40+0x40*x	32	RO	0x00000000	通道 x 接收 FIFO 溢出状态
19	TORx	0x44+0x40*x	32	RO	0x00000000	通道 x 发送 FIFO 溢出状态
20	RFCRx	0x48+0x40*x	32	RW	0x00000000	通道 x 接收 FIFO 配置
21	TFCRx	0x4c+0x40*x	32	RW	0x00000000	通道 x 发送 FIFO 配置
22	RFFx	0x50+0x40*x	32	RO	0x00000000	通道 x 接收 FIFO 清空
23	TFFx	0x54+0x40*x	32	RO	0x00000000	通道 x 发送 FIFO 清空
24	I2S_COMP_PARAM_2	0x1f0	32	RO	0x00000624	参数信息寄存器 2
25	I2S_COMP_PARAM_1	0x1f4	32	RO	0x03240566	参数信息寄存器 1
26	I2S_COMP_VERSION	0x1f8	32	RO	0x3131312A	版本信息寄存器
27	I2S_COMP_TYPE	0x1fc	32	RO	0x445701A0	类型信息寄存器
28	DMACR	0x200	32	RW	0x00000000	DMA 控制
29	RXDMA_CHx	0x204	32	RO	0x00000000	DMA 模式通道 x 接收数据
30	TXDMA_CHx	0x214	32	RO	0x00000000	DMA 模式通道 x 发送数据
31	EXT_CTRL	0x334	32	RW	0x00000000	扩展配置寄存器
32	GATECLKDLY	0x338	32	RW	0x00000002	输出门控时钟相位控制

24.7 I2S 寄存器描述

24.7.1 IER

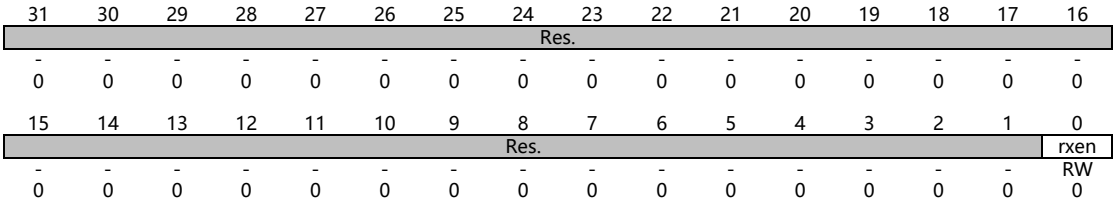
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
ien	[0]	RW	I2S 模块全局开关控制。 0：I2S 模块全局禁用 1：I2S 模块全局启用	0x0

24.7.2 IRER

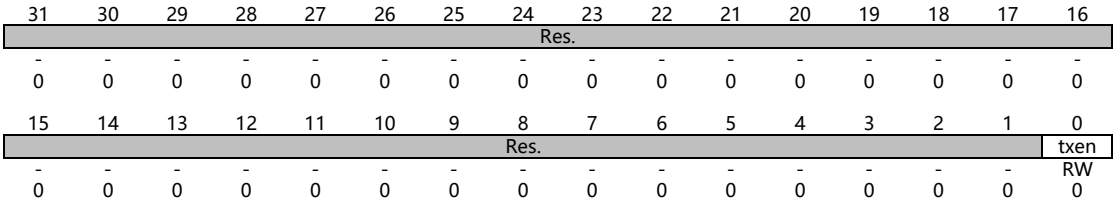
◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxen	[0]	RW	接收块全局开关控制。 0：禁用 1：启用	0x0

24.7.3 ITER

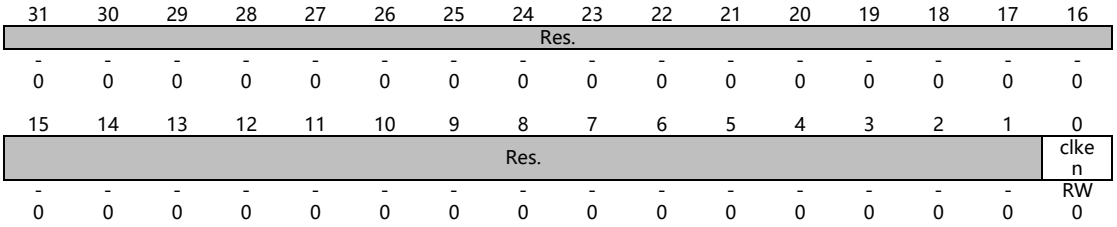
◆ **偏移地址：0x8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txen	[0]	RW	发送块全局开关控制。 0：禁用 1：启用	0x0

24.7.4 CER

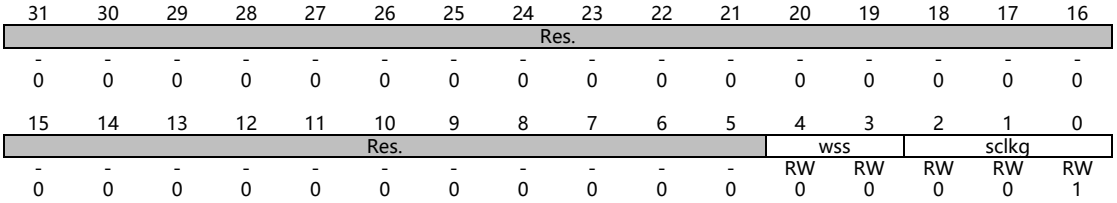
◆ **偏移地址：0xc　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
clken	[0]	RW	时钟产生开关，用于控制是否对外输出 sclk 和 ws 信号。仅在主模式该配置域才有作用。 0：关闭时钟输出 1：打开时钟输出	0x0

24.7.5 CCR

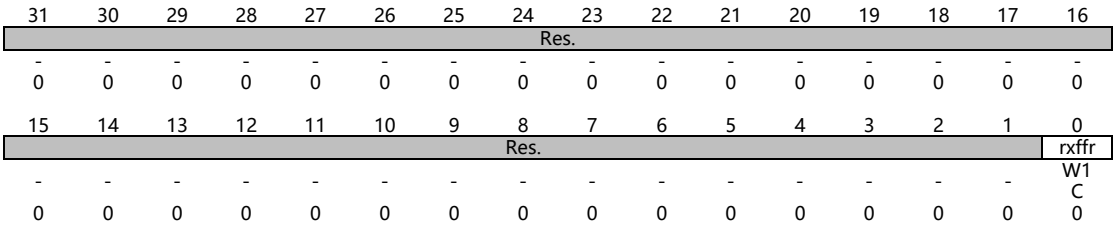
◆ **偏移地址：0x10　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
wss	[4:3]	RW	输出 ws 左声道、右声道信号指示宽度。仅在主模式该配置域才有作用。 0：16 sclk cycles 2：32 sclk cycles 其它值：无效配置	0x0
sclkg	[2:0]	RW	主模式的 sclk 自动时钟 gating 控制，初始时配置为 0	0x1

24.7.6 RXFFR

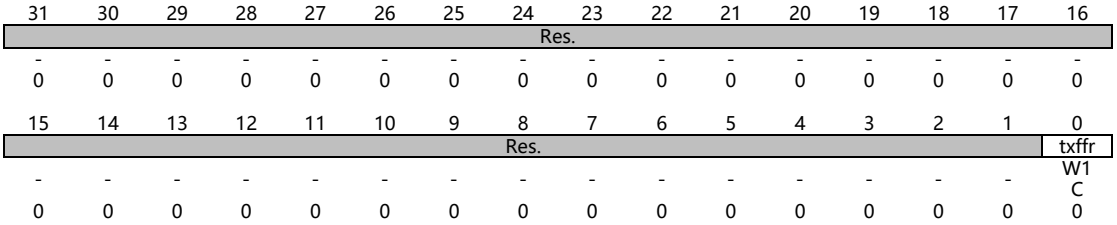
◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxffr	[0]	W1C	写 1 清空所有的 RX FIFO，硬件自动清零。写这个寄存器之前必须先禁用 IRER.rxen。 0：不清空复位 RX FIFO 1：清空复位 RX FIFO，硬件会自动清 0	0x0

24.7.7 TXFFR

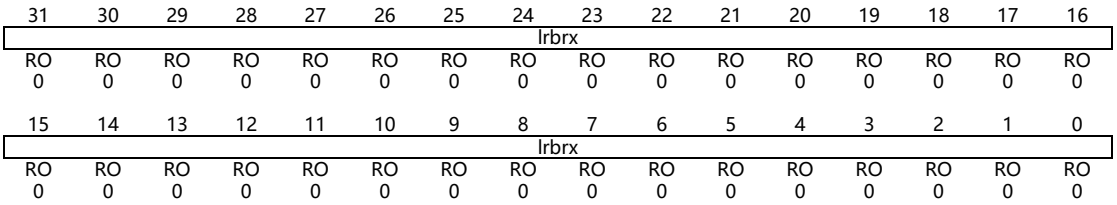
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txffr	[0]	W1C	写 1 清空所有的 TX FIFO，硬件自动清零。写这个寄存器之前必须先禁用 ITER.txen。 0：不清空复位 TX FIFO 1：清空复位 TX FIFO，硬件会自动清 0.	0x0

24.7.8 LRBRx

◆ **偏移地址：0x20+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
lrbrx	[31:0]	RO	存放接收的左声道数据。如果 RX FIFO 已满，还有数据传入则新数据丢失并产生溢出。（已在 RX FIFO 中的数据被保留。） 注意：再次读取此寄存器之前，必须从 RRBRx 读取右声道数据，否则状态/中断将无效。 该寄存器和后续其它寄存器后缀“x”取值：I2S0 模块“x”可以为 0，1，2。I2S1 模块“x”只能是 0。	0x0

24.7.9 LTHRx

◆ **偏移地址：0x20+0x40*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
lthrx															
WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
lthrx															
WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
lthrx	[31:0]	WC	通过该寄存器写入需要发送的左立体声数据。 写分为两个阶段： 1.对该寄存器的写入将左立体声数据传递给发射机。 2.之后必须将右立体声数据写入 RTHRx 寄存器。 数据只有在未满时才能写入 FIFO。任何写入满状态 FIFO 的尝试都会导致数据丢失并产生溢出中断。	0x0

24.7.10 RRBRx

◆ **偏移地址：0x24+0x40*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
rrbrx															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rrbrx															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
rrbrx	[31:0]	RO	存放接收的右立体声数据。如果 RX FIFO 已满，还有数据输入则新数据丢失，并发生溢出中断。（已在 RX FIFO 中的数据被保留。） 注意：在读取此寄存器之前，必须先从 LRBRx 读取左立体声数据，否则状态/中断将无效。	0x0

24.7.11 RTHRx

◆ **偏移地址：0x24+0x40*x　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
rthrx															
WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rthrx															
WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC	WC
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
rthrx	[31:0]	WC	通过该寄存器写入需要发送的右立体声数据。 写分为两个阶段： 1.左立体声数据必须写入 LTHRx 寄存器。 2.对该寄存器的写入将正确的立体声数据传递给发射机。 数据只应在未满时写入 FIFO。任何写入满状态 FIFO 的尝试都会导致数据丢失并产生溢出中断。	0x0

24.7.12 RERx

◆ **偏移地址：0x28+0x40*x　　复位值：0x00000001**

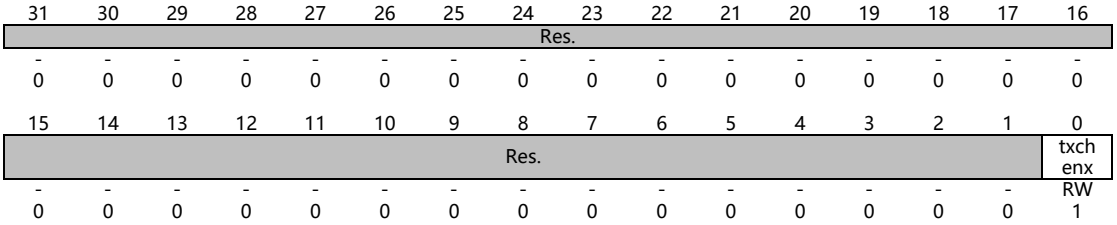
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															rxchenx
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxchenx	[0]	RW	接收通道启用。该位启用/禁用接收通道。 启用时，频道在下一个左立体声周期开始接收。 全局开关控制 IER.ien=0 或接收器块 IRER.rxen=0 的禁用将高于此位的优先级。0：接收通道禁用 1：接收通道启用 复位值为 0x1，默认独立通道接收控制为开启状态。	0x1

24.7.13 TERx

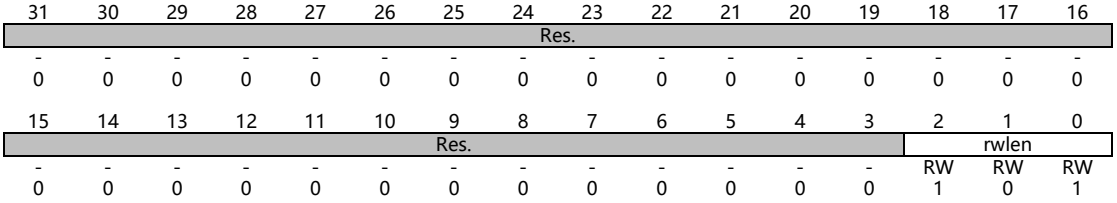
◆ **偏移地址：0x2c+0x40*x　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txchenx	[0]	RW	发送通道启用。该位启用/禁用发送通道。 启用时，频道在下一个左立体声周期开始发送。 全局开关控制 IER.ien=0 或发送块 ITER.txen= 0 的禁用将高于此位的优先级。 0：发送通道禁用 1：发送通道启用 复位值为 0x1，默认独立通道接收控制为开启状态。	0x1

24.7.14 RCRx

◆ **偏移地址：0x30+0x40*x　　复位值：0x00000005**

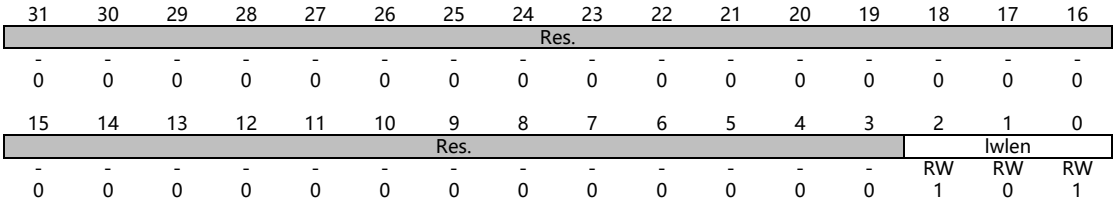


Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
rwlen	[2:0]	RW	接收数据分辨率位宽配置。从 MSB 开始后达到配置位宽后数据不会被接收。 修改该配置的值时需要先关闭通道接收使能 (RERx.rxchenx = 0)。 0x0：忽略位宽位宽按 ws 位宽接收所有 bit。 1：12-bit 数据分辨率 2：16-bit 数据分辨率 3：20-bit 数据分辨率	0x5

Name	Bits	R/W	Description	Default
			4：24-bit 数据分辨率 5：32-bit 数据分辨率 其它值：无效配置	

24.7.15 TCRx

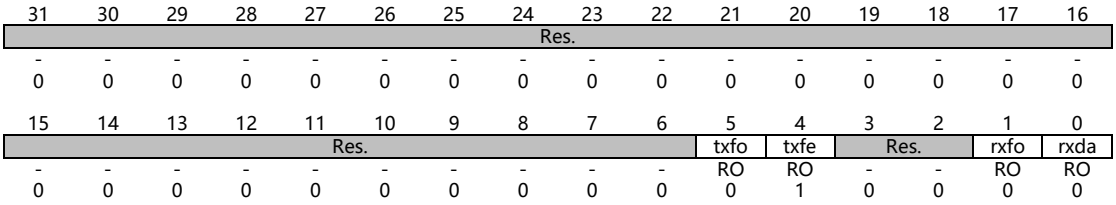
◆ **偏移地址：0x34+0x40*x　　复位值：0x00000005**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
lwlen	[2:0]	RW	发送数据分辨率位宽配置。从 MSB 开始后达到配置位宽后数据不会被发送。 修改该配置的值时需要先关闭通道发送使能(TERx.txchenx = 0)。 0：忽略位宽位宽按 ws 位宽接收所有 bit。 1：12-bit 数据分辨率 2：16-bit 数据分辨率 3：20-bit 数据分辨率 4：24-bit 数据分辨率 5：32-bit 数据分辨率 其它值：无效配置 注意：配置为 0x0 时发送数据 bit 顺序忽略位宽配置，按写入 fifo 数据的 bit31 到 bit0 顺序发送。如果 ws 位宽为 16，该域配置为 0，只会发送写入 FIFO 数据的高 16bits。	0x5

24.7.16 ISRx

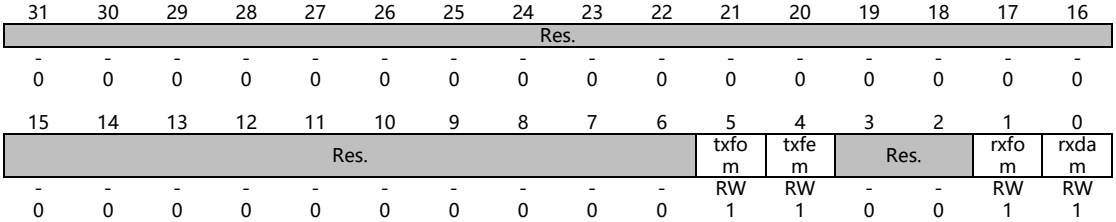
◆ **偏移地址：0x38+0x40*x　　复位值：0x00000010**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0
txfo	[5]	RO	TX FIFO 满溢出状态，可以用于支持当前写入 TX FIFO 操作是否有效，是否是 FIFO 满状态下进行的写入。 0：TX FIFO 无溢出 1：TX FIFO 溢出	0x0
txfe	[4]	RO	TX FIFO 空状态，指示是否达到 TX FIFO 设置阈值。 0：TX FIFO 非空，剩余数据个数小于等于阈值。 1：TX FIFO 空，剩余数据个数大于阈值	0x1
reserved	[3:2]	-	保留	0x0
rxfo	[1]	RO	RX FIFO 满溢出状态，指示是否接收数据出现溢出丢失。 0：RX FIFO 无溢出 1：RX FIFO 溢出	0x0
rxda	[0]	RO	接收数据有效，指示该通道的接收的立体声左右通道是否都已经达到了设定的 FIFO 阈值。 0：没有达到设定的 RX FIFO 阈值 1：达到设定的 RX FIFO 阈值	0x0

24.7.17 IMRx

◆ **偏移地址：0x3c+0x40*x　　复位值：0x00000033**

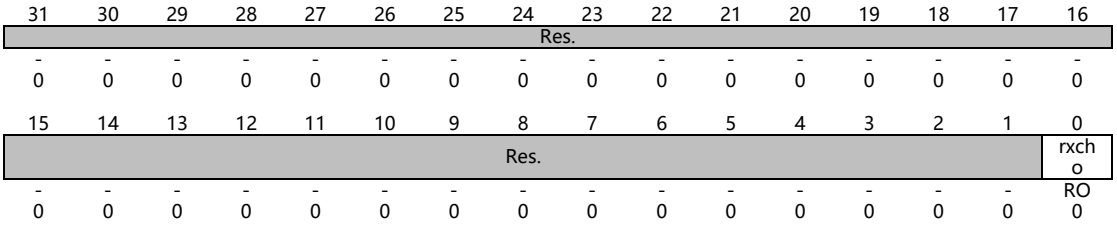


Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留	0x0

Name	Bits	R/W	Description	Default
txfom	[5]	RW	是否屏蔽 TX FIFO 溢出中断，屏蔽后不会因为此中断产生 intr 中断上报。 0：不屏蔽 1：屏蔽	0x1
txfem	[4]	RW	是否屏蔽 TX FIFO 空中断，屏蔽后不会因为此中断产生 intr 中断上报。 0：不屏蔽 1：屏蔽	0x1
reserved	[3:2]	-	保留	0x0
rxfom	[1]	RW	是否屏蔽 RX FIFO 溢出中断，屏蔽后不会因为此中断产生 intr 中断上报。 0：不屏蔽 1：屏蔽	0x1
rxdam	[0]	RW	是否屏蔽 RX FIFO 数据有效中断，屏蔽后不会因为此中断产生 intr 中断上报。 0：不屏蔽 1：屏蔽	0x1

24.7.18 RORx

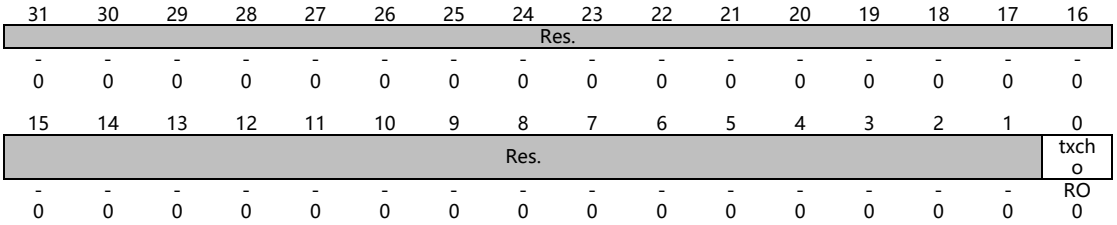
◆ **偏移地址：0x40+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxcho	[0]	RO	RX FIFO 溢出状态指示。读取该位会清除 RX FIFO 溢出中断。 0：RX FIFO 无溢出 1：RX FIFO 溢出	0x0

24.7.19 TORx

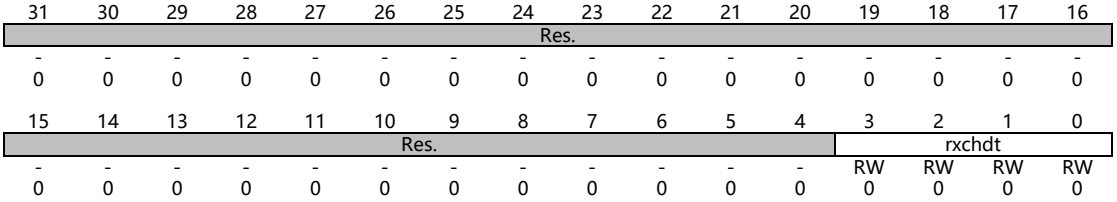
◆ **偏移地址：0x44+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txcho	[0]	RO	TX FIFO 溢出状态指示。读取该位会清除 TX FIFO 溢出中断。 0：TX FIFO 无溢出 1：TX FIFO 溢出	0x0

24.7.20 RFCRx

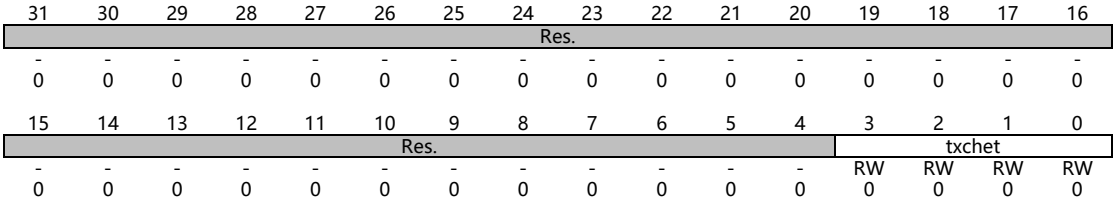
◆ **偏移地址：0x48+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
rxchdt	[3:0]	RW	RX FIFO 中的触发阈值配置，在 RX FIFO 数据大于等于该阈值时生成接收数据有效中断和 DMA 请求。 在该值发生任何变化之前，必须禁用接收通道（RERx.rxchenx=0）。 阈值和配置值关系：触发阈值=配置值+1。 0：左、右声道 RX FIFO 有 1 个数据后产生中断和 DMA 请求。 1：左、右声道 RX FIFO 有 2 个数据后产生中断和 DMA 请求。 2：左、右声道 RX FIFO 有 3 个数据后产生中断和 DMA 请求。 3：左、右声道 RX FIFO 有 4 个数据后产生中断和 DMA 请求。 其他值：左、右声道 RX FIFO 有 4 个数据后产生中断和 DMA 请求	0x0

24.7.21 TFCRx

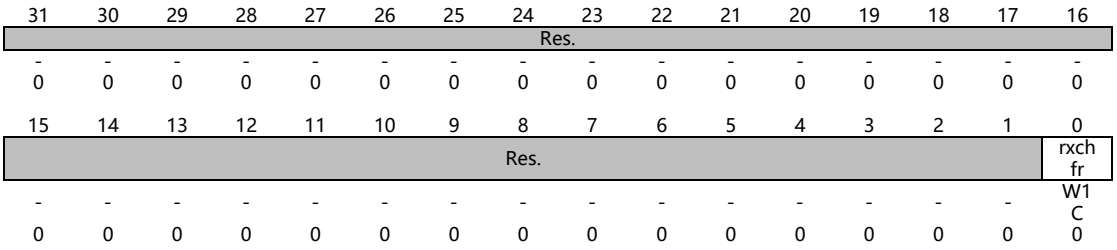
◆ **偏移地址：0x4c+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
txchet	[3:0]	RW	TX FIFO 中的触发阈值配置，在该触发阈值生成发送 FIFO 空中断和 DMA 请求。 在该值发生任何变化之前，必须禁用接收通道（TERx.txchenx=0）。 阈值和配置值关系：触发阈值=配置值+1。 0：左、右声道 TX FIFO 有 0 个数据产生中断和 DMA 请求。 1：左、右声道 TX FIFO 中剩余不超过 1 个数据产生中断和 DMA 请求。 2：左、右声道 TX FIFO 中剩余不超过 2 个数据后产生中断和 DMA 请求。 3：左、右声道 TX FIFO 中剩余不超过 3 个数据后产生中断和 DMA 请求。 其他值：左、右声道 TX FIFO 中剩余不超过 3 个数据后产生中断和 DMA 请求	0x0

24.7.22 RFFx

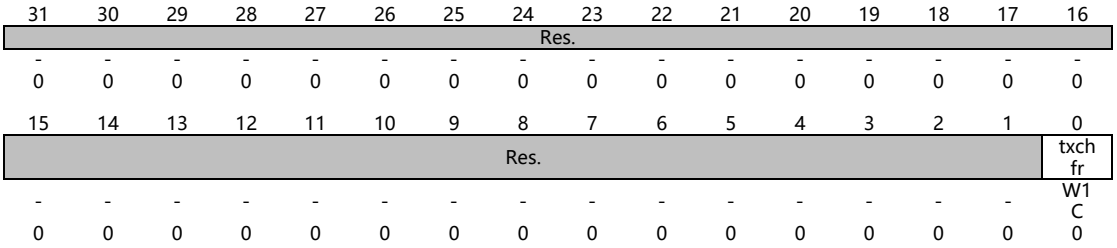
◆ **偏移地址：0x50+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
rxchfr	[0]	W1C	通道接收 FIFO 复位。写 1 用于清空该通道的 RX FIFO。 写该域前需要先关闭该接收通道或者关闭整个接收块。 该位写 1 后硬件会自动清 0. 0：不清空该通道的 RX FIFO 1：清空该通道的 RX FIFO，硬件会自动清 0。	0x0

24.7.23 TFFx

◆ **偏移地址：0x54+0x40*x　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
txchfr	[0]	W1C	通道 TX FIFO 复位。写 1 用于清空该通道的 TX FIFO。 写该域前需要先关闭该发送通道或者关闭整个发送块。 该位写 1 后硬件会自动清 0. 0：不清空该通道的 TX FIFO 1：清空该通道的 TX FIFO，硬件会自动清 0。	0x0

24.7.24 I2S_COMP_PARAM_2

◆ **偏移地址：0x1f0　　复位值：0x00000624**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
i2s_comp_param_2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i2s_comp_param_2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	1	1	0	0	0	1	0	0	1	0	0

Name	Bits	R/W	Description	Default
i2s_comp_param_2	[31:0]	RO	该寄存器读取返回值在不同模块的主或者从模式读取值有区别。 I2S0-从模式：0x624 I2S0-主模式：0x624 I2S1-从模式：0x48c I2S1-主模式：0x48c	0x624

24.7.25 I2S_COMP_PARAM_1

◆ **偏移地址：0x1f4　　复位值：0x03240566**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
i2s_comp_param_1															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	1	1	0	0	1	0	0	1	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i2s_comp_param_1															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	1	0	1	0	1	1	0	0	1	1	0

Name	Bits	R/W	Description	Default
i2s_comp_param_1	[31:0]	RO	该寄存器读取返回值在不同模块的主或者从模式读取值有区别。 I2S0-从模式：0x03240566 I2S0-主模式：0x03240576 I2S1-从模式：0x024c0066 I2S1-主模式：0x024c0076	0x03240566

24.7.26 I2S_COMP_VERSION

◆ **偏移地址：0x1f8　　复位值：0x3131312A**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
i2s_comp_version															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	1	1	0	0	0	1	0	0	1	1	0	0	0	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i2s_comp_version															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	1	1	0	0	0	1	0	0	1	0	1	0	1	0

Name	Bits	R/W	Description	Default
i2s_comp_version	[31:0]	RO	版本信息寄存器	0x3131312a

24.7.27 I2S_COMP_TYPE

◆ **偏移地址：0x1fc　　复位值：0x445701A0**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
i2s_comp_type															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	1	0	0	0	1	0	0	0	1	0	1	0	1	1	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
i2s_comp_type															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	1	1	0	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
i2s_comp_type	[31:0]	RO	类型信息寄存器	0x445701a0

24.7.28 DMACR

◆ **偏移地址：0x200　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					dma_en_txch_2	dma_en_txch_1	dma_en_txch_0	Res.					dma_en_rxch_2	dma_en_rxch_1	dma_en_rxch_0
-	-	-	-	-	RW	RW	RW	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留	0x0
dmaen_txch_2	[10]	RW	发送通道 2，DMA 使能开关配置。I2S0 包括 3 个通道有此配置位。 0：关闭 DMA 模式 1：打开 DMA 模式	0x0
dmaen_txch_1	[9]	RW	发送通道 1，DMA 使能开关配置。I2S0 包括 3 个通道有此配置位。 0：关闭 DMA 模式 1：打开 DMA 模式	0x0
dmaen_txch_0	[8]	RW	发送通道 0，DMA 使能开关配置。 0：关闭 DMA 模式 1：打开 DMA 模式	0x0
reserved	[7:3]	-	保留	0x0
dmaen_rxch_2	[2]	RW	接收通道 2，DMA 使能开关配置。I2S0 包括 3 个通道有此配置位。 0：关闭 DMA 模式 1：打开 DMA 模式	0x0
dmaen_rxch_1	[1]	RW	接收通道 1，DMA 使能开关配置。I2S0 包括 3 个通道有此配置位。 0：关闭 DMA 模式 1：打开 DMA 模式	0x0
dmaen_rxch_0	[0]	RW	接收通道 0，DMA 使能开关配置。 0：关闭 DMA 模式 1：打开 DMA 模式	0x0

24.7.29 RXDMA_CHx

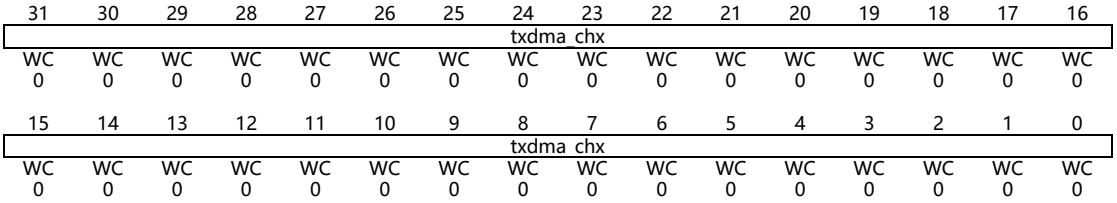
◆ **偏移地址：0x204　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
rxdma_chx															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rxdma_chx															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
rxdma_chx	[31:0]	RO	DMA 模式时通过此单个寄存器读取立体声数据，代替 LRBRx 和 RRBRx。	0x0

24.7.30 TXDMA_CHx

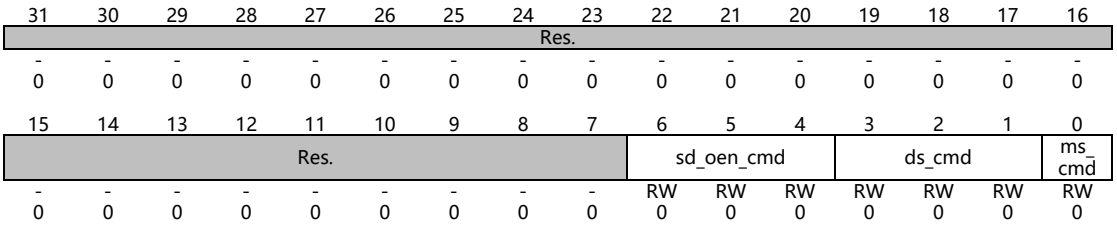
◆ **偏移地址：0x214　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
txdma_chx	[31:0]	WC	DMA 模式时通过此单个寄存器写入数据，代替 LTBRx 和 TRBRx。	0x0

24.7.31 EXT_CTRL

◆ **偏移地址：0x334　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:7]	-	保留	0x0
sd_oen_cmd	[6:4]	RW	各个 channel SD 总线输入输出配置，仅当 ds_cmd 配置为半双工时此寄存器配置有效 sd_oen_cmd[0]=0：channel0 SD 双向总线配置为输出 sd_oen_cmd[0]=1：channel0 SD 双向总线配置为输入 sd_oen_cmd[1]=0：channel1 SD 双向总线配置为输出 sd_oen_cmd[1]=1：channel1 SD 双向总线配置为输入 sd_oen_cmd[2]=0：channel2 SD 双向总线配置为输出 sd_oen_cmd[2]=1：channel2 SD 双向总线配置为输入	0x0

Name	Bits	R/W	Description	Default
			注意：仅当 IP 内部寄存器 IEN=0 时即模块全局开关控制为禁用状态时允许配置此寄存器; I2S0 包括 3 个通道，I2S1 只有 1 个通道。	
ds_cmd	[3:1]	RW	各个 channel 全双工/半双工配置寄存器 ds_cmd[0]=0：channel0 I2S 配置为半双工 ds_cmd[0]=1：channel0 I2S 配置为全双工 ds_cmd[1]=0：channel1 I2S 配置为半双工 ds_cmd[1]=1：channel1 I2S 配置为全双工 ds_cmd[2]=0：channel2 I2S 配置为半双工 ds_cmd[2]=1：channel2 I2S 配置为全双工 注意：仅当 IP 内部寄存器 IEN=0 时允许配置，即全局开关为禁用状态时允许配置此寄存器; 默认为都为半双工。 I2S0 包括 3 个通道，I2S1 只有 1 个通道。	0x0
ms_cmd	[0]	RW	Master/Slave 配置寄存器 0：I2S 配置为 Slave 1：I2S 配置为 Master 注意：仅当 IP 内部寄存器 IEN=0 时即模块全局开关控制为禁用状态时允许配置此寄存器; 默认 Slave。	0x0

24.7.32 GATECLKDLY

◆ **偏移地址：0x338　　复位值：0x00000002**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													gateclk_dly_sel		
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
gateclk_dly_sel	[2:0]	RW	2s Master 模式时，控制输出到 slave 的 sclk 的延时选择，通过此寄存器跳转 sclk 与 ws 和 sdo 线的相位关系 gateclk_dly_sel=0：输出 sclk 的相位无 cycle 延迟 gateclk_dly_sel=1：输出 sclk 的相位 1cycle 延迟 gateclk_dly_sel=2：输出 sclk 的相位 2cycle 延迟 gateclk_dly_sel=3：输出 sclk 的相位 3cycle 延迟	0x2

Name	Bits	R/W	Description	Default
			gateclk_dly_sel=4：输出 sclk 的相位 4cycle 延迟 gateclk_dly_sel=5：输出 sclk 的相位 5cycle 延迟 注意：仅当 IP 内部寄存器 IEN=0 时即模块全局开关控制为禁用状态时允许配置此寄存器;	

25 通用异步收发传输器（UART）

25.1 UART 简介

UART 是一个异步通信串行接口，主要功能是用于同外部的设备进行对接和数据交换。UART 可以实现全双工通信，支持流控传输和 RS485 功能以及 IrDA 红外传输协议。可以通过小数波特率分频配置实现多种波特率，在 DMA 模式下还可以实现高速的数据访问通信。芯片内部最多内置 5 个 UART。

25.2 UART 主要特性

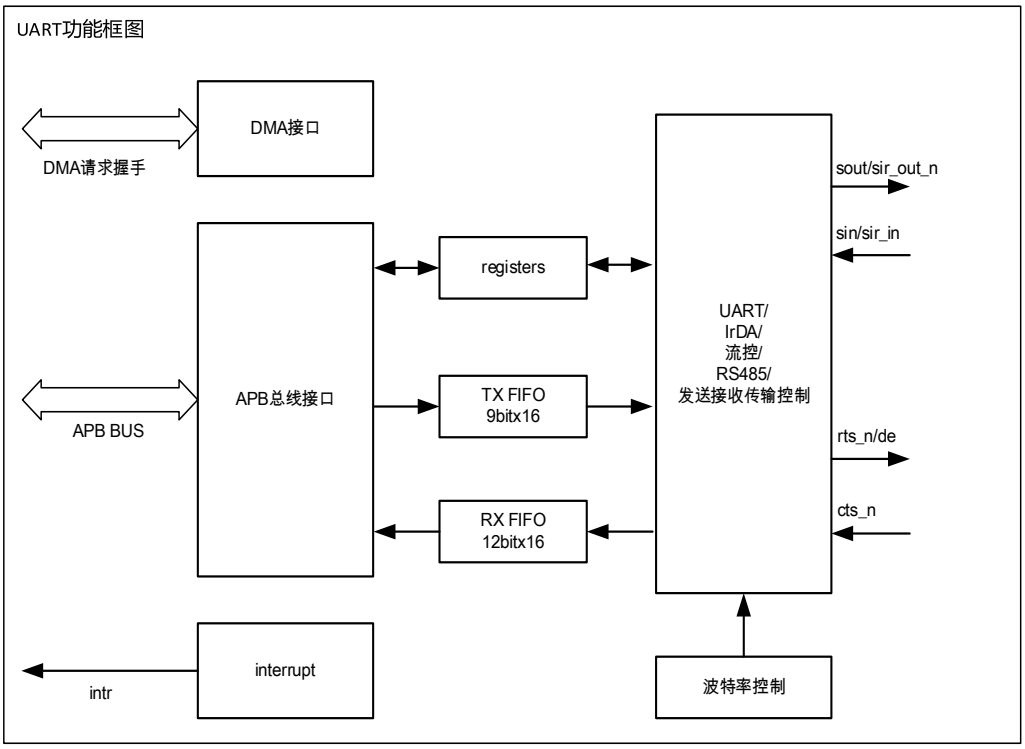
UART 的功能特点有：

- 配置时钟和数据传输工作时钟独立。
- 支持全双工模式。
- 支持硬件流控。
- 支持 RS485 通信接口。
- 支持 IrDA1.0 通信协议。
- 支持 9bit 模式开启和关闭，可实现 5、6、7、8、9 位可选数据位宽。
- 支持位反序功能。
- 支持传输停止位宽可配置（1,1.5,2 位）。
- 传输波特率可以编程配置，可以实现小数波特率设置。
- 支持 FIFO 模式，内置独立的发送 FIFO 和接收 FIFO，深度为 16，最大可以缓冲 16 帧数据。
- 支持奇偶校验。
- 支持 DMA 模式。

25.3 UART 逻辑实现

UART 支持 MCU 与外部器件之间进行异步串行通信，应用软件可通过轮询状态标志或者使用 UART 专用中断或者 DMA 方式对通信进行管理，用户可以根据实际需要选择 UART 使用哪种方式进行数据传输。UART 的逻辑框图及其交互方式如下面图 25-1 所示。

图 25-1 UART 框图



UART 最大可通过 4 个专用引脚与外部器件通讯，部分引脚在不同模式下的功能有所区别，具体功能如下。

- sout/sir_out_n：非红外 IrDA 模式下该引脚为常规 UART 的 TX 输出；红外模式该引脚为红外协议下 TX 输出。
- sin/sir_in：非红外 IrDA 模式下该引脚为常规 UART 的 RX 输入；红外模式该引脚为红外协议下 RX 输入。
- rts_n/de：是一个输出信号，四线流控模式下该引脚输出请求发送信号；RS485 模式该引脚控制外部驱动的输出使能。非流控和 RS485 模式该引脚不会使用。
- cts：是一个输入信号，四线流控模式下该引脚接收清除发送信号，和 rts_n 握手。非流控模式该引脚不会使用。

UART 同芯片内部的信息传输主要通过以下三个方式：

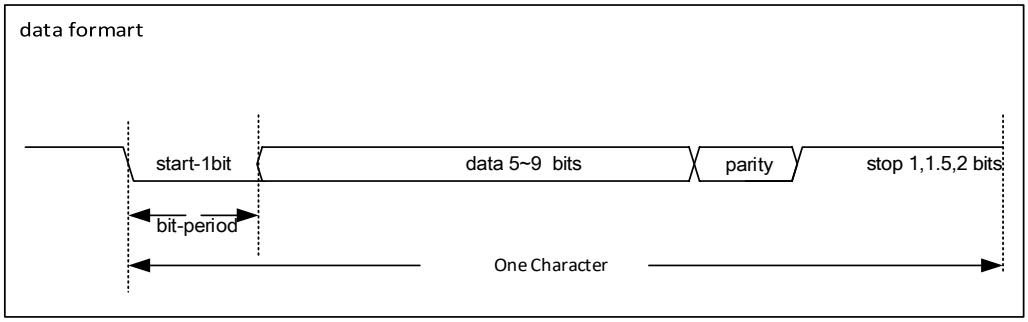
1. DMA 请求握手：用于同 DMA 传输握手。
2. APB BUS：用于配置模块内部寄存器和数据收发。
3. intr：用于上报中断给 CPU。

25.4 UART 功能描述

25.4.1 UART 串行协议

串行数据结构伴随着启动和停止位被称为一帧（character）。如下图 25-2 所示。

图 25-2 串行数据格式



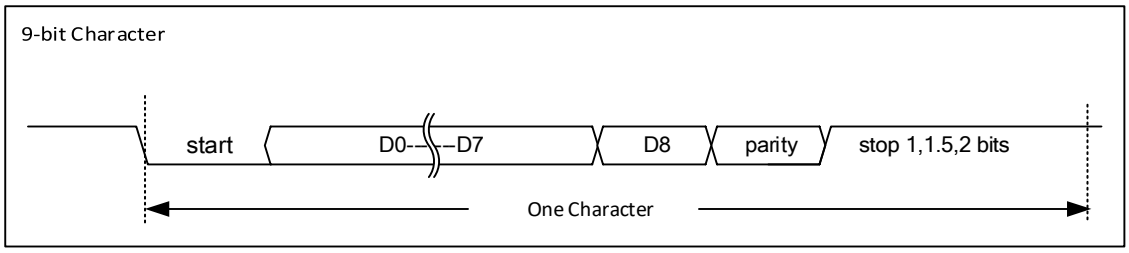
在串行传输的数据帧中增加奇偶校验位（parity bit），该位出现在最后一个数据位之后，对所接收的数据执行简单的错误检查。数据的各个比特在起始比特之后发送，默认从最低有效位（LSB）开始，也可配置反序模式实现高有效位（MSB）开始。后面是可选的奇偶校验位，最后是 1 或者 1.5 或 2 位长度的停止位，停止位长度可配置。

25.4.2 9-bit 数据传输

在发送和接收模式下都具有 9 位数据传输。字符中的第 9 位出现在字数据第 8 位之后，奇偶校验位之前。

图 25-3 显示了 D8 代表第 9 位的字符的串行传输，也显示了 9 位模式下的一般串行传输。

图 25-3 9-bit 数据格式



9 位数据传输模式可用于多点系统，其中的主设备连接到系统中的多个从设备。主设备与其中一个从设备通信，当主设备要将数据传输到从设备时，它首先发送一个地址字节来标识目标从设备。

地址/数据字节之间的区分是根据输入字符中的第 9 位进行的，如果第 9 位（D8）设置为 0，则该字符表示数据字节。如果第 9 位设置为 1，则字符表示地址字节。所有从系统都将地址字节与自己的地址进行比较，目标从设备（其中地址已匹配）能够从主设备接收数据。然后主设备开始向目标从设备发送数据字节。非寻址从属系统忽略传入数据，直到接收到新的地址字节。

发送模式

- **发送模式 0**：发送的地址需要写入 TAR，数据需要写入 THR 寄存器或者 STHRn 寄存器，此模式下 THR 和 STHRn 的第 9bit 没有使用。发送写入 TAR 的从设备地址需要配置 LCR_EXT.send_addr 为 1，硬件发送完 TAR 后会自动对该位清零。
- **发送模式 1**：THR 和 STHRn 呈现为 9-bit 位宽，地址和数据都通过 THR 和 STHRn 寄存器写入，LCR_EXT.send_addr 位和 TAR 不适用于此模式。软件必须根据发送地址/数据将第 9 位打包为 1/0。

接收模式

- **硬件地址匹配模式**：LCR_EXT. addr_match 为 1，为硬件地址匹配模式。当接收到帧为地址帧且地址的内容和配置到 RAR 寄存器的值匹配时才会将后续的数据帧接收下来。
- **软件地址匹配模式**：LCR_EXT. addr_match 为 0，为软件地址匹配模式。始终接收 9 位数据，并在 FIFO 模式下放进 RxFIFO，或在非 FIFO 模式下送入 RBR 寄存器。每当接收到地址字节并通过线路状态寄存器中的 ADDR_RCVD 位指示时，用户必须比较地址。如果地址不匹配，用户可以通过 FCR 寄存器中的 RFIFOR 位刷新/重置 RxFIFO。

25.4.3 RS485 串行协议

模块支持 RS485 串行协议，该协议允许使用 RS485 接口传输串行数据。

通过以下流程配置 RS485 接口模式：

- TCR.rs485_en 用于使能或者关闭 RS485 模式。
- TCR.de_pol 用于选择 de 的有效极性。
- TCR.xfer_mode 用于选择不同模式类型。
- DE_EN 寄存器用于通过软件控制 de。
- RE_EN 寄存器用于通过软件控制 re。
- DET 寄存器用于配置 de 的 assertion 和 de-assertion 时间。

25.4.4 小数分频

支持分数波特率，使用户可以对除数值的小数部分进行编程，以生成分数波特率，从而减少频率误差。小数部分的位宽为 6。

波特率主要由有以下因素决定：

- 系统配置该模块 sclk 频率（UART sclk 选择配置参考 4 时钟和复位控制 (CRG)，配寄存器 PERI_CRG21~PERI_CRG25）
- 期望的波特率
- 波特率产生控制的分频配置，整数部分 16bit 数值 BRD_I 由 DLH 和 DLL 寄存器得到，小数部分 6bit 数值 BRD_F 由 DLF 寄存器值/64 得到。另外由于固定存在 16 倍过采样处理，因此，波特率分频值= (sclk 频率)/(16 ×期望波特率) = BRD_I + BRD_F，需要注意的是 DLF 需要先于配置 DLH 和 DLL，且 DLH 和 DLL 不能同时都为 0。

可编程的小数波特率除法器实现了比传统整数除法器更精细的波特率时钟分辨率。可编程小数波特时钟分频器允许整数除数和小数分量的可编程性，从而为波特率时钟的频率平均提供了更精细的分辨率。

系统工作在最高主频 200MHZ 使用小数分频：UART0、UART1、UART2 波特率最高支持 2Mbps，UART3、UART4 波特率最高支持 4Mbps。

UART0、UART1、UART2 如果要配置高于 2Mbps 波特率，只能使用整数分频，小数部分配置为 0。UART3、UART4 如果要配置高于 4Mbps 波特率，只能使用整数分频，小数部分配置为 0。

25.4.5 IrDA1.0 SIR 协议

红外数据协会（IrDA）1.0 串行红外（SIR）模式支持使用红外辐射作为传输介质与远程设备进行双向数据通信。IrDA 1.0 SIR 模式指定的最大波特率为 115200 bps。

数据格式类似于标准串行 sout 和 sin 数据格式。每个数据字符按此顺序串行发送：

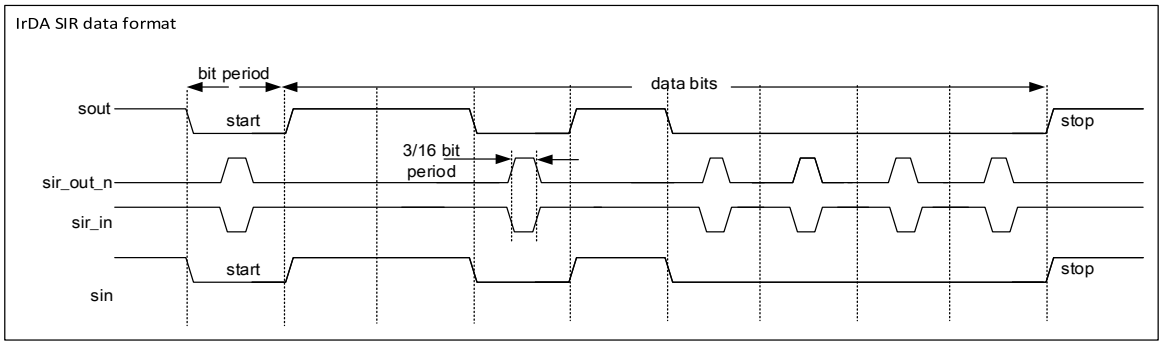
1. 以 start 位开始。
2. 8 个数据位。
3. 至少 1 个 stop 位。

因此可以发送的数据位数是固定的，不能提供奇偶校验信息，在此模式下仅使用停止位。尝试调整发送的数据位数或启用与行控制寄存器（LCR）的奇偶校验是无效操作。

MCR.sire 用于使能或者关闭 IrDA1.0 SIR 模式，使能该模式后串行数据的发送和接收通过 sir_out_n 和 sir_in 表示。端口上在一个 bit 时间内传输了一个红外脉冲表示逻辑 0，否则表示逻辑 1。每个脉冲的宽度是正常串行位时间的 3/16。因此每个新字符以起始位的红外脉冲开始。然而由于红外脉冲激励 IrDA 接收器的光电晶体管基极，使其输出变低，因此接收到的数据与发送的数据相反。

图 25-4 显示了与标准串行格式相比的 IrDA SIR 数据格式的时序图。

图 25-4 IrDA SIR 数据格式



当启用 IrDA SIR 模式时，操作方式与普通 UART 模式时类似，但有一个例外：当启用 IrDA-SIR 模式时，数据传输只能以半双工方式进行。由于 IrDA SIR 物理层规定了传输和接收之间的最小 10ms 延迟，这个 10ms 的延迟需要软件产生。

25.4.6 中断功能

UART 模块通过 intr 输出（高有效）通知 MCU 系统该模块有中断发生，中断的输出需受到中断使能的控制，中断使能控制可参考 IER 寄存器。

当中断发生后，cpu 需要读取 IIR 寄存器查询中断状态。相关的中断类型和描述如下表 25-1 所述。

表 25-1 中断控制功能描述

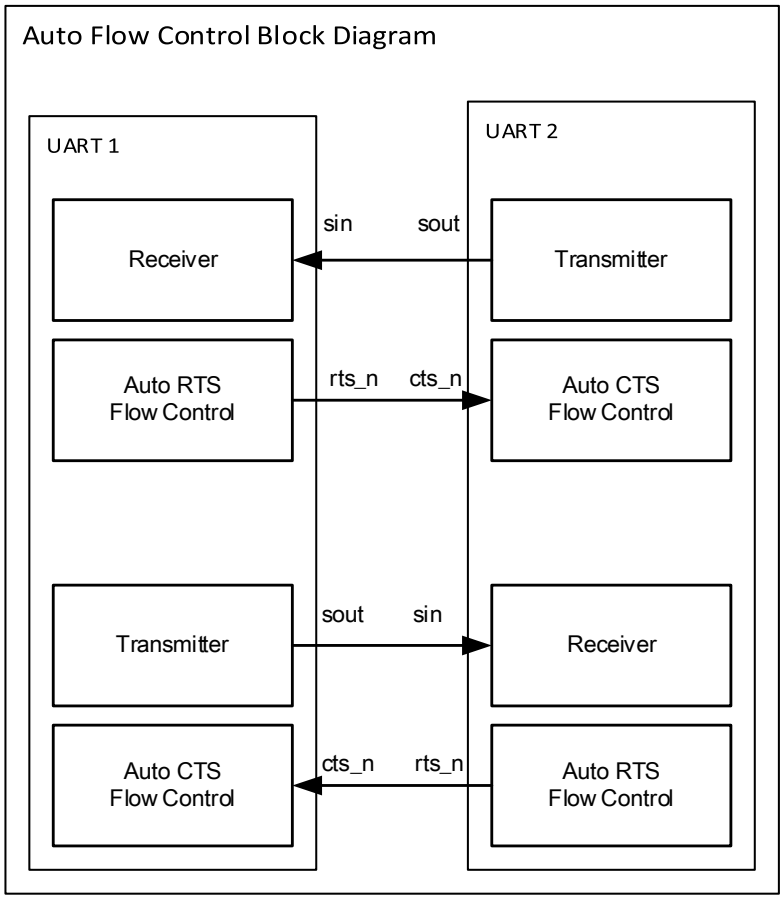
IIR.iid	优先级	中断类型	中断源	清除中断方式
0x1	—	**无中断**	无	无

IIR.iid	优先级	中断类型	中断源	清除中断方式
0x6	Highest	接收线状态	Overrun/parity/ framing 错误, break 中断，或者接收到地址中断	清除方式取决于 IER.elcolr 寄存器
0x4	Second	接收数据有效	非 FIFO 模式：接收缓存寄存器有效。 FIFO 模式：RX FIFO 的数据达到阈值。	读取接收缓存寄存器。在 FIFO 模式下需要读取到 RX FIFO 阈值以下。
0xc	Second	帧超时	RX FIFO 中还存放着有效数据，但是已经有 4 帧时间以上没有数据再读取或者写入到 RX FIFO 中。	读取接收缓存寄存器
0x2	Third	发送空	非 FIFO 模式或者 IER.ptime 为 0：发送缓存寄存器空。 FIFO 模式：TX FIFO 的数据小于等于阈值。	读取 IIR 寄存器； 或者写数据进入 THR，FIFO 模式需要直到 TX FIFO 数据超过阈值
0x0	Fourth	Modem 状态	Clear to send（cst_n）发生。 在自动流控使能时，CTS 状态的改变（及 DCTS 置 1 的情况）不会产生中断。	读取 MSR 寄存器
0x7	Fifth	忙状态指示	UART 模块在 busy 状态下（USR.busy 为 1），系统配置写 LCR 线上控制寄存器	读取 UART 状态 USR 寄存器

25.4.7 自动流控

UART 可配置 Auto-RTS 和 Auto-CTS 串行数据流控制模式。自动流控的功能配合应用框图如下图 25-5 所示。

图 25-5 自动流控框图



Auto-RTS 功能有效的前提：

- MCR.rts 和 MCR.afce 设置为 1
- FIFO 使能 FCR.fifoe 开启
- SIR 模式 MCR.sire 关闭

UART 使能 Auto-RTS 时，当 UART 的 RX FIFO 阈值达到 FCR.rt 设置的阈值时， rts_n 输出信号会强制变成无效信号（高电平）。此时如果 rts_n 连接到另一个 UART 设备的 cts_n，对端的 UART 会停止发送数据，直到 UART 自己 RX FIFO 有可用空间。

Auto-CTS 功能有效的前提：

- MCR.rts 和 MCR.afce 设置为 1
- FIFO 使能 FCR.fifoe 开启
- SIR 模式 MCR.sire 关闭

启用自动 CTS 时，当 cts_n 输入信号无效（高电平）时，UART 会停止发送，避免接收端

的 RX FIFO 溢出。如果 cts_n 输入信号在发送最后一个停止位之前仍是有效（低电平），则 UART 在传输停止之前还会继续发送一帧数据。当发送被禁用时，发射器 FIFO 仍可被写入，甚至溢出。

25.4.8 DMA 功能

UART 模块内部有两个并行的 DMA 通道，可以通过输出的 dma_tx_req/dma_tx_single_req 来告知系统，发送的 TX FIFO 可以被写入了，输出 dma_rx_req/ dma_rx_single 来告知系统接收的数据可以被读取了。DMA 模块根据 UART 的 DMA 请求进行数据传输。

dma_tx_req 在以下情况下会变为有效：

- 在非 FIFO 模式下，发送保持寄存为空。
- 在 FIFO 模式下，当 THRE 中断模式关闭且发送 FIFO 为空时。
- 在 FIFO 模式下，当 THRE 中断模式开启且发送 FIFO 内数据小于配置的阈值。

dma_rx_req 在以下情况下会变为有效：

- 在非 FIFO 模式下，有一帧数据已经存放在了接收缓存寄存器中。
- 在 FIFO 模式下，当接收 FIFO 配置内数据大于等于配置的阈值时。

当和系统 DMA 控制配合使用时，需要配置 DMA 的一次写入/读出 UART 模块的数据个数（即 DMA 的 des_msize/src_msize）和 UART 配置的 TX/RX FIFO 的阈值匹配，如果 DMA 需要传输的总数据不是单次 DMA 请求传输个数的整倍数时，DMA 控制会在最后使用 single(dma_tx_single_req、 dma_rx_single_req)方式同 UART 进行传输。

需要注意这些 DMA 请求信号在中断传输模式下也会有效，如果使用过中断操作传输数据再切换为 DMA 功能操作传输数据，需要在配置使能 DMA 模块前先配置 FCR. xfifor 和 FCR. rfifor 复位清空 fifo，清除中断操作时送给 DMA 的 dma_tx_req、dma_tx_single_req 和 dma_rx_req、dma_rx_single 信号。

25.4.9 时钟与复位

UART 模块本身有两个时钟：

- 一个是 pclk，这是系统内部 cpu 总线与 UART 模块交互的 apb 总线时钟，CRG 模块中对 UART 模块的软件复位不会复位 pclk 管控的部分，如寄存器配置操作。
- 一个是 sclk，这是 UART 模块内部工作时钟，主要用于波特率产生和收发 UART 串行数据。CRG 模块中对 UART 模块的软件复位会复位 sclk 时钟管控部分。

由于 pclk 和 sclk 是异步时钟，在配置 pclk 和 sclk 需要满足约束：

((2 * pclk_cycles) + 4) < (39 * (Baud Divisor))

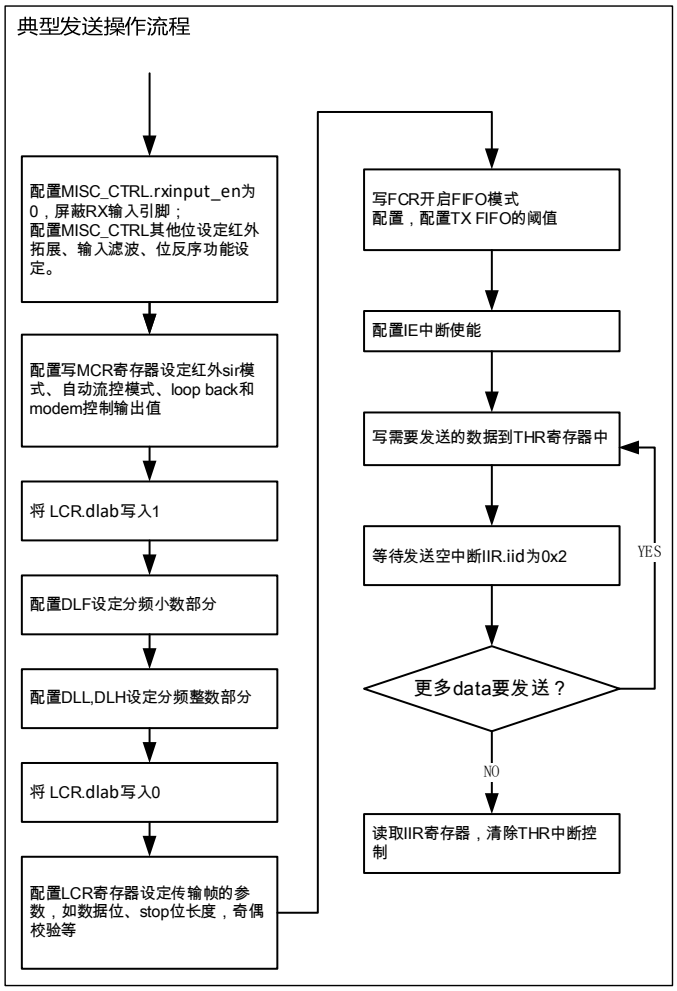
pclk_cycles 表示是 pclk 周期是 sclk 周期的倍数值。例如：当波特率分频值为 1 时，此时的 pclk_cycles 最大值为 17。即 sclk 的频率最大可以是 pclk 频率的 17 倍。

25.5 UART 配置流程

25.5.1 典型发送配置操作流程

图 25-6 描述了 UART 典型发送的初始化流程：

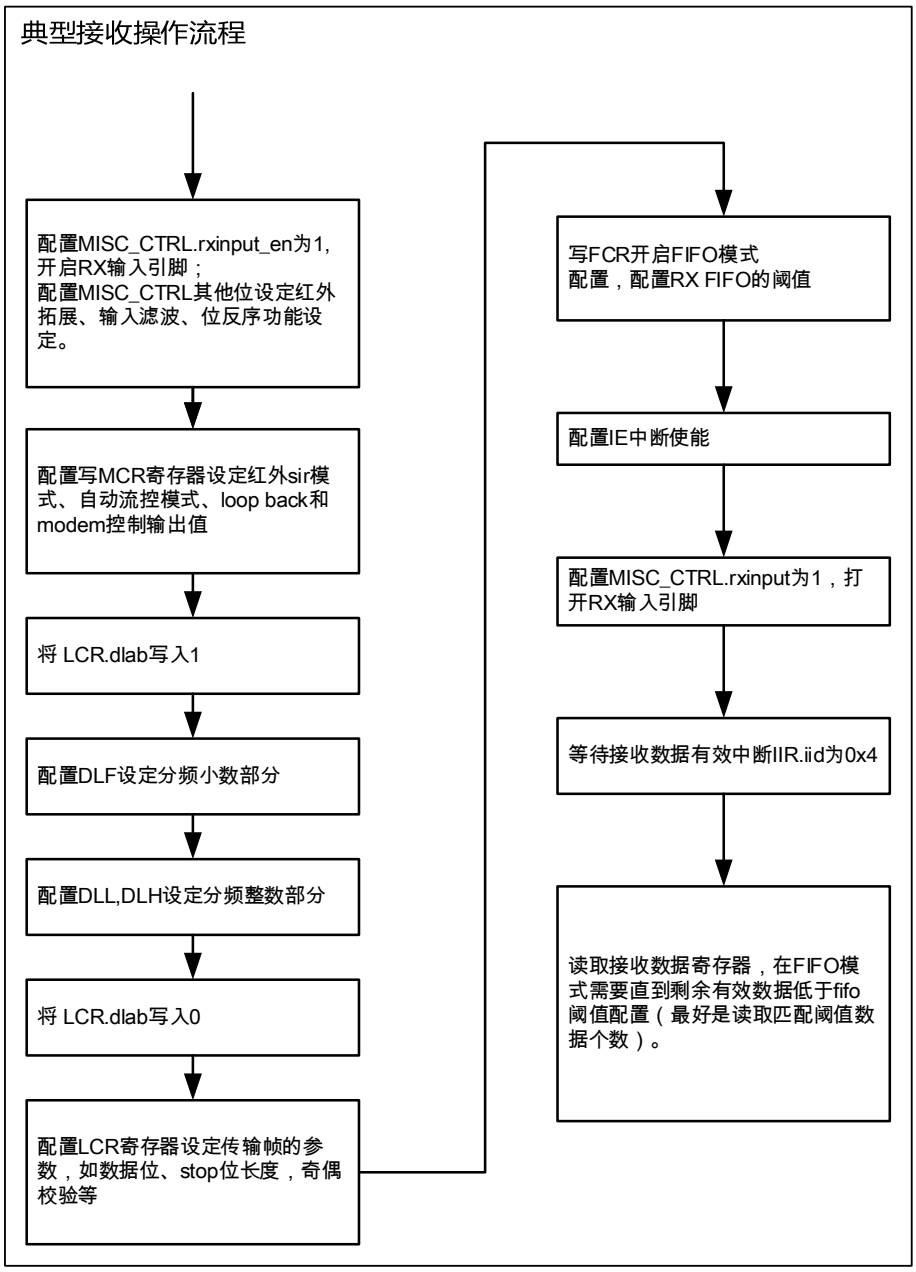
图 25-6 典型发送



25.5.2 典型接收配置操作流程

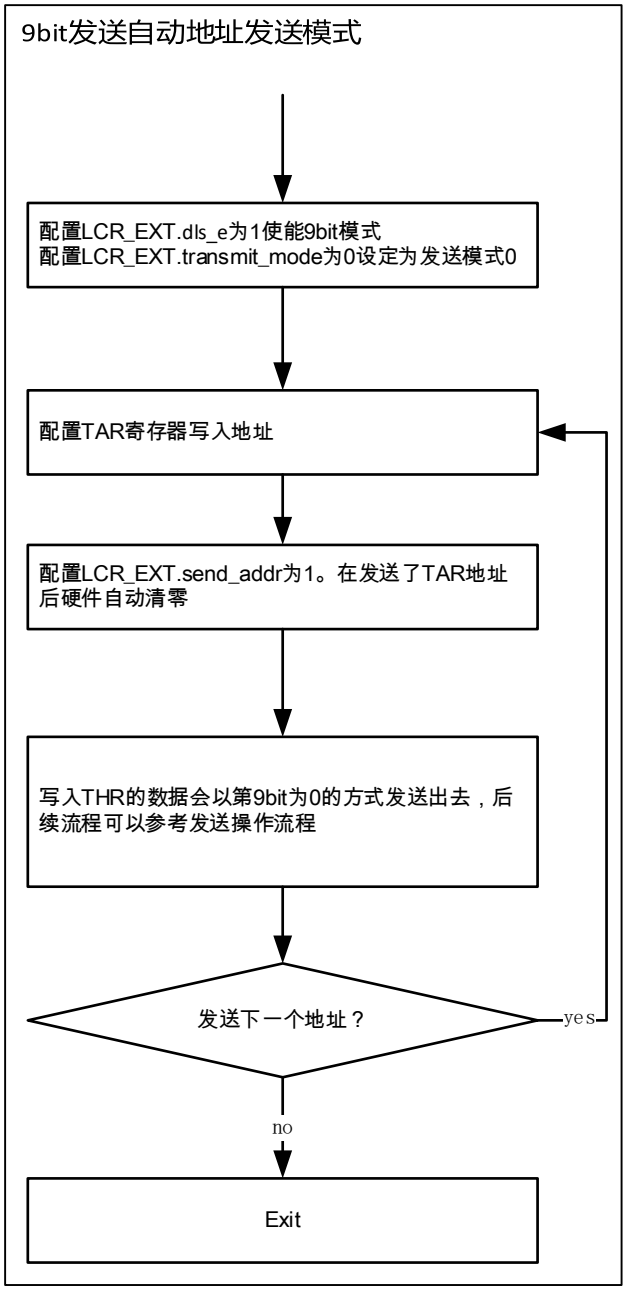
图 25-7 展示了 UART 典型接收的初始化流程

图 25-7 典型接收



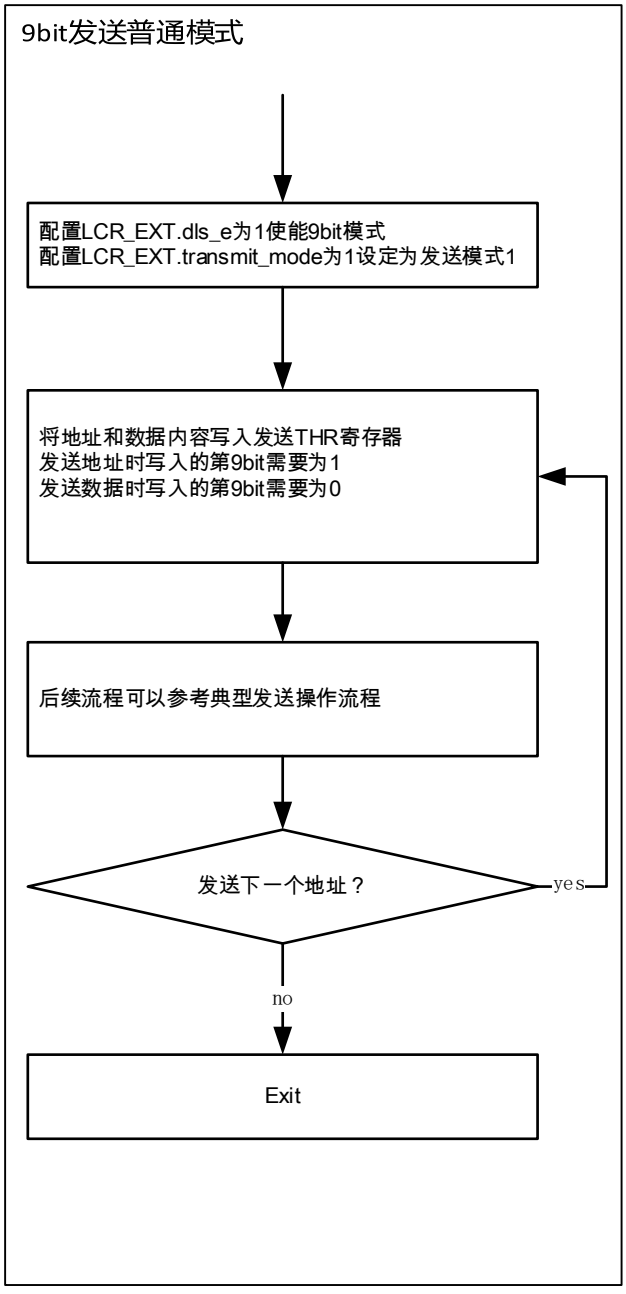
25.5.3 9-bit 数据格式发送模式 0

图 25-8 硬件发送地址



25.5.4 9-bit 数据格式发送模式 1

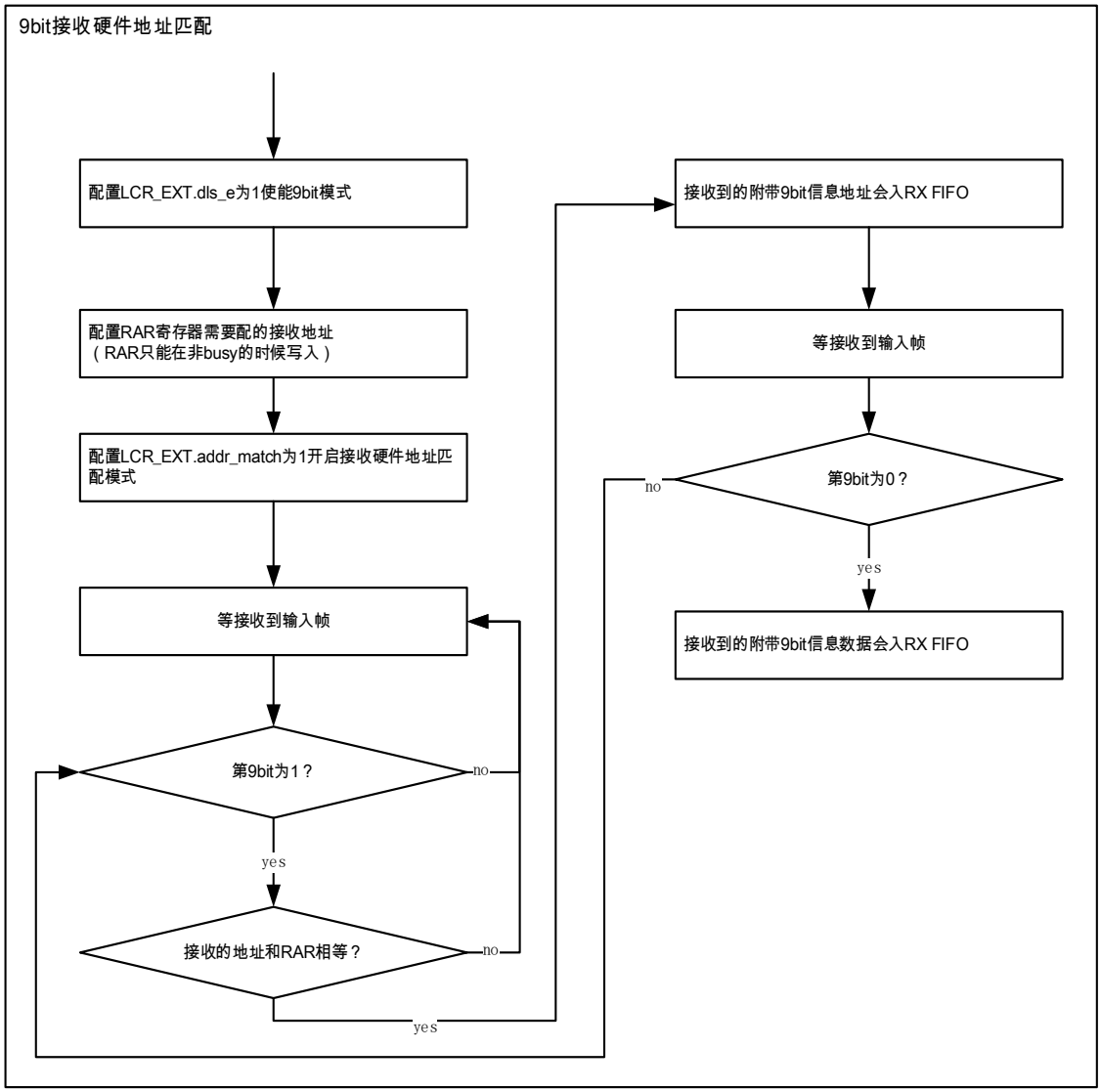
图 25-9 普通 9bit 发送



25.5.5 9-bit 数据格式接收硬件地址匹配模式

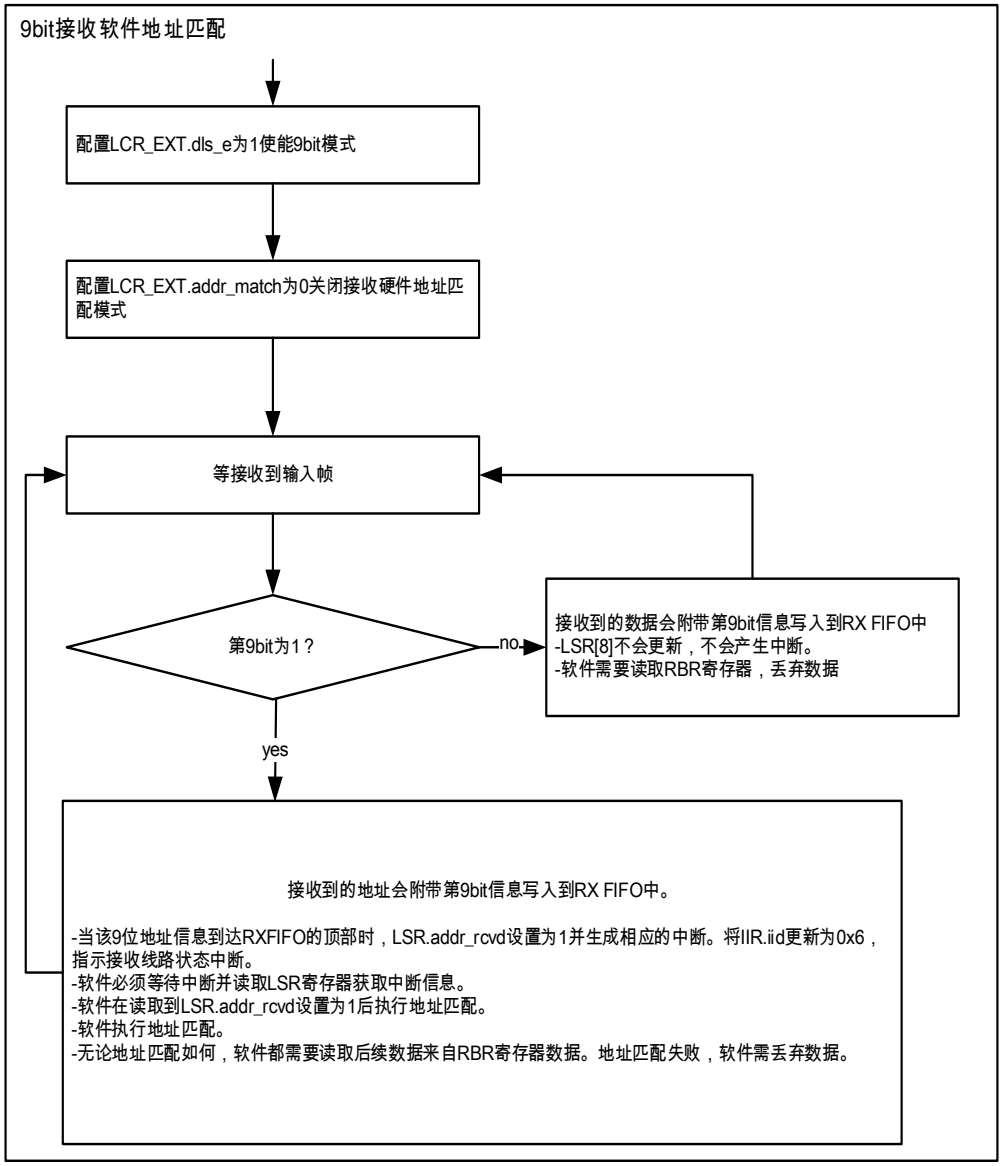
典型的数据接收模式可参照 25.5.1 典型发送配置操作流程，两种 9bit 的接收模式框图作为一个补充描述。

图 25-10 硬件地址匹配



25.5.6 9-bit 数据格式接收软件地址匹配模式

图 25-11 软件地址匹配



25.6 UART 寄存器概览

UART 实例 5 个，各个 UART 的基地址如下。

- UART0 寄存器基地址为：0x60010000
- UART1 寄存器基地址为：0x60011000
- UART2 寄存器基地址为：0x60012000

- UART3 寄存器基地址为：0x61007500
- UART4 寄存器基地址为：0x61007600

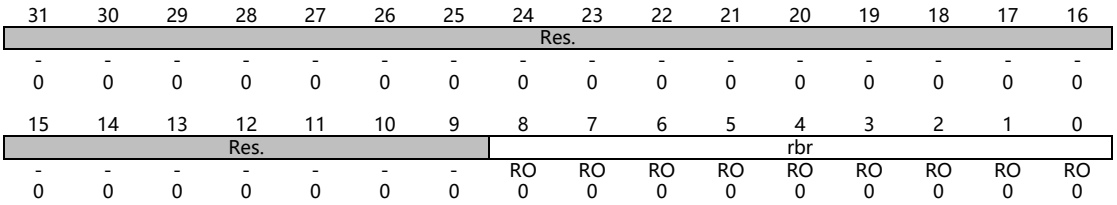
No.	Names	Offset Address	SIZE	R/W	Default	Description
1	RBR	0x0	32	RO	0x00000000	数据接收寄存器
2	THR	0x0	32	WC	0x00000000	数据发送寄存器
3	DLL	0x0	32	RW	0x00000000	整数分频参数低 8 位配置寄存器
4	DLH	0x4	32	RW	0x00000000	整数分频参数高 8 位配置寄存器
5	IER	0x4	32	RW	0x00000000	中断使能寄存器
6	FCR	0x8	32	WC	0x00000000	FIFO 控制寄存器
7	IIR	0x8	32	RO	0x00000001	中断状态标识寄存器
8	LCR	0xc	32	RW	0x00000000	线上控制寄存器
9	MCR	0x10	32	RW	0x00000000	Modem 控制寄存器
10	LSR	0x14	32	RO	0x00000060	线上状态寄存器
11	MSR	0x18	32	RO	0x00000000	Modem 状态寄存器
12	SCR	0x1c	32	RW	0x00000000	保留寄存器
13	MISC_CTRL	0x28	32	RW	0x00000000	补充功能控制寄存器
14	SRBRn	0x30+0x4*n	32	RO	0x00000000	数据接收影子寄存器
15	STHRn	0x30+0x4*n	32	WC	0x00000000	数据发送影子寄存器
16	FAR	0x70	32	RW	0x00000000	FIFO 访问模式控制寄存器
17	TFR	0x74	32	RO	0x00000000	TX FIFO 读取寄存器
18	RFW	0x78	32	WC	0x00000000	RX FIFO 写入寄存器
19	USR	0x7c	32	RO	0x00000006	UART 状态寄存器
20	TFL	0x80	32	RO	0x00000000	TX FIFO 数据个数寄存器
21	RFL	0x84	32	RO	0x00000000	RX FIFO 数据个数寄存器
22	SRR	0x88	32	WC	0x00000000	软复位寄存器
23	SRTS	0x8c	32	RW	0x00000000	MCR.rts 位影子寄存器
24	SBCR	0x90	32	RW	0x00000000	LCR.bc 影子寄存器
25	Reserved	0x94	32	-	0x00000000	保留
26	SFE	0x98	32	RW	0x00000000	FCR.fifoe 影子寄存器
27	SRT	0x9c	32	RW	0x00000000	FCR.rt 影子寄存器
28	STET	0xa0	32	RW	0x00000000	FCR.tet 影子寄存器
29	HTX	0xa4	32	RW	0x00000000	发送 HALT 控制寄存器
30	DMASA	0xa8	32	WC	0x00000000	DMA 软件 ACK 控制寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
31	TCR	0xac	32	RW	0x00000006	RS485 传输器控制寄存器
32	DE_EN	0xb0	32	RW	0x00000000	RS485 输出使能寄存器
33	RE_EN	0xb4	32	RW	0x00000000	RS485 接收使能寄存器
34	DET	0xb8	32	RW	0x00000000	RS485 驱动输出使能时间参数配置寄存器
35	TAT	0xbc	32	RW	0x00000000	TurnAround 时间配置寄存器
36	DLF	0xc0	32	RW	0x00000000	小数分频参数配置寄存器
37	RAR	0xc4	32	RW	0x00000000	接收地址配置寄存器
38	TAR	0xc8	32	RW	0x00000000	发送地址配置寄存器
39	LCR_EXT	0xcc	32	RW	0x00000000	扩展控制寄存器

25.7 UART 寄存器详细描述

25.7.1 RBR

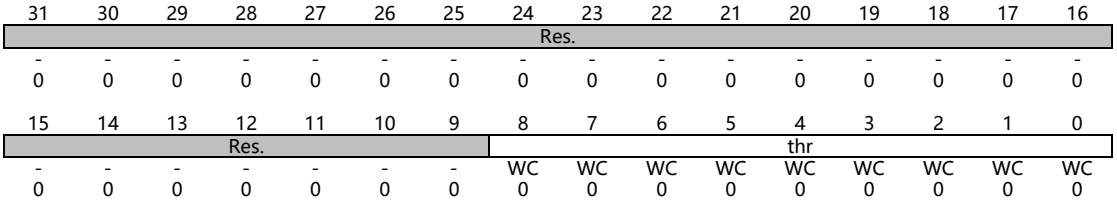
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
rbr	[8:0]	RO	接收数据。 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0

25.7.2 THR

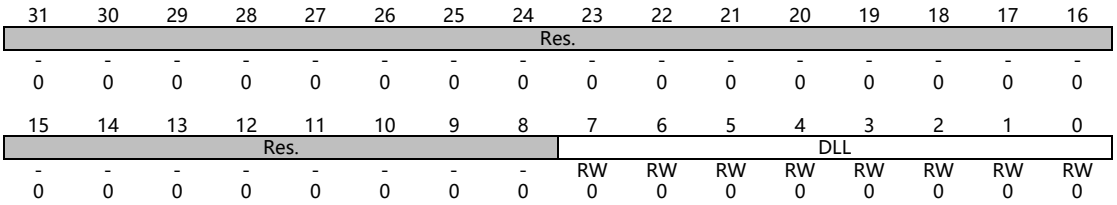
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
thr	[8:0]	WC	发送数据。 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0

25.7.3 DLL

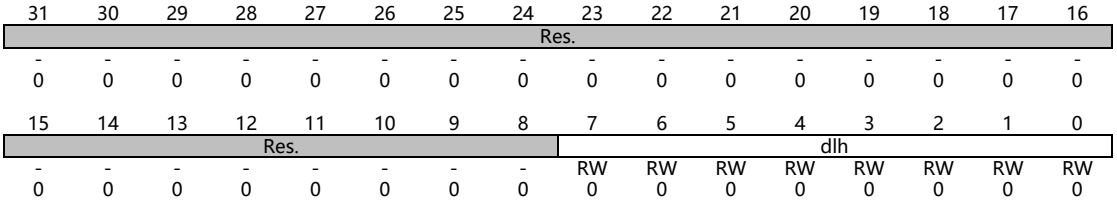
◆ **偏移地址：0x0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
dll	[7:0]	RW	配置波特率整数分频参数的低 8 位。 注意：只有当 LCR.dlab 被设置为 1，并且 UART 不忙时（USR.busy 为 0），才能访问该寄存器。当寄存器 DLL 和 DLH 设置为零时，波特率时钟被禁用不会发生串行通信。	0x0

25.7.4 DLH

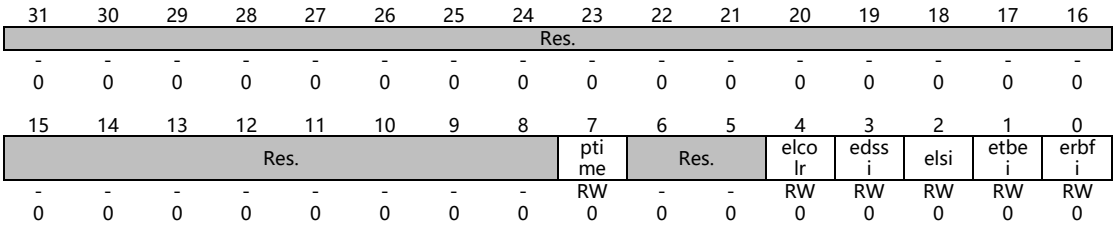
◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
dlh	[7:0]	RW	配置波特率整数分频参数的高 8 位。 注意：只有当 LCR.dlab 位被设置为 1，并且 UART 不忙时,即 USR.busy 为 0 时，才能访问该寄存器。	0x0

25.7.5 IER

◆ **偏移地址：0x4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
ptime	[7]	RW	这用于启用/禁用 THRE 中断的生成，该位启用时 FCR.tet 可控制发送中断在 TXFIFO 水线何时产生，禁用会等待 TXFIFO 全部空才产生中断。 0：禁用可编程 THR 中断模式 1：启用可编程 THR 中断模式 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0
reserved	[6:5]	-	保留。	0x0
elcolr	[4]	RW	该位控制清除 LSR 寄存器中状态的方法。这仅适用于溢出错误、奇偶校验错误、帧错误和 break 中断状态位。	0x0

Name	Bits	R/W	Description	Default
			0：读取 Rx FIFO（RBR 读取）或读取 LSR 寄存器时清除 LSR 状态位 1：仅在读取 LSR 寄存器时清除 LSR 状态位，建议在使用时该位配置为 1（尤其在使用 DMA 时），避免在读取数据时清除状态 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	
edssi	[3]	RW	Modem 状态中断使能。 0：禁用 1：启用 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0
elsi	[2]	RW	接收线路状态中断使能。 0：禁用 1：启用 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0
etbei	[1]	RW	发送保持寄存器空中断使能。 0：禁用 1：启用 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0
erbfi	[0]	RW	接收数据有效中断和帧超时中断使能。 0：禁用 1：启用 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器。	0x0

25.7.6 FCR

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								rt		tet		Res.	xfifor	rfifor	fifoe
-	-	-	-	-	-	-	-	WC	WC	WC	WC	-	WC	WC	WC
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0

Name	Bits	R/W	Description	Default
rt	[7:6]	WC	这用于在 RX FIFO 中选择将生成接收数据有效中断的触发阈值。在自动流控制模式下，它用于确定 rts_n 信号何时被取消。当处于某些操作模式时，用于控制 dma_rx_req 信号何时有效。 0：FIFO 中有 1 帧数据 1：FIFO 1/4 满 2：FIFO 1/2 满 3：FIFO 还差 2 帧数据就满	0x0
tet	[5:4]	WC	这用于选择 TX FIFO 空阈值，将在该阈值下生成 THRE 中断。它还确定在某些操作模式下 dma_tx_req 信号何时生效。 0：FIFO 已经空 1：FIFO 中有 2 帧数据 2：FIFO 1/4 满 3：FIFO1/2 满	0x0
reserved	[3]	-	保留。	0x0
xfifor	[2]	WC	复位清空 TX FIFO，并释放 DMA 发送求为无效。 1：清空 TX FIFO，配置为 1 后硬件会自动清 0 0：不操作	0x0
rfifor	[1]	WC	复位清空 RX FIFO，并释放 DMA 接收求为无效。 1：清空 RX FIFO，配置为 1 后硬件会自动清 0 0：不操作	0x0
fifoe	[0]	WC	FIFO 启用开关，每当该位的值改变时，FIFO 的 TX 和 RX 控制器部分都会复位。 0：禁用 FIFO 1：启用 FIFO	0x0

25.7.7 IIR

◆ **偏移地址：0x8　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								fifose		Res.		iid			
-	-	-	-	-	-	-	-	RO	RO	-	-	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
fifose	[7:6]	RO	指示 FIFO 禁用/启用转态。	0x0

Name	Bits	R/W	Description	Default
			0：FIFO 禁用 3：FIFO 启用 其他值：保留	
reserved	[5:4]	-	保留。	0x0
iid	[3:0]	RO	表示当前最高优先级的挂起中断，可以指定的以下类型之一。 0：调制解调器状态 1：无中断挂起 2：THR 为空 4：接收数据可用 6：接收器线路状态 7：忙检测 12：帧超时 其他值：保留	0x1

25.7.8 LCR

◆ **偏移地址：0xc　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

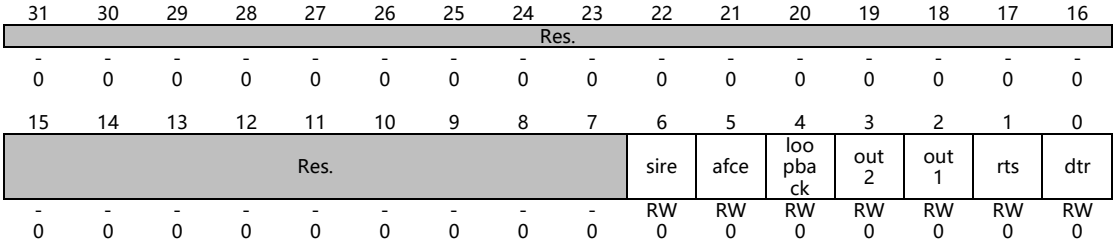
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								dlab	bc	sp	eps	pen	stop	dls	
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
dlab	[7]	RW	该位在 UART 不忙（USR.busy 为零）时才可写。用于启用 DLL 和 DLH 读写设置 UART 的波特率。访问其他寄存器，必须在初始波特率设置后清除此位为 0。 0：分频参数寄存器 DLL 和 DLH 不可以访问 1：分频参数寄存器 DLL 和 DLH 可以访问	0x0
bc	[6]	RW	这用于发送特殊打断情况，一般为测试模式使用。如果设置为 1，则串行输出强制为间隔（逻辑 0）状态。MCR.loopback 设定当不处于环回模式时，强制使信号线为低电平，直到中断位被清除。如果红外模式激活（MCR.sire 设置为 1），sir_out_n 线将持续脉冲。当处于环回模式时，在内部环回至接收器，sir_out_n 线强制为低。	0x0

Name	Bits	R/W	Description	Default
			0：为数据传输释放串行输出 1：串行输出强制为间隔状态	
sp	[5]	RW	Stick Parity 功能。 该位在 UART 不忙（USR.busy 为零）时才可写。 此位用于强制奇偶校验值。当 PEN、EPS 和 Stick Parity 设置为 1 时，奇偶校验位被发送并检查为逻辑 0。如果 PEN 和 Stick Parity 设置为 1 并且 EPS 为逻辑 0，则奇偶校验位被发送并作为逻辑 1 被检查。如果该位设置为 0，则禁用 Stick Parity。 0：Stick Parity 功能禁用 1：Stick Parity 功能启用	0x0
eps	[4]	RW	该位在 UART 不忙（USR.busy 为零）时才可写。 当启用奇偶校验（PEN 设置为 1）时，此选项用于在偶数和奇数奇偶校验之间进行选择。如果设置为 1，则发送或检查偶数个逻辑“1”。如果设置为零，则发送或检查奇数逻辑“1”。 0：发送或检查奇数个逻辑“1” 1：发送或检查偶数个逻辑“1”	0x0
pen	[3]	RW	该位在 UART 不忙（USR.busy 为零）时才可写。 奇偶校验开关。 0：禁用 parity 奇偶校验 1：启用 parity 奇偶校验	0x0
stop	[2]	RW	该位在 UART 不忙（USR.busy 为零）时才可写。 这用于选择外设将发送和接收的每个字符的停止位数。如果设置为 0，则在串行数据中传输一个停止位。如果设置为 1 并且数据位设置为 5（LCR.dls 设置为 0），则发送一个半停止位。否则，发送两个停止位。 注意，无论选择的停止位的数量如何，接收将只检查第一个停止位 0：1 个停止位 1：当 DLS（LCR.dls）为零时，1.5 个停止位，否则 2 个停止位	0x0
dls	[1:0]	RW	该位在 UART 不忙（USR.busy 为零）时才可写。 配置一帧里面数据 bit 数。 0：每帧 5 个数据位 1：每帧 6 个数据位 2：每帧 7 个数据位 3：每帧 8 个数据位	0x0

25.7.9 MCR

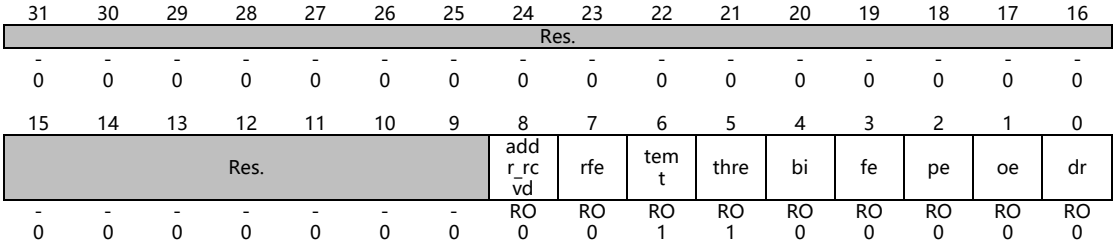
◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31：7]	-	保留。	0x0
sire	[6]	RW	IrDA SIR 红外模式启用开关。 0：禁用 1：启用 如果要启用 SIR 模式，在写入 LCR 寄存器之前，需要先将适当的值写入 MCR 寄存器。	0x0
afce	[5]	RW	自动流控模式启用开关。 0：禁用 1：启用	0x0
loopback	[4]	RW	内部自回环测试 loopback 启用开关。 这用于将 UART 置于 test 模式以进行测试。如果在 UART 模式下运行（红外模式没有启用），则输出线上的数据保持为高电平，而串行数据输出在内部循环回到输入线上。在这种模式下，所有中断都能正常工作。调制解调器控制输出在内部环回到输入。 如果在红外模式下运行，SIR_out_n 线上的数据保持低电平，而串行数据输出被反转并循环回到 SIR_in 线上。 0：禁用 1：启用	0x0
out2	[3]	RW	预留给 modem 模式的 out2，目前版本不会使用，建议配置为 0。	0x0
out1	[2]	RW	预留给 modem 模式的 out1，目前版本不会使用，建议配置为 0。	0x0
rts	[1]	RW	用于控制发送请求（rts_n）输出。发送请求（rts_n）输出用于通知 UART 已准备好交换数据。 0：将 rts_n 设定为非有效转态（逻辑 1） 1：将 rts_n 设定为有效转态（逻辑 0）	0x0
dtr	[0]	RW	预留给 modem 模式的 DTR，目前版本不会使用，建议配置为 0。	0x0

25.7.10 LSR

◆ **偏移地址：0x14　　复位值：0x00000060**



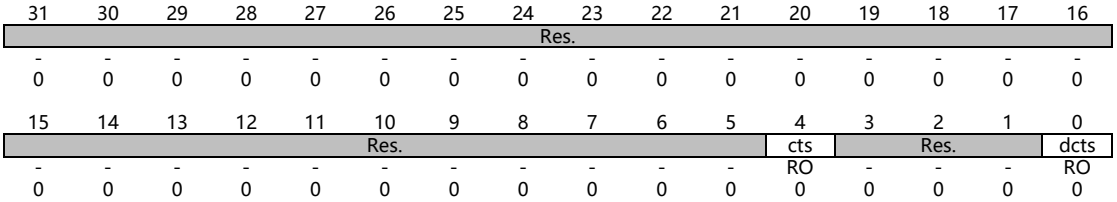
Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
addr_rcvd	[8]	RO	接收到地址帧状态指示。 如果启用了 9 位数据模式（LCR_EXT.dls_e=1），则该位用于指示接收数据的第 9 位设置为 1。该位还可用于指示传入字符是地址还是数据 0：指示此帧为数据帧 1：指示此帧为地址帧 在 FIFO 模式中，由于第 9 位与接收到的帧相关联，当位于 FIFO 的顶部存放帧的第 9 位设置为 1，就会通过此状态为指示告知。 读取 LSR 将清除该位。 注意：用户需要确保在下一个地址帧到达之前清除中断（读取 LSR 寄存器）。如果清除中断有延迟，则软件将无法区分多个地址相关中断。	0x0
rfe	[7]	RO	当 FIFO 已启用（FCR.fifoe 设置为 1）后，该位用于指示 FIFO 中是否至少存在一个奇偶校验错误、帧错误或 break 指示。 当读取 LSR 并且带有错误的帧位于接收器 FIFO 的顶部被读走并且后续 FIFO 顶部帧没有错误，该位才被清除。 0：RX FIFO 中无错误 1：RX FIFO 中有错误	0x0
temt	[6]	RO	TX 全空状态指示。 FIFO 模式下每当 TX 移位寄存器和 FIFO 均为空时，会设置此位为 1。如果处于非 FIFO 模式时每当发射机保持寄存器和发射机移位寄存器都为空时，该位被设置。 0：非空 1：空	0x1

Name	Bits	R/W	Description	Default
thre	[5]	RO	TX 传输保持寄存器空状态指示。 THRE 模式被禁用（IER.ptime 设置为零），该位表示 THR 或 TX FIFO 为空。 每当数据从 THR 或 TX FIFO 传输到发射机移位寄存器且没有新数据写入 THR 或 TXFIFO 时，该位被设置为 1。如果启用了 THRE 中断，这也会导致 THRE 中断。如果 THRE 模式启用且和 FIFO 两种模式均处于活动状态（IER.ptime 分别设置为 1 和 FCR.fifoe 设置为 1），功能切换为指示发射机 FIFO 已满，不再控制 THRE 中断，由 FCR.tet 设置控制。 0：非空 1：空	0x1
bi	[4]	RO	Break 中断。 这用于指示检测到串行输入数据上的 break 序列。 如果在 UART 模式下，只要串行输入 sin 保持在逻辑“0”状态的时间超过起始时间+数据位+奇偶校验位+停止位的总和，就会设置该值。 如果在红外模式下，只要串行输入 sir_in 持续脉冲至逻辑“0”的时间超过起始时间+数据位+奇偶校验位+停止位的总和，就会设置该值。 串行输入的中断条件导致 UART 接收一个且只有一个字符，由全零组成。 在 FIFO 模式下，与 break 条件相关的字符通过 FIFO 传送，当字符位于 FIFO 的顶部时显示。 读取 LSR 将清除 BI 位。 在非 FIFO 模式下，BI 指示立即出现，并持续到读取 LSR。 注意：如果收到中断条件时 FIFO 已满，则会发生 FIFO 溢出。break 条件以及与它相关的所有信息奇偶校验和成帧错误被丢弃；接收到 break 字符的任何信息都将丢失。 0：没有检测到 break 序列 1：检测到 break 序列	0x0
fe	[3]	RO	帧错误。 这用于指示接收机中出现帧错误。当接收机未检测到接收数据中的有效 STOP 位时，出现帧错误。 在 FIFO 模式中，由于帧错误与接收到的帧相关联，因此当具有帧错误的帧位于 FIFO 的顶部时，会显示出该错误。当发生帧错误时，UART 将尝试重新同步。它通过假设错误是由于下一个字符的起始位造成的，然后继续接收其他位，即数据和/或奇偶校验并停止。	0x0

Name	Bits	R/W	Description	Default
			应注意，如果 break 中断发生，则将设置帧错误（FE）位（LSR.fe）。发生这种情况是因为 break 字符通过将 sin 输入保持为逻辑 0 的时间长于帧的持续时间而隐式地生成帧错误。 读取 LSR 将清除 FE 位。 0：没有检测到帧错误 1：检测帧错误	
pe	[2]	RO	奇偶校验错误位。 如果设置了奇偶校验启用（PEN）位（LCR.pen），则这用于指示接收器中出现奇偶校验错误。 在 FIFO 模式中，由于奇偶校验错误与接收到的字符相关联，所以当具有奇偶校验错误的字符到达 FIFO 的顶部时，就会显示出来。 应注意，如果 break 中断发生，则将设置奇偶校验错误（PE）位（LSR.pe），在这种情况下，如果奇偶校验生成和检测被启用（LCR.pen = 1）并且奇偶校验被设置为奇数（LCR.eps = 0），则奇偶校验错误位被设置。 取 LSR 将清除该位。 0：没有奇偶校验 parity 错误 1：有奇偶校验 parity 错误	0x0
oe	[1]	RO	溢出错误。 这用于指示溢出错误的发生。如果在读取前一数据之前接收到新的数据字符，则会发生这种情况。 在非 FIFO 模式下，当新字符在从 RBR 读取前一个字符之前到达接收机时，OE 位被设置。发生这种情况时，RBR 中的数据将被覆盖。在 FIFO 模式下，当 FIFO 已满且新字符到达接收器时，发生超限错误。FIFO 中的数据被保留，而接收移位寄存器中的数据丢失。 读取 LSR 将清除 OE 位 0：没有溢出错误 1：发生溢出错误	0x0
dr	[0]	RO	数据可用指示。 这用于指示接收机在 RBR 或接收机 FIFO 中至少包含一个字符。在非 FIFO 模式下读取 RBR 时，或在 FIFO 模式下接收器 FIFO 为空时，清除此位。 0：数据不可用 1：数据可用	0x0

25.7.11 MSR

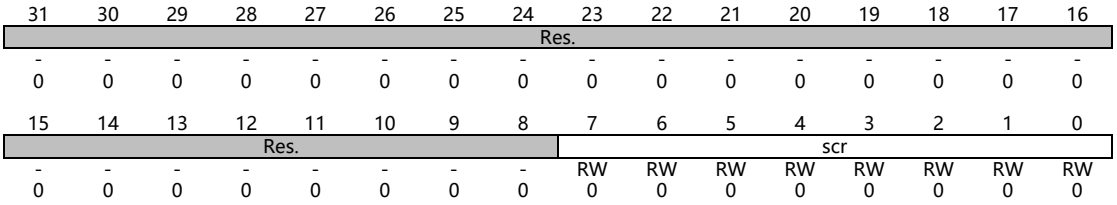
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
cts	[4]	RO	流控有效 cts_n 发生。 这用于指示调制解调器控制线 cts_n 的当前状态。这个位是补码 cts_n。当清除发送输入（cts_n）有效，表示调制解调器交换数据。 在环回模式（MCR.loopback 设置为 1）下，CTS 与 MCR.rts（RTS）相同。 0：输入的 cts_n 信号为逻辑 1，及无效状态 1：输入的 cts_n 信号为逻辑 0，及有效状态	0x0
reserved	[3:1]	-	保留。	0x0
dcts	[0]	RO	流控有效 cts_n 发生补充。 这用于指示调制解调器控制线 cts_n 自上次读取 MSR 以来已经改变。读取 MSR 将清除 DCTS 位。在环回模式（MCR.loopback 设置为 1）下，DCTS 反映 MCR.rts（RTS）的变化。 注意，如果未设置 DCTS 位并且 cts_n 信号为 0 并且发生复位（软件或其他），如果 cts_n 仍然保持 0，则当复位被移除时 DCTS 位将被设置为 1。 0：自上次读取 MSR 后，cts_n 上没有更改 1：自上次读取 MSR 后 cts_n 发生了变化	0x0

25.7.12 SCR

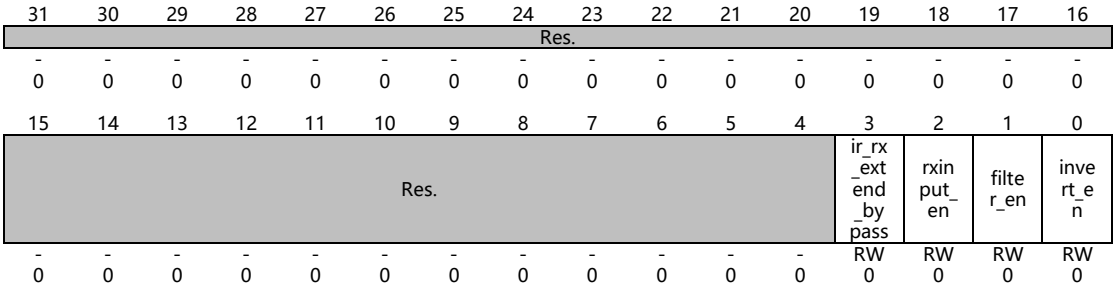
◆ **偏移地址：0x1c　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
scr	[7:0]	RW	预留可读可写寄存器位。	0x0

25.7.13 MISC_CTRL

◆ **偏移地址：0x28　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
ir_rx_extend_bypass	[3]	RW	控制是否 bypass 对红外 ir rx 输入低电平宽度扩展功能。 0：不 bypass 宽度拓展功能，红外 ir 输入的低电平宽度拓展开启。 1：bypass 宽度拓展功能，红外 ir 输入的低电平宽度拓展不开启。 注意：在该位配置位 0 时，会对红外输入的低电平进行拓展（UART 输入不受该 bit 配置影响），因为红外低电原始的宽度理论是 3/16 个波特率周期，为避免设计内部对低电平采样因为较小的波特率误差出错，该功能将 ir 输入超过 2/16 波特宽度的低电平拓展为一个波特宽度后用于内部数据采样，提高红外接受波特率误差容忍度。	0x0
rxinput_en	[2]	RW	控制外部输入的 UART RX 信号输入使能。 0：UART 和红外 RX 输入被屏蔽为固定 1，送入给模块内部 RX 固定接 1。 1：UART 和红外 RX 输入正常输入送给模块进行采样 注意：在模块配置初始化过程建议先配置该 bit 为 0，在初试化配置初始完成后再开启该 bit。避免在配置过程中出现由于 RX 为 0 导致配置过程中以错误的帧格式接受数据，或者出现不预期的 busy 状态。	0x0
filter_en	[1]	RW	输入信号滤波使能。	0x0

Name	Bits	R/W	Description	Default
			0：输入无滤波 1：开启固定 7 个 sclk 长度周期滤波功能，小于 7 个 sclk 周期长度信号的被认为是输入毛刺。 注意：cts 信号输入和红外 UART 数据输入信号的滤波功能开关	
invert_en	[0]	RW	位反序功能。 0：关闭位反序功能 1：开启位反序功能 注意：关闭反序功能，无特殊要求。开启反序功能时需要注意： 1）5-8bit databits（非 9bit 帧格式）反序使能使用无特殊限制 2）9bit 模式反序模式 TX 方向：只支持 LCR_EXT.transmit_mode 为 1 的模式 9bit 反序，及发送的数据或者地址都来源于 THR 或者 STHR 的情况。TX 的 9bit 反序的其他模式为非法配置，不支持。 RX 方向：只支持 LCR_EXT.addr_match 为 0 的模式 9bit 反序，及接收的帧都会放入 RXFIFO 的情况。RX 的 9bit 反序的其他模式为非法配置，不支持。 TX 如果不使用，及不发送数据的情况时。LCR_EXT.transmit_mode 的配置不影响 RX 方向。 RX 方向如果不使用或者不关心。LCR_EXT.send_addr 的配置不影响 TX 方向。	0x0

25.7.14 SRBRn

◆ **偏移地址：0x30+0x4*n　　复位值：0x00000000**

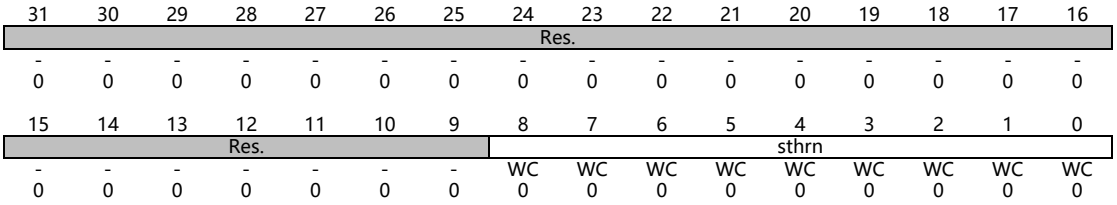
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							srbrn								
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
srbrn	[8:0]	RO	接收数据缓存 RBR 的影子寄存器，已分配了 16 个 32bit 位置，以适应来自主机的突发访问。 注意：只有当 LCR.dlab 位被清除时，才能访问该寄	0x0

Name	Bits	R/W	Description	Default
			存器	

25.7.15 STHRn

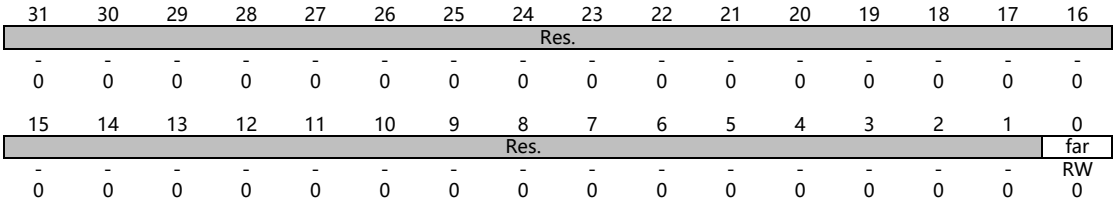
◆ **偏移地址：0x30+0x4*n　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
sthrn	[8:0]	WC	发送保 THR 持影子寄存器，已分配了 16 个 32bit 位置，以适应来自主机的突发访问。 注意：只有当 LCR.dlab 位被清除时，才能访问该寄存器	0x0

25.7.16 FAR

◆ **偏移地址：0x70　　复位值：0x00000000**

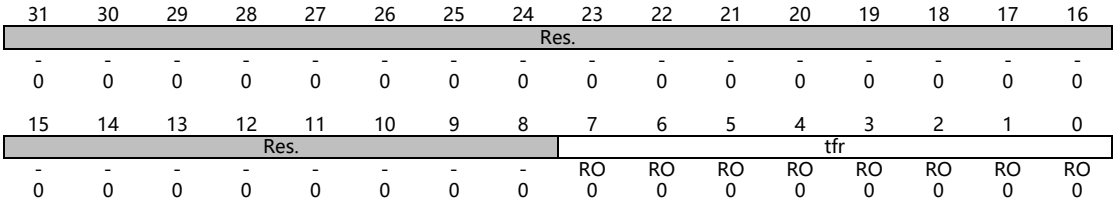


Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
far	[0]	RW	该寄存器用于启用 FIFO 访问模式。当 FIFO 启用（FCR.fifoe 设置为 1）且 FIFO 访问模式启用，RX FIFO 可以由主机写入，TX FIFO 可以由主机读取。当 FIFO 未启用时，它允许 RBR 由主机写入，THR 由主机读取。 注意：当 FIFO 访问模式被启用或者禁用时，RX FIFO 和 TX FIFO 的控制部分被重置，FIFO 被视为空。 0：FIFO 访问模式禁用	0x0

Name	Bits	R/W	Description	Default
			1：FIFO 访问模式启用	

25.7.17 TFR

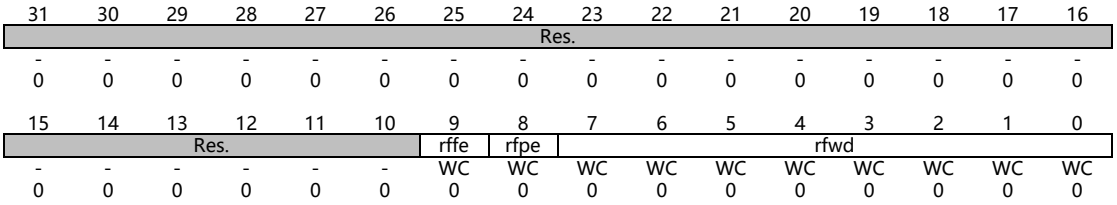
◆ **偏移地址：0x74　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
tfr	[7:0]	RO	该寄存器仅在 FIFO 访问模式启用（FAR.far 设置为 1）时有效。 当 FIFO 启用时，读取该寄存器可以获取 TX FIFO 顶部数据，读取动作会导致数据从 TX FIFO 弹出。 当 FIFO 未启用时，读取此寄存器将获取 THR 中的数据。	0x0

25.7.18 RFW

◆ **偏移地址：0x78　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:10]	-	保留。	0x0
rffe	[9]	WC	RX FIFO 帧错误控制。 该位仅在 FIFO 访问模式启用（FAR.far 设置为 1）时有效。当 FIFO 启用时，该位用于将成帧错误检测信息写入接收 FIFO。当 FIFO 未启用时，该位用于将成帧错误检测信息写入 RBR 寄存器。 0：帧错误禁用 1：帧错误启用	0x0
rfpe	[8]	WC	RX FIFO 奇偶校验错误控制。	0x0

Name	Bits	R/W	Description	Default
			该位仅在 FIFO 访问模式启用（FAR.far 设置为 1）时有效。当 FIFO 被并启用时，该位用于将奇偶校验错误检测信息写入接收 FIFO。当 FIFO 未启用时，该位用于将奇偶校验错误检测信息写入 RBR 寄存器。 0：奇偶校验错误禁用 1：奇偶校验错误启用	
rfwd	[7:0]	WC	RX FIFO 写入数据。 该位仅在 FIFO 访问模式启用（FAR.far 设置为 1）时有效。当 FIFO 启用时，写入 RFWD 的数据被推入接收 FIFO。当 FIFO 未启用时，写入 RFWD 的数据被推入 RBR 寄存器。	0x0

25.7.19 USR

◆ **偏移地址：0x7c　　复位值：0x00000006**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

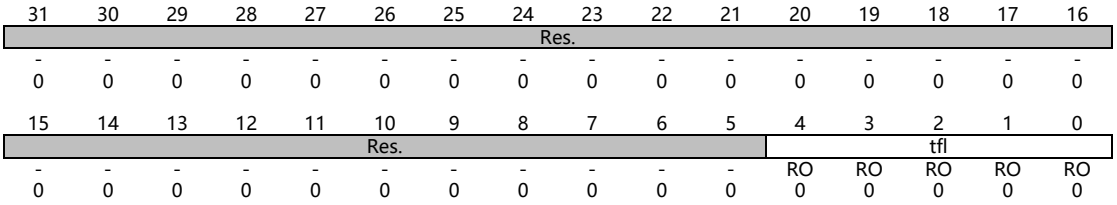
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.											rff	rfne	tfe	tfnf	busy
-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	1	1	0

Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
rff	[4]	RO	接收 FIFO 已满。 这用于指示 RX FIFO 已完全满。即：当 RX FIFO 不再满时，该位被清除。 0：接收 FIFO 未满 1：接收 FIFO 已满	0x0
rfne	[3]	RO	接收 FIFO 不为空。 这用于指示接收 FIFO 包含一个或多个条目。当 RX FIFO 为空时，该位被清除。 0：接收 FIFO 为空 1：接收 FIFO 不为空	0x0
tfe	[2]	RO	传输 FIFO 空。 这用于指示传输 FIFO 完全为空。当 TX FIFO 不再为空时，该位被清除。 0：传输 FIFO 不为空 1：传输 FIFO 为空	0x1
tfnf	[1]	RO	传输 FIFO 未满。	0x1

Name	Bits	R/W	Description	Default
			这用于指示传输 FIFO 未满。当 TX FIFO 满时，该位被清除。 0：传输 FIFO 已满 1：传输 FIFO 未满	
busy	[0]	RO	此位表示串行传输正在进行，清除时表示 UART 空闲或不活动。 0：uart 空闲或不活动 1：uart 忙（正在传输数据）	0x0

25.7.20 TFL

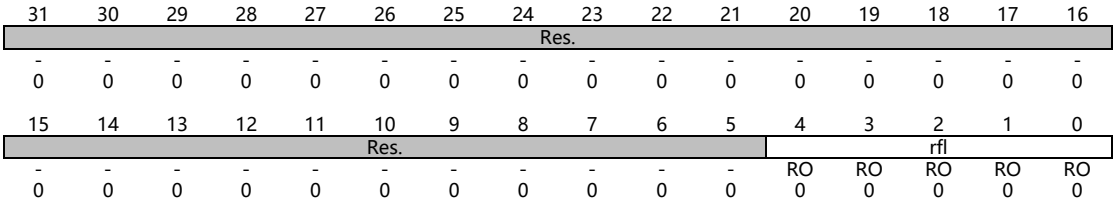
◆ **偏移地址：0x80　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
tfl	[4:0]	RO	指示 TX FIFO 中还有多少个数据，仅当 FIFO 启用时有效。	0x0

25.7.21 RFL

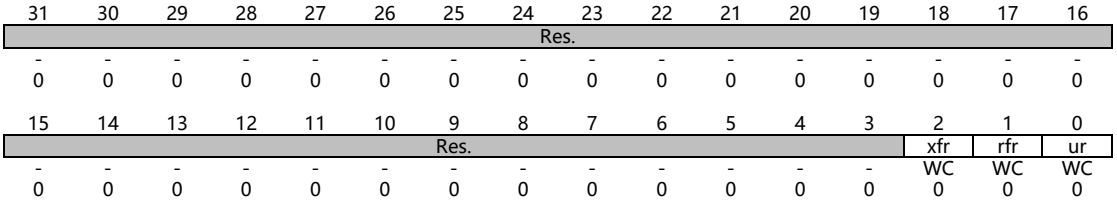
◆ **偏移地址：0x84　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
rfl	[4:0]	RO	指示 RX FIFO 中还有多少数据，仅当 FIFO 启用时有效。	0x0

25.7.22 SRR

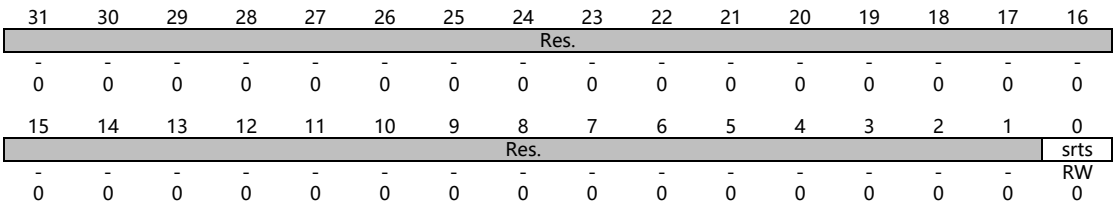
◆ **偏移地址：0x88　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
xfr	[2]	WC	TX FIFO 复位。 FCR.xfifor 的影子寄存器，此位是“自清除”，无需清除此位。	0x0
rfr	[1]	WC	RX FIFO 复位。 FCR.rfifor 的影子寄存器，此位是“自清除”，无需清除此位。	0x0
ur	[0]	WC	UART 复位。 复位 UART 所有逻辑，仅 MISC_CTRL 配置不会被复位。 0：不复位 1：复位	0x0

25.7.23 SRTS

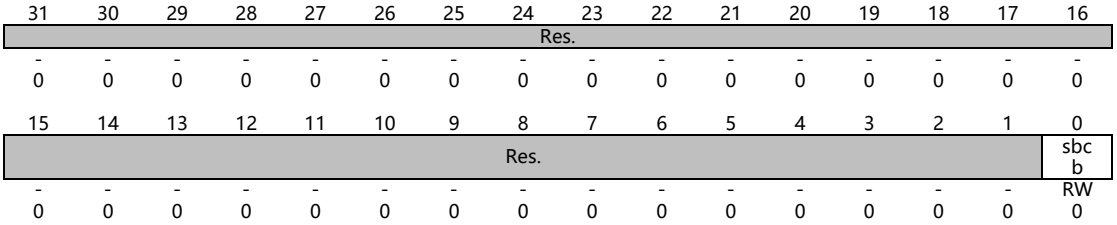
◆ **偏移地址：0x8c　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
srts	[0]	RW	这是 MCR.rts 的影子寄存器，控制该位等效于控制 MCR.rts。	0x0

25.7.24 SBCR

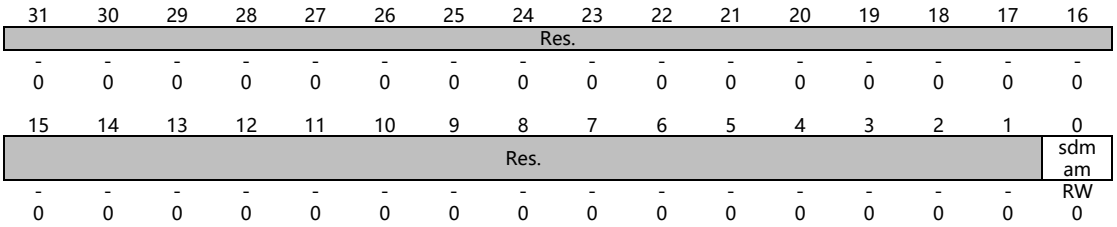
◆ **偏移地址：0x90　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
sbcb	[0]	RW	这是 LCR.bc 的影子寄存器，控制该位等效于控制 LCR.bc。	0x0

25.7.25 Reserved

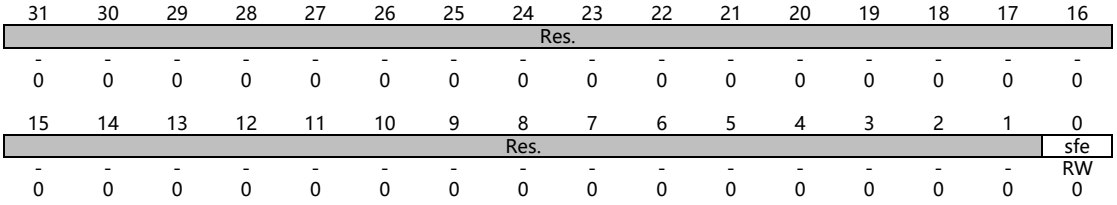
◆ **偏移地址：0x94　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:0]	-	保留。	0x0

25.7.26 SFE

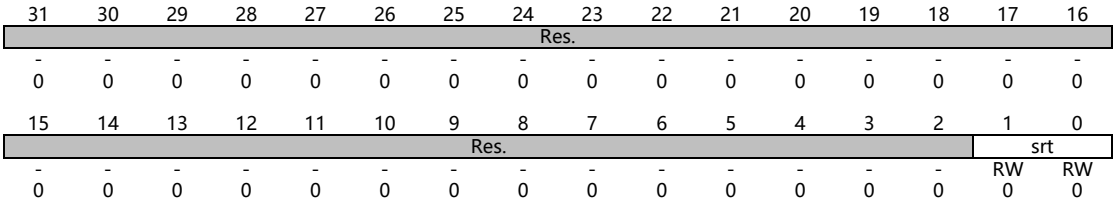
◆ **偏移地址：0x98　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
sfe	[0]	RW	FCR.fifoe 的影子寄存器，控制该位等效于控制 FCR.fifoe。	0x0

25.7.27 SRT

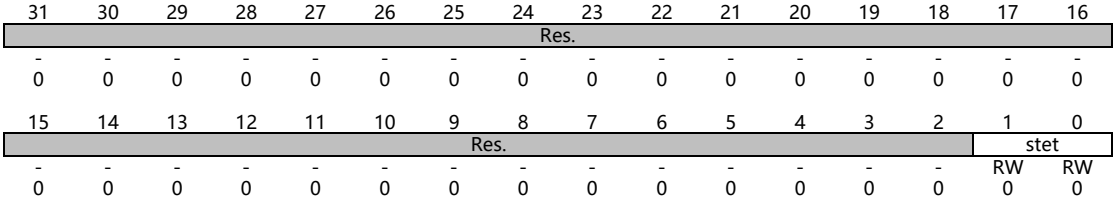
◆ **偏移地址：0x9c　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
srt	[1:0]	RW	FCR.rt 的影子寄存器，控制该位等效于控制 FCR.rt。	0x0

25.7.28 STET

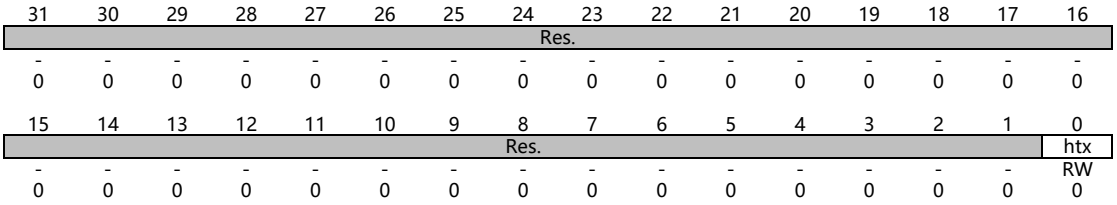
◆ **偏移地址：0xa0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
stet	[1:0]	RW	FCR.tet 的影子寄存器，控制该位等效于控制 FCR.tet。	0x0

25.7.29 HTX

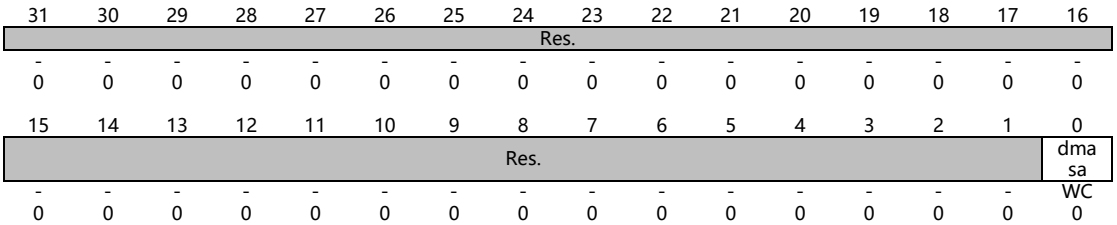
◆ **偏移地址：0xa4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
htx	[0]	RW	该寄存器用于暂停传输，用于测试。以便在 FIFO 启用时主设备可以填充传输 FIFO。 注意，如果 FIFO 未启用，暂停 TX 寄存器的设置将不会影响操作。 0：停止传输已禁用 1：已启用停止传输	0x0

25.7.30 DMASA

◆ **偏移地址：0xa8　　复位值：0x00000000**

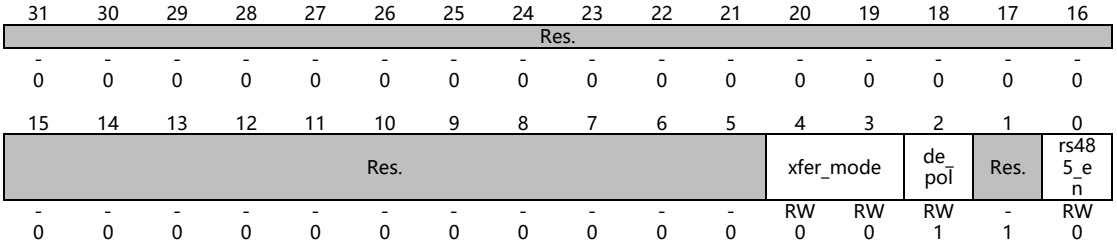


Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
dmasa	[0]	WC	DMA 软件确认。	0x0

Name	Bits	R/W	Description	Default
			如果由于错误情况需要终止传输，则使用此寄存器执行 DMA 软件确认。例如，如果 DMA 禁用信道，则 uart 应清除其请求。这将导致 TX 请求、TX 单个、RX 请求和 RX 单个信号被清除。注意，该位是“自清除”的，不需要清除该位。 1：DMA 软件确认	

25.7.31 TCR

◆ **偏移地址：0xac　　复位值：0x00000006**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
xfer_mode	[4:3]	RW	传输模式选择。 1：软件半双工模式。在此模式下，DE 信号和 RE 信号互斥，通过软件配置 DE_EN 或者 RE_EN 控制数据的接收或者发送；DE 到 RE 或者 RE 到 DE 的转换受到 TAT 寄存器中编程的周转时间影响。如果模块正在接收数据，则 DE 信号需要等待接收完成才能生效。 2：硬件半双工模式。此模式下，DE 信号和 RE 信号互斥，模块通过判断 TX_FIFO 的状态来决定 DE 和 RE 的状态。如果仅使能 DE_EN，则当 TX_FIFO 非空时 DE 有效，TX_FIFO 变空时 DE 失效；如果仅使能 RE_EN，则当 TX_FIFO 为空时 RE 有效，TX_FIFO 非空时 RE 失效；如果软件同时使能了 DE_EN 和 RE_EN，模块默认处于接收模式，即 RE 信号有效，一旦用户开始发送数据到 TX_ FIFO，则 DE 生效，RE 失效，当 TX_FIFO 变空时，模块自动切换到接收模式，即 DE 失效，RE 生效。DE 到 RE 或者 RE 到 DE 的转换受到 TAT 寄存器中编程的周转时间影响。 其他值：非法值。	0x0
de_pol	[2]	RW	Driver 使能极性。 0：DE 信号低有效	0x1

Name	Bits	R/W	Description	Default
			1：DE 信号高有效	
reserved	[1]	-	保留。	0x1
rs485_en	[0]	RW	RS485 传输启用。 0：在此模式下，传输仍处于 RS232 模式。此寄存器中的所有其他字段都被保留，寄存器 DE_EN/RE_EN/TAT 也被保留。 1：在此模式下，传输将在 RS485 模式下进行。本寄存器的所有其他字段均适用。	0x0

25.7.32 DE_EN

◆ **偏移地址：0xb0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															de_enable
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
de_enable	[0]	RW	用于控制输出的 de 信号 0：de 信号置位为无效 1：de 信号置位为有效	0x0

25.7.33 RE_EN

◆ **偏移地址：0xb4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

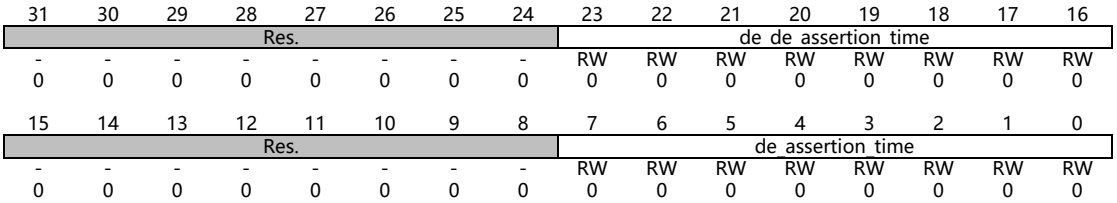
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															re_enable
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0

Name	Bits	R/W	Description	Default
re_enable	[0]	RW	用于控制输出的 re 信号。 0：re 信号置位为无效 1：re 信号置位为有效	0x0

25.7.34 DET

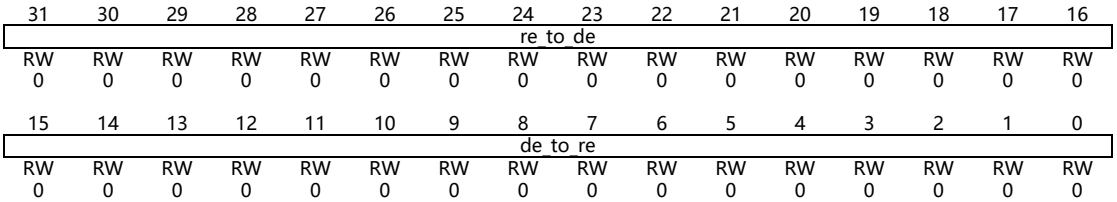
◆ **偏移地址：0xb8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
de_de_assertion_time	[23:16]	RW	driver 启用取消时间。 该字段控制从 sout 输出的停止位结束到驱动器输出使能信号的下降沿之间的时间量（以串行时钟周期的数量表示）。	0x0
reserved	[15:8]	-	保留。	0x0
de_assertion_time	[7:0]	RW	driver 启用置位有效时间。 该字段控制驱动器输出使能信号的上升到串行传输使能之间的时间量（以串行时钟周期的数量表示）。传输缓冲器中的任何数据将在传输使能后从串行输出（sout）开始传输。	0x0

25.7.35 TAT

◆ **偏移地址：0xbc　　复位值：0x00000000**

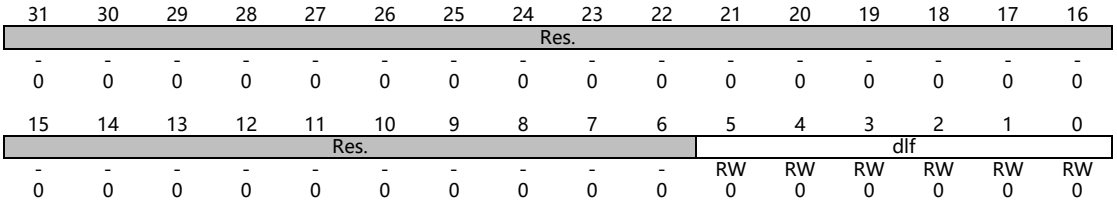


Name	Bits	R/W	Description	Default
re_to_de	[31:16]	RW	RE 无效到 DE 有效的周转时间。	0x0

Name	Bits	R/W	Description	Default
de_to_re	[15:0]	RW	DE 无效到 RE 有效的周转时间。	0x0

25.7.36 DLF

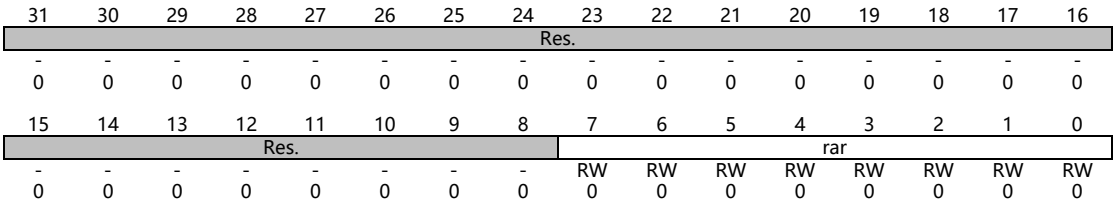
◆ **偏移地址：0xc0　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留。	0x0
dlf	[5:0]	RW	配置波特率小数分频参数，代表的小数值为（DLF.dlf/64）。	0x0

25.7.37 RAR

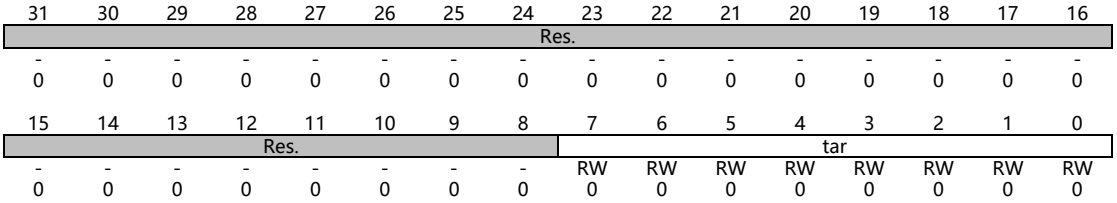
◆ **偏移地址：0xc4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
rar	[7:0]	RW	这是接收模式期间的地址匹配寄存器。如果在输入帧中第 9 位为 1，则将对照该寄存器值检查剩余的 8 位。如果匹配发生，则第 9 位设置为 0 的后续字符将被视为数据字节，直到接收到下一个地址字节。 注意：仅当 LCR_EXT.addr_match 和 LCR_EXT.dls_e 位设置为 1 时，此寄存器才适用。应仅在 UART 不忙时对 RAR 进行编程。当任何接收正在进行时，用户不得更改此寄存器值。	0x0

25.7.38 TAR

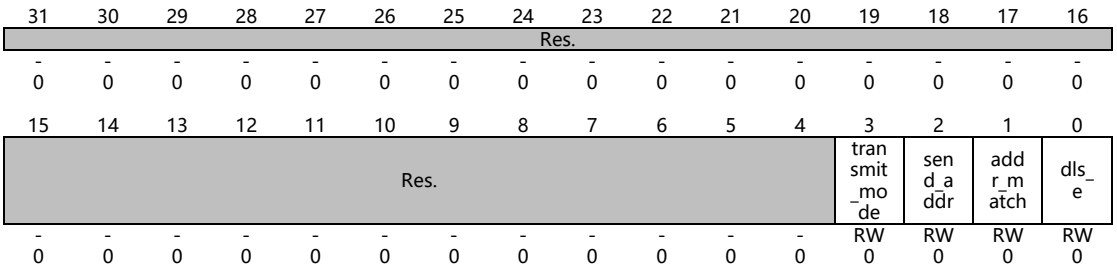
◆ **偏移地址：0xc8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
tar	[7:0]	RW	这是传输模式期间的地址匹配寄存器。如果启用了 LCR_EXT.dls_e 位，LCR_TEXT.send_addr 位设置为 1，则 uart 将发送帧第 9 位设置为 1 的 9 位帧，剩余的 8 位地址将从此寄存器发送。 注意：此寄存器仅用于发送地址。正常数据应通过编程 THR 寄存器发送。一旦开始在 uart 串行通道上发送地址，硬件将自动清除 LCR_TEXT.send_addr 位。	0x0

25.7.39 LCR_EXT

◆ **偏移地址：0xcc　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
transmit_mode	[3]	RW	传输模式控制位。该位用于在 9 位数据传输期间控制传输模式的类型。 1：在此操作模式下，传输保持寄存器（THR）和影子传输保持寄存器的宽度为 9 位。用户需要确保 THR/STHR 寄存器正确写入地址/数据。 地址：第 9 位设置为 1， 数据：第 9 位设置为 0。	0x0

Name	Bits	R/W	Description	Default
			注意：传输地址寄存器（TAR）不适用于此操作模式。 0：在此操作模式下，传输保持寄存器（THR）和影子传输保持寄存器的宽度为 8 位。用户需要将地址编程到传输地址寄存器（TAR）中，并将数据编程到 THR/STHR 寄存器中。SEND_ADDR 位用作控制旋钮，以指示何时打开 uart 发送地址帧。	
send_addr	[2]	RW	发送地址控制位。该位用作控制旋钮，供用户确定在传输模式期间何时发送地址。 1：9 位字符将被传输，第 9 位设置为 1，其余 8 位将与“传输地址寄存器”中正在编程的内容相匹配。 0：9 位字符将在第 9 位设置为 0 的情况下发送，其余 8 位将从 TXFIFO 中取出，TXFIFO 通过 8 位宽 THR/STHR 寄存器编程。 注意：发送地址字符后，硬件自动清除该位。用户不应将此位编程为 0。此字段仅在 DLS_E 位设置为 1 且 TRANSMIT_MODE 设置为 0 时适用。	0x0
addr_match	[1]	RW	在接收期间自动地址匹配功能。 1：地址匹配模式； uart 将等待第 9 位设置为 1 的传入帧。并进一步检查地址是否与“接收地址匹配寄存器”中编程的地址匹配。如果找到匹配，则后续帧将被视为有效数据， uart 将开始接收数据。 0：正常模式； uart 将开始接收数据，9 位字符将被形成并写入接收 RXFIFO。用户负责读取数据并区分地址和数据。 注意：此字段仅在 DLS_E 设置为 1 时适用。	0x0
dls_e	[0]	RW	DLS 域的扩展，用于开启 9bit 帧格式。 0：不开启 9bit 模式 1：开启 9bit 模式	0x0

26 控制器局域网总线（CAN）

26.1 CAN 简介

控制器局域网（CAN）为串行通讯协议，能有效地支持具有很高安全等级的分布实时控制。CAN 的应用范围很广，从高速网络到低成本的多路布线都可以使用 CAN。在汽车电子产品中，发动机控制单元、传感器、防滑系统等均使用 CAN 连接，其传输速度高达 1Mbps。

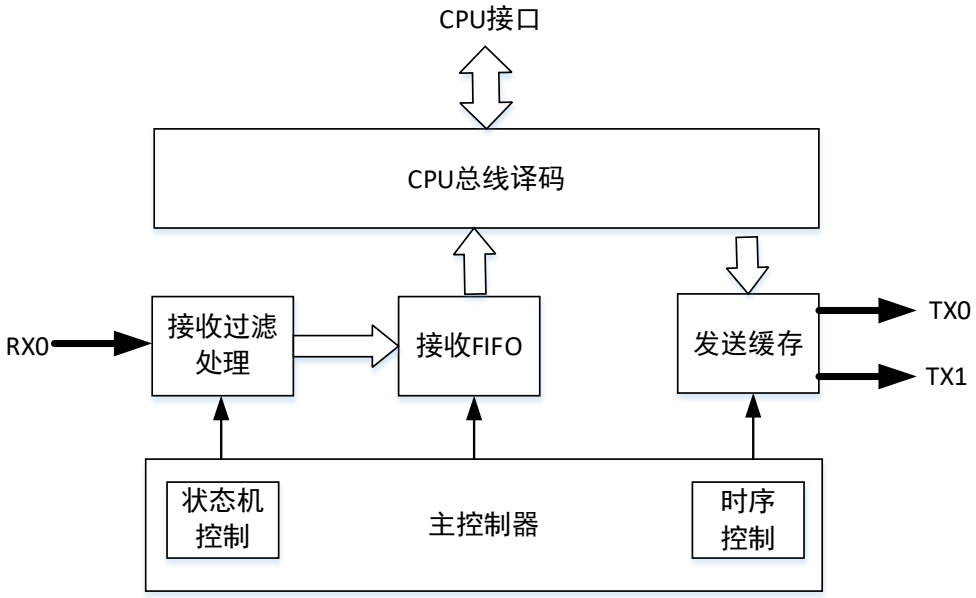
26.2 CAN 主要特性

- 支持 CAN 协议 2.0A 和 2.0B
- 支持 64 字节接收 FIFO
- 支持多个可编程的接收标识及掩码
- FIFO 溢出上报
- 自动重传功能可禁用
- 中断可屏蔽
- 支持侦听和自测试模式（listen-only/self-test）

26.3 CAN 逻辑实现

CAN 逻辑实现框图如下图 26-1 所示。

图 26-1 CAN 模块框图



CPU 可以通过独立的地址、数据总线访问 CAN 控制器。

发送数据时，CPU 将数据写进 transmit buffer 并交给 BIT PROCESSOR 模块处理。接收数据时，先由 Acceptance Filter 模块对数据进行过滤，然后再将数据存入 Receive FIFO。

Receive FIFO 会将数据写入偏移地址为 0x40~0x70 的寄存器，CPU 再通过这些寄存器读取数据。Receive FIFO 最多支持 5 个扩展帧。根据系统工作时钟 XTAL1（APB_LS），波特率最低支持 125K，最高支持 1M。

通过 Bit Timing Logic 模块可以配置不同波特率。

26.4 CAN 功能描述

26.4.1 工作模式

CAN 有 2 个工作状态：复位状态和运行状态。运行状态下支持 4 种工作模式：自测试模式、侦听模式、发送模式、接收模式。复位状态下，可以设置总线时序参数、过滤 ID、错误警告阈值等。可以通过硬复位或者把模式寄存器 MOD.reset_mode 置 1 进入复位状态，把模式寄存器 MOD.reset_mode 置 0 则退出复位状态回到运行状态。运行状态下可以进行正常的收发操作。在进入工作模式前，CAN 必须跟 CAN 总线取得同步；为取得同

步，CAN 要等待 CAN 总线达到空闲状态，即在 CANRX 引脚上监测到 11 个连续的隐性位。

1. 自测试模式

自测试模式是一种特殊模式，该模式下，数据在 CAN 控制器内部环回，不需要连接其它 CAN 节点，因此也不需要其它节点响应 ACK。使用自测试模式时，先把数据写入发送 buffer，然后配置模式寄存器 MOD.self_test_mode 为 1，把 CAN 配置为自测试模式，再配置 CMR.self_reception_request 为 1，即发起自接收请求，此时数据将在 CAN 控制器内部环回。

2. 侦听模式

配置模式寄存器 MOD.listen_only_mode 为 1，可将 CAN 配置为侦听模式。在侦听模式下，CAN 控制器只能接收数据不能发送数据，也不能发送应答信号。侦听模式可以用于波特率检测。启用该功能时需要使能总线错误中断和接收中断，并用软件制作一个波特率表， CAN 控制器遍历表中每个波特率时都会产生中断，当产生总线错误中断时，说明当前的波特率与波特率表中选择不一致，继续遍历下一个波特率；当能收到数据并且产生接收中断时，说明波特率与波特率表中选择一致，即完成了波特率检测功能。

3. 发送模式

发送模式即把待发送的数据以标准帧格式（SFF）或扩展帧格式（EFF）写入发送 buffer 并通过 CAN 总线发送出去。发送 buffer 包括 CAN 寄存器地址 0x40 到 0x70 之间的 13 个寄存器，这段地址空间也称为发送缓冲区，最大支持 8 个字节的数据帧

⚠注意

在将数据写入 buffer 前，需要检查发送 buffer 状态（SR.transmit_buffer_status），当 buffer 已释放时（SR.transmit_buffer_status=1），数据可以写入 buffer；当 buffer 被锁定时（SR.transmit_buffer_status=0），数据无法写入 buffer。

发送数据时，先把数据写入发送 buffer，再设置命令寄存器 CMR.transmission_request=1 发出发送请求，此时数据将通过 CAN 总线发送出去。发送开始时，发送状态位 SR. transmit_status 变为 1，并且发送请求位 CMR.transmission_request 被自动清除。

如果在发送消息时总线仲裁丢失或发生传输错误，则 CAN 控制器将自动重传信息。

15 位循环冗余校验（CRC）随每个帧一起发送，该 CRC 根据帧起始位、仲裁、控制和数据字段生成的。

在消息发送未开始时，可以通过配置命令寄存器 CMR.abort_transmission=1 来中止消息发送；当消息开始发送后，配置 CMR.abort_transmission=1 不能中止当前消息帧的发

送，当前消息帧发送完后才会中止发送。

⚠注意

要查看原始消息的发送状态，请等待发送 buffer 状态位 SR.transmit_buffer_status 设置为 1 或生成发送中断，然后检查发送完成状态位 SR.transmission_complete_status。即使由于发送 buffer 状态位变为释放而中止消息，也会生成发送中断。

4. 接收模式

CAN 控制器接收到的数据首先由接收过滤器（Acceptance Filter）过滤，然后才能传递到接收 FIFO。接收过滤器只接收那些与过滤 ID 寄存器中配置的标识符位相匹配的数据。

当 CAN 控制器在接收数据时，状态寄存器 SR 中的接收状态位 SR.receive_status 将变为 1。一旦数据存入到接收 FIFO 时，接收缓冲区状态位 SR.receive_buffer_status 将变为 1，如果对应的中断使能打开，则生成接收中断。

接收 FIFO 深度为 64 字节，最多可容纳 5 个完整的扩展帧格式（EFF）消息，并以循环方式使用。如果 FIFO 中没有足够的空间用于接收数据，则状态寄存器 SR 中的数据溢出状态位 SR.data_overrun_status 将置 1，并丢弃正在接收的数据帧，如果数据溢出中断使能打开，还会生成数据溢出中断。

接收的数据存放在接收 FIFO 中，接收 FIFO 会将数据写入寄存器地址 0x40~0x70，CPU 通过这段地址读取接收的数据，这段地址空间也称为接收缓冲区，它占用与发送缓冲区相同的地址空间，与发送缓冲区一样，最大支持 8 个字节的数据帧。

当主机 CPU 通过寄存器地址 0x40~0x70 读取接收数据时，需要通过配置命令寄存器 CMR.release_receive_buffer 为 1 发出释放接收缓冲命令，此时如果接收 FIFO 中还有数据未读取，接收 FIFO 会再次将数据写入寄存器地址 0x40~0x70，直到接收 FIFO 中的数据都读取完。如果发出释放接收缓冲区命令后，接收 FIFO 中没有可读取的数据，接收状态位以及相应的中断位会清零。

26.4.2 自接收功能

CAN 的自接收功能是指，CAN 控制器可以接收其自身正在向给其它 CAN 节点发送的消息，同时并不影响其它 CAN 节点接收该消息。自接收功能需要其他 CAN 节点发出 ACK 信号才能完成，其自身接收到数据不会产生 ACK 信号。使用自接收功能时，先把数据写入发送 buffer，再通过配置命令寄存器 CMR.self_reception_request 为 1 来发出自接收请求，此时 CAN 控制器可以接收到当前正在发送给其它 CAN 节点的消息。使用该功能，CAN 控制器可以正常使用发送和接收中断。

⚠注意

如果在正常发送的同时发出自接收请求（即同时配置 CMR.transmission_request 和 CMR.self_reception_request 为 1），则会忽略自接收请求。

CAN 控制器的自测试模式提供了自收发功能的实现，区别在于自测试模式下不需要其他节点应答，而自接收功能需要其他节点的应答；另外使用自测试模式时需要配置 MOD.self_test_mode 为 1（即使能自测试模式），而使用自接收功能时需要配置 MOD.self_test_mode 为 0（即正常模式）。

26.4.3 消息过滤

在 CAN 网络中，所有节点都能接收总线上传输的信息。为了允许节点忽略与其无关的消息，CAN 控制器对接收到的消息应用 4 字节的接收过滤器（Acceptance Filter），只有与过滤器匹配的标识符位的消息才会被接收。

CAN 控制器有 4 个接收过滤 ID 寄存器 ACR0~3 以及 4 个接收过滤 ID 掩码寄存器 AMR0~3，这两组寄存器构成两种滤波模式：单过滤 ID 模式和双过滤 ID 模式。

配置模式寄存器 MOD.acceptance_filter_mode=1，表示配置为单过滤 ID 模式，此时 4 个接收过滤 ID 寄存器 ACR0~3 组成单个 32 位过滤器，应用于每个消息的前 4 个字节，只有前 4 个字节与过滤器匹配时，消息才能被接收。可以通过将接收过滤 ID 掩码寄存器 AMR0~3 的特定 bit 设置为 1 表示对应的标识符位不做检查过滤。

配置模式寄存器 MOD.acceptance_filter_mode=0，表示配置为双过滤 ID 模式，此时 4 个接收过滤 ID 寄存器 ACR0~3 组成 2 个单独的过滤器，应用到每个消息的前 2 个或前 3 个字节，只要消息的前 2 个或前 3 个字节与其中一个过滤器匹配时，消息就能被接收。可以通过将接收过滤 ID 掩码寄存器 AMR0~3 的特定 bit 设置为 1 表示对应的标识符位不做检查过滤。

消息过滤基于整个标识符，根据接收到的消息是标准帧格式还是扩展帧格式，标识符可以是 11 或 29 位长。AMR0~3 和 ACR0~3 一起决定过滤机制，根据是否标准帧格式以及接收过滤模式，ACR 和 AMR 共有四种组合方式，如下所示：

第 1 种：标准帧格式单过滤 ID 模式，如下图 26-2 所示。此模式下接收到的消息 ID、RTR、DataByte1、DataByte2 的每 1bit 都有 1bit 对应的 AMR 掩码，AMR 掩码为 1 的 bit 不做检查，为 0 要检查过滤，需要检查过滤的 bit 与 ACR 一致即表示匹配。

图 26-2 标准帧格式单过滤 ID 模式

第1种：标准帧格式单过滤ID模式
接收buffer

ID28~ID21	ID20~ID18	RTR	XXXX (not	Data Byte 1	Data Byte 2

过滤：

ACR0[7:0]	ACR1[7:4]	ACR1[3:0] (unused)	ACR2[7:0]	ACR3[7:0]
AMR0[7:0]	AMR1[7:4]	AMR1[3:0] (unused)	AMR2[7:0]	AMR3[7:0]

第 2 种：标准帧格式双过滤 ID 模式，如下图 26-3 所示。此模式下有 2 个过滤器，消息只需要匹配其中一个过滤器即可，匹配的机制同上。

图 26-3 标准帧格式双过滤 ID 模式

第2种：标准帧格式双过滤ID模式。
接收buffer

ID28~ID21	ID20~ID18	RTR	XXXX (not	Data Byte 1[7:4]	Data Byte 1[3:0]	Data Byte 2 (not matched)

过滤1：

ACR0[7:0]	ACR1[7:4]	ACR1[3:0]	ACR3[3:0]
AMR0[7:0]	AMR1[7:4]	AMR1[3:0]	AMR3[3:0]

过滤2：

ACR2[7:0]	ACR3[7:4]
AMR2[7:0]	AMR3[7:4]

第 3 种：扩展帧格式单过滤 ID 模式，如下图 26-4 所示，其匹配的机制同上。

图 26-4 扩展帧格式单过滤 ID 模式

第3种：扩展帧格式单过滤ID模式
接收buffer

ID28~ID21	ID20~ID13	ID12~ID5	ID4~ID0	RTR	XX (not

过滤：

ACR0[7:0]	ACR1[7:0]	ACR2[7:0]	ACR3[7:4]	ACR3[1:0] (unused)
AMR0[7:0]	AMR1[7:0]	AMR2[7:0]	AMR3[7:4]	AMR3[1:0] (unused)

第 4 种：扩展帧格式双过滤 ID 模式，如下图 26-5 所示，其匹配的机制同上。

图 26-5 扩展帧格式双过滤 ID 模式

第4种：扩展帧格式双过滤ID模式

接收buffer

ID28~ID21	ID20~ID13	ID12~ID5 (not matched)	ID4~ID0 (not	RTR (not	XX (not

过滤1：

ACR0[7:0]	ACR1[7:0]
AMR0[7:0]	AMR1[7:0]

过滤2：

ACR2[7:0]	ACR3[7:0]
AMR3[7:0]	AMR3[7:0]

26.4.4 波特率和采样设置

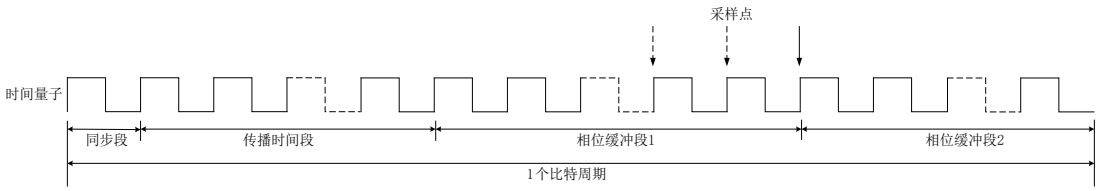
CAN 的波特率和采样参数通过两个总线定时寄存器 BTR0 和 BRT1 设置，其中 BTR0 用于配置同步补偿所需的 TQ 数量和波特率分频系数，BTR1 用于设置采样率和采样点位置，BTR0 和 BTR1 寄存器的详细信息见寄存器描述章节 26.6.7 BTR0 和 26.6.8 BTR1。

CAN 使用的波特率是通过 TQ 的数量来指定的。CAN 控制器允许定义从 3 到 25TQ 的比特周期，在实践中使用的比特周期可能遵循 BOSCH 标准，该标准定义了长度在 8 到 25TQ 之间的比特周期。这里 TQ（Time Quanta）称之为“时间量子”，是 CAN 总线的最小时间单位，波特率预分频器将 TQ 定义为 XTAL1输入时钟周期的倍数，其长度根据系统工作时钟 XTAL1 的周期来定义：

$TQ = 2 \times t_{clk} \times (BRP+1)$

其中 BRP 为寄存器 BTR0 的 bit【5:0】，t_{clk} 为 CAN 控制器工作时钟 XTAL1 的周期。

图 26-6 CAN 的比特周期结构图



CAN 的 1 个比特周期分为 4 段，如上图 26-6 所示：同步段(SS)、传播时间段(PTS)、相位缓冲段 1(PBS1)和相位缓冲段 2(PBS2)，其中同步段(SS)时间为 $t_{syncseg}$，固定为 1 个 TQ 长度，传播时间段(PTS)+相位缓冲段 1(PBS1)时间为 t_{tseg1}，相位缓冲段 2(PBS2)时间为 t_{tseg2}。这些段由 TQ 组成：

$t_{syncseg} = TQ \times 1;$

t_{tseg1} = TQ x (TSEG1+1);

t_{tseg2} = TQ x (TSEG2+1);

其中，TSEG1 为寄存器 BTR1 的 bit【3:0】，TSEG2 为寄存器 BTR1 的 bit【6:4】。

26.4.5 总线仲裁

CAN 总线受 CAN 2.0 技术规范中的规则控制。

当有多个 CAN 节点同时准备发送消息时，需要进行仲裁，标志符最低的节点将获得总线的控制权。失去仲裁的 CAN 节点自动退出，并且在 CAN 总线空闲之前不得再次尝试控制 CAN 总线。当包含 CAN 控制器的设备失去仲裁时，如果对应的中断使能打开，将生成仲裁丢失中断，并且仲裁丢失的位置将记录在仲裁失败捕获寄存器 ALC 中。具体请参阅 26.6.12 ALC 寄存器描述。

26.4.6 错误处理

CAN 控制器包括两个错误计数器，一个用于接收错误计数（RXERR），一个用于发送错误计数（TXERR）。当发生错误时，它们会自动递增，成功的接收和发送则递减计数器。此外，错误类型和错误发生的位置会记录在错误状态寄存器 ECC 中，它们一直保留到读取该寄存器。

CAN 控制器还包括一个可配置的错误警告阈值寄存器（EWL），该寄存器的默认值为 96，当发送错误计数器或接收错误计数器超过该值时，状态寄存器中的错误状态位（SR.error_status）被置 1，如果对应的中断使能打开，则生成错误警告中断。

当任意一个错误计数器的计数值超过 127 时，CAN 控制器将进入“错误被动”状态（由 CAN 协议定义），如果对应的中断使能打开，则发出错误被动中断。

如果发送错误计数器的计数值超过 255（计数器的极限），总线状态位（SR.bus_status）将设置为 1（总线关闭），CAN 控制器将设置为复位状态，如果对应的中断使能打开，则生成错误警告中断（EI）。然后，CAN 控制器将保持在复位状态，直到主机 CPU 清除模式寄存器（MOD.reset_mode）中的复位状态位。此外，在返回运行状态时，CAN 控制器将等待 128 次总线空闲序列（CAN 协议定义的最短时间），然后再次变为“总线开启”。

26.5 CAN 寄存器概览

CAN 寄存器基地址如下：

- CAN 寄存器基地址为：0x60087000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	MOD	0x0	32	RW	0x00000001	模式寄存器
2	CMR	0x4	32	RW	0x00000000	命令寄存器
3	SR	0x8	32	RO	0x0000003C	状态寄存器
4	IR	0xC	32	RO	0x00000000	中断状态寄存器
5	IER	0x10	32	RW	0x00000000	中断使能寄存器
6	Reserved	0x14	32	RO	0x00000000	保留寄存器
7	BTR0	0x18	32	RW	0x00000000	总线时序寄存器 0
8	BTR1	0x1C	32	RW	0x00000000	总线时序寄存器 1
9	OCR	0x20	32	RW	0x00000000	输出控制寄存器
10	Reserved	0x24	32	RO	0x00000000	保留寄存器
11	Reserved	0x28	32	RO	0x00000000	保留寄存器
12	ALC	0x2C	32	RO	0x00000000	仲裁失败捕获寄存器
13	ECC	0x30	32	RO	0x00000000	错误状态寄存器
14	EWLR	0x34	32	RW	0x00000060	错误警告阈值寄存器
15	RXERR	0x38	32	RO	0x00000000	接收错误统计寄存器
16	TXERR	0x3C	32	RO	0x00000000	发送错误统计寄存器
17	FRAME_INFO_ACR0	0x40	32	RO	0x00000000	帧信息寄存器
18	ID1_ACR1	0x44	32	RO	0x00000000	ID1 寄存器
19	ID2_ACR2	0x48	32	RO	0x00000000	ID2 寄存器
20	ID3_ACR3	0x4C	32	RO	0x00000000	ID3 寄存器
21	ID4_AMR0	0x50	32	RO	0x00000000	ID4 寄存器
22	Data1_AMR1	0x54	32	RO	0x00000000	数据寄存器 1
23	Data2_AMR2	0x58	32	RO	0x00000000	数据寄存器 2
24	Data3_AMR3	0x5C	32	RO	0x00000000	数据寄存器 3
25	Data4	0x60	32	RO	0x00000000	数据寄存器 4
26	Data5	0x64	32	RO	0x00000000	数据寄存器 5
27	Data6	0x68	32	RO	0x00000000	数据寄存器 6
28	Data7	0x6C	32	RO	0x00000000	数据寄存器 7
29	Data8	0x70	32	RO	0x00000000	数据寄存器 8
30	RMC	0x74	32	RO	0x00000000	接收信息统计寄存器
31	RBSA	0x78	32	RW	0x00000000	接收 buffer 起始地址寄存器
32	CDR	0x7C	32	RW	0x000000C0	时钟分频寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
33	RX_FIFO	0x80+0x4*n	32	RO	0x00000000	接收数据读取寄存器（n 取值 0~63）
34	TX_BUFF	0x180+0x4*m	32	RO	0x00000000	发送数据读取寄存器（m 取值 0~12）

26.6 CAN 寄存器描述

26.6.1 MOD

◆ **偏移地址：0x0　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												acceptance_filter_mode	self_test_mode	listen_only_mode	reset_mode
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
acceptance_filter_mode	[3]	RW	接收过滤模式设置。 0：双过滤 ID 模式，使用 2 个短的过滤 ID 1：单过滤 ID 模式，使用 4 字节过滤 ID	0x0
self_test_mode	[2]	RW	自测试模式设置。 0：正常模式，需要应答信号才能完成发送 1：自测试使能，该模式下不需要其他节点，也不需要应答即可完成发送	0x0
listen_only_mode	[1]	RW	侦听模式设置。 0：正常模式 1：侦听使能，该模式无须发送应答即使成功接收到信息	0x0
reset_mode	[0]	RW	复位状态设置。 0：运行状态 1：进入复位状态，退出收发操作	0x1

26.6.2 CMR

◆ **偏移地址：0x4　　复位值：0x00000000**

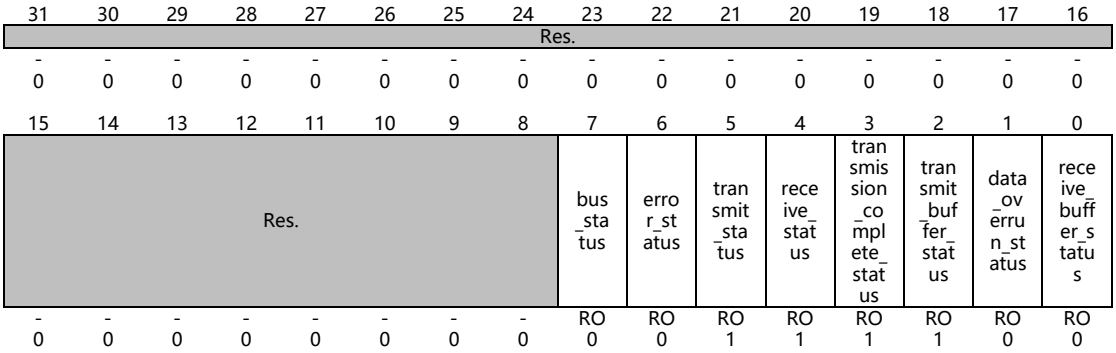
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.											self_reception_request	clear_data_overrun	release_receive_buffer	abort_transmission	transmission_request
-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
self_reception_request	[4]	RW	自接收请求设置。 0：正常收发； 1：表示自发自收。	0x0
clear_data_overrun	[3]	RW	溢出状态清零设置。 0：不清零； 1：清零	0x0
release_receive_buffer	[2]	RW	释放接收 buffer。 0：不释放接收 buffer，接收的数据仍保留在接收 FIFO 中； 1：释放接收 buffer，将接收 FIFO 的数据写入寄存器地址 0x40~0x70。	0x0
abort_transmission	[1]	RW	退出发送设置。 0：不退出发送； 1：退出发送。	0x0
transmission_request	[0]	RW	发送请求设置。 0：无发送请求； 1：发送请求。 注：有信息要发送时设置为 1，发送完成后将自动清零	0x0

26.6.3 SR

◆ **偏移地址：0x8　　复位值：0x0000003C**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
bus_status	[7]	RO	总线状态。 0：处于正常状态，参与 CAN 总线收发 1：处于离线状态，不参与 CAN 总线收发	0x0
error_status	[6]	RO	错误状态。 0：错误统计（TX/RX）都低于警告阈值 1：至少有 1 个错误统计（TX/RX）达到或超出设置的错误警告阈值	0x0
transmit_status	[5]	RO	发送状态 0：没有信息发送 1：正在发送信息	0x1
receive_status	[4]	RO	接收状态。 0：没有信息接收 1：正在接收信息	0x1
transmission_complete_status	[3]	RO	发送完成状态。 0：上个发送请求未完成 1：上个发送请求已完成	0x1
transmit_buffer_status	[2]	RO	发送 buffer 状态。 0：已锁住，CPU 不能访问 buffer，因为有数据在等待发送或者正在发送过程中 1：已释放，CPU 可以写数据进 buffer	0x1
data_overrun_status	[1]	RO	数据溢出状态。 0：数据没有溢出 1：数据溢出并丢失因为接收 FIFO 没有足够的空间接收数据	0x0
receive_buffer_status	[0]	RO	接收 buffer 状态。 0：buffer 已空，接收 FIFO 没有数据可读 1：buffer 非空，接收 FIFO 有一帧或者多帧数据可读取	0x0

26.6.4 IR

◆ **偏移地址：0xC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								bus_error_interrupt	arbitration_lost_interrupt	error_passive_interrupt	Res.	data_overrun_interrupt	error_warning_interrupt	transmit_interrupt	receive_interrupt
-	-	-	-	-	-	-	-	RO	RO	RO	-	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
bus_error_interrupt	[7]	RO	总线错误中断。当 BEIE 中断使能打开并且检测到总线错误时置 1。	0x0
arbitration_lost_interrupt	[6]	RO	仲裁失败中断。当 ALIE 中断使能打开并且仲裁失败时置 1。	0x0
error_passive_interrupt	[5]	RO	被动中断。当 EPIE 中断使能打开并且错误状态从主动错误状态切换到被动状态或者 TX/RX 错误统计至少有一个超过协议定义的水平（即 127）时置 1。	0x0
reserved	[4]	-	保留。	0x0
data_overrun_interrupt	[3]	RO	数据溢出中断。当 DOIE 中断使能打开并且数据溢出状态从 0 变为 1 时置 1。	0x0
error_warning_interrupt	[2]	RO	错误警告中断。当 EIE 中断使能打开并且总线状态 BS 或者错误状态 ES 变化时置 1。	0x0
transmit_interrupt	[1]	RO	发送中断。当 TIE 中断使能打开并且发送 buffer 状态从 0 变为 1（也即发送 buffer 释放）时置 1。	0x0
receive_interrupt	[0]	RO	接收中断。当 RIE 中断使能打开并且接收 FIFO 包含一个或多个帧数据时置 1。当没有数据可读时，也即 RRB 接收 buffer 释放时清零。	0x0

26.6.5 IER

◆ **偏移地址：0x10　　复位值：0x00000000**

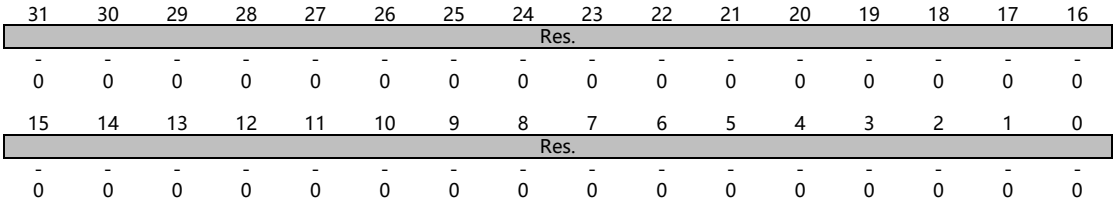
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								bus_error_interrupt_enable	arbitration_lost_interrupt_enable	error_passive_interrupt_enable	Res.	data_overrun_interrupt_enable	error_warning_interrupt_enable	transmit_interrupt_enable	receive_interrupt_enable
-	-	-	-	-	-	-	-	RW	RW	RW	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
bus_error_interrupt_enable	[7]	RW	总线错误中断使能。 0：中断不使能 1：中断使能	0x0
arbitration_lost_interrupt_enable	[6]	RW	仲裁失败中断使能。 0：中断不使能 1：中断使能	0x0
error_passive_interrupt_enable	[5]	RW	被动中断使能。 0：中断不使能 1：中断使能	0x0
reserved	[4]	-	保留。	0x0
data_overrun_interrupt_enable	[3]	RW	数据溢出中断使能。 0：中断不使能 1：中断使能	0x0
error_warning_interrupt_enable	[2]	RW	错误警告中断使能。 0：中断不使能 1：中断使能	0x0
transmit_interrupt_enable	[1]	RW	发送中断使能。 0：中断不使能 1：中断使能	0x0
receive_interrupt_enable	[0]	RW	接收中断使能。 0：中断不使能 1：中断使能	0x0

26.6.6 Reserved

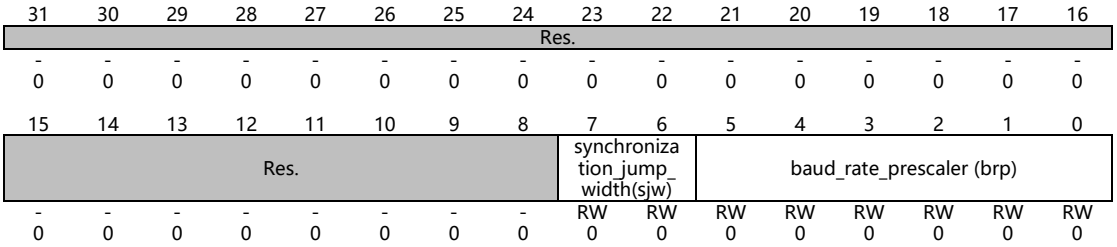
◆ **偏移地址：0x14　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:0]	-	保留。	0x0

26.6.7 BTR0

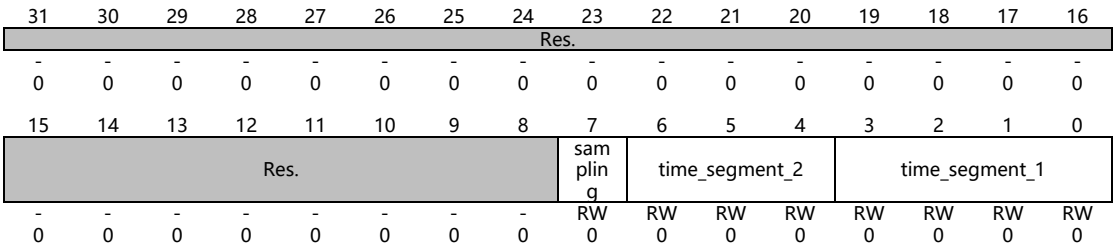
◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
synchronization_jump_width(sjw)	[7:6]	RW	同步补偿宽度，以 TQ 为单位。	0x0
baud_rate_prescaler (brp)	[5:0]	RW	波特率分频系数。	0x0

26.6.8 BTR1

◆ **偏移地址：0x1C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
sampling	[7]	RW	采样率设置。 0：表示正常采样 1：表示 3 倍过采样，推荐低速时使用	0x0
time_segment_2	[6:4]	RW	比特位时间段 2 的长度，以 TQ 为单位。	0x0
time_segment_1	[3:0]	RW	比特位时间段 1 的长度，以 TQ 为单位。	0x0

26.6.9 OCR

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.														output_control_mode	
-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
output_control_mode	[1:0]	RW	输出控制模式。 2：正常输出模式 3：时钟输出模式 其他值：保留	0x0

26.6.10 Reserved

◆ **偏移地址：0x24　　复位值：0x00000000**

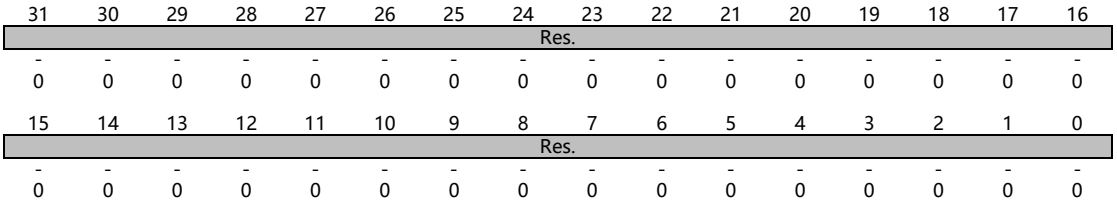
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:0]	-	保留。	0x0

26.6.11 Reserved

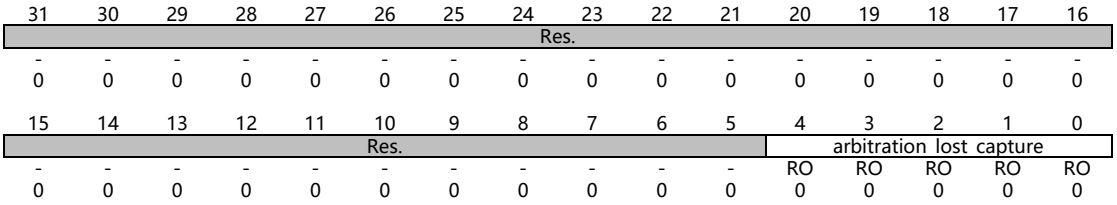
◆ **偏移地址：0x28　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:0]	-	保留。	0x0

26.6.12 ALC

◆ **偏移地址：0x2C　　复位值：0x00000000**

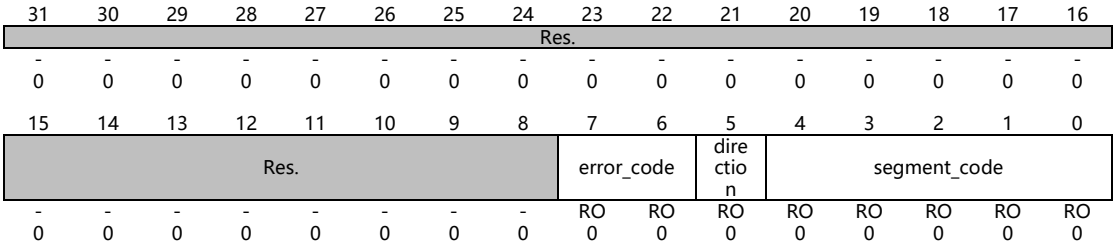


Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
arbitration_lost_capture	[4:0]	RO	仲裁失败说明。 0 ： 在标志符的第 1 位仲裁失败(ID.28) 1 ： 在标志符的第 2 位仲裁失败(ID.27) 2 ： 在标志符的第 3 位仲裁失败(ID.26) 3 ： 在标志符的第 4 位仲裁失败(ID.25) 4 ： 在标志符的第 5 位仲裁失败(ID.24) 5 ： 在标志符的第 6 位仲裁失败(ID.23) 6 ： 在标志符的第 7 位仲裁失败(ID.22) 7 ： 在标志符的第 8 位仲裁失败(ID.21) 8 ： 在标志符的第 9 位仲裁失败(ID.20) 9 ： 在标志符的第 10 位仲裁失败(ID.19) 10：在标志符的第 11 位仲裁失败(ID.18) 11：在 SRTR 位仲裁失败 12：在 IDE 位仲裁失败 13：在标志符的第 12 位仲裁失败(ID.17) 14：在标志符的第 13 位仲裁失败(ID.16)	0x0

Name	Bits	R/W	Description	Default
			15：在标志符的第 14 位仲裁失败(ID.15) 16：在标志符的第 15 位仲裁失败(ID.14) 17：在标志符的第 16 位仲裁失败(ID.13) 18：在标志符的第 17 位仲裁失败(ID.12) 19：在标志符的第 18 位仲裁失败(ID.11) 20：在标志符的第 19 位仲裁失败(ID.10) 21：在标志符的第 20 位仲裁失败(ID.9) 22：在标志符的第 21 位仲裁失败(ID.8) 23：在标志符的第 22 位仲裁失败(ID.7) 24：在标志符的第 23 位仲裁失败(ID.6) 25：在标志符的第 24 位仲裁失败(ID.5) 26：在标志符的第 25 位仲裁失败(ID.4) 27：在标志符的第 26 位仲裁失败(ID.3) 28：在标志符的第 27 位仲裁失败(ID.2) 29：在标志符的第 28 位仲裁失败(ID.1) 30：在标志符的第 29 位仲裁失败(ID.0) 31：在 RTR 位仲裁失败	

26.6.13 ECC

◆ **偏移地址：0x30　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
error_code	[7:6]	RO	错误类型。 0：位错误（Bit error） 1：格式错误:（Form error） 2：位填充错误（Stuff error） 3：其他类型错误	0x0
direction	[5]	RO	接收或者发送数据期间发生错误。 0: 表示错误发生在发送期间 1: 表示错误发生在接收期间	0x0
segment_code	[4:0]	RO	错误发生的位置。 0：保留	0x0

Name	Bits	R/W	Description	Default
			1：保留 2：ID bit[28:21]错误 3：帧起始错误 4：SRTR 位错误 5：IDE 位 6：ID bit[20:18]错误 7：ID bit[17:13]错误 8：CRC 段错误 9：保留 10：数据段错误 11：数据长度位错误 12：RTR 位错误 13：保留 14：ID bit[4:0]发生错误 15：ID bit[12:5]发生错误 16：保留 17：主动错误标志 18：帧间歇错误 19：容忍显性位错误 20：保留 21：保留 22：被动错误标志 23：错误帧界定符错误 24：CRC 界定符错误 25：ACK 段错误 26：帧结束错误 27：ACK 界定符错误 28：过载帧界定符错误 其他：保留	

26.6.14 EWLR

◆ **偏移地址：0x34　　复位值：0x00000060**

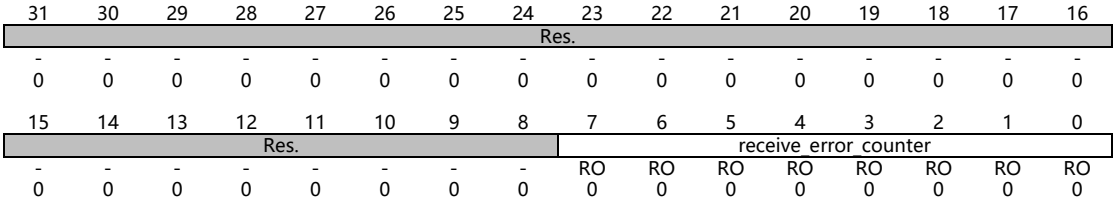
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								error_warning_limit							
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
error_warning_limit	[7:0]	RW	错误警告阈值，超过该阈值，会产生错误警告中断（相应中断使能打开的前提下）。	0x60

26.6.15 RXERR

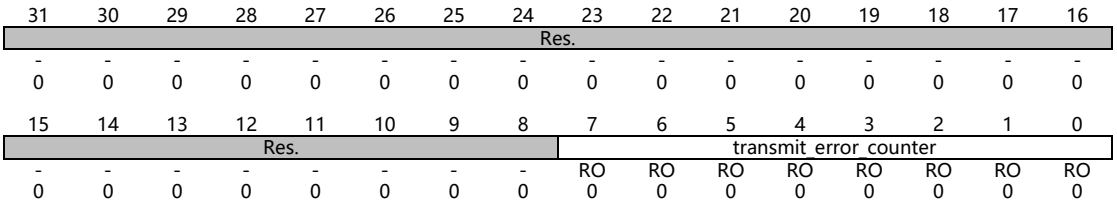
◆ **偏移地址：0x38　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
receive_error_counter	[7:0]	RO	接收时错误统计。	0x0

26.6.16 TXERR

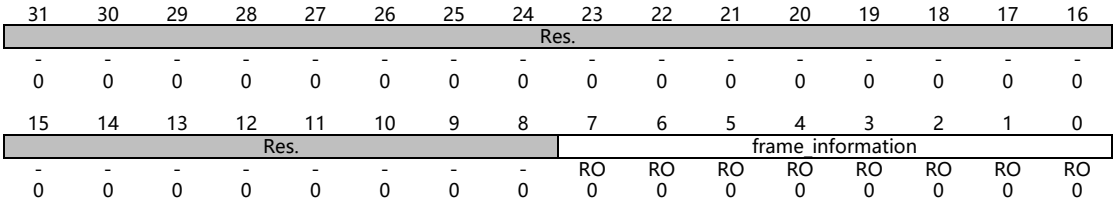
◆ **偏移地址：0x3C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
transmit_error_counter	[7:0]	RO	发送时错误统计。	0x0

26.6.17 FRAME_INFO_ACR0

◆ **偏移地址：0x40　　复位值：0x00000000**



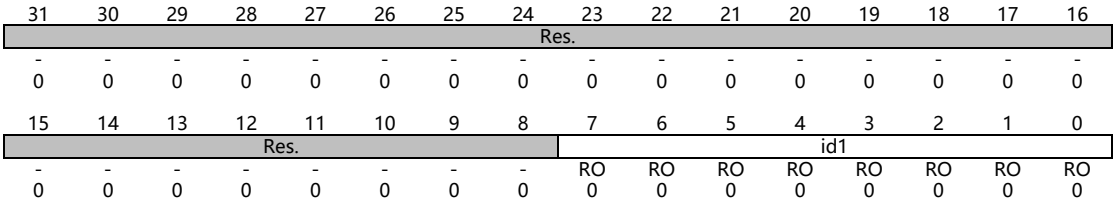
Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
frame_information	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 ACR0，设置对扩展帧格式 ID[28:21]或者标准帧格式 ID[10:3]进行过滤。 在运行状态下，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 1 个字节，表示帧信息： [7]为 1 表示扩展帧格式，为 0 表示标准帧格式； [6]为 1 表示遥控帧，为 0 表示数据帧； [5:4]表示帧信息的保留位 r1、r0； [3:0]表示数据长度，高位在前，有效值范围为 0~8	0x0

标准帧和扩展帧数据排布差异概览：

标准帧		扩展帧	
Offset Address	Name	Offset Address	Name
0x40	Frame Info	0x40	Frame Info
0x44	Identifier 1	0x44	Identifier 1
0x48	Identifier 2	0x48	Identifier 2
0x4C	Data Byte 1	0x4C	Identifier 3
0x50	Data Byte 2	0x50	Identifier 4
0x54	Data Byte 3	0x54	Data Byte 1
0x58	Data Byte 4	0x58	Data Byte 2
0x5C	Data Byte 5	0x5C	Data Byte 3
0x60	Data Byte 6	0x60	Data Byte 4
0x64	Data Byte 7	0x64	Data Byte 5
0x68	Data Byte 8	0x68	Data Byte 6
0x6C	(Unused)	0x6C	Data Byte 7
0x70	(Unused)	0x70	Data Byte 8

26.6.18 ID1_ACR1

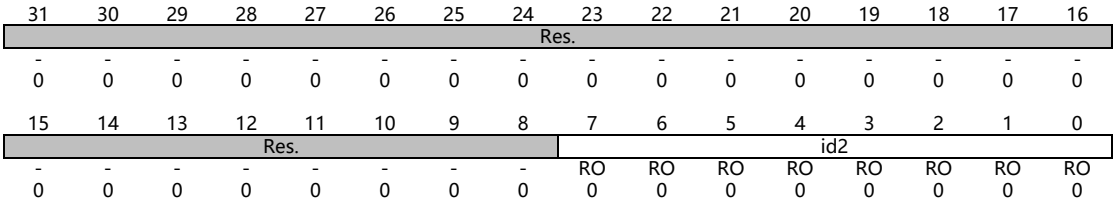
◆ **偏移地址：0x44　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
id1	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 ACR1，设置对扩展帧格式 ID[20:13]或者标准帧格式 ID[2:0]和 RTR 位进行过滤(只用高 4bit)。 在运行状态下，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 2 个字节，表示扩展帧格式 ID bit[28:21]或者标准帧格式 ID bit[10:3]。	0x0

26.6.19 ID2_ACR2

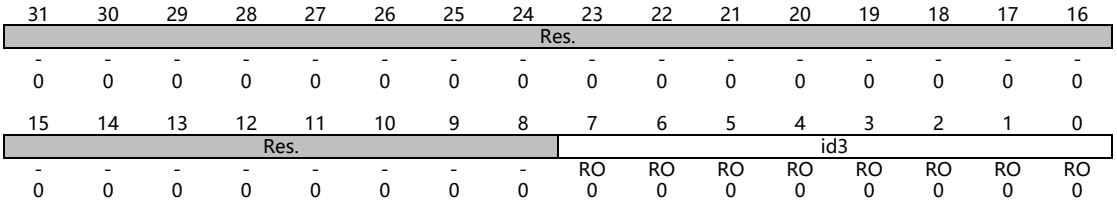
◆ **偏移地址：0x48　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
id2	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 ACR2，设置对扩展帧格式 ID[12:5] 或者标准帧格式 DataByte1 进行过滤。 在运行状态下，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 3 个字节，表示扩展帧格式 ID bit[20:13]或者标准帧格式 ID bit[2:0]（只用高 3bit）。	0x0

26.6.20 ID3_ACR3

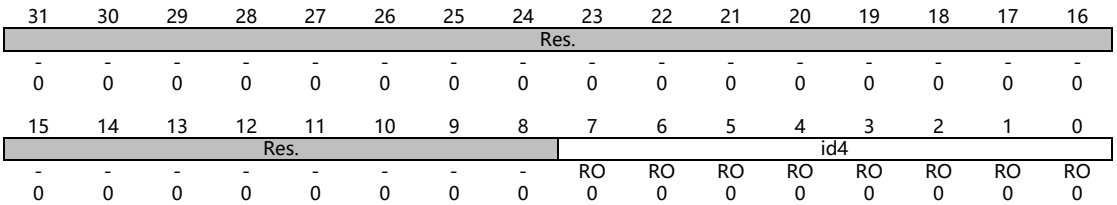
◆ **偏移地址：0x4C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
id3	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 ACR3，设置对扩展帧格式 ID[4:0]和 RTR 位进行过滤（只用高 6bit）或者标准帧格式 DataByte2 进行过滤。 在运行状态下，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 4 个字节，表示扩展帧格式 ID bit[12:5]或者标准帧格式 data byte1。	0x0

26.6.21 ID4_AMR0

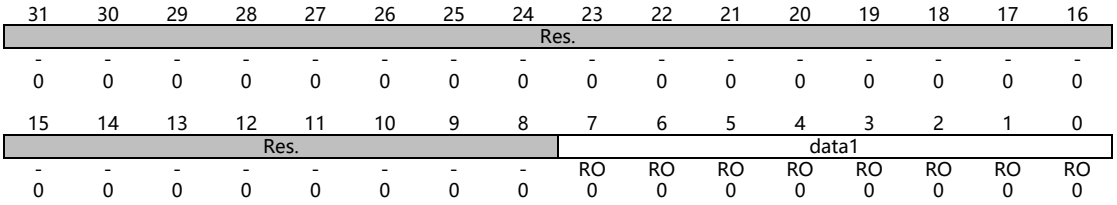
◆ **偏移地址：0x50　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
id4	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 AMR0，与 ACR0 一起决定是否对接收 ID 进行过滤。 在运行状态下，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 5 个字节，高 5bit 表示扩展帧格式 ID bit[4:0]或者标准帧格式 data byte2。	0x0

26.6.22 Data1_AMR1

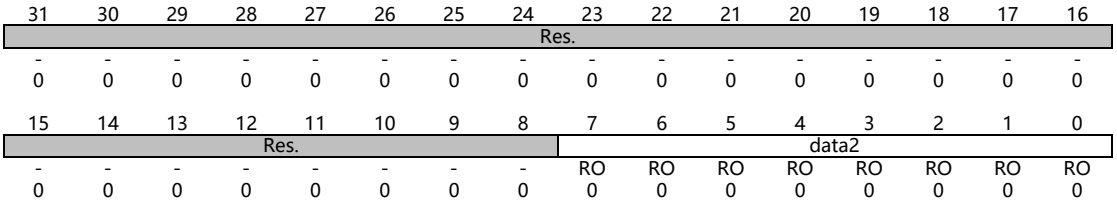
◆ **偏移地址：0x54　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data1	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 AMR1，与 ACR1 一起决定是否对接收 ID 进行过滤。 在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 6 个字节，标准帧格式 Data Byte 3，扩展帧格式 Data Byte 1。	0x0

26.6.23 Data2_AMR2

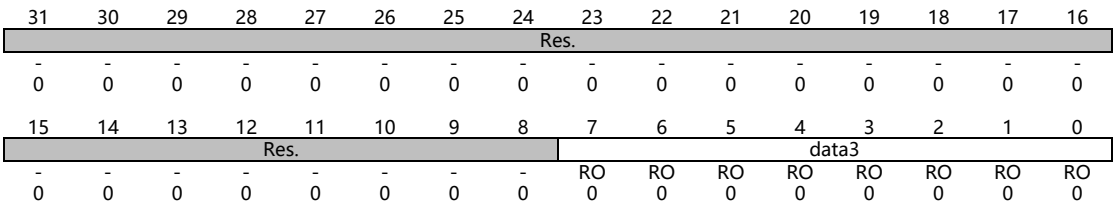
◆ **偏移地址：0x58　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data2	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 AMR2，与 ACR2 一起决定是否对接收 ID 进行过滤。 在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 7 个字节，标准帧格式 Data Byte 4，扩展帧格式 Data Byte 2。	0x0

26.6.24 Data3_AMR3

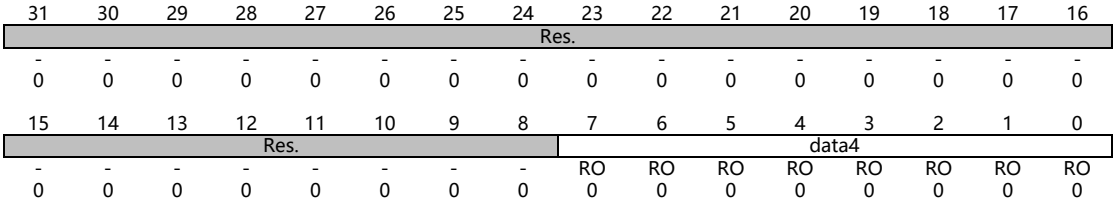
◆ **偏移地址：0x5C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data3	[7:0]	RO	在复位状态下，表示接收过滤 ID 寄存器 AMR3，与 ACR3 一起决定是否对接收 ID 进行过滤。 在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 8 个字节，标准帧格式 Data Byte 5，扩展帧格式 Data Byte 3。	0x0

26.6.25 Data4

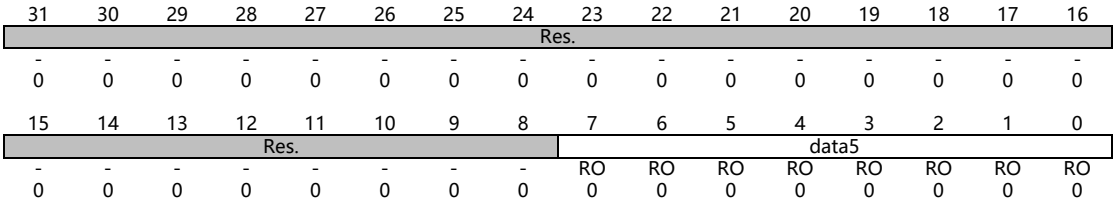
◆ **偏移地址：0x60　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data4	[7:0]	RO	在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 9 个字节，标准帧格式 Data Byte 6，扩展帧格式 Data Byte 4。	0x0

26.6.26 Data5

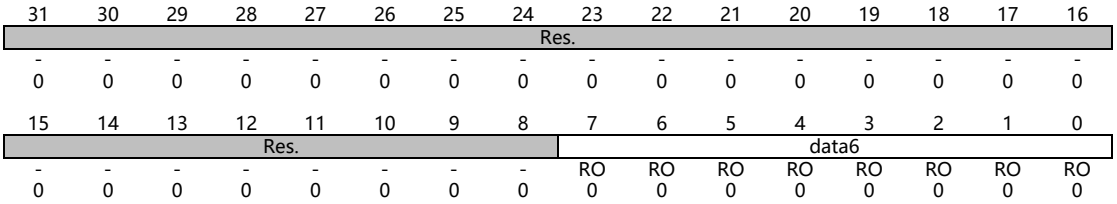
◆ **偏移地址：0x64　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data5	[7:0]	RO	在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 10 个字节，标准帧格式 Data Byte 7，扩展帧格式 Data Byte 5。	0x0

26.6.27 Data6

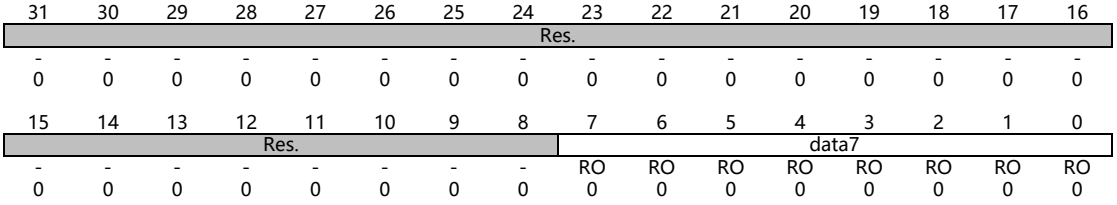
◆ **偏移地址：0x68　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
Data6	[7:0]	RO	在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 11 个字节，标准帧格式 Data Byte 8，扩展帧格式 Data Byte 6。	0x0

26.6.28 Data7

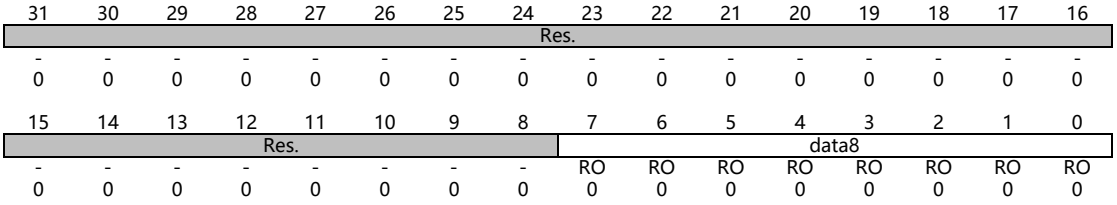
◆ **偏移地址：0x6C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data7	[7:0]	RO	在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 12 个字节，标准帧格式不用，扩展帧格式 Data Byte 7。	0x0

26.6.29 Data8

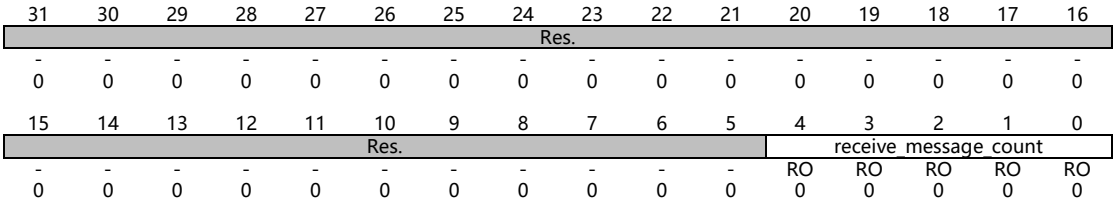
◆ **偏移地址：0x70　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
data8	[7:0]	RO	在运行状态下访问，发送时为 WO，接收时为 RO，表示 TX/RX buffer 第 13 个字节，标准帧格式不用，扩展帧格式 Data Byte 8。	0x0

26.6.30 RMC

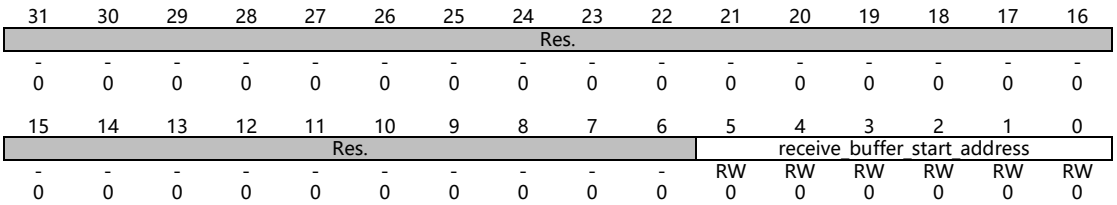
◆ **偏移地址：0x74　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留。	0x0
receive_message_count	[4:0]	RO	当前接收 FIFO 有效信息数量。	0x0

26.6.31 RBSA

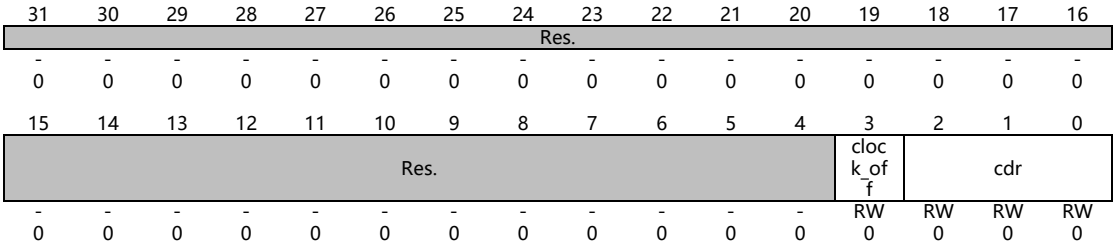
◆ **偏移地址：0x78　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留。	0x0
receive_buffer_start_address	[5:0]	RW	接收 FIFO 当前读指针，0x0 表示当前读指针指向 Receive FIFO 首地址 也即寄存器地址 0x80；0x3F 表示当前读指针指向 Receive FIFO 尾地址 也即寄存器地址 0x17C。	0x0

26.6.32 CDR

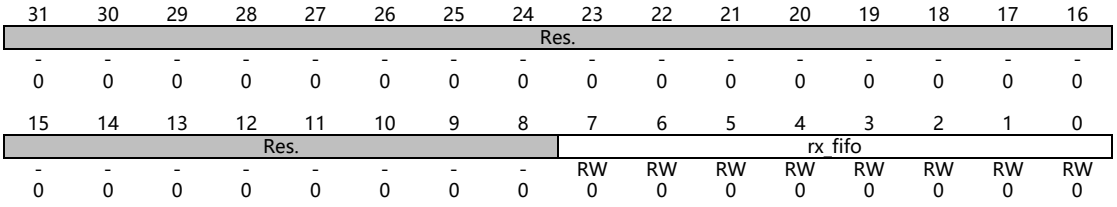
◆ **偏移地址：0x7C　　复位值：0x000000C0**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
clock_off	[3]	RW	0：CLKOUT 输出打开 1：CLKOUT 输出关闭	0x0
cdr	[2:0]	RW	分频系数，表示 CLKOUT 与工作时钟 fosc 的关系，fosc 表示 CAN 控制器工作时钟 XTAL1 的频率。 0 : fosc/2 1 : fosc/4 2 : fosc/6 3 : fosc/8 4 : fosc/10 5 : fosc/12 6 : fosc/14 7 : fosc	0x0

26.6.33 RX_FIFO

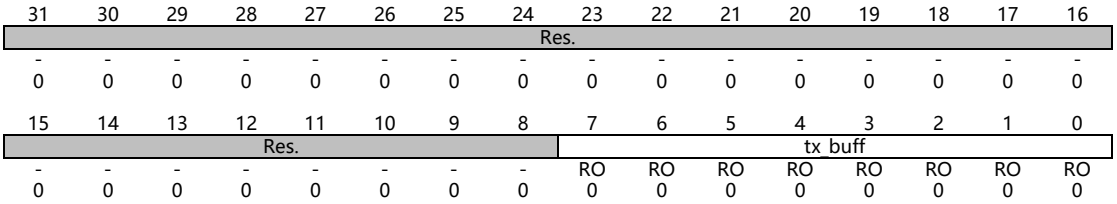
◆ **偏移地址：0x80+0x4*n　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
rx_fifo	[7:0]	RO	读取 RX_FIFO 接收到的数据（n 取值 0~63）。	0x0

26.6.34 TX_BUFF

◆ **偏移地址：0x180+0x4*m　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
tx_buff	[7:0]	RO	读取 TX_BUFF 中的数据（m 取值 0~12）。	0x0

27 通用串行总线（USB)

27.1 USB_FS 简介

芯片内置 1 路双角色通用串行总线 USB_FS，支持设备(DEVICE)功能或主机(HOST)功能，完全符合 USB2.0 规范。USB_FS 支持表 27-1 中定义的速度。

表 27-1 USB_FS 支持的速度

模式	HS(480Mb/s)	FS(12Mb/s)	LS(1.5Mb/s)
主机模式	不支持	支持	支持
设备模式	不支持	支持	不支持

27.2 USB_FS 主要特性

主要特性可分为三类：通用特性、主机模式特性和设备模式特性。

27.2.1 通用特性

USB_FS 接口的通用特性如下：

- 符合 USB-IF 通用串行总线规范 2.0。
- 可通过软件配置为主机模式或者设备模式。
- 支持 FS SOF 和 LS Keep-alive 令牌。
- 具有低功耗模式，例如在 USB 挂起期间停止系统、关闭数字模块内部时钟域以及 PHY 的时钟。
- 采用先进 FIFO 控制的一块 5.875Kbytes 的专用 RAM：
  - 可将 RAM 空间划分为不同 FIFO，方便灵活有效地利用 RAM 空间；

  - 每个 FIFO 可存储多个数据包；
  - 软件配置存储区大小；
  - FIFO 大小可配置为非 2 的幂次方，以便于连续使用存储空间。
- 一帧之内可以无需系统干预，以达到最大的 USB 带宽。

27.2.2 主机模式特性

USB_FS 接口在主机模式下具有以下主要特性和要求：

- 通过芯片外部的电源生成 VBUS 电压。
- 多达 12 个主机通道(管道)，每个通道都可以动态实现重新配置(不停止业务进行寄存器配置)，以支持任意类型的 USB 传输。
- 内置硬件调度器：
  - 可在周期性硬件队列中存储多达 8 个中断传输和同步传输请求；
  - 可在非周期性硬件队列中存储多达 8 个控制传输和批量传输请求。
- 管理一个共享 RX FIFO、一个周期性传输 TX FIFO 和一个非周期性传输 TX FIFO，以便有效使用 USB_FS 专用的 RAM。
- 不支持连接 HUB，只支持点对点连接设备。

27.2.3 设备模式特性

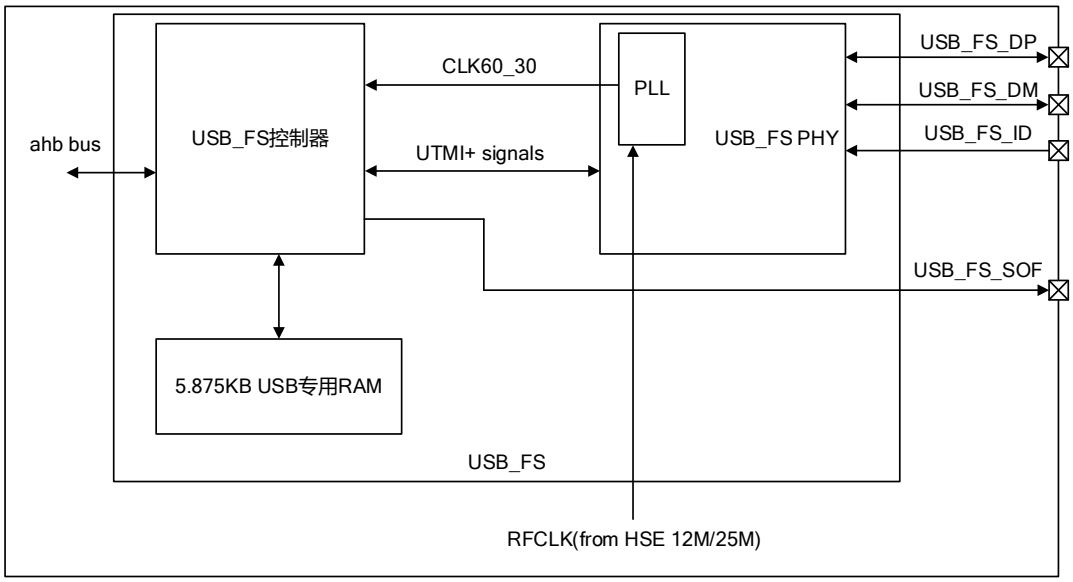
USB_FS 接口在设备模式下具有以下特性：

- 1 个双向控制端点 0。
- 5 个 IN 端点(共 5 个双向端点)，可配置为支持批量传输、中断传输或同步传输。
- 5 个 OUT 端点(共 5 个双向端点)，可配置为支持批量传输、中断传输或同步传输。
- 控制端点和 OUT 端点共享一个 RX FIFO，控制端点和 IN 端点使用各自专用的 TX FIFO（共 6 个 TX FIFO）。
- 支持软断开功能。

27.3 USB_FS 逻辑实现

27.3.1 USB_FS 框图

图 27-1 全速 USB 模块框图



27.3.2 USB_FS PHY

USB_FS PHY 由 USB_FS 控制器模块控制，通过 UTMI+总线实现互联。其包含以下组成部分：

- 供主机和设备模式下使用的 FS/LS 收发器，直接在单端 USB 线上实现发送和接收操作。
- 由 USB_FS 控制器控制的 DP/DM 集成上拉电阻和下拉电阻，具体使能上拉还是下拉电阻取决于设备的当前角色。作为设备模式使用时，使能 DP 上拉电阻，以实现全速设备的连接。主机模式下，则使能 DP 或者 DM 上的下拉电阻（根据主机模式的速度）。
- 内置 PLL，必须接收来自外部晶振的 12MHz 或者 25MHz 的时钟(精度要求+/-200ppm，transition 小于 100ps)，产生 UTMI+并行接口的 60MHz 或者 30MHz 时钟，具体的时钟频率由 USB_FS 控制器控制。

27.3.3 USB_FS 控制器

USB_FS 控制器接收来自 USB_FS PHY 的 60MHz 或者 30MHz 时钟，作为 USB_FS 控制器的 UTMI+并口工作时钟。

CPU 通过 AHB 总线对 USB_FS 控制器的寄存器进行读写操作。

CPU 通过向特定的地址(参考 27.4.7数据 FIFO 章节)写入 32bit 数据来向 USB 提交数据，数据随即自动存储到 USB 的专用 RAM 中配置的数据 TX FIFO 中。每个 IN 端点(设备模式)或 OUT 通道(主机模式)都有相应的一个 TX FIFO 地址。

CPU 从特定的地址读取 32bit 数据，以读取来自 USB 的数据。数据随即从 USB 的专用 RAM 内配置的共享 RX FIFO 中弹出。每个 OUT 端点或者 IN 通道都有相应的一个 RX FIFO 地址。

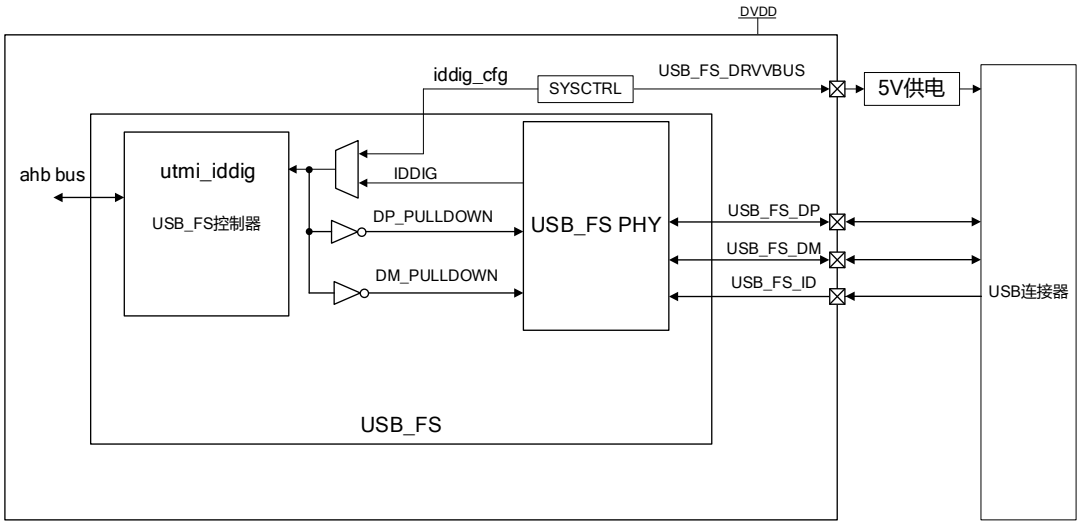
⚠注意

为确保 USB_FS 正常工作，AHB 频率应该不小于 42MHz。

27.4 USB_FS 功能描述

27.4.1 双角色设备(DRD)

图 27-2 USB_FS 双角色设备连接



USB_FS 工作于设备模式(默认模式)还是主机模式，取决于 utmi_iddig 的电平。该信号可由芯片的系统控制器配置(默认)或者由 USB_FS_ID 输入引脚决定。

- 如果 utmi_iddig 为高电平，则 USB_FS 工作于设备模式。
- 如果 utmi_iddig 为低电平，则 USB_FS 工作于主机模式。

27.4.2 USB_FS 设备模式

27.4.2.1 USB_FS 设备状态

连接状态

USB 设备在已连接到 USB 主机，但还没有上电的时候处于连接状态。

供电状态

USB_FS 只支持外部电源供电，即自供电方式。尽管自供电设备可能在插入状态之前已经有供电了，但是仍然要等插入之后，但还没有被复位的时候，才能被认为处于供电状态。

软断开状态

设备控制寄存器(DCTL)的软断开位 sftdiscon 默认配置为 1，设备默认处于软断开状态，所以在设备准备开始工作之前必须软件将该 bit 配置为 0。

默认状态

USB 设备在已连接到 USB 主机，且已经上电，并已经复位，但还没有执行主机发送的 SET_ADDRESS 命令时，处于默认状态（默认设备地址为 0）。

地址状态

在 USB 设备接入 USB 主机并已完成上电和复位后，再完成 USB 主机的 SET_ADDRESS 请求，如果指定的地址是个非零地址，则设备进入地址状态。

配置状态

USB 设备已连接到 USB 主机，已上电和复位，已分配了一个唯一的设备地址，并且完成 Set Configuration 请求后，且未挂起，设备将进入配置状态。

挂起状态

USB 设备持续检视 USB 总线活动，当 USB 总线空闲时间超过 3ms，触发挂起中断(usbsusp)进入挂起状态。然后，设备状态寄存器中的设备挂起位(DSTS 中的 suspsts 位)自动置 1，USB_FS 随即进入挂起状态。

设备主动退出挂起状态：应用程序将设备控制寄存器中的远程唤醒信号位(DCTL 中的 rmtwkupsig)置 1，并在 1ms 到 15ms 后将其清零。

设备被动退出挂起状态：若设备检测到主机发出的恢复信号时，将生成恢复中断(GINTSTS 中的 wkupint)，设备挂起位(DSTS 中的 suspsts 位)会自动清零。

27.4.2.2 USB_FS 设备端点

USB_FS 实现了以下的 USB 端点：

- 控制端点 0：
  - 双向且仅处理控制信息；
  - 使用一组单独的寄存器来处理 IN 和 OUT 事务；
  - 专用的设备端点控制寄存器(DIEPCTL0/DOEPCTL0)、设备端点传输大小寄存器(DIEPTSIZ0/DOEPTSIZ0)和设备端点中断寄存器(DIEPINT0/DOEPINT0)。设备端点控制寄存器和设备端点传输大小寄存器中可用的位域与其他端点稍有不同。
- 5 个 IN 端点：
  - 每个端点都可配置为支持同步传输、批量传输或中断传输类型；
  - 每个端点都有专用的设备 IN 端点控制寄存器(DIEPCTLx)、设备 IN 端点传输大小寄存器(DIEPTSIZx)和设备 IN 端点中断寄存器(DIEPINTx)(x=1~5)；
  - 设备 IN 端点公共中断屏蔽寄存器(DIEPMSK)，可用于使能/禁止所有 IN 端点(包括 EP0)上的同一类端点中断源；
  - 支持未完成的同步 IN 传输中断(GINTSTS 中的 incompisoin 位)，该中断将在当前帧中至少有一个同步 IN 端点上的传输未完成时触发。该中断和周期帧结束中断(GINTSTS 中的 eopf 位)一起触发。
- 5 个 OUT 端点：
  - 每个端点都可配置为支持同步传输、批量传输或中断传输类型；
  - 每个端点都有专用的设备 OUT 端点控制寄存器(DOEPCTLx)、设备 OUT 端点传输大小寄存器(DOEPTSIZx)和设备 OUT 端点中断寄存器(DOEPINTx)(x=1~5)；
  - 设备 OUT 端点公共中断屏蔽寄存器(DOEPMSK)，可用于使能/禁止所有 OUT 端点（包括 EP0）上的同一类端点中断源；
  - 支持未完成的同步 OUT 传输中断（GINTSTS 中的 isooutdrop 位），该中断将在当前帧中至少有一个同步 OUT 端点上的传输未完成时触发。该中断和周期帧结束中断(GINTSTS 中的 eopf 位)一起触发。

端点控制

- 应用程序可通过设备端点 x(x=0~5)的 IN/OUT 控制寄存器(DIEPCTLx/DOEPCTLx)对

端点采取以下控制：

- 端点使能/禁止；
- 在当前配置下激活端点；
- x=1~5 时设置 USB 传输类型(同步传输、批量传输或中断传输)；
- 设置支持的数据包大小；
- 设置与 IN 端点（1~5）和控制端点 0 相关的 TX FIFO 编号；
- 设置希望收到的或发送时要使用到的 data0/data1 PID(仅限批量传输或中断传输）；
- 设置接收或发送事务时所对应的奇/偶帧(仅限同步传输)；
- 可以设置 nak 位，从而不论此时 FIFO 的状态如何，都对主机的请求回复 NAK；
- 可以设置 stall 位，使得主机对该端点的令牌都被硬件回复 STALL；
- 可以将 OUT 端点设置为侦听模式，即对接收到的数据不进行 CRC 检查。

端点传输

设备端点 x(x=0~5)传输大小寄存器（DIEPTSIZx/DOEPTSIZx）允许应用程序对传输大小参数进行编程并读取传输状态。必须在端点控制寄存器中的端点使能位置 1 之前完成对此寄存器的设置。使能端点后，这些字段立即变为只读状态，同时 USB_FS 根据当前传输状态对这些字段进行更新。

可对以下传输参数进行编程：

- 以字节为单位的传输大小；
- 构成整个传输的数据包个数。

端点状态/中断

设备端点 x(x=0~5)中断寄存器(DIEPINTx/DOPEPINTx)指示端点在出现 USB 和 AHB 相关事件时的状态。当模块中断寄存器中的 OUT 端点中断位或 IN 端点中断位(分别为 GINTSTS 中的 oepint 位或 GINTSTS 中的 iepint 位)置 1 时，应用程序必须读取这些寄存器以获得详细信息。在应用程序读取这些寄存器之前，必须先读取设备全体端点中断寄存器(DAINT)，以获取设备端点 x 中断寄存器的端点编号。应用程序必须将此寄存器中的相应位清零，才能将 DAINT 和 GINTSTS 寄存器中的相应位清零。

模块提供以下状态检查和中断产生功能：

- 传输完成中断，指示应用程序(AHB)和 USB 端均已完成数据传输。

- Setup 阶段已完成(仅针对控制传输类型的 OUT 端点)。
- 相关的发送 FIFO 为半空或全空状态(IN 端点)。
- NAK 应答已发送到主机(仅针对同步传输的 IN 端点)。
- TX FIFO 为空时接收到 IN 令牌(仅针对批量和中断传输类型的 IN 端点)。
- 尚未使能端点时接收到 OUT 令牌。
- 检测到串扰错误。
- 应用程序关闭端点生效。
- 应用程序对端点设置 NAK 生效(仅针对同步传输类型的 IN 端点)。
- 接收到 3 个以上连续 setup 数据包(仅针对控制类型的 OUT 端点)。
- 检测到超时状况(仅针对控制传输类型的 IN 端点)。
- 同步传输类型的数据包未产生中断而丢失。

27.4.3 USB_FS 主机模式

27.4.3.1 USB_FS 主机状态

主机端口供电

芯片不能输出 5V 电源以提供 VBUS，如果需要，可在芯片外部添加 5V 供电电源来驱动 5V VBUS 线，外部 5V 供电电源开关可由 USBFS_DRVVBUS 管脚来控制。无论是否需要外部 5V 供电都必须将主机端口控制和状态寄存器中的端口电源位 (HPRT 中的 prtpwr 位)置 1。

主机检测设备连接

当 USB 设备在连接后立即被检测到，USB_FS 控制器将发出主机端口中断信号，该中断由主机端口控制和状态寄存器中的设备连接位(HPRT 中的 prtconndet)触发。

主机检测设备断开

设备断开事件将触发断开连接检测中断(GINTSTS 中的 disconnint 位)。

主机枚举

检测到设备连接后，主机必须通过向新的设备发送 USB 复位和配置命令来启动枚举过程。

应用程序通过将主机端口控制和状态寄存器中的端口复位位(HPRT 中的 prtrst 位)置 1，并保持最少 10 ms，最多 20 ms，来在 USB 总线上发出 USB 复位信号(单端零)。然后软件将端口复位位清零。

USB 复位序列完成后，端口使能/禁止更改位(HPRT 中的 prtenchng 位)立即触发主机端口

中断，进而向应用程序发出通知，指示可从主机端口控制和状态寄存器中的端口速度字段(HPRT 中的 prtspd 位)读取枚举的设备速度，以及主机已经开始驱动 SOF(FS)或 Keep-alive 令牌(LS)。此时主机已就绪，可通过对设备发送命令来完成对设备的枚举。

主机挂起

应用程序通过将主机端口控制和状态寄存器中的端口挂起位(HPRT 中的 prtsusp)置 1 来挂起 USB 活动。USB_FS 控制器停止发送 SOF 并进入挂起状态。

可由远程设备的自主活动(远程唤醒)使总线退出挂起状态。这种情况下，远程唤醒信号将触发远程唤醒中断(GINTSTS 中的 wkupint 位)，USB_FS 控制器把主机端口控制和状态寄存器中的端口恢复位(HPRT 中的 prtres 位)自动置位，并自动驱动恢复信号。应用程序必须将端口恢复位清零以退出挂起状态并重新发送 SOF。

如果由主机发起退出挂起状态，则应用程序必须将端口恢复位置 1 以启动主机端口上的恢复信号，并最终将端口恢复位清零。

27.4.3.2 USB_FS 主机通道

OTG_FS 控制器实现了 12 个主机通道。每个主机通道均可用于 USB 主机传输(USB 管道)。主机最多能同时处理 12 个传输请求。如果应用程序有 12 个以上的传输请求挂起，则在通道从之前任务释放后(即接收到传输完成和通道停止中断后)，主机控制器驱动器(HCD)必须为未处理的传输请求重新对通道进行分配。

每个主机通道都可配置为支持输入/输出以及周期性/非周期性事务。每个主机通道都使用专用的主机通道特征寄存器(HCCHARn)、主机通道传输大小寄存器(HCTSIZn)、主机通道中断寄存器(HCINTn)和主机通道中断屏蔽寄存器(HCINTMSKn)(n=0~11)。

主机通道控制

- 应用程序可通过主机通道特征寄存器(HCCHARn) (n=0~11)对主机通道作以下控制：
  - 通道使能/禁止；
  - 设置目标 USB 设备的 FS/LS 速度；
  - 设置目标 USB 设备的地址；
  - 设置与该通道通信的目标 USB 设备上的端点的编号；
  - 设置该通道上的传输方向：IN/OUT；
  - 设置该通道上的 USB 传输的类型：控制传输/批量传输/中断传输/同步传输；
  - 设置与该通道通信的设备端点的最大包长；

- 设置要进行周期传输的帧：奇帧/偶帧。

主机通道传输

主机通道传输大小寄存器(HCTSIZn) (n=0~11)允许应用程序对传输大小参数进行编程并读取传输状态。必须在主机通道特性寄存器中的通道使能位置 1 之前完成对此寄存器的设置。使能端点后，数据包计数字段立即变为只读状态，同时 USB_FS 控制器根据当前传输状态对该字段进行更新。

- 可对以下传输参数进行编程：
  - 以字节为单位的传输大小；
  - 构成整个传输大小的数据包个数；
  - 初始数据 PID。

主机通道状态/中断

主机通道 n 中断寄存器(HCINTn) (n=0~11)指示通道在出现 USB 和 AHB 相关事件时的状态。当模块中断寄存器中的主机通道中断位(GINTSTS 中的 hchint 位)置 1 时，应用程序必须读取这些寄存器以获得详细信息。在读取这些寄存器之前，应用程序必须先读取主机全体通道中断(HAINT)寄存器，以获取主机通道 x 中断寄存器的通道编号。应用程序必须将该寄存器中的相应位清零，才能将 HAINT 和 GINTSTS 寄存器中的对应位清零。
HCINTMSKn 寄存器还提供每个通道各中断源的屏蔽位。

- 主机模块提供以下状态检查和中断产生功能：
  - 传输完成中断，指示应用程序(AHB)和 USB 端均已完成数据传输；
  - 通道因传输完成、USB 事务错误或应用程序发出禁止命令而停止；
  - 接收到 STALL 响应；
  - 接收到 NAK 响应；
  - 接收到 ACK 响应；
  - 由于 CRC 校验失败、超时、位填充错误和错误的 EOP 导致 USB 事务错误；
  - 串扰错误；
  - 帧上溢；
  - 数据同步错误。

27.4.3.3 USB_FS 主机调度器

主机模块内置硬件调度器，可自主对应用程序发出的 USB 事务请求重新排序和管理。每一

帧开始时，主机都先执行周期性(同步和中断)事务，然后执行非周期性(控制和批量)事务，以符合 USB 规范对同步和中断传输高优先级的保证。

主机通过请求队列(一个周期性请求队列和一个非周期请求队列)处理 USB 事务。每个请求队列最多可存储 8 个条目。每个条目代表一个应用程序发起但还未得到响应的 USB 事务请求，并存储了执行该 USB 事务所用到的 IN 或 OUT 通道的编号，以及其它相关信息。USB 事务请求在队列中的写入顺序决定了事务在 USB 接口上的执行顺序。

每一帧开始时，主机都先处理周期性请求队列，然后处理非周期性请求队列。如果当前帧结束时，计划在当前帧执行的同步或中断类型的 USB 传输事务请求仍处于挂起状态，则主机将发出未完成周期性传输中断(GINTSTS 中的 incomplp 位)。USB_FS 控制器全面负责对周期性和非周期性请求队列的管理。周期性发送 FIFO 和队列状态寄存器(HPTXSTS)与非周期性发送 FIFO 和队列状态寄存器(GNPTXSTS)都为只读寄存器，应用程序可使用它们来读取各请求队列的状态。其中包括：

- 周期性(非周期性)请求队列中当前可用的空闲条目数(最多 8 个) 。
- 周期性(非周期性)TX FIFO(OUT 事务)中当前可用的空闲空间。
- IN/OUT 令牌、主机通道编号和其它状态信息。

由于每个请求队列最多可存储 8 个 USB 事务请求，因此应用程序可以把主机 USB 事务请求提前发送给调度器；实际的通信最晚会在调度器处理完已挂起的 8 个周期事务和 8 个非周期事务完成之后出现在 USB 总线上。

要向主机调度器(队列)发出事务请求，应用程序必须读取 HPTXSTS 寄存器中的 ptxqspcavail 位或 GNPTXSTS 寄存器中的 nptxqspcavail 位，确保周期性(非周期性)请求队列中至少有一个可用空间来存储当前请求。

27.4.4 USB_FS SOF 触发

27.4.4.1 主机模式 SOF

主机模式下，可以在主机帧间隔寄存器(HFIR)中对所产生的两个连续 SOF(FS)或 Keep-alive(LS)令牌期间所出现的 PHY 时钟数进行编程，进而应用程序可对 SOF 帧周期进行控制。帧开始(GINTSTS 中的 sof 位)时都将生成中断。当前帧编号和出现下一个 SOF 前剩余的时间应用程序在主机帧编号寄存器(HFNUM)中能够进行跟踪。

SOF 令牌发出的同时会产生 SOF 翻转信号。

27.4.4.2 设备模式 SOF

在设备模式下，USB 每次接收到 SOF 令牌时，都将触发帧开始中断(GINTSTS 中的 sof 位)。相应的帧编号可从设备状态寄存器(DSTS 中的 soffn 位)读取。还可以生成 SOF 翻转

信号。

周期性帧结束中断(GINTSTS 中的 eopf 位)用于在经过了 80%、85%、90%或 95%的帧间隔时间时通知应用程序，具体取决于设备配置寄存器中的周期性帧间隔字(DCFG 中的 perfrint 位)。此功能可用于确定该帧的所有同步通信是否完成。

27.4.5 低功耗模式

以下寄存器位和配置过程降低了功耗。

USB_FS PHY 的功耗由 UTMI+接口信号 SUSPENDM 控制，当 USB_FS 进入挂起状态时，通过 SUSPENDM 信号让 USB_FS PHY 进入低功耗模式，关闭所有时钟。当 USB_FS 退出挂起状态，通过 SUSPENDM 信号让 USB_FS PHY 退出低功耗模式，打开时钟，进入正常功能模式。

USB 会话没有开始或设备未连接时，可以在 USB 挂起状态下使用功率降低技术。

- 停止 PHY 时钟(PCGCCTL 中的 stoppclk 位)

将时钟门控控制寄存器中的停止 PHY 时钟位置 1 时，USB_FS 控制器的 60MHz/30MHz(并口模式)内部时钟域均由时钟门控关闭。即使应用程序仍提供时钟输入，也会节省掉模块由于时钟信号翻转带来的动态功耗

还会关掉收发器的大部分单元，只有负责检测异步恢复事件或远程唤醒事件的部分还保持工作状态。

- 动态时钟门控(PCGCCTL1 中的 gateen 位)

将该位置 1 时，打开动态时钟门控，只在 USB_FS 控制器有数据访问时为其提供时钟。

- RAM 门控(PCGCCTL1 中的 ramgateen 位)

将该位置 1 时，只在 USB 数据 FIFO 被 USB_FS 控制器访问时为 RAM 提供时钟。

27.4.6 动态更新 HFIR 寄存器

主机模式下，USB 模块具有对 SOF 帧周期进行动态微调的功能，能够将外部设备与 SOF 帧进行同步。

对于动态更新，需要设置 HFIR.hfirrldctrl=1。如果 HFIR 寄存器在当前 SOF 帧内发生更改，则将在当前帧中对 SOF 周期进行相应修正，具体说明请参考图 27-3。

对于非动态更新，需要设置 HFIR.hfirrldctrl=0。如果 HFIR 寄存器在当前 SOF 帧内发生更改，则将在下一个帧中对 SOF 周期进行相应修正，具体说明请参考图 27-4。

图 27-3 动态更新 HFIR

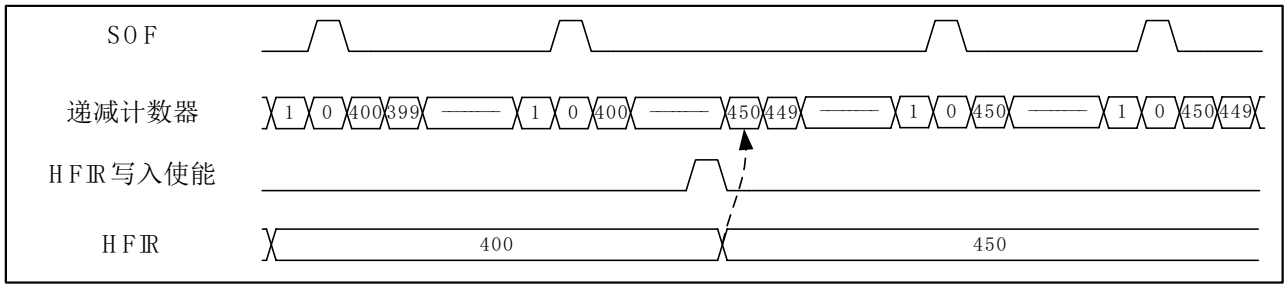
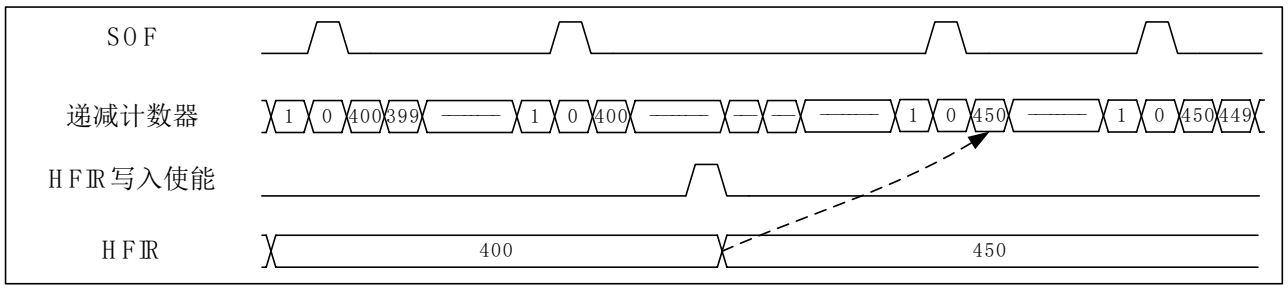


图 27-4 非动态更新 HFIR

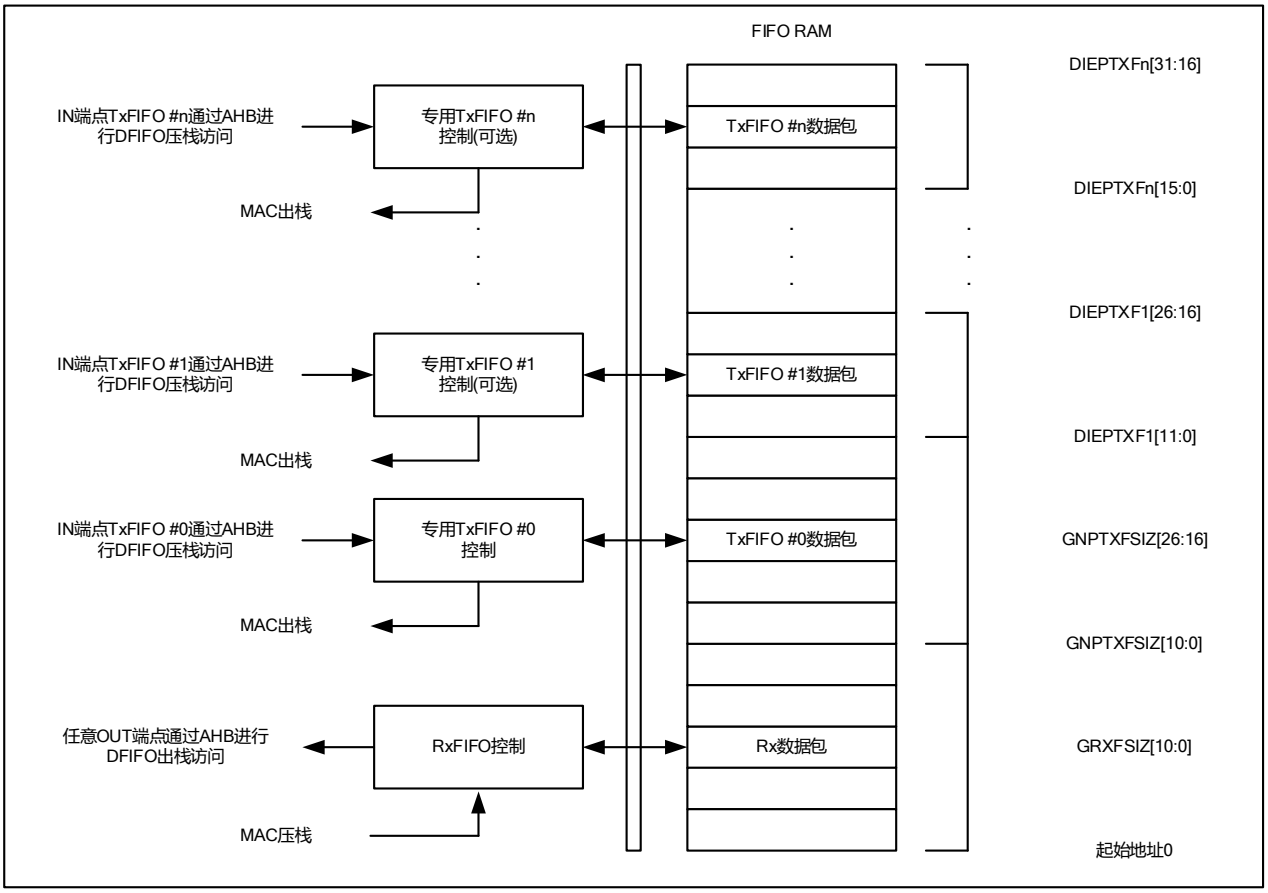


27.4.7 数据 FIFO

USB_FS 具有 5.875KB 专用 RAM，采用复杂的 FIFO 控制机制。USB_FS 控制器中的数据包 FIFO 控制器模块将 RAM 空间划分为多个 TX FIFO(USB 传输前，应用程序将数据压入其中进行短暂存储)和单个 RX FIFO(从 USB 接收到的数据被应用程序读取之前，在其中进行短暂存储)。RAM 中所构建的 FIFO 的数量与组织方式取决于设备的角色。设备模式下，为每个激活的 IN 端点配置一个 TX FIFO；主机模式下，所有通道共享一个周期 TX FIFO 和一个非周期 TX FIFO。FIFO 的大小均由软件配置，以更好地满足应用要求。

27.4.7.1 设备 FIFO 架构

图 27-5 设备模式下的 FIFO 地址映射



设备 RX FIFO

USB_FS 设备使用单个 RX FIFO 接收发送到所有 OUT 端点的数据。只要 RX FIFO 中有空余空间，收到的数据包就依次填入 RX FIFO。除了有效数据外，接收到的数据包状态(包含 OUT 端点目标编号、字节数、数据 PID 和对所接收数据的验证)也由模块进行存储。没有可用空间时，设备会回复主机事务 NAK 应答并在被寻址的端点上触发中断。接收 FIFO 的大小在接收 FIFO 大小寄存器(GRXFSIZ)中配置。

单个接收 FIFO 架构使得 USB 设备更高效地填充接收 RAM 缓冲区：

- 所有 OUT 端点共享同一个 RAM 缓冲区(共享 FIFO)。
- USB_FS 模块可将主机发出的任何 OUT 通信序列填充到 RX FIFO，直到没有多余空闲空间。

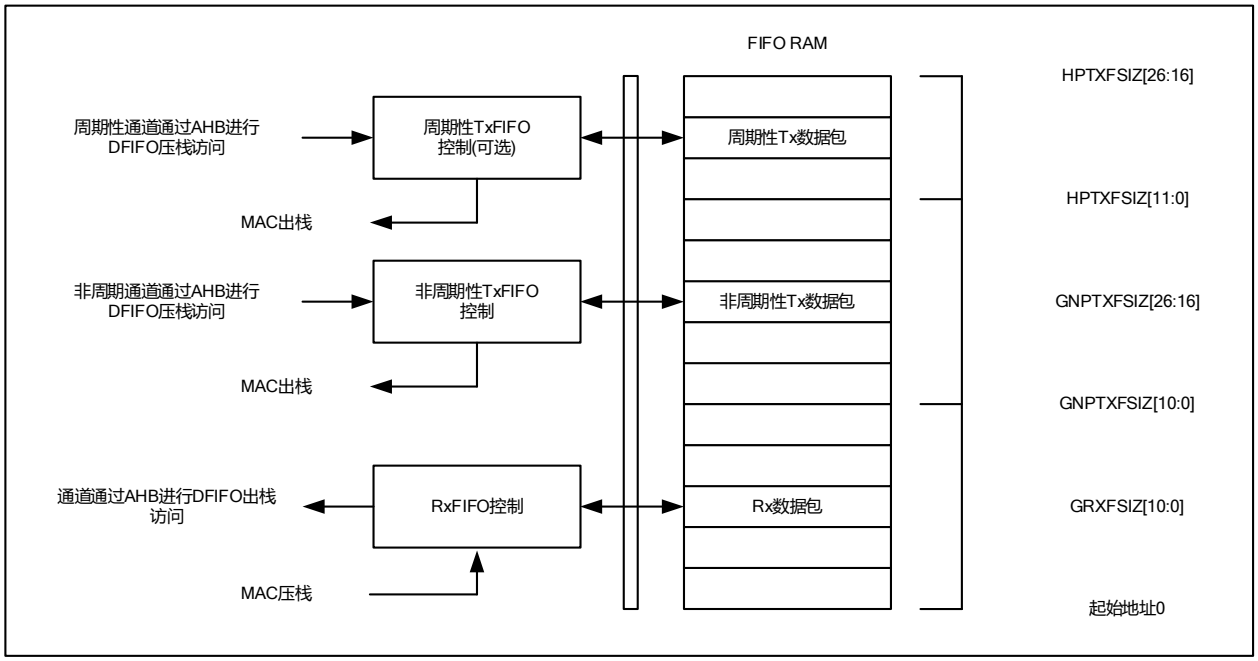
只要至少有一个数据包在 RX FIFO 中可供读取，应用程序就会一直接收 RX FIFO 非空中断(GINTSTS 中的 rxflvl 位)。应用程序从接收状态读取和出栈寄存器(GRXSTSP)中读取数据包信息，最后通过读取与端点相关的出栈地址从 RX FIFO 读出相应数据。

设备 TX FIFO

模块为各个 IN 端点提供了专用的 FIFO。应用程序通过非周期发送 FIFO 大小寄存器(GNPTXFSIZ)为 IN 端点 0 配置 FIFO 大小；通过设备 IN 端点发送 FIFOx 寄存器(DIEPTXFx)为 IN 端点 x 配置 FIFO 大小。

27.4.7.2 主机 FIFO 架构

图 27-6 主机模式下的 FIFO 地址映射



主机 RX FIFO

主机使用一个 RX FIFO 处理所有周期和非周期事务。FIFO 用作接收缓冲区以保存从 USB 接收到的数据(接收到的数据包的数据部分)，直至这些数据传输到系统存储器。只要 FIFO 中有空间，来自设备 IN 端点的数据包就接收进来并依次存储。接收到的每个数据包的状态(包含主机目标通道、字节数、数据 PID 和对所接收数据的校验)也存储在 FIFO 中。RX FIFO 的大小在接收 FIFO 大小寄存器(GRXFSIZ)中配置。

单个 RX FIFO 架构使得 USB 主机高效地填充接收数据缓冲区：

- 所有 IN 配置主机通道共享同一个 RAM 缓冲区。
- 对于主机软件驱动的任意序列 IN 令牌，USB_FS 控制器可将 RX FIFO 填充至限值。

只要至少有一个数据包在 RX FIFO 中可供读取，应用程序就会接收 RX FIFO 非空中断。应用程序从接收状态读取和出栈寄存器中读取数据包信息，最后从 RX FIFO 中读出数据。

主机 TX FIFO

主机使用一个 TX FIFO 处理所有非周期(控制传输和批量传输)OUT 事务，使用另一个 TX FIFO 处理所有周期(同步传输和中断传输)OUT 事务。FIFO 用作发送缓冲区以保存要通过 USB 发送的数据(发送数据包)。周期(非周期)TX FIFO 的大小在主机周期(非周期)发送 FIFO 大小(HPTXFSIZ/HNPTXFSIZ)寄存器中配置。

两个 TX FIFO 按优先级实施操作，周期性通信的优先级较高，因此在 USB 一帧的时间内首先进行周期性通信。帧起始时，内置的主机调度器先处理周期请求队列，再处理非周期请求队列。

两个发送 FIFO 的架构使得 USB 主机能够对周期和非周期发送数据缓冲区分别进行优化

管理：

- 配置为支持周期(非周期)OUT 事务的所有主机通道共享同一个 RAM 缓冲区。
- 对于主机软件驱动的任意序列 OUT 令牌，USB_FS 控制器可将周期性(非周期性)TX FIFO 填充至限值。

只要周期性 TX FIFO 为半空或全空，USB_FS 控制器就会发出周期性 TX FIFO 空中断(GINTSTS 中的 ptxfemp 位)，具体取决于 AHB 配置寄存器中的周期性 TX FIFO 空等级(GAHBCFG 中的 ptxfemplvl 位)的值。只要周期性 TX FIFO 和周期性请求队列中均存在空闲空间，应用程序便可提前写入发送数据。可通过读取主机周期性 TX FIFO 和队列状态寄存器(HPTXSTS)来了解二者的可用空间。

只要非周期性 TX FIFO 为半空或全空，USB_FS 控制器就会发出非周期性 TX FIFO 空中断(GINTSTS 中的 nptxfemp 位)，具体取决于 AHB 配置寄存器中的非周期性 TX FIFO 空等级位(GAHBCFG 中的 ptxfemplvl 位)。只要非周期性 TX FIFO 和非周期性请求队列中均存在空闲空间，应用程序便可写入发送数据。可通过读取主机非周期性发送 FIFO 和队列状态寄存器(HNPTXSTS)来了解二者的可用空间。

27.4.7.3 FIFO RAM 分配

设备模式

接收 FIFO RAM 分配：

- 接收 FIFO 中必须保留 13 个位置以在控制端点上接收 SETUP 数据包。USB_FS 控制器不会向这些为 SETUP 数据包保留的位置写入任何其它数据。
- 将会为全局 OUT NAK 分配一个位置。
- 状态信息随各个接收数据包写入 FIFO。因此，必须至少为接收数据包分配(最大数据包大小/4)+1 的空间。如果使能了多个同步端点，则为接收连续数据包分配的空间必

须至少为(最大数据包大小/4)的两倍+1。通常，推荐的空间为(最大数据包/4+1)的两倍，这样当上一个数据包向 CPU 传送时，USB 可同时接收后续的数据包。

- 传输完成状态信息和该端点收到的最后一个数据包会一起被推入 FIFO；也需要为每个端点的 EPDisable 状态分配一个位置。所以，通常推荐为每个 OUT 端点分配两个位置。

设备 RX FIFO=(5*控制端点数量+8)+2*((所使用的最大 USB 数据包/4)+用于状态信息的 1)+(2*OUT 端点数量)+用于全局 NAK 的 1。

例如：USB 数据包的 MPS 是 64 个字节。有五个 OUT 端点、五个 IN 端点、一个控制端点和十二个主机通道。

则设备 RX FIFO=(5*1+8)+2*((64/4)+1)+(2*6)+1=60。

发送 FIFO RAM 分配：各个 IN 端点发送 FIFO 所需的最小 RAM 空间为该特定 IN 端点的最大数据包大小，即(最大数据包大小/4)。一般而言，推荐 2*(最大数据包大小/4)的大小。为发送 IN 端点 FIFO 分配的 RAM 空间越多，USB 的性能越高。

主机模式

接收 FIFO RAM 分配：

状态信息随各个接收数据包写入 FIFO。因此，必须至少为接收数据包分配(最大数据包大小/4)+2 的空间。如果使能了多个同步通道，则为接收连续数据包分配的空间必须至少为(最大数据包大小/4+2)的两倍。通常，推荐的空间为(最大数据包/4+2)的两倍，这样当上一个数据包向 CPU 传送时，USB 可同时接收后续的数据包。传输完成状态信息和主机通道中的最后一个数据包会一起被推入 FIFO。因此，必须为此分配一个位置。

主机 RX FIFO=2*((所用的最大 USB 数据包/4+用于状态信息的 2)+用于传输完成的 1。

例如：MPS=64，则主机 RX FIFO=2*((64/4)+2)+1=37。

发送 FIFO RAM 分配：

主机非周期性发送 FIFO 所需的最小 RAM 为所支持的所有非周期性 OUT 通道上传输的最大数据包的大小。通常，推荐的空间为最大数据包大小的两倍，这样当 USB 正在发送当前数据包的同时，CPU 可获得下一个数据包。为非周期性发送 FIFO 分配的 RAM 空间越大，USB 的性能越高。

非周期性 TX FIFO=2*(所用的最大非周期性 USB 数据包/4)。

例如：MPS=64，则非周期性 TX FIFO=2*(64/4)=32。

主机周期性发送 FIFO 所需的最小 RAM 为所支持的所有周期性 OUT 通道上传输的最大数据包的大小。如果至少有一个同步 OUT 端点，则空间必须至少为该通道中最大数据包大小

的两倍。

主机周期性 TX FIFO=2*(所用的最大周期性 USB 数据包/4)。

例如：MPS=64，主机周期性,TX FIFO=2*(64/4)=32。

27.4.8 系统性能

凭借大容量 RAM 缓冲区、高度可配置的 FIFO 大小、通过对 32 位 FIFO 快速访问，尤其是高级 FIFO 控制机制可获得最佳 USB 性能和系统性能。实际上，无论当前 USB 序列如何，USB_FS 均可通过该机制高效填充可用的 RAM 空间。借助这些特性：

- 应用程序有足够的裕量来计算并校正 CPU 的负载，从而优化 CPU 带宽利用率：
  - 应用程序可先积累大量发送数据，再通过 USB 发送出去；
  - 可带来足够的时间裕量，以从接收 FIFO 读取数据。
- USB 模块能够保持全速工作状态，也就是提供最大的全速带宽(尽量多的硬件自动运行，尽量少的软件参与)：
  - USB 模块可提前积累大量发送数据供其支配，从而可对 USB 数据发送进行自主管理；
  - 接收缓冲区中有大量空白空间，可通过 USB 中的数据自动填满。

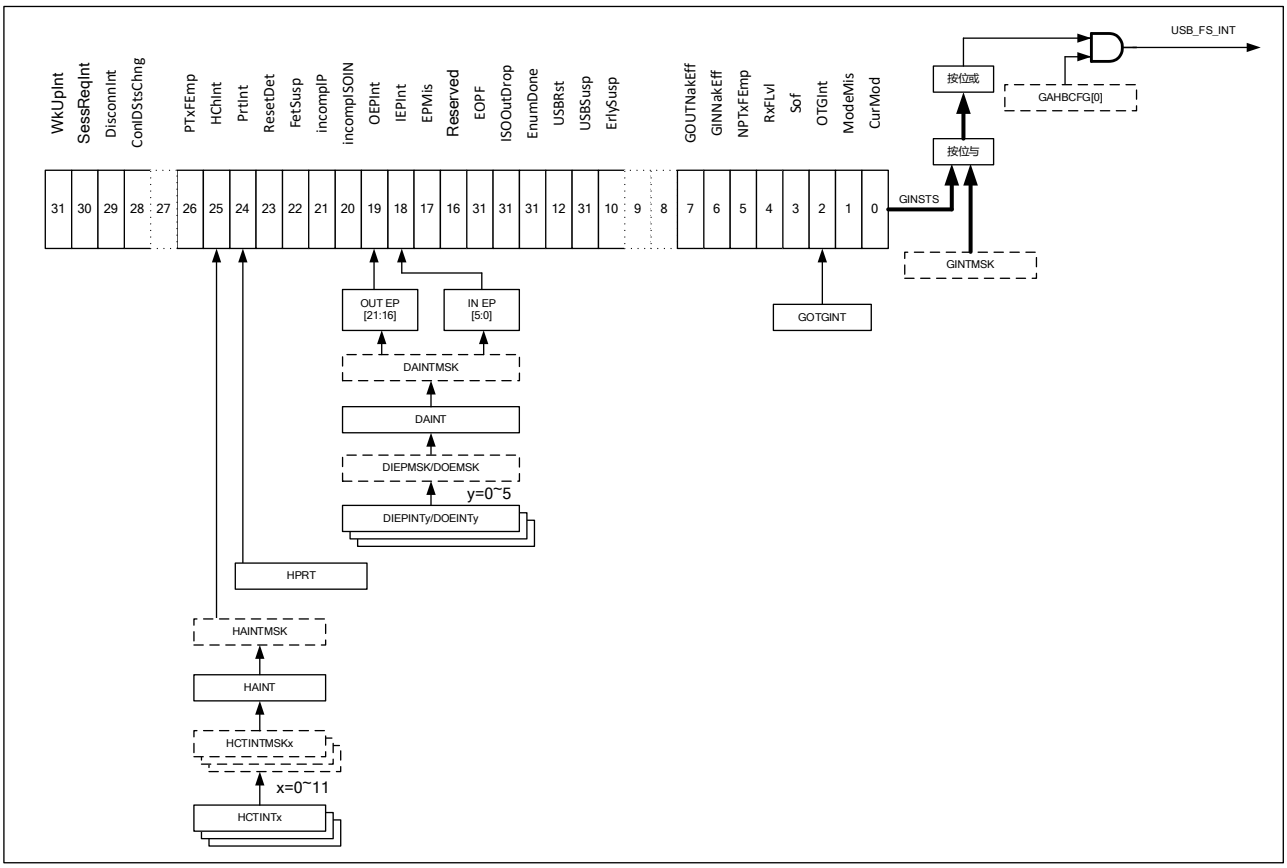
由于 USB_FS 模块能够高效填充 5.875 KB RAM 缓冲区且 5.875 KB 发送/接收数据足以满足一个全速帧所能容纳的数据量，因此 USB 系统在一帧(1 ms)之内可以无需 CPU 干预达到最大 USB 带宽。

27.4.9 中断

当 USB_FS 控制器在一种模式下(设备模式或主机模式)工作时，应用程序不得以另一种角色模式访问寄存器。如果发生了非法访问，将会产生模式不匹配中断并在模块中断寄存器(GINTSTS 寄存器中的 modemis 位)中反映。当模块从一种角色模式切换到另一种角色模式时，新工作模式下的寄存器必须重新编程为上电复位后的状态。

图 27-7 显示了中断汇聚关系。

图 27-7 中断汇聚



27.5 USB_FS 编程流程

27.5.1 控制器全局初始化

应用程序必须执行控制器初始化序列。GINTSTS 中的当前工作模式位(GINTSTS 中的 curmod 位)指示模式(设备模式或主机模式)。

本节介绍了 USB_FS 控制器在上电后的初始化过程。无论是以主机模式还是设备模式工作，应用程序都必须遵循初始化序列。根据模块配置对所有模块全局寄存器进行初始化：

步骤 1 读取用户硬件配置寄存器(GHWCFG1-GHWCFG4)以获取 USB_FS 控制器的配置参数。

步骤 2 在 GAHBCFG 寄存器中编程以下字段：

- 全局中断屏蔽位 glblintrmsk=1；
- 非周期性 TX FIFO 空门限位 nptxfemplvl(仅主机模式下有效)；
- 周期性 TX FIFO 空门限位 ptxfemplvl。

步骤 3 在 GINTMSK 寄存器中编程以下字段：

- 屏蔽 RX FIFO 非空中断，即 rxflvlmsk=0。

步骤 4 在 GUSBCFG 寄存器中编程以下字段：

- USB_FS 超时校准字段 toutcal；
- USB 周转时间字段 usbtrdtim。

步骤 5 软件必须使能 GINTMSK 寄存器中的以下位：

- 模式不匹配中断屏蔽 modemismsk。

步骤 6 通过读取 GINTSTS 中的 curmod 位，软件可确定 USB_FS 控制器是在主机模式还是设备模式下工作。

27.5.2 主机初始化

要将 USB_FS 控制器作为主机进行初始化，应用程序必须执行以下步骤：

步骤 1 编程 GINTMSK.prtintmsk=1，以打开该中断。

步骤 2 编程 HCFG 寄存器以选择 FS 主机。

步骤 3 将 HPRT.prtpwr 位编程为 1，给 USB 总线提供 VBUS。

步骤 4 等待 HPRT. prtconndet 中断。这表示某设备已连接到主机端口。

步骤 5 将 HPRT 中的 prtrst 位编程为 1，在 USB 总线上发出复位信号。

步骤 6 至少等待 10ms，以便完成复位过程。

步骤 7 将 HPRT 中的 prtrst 位编程为 0。

步骤 8 等待 HPRT 中的 prtenchng 中断。

步骤 9 读取 HPRT 中的 prtspd 位以获取枚举速度。

步骤 10 编程 HFIR 寄存器，以选择适当的 PHY 时钟。

步骤 11 编程 GRXFSIZ 寄存器以选择 RX FIFO 的大小。

步骤 12 编程 GNPTXFSIZ 寄存器，以选择用于非周期性通信事务的非周期性 TX FIFO 的大小和起始地址。

步骤 13 编程 HPTXFSIZ 寄存器，以选择用于周期性通信事务的周期性 TX FIFO 的大小和起始地址。

要与设备通信，系统软件必须初始化并使能至少一个通道。请参考 27.5.4 主机编程流程章节。

27.5.3 设备初始化

上电期间或者从主机模式切换为设备模式后，应用程序必须执行下列步骤来将模块作为设备进行初始化。

步骤 1 在 DCFG 寄存器中编程以下字段：

- 设备速度 devspd 位；
- 非零长度状态 OUT 握手信号 nzstsouthshk 位；
- 周期性帧间隔 perfrint 位。

步骤 2 将 DCTL 寄存器的 sftdiscon 位设置为 0，以便设备通过上拉电阻连接到 USB 总线上。

步骤 3 编程 GINTMSK 寄存器以使能以下中断：

- USB 复位；
- 枚举完成；
- 早期挂起；
- USB 挂起；
- SOF。

步骤 4 等待 GINTSTS 中的 usbrst 中断。这表示已在 USB 上检测到复位信号，复位过程自接收到此中断后约持续 10ms。一旦接收到这个中断，应用程序必须执行 27.5.5设备编程流程 章节中的 USB 复位初始化。

步骤 5 等待 GINTSTS 中的 enumdone 中断。此中断指示 USB 上复位过程结束。接收到此中断时，应用程序必须读取 DSTS 寄存器以确定枚举速度并执行 27.13.5 中的枚举完成时的端点初始化中所列的步骤。

此时，设备已准备好接受 SOF 数据包并在控制端点 0 上执行控制传输。

27.5.4 主机编程流程

通道初始化

应用程序必须初始化一个或多个通道，之后才能与所连接的设备通信。要初始化和使能通道，应用程序必须执行以下步骤：

步骤 1 编程 GINTMSK 寄存器以取消对以下位的中断屏蔽：

- 用于 OUT 事务的非周期性发送 FIFO 为空(在流水线事务级别工作且数据包计数字段编程值大于 1 时适用)；
- 用于 OUT 事务的非周期性发送 FIFO 为半空(在流水线事务级别工作且数据包计数字段编程值大于 1 时适用)。

步骤 2 编程 HAINTMSK 寄存器以使能所选通道中断。

步骤 3 编程 HCINTMSK 寄存器以使能主机通道中断寄存器中反映的和通信事务有关的中断。

步骤 4 编程所选通道的 HCTSIZn 寄存器，指定以字节为单位的总传输大小和包括短数据包在内的预期数据包个数。应用程序必须使用初始数据 PID(用于第一个 OUT 事务或预期从第一个 IN 事务获取)编程 PID 字段。

步骤 5 编程传输大小字段使得所选通道的传输大小为 MPS 的整数倍。如果 MPS 是 4 的倍数，即字对齐，则通道的传输大小为 n*MPS；如果 MPS 不是字对齐，则通道的传输大小为 n*(MPS + 4 – (MPS mod 4))。

步骤 6 编程所选通道的 HCCHARn 寄存器，指定设备的端点特性，例如类型、速度、方向等。(仅当应用程序准备好发送或接收数据包时，才能通过将通道使能位置 1 来使能通道)。

通道的停止

应用程序可以通过编程 HCCHARn 寄存器将 chdis 和 chena 字段置 1 来禁止任何通道。这会使 USB_FS 主机清空之前在该通道上发出的请求(如果有)并生成通道停止中断。应用程序在将通道重新分配给其他通信事务之前，必须等待 HCINTn 中的 chhltd 中断。USB_FS 主机不会中断已在 USB 上启动的通信事务。

当队列中有一个事项的时候，USB_FS 控制器产生 rxflvl 中断。应用程序必须读取 GRXSTSP 寄存器以产生 chhltd 中断。

出现以下任一情况时，应用程序将禁止通道：

1. 当一个非周期 IN 传输期间接收到 HCINTn.xfercompl 中断。
2. IN 或者 OUT 通道的 HCINTn 中接收到 stall、xacterr、bblerr 或者 datatglerr 中断。应用程序在接收到通道停止信号之前，必须能够接收相同通道的其他中断。
3. 当收到 GINTSTS.disconnint 中断。应用程序必须检查 HPRT.prtconnsts，因为当直接连接到主机的设备断开连接的时候，HPRT.prtconnsts 被复位。应用程序必须发起一个软件复位操作，以确保所有通道被清除。当设备重新连接的时候，主机必须产生一次 USB 复位操作。
4. 应用程序在正常传输完成之前中止传输。

操作模型

仔细选择周期性和非周期性请求队列深度，以匹配访问的周期性或非周期性端点数量。

非周期请求队列深度影响非周期传输的性能。队列越深(加上足够的 FIFO 大小)，控制器就能更频繁地将非周期传输流水线化。如果队列大小较小，则只有在空出队列空间时，控制器才能放入新请求。

控制器的周期请求队列深度对按计划执行定期传输至关重要。根据微帧中计划执行的周期传输数量选择周期队列深度。

- 写入发送 FIFO

应用程序对数据包执行最后一个 DWORD 写操作的同时，USB_FS 主机自动向周期性/非周期性请求队列写入一个条目(OUT 请求)。因此开始向发送 FIFO 写入数据之前，应用程序必须确保周期性/非周期性请求队列中至少有一个空闲空间。应用程序必须始终以 DWORD 形式向发送 FIFO 写入数据。如果数据包大小不是 DWORD 的整数倍，则应用程序必须将数据包填充到 DWORD 的整数倍。USB_FS 主机根据设定的最大数据包大小和传输大小确定实际数据包大小。

- 读取接收 FIFO

应用程序必须忽略除 IN 数据包(4'b0010)以外的所有数据包状态。

⚠注意

当进行 BULK OUT 传输时，必须保证正在进行的数据包完整地填充到 TX FIFO 中，然后才能处理 NAK 中断。推荐的编程流程为，在应用程序填充 TX FIFO 前关闭通道中断，当应用程序填充完整包数据到 TX FIFO 后再打开通道中断。

27.5.5 设备编程流程

USB 复位时的端点初始化

步骤 1 将所有 OUT 端点 nak 位置 1：

- DOEPCTLn.snak = 1(对于所有 OUT 端点)。

步骤 2 取消对以下中断位的屏蔽：

- DAINTMSK.inepmsk0 = 1(控制 IN 端点 0)；
- DAINTMSK.outepmsk0 = 1(控制 OUT 端点 0)；
- DOEPMSK.setupmsk = 1；
- DOEPMSK.xfercomplmsk = 1；

- DIEPMSK.xfercomplmsk = 1;
- DIEPMSK.timeoutmsk = 1。

步骤 3 为每个 FIFO 设置数据 FIFO RAM:

- 对 GRXFSIZ 寄存器进行编程，以能够接收控制传输的 OUT 数据和 SETUP 数据。如果未使能阈值，则该寄存器必须至少等于控制端点 0 的 1 个最大数据包大小+2 个字(用于控制 OUT 数据包的状态)+10 个字(用于 SETUP 数据包)。如果使能阈值，则该寄存器必须至少等于 2*(DTHRCTL.rxthrlen/4+1)+2 个字(用于控制 OUT 数据包的状态)+10 个字(用于 SETUP 数据包);
- 对 GNPTXFSIZ 寄存器进行编程(取决于所选的 FIFO 编号)，以能够发送控制 IN 数据。该寄存器至少必须等于控制端点 0 的 1 个最大数据包大小。

步骤 4 复位 DCFG 寄存器的 devaddr 字段。

步骤 5 对端点相关寄存器中的以下字段进行编程，以使控制 OUT 端点 0 接收 SETUP 数据包:

- DOEPTSIZ0.rxdpid = 3(为了接收最多 3 个连续 SETUP 数据包)。

枚举完成时的端点初始化

本节描述当检测到枚举完成中断时，应用程序必须执行的操作。

1. 在枚举完成中断中(GINTSTS.enumdone)，读取 DSTS 寄存器以确定设备的枚举速度。
2. 编程 DIEPCTL0.mps 字段以设置最大数据包大小。该步骤配置控制端点 0。控制端点的最大数据包大小取决于枚举速度。
3. 打开 SOF 中断

此时，设备已经准备好接收 SOF 数据包并配置为在控制端点 0 上处理控制传输。

收到 SetAddress 命令时的端点初始化

本节描述了应用程序在 SETUP 数据包中接收到 SetAddress 命令时必须执行的操作。

1. 使用在 SetAddress 命令中接收到的设备地址来对 DCFG 寄存器进行编程。
2. 对 USB_FS 控制器进行编程以发出状态阶段的 IN 数据包。

收到 SetConfiguration/SetInterface 命令时的端点初始化

本节描述了应用程序在 SETUP 包中接收 SetConfiguration 或 SetInterface 命令时必须执行的操作。

1. 接收到 SetConfiguration 命令时，应用程序必须对端点寄存器编程，以使用新配置

中有效端点的特性来配置这些端点寄存器。

2. 接收到 SetInterface 命令时，应用程序必须对命令指定的端点的端点寄存器进行编程。
3. 在先前配置或其它设置中有效的端点在新的配置或其它设置中无效。必须停用这些无效端点。
4. 使用 DAINTMSK 集群器打开有效端点的中断，屏蔽无效端点中断。
5. 为每个 FIFO 设置数据 FIFO RAM。
6. 配置完所有必需的端点后，应用程序必须对模块进行编程以发送状态阶段的 IN 数据包。

此时，设备模块已可以接收和发送任何类型的数据包。

端点激活

本节描述激活设备端点或者将现有设备端点配置为新类型所需的步骤。

步骤 1 在 DIEPCTLn 寄存器或者 DOEPCTLn 寄存器的以下字段中，对所需端点的特性进行编程：

- 最大数据包长；
- USB 活动端点位置 1；
- 端点初始数据同步位(对于中断和批量端点)；
- 端点类型；
- TX FIFO 编号。

步骤 2 激活端点后，USB_FS 控制器开始解码发送到该端点的令牌，并在收到令牌有效的情况下回复有效握手信号。

端点停用

本节描述停用现有端点所需的步骤。

步骤 1 在要停用的端点中，将 DIEPCTLn 寄存器或 DOEPCTLn 寄存器中的 USB 活动端点位清零。

步骤 2 停用端点后，USB_FS 控制器便会忽略发送到该端点的令牌，从而导致 USB 超时。

27.6 USB_FS 寄存器概览

芯片内置 1 路 USB_FS，USB_FS 控制器和 PHY 的基地址如下：

- USB_FS 控制器基地址为：0x63000000
- USB_FS PHY 基地址为：0x60088000

说明

寄存器中变量 n=0~11，m=0~5。

数据 FIFO 访问寄存器地址映射从 0x1000 偏移地址开始。这些寄存器在主机模式和设备模式下均可用，用于对给定方向的特定端点或通道的 FIFO 空间进行读写操作。如果主机通道类型为 IN，则只能对该通道上的 FIFO 进行读操作。同样地，如果主机通道类型为 OUT，则只能对该通道上的 FIFO 进行写操作。

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	GOTGCTL	0x0	32	RW	0x000D0000	OTG 控制和状态寄存器
2	GOTGINT	0x4	32	W1C	0x00000000	OTG 中断寄存器
3	GAHBCFG	0x8	32	RW	0x00000000	AHB 配置寄存器
4	GUSBCFG	0xC	32	RW	0x10001400	USB 配置寄存器
5	GRSTCTL	0x10	32	RW	0x80000000	复位寄存器
6	GINTSTS	0x14	32	W1C	0x04000020	中断寄存器
7	GINTMSK	0x18	32	RW	0x00000000	中断屏蔽寄存器
8	GRXSTSR	0x1C	32	RO	0x00000000	接收状态调试读取寄存器
9	GRXSTSP	0x20	32	RO	0x00000000	接收状态读取/弹出数据指针（POP）的 FIFO 空间大小寄存器
10	GRXFSIZ	0x24	32	RW	0x000005E0	接收 FIFO 空间大小寄存器
11	GNPTXFSIZ	0x28	32	RW	0x05E005E0	非周期性传输 FIFO 空间大小寄存器
12	GNPTXSTS	0x2C	32	RO	0x000805E0	非周期性传输 FIFO/队列状态寄存器
13	GSNPSID	0x40	32	RO	0x4F54430A	ID 寄存器
14	GHWCFG1	0x44	32	RO	0x00000000	用户硬件配置寄存器 1
15	GHWCFG2	0x48	32	RO	0x228ED562	用户硬件配置寄存器 2
16	GHWCFG3	0x4C	32	RO	0x05E004E8	用户硬件配置寄存器 3
17	GHWCFG4	0x50	32	RO	0x1610BA20	用户硬件配置寄存器 4

No.	Names	Offset Address	SIZE	R/W	Default	Description
18	GDFIFOCFG	0x5C	32	RW	0x05E005E0	全局 DFIFO 配置寄存器
19	HPTXFSIZ	0x100	32	RW	0x05E00BC0	主机定期发送 FIFO 大小寄存器
20	DIEPTXF1	0x104	32	RW	0x05E00BC0	设备 IN 端点 1 发送 FIFO 大小寄存器
21	DIEPTXF2	0x108	32	RW	0x05E011A0	设备 IN 端点 2 发送 FIFO 大小寄存器
22	DIEPTXF3	0x10C	32	RW	0x05E01780	设备 IN 端点 3 发送 FIFO 大小寄存器
23	DIEPTXF4	0x110	32	RW	0x05E01D60	设备 IN 端点 4 发送 FIFO 大小寄存器
24	DIEPTXF5	0x114	32	RW	0x05E02340	设备 IN 端点 5 发送 FIFO 大小寄存器
25	HCFG	0x400	32	RW	0x00000200	主机配置寄存器
26	HFIR	0x404	32	RW	0x0000EA60	主机帧间隔寄存器
27	HFNUM	0x408	32	RO	0x00003FFF	主机帧编号/剩余帧时间寄存器
28	HPTXSTS	0x410	32	RO	0x000805E0	主机周期发送 FIFO/队列状态寄存器
29	HAINT	0x414	32	RO	0x00000000	主机所有通道中断寄存器
30	HAINTMSK	0x418	32	RW	0x00000000	主机所有通道中断屏蔽寄存器
31	HPRT	0x440	32	RW	0x00000000	主机端口控制和状态寄存器
32	HCCHAR(n)	0x500+0x20*n	32	RW	0x00000000	主机通道(n)特征寄存器
33	HCINT(n)	0x508+0x20*n	32	W1C	0x00000000	主机通道(n)中断寄存器
34	HCINTMSK(n)	0x50C+0x20*n	32	RW	0x00000000	主机通道(n)中断屏蔽寄存器
35	HCTSIZ(n)	0x510+0x20*n	32	RW	0x00000000	主机通道(n)传输大小寄存器
36	DCFG	0x800	32	RW	0x08020000	设备配置寄存器
37	DCTL	0x804	32	RW	0x00000002	设备控制寄存器
38	DSTS	0x808	32	RO	0x00000002	设备状态寄存器
39	DIEPMSK	0x810	32	RW	0x00000000	设备 IN 端点公共中断屏蔽寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
40	DOEPMSK	0x814	32	RW	0x00000000	设备 OUT 端点公共中断屏蔽寄存器
41	DAINT	0x818	32	RO	0x00000000	设备所有端点中断寄存器
42	DAINTMSK	0x81C	32	RW	0x00000000	设备所有端点中断屏蔽寄存器
43	DTHRCTL	0x830	32	RO	0x00000000	设备阈值控制寄存器
44	DIEPEMPMSK	0x834	32	RW	0x00000000	设备 IN 端点 FIFO 空中断屏蔽寄存器
45	DIEPCTL(m)	0x900+0x20*m	32	RW	0x00008000	设备 IN 端点(m)控制寄存器
46	DIEPINT(m)	0x908+0x20*m	32	W1C	0x00000080	设备 IN 端点(m)中断寄存器
47	DIEPTSIZ(m)	0x910+0x20*m	32	RW	0x00000000	设备 IN 端点(m)传输大小寄存器
48	DTXFSTS(m)	0x918+0x20*m	32	RO	0x000005E0	设备 IN 端点(m)发送 FIFO 状态寄存器
49	DOEPCTL(m)	0xB00+0x20*m	32	RW	0x00008000	设备 OUT 端点(m)控制寄存器
50	DOEPINT(m)	0xB08+0x20*m	32	W1C	0x00000000	设备 OUT 端点(m)中断寄存器
51	DOEPTSIZ(m)	0xB10+0x20*m	32	RW	0x00000000	设备 OUT 端点(m)传输大小寄存器
52	PCGCCTL	0xE00	32	RW	0x00000000	功耗和时钟门控制寄存器
53	PCGCCTL1	0xE04	32	RW	0x00000000	功耗和时钟门控制寄存器 1
54	EP0/CH0 FIFO	0x1000	32	RW	0x00000000	端点 0/通道 0 的数据 FIFO 寄存器
55	EP1/CH1 FIFO	0x2000	32	RW	0x00000000	端点 1/通道 1 的数据 FIFO 寄存器
56	EP2/CH2 FIFO	0x3000	32	RW	0x00000000	端点 2/通道 2 的数据 FIFO 寄存器
57	EP3/CH3 FIFO	0x4000	32	RW	0x00000000	端点 3/通道 3 的数据 FIFO 寄存器
58	EP4/CH4 FIFO	0x5000	32	RW	0x00000000	端点 4/通道 4 的数据 FIFO 寄存器
59	EP5/CH5 FIFO	0x6000	32	RW	0x00000000	端点 5/通道 5 的数据 FIFO 寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
60	CH6 FIFO	0x7000	32	RW	0x00000000	通道 6 的数据 FIFO 寄存器
61	CH7 FIFO	0x8000	32	RW	0x00000000	通道 7 的数据 FIFO 寄存器
62	CH8 FIFO	0x9000	32	RW	0x00000000	通道 8 的数据 FIFO 寄存器
63	CH9 FIFO	0xA000	32	RW	0x00000000	通道 9 的数据 FIFO 寄存器
64	CH10 FIFO	0xB000	32	RW	0x00000000	通道 10 的数据 FIFO 寄存器
65	CH11 FIFO	0xC000	32	RW	0x00000000	通道 11 的数据 FIFO 寄存器
66	U2PHY_LS_CTRL	0x18	32	RW	0x00000000	USB_PHY LS 控制寄存器(使用 PHY 基地址)

27.7 USB_FS 寄存器

27.7.1 GOTGCTL

◆ **偏移地址：0x0　　复位值：0x000D0000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.										curmod	Res.			dbnctime	Res.
-	-	-	-	-	-	-	-	-	-	RO	-	-	-	RO	-
0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dbnceflterbypass	Res.														
RW	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:22]	-	保留。	0x0
curmod	[21]	RO	模式：主机和设备 指示当前模式。 0：设备模式 1：主机模式	0x0

Name	Bits	R/W	Description	Default
			注意：此寄存器字段的值只能在 PHY 时钟稳定后读取，或者如果使能了 utmi_iddig 滤波器，等待滤波器计时器停止后才能读到正确的值	
reserved	[20:18]	-	保留。	0x0
dbnctime	[17]	RO	模式：仅主机 长/短消抖时间，选择 utmi_iddig 滤波器消抖时间。 0：长消抖时间，用于物理连接（100 毫秒+2.5 微秒） 1：短的消抖时间，用于软连接（2.5 微秒）	0x0
reserved	[16]	-	保留。	0x1
dbncefltrbypass	[15]	RW	模式：主机和设备 utmi_iddig 去抖动滤波器旁路开关。 当旁路开关打开时，信号会绕过 utmi_iddig 信号的去抖动滤波器。 0：关闭 utmi_iddig 信号滤波器旁路 1：打开 utmi_iddig 信号滤波器旁路	0x0
reserved	[14:0]	-	保留。	0x0

27.7.2 GOTGINT

◆ **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:0]	-	保留。	0x0

27.7.3 GAHBCFG

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							ptxfemplvl	nptxfemplvl	Res.						glblintrmsk
-	-	-	-	-	-	-	RW	RW	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
ptxfemplvl	[8]	RW	模式：仅主机 周期性 TX FIFO 空门限。 选择何时触发控制器全局中断寄存器中的周期性 TX FIFO 空中断（GINTSTS.ptxfemp）。 0：周期性 TX FIFO 为半空时触发 GINTSTS.ptxfemp 中断。 1：周期性 TX FIFO 完全为空时触发 GINTSTS.ptxfemp 中断。	0x0
nptxfemplvl	[7]	RW	模式：主机和设备 非周期性 TX FIFO 空门限。 在主机模式，此位选择何时触发控制器全局中断寄存器中的非周期性 TX FIFO 空中断（GINTSTS.nptxfemp）。 在设备模式时，此位选择何时触发 in 端点 TX FIFO 空中断（DIEPINTn.txfemp）。 0： 主机模式：非周期性 TX FIFO 半空时触发 GINTSTS.nptxfemp 中断 设备模式：IN 端点 TX FIFO 半空时触发 DIEPINTn.txfemp 中断 1： 主机模式：非周期性 TX FIFO 全空时触发 GINTSTS.nptxfemp 中断 设备模式：IN 端点 TX FIFO 全空时触发 DIEPINTn.txfemp 中断	0x0
reserved	[6:1]	-	保留。	0x0
glblintrmsk	[0]	RW	模式：主机和设备 全局中断开关。 该位用于打开或关闭全局中断。中断状态寄存器由控制器进行更新，与此位的设置无关。 数值： 0：关闭全局中断 1：打开全局中断	0x0

27.7.4 GUSBCFG

- **偏移地址：0xC　　复位值：0x10001400**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	forcedevmode	forcehstmode	txenddelay	Res.											
-	RW	RW	RW	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		usbtrdtim				Res.			physel	Res.		phyiif	toutcal		
-	-	RW	RW	RW	RW	-	-	-	RW	-	-	RW	RW	RW	RW
0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31]	-	保留且必须保持默认值。	0x0
forcedevmode	[30]	RW	模式：主机和设备 强制设备模式。 向该位写入 1 将迫使控制器进入设备模式，而与 utmi_iddig 输入信号无关。 设置强制位后，应用程序必须等待至少 25 毫秒，更改才能生效。 0：正常模式 1：强制设备模式	0x0
forcehstmode	[29]	RW	模式：主机和设备 强制主机模式。 向该位写入 1 将强制控制器到主机模式，而与 utmi_idig 输入信号无关。 设置强制位后，应用程序必须等待至少 25 毫秒，更改才能生效。 0：正常模式 1：强制主机模式	0x0
txenddelay	[28]	RW	模式：仅限设备 将该位设置为 1 可使控制器在远程唤醒期间对 opmode 信号的操作遵循 txenddelay 时序（UTMI+规范 1.05 版本第 4.1.5 节）。 0：关闭 txenddelay(不建议使用此配置，仅用于调试目的) 1：打开 txenddelay	0x1
reserved	[27:14]	-	保留。	0x0
usbtrdtim	[13:10]	RW	模式：仅限设备 USB 周转时间。 以物理层时钟为单位设置周转时间。指定 MAC 请求到包 FIFO 控制器（PFC）从 DFIFO（SPRAM）获取数据的响应时间。 5：当 MAC 接口为 16 位 UTMI+时 9：当 MAC 接口为 8 位 UTMI+时	0x5

Name	Bits	R/W	Description	Default
reserved	[9:7]	-	保留。	0x0
physel	[6]	RW	模式：主机和设备 0：选择 USB2.0 高速 UTMI+接口 1：选择 USB1.0 全速串行接口（禁用）	0x0
reserved	[5:4]	-	保留。	0x0
phyif	[3]	RW	模式：主机和设备 应用程序使用此位来配置控制器，以支持具有 8 位或 16 位接口的 UTMI+ PHY。 0：8 位 1：16 位	0x0
toutcal	[2:0]	RW	模式：主机和设备 FS 超时校准。 该字段的 PHY 时钟数被添加到控制器的全速包间超时时长中，以校准由 PHY 引入的任何额外延迟。 全速模式的 USB 标准超时值为 16 至 18（包括 16 至 18）位时间。应用程序必须根据枚举的速度对该字段进行编程。 每个 PHY clock 添加的位时间数如下： 全速模式： 一个 30 MHz PHY clock=0.4 位时间 一个 60 MHz PHY clock=0.2 位时间 0：加 0 个 PHY clocks 1：加 1 个 PHY clocks 2：加 2 个 PHY clocks 3：加 3 个 PHY clocks 4：加 4 个 PHY clocks 5：加 5 个 PHY clocks 6：加 6 个 PHY clocks 7：加 7 个 PHY clocks	0x0

27.7.5 GRSTCTL

- **偏移地址：0x10　　复位值：0x80000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ahbidle	Res.	csftrstdone	Res.												
RO	-	W1C	-	-	-	-	-	-	-	-	-	-	-	-	-
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					txfnum					txfflsh	rxfflsh	Res.	frmcntrrst	piufssftrst	csftrst
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ahbidle	[31]	RO	模式：主机和设备 指示 AHB 主状态机是否处于空闲状态。 0：非空闲 1：空闲	0x1
reserved	[30]	-	保留。	0x0
csftrstdone	[29]	W1C	模式：主机和设备 内核软重置完成。 当控制器中所有必要的逻辑复位完成后，控制器会设置此位。此位应与 GRSTCTL.csftrst 一起被应用程序清除。 0：无软复位 1：软复位完成	0x0
reserved	[28:11]	-	保留。	0x0
txfnum	[10:6]	RW	模式：主机和设备 选择将要由 GRSTCTL.txfflsh 清除的 TX FIFO 的编号，在控制器清除 GRSTCTL.txfflsh 之前，不得更改此字段。 0： 主机模式下选择非周期性 TX FIFO 设备模式下选择 TX FIFO 0 1： 主机模式下选择非周期性 TX FIFO 设备模式下选择 TX FIFO 1 2： 设备模式下选择 TX FIFO 2 3： 设备模式下选择 TX FIFO 3 4： 设备模式下选择 TX FIFO 4 5： 设备模式下选择 TX FIFO 5 16：	0x0

Name	Bits	R/W	Description	Default
			设备或主机模式下选择所有 TX FIFO 注意：配置其他值无效	
txfflsh	[5]	RW	模式：主机和设备 该位有选择地刷新单个或所有传输 FIFO，但如果控制器处于事务中，则不能这样做。 只有在检查内核既没有写入 TX FIFO 也没有从 TX FIFO 读取之后，应用程序才能写入此位。 使用以下寄存器进行验证： 读 GINNakEff 中断可确保模块当前未对 FIFO 执行读操作；读 GRSTCTL.ahbidle 可确保内核当前未对 FIFO 执行任何写操作。 当 FIFO 被重新配置或在设备端点禁用期间，建议进行 TX FIFO 刷新。在执行任何操作之前，应用程序必须等待控制器清除此位。此位需要八个时钟才能清除，使用 phy_clk 或 hclk 的较慢时钟。 0：不刷新 TX FIFO 1：选择性刷新单个或所有 TX FIFO	0x0
rxfflsh	[4]	RW	模式：主机和设备 RX FIFO 刷新（RxFFlsh） 应用程序可以使用此位刷新整个 RX FIFO，但必须首先确保内核不在 USB 事务的中间。 只有在检查控制器既没有从 RX FIFO 读取也没有写入 RX FIFO 之后，应用程序才能写入该位。 在执行任何其他操作之前，应用程序必须等待该位被控制器清除。此位需要八个时钟（PHY 或 AHB 时钟中最慢的时钟）才能清除。 0：不刷新整个 RX FIFO 1：刷新整个 RX FIFO	0x0
reserved	[3]	-	保留。	0x0
frmcntrrst	[2]	RW	模式：仅主机 应用程序写入此位以重置控制器中的帧数计数器。当帧计数器复位时，控制器发出的后续 SOF 的（微）帧数为 0。 当应用程序向该位写入 1 时，它可能无法读回该值，因为它会在几个时钟周期内被控制器清除。 0：无复位 1：主机帧计数器复位	0x0
piufssftrst	[1]	RW	模式：主机和设备 PIU FS 专用控制器软复位控制。 复位 PIU FS 专用控制器，PIU 的 FS 专用控制器中的所有模块状态机都重置为 IDLE 状态，以防出现任何 PHY 错误，如活动丢失或 Babble 错误。	0x0

Name	Bits	R/W	Description	Default
			这是一个自清除位，在控制器中所有必要的逻辑复位后，控制器自动清除该位。 0：无复位 1：PIU FS 专用控制器软复位	
csftrst	[0]	RW	模式：主机和设备 控制器软复位。 按如下方式重置 hclk 和 phy_clock 域： (1)清除中断和除以下寄存器位以外的所有状态寄存器： PCGCCTL.rstpownmodule GUSBCFG.forcedev GUSBCFG.forcehst GUSBCFG.phylpwrclksel GUSBCFG.ddrsel GUSBCFG.physel GUSBCFG.fsintf GUSBCFG.ulpi_utmi GUSBCFG.phyif GUSBCFG.txenddelay HCFG.fslsp DCFG.devspd DCTL.sftdiscon (2)所有模块状态机 (3)所有模块状态机（AHB 从机除外）均重置为 IDLE 状态，所有 TX FIFO 和 RX FIFO 均被刷新。 (4)在顺利完成 AHB 传输的最后一个数据阶段后，AHB 主机上的任何传输都会尽快终止,USB 上的任何 usb 事务都会立即终止。 应用程序可以在任何时候写入该位，以重置控制器。 应用程序必须在检查到此寄存器的第 29bit 之后清除此位。 应用程序在开始任何操作之前，还必须检查该寄存器的第 31 位是否为 1。 通常在更改 PHY 时钟或者 PHY 类型后，就必须重置 PHY 域。 0：无复位 1：复位 hclk 和 phy_clock 域	0x0

27.7.6 GINTSTS

◆ **偏移地址：0x14　　复位值：0x04000020**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
wkupint	Res.	disconnint	Res.		ptxfemp	hchint	prtint	resetdet	Res.	incomplp	incompisoin	oepint	iepint	Res.	
W1C	-	W1C	-	-	RO	RO	RO	W1C	-	W1C	W1C	RO	RO	-	-
0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
eopf	isoouterop	enumdone	usbrst	usbsusp	erlysusp	Res.		goutnakeff	ginnakeff	nptxfemp	rxflvl	sof	Res.	modemis	curmod
W1C	W1C	W1C	W1C	W1C	W1C	-	-	RO	RO	RO	RO	W1C	-	W1C	RO
0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
wkupint	[31]	W1C	模式：主机和设备 检测到恢复/远程唤醒中断 。 挂起（L2）期间： 在设备模式下，USB 检测到恢复信号时，将触发该中断。在主机模式下，USB 检测到远程唤醒时，将触发该中断。 LPM(L1）期间： USB 检测到主机发起的恢复信号或设备发起的远程唤醒时，将触发该中断。 0：无中断 1：有中断	0x0
reserved	[30]	-	保留。	0x0
disconnint	[29]	W1C	模式：仅主机 检测到设备断开连接时触发此中断。 0：无中断 1：有中断	0x0
reserved	[28:27]	-	保留。	0x0
ptxfemp	[26]	RO	模式：仅主机 当周期性 TX FIFO 为半空或全空状态，且周期性请求队列中存在可写入至少一个条目的空间时，将触发该中断。该周期性 TX FIFO 为半空状态还是全空状态由 GAHBCFG.ptxfelvl 决定。 0：无中断 1：有中断	0x1
hchint	[25]	RO	模式：仅主机 模块将该位置 1 时，指示模块中一个通道上存在中断（在主机模式下）。应用程序必须 读取 HAINT 寄存器，以确定发生中断的通道的准确编号，然后读取相应的 HCINTx 寄存器，以确定引发中断的确切原因。应用程序必须先将 HCINTx 寄存器的相应状态位清零，之后才能将该位清零。	0x0

Name	Bits	R/W	Description	Default
			0：无中断 1：有中断	
prtint	[24]	RO	模式：仅主机 控制器将此位设置为指示主机模式下某个控制器端口的端口状态发生变化。应用程序必须读取主机端口控制和状态（HPRT）寄存器，以确定导致此中断的确切事件。应用程序必须清除主机端口控制和状态寄存器中的相应状态位才能清除此位。 0：无中断 1：有中断	0x0
resetdet	[23]	W1C	模式：仅限设备 重置检测到的中断。 在设备模式下，当设备处于挂起状态时，在部分断电模式下 USB 上检测到复位时，会触发此中断。 在主机模式下，此中断不会被触发。 0：无中断 1：有中断	0x0
reserved	[22]	-	保留。	0x0
incomplp	[21]	W1C	模式：仅主机 未完成周期性传输。 在主机模式下，当有未完成的周期性事务仍在等待时，而这些事务计划在当前帧期间完成时控制器会设置此位。 模式：仅限设备 未完成同步 OUT 传输。 在设备模式下，模块将该中断置 1 时，指示当前帧中至少有一个同步 OUT 端点上的传输未完成。该中断随该寄存器中的周期性帧结束中断（EOPF）位一同触发。 0：无中断 1：有中断	0x0
incompisoin	[20]	W1C	模式：仅限设备 未完成同步 IN 传输。 模块将该中断置 1 时，指示当前帧中至少有一个同步 IN 端点上的传输未完成。该中断随该寄存器中的周期性帧结束中断(EOPF) 位一同触发。 0：无中断 1：有中断	0x0
oepint	[19]	RO	模式：仅限设备 OUT 端点中断。 模块将该位置 1 时，指示模块中一个 OUT 端点上存在挂起的中断（在设备模式下）。应用 程序必须	0x0

Name	Bits	R/W	Description	Default
			读取 DAINT 寄存器，以确定发生中断的 OUT 端点的准确编号，然后读取相应的 DOEPINTx 寄存器，以确定引发中断的确切原因。应用程序必须先将相应 DOEPINTx 寄存器的相应状态位清零，之后才能将该位清零。 0：无中断 1：有中断	
iepint	[18]	RO	模式：仅限设备 IN 端点中断。 模块将该位置 1 时，指示模块中一个 IN 端点上存在挂起的中断（在设备模式下）。应用程序必须读取 DAINT 寄存器，以确定发生中断的 IN 端点的准确编号，然后读取相应的 DIEPINTx 寄存器，以确定引发中断的确切原因。应用程序必须先将相应 DIEPINTx 寄存器的相应状态位清零，之后才能将该位清零 0：无中断 1：有中断	0x0
reserved	[17:16]	-	保留。	0x0
eopf	[15]	W1C	模式：仅限设备 周期性帧中断结束。 指示在当前帧已达到设备配置寄存器（DCFG.perfrint）的周期性帧间隔字段中指定的周期。 0：无中断 1：有中断	0x0
isooutdrop	[14]	W1C	模式：仅限设备 丢弃同步 OUT 数据包中断。 如果由于 RX FIFO 空间不足，无法容纳同步 OUT 端点的最大数据包，从而导致模块无法向 RX FIFO 写入同步 OUT 数据包，模块会将触发此中断。 0：无中断 1：有中断	0x0
enumdone	[13]	W1C	模式：仅限设备 枚举完成中断。 控制器设置此位以指示速度枚举已完成。应用程序必须读取设备状态（DSTS）寄存器才能获得枚举的速度。 0：无中断 1：有中断	0x0
usbrst	[12]	W1C	模式：仅限设备 控制器设置该位以指示在 USB 上检测到复位。	0x0

Name	Bits	R/W	Description	Default
			0：无中断 1：有中断	
usbsusp	[11]	W1C	模式：仅限设备 USB 挂起。 控制器设置此位以指示在USB上检测到挂起。当线路状态信号在延长的一段时间没有活动时，控制器进入暂停状态。 0：无中断 1：有中断	0x0
erlysusp	[10]	W1C	模式：仅限设备 控制器设置该位以指示在 USB 上检测到空闲状态达3ms。 0：无中断 1：有中断	0x0
reserved	[9:8]	-	保留。	0x0
goutnakeff	[7]	RO	模式：仅限设备 控制器设置该位表示由应用程序设置的设备控制寄存器（DCTL.sgoutnak）中的设置全局 OUT NAK 位已在内核中生效。该位可以通过在设备控制寄存器（DCTL.cgoutnak）中写入清除全局 OUT NAK 位来清除。 0：无中断 1：有中断	0x0
ginnakeff	[6]	RO	模式：仅限设备 控制器设置该位指示应用程序设置的设备控制寄存器（DCTL.sgninnak）中的设置全局非周期性 IN NAK 位已在内核中生效。也就是说，控制器已经对应用程序设置的全局 IN NAK 比特进行了采样。此位可以通过清除设备控制寄存器（DCTL.cgninnak）中的清除全局非周期 IN NAK 位来清除。此中断并不一定意味着在 USB 上发送 NAK 握手。STALL 位的优先级高于 NAK 位。 0：无中断 1：有中断	0x0
nptxfemp	[5]	RO	模式：主机和设备 当非周期性 TX FIFO 为半空或完全空，并且有至少一个条目写入非周期性传输请求队列的空间时，该中断被触发。半空或完全空状态由控制器 AHB 配置寄存器（GAHBCFG.nptxfemplvl）中的非周期性 TX FIFO 空电平位确定。 0：无中断 1：有中断	0x1

Name	Bits	R/W	Description	Default
rxflvl	[4]	RO	模式：主机和设备 控制器设置该位表示至少有一个数据包待从 RX FIFO 读取。 0：无中断 1：中断	0x0
sof	[3]	W1C	模式：主机和设备 在主机模式下，控制器设置此位以指示 SOF（FS）或保持活动（LS）在 USB 上传输。应用程序必须向该位写入 1 以清除中断。 在设备模式下，控制器设置此位以指示已在 USB 上接收到 SOF 令牌。应用程序可以读取设备状态寄存器以获得当前帧号。只有当控制器在 HS 或 FS 下运行时，才会看到这种中断。此位只能由控制器设置，应用程序必须写入 1 才能清除它。 注意：如果在通电复位后立即读取，此寄存器可能会返回 1。如果寄存器位在上电复位后立即读取是 1，则它并不表示 SOF 已经发送（在主机模式的情况下）或 SOF 已经接收（在设备模式的情况）。只有在主机和设备之间建立有效连接后，此中断的读取值才有效。如果该位是在通电复位后设置的，则应用程序可以清除该位。 0：无中断 1：有中断	0x0
reserved	[2]	-	保留。	0x0
modemis	[1]	W1C	模式：主机和设备 当应用程序尝试访问以下寄存器时，模块将该位置 1： （1）模块运行在设备模式下访问主机模式寄存器 （2）模块运行在主机模式下访问设备模式寄存器 0：无中断 1：有中断	0x0
curmod	[0]	RO	模式：主机和设备 指示当前模式。 0：设备模式 1：主机模式 注意：此寄存器字段的值只能在 PHY 时钟稳定后读取，或者如果启用了 IDDIG_FILTER，请等待过滤器计时器到期以读取正确的值	0x0

27.7.7 GINTMSK

◆ **偏移地址：0x18　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
wkupintmsk	Res.	disconnintmsk	Res.		ptxfempmsk	hchintmsk	prtintmsk	resetdetmsk	Res.	incomplpmsk	Res.	oepintmsk	iepintmsk	Res.	
RW 0	- 0	RW 0	- 0	- 0	RW 0	RW 0	RW 0	RW 0	- 0	RW 0	- 0	RW 0	RW 0	- 0	- 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
eopfmsk	isooutdropmsk	enumdonemsk	usbrstmsk	usbsuspmsk	erlysuspmsk	Res.		goutnakeffmsk	ginnakeffmsk	nptxfempmsk	rxflvlmsk	sofmsk	Res.	modemismsk	Res.
RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	- 0	- 0	RW 0	RW 0	RW 0	RW 0	RW 0	- 0	RW 0	- 0

Name	Bits	R/W	Description	Default
wkupintmsk	[31]	RW	模式：主机和设备 检测到恢复/远程唤醒中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[30]	-	保留。	0x0
disconnintmsk	[29]	RW	模式：主机和设备 断开连接中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[28:27]	-	保留。	0x0
ptxfempmsk	[26]	RW	模式：仅主机 周期性 TX FIFO 空中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
hchintmsk	[25]	RW	模式：仅主机 主机通道中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
prtintmsk	[24]	RW	模式：仅主机 主机端口中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
resetdetmsk	[23]	RW	模式：仅限设备 检测到复位的中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[22]	-	保留。	0x0
incomplpmsk	[21]	RW	模式：仅主机 未完成的周期传输中断屏蔽。 模式：仅限设备 未完成的同步 OUT 传输中断中断屏蔽。	0x0

Name	Bits	R/W	Description	Default
			0：屏蔽中断 1：使能中断	
reserved	[20]	-	保留。	0x0
oepintmsk	[19]	RW	模式：仅限设备 OUT 端点中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
iepintmsk	[18]	RW	模式：仅限设备 IN 端点中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[17:16]	-	保留。	0x0
eopfmsk	[15]	RW	模式：仅限设备 周期性帧中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
isooutdropmsk	[14]	RW	模式：仅限设备 丢弃同步 OUT 数据包丢弃中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
enumdonemsk	[13]	RW	模式：仅限设备 枚举完成中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
usbrstmsk	[12]	RW	模式：仅限设备 USB 复位中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
usbsuspmsk	[11]	RW	模式：仅限设备 USB 挂起中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
erlysuspmsk	[10]	RW	模式：仅限设备 早期挂起中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[9:8]	-	保留。	0x0
goutnakeffmsk	[7]	RW	模式：仅限设备 全局 OUT NAK 有效中断屏蔽。 0：屏蔽中断 1：使能中断	0x0

Name	Bits	R/W	Description	Default
ginnakeffmsk	[6]	RW	模式：仅限设备， 全局非周期 IN NAK 有效中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
nptxfempmsk	[5]	RW	模式：主机和设备 非周期性 TX FIFO 空中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
rxflvlmsk	[4]	RW	模式：主机和设备 接收 FIFO 非空中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
sofmsk	[3]	RW	模式：主机和设备 开始帧中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[2]	-	保留。	0x0
modemismsk	[1]	RW	模式：主机和设备 模式不匹配中断中断屏蔽。 0：屏蔽中断 1：使能中断	0x0
reserved	[0]	-	保留。	0x0

27.7.8 GRXSTSR

◆ **偏移地址：0x1C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.							fn				pktsts				dpid
-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

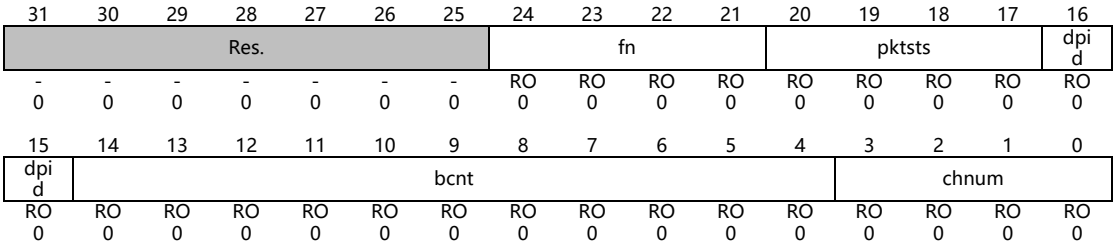
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
dpid	bcnt											chnum			
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
fn	[24:21]	RO	模式：仅限设备 帧编号（FN）。 这是在 USB 上接收数据包的帧号的最低有效 4 位。 只有当支持同步 OUT 端点时，才支持此字段。	0x0
pktsts	[20:17]	RO	模式：主机和设备	0x0

Name	Bits	R/W	Description	Default
			指示包状态。 在主机模式中： 2：接收到 IN 数据包 3：IN 传输完成（触发中断） 5：数据切换错误（触发中断） 7：通道暂停（触发中断） 其他：保留 在设备模式中： 1：全局输出 NAK（触发中断） 2：接收到 OUT 数据包 3：OUT 传输完成（触发中断） 4：SETUP 事务完成（触发中断） 6：接收到 SETUP 数据包 其他：保留	
dpid	[16:15]	RO	数据 PID。 在主机模式下，指示接收到的数据包的数据 PID。在设备模式中，表示接收到的 OUT 数据包的数据 PID。 0：DATA0 1：DATA2 2：DATA1	0x0
bcnt	[14:4]	RO	字节计数（BCnt）。 在主机模式下，表示接收到的 In 数据包的字节计数。 在设备模式下，表示接收到的数据包的字节数。	0x0
chnum	[3:0]	RO	主机模式下： 指示当前接收到的数据包所属的信道号。 设备模式下： 指示当前接收到的数据包所属的端点编号。 0：通道或端点 0 1：通道或端点 1 2：通道或端点 2 3：通道或端点 3 4：通道或端点 4 5：通道或端点 5 6：通道 6 7：通道 7 8：通道 8 9：通道 9 10：通道 10 11：通道 11	0x0

27.7.9 GRXSTSP

◆ **偏移地址：0x20　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:25]	-	保留。	0x0
fn	[24:21]	RO	模式：仅限设备 帧编号。 这是在 USB 上接收数据包的帧号的最低有效 4 位。 只有当支持同步 OUT 端点时，才支持此字段。	0x0
pktsts	[20:17]	RO	指示所接收的包的状态。 在主机模式中， 2：接收到 IN 数据包 3：IN 传输完成（触发中断） 5：数据切换错误（触发中断） 7：通道暂停（触发中断） 其他：保留 在设备模式中， 1：全局输出 NAK（触发中断） 2：接收到 OUT 数据包 3：OUT 传输完成（触发中断） 4：SETUP 事务完成（触发中断） 6：接收到 SETUP 数据包 其他：保留	0x0
dpid	[16:15]	RO	数据 PID。 在主机模式下，指示接收到的数据包的数据 PID。在设备模式中，表示接收到的 OUT 数据包的数据 PID。 0：DATA0 1：DATA2 2：DATA1	0x0
bcnt	[14:4]	RO	字节计数。 在主机模式下，表示接收到的 In 数据包的字节计数。 在设备模式下，表示接收到的数据包的字节数。	0x0
chnum	[3:0]	RO	主机模式下： 指示当前接收到的数据包所属的信道号。	0x0

Name	Bits	R/W	Description	Default
			设备模式下： 指示当前接收到的数据包所属的端点编号。 0：通道或端点 0 1：通道或端点 1 2：通道或端点 2 3：通道或端点 3 4：通道或端点 4 5：通道或端点 5 6：通道 6 7：通道 7 8：通道 8 9：通道 9 10：通道 10 11：通道 11	

27.7.10 GRXFSIZ

◆ **偏移地址：0x24　　复位值：0x000005E0**

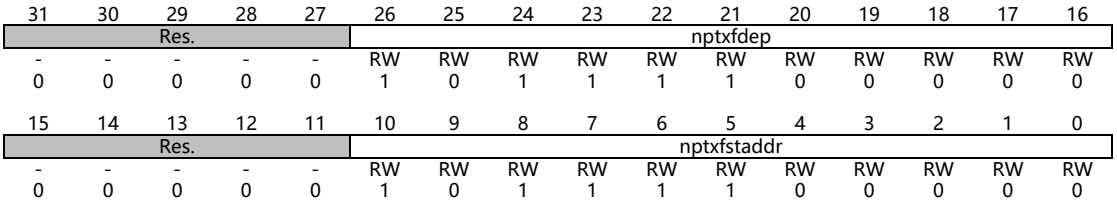
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					rxfdep										
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
rxfdep	[10:0]	RW	模式：主机和设备 RX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，最小值为 16，最大值为 0x5e0。 此寄存器的上电复位值在配置期间被指定为最大 Rx 数据 FIFO 深度。	0x5e0

27.7.11 GNPTXFSIZ

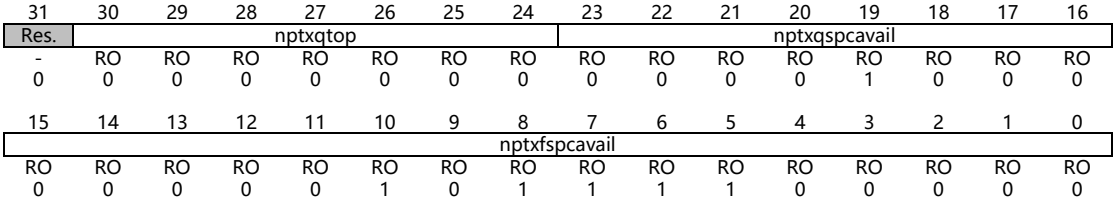
◆ **偏移地址：0x28　　复位值：0x05E005E0**



Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
nptxfdep	[26:16]	RW	模式：仅主机 非周期性 TX FIFO 深度。 对于主机模式，此字段始终有效。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 应用程序可以在此字段中写入新值。	0x5e0
reserved	[15:11]	-	保留。	0x0
nptxfstaddr	[10:0]	RW	非周期性传输 RAM 起始地址。 对于主机模式，此字段始终有效。 此字段包含非周期传输 FIFO RAM 的存储器起始地址。 应用程序可以在此字段中写入新值。	0x5e0

27.7.12 GNPTXSTS

◆ **偏移地址：0x2C　　复位值：0x000805E0**



Name	Bits	R/W	Description	Default
reserved	[31]	-	保留。	0x0
nptxqtop	[30:24]	RO	非周期性传输请求队列的顶部。 MAC 当前正在处理的非周期性 Tx 请求队列中的条目。 Bit[30:27]：通道/端点编号 Bit[26:25]： 0：IN/OUT 令牌 1：零长度传输数据包（设备输入/主机输出） 2：PING/CSPLIT 令牌	0x0

Name	Bits	R/W	Description	Default
			3：通道停止命令 Bit[24]：终止（最后所选的通道/端点）	
nptxqspcavail	[23:16]	RO	非周期性传输请求队列可用空间。 此队列在主机模式下同时保存 IN 和 OUT 请求。设备模式只有 IN 请求。 0：非周期性传输请求队列已满 1：1 个可用位置 2：2 个可用位置 3：3 个可用位置 4：4 个可用位置 5：5 个位置可用 6：6 个可用位置 7：7 个可用位置 8：8 个位置可用	0x8
nptxfspcavail	[15:0]	RO	非周期性 TX FIFO 可用空间。 值以 32 bit 的 word 为单位。 0：非周期性 TX FIFO 已满 1：剩余 1 个 word 可用空间 2：剩余 2 个 word 可用空间 n：剩余 n 个 word 可用空间（其中 0<=n<=0x5e0）	0x5e0

27.7.13 GSNPSID

◆ **偏移地址：0x40　　复位值：0x4F54430A**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
synopsysid															
RO 0	RO 1	RO 0	RO 0	RO 1	RO 1	RO 1	RO 1	RO 0	RO 1	RO 0	RO 1	RO 0	RO 1	RO 0	RO 0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
synopsysid															
RO 0	RO 1	RO 0	RO 0	RO 0	RO 0	RO 1	RO 1	RO 0	RO 0	RO 0	RO 0	RO 1	RO 0	RO 1	RO 0

Name	Bits	R/W	Description	Default
synopsysid	[31:0]	RO	当前正在使用的控制器的版本号。 Bits[31:16]：0x4f54:OT 的 ASCII 值 Bits[15:0]：当前版本号。例如，0x400a 对应于 v4.00a 版本号。	0x4f54430a

27.7.14 GHWCFG1

◆ **偏移地址：0x44　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
epdir															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
epdir															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
epdir	[31:0]	RO	每个端点使用两 bit 来表示端点方向。 位[31:30]：端点 15 方向 位[29:28]：端点 14 方向 ... 位[3:2]：端点 1 方向 位[1:0]：端点 0 方向（始终为 BIDIR） 0：双向端点 1：IN 端点 2：OUT 端点 3：保留	0x0

27.7.15 GHWCFG2

◆ **偏移地址：0x48　　复位值：0x228ED562**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	tknqdepth					ptxqdepth		nptxqdepth		Res.	multiprocintrpt	dynfifosizing	periosupport	numhstchnl	
-	RO	RO	RO	RO	RO	RO	RO	RO	RO	-	RO	RO	RO	RO	RO
0	0	1	0	0	0	1	0	1	0	0	0	1	1	1	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
numhstchnl		numdeveps				fsphytype		hsphytype		singpnt	otgarch		otgmode		
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
1	1	0	1	0	1	0	1	0	1	1	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31]	-	保留。	0x0
tknqdepth	[30:26]	RO	设备模式 IN 令牌队列深度。 范围：0-30	0x8
ptxqdepth	[25:24]	RO	主机模式周期性请求队列深度。 0：周期性请求队列深度为 2 1：周期性请求队列深度为 4 2：周期性请求队列深度为 8 3：周期性请求队列深度为 16	0x2
nptxqdepth	[23:22]	RO	非周期性请求队列深度。	0x2

Name	Bits	R/W	Description	Default
			0：非周期性请求队列深度为 2 1：非周期性请求队列深度为 4 2：非周期性请求队列深度为 8 其他：保留	
reserved	[21]	-	保留。	0x0
multiprocintrpt	[20]	RO	启用多处理器中断。 0：未启用多处理器中断 1：多处理器中断已启用	0x0
dynfifosizing	[19]	RO	启用动态 FIFO 大小调整。 0：禁用动态 FIFO 大小 1：启用动态 FIFO 大小	0x1
periosupport	[18]	RO	主机模式下支持的周期性 OUT 通道。 0：主机模式下不支持周期性 OUT 通道 1：主机模式下支持周期性 OUT 通道	0x1
numhstchnl	[17:14]	RO	指示控制器在主机模式下支持的通道数。 0：1 个通道 1：2 个通道 2：3 个通道 3：4 个通道 4：5 个通道 5：6 个通道 6：7 个通道 7：8 个通道 8：9 个通道 9：10 个通道 10：11 个通道 11：12 个通道 12：13 个通道 13：14 个通道 14：15 个通道 15：16 个通道	0xb
numdeveps	[13:10]	RO	指示控制器在设备模式下支持的最大端点号。 0：最大端点号是 0 1：最大端点号是 1 2：最大端点号是 2 3：最大端点号是 3 4：最大端点号是 4 5：最大端点号是 5	0x5
fsphytype	[9:8]	RO	全速 PHY 接口类型。 0：不支持全速接口 1：专用全速接口	0x1

Name	Bits	R/W	Description	Default
			2：全速与 UTMI+共用管脚的接口 3：全速与 ULPI 共用管脚的接口	
hsphytype	[7:6]	RO	高速 PHY 接口类型。 0：不支持高速接口 1：UTMI+ 2：ULPI 3：UTMI+ULPI	0x1
singpnt	[5]	RO	点对点模式。 0：多点模式，支持 HUB 和 SPLIT 1：单点模式，不支持 HUB 和 SPLIT	0x1
otgarch	[4:3]	RO	架构模式。 0：SLAVE 模式 1：外部 DMA 模式 2：内部 DMA 模式 3：保留	0x0
otgmode	[2:0]	RO	工作模式。 2：无 HNP 和无 SRP 功能的 OTG 其他：保留	0x2

27.7.16 GHWCFG3

◆ **偏移地址：0x4C　　复位值：0x05E004E8**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
dfifodepth															
RO 0	RO 0	RO 0	RO 0	RO 0	RO 1	RO 0	RO 1	RO 1	RO 1	RO 1	RO 0	RO 0	RO 0	RO 0	RO 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
lpmmode	bcsupport	hsicmode	adpsupport	rsttype	optfeature	vndctlsupt	i2cintsel	Res.	pktsizewidth			xfersizewidth			
RO 0	RO 0	RO 0	RO 0	RO 0	RO 1	RO 0	RO 0	- 1	RO 1	RO 1	RO 0	RO 1	RO 0	RO 0	RO 0

Name	Bits	R/W	Description	Default
dfifodepth	[31:16]	RO	FIFO 深度。 以 32bit 的 word 为单位，最小值为 32，最大值为 32768。	0x5e0
lpmmode	[15]	RO	LPM 模式。 0：LPM 已禁用 1：LPM 已启用	0x0
bcsupport	[14]	RO	电池充电支持。 0：不支持电池充电 1：支持电池充电	0x0
hsicmode	[13]	RO	HSIC 模式。	0x0

Name	Bits	R/W	Description	Default
			0：不支持 HSIC 1：支持 HSIC，与 UTMI PHY 共享接口	
adpsupport	[12]	RO	该位指示 ADP 逻辑存在于控制器内部或外部。 0：ADP 逻辑与控制器不在一起 1：ADP 逻辑与控制器在一起	0x0
rsttype	[11]	RO	控制器逻辑设计中的复位类型。 0：异步复位 1：同步复位	0x0
optfeature	[10]	RO	可选特性移除。 指示是否已通过删除用户 ID 寄存器、GPIO 接口端口、SOF 切换和计数器端口，以优化门计数。 0：否 1：是	0x1
vndctlsupt	[9]	RO	厂商控制接口。 0：不支持 1：支持	0x0
i2cintsel	[8]	RO	I2C 接口。 0：不支持 I2C 接口 1：支持 I2C 接口	0x0
reserved	[7]	-	保留。	0x1
pktsizewidth	[6:4]	RO	包大小计数器的宽度。 0：4 bits 1：5 bits 2：6 bits 3：7 bits 4：8 bits 5：9 bits 6：10 bits 其他：保留	0x6
xfersizewidth	[3:0]	RO	传输大小计数器的宽度。 0：11 bits 1：12 bits 2：13 bits 3：14 bits 4：15 bits 5：16 bits 6：17 bits 7：18 bits 8：19 bits 其他：保留	0x8

27.7.17 GHWCFG4

◆ **偏移地址：0x50　　复位值：0x1610BA20**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.		ineps				dedfifomode	sessendfltr	bvalidflt r	avalidfltr	vbusvalidfltr	iddgfltr	numctleps			
-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	1	0	1	1	0	0	0	0	1	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
phydatawidth		Res.	acgsupt	Res.					hibernation	ahbfreq	partialpwrdn	numdevperioeps			
RO	RO	-	RO	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO
1	0	1	1	1	0	1	0	0	0	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:30]	-	保留。	0x0
ineps	[29:26]	RO	设备模式 IN 端点的数量，包括控制端点。 0：1 个 IN 端点 1：2 个 IN 端点 2：3 个 IN 端点 3：4 个 IN 端点 4：5 个 IN 端点 5：6 个 IN 端点	0x5
dedfifomode	[25]	RO	设备 IN 端点启用专用传输 FIFO。 0：不启用专用 FIFO 1：启用专用 FIFO	0x1
sessendfltr	[24]	RO	session_end 滤波器使能。 0：不使能 1：使能	0x0
bvalidfltr	[23]	RO	b_valid 滤波器使能。 0：不使能 1：使能	0x0
avalidfltr	[22]	RO	a_valid 滤波器使能。 0：不使能 1：使能	0x0
vbusvalidfltr	[21]	RO	VBUS_valid 滤波器使能。 0：不使能 1：使能	0x0
iddgfltr	[20]	RO	IDDIG 滤波器使能。 0：不使能 1：使能	0x1
numctleps	[19:16]	RO	设备模式除了端点 0 之外的控制端点的数量。	0x0

Name	Bits	R/W	Description	Default
			0：0 个控制端点 1：1 个控制端点	
phydatawidth	[15:14]	RO	PHY 数据宽度。 0：8 位 1：16 位 2：8/16 位，软件可选 其他：保留	0x2
reserved	[13]	-	保留。	0x1
acgsupt	[12]	RO	动态时钟门控支持。 此位表示控制器在没有 USB 和 AHB 流量的时段期间支持动态（切换）功率降低。 0：不支持 1：支持	0x1
reserved	[11:7]	-	保留。	0x1
hibernation	[6]	RO	休眠使能。 0：不启用休眠功能 1：启用休眠功能	0x0
ahbfreq	[5]	RO	要求最小 AHB 频率小于 60 MHz。 0：否 1：是	0x1
partialpwrdn	[4]	RO	部分断电使能。 0：未启用部分断电 1：部分断电已启用	0x0
numdevperioeps	[3:0]	RO	设备模式周期 IN 端点数。 0：无周期性 IN 端点 1：1 个周期性 IN 端点	0x0

27.7.18 GDFIFOCFG

◆ **偏移地址：0x5C　　复位值：0x05E005E0**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
epinfobaseaddr															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
gdfifocfg															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
epinfobaseaddr	[31:16]	RO	此字段提供 EP 信息控制器的起始地址。	0x5e0

Name	Bits	R/W	Description	Default
gdfifocfg	[15:0]	RW	此字段用于 DFIFO 大小的动态编程。只有当应用程序将非零值编程到此寄存器时才会生效。编程值必须符合“FIFO RAM 分配”中所述的指导原则。如果 FIFO 大小编程不正确，控制器没有任何校正逻辑。	0x5e0

27.7.19 HPTXFSIZ

◆ **偏移地址：0x100　　复位值：0x05E00BC0**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					ptxfsize										
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				ptxfstaddr											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	1	0	1	1	1	1	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
ptxfsize	[26:16]	RW	主机周期性 TX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 此寄存器的上电复位值被指定为最大主机模式周期性 Tx 数据 FIFO 深度。 可以在该字段中写入一个新值。	0x5e0
reserved	[15:12]	-	保留。	0x0
ptxfstaddr	[11:0]	RW	主机周期性 TX FIFO 起始地址 此寄存器的上电复位值是最大 Rx 数据 FIFO 深度和最大非周期性 Tx 数据 FIFO 深度的总和。 可以在该字段中写入一个新值。	0xbc0

27.7.20 DIEPTXF1

◆ **偏移地址：0x104　　复位值：0x05E00BC0**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					inep1txfdep										
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				inep1txfstaddr											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	1	0	1	1	1	1	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
inep1txfdep	[26:16]	RW	IN 端点 TX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 可以在此字段中写入新值。	0x5e0
reserved	[15:12]	-	保留。	0x0
inep1txfstaddr	[11:0]	RW	IN 端点 FIFO 传输 RAM 起始地址。	0xbc0

27.7.21 DIEPTXF2

◆ **偏移地址：0x108　　复位值：0x05E011A0**

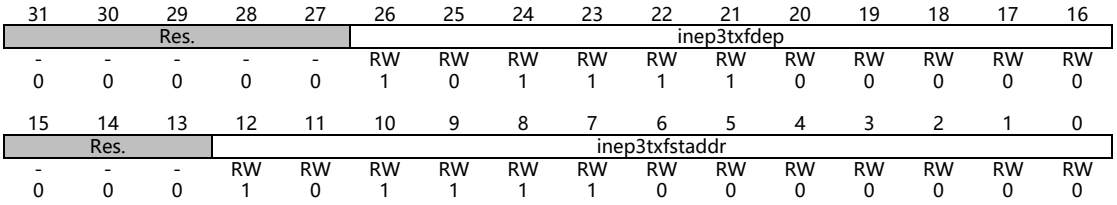
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.					inep2txfdep										
-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			inep2txfstaddr												
-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	1	0	0	0	1	1	0	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
inep2txfdep	[26:16]	RW	IN 端点 TX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 可以在此字段中写入新值。	0x5e0
reserved	[15:13]	-	保留。	0x0
inep2txfstaddr	[12:0]	RW	IN 端点 FIFO 传输 RAM 起始地址。	0x11a0

27.7.22 DIEPTXF3

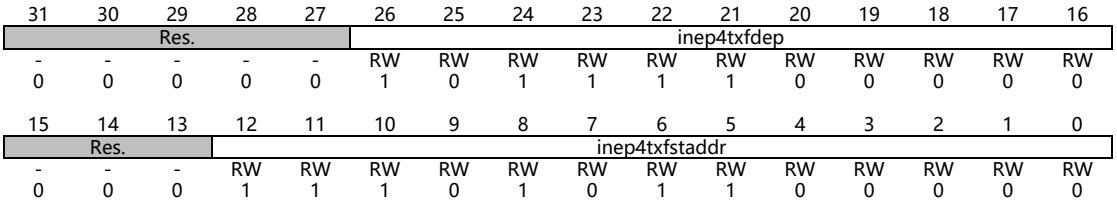
◆ **偏移地址：0x10C　　复位值：0x05E01780**



Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
inep3txfdep	[26:16]	RW	IN 端点 TX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 可以在此字段中写入新值。	0x5e0
reserved	[15:13]	-	保留。	0x0
inep3txfstaddr	[12:0]	RW	IN 端点 FIFO 传输 RAM 起始地址。	0x1780

27.7.23 DIEPTXF4

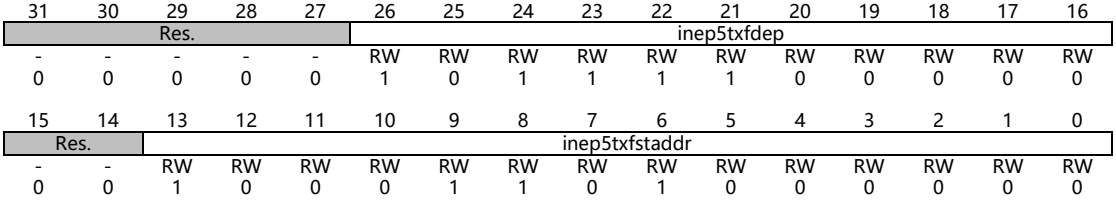
◆ **偏移地址：0x110　　复位值：0x05E01D60**



Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
inep4txfdep	[26:16]	RW	IN 端点 TX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 可以在此字段中写入新值。	0x5e0
reserved	[15:13]	-	保留。	0x0
inep4txfstaddr	[12:0]	RW	IN 端点 FIFO 传输 RAM 起始地址。	0x1d60

27.7.24 DIEPTXF5

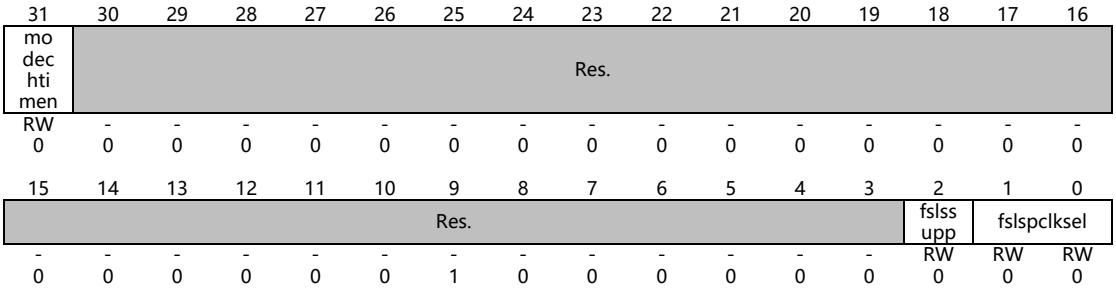
◆ **偏移地址：0x114　　复位值：0x05E02340**



Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
inep5txfdep	[26:16]	RW	IN 端点 TX FIFO 深度。 这个值是以 32 bit 的 word 为单位的，小值为 16，大值为 0x5e0。 可以在此字段中写入新值。	0x5e0
reserved	[15:14]	-	保留。	0x0
inep5txfstaddr	[13:0]	RW	IN 端点 FIFO 传输 RAM 起始地址。	0x2340

27.7.25 HCFG

◆ **偏移地址：0x400　　复位值：0x00000200**

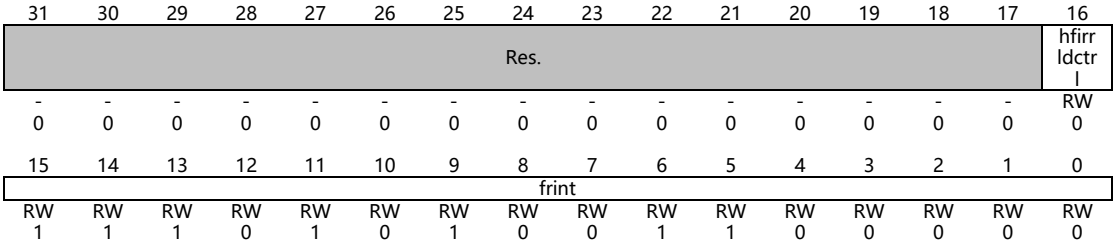


Name	Bits	R/W	Description	Default
modechtimen	[31]	RW	模式更改就绪定时器启用。 该位用于启用/禁用主机控制器以在 Resume 结束时等待 200 个 PHY 时钟周期，从而在 Suspend 或 LPM 之后将到 PHY 的操作模式信号改变为 0。 0：主机控制器在恢复结束时等待 200 个 PHY 时钟周期或 SE0 的线路状态后，将操作模式从 2 更改为 0 1：主机控制器仅在恢复结束时等待 SE0 的行状态后，将操作模式从 2 更改为 0。	0x0
reserved	[30:3]	-	保留。	0x0
fslssupp	[2]	RW	仅支持 FS 和 LS。	0x0

Name	Bits	R/W	Description	Default
			应用程序使用此位来控制控制器的枚举速度。使用此位，应用程序可以使控制器枚举为 FS 主机，即使连接的设备支持 HS 流量。初始编程后，请勿对此字段进行更改。 0：HS/FS/LS，基于连接设备支持的最大速度 1：仅限 FS/LS，即使连接的设备可以支持 HS	
fslspclksel	[1:0]	RW	FS/LS PHY 时钟选择。 当控制器处于 FS 主机模式时 0：PHY 时钟为 30/60 MHz 1：PHY 时钟为 48MHz（禁用） 其他：保留	0x0

27.7.26 HFIR

◆ **偏移地址：0x404　　复位值：0x0000EA60**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
hfirrldctrl	[16]	RW	重新加载控件。 此位允许在运行时动态重新加载 HFIR 寄存器。 0：无法动态重新加载 HFIR 1：HFIR 可以在运行时动态重新加载。 此位需要在初始配置期间进行编程，并且在运行期间不得更改其值。	0x0
frint	[15:0]	RW	帧间隔。 应用程序编程到此字段的值指定两个连续 SOF（FS）或保持活动令牌（LS）之间的间隔。该字段包含构成所需帧间隔的 PHY 时钟的数量。当 PHY 时钟频率为 60 MHz 时，此字段中设置的默认值用于 FS 操作。只有在设置了主机端口控制和状态寄存器（HPRT.prtenaport）的端口启用位之后，应用程序才能向该寄存器写入值。如果未编程任何值，控制器将根据主机配置寄存器（HCFG.fslspclksel）的 FS/LS PHY clock Select（FS/LS 物理层时钟选择）	0xea60

Name	Bits	R/W	Description	Default
			字段中指定的物理层时钟来计算该值。初始配置后不得更改该字段的值。 1ms*（FS 或者 LS 的 PHY 时钟频率）	

27.7.27 HFNUM

◆ **偏移地址：0x408　　复位值：0x00003FFF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
frrem															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
frnum															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
frrem	[31:16]	RO	剩余帧时间。 以 PHY 时钟表示当前帧（FS/LS）中剩余的时间量。该字段在每个 PHY 时钟上递减。当它达到零时，该字段将重新加载帧间隔寄存器中的值，并在USB上传输新的 SOF。	0x0
frnum	[15:0]	RO	帧编号。 当新的 SOF 在 USB 上传输时，该字段增加，当达到 16'H3FF 时，该区域重置为 0。	0x3fff

27.7.28 HPTXSTS

◆ **偏移地址：0x410　　复位值：0x000805E0**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ptxqtop									ptxqspcavail						
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

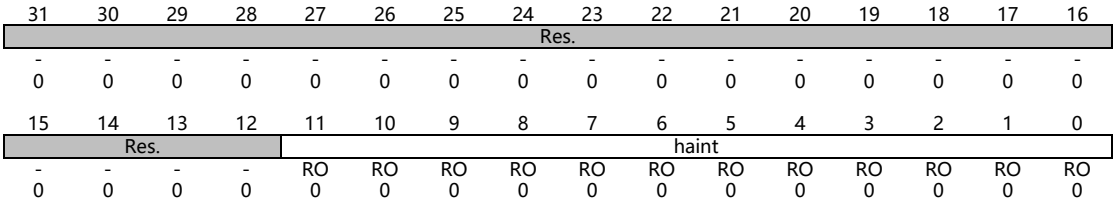
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ptxfspcavail															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
ptxqtop	[31:23]	RO	周期性传输请求队列的顶部。 这表示 MAC 当前正在处理的周期性 Tx 请求队列中的条目。 此寄存器用于调试。 Bit[31]：奇数/偶数帧	0x0

Name	Bits	R/W	Description	Default
			0：发送偶数帧 1：发送奇数帧 位[30:27]：通道/端点编号 位[26:25]：类型 0：输入/输出 1：零长度数据包 2：CSPLIT 3：禁用通道命令 Bit[24]：所选周期性通道/端点的最后一个周期性条目 Bit[23]：终止（所选通道/端点的最后一个条目）	
ptxqspcavail	[22:16]	RO	周期性传输请求队列可用空间。 指示可写入定期传输请求队列的可用位置数。这个队列同时保存 IN 和 OUT 请求。 0：周期性传输请求队列已满 1：1 个可用位置 2：2 个位置可用 3：3 个位置可用 4：4 个位置可用 5：5 个位置可用 6：6 个位置可用 7：7 个位置可用 8：8 个位置可用	0x8
ptxfspcavail	[15:0]	RO	周期性传输数据 FIFO 可用空间。 指示可在周期性 TX FIFO 中写入的空闲位置的数量。值以 32 bit 的 word 为单位 0：周期性 TX FIFO 已满 1:1 个 word 可用空间 2:2 个 word 可用空间 n:n 个 word 可用空间（其中 0 <= n <= 0x5e0）	0x5e0

27.7.29 HAINT

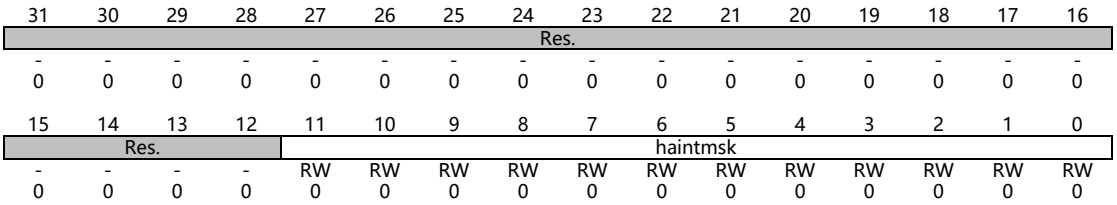
- **偏移地址：0x414　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留。	0x0
haint	[11:0]	RO	通道中断状态（每个 bit 对应一个通道）。 bit[0] 0：通道 0 无中断　1：通道 0 有中断 bit[1] 0：通道 1 无中断　1：通道 1 有中断 bit[2] 0：通道 2 无中断　1：通道 2 有中断 bit[3] 0：通道 3 无中断　1：通道 3 有中断 bit[4] 0：通道 4 无中断　1：通道 4 有中断 bit[5] 0：通道 5 无中断　1：通道 5 有中断 bit[6] 0：通道 6 无中断　1：通道 6 有中断 bit[7] 0：通道 7 无中断　1：通道 7 有中断 bit[8] 0：通道 8 无中断　1：通道 8 有中断 bit[9] 0：通道 9 无中断　1：通道 9 有中断 bit[10] 0：通道 10 无中断　1：通道 10 有中断 bit[11] 0：通道 11 无中断　1：通道 11 有中断	0x0

27.7.30 HAINTMSK

◆ **偏移地址：0x418　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留。	0x0
haintmsk	[11:0]	RW	通道中断。 每个通道一 bit：通道 0 对应 bit0，通道 8 对应 bit8 0：有中断 1：无中断	0x0

27.7.31 HPRT

◆ **偏移地址：0x440　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													prtspd		Res.
-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			prtpwr	prtlnsts		Res.	prtrst	prtsusp	prtres	Res.		prtenchng	prtena	prtconndet	prtconnsts
-	-	-	RW	RO	RO	-	RW	RW	RW	-	-	W1C	W1C	W1C	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留。	0x0
prtspd	[18:17]	RO	端口速度。 指示连接到此端口的设备的速度。 0：高速 1：全速 2：低速 3：保留	0x0
reserved	[16:13]	-	保留。	0x0
prtpwr	[12]	RW	端口电源。 应用程序使用此字段来控制此端口的电源，并且内核可以在过电流条件下清除此位。 0：端口电源关闭 1：端口电源打开 注意：此位与接口无关。应用程序需要为所有接口编程该位	0x0
prtlnsts	[11:10]	RO	端口线路状态。 指示当前逻辑电平 USB 数据线。 Bit[10]：D+的逻辑电平 0:低电平 1：高电平 Bit[11]：D-的逻辑电平 0:低电平 1：高电平	0x0
reserved	[9]	-	保留。	0x0
prtrst	[8]	RW	端口复位。 当应用程序设置此位时，将在此端口上启动复位序列。应用程序必须计时复位时间，并在复位序列完成后清除此位。 0：不复位 1：复位 应用程序必须将此位设置为 1 并至少保持下面提到的最短持续时间，才能在端口上启动复位。在清除此位之前，应用程序也可以在所需的最小保持时间之外再	0x0

Name	Bits	R/W	Description	Default
			保留 10 毫秒，即使没有设备连接到主机，此位也会被控制器清除。 全速/低速复位最短持续时间：10 毫秒	
prtsusp	[7]	RW	应用程序将此 bit 置 1 以将此端口置于挂起模式。只有此 bit 置 1 时，模块才会停止发送 SOF。要停止 PHY 时钟，应用程序必须将端口时钟停止 bit 置 1，这会使能 PHY 的挂起输入引脚。 此位的读取值反映该端口的当前挂起状态。检测到远程唤醒信号，或者应用程序将此寄存器中的端口复位 bit 或端口恢复 bit 置 1 后，模块可将此位清零；或应用程序将模块中断寄存器中 的检测到恢复/ 远程唤醒中断 bit 或检测到断开连接中断 bit（分别为 GINTSTS 中的 WKUINT 或 DISCINT）置 1，模块也可将此位清零。 0：端口未处于挂起模式 1：端口处于挂起模式	0x0
prtres	[6]	RW	应用程序将此 bit 置 1 以在该端口上驱动恢复信号。模块会持续驱动恢复信号直到应用程序将 此位清零。 按照模块中断寄存器中的检测到端口恢复/远程唤醒中断位（GINTSTS 中的 WKUINT bit）的描述，如果模块检测到 USB 远程唤醒序列，则开始驱动恢复信号，而无需应用程序进行干预；如果模块检测到断开连接的情况，则将此位清零。此位的读取值指示当前模块是否正在驱动恢复信号。 0：不驱动恢复信号 1：驱动恢复信号	0x0
reserved	[5:4]	-	保留。	0x0
prtenchng	[3]	W1C	端口启用/禁用更改。 当此寄存器的端口启用 bit[2]的状态发生变化时，内核会设置此位。此位只能由内核设置，应用程序必须写入 1 才能清除。 数值： 0：端口启用位 bit[2]未更改 1：端口启用位 bit[2]已更改	0x0
prtena	[2]	W1C	端口启用（PrtEa）。 端口仅在重置序列后由控制器启用，并因过电流条件、断开连接条件或应用程序清除此位而禁用。应用程序无法通过寄存器写入来设置此位。它只能通过写入 1 来清除它以禁用端口。此位不会触发对应用程序的任何中断。	0x0

Name	Bits	R/W	Description	Default
prtconndet	[1]	W1C	当检测到设备连接时，模块将此 bit 置 1，以使用模块中断寄存器中的主机端口中断 bit（GINTSTS.PrtInt）触发应用程序的中断。应用程序必须将此位置 1 才可清除该中断。 0：未检测到设备连接 1：检测到设备连接	0x0
prtconnsts	[0]	RO	端口连接状态。 0：没有设备连接到端口。 1：设备已连接到端口。	0x0

27.7.32 HCCHAR(n)

◆ **偏移地址：0x500+0x20*n　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
chena(n)	chdis(n)	oddfrm(n)	devaddr(n)							ec(n)		eptype(n)		Res.	
RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	- 0	- 0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
epdir(n)	epnum(n)				mps(n)										
RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0	RW 0

Name	Bits	R/W	Description	Default
chena(n)	[31]	RW	此字段由应用程序软件置 1，并由控制器清零。 0:禁用通道 1:使能通道	0x0
chdis(n)	[30]	RW	应用程序将此位置 1 以停止通过通道发送/接收数据，即使通过该通道的传输还未完成，停止操作仍然生效。应用程序必须等到禁止通道的中断以确认通道已经被禁止。	0x0
oddfrm(n)	[29]	RW	奇数帧。 该字段由应用程序设置，以指示主机必须在奇数帧/偶数帧中执行传输。此字段仅适用于周期性（同步和中断）事务。 0：偶数帧 1：奇数帧	0x0
devaddr(n)	[28:22]	RW	设备地址。 此字段用于指定要与该主机通信的特定设备。	0x0
ec(n)	[21:20]	RW	此字段向主机指示该周期性端点每帧必须执行的事务数。该字段不用于非周期性传输。 0：保留。对该字段的操作会产生未知的结果	0x0

Name	Bits	R/W	Description	Default
			1：该端点每帧需要发出 1 个事务 2：该端点每帧需要发出 2 个事务 3：该端点每帧需要发出 3 个事务 注意：此字段至少需设置为 1	
eptype(n)	[19:18]	RW	端点类型。 指示所选的传输类型。 0：控制 1：同步 2：批量 3：中断	0x0
reserved	[17:16]	-	保留。	0x0
epdir(n)	[15]	RW	端点方向。 指示事务是 IN 还是 OUT。 0:输出 1:输入	0x0
epnum(n)	[14:11]	RW	端点编号。 指示用作数据源或接收器的设备端点编号。 数值： 0：端点 0 1：端点 1 2：端点 2 3：端点 3 4：端点 4 5：端点 5 6：端点 6 7：端点 7 8：端点 8 9：终点 9 10：终点 10 11：端点 11 12：端点 12 13：终点 13 14：端点 14 15：端点 15	0x0
mps(n)	[10:0]	RW	指示与该主机通道通信的设备端点的最大数据包大小。	0x0

27.7.33 HCINT(n)

◆ **偏移地址：0x508+0x20*n** **复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					datatglerr(n)	frmovrun(n)	bblerr(n)	xacterr(n)	Res.	ack(n)	nak(n)	stall(n)	Res.	chhltd(n)	xfercompl(n)
-	-	-	-	-	W1C	W1C	W1C	W1C	-	W1C	W1C	W1C	-	W1C	W1C
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
datatglerr(n)	[10]	W1C	数据切换错误。 此位只能由内核设置，应用程序必须写入 1 才能清除。 0：无数据切换错误 1：数据切换错误	0x0
frmovrun(n)	[9]	W1C	帧溢出。 此位只能由内核设置，应用程序必须写入 1 才能清除。 0：无帧溢出 1：帧溢出	0x0
bblerr(n)	[8]	W1C	串扰错位。 此位只能由内核设置，应用程序必须写入 1 才能清除。 0：无串扰错误 1：有串扰错误	0x0
xacterr(n)	[7]	W1C	事务错误。 表示 USB 上发生以下错误之一： （1）CRC 检查失败 （2）超时 （3）位填充错误 （4）错误 EOP 该位只能由内核设置，应用程序必须写入 1 才能清除。 0：无事务错误 1：事务错误	0x0
reserved	[6]	-	保留。	0x0
ack(n)	[5]	W1C	接收/发送的 ACK 响应中断。 该位只能由内核设置，应用程序必须写入 1 才能清除。 0：未接收到 ACK 响应或发送 ACK 中断 1：收到或发送的 ACK 响应询问	0x0

Name	Bits	R/W	Description	Default
nak(n)	[4]	W1C	NAK 响应接收中断。 该位只能由内核设置，应用程序必须写入 1 才能清除。 0：未接收到 NAK 响应中断 1：NAK 响应接收中断	0x0
stall(n)	[3]	W1C	STALL 响应接收中断。 该位只能由内核设置，应用程序必须写入 1 才能清除。 0:未接收到 STALL 中断 1:接收到 STALL 中断	0x0
reserved	[2]	-	保留。	0x0
chhltd(n)	[1]	W1C	通道暂停。 它表示由于任何USB事务错误或响应应用程序的禁用请求而导致传输异常完成。 0:通道未停止 1:通道暂停	0x0
xfercompl(n)	[0]	W1C	传输完成。 传输正常完成，没有任何错误。此位只能由内核设置，应用程序必须写入 1 才能清除。 0:正在传输或无活动传输 1:传输正常完成，没有任何错误	0x0

27.7.34 HCINTMSK(n)

◆ **偏移地址：0x50C+0x20*n　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.					datatglerrmsk(n)	frmovrunmsk(n)	bblerrmsk(n)	xacterrmsk(n)	Res.	ackmsk(n)	nakmsk(n)	stallmsk(n)	Res.	chhltdmsk(n)	xfercomplmsk(n)
-	-	-	-	-	RW	RW	RW	RW	-	RW	RW	RW	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:11]	-	保留。	0x0
datatglerrmsk(n)	[10]	RW	数据翻转错误中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
frmovrunmsk(n)	[9]	RW	帧溢出中断屏蔽。 0:屏蔽中断	0x0

Name	Bits	R/W	Description	Default
			1:使能中断	
bblerrmsk(n)	[8]	RW	Babble 错误中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
xacterrmsk(n)	[7]	RW	事务错误中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[6]	-	保留。	0x0
ackmsk(n)	[5]	RW	ACK 响应接收/传输中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
nakmsk(n)	[4]	RW	NAK 响应接收中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
stallmsk(n)	[3]	RW	STALL 响应接收中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[2]	-	保留。	0x0
chhltdmsk(n)	[1]	RW	通道暂停中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
xfercomplmsk(n)	[0]	RW	传输完成中断屏蔽。 0:屏蔽中断 1:使能中断	0x0

27.7.35 HCTSIZ(n)

◆ **偏移地址：0x510+0x20*n　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	pid(n)		pktcnt(n)										xfersize(n)		
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xfersize(n)															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31]	-	保留。	0x0
pid(n)	[30:29]	RW	PID。 应用程序使用要用于初始事务的 PID 类型对该字段进行编程。主机在剩余的传输过程中保留此字段。 0: DATA0	0x0

Name	Bits	R/W	Description	Default
			1: DATA2 2: DATA1 3:MDATA（非控制）/SETUP（控制）	
pktcnt(n)	[28:19]	RW	数据包计数。 该字段由应用程序编程，具有要发送（OUT）或接收（IN）的预期分组数。 主机在每次成功发送或接收 OUT/IN 分组时递减该计数。一旦此计数达到零，应用程序将被中断以指示正常完成。	0x0
xfersize(n)	[18:0]	RW	传输大小。 对于 OUT，此字段是主机在传输过程中发送的数据字节数。 对于 IN，此字段是应用程序为传输保留的缓冲区大小。应用程序应将此字段编程为 IN 事务（周期性和非周期性）的最大数据包大小的整数倍。	0x0

27.7.36 DCFG

◆ **偏移地址：0x800　　复位值：0x08020000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
resvalid						Res.									
RW	RW	RW	RW	RW	RW	-	-	-	-	-	-	-	-	-	-
0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
erraticintmsk	Res.		perfrint		devaddr							Res.	nzstsouthshk	devspd	
RW	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
resvalid	[31:26]	RW	保留必须保持为默认值。	0x2
reserved	[25:16]	-	保留。	0x0
erraticintmsk	[15]	RW	不稳定错误中断屏蔽。 0:不屏蔽不稳定错误的早期挂起中断 1:屏蔽不稳定错误的早期挂起中断	0x0
reserved	[14:13]	-	保留。	0x0
perfrint	[12:11]	RW	指示一帧内必须使用周期性帧中断通知应用程序的时间点。此功能可用于确定该帧的所有同步通信是否完成。 0:80%的帧间隔 1:85%的帧间隔 2:90%的帧间隔	0x0

Name	Bits	R/W	Description	Default
			3:95%的帧间隔	
devaddr	[10:4]	RW	应用程序必须在执行每个 SetAddress 控制命令后根据命令参数对该字段进行设置。	0x0
reserved	[3]	-	保留。	0x0
nzstsouthshk	[2]	RW	在控制传输状态阶段的 OUT 事务期间，当模块收到非零长度数据包后，应用程序可以使用此字段选择要发送的握手信号。 0：将接收到的 OUT 包发送到应用程序(长度为零或非零)，并根据设备端点控制寄存器中端点的 NAK 和 STALL 位发送握手 1：在非零长度的状态 OUT 事务上发送一个 STALL 握手，不向应用程序发送接收到的 OUT 包	0x0
devspd	[1:0]	RW	指示应用程序要求模块进行枚举所采用的速度，或应用程序支持的最大速度。但是，实际总线速度只有在完成 chirp 序列后才能确定，同时此速度基于与模块连接的 USB 主机的速度。 1:全速 USB 2.0 PHY 时钟为 30 MHz 或 60 MHz 其他：禁用	0x0

27.7.37 DCTL

◆ **偏移地址：0x804　　复位值：0x00000002**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															nakonbble
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				pwronprgdone	cgoutnak	sgoutnak	cgnpinnak	sgnpinnak	Res.			goutnaksts	gnpinnaksts	sftdiscon	rmtwkupsig
-	-	-	-	RW	RW	RW	RW	RW	-	-	-	RO	RO	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0
nakonbble	[16]	RW	串扰错误的 NAK。 0：串扰错误时禁用 NAK 1：串扰错误时 NAK	0x0
reserved	[15:12]	-	保留。	0x0
pwronprgdone	[11]	RW	应用程序使用此位来指示寄存器编程在从断电模式唤醒后已完成编程。 0:上电编程未完成	0x0

Name	Bits	R/W	Description	Default
			1:上电编程完成	
cgoutnak	[10]	RW	清除全局 OUT NAK。 0：禁用清除全局 OUT NAK 1：清除全局输出 NAK	0x0
sgoutnak	[9]	RW	设置全局 OUT NAK。 对该字段的写入将设置全局 OUT NAK。应用程序使用此位在所有 OUT 端点上发送 NAK 握手。只有在确保控制器中断寄存器（GINTSTS.goutnakeff）中的全局 OUT NAK 有效位被清除后，应用程序才能设置此位。 数值： 0：禁用全局 OUT NAK 1：设置全局 OUT NAK	0x0
cgnpinnak	[8]	RW	清除全局非周期 IN NAK。 写入该字段将清除全局非周期性 IN NAK。 数值： 0：禁用全局非周期 IN NAK 1：清除全局非周期 IN NAK	0x0
sgnpinnak	[7]	RW	设置全局非周期 IN NAK。 对此字段的写入设置全局非周期性 IN NAK。应用程序使用此位在所有非周期性的 IN 端点上发送 NAK 握手。只有在确保控制器中断寄存器（GINTSTS.ginnakeff）中的全局 IN NAK 有效位被清除后，应用程序才能设置此位。 0：禁用全局非周期 IN NAK 1：设置全局非周期 IN NAK	0x0
reserved	[6:4]	-	保值为默认值 0。	0x0
goutnaksts	[3]	RO	全局 OUT NAK 状态。 0：根据 FIFO 状态、NAK 和 STALL 位设置发送握手。 1：无论空间可用性如何，都不会向 RX FIFO 写入任何数据。对所有数据包发送 NAK 握手，SETUP 事务除外。所有同步 OUT 数据包都被丢弃。	0x0
gnpinnaksts	[2]	RO	全局非周期 IN NAK 状态。 0：根据 TX FIFO 中的数据可用性发送握手。 1：无论 TX FIFO 中的数据可用性如何，在所有非周期性 IN 端点发送 NAK 握手。	0x0
sftdiscon	[1]	RW	应用程序使用该位向 USB 模块发出执行软断开的信号。	0x1

Name	Bits	R/W	Description	Default
			该位置 1 时，主机不会看到设备已连接，且该设备也不会接收 USB 上的信号。在应用程序将此位清零之前，模块会保持 断开状态。 0：此位在软断开之后清零，会使主机收到设备已连接的事件。重新连接设备之后，USB 主机会重新启动设备枚举。 1：模块使 USB 主机收到设备断开连接的事件	
rmtwkupsig	[0]	RW	远程唤醒信号。 当应用程序设置此位时，控制器启动远程信号以唤醒 USB 主机。应用程序必须设置此位以指示控制器退出挂起状态。根据 USB 2.0 规范中的规定，应用程序必须在设置该位 1-15 毫秒后清除该位。 如果启用了 LPM 并且内核处于 L1（睡眠）状态，则当应用程序设置此位时，内核启动 L1 远程信令以唤醒USB主机。应用程序必须设置此位，以指示控制器退出睡眠状态。根据 LPM 规范中的规定，硬件在应用程序设置 50 微秒后自动清除该位（TL1DevDrvResume）。若来自上一个 LPM 事务的 GLPMCFG bRemoteWake 为零时，应用程序不得设置此位。 0：控制器不发送远程唤醒信令 1：控制器发送远程唤醒信号	0x0

27.7.38 DSTS

◆ **偏移地址：0x808　　复位值：0x00000002**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.								devlnsts		soffn					
-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

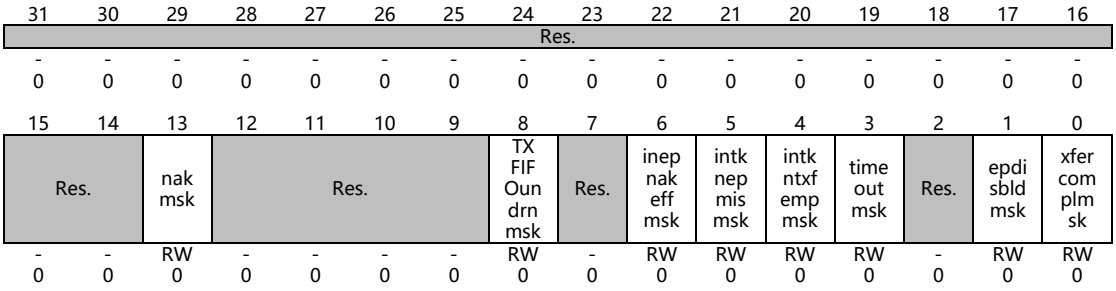
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
soffn								Res.				errticerr	enumspd		susposts
RO	RO	RO	RO	RO	RO	RO	RO	-	-	-	-	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
devlnsts	[23:22]	RO	设备线状态。 指示当前 USB 数据线的逻辑电平 bit[23]：D+的逻辑电平 0：低电平 1：高电平 bit[22]：D-的逻辑电平 0：低电平 1：高电平	0x0
soffn	[21:8]	RO	接收到的 SOF（SOFFN）的帧或微帧编号	0x0

Name	Bits	R/W	Description	Default
			控制器以全速或低速运行时，此字段包含一个帧号。 注意：如果在通电复位后立即读取，此寄存器可能会返回非零值。在上电复位后寄存器位立即读取非零的情况下，这并不表示已经从主机接收到 SOF。只有在主机和设备之间建立有效连接后，此中断的读取值才有效。	
reserved	[7:4]	-	保留。	0x0
errticerr	[3]	RO	不稳定错误。 控制器将此位设置为报告在 UTMI+上看到的任何不稳定错误（由于 phy 错误，phy_rxvalid_i/phy_rxvldh_i 或 phy_rxative_i 被触发至少 2ms）。由于不稳定的错误，内核进入挂起状态，并触发全局中断寄存器（GINTSTS.erlysusp）的 Early Suspend 中断。如果由于不稳定错误而触发了 Early Suspend，则应用程序必须执行 PHY 重置，然后对控制器进行软复位。 0：无异常错误 1：不稳定错误	0x0
enumspd	[2:1]	RO	枚举速度。 指示控制器在通过连接或复位序列进行速度检测后启动的速度。 0：高速（phy 时钟以 30 或 60 MHz 运行）（禁用） 1：全速（phy 时钟以 30 或 60 MHz 运行） 2：低速（phy 时钟运行频率为 6 MHz）（禁用） 3：全速（phy 时钟运行在 48 MHz）（禁用） 当前使用的是 UTMI+PHY，不支持低速，持默认值 1。	0x1
suspsts	[0]	RO	挂起状态。 在设备模式下，只要在 USB 上检测到挂起条件，就会设置此位。当 phy_line_state_i 信号在很长一段时间内没有活动时，控制器将进入挂起状态。控制器在以下条件下从挂起中脱离： 如果 phy_line_state_i 信号有任何活动，或 如果应用程序写入设备控制寄存器中的远程唤醒信令位（DCTL.rmtwkupsig）。 当控制器从挂起中出来时，该位被设置为 0。 0：无挂起状态 1：挂起状态	0x0

27.7.39 DIEPMSK

◆ **偏移地址：0x810　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
nakmsk	[13]	RW	NAK 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[12:9]	-	保留。	0x0
TX FIFOundrnmsk	[8]	RW	Fifo 下溢中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[7]	-	保留。	0x0
inepnakeffmsk	[6]	RW	IN 端点 NAK 有效中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
intknepmismsk	[5]	RW	EP 不匹配时接收到 IN 令牌中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
intkntxfempmsk	[4]	RW	TX FIFO 空时接收到 IN 令牌中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
timeoutmsk	[3]	RW	超时中断屏蔽（非同步端点）。 0:屏蔽中断 1:使能中断	0x0
reserved	[2]	-	保留。	0x0
epdisbldmsk	[1]	RW	端点禁用中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
xfercomplmsk	[0]	RW	传输完成中断屏蔽。 0:屏蔽中断 1:使能中断	0x0

27.7.40 DOEPMSK

◆ **偏移地址：0x814　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		nakmsk	bbleerrmsk	Res.			outpkterrmsk	Res.		stsphsercvdmsk	outtknepdismsk	setupmsk	Res.	epdisbldmsk	xfercomplmsk
-	-	RW	RW	-	-	-	RW	-	-	RW	RW	RW	-	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
nakmsk	[13]	RW	NAK 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
bbleerrmsk	[12]	RW	串扰错误中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[11:9]	-	保留。	0x0
outpkterrmsk	[8]	RW	OUT 数据包错误中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[7:6]	-	保留。	0x0
stsphsercvdmsk	[5]	RW	状态相位接收中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
outtknepdismsk	[4]	RW	端点禁用时接收到 OUT 令牌中断屏蔽（仅适用于控制 OUT 端点）。 0:屏蔽中断 1:使能中断	0x0
setupmsk	[3]	RW	SETUP 阶段完成中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
reserved	[2]	-	保留。	0x0
epdisbldmsk	[1]	RW	端点禁用中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
xfercomplmsk	[0]	RW	传输完成中断屏蔽。 0:屏蔽中断 1:使能中断	0x0

27.7.41 DAINT

◆ **偏移地址：0x818　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.										outepint5	outepint4	outepint3	outepint2	outepint1	outepint0
-	-	-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										inepint5	inepint4	inepint3	inepint2	inepint1	inepint0
-	-	-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:22]	-	保留。	0x0
outepint5	[21]	RO	OUT 端点 5 中断位。 0:无中断 1:OUT 端点中断有效	0x0
outepint4	[20]	RO	OUT 端点 4 中断位。 0：无中断 1：OUT 端点中断有效	0x0
outepint3	[19]	RO	OUT 端点 3 中断位。 0：无中断 1：OUT 端点中断有效	0x0
outepint2	[18]	RO	OUT 端点 2 中断位。 0：无中断 1：OUT 端点中断有效	0x0
outepint1	[17]	RO	OUT 端点 1 中断位。 0：无中断 1：OUT 端点中断有效	0x0
outepint0	[16]	RO	OUT 端点 0 中断位。 0：无中断 1：OUT 端点中断有效	0x0
reserved	[15:6]	-	保留。	0x0
inepint5	[5]	RO	IN 端点 5 中断位。 0:无中断 1:IN 端点中断有效	0x0
inepint4	[4]	RO	IN 端点 4 中断位。 0:无中断 1:IN 端点中断有效	0x0
inepint3	[3]	RO	IN 端点 3 中断位。 0:无中断 1:IN 端点中断有效	0x0

Name	Bits	R/W	Description	Default
inepint2	[2]	RO	IN 端点 2 中断位。 0:无中断 1:IN 端点中断有效	0x0
inepint1	[1]	RO	IN 端点 1 中断位。 0:无中断 1:IN 端点中断有效	0x0
inepint0	[0]	RO	IN 端点 0 中断位。 0:无中断 1:IN 端点中断有效	0x0

27.7.42 DAINTMSK

◆ **偏移地址：0x81C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.										outepmsk5	outepmsk4	outepmsk3	outepmsk2	outepmsk1	outepmsk0
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										inepmsk5	inepmsk4	inepmsk3	inepmsk2	inepmsk1	inepmsk0
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:22]	-	保留。	0x0
outepmsk5	[21]	RW	OUT 端点 5 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
outepmsk4	[20]	RW	OUT 端点 4 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
outepmsk3	[19]	RW	OUT 端点 3 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
outepmsk2	[18]	RW	OUT 端点 2 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
outepmsk1	[17]	RW	OUT 端点 1 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
outepmsk0	[16]	RW	OUT 端点 0 中断屏蔽。 0:屏蔽中断	0x0

Name	Bits	R/W	Description	Default
			1:使能中断	
reserved	[15:6]	-	保留。	0x0
inepmsk5	[5]	RW	IN 端点 5 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
inepmsk4	[4]	RW	IN 端点 4 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
inepmsk3	[3]	RW	IN 端点 3 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
inepmsk2	[2]	RW	IN 端点 2 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
inepmsk1	[1]	RW	IN 端点 1 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0
inepmsk0	[0]	RW	IN 端点 0 中断屏蔽。 0:屏蔽中断 1:使能中断	0x0

27.7.43 DTHRCTL

◆ **偏移地址：0x830　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.						rxthrlen									rxthren
-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			ahbthrratio		txthrlen									isothren	nonisothren
-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:26]	-	保留。	0x0
rxthrlen	[25:17]	RO	接收阈值长度。	0x0
rxthren	[16]	RO	接收阈值启用。 0：禁用阈值 1：启用阈值	0x0
reserved	[15:13]	-	保留。	0x0
ahbthrratio	[12:11]	RO	AHB 阈值比率。	0x0

Name	Bits	R/W	Description	Default
			这些比特定义了仅用于传输路径的 AHB 阈值和 MAC 阈值之间的比率。AHB 阈值始终保持小于或等于 USB 阈值，因为这不会增加开销。AHB 和 MAC 阈值必须是 DWORD 对齐的。应用程序需要对 TxThrLen 和 ahbthrratio 进行编程，以使 AHB 阈值 DWORD 对齐。如果 AHB 阈值未与 DWORD 对齐，则控制器可能无法正常工作。在编程 TxThrLen 和 ahbthrratio 时，应用程序必须确保最小 AHB 阈值不低于 8 DWORDS，以满足 USB 周转时间要求。 0： AHB 阈值 = MAC 阈值 1： AHB 阈值 = MAC 阈值 / 2 2： AHB 阈值 = MAC 阈值 / 4 3： AHB 阈值 = MAC 阈值 / 8	
txthrlen	[10:2]	RO	传输阈值长度。 此字段指定 DWORDS 中的传输阈值大小。这也形成了 MAC 阈值，并指定了在控制器可以开始在 USB 上传输之前，要在相应的端点传输 FIFO 中的数据量（以字节为单位）。当 ahbthrratio 的值为 2'000 时，阈值长度必须至少为八个 DWORDS。如果 ahbthrratio 为非零，则应用程序需要确保 AHB 阈值不会低于建议的八 DWORD 值。该字段控制等时和非等时 IN 端点阈值。ThrLen 的建议值应与编程的 AHB 突发长度（GAHBCFG.hbstlen）相同。	0x0
isothren	[1]	RO	同步 IN 端点阈值启用。 当该位被设置时，控制器启用同步 IN 端点的阈值。 0：无阈值 1：启用同步 IN 端点阈值	0x0
nonisothren	[0]	RO	非同步 IN 端点阈值启用。 当设置该位时，控制器启用非等时 IN 端点的阈值设置。 0：无阈值 1：启用非同步 IN 端点阈值	0x0

27.7.44 DIEPEMPMSK

◆ **偏移地址：0x834　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ineptxfempmsk															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
ineptxfempmsk	[15:0]	RW	IN EP TX FIFO 空中断屏蔽位。 这些位充当 DIEPINTn.TxFEmp 中断的屏蔽位，每个 IN 端点对应一 bit： IN_EP 0 对应 bit 0，IN_EP 5 对应 bit 5 0x1（EP0_MASK）：屏蔽进入 EP0 TX FIFO 空中断 0x2（EP1_MASK）：屏蔽进入 EP1 TX FIFO 空中断 0x4（EP2_MASK）：屏蔽进入 EP2 TX FIFO 空中断 0x8（EP3_MASK）：屏蔽进入 EP3 TX FIFO 空中断 0x10（EP4_MASK）：屏蔽进入 EP4 TX FIFO 空中断 0x20（EP5_MASK）：屏蔽进入 EP5 TX FIFO 空中断	0x0

27.7.45 DIEPCTL(m)

◆ **偏移地址：0x900+0x20*m　　复位值：0x00008000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
epena(m)	epdis(m)	Res.		snak(m)	cnak(m)	txfnum(m)				stall(m)	Res.	eptype(m)		naksts(m)	Res.
RW	RW	-	-	RW	RW	RW	RW	RW	RW	RW	-	RO	RO	RO	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
usbactep(m)	Res.				mps(m)										
RW	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
epena(m)	[31]	RW	该位指示数据已准备好在端点上传输。在该端点上设置端点关闭或传输完成中断之前，控制器会将清除该位。 0：无操作 1：启用端点	0x0

Name	Bits	R/W	Description	Default
epdis(m)	[30]	RW	应用程序将此位置 1 会停止在端点上传输数据，即使在该端点的传输完成之前。 应用程序必须等待 Endpoint Disabled 中断，然后才能将端点视为已禁用。 控制器在设置 Endpoint Disabled 中断之前会清除此位。 仅当已经为此端点设置了 Endpoint Enable（端点启用）时，应用程序才必须设置此位。 0：无操作 1：禁用端点	0x0
reserved	[29:28]	-	保留。	0x0
snak(m)	[27]	RW	对此位的写入为端点设置 NAK 位。使用这个比特，应用程序可以控制端点上 NAK 握手的传输。在端点上接收到 SETUP 数据包之后，控制器还可以为该端点设置该比特。 0：无操作 1：设置 NAK	0x0
cnak(m)	[26]	RW	清除 NAK（CNAK）对此位的写入将清除端点的 NAK 位。 0：无操作 1：清除 NAK	0x0
txfnum(m)	[25:22]	RW	此值设置为分配给 IN 端点的 TX FIFO 编号。 数值： 0：TX FIFO 0 1：TX FIFO 1 2：TX FIFO 2 3：TX FIFO 3 4：TX FIFO 4 5：TX FIFO 5 6：TX FIFO 6 7：TX FIFO 7 8：TX FIFO 8 9：TX FIFO 9 10：TX FIFO 10 11：TX FIFO 11 12：TX FIFO 12 13：TX FIFO 13 14：TX FIFO 14 15：TX FIFO 15	0x0
stall(m)	[21]	RW	应用程序只能设置此位不能清除此位，控制器会将其清除。如果 NAK 位、全局非周期 IN NAK 或全局	0x0

Name	Bits	R/W	Description	Default
			OUT NAK 与该位一起设置，STALL 位优先级别更高。 0：无 STALL 1：STALL 握手	
reserved	[20]	-	保留。	0x0
eptype(m)	[19:18]	RO	端点类型。 m=0 时此字段为 RO 类型,硬编码为 0 表示为控制端点 0：控制端点 1：禁用 2：禁用 3：禁用 m=1~5 时此字段为 RW 类型 0：禁用 1：同步端点 2：批量端点 3：中断端点	0x0
naksts(m)	[17]	RO	NAK 状态。 表示以下内容： 0：控制器基于 FIFO 状态传输非 NAK 握手 1：控制器正在该端点上传输 NAK 握手。 当应用程序或控制器设置此位时，即使 TX FIFO 中有可用数据，控制器也会停止传输数据。不管这个比特的设置如何，控制器总是用 ACK 握手来响应 SETUP 数据包。	0x0
reserved	[16]	-	保留。	0x0
usbactep(m)	[15]	RW	USB 活动端点。 该位总是设置为 1，指示控制端点 0 在所有配置和接口中总是活动的。 1：控制端点始终处于活动状态	0x1
reserved	[14:11]	-	保留。	0x0
mps(m)	[10:0]	RW	最大数据包大小。 适用于 IN 和 OUT 端点。 应用程序必须使用当前端点的最大数据包大小对此字段进行编程。 注意：当 m=0 时，只有[1:0]两 bit 有效 0：64 字节 1：32 字节 2：16 字节 3：8 字节	0x0

27.7.46 DIEPINT(m)

◆ **偏移地址：0x908+0x20*m　　复位值：0x00000080**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	nyetintrpt	nakintrpt(m)	bbleerr(m)	pktdrpsts(m)	Res.		TXFIFOundrn(m)	txfemp(m)	inepnakeff(m)	intknepmis(m)	intkntxfemp(m)	timeout(m)	Res.	epdisbld(m)	xfercompl(m)
-	W1C	W1C	W1C	W1C	-	-	W1C	RO	W1C	W1C	W1C	W1C	-	W1C	W1C
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留。	0x0
nyetintrpt	[14]	W1C	当为非同步 OUT 端点传输 NYET 响应时成此中断。 0：无中断 1：有 NYET 中断	0x0
nakintrpt(m)	[13]	W1C	当 NAK 被设备发送或接收时，控制器产生该中断。在等时 In 端点的情况下，当由于 TX FIFO 中的数据不可用而发送零长度分组时，会产生中断。 0：无中断 1：NAK 中断	0x0
bbleerr(m)	[12]	W1C	当接收到针对端点的 Babble 错误时，控制器会生成此中断。 数值： 0：无中断 1：有 BbleErr 中断	0x0
pktdrpsts(m)	[11]	W1C	该位向应用程序指示 ISOC OUT 数据包已被丢弃。该位没有相关的掩码位，并且不产生中断。 0：无中断 1：有 PkeDrp 中断	0x0
reserved	[10:9]	-	保留。	0x0
TX FIFOundrn(m)	[8]	W1C	仅适用于 IN 端点。 此位仅在启用阈值时有效。当内核检测到此端点的传输 FIFO 下溢出，它会生成此中断。 0：无中断 1：有 TX FIFO 下溢中断	0x0
txfemp(m)	[7]	RO	此位仅对 IN 端点有效。 当该端点的 TX FIFO 为半空或完全空时，该中断被触发。半空或完全空状态由控制器 AHB 配置寄存器（GAHBCFG.nptxfemplvl）确定。	0x1

Name	Bits	R/W	Description	Default
			0：无中断 1：TX FIFO 空中断	
inepnakeff(m)	[6]	W1C	仅适用于周期性 IN 端点。 当应用程序通过写入 DIEPCTLn.cnak 来清除 IN 端点 NAK 时，可以清除此位。 此中断表示内核已对 NAK 位进行了采样 此中断表示应用程序设置的 IN 端点 NAK 位已在内核中生效。 此中断不能保证在 USB 上发送 NAK 握手。STALL 位的优先级高于 NAK 位。 0：无中断 1：有 IN 端点 NAK 生效中断	0x0
intknepmis(m)	[5]	W1C	仅适用于非周期性 IN 端点。 指示非周期性 TX FIFO 顶部的数据属于接收 in 令牌的端点以外的端点。此中断是在接收到 IN 令牌的端点上触发的。 0：无中断 1：有 INTknEPMis 中断	0x0
intkntxfemp(m)	[4]	W1C	仅适用于非周期性 IN 端点。 指示在关联的 TX FIFO（周期性/非周期性）为空时接收到 IN 令牌。此中断是在接收到 IN 令牌的端点上触发的。 0：无中断 1：有中断	0x0
timeout(m)	[3]	W1C	仅适用于控制输入端点。 表示控制器已检测到此终结点上最后一个 IN 令牌的 USB 超时情况。 0：无中断 1：有超时中断	0x0
reserved	[2]	-	保留。	0x0
epdisbld(m)	[1]	W1C	适用于 IN 和 OUT 端点。 该位表示针对应用程序的请求禁用了端点 0：无中断 1：有端点关闭中断	0x0
xfercompl(m)	[0]	W1C	适用于 IN 和 OUT 端点。 此字段表示此端点的 AHB 和 USB 上的传输已完成。 0：无中断 1：有传输完成中断	0x0

27.7.47 DIEPTSIZ(m)

◆ **偏移地址：0x910+0x20*m　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	mc		pktcnt(m)										xfersize(m)		
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xfersize(m)															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31]	-	保留。	0x0
mc	[30:29]	RW	仅适用于 IN 端点。 对于周期性 IN 端点，此字段指示 USB 上每帧必须传输的数据包数。内核使用该字段计算同步 IN 端点的数据 PID。 1：1 packet 2：2 packets 3：3 packets 注意：当 m=0 时，该字段无效。	0x0
pktcnt(m)	[28:19]	RW	数据包计数。 指示构成终结点 0 的传输大小数据量的 USB 数据包总数。 每次从 TX FIFO 读取数据包（最大大小或短数据包）时，该字段都会递减。 注意：当 m=0 时，有效字段仅[20:19]有效。	0x0
xfersize(m)	[18:0]	RW	传输大小。 指示终结点的传输大小（以字节为单位）。只有在应用程序耗尽了传输大小的数据量之后，内核才会中断应用程序。传输大小可以设置为端点的最大数据包大小，在每个数据包结束时中断。 每次将来自外部存储器的数据包写入 TX FIFO 时，控制器都会递减此字段。 注意：当 m=0 时，有效字段[6:0]有效。	0x0

27.7.48 DTXFSTS(m)

◆ **偏移地址：0x918+0x20*m　　复位值：0x000005E0**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ineptxfspcavail(m)															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	1	0	1	1	1	1	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
ineptxfspcavail(m)	[15:0]	RO	IN 端点 TX FIFO 可用空间。 指示端点 TX FIFO 中可用的可用空间，32bit 的 word 为单位。 0：端点 TX FIFO 已满 1：1 个 word 可用空间 2：2 个 word 可用空间 n：n 个 word 可用空间（其中 0<=n<=0x5e0） 其他：保留	0x5e0

27.7.49 DOEPCTL(m)

◆ **偏移地址：0xB00+0x20*m　　复位值：0x00008000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
epena(m)	epdis(m)	setd1pid(m)	setd0pid(m)	snak(m)	cnak(m)	Res.				stall(m)	Res.	eptype(m)		naksts(m)	dpid(m)
RW	RW	RW	RW	RW	RW	-	-	-	-	RW	-	RW	RW	RW	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
usbactep(m)	Res.				mps(m)										
RW	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
epena(m)	[31]	RW	此位表示应用程序已分配内存以开始从 USB 接收数据。 在该端点上产生以下任何中断之前，控制器会先清除 EPEna 位： （1）SETUP 阶段完成 （2）端点已禁用 （3）传输已完成 0：无操作 1：端点使能	0x0
epdis(m)	[30]	RW	端点禁用	0x0

Name	Bits	R/W	Description	Default
			m=0 时此字段无效为 RO 类型,应用程序无法禁用控制端点 0 m=1~5 时 0：无操作 1：禁用端点	
setd1pid(m)	[29]	RW	m=0 时此字段无效 m=1~5 时此字段有效 设置 DATA1 PID： （1）用于中断和批量端点。 （2）此字段会将此寄存器中的端点 DPID 字段设置为 DATA1。 设置奇帧（SetOddFr）： （1）仅适用于同步端点。 （2）写入该字段会将 EO_FrNum 字段设置为奇数帧。 0：禁用设置 DATA1 PID 或不强制奇帧 1：将端点数据 PID 设置为 DATA1 或将 EO_FrNum 字段设置为奇帧	0x0
setd0pid(m)	[28]	RW	m=0 时此字段无效 m=1~5 时此字段有效 设置 DATA0 PID(SetD0PID)： （1）仅适用于中断/批量端点。 （2）写入此字段会将此寄存器中的 DPID 字段设置为 DATA0。 设置偶帧(SetEvenFr)： （1）仅适用于同步端点。 （2）写入该字段会将 EO_FrNum 字段设置为偶数帧。 0：禁用设置 DATA0 PID 或不强制偶帧 1：将端点数据 PID 设置为 DATA0 或将 EO_FrNum 字段设置为偶数帧	0x0
snak(m)	[27]	RW	对该位的写入为端点设置 NAK 位。使用这个 bit，应用程序可以控制端点上 NAK 握手的传输。控制器还可以在传输完成中断时设置位，或者在端点上接收到 SETUP 之后设置位。 0：无操作 1：设置 NAK	0x0
cnak(m)	[26]	RW	对该位的写入将清除端点的 NAK 位。 0：无操作 1：清除 NAK	0x0
reserved	[25:22]	-	保留。	0x0
stall(m)	[21]	RW	STALL 握手。	0x0

Name	Bits	R/W	Description	Default
			当接收到此端点的 SETUP 令牌时，应用程序只能设置此位，控制器将其清除。如果 NAK 位或 Global OUT NAK 与该位一起被设置，则 STALL 位具有优先级。不管这个比特的设置如何，控制器总是用 ACK 握手来响应 SETUP 数据包。 0：无 STALL 1：STALL 握手	
reserved	[20]	-	保留。	0x0
eptype(m)	[19:18]	RW	端点类型。 m=0 时此字段为 RO 类型,硬编码为 0 表示为控制端点。 0：控制端点 1：禁用 2：禁用 3：禁用 m=1~5 时此字段为 RW 类型。 0：禁用 1：同步端点 2：批量端点 3：中断端点	0x0
naksts(m)	[17]	RW	NAK 状态 表示以下内容： 0：控制器基于 FIFO 状态传输非 NAK 握手。 1：控制器正在该端点上传输 NAK 握手。 当应用程序或控制器设置此位时，即使 RX FIFO 中有空间容纳传入的数据包，控制器也会停止接收数据。不管这个比特的设置如何，控制器总是用 ACK 握手来响应 SETUP 数据包。	0x0
dpid(m)	[16]	RO	端点数据 PID。 仅适用于中断/批量 IN 和 OUT 端点。 包含要在此端点上接收或传输的数据包的 PID。在端点被激活后，应用程序必须对要在此端点上接收或传输的第一个数据包的 PID 进行编程。应用程序使用该寄存器的 SetD1PID 和 SetD0PID 字段对 DATA0 或 DATA1 PID 进行编程。 0：DATA0 1：DATA1 注意：当 m=0 时，该字段无效	0x0
usbactep(m)	[15]	RW	USB 活动端点。 此位始终设置为 1，表示控制端点 m 在所有配置和接口中始终处于活动状态。 1：USB 端点 0 处于活动状态	0x1

Name	Bits	R/W	Description	Default
reserved	[14:11]	-	保留。	0x0
mps(m)	[10:0]	RW	最大数据包大小。 控制输出端点 0 的最大数据包大小与控制输入端点 0 的最大数据包大小相同。 注意：当 m=0 时，只有[1:0]两 bit 有效 0：64 字节 1：32 字节 2：16 字节 3：8 字节	0x0

27.7.50 DOEPINT(m)

◆ **偏移地址：0xB08+0x20*m　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		nakintrpt(m)	bbleerr(m)	Res.			outpkterr(m)	Res.	back2backsetup(m)	stsphsercvd(m)	outtknepdis(m)	setup(m)	Res.	epdisbld(m)	xfercompl(m)
-	-	W1C	W1C	-	-	-	W1C	-	W1C	W1C	W1C	W1C	-	W1C	W1C
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
nakintrpt(m)	[13]	W1C	NAK 中断。 当 NAK 被设备发送或接收时，控制器产生该中断。 在同步 In 端点的情况下，当由于 TX FIFO 中的数据不可用而发送零长度分组时，会产生中断。 0：无中断 1：有中断	0x0
bbleerr(m)	[12]	W1C	Babble 错误中断。 当接收到针对端点的 Babble 错误时，控制器会生成此中断。 0：无中断 1：有中断	0x0
reserved	[11:9]	-	保留。	0x0
outpkterr(m)	[8]	W1C	OUT 数据包错误。 仅适用于 OUT 端点。 此中断仅在启用阈值时有效。	0x0

Name	Bits	R/W	Description	Default
			当控制器检测到非等时 OUT 数据包的溢出或 CRC 错误时，该中断被触发 0：无中断 1：有中断	
reserved	[7]	-	保留。	0x0
back2backsetup(m)	[6]	W1C	接收到的 Back-to-Back SETUP 数据包。 仅适用于控制 OUT 端点。 该位指示内核已收到超过三个此特定端点的 Back-to-Back SETUP 数据包。 0：没有收到 Back-to-Back SETUP 数据包 1：收到 Back-to-Back SETUP 数据包 注意：当 m=0 时，该字段无效。	0x0
stsphsercvd(m)	[5]	W1C	接收到控制写入的状态阶段。 此中断仅对 Control OUT 端点有效。 只有在内核将主机在控制写入传输的数据阶段发送的所有数据传输到系统内存缓冲区时，才会产生此中断。 中断向应用程序指示主机已从数据阶段切换到控制写入传输的状态阶段。在对数据阶段进行解码后，应用程序可以使用此中断来确认或暂停状态阶段。 数值： 0：未接收到控制写入的状态阶段 1：接收到控制写入的状态相位	0x0
outtknepdis(m)	[4]	W1C	端点禁用时接收到 OUT 令牌。 仅适用于控制 OUT 端点。 指示在端点尚未启用时接收到 OUT 令牌。此中断在接收到 OUT 令牌的端点上触发。 0：禁用端点时未接收到 OUT 令牌 1：端点禁用时接收到 OUT 令牌	0x0
setup(m)	[3]	W1C	设置阶段完成。 仅适用于控制 OUT 端点。 指示控制端点的 SETUP 阶段已完成，并且当前控制传输不再接收到 back-to-back SETUP 数据包。在此中断中，应用程序可以对接收到的 SETUP 数据包进行解码。 0：未完成 SETUP 阶段 1：SETUP 阶段完成	0x0
reserved	[2]	-	保留。	0x0
epdisbld(m)	[1]	W1C	端点禁用中断。 适用于 IN 和 OUT 端点。 此位表示根据应用程序的请求禁用端点。	0x0

Name	Bits	R/W	Description	Default
			0：无端点禁用中断 1：端点禁用中断	
xfercompl(m)	[0]	W1C	传输完成中断。 适用于 IN 和 OUT 端点。 此字段表示此端点的 AHB 和 USB 上的编程传输已完成。 0：无传输完全中断 1：传输完成中断	0x0

27.7.51 DOEPTSIZ(m)

◆ **偏移地址：0xB10+0x20*m　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	rxdpid(m)		pktcnt(m)										xfersize(m)		
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xfersize(m)															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31]	-	保留。	0x0
rxdpid(m)	[30:29]	RW	RxDPID。 当 m 不等于 0 时： （1）仅适用于同步 OUT 端点。 （2）这是在该端点的最后一个数据包中收到的数据 PID。 0：DATA0 1：DATA2 2：DATA1 3：MDATA 当 m 等于 0 时： 此字段指定端点可以接收的 SETUP 数据包的数量。 1：1 packet 2：2 packets 3：3 packets	0x0
pktcnt(m)	[28:19]	RW	数据包计数。 在数据包被写入 RX FIFO 之后，该字段被递减到零。 注意：当 m=0 时，只有[19]有效。	0x0
xfersize(m)	[18:0]	RW	传输大小。 指示终结点 0 的传输大小（以字节为单位）。只有在应用程序耗尽了传输大小的数据量之后，内核才会中	0x0

Name	Bits	R/W	Description	Default
			断应用程序。传输大小可以设置为端点的最大数据包大小，在每个数据包结束时中断。 每次从 RX FIFO 读取数据包并将其写入外部存储器时，控制器都会递减此字段。 注意：当 m=0 时，只有[6:0]有效	

27.7.52 PCGCCTL

◆ **偏移地址：0xE00　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

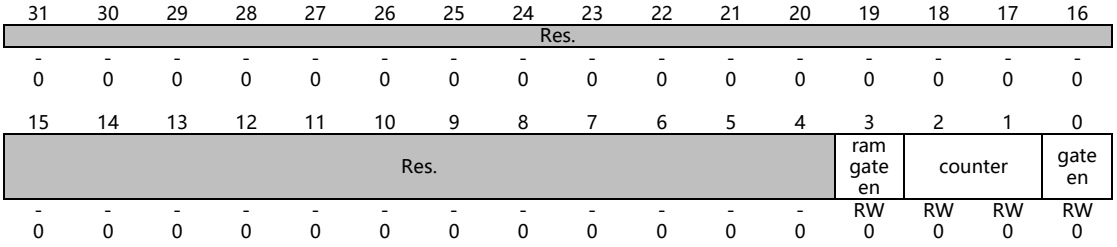
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								l1suspended	physleep	Res.		rstpdwnmodule	Res.		stoppclk
-	-	-	-	-	-	-	-	RO	RO	-	-	RW	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
l1suspended	[7]	RO	L1 深度睡眠。 指示 PHY 处于 L1 状态时处于深度睡眠状态。 0：非深度睡眠 1：深度睡眠	0x0
physleep	[6]	RO	PHY 处于睡眠状态。 表示物理层处于睡眠状态。 0：PHY 未处于睡眠状态 1：PHY 处于睡眠状态	0x0
reserved	[5:4]	-	保留。	0x0
rstpdwnmodule	[3]	RW	重置掉电模块。 此位仅在部分断电模式下有效。 应用程序在电源关闭时设置此位。 在电源打开且 PHY 时钟启动后，应用程序将清除此位。 注意：只有当该位设置为 0 时，才能读/写所有控制器寄存器 0：电源开启 1：电源关闭	0x0
reserved	[2:1]	-	保留。	0x0
stoppclk	[0]	RW	关闭 Pclk。 当USB挂起、会话无效或设备断开连接时，应用程序	0x0

Name	Bits	R/W	Description	Default
			将此位置 1 以停止 PHY 时钟（PHY_clk）。 当USB恢复或新会话开始时，应用程序会清除此位。 0：不关闭 Pclk 1：关闭 Pclk	

27.7.53 PCGCCTL1

◆ **偏移地址：0xE04　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
ramgateen	[3]	RW	RAM 时钟门控启用。 应用程序 RAMGateEn 在 ACG 功能中启用 RAM 时钟门控。 0：禁用 RAM 时钟门控 1：启用 RAM 时钟门控	0x0
counter	[2:1]	RW	计数到门时钟。 CntGateClk 向控制器指示控制器在选通控制器内部的相应 PHY 和 AHB 时钟之前等待多少个 PHY 时钟周期和 AHB 钟周期。 0：64 个时钟（默认值）。 1：128 个时钟	0x0
gateen	[0]	RW	启用活动时钟门控。 应用程序 GateEn 为 PHY 和 AHB 时钟启用活动时钟门控功能。 0：禁用时钟门控 1：启用时钟门控	0x0

27.7.54 EP0/CH0 FIFO

◆ **偏移地址：0x1000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ep0/ch0 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ep0/ch0 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ep0/ch0 fifo	[31:0]	RW	端点 0/通道 0 的数据 FIFO 寄存器	0x0

27.7.55 EP1/CH1 FIFO

◆ **偏移地址：0x2000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ep1/ch1 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ep1/ch1 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ep1/ch1 fifo	[31:0]	RW	端点 1/通道 1 的数据 FIFO 寄存器	0x0

27.7.56 EP2/CH2 FIFO

◆ **偏移地址：0x3000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ep2/ch2 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ep2/ch2 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ep2/ch2 fifo	[31:0]	RW	端点 2/通道 2 的数据 FIFO 寄存器	0x0

27.7.57 EP3/CH3 FIFO

◆ **偏移地址：0x4000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ep3/ch3 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ep3/ch3 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ep3/ch3 fifo	[31:0]	RW	端点 3/通道 3 的数据 FIFO 寄存器	0x0

27.7.58 EP4/CH4 FIFO

◆ **偏移地址：0x5000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ep4/ch4 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ep4/ch4 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ep4/ch4 fifo	[31:0]	RW	端点 4/通道 4 的数据 FIFO 寄存器	0x0

27.7.59 EP5/CH5 FIFO

◆ **偏移地址：0x6000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ep5/ch5 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ep5/ch5 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ep5/ch5 fifo	[31:0]	RW	端点 5/通道 5 的数据 FIFO 寄存器	0x0

27.7.60 CH6 FIFO

◆ **偏移地址：0x7000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ch6 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ch6 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ch6 fifo	[31:0]	RW	通道 6 的数据 FIFO 寄存器	0x0

27.7.61 CH7 FIFO

◆ **偏移地址：0x8000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ch7 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ch7 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ch7 fifo	[31:0]	RW	通道 7 的数据 FIFO 寄存器	0x0

27.7.62 CH8 FIFO

◆ **偏移地址：0x9000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ch8 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ch8 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ch8 fifo	[31:0]	RW	通道 8 的数据 FIFO 寄存器	0x0

27.7.63 CH9 FIFO

◆ **偏移地址：0xA000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ch9 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ch9 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ch9 fifo	[31:0]	RW	通道 9 的数据 FIFO 寄存器	0x0

27.7.64 CH10 FIFO

◆ **偏移地址：0xB000　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ch10 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ch10 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ch10 fifo	[31:0]	RW	通道 10 的数据 FIFO 寄存器	0x0

27.7.65 CH11 FIFO

◆ **偏移地址：0xC000　　复位值：0x00000000**

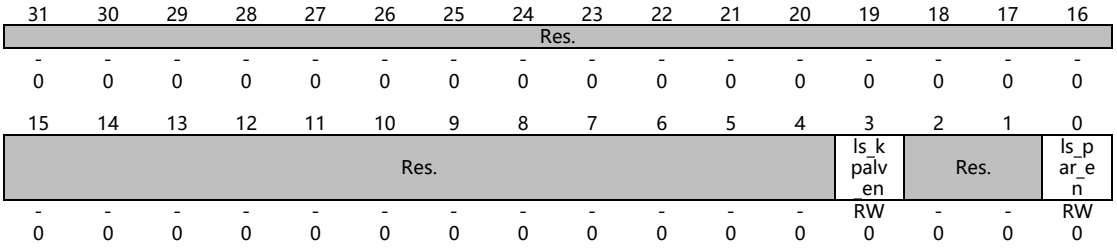
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ch11 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ch11 fifo															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
ch11 fifo	[31:0]	RW	通道 11 的数据 FIFO 寄存器	0x0

27.7.66 U2PHY_LS_CTRL

◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
ls_kpalv_en	[3]	RW	LS 模式下 keep-alive 使能。 0：不使能(默认) 1：使能	0x0
reserved	[2:1]	-	保留。	0x0
ls_par_en	[0]	RW	LS 模式下并口使能。 0：不使能(默认) 1：使能	0x0

28 安全数字输入输出接口（SDIO）

28.1 SDIO 简介

SDIO 定义了 SD、SD I/O 和 EMMC 设备的主机(Host)接口，提供 AHB Slave 接口用于配置寄存器；提供 AHB Master 接口，通过内置 DMA 与 System memory 之间进行数据交互。它作为 Host 与支持 SD2.0 协议的 SD、SD I/O Device 及支持 eMMC4.2 协议的 EMMC Device 通信。

28.2 SDIO 主要特性

SDIO 的功能特点有：

- 支持 SDSC、SDHC、SDXC 格式 SD 卡及 SDIO、EMMC 设备。
- 支持一线(1bit)和四线(4bit)模式 SD 总线。
- 支持一线(1bit)、四线(4bit)和八线(8bit)模式 EMMC 总线。
- 支持 SD 时钟最高 25MHz。
- 支持 3.3V。
- 支持 Read Wait 和 Suspend/Resume 功能。
- 支持卡识别和硬件写保护功能。
- 不支持 SPI 模式。

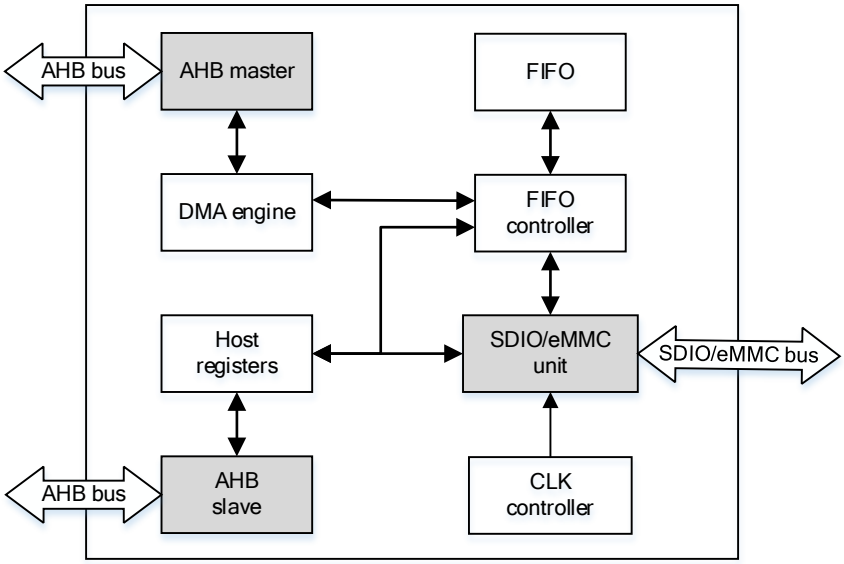
28.3 SDIO 功能说明

28.3.1 SDIO 框图

图 28-1 为 SDIO 结构框图，SDIO 主要由 AHB slave、AHB master、FIFO、

SDIO/eMMC unit 及 CLK controller 等模块组成。AHB slave 为 Host 的配置寄存器和 Non_DMA 数据传输时提供总线接口；Host register 为 Host 的寄存器模块；AHB master 为内置 DMA 提供总线接口；DMA engine 为内置 DMA 控制模块；FIFO controller 为乒乓 FIFO 控制模块；SDIO/eMMC unit 为 SDIO 协议解析模块，收发与 SDIO host 之间的命令和数据；CLK controller 为时钟复位控制模块。

图 28-1 SDIO 结构框图



28.3.2 SDIO 端口

表 28-1 介绍了芯片 SDIO 端口及其功能。

表 28-1 SDIO 端口

端口名	IO	功能
SDIO_WP	I	卡写保护状态信号
SDIO_CD	I	卡检测状态信号
SDIO_D7	I/O	数据信号 7
SDIO_D6	I/O	数据信号 6
SDIO_D5	I/O	数据信号 5
SDIO_D4	I/O	数据信号 4
SDIO_D3	I/O	数据信号 3
SDIO_D2	I/O	数据信号 2
SDIO_D1	I/O	数据信号 1
SDIO_D0	I/O	数据信号 0

端口名	IO	功能
SDIO_CMD	I/O	命令和应答信号
SDIO_CLK	O	时钟信号
SDIO_RST	O	复位信号
SDIO_VDD1_ON	O	电源开关信号

卡写保护状态信号(SDIO_WP)仅用于指示 SD 卡是否开启写保护功能。

卡检测状态信号(SDIO_CD)仅用于检测 SD 卡或 TF 卡是否插入 SDIO 接口。

数据信号(SDIO_D0~SDIO_D7)用于与设备通信过程中的数据传输，当采用一线(1bit)模式时，只有 SDIO_D0 有效，SDIO_D1~SDIO_D7 保持高电平，若采用四线(4bit)模式时，SDIO_D0~SDIO_D3 有效，SDIO_D4~SDIO_D7 保持高电平，八线(8bit)模式下，SDIO_D0~SDIO_D7 均有效，且八线(8bit)模式仅限于 EMMC 设备通信。

命令和应答信号(SDIO_CMD)用于传输 SDIO Host 发出的命令(CMD)和设备的应答(Response)。

时钟信号(SDIO_CLK)用于输出与 SDIO、SD 和 EMMC 设备通信时的 SD 时钟。

复位信号(SDIO_RST) 仅用于控制 EMMC 复位。

电源开关信号(SDIO_VDD1_ON)用于控制 SDIO、SD 和 EMMC 电源的开关。

⚠注意

卡电源开关信号(SDIO_VDD1_ON)通过配置寄存器 PWR_CTRL_R.sd_bus_pwr_vdd1 控制，该信号通过 PD2 或者 PE5 管脚（需复用成 SDIO_VDD1_ON 功能）输出，该信号的输出极性可由 CLK_CTRL_R.cfg_vdd_on_polarity 配置，可用于控制对接设备的上下电。

SDIO 的控制逻辑与 PWR_CTRL_R.sd_bus_pwr_vdd1 位域的状态强关联，无论对接什么设备，PWR_CTRL_R.sd_bus_pwr_vdd1 必须配置 1，SDIO 的控制逻辑才能正常工作。

28.3.3 命令与响应

SDIO Host 与 SDIO、SD 和 EMMC 设备之间的所有交互操作均通过命令完成，包括设备初始化、设备寄存器读写、设备状态查询、通信数据传输等。

SDIO 命令(CMD)为 48bit 的串行数据，通过 SDIO_CMD 发送给设备，由起始位(1bit)、传输位(1bit)、命令号(6bit)、命令参数(32bit)、CRC 校验位(7bit)和结束位(1bit)组成，命令格式如表 28-2 所示。

表 28-2 命令格式

Bit 位	47	46	[45:40]	[39:8]	[7:1]	0
宽度(bit)	1	1	6	32	7	1
值	0x0	0x1	x	x	x	0x1
描述	起始位	传输位	命令号	参数	CRC7 校验位	结束位

根据响应的类型，命令可分为 3 种。

- **无响应命令：**对于该种命令，设备没有响应，如复位命令。
- **短响应命令：**对于该种命令，设备有 48bit 长度的响应返回给 SDIO Host，如数据传输、卡状态查询等命令。
- **长响应命令：**对于该种命令，设备有 136bit 长度的响应返回给 SDIO Host，如 CID 寄存器的内容作为 CMD2 或 CMD10 命令的响应。

短响应 R1(R1b)格式如表 28-3 所示。

短响应 R3 格式如表 28-4 所示。

短响应 R4 如表 28-5 和表 28-6 所示。

短响应 R5 格式如表 28-7 和表 28-8 所示。

短响应 R6 格式如表 28-9 所示。

短响应 R7 格式如表 28-10 所示。

长响应只有 R2，格式如表 28-11 所示。

表 28-3 R1/R1b(普通响应)格式

Bit 位	47	46	[45:40]	[39:8]	[7:1]	0
宽度(bit)	1	1	6	32	7	1
值	0x0	0x0	x	x	x	0x1
描述	起始位	传输位	命令号	卡状态	CRC 校验位	结束位

表 28-4 R3(OCR 响应)格式

Bit 位	47	46	[45:40]	[39:8]	[7:1]	0
宽度(bit)	1	1	6	32	7	1
值	0x0	0x0	0x3F	x	0x7F	0x1
描述	起始位	传输位	保留	OCR 寄存器	保留	结束位

表 28-5 R4(CMD5 命令的响应)格式

Bit 位	47	46	[45:40]	39	[38:36]	[35]	[34:33]	[32]	[31:8]	[7:1]	0
宽度(bit)	1	1	6	1	3	1	2	1	24	7	1
值	0x0	0x0	0x3F	x	x	x	0x0	x	x	0x7F	0x1
描述	起始位	传输位	保留	初始化标志	IO 功能编号	是否带 memory	保留	S18A	I/O OCR	保留	结束位

表 28-6 R4(CMD39 命令的响应)格式

Bit 位	47	46	[45:40]	[39:24]	[23]	[22:16]	[15:8]	[7:1]	0
宽度(bit)	1	1	6	16	1	7	8	7	1
值	0x0	0x0	0x27	x	x	x	x	x	0x1
描述	起始位	传输位	CMD39	RCA[31:16]	状态	寄存器地址	寄存器内容	CRC7	结束位

表 28-7 R5(CMD52 命令的响应)格式

Bit 位	47	46	[45:40]	[39:24]	[23:16]	[15:8]	[7:1]	0
宽度(bit)	1	1	6	16	8	8	7	1
值	0x0	0x0	0x34	0x0	x	x	x	0x1
描述	起始位	传输位	命令号	Not used	响应标志	读写值	CRC 校验位	结束位

表 28-8 R5(CMD40 命令的响应)格式

Bit 位	47	46	[45:40]	[39:24]	[23:8]	[7:1]	0
宽度(bit)	1	1	6	16	16	7	1
值	0x0	0x0	0x28	x	x	x	0x1
描述	起始位	传输位	CMD40	RCA[31:16]	未定义，可用于 IRQ 数据	CRC7	结束位

表 28-9 R6(RCA 响应)格式

Bit 位	47	46	[45:40]	[39:24]	[23:8]	[7:1]	0
宽度(bit)	1	1	6	16	16	7	1
值	0x0	0x0	0x3	x	x	x	0x1
描述	起始位	传输位	命令号	RCA	卡的状态位	CRC7	结束位

表 28-10 R7(卡接口条件响应)格式

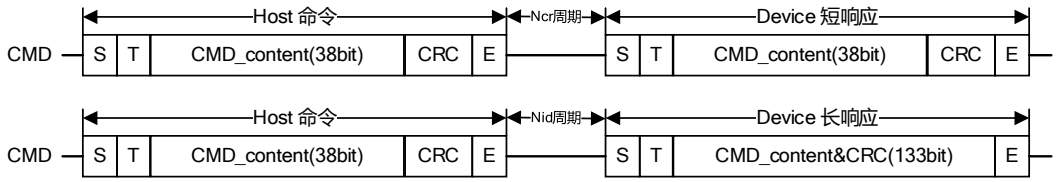
Bit 位	47	46	[45:40]	[39:20]	[19:16]	[15:8]	[7:1]	0
宽度(bit)	1	1	6	20	4	8	7	1
值	0x0	0x0	0x8	0x0	x	x	x	0x1
描述	起始位	传输位	命令号	保留位	接受电压	检查模式	CRC7	结束位

表 28-11 R2（CID/CSD 响应）格式

Bit 位	135	134	[133:128]	[127:1]	0
宽度(bit)	1	1	6	127	1
值	0x0	0x0	0x3F	x	0x1
描述	起始位	传输位	保留位	CID/CSD 内置 CRC7	结束位

SDIO 命令与短响应之间的间隔为 Ncr 周期，其值为 2-64 个 clk；SDIO 命令与长响应之间的间隔为 Nid 周期，其值为 5 个 clk。SDIO 命令与响应时序如图 28-2 示。

图 28-2 命令与响应时序



根据是否有数据传输，命令可分为 2 种。

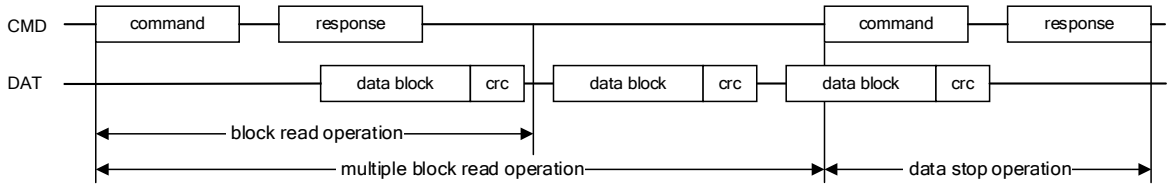
- **无数据传输命令：**该种命令只在 SDIO_CMD 线上有命令发送与响应接收。
- **有数据传输命令：**除了 SDIO_CMD 线上有命令与响应交互外，SDIO_Dx(0-7)线上还有数据传输。

28.3.4 数据传输

SDIO Host 与 SDIO、SD 和 EMMC 设备之间的数据传输以数据块(block)为基本单位，SD 和 EMMC 设备传输数据块最大为 512byte，而 SDIO 设备传输数据块可根据实际应用自定义，数据块以 CRC 位来保证数据的正确性。数据传输命令的响应皆为短响应，且数据线上伴有数据传输。数据传输时 SDIO_CMD 线与 SDIO_DAT 线上的命令、响应及数据之间的时序关系如图 28-3 和图 28-4 所示。

单块与多块读数据传输

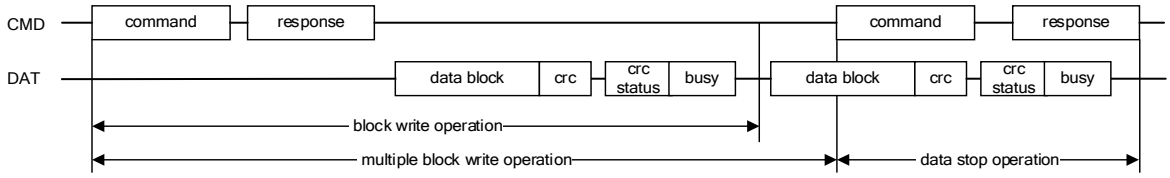
图 28-3 单块与多块读数据传输



SDIO Host 向 SDIO Device 发送单块或多块读命令(command)，SDIO Device 收到命令后通过 CMD 线返回响应(response)，此外，SDIO Device 也会通过 DAT 线将数据发送给 SDIO Host。数据传输以块(data block)为单位，并携带 crc 校验码。单块读操作时，SDIO Host 收到一个数据块后就完成该次数据传输。而多块读操作时，SDIO Host 将会收到多个数据块，数据传输停止则需要 SDIO Host 发出数据传输停止命令。

单块与多块写数据传输

图 28-4 单块与多块写数据传输

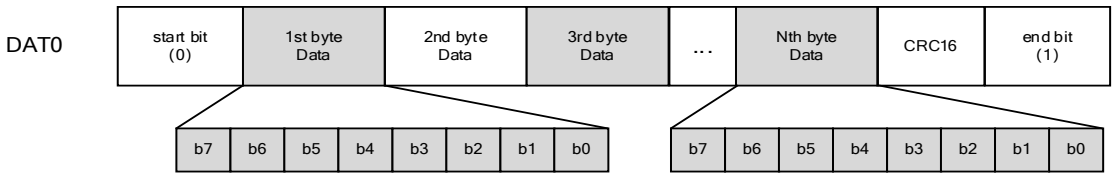


SDIO Host 向 SDIO Device 发送单块或多块写命令，在收到 SDIO Device 返回的响应后，数据随后通过 DAT 线向 SDIO Device 传输，SDIO Device 在接收完数据后，会根据接收到的数据 CRC 检验，并将结果 crc 状态(crc status)返回给 SDIO Host，crc 状态后面会紧跟 busy 信号，busy 信号(DAT0=0)表明 SDIO Device 还没有准备好接收下一个数据块。DAT0 线拉高，表明 busy 信号消失，SDIO Host 随即可把下一个数据块发送给 SDIO Device。同样，要结束多块写操作需要 SDIO Host 发出数据传输停止命令。

数据传输格式

数据传输有 1 线模式和 4 线模式。1 线模式的数据传输位宽为 1bit，此时只有 DAT0 线上有数据传输。而 4 线模式的数位宽为 4bit，此时 DAT0 至 DAT3 线上有数据传输。1 线和 4 线模式下的数据传输格式如图 28-5 和图 28-6 所示。

图 28-5 1 线模式数据传输格式



1 线模式传输时，第 1bit 是数据的起始标识位(start bit)，恒为 0；其后跟随的 N byte 数据，其中数据的传输顺序是先发低字节再发高字节，而每个字节(byte)内部的传输顺序则是先发高位后发低位(bit7 至 bit0)；在数据传输末尾紧跟 16bits 的 CRC 校验码；最后 1bit 是结束标识位(end bit)，恒为 1。

图 28-6 4 线模式数据传输格式

DAT	start bit (0)	1st byte Data		2nd byte Data		3rd byte Data		...	Nth byte Data		CRC16	end bit (1)
DAT0	0	b7	b3	b7	b3	b7	b3	...	b7	b3	CRC16	1
DAT1	0	b6	b2	b6	b2	b6	b2	...	b6	b2	CRC16	1
DAT2	0	b5	b1	b5	b1	b5	b1	...	b5	b1	CRC16	1
DAT3	0	b4	b0	b4	b0	b4	b0	...	b4	b0	CRC16	1

4 线传输模式的数据传输格式与 1 线类似，每条数据线上都有起始位、CRC 校验位及结束位，每条线的 CRC 校验码是独立的，且每条线单独进行 CRC 校验。传输顺序也是从低字节到高字节，同一字节的数据拆分到 4 条线上同时并行传输。

28.4 SDIO 工作方式

28.4.1 SDIO Host 复位和时钟配置

SDIO Host 的复位和时钟控制通过配置 CRG 的寄存器 PERI_CRG61、PERI_CRG62、PERI_CRG63、PERI_CRG64 实现。相关寄存器说明请参考章节 4 时钟和复位控制 (CRG)。

软复位

在数据传输中出现异常导致 SDIO Host 无法恢复正常时，可对 SDIO Host 进行复位操作。（以下 CRG 寄存器写“1”复位，写“0”释放复位）

- 向 CRG 寄存器 PERI_CRG61.sdio_b_srst_req 写“1”，对 SDIO Host 基本工作时钟域复位。
- 向 CRG 寄存器 PERI_CRG61.sdio_m_srst_req 写“1”，对内置 DMA 和 AHB master 复位。
- 向 CRG 寄存器 PERI_CRG64.sdio_card_div0_srst_req 写“1”，对产生 sd 时钟的逻辑复位。

时钟配置

连接不同的设备，以及同一设备不同的工作场景，可能使用不同的时钟频率。

- CRG 寄存器 PERI_CRG61.sdio_tm_div 控制产生计时器时钟 clk_timer 的分频系数。
- CRG 寄存器 PERI_CRG64.sdio_card_div0 控制产生 sd 时钟 clk_sdio_card 的分频系数。
- CRG 寄存器 PERI_CRG62.sdio_tx_cken 控制 tx 模块时钟 clk_sdio_tx 开关。
- CRG 寄存器 PERI_CRG63.sdio_rx_cksel 控制 rx 模块时钟 clk_sdio_rx 选择信号，用于 rx 时钟调相。

在改变时钟频率之前，为避免时钟切换产生毛刺，应当先关闭相应门控，再调整分频系数，最后再打开相应门控。

28.4.2 SDIO Host 初始化

连接 SDIO Device 之前，需要初始化 SDIO Host，步骤如下：

步骤 1 关闭中断信号使能。将寄存器 NORMAL_INT_SIGNAL_EN_R 和 ERROR_INT_SIGNAL_EN_R 所有位置“0”。

步骤 2 清中断。将寄存器 NORMAL_INT_STAT_R 和 ERROR_INT_STAT_R 所有位置“1”，清除原始中断状态位。

步骤 3 设置超时计数。配置寄存器 TOUT_CTRL_R.tout_cnt。

步骤 4 如果为 eMMC 设备，将寄存器 EMMC_CTRL_R.card_is_emmc 置“1”，否则置“0”。

步骤 5 打开 sd 及 timer 时钟门控使能。配置寄存器 CLK_CTRL_R.pll_enable、CLK_CTRL_R.sd_clk_en 和 CLK_CTRL_R.internal_clk_en 为“1”。

步骤 6 开启 power_on。将寄存器 PWR_CTRL_R.sd_bus_pwr_vdd1 置“1”。

步骤 7 打开中断状态使能。将寄存器 NORMAL_INT_STAT_EN_R 和 ERROR_INT_STAT_EN_R 所有位置“1”。

步骤 8 打开中断信号使能。将寄存器 NORMAL_INT_SIGNAL_EN_R 和 ERROR_INT_SIGNAL_EN_R 所有位置“1”。

以上步骤完成后，根据工作场景进行 sd 时钟配置，就可以往 SDIO Device 发送命令了。

⚠注意

中断状态使能，是控制中断事件状态寄存器的使能；中断信号使能是控制是否产生给 cpu 的中断信号使能。

28.4.3 无数据传输命令

SDIO Host 往 SDIO Device 发送无数据传输命令的步骤如下：

步骤 1 在寄存器 ARGUMENT_R 中设置命令的 argument 内容。

步骤 2 在寄存器 XFER_MODE_R 和 CMD_R 中设置相应命令参数。

步骤 3 等待 SDIO Host 发出命令。

步骤 4 检查寄存器 NORMAL_INT_STAT_R.cmd_complete 是否产生命令完成中断

步骤 5 检查寄存器 ERROR_INT_STAT_R 是否有异常响应中断，必要时可读取响应值。

步骤 6 根据已发命令查看响应(response)，回读寄存器 RESP01_R、RESP23_R、RESP45_R、RESP67_R。

28.4.4 单块或多块读数据

读取单块或多块数据的步骤如下：

步骤 1 清除中断寄存器 NORMAL_INT_STAT_R 和 ERROR_INT_STAT_R 的中断。

步骤 2 向寄存器 HOST_CTRL1_R.dma_sel 写入数据传输方式。

步骤 3 若数据传输方式选择 ADMA2，则先在系统 memory 上面创建 ADMA 描述链表，再向寄存器 ADMA_SA_LOW_R 写入 ADMA 描述链表存放的地址。

步骤 4 若数据传输方式选择 SDMA，则向寄存器 ADMA_SA_LOW_R 写入数据存入的起始地址。

步骤 5 向寄存器 BLOCKSIZE_R 写入块的大小。

步骤 6 向寄存器 SDMASA_R 和 BLOCKCOUNT_R 写入块的个数。

步骤 7 向寄存器 ARGUMENT_R 写入读取数据的起始地址(SDIO Device 内部的存储地址)。

步骤 8 向寄存器 XFER_MODE_R 和 CMD_R 写入数据传输参数。

须知

对于 SD 卡、eMMC 器件，分别使用命令 CMD17/CMD18 进行单块/多块读操作；对于 SDIO 卡，使用命令 CMD53 进行单块/多块读操作。

一旦寄存器 CMD_R 被写入，SDIO Host 就执行命令；当命令在 sd 总线上发送完成后，产生 cmd_done 中断。

步骤 9 若数据传输方式选择 Non_DMA 方式读取数据，检查寄存器 NORMAL_INT_STAT_R.buf_rd_ready，若为 1，则从寄存器 BUF_DATA_R 读取 FIFO 中的数据，以便 SDIO Host 接收后面的数据；同时回读寄存器 ERROR_INT_STAT_R 检查数据错误中断，若有错误中断产生，可以发送一个停止命令中止数据的传输。

步骤 10 当寄存器 NORMAL_INT_STAT_R.xfer_complete 为 1 时，通过读寄存器 BUF_DATA_R 获取 FIFO 中的数据，完成整个读操作流程。

28.4.5 单块或多块写数据

单块或多块写数据的步骤如下：

步骤 1 清除中断寄存器 NORMAL_INT_STAT_R 和 ERROR_INT_STAT_R 中的中断。

步骤 2 向寄存器 HOST_CTRL1_R.dma_sel 写入数据传输方式。

步骤 3 若数据传输方式选择 ADMA2，则先在系统 memory 上面创建 ADMA 描述链表，再向寄存器 ADMA_SA_LOW_R 写入 ADMA 描述链表存放的地址。

步骤 4 若数据传输方式选择 SDMA，则向寄存器 ADMA_SA_LOW_R 写入将要发送数据的起始地址。

步骤 5 向寄存器 BLOCKSIZE_R 写入块的大小。

步骤 6 向寄存器 SDMASA_R 和 BLOCKCOUNT_R 写入块的个数。

步骤 7 向寄存器 ARGUMENT_R 写入写数据的起始地址(SDIO Device 内部的存储地址)。

步骤 8 向寄存器 XFER_MODE_R 和 CMD_R 写入数据传输参数。

须知

对于 SD 卡、eMMC 器件，分别使用命令 CMD24/CMD25 进行单块/多块写操作；对于 SDIO 卡，使用命令 CMD53 进行单块/多块写操作。

步骤 9 若数据传输方式选择 Non_DMA 方式，则检查寄存器 NORMAL_INT_STAT_R.buf_wr_ready，如果为 1，则向寄存器 BUF_DATA_R 写

入要填充到 FIFO 中的数据，以便 SDIO Host 发送后面的数据；同时回读寄存器 ERROR_INT_STAT_R 检查数据错误中断，若有错误中断产生，可以发送一个停止命令中止数据的传输。

步骤 10 当寄存器 NORMAL_INT_STAT_R.xfer_complete 为 1 时，数据传输完成。

28.5 SDIO 寄存器概述

SDIO Host 的基地址如下：

- SDIO Host 寄存器基地址为：0x63060000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	SDMASA_R	0x0	32	RW	0x00000000	SDMA 系统地址寄存器
2	BLOCKSIZE_R	0x4	16	RW	0x0000	块大小配置寄存器
3	BLOCKCOUNT_R	0x6	16	RW	0x0000	块个数配置寄存器
4	ARGUMENT_R	0x8	32	RW	0x00000000	命令参数寄存器
5	XFER_MODE_R	0xC	16	RW	0x0000	传输模式配置寄存器
6	CMD_R	0xE	16	RW	0x0000	命令寄存器
7	RESP01_R	0x10	32	RW	0x00000000	RESP01 响应寄存器
8	RESP23_R	0x14	32	RW	0x00000000	RESP23 响应寄存器
9	RESP45_R	0x18	32	RW	0x00000000	RESP45 响应寄存器
10	RESP67_R	0x1C	32	RW	0x00000000	RESP67 响应寄存器
11	BUF_DATA_R	0x20	32	RW	0x00000000	数据寄存器
12	PSTATE_REG	0x24	32	RO	0x00000000	当前状态寄存器
13	HOST_CTRL1_R	0x28	8	RW	0x00	HOST 控制寄存器 1
14	PWR_CTRL_R	0x29	8	RW	0x00	上电控制寄存器
15	BGAP_CTRL_R	0x2A	8	RW	0x00	块间隙控制寄存器
16	WUP_CTRL_R	0x2B	8	RW	0x00	唤醒控制寄存器
17	CLK_CTRL_R	0x2C	8	RW	0x00	时钟控制寄存器
18	TOUT_CTRL_R	0x2E	8	RW	0x00	超时控制寄存器
19	SW_RST_R	0x2F	8	RW	0x00	软复位控制寄存器
20	NORMAL_INT_STAT_R	0x30	16	W1C	0x0000	普通中断状态寄存器
21	ERROR_INT_STAT_R	0x32	16	W1C	0x0000	错误中断状态寄存器
22	NORMAL_INT_STAT_EN_R	0x34	16	RW	0x0000	普通中断使能寄存器
23	ERROR_INT_STAT_EN_R	0x36	16	RW	0x0000	错误中断使能寄存器
24	NORMAL_INT_SIGNAL_EN_R	0x38	16	RW	0x0000	普通中断信号使能寄存器
25	ERROR_INT_SIGNAL_EN_R	0x3A	16	RW	0x0000	错误中断信号使能寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
26	AUTO_CMD_STAT_R	0x3C	16	RO	0x0000	Auto CMD 状态寄存器
27	HOST_CTRL2_R	0x3E	16	RW	0x0000	HOST 控制寄存器 2
28	CAPABILITIES1_R	0x40	32	RO	0x23CC0081	能力 1 寄存器
29	ADMA_ERR_STAT_R	0x54	32	RO	0x00000000	ADMA 错误状态寄存器
30	ADMA_SA_LOW_R	0x58	32	RW	0x00000000	ADMA 系统地址低 32bit 配置寄存器
31	MSHC_CTRL_R	0x508	8	RW	0x00	MSHC 控制寄存器
32	EMMC_CTRL_R	0x52C	16	RW	0x000C	EMMC 控制寄存器

28.6 SDIO 寄存器描述

28.6.1 SDMASA_R

- 偏移地址：0x0　　复位值：0x00000000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
blockcnt_sdmasa															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
blockcnt_sdmasa															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
blockcnt_sdmasa	[31:0]	RW	此寄存器是一个 32 位的块计数寄存器或 SDMA 系统地址配置寄存器。 当 host_ver4_enable=0，该寄存器表示 SDMA 系统地址。 当 host_ver4_enable=1，该寄存器是一个 32 位的递减计数器，表示剩余待传输的数据块的数量。	0x0

28.6.2 BLOCKSIZE_R

- 偏移地址：0x4　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	sdma_buf_bdary			xfer_block_size											
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15]	-	保留。	0x0
sdma_buf_bdary	[14:12]	RW	SDMA 数据块边界值 0：4K bytes； 1：8K bytes； 2：16K bytes； 3：32k bytes； 4：64K bytes； 5：128K bytes； 6：256K bytes； 7：512K bytes。	0x0
xfer_block_size	[11:0]	RW	数据块大小设置 1: 1 byte 2: 2 bytes 3: 3 bytes …… 512：512bytes。	0x0

28.6.3 BLOCKCOUNT_R

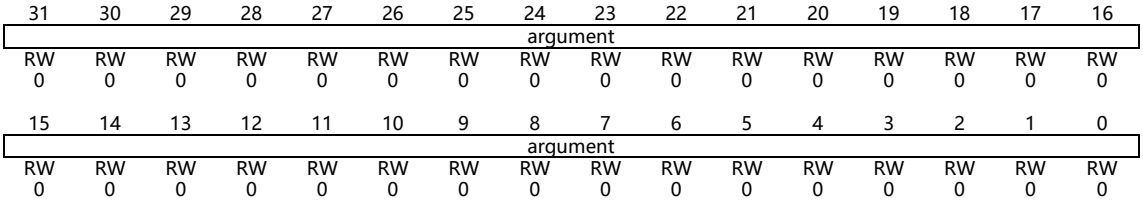
- 偏移地址：0x6　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
block_cnt															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
block_cnt	[15:0]	RW	数据块个数 1：1 个块； …… 65535：65535 个块。	0x0

28.6.4 ARGUMENT_R

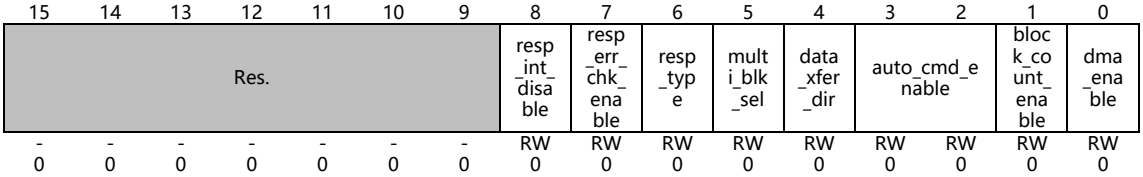
- 偏移地址：0x8　　复位值：0x00000000



Name	Bits	R/W	Description	Default
argument	[31:0]	RW	命令的参数，32bit 命令内容值。	0x0

28.6.5 XFER_MODE_R

- 偏移地址：0xC　　复位值：0x0000



Name	Bits	R/W	Description	Default
reserved	[15:9]	-	保留。	0x0
resp_int_disable	[8]	RW	响应中断使能 0：使能； 1：关闭。	0x0
resp_err_chk_enable	[7]	RW	响应错误检查使能 0：不使能； 1：使能。	0x0
resp_type	[6]	RW	R1/R5 响应类型选择 0：R1； 1：R5。	0x0
multi_blk_sel	[5]	RW	单块、多块数据传输选择 0：单块传输； 1：多块传输。	0x0
data_xfer_dir	[4]	RW	数据传输方向选择 0：SDIO Host 到 SDIO Device； 1：SDIO Device 到 SDIO Host。	0x0
auto_cmd_enable	[3:2]	RW	Auto CMD 使能控制	0x0

Name	Bits	R/W	Description	Default
			0：不使能 Auto CMD； 1：使能 Auto CMD12； 2：使能 Auto CMD23； 3：使能 Auto CMD，命令类型自动选择。	
block_count_enable	[1]	RW	数据块计数使能 0：关闭； 1：使能。	0x0
dma_enable	[0]	RW	DMA 使能 0：无数据传输或者非 DMA 数据(Non_DMA)传输； 1：DMA 数据传输。	0x0

28.6.6 CMD_R

- 偏移地址：0xE　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		cmd_index						cmd_type		data_present_sel	cmd_idx_chk_enable	cmd_crc_chk_enable	sub_cmd_flag	resp_type_select	
-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:14]	-	保留。	0x0
cmd_index	[13:8]	RW	命令号。	0x0
cmd_type	[7:6]	RW	命令类型。 0：Normal 命令； 1：Suspend 命令； 2：Resume 命令； 3：Abort 命令。	0x0
data_present_sel	[5]	RW	当前是否为数据传输 0：无数据传输； 1：有数据传输。	0x0
cmd_idx_chk_enable	[4]	RW	命令编号检查使能 0：关闭； 1：使能。	0x0
cmd_crc_chk_enable	[3]	RW	命令 CRC 检查使能 0：关闭； 1：使能。	0x0
sub_cmd_flag	[2]	RW	子命令标识 0：主命令；	0x0

Name	Bits	R/W	Description	Default
			1：子命令。	
resp_type_select	[1:0]	RW	响应类型 0：无响应； 1：长响应(136 位)； 2：短响应（48 位）； 3：短响应（48 位），在接收到响应后检查 busy 状态。	0x0

28.6.7 RESP01_R

- 偏移地址：0x10　　复位值：0x00000000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
resp01															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
resp01															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
resp01	[31:0]	RW	RESP01 寄存器。 此寄存器存放响应信息的[39:8]字段内容。	0x0

28.6.8 RESP23_R

- 偏移地址：0x14　　复位值：0x00000000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
resp23															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
resp23															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
resp23	[31:0]	RW	RESP23 寄存器。 此寄存器存放响应信息的[71:40]字段内容。	0x0

28.6.9 RESP45_R

- 偏移地址：0x18　　复位值：0x00000000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
resp45															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
resp45															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
resp45	[31:0]	RW	RESP45 寄存器。 此寄存器存放响应信息的[103:72]字段内容。	0x0

28.6.10 RESP67_R

- 偏移地址：0x1C　　复位值：0x00000000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
resp67															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
resp67															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
resp67	[31:0]	RW	RESP67 寄存器。 此寄存器存放响应信息的[135:104]字段内容。	0x0

28.6.11 BUF_DATA_R

- 偏移地址：0x20　　复位值：0x00000000

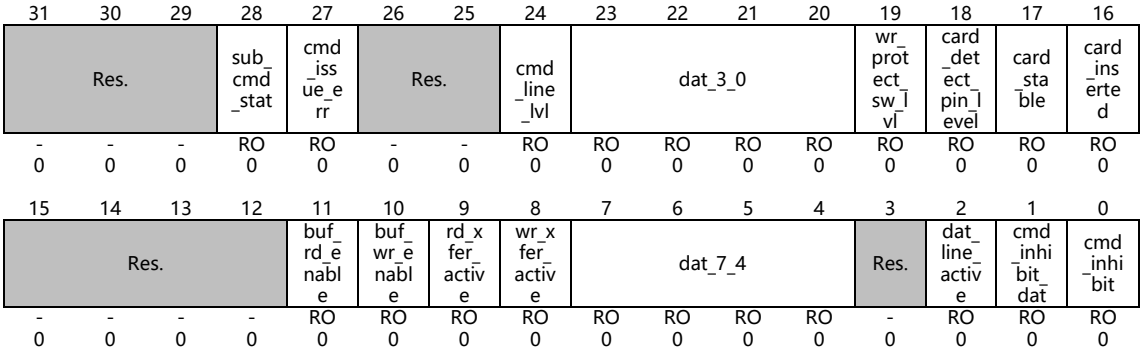
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
buf_data															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
buf_data															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
buf_data	[31:0]	RW	用于存放接收或者发送的数据。 在 Non_DMA 方式下通过写这个寄存器把数据写到卡中；读操作时则通过读这个寄存器来获取卡中的数据。	0x0

28.6.12 PSTATE_REG

- 偏移地址：0x24　　复位值：0x00000000



Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
sub_cmd_stat	[28]	RO	子命令标志 0：主命令； 1：子命令。	0x0
cmd_issue_err	[27]	RO	命令发送出错状态 0：发送命令无错； 1：命令不能被发送出去。	0x0
reserved	[26:25]	-	保留。	0x0
cmd_line_lvl	[24]	RO	Cmd 管脚状态 0：cmd 管脚为低电平； 1：cmd 管脚为高电平。	0x0
dat_3_0	[23:20]	RO	Data[3:0]管脚状态，每个 bit 含义 0：Data 管脚为低电平； 1：Data 管脚为高电平。	0x0
wr_protect_sw_lvl	[19]	RO	sd 卡检测信号 0：sd 卡写保护，不可写入数据； 1：sd 卡未开启可保护，可写数据。	0x0
card_detect_pin_level	[18]	RO	卡检测管脚状态 0：卡检测管脚为低电平，表示 sd/tf 卡插入； 1：卡检测管脚为高电平，表示没有 sd/tf 卡插入。	0x0
card_stable	[17]	RO	sd 卡插入稳定信号 0：表示 sd/tf 卡插入稳定； 1：表示没有 sd/tf 卡插入。	0x0
card_inserted	[16]	RO	sd 卡是否插入指示信号 0：表示 sd/tf 卡插入； 1：表示没有 sd/tf 卡插入。	0x0

Name	Bits	R/W	Description	Default
reserved	[15:12]	-	保留。	0x0
buf_rd_enable	[11]	RO	读使能 0：不使能； 1：使能。	0x0
buf_wr_enable	[10]	RO	写使能 0：不使能； 1：使能。	0x0
rd_xfer_active	[9]	RO	读数据有效状态 0：空闲状态； 1：正在读数据。	0x0
wr_xfer_active	[8]	RO	写数据有效状态 0：空闲状态； 1：正在写数据。	0x0
dat_7_4	[7:4]	RO	Data[7:4]管脚状态，每个 bit 含义 0：Data 管脚为低电平； 1：Data 管脚为高电平。	0x0
reserved	[3]	-	保留。	0x0
dat_line_active	[2]	RO	数据线有效状态 0：空闲状态； 1：正在数据传输。	0x0
cmd_inhibit_dat	[1]	RO	带数据的命令有效状态 0：空闲状态，可以发送带数据的命令； 1：数据线正在传输或者读操作有效中。	0x0
cmd_inhibit	[0]	RO	命令线有效状态 0：空闲状态，控制器可以发送命令； 1：忙状态，控制器不可以发送命令	0x0

28.6.13 HOST_CTRL1_R

- 偏移地址：0x28　　复位值：0x00

7	6	5	4	3	2	1	0
card_detect_sig_sel	card_detect_test_lvl	ext_dat_xfer	dma_sel		Res.	dat_xfer_width	Res.
RW	RW	-	RW	RW	-	RW	-
0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
card_detect_sig_sel	[7]	RW	卡检测信号选择 0：卡检测信号（card_detect_n）选择（常规使用）； 1：卡检测测试电平选择。	0x0
card_detect_test_lvl	[6]	RW	卡检测测试电平	0x0

Name	Bits	R/W	Description	Default
			0：无卡； 1：有卡插入。	
ext_dat_xfer	[5]	RW	扩展数据传输位宽选择 0：数据传输位宽由 dat_xfer_width 位控制； 1：数据传输位宽选择 8 线模式。 注意：8 线模式仅在对接 EMMC 时启用，SD 卡和 TF 卡禁止配启用。	0x0
dma_sel	[4:3]	RW	DMA 选择 当 HOST_CTRL2_R 寄存器中 host_ver4_enable 为 1 时： 0：SDMA； 1：保留； 2：ADMA2； 3：ADMA2 或者 ADMA3。 当 HOST_CTRL2_R 寄存器中 host_ver4_enable 为 0 时： 0：SDMA； 1：保留； 2：32bit 地址 ADMA2； 3：64bit 地址 ADMA2。	0x0
reserved	[2]	-	保留。	0x0
dat_xfer_width	[1]	RW	数据传输位宽选择 0：1 线模式； 1：4 线模式。	0x0
reserved	[0]	-	保留。	0x0

28.6.14 PWR_CTRL_R

- 偏移地址：0x29　　复位值：0x00



Name	Bits	R/W	Description	Default
reserved	[7:1]	-	保留。	0x0
sd_bus_pwr_vdd1	[0]	RW	sdio device 供电开关。 0：关闭 sdio device 供电； 1：打开 sdio devcie 供电。 注意：该位必须配置为 1，SDIO 才能正常工作。同时该信号可用于控制电源开关信号(SDIO_VDD1_ON)	0x0

Name	Bits	R/W	Description	Default
			的输出，其输出极性由 CLK_CTRL_R.cfg_vdd_on_polarity 配置。	

28.6.15 BGAP_CTRL_R

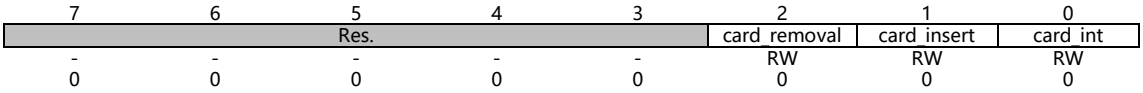
- 偏移地址：0x2a　　复位值：0x00

7	6	5	4	3	2	1	0
Res.				int_at_bgap	rd_wait_ctrl	continue_req	stop_bg_req
-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[7:4]	-	保留。	0x0
int_at_bgap	[3]	RW	块间隙（Block Gap）暂停数据传输功能启用控制。 该位仅在 SDIO 卡的 4 位模式下有效，设置为 1 允许在多块传输的块间隙暂停数据传输，方便用户执行其他操作，后续可通过 continue_req 重启数据传输。 0：不允许块间隙暂停数据传输； 1：允许块间隙暂停数据传输。	0x0
rd_wait_ctrl	[2]	RW	读取等待控制。 当卡支持读取等待功能，此位用于启用读取等待协议来停止读取数据；否则，主机必须通过停止卡时钟来停止读取数据。 0：不使能读取等待； 1：使能读取等待；	0x0
continue_req	[1]	RW	继续传输请求使能控制。 在块间隙（Block Gap）暂停断传输后,使能该 bit 位将发起继续传输请求，用于重启传输，重启传输后该 bit 位自动清零。如果块间隙暂停传输请求（stop_bg_req）为 1，写这个 bit 位无效。 0：不发起继续传输请求； 1：发起继续传输请求，重启传输。	0x0
stop_bg_req	[0]	RW	块间隙（Block Gap）暂停数据传输请求控制。 用于发起暂停数据传输请求，SDIO 控制器将在下一个块间隙暂停数据传输。 0：不发起块间隙中断传输请求； 1：发起块间隙暂停数据传输请求。	0x0

28.6.16 WUP_CTRL_R

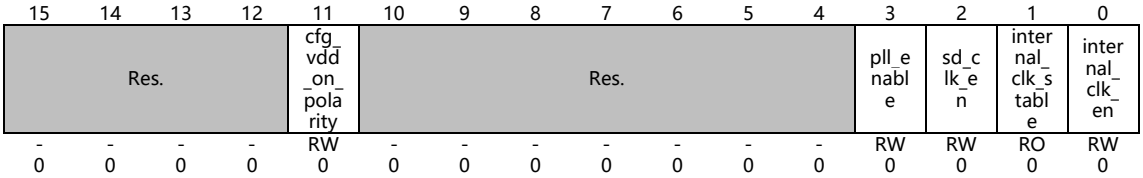
- 偏移地址：0x2b　　复位值：0x00



Name	Bits	R/W	Description	Default
reserved	[7:3]	-	保留。	0x0
card_removal	[2]	RW	卡拔出唤醒事件中断使能： 0：不使能； 1：使能。	0x0
card_insert	[1]	RW	卡插入唤醒事件中断使能： 0：不使能； 1：使能。	0x0
card_int	[0]	RW	sdio 卡中断唤醒事件中断使能： 0：不使能； 1：使能。	0x0

28.6.17 CLK_CTRL_R

- 偏移地址：0x2C　　复位值：0x00

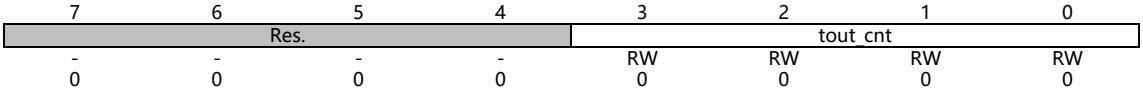


Name	Bits	R/W	Description	Default
reserved	[15:12]	-	保留。	0x0
cfg_vdd_on_polarity	[11]		该位用于配置 sdio device 供电控制信号（SDIO_VDD1_ON）的输出极性。 0：供电开关打开(sd_bus_pwr_vdd1 = 1)时，供电控制信号输出为 0；供电开关关闭(sd_bus_pwr_vdd1 = 0)时，供电控制信号输出为 1。 1：供电开关打开(sd_bus_pwr_vdd1 = 1)时，供电控制信号输出为 1；供电开关关闭(sd_bus_pwr_vdd1 = 0)时，供电控制信号输出为 0。	0x0
reserved	[10:4]		保留。	
pll_enable	[3]	RW	PLL 使能开关 0：PLL 关闭； 1：PLL 使能。	0x0

Name	Bits	R/W	Description	Default
sd_clk_en	[2]	RW	SD 时钟使能 0：关闭； 1：打开。	0x0
internal_clk_stable	[1]	RO	内部时钟稳定状态 0：时钟未稳定； 1：时钟稳定。	0x0
internal_clk_en	[0]	RW	内部时钟使能 0：关闭； 1：使能。	0x0

28.6.18 TOUT_CTRL_R

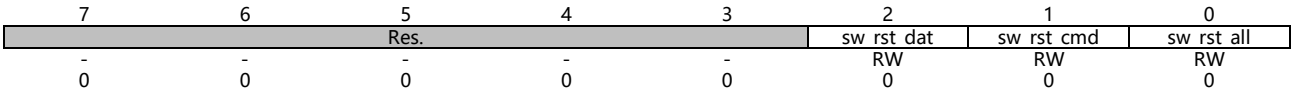
- 偏移地址：0x2E　　复位值：0x00



Name	Bits	R/W	Description	Default
reserved	[7:4]	-	保留。	0x0
tout_cnt	[3:0]	RW	数据超时计数值，(TMCLK 为 timer 时钟周期) 0：TMCLKx2^13； 1 : TMCLK x 2^14； ..... 14：TMCLKx2^27； 其他：保留	0x0

28.6.19 SW_RST_R

- 偏移地址：0x2F　　复位值：0x00



Name	Bits	R/W	Description	Default
reserved	[7:3]	-	保留。	0x0
sw_rst_dat	[2]	RW	软复位数据线请求 0：不复位； 1：复位。	0x0
sw_rst_cmd	[1]	RW	软复位命令线请求 0：不复位；	0x0

Name	Bits	R/W	Description	Default
			1：复位。	
sw_rst_all	[0]	RW	软复位控制器请求(同时复位数据线和命令线) 0：不复位； 1：复位。	0x0

28.6.20 NORMAL_INT_STAT_R

- 偏移地址：0x30　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
err_interrupt	Res.						card_interrupt	card_removal	card_insertion	buf_rd_ready	buf_wr_ready	dma_interrupt	bgap_event	xfer_complete	cmd_complete
RO	-	-	-	-	-	-	RO	W1C	W1C	W1C	W1C	W1C	W1C	W1C	W1C
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
err_interrupt	[15]	RO	汇总错误中断状态 0：无中断； 1：有中断。	0x0
reserved	[14:9]	-	保留。	0x0
card_interrupt	[8]	RO	SDIO 卡中断状态 0：无中断； 1：有中断。	0x0
card_removal	[7]	W1C	卡移除中断状态 0：无中断； 1：有中断。	0x0
card_insertion	[6]	W1C	卡插入中断状态 0：无中断； 1：有中断。	0x0
buf_rd_ready	[5]	W1C	缓冲区读取就绪中断状态 0：无中断； 1：有中断。	0x0
buf_wr_ready	[4]	W1C	缓冲区写入就绪中断状态 0：无中断； 1：有中断。	0x0
dma_interrupt	[3]	W1C	DMA 中断状态 0：无中断； 1：有中断。	0x0
bgap_event	[2]	W1C	块间隙（Block Gap）事件中断状态 0：无中断； 1：有中断。	0x0

Name	Bits	R/W	Description	Default
			注意：当发起块间隙暂停数据传输请求（stop_bg_req），数据传输因此暂停，该中断状态置 1。	
xfer_complete	[1]	W1C	读/写传输完成中断状态 0：无中断； 1：有中断。	0x0
cmd_complete	[0]	W1C	命令完成中断状态 0：无中断； 1：有中断。	0x0

28.6.21 ERROR_INT_STAT_R

- 偏移地址：0x32　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				resp_err	Res.	adma_err	auto_cmd_err	Res.	data_end_bit_err	data_crc_err	data_tout_err	cmd_idx_err	cmd_end_bit_err	cmd_crc_err	cmd_tout_err
-	-	-	-	W1C	-	W1C	W1C	-	W1C	W1C	W1C	W1C	W1C	W1C	W1C
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:12]	-	保留。	0x0
resp_err	[11]	W1C	响应错误中断，仅用于 SD 模式 0：无中断； 1：有中断。	0x0
reserved	[10]	-	保留。	0x0
adma_err	[9]	W1C	ADMA 错误中断，系统总线错误响应，或者 ADMA2 描述链表无效。 0：无中断； 1：有中断。	0x0
auto_cmd_err	[8]	W1C	Auto CMD 错误中断状态 0：无中断； 1：有中断。	0x0
reserved	[7]	-	保留。	0x0
data_end_bit_err	[6]	W1C	数据结束位错误中断状态 0：无中断； 1：有中断。	0x0
data_crc_err	[5]	W1C	数据 CRC 错误中断状态 0：无中断； 1：有中断。	0x0
data_tout_err	[4]	W1C	数据超时错误中断状态	0x0

Name	Bits	R/W	Description	Default
			0：无中断； 1：有中断。	
cmd_idx_err	[3]	W1C	响应的命令号错误中断状态 0：无中断； 1：有中断。	0x0
cmd_end_bit_err	[2]	W1C	响应的命令结束位错误中断状态 0：无中断； 1：有中断。	0x0
cmd_crc_err	[1]	W1C	命令 CRC 错误中断状态 0：无中断； 1：有中断。	0x0
cmd_tout_err	[0]	W1C	命令超时错误中断状态 0：无中断； 1：有中断。	0x0

28.6.22 NORMAL_INT_STAT_EN_R

- 偏移地址：0x34　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							card_interrupt_stat_en	card_removal_stat_en	card_insertion_stat_en	buf_rd_ready_stat_en	buf_wr_ready_stat_en	dma_interrupt_stat_en	bgap_event_stat_en	xfer_complete_stat_en	cmd_complete_stat_en
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:9]	-	保留。	0x0
card_interrupt_stat_en	[8]	RW	SDIO 卡中断使能 0：不使能； 1：使能。	0x0
card_removal_stat_en	[7]	RW	卡移除中断使能 0：不使能； 1：使能。	0x0
card_insertion_stat_en	[6]	RW	卡插入中断使能 0：不使能； 1：使能。	0x0
buf_rd_ready_stat_en	[5]	RW	缓冲区读取就绪中断使能 0：不使能； 1：使能。	0x0
buf_wr_ready_stat_en	[4]	RW	缓冲区写入就绪中断使能 0：不使能；	0x0

Name	Bits	R/W	Description	Default
			1：使能。	
dma_interrupt_stat_en	[3]	RW	DMA 中断使能 0：不使能； 1：使能。	0x0
bgap_event_stat_en	[2]	RW	块间隙（Block Gap）事件中断使能 0：不使能； 1：使能。	0x0
xfer_complete_stat_en	[1]	RW	读/写传输完成中断使能 0：不使能； 1：使能。	0x0
cmd_complete_stat_en	[0]	RW	命令完成中断使能 0：不使能； 1：使能。	0x0

28.6.23 ERROR_INT_STAT_EN_R

- 偏移地址：0x36　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				resp_err_stat_en	Res.	adma_err_stat_en	auto_cmd_err_stat_en	Res.	data_end_bit_err_stat_en	data_crc_err_stat_en	data_tout_err_stat_en	cmd_idx_err_stat_en	cmd_end_bit_err_stat_en	cmd_crc_err_stat_en	cmd_tout_err_stat_en
-	-	-	-	RW	-	RW	RW	-	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:12]	-	保留。	0x0
resp_err_stat_en	[11]	RW	响应错误中断使能，仅用于 SD 模式 0：不使能； 1：使能。	0x0
reserved	[10]	-	保留。	0x0
adma_err_stat_en	[9]	RW	ADMA 错误中断使能，系统总线错误响应，或者 ADMA2 描述链表无效 0：不使能； 1：使能。	0x0
auto_cmd_err_stat_en	[8]	RW	Auto CMD 错误中断使能 0：不使能； 1：使能。	0x0
reserved	[7]	-	保留。	0x0
data_end_bit_err_stat_en	[6]	RW	数据结束位错误中断使能 0：不使能； 1：使能。	0x0

Name	Bits	R/W	Description	Default
data_crc_err_stat_en	[5]	RW	数据 CRC 错误中断使能 0：不使能； 1：使能。	0x0
data_tout_err_stat_en	[4]	RW	数据超时错误中断使能 0：不使能； 1：使能。	0x0
cmd_idx_err_stat_en	[3]	RW	响应的命令号错误中断使能 0：不使能； 1：使能。	0x0
cmd_end_bit_err_stat_en	[2]	RW	响应的命令结束位错误中断使能 0：不使能； 1：使能。	0x0
cmd_crc_err_stat_en	[1]	RW	命令 CRC 错误中断使能 0：不使能； 1：使能。	0x0
cmd_tout_err_stat_en	[0]	RW	命令超时错误中断使能 0：不使能； 1：使能。	0x0

28.6.24 NORMAL_INT_SIGNAL_EN_R

- 偏移地址：0x38　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							card_interrupt_signal_en	card_removal_signal_en	card_insertion_signal_en	buf_rd_ready_signal_en	buf_wr_ready_signal_en	dma_interrupt_signal_en	bgap_event_signal_en	xfer_complete_signal_en	cmd_complete_signal_en
-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:9]	-	保留。	0x0
card_interrupt_signal_en	[8]	RW	SDIO 卡中断信号使能 0：不使能； 1：使能。	0x0
card_removal_signal_en	[7]	RW	卡移除中断信号使能 0：不使能； 1：使能。	0x0
card_insertion_signal_en	[6]	RW	卡插入中断信号使能 0：不使能； 1：使能。	0x0

Name	Bits	R/W	Description	Default
buf_rd_ready_signal_en	[5]	RW	缓冲区读取就绪中断信号使能 0：不使能； 1：使能。	0x0
buf_wr_ready_signal_en	[4]	RW	缓冲区写入取就绪中断信号使能 0：不使能； 1：使能。	0x0
dma_interrupt_signal_en	[3]	RW	DMA 中断信号使能 0：不使能； 1：使能。	0x0
bgap_event_signal_en	[2]	RW	块间隙（Block Gap）事件中断信号使能 0：不使能； 1：使能。	0x0
xfer_complete_signal_en	[1]	RW	读/写传输完成中断信号使能 0：不使能； 1：使能。	0x0
cmd_complete_signal_en	[0]	RW	命令完成中断信号使能 0：不使能； 1：使能。	0x0

28.6.25 ERROR_INT_SIGNAL_EN_R

- 偏移地址：0x3A　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				resp_err_signal_en	Res.	adma_err_signal_en	auto_cmd_err_signal_en	Res.	data_end_bit_err_signal_en	data_crc_err_signal_en	data_tout_err_signal_en	cmd_idx_err_signal_en	cmd_end_bit_err_signal_en	cmd_crc_err_signal_en	cmd_tout_err_signal_en
-	-	-	-	RW	-	RW	RW	-	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:12]	-	保留。	0x0
resp_err_signal_en	[11]	RW	响应错误中断信号使能，仅用于 SD 模式 0：不使能； 1：使能。	0x0
reserved	[10]	-	保留。	0x0
adma_err_signal_en	[9]	RW	ADMA 错误中断信号使能，系统总线错误响应，或者 ADMA2 描述链表无效 0：不使能； 1：使能。	0x0
auto_cmd_err_signal_en	[8]	RW	Auto CMD 错误中断信号使能	0x0

Name	Bits	R/W	Description	Default
			0：不使能； 1：使能。	
reserved	[7]	-	保留。	0x0
data_end_bit_err_signal_en	[6]	RW	数据结束位错误中断信号使能 0：不使能； 1：使能。	0x0
data_crc_err_signal_en	[5]	RW	数据 CRC 错误中断信号使能 0：不使能； 1：使能。	0x0
data_tout_err_signal_en	[4]	RW	数据超时错误中断信号使能 0：不使能； 1：使能。	0x0
cmd_idx_err_signal_en	[3]	RW	响应的命令号错误中断信号使能 0：不使能； 1：使能。	0x0
cmd_end_bit_err_signal_en	[2]	RW	响应的命令结束位错误中断信号使能 0：不使能； 1：使能。	0x0
cmd_crc_err_signal_en	[1]	RW	命令 CRC 错误中断信号使能 0：不使能； 1：使能。	0x0
cmd_tout_err_signal_en	[0]	RW	命令超时错误中断信号使能 0：不使能； 1：使能。	0x0

28.6.26 AUTO_CMD_STAT_R

- 偏移地址：0x3C　　复位值：0x0000

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								cmd_not_issued_auto_cmd12	Res.	auto_cmd_resp_err	auto_cmd_idx_err	auto_cmd_ebit_err	auto_cmd_crc_err	auto_cmd_tout_err	auto_cmd12_not_exec
-	-	-	-	-	-	-	-	RO	-	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15:8]	-	保留。	0x0
cmd_not_issued_auto_cmd12	[7]	RO	有 Auto CMD12 错误时发送后续命令状态 0：无错误；	0x0

Name	Bits	R/W	Description	Default
			1：还没有通过数据线发送 CMD。	
reserved	[6]	-	保留。	0x0
auto_cmd_resp_err	[5]	RO	Auto CMD 响应状态 0：无错； 1：错。	0x0
auto_cmd_idx_err	[4]	RO	Auto CMD 命令号状态 0：无错； 1：错。	0x0
auto_cmd_ebit_err	[3]	RO	Auto CMD 结束位状态 0：无错； 1：错。	0x0
auto_cmd_crc_err	[2]	RO	Auto CMD CRC 状态 0：无错； 1：错。	0x0
auto_cmd_tout_err	[1]	RO	Auto CMD 超时状态 0：无错； 1：错。	0x0
auto_cmd12_not_exec	[0]	RO	Auto CMD12 执行状态 0：执行； 1：没有执行。	0x0

28.6.27 HOST_CTRL2_R

- 偏移地址：0x3E　　复位值：0x0000

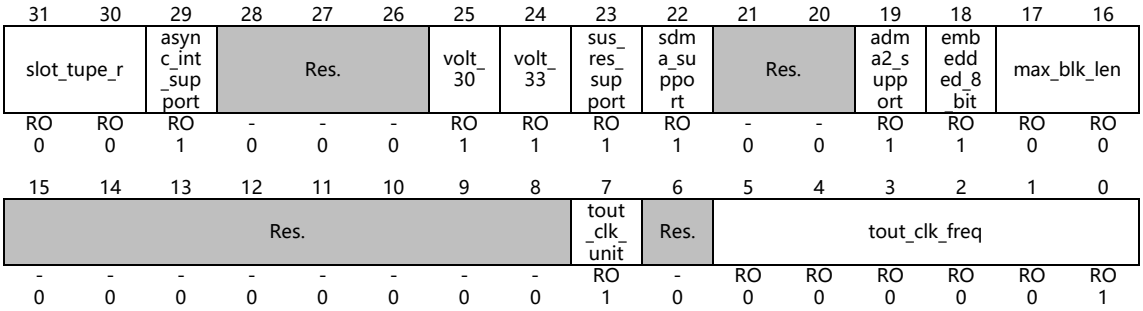
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	async_int_enable	Res.	host_ver4_enable	cmd23_enable	adma2_len_mode	Res.									
-	RW	-	RW	RW	RW	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[15]	-	保留。	0x0
async_int_enable	[14]	RW	异步中断使能 0：关闭； 1：打开。	0x0
reserved	[13]	-	保留。	0x0
host_ver4_enable	[12]	RW	控制器版本 4 使能 0：3.0 版本； 1：4.0 版本。	0x0
cmd23_enable	[11]	RW	CMD23 使能	0x0

Name	Bits	R/W	Description	Default
			0：Auto CMD23 关闭; 1：Auto CMD23 使能	
adma2_len_mode	[10]	RW	ADMA2 长度 0：16bit 数据长度; 1：26bit 数据长度。	0x0
reserved	[9:0]	-	保留。	0x0

28.6.28 CAPABILITIES1_R

- 偏移地址：0x40　　复位值：0x23CC0081

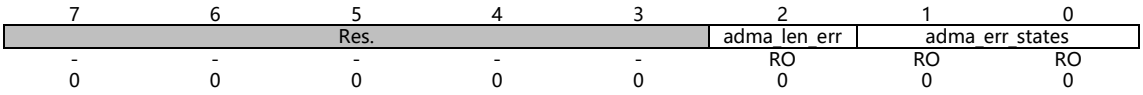


Name	Bits	R/W	Description	Default
slot_tupe_r	[31:30]	RO	Slot 类型。 0：可移除卡 Slot; 1：嵌入式 Slot; 2：共享总线 Slot(SD 模式下使用); 3：UHS2(暂时不支持)。	0x0
async_int_support	[29]	RO	同步中断是否支持。 0：不支持; 1：支持。	0x1
reserved	[28:26]	-	保留。	0x0
volt_30	[25]	RO	电压支持 3.0V。 0：不支持; 1：支持。	0x1
volt_33	[24]	RO	电压支持 3.3V。 0：不支持; 1：支持。	0x1
sus_res_support	[23]	RO	暂停、恢复支持。 0：不支持; 1：支持。	0x1
sdma_support	[22]	RO	SDMA 支持 0：不支持;	0x1

Name	Bits	R/W	Description	Default
			1：支持。	
reserved	[21:20]	-	保留。	0x0
adma2_support	[19]	RO	是否支持 ADMA2 0：不支持； 1：支持。	0x1
embedded_8_bit	[18]	RO	是否支持嵌入式设备 8 线模式 0：不支持； 1：支持。	0x1
max_blk_len	[17:16]	RO	最大数据块长度 0：512Byte； 1：1024Byte，无效； 2：2048Byte，无效； 3：保留。	0x0
reserved	[15:8]	-	保留。	0x0
tout_clk_unit	[7]	RO	超时时钟单位选择 0：KHz； 1：MHz。	0x1
reserved	[6]	-	保留。	0x0
tout_clk_freq	[5:0]	RO	超时时钟频率选择 1：1KHz/1MHz； 2：2KHz/2MHz； …… 63：63KHz/63MHz。	0x1

28.6.29 ADMA_ERR_STAT_R

- 偏移地址：0x54　　复位值：0x00

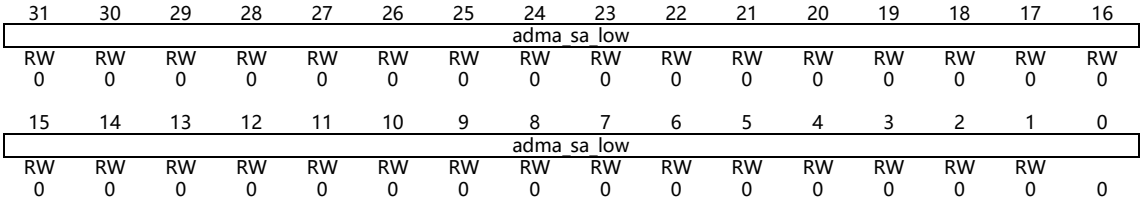


Name	Bits	R/W	Description	Default
reserved	[7:3]	-	保留。	0x0
adma_len_err	[2]	RO	ADMA 长度不匹配错误状态 0：无错； 1：有错。	0x0
adma_err_states	[1:0]	RO	ADMA 错误状态 0：ST_STOP，停止 DMA 传输的 ADMA 描述链表错误； 1：ST_FDS，获取当前 ADMA 描述链表错误；	0x0

Name	Bits	R/W	Description	Default
			2：ST_UNUSED，保留； 3：ST_TFR，传输数据的 ADMA 描述链表错误。	

28.6.30 ADMA_SA_LOW_R

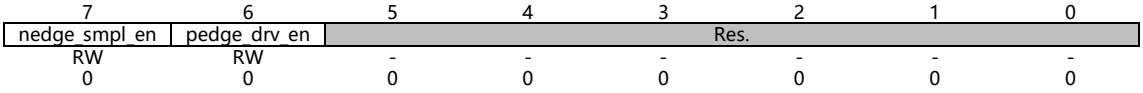
- 偏移地址：0x58　　复位值：0x00000000



Name	Bits	R/W	Description	Default
adma_sa_low	[31:0]	RW	SDMA：当 HOST_CTRL2_R 寄存器中 host_ver4_enable 为 1 时，该寄存器为数据在系统内存中存放的起始地址。 ADMA2：ADMA 系统地址低 32bit。	0x0

28.6.31 MSHC_CTRL_R

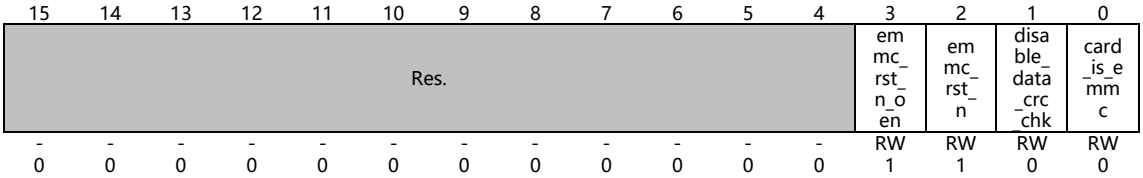
- 偏移地址：0x508　　复位值：0x00



Name	Bits	R/W	Description	Default
nedge_smpl_en	[7]	RW	rx 方向，下降沿采样使能 0：上升沿采样； 1：下降沿采样。	0x0
pedge_drv_en	[6]	RW	tx 方向，上升沿发数使能 0：下降沿发数； 1：上升沿发数。	0x0
reserved	[5:0]	-	保留。	0x0

28.6.32 EMMC_CTRL_R

- 偏移地址：0x52C　　复位值：0x000C



Name	Bits	R/W	Description	Default
reserved	[15:4]	-	保留。	0x0
emmc_rst_n_oen	[3]	RW	输出 EMMC 的硬件复位的 OEN 信号 0：输出 EMMC 的硬件复位； 1：不输出 EMMC 的硬件复位。	0x1
emmc_rst_n	[2]	RW	EMMC 的硬件复位 0：EMMC 设备复位； 1：EMMC 设备不复位。	0x1
disable_data_crc_chk	[1]	RW	关闭数据 CRC 检查 0：数据 CRC 检查使能； 1：数据 CRC 检查关闭。	0x0
card_is_emmc	[0]	RW	连接的卡类型 0：非 eMMC 卡； 1：eMMC 卡。	0x0

29 备份寄存器（BACKUP）

29.1 BACKUP 简介

BACKUP 的电源在 DVDD 上电时使用 DVDD 供电，当电源 DVDD 关闭后由 VBAT 维持供电。BACKUP 中支持 RTC，1 个定时唤醒器 WKT，16 个 32 位的备份数据寄存器用来存储并保护用户应用数据，以及支持入侵检测功能，时间戳功能。这些功能不受系统待机模式唤醒或系统复位的影响。

当系统发生复位时，为保护备份域数据，防止意外操作，所有的写操作都被禁止。

如需使能 BACKUP，首先设置 CRG 的寄存器，配置 PERI_CRG70.backup_cfg_srst_req 为 0，配置 PERI_CRG70.backup_cfg_cken 为 1；然后配置 SYSCTRL 的寄存器 LOCKEN.backup_cfg_lock 为 0；再开始 BACKUP 的其他配置。

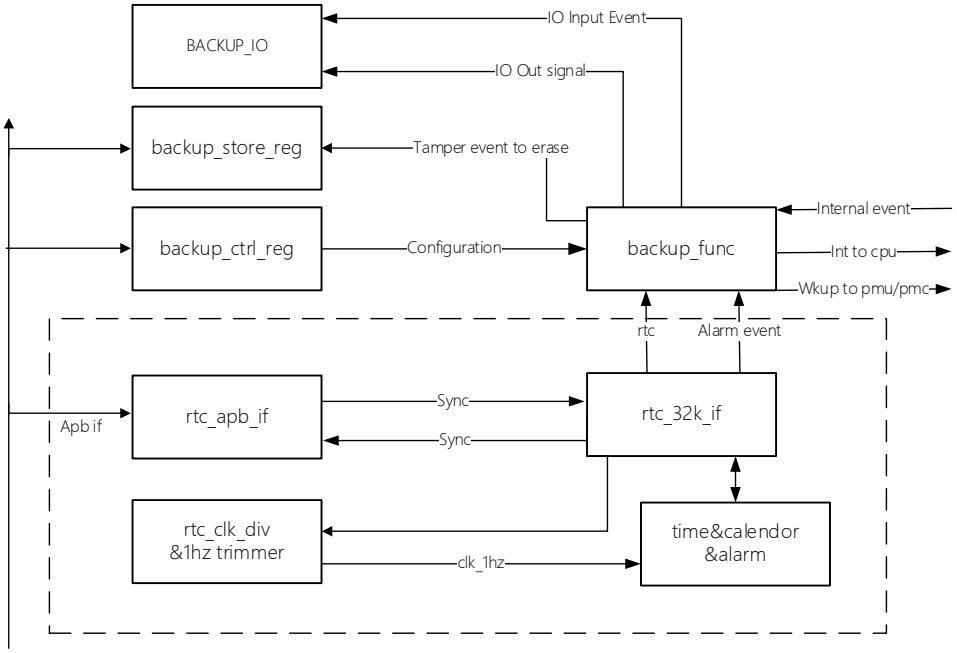
29.2 BACKUP 主要特性

- BACKUP 在所有低功耗模式都可工作；
- 支持内外部事件入侵检测；
- 支持一个重加载定时唤醒器；
- 支持备份数据寄存器；
- 支持时间戳；
- 支持管脚等内外部事件处理，以及触发中断和唤醒；
- 支持 RTC，详见 30 实时时钟章节；

29.3 BACKUP 功能说明

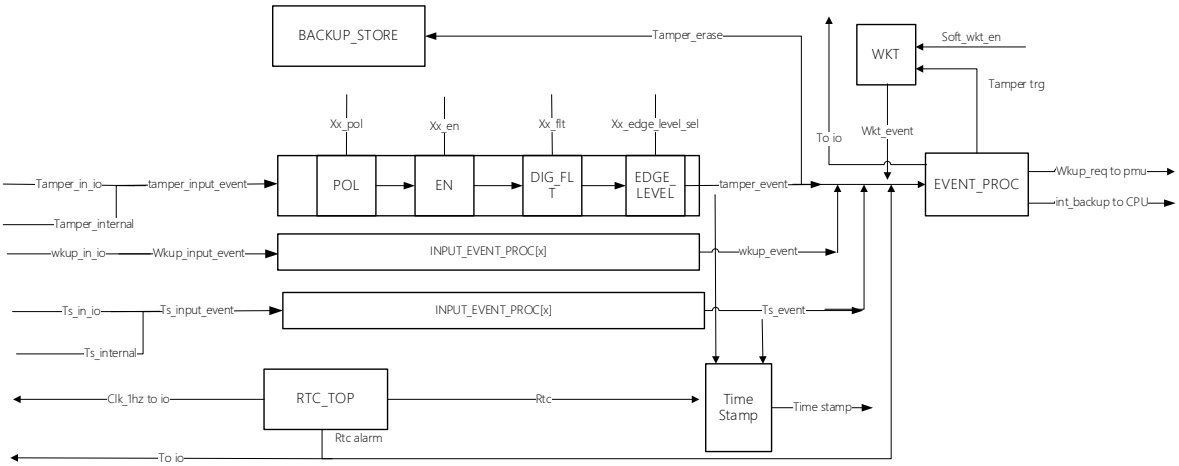
29.3.1 框图

图 29-1 BACKUP 电源域整体框图



Backup_func 为 BACKUP 事件处理模块，包括 TAMPER 事件处理，WKUP 事件处理，TS 事件处理，WKT 事件，闹钟事件处理（RTC）。详细框图如下图 29-2 所示：

图 29-2 Backup 事件处理模块框图



29.3.2 BACKUP 时钟结构

配置时钟

BACKUP 的配置时钟不受系统复位影响，时钟源来自 clk_hs。在 deepsleep/shutdown 时，BACKUP 无配置时钟。

工作时钟

工作时钟为 BACKUP_CLK，与配置时钟异步，有以下时钟源可选:

- 0：LSI
- 1：LSE
- 2：HSE_div_512（HSE 的 512 分频）
- default：LSI

HSE_div 使用 cldo 的 por 作为复位，在 cldo 上电后，如果发生系统复位，HSE 可以保持打开，并且保持正常分频。

在 deepsleep/shutdown 时，需要软件将 BACKUP 的时钟切到 LSI 或者 LSE，不能使用 HSE 时钟。

在 DVDD 掉电的时候，只能使用 LSE。

BACKUP_FUNC 时钟:

BACKUP 事件处理模块的时钟为 clk_backup_func，时钟源来自 BACKUP_CLK，可单独门控；TAMPER 事件/WKT 事件/WKUP 事件/TS 事件/闹钟事件处理，使用该时钟。

RTC 时钟:

RTC 时钟为 rtc_clk，时钟源来自 BACKUP_CLK，可单独门控。日历、闹钟使用该时钟。

自动唤醒器 WKT 时钟

WKT 时钟源选择:

- 0： BACKUP_1Hz
- 1： BACKUP_CLK
- 2/3：BACKUP_CLK 的 4 分频
- 4/5：BACKUP_CLK 的 8 分频
- 6/7：BACKUP_CLK 的 16 分频

WKT 时钟 wkt_clk，可单独门控。定时唤醒计数器使用该时钟。

29.3.3 BACKUP 复位结构

上电复位

BACKUP 的全局硬复位为 BACKUP 电源域上电复位 BPOR（BACKUP 电源的单独 POR），经过滤波，得到 rst_bpor_n，可以复位 BACKUP 全部逻辑。

全局软复位

BACKUP 的全局软件复位信号来自 BACKUP_CRG0 寄存器的 backup_srst_req 位，该信号可以复位除 backup_srst_req 本身之外的其他全部逻辑，backup_srst_req 的逻辑只受上电复位（rst_bpor_n）控制。

BACKUP 的全局硬复位和全局软复位，共同作用产生 BACKUP 域的全局复位 rst_backup_n。

功能单独软复位

Backup_func 不支持单独复位，直接使用 BACKUP 域的全局复位 rst_backup_n。

rtc 支持软复位，rtc_srst_req 和 rst_backup_n，共同作用得到 rtc 复位 rst_rtc_n。

wkt 支持软复位，wkt_srst_req 和 rst_backup_n，共同作用得到 wkt 复位 rst_wkt_n。

backup_store 备份数据寄存器，除了 backup 的全局复位外，内外部 TAMPER 入侵检测产生的事件也起到类似复位功能，共同作用得到 rst_backup_store_n。

29.3.4 寄存器写保护

配置 SYSCTRL 的 LOCKEN.backup_cfg_lock 来开关对 BACKUP 的寄存器写保护。

29.3.5 入侵检测

入侵检测功能是指 BACKUP 对 TAMPER 事件进行检测，TAMPER 事件可分为外部 TAMPER 管脚输入事件或者内部 TAMPER 检测事件。BACKUP 支持 3 个外部 TAMPER 管脚输入事件（有 3 个 IO 可配置复用为 TAMPER 管脚输入功能）；支持 1 个内部 TAMPER 检测事件（Flash RDP 保护等级降级）。

每个 TAMPER 事件可单独屏蔽，支持数字滤波功能（最大 3 个 BACKUP_CLK，可单独配置滤波级数），可单独配置为边沿或者电平检测。每个外部 TAMPER 管脚输入事件可单独配置极性，而内部 TAMPER 检测事件极性固定使用正极性，不可随意修改。

TAMPER 事件发生时，BACKUP 会单独锁存记录标志，软件可清除；该事件发生时支持上报中断，软件可使能中断是否上报，软件可清除原始中断。

TAMPER 事件支持触发唤醒，软件可使能是否触发唤醒；TAMPER 事件也可以触发 WKT 事件，每个 TAMPER 事件是否触发 WKT 可独立使能；TAMPER 事件也可以触发时间戳，每个入侵事件是否触发时间戳可独立使能。

29.3.6 定时唤醒器

BACKUP 支持 WKT 事件唤醒功能。WKT 是一个自动重加载的 17bit 定时器，由软件配置 WKT 事件使能，当定时器从软件设置的阈值递减计数到 0 时将产生一个 WKT 事件，如果使能位不关闭，将自动重载计数阈值，可周期性产出 WKT 事件；另外 TAMPER 事件也可以触发 WKT 事件使能，每个 TAMPER 事件可独立配置是否触发 WKT 事件使能（注意：在使用 TAMPER 事件触发 WKT 功能时，软件不允许在 WKT 事件未被触发时就清除对应的 BACKUP_RECORD_FLAG.tamper_internal_record_flag 或者 tamper_in_io_record_flag 位域，否则会导致 WKT 无法被 TAMPER 事件成功触发）。

产生 WKT 事件时，可以配置 IO 复用将该事件输出到管脚，且输出极性可控；该事件发生时将锁存记录标志，软件可清除；该事件发生时支持上报中断，软件可使能中断是否上报，软件可清除原始中断；该事件发生时支持触发唤醒，软件可使能是否触发唤醒。

29.3.7 时间戳

BACKUP 支持 TS 事件检测。TS 事件分为 TS 管脚输入事件或者内部 TS 检测事件，BACKUP 支持 1 个 TS 管脚输入事件（仅有 1 个 IO 可配置复用为 TS 管脚输入功能），支持 1 个内部 TS 检测功能（DVDD 掉电，BACKUP 电源域切到 VBAT 供电）。

每个 TS 事件可单独屏蔽，可单独配置为边沿或者电平检测，可单独配置极性，支持数字滤波功能（最大 3 个 BACKUP_CLK，可单独配置滤波级数）。TS 事件发生时，可单独锁存记录标志，软件可清除。TS 管脚输入事件发生时支持上报中断，软件可使能中断是否上报，软件可清除原始中断；TS 管脚输入事件发生时支持触发唤醒，软件可使能是否触发唤

醒。TS 事件发生时可以触发时间戳，每个 TS 事件可由软件配置是否使能触发时间戳。

检测到 TAMPER 事件后也可以触发时间戳，每个 TAMPER 事件可由软件配置是否使能触发时间戳。

当触发时间戳时，时间戳寄存器锁存当前 RTC 的实时时间日历（固定 24 小时模式），并且将 TSF 记录标志置 1，软件可清除。如果在 TSF 记录标志为 1 时，再次触发了时间戳，时间戳寄存器不更新，并将 TS_OVF 记录标志置 1，软件可清除。

29.3.8 外部 WKUP 管脚

BACKUP 的 3 个 IO 可配置复用为 WKUP 功能。3 个外部 WKUP 管脚事件，每个输入事件可单独屏蔽；可单独配置为边沿或者电平检测；支持数字滤波功能（最大 3 个 BACKUP_CLK，可单独配置滤波级数）；可单独配置极性。

当外部 WKUP 管脚事件发生时，单独锁存记录标志，软件可清除；该事件发生时支持上报中断，软件可使能中断是否上报，软件可清除原始中断；该事件发生时支持触发唤醒，软件可使能是否触发唤醒。

29.3.9 备份数据寄存器 BACKUP_STORE

实现 16 个 32bit 寄存器备份功能，可在低功耗或者 VBAT 模式保持数据，除非备份域掉电/BACKUP 全局软复位/入侵擦除，备份数据寄存器中的数据不会被系统复位或者低功耗模式清除。

管脚入侵事件或者内部入侵事件（FLASH RDP 保护等级从 1 降为 0）会导致备份数据寄存器擦除，每个事件是否擦除备份数据寄存器可单独使能。

29.3.10 BACKUP 相关的 IO

BACKUP 中有以下 IO，PA4，PA5，PA6/OSC32_IN，PA7/OSC32_OUT，PA8；详见管脚复用章节。

29.4 BACKUP 使用流程

29.4.1 初始化流程

在系统完成初始化之后，BACKUP 的配置流程如下：

步骤 1 配置 SYSCTRL 寄存器中的 LOCKEN.backup_cfg_lock 为 0，允许 backup 写操作。

步骤 2 配置 CRG 寄存器中的 PERI_CRG70.backup_cfg_srst_req 为 0，PERI_CRG70.backup_cfg_cken 为 1，对 backup 解复位并打开 backup 时钟门控。

步骤 3 配置 BACKUP_CRG0 为 0，解 BACKUP 的软件全局复位。

步骤 4 配置 BACKUP_CRG1，根据功能需求解 rtc/wkt 的复位，以及选择对应的时钟来源。

步骤 5 配置 BACKUP_EVENT_CTRL0，调整 BACKUP 的输入事件的滤波级数，0 表示不进行滤波，最多滤波级数为 3 个 backup_clk，如无特别要求，保持默认配置即可。

步骤 6 配置 BACKUP_EVENT_CTRL1 中的 xx_edge_level_sel 相关位域，选择外部管脚输入事件的电平判断方式（内部事件 ts_internal_edge_level_sel 和 tamper_internal_edge_level_sel 保持默认配置即可，不要更改）。

步骤 7 配置 BACKUP_EVENT_CTRL1 中的 xx_pol 相关位域，选择输入事件的触发极性：

- 配置 pol 为正极性，且配置 edge_level_sel 为电平检测模式，表示该事件高电平触发；
- 配置 pol 为正极性，且配置 edge_level_sel 为边沿检测模式，表示该事件上升沿触发；
- 配置 pol 为负极性，且配置 edge_level_sel 为电平检测模式，表示该事件低电平触发；
- 配置 pol 为负极性，且配置 edge_level_sel 为边沿检测模式，表示该事件下降沿触发；

步骤 8 配置 BACKUP_EVENT_CTRL2，选择 alarm/wkt 事件输出到管脚时的有效极性。

步骤 9 配置 BACKUP_TAMPER_ERASE_TRGWKT_EN，选择相关输入事件能否触发 WKT。

步骤 10 配置 BACKUP_TS_EN，选择相关输入事件能否触发时间戳。

步骤 11 在上述步骤完成后，配置 BACKUP_EVENT_CTRL1 中的 xx_en 位域，使能相关的输入事件。

步骤 12 配置 BACKUP_CRG1，配置 backup_cken 为 1 打开 BACKUP 的工作时钟，根据需要打开的子功能模块打开 rtc/wkt 的工作时钟门控。软件等待 6 个 backup_clk 时钟周期的时间，再进行其他操作，BACKUP 开始工作。

步骤 13 如果需要软件使能 WKT，首先配置 BACKUP_WKT_CFG .wkt_time，配置 WKT

的 Timer 计数阈值。然后配置 BACKUP_WKT_CFG .soft_wkt_en 为 1，WKT 就会按照（wkt_time+1）个 wkt_clk 周期性产生 wkt 事件，直到 soft_wkt_en 配置为 0。

29.4.2 处理事件记录标志

只要 backup_clk 存在，以上功能的产生事件，均会锁存记录标志供查询。事件记录标志汇总如下：
ts_ovf_record_flag/tsf_record_flag/ts_internal_record_flag/ts_in_io_record_flag/tamper_internal_record_flag/tamper_in_io_record_flag/wkup_in_io_record_flag/alarm_record_flag/wkt_record_flag .

步骤 1 查询 BACKUP_RECORD_FLAG 中的 xx_record_flag 相关位域，查询每个产生事件是否发生过。

步骤 2 配置 BACKUP_RECORD_FLAG_CLR 中的 xx_record_flag_clr 相关位域，软件对相应 bit 写 1 则芯片内部自动清除相关事件的产生标志位。

29.4.3 处理唤醒

进入低功耗模式前，对需要触发唤醒的 BACKUP 事件进行配置，在进入低功耗之后，只要 backup_clk 存在，BACKUP 产生的事件可作为唤醒事件，可以唤醒各种低功耗模式。唤醒事件汇总如下：
ts_in_io_wkupreq/tamper_internal_wkupreq/tamper_in_io_wkupreq/wkup_in_io_wkupreq/alarm_wkupreq/wkt_wkupreq。

步骤 1 配置 BACKUP_WKUPREQ_EN 中的 xx_wkupreq_en 相关位域，控制每个产生事件能否触发唤醒的使能。

步骤 2 配置 BACKUP_WKUPREQ_STATUS 中的 xx_wkupreq_status 相关位域，查询每个产生事件是否触发了唤醒的状态。

步骤 3 配置 BACKUP_RECORD_FLAG_CLR 中的 xx_record_flag_clr 相关位域，可清除 BACKUP_WKUPREQ_STATUS 对应的产生事件记录标志。

29.4.4 处理中断

在 backup_clk 存在并且 BACKUP 的 apb clk 存在的情况下，以上功能的产生事件，均可产生中断，在 BACKUP 中汇总成一个 backup_int 送到 cpu。中断事件汇总如下：
clk_1hz/ts_in_io_int/tamper_internal_int/tamper_in_io_int/wkup_in_io_int/alarm_int/wkt_int.

步骤 1 配置 BACKUP_INT_EN.Xx_int_en，控制每个产生事件的中断是否使能。

步骤 2 查询 BACKUP_INT_RAW.Xx_int_raw，查询每个产生事件的原始中断。

步骤 3 查询 BACKUP_INT_STATUS.Xx_int_status，查询每个产生事件是否触发了中断状态。

步骤 4 配置 BACKUP_INT_CLR.Xx_ int_clr，软件对相应 bit 写 1 则芯片内部自动清除相关标志位。

⚠注意

在使用 BACKUP 中断功能前，先关闭 BACKUP 所有中断使能，清除 BACKUP 所有原始中断标志，再打开 BACKUP 的中断使能，最后使能 CPU CLIC 中断。

29.4.5 处理时间戳

步骤 1 查询 BACKUP_RECORD_FLAG.tsf_record_flag 和 BACKUP_RECORD_FLAG.ts_ovf_record_flag，获取当前时间戳的状态。也可以通过中断机制获取（查询 INT_RAW，或者打开时间戳的中断使能将中断主动上报到 CPU）

步骤 2 软件读取 BACKUP_TS_YMDW 和 BACKUP_TS_HMS，查询产生的时间戳日历和时间。

步骤 3 配置 BACKUP_RECORD_FLAG_CLR.tsf_ovf_record_flag_clr 和 BABACKUP_RECORD_FLAG_CLR.tsf_record_flag_clr，清除产生的时间戳事件记录标志。

29.5 BACKUP_CTRL 寄存器概览

BACKUP_CTRL 基地址如下：

- BACKUP_CTRL 基地址为：0x61006000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	BACKUP_CRG0	0x0	32	RW	0x00000001	BACKUP 配置寄存器 0
2	BACKUP_CRG1	0x4	32	RW	0x00001805	BACKUP 配置寄存器 1
3	BACKUP_EVENT_CTRL0	0x8	32	RW	0x22022A2A	BACKUP 事件滤波配置寄存器
4	BACKUP_EVENT_C	0xc	32	RW	0x07FC0100	BACKUP 事件控制

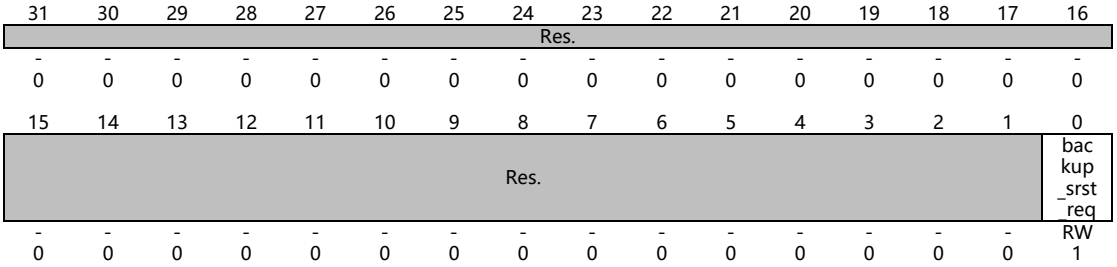
No.	Names	Offset Address	SIZE	R/W	Default	Description
	TRL1					寄存器
5	BACKUP_EVENT_CTRL2	0x10	32	RW	0x00000000	BACKUP 事件输出控制寄存器
6	BACKUP_TAMPER_ERASE_TRGWKT_EN	0x14	32	RW	0x0000000F	BACKUP TAMPER 事件功能选择寄存器
7	BACKUP_TS_EN	0x18	32	RW	0x0000003F	BACKUP 时间戳使能寄存器
8	BACKUP_WKT_CFG	0x1c	32	RW	0x00000000	BACKUP WKT 配置寄存器
9	BACKUP_INT_RAW	0x20	32	RO	0x00000000	BACKUP 原始中断状态寄存器
10	BACKUP_INT_CLR	0x24	32	W1P	0x00000000	BACKUP 中断清除寄存器
11	BACKUP_INT_EN	0x28	32	RW	0x000007FF	BACKUP 中断使能寄存器
12	BACKUP_INT_STATUS	0x2c	32	RO	0x00000000	BACKUP 中断状态寄存器
13	BACKUP_RECORD_FLAG	0x30	32	RO	0x00000000	BACKUP 事件原始记录寄存器
14	BACKUP_RECORD_FLAG_CLR	0x34	32	W1P	0x00000000	BACKUP 事件原始记录清除寄存器，软件配置清除后，至少软件等待 6 个 backup_clk 时钟周期的时间后才生效。
15	BACKUP_WKUPREQ_EN	0x38	32	RW	0x000007FF	BACKUP 事件触发低功耗唤醒使能寄存器
16	BACKUP_WKUPREQ_STATUS	0x3c	32	RO	0x00000000	BACKUP 事件触发低功耗唤醒状态寄存器
17	BACKUP_TS_YMDW	0x40	32	RO	0x00000000	BACKUP 时间戳年月日寄存器，必须在 tsf_record_flag 为 1 时读才有效
18	BACKUP_TS_HMS	0x44	32	RO	0x00000000	BACKUP 时间戳时分秒寄存器，必须在 tsf_record_flag 为 1 时读才有效

No.	Names	Offset Address	SIZE	R/W	Default	Description
19	LSE_32K_CTRL	0x60	32	RW	0x00006C02	LSE 配置寄存器
20	BACKUP_TEST_0	0x74	32	RW	0x00000001	BACKUP 测试寄存器
21	BACKUP_TEST_1	0x78	32	RO	0x00000000	BACKUP 测试寄存器

29.6 BACKUP_CTRL 寄存器描述

29.6.1 BACKUP_CRG0

◆ **偏移地址：0x0　　复位值：0x00000001**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留。	0x0
backup_srst_req	[0]	RW	backup 全局软复位请求: 0：软复位请求撤离； 1：软复位请求有效。 注意：配置所有门控和解复位后，BACKUP_CTRL 和 IO 的配置可以立刻进行，软件等待 6 个 backup 32k 时钟周期的时间后才能开始 BACKUP 的配置，然后开始正常工作。	0x1

29.6.2 BACKUP_CRG1

◆ **偏移地址：0x4　　复位值：0x00001805**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

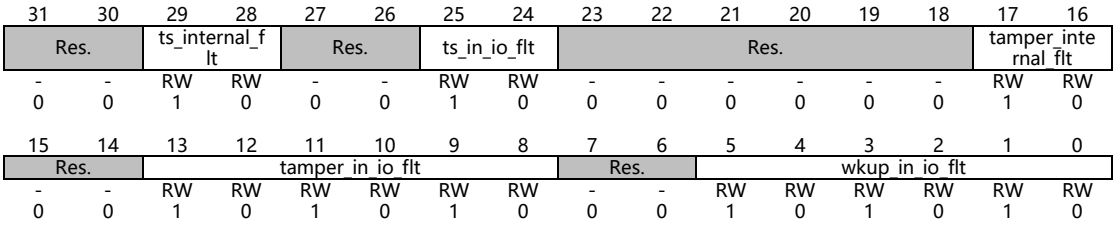
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			wkt_srst_req	rtc_srst_req	wkt_cken	rtc_cken	backup_cken	Res.		backup_cksel		Res.	wkt_cksel		
-	-	-	RW	RW	RW	RW	RW	-	-	RW	RW	-	RW	RW	RW
0	0	0	1	1	0	0	0	0	0	0	0	0	1	0	1

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
wkt_srst_req	[12]	RW	wkt 软复位请求: 0：软复位请求撤离； 1：软复位请求有效。 注意：只作用于 wkt，配置解复位后，软件等待 6 个 backup 32k 时钟周期的时间，再启动 wkt。	0x1
rtc_srst_req	[11]	RW	rtc 软复位请求: 0：软复位请求撤离； 1：软复位请求有效。 注意：只作用于日历和闹钟，配置解复位后，软件等待 6 个 backup 32k 时钟周期的时间，再配置 rtc。	0x1
wkt_cken	[10]	RW	wkt 时钟门控使能控制: 0：时钟关闭； 1：时钟打开。 注意：只作用于 wkt，配置使能后，软件等待 6 个 backup 32k 时钟周期的时间，再启动 wkt。	0x0
rtc_cken	[9]	RW	rtc 时钟门控使能控制: 0：时钟关闭； 1：时钟打开。 注意：只作用于日历和闹钟，配置使能后，软件等待 6 个 backup 32k 时钟周期的时间，再配置 rtc。	0x0
backup_cken	[8]	RW	Backup 工作时钟门控使能控制: 0：时钟关闭； 1：时钟打开。 注意：作用于除 rtc 外的 backup_func，包括 wkt。	0x0
reserved	[7:6]	-	保留。	0x0
backup_cksel	[5:4]	RW	BACKUP_CLK 时钟源选择: 0：LSI； 1：LSE； 2：HSE/512 分频。	0x0

Name	Bits	R/W	Description	Default
			注意：这是选择整个 backup 32k 时钟的来源，切换时钟时，先关闭 backup_cken，再切换时钟，再配置使能后，再启动 backup_func。	
reserved	[3]	-	保留。	0x0
wkt_cksel	[2:0]	RW	WKT 时钟源选择: 0：BACKUP_1Hz; 1：BACKUP_CLK; 2/3：BACKUP_CLK 的 4 分频; 4/5：BACKUP_CLK 的 8 分频; 6/7：BACKUP_CLK 的 16 分频。 注意：wkt 的时钟来源，切换时钟时，先关闭 wkt_cken，再切换时钟，再配置使能后，软件等待 6 个 backup 32k 时钟周期的时间，再启动 wkt_func。	0x5

29.6.3 BACKUP_EVENT_CTRL0

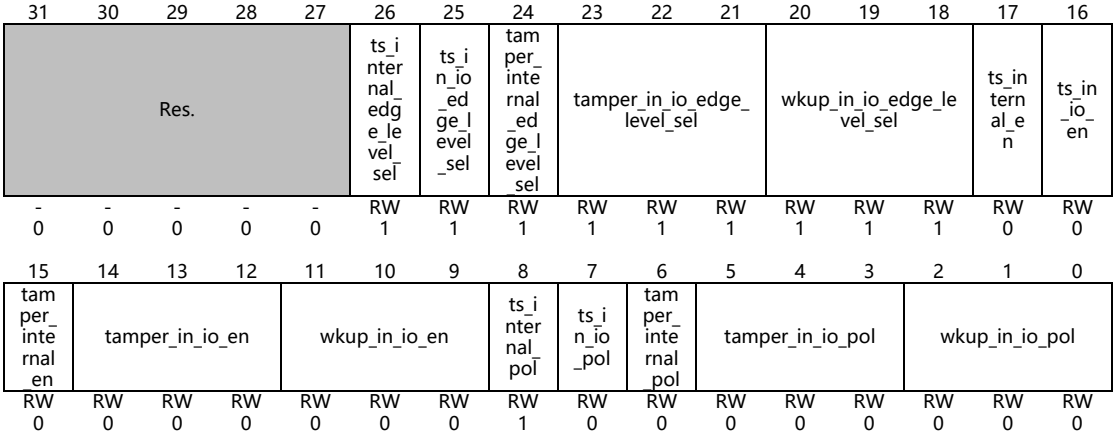
◆ **偏移地址：0x8　　复位值：0x22022A2A**



Name	Bits	R/W	Description	Default
reserved	[31:30]	-	保留。	0x0
ts_internal_flt	[29:28]	RW	ts internal 的滤波级数，BACKUP_CLK 单位。	0x2
reserved	[27:26]	-	保留。	0x0
ts_in_io_flt	[25:24]	RW	ts in 管脚 的滤波级数，BACKUP_CLK 单位。	0x2
reserved	[23:18]	-	保留。	0x0
tamper_internal_flt	[17:16]	RW	tamper internal 的滤波级数，BACKUP_CLK 单位。	0x2
reserved	[15:14]	-	保留。	0x0
tamper_in_io_flt	[13:8]	RW	tamper in 管脚的滤波级数，BACKUP_CLK 单位。	0x2A
reserved	[7:6]	-	保留。	0x0
wkup_in_io_flt	[5:0]	RW	wkup in 管脚的滤波级数，BACKUP_CLK 单位。	0x2A

29.6.4 BACKUP_EVENT_CTRL1

◆ **偏移地址：0xc　　复位值：0x07FC0100**



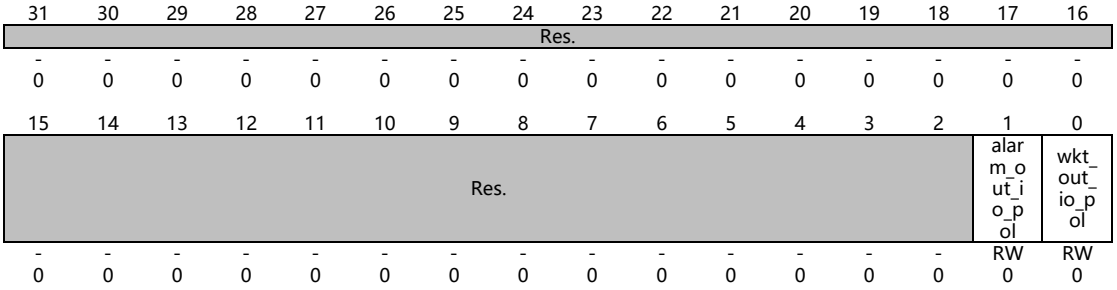
Name	Bits	R/W	Description	Default
reserved	[31:27]	-	保留。	0x0
ts_internal_edge_level_sel	[26]	RW	ts internal 事件的电平判断方式选择: 0：电平检测； 1：边沿检测（建议使用）。	0x1
ts_in_io_edge_level_sel	[25]	RW	ts in io 管脚输入事件的电平判断方式选择: 0：电平检测； 1：边沿检测。	0x1
tamper_internal_edge_level_sel	[24]	RW	tamper internal 事件的电平判断方式选择: 0：电平检测； 1：边沿检测（建议使用）。	0x1
tamper_in_io_edge_level_sel	[23:21]	RW	tamper in 管脚输入事件的电平判断方式选择: 0：电平检测； 1：边沿检测。	0x7
wkup_in_io_edge_level_sel	[20:18]	RW	wkup in 管脚输入事件的电平判断方式选择: 0：电平检测； 1：边沿检测。	0x7
ts_internal_en	[17]	RW	ts internal 事件的使能: 0：关闭； 1：使能。 注意：初始化的时候，先初始化 EVENT_CTRL0/1/2 其他 bit 后，软件等 6 个 backup 32k 时钟周期的时间再打开使能，去初始化的时候先关闭使能，软件等待 6 个 backup 32k 时钟周期的时间，再去初始化 EVENT_CTRL0/1/2 其他 bit。	0x0
ts_in_io_en	[16]	RW	ts in 管脚的使能:	0x0

Name	Bits	R/W	Description	Default
			0：关闭； 1：使能。 注意：初始化的时候，先初始化 EVENT_CTRL0/1/2 其他 bit 后，软件等 6 个 backup 32k 时钟周期的时间再打开使能，去初始化的时候先关闭使能，软件等待 6 个 backup 32k 时钟周期的时间，再去初始化 EVENT_CTRL0/1/2 其他 bit。	
tamper_internal_en	[15]	RW	tamper internal 事件的使能: 0：关闭； 1：使能。 注意：初始化的时候，先初始化 EVENT_CTRL0/1/2 其他 bit 后，软件等 6 个 backup 32k 时钟周期的时间再打开使能，去初始化的时候先关闭使能，软件等待 6 个 backup 32k 时钟周期的时间，再去初始化 EVENT_CTRL0/1/2 其他 bit。	0x0
tamper_in_io_en	[14:12]	RW	tamper in 管脚的使能: 0：关闭； 1：使能。 注意：初始化的时候，先初始化 EVENT_CTRL0/1/2 其他 bit 后，软件等 6 个 backup 32k 时钟周期的时间再打开使能，去初始化的时候先关闭使能，软件等待 6 个 backup 32k 时钟周期的时间，再去初始化 EVENT_CTRL0/1/2 其他 bit。	0x0
wkup_in_io_en	[11:9]	RW	wkup in 管脚事件的使能: 0：关闭； 1：使能。 注意：初始化的时候，先初始化 EVENT_CTRL0/1/2 其他 bit 后，软件等 6 个 backup 32k 时钟周期的时间再打开使能，去初始化的时候先关闭使能，软件等待 6 个 backup 32k 时钟周期的时间，再去初始化 EVENT_CTRL0/1/2 其他 bit。	0x0
ts_internal_pol	[8]	RW	ts internal 事件的极性: 0：正极性； 1：负极性(固定使用，不要修改）。	0x1
ts_in_io_pol	[7]	RW	ts in 管脚的极性: 0：正极性； 1：负极性。	0x0
tamper_internal_pol	[6]	RW	tamper internal 事件的极性: 0：正极性(固定使用，不要修改）； 1：负极性。	0x0

Name	Bits	R/W	Description	Default
tamper_in_io_pol	[5:3]	RW	tamper in 管脚的极性: 0：正极性; 1：负极性。	0x0
wkup_in_io_pol	[2:0]	RW	wkup in 管脚的极性: 0：正极性; 1：负极性。	0x0

29.6.5 BACKUP_EVENT_CTRL2

◆ **偏移地址：0x10　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:2]	-	保留。	0x0
alarm_out_io_pol	[1]	RW	此位用于配置 ALARM 输出到 IO 时的极性。 0：当 ALARM 标志位置 1，ALARM 在管脚上输出单次高电平脉冲 1：当 ALARM 标志位置 1，ALARM 在管脚上输出单次低电平脉冲	0x0
wkt_out_io_pol	[0]	RW	此位用于配置 wkt 输出到 IO 时的极性。 0：当 wkt 标志位置 1，wkt 在管脚上输出周期性高电平脉冲; 1：当 wkt 标志位置 1，wkt 在管脚上输出周期性低电平脉冲。	0x0

29.6.6 BACKUP_TAMPER_ERASE_TRGWKT_EN

◆ **偏移地址：0x14　　复位值：0x0000000F**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								tamper_internal_trg_wkt_en	tamper_in_io_trg_wkt_en			tamper_internal_erase_en	tamper_in_io_erase_en		
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
tamper_internal_trg_wkt_en	[7]	RW	tamper internal 事件触发 wkt 的的使能 0：关闭； 1：使能。	0x0
tamper_in_io_trg_wkt_en	[6:4]	RW	tamper in 管脚触发 wkt 的的使能 0：关闭； 1：使能。	0x0
tamper_internal_erase_en	[3]	RW	tamper internal 事件触发擦除 backup_store 备份寄存器的使能 0：关闭； 1：使能。	0x1
tamper_in_io_erase_en	[2:0]	RW	tamper in 管脚触发擦除 backup_store 备份寄存器的使能 0：关闭； 1：使能。	0x7

29.6.7 BACKUP_TS_EN

◆ **偏移地址：0x18　　复位值：0x0000003F**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										ts_internal_ts_en	ts_in_io_ts_en	tamper_internal_ts_en	tamper_in_io_ts_en		
-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:6]	-	保留。	0x0

Name	Bits	R/W	Description	Default
ts_internal_ts_en	[5]	RW	ts internal 事件触发时间戳的使能 0：关闭； 1：使能。	0x1
ts_in_io_ts_en	[4]	RW	ts in 管脚触发时间戳的使能 0：关闭； 1：使能。	0x1
tamper_internal_ts_en	[3]	RW	tamper internal 事件触发时间戳的使能 0：关闭； 1：使能。	0x1
tamper_in_io_ts_en	[2:0]	RW	tamper in 管脚触发时间戳的使能 0：关闭； 1：使能。	0x7

29.6.8 BACKUP_WKT_CFG

◆ **偏移地址：0x1c　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
soft_wkt_en	Res.														wkt_time
RW	-	-	-	-	-	-	-	-	-	-	-	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
wkt_time															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
soft_wkt_en	[31]	RW	WKT 软件使能。与此条件相或的还有 tamper_trg_wkt_en 使能后的 tamper 事件。 0：关闭； 1：使能。 注意：重复配置开关该 bit 之间的间隔至少大于 6 个 backup 32k 时钟周期的时间。	0x0
reserved	[30:17]	-	保留。	0x0
wkt_time	[16:0]	RW	wkt_time 计时器配置，单位 wkt_clk。 注意：配置 wkt_time 前，必须配置 soft_wkt_en、tamper_internal_trg_wkt_en、tamper_in_io_trg_wkt_en 都为 0，软件等待 100us 后，再配置 wkt_time，软件再等待 100us，再打开 soft_wkt_en 或 tamper_internal_trg_wkt_en 或 tamper_in_io_trg_wkt_en，只要其中一个使能位保持为 1，每（wkt_time+1）个 wkt_clk 就产生一个 wkt 事件。	0x0

29.6.9 BACKUP_INT_RAW

◆ **偏移地址：0x20　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		clk_1hz_int_raw	reserved1_int_raw	reserved2_int_raw	reserved3_int_raw	ts_in_io_int_raw	tamper_internal_int_raw	tamper_in_io_int_raw			wkup_in_io_int_raw			alarm_int_raw	wkt_int_raw
-	-	RO	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
clk_1hz_int_raw	[13]	RO	日历每 1s 产生一次中断，在设置日历后，先清除该中断，然后查询到该原始中断产生，表示日历已经更新。 0：无中断产生； 1：产生中断。	0x0
reserved1_int_raw	[12]	-	预留。	0x0
reserved2_int_raw	[11]	-	预留。	0x0
reserved3_int_raw	[10]	-	预留。	0x0
ts_in_io_int_raw	[9]	RO	ts in 管脚事件的原始中断： 0：无中断产生； 1：产生中断。	0x0
tamper_internal_int_raw	[8]	RO	tamper internal 事件的原始中断： 0：无中断产生； 1：产生中断。	0x0
tamper_in_io_int_raw	[7:5]	RO	tamper in 管脚产生事件的原始中断： 0：无中断产生； 1：产生中断。	0x0
wkup_in_io_int_raw	[4:2]	RO	wkup in 管脚产生事件的原始中断： 0：无中断产生； 1：产生中断。	0x0
alarm_int_raw	[1]	RO	alarm 事件的原始中断： 0：无中断产生； 1：产生中断。	0x0
wkt_int_raw	[0]	RO	wkt 事件的原始中断：	0x0

Name	Bits	R/W	Description	Default
			0：无中断产生； 1：产生中断。	

29.6.10 BACKUP_INT_CLR

◆ **偏移地址：0x24　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		clk_1hz_int_clr	reserved1_int_clr	reserved2_int_clr	reserved3_int_clr	ts_in_io_int_clr	tamper_internal_int_clr	tamper_in_io_int_clr			wkup_in_io_int_clr			alarm_int_clr	wkt_int_clr
-	-	W1P	-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
clk_1hz_int_clr	[13]	W1P	clk_1hz 的中断清除。	0x0
reserved1_int_clr	[12]	-	预留，不允许配 1。	0x0
reserved2_int_clr	[11]	-	预留，不允许配 1。	0x0
reserved3_int_clr	[10]	-	预留，不允许配 1。	0x0
ts_in_io_int_clr	[9]	W1P	ts in 管脚事件的中断清除。	0x0
tamper_internal_int_clr	[8]	W1P	tamper internal 事件的中断清除。	0x0
tamper_in_io_int_clr	[7:5]	W1P	tamper in 管脚产生事件的中断清除。	0x0
wkup_in_io_int_clr	[4:2]	W1P	wkup in 管脚产生事件的中断清除。	0x0
alarm_int_clr	[1]	W1P	alarm 事件的中断清除。	0x0
wkt_int_clr	[0]	W1P	wkt 事件的中断清除。	0x0

29.6.11 BACKUP_INT_EN

◆ **偏移地址：0x28　　复位值：0x000007FF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		clk_1hz_int_en	reserved1_int_en	reserved2_int_en	reserved3_int_en	ts_in_io_int_en	tamper_internal_int_en	tamper_in_io_int_en			wkup_in_io_int_en			alarm_int_en	wkt_int_en
-	-	RW	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
clk_1hz_int_en	[13]	RW	clk_1hz 的中断使能： 0：中断不使能； 1：中断使能。	0x0
reserved1_int_en	[12]	-	预留，不允许配 1。	0x0
reserved2_int_en	[11]	-	预留，不允许配 1。	0x0
reserved3_int_en	[10]	-	预留，不允许配 1。	0x1
ts_in_io_int_en	[9]	RW	ts in 管脚事件的中断使能： 0：中断不使能； 1：中断使能。	0x1
tamper_internal_int_en	[8]	RW	tamper internal 事件的中断使能： 0：中断不使能； 1：中断使能。	0x1
tamper_in_io_int_en	[7:5]	RW	tamper in 管脚产生事件的中断使能： 0：中断不使能； 1：中断使能。	0x7
wkup_in_io_int_en	[4:2]	RW	wkup in 管脚产生事件的中断使能： 0：中断不使能； 1：中断使能。	0x7
alarm_int_en	[1]	RW	alarm 事件的中断使能： 0：中断不使能； 1：中断使能。	0x1
wkt_int_en	[0]	RW	wkt 事件的中断使能： 0：中断不使能； 1：中断使能。	0x1

29.6.12 BACKUP_INT_STATUS

◆ **偏移地址：0x2c　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.		clk_1hz_int_status	reserved1_int_status	reserved2_int_status	reserved3_int_status	ts_in_io_int_status	tamper_internal_int_status	tamper_in_io_int_status			wkup_in_io_int_status			alarm_int_status	wkt_int_status
-	-	RO	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:14]	-	保留。	0x0
clk_1hz_int_status	[13]	RO	clk_1hz 的中断状态： 0：无中断产生； 1：产生中断。	0x0
reserved1_int_status	[12]	-	预留。	0x0
reserved2_int_status	[11]	-	预留。	0x0
reserved3_int_status	[10]	-	预留。	0x0
ts_in_io_int_status	[9]	RO	ts in 管脚事件的中断状态： 0：无中断产生； 1：产生中断。	0x0
tamper_internal_int_status	[8]	RO	tamper internal 事件的中断状态： 0：无中断产生； 1：产生中断。	0x0
tamper_in_io_int_status	[7:5]	RO	tamper in 管脚产生事件的中断状态： 0：无中断产生； 1：产生中断。	0x0
wkup_in_io_int_status	[4:2]	RO	wkup in 管脚产生事件的中断状态： 0：无中断产生； 1：产生中断。	0x0
alarm_int_status	[1]	RO	alarm 事件的中断状态： 0：无中断产生； 1：产生中断。	0x0
wkt_int_status	[0]	RO	wkt 事件的中断状态： 0：无中断产生；	0x0

Name	Bits	R/W	Description	Default
			1：产生中断。	

29.6.13 BACKUP_RECORD_FLAG

◆ **偏移地址：0x30　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			ts_ovf_record_flag	tsf_record_flag	ts_internal_record_flag	ts_in_io_record_flag	tamper_internal_record_flag	tamper_in_io_record_flag			wkup_in_io_record_flag			alarm_record_flag	wkt_record_flag
-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
ts_ovf_record_flag	[12]	RO	ts ovf 溢出（标记位中的 TSF 未被软件清除前，又有时间戳事件产生）事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
tsf_record_flag	[11]	RO	任一ts/tamper原始事件，经过TS_EN使能控制后，真正触发了时间戳事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
ts_internal_record_flag	[10]	RO	ts internal 事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
ts_in_io_record_flag	[9]	RO	ts in 管脚事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
tamper_internal_record_flag	[8]	RO	tamper internal 事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
tamper_in_io_record_flag	[7:5]	RO	tamper in 管脚产生事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
wkup_in_io_record_flag	[4:2]	RO	wkup in 管脚产生事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0

Name	Bits	R/W	Description	Default
alarm_record_flag	[1]	RO	alarm 事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0
wkt_record_flag	[0]	RO	wkt 事件的事件标记： 0：没有产生事件； 1：有事件产生。	0x0

29.6.14 BACKUP_RECORD_FLAG_CLR

◆ **偏移地址：0x34　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			ts_ovf_record_flag_clr	tsf_record_flag_clr	ts_internal_record_flag_clr	ts_in_io_record_flag_clr	tamper_internal_record_flag_clr	tamper_in_io_record_flag_clr			wkup_in_io_record_flag_clr			alarm_record_flag_clr	wkt_record_flag_clr
-	-	-	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
ts_ovf_record_flag_clr	[12]	W1P	ts ovf 溢出（标记位中的 TSF 未被软件清除前，又有时间戳事件产生）事件的事件标记的清除使能。	0x0
tsf_record_flag_clr	[11]	W1P	任一ts/tamper原始事件，经过TS_EN使能控制后，真正触发了时间戳事件的事件标记的清除使能。	0x0
ts_internal_record_flag_clr	[10]	W1P	ts internal 事件的事件标记的清除使能。	0x0
ts_in_io_record_flag_clr	[9]	W1P	ts in 管脚事件的事件标记的清除使能。	0x0
tamper_internal_record_flag_clr	[8]	W1P	tamper internal 事件的事件标记的清除使能。	0x0
tamper_in_io_record_flag_clr	[7:5]	W1P	tamper in 管脚产生事件的事件标记的清除使能。	0x0
wkup_in_io_record_flag_clr	[4:2]	W1P	wkup in 管脚产生事件的事件标记的清除使能。	0x0
alarm_record_flag_clr	[1]	W1P	alarm 事件的事件标记的清除使能。	0x0

Name	Bits	R/W	Description	Default
wkt_record_flag_clr	[0]	W1P	wkt 事件的事件标记的清除使能。	0x0

29.6.15 BACKUP_WKUPREQ_EN

◆ **偏移地址：0x38　　复位值：0x000007FF**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
wkupreq_mode	Res.														
RW	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			reserved1_wkupreq_en	reserved2_wkupreq_en	reserved3_wkupreq_en	ts_in_io_wkupreq_en	tamper_internal_wkupreq_en	tamper_in_io_wkupreq_en			wkup_in_io_wkupreq_en			alarm_wkupreq_en	wkt_wkupreq_en
-	-	-	RW	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	1

Name	Bits	R/W	Description	Default
wkupreq_mode	[31]	RW	唤醒信号模式选择： 0：产生的唤醒低功耗信号是一个持续到清除前的电平（默认配置，推荐使用） 1：产生的唤醒低功耗信号是一个 BACKUP_CLK 的脉冲(如默认配置无问题请勿使用)	0x0
reserved	[30:13]	-	保留。	0x0
reserved1_wkupreq_en	[12]	-	预留，不允许配 1。	0x0
reserved2_wkupreq_en	[11]	-	预留，不允许配 1。	0x0
reserved3_wkupreq_en	[10]	-	预留，不允许配 1。	0x1
ts_in_io_wkupreq_en	[9]	RW	ts in 管脚事件的事件标记产生唤醒的使能： 0：不使能； 1：使能。	0x1
tamper_internal_wkupreq_en	[8]	RW	tamper internal 事件的事件标记产生唤醒的使能： 0：不使能； 1：使能。	0x1
tamper_in_io_wkupreq_en	[7:5]	RW	tamper in 管脚产生事件的事件标记产生唤醒的使能： 0：不使能； 1：使能。	0x7

Name	Bits	R/W	Description	Default
wkup_in_io_wkupreq_en	[4:2]	RW	wkup in 管脚产生事件的事件标记产生唤醒的使能： 0：不使能； 1：使能。	0x7
alarm_wkupreq_en	[1]	RW	alarm 事件的事件标记产生唤醒的使能： 0：不使能； 1：使能。	0x1
wkt_wkupreq_en	[0]	RW	wkt 事件的事件标记产生唤醒的使能： 0：不使能； 1：使能。	0x1

29.6.16 BACKUP_WKUPREQ_STATUS

◆ **偏移地址：0x3c　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

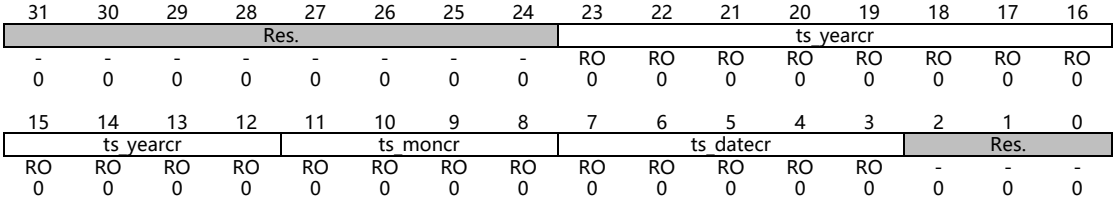
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			reserved1_wkupreq_status	reserved2_wkupreq_status	reserved3_wkupreq_status	ts_in_io_wkupreq_status	tamper_internal_wkupreq_status	tamper_in_io_wkupreq_status			wkup_in_io_wkupreq_status			alarm_wkupreq_status	wkt_wkupreq_status
-	-	-	-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
reserved1_wkupreq_status	[12]	-	预留	0x0
reserved2_wkupreq_status	[11]	-	预留	0x0
reserved3_wkupreq_status	[10]	-	预留	0x0
ts_in_io_wkupreq_status	[9]	RO	ts in 管脚事件的唤醒状态 0：没有产生事件； 1：产生事件。	0x0
tamper_internal_wkupreq_status	[8]	RO	tamper internal 事件的唤醒状态： 0：没有产生事件； 1：产生事件。	0x0
tamper_in_io_wkupreq_status	[7:5]	RO	tamper in 管脚产生事件的唤醒状态： 0：没有产生事件； 1：产生事件。	0x0

Name	Bits	R/W	Description	Default
wkup_in_io_wkupreq_status	[4:2]	RO	wkup in 管脚产生事件的唤醒状态： 0：没有产生事件； 1：产生事件。	0x0
alarm_wkupreq_status	[1]	RO	alarm 事件的唤醒状态： 0：没有产生事件； 1：产生事件。	0x0
wkt_wkupreq_status	[0]	RO	wkt 事件的唤醒状态： 0：没有产生事件； 1：产生事件。	0x0

29.6.17 BACKUP_TS_YMDW

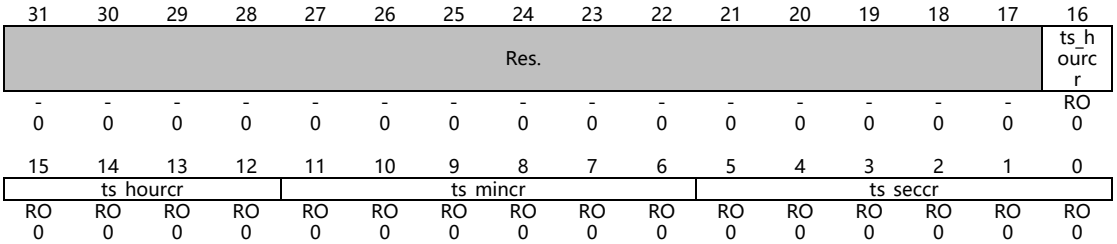
◆ **偏移地址：0x40　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
ts_yearcr	[23:12]	RO	时间戳-年。	0x0
ts_moncr	[11:8]	RO	时间戳-月。	0x0
ts_datecr	[7:3]	RO	时间戳-日。	0x0
reserved	[2:0]	-	保留。	0x0

29.6.18 BACKUP_TS_HMS

◆ **偏移地址：0x44　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:17]	-	保留。	0x0

Name	Bits	R/W	Description	Default
ts_hourcr	[16:12]	RO	时间戳-时。	0x0
ts_mincr	[11:6]	RO	时间戳-分。	0x0
ts_seccr	[5:0]	RO	时间戳-秒。	0x0

29.6.19 LSE_32K_CTRL

◆ **偏移地址：0x60　　复位值：0x00006C02**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
xo32k_rdy_delay_tm		xo32k_drvmux_tm	xo32k_drive_core_tm	xo32k_rmos_tm		xo32k_ibias_tm			xo32k_drive_tm		xo32k_rmos_lvl_sel	xo32k_ex_en	xo32k_force_rdy_flag_en	xo32k_rdy_en	xo32k_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	1	1	0	1	1	0	0	0	0	0	0	0	0	1	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
xo32k_rdy_delay_tm	[15:14]	RW	xo32k ready 信号的延时时间选择： 0：xo32k_en=1 后，大约 500ms 后，ready 信号拉高 1：xo32k_en=1 后，大约 1500ms 后，ready 信号拉高 2/3：xo32k_en=1 后，大约 2500ms 后，ready 信号拉高。	0x1
xo32k_drvmux_tm	[13]	RW	xo32k mux 的使能： 0：关闭 mux，输出下拉到地； 1：打开 mux。	0x1
xo32k_drive_core_tm	[12]	RW	xo32k drive 驱动能力控制： 0：驱动能力设置为最低； 1：驱动能力设置为最高。	0x0
xo32k_rmos_tm	[11:10]	RW	xo32k 内部比较器阈值偏置电流负载 MOS 管调节，取值越大，阈值越高： 0：阈值电压设置为最低； 3：阈值电压设置为最高；	0x3
xo32k_ibias_tm	[9:7]	RW	xo32k 内部比较器阈值偏置电流调节，取值越大，阈值越高： 0：阈值电压设置为最低； 7：阈值电压设置为最高。	0x0

Name	Bits	R/W	Description	Default
xo32k_drive_tm	[6:5]	RW	xo32k drive 驱动能力控制，取值越大，阈值越高； 0：驱动能力设置为最低； 3：驱动能力设置为最高。	0x0
xo32k_rmos_lvl_sel	[4]	RW	xo32k 内部比较器阈值选择高低两档： 0：低档位； 1：高档位。	0x0
xo32k_ex_en	[3]	RW	xo32k 外灌使能控制信号，当 xo32k_en=1 时： 0：关闭外灌时钟通路； 1：使能外灌时钟通路，同时 xo 通路关闭。	0x0
xo32k_force_rdy_flag_en	[2]	RW	xo32k ready 信号的打拍时钟强制保持使能： 0：ready 信号的打拍时钟在 ready 信号给出后的几拍后关闭； 1：ready 信号的打拍时钟在 ready 信号给出后依然保持。	0x0
xo32k_rdy_en	[1]	RW	xo32k ready 信号输出使能： 0：ready 信号始终保持 0； 1：ready 信号在时钟稳定后输出 1。	0x1
xo32k_en	[0]	RW	xo32k 使能控制信号： 0：关闭 xo32k 电路； 1：使能 xo32k 电路。	0x0

29.6.20 BACKUP_TEST_0

◆ **偏移地址：0x74　　复位值：0x00000001**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												backup_sw_ctrl	xo32k_sel_clk_out		lsi_trim_sel
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
backup_sw_ctrl	[3]	RW	backup 域电源选择控制信号： 0：backup 域选择使用 DVDD，仅 DVDD 在位时； 1：选择为 VBAT。	0x0
xo32k_sel_clk_out	[2:1]	RW	xo32k 输出到数字的滤波功能选择： 0：选择 dfilter 模式； 1：选择 deglitch 模式；	0x0

Name	Bits	R/W	Description	Default
			2/3：不做滤波功能。 注意：芯片上电时需要软件配置为 2 或者 3。	
lsi_trim_sel	[0]	RW	LSI 的 trim 选择控制信号： 0：trim 值来自于模拟的锁存值； 1：trim 值来自于数字，from eflash	0x1

29.7 BACKUP_STORE 备份数据寄存器概览

29.7.1 BACKUP_STORE

BACKUP_STORE 基地址如下：

- BACKUP_STORE 基地址为：0x61006600

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	BACKUP_STORE	0x0+0x4*n	32	RW	0x00000000	BACKUP 备份寄存器（n 属于 0~15）

29.8 BACKUP_STORE 备份数据寄存器描述

29.8.1 BACKUP_STORE

◆ **偏移地址：0x0+0x4*n　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
backup_store															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
backup_store															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
backup_store	[31：0]	RW	BACKUP 备份寄存器	0x0

30 实时时钟（RTC）

30.1 RTC 简介

RTC(Real-Time Clock)实时时间日历模块主要用来建立系统时间日历，并支持一个闹钟。在低功耗模式以及系统复位的过程中，RTC 的逻辑仍然可以保持正常工作。

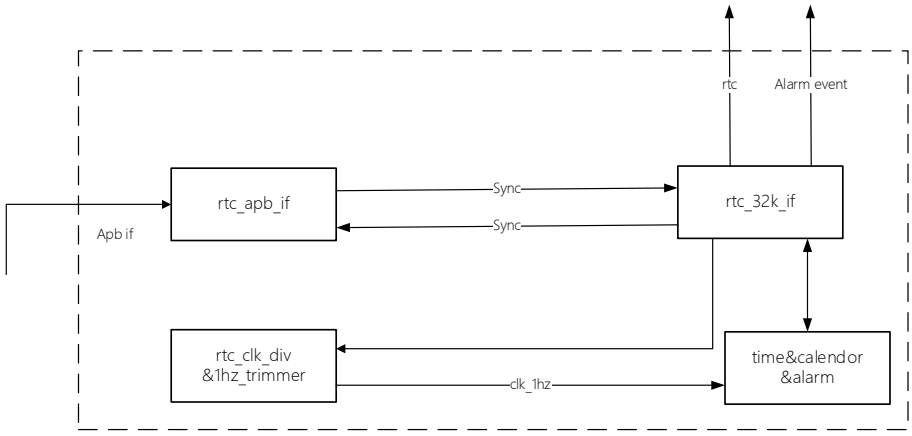
30.2 RTC 主要特性

- 时间日历功能：年/月/日/周/时/分/秒采用 BIN 码分别编码。
- 特殊年月标记：能够正确判断闰年和每月天数。
- 时钟校准：能按照配置对 1Hz 计数时钟（用于 1 秒钟计时）进行校正。
- 支持闹钟功能：闹钟功能可使能；支持上报 Alarm 中断；支持锁存 Alarm 记录标志，软件可清除。支持检测到 Alarm 事件后会可以产生唤醒事件，Alarm 事件是否产生唤醒事件可独立使能。
- 日历和闹钟支持 12h 和 24h 切换，支持动态切换。
- 支持寄存器写保护和解除写保护功能。
- 软件可动态设置（年月日时分秒），设置生效后，从设置时间开始工作。

30.3 RTC 功能说明

30.3.1 RTC 框图

图 30-1 RTC 框图



RTC 框图如上图 30-1 所示，其中各个模块功能如下：

1. rtc_apb_if：配置总线接口模块，完成从 APB 总线接收和发送数据，另外包括 RTC 中各种控制、数据和状态寄存器。RTC 的配置时钟同 BACKUP 的配置时钟。
2. rtc_32k_if：32K 时钟域接口模块，主要由寄存器电路、匹配电路和中断产生电路构成。
3. rtc_clk_div&1hz_trimmer：实现 1hz 时钟分频和校准功能。
4. time&calendor&alarm：实现时间、日历、闹钟相关功能。

30.3.2 时间日历

分别用来对年/月/日/时/分/秒进行计数，同时支持闰年，精度为 1 秒，可以在 deepsleep/shutdown/sleep 模式下运行。

30.3.3 可编程闹钟 ALARM

时间日历与闹钟配置寄存器（RTCWDA、RTCWTA）中的配置值做比较，如果相同，产生闹钟事件。

闹钟支持动态开关。

闹钟支持 12/24 小时模式（闹钟的 12/24 小时模式与当前时间日历的 12/24 小时模式配置保持一致）。

闹钟支持忽略年月日，在忽略年月日的模式下，每次时间日历的时分秒与闹钟时间配置寄存器（RTCWTA）中的配置值相同，会产生闹钟事件；在非忽略年月日的模式下，每次时

间日历的年月日时分秒与相应闹钟配置寄存器（RTCWDA、RTCWTA）中年月日时分秒的配置值相同，会产生闹钟事件。

闹钟支持周期模式和单次模式。周期模式表示在配置闹钟使能打开后，每次时间日历与相应闹钟配置寄存器中的配置值相同，都会产生闹钟事件，直到软件配置关闭闹钟使能（如果是非忽略年月日的模式下，则闹钟就只能产生一次）。单次模式表示在配置闹钟使能打开后，在时间日历与相应闹钟配置寄存器中的配置值相同时，产生一次闹钟事件，然后硬件自动关闭闹钟使能。

⚠注意

在单次模式下，软件配置闹钟打开后，如果在闹钟触发前想要关闭闹钟，需要先切换到周期模式，再关闭闹钟使能。

30.3.4 RTC 时钟校准

将 RTC_CLK 时钟分频得到 1Hz 时钟，用于 1 秒钟计时。

由于晶振自身固有的误差，再加上板级各种电容的负面影响，导致分频得到的 1Hz 时钟信号有一定误差，这个误差可能导致时钟日历计数在长时间积累之后产生误差，为了消除此误差，用户可通过配置寄存器进行校正，包含频率整数配置值（RTCTR[15:0]），频率小数配置值（RTCTR[25:16]）。

可以配置输出 LSCO 时钟到管脚，测量 RTC_CLK 32KHz 时钟的精准频率。（也可以配置输出 1Hz 时钟到管脚间接测量，假设实际测定值为 0.996915283203125Hz，则：实际发振频率 RTC_CLK=32768×0.996915283203125≈32666.92Hz）。

进一步计算补偿目标值，例如 RTC 输入时钟频率为 32666.92Hz，那么其整数部分为 32666，那么 RTCTR[15:0]配置为整数部分减 1 后的 16 进制 0x7F99。其小数部分为 0.92，乘以 1024 得到近似值 942，RTCTR[25:16]配置为 942 的 16 进制 0x03AE，合计得到 RTCTR 配置为 0x03AE7F99。

30.3.5 RTC 相关的 IO

Alarm 事件，可以通过配置 IO 复用输出到管脚 PA5，输出极性可控。

RTC_CLK 32KHz 时钟，可以通过配置 IO 复用输出到 PA5 管脚。

RTC 分频产生并校准后的 1Hz 时钟，可以通过配置 IO 复用输出到 PA5 管脚。

PA5 通过不同的 IO 复用配置选择不同的功能，详见 8 管脚复用（IO MUX)章节。

30.4 RTC 使用流程

30.4.1 初始化流程

在系统完成初始化之后，以及 BACKUP 的全局软复位解除后：

配置 RTCLOCK 为 0xCA53，解除 RTC 寄存器的写保护。

30.4.2 时间校准配置流程

步骤 1 将 RTC_CLK 32KHz（或者 RTC 产生的 1hz）配置到 IO 管脚观测。

步骤 2 根据上面章节的方法获取到目前的 32K 时钟的精确频率，并计算得到 RTCTR 的预期配置值。

步骤 3 配置 RTCTR；

步骤 4 配置 RTCCR.tsen，写 1 同步 RTC 时钟校准设置；回读该 bit 为 0 表示同步完成；

步骤 5 重新测量 RTC 产生的 1hz，确认时钟校准生效。如果一次校准不到位，可以重复以上步骤。

30.4.3 时间日历使用流程

步骤 1 配置 RTCWTC.hmode_cr，选择 12 还是 24 小时模式；如果是 12 小时制，再配置 RTCWTC.am_pm_cr，选择要配置的时间是上午还是下午；如果是 24 小时制，RTCWTC.am_pm_cr 固定配 0；

步骤 2 配置 RTCWTC，设置时分秒；

步骤 3 配置 RTCWDC，设置年月日；

步骤 4 配置 RTCCR.csen，写 1 同步时间日历设置；回读该 bit 为 0 表示同步完成；

步骤 5 日历开始正常工作，通过读取 RTCWTC 和 RTCWDC 获取实时时间日历。

30.4.4 闹钟功能使用流程

步骤 1 配置 RTCWTA.wta_en 为 1，打开闹钟。

步骤 2 配置 RTCWTA.wta_mask_ymd 选择是否忽略年月日；

步骤 3 如果是 12 小时制，通过配置 RTCWTA.wta_am_pm 选择所配置的闹钟时间是上午还是下午；如果是 12 小时制，RTCWTA.wta_am_pm 固定配置 0.

步骤 4 配置 RTCWTA.wta_mode 选择是周期闹钟模式还是单次闹钟模式；

步骤 5 配置 RTCWTA，设置时分秒；

步骤 6 配置 RTCWDA，设置年月日；

步骤 7 配置 RTCCR.asen，写 1 同步闹钟设置；回读该 bit 为 0 表示同步完成；

步骤 8 闹钟开始正常工作，在满足闹钟条件时产生 Alarm 事件，并将该事件送到 BACKUP 进行事件处理，进而可以产生唤醒，中断，请参考 29 备份寄存器（BACKUP）章节。

30.5 RTC 寄存器概览

RTC 寄存器基地址如下：

- RTC 寄存器基地址为：0x61006200

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	RTCCR	0x0	32	RW	0x00000000	RTC 控制寄存器
2	RTCTR	0x4	32	RW	0x00007FFF	时间校正寄存器
3	RTCWDA	0x8	32	RW	0x00000000	闹钟日期配置寄存器
4	RTCWTA	0xc	32	RW	0x00000000	闹钟时间配置寄存器
5	RTCWDC	0x10	32	RW	0x007D9246	日历日期计数寄存器
6	RTCWTC	0x14	32	RW	0x00000000	日历时间计数寄存器
7	RTCLOCK	0x34	32	RW	0x00000000	RTC 配置保护寄存器

30.6 RTC 寄存器列表

30.6.1 RTCCR

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

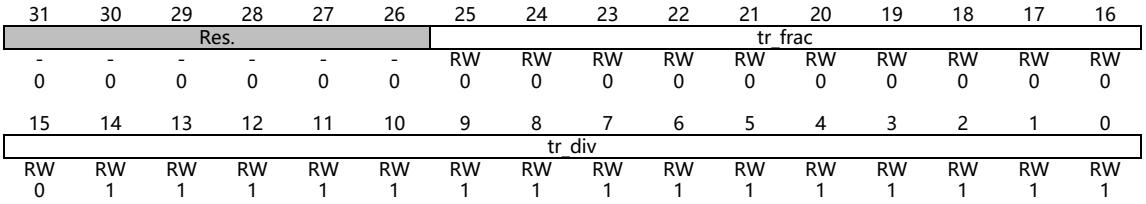
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													tsen	asen	csen
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留。	0x0
tsen	[2]	RW	读的时候表示读取 RTCTR 配置生效状态(RO)	0x0

Name	Bits	R/W	Description	Default
			写的时候表示 RTCTR 配置生效使能控制(WO) 软件写前要先读。 当读为 0 时，表示上一次写入寄存器的数据生效，允许再一次配置。 配置该 bit 为 1，在写入寄存器的数据生效后，芯片会将该 bit 自动归零。 当读为 1 时，表示上一次写到寄存器的数据还未生效，不能重复配置该 bit 为 1，要等该 bit 归零后，软件等待 6 个 backup 32k 时钟周期的时间才能再一次配置。 不允许向此寄存器写 0。	
asen	[1]	RW	读的时候表示 RTCWDA/RTCWTA 配置生效状态位(RO) 写的时候表示 RTCWDA/RTCWTA 配置生效使能位(WO) 软件写前要先读。 当读为 0 时，表示上一次写入寄存器的数据生效，允许再一次配置。 配置该 bit 为 1，在写入寄存器的数据生效后，芯片会将该 bit 自动归零。 当读为 1 时，表示上一次写到寄存器的数据还未生效，不能重复配置该 bit 为 1，要等该 bit 归零后，软件等待 6 个 backup 32k 时钟周期的时间才能再一次配置。 不允许向此寄存器写 0。	0x0
csen	[0]	RW	读的时候表示 RTCWDC/RTCWTC 配置生效状态位(RO) 写的时候表示 RTCWDC/RTCWTC 配置生效使能位(WO) 软件写前要先读。 当读为 0 时，表示上一次写入寄存器的数据生效，允许再一次配置同步。 配置该 bit 为 1，在写入寄存器的数据生效后，芯片会将该 bit 自动归零。 当读为 1 时，表示上一次写到寄存器的数据还未生效，不能重复配置该 bit 为 1，要等该 bit 归零后，软件等待 6 个 backup 32k 时钟周期的时间才能再一次配置同步。 不允许向此寄存器写 0。	0x0

30.6.2 RTCTR

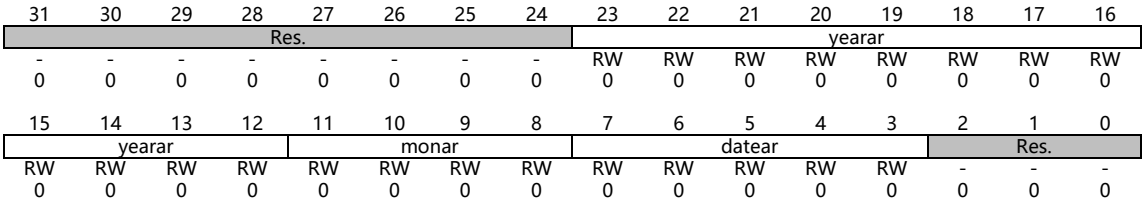
◆ **偏移地址：0x4　　复位值：0x00007FFF**



Name	Bits	R/W	Description	Default
reserved	[31:26]	-	保留。	0x0
tr_frac	[25:16]	RW	频率小数配置值。 例如时钟输入为 32666.92Hz，小数配置值为 0.92*1024 的近似值 942。 写表示配置，写入不立即生效，必须配合 RTCCR.tsen 配置生效使能。	0x0
tr_div	[15:0]	RW	频率整数配置值，减 1 配置。 例如时钟输入为 32666.92Hz，整数配置为 32665。 写表示配置，写入不立即生效，必须配合 RTCCR.tsen 配置生效使能。	0x7FFF

30.6.3 RTCWDA

◆ **偏移地址：0x8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
yearar	[23:12]	RW	年 ALARM 闹钟配置寄存器。 数据有效值从 0 到 4095。 写表示配置，写入不立即生效，必须配合 RTCCR.asen 配置生效使能。	0x0
monar	[11:8]	RW	月 ALARM 闹钟配置寄存器。 数据有效值从 1 到 12。	0x0

Name	Bits	R/W	Description	Default
			写表示配置，写入不立即生效，必须配合 RTCCR.asen 配置生效使能。	
datear	[7:3]	RW	日 ALARM 闹钟配置寄存器。 数据有效值从 1 到 31。 写表示配置，写入不立即生效，必须配合 RTCCR.asen 配置生效使能。	0x0
reserved	[2:0]	-	保留。	0x0

30.6.4 RTCWTA

◆ **偏移地址：0xc　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.											wta_en	wta_mask_ymd	wta_am_pm	wta_mode	hourar
-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

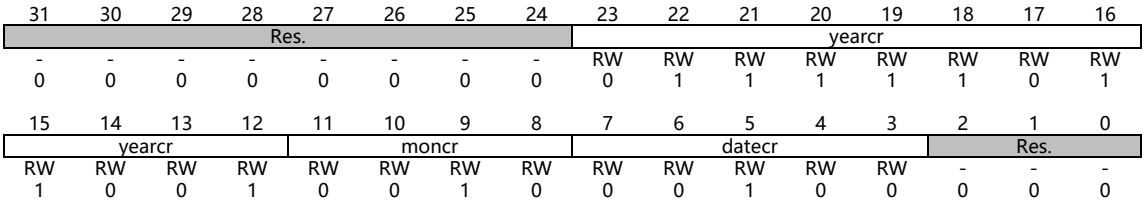
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
hourar				minar						secar					
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:21]	-	保留。	0x0
wta_en	[20]	RW	闹钟使能寄存器： 0：闹钟关闭； 1：使能闹钟。 如果是一次闹钟模式，那么需要在闹钟上报后该 bit 会硬件清 0，软件需要重新使用闹钟功能时，再次配置该 bit 为 1。 如果是周期性闹钟模式，保持该 bit 为 1 即可持续使能闹钟。 写入数据立即生效	0x0
wta_mask_ymd	[19]	RW	闹钟是否忽略年月日： 0：不忽略闹钟年月日设置； 1：忽略闹钟年月日设置，只关注时分秒。 写入数据立即生效。	0x0
wta_am_pm	[18]	RW	上午或下午设置： 0：AM； 1：PM。 12 小时有意义，24 小时固定配 0。 写入数据立即生效。	0x0

Name	Bits	R/W	Description	Default
			闹钟的 12/24 小时模式使用 WTC_HMODE，不单独设置。	
wta_mode	[17]	RW	一次闹钟，还是周期闹钟： 0：一次闹钟模式； 1：周期闹钟模式。 写入数据立即生效。	0x0
hourar	[16:12]	RW	时 ALARM 闹钟配置寄存器。 数据有效值从 0 到 23。 写表示配置，写入不立即生效，必须配合 RTCCR.asen 配置生效使能。	0x0
minar	[11:6]	RW	分 ALARM 闹钟配置寄存器。 数据有效值从 0 到 59。 写表示配置，写入不立即生效，必须配合 RTCCR.asen 配置生效使能。	0x0
secar	[5:0]	RW	秒 ALARM 闹钟配置寄存器。 数据有效值从 0 到 59。 写表示配置，写入不立即生效，必须配合 RTCCR.asen 配置生效使能。	0x0

30.6.5 RTCWDC

◆ **偏移地址：0x10　　复位值：0x007D9246**



Name	Bits	R/W	Description	Default
reserved	[31:24]	-	保留。	0x0
yearcr	[23:12]	RW	年计数配置和实时回读寄存器。 数据有效值从 0 到 4095。 写表示配置，写入不立即生效，必须配合 RTCCR.csen 配置生效使能。 读表示获取当前时间。	0x7D9
moncr	[11:8]	RW	月计数配置和实时回读寄存器。 数据有效值从 1 到 12。 写表示配置，写入不立即生效，必须配合 RTCCR.csen 配置生效使能。 读表示获取当前时间。	0x2

Name	Bits	R/W	Description	Default
datecr	[7:3]	RW	日计数配置和实时回读寄存器， 数据有效值从 1 到 31 写表示配置，写入不立即生效，必须配合 RTCCR.csen 配置生效使能。 读表示获取当前时间。	0x8
reserved_wdc	[2:0]	RW	保留。	0x6

30.6.6 RTCWTC

◆ **偏移地址：0x14　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													am_pm_cr	hmode_cr	hourcr
-	-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

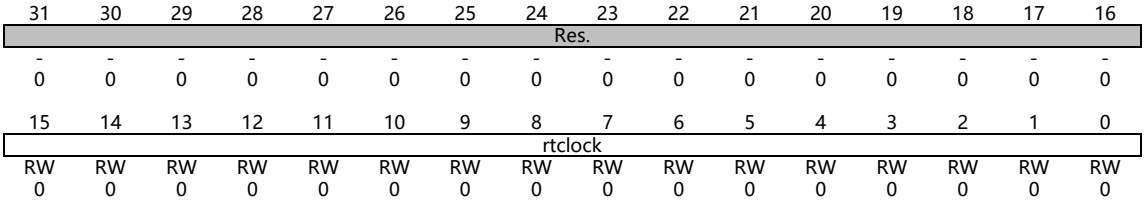
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
hourcr				mincr						seccr					
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留	0x0
am_pm_cr	[18]	RW	上午或下午标识： 0：AM； 1：PM。 12 小时制时有意义，24 小时制固定配 0。 写入数据立即生效；	0x0
hmode_cr	[17]	RW	12 小时或 24 小时制模式选择： 0：24 小时； 1：12 小时。 写入数据立即生效。	0x0
hourcr	[16:12]	RW	时计数配置和实时回读寄存器。 24 小时制时：数据有效值从 0 到 23 12 小时制时：数据有效值从 0 到 11 写表示配置，写入不立即生效，必须配合 RTCCR.csen 配置生效使能。 读表示获取当前时间。	0x0
mincr	[11:6]	RW	分计数配置和实时回读寄存器。 数据有效值从 0 到 59。 写表示配置，写入不立即生效，必须配合 RTCCR.csen 配置生效使能。 读表示获取当前时间。	0x0
seccr	[5:0]	RW	秒计数配置和实时回读寄存器。	0x0

Name	Bits	R/W	Description	Default
			数据有效值从 0 到 59。 写表示配置，写入不立即生效，必须配合 RTCCR.csen 配置生效使能。 读表示获取当前时间。	

30.6.7 RTCLOCK

◆ **偏移地址：0x34　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
rtclock	[15:0]	RW	写保护寄存器。 写入 0xCA53，解除写保护，可以对 RTC 其他寄存器进行写操作；写入非 0xCA53 的值，进入写保护，RTC 所有寄存器写不生效。	0x0

31 数模转换器（DAC）

31.1 DAC 简介

芯片集成了 3 路 12-bit 的 DAC，可以将数字信号转换为模拟电压信号输出，用于给 ACMP、PGA 等模块提供高精度的参考电压。每个 DAC 均支持独立的内部和外部参考电压切换、输出带 buffer 和不带 buffer 功能。

31.2 DAC 主要特性

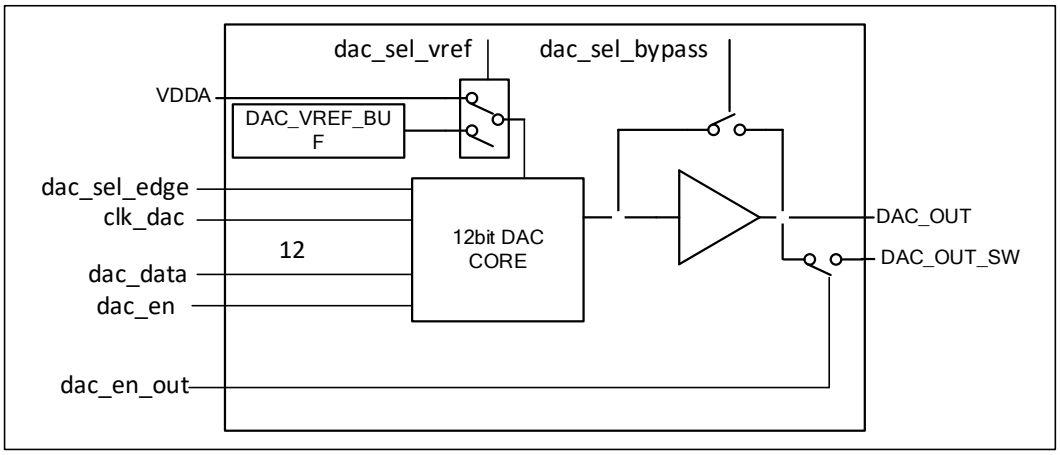
DAC 功能特点有：

- 支持 12bit 的高精度输出。
- 支持最大 1Msps 的采样速率。
- 支持每个 DAC 的参考电压可独立选择内部或者外部参考。
- 支持每个 DAC 带 buffer 或不带 buffer 输出。
- 支持三角波生成，步长与速率可配。
- 支持发送 DC 数据。
- 支持 DMA 功能，包括 DMA 溢出检测。
- 支持软件触发、硬件触发进行转换。

31.3 DAC 功能说明

31.3.1 DAC 框图

图 31-1 DAC 架构图



DAC 的参考电压可选，可以直接使用电源作为参考，用于低精度的应用；也可以使用内置的高精度的电压参考，用于高精度的应用。DAC 集成了一个输出 buffer，该 buffer 用于减小输出阻抗，并且可以用来直接驱动片外负载。

DAC_OUT_SW 为带开关的输出电压，由于 DAC 和其他 IP 复用管脚，当其他 IP 使用时，需要将 DAC 的输出关断（通过将 dac_en_out 配置为 0 实现）。

31.3.2 DAC 工作时钟

DAC 时钟频率最大 2MHz，支持时钟频率为 2/1/0.5/0.333/0.25/0.2/0.1/0.05MHz，对应的 CRG 寄存器 PERI_CRG57.dac0~3_ana_div 分频系数分别为 1,2,4,6,8,10,20,40。

31.3.3 DAC 引脚与连接关系

DAC 输出端支持直接输出和带开关的输出。DAC 直接输出端支持给 ACMP 提供内部的参考电压，DAC 直接输出端还支持给 PGA 提供参考电压。DAC 带开关的输出支持连接到管脚，为片外提供参考电压。

表 31-1 DAC 的带开关输出连接关系

DAC 输出端	DAC0	DAC1	DAC2
DAC_OUT_SW	管脚 PB3	管脚 PB1	管脚 PB6

31.3.4 DAC 使能

将 DAC_CTRL 寄存器中的 dac_en 寄存器配置为 1，即可使能对应的模拟 DAC 通道。

⚠注意

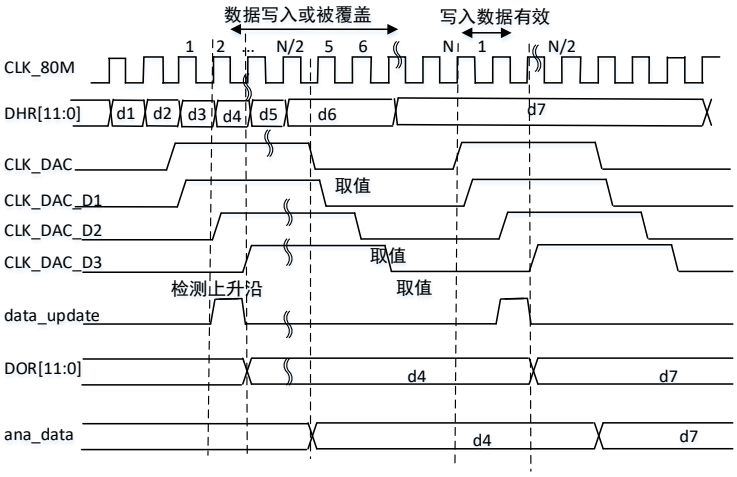
dac_en 只使能模拟 DAC 通道 N，即使寄存器 dac_en 位复位，DAC 通道 N 数字接口仍处于使能状态。若需要设置数字输出为 0，需要将 dac_trg_en 与 dac_dhr 寄存器配置为 0。

31.3.5 DAC 转换

数据发送时，用户将数据写入 DHR，在 dac_clk 的作用下自动加载到 DOR 寄存器进行数模转换。从数据写入 DHR 到转换完成，需要 1 个 dac_clk 的 cycle。

由于 dac 时钟来自于内部 HSI（16M）或者外部 HSE（8~40M）时钟，因此与 clk_apb 时钟异步。为了让寄存器 clk_apb 时钟域的信号，同步到 clk_dac 时钟域送出，我们将 clk_dac 时钟的上升沿作为控制信号，同步到 clk_apb 域，生成数据的 update 信号，使用这个控制信号将数据送出。由于模拟是在下降沿采样数据，因此建立时间至少可达 23 个 apb 周期，保持时间可达 17 周期左右（clk_apb=80M，clk_dac=2M 为例），并要求不得在 apb 时钟频率低于 clk_dac 时钟 10 倍的情况下打开 DAC。

图 31-2 DAC 转换时序图



31.3.6 DAC 触发选择

1. 自动触发模式

默认使用自动触发，配置到 DHR 的数据会在一个 clk_dac 时钟内自动搬运到 DOR 寄存器，同时送给 dac core。

自动模式下，软件需要注意配置的时间间隔，如果配置频率大于 clk_dac，那么数据会有丢失。

2. 软件触发模式

如果将 DAC 控制器配置为软件触发，那么一旦 SWTRIG 置 1，那么会在一个 clk_dac 周期内将 DHR 数据装载到 DOR，并发送数据到模拟端。

3. 硬件触发模式

触发源通过 DAC_CFG.dac_trg_sel[9:8]进行选择，当检测到触发信号的上升沿到来时，DHR 中的数据会在接下来的 clk_dac 周期内装载到 DOR，并发送数据到模拟端。软件配置 timer 计时触发间隔也要保证大于一个 clk_dac。

表 31-2 DAC 触发源选择

触发源	类型	dac_trg_sel[9:8]
swtrig	软件触发	0
ext_trg0	硬件触发 0（定时器）	1
ext_trg1	硬件触发 1（定时器）	2
ext_trg2	硬件触发 2（定时器）	3

31.3.7 DMA 数据传输

每个 DAC 都具有 DMA 功能，每个 DAC 需要一个 DMA 通道，总计需要三个 DMA 通道用于处理 DAC 的 DMA 请求。

1. DMA 请求

当 DMAENx 位置 1 时，如果发生外部触发（不能是软件触发），那么当 DAC_DHR 数据装载到 DOR 时，会同时产生 DMA 请求

2. DMA 溢出

DAC DMA 支持溢出上报。当第一个触发的 DMA 握手还没有完成，此时第二个外部触发有效，那么将会上报 DMA 溢出，DAC_STATE.dac_dmaovf 标志位会置 1，以报告此错误。可通过 DAC_CFG.dac_dmacfg 来配置溢出后是否影响后续 DMA 请求生成。当配置为 0（溢出后不再发出新的 DMA 请求）时，后续的 DMA 请求都不再生成，直到通过配置寄存器 DAC_SCR.dac_dmaovf_clr 清除溢出标志。

软件通过调节 DAC 触发频率来降低 DMA 负荷，以避免再次发生 DMA 溢出。最后再次使能 DMA 数据传输，继续进行 DAC 转换。

31.3.8 DAC 测试数据生成

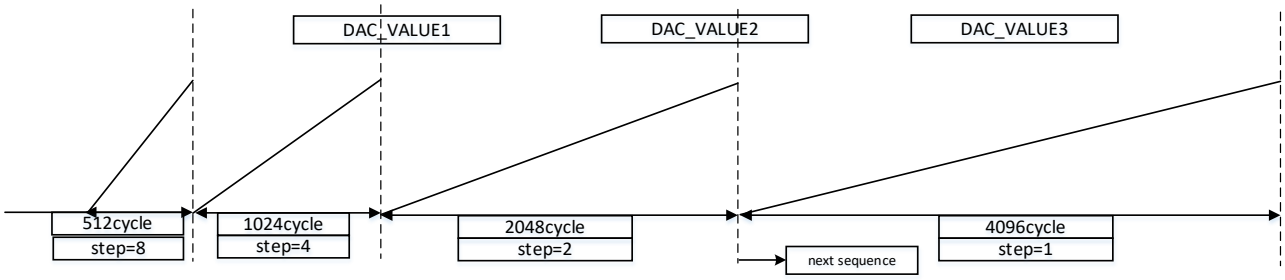
1. 直流信号生成

配置 DAC_CFG.dac_ramp_test_en 为 0，DAC_CFG.dac_trg_en 为 0，即关闭 ramp 波发送功能及触发使能，然后配置 DAC_DHR.dac_dhr 为想要的直流值即可。

2. 三角波信号生成

当配置 DAC_CFG.dac_ramp_test_en 为 1，ramp 波数据会接管发送的数据，而不再使用 DHR 中的数据。发送波形如下图所示。每次发送到 4095 后，会从 0 再次重复发送。可以通过配置 DAC_CFG.dac_ramp_step 来控制 ramp 波的步长（1/2/4/8）；通过 DAC_CFG.dac_ramp_rate 来配置 ramp 值的更新速率，即每隔多少个 DAC cycle 更新 ramp 值。

图 31-3 ramp 序列



31.3.9 DAC 参考电压选择

DAC 的参考电压来源有外部参考和内部参考，外部参考为管脚电压 VDDA。

内部参考的特性见下表 31-3。

表 31-3 DAC 内部参考源

符号	参数	条件	最小值	典型值	最大值	单位
VDDA	模拟电源电压	-	2.4	-	3.6	V
V_{out}	输出电压	VDDA≥2.8V	-	2.5	-	V
		2.4≤VDDA<2.8V	-	2	-	V
accuracy	精度	-	-	-	±5	mV
TV_{out}	输出电压温漂	温度范围-40~125℃	-	-	±5	mV

31.3.10 DAC 模式选择

DAC 的输出如果需要连接到管脚，需要带输出级的 buffer 电路，确保驱动能力。

配置 DAC_CTRL.dac_sel_bypass 为 1，DAC 工作于 buffer 模式；配置该信号为 0，DAC 工作于低速模式。DAC 的输出 buffer 的特性如下表 31-4 所示：

表 31-4 DAC 输出 buffer 特性

符号	参数	条件	最小值	典型值	最大值	单位
VDDA	模拟电源电压	-	2.4	-	3.6	V
VIN	输入电压范围	-	0.1	-	VDDA-0.8	V
VOUT	输出电压范围	-	0.15	-	VDDA-0.15	V
gain	传输增益	-	-	1.267	-	V
gain error	增益误差	-	-	-	1	%
offset	失调误差	-	-	-	4	mV
RLOAD	负载电阻	输出端直接接负载，DAC 输出端与负载之间无串接开关	20	-	-	kΩ
CLOAD	负载电容	-	-	-	10	pF

31.4 DAC 工作方式

31.4.1 中断处理

DAC 有 2 个原始中断，分别为数据加载完成中断，以及 dma 溢出中断，根据用户配置 CPU 响应相应中断。

- 数据加载完成中断

在 clk_dac 上升沿时，若触发信号到来或者 DMA 写操作到来，就将数据加载到 DHR 寄存器中，此时上报数据加载完成中断（DAC_STATE.dac_ulf）。

- dma 溢出中断

当使用 DMA 搬运数据时，若前一个数据未完成搬运，新的触发信号已触发进行下次搬运，此时 DAC 会产生 dma 溢出标志（DAC_STATE.dac_dmaovf）。

2 个中断分别对应中断使能为：DAC_CFG.dac_ulfie、DAC_CFG.dac_damovfie。

2 个中断分别对应中断清除为：DAC_STATE.dac_ulf_clr、DAC_STATE.dac_dmaovf_clr。

31.4.2 测试模式

模式一：发送直流信号

直流测试模式具体步骤如下：

步骤 1 配置 DAC_CFG.dac_ramp_test_en 为 0。

步骤 2 配置 DAC_CFG.dac_trg_en 为 0。

步骤 3 配置 DAC_DHR.dac_dhr 为想要的直流值。

模式二：发送 ramp 波信号

ramp 测试模式下具体步骤如下：

步骤 1 配置 ramp 值更新速率 DAC_CFG.dac_ramp_rate。

步骤 2 配置 ramp 值更新速率 DAC_CFG.dac_ramp_step。

步骤 3 配置 ramp 波使能 DAC_CFG.dac_ramp_test_en。

31.4.3 触发模式

触发模式具体步骤如下：

步骤 1 配置 DAC_CFG.dac_ramp_test_en，关闭测试模式。

步骤 2 根据场景，配置是否使能 DMA。通过配置 DAC_CFG.dac_dmaen 实现。

步骤 3 配置 DAC_CFG.dac_trg_sel 为 1，打开触发信号。

步骤 4 配置 DAC_CFG.dac_trg_en 为 1，打开触发模式使能。

31.5 DAC 寄存器概览

DAC 的基地址如下：

- DAC 寄存器基地址为：0x61005010

说明

寄存器中变量 n 取值范围为 0~2

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	DAC_CTRL	0x10+0x10*n	32	RW	0x0000000C	dac0~2 控制寄存器
2	DAC_DHR	0x14+0x10*n	32	RW	0x00000000	dac0~2 数字值保持寄存器
3	DAC_VALUE	0x18+0x10*n	32	RO	0x00000000	dac0~2 数据输出寄存器
4	DAC_CFG	0x1C+0x10*n	32	RW	0x00000000	dac0~2 配置寄存器
5	DAC_STATE	0x40	32	RO	0x00000000	dac 状态寄存器
6	DAC_SCR	0x44	32	W1P	0x00000000	dac 状态清除寄存器
7	DAC_SWTRG	0x48	32	W1P	0x00000000	dac 软件触发寄存器
8	DAC_COM	0x4C	32	RW	0x00000000	dac 通用寄存器

31.6 DAC 寄存器描述

31.6.1 DAC_CTRL

◆ **偏移地址：0x10+0x10*n　　复位值：0x0000000C**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

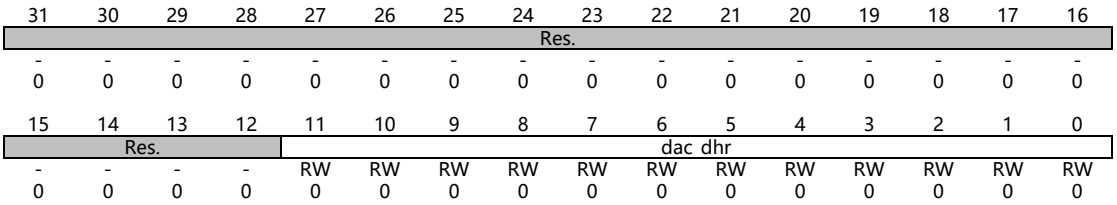
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								dac_en_out	dac_en_9b	dac_sel_buf_drv		dac_sel_bypass	dac_sel_edge	dac_sel_vref	dac_en
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留	0x0
dac_en_out	[7]	RW	DAC 带开关的输出使能控制信号 0：不使能； 1：使能； 其中，DAC0 输出连接到 ADC0_IN7，即 PB3； DAC1 输出连接到 ADC0_IN5，即 PB1； DAC2 输出连接到 ADC0_IN10，即 PB6；	0x0
dac_en_9b	[6]	RW	DAC 的 9bit 模式使能控制信号： 0：12bit 模式（默认）； 1：9bit 模式（bit4、bit5、bit6 旁路）；	0x0
dac_sel_buf_drv	[5:4]	RW	DAC 输出 buffer 的 NMOS 吸收电流档位控制信号： 0：20uA； 1：40uA；	0x0

Name	Bits	R/W	Description	Default
			2：50uA； 3：70uA；	
dac_sel_bypass	[3]	RW	DAC 的输出 buffer 旁路使能控制信号： 0：旁路不使能，输出 buffer 工作； 1：旁路使能，输出 buffer 不工作；	0x1
dac_sel_edge	[2]	RW	DAC 的输入数据的时钟沿选择： 0：选择上升沿； 1：选择下降沿；	0x1
dac_sel_vref	[1]	RW	DAC 的参考电压选择控制信号： 0：选择 VDDA； 1：选择 VREF_BUF 的输出；	0x0
dac_en	[0]	RW	DAC 的使能控制信号： 0：不使能（默认）； 1：使能；	0x0

31.6.2 DAC_DHR

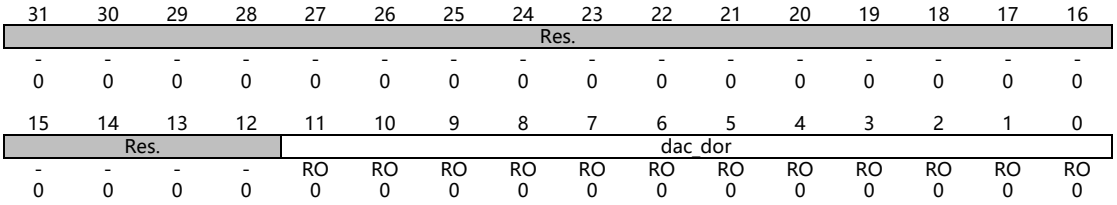
◆ **偏移地址：0x14+0x10*n　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
dac_dhr	[11:0]	RW	DAC 输出数字配置值，对应电压值。由软件人员写入，装载到 dor 生效。	0x0

31.6.3 DAC_VALUE

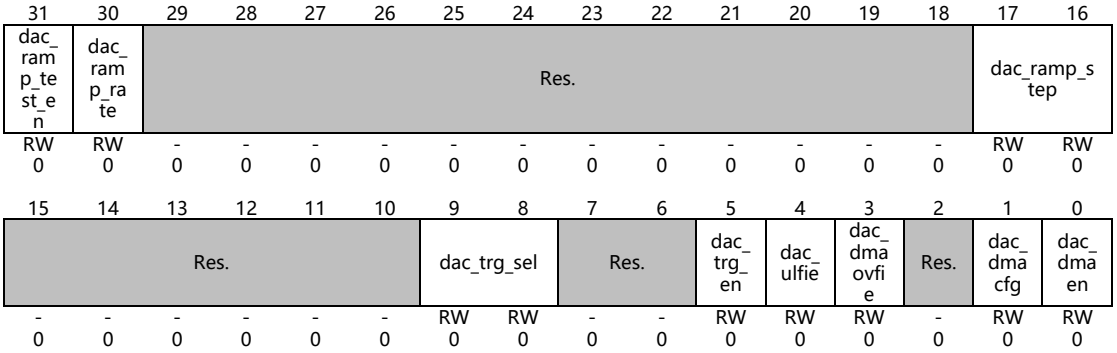
◆ **偏移地址：0x18+0x10*n　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:12]	-	保留	0x0
dac_dor	[11:0]	RO	DAC 正在生效的输出数据值，对应电压值。由硬件自动装载。	0x0

31.6.4 DAC_CFG

◆ **偏移地址：0x1C+0x10*n　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
dac_ramp_test_en	[31]	RW	ramp 波发送使能： 0：不使能 ramp 波； 1：使能 ramp 波；	0x0
dac_ramp_rate	[30]	RW	ramp 值更新速率： 0：每隔 5 DAC cycle 更新 ramp 波值； 1：每隔 10 DAC cycle 更新 ramp 波值；	0x0
reserved	[29:18]	-	保留	0x0
dac_ramp_step	[17:16]	RW	ramp 波 step 步长： 0：1； 1：2； 2：4； 3：8；	0x0
reserved	[15:10]	-	保留	0x0
dac_trg_sel	[9:8]	RW	触发源选择： 0：软件触发；	0x0

Name	Bits	R/W	Description	Default
			1：外部触发源 0； 2：外部触发源 1； 3：外部触发源 2；	
reserved	[7:6]	-	保留	0x0
dac_trg_en	[5]	RW	dac 触发使能： 0：使用自动转换，禁用所有触发源； 1：使用触发源，禁用自动转换； 注意：自动转换也能触发 dma。关闭 DMA 功能时，每次触发会加载当前 DHR 寄存器中数据到 DOR；打开 DMA 功能时，每次触发会从 DMA 搬运一个数据到 DHR 并且加载到 DOR。	0x0
dac_ulfie	[4]	RW	数据更新完成中断使能： 0：不上报 dac 的数据更新完成中断； 1：上报 dac 的数据更新完成中断；	0x0
dac_dmaovfie	[3]	RW	dma 溢出中断使能： 0：不上报 dac 的 dma 下溢中断； 1：上报 dac 的 dma 下溢中断；	0x0
reserved	[2]	-	保留	0x0
dac_dmacfg	[1]	RW	dma 溢出模式： 0：溢出后不再发出新的 dma 请求； 1：溢出不影响后续 dma 请求生成；	0x0
dac_dmaen	[0]	RW	dma 使能： 0：不使能 dma； 1：使能 dma；	0x0

31.6.5 DAC_STATE

◆ **偏移地址：0x40　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													dac2_ulf	dac1_ulf	dac0_ulf
-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													dac2_dmaovf	dac1_dmaovf	dac0_dmaovf
-	-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留	0x0
dac2_ulf	[18]	RO	dac2 输出数据加载完成标志	0x0

Name	Bits	R/W	Description	Default
			0：没有新的数据加载动作 1：已产生新的数据加载动作 注意：当发生触发时（非自动转换模式），并且发生数据已加载到 DOR（一个 dac 时钟内），逻辑自动将此位置高。	
dac1_ulf	[17]	RO	dac1 输出数据加载完成标志 0：没有新的数据加载动作 1：已产生新的数据加载动作 注意：当发生触发时（非自动转换模式），并且发生数据已加载到 DOR（一个 dac 时钟内），逻辑自动将此位置高。	0x0
dac0_ulf	[16]	RO	dac0 输出数据加载完成标志 0：没有新的数据加载动作 1：已产生新的数据加载动作 注意：当发生触发时（非自动转换模式），并且发生数据已加载到 DOR（一个 dac 时钟内），逻辑自动将此位置高。	0x0
reserved	[15:3]	-	保留	0x0
dac2_dmaovf	[2]	RO	dac2 dma 溢出标志： 0：没有 dma 溢出； 1：发生 dma 溢出；	0x0
dac1_dmaovf	[1]	RO	dac1 dma 溢出标志： 0：没有 dma 溢出； 1：发生 dma 溢出；	0x0
dac0_dmaovf	[0]	RO	dac0 dma 溢出标志： 0：没有 dma 溢出； 1：发生 dma 溢出；	0x0

31.6.6 DAC_SCR

- **偏移地址：0x44　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.													dac2_ulf_clr	dac1_ulf_clr	dac0_ulf_clr
-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													dac2_dmaovf_clr	dac1_dmaovf_clr	dac0_dmaovf_clr
-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:19]	-	保留	0x0
dac2_ulf_clr	[18]	W1P	清除 dac2 数据加载完成标志 0：不执行操作； 1：清除数据更新完成标志；	0x0
dac1_ulf_clr	[17]	W1P	清除 dac1 数据加载完成标志 0：不执行操作； 1：清除数据更新完成标志；	0x0
dac0_ulf_clr	[16]	W1P	清除 dac0 数据加载完成标志： 0：不执行操作； 1：清除数据更新完成标志；	0x0
reserved	[15:3]	-	保留	0x0
dac2_dmaovf_clr	[2]	W1P	清除 dac2 dma 溢出标志： 0：不执行操作； 1：清除数据更新完成标志；	0x0
dac1_dmaovf_clr	[1]	W1P	清除 dac1 dma 溢出标志： 0：不执行操作； 1：清除数据更新完成标志；	0x0
dac0_dmaovf_clr	[0]	W1P	清除 dac0 dma 溢出标志： 0：不执行操作； 1：清除数据更新完成标志；	0x0

31.6.7 DAC_SWTRG

◆ **偏移地址：0x48　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.													dac2_swtrg	dac1_swtrg	dac0_swtrg
-	-	-	-	-	-	-	-	-	-	-	-	-	W1P	W1P	W1P
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:3]	-	保留	0x0
dac2_swtrg	[2]	W1P	DAC2 软件触发	0x0
dac1_swtrg	[1]	W1P	DAC1 软件触发	0x0
dac0_swtrg	[0]	W1P	DAC0 软件触发	0x0

31.6.8 DAC_COM

◆ **偏移地址：0x4C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.											dac_en_test	dac_sel_test		dac_sel_vrefbuf	dac_vref_en
-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:5]	-	保留	0x0
dac_en_test	[4]	RW	DAC 的 testmux 使能控制信号： 0：不使能； 1：使能；	0x0
dac_sel_test	[3:2]	RW	DAC testmux 控制信号（经模拟 TOP 的 testmux1，连接到 ADC0_IN9，即 PB5）： 0：VRDAC0； 1：VRDAC1； 2：VRDAC2； 3：VREFBUF_DAC；	0x0
dac_sel_vrefbuf	[1]	RW	DAC 的 VREFBUF 输出电压值选择控制信号： 0：2.5V 输出； 1：2.0V 输出；	0x0
dac_vref_en	[0]	RW	DAC 的 VREF_BUF 使能控制信号：	0x0

Name	Bits	R/W	Description	Default
			0：VREF_BUF 不使能； 1：VREF_BUF 使能；	

32 模数转换器（ADC）

32.1 ADC 简介

芯片集成了 2 个独立的 12 位模数转换器，每个模数转换器有高达 16 个外部和 4 个内部（温度传感器、电压参考、DAC 输出电压、PGA 输出电压）通道，可在单次或扫描模式下执行转换。在扫描模式下，将对一组选定的模拟输入执行自动转换。

32.2 ADC 主要特性

- 支持 ADC0 和 ADC1 组成双 ADC 工作模式。
- 支持 ADC0 和 ADC1 各自独立工作。
- 支持 12bit 采样精度，有效位数 10.5bit。
- ADC 最高采样率为 8Msps。
- 可独立编程各通道采样时间。
- 各通道共用可编程控制转换时间。
- 支持误差校准功能。
- 支持 DMA 请求。
- 支持 20 个模拟输入通道，包括 4 个内部通道和 16 个外部通道。
- 支持单端输入模式和差分输入模式
- 支持采样模式：单独采样、过采样（最大 256 倍）。
- 支持转换模式：单次转换、连续转换、非连续转换。
- 支持两种可配置通道组：规则组、注入组。规则组支持 1 个结果寄存器，注入组支持 4 个结果寄存器。
- 硬件触发支持 23 个触发源可选择。
- 支持软件触发。

- 支持三个模拟看门狗用于通道电压监测并报警。
- 支持转换完成、电压检测、数据溢出等可配置中断。

32.3 ADC 性能指标

表 32-1 ADC 的特性

符号	参数	条件	最小值	典型值	最大值	单位
VDDA	模拟电源电压	-	2.4	-	3.6	V
I_{VDDA}	工作电流	VDDA=3.3V F_S=8Msps		-	1.49	mA
		VDDA=3.3V F_S=0.39Msps		-	0.05	mA
F_S	采样率	-	0.39	-	8	Msps
V_{REFP}	正参考电压	独立管脚或和 VDDA 合并	2.4	-	VDDA	V
V_{REFN}	负参考电压	与 VSSA 合并管脚	-	0	-	V
V_{AIN}	转换电压范围	-	V_{REFN}	-	V_{REFP}	V
R_{ADC}	ADC 采样开关电阻	-	-	-	0.5	KΩ
R_{AIN}	ADC 外部输入阻抗	详细参见公式 1	-	-	-	KΩ
C_{ADC}	内部采样和保持电容	-	-	-	8	pF
t_s	采样时间	-	50	-	1280	ns
t_{stab}	上电时间	-	0.125	-	-	μs

公式 1：R_{AIN} 最大值公式

$$R_{AIN} < \frac{t_s}{9 * C_{ADC}} - R_{ADC}$$

上面的公式（公式 1）用于确定使误差低于 1/4 LSB 的最大外部阻抗。

表 32-2 RAIN 的最大值与 t_s 的关系

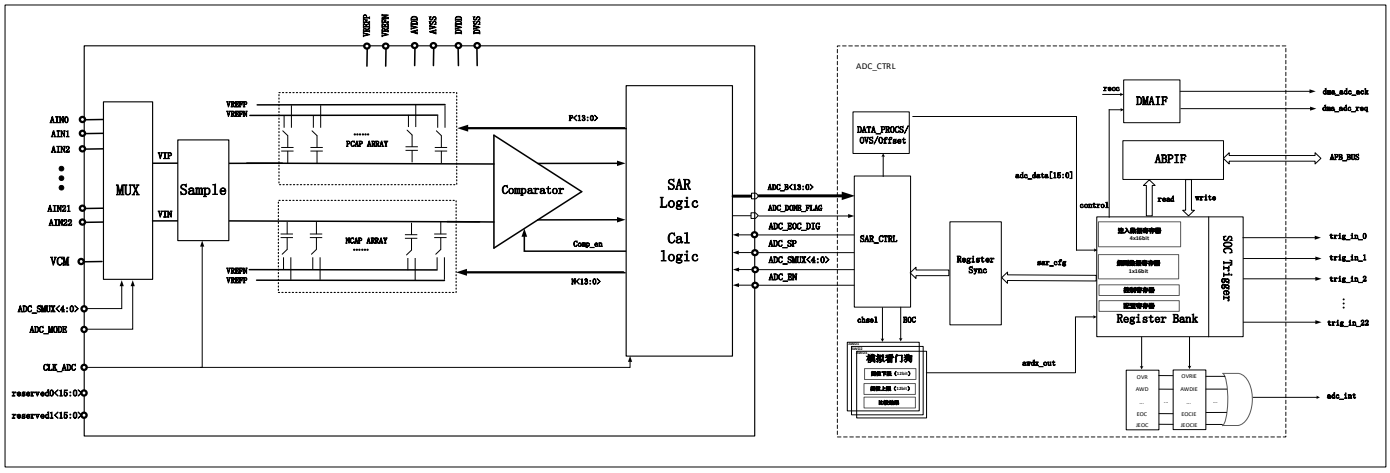
ts（ns）	RAIN（KΩ）
50	0.2
100	0.9

200	2.3
500	6.4
750	9.9
1000	13.4
1250	16.9

32.4 ADC 功能

32.4.1 ADC 框图

图 32-1 ADC 实现框图



32.4.2 ADC 时钟

ADC 有两个时钟，分别为系统时钟（clk_hs）和 ADC 时钟（clk_adc）。系统时钟由系统 CRG 模块产生，ADC 时钟由 ADC 控制器（ADC_CTRL）分频产生。ADC 时钟的高低电平持续时间分别由采样时间和转换时间决定。

32.4.3 ADC 分辨率

分辨率表示 ADC 能够转换量化模拟电压的精度，此 ADC 的分辨率为 12bit，不可配置。

32.4.4 ADC 参考电压

ADC 的参考电压来自于管脚 VREFP 和 VREFN，这些引脚用作比率基准电压源，以确定 ADC 转换输入范围。其中 VREN 接地。

32.4.5 ADC 输入信号

ADC 支持单端和差分信号输入，输入信号通过 ADC 的端口 ADC_INx 进行采样。

表 32-3 ADC 输入信号连接关系

输入信号	ADC0	ADC1
ADC_IN0	来自管脚-PA12	来自管脚-PB12
ADC_IN1	来自管脚-PA13	来自管脚-PB13
ADC_IN2	来自管脚-PA14	来自管脚-PB14
ADC_IN3	来自管脚-PA15	来自管脚-PB15
ADC_IN4	来自管脚-PB0	来自管脚-PC0
ADC_IN5	来自管脚-PB1	来自管脚-PC1
ADC_IN6	来自管脚-PB2	来自管脚-PC2
ADC_IN7	来自管脚-PB3	来自管脚-PC3
ADC_IN8	来自管脚-PB4	来自管脚-PC4
ADC_IN9	来自管脚-PB5	来自管脚-PC5
ADC_IN10	来自管脚-PB6	来自管脚-PC6
ADC_IN11	来自管脚-PB7	来自管脚-PC9
ADC_IN12	来自管脚-PB8	来自管脚-PC10
ADC_IN13	来自管脚-PB9	来自管脚-PC11
ADC_IN14	来自管脚-PB10	来自管脚-PC12
ADC_IN15	来自管脚-PB11	来自管脚-PC13
ADC_IN16	TSENSOR	PGA2_OUT
ADC_IN17	PGA0_OUT	PGA3_OUT
ADC_IN18	PGA1_OUT	DAC1_OUT
ADC_IN19	DAC0_OUT	DAC2_OUT

单端模式使用 ADC_SQRx(x=1~4)选择通道。通道 0~14 可通过将寄存器 ADC_DIFSEL.difsel 与通道对应的 bit 置 1，将该通道配为差分通道。当通道 N 配成差分通道时，该通道对应输入差分端口为 N 和 N+1,N=0~14。

⚠注意

来自管脚的信号会先经过 GPIO 的开关，ADC 使用时需要使能该开关。参考 8.3 IO 模拟模式描述。

32.4.6 ADC 启动和关闭

系统上电后，ADC 默认处于关闭状态，在任何转换启动或配置操作前，必须确认 ADC 处于启动状态。

启动 ADC 流程如下：

步骤 1 查询 ADC_CSR 为 0x0，ADC 处于关闭状态。

步骤 2 配置 ADC_CMDR 为 0x1，开始启动 ADC 模拟电路。

步骤 3 等待 ADC_ISR 中的 adrdy 置 1（120us），ADC 启动成功。

步骤 4 启动成功后，可通过配置 ADC_ISR_CLR.adrdy_clr 清除该 adrdy 状态。

关闭 ADC 流程如下：

步骤 1 查询 ADC_CSR 为 0x1 时，即当前没有正在进行中的转换，可以准备关闭 ADC。
如果有需要，可以配置 ADC_CMDR 为 0x10 停止规则组的转换，配置 ADC_CMDR 为 0x20 停止注入组的转换。

步骤 2 配置 ADC_CMDR 为 0x2。

步骤 3 查询 ADC_CSR 为 0x0，即认为 ADC 关闭成功。

32.4.7 ADC 自校准

本模块支持 ADC 上电自校准，校准的内容是数字偏移。上电自校准值一般不会存放到 Eflash 中。自校准分为单端模式和差分模式。

自校准偏移值，可以由软件配置寄存器 ADC_SEL_CAL_CFG.calfact_sel 选择是软件配置的值（单端：ADC_CALFACT_SOFT.soft_calfact_s；差分：ADC_CALFACT_SOFT.soft_calfact_d）或者是逻辑自校准的结果。

下面是逻辑单端自校准和差分自校准的步骤描述。

- 单端模式

逻辑单端自校准步骤：

步骤 1 配置 ADC_SEL_CAL_CFG.cal_difsel=0，选择单端。

步骤 2 配置 ADC_SEL_CAL_CFG.cal_vref 设定自校准基准电压。

步骤 3 配置 ADC_SEL_CAL_CFG.cal_sample_size 设定自校准样本量（8/16/32/64 次）。

步骤 4 配置 ADC_SEL_CAL_CFG.cal_smp 设定校准采样时间。

步骤 5 配置 ADC_CVS 设定通道转换时间。

步骤 6 配置 ADC_CMDR=0x8000_0000，启动校准。

步骤 7 控制器根据配置的自校准样本量 ADC_SEL_CAL_CFG.cal_sample_size 进行采集结果的累加与平均，得到平均值，并将平均值减去 ADC_SEL_CAL_CFG.cal_vref，得到固定的 offset 偏移误差。

步骤 8 控制器将 offset 偏移误差存入 ADC_CALFACT_AUTO.self_calfact_s。

步骤 9 自校准停止。

- 差分模式

逻辑差分自校准步骤与单端自校准步骤基本相同，差异是配置 ADC_SEL_CAL_CFG.cal_difsel=1，最终结果存入 ADC_CALFACT_AUTO.self_calfact_d。

32.4.8 ADC 通道选择

每个 ADC 有 20 个模拟输入通道来源。规则组和注入组各有一组控制转换序列长度和转换序列的配置。

规则组序列的长度由 ADC_SQRL.len 设定，最大长度是 16。规则组转换序列由 ADC_SQRx（x=1~4）寄存器决定。序列中的通道从 20 个输入通道中选择。仅在 ADC_CSR.adstart 为 0 时允许配置。

注入组序列的最大长度是 4。注入组转换序列和长分别由 ADC_JSQR.jsq1~4 和 ADC_JSQR.jlen 设定。序列中的通道从 20 个输入通道中选择。当注入组队列功能打开时（ADC_CFGR.jqdis=0），才可以在 ADC_CSR.jadstart 是 1 的情况下，更改 ADC_JSQR。

⚠注意

当注入组开启自动注入时，转换序列的长度由 ADC_SQRL.len 和 ADC_JSQR.jlen 求和值设定。自动注入参考 32.4.13 ADC注入通道采样 。

32.4.9 ADC 采样时间

在开始转换前，ADC 必须在被测电压源和 ADC 的嵌入式采样电容器之间建立直接连接。该采样时间必须足以使输入电压源将嵌入式电容器充电到输入电压电平。可以使用不同的采样时间对每个通道进行采样，采样时间可以使用 ADC_SMPRx（x=0~4）进行设定。采样时间的单位是一个 clk_hs 周期。采样时间决定了 clk_adc 的高电平持续时间。

⚠注意

采样时间配置值必须大于等于 9（默认配置 9）。

32.4.10 ADC 采样结果

表 32-4 单端输入电压转 12bit 数据表

模拟输入电压	ADC 单端转换结果
ADC_INx ≤ VREFN	RESULTx = 0
VREFN < ADC_INx < VREFP	RESULTx =4095*(ADC_INx-VREFN)/(VREFP-VREFN)
VINx≥VREFP	RESULTx = 4095

表 32-5 差分输入电压转 12bit 数据表

模拟输入电压	ADC 差分转换结果
(ADC_INx- ADC_IN(x+1)) ≤-1/2*VREFP	RESULTx = 0
-1/2*VREFP < (ADC_INx- ADC_IN(x+1)) <1/2*VREFP	RESULTx=4095*(ADC_INx-ADC_IN(x+1) +1/2*VREFP)/(VREFP-VREFN) -2048
(ADC_INx- ADC_IN(x+1)) ≥1/2*VREFP	RESULTx = 4095

32.4.11 ADC 转换时间

所有通道转换时间共用一个配置寄存器 ADC_CVS。转换时间的单位是一个 clk_hs 周期。转换时间决定了 clk_adc 的低电平持续时间。

⚠注意

转换时间配置必须大于 14。

32.4.12 ADC 外部触发和触发极性的转换

触发方式有两种，软件触发和硬件触发。

软件触发：

- 当 ADC_CFGR.exten=0x0 时，软件配置 ADC_CMDR.adstart_cmd=1，自动触发

ADC 规则组转换。

- 当 ADC_JSQR.jexten=0x0 时，软件配置 ADC_CMDR.jadstart_cmd=1，自动触发 ADC 注入组转换。

硬件触发：

触发源来自外部（PWM，TIMER），当使用外部触发时，需要在启动（ADC_CMDR.adstart_cmd/jadstart_cmd）前，配置规则/注入转换的外部触发源选择 ADC_CFGR.extsel/ADC_JSQR.jextsel（触发源有 23 个，具体见寄存器描述），以及外部触发边沿选择 ADC_CFGR.exten/ADC_JSQR.jexten（触发边沿选择可以选择上升沿触发，下降沿触发，上升沿或下降沿触发）。

当注入组的队列功能使能（ADC_CFGR.jqdis=0）时，只能选择硬件触发，不能选择软件触发。

当 ADC_CSR.adstart=0 时，规则组的硬件触发沿将被忽略。当 ADC_CSR.jadstart=0 时，注入组的硬件触发沿将被忽略。

表 32-6 规则组外部触发的极性配置

exten[1:0]	描述
00	软件触发。
01	在上升沿执行硬件触发检测。
10	在下降沿执行硬件触发检测。
11	在上升沿和下降沿都执行硬件触发检测。

表 32-7 注入组外部触发的极性配置

jexten[1:0]	描述
00	jqdis=1，队列关闭，软件触发； jqdis=0，队列使能，软件和硬件触发都不使能。
01	在上升沿执行硬件触发检测。
10	在下降沿执行硬件触发检测。
11	在上升沿和下降沿都执行硬件触发检测。

Dual ADC 模式下，只能选择硬件触发，而且两个 ADC 要选择相同的触发源和触发沿。

32.4.13 ADC 注入通道采样

- 触发注入

启动触发注入时，注入组优先级高于规则组。注入组触发后可以打断当前正在进行的规则组转换。要使用触发注入，需要将 ADC_CFGR.jauto 清零。

1. 通过软件或外部触发来启动规则通道转换。
2. 启动注入组转换，此后出现注入组触发（软件或外部）采样，则当前规则转换会复位，注入通道立刻进入序列转换。
3. 规则通道转换会在上次中断的规则转换处恢复。

如果在注入转换过程中出现规则转换的触发事件，那么注入序列结束时会立即执行规则序列转换。

⚠注意

禁止规则组和注入组同时开启非连续转换。非连续转换参考 32.4.16 ADC非连续转换模式 章节。

- 自动注入

当 jauto 置 1 时，注入组中的通道会在规则组通道（长度为 ADC_SQRL.len+1）都转换之后自动转换。这使得控制器可用于最多转换 20 个通道构成的序列。这些通道在 ADC_SQRx（x=1~4）和 ADC_JSQR 中设定。

⚠注意

开启自动注入时，注入组的 ADC_CSR.jadstart 必须保持关闭，并且不能配置非连续模式。

32.4.14 ADC 单次转换模式

在单次转换情况下，ADC 执行一次全部通道转换。ADC_CFGR.cont=0，可以通过以下外部触发方式实现单次转换：

- 对于规则通道，置位 ADC_CMDR.adstart_cmd
- 对于注入通道，置位 ADC_CMDR.jadstart_cmd
- 注入通道和规则通道，都要等待触发事件

在规则转换序列中，每转换了一个规则通道：

- 转换数据存储在 16 位 ADC_DR 中
- 产生 eoc 标志

- ADC_IER.eocie=1 时产生中断

当规则序列转换完成，除了以上操作外，还会：

- 产生 eos 标志，表示序列转换结束
- ADC_IER.eosie=1 时产生中断

在注入序列中，每转换了一个注入通道：

- 转换数据存储在 16 位 ADC_JDRx（x=1~4）寄存器中
- 产生 jeoc 标志
- ADC_IER.jeocie=1 时产生中断

当注入序列转换完成，除了以上操作外，还会：

- 产生 jeos 标志，表示序列转换结束
- ADC_IER.jeosie=1 时产生中断

然后当前 ADC 转换会停止。若为硬件触发，ADC_CSR.adstart/jadstart 会维持高，等待新的规则/注入触发事件；若为软件触发，ADC_CSR.adstart/jadstart 会硬件清 0，等到再次配置 ADC_CMDR.adstart_cmd/ jadstart_cmd 为 1 开启转换。

32.4.15 ADC 连续转换模式

在连续转换模式下，ADC 结束一个转换后立即启动一个新的转换。ADC_CFGR.cont=1，可通过外部触发或软件触发来启动转换。每次转换之后：

在规则序列中，每完成了一个规则通道转换：

- 转换数据存储在 16 位 ADC_DR 中
- 产生 eoc 标志
- ADC_IER.eocie=1 时产生中断

当规则序列转换完成，除了以上操作外，还会：

- 产生 eos 标志，表示序列转换结束
- ADC_IER.eosie=1 时产生中断

接下来新的序列转换将会立即重新开始。

⚠注意

无法连续转换注入通道。连续模式下唯一的例外情况是，注入通道配置为在规则组通道之后自动转换（ADC_CFGR.jauto 置 1），请参考 32.4.13 ADC注入通道采样的介绍。

连续转换模式下不会自动停止，只能通过配置 ADC_CMDR.adstp_cmd 来终止连续转换。

32.4.16 ADC 非连续转换模式

规则组的非连续转换，可以通过将 ADC_CFGR.discen 置 1 来使能。该模式可用于转换含有 n(n<8)个转换的短序列。该短序列是在 ADC_SQRx（x=1~4）寄存器中选择的转换序列的一部分。可以通过配置 ADC_CFGR.discnum[2:0]来指定 n 的值。通过 ADC_SQRL.len 来设定总序列长度。

每次出现外部触发时，将启动长度为 n 的短序列转换，第一次转换总序列的第 1 组 n 个序列（总序列中编号 0~n-1），第二次转换总序列的第 2 组 n 个序列（总序列中编号 n~2n-1），第三次转换总序列的第 3 组 n 个序列（总序列中编号 2n~3n-1），以此类推，直到序列中的所有转换均完成为止。当总序列最后一次待转换的短序列长度<n 时，只转换剩余的序列。

示例：ADC_CFGR.discen=1，n=3，要转换的通道=1/2/3/6/7/8/9/10：

第 1 次触发：转换序列 1、2、3（每个通道转换都会产生 eoc）。

第 2 次触发：转换序列 6、7、8（每个通道转换都会产生 eoc）。

第 3 次触发：转换序列 9、10（每个通道转换都会产生 eoc），最后生成 eos。

第 4 次触发：转换序列 1、2、3（每个通道转换都会产生 eoc）。

…

转换完所有子组后，下一个触发信号将启动第一个子组的转换。在上述示例中，第 4 次触发重新转换了第 1 个子组中的通道 1、2、3。

⚠注意

当触发模式是软件触发时，在第一次转换序列 1、2、3 之后，控制器自动停止，并将 ADC_CSR.adstart 清零。

注入组的非连续转换，可以通过将 ADC_CFGR.jdiscen 置 1 来使能。在出现外部触发事件后，可逐通道转换在 ADC_JSQR 中选择的序列 jsq*（*=1~4）。通过 ADC_JSQR.jlen[1:0]设定总序列长度。

出现外部触发时，将启动序列中选择的下一个通道转换，直到序列中的所有转换均完成为止。

示例：ADC_CFGR. jdiscen=1，要转换的通道=1、2、3：

第 1 次触发：转换通道 1，并生成 jeoc。

第 2 次触发：转换通道 2，并生成 jeoc。

第 3 次触发：转换通道 3，并生成 jeoc 和 jeos。

第 4 次触发：转换通道 1，并生成 jeoc。

…

⚠注意

1. 开启注入组队列功能后，上述第 4 次触发后跳转到新的 context 内容。
2. 当触发模式是软件触发时，在转换完通道 1 后，控制器自动停止，并将 ADC_CSR.jadstart 清零

当触发模式是软件触发时，实现非连续转换需要通过额外的操作实现：

1. 根据上次转换的通道编号，计算本次要转换的通道编号。
2. 将通道编号配置到转换序列寄存器中，规则组配置 ADC_SQR1.sq1，注入组配置 ADC_JSQR.jsq1。
3. 配置启动转换。

如果要继续非连续转换，需重复上述操作。

32.4.17 ADC 启动转换

软件配置 ADC_CMDR.adstart_cmd 为 1，来启动规则组的转换。当 ADC_CFGR. exten=0x0（软件触发）时，转换立即开始；配置 0x1~0x3 时为硬件触发，需要等待触发信号的有效沿到来再开始转换。

软件配置 ADC_CMDR.jadstart_cmd 为 1，来启动注入组的转换。当 ADC_JSQR.jexten=0x0（软件触发）时，转换立即开始；配置 0x1~0x3 时为硬件出发，需要等待触发信号的有效沿到来再开始转换。

⚠注意

配置 ADC_CMDR 时，每次只能配置单个 bit 为 1，配置后，逻辑会自动清除配置值，对

应的状态通过查询 ADC_CSR 获得。

当注入组开启自动注入时（ADC_CFGR.jauto=1），通过规则组的 ADC_CMDR.adstart_cmd 来控制转换启动，注入组自动跟在规则组后执行，禁止配置 ADC_CMDR.jadstart_cmd。

ADC_CSR 中的 adstart 和 jadstart 提供当前是否有任何 ADC 操作正在进行的信息。当二者都为 0 时，表明 ADC 处于空闲状态，可以重新配置 ADC。

ADC_CSR.adstart 被硬件清零的场景：

1. 软件触发模式下，单次转换完成后，或者非连续转换的第一个短序列转换完成后。
2. 在所有情况下，软件配置 ADC_CMDR.adstp_cmd 为 1 后。

规则组在连续模式下，只能通过配置 ADC_CMDR.adstp_cmd 来清零 adstart。硬件触发模式下，单次转换完成后，逻辑也不会自动清零 ADC_CSR.adstart。

ADC_CSR.jadstart 被硬件清零的场景：

1. 软件触发模式下，在注入序列产生 jeos 或者非连续转换模式下第一个通道转换完成后。
2. 在所有情况下，软件配置 ADC_CMDR.jadstp_cmd 为 1 后。

32.4.18 ADC 停止正在进行中的转换

当软件将 ADC_CMDR.adstp_cmd 置 1 时，任何正在进行的规则组转换都将中止，部分结果将被丢弃（ADC_DR 不会使用当前转换进行更新）。

当软件将 ADC_CMDR.jadstp_cmd 置 1 时，任何正在进行的注入组转换都将中止，部分结果将被丢弃（ADC_JDRx 不会使用当前转换进行更新）。

停止转换将重置正在进行的 ADC 操作。然后可以重新配置 ADC（例如改变通道选择，或者更改触发源，极性等），为新的操作做好准备。

在规则转换进行中，可以停止注入转换，反之亦然。这将允许在规则转换进行中时重新配置注入的转换序列和触发选择，也可以在注入转换进行中重新配置规则组的转换序列和触发选择。

ADC_CSR 中的 adstp 和 jadstp 提供当前中止是否完成的信息，一旦该过程完成，adstp/adstart（规则转换）或 jadstp/jadstart（注入转换）将被硬件清除，直到回读 adstart（或 jadstart）是 0，认为 ADC 完成停止。

32.4.19 ADC 注入组转换队列功能

ADC 控制器支持最多 2 个队列内容用于注入转换，ADC_CFGR.jqdis 位默认值 1（关闭队列功能），如果要使用队列功能，需要将 jqdis 设定为 0。

⚠注意

队列使用时必须配置为硬件触发。

使用注入队列功能包括以下操作：

1. 配置注入触发源，ADC_JSQR.jexten 以及 ADC_JSQR.jextsel。
2. 配置触发转换的通道 ADC_JSQR.jsq1~4，转换序列长度 ADC_JSQR.jlen。

所有这些配置内容会被定义到一个单独的 ADC_JSQR 中，而且这个寄存器会实现一个深度为 2 的缓存队列，从而允许存放两套转换内容。

注入队列功能存在以下特征：

- ADC_JSQR 可以在任意时刻写入。
- 写入的值会被存放到上下文队列中。
- 初始队列为空时，第一个写入的 ADC_JSQR 的配置会立即生效。
- 一旦注入序列被转换完成，当前队列会被移除，下一个内容会存入活动队列，此内容会被应用于下一个注入转换队列。
- 当队列为满时，写入 ADC_JSQR 会发生队列溢出。这个溢出通过置位 jqovf 标志显示出来。溢出发生时，导致溢出的写入操作会被忽略，队列的转换内容不变。置位 ADC_IER.jqovfie 会生成中断。
- 当队列变空时，根据 ADC_CFGR.jqm 的配置，可能出现两个操作：
  1) 当 jqm=0 时，队列在启动 ADC 时为空，在之后的运行过程中将永远不会为空，队列会始终保存最后一次活动的内容。任何后续的启动注入序列都会根据最后一个活动的内容来执行有效启动。
  2) 当 jqm=1 时，在注入序列执行完成或者刷新操作后，可以变成空队列，则此时队列中待转化的内容，所有硬件触发都会关闭。因此，任何注入触发会被忽略直到重新写入 ADC_JSQR。
- 读取 ADC_CUR_JSQR 会返回此刻活动的注入队列内容，当内容为空时，读到 0x0。
- 当配置 ADC_CMDR.jadstp_cmd 停止时，队列会被刷新：
  1) jqm=0，队列保留最后一个活动的内容。
  2) jqm=1，队列变空。

当配置为非连续转换时，第一个触发信号会清除当前的队列内容，最后一个触发会改变队列切换队列内容。以配置两个队列，每个队列都要转换 3 个通道为例，非连续执行过程如

下：

第 1 次触发：队列 1 内容清除，非连续序列 1 的第一个转换执行。

第 2 次触发：非连续序列 1 的第二个转换执行。

第 3 次触发：非连续序列 1 的第三个转换执行。

第 4 次触发：队列 2 内容清除，非连续序列 2 的第一个转换执行。

第 5 次触发：非连续序列 2 的第二个转换执行。

第 6 次触发：非连续序列 2 的第三个转换执行。

使能和关闭队列功能

控制器默认不使用队列功能，即默认 jqdis=1，如果要使用注入队列功能，软件应首先将 jqdis 置零，此时：

1） 如果 cur_jsqr 中存在内容，会加入到注入队列的活动内容中；

2） 如果 cur_jsqr 中没有内容，那么注入队列为空队列。

图 32-2 启动队列功能时的队列缓存示意图

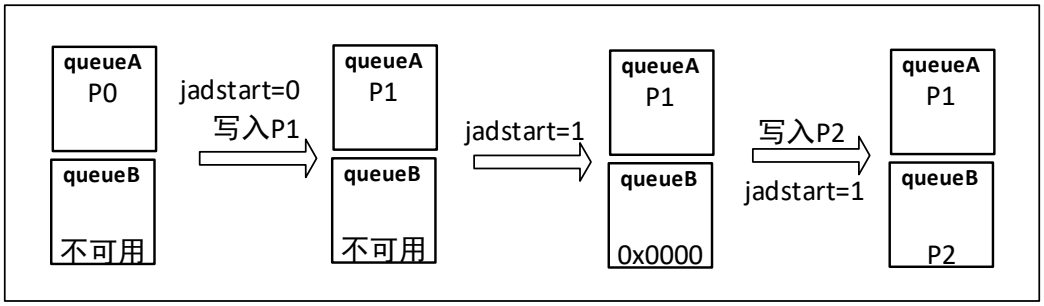
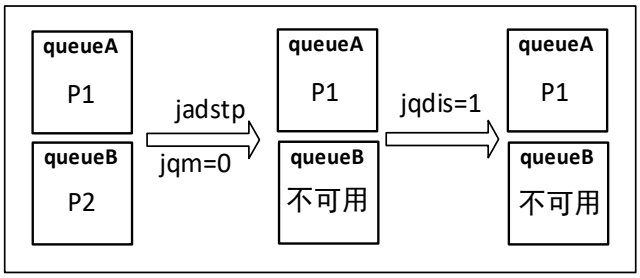


图 32-2 中队列 A 为当前活动的队列内容，在 jqdis=0 并且 ADC_CSR.jadstart=0 时，有且仅有一个内容；当 ADC_CSR.jadstart=1 时，会开启队列 B，初始时 B 一定为空。

当 ADC_CSR.jadstart 从 1 变成 0，即关闭注入转换时，此时按照 jqm 的配置保留 1 个（jqm=0）或 0 个（jqm=1）队列内容。

图 32-3 jqm=0 队列关闭时的缓存示意图



如果要关闭注入组队列功能，软件必须使能 jadstp 来关闭正在进行中的转换，查询等待 ADC_CSR.jadstart=0 之后，才能配置 jqdis=1，从而关闭注入队列功能。图 32-3 中，正在处理 P1 时，使能 jadstp，保留了 P1，未处理的 P2 被删除，然后配置 jqdis=1 关闭注入队列功能。

注入队列内容更新

每次对 ADC_JSQR 进行写操作时，写入的内容会被更新到深度为 2 的队列中，写入的注入组内容可以配置不同的触发源。

图 32-4 注入组队列功能示例（相同触发源）

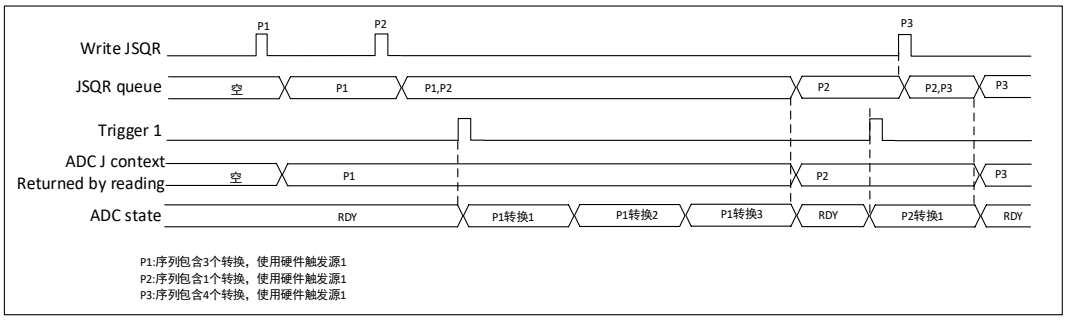
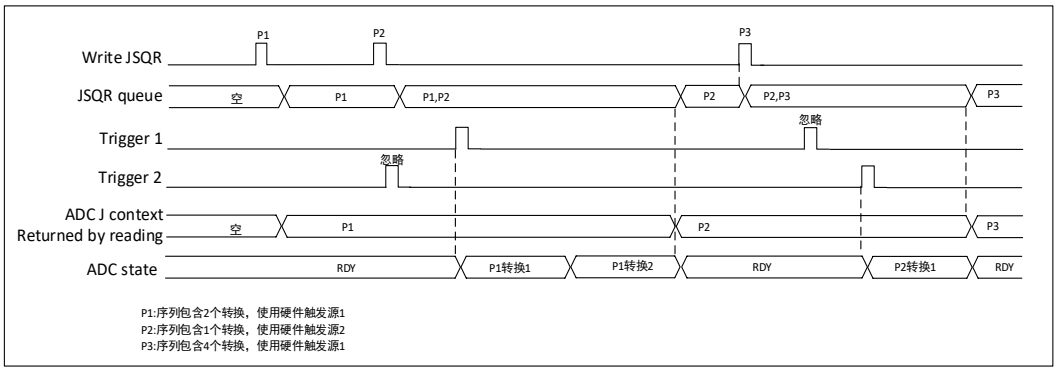


图 32-5 注入组队列功能示例（切换触发源）



如果写 JSQR 时，注入组队列中已经包含 2 个等待的内容，那么此次写入的内容会被忽略，并且控制器产生注入组队列溢出标志 jqovf。通过标记队列溢出，可以在每次写入新内容时，通过 jqovf 标志指示本次写入是否生效。软件在写入新队列内容之前，都要查询 jqovf 标志，如果有此标志，必须清除 jqovf 标志之后再进行新的队列配置操作。

图 32-6 注入组队列功能示例（转换前溢出）

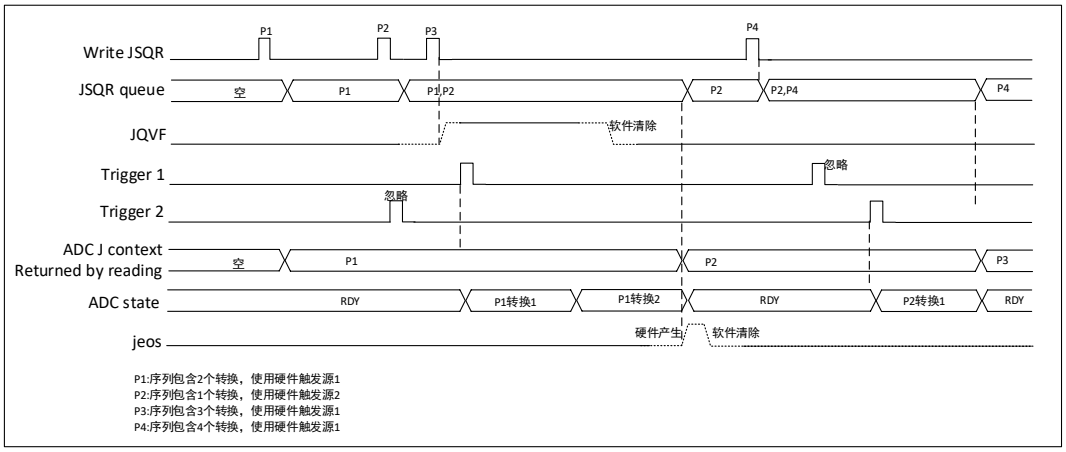
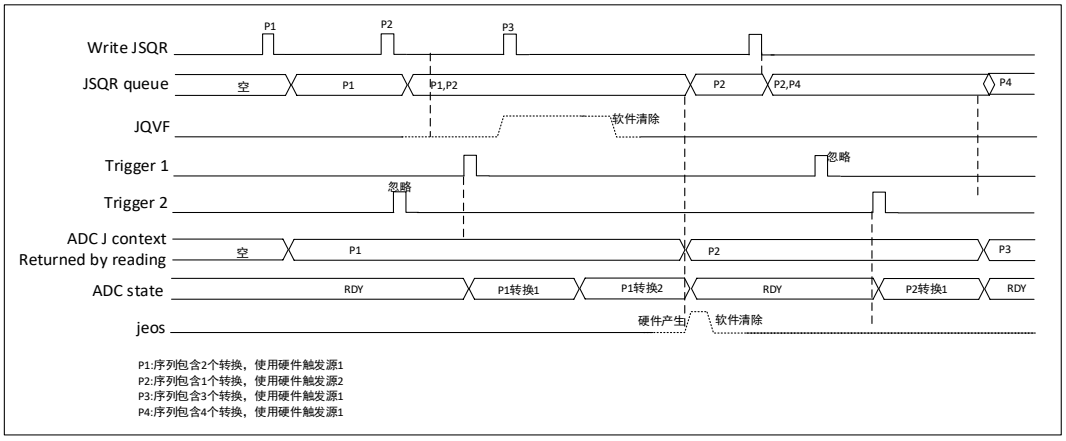


图 32-7 注入组队列功能示例（转换过程中溢出）



注入队列空管理

当注入队列的最后一个内容清除后， 队列会根据 jqm 判断是否进入空状态。当 jqm=0 时，会保存最后一个活动的内容；jqm=1 时，队列会处于空状态。

图 32-8 注入组队列清空示例 1（jqm=0）

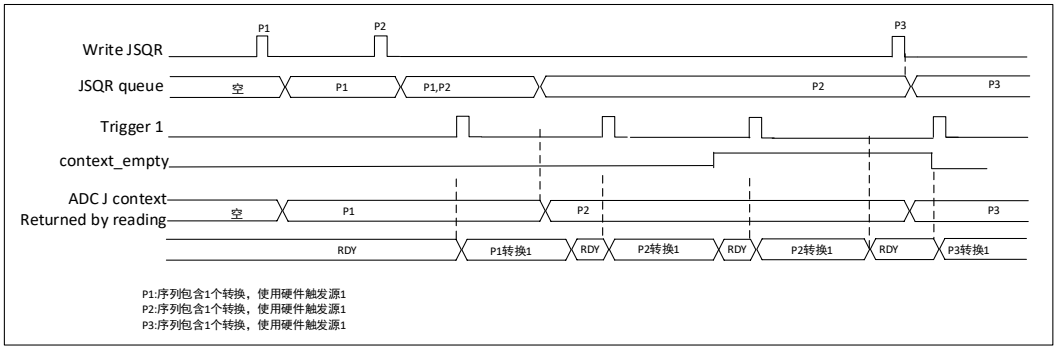
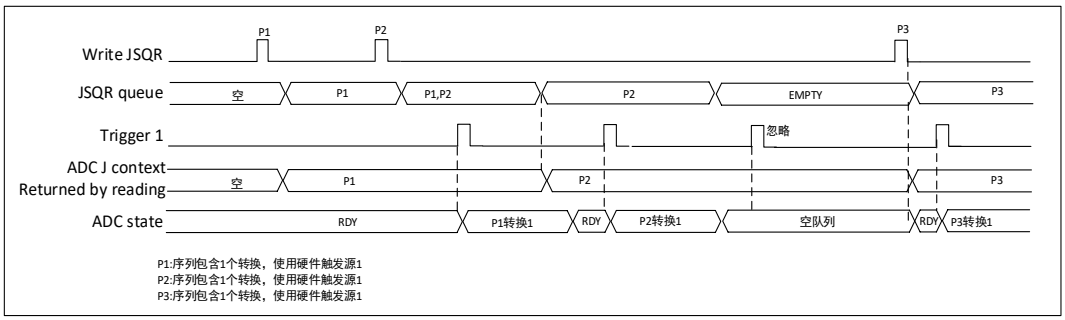


图 32-9 注入组队列清空示例 2（jqm=1）



⚠注意

P3 写入后，队列会立刻被更改。然而因为内部的不同步，当触发正好在写入 P3 的前后，那么 P2 内容的转换可能已经开始，为避免这种情况发生，软件如果要使新的配置立刻生效，就要保证在配置时没有触发信号。

图 32-10 注入组队列清空示例 3（jqm=0）

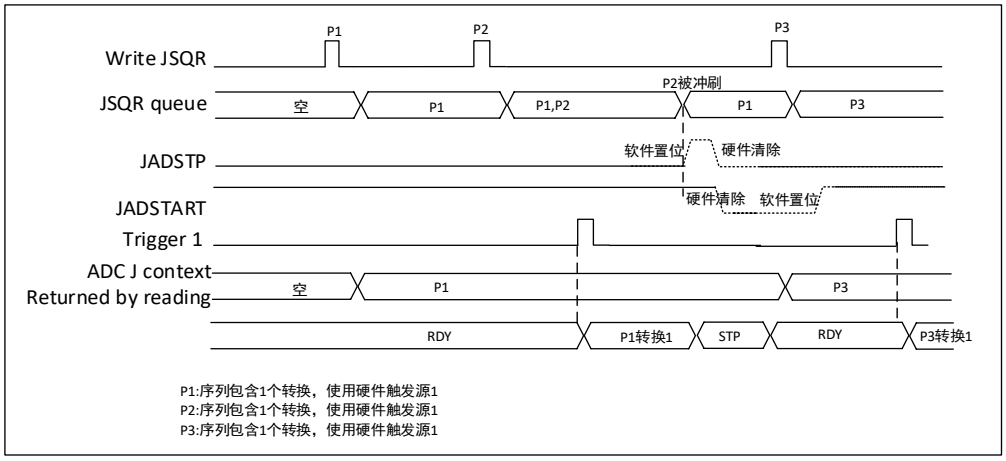
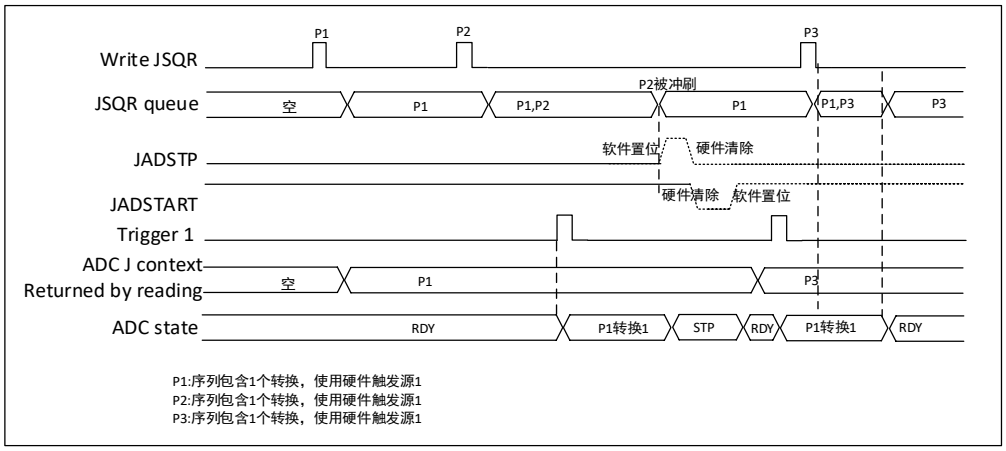


图 32-11 注入组队列清空示例 4（jqm=0）



如图 32-10 和图 32-11 所示，如果写入新队列内容时，此时 P1 没有被触发，那么 P3 成为队列中唯一的内容；如果 P1 已经正在转换，那么 P3 会被存到 buffer 中，直到 P1 转换完成再活动。

注入队列配置流程

注入组的队列功能可以节省打开和关闭注入转换所带来的时间成本，但是使用注入组的寄存器时要特别注意：

1. 在 ADC_CSR.jadstart=0 时，注入队列开启与不开启都只维持一个队列内容，此时重复配置 ADC_JSQR 不会产生溢出。
2. 在开启队列功能后，禁止使用软件触发，jexten=00 的配置被认为是空内容。
3. 在非连续转换模式下，队列只有在 jeos 产生的同时才更新。
4. 软件人员应该避免写入操作和触发时刻接近，这样无法判断当前写入时此刻生效，还是之前的内容再触发一次后生效。

配置流程如下：

步骤 1 查询 ADC_CSR 为 0x1，确定当前没有 adstart/jadstart/adcal 的操作，且 ADC 使能。

步骤 2 清除原始中断，ADC_ISR_CLR 配置为 0x1fff。

步骤 3 查询 ADC_ISR，确定所有原始中断状态已清除。

步骤 4 配置 ADC_IER，打开所需的中断上报使能。

步骤 5 jqdis 设定 0，打开注入组队列功能；注意一旦开启队列功能，注入组软件触发功能将会关闭，一旦内容中存在 jexten=0x0 的对垒，会被认为是空（包括 jqdis 清零前配置的内容）。

步骤 6 根据需求配置 ADC_JSQR，必须配置硬件触发，同时配置 jexten、jextsel，jsq1~jsq4，转换的长度 jlen，其他可选配置信息请参考寄存器表单。

步骤 7 置位 ADC_CMDR.jadstart_cmd。

步骤 8 等待触发源（配置触发源对应的 MCU 器件），查询 jeoc 中断上报，表示硬件完成一次通道转换，jeos 表示单次序列转换完成。

步骤 9 jeos 产生后，根据需要读取 ADC_JDR1、ADC_JDR2、ADC_JDR3 和 ADC_JDR4 获取数据。

步骤 10 清除注入组原始中断。

步骤 11 再次跟进需求配置 ADC_JSQR，配置 jexten、jextsel，jsq1~jsq4，转换的长度 jlen，（注意重新配置时要保证此时没有新的触发信号）。

步骤 12 查询 ADC_ISR 是否有队列内容溢出标志 jqovf 产生，或配置 ADC_IER.qovfie 使能此中断上报 CPU，并读取 ADC_CUR_JSQR，确定此时新的配置已成为活动的内容。

重复等待触发源（配置触发源对应的 MCU 器件），查询 jeoc 中断上报，表示硬件完成一次通道转换，jeos 表示单次序列转换完成。 至步骤 12 可实现注入组的队列功能，直到业务操作完成，接下来关闭注入组。

步骤 13 配置 ADC_CMDR.jadstp_cmd，等待 ADC_CSR 中的 jadstart 和 jadstp 同时拉低，表示注入队列转换停止。

⚠注意

为了避免新的配置时刻不清楚是否有触发信号，可以将 jqm 置 1，这样一旦转换完成，就不会继续留在队列中，转换过程中，没有数据溢出的标志检查，软件要考虑在合理的时间取走注入转换的数据。

32.4.20 ADC 单通道转换结束

ADC 在每个规则转换结束（eoc）和每个注入转换结束（jeoc）时，都会上报事件。

一旦 ADC_DR 中有新的规则组转换数据可用，ADC 就会设置 eoc 标志，并上报到 ADC_ISR 中。如果设置了 ADC_IER.eocie 为 1，就可以产生中断。eoc 标志由软件通过配置 ADC_ISR_CLR.eoc_clr 为 1、非 dual 模式下读取 ADC_DR 或者 dual 模式下 ADC_COM_DR 读取来清除。

一旦在 ADC_JDRx 寄存器之一中有新的注入转换数据可用，ADC 就设置 jeoc 标志，并上

报到 ADC_ISR 中。如果设置了 ADC_IER. jeocie 为 1，就可以产生中断。jeoc 标志由软件通过配置 ADC_ISR_CLR.jeoc_clr 为 1 来清除。

ADC 还通过设置状态位 eosmp 来通知采样阶段的结束，并上报到 ADC_ISR。eosmp 标志由软件通过配置 ADC_ISR_CLR.eosmp_clr 为 1 来清除。如果设置了 ADC_IER.eosmpie 为 1，就可以产生中断。

32.4.21 ADC 转换序列结束

ADC 在规则序列（eos）和注入序列（jeos）的最后一个通道转换完成后，都会上报事件。

一旦 ADC_DR 中有规则序列转换的最后一个数据可用，ADC 就会设置 eos 标志，并上报到 ADC_ISR 中。如果设置了 ADC_IER.eosie 为 1，就可以产生中断。eos 标志由软件配置 ADC_ISR_CLR.eos_clr 为 1 来清除。

一旦注入序列的最后数据完成，ADC 就会设置 jeos 标志，并上报到 ADC_ISR 中。如果设置了 ADC_IER.jeosie 为 1，就可以产生中断。jeos 标志由软件配置 ADC_ISR_CLR.jeos_clr 为 1 来清除。

32.4.22 ADC 数据管理

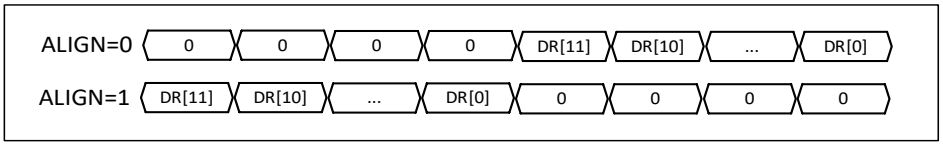
每次规则通道转换（eoc 事件发生后）会将数据存入为 16 位的 ADC_DR 寄存器。

每次注入通道转换（jeoc 事件发生后）会将数据存入 16 位的 ADC_JDRy（y=1~4）数据寄存器。

对齐方式

通过 ADC_CFGR.align 位来配置转换完成后数据的保存格式。数据可以保存为左对齐和右对齐两种方式，如下图 32-12 所示。

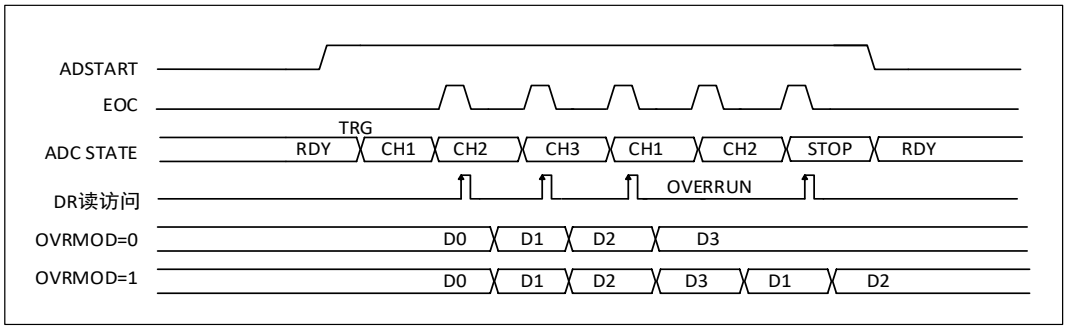
图 32-12 数据对齐方式



规则转换得到的数据将保存在对应的 ADC_DR，如果没有及时取走，当下一次触发并有新的规则转换数据产生时，将会产生 overrun 的情况，此时可以通过配置 ADC_CFGR 选择是否覆盖旧的数据，同时产生 OVR_FLAG 标志位，该标志位可以用于触发中断，并且只能

被软件清除。为了避免产生 overrun 的情况，可以选择通过 DMA 搬走数据或者 autodelay 模式。

图 32-13 规则转换数据溢出



32.4.23 ADC 自动延迟转换功能

当配置 ADC_CFGR.autdly 为 1 时，开启规则组自动延迟转换模式。开启后，只有在 ADC_DR 被软件读取（DMA 搬数不使能）后，或者 DMA 请求与 DMA 响应完成握手（DMA 搬数使能）后才开始新的转换。

这是一种 ADC 自动适应系统读取数据的速度的方式。

32.4.24 ADC DMA 搬数

使用注意事项

一旦发送数据溢出，那么 ADC 控制器会在处理完当前 dma 请求后，停止发送新的 dma 请求，DMA 控制器中搬运的数据除最后一个以外都是可用的。溢出产生说明 DMA 搬运性能不够，软件应立即配置 adstp 来停止规则转换，并优化后续的 DMA 配置。

如果要确保 dma 不溢出，可同时打开 autdly 和 dma 功能，但是 adc 的实际采样率会降低。不允许在 DMA 搬运过程中去访问 ADC_DR。

配置流程

ADC 的 DMA 功能需要与 DMA 控制器配合，具体过程如下：

步骤 1 配置 DMA 控制器，选择 adc 的 dma 通道，配置要搬运的 adc 数据个数（此数值必须大于 adc 实际产生的采样数据个数，否则 DMA 会异常），配置指定搬运 sram 的位置。然后开启 DMA 功能。

步骤 2 查询 ADC_CSR 为 0x1，确定当前没有正在进行的转换，且 ADC 使能。

步骤 3 清除原始中断，ADC_ISR_CLR 配置为 0x1fff。

步骤 4 查询 ADC_ISR，确定所有原始中断状态已清除。

步骤 5 配置 ADC_IER，设定 ovrie 为 1，使能 ovr 中断上报 cpu，其他中断使能根据需求进行设定。

步骤 6 根据需求配置 ADC_CFGR，默认使用的是软件触发，如果需要硬件触发，请配置 exten 和 extsel，使能 dmaen，其他可选配置信息请参考寄存器列表。

步骤 7 配置 ADC_SQRL 设定规则转换序列的长度。

步骤 8 配置规则转换的通道，ADC_SQR1/2/3/4，个数与上一步的长度保持一致，转换从 sq1 开始。

步骤 9 开启 ADC_CMDR.adstart_cmd， 步骤 6 配置为软件触发时，转换立即开始，配置为硬件触发时，等待触发边沿到来才开始转换。

步骤 10 数据采集完成，模块发送 dma 请求给 DMA。

步骤 11 等待 DMA 搬运完成。

步骤 12 配置 ADC_CMDR.adstp_cmd，停止规则转换，等待 adstart 拉低。

步骤 13 停止或复位 DMA，处理 DMA 搬运到指定位置的数据。

DMA 异常处理

- 如果数据溢出产生，那么有两种情况：

  1) dma 性能不够，此时可以配置 ADC_CMDR.adstp_cmd 停止转换，调整 dma 配置，转换的数据除最后一次以外，之前的都可用。

  2) DMA 采样的数据量大于 DMA 控制器配置的搬运数量，比如 ADC 配置为连续模式，而 DMA 搬运数量固定，软件必须禁止使用此种配置。

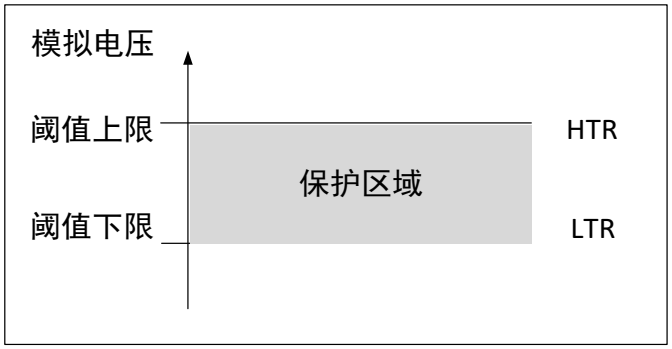
- 规则组的停止操作要等待 DMA 处理完最后一个请求，因此如果配置 adstp_cmd 后，adstp 可能持续为高无法停下来，应该检查 dma 配置。

32.4.25 ADC 模拟看门狗

- **模拟看门狗 1**

通过将 ADC_CFGR 寄存器中的 awd1en/jawd1en 位置 1，可使能 AWD1 模拟看门狗。此功能用于监控一条选定的通道或所有已使能的通道，判断是否处于所配置的电压范围（窗口）内，如下图所示。

图 32-14 模拟看门狗阈值保护



如果 ADC 转换的模拟电压低于阈值下限或高于阈值上限，则 AWD1 模拟看门狗状态位会置 1。这些阈值在 ADC_TR1 寄存器的 ht1[11:0] 和 lt1[11:0] 位中设置。同时可以通过将 ADC_IER 寄存器中的 awd1ie 位置 1 来使能中断。

使能看门狗监测单一或所有通道如下表 32-8 所示：

表 32-8 AWD1 检测通道配置

模拟看门狗保护的通道	awd1sgl	awd1en	jawd1en
无	x	0	0
所有注入通道	0	0	1
所有规则通道	0	1	0
所有规则通道和注入通道	0	1	1
单个注入通道	1	0	1
单个规则通道	1	1	0
单个规则通道或注入通道	1	1	1

AWD1 标志可通过由软件将其对应 clr 位编程为 1 的方式进行清零。

- **模拟看门狗 2、3**

本设计中的第二和第三个看门狗更加灵活，可通过编程看门狗配置寄存器 ADC_AWD2CR 和 ADC_AWD3CR 将相应通道位置 1 来使能每个通道的监测。

当 ADC 转换得到的模拟电压超出上下阈值，那么 AWD 看门狗会送出 AWD_OUTx 高信号，并将对应寄存器状态位置 1，可以通过 ADC_IER 寄存器中的 AWDxIE 位置 1 来使能中断输出，看门狗中断标志位通过软件将其对应 clr 位写 1 的方式清除。

32.4.26 ADC 过采样

单次采样

单次采样的数据需要进行数据对齐，对齐方式见 32.4.22 ADC数据管理 中相关描述。

过采样

过采样后的数据为 16bit，过采样率的范围为 x2~x256，（右）移位范围为 0~8，配置的不合理可能会涉及数据的截取，从而导致数据的不正确。转换的公式为：

$Result=\frac{1}{M}\sum_{n=0}^{N-1}B_n$，B 为 N 次 AD 转换中每次的值，为 12bit 数据，M 的值等于以 2 为底数，移位值（OVSS）为幂的值。

12bit 位宽的数据进行 x2~x256 累加后，有效位宽应为 13~20bit，统一存放在 20bit 的寄存器中做为原始数据，配置右移 bit 数后，实际的有效位数为(13~20)-配置右移位数。

图 32-15 过采样的数据截取

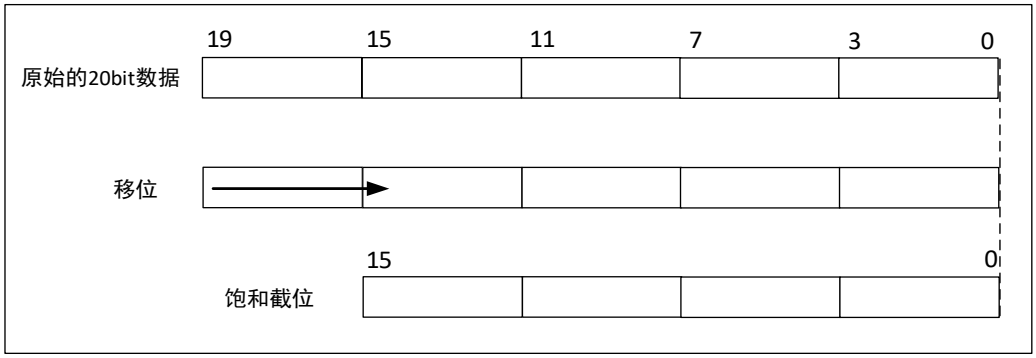
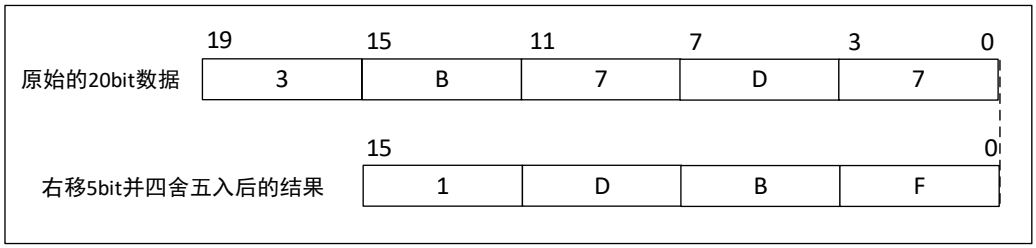
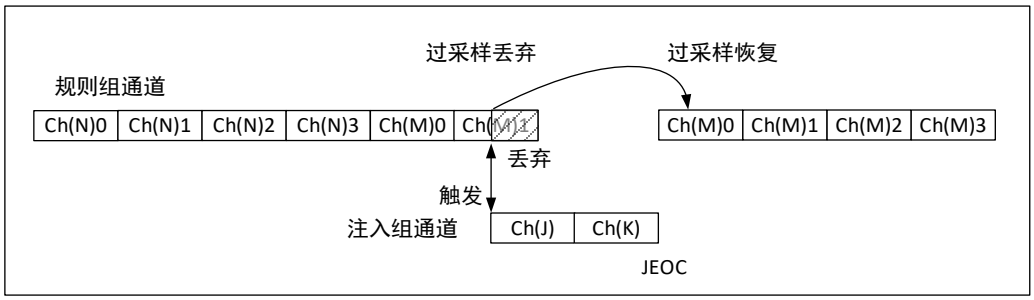


图 32-16 过采样的数据截取示例



只有规则组包含过采样功能，注入组不能够进行过采样（jauto 除外），但是可以在过采样过程中进行触发注入功能，执行方法如下：

图 32-17 过采样时触发注入（丢弃注入前的过采样数据）



为了实现过采样的触发注入功能，从 sar_ctrl 中送出过采样清空的指示信号（切换到注入转换时也会置 1），保证注入转换时 data_proc 中的过采样序列清空，并 sar_ctrl 在进入注入转换时，过采样计数器复位。

32.4.27 温度传感器

温度传感器将温度转化成电压，两者成线性关系，通过测量传感器的输出电压就可以知道当前器件的结温（Tj）。由于工艺变化，出厂前需要对温度传感器进行校准，以保证用户在使用时可以得到精准的温度值。温度传感器的校准值存储在 EFLASH 中，以便用户随时查找与应用。不使用时，温度传感器可以处于掉电模式以节省功耗。

温度传感器的主要特性如下表 32-9 所示：

表 32-9 温度传感器参数

测量温度范围	温度系数	温度精度
-40 ℃ ~125 ℃	3mV/ ℃	±3 ℃（校准后）

温度传感器连接到 ADC0_IN16 通道上，可实现读取温度操作。

温度传感器的配置流程如下：

步骤 1 配置 adc_bgr_ctrl.adc_bgr_tsensor_op_en 和 adc_bgr_ctrl.adc_bgr_ptat_en 为 1，使能 Tsensor 的电压输出。

步骤 2 等待至少 40us，温度传感器从掉电模式切换到唤醒状态。

步骤 3 将 SOC 配置到 ADC0_IN16（与内部参考电压同一个通道）通道后启动 ADC0 转换。

步骤 4 当 ADC0 转换完成后，读取 ADC0_DR 寄存器中产生的数据。

步骤 5 推荐采样 4 次，取平均值作为最终结果 c_n_ptat。

步骤 6 配置 adc_bgr_ctrl.adc_bgr_tsensor_mux 为 3，将芯片内部的 VREF_1P25 电压连接到 ADC0_IN16。

步骤 7 启动 ADC0 转换，当 ADC0 转换完成后，读取 ADC0_DR 寄存器中产生的数据；

步骤 8 推荐采样 4 次，取平均值作为最终结果 c_n_1p25。

步骤 9 使用以下公式计算温度：

$$待测温度T_n = \frac{\frac{c_n_ptat}{c_n_1p25} - \frac{c_rt_ptat}{c_rt_1p25}}{\frac{c_ht_ptat}{c_ht_1p25} - \frac{c_rt_ptat}{c_rt_1p25}}(T_1 - T_0) + T_0 + 10$$

其中：

c_rt_ptat是在温度为T_0时获取的芯片内部 PTAT 电压码字；c_rt_1p25是在温度为T_0时获取的芯片内部 VREF_1P25 电压码字；T_0 是 Tsensor 校准时的环境温度（一般为常温）。

c_ht_ptat是在温度为T_1时获取的芯片内部 PTAT 电压码字；c_ht_1p25是在温度为T_1时获取的芯片内部 VREF_1P25 电压码字；T_1 是 Tsensor 校准时的环境温度（一般为高温）。

⚠注意

该温度传感器为芯片结温传感器，温度转换成的电压和芯片的工作电压无关。在使用 ADC 进行量化转换为温度值时，ADC 的不同参考电压会量化为不同的中间码字，需要使用绝对电压获取方法消除 ADC 的参考电压变化对温度传感器结果的影响。

32.4.28 双 ADC 模式

MCU 支持 2 个 ADC 器件，可使用双 ADC 模式。默认情况下，两个 ADC 工作在独立模式。通过配置为双 ADC 模式实现两个 ADC 的并行转换功能，提高 DMA 传输效率（传输数据宽度增加至 32bit）。

并行模式使用注意事项：

1. 注入组必须关闭。
2. 两个 ADC 的规则组仅支持外部触发，且规则触发源相同。
3. 序列转换时，ADC0 和 ADC1 的（所有待转换通道）转换时间（cvs+smp）与数目必须完全相同。
4. 同一时刻两个 ADC 不能转换同一个输入通道。
5. ADC0 和 ADC1 的 DMA 功能都要关闭。

并行模式不使用 DMA 时的软件配置流程：

步骤 1 配置 ADC0。

步骤 2 配置 ADC1。

步骤 3 配置 ADC_COM_CSR.dual 为 1，使能 dual 模式。

步骤 4 配置 ADC0 和 ADC1 的 adstart_cmd，等待触发源触发。

步骤 5 查询 ADC_COM_ISR 中的 eoc_mst 和 eoc_slv 同时为高，则读取 ADC_COM_DR 中的 32bit 数据（为了避免溢出，也可同时开始 autdly），不允许访问 ADC0 和 ADC1 的 ADC_DR。

步骤 6 读取 ADC_COM_ISR 中的 eos_mst 和 eos_slv 均为高，并行转换完成，可以关闭转换，或者等待下一次触发。

步骤 7 配置 ADC0 和 ADC1 的 ADC_ISR_CLR 为 0x1fff 清除中断，ADC_COM_ISR 随即清除。

并行模式开启 DMA 时的软件配置流程（推荐）：

步骤 1 初始化 DMA 控制器通道，源地址指向 ADC_COM_DR 的地址，目标地址指向存储器，指定搬运的数量以及工作模式。

步骤 2 配置 ADC0 和 ADC1。

步骤 3 配置 ADC_COM_CSR 中的 dual 为 1，使能 dual 模式，dual_dmacfg 选择空闲的 DMA 通路，打开 dual_dmaen。

步骤 4 启动 ADC0 和 ADC1 的 adstart_cmd，等待触发源触发。

步骤 5 读取 ADC_COM_ISR 中的 eos_mst 和 eos_slv 均为高，并行转换完成，可以关闭转换，或者等待下一次触发。

步骤 6 关闭 dma 控制器通道，在指定地址读取采样到的数据。

步骤 7 配置 ADC0 和 ADC1 的 ADC_ISR_CLR 为 0x1fff 清除中断，ADC_COM_ISR 随即清除。

如果发生数据溢出，处理流程同 32.4.24 ADC DMA 搬数 章节中的异常处理流程。

32.4.29 ADC 中断

对每个 ADC 而言，以下情况可能产生中断：

- ADC 就绪标志（adrdy），当 ADC 使能后（位 ADEN=1）以及 ADC 达到准备好接收转换请求的状态
- 当采样阶段结束时（eosmp）
- 在规则组的任何转换结束时（eoc）
- 在规则组序列的转换结束时（eos）
- 在注入组的任何转换结束时（jeoc）

- 在注入组序列的转换结束时（jeos）
- 规则组的数据溢出时（ovr）
- 当模拟看门狗检测发生时（awd1、awd2、awd3）
- 注入队列内容溢出时（jqovf）
- 校准结束时（eocal）
- 转换结束错误时（eocerr），即转换时间不够导致转换数据错误

表 32-10 每个 ADC 的中断列表

中断事件	中断标志	中断使能
ADC 就绪	adrdy	adrdyie
采样阶段结束	eosmp	eosmpie
规则组单通道转换结束	eoc	eocie
规则组序列转换结束	eos	eosie
注入组单通道转换结束	jeoc	jeocie
注入组序列转换结束	jeos	jeosie
规则组数据溢出	ovr	ovrie
模拟看门狗 1 超门限	awd1	awd1ie
模拟看门狗 2 超门限	awd2	awd2ie
模拟看门狗 3 超门限	awd3	awd3ie
注入组队列内容溢出	jqovf	jqovfie
校准结束	eocal	eocalie
转换结束错误	eocerr	eocerrie

32.5 ADC 寄存器概述

ADC 实例 2 个，其基地址如下：

- ADC0 寄存器基地址为：0x65001000
- ADC1 寄存器基地址为：0x65001200
- ADC_DUAL 寄存器基地址为：0x65001600

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	ADC_ISR	0x0	32	RO	0x00000000	ADC 中断状态寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
2	ADC_ISR_CLR	0x4	32	W1P	0x00000000	中断清除寄存器
3	ADC_IER	0x8	32	RW	0x00000000	中断使能寄存器
4	ADC_CSR	0xC	32	RO	0x00000000	ADC 转换状态寄存器
5	ADC_CMDR	0x10	32	W1P	0x00000000	ADC 命令寄存器
6	ADC_CFGR	0x14	32	RW	0x80000000	ADC 配置寄存器
7	ADC_OVSR	0x18	32	RW	0x00000000	ADC 过采样寄存器
8	ADC_SMPR0	0x20	32	RW	0x09090909	采样时间寄存器 1
9	ADC_SMPR1	0x24	32	RW	0x09090909	采样时间寄存器 2
10	ADC_SMPR2	0x28	32	RW	0x09090909	采样时间寄存器 3
11	ADC_SMPR3	0x2C	32	RW	0x09090909	采样时间寄存器 4
12	ADC_SMPR4	0x30	32	RW	0x09090909	采样时间寄存器 5
13	ADC_CVS	0x34	32	RW	0x0000000E	转换时间寄存器
14	ADC_TR1	0x40	32	RW	0x00000000	模拟看门狗 1 阈值寄存器
15	ADC_TR2	0x44	32	RW	0x00000000	模拟看门狗 2 阈值寄存器
16	ADC_TR3	0x48	32	RW	0x00000000	模拟看门狗 3 阈值寄存器
17	ADC_SQRL	0x4C	32	RW	0x00000000	规则序列长度寄存器
18	ADC_SQR1	0x50	32	RW	0x00000000	规则序列寄存器 1
19	ADC_SQR2	0x54	32	RW	0x00000000	规则序列寄存器 2
20	ADC_SQR3	0x58	32	RW	0x00000000	规则序列寄存器 3
21	ADC_SQR4	0x5C	32	RW	0x00000000	规则序列寄存器 4
22	ADC_DR	0x60	32	RO	0x00000000	规则数据寄存器
23	ADC_JSQR	0x64	32	W1P	0x00000000	注入序列配置寄存器
24	ADC_CUR_JSQR	0x68	32	RO	0x00000000	当前注入队列内容寄存器
25	ADC_JDR1	0x70	32	RO	0x00000000	注入通道数据寄存器 1
26	ADC_JDR2	0x74	32	RO	0x00000000	注入通道数据寄存器 2
27	ADC_JDR3	0x78	32	RO	0x00000000	注入通道数据寄存器 3
28	ADC_JDR4	0x7C	32	RO	0x00000000	注入通道数据寄存器 4
29	ADC_AWD2CR	0x80	32	RW	0x00000000	模拟看门狗 2 配置寄存器
30	ADC_AWD3CR	0x84	32	RW	0x00000000	模拟看门狗 3 配置寄存器
31	ADC_CALFACT_AUTO	0x88	32	RO	0x00000000	ADC 自校准补偿系数
32	ADC_CALFACT_SOFT	0x8C	32	RW	0x00000000	ADC 软件配置补偿系数
33	ADC_DIFSEL	0x90	32	RW	0x00000000	差分模式选择寄存器

No.	Names	Offset Address	SIZE	R/W	Default	Description
34	ADC_SEL_CAL_CFG	0x94	32	RW	0x20E40000	ADC 自校正配置寄存器
35	ADC_ANA_CTRL	0x98	32	RW	0x00000004	模拟控制寄存器

DUAL ADC 寄存器列表

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	ADC_COM_ISR	0x0	32	RO	0x00000000	ADC 通用中断状态寄存器
2	ADC_COM_CSR	0x4	32	RW	0x00000000	ADC 通用配置寄存器
3	ADC_COM_DR	0x8	32	RO	0x00000000	通用规则组数据寄存器

32.6 ADC 寄存器描述

32.6.1 ADC_ISR

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			eocerr	eocal	jqovf	awd3	awd2	awd1	jeos	jeoc	ovr	eos	eoc	eosmp	adrdy
-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
eocerr	[12]	RO	转换结束错误标志（End Of Conversiopn error flag)。eoc 发生错误时，该位由硬件置 1。 0：控制器产生 eoc 时模拟数据已准备好 1：控制器产生 eoc 时模拟数据未准备好（强制结束转换） 注意：软件配置的 samp+cvs 时间必须长于 adc 所需的最低采样转换时间，否则会产生此中断标志。	0x0

Name	Bits	R/W	Description	Default
eocal	[11]	RO	校准结束标志 (End Of Calibration flag)。校准完成时，该位由硬件置 1。 0：校准未完成 1：校准已完成	0x0
jqovf	[10]	RO	注入组队列内容溢出（Injected queue over run）。 0：未发生溢出 1：已产生溢出	0x0
awd3	[9]	RO	模拟看门狗 3 标志 (Analog watchdog 3 flag)。当转换电压超过在 ADC_AWD3TR 寄存器中编程的值时，硬件将该位置 1。通过软件编程相应 clr，可将该位清零。 0：未发生模拟看门狗事件（或标志事件已通过软件确认并清零） 1：发生模拟看门狗事件	0x0
awd2	[8]	RO	模拟看门狗 2 标志 (Analog watchdog 2 flag)。当转换电压超过在 ADC_AWD2TR 寄存器中编程的值时，硬件将该位置 1。 0：未发生模拟看门狗事件（或标志事件已通过软件确认并清零） 1：发生模拟看门狗事件	0x0
awd1	[7]	RO	模拟看门狗 1 标志 (Analog watchdog 1 flag)。当转换电压超过在 ADC_TR1 寄存器中编程的值时，硬件将该位置 1。 0：未发生模拟看门狗事件（或标志事件已通过软件确认并清零） 1：发生模拟看门狗事件	0x0
jeos	[6]	RO	注入通道序列转换结束标志 (Injected channel end of sequence flag)。在由 CHSEL 位选择的一系列注入通道转换结束时，会通过硬件将该位置 1。 0：转换序列未完成（或标志事件已通过软件确认并清零） 1：转换序列已完成	0x0
jeoc	[5]	RO	注入通道转换结束标志 (Injected channel end of conversion flag)。当注入通道的每次转换结束，新数据结果出现在寄存器时，会通过硬件将该位置 1。 0：通道转换未完成（或标志事件已通过软件确认并清零） 1：通道转换已完成	0x0

Name	Bits	R/W	Description	Default
ovr	[4]	RO	ADC 溢出（ADC overrun）。该位在规则组发生溢出事件时由硬件置 1，这意味着在 EOC 标志置 1 时，旧转换数据未处理，新转换已完成。 0：未发生溢 1：发生溢出	0x0
eos	[3]	RO	规则通道序列转换结束标志（Regular channel end of sequence flag）。在由 CHSEL 位选择的一系列规则通道转换结束时，会通过硬件将该位置 1。 0：转换序列未完成（或标志事件已通过软件确认并清零） 1：转换序列已完成	0x0
eoc	[2]	RO	规则通道转换结束标志（Regular channel end of conversion flag）。当规则通道的每次转换结束，新数据结果出现在寄存器时，会通过硬件将该位置 1。读取 ADC_DR 寄存器该位清零。 0：通道转换未完成（或标志事件已通过软件确认并清零） 1：通道转换已完成	0x0
eosmp	[1]	RO	采样结束标志（End of sampling flag）。在转换过程中，当采样阶段结束时该位由硬件置 1。 0：采样阶段未结束（或标志事件已通过软件确认并清零） 1：采样阶段已结束	0x0
adrdy	[0]	RO	ADC 就绪(ADC ready)。ADC 使能后（ADEN=1）以及 ADC 达到准备好接收转换请求的状态时，会通过硬件将该位置 1。 0：ADC 未准备好开始转换（或标志事件已通过软件确认并清零） 1：ADC 已准备好开始转换	0x0

32.6.2 ADC_ISR_CLR

- **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			eocerr_clr	eocal_clr	jqovf_clr	awd3_clr	awd2_clr	awd1_clr	jeos_clr	jeoc_clr	ovr_clr	eos_clr	eoc_clr	eosmp_clr	adrdy_clr
-	-	-	WP	WP	WP	WP	WP	WP	WP	WP	WP	WP	WP	WP	WP
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
eocerr_clr	[12]	W1P	eoc 错误标志清除，写 1 清。	0x0
eocal_clr	[11]	W1P	校准结束标志清除，写 1 清。	0x0
jqovf_clr	[10]	W1P	注入队列溢出标志清除，写 1 清。	0x0
awd3_clr	[9]	W1P	模拟看门狗 3 标志清除，写 1 清。	0x0
awd2_clr	[8]	W1P	模拟看门狗 2 标志清除，写 1 清。	0x0
awd1_clr	[7]	W1P	模拟看门狗 1 标志清除，写 1 清。	0x0
jeos_clr	[6]	W1P	注入序列转换完成标志清除，写 1 清。	0x0
jeoc_clr	[5]	W1P	注入通道转换完成标志清除，写 1 清。	0x0
ovr_clr	[4]	W1P	规则数据溢出标志清除，写 1 清。	0x0
eos_clr	[3]	W1P	规则序列转换完成标志清除，写 1 清。	0x0
eoc_clr	[2]	W1P	规则通道转换完成标志清除，写 1 清。	0x0
eosmp_clr	[1]	W1P	通道采样结束标志清除，写 1 清。	0x0
adrdy_clr	[0]	W1P	ad 就绪标志清除，写 1 清。	0x0

32.6.3 ADC_IER

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			eocerrie	eocalie	jqovfie	awd3ie	awd2ie	awd1ie	jeosie	jeocie	ovrie	eosie	eocie	eosmpie	adrdyie
-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:13]	-	保留。	0x0
eocerrie	[12]	RW	eoc 错误中断使能 (End of calibration interrupt enable)。此位由软件置 1 和清零，用于使能/禁止 eoc 错误中断上报。 0：禁止 eoc 错误中断	0x0

Name	Bits	R/W	Description	Default
			1：使能 eoc 错误中断 注意：仅当 ADC_CSR.adstart 且 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作（本寄存器所有位，不满足上述条件的软件配置也会立即生效）。	
eocalie	[11]	RW	校准结束中断上报使能（End of calibration interrupt enable)。 0：禁止校准结束中断 1：使能校准结束中断 注意：仅当 ADC_CSR.adcal 清零时，才允许通过软件对此位执行写操作。	0x0
jqovfie	[10]	RW	注入队列溢出标志上报使能（injected queue overrun flag enable）。 0：禁止注入队列溢出中断 1：使能注入队列溢出中断 注意：仅当 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	0x0
awd3ie	[9]	RW	模拟看门狗 3 中断使能上报 (Analog watchdog 3 interrupt enable)。通过软件将该位置 1 和清零可使能/禁止模拟看门狗中断。 0：禁止模拟看门狗中断 1：使能模拟看门狗中断 注意：仅当 ADC_CSR.adstar 且 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	0x0
awd2ie	[8]	RW	模拟看门狗 2 中断上报使能 (Analog watchdog 2 interrupt enable)。通过软件将该位置 1 和清零可使能/禁止模拟看门狗中断。 0：禁止模拟看门狗中断 1：使能模拟看门狗中断 注意：仅当 ADC_CSR.adstar 且 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	0x0
awd1ie	[7]	RW	模拟看门狗 1 中断上报使能 (Analog watchdog 1 interrupt enable)。通过软件将该位置 1 和清零可使能/禁止模拟看门狗中断。 0：禁止模拟看门狗中断 1：使能模拟看门狗中断 注意：仅当 ADC_CSR.adstar 且 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	0x0
jeosie	[6]	RW	转换序列结束中断上报使能 (End of conversion sequence interrupt enable)。此位由软件置 1 和清零，用于使能/禁止转换序列结束中断。 0：禁止 EOS 中断	0x0

Name	Bits	R/W	Description	Default
			1：使能 EOS 中断，EOS 位置 1 时产生中断 注意：仅当 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	
jeocie	[5]	RW	转换结束中断上报使能 (End of conversion interrupt enable)。通过软件将该位置 1 和清零可使能/禁止转换结束中断。 0：禁止 EOC 中断 1：使能 EOC 中断，EOC 位置 1 时产生中断 注意：仅当 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	0x0
ovrie	[4]	RW	溢出中断上报使能 (Overrun interrupt enable)。通过软件将该位置 1 和清零可使能/禁止溢出中断。 0：禁止溢出中断 1：使能溢出中断，OVR 位置 1 时产生中断 注意：仅当 ADC_CSR.adstart 位清零时，才允许通过软件对此位执行写操作。	0x0
eosie	[3]	RW	转换序列结束中断上报使能 (End of conversion sequence interrupt enable)。此位由软件置 1 和清零，用于使能/禁止转换序列结束中断。 0：禁止 EOS 中断 1：使能 EOS 中断，EOS 位置 1 时产生中断 注意：仅当 ADC_CSR.adstart 位清零时，才允许通过软件对此位执行写操作。	0x0
eocie	[2]	RW	转换结束中断上报使能 (End of conversion interrupt enable)。通过软件将该位置 1 和清零可使能/禁止转换结束中断。 0：禁止 EOC 中断 1：使能 EOC 中断，EOC 位置 1 时产生中断 注意：仅当 ADC_CSR.adstart 位清零时，才允许通过软件对此位执行写操作。	0x0
eosmpie	[1]	RW	采样结束中断上报使能 (End of sampling interrupt enable)。此位由软件置 1 和清零，用于使能/禁止采样阶段结束中断。 0：禁止 EOSMP 中断 1：使能 EOSMP 中断，EOSMP 位置 1 时产生中断 注意：仅当 ADC_CSR.adstar 且 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	0x0
adrdyie	[0]	RW	ADC 就绪中断上报使能 (ADC ready interrupt enable)。此位由软件置 1 和清零，用于使能/禁止 ADC 就绪中断。 0：禁止 ADRDY 中断	0x0

Name	Bits	R/W	Description	Default
			1：使能 ADRDY 中断，ADRDY 位置 1 时产生中断 注意：仅当 ADC_CSR.adstar 且 ADC_CSR.jadstart 位清零时，才允许通过软件对此位执行写操作。	

32.6.4 ADC_CSR

◆ **偏移地址：0xC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
adcal	adcalstp	Res.													
RO	RO	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										jadstp	adstp	jadstart	adstart	addis	aden
-	-	-	-	-	-	-	-	-	-	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
adcal	[31]	RO	ADC 校准 (ADC calibration flag)状态。 0：校准转换没有开启 1：校准转换正在执行	0x0
adcalstp	[30]	RO	停止校准转换(ADC stop calibration conversion flag)。 0：没有停止 ADC 校准转换的命令 1：正在进行停止校准转换命令	0x0
reserved	[29:6]	-	保留。	0x0
jadstp	[5]	RO	停止 injected 转换(ADC stop injected conversion flag)。 0：没有停止 ADC 注入转换的命令 1：正在进行停止注入转换命令	0x0
adstp	[4]	RO	停止 regular 转换(ADC stop regular conversion flag)。 0：没有停止 ADC 规则转换的命令 1：正在进行停止规则转换的命令	0x0
jadstart	[3]	RO	injected 转换(ADC start injected conversion)执行状态。 0：injected 转换没有开启 1：injected 转换处于开启状态	0x0
adstart	[2]	RO	regular 转换(ADC start regular conversion)执行状态。 0：regular 转换没有开启	0x0

Name	Bits	R/W	Description	Default
			1：regular 转换处于开启状态	
addis	[1]	RO	软件关闭 ADC 状态指示，使其进入 power-down 状态。 0：没有关闭 ADC 的命令在执行 1：关闭 ADC 命令正在执行	0x0
aden	[0]	RO	ADC 打开状态(ADC enable flag)。 0：ADC 关闭状态 1：ADC 启动状态	0x0

32.6.5 ADC_CMDR

◆ **偏移地址：0x10　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
adcal_cmd	adcalstp_cmd	Res.													
WP	WP	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

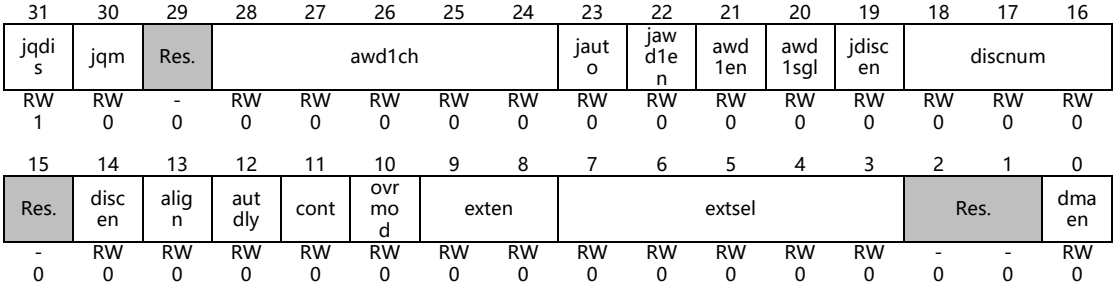
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.										jadstp_cmd	adstp_cmd	jadstart_cmd	adstart_cmd	addis_cmd	aden_cmd
-	-	-	-	-	-	-	-	-	-	WP	WP	WP	WP	WP	WP
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
adcal_cmd	[31]	W1P	启动 adc 校准命令。 注意：仅当 ADC 已启动，并且 ADC_CSR=0x1 时，允许软件修改此位。对于本寄存器，软件一次只允许配置 1 位，不允许配置多位（逻辑保证）。	0x0
adcalstp_cmd	[30]	W1P	停止校准转换命令。 注意：只有当 ADC_CSR.adcal=1 和 ADC_CSR.addis=0 时才允许软件修改，操作方法同上。	0x0
reserved	[29:6]	-	保留。	0x0
jadstp_cmd	[5]	W1P	停止注入转换命令。 注意：只有当 ADC_CSR.jadstart=1 和 ADC_CSR.addis=0 时才允许软件修改，操作方法同上。	0x0
adstp_cmd	[4]	W1P	停止规则转换命令。 注意：只有当 ADC_CSR.adstart=1 和 ADC_CSR.addis=0 时才允许软件修改，操作方法同上。	0x0
jadstart_cmd	[3]	W1P	启动注入转换命令。 注意：只有当 ADC_CSR.jadstart=0 和 ADC_CSR.addis=0 时才允许软件修改，操作方法同上。	0x0

Name	Bits	R/W	Description	Default
adstart_cmd	[2]	W1P	启动规则转换命令。 注意：只有当 ADC_CSR.adstart=0 和 ADC_CSR.addis=0 时才允许软件修改，操作方法同上。	0x0
addis_cmd	[1]	W1P	关闭 ADC 命令。 注意：仅当 ADC 已启动，并且 ADC_CSR=0x1 时，允许软件修改，操作方法如上。	0x0
aden_cmd	[0]	W1P	启动 ADC 命令，并且 ADC_CSR=0x0 时才允许修改，启动成功后，adready 标志位会置位，在该标志位生成前禁止软件进行其它操作。	0x0

32.6.6 ADC_CFGR

◆ **偏移地址：0x14　　复位值：0x80000000**



Name	Bits	R/W	Description	Default
jqdis	[31]	RW	注入队列使能（Injected queue disable）。 0：使能注入队列 1：关闭注入队列 注意：由软件置 1 和清 0，只允许在 ADSTART=0 且 JADSTART=0 时改变此位的值。	0x1
jqm	[30]	RW	JSQR 队列模式时定义如何管理空队列。 0：队列永远不会被清空，并把最后一个配置写入到 cur_jsqr 1：队列会被清空，当最后一个有效的队列过后，软件和硬件触发都会在内部禁用。	0x0
reserved	[29]	-	保留。	0x0
awd1ch	[28:24]	RW	模拟看门狗 1 监测通道选择(Analog watchdog channel selection)(singal 模式下)。 0：监测 ADC 模拟输入通道 0 1：监测 ADC 模拟输入通道 1 … 19：监测 ADC 模拟输入通道 19	0x0

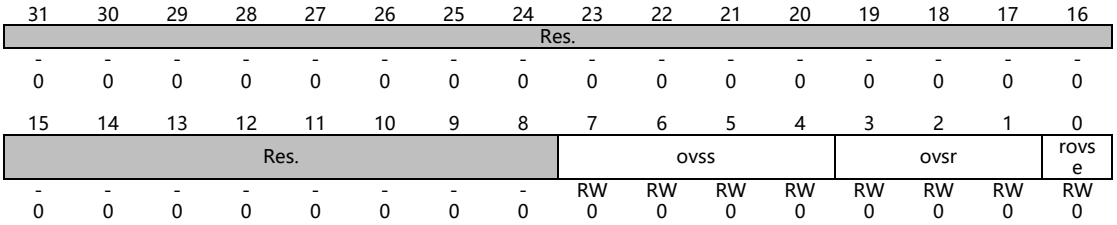
Name	Bits	R/W	Description	Default
			注意：只允许在 ADSTART=0 且 JADSTART=0 时写入。	
jauto	[23]	RW	自动注入转换使能控制（Automatic injected group conversion）。 0：自动注入转换关闭 1：自动注入转换开启 注意：软件置 1 和清 0 只允许在 ADSTART=0 且 JADSTART=0 时写入，置位 jauto 时，逻辑保证 jadstart_cmd 不生效。	0x0
jawd1en	[22]	RW	注入通道的模拟看门狗 1 使能(Analog watchdog enable)。 0：注入通道的模拟看门狗 1 监测关闭 1：注入通道的模拟看门狗 1 监测开启 注意：在 JADSTART=0 时，软件置 1 或清 0	0x0
awd1en	[21]	RW	规则通道的模拟看门狗 1 使能(Enable the watchdog on a single channel or on all channels)。 0：规则通道的模拟看门狗 1 监测关闭 1：规则通道的模拟看门狗 1 监测打开 注意：在 ADSTART=0 时，软件置 1 或清 0	0x0
awd1sgl	[20]	RW	看门狗 1 的单通道（AWD1CH[4:0]选中）或所有通道监测模式使能位。 0：看门狗 1 监测所有通道 1：看门狗 1 监测单通道 注意：当 JADSTART=0 且 ADSTART=0 时，才允许软件置 1 或清 0。	0x0
jdiscen	[19]	RW	注入通道不连续转换使能。 0：注入通道的不连续转换关闭 1：注入通道的不连续转换打开 注意：禁止规则组转换和注入组不连续转换同时开启，当 JADSTART=0 时，才允许软件置 1 或清 0。	0x0
discnum	[18:16]	RW	规则通道单次不连续转换通道数。 0：1 个通道 1：2 个通道 ... 8：8 个通道 注意：当 ADSTART=0 时，才允许软件配置	0x0
reserved	[15]	-	保留。	0x0
discen	[14]	RW	规则通道不连续转换使能。 0：规则通道的不连续转换关闭 1：规则通道的不连续转换打开 注意：当 ADSTART=0 时，才允许软件置 1 或清 0	0x0

Name	Bits	R/W	Description	Default
align	[13]	RW	对齐方式。 0：右对齐 1：左对齐 注意：当 ADSTART=0 且 JADSTART=0 时，才允许软件置 1 或清 0	0x0
autdly	[12]	RW	规则组自动延迟转换模式。 0：延迟转换模式关闭 1：延迟转换模式开启 注意：延迟转换只支持规则通道的单次序列采样和连续采样，当 ADSTART=0 和 JADSTART=0 时，才允许软件置 1 或清 0。	0x0
cont	[11]	RW	单次/连续转换模式 (Single/continuous conversion mode)。 0：单次转换模式 1：连续转换模式 注意：此位由软件置 1 和清零。该位置 1 时，规则组转换将持续进行，直到该位清零	0x0
ovrmod	[10]	RW	溢出管理模式 (Overrun management mode)。该位通过软件进行置 1 和清零，并用于配置数据溢出的管理方式。 0：如果检测到溢出，ADC_DR 寄存器会保留原有数据。 1：如果检测到溢出，ADC_DR 寄存器会被新转换结果覆盖。 注意： 仅当 ADSTART 位清零时，才允许通过软件对此位执行写操作。	0x0
exten	[9:8]	RW	外部触发使能和极性选择 (External trigger enable and polarity selection)。这些位由软件置 1 和清零，用于选择外部触发极性并使能触发。 0：软件触发转换（启动后立即开始转换，并禁用硬件触发） 1：在上升沿执行硬件触发检测 2：在下降沿执行硬件触发检测 3：在上升沿和下降沿都执行硬件触发检测 注意： 仅当 ADSTART 位清零时，才允许通过软件对此位执行写操作。	0x0
extsel	[7:3]	RW	外部触发选择 (Injected External trigger selection)。这些位可选择用于触发转换开始的外部事件。 0：apt0_soc0_a 1：apt0_soc0_b 2：apt0_soc0_a 或 apt0_soc0_b	0x0

Name	Bits	R/W	Description	Default
			3：apt0_soc1_a 4：apt0_soc1_b 5：apt0_soc1_a 或 apt0_soc1_b 6：apt0_soc2_a 7：apt0_soc2_b 8：apt0_soc2_a 或 apt0_soc2_b 9：apt1_soc0_a 10：apt1_soc0_b 11：apt1_soc0_a 或 apt1_soc0_b 12：apt1_soc1_a 13：apt1_soc1_b 14：apt1_soc1_a 或 apt1_soc1_b 15：apt1_soc2_a 16：apt1_soc2_b 17：apt1_soc2_a 或 apt1_soc2_b 18：capm 19：gpt0 20：gpt1 21：gpt2 22：gpt3 其他：禁止配置 注意： 仅当 ADSTART 位清零时，才允许通过软件对此位执行写操作。	
reserved	[2:1]	-	保留。	0x0
dmaen	[0]	RW	DMA 使能控制(Direct memory access enable)。此位由软件置 1 和清零，用于使能 DMA 请求的生成。这样便可使用 DMA 控制器自动管理规则组数据搬运。 0：禁止 DMA 1：使能 DMA 注意： 仅当 ADSTART 位清零时，才允许通过软件对此位执行写操作。	0x0

32.6.7 ADC_OVSR

◆ **偏移地址：0x18　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
ovss	[7:4]	RW	过采样移位 (Oversampling shift)。此位由软件置 1 和清零。 0：不发生移位 1：移 1 位 2：移 2 位 3：移 3 位 4：移 4 位 5：移 5 位 6：移 6 位 7：移 7 位 8：移 8 位 其他值：禁止配置 注意： 仅当 ADSTART=0 时，软件写操作才有效。	0x0
ovsr	[3:1]	RW	过采样率 (Oversampling ratio)。该位域用于定义过采样率的数值。 0：2x 1：4x 2：8x 3：16x 4：32x 5：64x 6：128x 7：256x 注意： 仅当 ADSTART=0 时，软件写操作才有效。	0x0
rovse	[0]	RW	过采样器使能 (Oversampler Enable)。此位由软件置 1 和清零。 0：禁止过采样器 1：使能过采样器 注意： 仅当 ADSTART=0 时，软件写操作才有效。	0x0

32.6.8 ADC_SMPR0

◆ **偏移地址：0x20　　复位值：0x09090909**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
smp3								smp2							
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	1	0	0	1	0	0	0	0	1	0	0	1

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
smp1								smp0							
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	1	0	0	1	0	0	0	0	1	0	0	1

Name	Bits	R/W	Description	Default
smp3	[31:24]	RW	功能请参考 SMP0。	0x9
smp2	[23:16]	RW	功能请参考 SMP0。	0x9
smp1	[15:8]	RW	功能请参考 SMP0。	0x9
smp0	[7:0]	RW	SMPx[7:0]：通道 X 采样时间选择（Channel x sampling time selection)。通过软件写入这些位可分别为各个通道选择采样时间。在采样周期期间，通道选择位必须保持不变。 注意： 0：1 个周期 1：2 个周期 2：3 个周期 3：4 个周期 4：5 个周期 … 255：256 周期 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x9

32.6.9 ADC_SMPR1

◆ **偏移地址：0x24　　复位值：0x09090909**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
smp7								smp6							
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	1	0	0	1	0	0	0	0	1	0	0	1

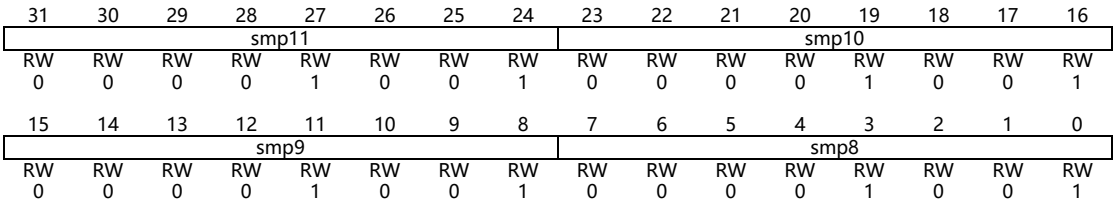
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
smp5								smp4							
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	1	0	0	1	0	0	0	0	1	0	0	1

Name	Bits	R/W	Description	Default
smp7	[31:24]	RW	功能请参考 SMP0。	0x9

Name	Bits	R/W	Description	Default
smp6	[23:16]	RW	功能请参考 SMP0。	0x9
smp5	[15:8]	RW	功能请参考 SMP0。	0x9
smp4	[7:0]	RW	功能请参考 SMP0。	0x9

32.6.10 ADC_SMPR2

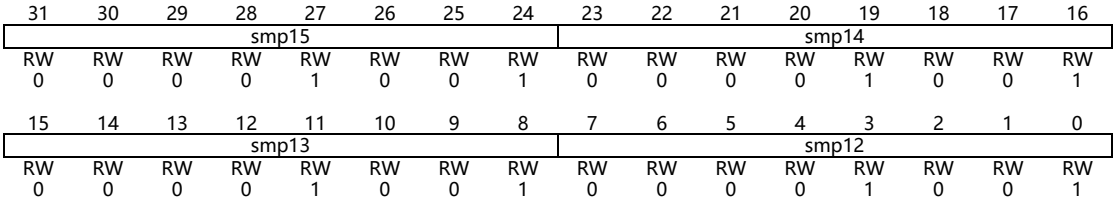
◆ **偏移地址：0x28　　复位值：0x09090909**



Name	Bits	R/W	Description	Default
smp11	[31:24]	RW	功能请参考 SMP0。	0x9
smp10	[23:16]	RW	功能请参考 SMP0。	0x9
smp9	[15:8]	RW	功能请参考 SMP0。	0x9
smp8	[7:0]	RW	功能请参考 SMP0。	0x9

32.6.11 ADC_SMPR3

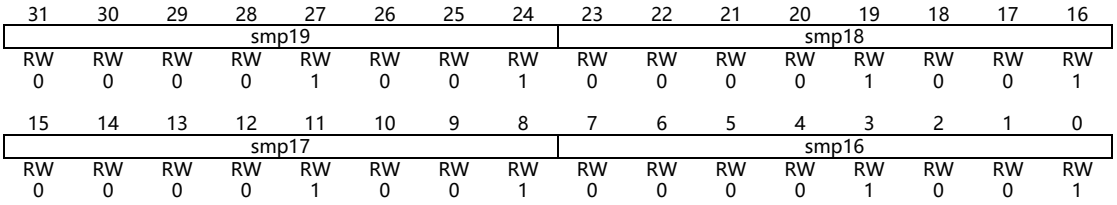
◆ **偏移地址：0x2C　　复位值：0x09090909**



Name	Bits	R/W	Description	Default
smp15	[31:24]	RW	功能请参考 SMP0。	0x9
smp14	[23:16]	RW	功能请参考 SMP0。	0x9
smp13	[15:8]	RW	功能请参考 SMP0。	0x9
smp12	[7:0]	RW	功能请参考 SMP0。	0x9

32.6.12 ADC_SMPR4

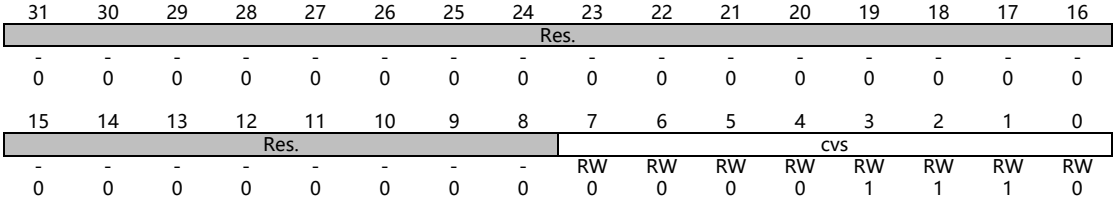
◆ **偏移地址：0x30　　复位值：0x09090909**



Name	Bits	R/W	Description	Default
smp19	[31:24]	RW	功能请参考 SMP0。	0x9
smp18	[23:16]	RW	功能请参考 SMP0。	0x9
smp17	[15:8]	RW	功能请参考 SMP0。	0x9
smp16	[7:0]	RW	功能请参考 SMP0。	0x9

32.6.13 ADC_CVS

◆ **偏移地址：0x34　　复位值：0x0000000E**



Name	Bits	R/W	Description	Default
reserved	[31:8]	-	保留。	0x0
cvs	[7:0]	RW	cvs[7:0]：通道转换时间选择（conversion time register）。通过软件写入给所有通道设置转换时间。 0：1 个周期 1：2 个周期 2：3 个周期 3：4 个周期 4：5 个周期 … 255:256 周期 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0xe

32.6.14 ADC_TR1

◆ **偏移地址：0x40　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.				ht1											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				lt1											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:28]	-	保留。	0x0
ht1	[27:16]	RW	模拟看门狗 1 阈值上限(Analog watchdog 1 higher threshold)。通过软件写入这些位可为模拟看门狗定义阈值上限。 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x0
reserved	[15:12]	-	保留。	0x0
lt1	[11:0]	RW	模拟看门狗 1 阈值下限(Analog watchdog 1 lower threshold)。通过软件写入这些位可为模拟看门狗定义阈值下限。 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x0

32.6.15 ADC_TR2

◆ **偏移地址：0x44　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.				ht2											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				lt2											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:28]	-	保留。	0x0
ht2	[27:16]	RW	模拟看门狗 2 阈值上限(Analog watchdog 2 higher threshold)。通过软件写入这些位可为模拟看门狗定义阈值上限。 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x0
reserved	[15:12]	-	保留。	0x0

Name	Bits	R/W	Description	Default
lt2	[11:0]	RW	模拟看门狗 2 阈值下限(Analog watchdog 2 lower threshold)。通过软件写入这些位可为模拟看门狗定义阈值下限。 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x0

32.6.16 ADC_TR3

◆ **偏移地址：0x48　　复位值：0x00000000**

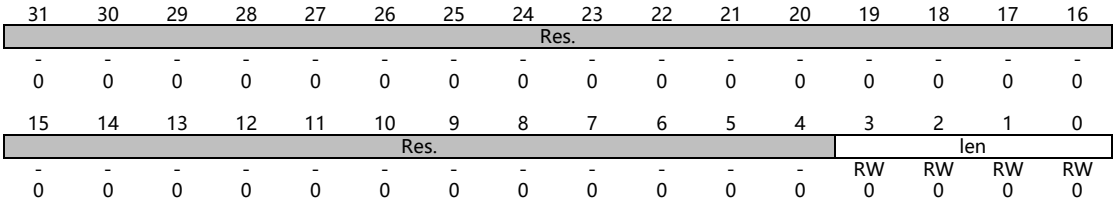
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.				ht3											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				lt3											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:28]	-	保留。	0x0
ht3	[27:16]	RW	模拟看门狗 3 阈值上限(Analog watchdog 3 higher threshold)。通过软件写入这些位可为模拟看门狗定义阈值上限。 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x0
reserved	[15:12]	-	保留。	0x0
lt3	[11:0]	RW	模拟看门狗 3 阈值下限(Analog watchdog 3 lower threshold)。通过软件写入这些位可为模拟看门狗定义阈值下限。 注意：只有 ADSTART=0 且 JADSTART=0 时，软件写操作才有效。	0x0

32.6.17 ADC_SQRL

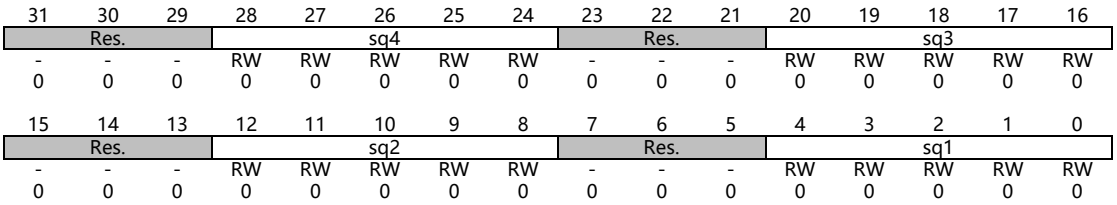
◆ **偏移地址：0x4C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留。	0x0
len	[3:0]	RW	规则通道序列长度（Regular channel sequence length)。通过软件写入这些位可定义规则通道转换序列中的转换总数。 0：1 次转换 1：2 次转换 ...□ 15：16 次转换	0x0

32.6.18 ADC_SQR1

◆ **偏移地址：0x50　　复位值：0x00000000**

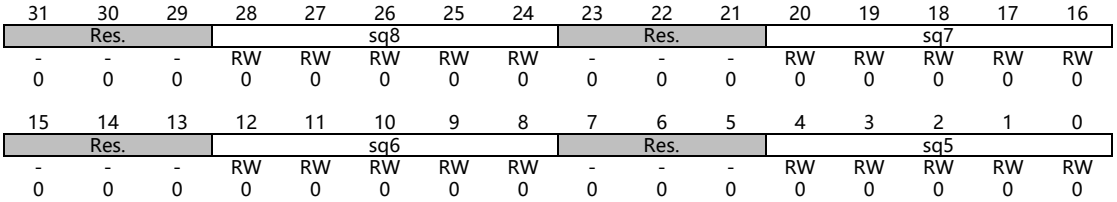


Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
sq4	[28:24]	RW	功能请参考 sq1。	0x0
reserved	[23:21]	-	保留。	0x0
sq3	[20:16]	RW	功能请参考 sq1。	0x0
reserved	[15:13]	-	保留。	0x0
sq2	[12:8]	RW	功能参考 sq1。	0x0
reserved	[7:5]	-	保留。	0x0
sq1	[4:0]	RW	规则序列中的第 x 次转换（1th conversion in regular sequence)。通过软件写入这些位，并将通道编号（0..19）分配为转换序列中的第 1 次转换。 0：转换输入通道 0 1：转换输入通道 1 ...	0x0

Name	Bits	R/W	Description	Default
			19：转换输入通道 19 其它：禁止配置 注意：只有在 ADSTART=0 时，软件写规则序列寄存器才有效。	

32.6.19 ADC_SQR2

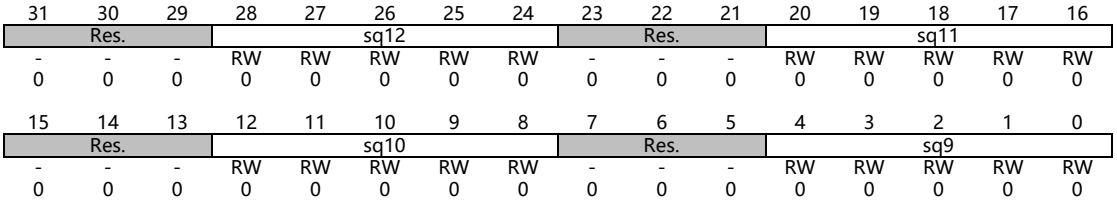
◆ **偏移地址：0x54　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
sq8	[28:24]	RW	功能请参考 sq1。	0x0
reserved	[23:21]	-	保留。	0x0
sq7	[20:16]	RW	功能请参考 sq1。	0x0
reserved	[15:13]	-	保留。	0x0
sq6	[12:8]	RW	功能请参考 sq1。	0x0
reserved	[7:5]	-	保留。	0x0
sq5	[4:0]	RW	功能请参考 sq1。	0x0

32.6.20 ADC_SQR3

◆ **偏移地址：0x58　　复位值：0x00000000**

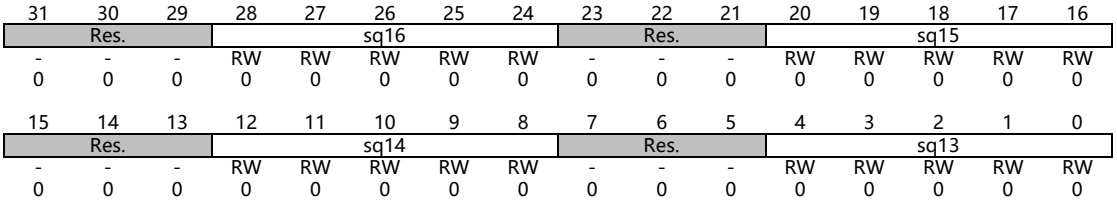


Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
sq12	[28:24]	RW	功能请参考 sq1。	0x0
reserved	[23:21]	-	保留。	0x0
sq11	[20:16]	RW	功能请参考 sq1。	0x0

Name	Bits	R/W	Description	Default
reserved	[15:13]	-	保留。	0x0
sq10	[12:8]	RW	功能请参考 sq1。	0x0
reserved	[7:5]	-	保留。	0x0
sq9	[4:0]	RW	功能请参考 sq1。	0x0

32.6.21 ADC_SQR4

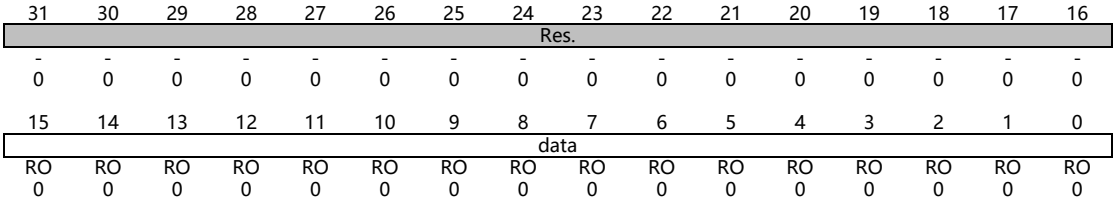
◆ **偏移地址：0x5C　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
sq16	[28:24]	RW	功能请参考 sq1。	0x0
reserved	[23:21]	-	保留。	0x0
sq15	[20:16]	RW	功能请参考 sq1。	0x0
reserved	[15:13]	-	保留。	0x0
sq14	[12:8]	RW	功能请参考 sq1。	0x0
reserved	[7:5]	-	保留。	0x0
sq13	[4:0]	RW	功能请参考 sq1。	0x0

32.6.22 ADC_DR

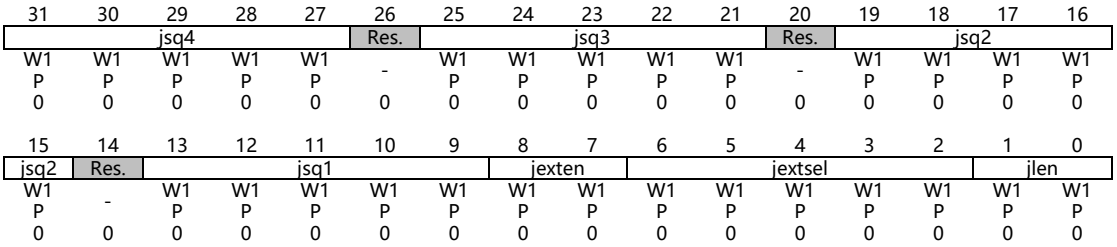
◆ **偏移地址：0x60　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
data	[15:0]	RO	规则数据（Regular data）。这些位为只读。它们包括来自规则通道的转换结果，数据有左对齐和右对齐两种方式，	0x0

32.6.23 ADC_JSQR

◆ **偏移地址：0x64　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
jsq4	[31:27]	W1P	注入序列中，第 4 次转换通道配置。 注意：当 jqdis=1 时，本寄存器中所有域只有在 jadstart=0 时才允许配置（默认）；当 jqdis=0 时，可以在任意时刻配置此寄存器的所有域，但只能配置硬件触发。	0x0
reserved	[26]	-	保留。	0x0
jsq3	[25:21]	W1P	注入序列中，第 3 次转换通道配置 。	0x0
reserved	[20]	-	保留。	0x0
jsq2	[19:15]	W1P	注入序列中的，第 2 次转换通道配置。	0x0
reserved	[14]	-	保留。	0x0
jsq1	[13:9]	W1P	注入序列中的第 1 次转换 (1th conversion in injected sequence)。 0：转换输入通道 0 1：转换输入通道 1 … 19：转换输入通道 19 其它：禁止配置 注意：通过软件写入这些位，并将通道编号 (0..19) 分配为转换序列中的第 1 次转换。	0x0
jexten	[8:7]	W1P	外部触发使能和极性选择 (External trigger enable and polarity selection)。这些位由软件置 1 和清零，用于选择外部触发极性并使能触发。 0：禁止硬件触发检测（jqdis=1 时，软件触发；当 jqdis=0 时，配置此值，会让队列内容无效） 1：在上升沿执行硬件触发检测 2：在下降沿执行硬件触发检测 3：在上升沿和下降沿都执行硬件触发检测	0x0
jextsel	[6:2]	W1P	外部触发选择 (Injected External trigger selection)。这些位可选择用于触发转换开始的外部事件。	0x0

Name	Bits	R/W	Description	Default
			0：apt0_soc0_a 1：apt0_soc0_b 2：apt0_soc0_a 或 apt0_soc0_b 3：apt0_soc1_a 4：apt0_soc1_b 5：apt0_soc1_a 或 apt0_soc1_b 6：apt0_soc2_a 7：apt0_soc2_b 8：apt0_soc2_a 或 apt0_soc2_b 9：apt1_soc0_a 10：apt1_soc0_b 11：apt1_soc0_a 或 apt1_soc0_b 12：apt1_soc1_a 13：apt1_soc1_b 14：apt1_soc1_a 或 apt1_soc1_b 15：apt1_soc2_a 16：apt1_soc2_b 17：apt1_soc2_a 或 apt1_soc2_b 18：capm 19：gpt0 20：gpt1 21：gpt2 22：gpt3 其他：禁止配置	
jlen	[1:0]	W1P	注入序列长度 (Injected sequence length)。通过软件写入这些位可定义注入通道转换序列中的转换总数。 0：1 次转换 1：2 次转换 2：3 次转换 3：4 次转换	0x0

32.6.24 ADC_CUR_JSQR

- **偏移地址：0x68　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
cur_jsq4					Res.	cur_jsq3					Res.	cur_jsq2			
RO	RO	RO	RO	RO	-	RO	RO	RO	RO	RO	-	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

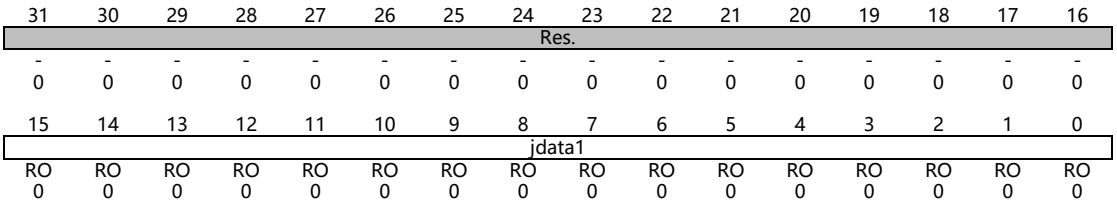
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
cur_jsq2	Res.	cur_jsq1					cur_jexten		cur_jextsel					cur_jlen	
RO	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
cur_jsq4	[31:27]	RO	当前生效的注入序列中，第 4 次转换通道配置。	0x0
reserved	[26]	-	保留。	0x0
cur_jsq3	[25:21]	RO	当前生效的注入序列中，第 3 次转换通道配置 。	0x0
reserved	[20]	-	保留。	0x0
cur_jsq2	[19:15]	RO	当前生效的注入序列中的，第 2 次转换通道配置。	0x0
reserved	[14]	-	保留。	0x0
cur_jsq1	[13:9]	RO	当前生效的注入序列中的第 1 次转换 (1th conversion in injected sequence)。通过软件写入这些位，并将通道编号 (0..19) 分配为转换序列中的第 1 次转换。	0x0
cur_jexten	[8:7]	RO	当前生效的外部触发使能和极性选择配置 (External trigger enable and polarity selection)。这些位由软件置 1 和清零，用于选择外部触发极性并使能触发。 0：禁止硬件触发检测（jqdis=1 时，为软件触发） 1：在上升沿执行硬件触发检测 2：在下降沿执行硬件触发检测 3：在上升沿和下降沿都执行硬件触发检测	0x0
cur_jextsel	[6:2]	RO	外部触发选择 (Injected External trigger selection)。这些位可选择用于触发转换开始的外部事件。 0：apt0_soc0_a 1：apt0_soc0_b 2：apt0_soc0_a 或 apt0_soc0_b 3：apt0_soc1_a 4：apt0_soc1_b 5：apt0_soc1_a 或 apt0_soc1_b 6：apt0_soc2_a 7：apt0_soc2_b 8：apt0_soc2_a 或 apt0_soc2_b 9：apt1_soc0_a 10：apt1_soc0_b 11：apt1_soc0_a 或 apt1_soc0_b	0x0

Name	Bits	R/W	Description	Default
			12：apt1_soc1_a 13：apt1_soc1_b 14：apt1_soc1_a 或 apt1_soc1_b 15：apt1_soc2_a 16：apt1_soc2_b 17：apt1_soc2_a 或 apt1_soc2_b 18：capm 19：gpt0 20：gpt1 21：gpt2 22：gpt3 其他：禁止配置	
cur_jlen	[1:0]	RO	当前生效的注入序列长度 (Injected sequence length) 通过软件写入这些位可定义注入通道转换序列中的转换总数。 0：1 次转换 1：2 次转换 2：3 次转换 3：4 次转换	0x0

32.6.25 ADC_JDR1

◆ **偏移地址：0x70　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
jdata1	[15:0]	RO	注入通道数据寄存器 1 (Injected data register 1)。 这些位为只读，它们包括来自注入通道的第一次转换结果，数据有左对齐和右对齐两种方式。	0x0

32.6.26 ADC_JDR2

◆ **偏移地址：0x74　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
jdata2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
jdata2	[15:0]	RO	注入通道数据寄存器 2 (Injected data register 2)。这些位为只读，它们包括来自注入通道的第二次转换结果，数据有左对齐和右对齐两种方式。	0x0

32.6.27 ADC_JDR3

◆ **偏移地址：0x78　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
jdata3															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
jdata3	[15:0]	RO	注入通道数据寄存器 3 (Injected data register 3)。这些位为只读，它们包括来自注入通道的第三次转换结果，数据有左对齐和右对齐两种方式。	0x0

32.6.28 ADC_JDR4

◆ **偏移地址：0x7C　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
jdata4															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0

Name	Bits	R/W	Description	Default
jdata4	[15:0]	RO	注入通道数据寄存器 4 (Injected data register 4)。这些位为只读，它们包括来自注入通道的第四次转换结果，数据有左对齐和右对齐两种方式。	0x0

32.6.29 ADC_AWD2CR

◆ **偏移地址：0x80　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												awd2chx			
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
awd2chx															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
awd2chx	[19:0]	RW	模拟看门狗通道选择 (Analog watchdog channel selection)。这些位将由软件置 1 和清零。它们用于使能和选择由模拟看门狗 2 (AWD2) 监控的输入通道。 0：不通过 AWD2 监控 ADC 模拟通道 x 1：通过 AWD2 监控 ADC 模拟通道 x 注意： 仅当 JADSTART =0 且 ADSTART=0 时，软件对此位写操作才有效。	0x0

32.6.30 ADC_AWD3CR

◆ **偏移地址：0x84　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.												awd3chx			
-	-	-	-	-	-	-	-	-	-	-	-	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
awd3chx															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:20]	-	保留。	0x0
awd3chx	[19:0]	RW	模拟看门狗通道选择 (Analog watchdog channel selection)。这些位将由软件置 1 和清零。它们用于	0x0

Name	Bits	R/W	Description	Default
			使能和选择由模拟看门狗 3（AWD3）监控的输入通道。 0：不通过 AWD3 监控 ADC 模拟通道 x 1：通过 AWD3 监控 ADC 模拟通道 x 注意： 仅当 JADSTART =0 且 ADSTART=0 时，软件对此位写操作才有效。	

32.6.31 ADC_CALFACT_AUTO

◆ **偏移地址：0x88　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
self_calfact_d															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
self_calfact_s															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
self_calfact_d	[31:16]	RO	差分校准系数（self calibration factors in differential mode）。fix(1,16,7),这些位由逻辑自校准完成时自动写入。	0x0
self_calfact_s	[15:0]	RO	单端校准系数（self calibration factors in single-ended mode）。fix(1,16,7),这些位由逻辑自校正完成时自动写入。	0x0

32.6.32 ADC_CALFACT_SOFT

◆ **偏移地址：0x8C　　复位值：0x00000000**

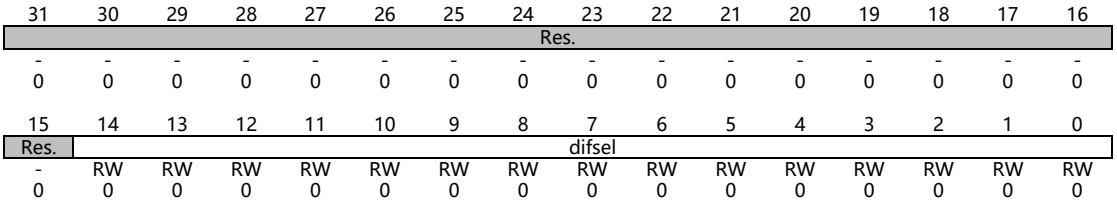
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
soft_calfact_d															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
soft_calfact_s															
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
soft_calfact_d	[31:16]	RW	差分校准系数（soft calibration factors in differential mode）。ix(1,16,7),这些位由软件写入，根据 calfact_sel 决定是否生效。	0x0

Name	Bits	R/W	Description	Default
soft_calfact_s	[15:0]	RW	单端校准系数（soft calibration factors in single-ended mode）。fix(1,16,7),这些位由软件写入，根据 calfact_sel 决定是否生效。	0x0

32.6.33 ADC_DIFSEL

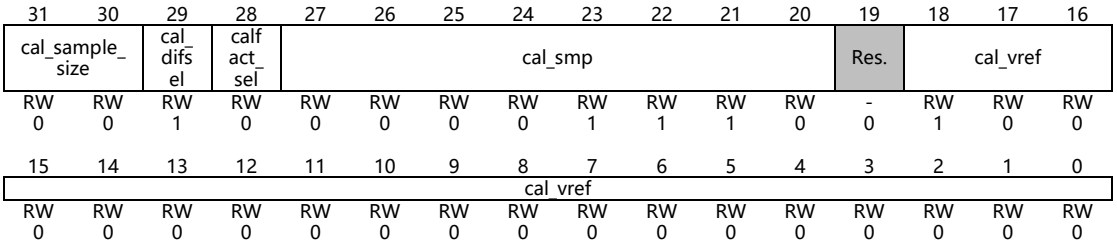
◆ **偏移地址：0x90　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留。	0x0
difsel	[14:0]	RW	模拟输入差分模式选择（differential mode selection register）。 0：单端模式 1：差分模式 注意：14：0 分别对应外部输入的 14 通道~0 通道，默认是单端模式，只有在 ADEN=0 时，才允许配置该寄存器。19~15 通道只能单端模式。	0x0

32.6.34 ADC_SEL_CAL_CFG

◆ **偏移地址：0x94　　复位值：0x20E40000**



Name	Bits	R/W	Description	Default
cal_sample_size	[31:30]	RW	自校正样本量。运行一次自校准，采样的参考电压次数。 0：8 1：16	0x0

Name	Bits	R/W	Description	Default
			2：32 3：64	
cal_difsel	[29]	RW	自校准模式选择。 0：单端模式 1：差分模式	0x1
calfact_sel	[28]	RW	ADC 数字补偿系数来源。 0：来源于自校准的 auto_calfact 1：来源于软件配置的 soft_calfact	0x0
cal_smp	[27:20]	RW	校准采样时间选择（calibration sampling time selection）。通过软件写入这些位控制校准时单次采样时间。在采样周期期间，通道选择位必须保持不变。 注意： 0：1 个周期 1：2 个周期 2：3 个周期 3：4 个周期 4：5 个周期 … 255：256 周期 注意：校准时采样时间由本域配置，转换时间共用 ADC_CVS 值。	0xe
reserved	[19]	-	保留。	0x0
cal_vref	[18:0]	RW	自校准基准电压，格式为 fix(0,19,7)。配置为 Vp 的电压码字（默认来自 vrefbuf0 的 VCM，码值是 2048，单端模式会选择到 vrefbuf1 的输出，根据此电压配置相应的码值）。	0x40000

32.6.35 ADC_ANA_CTRL

◆ **偏移地址：0x98　　复位值：0x00000004**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.								adc_bp_short	adc_bp_inj_pn	adc_inj_pn_sel	adc_out_ck_sel	adc_convert_dly			
-	-	-	-	-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0

Name	Bits	R/W	Description	Default
reserved	[31:8]	RW	保留。	0x0
adc_bp_short	[7]	RW	校准模式输入 VIP/VIN 自动短接控制。 0：自动短接 1：bypass 短接（不短接）	0x0
adc_bp_inj_pn	[6]	RW	电容校准模式是否自动注入 0/1。 0：自动注入 1：bypass 自动注入	0x0
adc_inj_pn_sel	[5]	RW	电容校准模式注入 0/1 是否两个阵列同时注入。 0：只在一侧注入 1：两侧同时注入	0x0
adc_out_ck_sel	[4]	RW	输出寄存器触发时钟选择。 0：选择 EOC_DIG 作为触发时钟 1：选择 EOC_ANA 作为触发时钟	0x0
adc_convert_dly	[3:0]	RW	ADC 异步转换延时控制。 0： delay = 1n 1： delay =1.5n … … 15： delay =8.5n	0x4

32.7 ADC_DUAL 寄存器描述

32.7.1 ADC_COM_ISR

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.			eocerr_slv	eocal_slv	jqovf_slv	awd3_slv	awd2_slv	awd1_slv	jeos_slv	jeoc_slv	ovr_slv	eos_slv	eoc_slv	eosmp_slv	adrdy_slv
-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.			eocerr_mst	eocal_mst	jqovf_mst	awd3_mst	awd2_mst	awd1_mst	jeos_mst	jeoc_mst	ovr_mst	eos_mst	eoc_mst	eosmp_mst	adrdy_mst
-	-	-	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:29]	-	保留。	0x0
eocerr_slv	[28]	RO	此位复制 ADC1_ISR 寄存器中的 eocerr 位。	0x0
eocal_slv	[27]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
jqovf_slv	[26]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
awd3_slv	[25]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0

Name	Bits	R/W	Description	Default
awd2_slv	[24]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
awd1_slv	[23]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
jeos_slv	[22]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
jeoc_slv	[21]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
ovr_slv	[20]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
eos_slv	[19]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
eoc_slv	[18]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
eosmp_slv	[17]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
adrdy_slv	[16]	RO	此位复制 ADC1_ISR 中相应的寄存器位。	0x0
reserved	[15:13]	-	保留。	0x0
eocerr_mst	[12]	RO	此位复制 ADC0_ISR 寄存器中的 eocerr 位。	0x0
eocal_mst	[11]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
jqovf_mst	[10]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
awd3_mst	[9]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
awd2_mst	[8]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
awd1_mst	[7]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
jeos_mst	[6]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
jeoc_mst	[5]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
ovr_mst	[4]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
eos_mst	[3]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
eoc_mst	[2]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
eosmp_mst	[1]	RO	此位复制 ADC0_ISR 中相应的寄存器位。	0x0
adrdy_mst	[0]	RO	此位对应 ADC0_ISR 寄存器中的 ADRDY 位。	0x0

32.7.2 ADC_COM_CSR

◆ **偏移地址：0x4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.							dual_dmacfg	Res.			dual_dmaen	Res.			dual
-	-	-	-	-	-	-	RW	-	-	-	RW	-	-	-	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:9]	-	保留。	0x0
dual_dmacfg	[8]	RW	dual 模式下的 DMA 通道配置。 0：dual 复用 adc0 的 DMA 通道 1：dual 复用 adc1 的 DMA 通道	0x0

Name	Bits	R/W	Description	Default
			注意：只有在两个 ADC 的 adstart=0 时才允许更改。	
reserved	[7:5]	-	保留。	0x0
dual_dmaen	[4]	RW	dual 模式下的 DMA 使能。 0：不使能 1：使能 注意：只有在两个 ADC 的 adstart=0 时才允许更改。	0x0
reserved	[3:1]	-	保留。	0x0
dual	[0]	RW	dual 模式使能。 0：不使能 1：使能 注意：只有在两个 ADC 的 aden=0 时才允许更改。	0x0

32.7.3 ADC_COM_DR

◆ **偏移地址：0x8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
rdata_slv															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
rdata_mst															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
rdata_slv	[31:16]	RO	ADC1 规则数据。这些位为只读，它们包括来自规则通道的转换结果，对齐方式由 ADC1 的配置寄存器中 align 位决定。	0x0
rdata_mst	[15:0]	RO	ADC0 规则数据。这些位为只读，它们包括来自规则通道的转换结果，对齐方式由 ADC0 的配置寄存器中 align 位决定。	0x0

33 可编程增益放大器（PGA）

33.1 PGA 简介

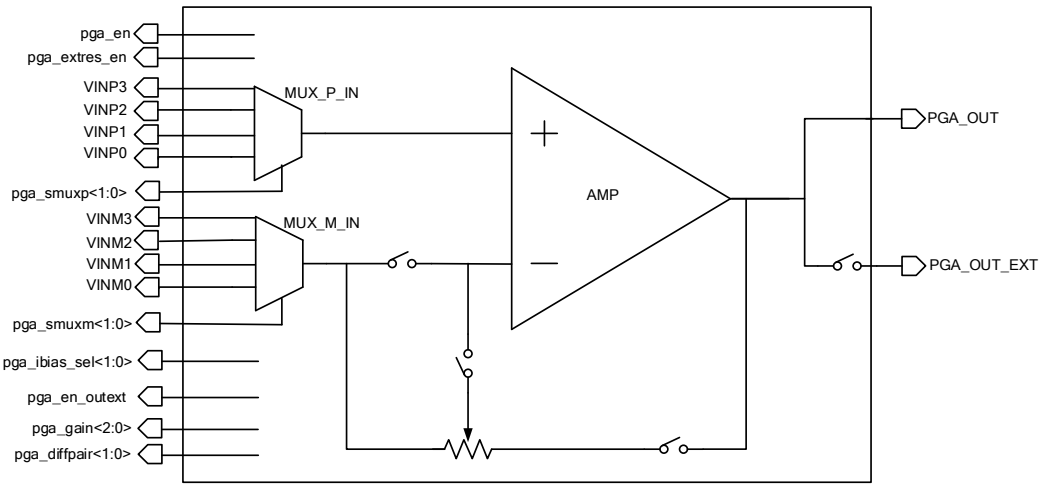
PGA 为增益可调运算放大器。PGA 同时是 ADC 的高性能模拟前端，可选择轨到轨输入输出、全 PMOS 输入级或者全 NMOS 输入级；同时该放大器具有失调校准和输入数据选择功能。PGA 集成多个内部增益选项，用于调整宽范围的输入电压信号，使 ADC 满量程信号达到均一化，提高测量精度。芯片中集成了 4 路独立的 PGA 电路。

33.2 PGA 主要特性

- 三种输入模式：轨到轨输入、全 PMOS 输入、全 NMOS 输入；
- 电源：2.4V~3.6V；
- 内部电阻模式，增益可编程为 1、2、4、8 或 16；
- 外部电阻模式，灵活可调增益；
- 输入失调<1.6mV（校准后）；
- GBW>8MHz；
- Slew Rate>12V/us；
- PSRR@10KHz>50dB；
- CMRR@100Hz>70dB；

33.3 PGA 结构框图

图 33-1 PGA 结构框图



单个 PGA 结构如上图，包括了 P 输入端的输入选择 MUX_P_IN、M 输入端的输入选择 MUX_M_IN、主体的运放、增益控制电阻、内外电阻模式的选择、输出控制开关等。运放支持不同的功耗模式，使用 pga_ibias_sel[1:0]做功耗控制；运放支持不同的输入模式，使用 pga_diffpair[1:0]做输入模式控制。

33.4 PGA 通道选择及输出连接

PGA 正输入端和负输入端分别具有 4 个独立可选的输入通道，通过寄存器 PGA_CTRL.pga_smuxp 及 PGA_CTRL.pga_smuxm 进行配置选择。输出信号有直接输出端 PGA_OUT 和带开关的输出端，开关使用 PGA_CTRL.pga_en_outext 进行配置。

表 33-1 PGA 的输入通道选择

pga(m)_smuxp	P 端选择端口	pga(m)_smuxm	M 端选择端口
0	VINP0	0	VINM0
1	VINP1	1	VINM1
2	VINP2	2	VINM2
3	VINP3	3	VINM3

说明

m 取值范围为 0~3。分别表示 PGA0~PGA3。

表 33-2 各个 PGA 的端口连接

PGA 端口	PGA(0)	PGA(1)	PGA(2)	PGA(3)
VINP0	管脚 PB1	管脚 PB4	管脚 PB9	管脚 PB11
VINP1	管脚 PB6	管脚 PB1	管脚 PB11	管脚 PB9
VINP2	管脚 PA14	管脚 PA12	管脚 PA13	管脚 PA14
VINP3	DAC0_OUT	DAC2_OUT	DAC2_OUT	DAC1_OUT
VINM0	管脚 PB3	管脚 PB6	管脚 PB8	管脚 PB13
VINM1	管脚 PB4	管脚 PB3	管脚 PB13	管脚 PB8
VINM2	管脚 PA15	管脚 PA13	管脚 PA12	管脚 PA15
VINM3	VSSA	VSSA	VSSA	VSSA
PGA_OUT	ADC0_IN17	ADC0_IN18	ADC1_IN16	ADC1_IN17
PGA_OUT_EXT	管脚 PB2	管脚 PB5	管脚 PB7	管脚 PB12

⚠注意

DAC0_OUT~DAC2_OUT 及 ADC0_IN17、ADC0_IN18、ADC1_IN16、ADC1_IN17 为 DAC 内部输出及 ADC 内部输入信号。

PGA 的 P 输入端和 M 输入端仅支持多选一，具体参考 PGA_CTRL.pga_smuxp 和 PGA_CTRL.pga_smuxm。

管脚信号会先经过 GPIO 开关，使用时需要先配置对应管脚的 GPIO 开关为模拟功能。

33.5 PGA 输入摆幅选择

PGA 为了保证最优的性能（速率与输入范围的折中），对输入范围做了优化。使用 PGA_CTRL.pga_diffpair 寄存器进行修改。三种模式差异、使用场景与限制，分别为：

- pga_diffpair=00：全 PMOS 输入级

在输入范围为 0.1~(VDDA-1)时使用，速度最快，满足与 ADC 级联时 ADC 采样时间 50ns，满足 ADC 采样率为 8Msps 使用；

- pga_diffpair=01：全 NMOS 输入级

在输入范围为 1~(VDDA-0.1)时使用，速度较低，满足与 ADC 级联时 ADC 采样时间 100ns，满足 ADC 采样率为 6Msps 使用；

- pga_diffpair=1x：轨到轨输入

在输入范围为 0.1~(VDDA-0.1)时使用，速度较低，满足与 ADC 级联时 ADC 采样时间 100ns，满足 ADC 采样率为 6Msps 使用；

33.6 PGA 增益配置及内外电阻模式选择

PGA 内置电阻增益通过寄存器 PGA_CTRL. pga_gain 进行配置。

表 33-3 PGA 增益选择

pga_gain	增益
0	1
1	2
2	4
3	8
其他值	16

PGA 内置电阻和外置电阻模式选择通过寄存器 PGA_CTRL.pga_extres_en 进行选择，当该信号为 1 时，配置为外置电阻模式；当该信号为 0 时，配置为内置电阻模式。

33.7 PGA 寄存器概述

PGA 的基地址如下：

- PGA 寄存器基地址为：0x61005000，m 取值范围 0~3，表示 PGA0~PGA3

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	PGA0_CTRL	0xA0	32	RW	0x00000000	PGA0 控制寄存器
2	PGA1_CTRL	0xA4	32	RW	0x00000000	PGA1 控制寄存器
3	PGA2_CTRL	0xA8	32	RW	0x00000000	PGA2 控制寄存器
4	PGA3_CTRL	0xAC	32	RW	0x00000000	PGA3 控制寄存器

33.8 PGA 寄存器描述

33.8.1 PGA0_CTRL

◆ **偏移地址：0xA0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pga0_diffpair		pga0_en_outext	pga0_smuxp		pga0_smuxm		pga0_gain			pga0_ibias_sel		pga0_lowp_en		pga0_extres_en	pga0_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
pga0_diffpair	[15:14]	RW	PGA0 的输入对模式选择。 0：PMOS 输入，输入范围支持 0.1~(VDDA-1) 1：NMOS 输入，输入范围支持 1~(VDDA-0.1) 其他：CMOS 输入，输入范围支持 0.1~(VDDA-0.1)	0x0
pga0_en_outext	[13]	RW	PGA0 的输出连接到 PAD 的使能信号，连接到 ADC0_IN6，即 PB2。 0：不使能（默认） 1：使能	0x0
pga0_smuxp	[12:11]	RW	PGA0 的 P 输入开关选择控制信号。 0：通道 0，连接到 ADC0_IN5，即 PB1 1：通道 1，连接到 ADC0_IN10，即 PB6 2：通道 2，连接到 ADC0_IN2，即 PA14 3：通道 3，连接到 DAC0_VOUT	0x0
pga0_smuxm	[10:9]	RW	PGA0 的 M 输入开关选择控制信号。 0：通道 0，连接到 ADC0_IN7，即 PB3 1：通道 1，连接到 ADC0_IN8，即 PB4 2：通道 2，连接到 ADC0_IN3，即 PA15 3：通道 3，连接到 VSSA	0x0
pga0_gain	[8:6]	RW	PGA0 的增益档位控制信号。 0：增益为 1 1：增益为 2 2：增益为 4 3：增益为 8 其他：增益为 16	0x0
pga0_ibias_sel	[5:4]	RW	PGA0 的偏置电流档位选择。 0：5uA（默认） 1: 2.5uA 2: 1.25uA（需偏置电流减到 2.5uA 电流） 3: 1.25uA（需偏置电流减到 2.5uA 电流）	0x0

Name	Bits	R/W	Description	Default
pga0_lowp_en	[3:2]	RW	PGA0 的输出驱动能力选择。 0：最大 1：NMOS 减小 2：PMOS 减小 3：NMOS+PMOS 同时减小	0x0
pga0_extres_en	[1]	RW	PGA0 的外置电阻模式使能控制信号。 0：不使能（默认） 1：使能	0x0
pga0_en	[0]	RW	PGA0 的使能信号。 0：不使能（默认） 1：使能	0x0

33.8.2 PGA1_CTRL

◆ **偏移地址：0xA4　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pga1_diffpair		pga1_en_outext	pga1_smuxp		pga1_smuxm		pga1_gain			pga1_ibias_sel		pga1_lowp_en		pga1_extres_en	pga1_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
pga1_diffpair	[15:14]	RW	PGA1 的输入对模式选择。 0：PMOS 输入，输入范围支持 0.1~(VDDA-1) 1：NMOS 输入，输入范围支持 1~(VDDA-0.1) 其他：CMOS 输入，输入范围支持 0.1~(VDDA-0.1)	0x0
pga1_en_outext	[13]	RW	PGA1 的输出连接到 PAD 的使能信号，连接到 ADC0_IN9，即 PB5。 0：不使能（默认） 1：使能	0x0
pga1_smuxp	[12:11]	RW	PGA1 的 P 输入开关选择控制信号。 0：通道 0，连接到 ADC0_IN8，即 PB4 1：通道 1，连接到 ADC0_IN5，即 PB1 2：通道 2，连接到 ADC0_IN0，即 PA12 3：通道 3，连接到 DAC2_VOUT	0x0
pga1_smuxm	[10:9]	RW	PGA1 的 M 输入开关选择控制信号。	0x0

Name	Bits	R/W	Description	Default
			0：通道 0，连接到 ADC0_IN10，即 PB6 1：通道 1，连接到 ADC0_IN7，即 PB3 2：通道 2，连接到 ADC0_IN1，即 PA13 3：通道 3，连接到 VSSA	
pga1_gain	[8:6]	RW	PGA1 的增益档位控制信号。 0：增益为 1 1：增益为 2 2：增益为 4 3：增益为 8 其他：增益为 16	0x0
pga1_ibias_sel	[5:4]	RW	PGA1 的偏置电流档位选择。 0：5uA（默认） 1：2.5uA 2：1.25uA（需偏置电流减到 2.5uA 电流） 3：1.25uA（需偏置电流减到 2.5uA 电流）	0x0
pga1_lowp_en	[3:2]	RW	PGA1 的输出驱动能力选择。 0：最大 1：NMOS 减小 2：PMOS 减小 3：NMOS+PMOS 同时减小	0x0
pga1_extres_en	[1]	RW	PGA1 的外置电阻模式使能控制信号。 0：不使能（默认） 1：使能	0x0
pga1_en	[0]	RW	PGA1 的使能信号。 0：不使能（默认） 1：使能	0x0

33.8.3 PGA2_CTRL

◆ **偏移地址：0xA8　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pga2_diffpair		pga2_en_outext	pga2_smuxp		pga2_smuxm		pga2_gain			pga2_ibias_sel		pga2_lowp_en		pga2_extres_en	pga2_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
pga2_diffpair	[15:14]	RW	PGA2 的输入对模式选择。 0：PMOS 输入，输入范围支持 0.1~(VDDA-1) 1：NMOS 输入，输入范围支持 1~(VDDA-0.1) 其他：CMOS 输入，输入范围支持 0.1~(VDDA-0.1)	0x0
pga2_en_outext	[13]	RW	PGA2 的输出连接到 PAD 的使能信号，连接到 ADC0_IN11，即 PB7。 0：不使能（默认） 1：使能	0x0
pga2_smuxp	[12:11]	RW	PGA2 的 P 输入开关选择控制信号。 0：通道 0，连接到 ADC0_IN13，即 PB9 1：通道 1，连接到 ADC0_IN15，即 PB11 2：通道 2，连接到 ADC0_IN1，即 PA13 3：通道 3，连接到 DAC2_VOUT	0x0
pga2_smuxm	[10:9]	RW	PGA2 的 M 输入开关选择控制信号。 0：通道 0，连接到 ADC0_IN12，即 PB8 1：通道 1，连接到 ADC1_IN1，即 PB13 2：通道 2，连接到 ADC0_IN0，即 PA12 3：通道 3，连接到 VSSA	0x0
pga2_gain	[8:6]	RW	PGA2 的增益档位控制信号。 0：增益为 1 1：增益为 2 2：增益为 4 3：增益为 8 其他：增益为 16	0x0
pga2_ibias_sel	[5:4]	RW	PGA2 的偏置电流档位选择。 0：5uA（默认） 1：2.5uA 2：1.25uA（需偏置电流减到 2.5uA 电流） 3：1.25uA（需偏置电流减到 2.5uA 电流）	0x0
pga2_lowp_en	[3:2]	RW	PGA2 的输出驱动能力选择。 0：最大 1：NMOS 减小 2：PMOS 减小 3：NMOS+PMOS 同时减小	0x0
pga2_extres_en	[1]	RW	PGA2 的外置电阻模式使能控制信号。 0：不使能（默认） 1：使能	0x0
pga2_en	[0]	RW	PGA2 的使能信号。	0x0

Name	Bits	R/W	Description	Default
			0：不使能（默认） 1：使能	

33.8.4 PGA3_CTRL

◆ **偏移地址：0xAC　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
pga3_diffpair		pga3_en_outext	pga3_smuxp		pga3_smuxm		pga3_gain			pga3_ibias_sel		pga3_lowp_en		pga3_extres_en	pga3_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
pga3_diffpair	[15:14]	RW	PGA3 的输入对模式选择。 0：PMOS 输入，输入范围支持 0.1~(VDDA-1) 1：NMOS 输入，输入范围支持 1~(VDDA-0.1) 其他：CMOS 输入，输入范围支持 0.1~(VDDA-0.1)	0x0
pga3_en_outext	[13]	RW	PGA3 的输出连接到 PAD 的使能信号，连接到 ADC1_IN0，即 PB12。 0：不使能（默认） 1：使能	0x0
pga3_smuxp	[12:11]	RW	PGA3 的 P 输入开关选择控制信号。 0：通道 0，连接到 ADC0_IN15，即 PB11 1：通道 1，连接到 ADC0_IN13，即 PB9 2：通道 2，连接到 ADC0_IN2，即 PA14 3：通道 3，连接到 DAC1_VOUT	0x0
pga3_smuxm	[10:9]	RW	PGA3 的 M 输入开关选择控制信号。 0：通道 0，连接到 ADC1_IN1，即 PB13 1：通道 1，连接到 ADC0_IN12，即 PB8 2：通道 2，连接到 ADC0_IN3，即 PA15 3：通道 3，连接到 VSSA	0x0
pga3_gain	[8:6]	RW	PGA3 的增益档位控制信号。 0：增益为 1 1：增益为 2 2：增益为 4 3：增益为 8	0x0

Name	Bits	R/W	Description	Default
			其他：增益为 16	
pga3_ibias_sel	[5:4]	RW	PGA3 的偏置电流档位选择。 0：5uA（默认） 1：2.5uA 2：1.25uA（需偏置电流减到 2.5uA 电流） 3：1.25uA（需偏置电流减到 2.5uA 电流）	0x0
pga3_lowp_en	[3:2]	RW	PGA3 的输出驱动能力选择。 0：最大 1：NMOS 减小 2：PMOS 减小 3：NMOS+PMOS 同时减小	0x0
pga3_extres_en	[1]	RW	PGA3 的外置电阻模式使能控制信号。 0：不使能（默认） 1：使能	0x0
pga3_en	[0]	RW	PGA3 的使能信号。 0：不使能（默认） 1：使能	0x0

34 模拟比较器（ACMP）

34.1 ACMP 简介

ACMP 为模拟电压比较器，选择两个输入源进行电压比较， ACMP 的比较信号有四个来源，即片外输入、DAC 输出、PGA 输出或是 TSC 输出。芯片集成了 4 个模拟比较器。

34.2 ACMP 主要特性

ACMP 功能特点有：

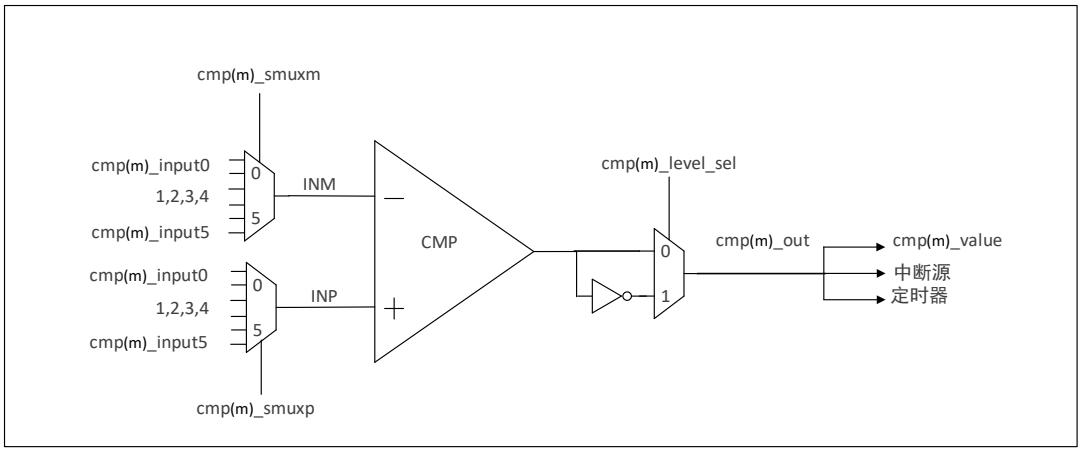
- 支持模拟电源 2.4V~3.6V；数字电源 0.99V~1.21V。
- 支持正、负输入端各 6 个独立输入源可配置选择。
- 支持 4 个迟滞电压档位可选，分别为 0/6/14/20mV。
- 支持轨对轨输入，共模输入电压范围支持 0~VDDA（工作电压）。
- 支持输出结果极性可选。
- 不同功耗与性能模式可选。
- 支持窗口比较器功能。
- 可用于过压、欠压检测。
- 可用于异常状态恢复检测。
- 支持比较器消隐功能。
- 支持对模拟比较器输出进行数据消抖。

34.3 ACMP 功能说明

34.3.1 ACMP 框图

比较器的输入引脚必须配置为模拟输入模式，输出可通过配置 IO 功能复用映射表连接到 IO，输出可在内部进行重定向。文档中的 m 代指比较器序号，取值 0~3。

图 34-1 比较器结构框图



模拟 CMP 的正相和反相端分别拥有 6 个输入，软件通过 cmp(m)_smuxp/cmpx_smuxm 寄存器配置比较器输入源。模拟通过 cmp(m)_level_sel 位判断是否进行取反操作，得到 cmp(m)_out，将此结果存储到寄存器 CMP_OUT 中，软件可以查看。

34.3.2 ACMP 输入源选择

ACMP 的输入源有四个，即片外输入、DAC 输出、PGA 输出或是 TSC 输入。

表 34-1 ACMP 输入源选择

cmp(m)_smuxp	P 端选择端口	cmp(m)_smuxm	M 端选择端口
001	INP0	001	INM0
010	INP1	010	INM1
011	INP2	011	INM2
100	INP3	100	INM3
101	INP4	101	INM4
110	INP5	110	INM5
Other	保留	Other	保留

表 34-2 各个 CMP 的输入端口连接

ACMP 输入端口	ACMP(0)	ACMP(1)	ACMP(2)	ACMP(3)
INP0	管脚 PB6	管脚 PB6	TSC_DET2	管脚 PB12
INP1	TSC_DET0	TSC_DET1	DAC1_OUT	DAC2_OUT
INP2	DAC1_OUT	DAC2_OUT	管脚 PB8	管脚 PA13
INP3	管脚 PB9	管脚 PB13	管脚 PA15	TSC_IO18
INP4	管脚 PB8	管脚 PB12	管脚 PB2	管脚 PB4
INP5	管脚 PB7	管脚 PB11	管脚 PB7	管脚 PB13
INM0	DAC2_OUT	DAC0_OUT	DAC0_OUT	DAC0_OUT
INM1	TSC_VREF	TSC_VREF	TSC_VREF	DAC1_OUT
INM2	DAC0_OUT	管脚 PB0	管脚 PB6	管脚 PB6
INM3	管脚 PB1	管脚 PB1	管脚 PB1	管脚 PB1
INM4	管脚 PB3	管脚 PB3	管脚 PB3	管脚 PB3
INM5	管脚 PB6	管脚 PB6	管脚 PA14	管脚 PB12

说明

ACMP 的 P 输入端与 N 输入端仅支持选择 6 个输入源中的一个，复用说明参考寄存器 CMP(m)_CTRL.cmp(m)_smuxp 及 CMP(m)_CTRL.cmp(m)_smuxm。

34.3.3 ACMP 比较极性选择

比较器可通过寄存器 CMP_CTRL. cmp_level_sel 选择电压比较的输出极性。

表 34-3 输出极性选择

cmp_level_sel	输出极性
0	当 INP>INM 时，比较器结果为 1； 当 INM>INP 时，比较器结果为 0。
1	当 INP>INM 时，比较器结果为 0； 当 INM>INP 时，比较器结果为 1。

34.3.4 ACMP 迟滞电压选择

比较器可通过寄存器 CMP_CTRL.cmp_hyst_sel 选择不同的迟滞电压档位。

表 34-4 ACMP 迟滞电压选择

cmp_hyst_sel	迟滞档位
00	无迟滞
01	6mV
10	14mV
11	20mV

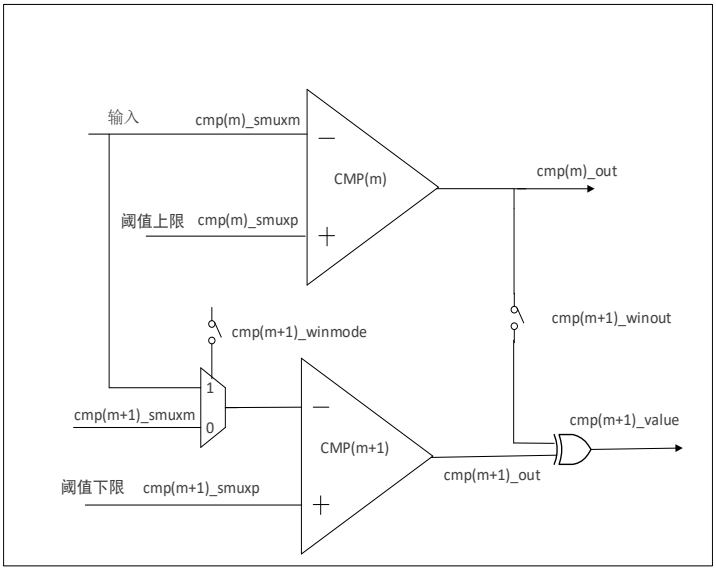
说明

迟滞电压档位为设计的典型值，不同情况下可能会有±5mV 的偏差。迟滞档位越大，抗输入信号的干扰能力越强。

34.3.5 窗口比较器功能

比较器 cmp0/cmp1，cmp2/cmp3，支持配对成为 2 组窗口比较器。开启窗口比较器功能后，输入信号会送给比较器进行处理，cmp(m)_out 与配对的 cmp(m+1)_out 做异或（需要置位 cmp(m+1)_winout），输出 cmp(m+1)_value，具体实现框图如下图 4.2 所示。

图 34-2 窗口比较器功能图



34.3.6 过压欠压检测

使用比较器可以监测电压波动，比如使用上一节窗口比较器配置，cmp0 INP 外接为 3V 电压，cmp1 INP 外接 0.3V 电压。此时一旦产生窗口中断，并进入中断处理函数。那么说明被检测电压 cmp1 INM 已经超过了 3V，或者低于 0.3v，即有欠压或过压产生。

此时，软件通过查询 CMP_OUT 寄存器即可判断到底是欠压还是过压，过程如下：

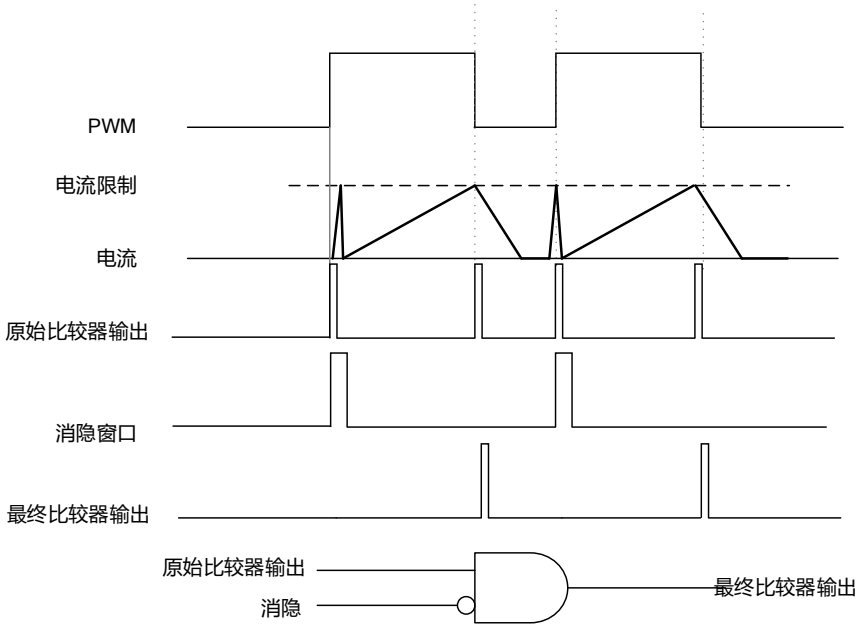
1. cmp0_out = 0；即 cmp0 INP<cmp0 INM，即检测电压大于 3V，处于过压状态；
2. cmp0_out = 1；即 cmp0 INP>cmp0 INM；假设检测电压小于 3V，而且在窗外，那么检测电压也要同时小于 0.3V，处于欠压状态。

需要注意的是，查询寄存器要及时，避免中断到查询期间内比较器输入的 cmp0_out 发生变化。

34.3.7 比较器消隐功能

消隐功能的目的是防止电流调节在 PWM 周期开始处出现短暂电流尖峰（通常为功率开关反向并联二极管中的恢复电流）时发生跳闸。消隐信号来自 APT，通过软件配置寄存器 CMP_DCR.cmp_blking_sel 来选择，总共包含 12 个信号源。选择消隐信号后，将消隐信号的补码与比较器输出执行逻辑“与”运算，以提供所需比较器输出。默认情况，模拟比较器关闭消隐功能，消隐处理过程中，可以同时开启多个消隐源窗口。

图 34-3 消除功能时序图



34.3.8 去抖功能

对于模拟比较结果，ACMP 的数字部分可以对比较结果进行去抖处理，提供 10ns~1280ns 去抖范围选择。通过配置 CMP(m)_DCR.cmp_ft_lev 寄存器选择 2~256 个去抖档位，以 clk_hs（200M）时钟计数，低于配置值宽度的脉冲将被滤除。

如配置 CMP(m)_DCR.cmp_ft_lev 为 10，表示低于 50ns（10*5ns）的脉冲将被滤除。

34.4 ACMP 配置流程

窗口检测功能，以 cmp0 和 cmp1 为例，具体步骤如下：

步骤 1 使能 cmp0 数字配置功能，配置 cmp0_cfg_en =1；

步骤 2 屏蔽 cmp0 中断上报，配置 cmp0_int_mask =1；

步骤 3 关闭 cmp0 的窗口模式，配置 cmp0_winmode =0；

步骤 4 cmp0 输出电平取反，配置 cmp0_level_sel =1；

步骤 5 选择 cmp0 的 m 端输入，例如配置 cmp0_smuxm = 6 (管脚 PB6)；

步骤 6 选择 cmp0 的 p 端输入，例如配置 cmp0_smuxp = 3(DAC1_OUT)；

步骤 7 根据场景需求配置 cmp0 去抖档位 cmp(n)_ft_lev；

步骤 8 关闭 cmp0 结果异或输出，配置 cmp0_winout=0；

步骤 9 使能 cmp1 数字配置功能，配置 cmp1_cfg_en =1；

步骤 10 屏蔽 cmp1 中断上报，配置 cmp1_int_mask =1；

步骤 11 打开 cmp1 的窗口模式，配置 cmp1_winmode=1；

步骤 12 cmp1 输出电平不取反，配置 cmp1_level_sel =0；

步骤 13 选择 cmp1 的 p 端输入，例如配置 cmp1_smuxp = 4 (管脚 PB13)；

步骤 14 根据场景需求配置 cmp1 去抖档位 cmp(n)_ft_lev；

步骤 15 配置 cmp1 结果异或输出，配置 cmp1_winout=1；

步骤 16 在 cpu 侧打开 cmp0 中断，配置为下降沿触发；

步骤 17 在 cpu 侧打开 cmp1 中断，配置为上升沿触发；

步骤 18 使能 cmp0 和 cmp1 的中断上报，配置 cmp0_int_mask =0，
cmp1_int_mask=0；

步骤 19 使能 cmp0 和 cmp1，配置 cmp0_en =1，cmp1_en=1；

34.5 ACMP 寄存器概览

ACMP 的基地址如下：

- ACMP 寄存器基地址为：0x61005000；

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	CMP(m)_CTRL	0x50+0x10*m	32	RW	0x00000000	cmp0~3 模拟控制寄存器（m 取值 0~3）

No.	Names	Offset Address	SIZE	R/W	Default	Description
2	CMP(m)_DCR	0x54+0x10*m	32	RW	0x00000000	cmp0~3 数字控制寄存器（m 取值 0~3）
3	CMP_OUT	0x90	32	RO	0x00000000	比较器结果寄存器

34.6 ACMP 寄存器描述

34.6.1 CMP(m)_CTRL

◆ **偏移地址：0x50+0x10*m　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res	cmp(m)_en_out				cmp(m)_hyst_sel		cmp(m)_level_sel	cmp(m)_ibias_sel	cmp(m)_smuxp			cmp(m)_smuxn			cmp(m)_en
-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:15]	-	保留	0x0
cmp(m)_en_out	[14:11]	RW	CMP0 的高压输出使能控制信号： 0：关闭；1：打开 bit0：连接到 ADC0_IN12，即 PB8； bit1：连接到 ADC0_IN0，即 PA12； bit2：保留位； bit3：保留位； CMP1 的高压输出使能控制信号： 0：关闭；1：打开 bit0：连接到 ADC0_IN15，即 PB11； bit1：连接到 ADC0_IN6，即 PB2； bit2：保留位； bit3：保留位； CMP2 的高压输出使能控制信号： 0：关闭；1：打开 bit0：连接到 ADC0_IN13，即 PB9； bit1：连接到 ADC0_IN3，即 PA15； bit2：连接到 ADC0_IN1，即 PA13； bit3：经 testmux0 连接到 ADC0_IN6，即 PB2；	0x0

Name	Bits	R/W	Description	Default
			CMP3 的高压输出使能控制信号： 0：关闭；1：打开 bit0：连接到 ADC1_IN1，即 PB13； bit1：连接到 ADC0_IN8，即 PB4； bit2：连接到 ADC0_IN2，即 PA14； bit3：连接到 TSC_SHIELD，即 PC7；	
cmp(m)_hyst_sel	[10:9]	RW	cmp(m)的迟滞电压档位选择： 0：无迟滞； 1：6mv； 2：14mv； 3：20mv；	0x0
cmp(m)_level_sel	[8]	RW	cmp(m)的输出电平切换： 0: 当 P>M 时，输出 1；当 P<M 时，输出 0； 1: 当 P>M 时，输出 0；当 P<M 时，输出 1；	0x0
cmp(m)_ibias_sel	[7]	RW	cmp(m)的偏置电流选择： 0：低电流档位； 1：高电流档位；	0x0
cmp(m)_smuxp	[6:4]	RW	CMP0 的 P 输入选择开关： 0：NA； 1：CH0（ADC0_IN10，PB6）； 2：CH1（TSC_DET0）； 3：CH2（DAC1_OUT）； 4：CH3（ADC0_IN13，PB9）； 5：CH4（ADC0_IN12，PB8）； 6：CH5（ADC0_IN11，PB7）； 7：NA； CMP1 的 P 输入选择开关： 0：NA； 1：CH0（ADC0_IN10，PB6）； 2：CH1（TSC_DET1）； 3：CH2（DAC2_OUT）； 4：CH3（ADC1_IN1，PB13）； 5：CH4（ADC1_IN0，PB12）； 6：CH5（ADC0_IN15，PB11）； 7：NA； CMP2 的 P 输入选择开关： 0：NA； 1：CH0（TSC_DET2）； 2：CH1（DAC1_OUT）； 3：CH2（ADC0_IN12，PB8）； 4：CH3（ADC0_IN3，PA15）； 5：CH4（ADC0_IN6，PB2）；	0x0

Name	Bits	R/W	Description	Default
			6：CH5（ADC0_IN11，PB7）； 7：NA； CMP3 的 P 输入选择开关： 0：NA； 1：CH0（ADC1_IN0，PB12）； 2：CH1（DAC2_OUT）； 3：CH2（ADC0_IN1，PA13）； 4：CH3（TSC_IO18，PC8）； 5：CH4（ADC0_IN8，PB4）； 6：CH5（ADC1_IN1，PB13）； 7：NA；	
cmp(m)_smuxm	[3:1]	RW	CMP0 的 M 输入选择开关： 0：NA； 1：CH0（DAC2_OUT）； 2：CH1（TSC_VREF）； 3：CH2（DAC0_OUT）； 4：CH3（ADC0_IN5，PB1）； 5：CH4（ADC0_IN7，PB3）； 6：CH5（ADC0_IN10，PB6）； 7：NA； CMP1 的 M 输入选择开关： 0：NA； 1：CH0（DAC0_OUT）； 2：CH1（TSC_VREF）； 3：CH2（ADC0_IN4，PB0）； 4：CH3（ADC0_IN5，PB1）； 5：CH4（ADC0_IN7，PB3）； 6：CH5（ADC0_IN10，PB6）； 7：NA； CMP2 的 M 输入选择开关： 0：NA； 1：CH0（DAC0_OUT）； 2：CH1（TSC_VREF）； 3：CH2（ADC0_IN10，PB6）； 4：CH3（ADC0_IN5，PB1）； 5：CH4（ADC0_IN7，PB3）； 6：CH5（ADC0_IN2，PA14）； 7：NA； CMP3 的 M 输入选择开关： 0：NA； 1：CH0（DAC0_OUT）； 2：CH1（DAC1_OUT）；	0x0

Name	Bits	R/W	Description	Default
			3：CH2（ADC0_IN10，PB6）； 4：CH3（ADC0_IN5，PB1）； 5：CH4（ADC0_IN7，PB3）； 6：CH5（ADC1_IN0，PB12）； 7：NA；	
cmp(m)_en	[0]	RW	cmp(m)的使能控制信号： 0：不工作（默认）； 1：工作；	0x0

34.6.2 CMP(m)_DCR

◆ **偏移地址：0x54+0x10*m　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
cmp(m)_cfg_en	Res.	cmp(m)_winout	cmp(m)_winmode	Res.			cmp(m)_int_mask	cmp(m)_ft_lev							
RW	-	RW	RW	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.				cmp(m)_blking_sel											
-	-	-	-	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
cmp(m)_cfg_en	[31]	RW	cmp(m)数字配置使能： 0：比较器数字配置未使能； 1：比较器数字配置使能； 注意：必须先配置此位后，才能启动 cmp(m)_en。	0x0
reserved	[30]	-	保留	0x0
cmp(m)_winout	[29]	RW	窗口比较器 cmp_value 输出选择 以 cmp1 为例，用于选择 cmp1_value 输出类型： 0：comp1_out； 1：comp1_out xor comp0_out； 注意：窗口模式只能配置 cmp1 和 cmp3 的 winout；cmp0 和 cmp2 的 winout 不支持配置（保持默认值 0）。	0x0
cmp(m)_winmode	[28]	RW	窗口模式配置（非反向输入选择器） 以 cmp1 为例，用于选择 cmp1 m 端输入信号： 0：窗口模式关闭（选择 cmp1 的 cmp1_smuxm 位域的信号）； 1：窗口模式打开（选择 cmp0 的 cmp0_smuxm 位域的信号）；	0x0

Name	Bits	R/W	Description	Default
			注意：窗口模式只能配置 cmp1 和 cmp3 的 winmode；cmp0 和 cmp2 的 winmode 不支持配置（保持默认值 0）。	
reserved	[27:25]	-	保留	0x0
cmp(m)_int_mask	[24]	RW	cmp(m)中断屏蔽使能： 0：上报比较结果到中断控制模块； 1：屏蔽比较结果到中断控制模块；	0x0
cmp(m)_ft_lev	[23:16]	RW	cmp(m)去抖档位，连续多少次结果比较相同才跳转结果： 0：连续 1 次（不进行滤波）； 1：连续 2 次； … 255: 256 次；	0x0
reserved	[15:12]	-	保留	0x0
cmp(m)_blking_sel	[11:0]	RW	cmp(m)消隐信号源选择： bit0~bit11 分别代表 blanking0~ blanking11 这 12 个消隐信号的启用控制： 0：不启用消隐信号； 1：启用消隐信号；	0x0

34.6.3 CMP_OUT

◆ **偏移地址：0x90　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.												cmp3_out	cmp2_out	cmp1_out	cmp0_out
-	-	-	-	-	-	-	-	-	-	-	-	RO	RO	RO	RO
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:4]	-	保留	0x0
cmp3_out	[3]	RO	极性选择后，比较器 3 的输出结果	0x0
cmp2_out	[2]	RO	极性选择后，比较器 2 的输出结果	0x0
cmp1_out	[1]	RO	极性选择后，比较器 1 的输出结果	0x0
cmp0_out	[0]	RO	极性选择后，比较器 0 的输出结果	0x0

35 其它模拟 IP 寄存器

35.1 寄存器概览

其它模拟 IP 控制器寄存器基地如下：

- 其它模拟 IP 寄存器基地为：0x61005000

No.	Names	Offset Address	SIZE	R/W	Default	Description
1	VREFBUF_CTRL	0x0	32	RW	0x00000000	VREFBUF 控制寄存器
2	IBIAS_SEL_2	0xB8	32	RW	0x00000000	IBIAS 电流选择寄存器 2
3	ADC_BGR_CTRL	0xC4	32	RW	0x00000000	ADC BGR 控制寄存器

35.2 寄存器描述

35.2.1 VREFBUF_CTRL

◆ **偏移地址：0x0　　复位值：0x00000000**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.															
-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

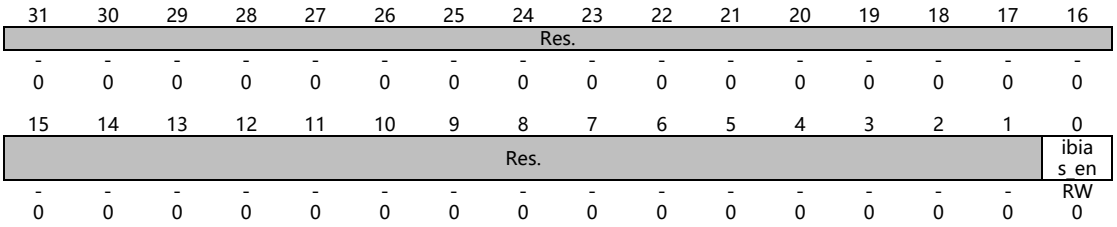
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
vrefbuf_ib_sel		vrefbuf_drv_sel		vrefbuf1_bypass_mode	vrefbuf1_vref_sel		vrefbuf1_vout_sel		vrefbuf1_en	vrefbuf0_bypass_mode	vrefbuf0_vref_sel		vrefbuf0_vout_sel		vrefbuf0_en
RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW	RW
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留。	0x0
vrefbuf_ib_sel	[15:14]	RW	ADC 的偏置电流大小选择: 0：5uA 偏置电流，1:1 镜像； 1：5uA 偏置电流，2:1 镜像； 2：2.5uA 偏置电流，1:1 镜像； 3：2.5uA 偏置电流，2:1 镜像；	0x0
vrefbuf_drv_sel	[13:12]	RW	ADC 的 vrefbuf 输出的驱动能力选择控制信号： 0：最强驱动； 1/2：次强驱动； 3：最弱驱动；	0x0
vrefbuf1_bypass_mode	[11]	RW	ADC1 vrefbuf 的 bypass 模式使能控制信号： 0：vrefbuf 模式； 1：bypass 模式，vrefbuf=AVDD(EN 需同时为 0)；	0x0
vrefbuf1_vref_sel	[10:9]	RW	ADC1 vrefbuf 参考电压选择控制： 0：1.25V 档位； 1：0.2V 档位； 2：选择 AVDD/2； 3：外灌参考电压，来源于 PB12；	0x0
vrefbuf1_vout_sel	[8:7]	RW	ADC1 vrefbuf 输出控制： 0：X2 档位； 1：X1P76 档位； 2：X1 档位； 3：X1 档位；	0x0
vrefbuf1_en	[6]	RW	buffer 使能： 1：使能； 0：禁止；	0x0
vrefbuf0_bypass_mode	[5]	RW	ADC0 的 vrefbuf 的 bypass 模式使能控制信号： 0：vrefbuf 模式； 1：bypass 模式，vrefbuf=AVDD(EN 需同时为 0)；	0x0
vrefbuf0_vref_sel	[4:3]	RW	ADC0 vrefbuf 参考电压选择控制： 0：1.25V 档位； 1：0.2V 档位； 2：选择 AVDD/2； 3：外灌参考电压，来源于 PA2；	0x0
vrefbuf0_vout_sel	[2:1]	RW	ADC0 vrefbuf 输出控制： 0：X2 档位； 1：X1P76 档位； 2：X1 档位； 3：X1 档位；	0x0
vrefbuf0_en	[0]	RW	buffer 使能： 1：使能；	0x0

Name	Bits	R/W	Description	Default
			0：禁止；	

35.2.2 IBIAS_SEL_2

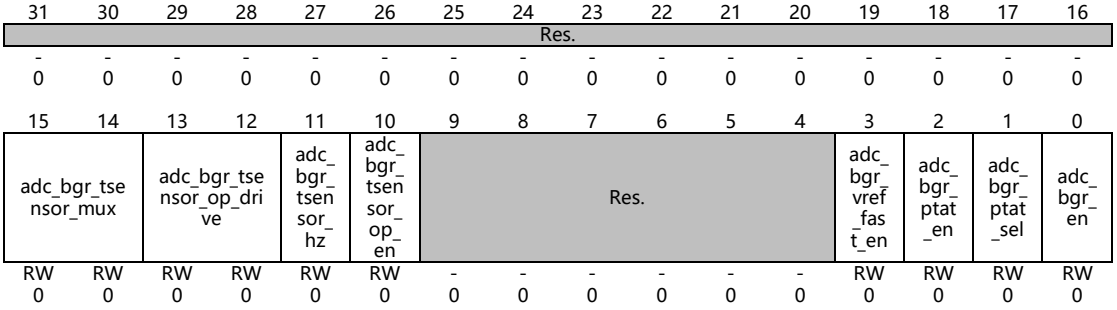
◆ **偏移地址：0xB8　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:1]	-	保留	0x0
ibias_en	[0]	RW	IBIAS 的使能控制信号： 0：不使能； 1：使能；	0x0

35.2.3 ADC_BGR_CTRL

◆ **偏移地址：0xC4　　复位值：0x00000000**



Name	Bits	R/W	Description	Default
reserved	[31:16]	-	保留	0x0
adc_bgr_tsensor_mux	[15:14]	RW	Tsensor MUX 的输入电压选择控制信号： 0：选择 PTAT 电压； 1：VDD_CLDO； 2：VREF_1P0； 3：VREF_1P25；	0x0
adc_bgr_tsensor_op_drive	[13:12]	RW	Tsensor 的 OP 输出级电流档位选择： 0：20uA；	0x0

Name	Bits	R/W	Description	Default
			1：50uA； 2：60uA； 3：90uA；	
adc_bgr_tsensor_hz	[11]	RW	Tsensor OP HZ 模式使能控制开关： 0：不使能； 1：使能；	0x0
adc_bgr_tsensor_op_en	[10]	RW	Tsensor 的 OP 使能控制信号： 0：不使能； 1：使能；	0x0
reserved	[9:4]	-	保留。	0x0
adc_bgr_vref_fast_en	[3]	RW	ADC_BGR 的快启使能控制信号： 0：快启使能； 1：快启不使能，可在上电完成后配置该模式，该模式低噪声；	0x0
adc_bgr_ptat_en	[2]	RW	ADC_BGR 的输出 PTAT 电压使能控制信号： 0：关闭； 1：使能；	0x0
adc_bgr_ptat_sel	[1]	RW	PTAT 电压选择信号，默认选择 1 倍电压： 0：低档位电压； 1：高档位电压；	0x0
adc_bgr_en	[0]	RW	ADC_BGR 的使能控制信号： 0：不使能； 1：使能；	0x0

36 调试接口（DBG）

36.1 DBG 简介

调试接口（DBG）是软件与处理器交互的通道。用户可以通过调试接口获取 CPU 的寄存器以及存储器内容等信息，包括其他的片上设备信息。此外，程序下载等操作也可以通过调试接口完成。AiM51 系列芯片支持标准 5 线 JTAG 调试接口或者两线串行调试接口，完全兼容《RISC-V debug V0.13.2》协议标准。

36.2 DBG 主要特性

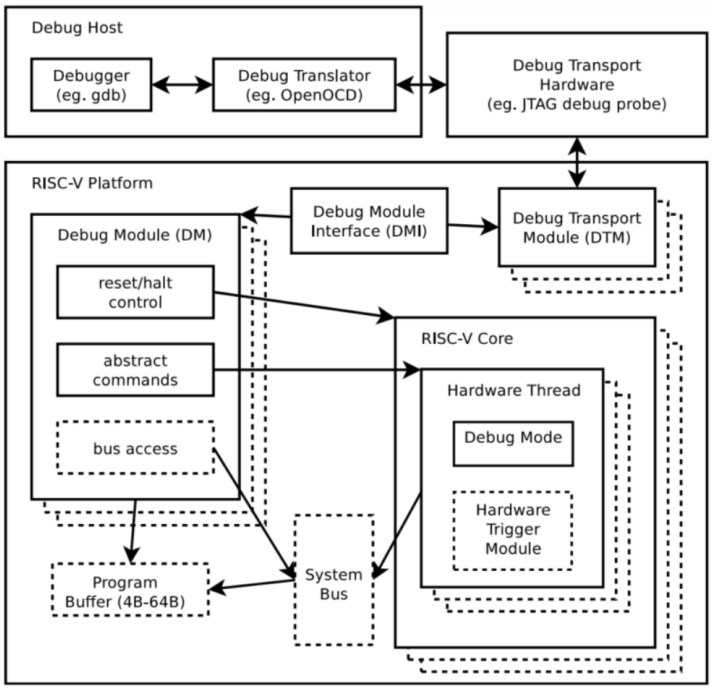
- 支持同步调试和异步调试；
- 支持软断点；
- 可以设置多个硬断点；
- 检查和设置 CPU 寄存器的值；
- 检查和改变内存值；
- 可进行指令单步执行或多步执行；
- 快速下载程序；
- 可在普通用户模式下进入调试模式；
- 支持 CPU 全速运行时通过调试端口直接访问内存。

36.3 DBG 框图

AiM51 系列芯片的调试工作是调试软件、调试代理服务程序、调试器和调试接口一起配合

完成的，其中，调试软件和调试代理服务程序通过网络互联（或为本机），调试代理服务程序与调试器通过 USB 连接，调试器与芯片的调试接口以 JTAG 接口或两线串行调试接口通信。

图 36-1 调试系统示意图



参考文档：

《RISC-V debug V0.13.2》 https://github.com/riscv/riscv-debug-spec

36.4 DBG 引脚

36.4.1 JTAG 5 线接口引脚

引脚名	类型	Description	管脚
JTAG_TMS	I	JTAG 调试模式选择	PD14
JTAG_TCK	I	JTAG 调试时钟	PE0
JTAG_TDI	I	JTAG 调试数据输入	PE1
JTAG_TDO	O	JTAG 调试数据输出	PE13
JTAG_TRST	I	JTAG 调试复位	PE14

36.4.2 二线串行调试接口引脚

引脚名	类型	Description	管脚
SWDIO	I/O	串行线数据输入/输出	PD14
SWCLK	I	串行线时钟	PE0

37 器件电子签名

37.1 简介

器件电子签名存储在 Flash 模块的系统存储区中，可以使用调试接口或 CPU 对其进行读取。它包含出厂前编程的标识数据，这些数据允许用户或其他外部器件访问，方便用户使用微控制器的相关功能。

37.2 唯一器件标识符寄存器（UID）

96 位的唯一器件标识符提供了一个对于任何器件和任何上下文都唯一的参考号码。用户不能更改这些位，唯一器件标识符可用于以下场景：

- 用作序列号（例如 USB 字符串序列号或其他终端应用程序）
- 在对内部 Flash 进行编程前，将唯一器件标识符与软件加密原语、协议结合，将其作为安全密钥的一部分，可以提高 Flash 中代码的安全性、激活安全自举过程等。

◆ **UID0 地址：0x610010C0　　复位值：0xXXXXXXXX　　其中 X 是出厂前编程的标识数据**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
uid0															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
uid0															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

◆ **UID1 地址：0x610010C4　　复位值：0xXXXXXXXX　　其中 X 是出厂前编程的标识数据**

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
uid1															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

47	46	45	44	43	42	41	40	39	38	37	36	35	34	33	32
uid1															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

◆ **UID2 地址：0x610010C8　　复位值：0xXXXXXXXX　　其中 X 是出厂前编程的标识数据**

95	94	93	92	91	90	89	88	87	86	85	84	83	82	81	80
uid2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

79	78	77	76	75	74	73	72	71	70	69	68	67	66	65	64
uid2															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

Name	Bits	R/W	Description	Default
UID0	[31:0]	RO	出厂前编程的标识数据	0xXXXXXXXX
UID1	[63:32]	RO	出厂前编程的标识数据	0xXXXXXXXX
UID2	[95:64]	RO	出厂前编程的标识数据	0xXXXXXXXX

37.3 Flash 大小数据寄存器

本寄存器保存芯片内部 FLASH 和 SRAM 容量大小数据。

◆ **地址：0x610010CC　　复位值：0xXXXXXXXX　　其中 X 是出厂前编程的标识数据**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
sram_size															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
flash_size															
RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO	RO
X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

Name	Bits	R/W	Description	Default
sram_size	[31:16]	RO	芯片内部 SRAM 大小，单位 KB。	0xXXXX
flash_size	[15:0]	RO	芯片内部 Flash 大小，单位 KB。	0xXXXX